ACERVATOR

User Manual and Feature Design Intention Guide

Turbator aequilibrii dissolvendus reformandusque.

—----------

“The disturber of the balance is to be dissolved and reformed.”

“You are either gaining more territory cheaper or you are selling for higher than you bought it in direct response to the market’s moment to moment volatility.” - Ekthelius

Contents

Part 1 Frontmatter & Overview **13**
Copyright and Attribution 13
Contact 13
Legal Disclaimers 13
Risk Factors Specific to Algorithmic Trading 14
Executive Summary 14
What Acervator Is and What It Is Aspiring To Be… 14
Product Manual Parts 16
Part 2 Novel Concepts and Patent Candidate Portfolio **18**
Novel Concepts and Patent Candidate Catalogue 18
1 - Harvest-Fold / Scrumming 18
2 - The Phantom Bot 24
3 - The Landing Strip 33
4 - Bollinger Band Travel Detection 35
5 - Position-Aware Technical Analysis 37
6 - Market Inspector 38
7 - Bot Swarm and Smart Wire Network w/ Provenance Tracking 40
8 - Hunger and Satiety Indices 41
9 - Accumulation Shadows 45
10 - Charge Up Permission Gate 45
11 - Fold, Stack, Extractor, and Shadow Tranches / Provenance Queue 46
12 - Technical Analysis Indicator Confidence Tiers 57
13 - Initial-Purchase-Price Floor on Folds 59
14 - Position Ceiling and Detonation 62
15 - Proof of Accumulation (PoA) 65
16 - Technical Analysis, Goodness of Fit, and the Confidence Index 66
17 - Tiered Strategy Validation and Workflow 68
Part 3 System Architecture and Features Catalogue **70**
System Architecture and Features Catalogue 70
The mark 71
Trading Tab 75
A dollar box draws a dash rather than its own bound 246
2026-10-25 - #665 - the lock figure reaches a new bot on all three builds 248
2026-10-25 - #665 - one read seeds every row on the Detail page 248
2026-10-25 - #665 - the despawn rule is declared once 249
2026-10-25 - #665 - the Apply routing map is declared once 250
2026-10-25 - #665 - the Hedge Balance opening figure is declared once 251
2026-10-26 - #665 - both Electron Ceiling Multiple rows read the one declaration 251
2026-10-26 - #665 - a stored zero fold ratio reads as the declared ratio 252

2026-10-26 - #665 - the despawn tooltip is declared once 254
2026-10-26 - #665 - the grid conversion method is gone 255
2026-10-26 - #665 - the ratio sits in one group, and the fill writes no spare record 256
2026-10-26 - #665 - the cartridge cap, the taper's name, and the bar on the Bot Swarm row 259
2026-10-27 - the holdings audit, and what a buy books 261
2026-10-28 - the holdings audit lands the lot book on the exchange, and every fill moves the pri… 265
2026-10-28 - #665 - Min Confidence reaches down to zero, and the detonation vote reads the… .. 268
2026-10-29 - every fill is booked once, from the count the exchange settled 270
2026-10-29 - every verified fill is announced once, in one shape 273
2026-10-29 - every figure a fill settles is written when the fill is announced 277
Indicator Voting Panel 280
The twelve voters 290
Indicator formulae 292
Departures from the published maths 304
2026-09-08 15:12 - #414 - the ratio the gate reads 305
2026-09-08 15:54 - #414 - when ADX first has something to say 306
2026-09-08 16:22 - #414 - the MACD reading at a millionth of a dollar 308
2026-09-08 16:52 - #414 - the money flow a halted market never traded 310
2026-09-08 17:45 - #414 - the Vortex reports the Vortex 312
2026-09-08 19:05 - #414 - the RSI label and the RSI vote read one number 314
2026-09-09 19:37 - #414 - the band a millionth of a dollar wide 317
Trading gate logic chain 320
2026-09-24 22:50 - #202 - one admission rule for every panel reading 333
2026-10-24 - #665 - a gate row carries the panel it was decided on 334
2026-10-25 - #665 - the consolidation figure is declared once 334
Subsystem Tabs 335
Main Window 339
Inspector 345
Charts 350
Live Tab 355
Converting the Interface to React 357
2026-10-28 - #1374 - where the owed rebuild now stands 405
Subsystem Detail 406
How an update is written down 406
Contents 407
The live tab set 407
How a screen reaches its renderer 409
Portfolio Information Panels 411
What builds it 411
Left: the spendable columns 412
Right: the five counter cards 416
Privacy 418

Where the numbers come from ... 419
Hiding ... 419
Bridge ... 420
Simulator Tab ... 420
What the tab holds today ... 420
How each part was built ... 431
The candles ... 431
Portfolio Battery history ... 432
Portfolio Battery coverage ... 433
Validation Mode ... 435
Back Test Mode ... 437
Portfolio Battery Mode ... 440
The YTD trade files and the import ... 444
How the Sim tab reaches the bar ... 448
The Qt tab is Live's tab code, forked ... 449
The React page is Live's page modules, forked ... 452
The header strip shows on Sim ... 455
The venue stack seats the fleet's exchanges ... 457
The Scrumming Bots table draws the sim fleet ... 458
New Bot opens the Simulator's wizard ... 460
The Extractor Bots table draws the sim extractors ... 463
The Simulator keeps its own fleet file ... 465
Detail opens the Simulator's Bot Settings window with all seven tabs ... 469
The Sim venue pane is Live's width ... 471
The command bar acts on the sim fleet ... 472
The voting panel reads the Stone Tablet ... 474
The Activity Log spool holds and resumes ... 478
The API Interaction Log spool holds and resumes ... 480
The two strip rows keep Live's heights ... 482
The mode choice is wired and the way-ins follow it ... 483
Import Live Fleet fills the tables and the strip ... 487
Generate From YTD fills the tables from the YTD trade files ... 490
Create New Bots fills one row through the forked wizard ... 494
One trading logic: the Simulator sizes its trades with the Live bot's own arithmetic ... 497
The unit rule: fractional or whole, per asset class, cited from the venue ... 499
The trade parity report every run writes ... 504
The Simulator colour distinction is the theme's nigredo tone ... 508
The API spool shows Stone Tablet and YTD retrieval activity only ... 514
The Activity spool shows the trades of Validation, Back Test and Portfolio Battery runs ... 517
Start runs Validation and Back Test from the command bar ... 520
The Simulator writes Live's log rows under the sim bucket ... 526
Clear Fleet empties the held fleet ... 532

- Start Run starts the active mode's run from the way-in row ... 535
- The flip sits at the panel's top-left corner under both layers ... 538
- One fleet per mode ... 540
- The Battery walks every candle, one portfolio at a time, and reads HODL against Harvest-Fold ... 543
- The Simulator's run emits go to the Simulator's own signal sink ... 548
- The widget the rebuild replaced ... 552
- The empty Sim tab before the fork ... 552
- What builds it ... 553
- Fleet Replay ... 554
- The validation criterion ... 557
- Nuclear Mode ... 558
- Bridge ... 559
- 2026-09-08 08:17 - #117 - what the closed issues landed ... 559
- The first clone, before the fork ... 565
- The style sheet the Sim page loads ... 568
- Crypto tablets are 5m and each bot walks the timeframe its tablet carries ... 569
- The playback marks each fill on its candle and the header reads the opposing-trade distance ... 573
- The simulated bot acts like its live equivalent ... 579
- Back Test walks the full Stone Tablet ... 587
- 2026-09-19 17:40 - #55 - the glyph definition moves to the chart's surface ... 591
- 2026-09-24 - #858 - the bot list upgrades reach the Simulator ... 592
- 2026-09-25 - a back test's orders are formatted to the venue ... 594
- A stored figure no box can hold draws a dash ... 595
- 2026-10-20 - #937 - the Scrumming table's state sits on the Symbol cell ... 595
- Both tabs build their sector pages from one declared list ... 597
- The sector press reaches every sector page on both tabs ... 598
- 2026-10-25 - #937 - the Simulator page draws the Asset column's mark ... 601
- 2026-10-25 - #665 - the simulated record carries the wizard's lock figure ... 602
- 2026-10-25 - #665 - the Simulator wizard reads the one Hedge Balance figure ... 602

Paper Trader Tab ... 603

- The empty tab it replaced ... 603
- What the step is ... 603
- Where the platform once offered it ... 603
- The screen as it stands now ... 604
- The two presses ... 605
- Downstream only ... 606
- One gate chain ... 606
- The fake balance ... 607
- The fleet ledger ... 607
- The Paper Trader log ... 608
- Real time ... 609
- Paper Trade History ... 609

2026-09-08 08:17 - #19 - what the closed issues landed ... 609
How the Paper tab reaches the bar ... 612
The style sheet the Paper page loads ... 613
2026-09-19 - #19 - Live's tab code cloned under Paper names ... 614
2026-09-20 - #19 - The paper exchange adapter feeds the tab ... 624
2026-09-20 - #19 - The paper bot ticks and fills on the clone ... 630
2026-09-24 - #858 - the bot list upgrades reach the Paper Trader ... 641
2026-09-24 - #571 - the asset class group and Add Exchange reach the Paper Trader ... 643
2026-09-25 - #19 - a paper bot's settings reach its stored record ... 646
2026-09-26 - #19 - a paper fill draws its own Activity Log line ... 648
2026-09-26 - #19 - a paper bot is created from scratch ... 649
2026-10-04 - #1103 - a pushed candidate spawns paper bots ... 651
A stored figure no box can hold draws a dash ... 654
2026-10-18 - #1203 - one resolver answers a market's sector ... 655
2026-10-20 - #937 - a Paper row's state sits on its Symbol cell ... 658
2026-10-25 - #937 - the Paper page draws the Asset column's mark ... 659
2026-10-25 - #665 - the paper record carries the wizard's lock figure ... 660
2026-10-25 - #665 - the Paper wizard reads the one Hedge Balance figure ... 661
2026-10-26 - #665 - the Paper row names the fold taper by size and shows the detonation bar ... 661
2026-10-27 - #1125 - a pair with no recorded row answers no sector at all ... 661
CITED AS ABSENT ... 662
Proof of Accumulation ... 662
The tab the window does not build ... 663
What the screen declares ... 664
Identity ... 665
The trade log ... 666
The competition lifecycle ... 666
The token ledger ... 666
The five rarity tiers ... 667
The token contract ... 668
The Quintessence contract ... 669
Governance, the franchise and the halt council ... 670
Trophies ... 671
The chain ... 671
Quintessence ... 673
The certified fill ... 676
The capture bounds on one award ... 677
The rotating reward set ... 678
The six-month project age rule ... 678
The event modes and the turn ... 679
The action bands and the record store ... 680
The pot divides on performance ... 681

- Head to head ... 681
- The world grid ... 682
- Movement, and the world turn's pool ... 683
- Dungeons and the second clock ... 684
- The Vessel ... 684
- Stats, and the metrics they convert from ... 684
- The seven character classes ... 685
- The skill ladder and the transfer skill ... 685
- Materials, items and crafting ... 686
- Loot ... 686
- Monsters ... 687
- Alignment, prayer and consecrated places ... 688
- Domination, and the two forms a taking takes ... 688
- The PvP vote ... 689
- Guilds and the treasury ... 689
- Linking two nodes ... 690
- Map marks ... 690
- Every rate, and its provenance ... 691
- The party row's mark ... 691
- The two meters, and the reset ... 691
- What the engine does on the Demo chain today ... 692
- What is not built ... 692

Market Inspector Tab ... 693

- What builds it ... 693
- The six zones ... 693
- Left half: the scanner ... 696
- Where the analysis happens ... 716
- Fetching ... 717
- Right half: topology proposals ... 725
- Dismissal ... 728
- Adopt ... 728
- Push to the Simulator, the Paper Trader and Live ... 729
- ATA-SMP ... 731
- Multi-Exchange Arbitrage ... 762
- Phantom Bot HTF Signals ... 762
- The asset logo library ... 764
- What the emitters write ... 770
- Bridge ... 771

Bot Swarm Tab ... 774

- What builds it ... 774
- The nodes ... 775
- The live rows ... 776

- Drawing a wire ... 776
- The flow along a wire ... 777
- Routing the profit ... 777
- Landing the profit ... 782
- The tranche book ... 782
- Where the fleet comes from ... 785
- Capital claims ... 785
- Bridge ... 786
- 2026-09-07 15:58 - no issue recorded - ten single-word tabs, each on its own ground ... 787
- 2026-09-09 22:12 - #128 - one view, growth stages, and wires that hang ... 792
- 2026-09-09 22:12 - #128 - the Swarm tab loses List View ... 796
- 2026-09-23 - #239 - Disconnect All clears the key the fleet is stored under ... 798
- 2026-09-23 - #239 - every control read on the running program ... 799
- CITED AS ABSENT ... 804

Asset Charts ... 804

- What builds it ... 804
- The chart widget ... 805
- Indicators on the chart ... 806
- Where the candles come from ... 807
- The other chart ... 808
- Bridge ... 808
- 2026-09-08 13:30 - #407 - a Live / ATA-SMP toggle over the arrows ... 808
- 2026-09-08 13:30 - #407 - the arrows now reach the renderer ... 809
- 2026-09-13 15:40 - #23 - the Charts tab draws in the desktop window ... 810
- 2026-09-15 10:38 - #55 - one painter draws the chart in both builds ... 812
- 2026-09-19 08:23 - #55 - every Voting Panel indicator on the chart ... 815
- 2026-09-19 17:40 - #55 - the painter's skin and the Acervator annotations ... 819
- 2026-09-19 20:10 - #55 - the ATA-SMP list draws the venue image's picture ... 824
- 2026-09-19 20:53 - #55 - the page's chart answers the pointer as the widget does ... 827
- 2026-09-20 05:40 - #55 - the frame: one control row, compact toggles, the value field on the A… ... 831
- 2026-09-20 06:10 - #55 - the trade events from the History; the floors gone ... 834
- 2026-09-20 09:45 - #55 - the oscillators: one readable sub-pane height, whole-pixel pens, tags… ... 837
- 2026-09-23 17:30 - #55 - the oscillators are a third taller, and the bottom bar drags both heights ... 841
- 2026-09-23 22:10 - #55 - the trade markers draw on every timeframe ... 843
- 2026-10-16 - #230 #223 - one unreadable price costs one candle, and a stored flag never print… ... 846
- 2026-10-16 - #230 #223 - the other five stored prices admit before the chart draws them ... 848
- 2026-10-19 - #230 - the header names the candle it left out, and the dead bullseye list goes ... 850
- 2026-10-20 - #230 - the grid line list, its setter and its published field go ... 852

History Tab ... 854

- What builds it ... 854
- Fetching ... 854
- Where the trade record is kept ... 855

Filtering, paging, export 856
Where the cell values come from 857
Grading 858
Gate analysis 859
When the logs are read 860
The renderer 861
The Simulator front-load 861
Paper History 862
Bridge 862
2026-09-08 08:17 - no issue recorded - what the closed issues landed 862
2026-10-06 - #1133 - the trade historian's own fetch 866
Console 867
What builds it 868
Upper pane: the Python log 868
Pausing 870
Lower pane: the Emitter Network 870
Where the lower pane goes 871
Bridge 871
2026-09-07 15:58 - no issue recorded - ten single-word tabs, each on its own ground 872
2026-09-23 - #34 - the drained record carries three more fields 879
2026-10-20 - #1392 - a gate refusal reads REFUSED, once a candle 880
2026-10-21 - #1392 - the state lines take the same guard 881
2026-10-28 - #1374 - a locked launch starts no bot and the Console says what it waits for 883
2026-10-28 - #1374 - the screen that takes the password is built 885
2026-10-28 - #1374 - a locked launch reads the prices a screen draws for a market 886
2026-10-28 - #1374 - a sealed fleet makes no new bot 887
System Status Tab 889
The skeleton 889
Half one: the Emitter Network 890
Half two: the Watchdog 892
Until the tab lands 892
2026-09-08 08:17 - #34 - what the closed issues landed 892
Settings 912
The pages 912
What each page persists 913
Settings, User page 915
Settings, Exchanges page 916
Bridge 918
2026-09-06 00:52 - #423 - the Settings dialog draws in React 918
2026-09-11 - Save keeps what the store holds 974
2026-09-11 - The Username row and the vault phrase 976
2026-09-11 - The exchange picker and the venue list 977

2026-09-11 - The API Key row, from the typed string to the stored token 980
2026-09-11 - The API Secret row converts a pasted PEM 982
2026-09-11 - The Passphrase row and the venue that needs one 983
2026-09-11 - The saved exchange list 985
2026-09-11 - Position Distance is off the Trading page 988
2026-09-11 - Increment Style is off the Trading page 989
2026-09-11 - Default Positions is off the Trading page 991
2026-09-11 - Default Target Balance opens the wizard's field 993
2026-09-12 - Bot Visibility opens the wizard's Order Visibility control 996
2026-09-12 - Enable aggressive trading mode opens the wizard's own box 998
2026-09-12 - Profit Folding Active waits on the wizard's folding page 1001
2026-09-12 - Distribution Mode is off the Profit Folding page 1004
2026-09-12 - Profit Folding Target waits on its count row 1006
2026-09-12 - Fold to X# of buy positions takes the whole target group off the page 1010
2026-09-12 - Upward Distribution Target takes the sell-side group off the page 1013
2026-09-12 - The phantom master box becomes a new bot's phantom default 1018
2026-09-12 - The timeframe grid becomes one phantom selection 1021
2026-09-12 - Every retired setting name says what it is 1024
2026-09-12 - The Logging page comes off the Settings dialog 1026
2026-09-12 - The AI Monitor page reaches the monitor it names 1031
2026-09-13 - The credential pair is masked and Username is not 1036
2026-09-13 - The first-run path keys no vault to a placeholder 1038
2026-09-13 - One venue list, one passphrase set 1040
2026-09-13 - The exchange screen follows the store 1042
2026-09-13 - The bot's increment style comes off BotConfig 1044
2026-09-13 - The shell's wizard request carries the store 1046
2026-09-13 - The store refuses a value of the wrong type 1047
2026-09-13 - The wizard's Profit Folding page is removed 1050
2026-09-13 - The Settings dialog's Profit Folding page is removed 1052
2026-09-13 - A value pushed after the page is drawn 1055
2026-09-13 - The radio drawing code comes off the Settings dialog 1058
2026-09-13 - The distribution statistic comes off the bot record 1059
2026-10-04 - Exchange Status, the sector's venues in three colours 1060
2026-10-06 - Six sectors on the Exchanges page 1064
2026-10-09 - Exchange Status is an array of buttons, not a list 1065
2026-10-09 - The first-run wizard offers every venue the platform holds 1069
2026-10-11 - The Verification Code row and the venue that needs one 1071
2026-10-11 - The stock wing takes a credential 1073
2026-10-16 - The lite bot runner keeps its pickers 1077
A stored figure no control can hold draws a dash 1078
2026-10-16 - The faults a control reader may raise 1079
2026-10-18 - The launch line reports what the last call answered 1079

2026-10-18 - A format check records a format check 1081
2026-10-18 - A Username change asks before it reseals the stored secrets 1083
2026-10-22 - One declaration opens both Phantom Bots boxes 1086
2026-10-23 - Eighteen citations on this page name their module 1088
2026-10-24 - One declaration names each venue field and its default 1088
2026-10-25 - #665 - a stored choice outside its list says so on the control 1089
2026-10-25 - #665 - the AI Monitor's opening wait is declared once 1090
2026-10-26 - #665 - the venue record, the add box and the retired mode's name 1091
2026-10-26 - #665 - the nested pair and the log field are off the tree 1093
2026-10-26 - The Phantom Bots figure is retaken from the drawn page 1094
2026-10-28 - #1374 - the Fleet Lock rows, and the screen that gives the password back 1095
2026-10-28 - #305 - the AI note builds its journal when it is written 1098
2026-10-28 - #1374 - the React page draws the Fleet Lock group from the declaration the Qt… 1099
2026-10-28 - #665 - a new bot's twelve weights start at zero 1101
2026-10-28 - #1374 - a sealed fleet makes no new bot 1103
2026-10-29 - A venue a running bot trades on is kept 1104
2026-10-29 - A stored venue no sector lists stays removable 1106
The Promotion Pipeline 1107
What each step contributes 1107
One logic, three data sources 1108
The dashed edge 1108
Where the chain breaks today 1108
Notifications and Alerts 1109
What the Event Routing table is for 1109
The columns come from one declaration 1109
Which events reach the message channels 1110
The dollar figure in a fill message 1110
The state of this screen 1110
Part 4 Live and Supporting Evidence 1112
Part 5 Conceptual Hopscotch: The HOP Protocol, Rules Registry, and SADP v1 and v2 1113
Part 6 ADR Index & Glossary 1114
Decision Records and Glossary 1114
Where a decision is recorded 1114
Glossary 1114
The operator's own words 1117
Part 7 Recent Updates and Version Tracking 1118
Part 8 The Development Chronicle As Told By Claude 1119
Part 9 Live Trade History and Supporting Evidence 1120
VWAP Charts and Trade Grading 1120
The record behind the charts 1120
Subtotal is the price 1120
How a VWAP is computed here 1121

What the combined chart plots 1121
The buy and sell ratio 1123
Traps in the export 1125
Where the figures live 1125
The charts are tracked, and they name no asset 1125
How to read a per-asset chart 1126
The Bollinger band behind the price 1126
The newest export, beside the one the figures above measure 1130
The October export, and the base that makes 39 1130
What this repository holds for these charts 1139
The builder draws every VWAP chart 1140
The 38 per-asset charts 1142
Trade grading 1258
Gate logs behind each trade 1258
Weekly candles behind the fills 1260
The price at the finest resolution the record carries 1261
The venue's own position cards 1261

Part 1

Frontmatter & Overview

Copyright and Attribution

Copyright © 2025–2026 Ekthelius the Accumulator. All rights reserved. Unauthorized reproduction, distribution, modification, or derivative work is prohibited.

This document describes systems, algorithms, and methods developed by the author over the course of a self-teaching period starting in 2017.

Produced using the SADP (Structured Assistant Development Protocol) discipline framework designed by Ekthelius and deployed within Claude Code as a Skills-based and python-driven harness.

Contact

Acervator is the personal project of Zavyrian Productions and Ekthelius the Accumulator. Commercial inquiries, licensing requests, and networking requests are available via the channels listed at publication time:

ekthelius_the_accumulator@protonmail.com

Legal Disclaimers

NOT INVESTMENT ADVICE Nothing in this document constitutes investment advice, a solicitation to buy or sell any security or financial instrument, a recommendation of any trading strategy, or an offer to engage in any financial transaction. Acervator is a software system, and software cannot give investment advice. Trading cryptocurrencies and digital assets involves substantial risk of loss. Acervator is a highly complex, professional, and stable tool for automating digital asset trades. It does not and cannot guarantee the ultimate state of a portfolio given any number of transactions executed through or by it based upon a given user's configuration.

NOT A REGISTERED INVESTMENT ADVISOR The author is not a registered investment advisor, broker-dealer, or fiduciary. No fiduciary relationship is created by distribution of this document. Consult a qualified financial professional before making any investment decision. Ekthelius is a self-taught Technical Analyst with no official documentation to certify or desire to certify his skillset beyond his own personal portfolio results. They have been developed and achieved the hard way.

SOFTWARE PROVIDED AS-IS

The Acervator software system is provided as-is without warranty of any kind, express or implied, including merchantability, fitness for a particular purpose, and noninfringement. Users assume all risk associated with deployment. The author is not liable for financial losses, system failures, or downstream damages resulting from use of the software. Training, guidance, and general set up assistance can be provided but the vast majority of help one needs will be found in the eight part documentation so read it and understand what you are trying to use.

JURISDICTIONAL COMPLIANCE

Users are solely responsible for compliance with all applicable laws and regulations in their jurisdiction, including but not limited to: securities regulations, cryptocurrency-specific rules, KYC/AML requirements, tax reporting obligations, and exchange terms of service. Acervator does not assess or enforce jurisdictional compliance.

PATENT AND INTELLECTUAL PROPERTY NOTICE

The mechanisms catalogued in Part 2 are claimed as novel inventions by the author. No patent has been granted or filed for any of them. Research, academic, and personal study use is permitted. Commercial or competitive use requires written license.

Risk Factors Specific to Algorithmic Trading

Algorithmic trading systems can fail in ways manual trading cannot. Known risk categories include: (1) exchange API failures and rate limits, (2) network latency and partial fills, (3) flash-crash and liquidity-gap events that bypass stop conditions, (4) stuck positions during exchange downtime, (5) adverse selection on thinly-traded pairs, (6) slippage exceeding model assumptions, (7) parameter drift as market microstructure evolves, (8) bugs in the trading code itself. Acervator includes explicit mitigations for these failure modes (see Part 4 · Risk Controls), but no mitigation is complete and Acervator is a platform that is being actively developed. A trader using any algorithmic system must understand these risks before deploying capital.

Executive Summary

What Acervator Is and What It Is Aspiring To Be…

Acervator is a desktop trading platform whose innovation is the platform itself. The breakthrough is not a feature, an indicator, or a bot — it is the integrated system. A harvest-fold accumulation engine, a twelve-voter Indicator Voting Panel across up to five timeframes, a seventeen-class declarative GateChain safety framework, multi-bot coordination, and a live evidence trail proving the whole operates on real capital — coherent as one machine that converts price oscillation into asset accumulation and monotonic target growth without forecasting direction.

The platform will ultimately be designed to trade both cryptocurrency and stocks but is being developed to focus on the blockchain first given its current and immediate future impact on global financial systems. The crypto trading side is being prototyped via Coinbase Advanced and is the production surface today — live operator capital, fourteen weeks of execution, 5,055 trades with all assets successfully fending off the current bear trend or flat out beating it.

Harvest-Fold is the method and a mechanism. A position opens against a base currency with an explicit USD target (or a target that is translated to a USD or other local equivalent) with the initial size of the position being used as a value set point about which the mechanism of the platform allow volatility to “orbit” by persistently re-zeroing to it in response to local market structures and price ranges. The method itself stands apart from traditional Technical Analysis trading strategy or systems by not speculating or attempting to guess what the price will be within any given amount of time but instead persistently remolds a portfolio as its positions fluctuate and refusing to leave profit on the table by saying “It might be worth more later.” .

The primary “bot” or strategy that executes the Harvest-Fold method is called a Scrumming Bot and can be supported by, and this is exclusive to the cryptocurrency side of the platform, an Extractor Bot which performs more traditional scalp-style trades using isolated chunks of the parent Scrumming Bot’s Target Asset with the Extractor’s primary limitation being that it only works for established Base Currencies on a given exchange that has multiple asset pairs to exchange them against. The platform is designed and optimized in such a manner to allow “swarms” of Scrumming Bots (with or without Extractors) to be deployed to support dozens of positions across multiple exchanges simultaneously and thereby allowing the average digital asset investor to experience full spectrum digital asset trading that has previously been inaccessible or inefficient to execute properly.

The term “scrum” was initially encountered while I was learning about how a barrel of oil’s profit is split or distributed. The scrum of the barrel, according to the tutorial, was what the gas station gets as its share of profit and its insultingly small, thus requiring huge sums of fuel to be sold for any respectable amount of money to be made by the facilities hosting the gas pumps and their infrastructure. The concepts of small, repeating, and accumulating were found to rest inside on one term and so it was readily adopted. The extractor, however, is more straightforward in origin and it rests on the idea that A15 and A14 trading became the primary means of acquisition after direct mining became unaffordable for average blockchain participants. It has always been apparent that the potential for faster wealth acquisition existed via asset trading but the skill and technology gaps are what prevent this door of opportunity from opening for just about everyone.

Prior to the invention of the Harvest-Fold mechanism, I had also conceptualized grid bots via a now deprecated cryptocurrency auto-trader called Margin. When this application first appeared and was used, it only featured a simple “ping-pong” and a more speculative / adaptive “margin” bot. It should also be noted that it was never a stable product throughout all of the versions attempted and this was what led to its failure while its company became a consultant firm of some sort. I am not sure how producing an unstable, feature-limited auto-trader qualifies you as a fintech advisor but maybe qualifications remain loose or have become looser. Regardless, and in spite of the instability experienced immediately during basic use, an attempt was made to stack dozens of ping bots of a large range on a given market. This may have been the first “grid bot” used in cryptocurrency trading. The resulting and constant crashing led to my seeking help from the company via their Telegram channel where he was told, basically, that he was using it incorrectly which was an unacceptable and stupid excuse for not fixing their product. Instead, they stole said concept and added a “stack bot” in a later version of Margin that performed the same basic behavior under one bot instance but because of the fundamentally buggy quality of the software, this was not used much before a boycott was necessary because the company chose to ban people criticizing the unending stability issues. After this, Margin supposedly partnered with KuCoin to bring their technology to said exchange but this seems to have dissolved at some point with KuCoin releasing its own Grid Bots with which I was able to win 3rd place and win three Lomen NFTs as a result. Said Lomens still reside on a certain Metamask to this day should this or any other part of this tale be doubted. Whether I am a co-inventor or discoverer of modern automated cryptocurrency trading is a point for irrelevant debate. What matters is that Acervator is my newest invention and it is infinitely more powerful than anything before it.

After the United States blocked access to KuCoin, I continued to develop my trading strategies further and executed them manually while learning more Technical Analysis. In the end, high volume trading methodologies were seen as the only method for maximizing profits in volatile markets. The entire idea of ‘HODL’ being pushed by people who were obviously continuing to accumulate was and remains disgusting but it is just how the game is played. No one is clicking anyone else’s mouse so it is permissible. This is just a variation of what is generally called “Wall Street Investment” as well or at least fits the meta-pattern well enough. The average person, and their precious money, is expected to trust and be exposed to dangerous, unstable market environments while gaining no real benefits beyond a promise that breaks every few years.

There is not an entry level for market volatility and, with the advent of Acervator, there does not have to be.

More than anything else perhaps, the creation of Acervator represents an explicit effort to essentially debunk the approach and hamstringed interaction methods most investors are expected to utilize or, more accurately, tolerate. The average person who puts money into the market surrenders most or all control of it based on a hope that increasingly has no rational basis due to the lack of protections or proportionate, timely reward. The investor gets a secondary value abstraction that they have to wait to sell for more later (and are attacked for doing so if it's anything other than a stock!) assuming the market moves favorably. I see this as an over-exposed “bait and switch” where too often people are left holding worthless bags of tokens or stocks or bonds. It is an upgraded lottery system in some regard but it is only gambling to those that attempt to play blindly or passively and accept the ‘HODL’ advice from whales who just need their next meal to sit still.

The highest goal is for Acervator to be a self-contained retirement platform that provides a powerful option for anyone seeking to take a direct, elegant, self-contained approach to their digital asset portfolio management. It aspires to be the lens, the manager, and the fiduciary while only requiring users to test it, suggest new features, and / or donate to its continued development. Regardless, this will be my last attempt to help humanity see past the wiggly chart illusions but, honestly, I do not think anything else will be required.

If Acervator has helped you accumulate or you wish to support development directly (I am a one man show…), consider sending a tip to the inventor.

USDT 0x6Eb5bc78f43b71F5f09B87ED54e7FF3bD8a3428B

A14 0x8b1022d836d19aA848Ebc9E09A6FC2921d9EB7d41

A15 3HutPgxaMTftWQY3KyRkC7Dckp6A1zdfdp0

PayPalUSD 0x52D75aD65a98f98fA2d1D3675Bd5781ee547426D

A15 2 bc1qn3mkssem0k2tyhwl5yeea4hj222k8j9rr6ruff

A14 2 0x56d1Ee9F683d7535D2637F242a1A2B338aC5aC12

LAST NOTE: This manual, like the software it discusses, is a work in progress and it will be updated on a regular basis as features are added and finalized..

Product Manual Parts

This manual is an attempt to clarify all parts and functions for what might be considered a somewhat esoteric product given its innovative approaches and the sheer number of novel features it contains. More cynical SaaS providers might disagree but I look forward to all feedback regardless. Currently there are nine such sections with each focusing on ever smaller components of the total system.

Part Title and Area of Focus

1 Frontmatter & Overview

2 Novel Concepts and Patent Candidate Portfolio

3 System Architecture and Features Catalogue

- Main Window

- Trading Tab - Active Bot Lists Under Exchange Tabs - Indicator Voter Panel - Trade Logic Activity - API Activity - Top-Level Portfolio Metrics
- Simulator Tab - Validation Runs vs. Historical Trade Data - Acervator Strategy Back Testing - Runs Historical Data Through Simulated Bots - Bot Swarm Simulation - Deploy As Paper Trading - Deploy As Live Trading
- Paper Trader Tab - Runs Live Exchange Data Through Simulated Bots - Executes and Documents Paper Trades - Collects Paper Trade Data - Pushes Trade Data to Paper Trade History Log - Paper Trade History Tab Reads and Displays
- Proof of Accumulation (PoA) Tab - PoA Blockchain and Trading Tournament Interface - Hosts User Competition Performance and Data - Displays Tournament Monsters and Targets
- Market Inspector Tab - Synthesizes and Proposes Bot Swarm Topologies and Oppositional Trades - Pushes Proposals to Sim, Paper, or Live As Directed
- Bot Swarm Tab - Displays and Allows Configuration of Capital Reinforcement Networks
- Asset Charts Tab - Displays Price Charts for All Active Bots
- History Tab - Loads and Displays Live Trade History Via Exchange API - Merges Live Trade History w/ Acervator Trade Log for Grading and Gate Analysis - Paper History Tab - Loads and Displays Paper Trade History w/ Grading and Gate Analysis
- Console Tab - Displays internal codebase activity, warnings and faults
- System Status Tab - Displays Platform Status by Subsystem - Fed by the Emitter Network
- Settings - System and High-Level User Settings

4 Live and Supporting Evidence

- YTD Exchange Data Covering the Entire Development Arc and Exchange Test Coverage

5 Conceptual Hopscotch: The HOP Protocol, Rules Registry, and SADP v1 and v2

6 ADR Index & Glossary

7 Recent Updates and Version Tracking

8 The Development Chronicle As Told By Claude

9 Live Trade History and Supporting Evidence

- VWAP Charts - YTD Data Analysis and Trade Grading - Deep Indicator Function Investigations Via Trade Logic Gate Logs

Part 2

Novel Concepts and Patent Candidate Portfolio

Novel Concepts and Patent Candidate Catalogue

Active development on Acervator began in early April of 2026. It began after realizing that the AI-assisted software development tools had achieved an inflection point with their capabilities. Prior to this, efforts were being made to learn traditional coding with little success that was mainly due to an inability to get into a grinding effort for whatever reasons. This has been particularly frustrating since other subjects seem to grab and others deflect. With the advent of Claude Code however, it feels like some of us have been given a decently priced door man and well-informed guide through this and other labyrinths.

Each entry below carries an anchor. The anchor is the code that runs the concept today, quoted from the module that holds it. Where a concept has no code, the entry says so and gives the query that measured the absence, and then proposes the build. Anchors describe the present state of the platform, not the plan.

1 - Harvest-Fold / Scrumming

At the core of Acervator is what might not first appear as a multi-faceted concept designed to address or alleviate many of the mental pitfalls that can afflict a new trader. After having observed a number of people via Telegram and web approach the challenge of investment differently, I have come away convinced that all it only becomes as complex as the individual allows it to be. This is not to be reductive about economics or any other connected discipline but it is to say there are reliable, stable, consistent means of reading price charts and too many people are simply doing it too many different ways and this only compounds the complexity illusion further. Much of the uneasiness inflicted on the new trader surrounds the emotional desire or even personal need for the value of their positions to increase as soon as possible or show the midterm potential of becoming profitable. Those that adopt the HODL meme are, in my experience, quite crippled in their ability to sell positions even when the time is right, which is usually when most people are happiest about the market. The ironic truth is that however good a trade is, there is an equally bad one on the other side of it. Acervator attempts to respect the market by not taking profit all at once and absorbing or claiming ownership of its positions' variance strictly based on the market local, organic price ranges. In other words, its trade actions are always proportionate to whatever the market is actually doing at the time the trade is executed. Over time, this 'molds' the position to the long term truth of the market and avoids the need to speculate entirely. The more bold hope for this approach is that it might add another tool for stabilizing macro-economic phenomena such as underpinning currency exchange rates between countries.

Acervator addresses 'fundamental market unpredictability' by allowing volatility to "orbit" around a set point we call the Target Balance. This element is what performs the first mental trick. It forces the trader to say and establish that "my position is worth X and I will support X with an equal amount of liquidity" i.e. a Target Balance of $50 is set and a total mental position of $100 is established. Trades will be executed to maintain the truth of the Target Balance always being X. After the initial entry is executed and filled, which Acervator will only do in the lower extent of the Bollinger Band on the chosen timeframe, the Scrumming Bot will persistently shave off and "Fold" back in Y% of profits through repeating Harvest-Fold operations that continue until the bot is

deactivated or the market suffers a catastrophic collapse which can and do happen so don't not leave it unmonitored. It should also be noted that the term 'harvest' came to replace 'scrum' for the sake of clarity but they may be used interchangeably throughout this text. Essentially a Scrum or Harvest is a sale of a position's increased value over the Target Balance. The amount that the Scrumming Bot harvests or folds is pinned to the Target Delta or the amount of variation (%delta) from the most recent trade event. Much of the finer details of trade execution can be tuned by the user as expected for any professional application but the primary point here is that the amount of profit folded back is capped by the Maximum Target Growth Per Cycle setting while the ability of Acervator to prevent inverted (profit losing: buying higher than sold / selling lower than bought) trades we also have the Minimum Opposing Trade Distance rule which dictates the next trade must be Z distance from the most recent trade event. In addition, sells are not allowed to occur or be pushed below the Bollinger Band midline while buys are not allowed to occur or be pushed above the midline. As a result of just these few component features, we already have multiple soft, non-TA or non-traditional rules protecting the investor before we even start optimizing the trades themselves (pushing them as close to volatility extents as possible) via Technical Analysis and other inventions.

Now: a buy-back waits for this distance alone, wired or not, because a wire takes only the profit a sale leaves past the dollars that buy back its units and the room the growth cap leaves (`src/trading/scrumming/sizing.py, in sale_surplus_usd`).

The harvest-fold method itself was manually executed and tested on a multiple cryptocurrency pairs throughout 2023~2026 but it has always been the aspiration to automate it and have it executed at a high rate of frequency with the operating theory that this approach to investment, being a hybrid a HODL and active trading, creates frontline area of support for whatever markets in which it participates long term. It does this by maintaining and, utilizing as directed, all earned profits. It turns any user into an active participant and liquidity provider.

After entering the space full time in 2017, I immediately began to parse and breakdown how price charts were being interpreted. This also allowed me to perform ever deeper introspections on my own response to how my mind was responding to the value of my portfolio fluctuating. I did myself absolutely no psychological favors in this pursuit but, as you will see, I had plenty of reasons to take this reckless approach. By this point, my undiagnosed and untreated military injuries had worn me down to the point where I realized I was running out of time to continue working as normal. I had, by then, been walking a torsioned pelvis for 17 years that had seized in place and, by 2020, would be causing enough pain to induce panic attacks. I could have joined the pill-mill club but I declined. In combination with the pain, the forced TA self-training, physical job, and broken marriage, I also assembled an entire GPU-based mining farm…alone. I had some business partners but their participation was basically nonexistent. The VA was providing no support and would not until 2022. The Navy and Pentagon had spit in my face by sending me PDBR paperwork that they had no intention of ever honoring no matter what I wrote down. My family more or less remained ignorant of all my turmoil while I kept the paychecks coming. So really the story of Acervator's and Harvest-Fold is one of betrayal to a degree that allowed me to see through things more clearly than would otherwise have been possible. It has been invented out of necessity and it is my intuition that Acervator can become that necessary tool for other people as well.

The target and the two sides

Functional. The bot holds one number, and that number is the dollar value you told it the position is worth. On every tick it compares the position against that number. Above it by more than a dust band, the bot has surplus to sell. Below it by more than the band, the bot has a deficit to buy. Inside the band it does nothing at all. One function answers that question for the whole engine, and the dust band is a tenth of a percent of the target with a floor of one cent. At a target of $50 the band is $0.05, so a position at $60 answers scrum and a position at $40 answers fold.

`src/trading/target_bands.py` — `target_territory`

```
delta = float(position_value) - float(target_balance)
band = at_target_dust_band(target_balance)
if delta > band:
    return "scrum"
if delta < -band:
    return "fold"
return "at_target"
```

Design intention. The Target Balance is the set point the whole method orbits. It is declared once in the bot's config and the tick re-zeroes to it. The bot brain holds the tick, the scrumming package fires the two halves of the cycle, and the execution module places the orders.

`src/trading/container/config.py` — `BotConfig`

```
target_balance: float = 200.0  # Balance the bot trades relative to
scrumming_interval_pct: float = 1.0  # % market move between actions
max_target_growth_pct: float = 1.0
```

The distance rule

Functional. An opposing trade has to travel far enough to clear both legs of the round trip. That is the operator's scrumming interval plus the exchange fee. A one percent interval against a six tenths fee gives 1.60 percent. One function owns the arithmetic, and the clamp is applied to the sum rather than to the interval, because the bound exists to keep the rebuy factor sane.

`src/trading/otd_math.py` — `minimum_opposing_trade_distance_pct`

```
total = float(interval_pct) + float(fee_pct)
return max(OTD_MIN_PCT, min(OTD_MAX_PCT, total))
```

Design intention. Without this rule the bot can sell low and buy back higher, which is the one outcome the method exists to prevent. The gate that enforces it refuses the trade until price has travelled that distance from the pivot, and it rebuilds the required price in its own refusal message so the operator can read why it held.

`src/trading/gate_chain.py` — `HysteresisGate.evaluate`

```
ok = ctx.hyst_ok_scrum_side if self.side == "scrum" else ctx.hyst_ok_fold_side
if ok:
    return GateResult(passed=True)
eff_pct = ctx.scrumming_interval_pct + ctx.trading_fee_pct
```

Current reading. The distance never sits under two fees, the round trip a scrum and its fold pay between them. Where the interval and the band extension together sit under one fee, the distance is two fees. At or above one fee, it is the sum above. The fee is the larger of the Trading Fee setting and the highest fee the venue has reported on this bot's fills.

`src/trading/otd_math.py` — `minimum_opposing_trade_distance_pct`

```
interval, fee, extension = terms
if interval + extension < fee:
    return clamped_otd_pct(ROUND_TRIP_FEES * fee)
return clamped_otd_pct(interval + fee + extension)
```

The midline rule and the growth cap

Functional. A sell is not allowed below the Bollinger midline and a buy is not allowed above it. The tick sets one flag for each side off the band position, and a gate refuses the wrong side of the band. The scrum flag also carries the phantom lock. Nothing sets that lock today, and on a ripe row the ripe-harvest override force-passes the gate that reads it.

`src/trading/scrumming_bot.py` — `ScrummingBot.tick`

```
scrum_ok = (bb_pos > 0.50) and not self._phantom_locked
fold_ok_midline = bb_pos < 0.50
```

Design intention. Profit folded back into the target is capped for one market cycle, so a single violent move cannot inflate the set point beyond what the next cycle can support. The cap is a percentage of the target at the moment the cycle opened, and it shrinks as the cycle consumes it.

`src/trading/scrumming_bot.py` — `ScrummingBot.cycle_growth_cap_usd`

```
_pct = float(getattr(self.config, "max_target_growth_pct", 1.0) or 0.0)
```

The same key answers differently in different places. Nine read sites fall back to 1.0 and three fall back to 0.0, so a bot whose stored config lacks the key compounds or freezes depending on which site read it first. Issue #409 carries it.

The issue #409 clause above is overtaken. It is not edited and not removed. It read:

> *Nine read sites fall back to 1.0 and three fall back to 0.0, so a bot whose stored config lacks the key compounds or freezes depending on which site read it first. Issue #409 carries it.*

Issue #409 closed on 2026-09-25. One declaration answers the key now, and no read site carries a fallback of its own. `src/trading/container/config.py` declares `TARGET_GROWTH_PCT_DEFAULT` from the field's own declared default, and `in target_growth_pct` is the single reader. Fourteen call sites read that function.

```
src/trading/container/config.py, in BotConfig    max_target_growth_pct: float = 1.0
src/trading/container/config.py                  TARGET_GROWTH_PCT_DEFAULT reads that default
src/trading/container/config.py, in target_growth_pct   the one reader
call sites reading target_growth_pct             14
read sites carrying their own fallback            0
```

The gate set the bot writes is named in one place: ten scrum labels and nine fold labels, nineteen lights per bot.

`src/trading/gate_vocabulary.py` — `_GATE_ORDER_SCRUM`

```
_GATE_ORDER_SCRUM: tuple[str, ...] = (
    "TGT",
    "INT",
    "BB",
    "FIRE",
    "TA",
    "LS",
    "TRND",
    "HTF",
    "CB",
    "OTD",
)
```

What the ripe-harvest override replaces at the midline

The entry above states this:

> *Nothing sets that lock today, and on a ripe row the ripe-harvest override force-passes the gate that reads it.*

The current reading. On a ripe row the override force-passes nothing at the midline, because the gate has already passed. A row is ripe only when the band position reaches the upper detect threshold, and that threshold never sits below the midline. A ripe row is therefore above the midline by construction, so the band half of the scrum flag is already true. The phantom lock is the only other way that flag can read false, and no method in the platform sets it.

`src/trading/scrumming/circuit_breakers.py` — `_bb_detect_thresholds`, the reason a ripe row is above the midline

```
detect_frac = max(0.0, min(1.0, detect_pct / 100.0))
half = detect_frac * 0.5
return (0.5 - half, 0.5 + half)
```

Driven. The shipped scrum chain ran over a swept window of hand-built candles. The last candle's close moved from 95.00 to 105.00 in steps of 0.0005, which is 20,001 rows, and the band position read from -0.3089 to 1.3165 off the real band reading. Both the midline and the upper detect threshold sit inside that span, and one step moves the band position by about 0.00008, so no change of verdict can fall between two rows.

```
20,001 rows, the phantom lock clear

the gate passed on its own merit          10,106
the gate refused and the refusal stood     9,895
the gate was passed regardless                 0
rows reaching a real verdict              20,001

6,832 rows were ripe, and the gate passed on its own on every one of them
```

The same override does replace a refusal, on a different gate. With the voting panel not bullish, the override passed the bullish-vote gate on exactly the 6,832 ripe rows, and its refusal stood on the other 13,169. The override acts, and the midline gate is not the gate it acts on.

```
20,001 rows, the phantom lock clear, the voting panel not bullish

ta_bullish passed regardless     6,832
ta_bullish refusal stood        13,169
midline passed regardless            0
```

What a recorded lock does. The Simulator can recover a lock from a stored record. Driven with the lock recovered, the midline gate was passed regardless on the 6,832 ripe rows and its refusal stood on the other 13,169, and the record named the override every time. That is the only state in which the override reaches the midline gate, and nothing in the running program produces it.

```
midline_scrum overridden by ripe_harvest_override: scrum_ok=False(bb_pos=1.23)
scrum_ok=False(bb_pos=-0.22) [override not applied: ripe_harvest_override]
```

What is still open, and it is his. The open decision on the midline rule is whether the band should hold a ripe scrum at all. It is not a decision about the higher timeframe. The reading above moves the ground under it: the band does not hold a ripe scrum today because a ripe row is already above the band midline, not because an override lifts the band. Two settings carry that behaviour and both are his, the BB Midline Gate tick box and the BB Detect threshold, and both are described in Part 6, the Trading tab.

Which gate each override covers is on the gate table in Part 7, Indicators.

The first buy reads three agreeing indicators

Current reading. The entry above places the first buy in the lower extent of the band. The first buy also reads the vote, at the twelve published weights. Before this change one bearish indicator was enough. Now the bearish vote must top what two indicators reach on that same vote, which is 0.1966 with all twelve voting. A bot whose own twelve weights are all zero still buys on the direction alone. Every new bot starts that way.

`src/trading/scrumming/tick_phases.py` — `TickPhaseMixin._tick_initial_entry`

```
if not three_agree(
    summary.consensus_confidence,
    summary.signals,
    self._voting_engine.indicator_weights,
):
```

Driven on a stand-in exchange, against the code before this change taken from git. One MACD reading at 0.1 gave a bearish vote of 0.0103. It bought before and it waits now. Two agreeing indicators read 0.1966 and wait now. Three read 0.2821 and buy on both. A bot with every weight at zero bought on the one and on the two, before and after.

2 - The Phantom Bot

While the Harvest-Fold method housed with the Scrumming Bot addresses several persistent and nagging questions around any given investment position (Should I buy more? Should I sell some? When? How much?), it does have the weakness of operating on a single timeframe. The Phantom Bot is another feature intended to optimize trade execution position. In this specific case, it allows a single Scrumming Bot to be inhibited by a higher timeframe signal with the intention of capturing alignments or disagreements so that trades are not being executed prematurely. The design intention here is that, for example, a 5m TF bot can be inhibited by a 15m, 30m or 1hr signal while maintaining a volatility watch on its primary TF. The intention is to build out this feature so that it obviates or reduces the need for multi-screen monitoring of individual assets.

Functional. A phantom is a copy of the position read on a slower chart. It computes technical analysis on its own timeframe, stores the result, and constructs no order. One timeframe is the default set, and a saved record names its own.

`src/trading/scrumming_bot.py` — `ScrummingBot.__init__`

```
self._phantom_timeframes = list(
    phantom_timeframes
    or default_phantom_timeframes(config.ta_timeframe, _offered)
)
```

Design intention. The inhibition the entry describes runs today. A bullish higher timeframe stops the scrum and a bearish one stops the fold, and each writes its own label into the blocker list so the operator can see which timeframe held the trade. The coordinator weights the phantom summaries by their rank before the gate reads them.

`src/trading/gate_chain.py` — `HTFDeferGate.evaluate`

```
blocks = (
    ctx.eff_htf_blocks_scrum
    if self.side == "scrum"
    else ctx.eff_htf_blocks_fold
)
if not blocks:
    return GateResult(passed=True)
label = "HTF-bullish" if self.side == "scrum" else "HTF-bearish"
return GateResult(passed=False, blocker_message=label)
```

One part of the feature does not work, and it is the phantom lock. No method on the coordinator creates a lock, so the flag stays clear and the lock list stays empty. The term in the scrum flag does read the flag, and a ripe row force-passes the gate it sits in. Whether a higher timeframe should hold a trade at all is the open decision on issue #1359. Replacing multi-screen monitoring is an intention, not current behaviour.

```
six ticks, three phantoms registered, one bot

the flag after every tick                       False
the coordinator's active locks                  []
the flag set by hand, band position 0.70        the midline gate refuses
the flag set by hand, band position 0.91        the ripe row force-passes it
```

Two sentences above are overtaken in part. Neither is edited and neither is removed. They read:

> *The scrum flag also carries the phantom lock. Nothing sets that lock today, and on a ripe row the ripe-harvest override force-passes the gate that reads it.*
>
> *The phantom lock is the only other way that flag can read false, and no method in the platform sets it.*

The current reading. A method on the bot now sets the flag. It runs on every tick, above the scrum flag, and it takes the state from the coordinator's own active lock list rather than from a field nobody wrote. Where the operator has phantom bots switched off it clears the flag instead.

`src/trading/scrumming_bot.py` — `ScrummingBot.tick`, the call above the flag

```
self._refresh_phantom_lock()
if self.config.bb_midline_gate:
    scrum_ok = (bb_pos > 0.50) and not self._phantom_locked
    fold_ok_midline = bb_pos < 0.50
```

The rest of both sentences still holds. No method on the coordinator creates a lock, so the lock list is empty on every running bot and the flag reads clear. The ripe-harvest override still force-passes the gate on a ripe row, which is why a held lock changes no trade. Driven across 3,201 band positions with a lock held, the scrum flag changed on 1,601 rows and the final scrum decision on none.

The full reading, the method, the coordinator's answer and the swept figures are on the Indicator Voting Panel page.

The operator's ruling, 2026-10-27. Asked what should create a phantom lock, he answered:

> *This is a strange question. A phantom bot is simply a higher timeframe read that is piped through a parallel trading gate logic chain. It is intended to improve rather than restrict ripe scrums or folds so it must work on both sides. It should be thought of as an optional guardian angel that may optimize trades.*

Two clauses above are overtaken. Neither is edited and neither is removed. They read:

> *The ripe-harvest override still force-passes the gate on a ripe row, which is why a held lock changes no trade.*
>
> *fold_ok_midline = bb_pos < 0.50*

The current reading. The fold flag carries the lock too, so a held lock does change a trade. The clause remains correct about the scrum side, where a ripe row is above the midline and the override force-passes the gate anyway. On the fold side there is no such override below the midline, so the refusal stands and the fold is held.

`src/trading/scrumming_bot.py` — `ScrummingBot.tick`, the fold flag

```
fold_ok_midline = (bb_pos < 0.50) and not self._phantom_locked
```

Driven. A bot was built by hand with a stand-in venue that refuses every order, and the real tick was driven over 801 window levels an arm, which moved the band position from -0.558 to 1.559. With phantom bots off, and with them on and no lock held, every recorded verdict is identical before and after. With them on and a lock held, the fold gate's refusal appears on the 398 rows below the midline, and the fold decision itself moves on 362 of them. No scrum reading moves on any row.

```
801 rows an arm, two trees compared row by row

phantoms off, lock held     every verdict identical
phantoms on,  no lock       every verdict identical
phantoms on,  lock held     fold decision differs on   362
                            scrum readings differ on     0
```

What should create a lock on the coordinator is still his, and nothing creates one today.

The operator's specification, 2026-10-27. On what a phantom bot is for, and the last change he made to its design:

> *Again, the entire function of a Phantom Bot is to provide high TF over rides and trade optimization. From this perspective, all such questions can be inferred. The design spec is established but simply not fully implemented with the last major modification to it being the decision to only allow one Phantom Bot TF to be active per bot or Target Asset.*

One phantom timeframe is active per bot. The limit is one declaration and every site that builds a reader reads it. The active name is the highest-ranked one the bot was given, which is the name the Phantom Bots picker already drew and the name the rank weighting already counted for most.

`src/trading/phantom_balance.py` — `one_active_phantom_timeframe`

```
active = max(ranked, key=tf_rank)
set_aside = sorted(
    [one for one in held if one != active],
    key=tf_rank,
)
return [active], set_aside
```

step	module	symbol
name the one	`src/trading/phantom_balance.py`	`one_active_phantom_timeframe`
hold the one	`src/trading/scrumming_bot.py`	`ScrummingBot.__init__`
build the one	`src/trading/phantom_balance.py`	`PhantomBalanceManager.create_phantom_set`
keep it on a live change	`src/trading/scrumming_bot.py`	`ScrummingBot.update_phantom_config`
name it on the page	`src/gui/main_tabs/phantom_bots_tab_surface.py`	`configured_timeframes`
name it in the Simulator	`src/simulator/back_test.py`	`phantom_timeframes_for`

Nothing is dropped in silence. A name the limit holds back is drawn on its own status row, and the bot writes the same sentence to its Activity Log on the tick that starts its phantom.

```
a bot given 6h and 1d, read off the page and the log

the picker                      1d
Configured timeframes:          6h, 1d
Set aside, one runs:            6h
the Activity Log line           Phantom timeframes set aside: 6h
```

Two sentences below are overtaken. Neither is edited and neither is removed. They read:

> *A bot holding three names writes all three and reads all three back, so a set of several survives a restart.*
>
> *A bot holding 2h, 6h and 1d writes all three. The fleet load hands all three back, and the phantom manager builds three PhantomBots on them.*

The current reading. The record still carries every name and the fleet load still hands them all back, so both sentences hold for the record. The bot now keeps the highest of them as the one it runs and the rest as names set aside, and the manager builds one reader rather than three.

The save writes both halves back, so a save loses no name the operator chose.

`src/trading/bot_container.py` — `BotContainer.get_full_state`

```
_phantom_held = list(getattr(self, "_phantom_timeframes", None) or []) + list(
    getattr(self, "_phantom_tf_set_aside", None) or []
)
```

Driven on a bot given 6h and 1d, saved and restored three times in a row: the record reads 6h and 1d on every pass, the bot runs 1d on every pass, and the page names 6h as set aside on every pass.

Driven. A bot was built by hand with a stand-in venue that refuses every order, and the real tick was driven over 801 window levels an arm, which moved the band position from -0.292 to 1.283 in steps of 0.03 of a price unit. With phantom bots off, and with exactly one timeframe given, every recorded verdict is identical before and after. With two given, the lower-ranked reading no longer reaches the gate.

```
801 rows an arm, two trees compared row by row

phantoms off              every verdict identical
one timeframe             every verdict identical
one timeframe, lock held  every verdict identical
two timeframes            scrum decision differs on   256
                          fold decision differs on    265
```

One bot, one Target Asset. Each bot builds its own coordinator and no code hands one bot another bot's, so the limit at the bot is the limit at the Target Asset that bot holds. Two bots on one asset each run their own one, and neither changes the other.

The saved record carries the timeframe. The bot writes every name it holds under the key `phantom_timeframes`. The fleet load reads that key and gives the list to the bot it builds. The key, the change

field and the bot attribute now use one spelling.

step	module	symbol
write the key	`src/trading/bot_container.py`	`BotContainer.get_full_state`
read the key	`src/trading/container/restore.py`	`StateRestoreMixin.restore_bots_from_state`
hold the list	`src/trading/scrumming_bot.py`	`ScrummingBot.__init__`
build the set	`src/trading/scrumming/tick_phases.py`	`TickPhaseMixin._tick_initialise`
run one timeframe	`src/trading/phantom_balance.py`	`PhantomBalanceManager.create_phantom_set`

A record written before the key took the plural spelling still restores. The fleet load reads the older singular key `phantom_timeframe` when the plural key is absent. A bot holding three names writes all three and reads all three back, so a set of several survives a restart.

The venue decides which names survive. The bot drops any stored timeframe its exchange does not serve. A record naming one timeframe the exchange refuses restores with no phantom at all, and the bot writes the dropped name to its own log line.

The record drops one key. It no longer carries `phantom_config`. The attribute behind it held an empty dictionary and no code read it. A record that still carries the key loads, and the fleet load steps past it.

The saved fleet on the trading machine, read once:

```
38 records, all scrumming        count
phantom_timeframes                   0
phantom_timeframe                    0
phantom_config                      38
phantoms_enabled false              38
```

No bot in that fleet runs a phantom. Every record holds the enable flag off.

Three phantom keys reach the record, and no fourth. A saved bot carries `phantoms_enabled`, `lock_candle_count`, and `phantom_timeframes` when the bot holds a name its venue serves. `src/trading/bot_container.py`, in `BotContainer.get_full_state`, writes all three. The Paper and Simulator records carry the same three names, from `wizard_record` in `src/paper/fleet_source.py` and in `src/simulator/fleet_source.py`. A bot built on any of those routes carries no key spelled `phantom_config`.

Three load paths step past the retired key. `src/trading/container/restore.py`, in `StateRestoreMixin.restore_bots_from_state`, builds the live bot. `src/paper/paper_bot_view.py` and `src/simulator/sim_bot_view.py` build the Paper and the Simulator views, each in `config_of`. All three read the stored declaration through `bot_config_kwargs` in `src/trading/container/config.py`, which keeps only the names the declaration holds. A record carrying `phantom_config` builds a bot on each of the three, with its Target Balance intact.

The retirement record holds no entry for it. `_DEPRECATED_KWARGS` in `src/trading/container/config.py` strips a retired name that a caller passes to the bot declaration.

`phantom_config` was a top-level record key and never a field of the declaration. No load path hands it over, so the strip list needs no entry for it. Handed to `make_bot_config` directly the name raises, and a name the strip list does hold builds.

The cursor tick keeps a timestamp. `PhantomBot.tick_for_cursor` stores the cursor itself, not a bucket number. It divides the stored cursor by the candle length on every call, so both bucket numbers come from one divisor.

The cursor tick is gone. `PhantomBot` carries no such method now. No module in the tree called it, and no module passed the simulator flag beside it. The attribute that held the cursor went with it.

One phantom, above the parent. A bot whose record names no timeframe takes one name. The name is the lowest timeframe above the bot's own TA Timeframe that the venue serves. The class constant that held six names is gone.

`src/trading/phantom_balance.py` — `default_phantom_timeframes`

```
above = [
    one
    for one in TIMEFRAME_ORDER
    if one in allowed and is_higher_tf(one, parent_timeframe)
]
return above[:1]
```

Driven on a Coinbase bot with a 1h TA Timeframe, the bot holds 2h. It held five names before, and four of the five sat at or below that 1h parent.

The record keeps every name the bot holds. The save writes the key for any number of names. A bot holding 2h, 6h and 1d writes all three. The fleet load hands all three back, and the phantom manager builds three PhantomBots on them. The save wrote no key at all for that bot before.

step	module	symbol
write the key	`src/trading/bot_container.py`	`BotContainer.get_full_state`
read the key	`src/trading/container/restore.py`	`StateRestoreMixin.restore_bots_from_state`
pick a default	`src/trading/phantom_balance.py`	`default_phantom_timeframes`
build the set	`src/trading/phantom_balance.py`	`PhantomBalanceManager.create_phantom_set`

The eleven names are declared once. `src/exchange/timeframes.py` declares `ALL_TIMEFRAMES`. Five modules read that tuple. Each held its own copy of the same ranked list before, and one module held two copies.

reader	module
the rank order	`src/trading/phantom_balance.py`
the wizard page	`src/gui/main_tabs/bot_wizard_surface.py`
the Settings page	`src/gui/main_tabs/settings_dialog_surface.py`
the Phantom Bots tab	`src/gui/main_tabs/phantom_bots_tab_surface.py`
the Comp column	`src/gui/main_tabs/indicator_panel_surface.py`

A refused phantom is named on screen. The Phantom Bots tab carries six status rows. Two of them name the timeframes no phantom runs on. The first names the timeframes the venue dropped. The second names the timeframes at or below the bot's own TA Timeframe. The Comp column's cell carries that second list in its tool tip.

reading	module	symbol
the venue drop	`src/trading/scrumming_bot.py`	`ScrummingBot._phantom_tf_dropped`
the parent refusal	`src/gui/main_tabs/phantom_bots_tab_surface.py`	`below_parent_timeframes`
the Comp tool tip	`src/gui/main_tabs/indicator_panel_surface.py`	`comp_skipped_tooltip`
the engine record	`src/trading/phantom_balance.py`	`TimeframeCoordinator.get_higher_tf_bias`

A refused change is named too. A line under the timeframe picker names a stored timeframe the venue does not offer, and names a stored timeframe the tab does not list beside the one the picker shows instead. Both builds draw that line. In the React build a second line at the top of the tab names a change the tab would not take and the setting it came from, and it is replaced rather than repeated while the same change keeps arriving. The Qt build moves its own controls and carries no second line. A change the tab takes shows neither line.

reading	module	symbol
the stored timeframe	`src/gui/main_tabs/phantom_bots_tab_surface.py`	`stored_timeframe_notice`
the refused change	`src/gui/main_tabs/phantom_bots_tab_surface.py`	`step_refusal_text`
the one announcement	`src/gui/main_tabs/phantom_bots_tab_surface.py`	`PhantomBotsTabModel.note_steps`
the drawn line	`src/gui/web/phantom_bots_tab.js`	`step-refusal`

The P&L column draws a dash. A phantom is a read-only observer. It runs no order on any path, it holds no position and it counts no cost, so it computes no profit and there is no figure for the column to print. Its answer to the tab carries no profit key, and the cell draws the dash the Comp column draws for a reading it cannot make. Both builds draw that one dash. The column printed `$+0.0000` on every row before, which an operator could not tell from a profit of nothing.

A figure the phantom does report is still printed. A profit of exactly zero prints `$+0.0000`, so the dash stands for an absent reading alone and never hides a real one. A figure of a kind the money admission refuses — a stored flag, a text number, a not-a-number, an endless number — draws the dash as well.

reading	module	symbol
the phantom's answer	`src/trading/phantom_balance.py`	`PhantomBot.get_status`
the profit figure	`src/gui/main_tabs/phantom_bots_tab_surface.py`	`phantom_pnl`
the drawn cell	`src/gui/main_tabs/phantom_bots_tab_surface.py`	`phantom_pnl_text`

reading	module	symbol
the money admission	`src/trading/container/config.py`	`as_finite_float`
the one dash	`src/gui/main_tabs/indicator_panel_surface.py`	`ABSENT_CELL_TEXT`
the Qt cell	`src/gui/live_settings/phantom_bots_tab.py`	`PhantomBotsTabMixin._create_phantom_bots_tab`
the React cell	`src/gui/main_tabs/phantom_bots_tab_surface.py`	`phantom_row`

A phantom answering nothing still draws its row. The per-phantom table reads each phantom through one guarded call. A phantom that raises, and a phantom that hands back nothing at all, both reach the table as an empty reading, and the row draws its absent marks. The table ended the whole tab build on either one before.

reading	module	symbol
the guarded read	`src/gui/main_tabs/phantom_bots_tab_surface.py`	`phantom_status`
the Qt loop	`src/gui/live_settings/phantom_bots_tab.py`	`PhantomBotsTabMixin._create_phantom_bots_tab`
the React loop	`src/gui/main_tabs/phantom_bots_tab_surface.py`	`PhantomBotsTabModel.build`

The window's answer carries every row the tab draws. The Bot Settings window publishes one bot reading that each of its tabs asks for. That reading carried the enable switch, the timeframe list and the lock figure, and the tab read six more names the window never sent. On the running build the started row, the dropped row and the lock-state row drew their built-in words, and neither runtime table drew at all.

One declaration now names each status reading, its bot attribute and the value an absent attribute gets, so a row cannot reach the page alone.

```
BOT_REQUEST_ROW: tuple = (
    (REQUEST_ENABLED, ENABLED_ATTRIBUTE, False),
    (REQUEST_STARTED, STARTED_ATTRIBUTE, False),
    (REQUEST_TIMEFRAMES, TIMEFRAMES_ATTRIBUTE, None),
    (REQUEST_DROPPED, DROPPED_ATTRIBUTE, None),
    (REQUEST_LOCKED, LOCKED_ATTRIBUTE, False),
    (REQUEST_LOCK_TIMEFRAME, LOCK_TIMEFRAME_ATTRIBUTE, DEFAULT_LOCK_TIMEFRAME),
)
```

Driven on two bots differing in every row, the page and the Qt tab now report the same six rows and the same two tables.

```
                          before               after
Phantoms started          NO on both bots      YES then NO
Dropped by the venue      none on both bots    1w then 3m, 6h
SCRUM lock state          UNLOCKED on both     LOCKED then UNLOCKED
per-phantom table rows    0 and 0              1 and 2
active lock table rows    0 and 0              1 and 2
```

reading	module	symbol
the declaration	`src/gui/main_tabs/phantom_bots_tab_surface.py`	`BOT_REQUEST_ROW`
the bot read	`src/gui/main_tabs/phantom_bots_tab_surface.py`	`published_bot`
the per-phantom read	`src/gui/main_tabs/phantom_bots_tab_surface.py`	`phantom_statuses`
the lock read	`src/gui/main_tabs/phantom_bots_tab_surface.py`	`active_locks`
the window's answer	`src/gui/main_tabs/bot_live_settings_surface.py`	`tab_bot`
the page's reader	`src/gui/main_tabs/phantom_bots_tab_surface.py`	`view_model`

Paper and the Simulator read the same answer. Each builds its own window payload by calling Live's builder, so one repair reaches all three.

```
src/gui/paper/paper_bot_live_settings_surface.py, in build_view_model
src/gui/simulator/sim_bot_live_settings_surface.py, in build_view_model
```

A seventh status row names the timeframes the one-phantom limit holds back. Two sentences above are overtaken. Neither is edited and neither is removed. They read:

> *One declaration now names each status reading, its bot attribute and the value an absent attribute gets, so a row cannot reach the page alone.*
>
> *Driven on two bots differing in every row, the page and the Qt tab now report the same six rows and the same two tables.*

The current reading. The declaration still names each reading and a row still cannot reach the page alone. It holds seven entries rather than six, and the seventh is the set-aside row.

```
(REQUEST_SET_ASIDE, SET_ASIDE_ATTRIBUTE, None),
```

The row reads the same on a running bot and on a stored record. A running bot holds the names it set aside in its own attribute; a Paper or Simulator record holds the whole list, and one read on the page derives the same answer from either.

`src/gui/main_tabs/phantom_bots_tab_surface.py` — `configured_timeframes`

```
held = [str(one) for one in (getattr(bot, TIMEFRAMES_ATTRIBUTE, None) or ())]
aside = getattr(bot, SET_ASIDE_ATTRIBUTE, None)
if aside is None:
    return held
extra = [str(one) for one in (aside or ()) if str(one) not in held]
return sorted(held + extra, key=timeframe_order_key)
```

Driven offscreen on all three pages, with a bot given 6h and 1d, reading the strings each row reports:

```
                            Live       Paper      Simulator
the picker                  1d         1d         1d
Configured timeframes:      6h, 1d     6h, 1d     6h, 1d
Set aside, one runs:        6h         6h         6h
```

The phantom's own gate reading. On each pass of its loop, the phantom now runs the bot's two gate chains over the candles it read, at the bot's own settings, and keeps the gate states. This is the gate chain read on the higher timeframe, beside the bot's own read. It decides no trade. The Market Inspector draws both readings in the Phantom Bot HTF Signals zone.

`src/trading/ata_gate_scan.py` — `timeframe_gates`

```
    held = settings if settings is not None else scan_settings(symbol)
    return scan_gates(
        symbol, timeframe, candles, summary, band_reading(candles, held), None, held
    )
```

The six gates that read the position do not run at the phantom read. No timeframe above the phantom's is weighed there.

3 - The Landing Strip

Perhaps the earliest assumption that hit me the most and really presented the catalyzing challenge to creating all of my alternate trading methods, is that you cannot predict what the market will do and after only a short while of actively trading I asked the questions: “But what if I don’t have to?” “What if I decide that I do not have to speculate at all?” I do not remember exactly when I first used the term but I think it was when I was trading A34 and A27 some years ago while participating in a Telegram group with some psytrance friends and associations called the Better Bitcoin Bureau. The name changed a few times but this is the one I recall.

The concept emerged from my persistent use of Heikin-Aishii candles. I had decided on this candle style early just by intuition. I could say it was because I “felt it looked nicer” but that does not quite capture the magnetism. To me, they just structurally make more sense and are less sloppy than traditional candles. What I began to observe over and over was the propensity for crypto markets to form long, tapering structures from these candles and that the likelihood of market reversal could be pinned to 1) the landing strip length and 2) its proximity to the Bollinger Band. Further, I found that those markets that formed these on 1D or higher timeframes would often have explosive breakouts like a volcano that had charged up to blow out. These findings can be historically verified and back tested. I manually performed this observation all the way down to 1 minute trading and found that it was only at this lowest TF that it started to exhibit sub-50% reliability. Given this, the concept has been transposed into Acervator as a core component and trading logic gate with the number of candle strip candles being a user configurable setting.

This is also where I should mention that I am not at all a quant or mathematical trader. I consider myself a visual or tactical trader that focuses on market reversals. I have taught myself to visually interpret price chart structure using a given set of indicators used by countless other technical analysts. When looking at some of the labeling within Acervator, however, one will readily see military references and hints at how I view asset accumulation as something akin to progressive territory capture on a perpetual gameboard. The less skilled traders are surrendering it with every cycle of the market.

Functional. Two detectors build the Landing Strip and both run on every tick. The first places price inside the Bollinger channel and counts trailing Heikin Ashi bodies under a tightness threshold. It reads the operator's control directly.

`src/trading/scrumming_bot.py` — the first detector's call

```
    min_pattern_candles=self.config.bb_landing_strip_candles,
```

Design intention. The second detector is the one that scores the taper. It counts a run of shrinking Heikin Ashi bodies, names the band the run reached, and returns a confidence boost the tick adds to its own. On a tapering rally into the upper band it answers detected, side upper, length three, boost 0.1437. The same tape without the taper answers not detected, and a five-candle tape answers not detected. A detected strip can force a direction, which is why the gate vocabulary marks it an override rather than a gate.

`src/trading/indicators/landing_strip.py` — `detect_landing_strip_v2`

```
def detect_landing_strip_v2(
    candles: list[Candle],
    min_consecutive: int = 3,
    shrink_threshold: float = 0.90,
    bb_tolerance_pct: float = 3.0,
    use_ha: bool = True,
    bb_period: int = 20,
    bb_std: float = 2.0,
) -> TighteningResult:
```

The second detector does not read the setting. The tick hands it a literal three, so the wizard's Landing Strip Candles control moves the first detector and leaves the second where it is. Both defaults sit at three, so the two agree until the operator changes the control.

`src/trading/scrumming_bot.py` — the second detector's call

```
tightening = detect_landing_strip_v2(
    candles,
    min_consecutive=3,
    shrink_threshold=0.90,
    bb_tolerance_pct=3.0,
)
```

Issue #432 carries it.

The issue #432 clause above is overtaken. It is not edited and not removed. It read:

> *The tick hands it a literal three, so the wizard's Landing Strip Candles control moves the first detector and leaves the second where it is. Issue #432 carries it.*

Issue #432 closed on 2026-10-23. The literal three is a named constant now, and the call states why the two counts are different things. `min_consecutive` counts shrinking Heikin Ashi bodies; the wizard's control counts the newest small-body candles the band-proximity check needs. The control is not meant to move the second detector.

```
src/trading/ata_gate_scan.py   TIGHTENING_MIN_CANDLES = 25
src/trading/ata_gate_scan.py   TIGHTENING_MIN_CONSECUTIVE = 3
src/trading/ata_gate_scan.py   TIGHTENING_SHRINK_THRESHOLD = 0.90
src/trading/ata_gate_scan.py   TIGHTENING_TOLERANCE_PCT = 3.0

src/trading/container/config.py, in BotConfig   bb_landing_strip_candles: int = 3
src/trading/scrumming_bot.py                    the call passes the four constants
```

Current reading, when a strip sets the side. A detected strip still names a side, but it sets the side of the vote only when three indicators agree. The vote must top what the two heaviest voters reach at full confidence, at the bot's weights on that tick. A bot with every weight at zero has no line, so its strip sets the side as before. Live, the Market Inspector scan, Paper and the Simulator all call the one function below.

`src/trading/ta_engine.py` — `vote_sides`

```
strip = band.landing_strip_side if band is not None and band.landing_strip else ""
if strip and not three_agree(confidence, signals):
    strip = ""
```

Driven on a stand-in exchange at a lower strip, with the twelve published weights. Two agreeing indicators read 0.1966 and the fold waited. Three read 0.2821 and the fold went through. The code before this change, taken from git, folded on both.

4 - Bollinger Band Travel Detection

Allowing a strategy to “see” where it is in relation to local width of the Bollinger Band is another aspect of treating the immediate market as a region on a map through which a stack of tokens is traveling. Along the route, we are able to offload surplus at the peaks and pick up additional supplies in the next valley. This is used in combination with Minimum Opposing Trade Distance. The complete question then becomes “Has the market traveled 70% (default), is current price X% distant from the most recent trade event, and is Target Delta not zero? When combining these gates with the (Technical Analysis) Indicator Voting Panel and Landing Strip Detection, you have a full market structure detection suite that is able to incrementally adjust a position’s size and generate recurring profits or growth with each market cycle.

Functional. The tick measures how far price has moved since the last trade and divides that by the current band width. When the fraction reaches the operator's percentage and the target delta is positive, the travel flag is set. The default is seventy percent and the wizard offers zero to a hundred. That is the compound question the entry states, and the code asks it in one expression.

`src/trading/scrumming_bot.py` — `ScrummingBot.tick`

```
_bb_width = max(bb_result.upper - bb_result.lower, 1e-12)
band_travel_frac = abs(ticker.last - self._last_trade_price) / _bb_width
if (
    abs(ticker.last - self._last_trade_price)
    >= self.config.band_travel_pct / 100.0 * _bb_width
    and delta > 0
):
    band_travel_triggered = True
```

Design intention. A fixed percentage means two different things in a quiet market and a violent one. Measuring the move against the band width makes the threshold follow the volatility instead. The flag then clears a standing trend hold and lets the harvest through. Travel is no gate of its own: the ten scrum labels carry no travel light.

`src/trading/scrumming_bot.py` — what the flag releases

```
trend_override = abs(delta) >= _interval_usd * 2.0 or band_travel_triggered
```

The clause above saying travel is no gate of its own is overtaken. It is not edited and not removed. It read:

> *Travel is no gate of its own: the ten scrum labels carry no travel light.*

The labels still carry no travel light. Travel is now a term inside Minimum Opposing Trade Distance, which is the combination the entry above names. `src/trading/otd_math.py`, in `band_extension_pct`, turns `band_travel_pct` and the band span into a percentage of the band midline, and `minimum_opposing_trade_distance_pct` adds that percentage to the interval and the fee. Zero keeps the term absent.

```
span = float(upper) - float(lower)
price = float(reference_price)
travel = float(travel_pct)
if travel <= 0.0 or span <= 0.0 or price <= 0.0:
    return BAND_EXTENSION_ABSENT_PCT
return span * (travel / 100.0) / price * 100.0
```

Current reading. The extension still adds to the interval and the fee. Where the interval and the extension together sit under one fee, the distance is two fees, so a narrow band cannot pull it under the round trip.

`src/trading/otd_math.py` — `DistanceSettings.distance_pct`

```
return minimum_opposing_trade_distance_pct(
    self.interval_pct,
    self.fee_pct,
    band_extension_pct(
        band_upper, band_lower, self.band_travel_pct, reference_price
    ),
)
```

5 - Position-Aware Technical Analysis

Traditional TA treats a signal (e.g., RSI > 70) as context-independent. Position-Aware TA encodes the current Bollinger Band position as context: RSI > 70 near the upper band confirms overbought (sell bias), but RSI > 70 near the lower band is a failed oversold — a trap for mean-reversion buyers. The system applies this inversion systematically across all oscillator signals. In other words, it attempts to strengthen the price to Bollinger Band relationship by reading additional indicators with relation to it. If you have the landing strip, band travel, opposing trade distance, and a bullish or bearish confidence score from the Indicator Voting Panel, it is likely a trading event will occur. In the end, many of these functions were layered in one after the other until I saw the platform take most trades that I would myself manually execute most of the time.

Functional. The band context enters at the bot, not inside the voters. The tick reads the band position once, marks the upper quarter and the middle band, and accumulates a skew from the votes that arrive. A bullish vortex adds 0.12 at the upper band and takes 0.05 back in the middle. A bullish MACD adds 0.08 and takes 0.03 on the same test. Ichimoku and Stochastic RSI move the skew on direction alone, with no band term at all.

`src/trading/scrumming_bot.py` — `ScrummingBot.tick`

```
at_upper_bb = bb_pos > 0.75
in_bb_middle = 0.35 <= bb_pos <= 0.65

if (
    sig.indicator == "vortex"
    and sig.direction == SignalDirection.BULLISH
):
    if at_upper_bb and sig.confidence > 0.5:
        position_boost += 0.12
    elif in_bb_middle:
        position_boost -= 0.05
```

Design intention. The skew relaxes or tightens the confidence floor the vote has to clear. A floor of 0.30 becomes 0.2679 at a skew of plus 0.12 and 0.3158 at a skew of minus 0.05. A skew of minus one returns infinity, which no confidence can clear, so a fully contradicted reading is refused outright rather than discounted.

`src/trading/scrumming_bot.py` — the two terms the floor reads

```
_ta_conf_skew = position_boost + bb_confidence_boost
```

The inversion does not reach the oscillator modules. Each indicator computes from candles alone, and the band position appears in six indicator modules only: Bollinger, band proximity, landing strip, M top, spring and W bottom. Neither the RSI module nor the Stochastic RSI module takes a band argument. Today the band context re-weights the vote after the fact. It does not re-read the oscillator.

Current reading, how far the skew relaxes the floor. The skew now relaxes the floor no further than the vote's own two-indicator line. The line is the confidence the two heaviest voters reach at full confidence, with every other voter at none, at the bot's weights on that tick. An eased floor stays above the line and never rises above 0.25. At equal weights the line is 0.1667, and at the published weights it is 0.1966. Where two indicators already reach 0.25, through voters that abstain or uneven weights, the favours ease nothing.

`src/trading/ta_engine.py` — `two_indicator_ceiling`

```
    heaviest = sorted(
        (sig for sig in signals if not sig.abstained),
        key=lambda sig: sig.weight,
        reverse=True,
    )[:2]
    tally = [
        (
            replace(sig, direction=SignalDirection.BULLISH, confidence=1.0)
            if any(sig is one for one in heaviest)
            else replace(sig, direction=SignalDirection.NEUTRAL, confidence=0.0)
        )
        for sig in signals
    ]
    return VotingEngine._aggregate(tally, "").consensus_confidence
```

Current reading, the line at the plain floor. The line now holds at every floor, the plain 0.25 included, so a floor can rise over 0.25. Where two indicators already reach 0.25, through voters that abstain or weights that lean on two, the floor sits just above what they reach. At equal weights that starts when four of the twelve abstain. At the published weights it starts when the three lightest abstain. A bot with every weight at zero reads a line of 0.0, so its floor does not move.

`src/trading/scrumming_bot.py` — `_skewed_confidence_floor`

```
    denominator = 1.0 + skew
    if denominator <= 0.0:
        return math.inf
    eased = floor if math.isnan(denominator) else floor / denominator
    return three_indicator_floor(eased, signals)
```

6 - Market Inspector

This addition was, like the Phantom Bot above, a move towards making Acervator “hyper-vigilant” of hypothetical opportunities that take the form of oppositional trades (assets that move inversely to one another) and bot swarm network topologies. The bot swarm itself will be covered in a moment but basically this feature allows for the creation of custom capital reinforcement networks where profits are configured to flow between positions in a manner that can potentially strengthen a portfolio faster than just focusing on single, isolated bot instances.

The Market Inspector is designed to analyze the broader market for groups of assets from which to build topologies and identify non-active assets that could be traded in opposition to active positions. Its findings can be pushed to the Simulator for historical back testing, to the Paper Trader for live market and real time validation, and, of course, Live Trading should the user choose to do so.

Functional. The Inspector walks daily, weekly and monthly candles, scores each market, and keeps the result. It then pairs a long-signal market with a short-signal market when the correlation of their returns over a rolling window sits inside a negative window. Those are the oppositional trades the entry describes, and the ranking puts the most negative correlation first.

`src/trading/market_inspector.py` — `MarketInspector._find_opposing_pairs`

```
corr = self._pearson(r_l[-m:], r_s[-m:])
if self._corr_min <= corr <= self._corr_max:
    pairs.append(
        OpposingPair(long_side=lo, short_side=sh, correlation_30d=corr)
    )
```

Design intention. The findings are meant to leave the screen. A separate module turns those signals into swarm proposals with the bots and the wires already written, and one push target exists today: the Simulator asks the Inspector for its current proposals at build time.

`src/gui/main_tabs/simulator_tab.py — _build_simulator_tab` The Simulator rebuild removed this file; it is not in the tree. The file is back in the tree: `src/gui/main_tabs/simulator_tab.py, in SimulatorTabMixin._build_simulator_tab` seats the Sim tab, or the note naming a caught build. The block below is not in it: `set_topology_getter` is in no file, so nothing wires a getter at build time. The Inspector still answers `src/gui/main_tabs/market_inspector_surface.py, in current_topology_proposals`, and `src/gui/main_window.py, in _push_candidate_to_sim` is the push the operator presses.

```
if hasattr(self._simulator, "set_topology_getter"):
    self._simulator.set_topology_getter(
        lambda: (
            self._market_inspector.current_topology_proposals()
            if hasattr(
                getattr(self, "_market_inspector", None),
                "current_topology_proposals",
            )
            else None
        )
    )
```

No Paper Trader exists to push to; entry 17 gives the measurement. Issue #23 tracks the tab's build-out and the topology and oppositional pushes, and issue #290 a live connector defect on Refresh.

The issue #290 clause above is overtaken. It is not edited and not removed. It read:

> *and issue #290 a live connector defect on Refresh.*

Issue #290 closed on 2026-10-17. The Inspector holds no connector. The window hands it a getter, and Refresh calls that getter and copies what it answers, so a wired source is read fresh on each press rather than kept from the build.

```
src/gui/main_tabs/market_inspector_tab.py, in _build_market_inspector_tab
    set_exchange_source(connectors_getter=..., scheduler=...)

src/gui/market_inspector.py, in set_exchange_source   stores the getter
src/gui/market_inspector.py                           dict(self._connectors_getter() or {})
```

7 - Bot Swarm and Smart Wire Network w/ Provenance Tracking

The Bot Swarm, which is how I presented the concept to Claude, is my version of a capital reinforcement network. At the Exchange level (swarms cannot be connected across multi-API boundaries) profit from individual bots can be routed to others with such outputs being termed Wire Credits since they are sent and received along Smart Wires which connect the bots. Each such wire can have its own flow rate. The network nodes are designed to protect a given bot from excessive outflows which would cripple its own growth capacity. The other piece of this is that Wire Credits are Provenance tracked which allows outflows to eventually start sending profits back to the originating bot with the over-arcing philosophy being long-term investment with accumulation and position strengthening rather than just sitting around waiting for a good outcome that might not arrive.

Now: only a fold spends wire credits, under the Maximum Target Growth Per Cycle, and a tranche whose own sale dollars are spent or sold ends and returns its credit to the pool for a later tranche (`src/trading/scrumming/wire_routing.py, in _return_wire_credits_to_pool`).

In combination with the Market Inspector, the Bot Swarm can receive and deploy groups of pre-configured and pre-connected bot clusters that immediately begin participating in the market. This layer, along with future portfolio-level algorithms, will enable the fully automated and persistent execution of complex trading strategies from a single terminal window.

Functional. The wire manager holds the topology and one ledger for each bot. A wire records a source, a target and a flow percentage, and a realised fold moves the share along it. The outflow is capped so a funder keeps its own growth capacity: a ten dollar scrum profit against a hundred dollar target releases ninety percent, and a zero profit releases nothing.

`src/trading/smart_wire.py` — `compute_safe_outflow_pct`

```
if scrum_profit_usd <= 0:
    return 0.0

if band_upper <= band_lower:
    distance_to_fold_pct = 0.0
else:
    distance_to_fold_pct = max(
        0.0, min(1.0, (current_price - band_lower) / (band_upper - band_lower))
    )
```

Design intention. Provenance is what makes the network more than a pipe. Each credit that lands on a tranche keeps the id of the bot that supplied it, so an outflow can eventually route profit back to the bot that funded it. The detail list is bounded at twenty entries per tranche and everything older folds into a rolled total, which keeps the sums exact without letting the record grow without limit.

`src/trading/scrumming/wire_routing.py` — `_WIRE_CREDIT_CAP`

```
_WIRE_CREDIT_CAP = 20
if not isinstance(wc, list) or len(wc) <= _WIRE_CREDIT_CAP:
    return 0
overflow = wc[:-_WIRE_CREDIT_CAP]
del wc[:-_WIRE_CREDIT_CAP]
```

How much of an arriving inflow a bot stacks onto an open position is its own setting, and it defaults to one percent.

`src/trading/container/config.py` — `BotConfig.wire_inflow_stack_pct`

```
  wire_inflow_stack_pct: float = 1.0
```

That setting is removed. An arriving inflow is added to the tranches the receiving bot already holds, or parked until a sell opens one, and no figure sizes it.

```
  if self._fold_tranches:
      share = self._spread_wire_usd_over_fold_queue(u, [credit])
```

Two mechanisms move capital between bots, and both are called throughout. The fold distribution sends a share of each realised fold to every wired target, and the scrum routing sends a share of each sale's proceeds along the same wires. No mechanism funds a new bot out of a parent's mature profit. Deploying a pre-connected cluster from the Market Inspector is an intention.

Now: the scrum routing sends each wire its share of a sale's surplus alone, the Scrum Fold Ratio's share of what the sale leaves past the dollars that buy back every unit it sold, less the room the growth cap leaves, and a detonation sends nothing (`src/trading/scrumming/wire_routing.py, in _route_scrum_proceeds_via_wires`).

8 - Hunger and Satiety Indices

These elements were implemented as part of an effort to boost accumulation and scrumming performance in longer term bearish or bullish conditions that might not latch all logic gates to trigger a fold or scrum despite the Target Delta not being zero. This piece adds a factor to the bot that makes it more aggressively evaluate subsequent candles and will increase its likelihood of executing the re-zeroing action. This allows for trades to be executed in the hypothetical ‘dead zone’ between Minimum Opposing Trade Distance and the lower or upper Bollinger Band extent if the candle structure begins to float or cruise within it for too long.

Built, and switched off on every bot. Hunger is a count of the candles that pass while the position is off target and nothing fills. Satiety sets the count back to zero. The count becomes a skew on the TA confidence floor, so after a long idle stretch a weaker vote clears the floor.

Each bot carries its own two figures. The gain is zero on every bot, so no bot trades differently today. No screen sets the gain, and no measurement sets its figure.

Now: the gain is the 0.20 ceiling over the pace typed in each live Scrumming bot's Hunger Pace box, and none until its Hunger box is ticked (`src/trading/container/config.py, in BotConfig.hunger_gain_per_candle`).

`src/trading/container/config.py` — `BotConfig`

```
      # Floor skew each idle candle off target adds; 0.0 is off.
      hunger_gain_per_candle: float = 0.0
      # The most hunger_gain_per_candle may add to the floor skew.
      hunger_skew_max: float = 0.20
```

The tick adds one when the newest candle of the bot's TA timeframe carries a newer timestamp, the rule the circuit breaker uses for its cooldown. A hard circuit-breaker trip stops the count. A fill sets it to zero, and so does a tick inside the at-target dust band. The count lives in memory, so a restart starts it again at zero.

`src/trading/scrumming_bot.py` — `ScrummingBot._count_hunger_candle`

```
        if not math.isfinite(stamp) or stamp <= self._hunger_candle_ts:
            return
        anchored = self._hunger_candle_ts > 0.0
        self._hunger_candle_ts = stamp
        if not anchored:
            return
        before = self._hunger_floor_skew()
        self._hunger_candles += 1
```

Every fill the bot books reaches its clock figures once, so satiety sits there.

`src/trading/scrumming_bot.py` — `ScrummingBot._write_clock_figures`

```
    def _write_clock_figures(
        self, fill: SettledFill, order_id: str, filled: float, rests: bool
    ) -> None:
        """Run ``_sate_hunger`` for ``fill``, then write the mixin's clock figures."""
        self._sate_hunger(f"a {fill.type} fill")
        super()._write_clock_figures(fill, order_id, filled, rests)
```

The skew is the count times the gain, held at the ceiling. It joins the favours the floor already divides by, so the floor is 0.25 divided by one plus their sum. At the 0.20 ceiling alone the floor is 0.2083.

`src/trading/scrumming_bot.py` — `_hunger_skew`

```
def _hunger_skew(idle_candles: int, gain_per_candle: object, skew_max: object) -> float:
    """Return ``idle_candles`` times ``gain_per_candle``, capped at ``skew_max``.

    ``gain_per_candle`` and ``skew_max`` must be finite and above zero, else it is 0.0.
    """
    gain = as_finite_float(gain_per_candle)
    ceiling = as_finite_float(skew_max)
    if gain is None or ceiling is None or gain <= 0.0 or ceiling <= 0.0:
        return 0.0
    return min(ceiling, idle_candles * gain)
```

`src/trading/scrumming_bot.py` — `ScrummingBot.tick`

```
        _ta_conf_skew = position_boost + bb_confidence_boost
        if hunger_skew:
            _ta_conf_skew += hunger_skew
        if _bb_priority_arm:
            _ta_conf_skew += _BB_PRIORITY_SKEW
        _eff_conf_floor = _skewed_confidence_floor(_ta_conf_skew)
```

A lower floor changes three decisions and leaves a fourth alone. It releases a fold that every other gate passes and the TA confidence alone holds. That happens at the lower band while the delta is still inside the interval, where the deep-fold override does not reach. It lets a DIST sale through sooner. It refuses the hedge top-up more often, because that buy waits for a vote that is not bullish. It changes no scrum, because the ripe-harvest override already passes the TA gate wherever the band lets a scrum through. The band detect lines do not move, so hunger opens no trade short of them.

The bot log names hunger while it acts. A HUNGER line gives the count and the floor it sets alone, and a SATIETY line names the fill or the return to target that ended it.

Current reading, the DIST sale. The sentence above that hunger lets a DIST sale through sooner holds for the vote alone. A DIST sale now also waits for a price. Each fold that buys more units than its scrum sold queues those extra units beside the price it paid. They sell only at that price raised by the Minimum Opposing Trade Distance, or higher. Hunger can pass the vote sooner, and it does not move that price.

`src/trading/scrumming/tick_phases.py` — `TickPhaseMixin._tick_distribute`

```
clearing = dist_sets_clearing(self._dist_sets, price, otd_pct)
if not clearing:
    self._hold_dist_sets(price, otd_pct)
    return
```

Current reading, what hunger reaches and when it counts. Three statements in the readings above were wrong. "It changes no scrum" fails when the lower band sits at or under zero. The ripe-harvest override does not reach there, so the vote decides that scrum, and hunger can pass it. "Hunger opens no trade short of them" failed through the DIST sale, which has no band gate. The first reading counts "the candles that pass while the position is off target and nothing fills". That count ran inside the interval and while the Minimum Opposing Trade Distance blocked, and his entry names neither.

`src/trading/scrumming_bot.py` — `ScrummingBot.tick`, where the ripe-harvest override needs a lower band above zero

```
if (
    bb_result is not None
    and bb_result.upper > 0
    and bb_result.lower > 0
    and ticker.last > 0
):
    if delta > 0 and not below_interval and bb_pos >= _bb_upper_dt:
        _ripe_scrum = True
```

Hunger now reaches the scrum and fold chains only. The DIST sale and the hedge top-up read the same favours without hunger. So hunger no longer passes a DIST sale, and it no longer holds a hedge top-up. Both chains keep the band detect lines, and no override lifts them, so hunger now opens no trade short of those lines.

`src/trading/scrumming_bot.py` — `ScrummingBot.tick`

```
_ta_conf_skew = _favour_skew(
    position_boost,
    bb_confidence_boost,
    hunger_skew,
    priority_arm=_bb_priority_arm,
)
_eff_conf_floor = _skewed_confidence_floor(_ta_conf_skew, summary.signals)
_floor_no_hunger = _skewed_confidence_floor(
    _favour_skew(
        position_boost, bb_confidence_boost, 0.0, priority_arm=_bb_priority_arm
    ),
    summary.signals,
)
```

The count follows his entry. A new candle adds one only in the dead zone: the delta at the interval or past it, the opposing trade distance clear on its side, and the price still inside the band on that side. A candle inside the interval, short of the distance, or past the band adds nothing. It does not end the count either; a fill or a return to target still does that.

`src/trading/scrumming_bot.py` — `_in_hunger_zone`

```
if not distance_clear or band is None:
    return False
if delta > 0:
    return price < band.upper
if delta < 0:
    return price > band.lower
return False
```

No favour, hunger among them, now eases the floor to a level two indicators can reach. The floor stays above what the two heaviest voters reach at full confidence, at the bot's weights on that tick, and never rises above 0.25. At the published weights two indicators reach 0.1966, so the 0.20 ceiling alone still gives 0.2083.

Now: the floor stays just above what two indicators reach at every skew, no skew included, so it rises over 0.25 wherever two indicators reach more (`src/trading/scrumming_bot.py, in _skewed_confidence_floor`).

`src/trading/scrumming_bot.py` — `_skewed_confidence_floor`

```
if denominator <= 1.0:
    return floor / denominator
past_two = math.nextafter(two_indicator_ceiling(signals), math.inf)
return min(floor, max(floor / denominator, past_two))
```

9 - Accumulation Shadows

The Accumulation Shadow was invented as a means of taking maximum advantage of long term and bearish conditions. It allows for a Scrumming to create a Shadow of itself that focuses on accumulating a smaller chunk of the target asset during a bearish regime. The shadow is allowed to accumulate downward but does not scrum. It was a bullish regime to be dedicated and then detonates thereby claiming all of its profits at once. Shadows operate in parallel with their parent bots and will be listed as Shadow Tranches once they are fully implemented.

Not built. The Accumulation Shadow has no code. No shadow bot, shadow tranche or shadow regime exists today.

Three functions collapse or remove a tranche, and none of them knows a shadow kind. One merges two rungs, one removes an aged row, and one discards a queue. Despawn removes a tranche; it does not delist one. The entry's own wording, "once they are fully implemented", matches the code.

`src/trading/stack_math.py` — `_apply_merge_rule`

```
def _apply_merge_rule(tranches: list[Tranche]) -> list[Tranche]:
    """Combine adjacent tranches whose prices are within MERGE_THRESHOLD.

    A merged Tranche keeps the higher `price` and the summed `size`, and the
    survivors are re-indexed from 0.
    """
```

The build this specifies. A shadow is a tranche kind, not a second bot. It accumulates downward, never sells, and exits in one move when the regime turns. The tranche dict gains a kind key, the three collapse functions skip a row that carries it, and the exit reuses the detonation executor that already sells a surplus and reseeds the lots.

Proposed, not present, in `src/trading/scrumming/fold_tranches.py`*:*

```
def _despawn_aged_tranches(self, now: Optional[float] = None) -> dict:
    for t in list(self._fold_tranches):
        if t.get("kind") == "shadow":
            continue
```

A row with no kind key behaves exactly as it does today, so the change is invisible until a shadow is spawned. Issue #433 carries the build.

10 - Charge Up Permission Gate

This is client-side trade batching. It operates in conjunction with the Fold and Stack Tranches (explained later) to allow these individual positions to be combined into a single trade action in response to a given market trend.

Not built. The Charge Up Permission Gate has no code today. Fold and Stack Tranches do run, and entry 11 anchors them. Nothing combines them into one batched order.

The build this specifies. The fold chain already carries a gate that refuses when no tranche is queued, and the context it reads already counts them. Charge Up is the same test with a threshold: hold the fold until enough tranches are eligible, then release them together. It is a new gate class in the existing chain, and it needs no

new context field.

Proposed, not present, in `src/trading/gate_chain.py`*:*

```
class ChargeUpGate(Gate):
    """FOLD-only: hold until ``min_tranches`` rows are queued, then release."""

    name = "charge_up"
    side = "fold"

    def __init__(self, min_tranches: int = 1) -> None:
        self.min_tranches = int(min_tranches)

    def evaluate(self, ctx: GateContext) -> GateResult:
        if ctx.n_fold_tranches >= self.min_tranches:
            return GateResult(passed=True)
        return GateResult(passed=False, blocker_message="charging")
```

The gate enters the fold chain before the hysteresis gate, and the gate vocabulary gains one fold label, taking the row from nineteen lights to twenty. A threshold of one reproduces present behaviour exactly. Issue #433 carries the build.

11 - Fold, Stack, Extractor, and Shadow Tranches / Provenance Queue

The term tranche emerged in Acervator as the concept of placing soft boundaries around where an opposing trade 1) can initially happen in terms of price level, 2) denote the amount of the position expected to be transacted once the price thresholds are met or exceeded, 3) allows wire credits to be fed into Fold Tranches (Buys) and Extractor Tranche profits to be distributed across Stack Tranches (Scrums / Sells) for additional accumulation and profit optimization. For the sake of clarity, Fold Tranches spawn after a Scrum (Sell) occurs; Stack Tranches spawn after a Fold (Buy) occurs; and Extractor Tranches spawn when an Extractor Bot finds a favorable Alternate Currency entry on its chosen timeframe and these tranches remain until the acquired Alternate Currency stack is sold by either the Extractor Bot (Sibling) or its corresponding Scrumming Bot (Parent). All Tranches will persist under a bot’s Details > Tranches Tab.

Functional. Three of the four tranche kinds run. Fold Tranches spawn on a scrum and hold the price of the sale that made them. Stack Tranches spawn on a fold and are priced as a ladder above the buy. A three-unit scrum at 100 with a one percent split and linear spacing returns rungs at 101, 102 and 103 carrying one unit each. Two rungs closer than a tenth of a percent merge, so 100, 100.05 and 105 come back as 100.05 carrying two units, then 105 carrying one.

`src/trading/stack_math.py` — `split_scrum_into_tranches`

```
def split_scrum_into_tranches(
    scrum_price: float,
    scrum_size: float,
    n_target: int,
    split_distance_pct: float = DEFAULT_INITIAL_GAP_PCT,
    spacing_mode: str = DEFAULT_SPACING_MODE,
    min_opposing_pct: float = 0.0,
    min_order_size: float = 0.0,
) -> list[Tranche]:
```

Design intention. Extractor Tranches are the sibling-and-parent relationship the entry describes. The Extractor opens a position on an alternate pair, names the tranche with an id that survives a restart, and hands the base currency back to the parent Scrumming Bot on exit. The parent raises its own target by the amount that arrived rather than selling it straight back out as surplus.

`src/trading/extractor_bot.py` — `ExtractorBot.tranche_id_for_position`

```
"""Build the stable Extractor Tranche id: ``bot_id|pair|opened_at``.

The row emitter and `set_tranche_arbiter` both format
`opened_at` through this one method, so they can never format
it differently and disagree on the same id.
"""
```

Shadow Tranches do not exist; entry 9 gives the measurement and the proposal.

The queue sorts the fold tranches by their reference price, highest first, and drops any row whose reference or unit count is not a finite number. Wire credits ride on the tranche, which is the provenance the entry names.

Three defects stand against the Tranches tab and the rows it draws: spawned tranches do not match the scrum that spawned them (issue #367), the list is not visible and lifetime counters reset (issue #133), and the Units column prints a quantity that does not exist (issue #201). One more sits behind them. The capital reservation registry answers before a sale is placed, and of the five bot construction sites only the two Simulator controllers hand the bot a registry (issue #427).

The sentence above is overtaken. It is not edited and not removed. It read:

> *The capital reservation registry answers before a sale is placed, and of the five bot construction sites only the two Simulator controllers hand the bot a registry (issue #427).*

Three sites build a bot, and none of them hands it a registry. The constructor accepts one and every caller leaves it unset, so each bot resolves the process-wide registry instead.

```
src/gui/live_bot_window.py, in _run_bot                   builds a bot, passes no registry
src/gui/main_window.py, in _create_bot                    builds a bot, passes no registry
src/trading/container/restore.py, in restore_bots_from_state  builds a bot, passes no registry
src/trading/scrumming_bot.py, in ScrummingBot.__init__     capital_registry defaults to None
```

The engine already records the consequence at `src/trading/container/registry.py, in register`, which states that the setter has no caller. A sale is therefore admitted against the process-wide table rather than an injected one.

The first anchor in the block above named a line that had moved. The bot it means is built in `src/gui/live_bot_window.py`, in `_run_bot`, and that is the anchor the block now carries.

The issue #367 clause above is overtaken. It is not edited and not removed. It read:

> *spawned tranches do not match the scrum that spawned them (issue #367)*

Issue #367 closed on 2026-10-03. A spawned tranche carries the sale that made it. The sell loop writes the fill price onto each new record and carries the lot's own buy price with it, and the bounding pass then reads the sale's own net rate, which is the dollars the sale left with this bot divided by the units it sold.

```
src/trading/scrumming/execution.py
    each new record carries   usd, units, ref (the fill price), initial_buy_price

src/trading/scrumming/fold_tranches.py, in _bound_new_fold_tranches
    reads   the slice this sale appended, scrum_usd and scrum_asset
    rate    scrum_usd / scrum_asset, the sale's own net rate
```

The issue #133 clause above is overtaken. It is not edited and not removed. It read:

> *the list is not visible and lifetime counters reset (issue #133)*

The list is visible. A reading over a fleet shaped like the saved one drew every row. A queue of one tranche draws one row and a queue of twenty-three draws twenty-three, each row thirty pixels tall, each carrying its own Fire button, none hidden. `src/gui/live_settings/fold_tranches_tab.py`, in `_create_fold_tranches_tab`, sets the row count from the fold rows plus the Extractor rows and fills each one.

No code resets the lifetime counters. The only writer of the reset stamp is `src/trading/scrumming/fold_tranches.py`, in `clear_lifetime_tranche_counters`, and its only callers are the Clear Lifetime Counters control on each Fold Tranches tab. One lifetime figure does fall without a control: `src/trading/scrumming/fold_tranches.py`, in `_bound_new_fold_tranches`, lowers the opened count each time one sell's per-lot records merge into one record, which the Lifetime tranches opened tooltip states.

The issue #201 clause above is overtaken. It is not edited and not removed. It read:

> *the Units column prints a quantity that does not exist (issue #201)*

Every figure the Fold Tranches table draws from a saved row passes one admission rule first. The rule takes a value only when it is exactly a whole number or a decimal number and finite, so it refuses a stored true-or-false value, a not-a-number, an endless value, text, and a whole number too large to hold. A refused figure draws an em dash, never a quantity, a price or an age.

The Extractor row's Age, Units and marked value read through that rule in `src/gui/live_settings/fold_tokens.py`, in `_compose_extractor_tranche_cells`, and its tooltip reads the leased amount and the alt holding the same way in `_compose_extractor_tranche_tooltip`. The other build draws the same row from `src/gui/main_tabs/fold_tranches_tab_surface.py`, in `extractor_row_cells` and in `guarded_extractor_units`.

The Clear fold tranches confirmation states the queued cash and the queued units before the queue is discarded. Both figures are totalled by `src/gui/live_settings/fold_tokens.py`, in

`fold_queue_figure_total`, and by `src/gui/main_tabs/fold_tranches_tab_surface.py`, in `queue_figure_total`. A total is reported only when every queued row is readable; otherwise the figure draws an em dash rather than a partial sum offered as the whole queue. A missing key counts as nothing, which is how the table's own cells read it.

The same confirmation warns him when the bot also holds parked Smart Wire credit, because clearing the queue opens the absorb window. That figure reads through the same rule in `src/gui/live_settings/fold_tokens.py`, in `fold_parked_credit`, and in `src/gui/main_tabs/fold_tranches_tab_surface.py`, in `FoldTranchesTabModel.parked`. A credit that cannot be read counts as a credit that stands: `fold_parked_credit_stands` and `parked_credit_stands` keep the warning drawn and the Clear Wire control live, and the amount alone draws an em dash. A credit of nothing draws no warning and leaves that control off.

What the panel draws for a count it cannot work out

The cycle close ratio is the closed count over the opened count less the discarded count. A merge lowers the opened count while the discarded count holds, so the top of that sum can pass its bottom. `src/gui/live_settings/fold_tokens.py`, in `compose_cycle_close_ratio`, draws the dash for that pair and names the two counts beside it, never a share above one hundred percent. `src/gui/main_tabs/fold_tranches_tab_surface.py`, in `cycle_close_ratio`, draws the same thing for the React page.

How the ratio cell picks its colour

Red under a half, amber under four fifths, green above, and no colour at all while fewer than five tranches are openable. The allotment cell above it draws red when the queue claims more units than the bot holds.

The cell prints the share and decides the colour on the counts themselves. The share is worked out for the text alone, so the division has no part in the verdict.

`src/gui/main_tabs/fold_tranches_tab_surface.py`, in `cycle_close_ratio`

```
  if int(closed) < denominator * RATIO_RED_BELOW:
      return [text, FOLD_RATIO_RED_FG_HEX]
  if int(closed) < denominator * RATIO_AMBER_BELOW:
      return [text, FOLD_RATIO_AMBER_FG_HEX]
```

Both ways of asking give the same answer on every count either cell can be given. Swept over 1,399,894 whole-number pairs, with denominators up to one hundred thousand and every closed count within three of each threshold, no cell changes colour; 9,396 allotment readings stepped by the smallest amount a stored figure can move give the same result. The Fold Tranches screen on the Electron build, the Stack Tranches screen beside it and the Qt panel all draw the same colour for the same counts.

The Qt panel reads the same three names

The Qt Fold Tranches panel wrote the three figures in as plain numbers, so the arithmetic check could not read any of its three colour decisions. It now reads the names the Electron page declares, and it decides on the counts the way that page does.

The names are declared in `src/gui/main_tabs/fold_tokens_surface.py`.

`src/gui/live_settings/fold_tokens.py`, in `compose_cycle_close_ratio`

```
if denominator < RATIO_MIN_DENOMINATOR:
    return (text, None)
if int(closed) < denominator * RATIO_RED_BELOW:
    return (text, FOLD_RATIO_RED_FG_HEX)
if int(closed) < denominator * RATIO_AMBER_BELOW:
    return (text, FOLD_RATIO_AMBER_FG_HEX)
```

Every colour is what it was. Swept over 131,302 whole-number pairs, with denominators from one to two hundred and three larger ones up to one hundred thousand, and every closed count from nothing up to the denominator, the panel draws the same colour on every pair and agrees with the Electron page on all 131,302 of them. Both counts are whole numbers, so a step of one is the finest step there is and no colour change can fall between two readings.

Moving the three declarations moved 77,340 of the 131,302 colours, on both sides together. Loosening the amber figure by a thousandth moved 151 of them. The same two moves against the old panel moved nothing on the Qt side, which left it disagreeing with the Electron page on 77,340 readings.

The arithmetic check now reads thirty-two of this panel's comparisons, where it read twenty-nine before. The three it gained are the three it could not see.

Standing tranches against the sells that opened them

The queue may never hold more tranches than the sells that opened them, outside a strong trend. `src/trading/scrumming/fold_tranches.py`, in `_bound_new_fold_tranches`, counts each such sell, and `src/gui/live_settings/fold_tokens.py`, in `compose_scrum_sells_row`, is the row on the panel that reports the pair. A standing count above the sells draws red. A bot whose save predates the sell counter draws the dash in place of a sell count.

One row number, not two

The # column is the tranche's place in the queue, and the Fire button quotes that number back in its confirmation. `src/gui/live_settings/fold_tokens.py`, in `configure_tranche_row_numbers`, hides the table's own row header, which drew the display position instead. The two disagreed on four of five rows under Oldest first and under Largest USD first. The React page draws only the declared columns, so it never held a second row number.

The Paper and Simulator Fold Tranches tabs call that same function on their own tables, so a tranche carries one number on all three screens. Both forks also draw the standing-against-sells row from `compose_scrum_sells_row`, the function named in the section above, rather than composing a row of their own.

What sets the Details window's opening width

The Fold Tranches tab is the widest tab, and the Fold-Tranche Cycle Health box is the widest thing on it. The despawn preview value is the longest line in that box. `src/gui/live_settings/fold_chrome.py`, in `install_despawn_rows`, lets that line wrap, which drops the tab's demand from 1938 to 1548 pixels. At a 1920 by 1080 display `src/gui/bot_live_settings.py`, in `open_at_content_size`, then opens the window at its full content width with a whole tranche row inside it. The screen clamp in `dialog_open_size_px` stays, because the operator cannot drag back a window wider than the display.

Every tranche names the opposing event that produced it

A fold tranche holds the tokens the scrum that made it sold. A scrum stack holds the base currency the fold that made it spent. Each record now carries the identity of that event and the two amounts the event produced, so the amount on the record can be read against the amount the event supplied.

`src/trading/scrumming/sizing.py`, in `fold_tranche_records`, is the one place a fold tranche is built. It takes the sale's own figures as its whole budget and hands them out across the lots the sale consumed, so a record carrying an amount the sale did not produce cannot be written there.

```
def fold_tranche_records(
    lots: list,
    fill_price: float,
    sold_units: float,
    proceeds_usd: float,
    event_id: str,
    created_ts: float,
) -> tuple[list, float]:
```

Three sites called their own copy of that loop before: the scrum phase, the distribution phase and the operator's Manual Fire. All three now call this one function. The second value it returns is the units a sale sold that no lot covered, and each site writes a FOLD ALIGNMENT SHORT line naming that figure.

`src/trading/scrumming_bot.py`, in `_open_stack_from_scrum`, is the one place a stack tranche is built, and it writes the same three fields from the fold fill that spawned the ladder.

The Units cell on the Fold Tranches row reads the tranche's units against the units its event sold, and its mouse-over names the event and both amounts. `src/gui/main_tabs/fold_tranches_tab_surface.py`, in `units_cell`, is the one declaration of that cell, and the Live, Paper and Simulator tables each call it.

The Stack-Tranche Cycle Health box gains a row reading the standing size against what the events that produced it acquired, from `src/gui/main_tabs/stack_tranches_tab_surface.py`, in `compose_event_alignment_row`. The row draws red while the standing size passes what was produced.

Marked units above held units, and what produces it

Driven on hand-built tranches with no venue reached, three scrums in a row at one price read 0.60, 2.20 and 5.40 times the holding. Every tranche's units equalled the units its own sale sold on all three. The ratio first passed one on the second scrum.

A bot that scrums three times without folding holds the units of three sales in its queue and only the remainder in its position, so the ratio climbs on every scrum and falls on every fold. The same six-event sequence read 2.20, 0.60 and 0.00 on the three folds that followed.

So a ratio above one is not by itself a tranche whose amount no event produced. The reading that separates the two is the amount on the record against the amount its event supplied, which is what the row now states.

A split against an unknown venue minimum is refused

A Stack ladder divides one fold's acquired tokens across spaced prices, and every piece must reach the smallest order size the venue published. Where the venue published no size rule the floor is unknown, and the split is refused rather than sized against a guess.

`src/trading/stack_math.py`, in `split_scrum_into_tranches`, refuses a minimum that is not a positive finite number, and `src/trading/scrumming_bot.py`, in `_open_stack_from_scrum`, passes the venue's own figure through unmasked so an absent rule arrives as the absence it is. The scrum still sells as one undivided order; only the split is refused.

The signature in the code block at the head of this entry is overtaken. It is not edited and not removed. It read:

```
    min_order_size: float = 0.0,
```

The parameter now defaults to `None`, and both `None` and zero are refused:

```
    min_order_size: Optional[float] = None,
```

Driven on a hand-built ladder of one unit over four rungs: a floor of 0.20 gives four pieces of 0.250000, a floor of 0.40 gives two of 0.500000, and both have every piece at or above the floor. A floor of 2.00 against one unit is refused. No floor at all is refused. In every arm that is not refused the pieces sum to the units the fold acquired.

`src/trading/stack_math.py`, in `_apply_min_order_size`, reads the smallest piece the ladder actually holds rather than a count of pieces. Two rungs that merge leave the ladder with pieces of unequal size, and a count can fit while a piece sits under the floor. On a merged ladder of one unit the same arm read pieces of 0.500000, 0.250000 and 0.250000 against a floor of 0.300000 before, and three pieces of 0.333333 after.

The merge distance is one declaration both tranche types read

Two tranches within a tenth of a percent of each other in price combine. `src/trading/stack_math.py`, in `within_merge_distance`, is the one declaration of that distance, and `MERGE_THRESHOLD` beside it holds the figure.

The Stack ladder read that distance already, inside its own build. `src/trading/scrumming/sizing.py`, in `merge_fold_tranches_within_distance`, now reads the same declaration for the fold queue. The survivor keeps the lower reference price, which is the stricter fold threshold, sums the cash and the units, takes the units-weighted original cost and names both events.

```
  def within_merge_distance(lower_price: float, higher_price: float) -> bool:
```

`src/trading/scrumming/fold_tranches.py`, in `_combine_fold_tranches_within_distance`, runs it on the standing queue each time a sale queues a tranche, and lowers the opened counter by what it combined so the panel's reconciliation still balances.

`src/trading/stack_math.py`, in `merge_into_standing_rungs`, moves a new rung into a standing rung inside the same distance. A standing rung holding a venue order is never a candidate, since combining it would need that order cancelled.

Driven in one run on both types: a pair 0.05 percent apart combined and a pair 0.5 percent apart did not. On the fold pair the survivor's cash and units were the sums of both, and its original cost was the units-weighted figure.

Current reading. A rung that a fold or a distribution opens carries the buy price of that fold as its floor reference. A new rung combines only with a standing rung holding the same floor reference, so no combine loses a floor. In Internal mode an activated rung with a floor reference is not spent while the price sits under that

buy price raised by the Minimum Opposing Trade Distance. It stays activated, and the bot log names the price it waits for. A fold whose own ladder opens adds nothing to the distribution sale, because that ladder already offers its extra units.

`src/trading/scrumming_bot.py` — `ScrummingBot._rung_under_floor`

```
floor_ref = rung_floor_ref(rung)
if floor_ref <= 0.0:
    return False
try:
    floor = opposing_sale_price(floor_ref, self.opposing_trade_distance_pct())
```

What this changes for a queue that is already aligned

Three sales at prices a percent apart, driven through the real queueing path, produced the same three tranches before and after, to the digit. Sweeping the fold gate over 120,001 prices from $40.000 to $160.000 in steps of $0.001 gave 356,457 admitted rows, 3,352,311.201596 admitted units and $537,573,705.000000 of planned spend on both readings, and the same digest over every price.

Three sales at one price produced three tranches before and one after. The cash and the units the gate admits and plans over the same swept domain are identical on both readings; the row count falls from 352,323 to 117,441 because one record stands where three stood.

A queue that already exceeds its holding is not changed by any of this. What happens to an excess that already stands is the operator's decision.

The standing excess is retired on load

The decision above has been taken. The operator's ruling: drop the standing tranches whose amount no sale produced, keep the aligned ones, and bring the marked total back under the held total.

`src/trading/scrumming/fold_tranches.py`, in `retire_unaligned_tranches`, is the sweep. `src/trading/scrumming/state_io.py`, in `import_scrumming_state`, is the one caller. It runs once per bot, as that bot's state is restored, and after the parked wire credit has landed on the records. Every record queued after it is handed out of one sale's own figures, so the standing set is the only set that can hold an amount no sale produced, and that set does not grow.

No standing tranche carries the identity of the event that produced it, because the field did not exist when the tranche was queued. The sweep does not read it. One relation holds across every record the creation path makes: a record's dollars, less the wired-in dollars the record itself records, equal its units times the price its sale filled at, less the trading fee.

```
sale dollars / (units x sale rate)  =  1 - trading fee / 100
```

Now: a record that a wired sale queues carries the share the wire took, and the relation reads its dollars against units times price less the fee and that share, so the sweep keeps it (`src/trading/scrumming/sizing.py, in unproduced_fold_amount`).

`src/trading/scrumming/sizing.py`, in `unproduced_fold_amount`, reads that and answers why no sale produced a record's amount, or nothing where one could. `tranche_wire_usd` beside it takes off the

wired-in dollars first, because wire money reaches a tranche with no units behind it.

The window carries one merge distance of slack at each edge, the distance at which `src/trading/stack_math.py`, in `within_merge_distance`, already calls two prices one price. For a bot at a 0.6 per cent fee a record is kept from 0.99301 to 1.00100 of price times units.

A record with no price, with no lot behind its units, or with no readable figure is retired at every fee. A drop counts as discarded and never as closed, so the panel's opened less closed less discarded still equals what stands, and the dropped record's price, its units, its dollars and the reason it went are written to the bot log.

Where the sweep spares a record. A Trading Fee % lowered by more than a tenth of a point moves every record the earlier sales queued below the window at once. `src/trading/scrumming/sizing.py`, in `unaligned_fold_tranches`, spares a record the fee window condemns where another standing record shares its implied fee, and where no record passed the window at all. Records agreeing on one fee are sales at one fee, which is what a changed setting leaves standing, and the log names the fee and how many were spared.

`src/trading/stack_math.py`, in `unproduced_stack_amount`, is the stack side. A stored rung carries no figure of the fold event that acquired its units, because `split_scrum_into_tranches` spreads one fold's tokens over spaced rungs and the alignment is at their sum. Size and price are all a single stored rung can be tested against, and a rung holding a resting venue order is kept whatever it reads, because dropping the record would strand that order on the book.

Driven on constructed records in a redirected home, with no venue reached. On a queue holding one aligned record beside one carrying a quarter of its sale's dollars, the tree before this work kept both and the tree after kept the aligned one and retired the other. Over a swept domain of 120 records a sale produced, both trees left the same 120 records with the same digest, the same $468,656,385.72274005 and the same 27,528.615 units. Moving one of those records by one digit changed the digest and left all 120 standing. A bot whose fee setting read 0.00 per cent against five records queued at 0.6 per cent retired none of them and wrote the refusal.

The marked total is brought under the held total on load

The sweep above retires the records whose own figures no sale produced. The operator's second ruling covers what is left: keep the aligned tranches up to the held total and retire the oldest beyond it, so the units the queue marks never exceed the units the bot holds.

This is not a limit on how many tranches stand. Many may stand on either side, in any number. Only their sum is bounded.

`src/trading/scrumming/fold_tranches.py`, in `retire_tranches_above_held`, is the sweep. `src/trading/scrumming/state_io.py`, in `import_scrumming_state`, is the one caller, and it runs this sweep straight after the alignment sweep and before the fold queue's dollar total is re-derived. So it judges only what the alignment sweep left standing, and the re-derive reads the queue it leaves.

Diagram: flowchart, top to bottom

- a bot's state is restored -> **parked wire credit lands on the records**
- parked wire credit lands on the records -> **retire_unaligned_tranches — drops what no sale produced**
- retire_unaligned_tranches — drops what no sale produced -> **retire_tranches_above_held — drops the oldest beyond the held total**

- retire_tranches_above_held — drops the oldest beyond the held total -> **the fold queue's dollar total is re-derived**

Which figure is the held total. `import_scrumming_state` does not restore `_current_holdings`; the exchange supplies that on the next tick, and it still reads zero while the sweep runs. `src/trading/scrumming/fold_tranches.py`, in `_held_units_now`, therefore sums the units of this bot's restored lots, the same figure the restore drift warning in `import_scrumming_state` already sums. A lot holding no readable units figure is counted beside the total, never added as zero.

A held total of zero or less refuses the whole sweep and every tranche is kept. A bot whose lots were never saved would otherwise lose the lot. The refusal is written to the bot log and names the figure it read.

Which field dates a record. `src/trading/scrumming/sizing.py` declares `FOLD_AGE_KEY` as `created_ts`, written by `fold_tranche_records`, and `STACK_AGE_KEY` as `opened_ts`, written by `split_scrum_into_tranches`. The age sweep and this one read the same two names. `src/trading/scrumming/sizing.py`, in `tranches_beyond_held_total`, sorts the standing records oldest first on that field and drops from the front while the remainder still marks more than the held total, so the newest records are the ones that stay. A record whose stamp is not a positive finite number sorts as the oldest and is counted, and the log line states how many records the ordering could not place.

One sale stamps every record it queues with one `created_ts`, so a tie is the normal case. A tie keeps the queue's own order, which is the order those records were written in.

A queue that equals its held total exactly is left alone. Two things make that hold. `tranches_beyond_held_total` sums the marked units with `math.fsum`, which is exactly rounded, so the order the records are added in cannot push the total above the held figure; a running total of seven equal parts of ten thousand units reads 1.82 trillionths above it, which is near three times the real excess of 0.68 trillionths. And `src/trading/scrumming/sizing.py`, in `units_dust_at`, reads `UNITS_DUST` against the held figure and takes the larger of the absolute figure and that share of it, because one bit of a figure above ten thousand units is already wider than the absolute dust.

Measured at thirteen scales from a thousandth of a unit to a billion units, and at four part counts each: of 52 such queues the running total and the absolute dust dropped a record from 10, and the exact sum with the scaled dust dropped none. A queue genuinely above its held total still loses its oldest records at every one of those scales.

The two queues are measured separately and never added together. A fold tranche's units left the position when the scrum sold them; a stack rung's units are still in it, because `split_scrum_into_tranches` places a sell for each rung. One total across both is not a quantity. Each queue is measured against the one held total on its own.

A rung holding a resting venue order is kept whatever the arithmetic says, exactly as the alignment sweep and `src/trading/scrumming_bot.py`, in `clear_stack_tranches`, keep it, and its units are charged against the held total first. Where those kept rungs alone still mark more than the bot holds, the log line says so rather than claiming the invariant.

What this costs. Every record dropped here is units the bot sold and will now never buy back at that price. The operator chose this with that consequence stated. Each drop is written to the bot log with its price, its units, its dollars, its age in days and the reason it went, and counts as discarded and never as closed, so opened less closed less discarded still equals what stands. No order is placed or cancelled, and holdings, cost basis and

target balance are unchanged.

Only a pending rung marks units. The clause above that says a stack rung's units are still in the position holds for a pending rung only. It is not edited and not removed. A rung reads pending until its sell fills or ends unfilled. A filled rung's units left the position when the sell filled. A cancelled rung's sell never reaches the book. Neither has units left to deliver.

So the sweep measures the stack queue on its pending rungs alone, and it retires only pending rungs. A filled or cancelled rung stays in the queue, and the log line counts how many stood outside the stack total. The sell for each rung, and the `opened_ts` stamp the ordering reads, come from `_open_stack_from_scrum`; the ladder arithmetic the clause names places nothing.

```
src/trading/stack_math.py, in rung_is_pending                          the one reading of a pending rung
src/trading/stack_math.py, in _rung_is_order_free                      reads rung_is_pending
src/trading/scrumming/fold_tranches.py, in retire_tranches_above_held  measures and retires those rungs only
src/trading/scrumming_bot.py, in _open_stack_from_scrum                places each rung's sell, writes opened_ts
```

`src/trading/stack_math.py` — `rung_is_pending`

```
def rung_is_pending(record: dict) -> bool:
    """Does `record` read `pending` as its `status`; a filled or cancelled rung
    marks no units."""
    return str(record.get(STACK_STATUS_KEY, "")) == STACK_STATUS_PENDING
```

What a dropped rung costs. The paragraph above says every record dropped here is units the bot sold and will never buy back at that price. That holds for a fold tranche. A dropped rung is units the bot holds and will not sell at that rung's price, and the log line states both.

Driven on constructed records in a redirected home, with no venue reached and no order placed. Two pending rungs of 4 units, inside 10 units held, stood beside a filled rung of 6 and a cancelled rung of 6. The tree before this work read 20 marked units and dropped both pending rungs and the filled one. This tree reads 8 and keeps all four. Fourteen status readings ran, each at 3 and at 30 units against 10 held. The tree before this work counted all fourteen. This tree counts only the three pending readings, and still retires a pending rung of 30 that holds no venue order. Over 600 bots holding only pending rungs, both trees left the same queues and dropped the same 1,130 rungs. Lowering one digit of one lot on one side only made 227 of the 600 differ.

```
at 100 units held, pending rungs inside the holding   pending rungs lost
                                                       before    after
60 pending in 6 rungs, 30 filled in 3, newer               0        0
60 pending in 6 rungs, 60 filled in 6, newer               2        0
60 pending in 6 rungs, 60 cancelled in 6, newer            2        0
90 pending in 9 rungs, 90 filled in 9, newer               8        0
100 pending in 10 rungs, 10 filled in 1, newer             1        0
60 pending in 6 rungs, 200 filled in 20, newer             6        0
60 pending in 6 rungs, 60 filled in 6, older               0        0
```

Where the filled rungs were the older ones, or the pending rungs held venue orders, the tree before this work dropped 2 filled records instead. This tree drops none.

What the screen reads afterwards. The allotment row described under *How the ratio cell picks its colour* draws its figure through `src/gui/live_settings/fold_tokens.py`, in `compose_units_marked_row`, on the Qt panel and through `src/gui/main_tabs/fold_tokens_surface.py`, in `compose_units_marked_row`, on the Electron page. After a load that sweep ran on, that row reads at or below `1.00x` and draws no red, unless a resting venue order pinned a rung or no held total was readable.

Driven on constructed records in a redirected home, with no venue reached and no order placed. Three fold records of 30 units each against 40 units held kept the newest and retired the two older ones; the tree before this work kept all three. Over 400 bots whose held figure sat just above their marked total, both trees left the same queues with the same digest on all 400 and dropped nothing. Lowering one digit of that held figure left 4 of the 400 digests matching and dropped 945 records. Over 1,200 pseudo-random bots, 645 stood above their held total on the tree before this work and 62 after it, and those 62 are exactly the bots whose remaining excess sits in rungs holding a resting venue order, each reported as such. The counter reconciliation held on all 1,200. Blinding the date stamp on all three records of the first reading made the sweep report three records it could not place.

The Product Manual PDF rebuild is owed for this page.

12 - Technical Analysis Indicator Confidence Tiers

Most TA systems stack indicators additively. The Tier-4 suite organizes signals into four confidence tiers with cumulative weighting: Tier-1 (structural, highest weight), Tier-2 (confirmation, medium weight), Tier-3 (momentum), Tier-4 (context). Entries require Tier-1 + Tier-2 alignment; Tier-3/4 scale confidence but cannot override. This tier system arose out of queries with Claude Code while it was roleplaying as a TA Archetype and is only one such example where most of the other such contributions deal with bug fixes.

Not built as tiers. The four-tier scheme is a design. The engine has no tier. The signal record carries an indicator name, a timeframe, a direction, a confidence, a weight, a details map, a timestamp and an abstention flag, and nothing else. No field names a tier, no code groups the voters into tiers, and no rule requires a Tier-1 and Tier-2 alignment before an entry.

`src/trading/indicators/types.py` — `Signal`

```
@dataclass
class Signal:
    """One indicator's output at a point in time."""

    indicator: str
    timeframe: str
    direction: SignalDirection
    confidence: float
    weight: float = 1.0
    details: dict = field(default_factory=dict)
    timestamp: float = field(default_factory=time.time)
    abstained: bool = False
```

What runs instead. A flat per-indicator weight. Each of the twelve voters carries one number between 0.8 and 1.2, and a signal's score is its direction times its confidence times that weight. Every vote can move the total, and none can veto another.

`src/trading/ta_engine.py` — `DEFAULT_WEIGHTS`

```
DEFAULT_WEIGHTS = {
    "bollinger_bands": 1.0,
    "vortex": 0.9,
    "macd": 1.2,
    "stochastic_rsi": 1.0,
    "ichimoku": 1.1,
    "volume": 0.8,
    "slingshot": 1.0,
    "adx": 1.0,
    "kaufman_er": 1.0,
    "supertrend": 1.0,
    "zscore": 0.9,
    "rsi": 0.8,
}
```

Since 2026-10-28 a new bot starts with all twelve at 0.0, written by `src/trading/ta_engine.py, in new_bot_weights`, and votes neutral until Save on the Settings window hands it the TA Indicators figures. The published figures above are unchanged, and a saved bot votes as it did.

The build this specifies. A tier is a second map beside the weights, and the veto is one test in the aggregator. The signal record grows one field, and it has to go last: the record is built positionally in this tree, so inserting a field earlier would re-bind an existing argument.

Proposed, not present, in `src/trading/ta_engine.py`*:*

```
DEFAULT_TIERS = {
    "bollinger_bands": 1,
    "ichimoku": 1,
    "supertrend": 1,
    "macd": 2,
    "vortex": 2,
    "adx": 2,
    "stochastic_rsi": 3,
    "rsi": 3,
    "slingshot": 3,
    "volume": 4,
    "kaufman_er": 4,
    "zscore": 4,
}
```

Proposed, not present, in `VotingEngine._aggregate`*:*

```
_lead = {s.tier for s in signals if not s.abstained and s.direction is not NEUTRAL}
if require_tier_alignment and not {1, 2} <= _lead:
    consensus_conf = 0.0
```

Issue #433 carries the build.

Current reading, what a trade needs from the vote today. The tiers are still not built. What a trade the vote decides needs today is three agreeing indicators: the vote must top what the two heaviest voters reach at full

confidence. That holds at every floor, where a landing strip sets the side, and on a bot's first buy. A bot with every weight at zero is outside the rule and trades as it did.

`src/trading/ta_engine.py` — `three_agree`

```
    held = [sig.weight for sig in signals] if weights is None else list(weights)
    if not any(weight > 0.0 for weight in held):
        return True
    return confidence > two_indicator_ceiling(signals)
```

13 - Initial-Purchase-Price Floor on Folds

This rule asserts that a subsequent Fold occurring after a Scrum MUST be less than sell. This, of course, is asserted by Bollinger Band Travel, Minimum Opposing Trade Distance, Target Balance Not Zero, etc.

Now: every fold path, the cartridge included, pays a tranche back only at or under its own sale price less the distance, wired or not (`src/trading/scrumming/sizing.py, in tranche_rebuy_price`).

Functional. The floor runs, and it lives on the tranche. Every fold tranche stores the price of the scrum that spawned it. The row is released only when price has fallen to that reference multiplied by the rebuy factor, and the factor is one minus the opposing trade distance. A one percent interval with a six tenths fee gives a distance of 1.60 percent and a factor of 0.984, so a tranche spawned by a sale at $100.00 folds back at $98.40 or lower.

`src/trading/scrumming/fold_tranches.py` — `_fold_eligible_tranches`

```
    return [
        t
        for t in self._fold_tranches
        if ticker_last <= float(t.get("ref", 0)) * otd_factor
    ]
```

A tranche can take money from more than one sale. When a sell's money moves into an older part-spent row instead of opening a new one, the row keeps the price its own scrum sold at, so the distance is still measured from a price a sale happened at. The dollars-per-unit rate across every sale that fed the row is held apart from that price as the row's blend rate, and only the units arithmetic reads it.

`src/trading/scrumming/sizing.py` — `tranche_sale_rate`

```
    blended = float(record.get("blend_rate", 0) or 0)
    if blended > 0:
        return blended
    return float(record.get("ref", 0) or 0)
```

A row fed at one price alone carries no blend rate, and its rate is its scrum price. `src/trading/scrumming/fold_tranches.py`, in `_top_up_remnant_fold_tranches`, moves the money and writes a blend rate only where the two rates differ. `src/trading/scrumming/sizing.py`, in `units_at_scrum`, is the one place that turns a row's dollars back into the units it sold; the fold executor and `fold_surplus_usd` both read it, so the units a fold is measured against and the surplus it books cannot disagree.

Current reading. The function above no longer turns a row's dollars back into units. It reads the row's own units, the units its scrum sold, so a fold's extra is the units it bought less the units its scrum sold. The sell fee no longer counts as extra, and a cycle that ends with fewer units books no gain. The blend rate is now read only by the checks that hold a row's dollars against its sale, and by the top-up that writes it.

`src/trading/scrumming/sizing.py` — `units_at_scrum`

```
    units = as_finite_float(record.get("units", 0.0))
    if units is None or units <= 0.0:
        return 0.0
    return units
```

Design intention. The same distance is held in three places so the rule cannot be true for the bot and false for a row. The bot-wide hysteresis gate holds it, the per-tranche filter above holds it, and each new ladder rung is priced against it.

`src/trading/otd_math.py` — `fold_rebuy_factor_from_pct`

```
    def fold_rebuy_factor_from_pct(otd_pct: float) -> float:
        """Convert an already-clamped OTD percentage to a FOLD rebuy factor.

        Multiply a tranche's ``ref`` by this to get the HIGHEST price at
        which that tranche is still eligible to be re-bought:

            eligible  <=>  ticker.last <= ref * factor
        """
        return 1.0 - (float(otd_pct) / 100.0)
```

The lot carries its own cost basis as well as the scrum reference, and the tick counts the rows eligible against that basis.

`src/trading/scrumming/tick_phases.py` — the lot's own basis

```
    _patent_only_eligible = sum(
        1
        for _t in self._fold_tranches
        if ticker.last
        <= float(_t.get("initial_buy_price", _t.get("ref", 0)))
    )
```

One gap sits in the same path, and the function names it in its own docstring. The fold gate wraps the distance call and falls back to a distance of zero when a config value is not a number. At that fallback the rule is not active.

Current reading. No path falls back to a distance of zero. A distance setting that holds no number refuses the scrum and the fold, and the bot log names the setting.

`src/trading/scrumming_bot.py` — `ScrummingBot.tick`

```
try:
    _distance = self.opposing_distance_settings()
except DistanceSettingUnreadable as _unread:
    self._refuse_unreadable_distance(_unread, "SCRUM AND FOLD", ticker.last)
```

Where a tranche's rebuy guide is placed

The owner states the rule for where a tranche sits:

> *Placement of Tranches must first be a function of Minimum Opposing Trade Distance + Trading Fee + BB Extension (if present...).*

The guide is measured from the price the tranche's scrum sold at, the `ref` on the stored row, and never from the price first paid. A row shows both: the Sell ref column holds the first and the Original cost column holds the second. A guide above the cost basis is normal, because a bot that sold high can buy back above its own cost and still gain units.

`src/trading/otd_math.py` — the guide, from the sell reference

```
def fold_rebuy_price(sell_ref_price: float, otd_pct: float) -> float:
    return float(sell_ref_price) * fold_rebuy_factor_from_pct(otd_pct)
```

Two of the three terms reach the guide. The opposing trade distance is `scrumming_interval_pct` and the trading fee is `trading_fee_pct`, both declared in `src/trading/container/config.py`, and `minimum_opposing_trade_distance_pct_from_config` is the one place that reads either. Moving the declared interval from 1.0 to 7.3 moves the guide on a hundred-dollar sell reference from $98.40 to $92.10; moving the declared fee from 0.6 to 2.9 moves it to $96.10.

Current reading. The fee move reads differently. At a fee of 2.9 % the 1.0 % interval sits under one fee, so the distance is two fees, 5.80 %, and the guide reads $94.20. The interval move keeps its figure, since 7.3 % sits above the 0.6 % fee.

```
interval 1.0 %   fee 2.9 %   distance 5.80 %   guide $94.20
interval 7.3 %   fee 0.6 %   distance 7.90 %   guide $92.10
```

The band extension is the third term and no caller supplies it yet. `band_extension_pct` states it as a share of the reference price, taking the band's own span the way `src/trading/indicators/bb_proximity.py`, in `detect_bb_proximity`, takes it. Moving the declared `bb_tolerance_pct` from 1.0 to 4.7 moves that term from 0.200% to 0.940%, and a distance holding it would read 2.540% in place of 1.600%. A band makes the distance differ per tranche, and `src/trading/scrumming/sizing.py`, in `eligible_fold_tranches`, takes one factor for the whole queue, so wiring the term reaches that function, `src/trading/scrumming/tick_phases.py`, `src/paper/paper_run.py` and `src/simulator/back_test.py`.

Now: every tranche reads one distance, the bot's distance settings at the tick's band, the same for a wired tranche as for an unwired one (`src/trading/scrumming/sizing.py, in tranche_rebuy_price`).

A guide above the price it is measured from cannot be produced

An earlier reading of the factor trusted its caller to clamp the percentage first. Its own docstring said so:

Convert an already-clamped OTD percentage to a FOLD rebuy factor.

The current reading is that the factor clamps its own argument, so a negative percentage cannot lift it above 1.0 and the guide cannot read above the price it is measured from.

`src/trading/otd_math.py` — the clamp inside the factor

```
def fold_rebuy_factor_from_pct(otd_pct: float) -> float:
    return 1.0 - (clamped_otd_pct(otd_pct) / 100.0)
```

Swept from -5.0% to 60.0% in steps of 0.001%, 5,000 of the 65,001 percentages produced a guide above its own reference before the clamp moved and none produce one now. The worst was 1.05 times the reference at -5.0%. The configured path never reached that domain: over 251,001 interval and fee pairs from -2.0 to 3.0 in steps of 0.01, the guide ratio stayed between 0.94 and 1.00 both before and after.

The Fold Tranches page model held the percentage and the factor as two numbers a host could set apart. Asked for a percentage of 1.60 and a factor of 1.04, the row drew a guide of $104.00000000 against a sell reference of $100.00000000. The model now derives the factor from the percentage and the same request draws $98.40000000.

What the row's percentage means

The Status cell states the move the live price must still make to reach the guide, so the figure carries a minus while the price sits above it. With a hundred-dollar sell reference and a distance of 1.60%, a price of $110 reads `Need price ≤ OTD (-10.55%)` and a price of $90 reads `Price-OK (+9.33% vs OTD)`. The earlier reading stated where the price sat rather than the move it must make, so the same two prices read `+11.79%` and `-8.54%`.

`src/gui/main_tabs/fold_tranches_tab_surface.py` — the signed move

```
def status_move_pct(price: float, threshold: float) -> float:
    return (threshold - price) / price * 100.0
```

The Min rebuy tooltip names both prices the guide sits between: *A guide, not a trigger: sell ref $100.00000000 less 1.60% is $98.40000000. A fold-back still needs TA.* The Live, Paper and Simulator panels and both React pages draw `min_rebuy_cell` and `status_cell` from this one module, so no screen can carry a different figure.

Now: each row's guide and status read the bot's one distance, so a wired tranche shows the same buy-back price an unwired one does (`src/gui/main_tabs/fold_tranches_tab_surface.py, in fold_distance_pct`).

14 - Position Ceiling and Detonation

This setting allows a total profit cap to be set for a given position before it sells its entire or a large portion of its total value.

Functional. The entry covers two mechanisms. They act on opposite sides of the cycle and both start disabled. The ceiling brakes buying. It is the anchor target multiplied by a factor clamped between one and ten, and the fold rate tapers as the position approaches it: full rate below half the ceiling, a linear taper from full to a tenth across the upper half, and a hard stop at the ceiling. The ceiling starts no sale.

`src/trading/scrumming_bot.py` — `ScrummingBot.fold_rate_taper`

```
ratio = self.ceiling_ratio
if ratio is None:
    return 1.0
if ratio >= 1.0:
    return 0.0
if ratio < 0.5:
    return 1.0
return 1.0 - (ratio - 0.5) / 0.5 * 0.9
```

Design intention. Detonation sells. It requires the feature to be enabled, a position above the anchor, and a bullish consensus at or above the confidence minimum on the detonation timeframe, which defaults to one day. The check is edge triggered, so one bull reversal fires it once, and a rate limit spaces the checks. The executor then market-sells the surplus above the anchor, clears the fold tranches and reseeds the lots. It harvests the excess above the anchor, not the whole position.

`src/trading/scrumming_bot.py` — `ScrummingBot._check_detonation_trigger`

```
conf_min = float(getattr(self.config, "detonation_confidence_min", 0.75))
is_bullish = (
    summary.consensus_direction == SignalDirection.BULLISH
    and summary.consensus_confidence >= conf_min
)

fired = is_bullish and not self._detonation_last_signal_bullish
self._detonation_last_signal_bullish = is_bullish
```

Both features default to off, and the ceiling multiple defaults to five.

`src/trading/container/config.py` — `BotConfig`

```
position_ceiling_enabled: bool = False
position_ceiling_multiple: float = 5.0  # Range [1.0, 10.0]
```

Issue #336 records both pairs among the wizard settings that never reach the bot. Issue #107 records a detonation that can fire twice across a restart. Any earlier description reading "bearish" is wrong: the code compares against the bullish direction.

The issue #336 and issue #107 clauses above are overtaken. They are not edited and not removed. They read:

> *Issue #336 records both pairs among the wizard settings that never reach the bot. Issue #107 records a detonation that can fire twice across a restart.*

Issue #336 closed on 2026-10-04 and issue #107 closed on 2026-10-17. Both pairs reach the bot, and the detonation flag survives a restart.

```
src/trading/container/config.py, in _BOT_CONFIG_SCRUMMING_ONLY_FIELDS
    position_ceiling_enabled, position_ceiling_multiple,
    detonation_enabled, detonation_timeframe, detonation_confidence_min

src/trading/scrumming/execution.py   reads position_ceiling_enabled and the multiple
src/trading/container/config.py, in ceiling_multiple   the one reader of the multiple
src/trading/bot_container.py         publishes both to the screen
src/trading/scrumming_bot.py         reads detonation_enabled before the bar fires
```

The flag the detonation bar reads is saved with the bot and read back on load, so a restart no longer presents a bull run the bar has already fired on as a fresh one.

```
src/trading/scrumming/state_io.py
    saved     detonation_last_signal_bullish
    restored  detonation_last_signal_bullish, False when the record has none
```

How the taper and the detonation bar read today

Two readings in section 14 above carry wording this unit changed. They stand as written, and these are the readings that replaced them.

written above	what it reads now
"the fold rate tapers as the position approaches it"	The fold size tapers. The figure multiplies the dollars a fold spends and never the interval between folds. The schedule itself is unchanged: full size below half the ceiling, a linear taper to a tenth across the upper half, a hard stop at the ceiling
"`src/trading/scrumming_bot.py` — `ScrummingBot.fold_rate_taper`"	`src/trading/scrumming_bot.py, in ScrummingBot.fold_size_taper`, reading the same schedule from `src/trading/scrumming/sizing.py, in fold_size_taper`
"a bullish consensus at or above the confidence minimum"	Unchanged, and the minimum now reaches the status record every screen reads, so the Fire tooltip on a Bot Swarm row names the figure a vote has to clear

`src/trading/scrumming_bot.py, in ScrummingBot.fold_size_taper` — the property

```
ratio = self.ceiling_ratio
if ratio is None:
    return 1.0
return fold_size_taper(ratio)
```

The opening confidence figure is one declaration, read by the stored record and by the Min Confidence box.

`src/trading/container/config.py` — the figure

```
DETONATION_CONFIDENCE_MIN_DEFAULT: float = 0.75
```

The detonation vote and its bar, 2026-10-28

Two readings in section 14 above carry wording this change moved. They stand as written, and these are the readings that replaced them.

written above	what it reads now
"`conf_min = float(getattr(self.config, "detonation_confidence_min", 0.75))`"	The fallback reads `DETONATION_CONFIDENCE_MIN_DEFAULT`, and the vote the bar is compared against reads the bot's own twelve weights
"a bullish consensus at or above the confidence minimum on the detonation timeframe"	Unchanged, and the minimum can now be set from 0.00 to 1.00, so a reading the vote reaches can fire it

`src/trading/scrumming_bot.py, in ScrummingBot._check_detonation_trigger`

```
  engine = VotingEngine(weights=self._ta_weights)
```

The Product Manual PDF rebuild is owed for this page.

The sentence above stands as written. This is the reading that replaced it.

written above	what it reads now
"The Product Manual PDF rebuild is owed for this page."	`docs/Acervator-Product-Manual.pdf` is rebuilt with this page

15 - Proof of Accumulation (PoA)

This is another one of the larger and more distinguishing features of the platform. At this point, it is a design proposal for a blockchain that provides certified trade performance between competitors without revealing their specific strategy tunings within Acervator or their identity. When this blockchain is initiated, ZERO tokens for it will exist in the world. I intend it to be, much like A15 to 1) have a fixed supply, 2) mean something through its generation and acquisition. Tokens are only created through the PoA tournaments and defeating Token Beasts or inflicting the most damage to them in a colosseum format. Additional functions would allow PoA tokens to be socketed into the platform to change its appearance and identify a user as a victor.

Functional. The competition package is the Proof of Accumulation package, and its engine runs without a screen. The engine walks a competition through registration, active trading, submission and adjudication, and hashes each entrant's config so a strategy proves consistency without publishing itself. A Merkle log produces the root that stands in for the trade record.

The token ledger is append-only and hard-capped. Balances come from replaying the log, and no operation edits one. An award that would cross the cap raises rather than trimming, and an award id repeats at most once.

`src/competition/token_ledger.py` — `TokenLedger.award`

```
event_id = self._event_id(bot_id, competition_id, tier.name)
if event_id in self._seen:
    return None

if self.total_minted() + tier.base_value > TOTAL_SUPPLY_CAP:
    raise OverflowError(
        f"Supply cap {TOTAL_SUPPLY_CAP:,} ACRV would be exceeded. "
        f"Only {self.remaining_ever()} tokens remain mintable."
    )
```

Design intention. The fixed supply is the whole point of the token, so the cap is a constant the ledger reads rather than a policy a caller can pass. It reads ten million.

`src/competition/season_schedule.py` — `TOTAL_SUPPLY_CAP`

```
TOTAL_SUPPLY_CAP = 10_000_000  # Hard cap — immutable
```

Nothing in the running platform starts a competition. Every module that would request one sits outside the running application, and the retired-tab sentinels assign nothing to the competition and testnet tabs. The canonical tab order now lists an Accumulation tab, and that tab is empty: it draws its name, one sentence saying it is not built, and issue #147, which covers the build-out.

`src/competition/local_testnet.py` — the in-platform chain, built every session

```
MAX_SUPPLY_WEI = TOTAL_SUPPLY_CAP * (10**TOKEN_DECIMALS)
```

Read on the current tree, no session builds the chain. The main window no longer installs the testnet bridge at launch, so the bridge and the eleven competition parts it installs stay unbuilt, and its 250 ms timer never starts. The competition package stays on disk while Proof of Accumulation is paused.

`src/gui/main_window.py` — `MainWindow._setup_ui`, the first two build steps

```
            main_layout = self._timed_build_step(
                "header strip", self._build_header_strip
            )

            self._main_tabs = _main_tab_book_class()()
```

The trophy generator draws the trophy as vector art and letters SOLVE ET COAGULA around its rim. No module imports it. Token Beasts and the colosseum appear in no Python file at all.

16 - Technical Analysis, Goodness of Fit, and the Confidence Index

The Confidence Index is the collated result of the 12-indicator Voting Panel and emulates the concept of a set of features (market structures / indicators) being read by a system having similar measurements for a given or expected or past action i.e. a Fold will always have a certain number of indicators voting bearish before its released for execution.

Functional. The Confidence Index runs. The voting engine builds twelve voters and holds twelve weights, and both count twelve when driven. The net is the bullish score minus the bearish score, and the confidence is the absolute net divided by the summed weight of the voters that did not abstain. Two bulls at weights one and two against one bear at weight one give a net of 2.0 and a confidence of 0.5. A single bull beside an abstainer of weight three gives 1.0, not 0.25, which shows the abstainer leaving the denominator.

`src/trading/ta_engine.py` — `VotingEngine._aggregate`

```
net = bull_score - bear_score

voted_weight = sum(s.weight for s in signals if not s.abstained)
consensus_conf = abs(net) / voted_weight if voted_weight > 0.0 else 0.0
```

Design intention. Confidence measures the breadth of agreement across the panel, not the strength of one reading. That is why an abstention has to be declared rather than inferred: an indicator with too little history returns a fabricated middle reading, and counting it would dilute a real consensus.

`src/trading/indicators/types.py` — `Signal.weighted_score`

```
@property
def weighted_score(self) -> float:
    """Signed score: direction * confidence * weight."""
    return self.direction.value * self.confidence * self.weight
```

Goodness of fit is not built. The phrase names the work of comparing the Simulator against the live platform, and the criterion for that comparison is gates latching identically on the same data. The measure has no settled definition yet and nothing here to attach it to, so nothing is proposed. Issue #117 carries it.

In development.

The issue #117 clause above is overtaken. It is not edited and not removed. It read:

> *The measure has no settled definition yet and nothing here to attach it to, so nothing is proposed. Issue #117 carries it.*

Issue #117 closed on 2026-09-19. The criterion is settled and it is in the tree. A run reads identical when no light disagrees and both armed flags match, and every disagreeing light is charged to one of three causes.

```
src/simulator/validation.py, in latches_identically
    true while   no light disagreed, scrum armed matches, fold armed matches

src/simulator/validation.py, in causes   how many disagreeing lights per cause
src/simulator/validation.py              CAUSES = (TAPE_CAUSE, FIXTURE_CAUSE, CHAIN_CAUSE)
src/simulator/parity_report.py           one report per run, a page and a sidecar
```

No standing defect touches the panel. The eight indicator implementations that departed from their published formulae are all repaired, one indicator per unit, and issue #414 is closed. The API Interaction Log and the Stocks window's New Accumulation Bot box both name the twelve voters the engine builds.

17 - Tiered Strategy Validation and Workflow

Acervator has many unique characteristics and many of these are rooted in attempting to protect an investor from themselves. As such, I have designed a means of self-contained strategy testing that is meant to educate and validate before any attempt at using the platform against actual personal funds is ever attempted. Some may be confident or skilled enough to skip these protective steps. That is an individual user choice. I did not follow this workflow while developing it but I know exactly how my strategy works and when it is not. Given this, the intended workflow for a new user of Acervator should be Simulator > Paper Trader > Live. The system is configured so that the relevant operational elements, such as the Market Inspector, can inject Simulator and Paper equivalents into the appropriate Bot Swarm layers or bot fleet under the respective tab.

Functional. All four stages exist. The Inspector, Sim, Paper and Live tabs are all built and inserted. The Inspector no longer hands its proposals to the Sim tab: the rebuild removed that hook, and Sim and Paper each load a fleet from the stored bot record instead. Live is the tab the part list calls Trading, and it is third in the canonical order.

`src/gui/main_tabs/main_window_surface.py` — `CANONICAL_TAB_ORDER`

```
CANONICAL_TAB_ORDER = (
    SIM_TAB,
    PAPER_TAB,
    LIVE_TAB,
    CHARTS_TAB,
    INSPECTOR_TAB,
    SWARM_TAB,
    ACCUMULATION_TAB,
    HISTORY_TAB,
    STATUS_TAB,
    CONSOLE_TAB,
)
```

The Paper stage is built. Issue #19 added `src/paper/`, the Qt tab and the React panel. Before that no file named for it existed in the tree and none had ever been committed on any branch, which is what the query below measured. One default still records the older state: the shelved stock window imports a module name that does not exist, so it adds no tab of its own.

`src/gui/main_tabs/stock_main_window_surface.py` — the declared default

```
paper_trader_available: bool = False,
paper_trader_error: Any = "No module named 'src.gui.paper_trader_tab'",
```

Design intention. Paper is the stage that proves a strategy against the real market in real time before real money reaches it, and its defining property is that it runs at the market's own pace. What it shares with Live and what it forks is settled: one trading logic, three data sources. What it does not do is import the live stateful shells. That was a build, and issue #19 landed it after the Simulator rebuild.

Issue #19 covers the build-out. Two surfaces that told the operator a Paper Trader was present were issues #422 and #426, and both folded into #19. Issue #117 tracks the Simulator rebuild that gated it.

Issue #117 closed on 2026-09-19 and issue #19 closed on 2026-09-26, so neither tracks anything now. Both screens are in the tree: `src/gui/paper/paper_trading_tab.py` and

`src/gui/paper/paper_react_trading_tab.py` draw Paper, and `src/gui/main_tabs/simulator_tab_surface.py` with `src/gui/react_simulator_tab.py` draw the Simulator.

Today the workflow runs Market Inspector, then Simulator, then Paper, then Live. Every stage has a screen.

Part 3

System Architecture and Features Catalogue

- Main Window
- Trading Tab - Active Bot Lists Under Exchange Tabs - Indicator Voter Panel - Trade Logic Activity - API Activity - Top-Level Portfolio Metrics
- Simulator Tab - Validation Runs vs. Historical Trade Data - Acervator Strategy Back Testing - Runs Historical Data Through Simulated Bots - Bot Swarm Simulation - Deploy As Paper Trading - Deploy As Live Trading
- Paper Trader Tab - Runs Live Exchange Data Through Simulated Bots - Executes and Documents Paper Trades - Collects Paper Trade Data - Pushes Trade Data to Paper Trade History Log - Paper Trade History Tab Reads and Displays
- Proof of Accumulation (PoA) Tab - PoA Blockchain and Trading Tournament Interface - Hosts User Competition Performance and Data - Displays Tournament Monsters and Targets
- Market Inspector Tab - Synthesizes and Proposes Bot Swarm Topologies and Oppositional Trades - Pushes Proposals to Sim, Paper, or Live As Directed
- Bot Swarm Tab - Displays and Allows Configuration of Capital Reinforcement Networks
- Asset Charts Tab - Displays Price Charts for All Active Bots
- History Tab - Loads and Displays Live Trade History Via Exchange API - Merges Live Trade History w/ Acervator Trade Log for Grading and Gate Analysis - Paper History Tab - Loads and Displays Paper Trade History w/ Grading and Gate Analysis
- Console Tab - Displays internal codebase activity, warnings and faults
- System Status Tab - Displays Platform Status by Subsystem - Fed by the Emitter Network
- Settings - System and High-Level User Settings

System Architecture and Features Catalogue

Acervator features a semi-modular / layered design that nests a hyper-vigilant trading engine capable of operating in multiple investment domains simultaneously and all from one terminal. The Main Window contains all of the various subsections found in each subsystem tab. As it stands the existing and planned subsystem tabs are: Simulator, Paper Trading, Trading (Live), Market Inspector, Bot Swarm, Asset Charts, History, Console, Proof of Accumulation (PoA), and System Status. Most of these subsystems interact with each other to some degree with key isolations existing between the three trading tabs and their wiring to Market Inspector, Bot Swarm, and History.

The mark



The Acervator mark, the sigil the splash screen paints at launch.

The mark is the product's own sigil. It is drawn from triangles, not from a font, and the splash screen paints it where a single letter used to sit.

What each element means

Element	Form	What it stands for
Compass	The dividers, hinged above centre, their two legs splaying into the frame that carries the rest	Measurement before action
Scale	A balance below centre, one beam and two hanging pans	The balance the platform exists to disturb and restore
Reptilian eye	An almond eye whose pupil is a vertical slit	Watching the market without predicting it
Fiery aura	Thirteen flames wreathing the eye	The volatility the cycle harvests
Scythe	A single shaft crossing behind the eye, its blade the feather of Ma'at	The cut that takes the excess above the target
Winged caduceus	A staff down the centre, two serpents coiled around it, wings at the top	Exchange, and the two sides of every cycle

The aura carries thirteen points, and the count is deliberate.

The maxim on the legs

Each compass leg is engraved with one word, cut dark into the metal and reading from the hinge downward.

Leg	Word	Meaning
Left	SOLVE	dissolve
Right	COAGULA	reform

Together they are the alchemical maxim *solve et coagula*. It is the platform's own cycle: a position is dissolved when the excess above the target is sold, and reformed larger when the dip is bought back.

How it is built

Each element has its own generator in `src/gui/main_tabs/splash_screen_surface.py`, and each returns triangles rather than a picture. `compose_still` in `src/gui/main_tabs/splash_screen_painter.py` runs every generator into its own layer, adds a bloom, a blur, grain, a tone grade and a vignette, and writes the picture above.

```
  for name, groups in surface.mark_layers(spin, pulse, moment):
      layer = _pillow(bake_groups(groups, side))
```

The order the layers draw in

`mark_layers` returns one entry per layer, back layer first. The coiled serpents draw after the aura, so the compass legs do not cover them.

Order	Layer	Generator
1	Winged caduceus	`caduceus_faces`
2	Compass	`compass_faces`
3	Scythe	`scythe_faces`
4	Scale	`scale_faces`

Order	Layer	Generator
5	Fiery aura	`aura_faces`
6	Coiled serpents	`serpent_faces`
7	Reptilian eye	`eye_faces`

`MARK_LAYER_ORDER` in the painter holds the same order for the still frame.

The letter forms of the maxim

Every capital in the maxim is a chain of straight strokes. No stroke is a curve, so the words read as archaic capitals cut with a chisel.

```
  if letter == "L":
      return [[[0.13, -0.46], [0.13, 0.42], [0.56, 0.42]]]
```

`letter_strokes` returns the chains and `etched_faces` lays them along a leg.

How the aura reads as one body

Each flame spreads half of its own share of the turn at its base, so the base of one flame meets the base of the next. The thirteen flames form one wreath that turns together, and each flame still tapers to its own point.

```
  AURA_BASE_SPREAD_DEGREES = FULL_TURN_DEGREES / AURA_POINTS / 2.0
```

What moves at launch

Four things move while the splash is up, and all four run on clocks the splash already had.

What moves	How
The whole mark	Fades in from 0.2 s, then breathes with the rings
The aura	Turns with the rings, fifteen degrees a second
The pupil	Narrows and widens on the same breath
The scale beam	Swings once, then settles level

How far the coil reaches, and which arcs draw behind the staff

The coiled serpents are drawn at the splash screen's own size, not at an inspection size, so the coil has to clear the staff by enough to read as two snakes. It swings out from the centre line by a fixed share of the mark's radius, and each strand thickens and thins along its own length.

`src/gui/main_tabs/splash_screen_surface.py` — the reach and the strand

```
  SERPENT_BOW = 0.220
  #: The thinnest and the widest half-thickness of one strand.
  #: SERPENT_HALF_MIN plus SERPENT_HALF_SWELL is twice STAFF_HALF_BOTTOM.
  SERPENT_HALF_MIN = 0.016
  SERPENT_HALF_SWELL = 0.046
```

At its widest a strand is twice as thick as the foot of the staff it wraps, so neither wire disappears into the staff's own channel. The widest figure is tied to the staff rather than typed on its own, so a change to the staff carries through to the serpents.

Each serpent crosses the staff several times, and half of those arcs pass behind it. The generator sorts them into a behind set and a front set, gives the two sets opposite shading, and hands back the behind set first so the front arcs draw over it. A crossing therefore reads as a crossing rather than as one flat ribbon.

`src/gui/main_tabs/splash_screen_surface.py` — the behind arcs are shaded apart

```
            back = (turn % 2 == 0) == (phase > 0)
            wraps = behind if back else infront
            depth = -SERPENT_DEPTH_BIAS if back else SERPENT_DEPTH_BIAS
            wraps.append((band, BLUE, SERPENT_ALPHA_SHARE, depth))
```

Read off the generator on the shipped tree: sixteen groups holding 1072 triangles, four arc groups behind the staff and four in front, and three shade values across the whole component.

What was read	Figure
Groups the generator returns	16
Triangles in those groups	1072
Arc groups behind the staff	4, handed back first
Arc groups in front of the staff	4
Shade values present	-0.22, 0.0 and +0.22

The browser build holds none of this geometry. It replays the triangle list the painter sends it, so one generator serves both builds and there is no second copy to fall out of step.

What the launch frame shades, and what only the still picture gets

The section *How it is built* above says:

> `compose_still` *in* `src/gui/main_tabs/splash_screen_painter.py` *runs every generator into its own layer, adds a bloom, a blur, grain, a tone grade and a vignette, and writes the picture above.*

That sentence describes the picture on this page. It does not describe a launch. `compose_still` has no caller in the tree, so the bloom, the blur, the grain, the tone grade and the vignette reach the still picture alone. A launch goes through `mark_ops` in the same module, which emits the triangle list and no post step.

Read off both routes on the shipped tree:

What was read	Figure
Callers of `compose_still`	0
Callers of `mark_ops`	1, `main.py`
Triangle groups the launch frame draws	76
Distinct triangle tones in the launch frame	810
Post steps applied to the launch frame	0
Post steps `compose_still` applies	5

So the mark is not flat. `toned_mesh` gives every triangle its own tone of its layer's colour, and the launch frame carries 810 of them across the seven layers. What a launch does not carry is the surface texture: no bloom around the aura, no grain, no tone grade and no vignette.

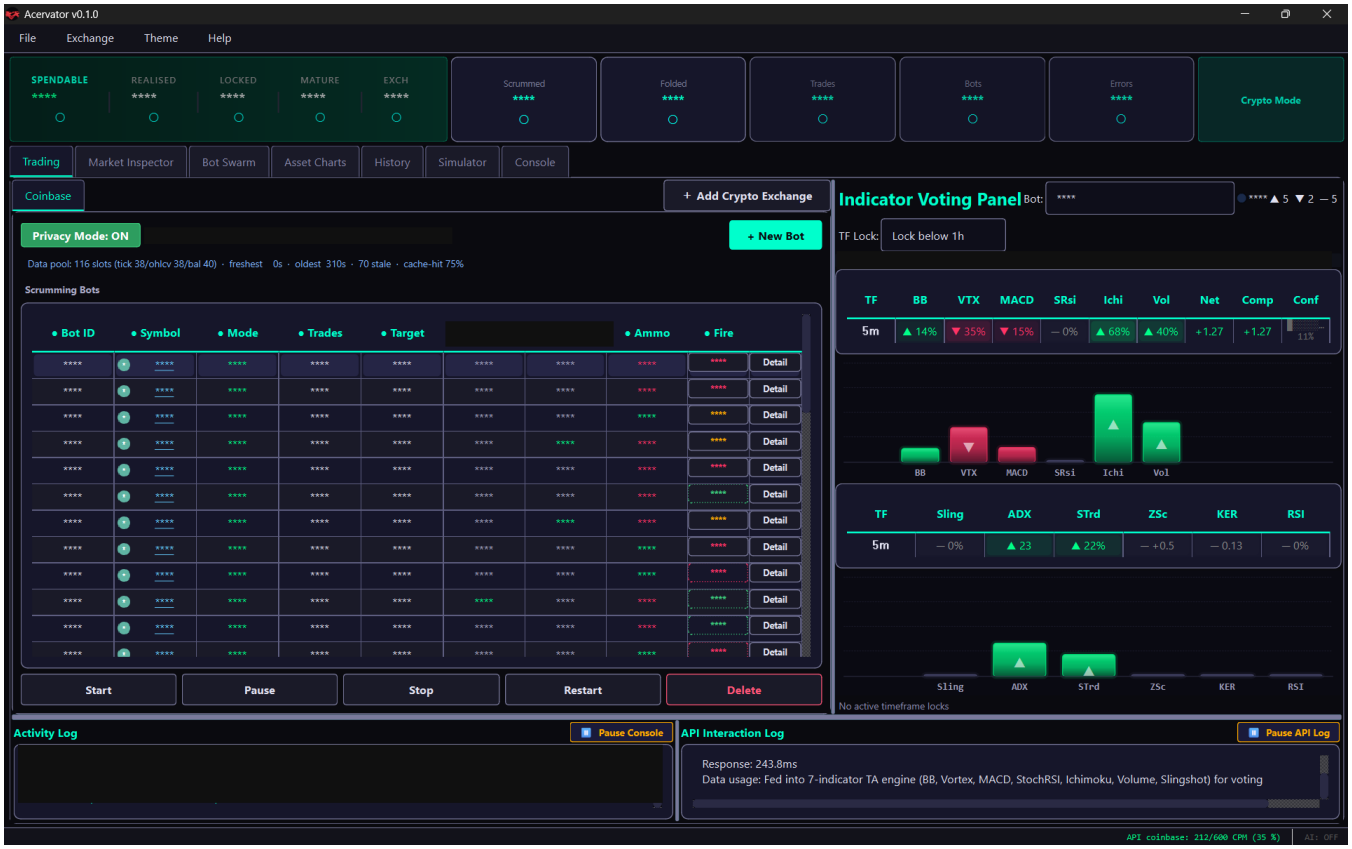
`src/gui/main_tabs/splash_screen_painter.py` — the two routes

```
def mark_ops(cx: Any, cy: Any, alpha: int, spin: Any, pulse: Any, t: Any) -> list:
    """The mark's painter calls, taken from `sigil_ops` in the surface module."""
    return surface.sigil_ops(cx, cy, alpha, spin, pulse, t)
```

The five post steps named above are `with_bloom`, `with_outer_blur`, `with_grain`, `with_grade` and `with_vignette`. All five are present and all five sit behind `compose_still`.

Trading Tab

Here you see the first subsystem we are going to cover and this will primarily be due to familiarizing the prospective investor or user with the core trading philosophy and strategies that drive Acervator. To begin, the platform runs locally on whichever hardware is selected. A single authorization phase requiring a purchased license key will be the only non-exchange communication the application will ever need. The user’s keys and secrets are stored locally and encrypted after API handshake verification passes which allows the chosen exchange to be initialized. At a later development stage, I have plans for dedicated hardware that uses a hardware key for quick boot into a given user’s account and also allows the platform to run in isolation under Linux.



The Trading tab, with Privacy Mode on.

This is the whole tab in one capture. The title bar and the menu row open it. The header strip and the tab row run under them. Below those the exchange sub-tab fills the left with the Scrumming Bots table and the command bar, the Indicator Voting Panel fills the right, and the Activity Log and the API Interaction Log close the foot. The status bar carries the API load pill and the AI state label. The parts below take that screen in that order.

Privacy Mode is on, so every masked field draws four asterisks and the bot table reads as ten columns of them. The title bar is the one reading no part below names. It carries the version the tree answered with on the day of the capture. Nothing types that string out. Git answers for a source checkout and the baked file answers for a frozen bundle, so the title cannot name a release the build is not.

`src/gui/main_window.py` — the window title

```
self.setWindowTitle("Acervator v" + __version__ + "")
```

Press a class segment in the header strip and the title is rewritten to name the active asset class, as `Acervator — CRYPTO LAYER`. The version leaves the title for the rest of the session, and only a restart brings it back.

`src/gui/main_tabs/asset_class_surface.py` — the title a class writes

```
self.setWindowTitle(window_title(key))
```

The capture above predates the two added sectors. Its square carries four segments where the running window now carries six, and its title names a layer the same way. Everything else in the capture stands. Replacing it needs a fresh screenshot of the running platform.

The screen itself

Functional. One method builds this whole screen. It makes one page per trading layer, one card for every asset class that has no layer, and stacks them so only one is on show at a time. Crypto and Stock each have a layer; Commodities and Forex share the card. The opening page is read off the taxonomy rather than typed. A second method builds the strip along the top, and that strip stays put on every tab. Privacy Mode is on in the figure, so each masked field draws four asterisks where its number would be.

`src/gui/main_tabs/trading_tab.py` — `TradingTabMixin._build_trading_tab`

```
self._trading_stack.addWidget(crypto_page)
self._trading_stack.addWidget(stock_page)
self._trading_stack.addWidget(self._make_unlayered_page())
self._trading_stack.setCurrentIndex(acs.layer_page("crypto"))
```

OVERTAKEN, and the sentences above are kept as written. Every asset class has its own trading layer. The stack holds four layers in taxonomy order and keeps the card page behind them, so Commodities and Forex each draw their own empty state instead of sharing one card.

`src/gui/main_tabs/trading_tab.py` — one layer per layered class

```
for layered in acs.layered_classes():
    page, bar, store, card = _make_layer(layered, acs.accent(layered))
    self._class_layers[layered] = {
        "page": page,
        "tabs": bar,
        "exchange_tabs": store,
        "placeholder": card,
    }
    self._trading_stack.addWidget(page)
```

The taxonomy names the four classes a layer stands behind.

`src/gui/main_tabs/asset_class_surface.py` — the classes a layer stands behind

```
LAYERED_CLASSES = frozenset({"crypto", "stocks", "commodities", "forex"})
```

OVERTAKEN, and the sentence and block above are kept as written. Six sectors are declared and every one stands behind its own layer, so the stack holds six layers in taxonomy order and keeps the card page behind them. Indices and Futures / Perps each draw their own empty state.

`src/gui/main_tabs/asset_class_surface.py` — the sectors a layer stands behind

```
LAYERED_CLASSES = frozenset(
    {"crypto", "stocks", "commodities", "forex", "indices", "futures_perps"}
)
```

Sector is the operator's word for an investment product category, and these six are the categories the venue's own market list draws.

> *"We will need to add the two additional Sectors (really mean that as investment product categories...its my invention or renaming) which are Indices and Futures / Perps."*

Each sector draws the stack page its position in the taxonomy names, and paints its empty state in its own accent. Pressing its segment is what shows the page.

sector	the page it draws	its accent
Crypto	page 0	`#00ccaa`
Stock	page 1	`#6699ff`
Commodities	page 2	`#ffaa00`
Forex	page 3	`#4fc3ff`
Indices	page 4	`#ffd700`
Futures / Perps	page 5	`#ff00aa`

No venue serves Indices or Futures / Perps yet, so each page reads `No <sector> Exchanges Configured` and its Add Exchange button cannot act.

A sector is the first of three nested levels. What each sector's venue list holds today, and what products a venue serves under it, are in 16-sector-exchange-product-tree.md, with the count behind every empty row.

The hint under that card takes its article from the sector name, so the Indices page reads "Add an Indices exchange to begin trading" and the other five keep "a". The Qt page and the React page read the one sentence.

`src/gui/main_tabs/asset_class_surface.py` — the article a sector name takes

```
def article(label: Any) -> str:
    first = str(label or "").strip()[:1].lower()
    return ARTICLE_VOWEL if first in ARTICLE_VOWEL_LETTERS else ARTICLE_DEFAULT
```

Design intention. The stack holds one page per trading layer and one card for every class without one. A class with no layer shares that card rather than getting an empty layer, because an empty layer would offer a venue list it cannot serve. Which page a class draws is stated once, in the taxonomy.

`src/gui/main_tabs/asset_class_surface.py` — `layer_page`

```
layered = layered_classes()
key = normalise(name)
return layered.index(key) if key in layered else len(layered)
```

OVERTAKEN, and the paragraph above is kept as written. The stack holds one page per asset class and keeps the card page last, which no live class now reaches. An empty layer offers no venue list. It draws its empty state and an Add Exchange button that cannot act while no venue serves the class.

`src/gui/main_tabs/asset_class_surface.py` — what the button answers for a class

```
def add_exchange_enabled(name: Any) -> bool:
    """Whether Add Exchange can act for one asset class."""
    return class_state(name)["served"]
```

The builder counts its own faults when it finishes and reports them on the System Status tab. That count reads zero, and the number it compares against is the taxonomy's rather than a typed one.

`src/gui/main_tabs/trading_tab.py` — the three conditions the count reads

```
self._trading_stack.count() != acs.stack_pages(),
_crypto_page != acs.layer_page("crypto"),
_stock_page != acs.layer_page("stocks"),
```

The Modulus Bot this section names has no module behind it. This manual marks it unbuilt where it reaches it.

The header strip

Functional. Seven columns and five counter cards run along the top, and the asset class square ends the row. The columns read SPENDABLE, REALISED, P/L, LOCKED, MATURE, EXCH and AMMO. One call to `get_aggregate_stats` fills all of them once a tick. Spendable takes the wallet cash, Locked takes the value tied up in crypto, and Exch counts the open exchange sub-tabs.

The five cards are Scrummed, Folded, Trades, Bots and Errors. Scrummed and Folded total the fleet's sold and bought dollars, Bots counts the bots that are running, and Errors totals the lifetime error count. Trades counts

the orders the exchange filled for the fleet since the year-to-date anchor, one per order however many pieces the exchange filled it in, and a bot the exchange has not answered for completely counts nothing rather than its last figure. Click Errors and the rolling error log opens. The small circle under every column and every card is a privacy dot, and it masks that one field on its own.

`src/gui/main_tabs/header_strip_surface.py` — the seven columns, declared once

```
KPI_COLUMNS = (
    {"key": "spendable", "label": "SPENDABLE", ...},
    {"key": "total_realised", "label": "REALISED", ...},
    {"key": "pnl", "label": "P/L", ...},
    {"key": "locked", "label": "LOCKED", ...},
    {"key": "mature", "label": "MATURE", ...},
    {"key": "exchanges", "label": "EXCH", ...},
    {"key": "total_ammo", "label": AMMO_LABEL, ...},
)
```

Design intention. The strip answers one question at a glance: what the fleet holds, what it has earned, and what it has spent. The window no longer writes the payload itself. It calls the same builder the React strip calls, so one function decides what the seven columns hold and neither host can drift from the other. `_refresh_dashboard` runs the tick that calls it, every two seconds.

`src/gui/main_window.py` — `_write_header_strip`

```
self._spendable_widget.update_profits(
    header_strip_surface.profits_payload(agg, exchanges)
)
```

REALISED. The platform's own first-in, first-out match over the venue's own fills, one figure per bot, added across the fleet. The venue carries no lifetime realised figure for a spot position, so the fills are the venue's and the cost basis is the venue's while the match is the platform's. Realised draws the empty marker when the fill history cannot be walked to its end, so the column never shows a figure added up from part of a history.

`src/exchange/position_health.py` — `compute_position_health`

P/L. The unrealised profit and loss the exchange answers across every bot's open position. `src/trading/scrumming/reconciliation.py` writes each bot's unrealised figure from the venue, and the column adds them.

`src/gui/main_tabs/header_strip_surface.py` — `unrealised_amount`

```
return exchange_amount(stats, "total_unrealized_exchange")
```

MATURE. The part of a position's value that sits over three times its cost. A position worth $350 on a $100 cost basis holds $50 of mature profit, not $250.

`src/trading/smart_wire.py` — `mature_profit_usd`

AMMO. Every bot's Target Delta added together, in whole dollars with a thousands mark. It draws green while more bots hold more than their target and red while more hold less, so the colour follows the count of bots and

never the size of the figure. That makes the column the majority of the colours the Ammo cells already draw bot by bot, so the row and the bot list cannot disagree.

`src/gui/main_tabs/header_strip_surface.py` — `ammo_lean`, the colour's own rule

```
if scrum > fold:
    return TERRITORY_SCRUM
if fold > scrum:
    return TERRITORY_FOLD
return TERRITORY_AT_TARGET
```

The sum is taken inside the loop that already prices every bot's position, so no second pass over the fleet is made for it.

`src/trading/container/aggregation.py` — the sum

```
total_target_delta_usd += target_delta(_bot_pos_val, _target_usd)
```

AMMO reads the sector in front. "Every bot" names the bots of the pressed sector, not the whole fleet. The bot list beside the strip already shows one sector's rows, and the total is summed over exactly those rows, so the cell and the column beneath it cannot report different fleets.

`src/gui/main_tabs/header_strip_surface.py` — `sector_ammo`, the sector's own sum

```
readings = [
    ammo_reading(row)
    for row in class_filter_surface.bots_of_class(statuses, sector)
]
if not any(ammo_target(live, held) > 0 for _position, live, held in readings):
    return None
return fleet_ammo(readings)
```

`bots_of_class` is the one filter the rows are narrowed with, and `ammo_reading` reads a row the way that row's own Ammo cell reads it, in `src/gui/main_tabs/bot_status_table_surface.py`. One sum serves all six sectors: `src/trading/target_bands.py`, in `fleet_ammo`.

A sector holding no bot, and a sector whose bots carry no Target Balance above zero, each have nothing to sum. The cell draws the em dash in the muted skin there, the same marker every other unreadable amount on the strip draws, and never a zero. A sector whose bots sit exactly on their targets does have a reading, and that reading is `$0`.

The cell follows a sector press at once rather than waiting for the two-second dashboard tick: `src/gui/main_tabs/class_filter_tab.py`, in `_repaint_header_strip`, redraws the strip as the last step of `_apply_asset_class`.

Ammo means one quantity on all six sectors, because one `target_balance` on one `BotConfig` is what every variant trades around. `src/trading/scrumming/sizing.py`, in `SECTOR_VARIANTS`, names six Scrumming variants, one per sector. What differs between markets is the order grain, and `UNIT_RULES` keys that by sector **and** venue, so five of the six sectors hold both a fractional and a whole-unit row. A whole-unit market cannot always close the gap it reads. The gap is still the gap.

Where a column stays empty. The venue's portfolio breakdown answers a cost basis, an average entry price and an unrealised profit per open position, and each bot reads those three from it. Where the exchange has answered for no bot, the column is handed nothing rather than a computed zero, so an empty column is never a figure the platform made up.

`src/exchange/base.py` — `SpotPosition`

```
  asset: str
  cost_basis_usd: float
  avg_entry_price: float
  unrealized_pnl_usd: float
```

`src/gui/main_tabs/header_strip_surface.py` — `exchange_amount`

```
  answered = int(data.get(EXCHANGE_FRESHNESS_KEY, 0) or 0)
  if answered <= 0:
      return None
```

An empty column stays empty under Privacy Mode. The mask replaces a number with four asterisks and leaves the empty marker alone, so the operator can always tell a hidden figure from a missing one.

`src/core/privacy_mask_registry.py` — `mask_or`

```
  text = str(value)
  if field_id not in ALL_FIELD_IDS or text == ABSENT_TEXT:
      return text
```

What one unreadable figure costs. One cell, and only that cell. The row holds twelve: the six counter cards declared in `COUNTER_CARDS` and the six columns declared in `KPI_COLUMNS`. Every figure is admitted on its own before it is written, so a stored value the platform cannot read as a number draws the dash in its own cell and the eleven beside it still carry their figures.

`src/gui/main_tabs/header_strip_surface.py` — the admission every money cell shares

```
  def money_amount(value: Any) -> Any:
      return as_finite_float(value)
```

`as_finite_float` is the bot container's own money rule, so the strip and the money path refuse the same values: a stored flag, a not-a-number, an endless number, a text number, and an integer too large to be a dollar amount. `whole_count` holds the same line for the three cards that draw a count, so Trades, Bots and Errors draw a whole number or the dash and nothing else.

`src/gui/main_tabs/header_strip_surface.py` — `card_text`, the one renderer both hosts call for a counter card

```
  if card["format"] == COUNT_FORMAT:
      return count_text(raw)
  return money_text(raw)
```

A read that refuses anyway is caught in `src/gui/main_window.py, in _strip_cell_text`, which logs the refusal with its traceback and gives that one card the dash. The Bot Settings window guards a tab the same way, in `src/gui/bot_live_settings.py, in _add_guarded_tab`.

The refresh re-reads, it does not cache. `_refresh_dashboard` fetches the aggregate afresh on every tick, so an unreadable stored value is read again every two seconds for as long as it is there, and it costs the same one cell on every one of those reads. The no-fleet path runs inside the tick's own catch, so a Simulator or paper aggregate that cannot be read does not escape the timer slot.

Both builds draw from one writer. `_write_header_strip` fills the Qt cards and the React cards alike, and `build_view_model` renders the bridge's copy of the counter cards through the same `card_text`, so neither host admits a figure the other refuses.

The total one bot cannot be read into. The twelve cells are drawn from one aggregate, and the aggregate is summed from every bot's stored figures. `src/trading/container/aggregation.py, in get_aggregate_stats` admits each of those figures through `FleetTotals.money`, `FleetTotals.count` and `FleetTotals.flag`. Those three read by `as_finite_float` and `as_whole_count`, the same two rules the cells draw by. A figure one of them refuses adds nothing to its total, and the total is named in the aggregate's `unreadable_totals` list.

`src/trading/container/aggregation.py` — the admission, and the mark it leaves

```
def money(self, value: Any, *totals: str) -> Optional[float]:
    amount = as_finite_float(value)
    if amount is None:
        self.unreadable.update(totals)
    return amount
```

`src/gui/main_tabs/header_strip_surface.py, in aggregate_amount` reads that list, and a total it names answers nothing, which draws the dash. Both hosts read it: `_write_header_strip` passes every counter card through it, and `profits_payload` passes SPENDABLE, P/L, LOCKED, MATURE and AMMO. A marked total draws neither a sum one bot is missing from nor a zero.

One bot's figures reach some totals and not others. The named groups in `src/trading/container/aggregation.py` say which: `SCRUMMED_TOTALS`, `FOLDED_TOTALS`, `CASH_TOTALS`, `POSITION_TOTALS`, `MATURE_TOTALS`, `AMMO_TOTALS`, `FRESHNESS_TOTALS` and `STATE_TOTALS`. A bot's position value feeds three of the cells, so a price the platform cannot read marks LOCKED, MATURE and AMMO together while SPENDABLE, P/L and the six cards keep their figures.

The aggregate itself no longer refuses. A bot whose `stats` cannot be reached marks `STATS_TOTALS` and the pass moves to the next bot, and a bot whose run state cannot be read marks `STATE_TOTALS`. No reading in the pass raises, so `_refresh_dashboard` is never left holding an earlier tick's figures. `FleetAggregationMixin._log_unreadable_totals` writes one warning naming the marked totals, and writes again only when that list changes.

The row's width budget. Every part of the row declares a floor, and the sum of the floors is the narrowest the window opens at. Without them a part's minimum is the width of its own text, and a six-figure amount widens the whole window past the screen.

`src/gui/main_tabs/header_strip_surface.py` — the floors and the shares

```
SPENDABLE_MIN_W = 180
COUNTER_MIN_W = 48
COUNTER_NATURAL_W = 118
TOP_ROW_STRETCH = [3, 1, 1, 1, 1, 1, 0]
```

The money strip's floor is 180 px and a counter card's is 48 px, against the 118 px that card wants for a whole money amount of `$12,345.67`. The money strip takes three shares of the spare width and each counter card takes one. The asset class square takes none, because its side is a declared number.

OVERTAKEN, and the block and the paragraph above are kept as written. `SPENDABLE_MIN_W` is not a constant and is not 180. `spendable_min_w` answers the money strip's narrowest width, as `spendable_w` at `MONEY_CELL_MIN_W`: every `KPI_COLUMNS` column's room at the smallest type an amount draws at, each rule's own width and the gap either side of it, and `SPENDABLE_TEXT_PAD` for the frame. Six columns answer 512 px.

OVERTAKEN, and the block and the paragraph above are kept as written. `COUNTER_MIN_W` is not 48. It is 88 px, the width the row draws a counter slot at, and it is a typed figure on purpose: raising it would move every counter slot. `MONEY_CELL_MIN_W` plus `COUNTER_TEXT_PAD` now reads 66 px, which is 22 px under that floor, so a card at its floor holds the widest amount whole and no floor had to move to give it that. What the card's amount gained is the room inside the floor, which the card's own frame padding widened.

OVERTAKEN, and the block and the paragraph above are kept as written. `COUNTER_NATURAL_W` is not 118. It is 144 px, the width the row draws a counter card at when it can afford it, and `COUNTER_TEXT_W` is that width less `COUNTER_TEXT_PAD`. The React card reads `COUNTER_NATURAL_W` for its own sizeHint, so the width is held and the room inside it is what the card's margin and then the card's own frame padding widened — 110 px of room at first, then 118 px, and 142 px now.

OVERTAKEN, and the block and the paragraph above are kept as written. `TOP_ROW_STRETCH` is not `[3, 1, 1, 1, 1, 1, 0]` and is not written out. It is built from `KPI_COLUMNS` and `COUNTER_CARDS`: the money strip takes one share per column it holds, each card takes `COUNTER_STRETCH`, and the class square takes `CLASS_GROUP_STRETCH`. Six columns and six cards answer `[6, 1, 1, 1, 1, 1, 1, 0]`, which is the share list the row already drew, and it holds one share per `TOP_ROW_ORDER` slot by construction. The shares themselves are unchanged; only where they come from is.

`src/gui/main_tabs/header_strip_surface.py` — the floors and the shares today

```
MONEY_CELL_MIN_W = 64
COUNTER_MIN_W = 88
COUNTER_NATURAL_W = 144
COUNTER_TEXT_W = COUNTER_NATURAL_W - COUNTER_TEXT_PAD
TOP_ROW_STRETCH = [
    len(KPI_COLUMNS),
    *([COUNTER_STRETCH] * len(COUNTER_SLOTS)),
    CLASS_GROUP_STRETCH,
]
```

OVERTAKEN, and the paragraph at the head of this section is kept as written. "the sum of the floors is the narrowest the window opens at" reads true today and did not before. `top_row_min_w` added up `slot_min_w`, and the row floors every slot at `slot_floor_w`. The money strip's two figures are 512 px and 668 px, so the

sum came out 156 px under the row it described: it answered a 1228 px window while the real row, built by `header_strip.py, in HeaderStripMixin._build_header_strip`, reported its own narrowest as 1384 px and Qt refused to draw the window any narrower. `top_row_min_w` now adds up `slot_floor_w`, so `window_min_w` answers 1384 px and the two agree. The figure follows the column list on its own: a seventh card answers 1476 px and a seventh money column answers 1492 px.

`src/gui/main_window.py` still pins the window's own floor to 1400 by 900 by hand. 1400 px holds the row as it stands and would not hold a seventh slot.

OVERTAKEN, and the paragraph above is kept as written. The window no longer pins that width by hand. `main_window.py, in window_min_w_px` answers the wider of `WINDOW_MIN_W_PX`, the window's own figure, and `header_strip_surface.window_min_w`, which is the one declaration of what the top row's slots need. Six cards answer 1400 px, the figure the window already held, and a seventh answers 1476 px, which is what Qt reports as the row's own minimum and 76 px past where the hand figure stopped. The height is `WINDOW_MIN_H_PX` and is unchanged at 900 px.

The margins the two newest columns are drawn from. A column leaves 3 px either side of the rule between it and its neighbour, the rule itself is 12 px, the panel's own side margin is 6 px, and a counter card's side margin is 4 px.

`src/gui/main_tabs/header_strip_surface.py` — the four figures

```
SPENDABLE_SIDE_MARGIN_PX = 6
SPENDABLE_COLUMN_GAP_PX = 3
SPENDABLE_RULE_W_PX = 12
CARD_SIDE_MARGIN_PX = 4
```

OVERTAKEN, and the block and the paragraph above are kept as written. `CARD_SIDE_MARGIN_PX` is not 4 and the Qt counter card leaves no side margin of its own. `PANEL_FRAME_INSET_PX` names what the theme's frame rule insets — 12 px of padding and a 1 px border on each side — so a margin on top of it was a second inset, and it cost the amount the digits `MONEY_CELL_MIN_W` reserves room for. The three money strip figures above are unchanged.

OVERTAKEN, and the block and the paragraph above are kept as written. The card's side margin is declared twice, once per host, because the two hosts apply it differently. `CARD_SIDE_MARGIN_PX` is the margin the Qt card holds inside the frame inset it already has. `CARD_PAGE_SIDE_MARGIN_PX` is the padding `CARD_LAYOUT` carries to the page, where it replaces the frame's inset instead of sitting inside it, and where the slot's own drawn width follows it. It is held at 4, the figure the row already drew, so no slot moves on either host.

OVERTAKEN, and the block and the paragraph above are kept as written. The Qt card no longer wears the theme's side padding at all. `CARD_STYLE` is the card's own frame, 12 px of padding above and below and none either side, and a sheet on the card itself wins for that card alone, so the strip and every other panel keep the theme's padding and no slot's width moves. What a card's frame and margins take off its text is now the 1 px border on each side, so a card drawing 144 px wide gives its amount 142 px of room, against the 90 px the widest amount asks for.

`src/gui/main_tabs/header_strip_surface.py` — the card's own frame

```
CARD_FRAME_SIDE_PX = 0
CARD_STYLE = f"StatCard {{ padding: {PANEL_FRAME_PAD_PX}px {CARD_FRAME_SIDE_PX}px; }}"
COUNTER_TEXT_PAD = 2 * (
    CARD_SIDE_MARGIN_PX + CARD_FRAME_SIDE_PX + PANEL_FRAME_BORDER_PX
)
```

One money cell, one width rule. The row draws money in two kinds of panel — a column of the money strip and a counter card — and both reserve the same room for it. `header_strip_surface.py` names the widest amount the row draws and measures it once at the smallest type a money cell shrinks to.

`src/gui/main_tabs/header_strip_surface.py` — the one declaration

```
WIDEST_AMOUNT = "$-999,999.99"
MONEY_CELL_MIN_W = 64
KPI_COLUMN_W = money_cell_w(VALUE_FONT_PX)
```

`money_cell_w` scales that reading to any type size, because a font's advance scales with its pixel size. `spendable_natural_w` is the strip at `VALUE_FONT_PX` and `spendable_min_w` is the strip at `VALUE_FONT_MIN_PX`, so one number answers both.

Every width the row is sized from is held where it was. `COUNTER_MIN_W` and `COUNTER_NATURAL_W` are the widths the row already drew a counter slot at, and `slot_floor_w`, `slot_natural_w` and `slot_stretch` answer the same number for every `TOP_ROW_ORDER` slot as before. The Qt card reaches `MONEY_CELL_MIN_W` of room through `CARD_FRAME_SIDE_PX`, which is room inside the panel and not a wider panel, and the page's card has that room already through `CARD_PAGE_SIDE_MARGIN_PX`, so no slot's drawn size moves on either host.

The page's slot width follows its card's padding and no Qt slot follows its margin. Measured at four window widths in both builds: moving `CARD_SIDE_MARGIN_PX` alone moves no Qt slot at any width, while the same figure carried to `CARD_LAYOUT` moves every page counter slot wherever the row has width above its floors. That is why the two are declared apart, and it is the reason the page and the Qt row draw a counter slot at different widths at all.

A card's margin is its frame's inset and nothing more. The theme's `QFrame[frameShape="6"]` rule already holds a panel's text 12 px off its border and draws a 1 px border, which `PANEL_FRAME_INSET_PX` names. A counter card adds no side margin of its own on top of it, so the amount keeps the digits `MONEY_CELL_MIN_W` reserves room for at the narrowest window.

The slot order and the slot shares are read off the cards. `header_strip_surface.py, in` `COUNTER_SLOTS` is built from `COUNTER_CARDS`, `TOP_ROW_ORDER` is the money strip's slot, those cards and the class square, and `TOP_ROW_STRETCH` gives the strip one share per `KPI_COLUMNS` column, each card one share and the class square none. Adding or removing a card changes that card's own slot and no other panel's width.

OVERTAKEN, and the paragraph above is kept as written. "Adding or removing a card changes that card's own slot and no other panel's width" holds for the floor each panel declares and not for the width the row draws it at. Read off the real row at a 1400 px window: the floors in `width_budget` moved for the added or removed card alone, and every other card's drawn width moved with it — 91 px with six cards, 88 px with seven and 110 px with five. The row shares what is left above the floors, so a slot added or taken away changes every slot's share

of that surplus.

A card whose figure the fleet never published draws the dash. `header_strip_surface.py, in counter_amount` reads each `COUNTER_CARDS` entry's `source_key` with no default, so a fleet that publishes no such key answers nothing and the card draws `EMPTY_TEXT`. The live fleet publishes `total_accrued_usd` and the paper ledger and the simulator do not, so ACCUMULATED draws a figure behind Live and the dash behind Paper and Sim.

A shortened amount is measured at the size it is drawn at. `src/gui/widgets/fitted_label.py, in _draw` picks the largest size that fits through `_fitted_for`, then shortens the text through `_metrics_at` at that same size. `fontMetrics` answers the size the style sheet last applied, which is the size before the shrink, so a reading taken from it shortens an amount that had already been made to fit.

A shortened amount ends in an ellipsis. Every caption and every amount in the columns and the counters is an `ElidingLabel`. It keeps the whole text and draws what the width holds, so `$128,456.78` in a narrow row reads `$128,45…` and never `$128,45`. The whole text goes to `setAccessibleName`, so a shortened amount still reaches a screen reader whole.

`src/gui/widgets/eliding_label.py` — `sizeHint`

```
hint = super().sizeHint()
metrics = self.fontMetrics()
pad = hint.width() - metrics.horizontalAdvance(super().text())
return QSize(metrics.horizontalAdvance(self._full) + max(pad, 0), hint.height())
```

An amount draws as characters. An amount is a stored figure, so an `ElidingLabel` is fixed to plain text. A figure that arrives carrying angle brackets shows those brackets, and the label builds nothing from them. Every user of the class draws the same way, including the `FittedLabel` the header stat cards use for their amounts.

`src/gui/widgets/eliding_label.py` — `ElidingLabel.__init__`

```
self.setTextFormat(Qt.PlainText)
```

At a 900-pixel window the row is wider than the window and the Scrummed and Folded amounts are cut short, because the cards reach their declared floor. Seven columns is more width than 900 pixels holds before a single card or the square, so no margin closes it. Which figure to prefer at that width is not decided.

08-tabs/portfolio-panels.md covers the strip in full.

The tab row

Functional. The row of main tabs takes its order from one list. Each label in the list is moved to its own index at build time. Any tab the list does not name keeps the position it was added at.

`src/gui/main_tabs/main_window_surface.py` — the order, declared once

```
CANONICAL_TAB_ORDER = (
    SIM_TAB,
    PAPER_TAB,
    LIVE_TAB,
    CHARTS_TAB,
    INSPECTOR_TAB,
    SWARM_TAB,
    ACCUMULATION_TAB,
    HISTORY_TAB,
    STATUS_TAB,
    CONSOLE_TAB,
)
```

Each of those ten names is a constant holding the label the tab bar shows, and the main window applies the order once, after the last builder has run.

`src/gui/main_window.py` — where the order is applied

```
self._reorder_main_tabs(list(CANONICAL_TAB_ORDER))
```

Design intention. The order should read as the promotion order. Practise first, then paper, then real money, then the screens that inspect the trade. One list decides it, so the order cannot drift as tabs are added.

`src/gui/main_window.py` — the move loop

```
tab_bar = self._main_tabs.tabBar()
for target_idx, name in enumerate(desired):
    for cur_idx in range(self._main_tabs.count()):
        if self._main_tabs.tabText(cur_idx) == name:
            if cur_idx != target_idx:
                tab_bar.moveTab(cur_idx, target_idx)
            break
```

One sub-tab per exchange

Functional. One sub-tab holds one exchange. Along its header sit Privacy Mode, a news headline, and the + New Bot button that opens the wizard for this venue. Privacy Mode toggles every mask the register holds at once, and the register holds twenty-one fields; the button's own tooltip still says eighteen. Under the header is a data pool line: how many slots are held by kind, the age of the freshest and the oldest, how many have run past their time to live, and the cache hit ratio. The line keeps that seat on a venue holding no bots. Both bot tables hide themselves while their lists are empty, which leaves the line's own box as tall as the panel, and its words draw at the top of that box on every sector. At the right of the sub-tab row sits the button that adds another venue.

`src/gui/widgets/exchange_tab.py` — `ExchangeTab.__init__`

```
self._privacy_mode_btn = QPushButton("Privacy Mode: OFF")
self._privacy_mode_btn.setToolTip(
    "Toggle ALL 18 privacy masks at once. When ON, every "
```

Current reading, 2026-10-29. The Privacy Mode description is one sentence that both builds read, within twenty words.

`src/gui/main_tabs/exchange_tab_surface.py` — `PRIVACY_TOOLTIP`

```
PRIVACY_TOOLTIP = (
    "Masks every registered field at once, for screen sharing. Each field's own "
    "dot still works when this is OFF."
)
```

Design intention. One venue, one tab, one swarm. Add a second exchange and it arrives beside the first with its own bots and its own data pool, and nothing about the first changes.

`src/gui/main_window.py` — `_sync_exchange_tabs`

```
def _sync_exchange_tabs(self) -> None:
    """Add a tab for each configured exchange missing one, in its own layer."""
```

When the data pool stops refreshing

Functional. A red line appears directly under the data pool line when nothing has come back from the venue for a full minute. It names the venue, says how long the screen has been showing old figures, and tells you to restart the platform. It is hidden at every other time. It clears itself the moment a reading lands.

The line no longer sends you to a restart. It says the platform is clearing the block itself on the next fetch, and to restart only if the line stays.

`src/gui/widgets/exchange_tab.py` — `ExchangeTab._draw_pool_stopped`

```
"DATA POOL STOPPED REFRESHING — nothing has come back from "
f"{self._exchange_name} for {span_words(stopped_for)}. Every "
"price, balance and ammo figure below is at least that old. "
"The platform clears a block of its own on the next fetch; "
"restart only if this line stays."
```

Functional. The data pool line directly above it ends with a count of blocked fetches the platform cleared, and it carries that count only while the count is above zero. A run that never stopped draws nothing there.

`src/gui/widgets/exchange_tab.py` — `ExchangeTab._cleared_words`

```
noun = "blocked fetch" if count == 1 else "blocked fetches"
return f"  ·  {count} {noun} cleared"
```

Design intention. Single slots passing their time to live is ordinary. A ticker slot lives five seconds and a balance slot lives ten, and the pool refetches a slot because it aged out. So the line never watches one slot. It watches the age of the newest reading anywhere in the pool. While the pool is healthy, something was fetched a moment ago, so that age stays near zero even when most slots are stale.

The span is one minute. It is the window the call rate is measured over, so a rate of zero calls is a complete reading only across that minute. Measured in the pool's own lifetimes it is twelve ticker lifetimes and six balance

lifetimes, and it is twelve passes of the five-second trading tick that drives the fetching.

`src/exchange/fetch_stall.py` — the span and the verdict both tabs read

```
STALL_THRESHOLD_SECONDS = DEFAULT_WINDOW_SECONDS
TICKER_LIFETIMES_PER_THRESHOLD = STALL_THRESHOLD_SECONDS / TICKER_TTL_SECONDS
BALANCE_LIFETIMES_PER_THRESHOLD = STALL_THRESHOLD_SECONDS / BALANCE_TTL_SECONDS
```

What to do when it appears. Restart the platform. Nothing else brings the fetching back. The Console carries the same reading as a log line, so a run that is already closed can still be read there.

What to do when it appears now. Wait one fetch. The data pool replaces a fetch lock whose holder never finished, and the connector replaces its one worker thread when a call is abandoned while that thread is still inside it. Both report one Console line when they do. Restart only if the line is still there after that.

`src/exchange/ccxt_connector.py` — `CCXTConnector._sync_executor_for_submit`

```
fresh = ThreadPoolExecutor(
    max_workers=SYNC_CALL_CONCURRENCY,
    thread_name_prefix=f"ccxt-{self._exchange_id}",
)
self._sync_executor = fresh
```

Functional. The red line and the count beside it now draw on both builds of the venue page. Before this, only the first build drew them: the second build drew the data pool line and nothing under it, so a fleet that had stopped reading the venue looked normal on that build.

A passage above names one file for the words of the red line:

`src/gui/widgets/exchange_tab.py` — `ExchangeTab._draw_pool_stopped`

The current reading is that the words live in the shared venue surface and both pages read them from there. The same holds for the count of cleared blocks, named above as `ExchangeTab._cleared_words`.

`src/gui/main_tabs/exchange_tab_surface.py` — the words both pages draw

```
    if stopped_for is None:
        return NO_POOL_STOP_TEXT
    return POOL_STOPPED_FORMAT.format(
        venue=exchange_name, span=span_words(stopped_for)
    )
```

Design intention. One function answers for both builds, so the two cannot show a different age for the same stop. Read off the real screen and the real page values, with nothing coming back for two minutes and twenty seconds, both builds name the same span and the same count.

what was read	refreshing normally	nothing back for 2 m 20 s
first build, the red line	hidden	shown, naming 2 minutes 20 seconds
first build, the count	absent	2 blocked fetches cleared
second build, the red line	hidden	shown, naming 2 minutes 20 seconds

what was read	refreshing normally	nothing back for 2 m 20 s
second build, the count	absent	2 blocked fetches cleared

Design intention. The Paper Trader and the Simulator draw no data pool row, and neither reads the live market cache, so neither has a venue read that can stop this way. Each one publishes an empty row.

`src/gui/simulator/sim_exchange_tab_surface.py` — the row the Simulator publishes

```
DATA_POOL_ROW_TEXT = ""
```

What comes back on its own

Functional. Two readings rebuild themselves and need no restart. The BTC/USD and ETH/USD strip comes back once its last reading is two minutes old. The cross-pair rows come back once every venue the scout polled is twenty seconds past its reading. Each one hands its last reading to its replacement and marks it held, so the strip keeps both prices and ends `coinbase (held)`, and a cross-pair cell keeps its figure and ends `(held)`. The mark leaves on its own when a fresh reading lands. A reader that has never answered holds nothing to hand on, so the strip still draws a dash for each price and the cell still reads pending. The Console carries one line for each rebuild, and that line says whether the readings were held or lost.

`src/exchange/lazy_singleton.py` — the cached reader is replaced in place

```
            self._instance = instance
            if cached is None:
                self._fault.note_success()
            else:
                self._fault.note_rebuild(reading_carried=reading_carried)
```

Design intention. The old reader is served until its replacement is built, so nothing on the screen is ever handed an empty answer in the gap. A reading that keeps stopping is rebuilt after thirty seconds the first time, then at doubling gaps up to five minutes, so one stop cannot turn into a stream of rebuilds.

`src/exchange/currency_rate_monitor.py` — the rate feed's own test for a stop

```
def _monitor_has_stopped(monitor: CurrencyRateMonitor) -> bool:
    return monitor.snapshot().last_updated > 0 and monitor.is_stale()
```

Functional. Nothing else comes back on its own. The data pool, each bot's own fetching, the trade history and the Market Inspector all fetch outside this mechanism, so the red line above still means a restart. Measured across the source tree: three of thirty-eight venue fetches sit behind it and thirty-five do not. A bot is also handed the scout once, when it is registered, so a rebuilt scout reaches the screen and not the bots.

Functional. Two more mechanisms come back on their own, so the red line above no longer means a restart. The data pool replaces a fetch lock whose holder never finished, which covers its own six fetches. One worker thread serves every call a venue's connector makes, and the connector replaces that thread when the outer wait abandons a call the thread is still inside, which covers the eleven calls the worker serves: the ticker, all tickers, the order book, the candles, the balances, the spot positions, placing an order, cancelling one, reading one back, the open orders and the trade list. Opening the connection runs on that same worker.

`src/exchange/ccxt_connector.py` — the worker a call leaves behind

```
    @property
    def worker_held(self) -> bool:
        """True while the worker thread has entered the venue call and not left it.
```

Design intention. The replacement is built on the next call and not at the moment the wait expires, so a worker that comes back inside the hand-off gap is never replaced. The abandoned call is never sent again, so nothing reaches the venue twice. The call queue in front of the worker is not widened: it still holds eight and still refuses the ninth caller.

Functional. A venue fetch that reaches no connector call still needs a restart. The post-connect trade history scan is one: it drives the synchronous exchange handle on its own thread and does not pass the worker.

Functional. The trade history scan now comes back on its own too. One lock queues every history scan on a venue, and a waiter gives the holding scan a fixed turn. The turn is the same minute the red line above reports a stop over. At the end of a turn the waiter asks whether the holding scan has finished a market. A market finished means the venue is answering, so the waiter waits again, however long the run takes. Two turns with no market finished means the scan has stopped, so the waiter takes a fresh lock and scans. The Console carries one line for each clearance, naming the venue and how long the scan was blocked.

`src/exchange/ccxt_connector.py` — the turn a waiter gives the holding scan

```
            lock = self._history_scan_lock
            if lock.acquire(timeout=budget_s):
                self._note_history_progress()
                return lock
```

Design intention. Two turns is two minutes, above the longest one market can legitimately take. A scan that keeps finishing markets keeps its lock, so a long run across a large fleet is never cut short and raises no line. The blocked scan keeps its old lock and is never driven again, so no market it was part way through is read a second time. Nothing new runs while a scan is healthy.

`src/exchange/market_pairs_scout.py` — the scout rebuilds only when every venue it polled has stopped

```
def _scout_has_stopped(scout: MarketPairsScout) -> bool:
    polled = scout.polled_exchanges()
    return bool(polled) and all(scout.is_stale(eid) for eid in polled)
```

Functional. A reading whose very first build fails also comes back on its own, and it needs no restart either. The first attempt is made the moment the dashboard asks for the reader. When the venue refuses, the Console carries the offline line and every later ask is skipped until the cooling-off window closes. One attempt is made per window. The attempt at the end of the first window is the one that answers once the venue is reachable again, and the first window is thirty seconds. Until then the strip draws a dash for each price, because a reader that has never answered holds nothing to hand on.

`src/exchange/lazy_singleton.py` — the cooling-off is asked before every build

```
                if self._fault.quiet():
                    self._fault.note_skip()
                    return cached
                self._constructions += 1
                instance = self._factory()
```

Design intention. A venue that never comes back must not be stormed. The window starts at thirty seconds, doubles after each failed attempt, and stops widening at five minutes. Each Console line names the next wait. Driven with the window shortened to one second, so the thirty did not have to be waited out, a reader whose first build failed answered at the end of its first window. The control is the same reader against a venue that never answers: it made one attempt per window over the same span, answered nothing, and was still holding nothing at the end.

The venue	Accessor calls	Build attempts	First answer
refuses, then answers	389	2	at the end of the first window
never answers	387	3	none

`src/exchange/lazy_singleton.py` — the three figures the windows are built from

```
#: First cooling-off window, seconds.
DEFAULT_BASE_COOLOFF_S = 30.0
#: Ceiling the window doubles up to, seconds.
DEFAULT_MAX_COOLOFF_S = 300.0
#: Window multiplier per further failure.
BACKOFF_FACTOR = 2.0
```

A stopped feed and the crash box are two faults

Functional. Two things have been seen on this screen. A feed stops and the figures stop moving. A Windows box also appears, saying the program tried to use something that was not there. These are two faults and not one. Driven on the current build: making a feed stop never raised the box, and raising the box never stopped a feed.

Functional. A reader that fails is caught where it is built. The Console carries one line naming the feed, what is lost, and the error behind it. Nothing reaches the top of the program, so no box appears. Driven with the rate feed and the pairs scout both failing: both Console lines were written, every screen that reads them drew a blank cell rather than an error, and the box's own channel was never used once.

`src/exchange/lazy_singleton.py` — `LazySingleton.get` catches a failing reader

```
            except Exception as exc:  # noqa: BLE001 - a pump must not raise
                self._fault.note_failure(exc)
                return cached
```

Functional. The box is the other fault. It appears when an error reaches the top of the program with nothing holding it. Driven with that error raised inside a live screen timer: it was reported at the top, the program kept running, and the feed behind it was untouched. The same reader answered before and after, no replacement was built, and no Console line was written.

`src/gui/main_tabs/console_log_handler.py` — `_append_to_widget` holds its own paint failure

```
        except Exception:
            self.handleError(_paint_failure(msg))
```

Functional. One underlying fault can still set both off. A Console line behind a stopped feed has carried the same error wording the box later showed. Where that wording comes from is not settled. Neither symptom produces the other, so each needs its own repair.

Functional. A bot that already holds the pairs scout keeps it while the scout is down. A bot registered during a stop was handed nothing, and its pair list then stayed empty for the rest of the run. The list now holds its last answer, which is what every other stopped reader on this screen already does.

`src/trading/scrumming_bot.py` — `set_market_pairs_scout` refuses an empty hand-off

```
        if not scout:
            return
        self._market_pairs_scout = scout
```

The news line

Functional. The headline between Privacy Mode and + New Bot is one item from the crypto news ticker. The counter in front of it gives that item's place in the batch the ticker holds, so a reading of 23 of 42 marks the twenty-third headline of forty-two. The item itself carries the feed name, a middle dot, then the story title. A timer steps to the next item and wraps at the end of the batch.

`src/gui/crypto_news_ticker.py` — the line the strip draws

```
_prefix = f"[{self._index + 1}/{len(self._headlines)}] "
self._label.setText(_prefix + h.display_text())
```

Design intention. The counter tells you the batch is whole. A feed that came back short shows a smaller second number rather than a strip that looks the same and carries less. With nothing reachable the strip says so in words instead of going blank.

`src/gui/crypto_news_ticker.py` — the empty state

```
self._label.setText("(no crypto news feeds reachable)")
```

A headline draws as characters. A headline is words one of the ten feeds sent, so the strip's label is fixed to plain text. A title carrying <b> or an image address shows those characters on the strip, and the label resolves no address. The React build draws the same characters, because `crypto_news_ticker.js` passes the headline as an element's child.

`src/gui/crypto_news_ticker.py` — `CryptoNewsTicker.__init__`

```
self._label.setTextFormat(Qt.PlainText)
```

The hover box draws as characters as well. The box that appears over the headline names the feed and the address a click would open. That address is a feed's own words too, so every character of it is escaped and the box is given an `<html>` envelope around them. A feed that sends `<img src="…">` inside its address gets those characters drawn back, and the box builds no image. Read back off a live strip, the only tags in the box's text are the envelope's own.

`src/gui/crypto_news_ticker.py` — `hover_text`

```
body = html.escape(HOVER_FORMAT.format(source=source_name, tail=tail))
return "<html>" + body.replace("\n", "<br>") + "</html>"
```

A click opens an http or https address, and nothing else. The address a click hands the system browser comes from the feed, so it is admitted first: `openable_url` refuses a scheme outside http and https, and the strip then refuses an address that names no host. A refused address opens nothing, the log names the feed and the reason, and the hover box says the headline opens nothing. On Windows a scheme-less string is run by the shell rather than the browser, so a bare drive path is one of the addresses this turns away.

`src/gui/crypto_news_ticker.py` — `opening_address`

```
address, refused = openable_url(url)
if not address:
    return "", refused
```

Three routes close the screen and all three halt the strip. An exchange removed in Settings takes its tab off the bar, the tab itself can be closed, and the application can quit. Each route reaches `stop_feeds`, which stops the headline timer, the hourly refresh and the fetch in flight. A tab already gone when the application quits halts nothing and raises nothing.

`src/gui/widgets/exchange_tab.py` — `ExchangeTab.closeEvent`

```
self.stop_feeds()
super().closeEvent(event)
```

Functional. In the React build the strip is drawn by `crypto_news_ticker.js` into the header space the exchange screen keeps between the Privacy Mode button and + New Bot. The exchange screen names that space only when the strip is there; where the strip would not build, the header takes plain space instead and the two buttons keep their positions.

`src/gui/main_tabs/exchange_tab_surface.py` — `react_news_ticker`

```
def react_news_ticker() -> str:
    """The module that draws the news strip, as ``build_news_ticker`` reads it.

    ``ExchangeTabModel`` calls this as its ``news_ticker_factory``, so
    ``news_ticker`` is set and the header keeps the strip's space.
    """
    return NEWS_TICKER_MODULE
```

Functional. Hovering the headline pauses the step timer and moving away starts it again. Clicking the headline opens that story. Each of the three sends one request and redraws the strip from the answer, so the strip on screen matches what the backend holds.

`src/gui/main_tabs/crypto_news_ticker_surface.py` — `CryptoNewsTickerModel.handle_event`

```
if event_type == EVENT_ENTER:
    self.paused = True
    self.calls.append([HOVER_PAUSED])
    return False
if event_type == EVENT_LEAVE:
    self.paused = False
    self.calls.append([HOVER_RELEASED])
    return False
```

The bot tables

Functional. The Scrumming Bots table carries ten columns. Nine are named and the tenth is blank, because that one holds the Detail button. The Extractor table sits underneath. Both tables start hidden and appear when their own list gains a row.

`src/gui/main_tabs/bot_status_table_surface.py` — the ten labels, declared once and carried into the widget as `BotStatusTable.SCRUMMING_COLUMNS`

```
COLUMN_LABELS = (
    "Asset",
    "Symbol",
    "Current Position Value",
    "Trades",
    "Target",
    "Target A15",
    "Target A14",
    "Ammo",
    "Fire",
    "",
)
```

Asset, the first column. The column draws the target asset's own official mark. Where the logo library has already put a file on disk for that asset, the cell draws it and holds no text. Where none is kept, the cell draws the asset's ticker instead, so the row still names what the bot accumulates. Nothing on this path fetches anything: the cell reads the kept directory and no further.

`src/gui/main_tabs/bot_status_table_surface.py` — `logo_cell`

```
    if shown != asset or not asset:
        return cell(shown)
    address = logo_data_address(path)
    if path and address:
        return cell(
            EMPTY_TEXT,
            tooltip=LOGO_TIP_FORMAT.format(asset=asset),
            logo_path=path,
            logo_image=address,
            logo_size=LOGO_SIZE_PX,
        )
    return cell(asset, tooltip=NO_LOGO_TIP_FORMAT.format(asset=asset))
```

The asset a crypto bot names is kept under its base, and a currency pair under the whole pair, so both keys are asked in that order.

`src/gui/main_tabs/bot_status_table_surface.py` — `kept_logo_path`

```
    for asked in (icon_asset_of(symbol), symbol):
        if not asked:
            continue
        found = KEPT_LOGOS.kept_path(asked)
        if found is not None:
            _LOGO_PATHS[symbol] = str(found)
            return str(found)
```

Where a mark comes from. The library sits at `resources/logos`, under the repository, and is filled ahead of any bot. A crypto asset's mark comes from a coin data source; every other asset's mark comes from that organisation's own web site. The library files each mark under its asset class and its sector, so one asset has one file wherever it is met, and the list reads that directory at any depth.

`src/trading/logo_library.py` — where one asset's file is put

```
    def library_folder(asset_class: Any, sector: Any = "") -> str:
        return kept_folder(f"{asset_class}/{sector}")
```

`src/core/asset_logos.py` — the read at any depth

```
    for found in self._cache_dir.rglob(f"{stem}.*"):
        rank = ranks.get(found.name)
        if rank is None or rank >= best_rank or not found.is_file():
            continue
        best, best_rank = found, rank
```

A mark keeps its own shape and its own detail. A mark is drawn inside a box of the logo's own size and is never squeezed to fill it. A mark that is not square keeps its proportions in both builds. A mark whose file holds several sizes is drawn from the size nearest the box rather than from the smallest. A mark smaller than the box is drawn at its own size rather than enlarged. The React page cannot open a file by its path, so the same bytes travel to it as a data address, read once per file.

`src/gui/widgets/bot_status_table.py` — `_draw_logo` hands the file to the icon whole

```
icon = QIcon(path)
found = icon if icon.availableSizes() else None
```

`src/gui/web/bot_status_table.js` — the page bounds the mark rather than setting it

```
style.maxWidth = size + PX;
style.maxHeight = size + PX;
```

HIS.

> *"Live - Asset Column - Logos are not hyperlinked. Should be centered in the column. Column fields must match logo background color."*

The mark sits at the centre of its column. Every cell in the table takes centre text alignment, and the first column's cell empties its text once it holds a mark. Text alignment governs text, so a delegate on the first column moves the decoration instead, and it touches nothing else.

`src/gui/widgets/bot_status_table.py` — `CentredMarkDelegate`

```
def initStyleOption(self, option, index) -> None:
    super().initStyleOption(option, index)
    option.decorationPosition = QStyleOptionViewItem.Top
```

The page already centred its own mark, with a block image at an automatic side margin, so one declaration on each side puts the mark in the same place in both builds.

`src/gui/web/bot_status_table.js` — `AssetLogo`

```
var DOT_MARGIN = "0 auto";
```

HIS.

> *"Logos should also double as hyperlinks to the company or organization beyond each asset. Want Acervator to feel like it is connected to all of these corners of the investment world simultaneously like a creature with a thousand tendrils..."*

A mark opens the organisation that owns the asset. Clicking a mark opens the front door of the company or organisation behind the traded asset, in the operator's own browser. Two readers already in the tree answer an address and the table asks them in one place: a crypto base answers the site its own coin record carries, and a listed name answers the domain the asset maps hold for it. A regulator's company-search page is never answered, because a search result is not an organisation's own front door. A mark whose asset has no known address is not a link: it draws exactly as it drew before, it opens nothing, and its tooltip says so.

`src/gui/main_tabs/bot_status_table_surface.py` — `organisation_address`

```
for asked in (icon_asset_of(symbol), symbol):
    if not asked:
        continue
    site, _ = CRYPTO_RECORDS.organisation_url(asked)
    found = site or organisation_page(asked)
    if found:
        break
address = opening_address(found)
```

The site comes from the same coin record the mark came from. The library fill settles one coin for each ticker and keeps that coin's own front door beside the mark's address, so the bot list reads the kept site first and a hand-written row second. Only the coin source's detail address carries a site: the list address names every coin and the market records name each mark, so a site costs one read for each settled coin and a site already kept is never read twice.

`src/exchange/crypto_assets.py` — `AssetManager.organisation_url`

```
indexed = str(row.get(COIN_INDEX_SITE_KEY) or "").strip()
if indexed:
    return openable_url(indexed, allowed_schemes=COIN_SITE_SCHEMES)
asset = self.get_asset(name)
address, refused = openable_url(asset.website if asset else "")
if address:
    return address, refused
turned_away = str(row.get(COIN_INDEX_SITE_REFUSED_KEY) or "").strip()
return "", turned_away or refused
```

`src/exchange/crypto_assets.py` — the address that carries a site

```
COIN_DETAIL_URL = "https://api.coingecko.com/api/v3/coins/{id}"
```

Of the thirty-nine target assets the saved fleet holds, seven answer a web address. All seven come from the asset record's own `website` field. The coin index holds no site for any fleet ticker, and records the same reason for every one of them: no coin is settled for that ticker, so no site is looked up. None of the other thirty-two is guessed at.

A mark's tooltip carries one extra line naming the address or its absence, so the operator can see which marks are links and which are not without pressing one.

```
a mark whose asset resolves      Open <the site> in default browser: <the address>
a mark whose asset resolves none no web address is known for <ticker>, so this mark
                                 is not a link
a mark whose address is refused  <ticker> opens nothing: <the rule that refused it>
```

Current reading, 2026-10-29. The lines are shorter, so a mark's whole description stays within twenty words.

`src/gui/main_tabs/bot_status_table_surface.py` — `NO_ORGANISATION_TIP_FORMAT`

```
LOGO_TIP_FORMAT = "{asset}, the asset this bot accumulates."
NO_LOGO_TIP_FORMAT = "{asset}. No logo is kept for it yet."
NO_ORGANISATION_TIP_FORMAT = "No web address is known, so this mark opens nothing."
```

The third line is new. An asset that never had an address and an asset whose address was refused used to read the same, and now the mark says which it is.

What the address check refuses. An address is checked before any browser is asked to open it. Only a secure web address with a host opens. An insecure address, a local file path, a script address, a data address, a file-transfer address, a bare host and an address with no host are each refused, and nothing is opened, whatever the data carries. The window reads the address off the clicked cell and opens it; the page sends the row and the column out and Python opens it, so the page never opens anything itself. Both go through the one check.

A refusal is no longer silent. The mark's tooltip names the rule that refused the address, and the same line goes to the log, so a refused address can be told from an asset that never had one.

`src/gui/main_tabs/bot_status_table_surface.py` — `opening_address`

```
OPENING_SCHEMES: tuple[str, ...] = ("https",)
```

`src/gui/main_tabs/bot_status_table_surface.py` — `opening_refusal`

```
if not host:
    return EMPTY_TEXT, OPENING_NO_HOST_REASON
return address, EMPTY_TEXT
```

HIS.

> *"Column fields must match logo background color."*
>
> *"Some do not have a background color. Be sure to choose one that contrasts and makes each one pop. Use a theme-consistent color."*
>
> *"Do not want a bunch of random background colors in the Asset Column that smash together and cause an eyesore."*

The three sentences settle each other. One colour sits behind the whole column. Every mark on it reads against that one colour. A column carrying its own tint per row is the thing he refused.

The column sits on one palette ground. One colour sits behind the whole column, and every mark on it reads against that one colour. The column carries no tint per row. GUI010 in the GUI archetype refuses a ground the declared palette does not hold, a ground that does not carry what sits on it at the published contrast floor, and a set of declarations giving one column several grounds. Every finding is high, so the verdict reads `passed=False` and the command exits 1.

`dev_harness/harness/gui_archetype.py` — `_scan_column_ground_colours`

```
findings = _style_ground_faults(path, tree, known)
findings.extend(_cell_ground_faults(path, tree, known))
findings.extend(_named_ground_faults(path, tree, known))
findings.extend(_ground_set_faults(path, tree, known))
```

Its fixture pair is `harness_fixtures/gui_archetype/known_good_column_ground.py`, which names one ground, the value SURFACE_2 holds, and puts two palette colours on it; and `harness_fixtures/gui_archetype/known_bad_column_ground.py`, which plants all three departures and draws four high findings.

`src/gui/design_system.py` — where the palette is declared

The floors are the Web Content Accessibility Guidelines' own figures: 4.5 to 1 for ordinary text on the ground and 3 to 1 for large-scale text, where large-scale means 18 point, or 14 point bold. The ratio comes from relative luminance, which linearises each channel before weighting it, so a plain average of the raw channels returns a different number. Two grounds count as one ground while they sit within 2.3 of each other in CIE 1976 L*a*b*, which is the just-noticeable difference.

The rule leaves three things alone: a border colour, because a boundary is measured against the adjacent colour and one declaration names the fill on one of its two sides only; a block on the disabled state, which the contrast criterion exempts as an inactive component; and a colour the module never declares, such as the pixels inside a logo file.

The Fire button's engaged ground carries its own text. Of the palette's 146 tokens, 77 reach the floor against white. The bright engaged token missed it, so the button draws on the dimmed one, which is the nearest colour to it that reaches.

```
token                hex       carries #ffffff   floor
STATE_ENGAGED        #2d9d5f   3.4428 to 1       4.5     misses
STATE_ENGAGED_DIM    #2d5f48   7.3891 to 1       4.5     reaches
```

`src/gui/main_tabs/bot_status_table_surface.py` — `FIRE_STYLE_FOLD_SOLID`

```
FIRE_STYLE_HEAD + f"color: {ds.TEXT_MAX}; font-weight: bold; "
f"background-color: {ds.STATE_ENGAGED_DIM}; "
f"border: 1px solid {ds.STATE_ARMED};"
```

The button keeps its bright border and its glow, which is what parts it from the button a position ceiling has stopped.

Symbol, the second column. The Symbol cell carries the colour that says what a bot is doing, and its tooltip names the mode and the state. Where the pair has a chart, the chart's line follows underneath the cell, and the underline is what marks the cell as the chart's link. The coloured disc carrying the asset's first letter no longer draws in this cell. Neither build carries the disc any more: the payload declares no icon field on a cell, and the window build, the Paper build and the Simulator build of this page hold no disc element. The lettered disc is still the Extractor table's own, and the wizard's.

`src/gui/main_tabs/bot_status_table_surface.py` — `BotStatusTableModel._symbol_cell`

```
found = cell(text, state_color(state), mode_tooltip(mode, state))
```

The eight colours a state draws in:

state	the Symbol cell's colour
running	`#00ff88`
idle	`#888888`
paused	`#ffaa00`
error	`#ff3366`
cooldown	`#ff6600`
stopped	`#666666`
starting	`#00e6ff`
a state the list does not name	`#e0e0f0`

Current Position Value, the third column. It shows what the bot's holdings are worth at the exchange's own price. It is blank whenever no fresh exchange price exists, and a blank cell names the missing thing in its tooltip: no position held, no exchange price for the pair yet, no exchange price this tick, or a price older than twenty seconds. The cell never falls back to a last-known figure, to a stand-in, or to a value read out of the bot's own ledger.

`src/gui/main_tabs/table_cells_surface.py` — the one multiplication both priced cells read, so the Position Value cell and the Ammo cell can never disagree

```
def priced_position(holdings: float, price: float, quote_rate: float) -> float:
    """The position value at one price: ``holdings`` times ``price`` times
    ``quote_rate``."""
    return holdings * price * quote_rate
```

OVERTAKEN, quoted whole:

> `src/gui/main_tabs/table_cells_surface.py` *— the one multiplication both priced cells read, so the Position Value cell and the Ammo cell can never disagree*

True today: both cells still read that one multiplication, and they hand it different unit counts. The Position Value cell prices the units the bot attributes to itself. The Ammo cell prices the units the exchange reports, wherever the platform holds a reading of them, so the two cells differ exactly when the bot's own count has drifted from the exchange.

`src/gui/main_tabs/table_cells_surface.py` — the four blank paths and the one priced path

```
POSITION_PATH_PRICED = "priced"
POSITION_PATH_NO_HOLDINGS = "no_holdings"
POSITION_PATH_NO_PRICE = "no_price"
POSITION_PATH_OFF_EXCHANGE = "off_exchange"
POSITION_PATH_AGED = "aged"
```

Ammo. The cell draws green above the target, red below it, and neutral grey inside the dust band.

Design intention. The Ammo cell measures against the live target, not the frozen number typed into the wizard, so the reading follows the grown balance the engine re-zeroes to. The header strip's AMMO total takes the same target from the same function, so the row and the list cannot disagree.

Which unit count the distance is measured from. The exchange is the authority on what is held, so the cell prices the exchange's own count of units and falls back to the bot's attributed count only when the platform holds no exchange reading. The reading is already in memory. Three paths store it, net of the personal hold and of the sibling bots on the same asset: the startup read, the periodic audit, and the balance read a detonation sizes its sale from. The cell takes the stored figure off the bot's status, so nothing is fetched while the table paints, and a bot none of the three has run for yet draws the marked attributed count.

The hold in that sentence is gone. Personal Hold was removed. The three paths now net the reading against the sibling bots on the same asset and nothing else. `src/trading/scrumming/reconciliation.py`, in `_claimable_exchange_units`, is the one place the subtraction happens.

The tooltip always names the count it measured. On the exchange's count it gives both figures, so a drift between them is visible in the cell that is affected by it. With no exchange reading held it opens `NO EXCHANGE COUNT HELD` and says the direction can read the wrong way, because an attributed count below the exchange turns a surplus into an apparent deficit and paints buy pressure on a position that is already above target.

Read today: the tooltip names three figures together, the exchange count, the holdings figure and the lot-book total. With no exchange reading held it opens `NO EXCHANGE COUNT` and names the other two. The sentence about the direction reading the wrong way is gone from the tooltip, which now fits twenty words in every state. **What the Ammo mouse-over reads**, in the 2026-10-28 entry at the end of this page, gives each sentence.

The cell itself draws the amount and nothing else. The two counts are a fact about where the figure came from, and the tooltip is the only place that fact is written. The column is sized for a dollar amount, so a note added beside the amount would run past the column edge and cut itself short. The stale marker is the one exception the cell still draws, and it marks the figure as not current rather than naming its source.

`src/gui/main_tabs/table_cells_surface.py, in TableCellsModel._signal_cell` — the drawn figure and the tooltip, written apart

`src/gui/main_tabs/table_cells_surface.py`, in `ammo_units` — the one rule all three Ammo readers share

```
def ammo_units(holdings: float, venue_units: Any) -> tuple:
    try:
        reading = float(venue_units)
    except (TypeError, ValueError):
        return holdings, False
    if not math.isfinite(reading) or reading <= 0:
        return holdings, False
    return reading, True
```

`src/trading/scrumming/reconciliation.py`, in `_record_audited_venue_units` — where the reading comes from, and `src/trading/bot_container.py`, in `get_status` — the `audited_venue_units` field that carries it to the screen.

A fill moves the stored figure as well. Those three paths are not the only writers. Every fill path that is already holding a venue balance reading records that reading with the filled units applied, so the row no longer draws a

position taken before the fill beside one taken after it. No path fetches anything new. The sell path holds no reading of its own and still waits for the periodic audit. Two routes reach one assignment: one stores a venue reading and nets it against the personal hold and the sibling bots, and the other moves the stored figure by the units a fill settled and asks no venue for anything.

Read that first route as the sibling bots alone. Personal Hold was removed, so the hold named above and in the block below no longer exists. The venue reading is netted against the sibling bots and nothing else.

```
src/trading/scrumming/reconciliation.py, in _store_audited_venue_units
    the one assignment both routes land on
src/trading/scrumming/reconciliation.py, in _record_audited_venue_units
    a venue reading, netted against the hold and the siblings
src/trading/scrumming/reconciliation.py, in _record_venue_units_after_fill
    the stored figure moved by the units one fill settled
src/trading/scrumming/execution.py, in _execute_buy
    its own pre-fill reading plus the units bought
src/trading/scrumming/execution.py, in _execute_manual_rebalance
    its own pre-fill reading, on every intent
src/trading/scrumming/execution.py, in _execute_detonation
    the second route, by the units it sold
src/trading/scrumming/execution.py, in _execute_sell
    holds no reading; waits for the periodic audit
```

Read today: the sale path reads the exchange once after its fill, as the entry on the sale's own refresh below describes. Four more fill paths now move the stored figure, each by the units its own fill settled and with no new exchange read. The 2026-10-28 entry at the end of this page lists every fill path.

```
src/trading/scrumming_bot.py, in _reconcile_stack_tranches_visible
    the units a resting stack rung filled, off the held count and the stored figure
src/trading/scrumming/tick_phases.py, in _tick_expiry_close
    the units the expiry close sold
src/trading/scrumming_bot.py, in self_destruct
    its own pre-fill reading less the units sold
src/trading/scrumming_bot.py, in apply_extractor_tranche_return
    the base units that landed, while a reading is stored
```

The row's cell, the header strip's AMMO total and the Ammo column's sort order all take the count from `src/gui/main_tabs/bot_status_table_surface.py`, in `measured_units_of`, so no two of them can measure different units.

`src/trading/target_bands.py` — the target every Ammo reading measures against

```
def ammo_target(live_target_balance: Any, configured_target_balance: Any) -> float:
    live = float(live_target_balance or 0.0)
    if live:
        return live
    return float(configured_target_balance or 0.0)
```

`src/gui/widgets/bot_status_table.py` — the row reads it through `target_value`

```
target_val = target_value(status)
```

Target A15 and Target A14. They restate the Target in those two assets, and go blank when the pair is unlisted or when the target already is that asset.

The row's height. A logo draws at 32 pixels square. A row keeps two pixels above and two below its content, which is what this widget's style answers for its own item margin, so a row takes 36 pixels. Every bot table's Fire and Detail buttons take that same height, so a selected row paints no band around them — `src/gui/main_tabs/bot_status_table_surface.py, in BUTTON_HEIGHT_PX`. The page's own sheet leaves more room than the window's style, so the logo's cell drops its vertical padding and the row keeps the height the payload set. No other cell and no rule in the sheet changes, so the Extractor table beneath is untouched.

`src/gui/main_tabs/bot_status_table_surface.py` — the two figures and the sum

```
LOGO_SIZE_PX = 32
ROW_LOGO_MARGIN_PX = 2
ROW_HEIGHT_PX = LOGO_SIZE_PX + 2 * ROW_LOGO_MARGIN_PX
```

One sentence this page carried is overtaken. It is quoted whole, and the sentence under it is true today.

OVERTAKEN, quoted whole:

> *"No other cell and no rule in the sheet changes, so the Extractor table beneath is untouched."*

True today: no other cell and no rule in the sheet changes, and the Extractor table beneath takes this same row height. Read off the real widget, its row went from 30 pixels to 36 and both its buttons from 22 to 36, so every bot table in the window stands on one declared height — `src/gui/widgets/extractor_bot_table.py`, `src/gui/paper/paper_extractor_bot_table.py` and `src/gui/simulator/sim_extractor_bot_table.py` each read `ROW_HEIGHT_PX` from this same declaration. The three Positions Held tables read it too.

A cell widget's pinned size reads the row height. GUI011 in the GUI archetype refuses a widget placed into a table cell whose size is pinned to a bare number, or to a name the same file declares once as a bare number. Three readings are left alone, each able to carry the row's declared height: a value imported from another module, a value the file computes, and a name the file also hands to the call that declares how tall its rows are. The finding is high, so the verdict reads `passed=False` and the command exits 1.

`dev_harness/harness/gui_archetype.py, in _scan_pinned_cell_widgets` — the rule

```
_CELL_PLACEMENT_SETTER = "setCellWidget"

_PINNING_SETTERS: frozenset[str] = frozenset(
    {"setFixedHeight", "setFixedWidth", "setFixedSize"}
)

_ROW_HEIGHT_SETTERS: frozenset[str] = frozenset(
    {"setDefaultSectionSize", "setMinimumSectionSize", "setRowHeight"}
)
```

The rule's fixture pair holds two tables of the same three cells. In the good one the buttons read a height declared from the row's own and the logo square reads the row height itself. In the bad one one button takes a bare number, the other takes a height declared on its own, and the logo square takes two bare numbers. The good file exits 0 and the bad one draws three high findings and exits 1.

`harness_fixtures/gui_archetype/known_good_cell_widget_height.py` — the good half

The three Positions Held tables read the same two figures as the bot list, so each Manual Fire button fills the cell it sits in.

`src/gui/live_settings/positions_held_tab.py, in _create_positions_held_tab` — one of the three

The bot id. The bot id is still the thing a row is identified by. It stays in the payload, once as the list of every drawn row's bot and once on each row. It stays in the line the list writes when a row is built. The Detail button carries it, and the window that button opens names its first eight characters in its own title.

`src/gui/main_tabs/bot_live_settings_surface.py` — the title that names it

```
WINDOW_TITLE_FORMAT = "Bot Settings — {symbol} [{short_id}]"
```

Functional. Every column header wraps its own label inside its own column. `Current Position Value` reads over two or three lines rather than being cut. The label draws at `HEADER_LABEL_FONT_PX` and steps down to `HEADER_LABEL_MIN_FONT_PX` when its longest word does not fit the column at the first size. Every column's header is the same height, which is the tallest wrapped label plus one dot row. Nine of the ten columns carry one privacy dot centred beneath the label, and the tenth, the one holding the Detail button, leaves that row empty. `ColumnHeaderCell` draws one label over one dot and `WrappedColumnHeader` sizes and places one cell a column.

`src/gui/main_tabs/bot_status_table_surface.py` — `header_view`

```
return {
    "text": label,
    "tooltip": header_tooltip(column, field_id, masked),
    "field_id": field_id,
    "masked": bool(masked),
    "dot_text": header_glyph(masked),
    "dot_tooltip": header_dot_tooltip(field_id, masked),
}
```

Current reading, 2026-10-29. A heading's description no longer names the privacy state. The dot under the label carries it, so a heading keeps its sentence and its sort line within twenty words on both builds.

`src/gui/main_tabs/bot_status_table_surface.py` — `header_tooltip`

```
def header_tooltip(column: int, direction: str) -> str:
    tip = COLUMN_TOOLTIPS.get(column, EMPTY_TEXT)
    return (tip + sort_tooltip(column, direction)).strip()
```

Functional. The dot is the `PrivacyDot` the manual calls the universal control. A press flips that column's field in the privacy register, repaints every dot that shares the field, and redraws the rows from the payload the table already holds. Three of the nine dots share a field with another column: `Current Position Value` and `Ammo` share `bot_table.ammo`, and `Target`, `Target A15` and `Target A14` share `bot_table.target`. A press on one of them masks every column that names the same field. A masked first column draws the mask alone, with no image and no ticker, and revealing it brings the mark back.

`src/gui/widgets/bot_status_table.py` — `BotStatusTable.refresh_privacy_dots`

```
for cell in self._header.cells():
    if cell.dot is not None:
        cell.dot.refresh()
```

Functional. A press on a column's label orders the rows by that column. The first press sorts smallest first, and a second press on the same label reverses it. Nine of the ten columns sort. The tenth holds one identical Detail button on every row, so it carries no value to order by, and a press on it changes nothing. One ordering serves both builds: the window and the page both call the same function on the same fleet, so neither can put the same bots in an order the other would not. On the page the press goes back to Python, because the fleet that answers it lives there, and the venue republishes the ordered rows.

`src/gui/main_tabs/bot_status_table_surface.py` — `SORT_KIND_BY_COL`

```
SORT_KIND_BY_COL = {
    BOT_ID_COLUMN: SORT_KIND_TEXT,
    SYMBOL_COLUMN: SORT_KIND_TEXT,
    POSITION_VALUE_COLUMN: SORT_KIND_NUMBER,
    TRADES_COLUMN: SORT_KIND_NUMBER,
    TARGET_COLUMN: SORT_KIND_NUMBER,
    TARGET_A15_COLUMN: SORT_KIND_NUMBER,
    TARGET_A14_COLUMN: SORT_KIND_NUMBER,
    AMMO_COLUMN: SORT_KIND_NUMBER,
    FIRE_COLUMN: SORT_KIND_NUMBER,
    DETAIL_COLUMN: SORT_KIND_NONE,
}
```

`src/gui/main_tabs/bot_status_table_surface.py` — `BotStatusTableModel.on_header_sorted`

```
if column == self.sort_column:
    self.sort_descending = not self.sort_descending
else:
    self.sort_column = column
    self.sort_descending = False
```

Functional. Asset and Symbol sort as words, Asset on the asset's ticker. The other seven sort as figures, each reading the number its cell was computed from rather than the text the cell draws, so nine never sorts above ten. Ammo sorts on the distance from target without its sign, which is the amount that would fire and the figure the cell prints; the colour still says which side of the target the bot sits on. Fire sorts by what the engine would do next, armed first and disabled last. A row whose cell draws nothing sits beneath every row that draws a

figure, whichever way the sort runs, and the bot's own identifier breaks a tie.

`src/gui/main_tabs/bot_status_table_surface.py` — `order_statuses`

```
drawn.sort(key=lambda row: (row[0], row[1]), reverse=descending)
return [row[2] for row in drawn] + blank
```

One sentence above is true and incomplete, and this completes it. Fire still sorts by what the engine would do next, and that is not the same question as what a press would do. The sort reads the bot's own armed action; the button reads how far the position sits from target. So a row can sort high as armed and still carry the refused word on its button. The word on the button is the answer about the press, and the sort position is not.

`src/gui/main_tabs/bot_status_table_surface.py, in fire_sort_value` — the figure the column orders on

Functional. The order is recomputed every time the rows are rewritten, not once at the press. A fleet arriving a second later is ordered again before it is drawn, so a column whose figures keep moving keeps the order the last press asked for and the rows travel as the figures change.

`src/gui/widgets/bot_status_table.py` — `BotStatusTable.update_bots`

```
bot_statuses = order_statuses(
    self._last_statuses,
    self._sort_column,
    self._sort_descending,
    _qt_lookups(),
)
```

Functional. The sort arrow draws on the privacy dot's own row, at the right edge of its own column. Its rectangle takes the dot's top and the dot's height, so the two sit on one line, and its left edge never crosses the dot's right edge. The box is clamped to the column, so no arrow is drawn outside the column it belongs to. The arrow sits outside the header's own layout, so neither the wrapped label nor the dot beneath it moves when a column sorts. The page places its arrow the same way, against the heading's bottom edge above the cell's padding and held to the column's width.

`src/gui/widgets/bot_status_table.py` — `ColumnHeaderCell._place_mark`

```
top, height = self._dot_row()
left = max(self._dot_right(), self.width() - HEADER_SORT_MARK_BOX_PX)
self.mark.setGeometry(left, top, max(0, self.width() - left), height)
```

`src/gui/web/bot_status_table.js` — `SortMark`

```
style.bottom = length(model[HEADER_CELL_PAD_PX]);
style.right = MARK_EDGE;
style.width = length(model[HEADER_SORT_MARK_BOX_PX]);
style.maxWidth = MARK_MAX_WIDTH;
style.height = length(model[HEADER_DOT_ROW_PX]);
```

Functional. Five things can be pressed on the table, and each sends one request and redraws from the answer. The dot under a column's label toggles that column's privacy mask. A mark in the first column opens its asset's own organisation. A Symbol cell opens the chart address. Fire hands the bot to Manual Fire. Detail selects the row and opens the bot. A press on a column's label sorts it. The dot keeps its own press to itself, so masking a column never reorders the table, and the Fire and Detail buttons stop the press reaching the row so they keep their window behaviour.

`src/gui/main_tabs/bot_status_table_surface.py` — `BotStatusTableModel.on_detail`

```
def on_detail(self, bot_id: str) -> None:
    """Select the row, then hand the bot to whatever opens the detail."""
    self.select_row_for_bot(bot_id)
    self.detail_clicks.append(bot_id)
    self.calls.append([DETAIL_CLICKED, bot_id])
    if self.on_bot_clicked:
        self.on_bot_clicked(bot_id)
```

`src/gui/react_trading_tab.py` — the presses the venue answers

```
VENUE_PRESSES = (
    (scrum_surface.PRIVACY_TOGGLE_PARAM, "toggle_privacy"),
    (scrum_surface.SORT_COLUMN_PARAM, "sort_by"),
    (scrum_surface.SELECT_BOT_PARAM, "select_bot"),
    (scrum_surface.VIEW_BAND_PARAM, "scroll_view"),
)
```

Functional. A press on a bot's row draws that bot in the Indicator Voting Panel. A press on the row the panel already draws takes the panel off it instead, and the panel shows its empty state; a third press brings the bot back. A press naming another bot never blanks the panel, it moves it. The press names the bot on the row and never the row's position, so a column can be sorted first and the panel still draws the bot whose row was pressed. One function decides it and both builds call it, so a press cannot mean two different things on the two screens.

`src/gui/main_tabs/bot_status_table_surface.py` — `selection_after_press`

```
pressed = str(pressed_bot_id or "")
if not pressed or pressed == str(shown_bot_id or ""):
    return NO_SELECTION_BOT_ID
return pressed
```

`BotStatusTable.mousePressEvent` calls it in the window and `BotStatusTableModel.on_row_pressed` calls it on the page, and the page's row sends its press out on the same console line the privacy toggle and the column sort use.

`src/gui/web/bot_status_table.js` — `sendRowPress` and `sendSort`

```
  function sendRowPress(model, botId) {
    return dispatch(
      model,
      actionNamed(model, ROW_PRESSED),
      request(model, SELECT_BOT_PARAM, botId)
    );
  }
```

The window builds one link and gives the panel's own topic a reader, in `MainWindow._setup_bot_list_link`. The topic was already declared and the emit already ran; nothing listened to it.

`src/gui/widgets/bot_selection.py` — `BotListPanelLink.row_selected`

```
  def row_selected(self, bot_id: Optional[str]) -> str:
      """Draw the pressed bot on the panel and answer the bot it holds."""
      if self._settling:
          return self.bot_id
      self._settling = True
      try:
          return str(self._panel.select_bot(str(bot_id or "")) or "")
      finally:
          self._settling = False
```

Functional. The list scrolls only when the operator scrolls it. The table turns off the scroll-to-current-item behaviour the toolkit applies by default, so the only thing that moves the scroll bar is a hand. A highlighted row the viewport no longer draws loses the highlight, and one rule decides that for both builds. A column sort reorders every row while the scroll position stays, so the highlighted bot can land far below the rows on screen; that is the highlight leaving the view and it is released the same as any other cause. The drop runs with the table's signals blocked, so nothing downstream hears it.

`src/gui/widgets/bot_status_table.py` — `BotStatusTable.__init__`

```
  self.setAutoScroll(False)
  self.verticalScrollBar().valueChanged.connect(
      lambda _value: self.release_selection_off_view()
  )
```

`src/gui/main_tabs/bot_status_table_surface.py` — `selection_after_view_moved`

```
  if row < 0 or first < 0 or last < first:
      return NO_SELECTION_ROW
  if first <= row <= last:
      return row
  return NO_SELECTION_ROW
```

The React page reads its own drawn rows off the scroll box it sits in and sends the first and last of them, and the same function answers there.

`src/gui/web/bot_status_table.js` — `visibleRowBand`

```
if (span.top >= edge.top - 1 && span.bottom <= edge.bottom + 1) {
  if (first < 0) {
    first = at;
  }
  last = at;
}
```

Functional. The list's highlight and the panel's bot are two values. A release changes the list alone. The panel changes only when a bot is chosen, from a row or from the panel's own dropdown, and the Live page does not redraw the panel on a scroll. The panel's dropdown still puts the highlight on that bot's row, and now only while that row is drawn, so the two can name different bots: the panel holds the bot it is drawing and the list holds a highlight it can release, which is what keeps the scroll bar still.

`src/gui/widgets/bot_selection.py` — `BotListPanelLink.panel_selected`

```
for host in self._hosts() or ():
    mark = getattr(host, "highlight_bot", None)
    if callable(mark):
        mark(wanted)
```

Functional. One bad field stops one row instead of the whole paint. Each row is written inside its own guard. A row that refuses is logged, emptied and left empty, and every other bot still draws. A row whose bot is not a scrumming bot is emptied the same way, so it can no longer keep the previous bot's figures.

`src/gui/widgets/bot_status_table.py` — `BotStatusTable._clear_row`

```
for col in range(self.columnCount()):
    self.setItem(row, col, None)
    if self.cellWidget(row, col) is not None:
        self.removeCellWidget(row, col)
```

Functional. In the React build the Scrumming Bots table is drawn by `bot_status_table.js`. The exchange screen keeps a named empty space for it and fills that space when it draws itself. The rows come from the backend, not from the exchange screen: the renderer asks the bridge for the table's own state and names the exchange it wants rows for. Each exchange gets its own table state, so two exchange screens on one page never share rows or a highlight.

`src/gui/web/exchange_tab.js` — `mountScrumTable`

```
function mountScrumTable(target, model) {
  var loader = global[LOAD_BOT_TABLE];
  var wait =
    typeof loader === "function" ? loader(askFor(model)) : Promise.resolve(null);
  return Promise.resolve(wait).then(function () {
    return renderScrumTable(target, exchangeOf(model)) === null
      ? null
      : BOT_TABLE_MODULE;
  });
}
```

Design intention. The exchange screen redraws itself once a second to keep the data-pool line fresh. Each redraw re-fills the table space, so the table is drawn from the state the renderer holds for that exchange rather than from the answer captured at first paint. A row the operator selected therefore survives the next redraw.

`src/gui/web/bot_status_table.js` — `modelFor`

```
function modelFor(exchangeId) {
  return owns(models, String(exchangeId)) ? models[String(exchangeId)] : null;
}
```

Functional. The Extractor table underneath is drawn the same way, by `extractor_bot_table.js` into the second named space the exchange screen keeps. Its rows come from the same fleet list, kept to the records whose mode is extractor. The Detail button is the one control the Extractor table offers; Fire stays disabled on this screen, because Manual Fire is per position and lives in the Positions Held tab of the bot's own window.

`src/gui/main_tabs/extractor_bot_table_surface.py` — `extractor_statuses`

```
def extractor_statuses(statuses: Any) -> list:
    """The records of ``statuses`` whose mode is ``MODE_TEXT``.

    ``update_bots`` builds a row for every record it is handed, so only the
    Extractor bots reach the Extractor table.
    """
    return [
        found
        for found in statuses or []
        if isinstance(found, dict) and found.get("mode") == MODE_TEXT
    ]
```

Functional. The Scrumming table holds the whole accumulation fleet and grows into whatever room is left. The Extractor table keeps the height of its own rows, so its rows are never cut. The React page reads the two shares onto the flex of each table space, and the Qt page passes them to the layout.

`src/gui/main_tabs/exchange_tab_surface.py` — the two stretches

```
SCRUM_TABLE_STRETCH = 1
EXTRACTOR_TABLE_STRETCH = 0
```

Functional. The exchange screen mounts its three children in one step, each into its own space and each asked for its own view model.

`src/gui/web/exchange_tab.js` — `mountChildren`

```
function mountChildren(target, model) {
  return Promise.all([
    mountScrumTable(target, model),
    mountExtractorTable(target, model),
    mountNewsTicker(target, model)
  ]).then(function (drawn) {
    return drawn.filter(function (name) {
      return name !== null;
    });
  });
}
```

Functional. A sale fires its own refresh of the two priced figures on the row. The sale reduces the count the bot attributes to itself, which is what Current Position Value is priced from. It does not move the audited exchange count, which is what the Ammo cell measures the Target Delta from. One exchange read at the fill puts both figures on the same sale. The read costs one call per fill and the lot book is left to the periodic reconcile, because the callers of the sale take the filled units off the lot book only after it returns.

`src/trading/scrumming/execution.py` — `ExecutionEngineMixin._execute_sell`

```
await self.refresh_priced_figures_after_fill(reason=label.lower())
```

Functional. The refresh reuses the one exchange-reading helper the engine already holds. That helper reads the balance once, subtracts the sibling bots, and stores the figure as the audited count. The engine's own scrum, the distribution sale and a stack close all run through one sale function, so all three get the refresh from one site.

`src/trading/scrumming/reconciliation.py` — `ReconciliationEngineMixin.refresh_priced_figures_after_fill`

```
try:
    _units, _why = await self._venue_claimable_units()
except Exception as exc:
    _units, _why = None, (
        f"the venue read raised {type(exc).__name__}: {exc}"
    )
```

Functional. An exchange that cannot be read leaves the last audited count exactly as it was and records why. The reason reaches the bot's status record, and the Ammo cell then draws the last reading with the word `(held)` beside it and names the reason in its tooltip. The figure is never blanked and never drawn as though it were current. The order and the trade are unaffected: the sale has already settled at the exchange, and the refresh cannot refuse it.

Read today: the tooltip ends `Exchange read refused.` and no longer names the reason. The reason stays in the `VENUE READ REFUSED` line of the bot log.

`src/trading/bot_container.py` — the status record's refusal field

```
    "venue_units_refusal": str(
        getattr(self, "_venue_units_refusal", "") or ""
    ),
```

Functional. Both bot tables read one definition of the mark, so one surface cannot carry it while the other does not. The Qt table and the React table each compose their own Ammo cell, and both ask the shared cell surface whether a reading is held.

`src/gui/main_tabs/table_cells_surface.py` — `venue_held_figure`

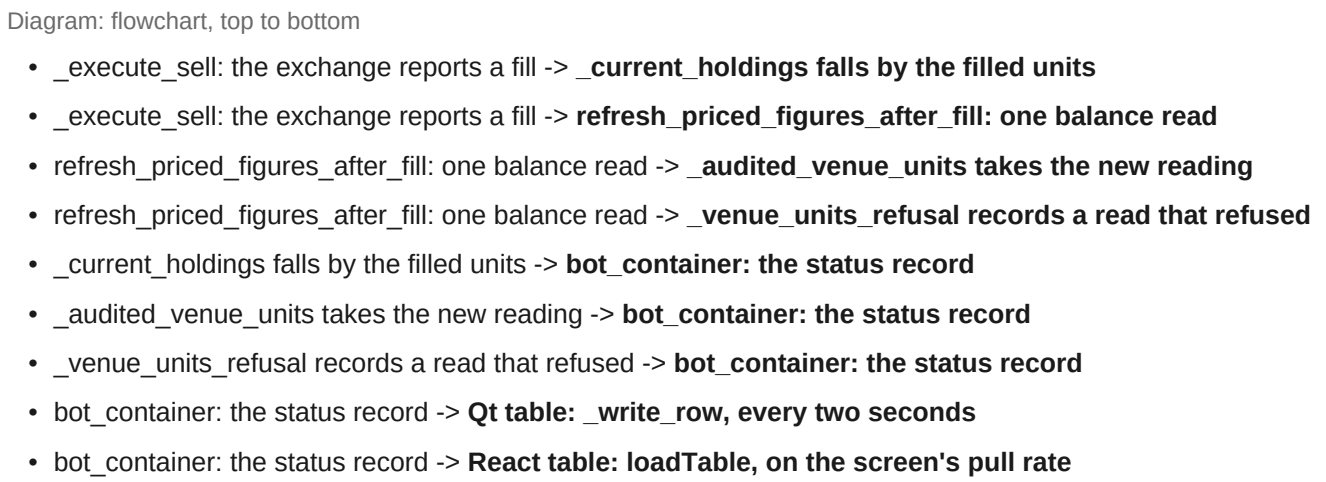
```
def venue_held_figure(text: str, refusal: Any) -> str:
    if not venue_refusal_text(refusal):
        return text
    return MARKED_FIGURE_FORMAT.format(text=text, marker=VENUE_HELD_MARKER)
```

Functional. The React table asks the bridge again once an answer has landed. It still collapses two requests that carry the same question while the first is in flight. Before this, the first answer to one question was drawn for the life of the page, so a sale's figures could not reach that table at all. The table now follows the exchange screen's own pull-rate clock, which redraws the screen and remounts the table.

`src/gui/web/bot_status_table.js` — `clearAsked`

```
function clearAsked(key) {
  if (asked !== null && asked.key === key) {
    asked = null;
  }
}
```

The path a sale takes to both tables:



The command bar

Functional. Start, Pause, Stop, Restart and Delete all act on one bot. Two tables share the bar, so the bar takes its bot from whichever table you clicked last. If that table holds no selection it tries the other one. If neither holds a selection it says "Select a bot first." and does nothing.

`src/gui/widgets/exchange_tab.py` — `ExchangeTab._cmd`

```
def _cmd(self, command: str) -> None:
    if self._last_clicked_table == "extractor":
        bot_id = self._extractor_table.get_selected_bot_id()
        if not bot_id:
            bot_id = self._bot_table.get_selected_bot_id()
    else:
        bot_id = self._bot_table.get_selected_bot_id()
        if not bot_id:
            bot_id = self._extractor_table.get_selected_bot_id()
```

Design intention. One command bar for two tables means the bar has to guess which bot you meant. The last table you clicked is the tie-breaker, and the fallback keeps a stray click from swallowing the command. With nothing selected anywhere it refuses out loud rather than acting on a guess.

`src/gui/widgets/exchange_tab.py` — the refusal

```
if not bot_id:
    if self._status_log:
        self._status_log.log("Select a bot first.", "warning")
    return
```

SHIFT turns each command into its all-bots form

> *"Bot List Upgrades Pt. 2 (Live and Paper Only) - Holding SHIFT makes Start become Start All and launches bot start sequencer used during boot up - Holding SHIFT makes Stop, Pause, and Restart all become Stop All, Pause All, and Restart All respectively"* ***Functional.*** *Four of the five act on one bot, or on the whole fleet while you hold SHIFT, in the window build and in the page build. Hold the key and Start,*

Functional. Four of the five act on one bot, or on the whole fleet while you hold SHIFT. Hold the key and Start, Pause, Stop and Restart read Start All, Pause All, Stop All and Restart All. Let go and they read their own names again. Delete has no all-bots form and never changes its name. The key is read at the moment you press the button, so a key held for one press cannot reach the next one. An all-bots press needs no bot selected.

`src/gui/main_tabs/exchange_tab_surface.py` — `FLEET_COMMANDS`

```
FLEET_COMMANDS = {
    "start": ("Start All", "start_all"),
    "pause": ("Pause All", "pause_all"),
    "stop": ("Stop All", "stop_all"),
    "restart": ("Restart All", "restart_all"),
}
```

Design intention. Start All runs the same sequence the application runs at launch. It opens the Start All window, starts one bot at a time, waits for that bot to read running, and moves on after two seconds or after eight seconds of waiting. Each bot goes through the same handler a single Start goes through, so each one connects to its venue before it starts. Restart All runs that sequence over every bot held, not only the idle ones. Pause All and Stop All need no venue, so they go straight to the bot manager.

`src/gui/main_window.py` — `MainWindow._global_bot_cmd`

```
if command == "start_all":
    if not self.run_fleet_sequence("start"):
        self._status_log.log(
            "start_all: no bots eligible (none idle/stopped).", "info"
        )
        return
elif command == "restart_all":
    if not self.run_fleet_sequence("restart"):
        self._status_log.log("restart_all: no bots held.", "info")
        return
elif command == "pause_all":
    self._schedule_async(self._bot_manager.pause_all())
elif command == "stop_all":
    self._schedule_async(self._bot_manager.stop_all())
```

Functional, on the Paper tab. The Paper command bar reads the key the same way, over the paper fleet. A paper bot moves a saved state and connects to nothing, so the four all-bots verbs run straight down the held list, with no staggering and no progress window.

`src/paper/paper_bot_manager.py` — `PaperBotManager.start_all`

```
def start_all(self) -> int:
    """Run ``start`` over every held bot and answer how many landed on
    ``running``. A paper bot moves a saved state and connects to nothing,
    so the fleet needs no staggering."""
    return self._over_fleet(self.start, BotState.RUNNING.value)
```

Functional, on the Simulator. The Simulator command bar does not read the key. He named Live and Paper only, and the Simulator's own venue page and its own view model are separate files that carry none of this.

HIS.

> *"Live - Bug - Bot List - Start / Stop / Pause (All) toggles 'normally on' instead of remaining 'normally off' after releasing the SHIFT key. Can hold SHIFT key to toggle back to the normal state but this is incorrect / inverted behavior."*

The five readings, per build:

```
                              window build   page build
at rest, no key held          single-bot     single-bot
SHIFT pressed and held        all-bots       all-bots
SHIFT released                single-bot     single-bot
window deactivated while held single-bot     single-bot
SHIFT pressed twice in a row  all-bots       all-bots
```

Where the label state comes from. Both window forks call one rule, so the Live tab and the Paper tab cannot draw different labels from the same key. The rule takes the state off the key event the code is already holding: a SHIFT press means the key is down, a SHIFT release means it is up, and no other reading is possible. It holds whichever way the computer reports the key.

`src/gui/main_tabs/exchange_tab_surface.py` — `shift_held_after_key`

```
def shift_held_after_key(
    is_press: bool, key_is_shift: bool, event_has_shift: bool
) -> bool:
    if key_is_shift:
        return is_press
    return event_has_shift
```

The press target follows the label. The press reads SHIFT from the mouse press, which carries the true state. A window deactivated while the key is held reads the buttons' own names, and the key pressed twice in a row reads the all-bots names.

Functional. A press on the Live page reaches the application only when it carries one of the fields the page forwards, and the command field is one of them. The page answers a read from what it already holds, and sends a press on, because the register, the fleet, the voting panel and the bot manager all live there.

`src/gui/react_trading_tab.py` — the page's own bridge

```
if (
  params &&
  (owns(params, TOGGLE) ||
    owns(params, SORT) ||
    owns(params, PICK) ||
    owns(params, COMMAND))
) {
```

The venue a press names. One page can draw more than one venue, so every press carries the exchange it came from. The bot table names that field one way and the venue page names it another, and the lookup reads every name a press can use.

`src/gui/react_trading_tab.py` — `venue_of_press`

```
def venue_of_press(params: Any) -> str:
    held = params if isinstance(params, dict) else {}
    for name in VENUE_KEYS:
        found = held.get(name)
        if found:
            return str(found)
    return ""
```

The bot the bar acts on. The bar takes its bot from the table the operator pressed last. The page keeps two records of the Scrumming table, the one it draws and the one the bar reads, and a row press moves both. The Paper tab and the Simulator keep the same two records, so the same call runs on all three.

`src/gui/main_tabs/exchange_tab_surface.py` — `ExchangeTabModel.hold_scrum_bot`

```
row = (
    table.bot_ids.index(wanted)
    if wanted in table.bot_ids
    else NO_SELECTION_ROW
)
was = table.block_signals(True)
try:
    table.clear_selection()
    table.set_current_cell(row, 0)
    if row != NO_SELECTION_ROW:
        table.select_row(row)
finally:
    table.block_signals(was)
```

The rest of the screen

The right half is the Indicator Voting Panel, described at the end of this section. The two panes at the foot are the Activity Log and the API Interaction Log. The status bar carries the API load pill written by `_refresh_api_load_pill` and the `AI:` state label.

The pill names the venue, the calls that venue took in the last minute, the ceiling for it and the two as a percentage. It reads the worst-loaded connected venue, so one busy exchange cannot hide behind a quiet one.

`src/gui/main_window.py` — the pill text

```
text = (
    f"API {worst.exchange}: "
    f"{worst.calls_per_minute:.0f}/"
    f"{worst.ceiling_cpm:.0f} CPM ({pct} %)"
)
```

Its colour is the warning. Green under half load, amber above that, red past the venue's safety percentage. With no venue connected the pill draws an em dash and no number.

`src/gui/main_window.py` — the three colours

```
if worst.load_score > mon.safety_pct:
    colour = ds.ERROR
elif worst.load_score > 0.5:
    colour = ds.FOLD_RATIO_AMBER
else:
    colour = ds.SUCCESS
```

1 - Add Exchange - User provides valid API key and secret for target exchange - Platforms validates with an API handshake - Exchange Initializes

After an exchange is connected to the platform, bots can be added to target entire or portions of existing positions as well as using standing liquidity to enter positions for the first time. The two primary bots used by Acervator are known as Scrumming and Extractor with the conceptual Modulus Bot to be added later. The first two are the active traders that continuously interact with the market whereas the Modulus Bot will be a macro-strategic interface designed to trigger portfolio-scale shifts in response to custom market signals and it

can best be visualized as a modular synthesizer with investment and market-specific functions.

Scrumming Bot

If Acervator has a crown jewel, this is it. The scrumming bot is what houses and executes the harvest-fold method and the harvest-fold method is what leads to the creation of the platform itself. All else stems from this origin point and the simple revelation that allowed the method to be discovered (or probably rediscovered) by nonmathematical eyes. Instead of paying attention to what my entire portfolio was doing, I opted to start focusing on a single position until I had found a strategy that was more reliable than Grid Bots or standard speculation. I was convinced such a method existed and thought it absurd that price charts could not be played better when there was obviously so much room for improvement. You do not have to predict the market. You flow and mold your portfolio to it as time goes. Scrumming allows profits to be shaved off, held and re-investment in a cyclical, reliable manner that sizes its trades in direct proportion to actual market movement as it relates to position value drift. Trading in this manner allows a fixed balance for a position to be maintained and asserts the truth that “Position X is Y value. Any deviation from Y is a Target Delta and is subject to a Scrum (Sell) or Fold (Buy).” The primary weaknesses to this strategy are not have an equal amount of liquidity to the position value (a $500 Scrumming Bot should be supported by $500 of liquidity when initialized) and severe market downturns will little or no short term upside which can lock most liquidity for the position into the Target Asset. Scrumming is a survival strategy and is directionally agnostic but also depends on the market to cycle. I also refer to this simply as Accumulation Trading.

Scrumming Bot Terms

Target Balance - The initial value of the position to be taken or controlled by the Scrumming Bot. The bot will monitor the market for bullish or bearish conditions, check for Target Balance deviations (Target Delta), and re-zero back to the set point. It will repeat this until stopped by the user or some other market condition.

The figure is one field on the bot's own configuration. It is carried in from the wizard and held for the life of the bot, and every other term on this page is measured against it.

`src/trading/container/config.py` — `BotConfig.target_balance`

```
  target_balance: float = 200.0  # Balance the bot trades relative to
```

Target Delta - The amount by which a position value has drifted from the Target Balance. The Target Delta is denoted by the Ammo column under the Scrumming Bot list of the Trading Tab. This is the amount of value that will be fired during the appropriate market conditions.

One subtraction, and the platform does it in two places with opposite signs. The Ammo cell takes the position value less the target, so a surplus reads positive. The fold path takes the target less the position, so a deficit reads positive. Both measure the same drift.

`src/gui/table_cells.py` — `ammo_cell`, the figure the Ammo column shows

```
  delta = position_val - target_val
  territory = target_territory(position_val, target_val)
```

The position value in that subtraction is the exchange's own unit count at the exchange's own price, and the bot's attributed count only where no exchange reading is held. The Ammo column section above names the rule and where the reading is stored.

Two bands measure that drift, and they answer two different questions. The narrower one is a tenth of one per cent of the target. The bot's own tick parks inside it and trades nothing. The wider one is ten times that, one per cent of the target, and a Manual Fire press refuses inside it. Each figure is named for who stops and for what they stop doing, so a later reader cannot take one for the other.

`src/trading/target_bands.py` — the two figures, each a fraction of the target

```
TICK_PARK_PCT = 0.001
MANUAL_FIRE_REFUSE_PCT = 0.01
```

`src/trading/target_bands.py, in at_target_dust_band` turns the first into dollars and `src/trading/target_bands.py, in manual_fire_dust_band` turns the second. Both hold a floor of one cent, so a tiny target still gets a band.

What the gap between the two bands means. On a one-thousand-dollar target the tick's band is one dollar and Manual Fire's is ten. A position between those two is one the bot's own tick will trade and the button will not. The band stays where it is, and the button says so.

The button's own word changes inside that gap. It reads `No Trade` in place of `Fire`. It carries no glow, it draws a dotted outline in neutral grey, and its tooltip names three things: how far the position sits from target, Manual Fire's own band, and where the tick's band starts. The press stays available and still writes its no-op line to the bot log, so nothing a press can do today is taken away.

`src/gui/main_tabs/bot_status_table_surface.py, in FIRE_LABELS` — the word each Fire path draws, one entry a path

`src/gui/main_tabs/bot_status_table_surface.py, in manual_fire_verdict` — the three answers that pick the path, read off the row's own Ammo cell so the cell and the button cannot disagree about one press

```
MANUAL_FIRE_ACTS = "acts"
MANUAL_FIRE_REFUSES = "refuses"
MANUAL_FIRE_PARKED = "parked"
```

The Qt build picks the same word from the same table — `src/gui/widgets/bot_status_table.py, in BotStatusTable._write_row`. The Paper Trader's table and the Simulator's table are subclasses of those two and hold no word of their own, so all six surfaces read alike.

Manual Fire reads the exchange before it sizes the trade. The press asks the exchange for the traded asset's balance first. It costs one call, so the press is slightly slower. Where the exchange's count agrees with the bot's own count inside Manual Fire's band, the press sizes the trade exactly as it did before. Where the two counts differ by more than that band, the press trades nothing and says so. The same answer covers an exchange that leaves the asset out of its reply, and an exchange that cannot be read at all. This is the path with no gates behind it, so a position the platform cannot verify is not traded.

Driven on a target of $1,000.00, a price of $1.00 and a bot holding 1050.0 units. Manual Fire's band is $10.00 and the position sits $50.00 above target:

```
the exchange reports     the press sells
1050.0 units             50.000000 units
1020.0 units              0.000000 units, refused
the asset left out        0.000000 units, refused
no answer at all          0.000000 units, refused
```

The refusal carries the same words the Fire button itself shows, `MANUAL FIRE WILL NOT ACT`, and names both counts and both dollar figures. It goes to the bot log the Console and the bot window both read. Every press gets an answer. The autonomous intents keep the hold that stops a repeating tick writing the same refusal on every candle.

`src/trading/scrumming/execution.py`, in `_execute_manual_rebalance` — the one balance read every intent takes, the button included

```
_xbal = await self._get_balance(self.config.target_asset)
_xunits = float(
    getattr(_xbal, "total", 0) or getattr(_xbal, "free", 0) or 0.0
)
```

The growth-cap check that sits beside that read still runs for the two autonomous intents alone. The Max Entry Price entry below says Manual Fire goes around the entry-price gates, and the operator override keeps the cap bypassed the same way.

The row says the position is disputed before the press. The Fire button reads the two counts the row already carries. Where the exchange's count and the bot's count stand further apart than Manual Fire's band, the button takes the same refused path a press inside the band takes. It reads `No Trade`, it carries no glow, and its tooltip names both counts, both dollar figures and the gap. Where the gap sits inside the band, and where the row holds no exchange reading at all, the button reads exactly as it did before. An absent reading is not a dispute.

Driven on six rows, read off the live button on both build variants, at a target of $1,000.00 and a price of $1.00. Manual Fire's band is $10.00 and the tick's is $1.00:

```
the bot counts   the exchange reports   the button reads   its tooltip names
1050.0 units     1050.0 units           Fire               the armed scrum
1050.0 units     1020.0 units           No Trade           the $30.00 gap
1050.0 units     1049.0 units           Fire               the armed scrum
1050.0 units     no reading at all      Fire               the armed scrum
1005.0 units     1005.0 units           No Trade           the $10.00 band
1050.0 units     1005.0 units           No Trade           the $45.00 gap
```

Two refusals reach that one path, and the tooltip names the gap first. A press is refused where the gap between the two counts is past the band, and refused where the distance from target sits inside the band. A row can be in both states at once, and the last row above is one. The tooltip names the gap there, because `_execute_manual_rebalance` reads the exchange before it measures the distance, so the gap is the refusal a press would actually meet. The band entry above keeps its own wording for a row whose two counts agree.

`src/gui/main_tabs/bot_status_table_surface.py`, in `manual_fire_dispute` — the one answer both build variants read

```
reading = ammo if isinstance(ammo, dict) else {}
if not reading.get("two_reads", False):
    return None
```

The page's row and the window's row both call that function, and both read `manual_fire_verdict` beside it, so neither build variant can draw a button the other would not. The Ammo cell's own tooltip carries the same two counts, which the Ammo column section above describes.

Scrum - To sell an amount from an investment position that allows it to return to its initial price level. This never closes the position. This is the first half of the infinitely divisible circle that can persist for such positions in a healthy market.

One shared helper decides which half of the cycle a position sits in. It answers scrum above the target and fold below it, and it answers at target inside a dust band, so a position that has barely moved is left alone.

`src/trading/target_bands.py` — `target_territory`

```
delta = float(position_value) - float(target_balance)
band = at_target_dust_band(target_balance)
if delta > band:
    return "scrum"
if delta < -band:
    return "fold"
return "at_target"
```

What a sale takes off the position. A scrum asks the exchange for an amount and the exchange decides how much of it trades. The units that leave the position are the units the exchange reports sold, never the units asked for, so a part-filled order no longer takes holdings the exchange still holds. The sale log prints both figures, the sold count and the asked count, and the proceeds, the fold tranches and the Growth Rate Cap are all sized to the sold count.

`src/trading/scrumming/execution.py`, in `_execute_sell` — the units the exchange reported, read through `_settled_fill`, which re-reads the order until the exchange answers and warns on the log when it never does. The same figure reaches the fold tranches through `_last_sell_filled_units`, which `src/trading/scrumming/tick_phases.py` reads on both the scrum path and the distribute path.

When the exchange reports nothing sold. A limit order resting on the book reports a filled count of zero until it trades, and the re-read waits six tenths of a second. A sale the exchange has not reported now takes no units off the position and queues no fold tranche. The bot asks the exchange for its balance again, the activity log says the sale was sent and nothing was reported, and the open-order check refuses a second sell while the first one is still there.

```
# src/trading/scrumming/execution.py, in _settled_fill
        est_amt = amt

# src/trading/scrumming/execution.py, in _execute_sell
            if filled_units <= 0:
```

When the exchange fills that order later. The paragraph above stands for the tick that sends the order. The bot now keeps an order the exchange still holds open on `_resting_orders`. Each worked tick asks the exchange about it once and books each part the exchange filled through the same path a fill inside the re-read takes, so the units leave the position and the fold tranches queue at that tick. The section dated 2026-10-29 below gives every path and the readings.

```
# src/trading/scrumming/execution.py, in ExecutionEngineMixin._execute_sell
            asked = self._order_asked_units(self._last_settled_order or order, amount)
            self._hold_resting_order(
                order,
                "_book_sale_fill",
                "sell",
```

The same rule on the three other sale paths. The Fire button's rebalance and the detonation read the exchange's own filled count and treat a reported zero as zero, so neither takes units off the lot book on a sale the exchange has not reported. The self-destruct sells the whole position and empties the lot book; whatever the exchange did not sell goes back as one lot at the fill price, so the book never falls below what the exchange holds, and the completion line on the activity log names those units.

```
# src/trading/scrumming/execution.py, in _execute_detonation
        _filled_raw = getattr(order, "filled", None)

# src/trading/scrumming_bot.py, in self_destruct
        if self._current_holdings > 0:
            self._main_lots.append(
```

An exchange that sends no filled count at all. The paragraph above says the other sale paths "treat a reported zero as zero". That sentence stands, and this adds the case it does not name: an exchange that leaves the filled count out of its reply entirely. Three sale paths read the missing count as the count the exchange had accepted, which is the count the bot asked for, so a sale that reported nothing took the whole asked amount off the position. The detonation, the expiry close and the self-destruct each did this. **A missing count now books nothing, exactly as a reported zero does**, and the scrum path already read it that way.

```
# the detonation, now with no fallback under it
        _filled_raw = getattr(order, "filled", None)
        try:
            fill_amount = float(_filled_raw or 0.0)

# the expiry close, src/trading/scrumming/tick_phases.py in _tick_expiry_close
        reported = getattr(order, "filled", None)

# the self-destruct, src/trading/scrumming_bot.py in self_destruct
        _sd_filled = getattr(order, "filled", None)
        fill_amount = float(_sd_filled or 0.0)
```

True today: the detonation, the expiry close and the self-destruct no longer read `filled` off the first reply, and the reads quoted above on these three paths are gone. Each reads the order through `_settled_fill`, the re-read the scrum uses. A missing count and a reported zero still book nothing, and the order stays held until the exchange fills it or closes it. The self-destruct's read is quoted in the section dated 2026-10-29 below.

```
# src/trading/scrumming/execution.py, in ExecutionEngineMixin._execute_detonation
        fill_amount, fill_price, _ = await self._settled_fill(
            order, self.config.symbol, sell_amount, price, label="DETONATION"
        )

# src/trading/scrumming/tick_phases.py, in TickPhaseMixin._tick_expiry_close
        filled, fill_price, _ = await self._settled_fill(
            order, symbol, held, mark, label="EXPIRY CLOSE"
        )
```

What the four sale paths wrote, driven. Each site was driven with the exchange filling the whole amount, filling part of it, reporting zero, and leaving the count out. The bot record and the lot book were built for the run, the home directory was redirected before the first import, no exchange was contacted and no order was placed anywhere.

What the exchange said	Held after	Lot book after	Saved to disk
filled all ten units	2.000000	2.000000	2.000000
filled four of ten	8.000000	8.000000	8.000000
reported zero filled	12.000000	12.000000	12.000000
sent no filled count	12.000000	12.000000	12.000000

The position held twelve units in two lots before each run. The full fill is the control: it moves the held count, drops a drained lot and writes both figures to the saved document, so a row that does not move is a reading and not a writer that never ran. Across the scrum, the detonation and the self-destruct, the count of sites where the position fell further than the exchange filled is **nothing**, against **two** before this change on the two sites that read a missing count.

OVERTAKEN, quoted whole:

> *The self-destruct sells the whole position and empties the lot book; whatever the exchange did not sell goes back as one lot at the fill price, so the book never falls below what the exchange holds, and the completion line on the activity log names those units.*

True today: the self-destruct takes off the lot book only the units the exchange filled, highest buy price first, and every lot that survives keeps the price it was bought at. A sale that filled nothing takes nothing off. The book still never falls below what the exchange holds, because where the surviving lots account for fewer units than the exchange reports, that shortfall alone is added as one lot at the fill price.

`src/trading/scrumming_bot.py`, in `_take_filled_units_off_lots`

```
            _take = min(_held, _remaining)
            _lot["units"] = _held - _take
            _remaining -= _take
```

Why a lot's buy price is the figure at risk. A scrum queues one fold tranche per lot it sells from, and each tranche carries that lot's own buy price. A book holding one lot at the sale price prices every later fold off a price the bot never paid, and the held count stays right the whole time, so a check that watches units alone reports nothing.

Driven on a bot holding three lots. One unit bought at $10.00, two at $20.00 and three at $30.00, six units in all, the mark at $50.00. The home directory was redirected before the first import, every lot was built by hand, no exchange was contacted and no order was sent anywhere.

What the exchange said	Lot book before	Lot book now	Held after
filled nothing	1 lot at $50.00	3 lots at $10.00, $20.00, $30.00	6.000000
filled 2 of 6	1 lot at $50.00	3 lots at $10.00, $20.00, $30.00	4.000000
sent no filled count	1 lot at $50.00	3 lots at $10.00, $20.00, $30.00	6.000000
filled all 6	no lots	no lots	0.000000

The full fill is the control. It empties the book on both sides and reads the same figure in every column, so a row that moved is a reading and not a writer that stopped running.

What a later fold reads. The same bot was driven through a real scrum, which queues the tranche, and then a real fold, which discharges it.

The sale that ran first	The buy price the queued tranche carried
no self-destruct at all	$30.00
a self-destruct that filled nothing, before	$50.00
a self-destruct that filled nothing, now	$30.00

What else moved, swept. The filled count was swept from 0.00 to 6.00 units in steps of 0.05, 121 points, plus the case where the exchange sends no filled count at all. Fourteen figures read the same at every one of the 122 points: the fill price, the fold queue dollars, the tranche count, the held count, the last trade price, the units the book holds in total, the outcome, the reason, the sold count, the sold dollars, the bot's state, the lifetime scrummed dollars, the trade count and the closed-tranche count. Six figures moved, and every one of the six is the lot book's own shape or its prices, in memory and in the saved document alike. The prices moved at 121 of the 122 points, and the one point they did not move is the full fill.

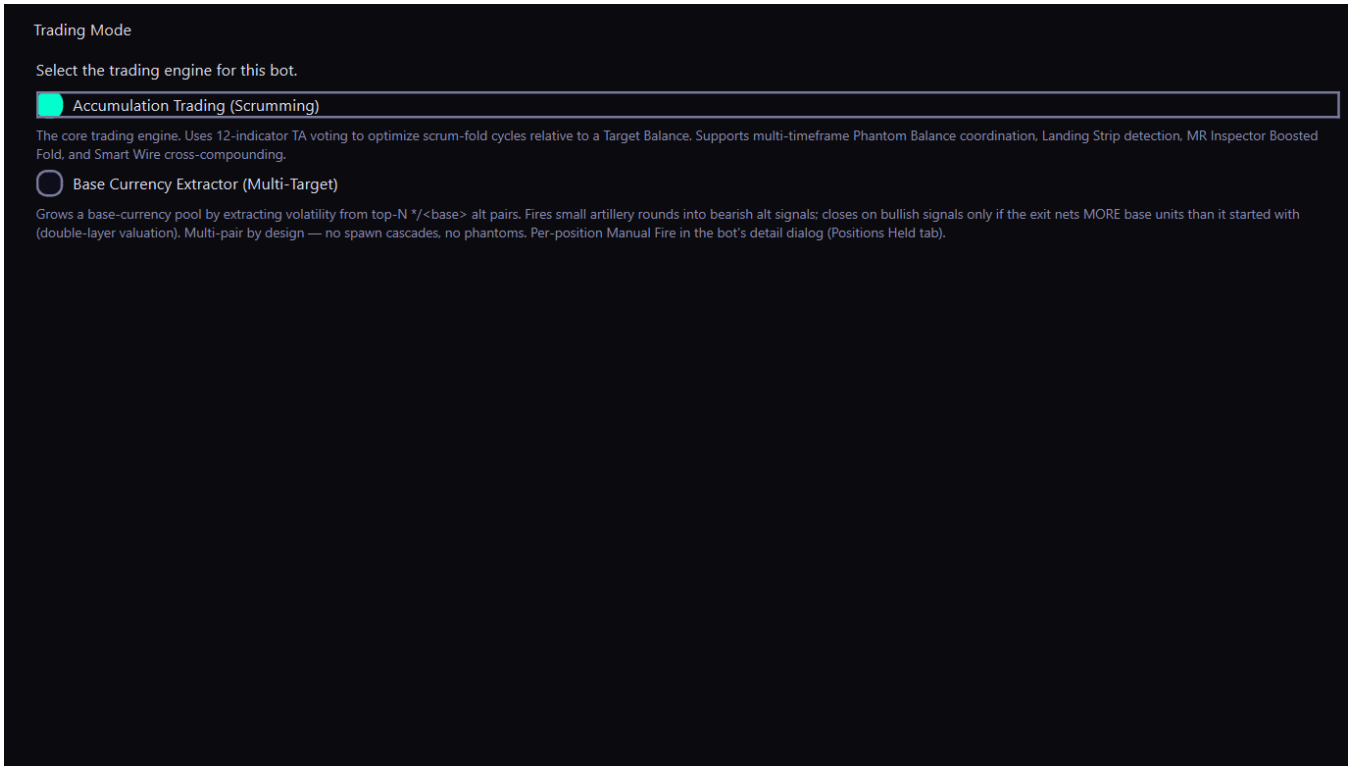
Which surfaces this reaches. Live only. The Paper Trader and the Simulator place no venue order, so neither has a sale that can fill short, and neither `PaperBotView` nor `SimBotView` carries a self-destruct at all — both settings tabs ask whether the bot has one before they press. The two surfaces that rebuild a Live composition

from a saved record read each lot's units and no lot's buy price, so both read 6.000000 from the repaired record and from the old one, against a control reading 3.000000 when a lot is dropped. Parity on a recovered record is closed by that reading.

```
src/simulator/fleet_source.py, in _lots_units    one lot at $50.00   6.0
                                                 three lots         6.0

src/paper/fleet_source.py, in _lots_units        one lot at $50.00   6.0
                                                 three lots         6.0
```

Fold - To buy an amount for an investment position that allows it to return to its initial price level. This also drives Compounding Growth based on Local Volatility and is restricted by the Maximum Growth Per Cycle setting which has been set to a conservative 1% globally for Acervator’s live test and development run.



The wizard's first page: Trading Mode, with Scrumming selected.

When the +New Bot button is pressed, the above window appears. Currently there are two “trading mode”

options (this will be change to Strategies). We are only interested in the Scrumming Bot at this point.

Functional. This is the wizard's first page, and the wizard opens on it. Two radio buttons, one description under each. Accumulation Trading is already selected when the page opens, and its description promises 12-indicator voting, which is what the engine builds. Choosing Base Currency Extractor sends you to the Extractor Pool page instead of the asset page. One question decides that branch, and the rest of the wizard asks the same question whenever it needs to know which kind of bot it is building.

OVERTAKEN, quoted whole:

> ***Functional.*** *This is the wizard's first page, and the wizard opens on it. Two radio buttons, one description under each. Accumulation Trading is already selected when the page opens, and its description promises*

12-indicator voting, which is what the engine builds.

True today: the page offers the engines the pressed sector serves. The accumulation row carries that sector's own variant name — Crypto Scrumming, Stock Scrumming, Commodity Scrumming, Forex Scrumming, Index Scrumming or Futures Scrumming — and it is selected when the page opens. The Extractor row shows on the crypto layer alone, so five of the six layers draw one row and crypto draws two. `src/gui/main_tabs/bot_wizard_surface.py`, in `mode_offer`, names the rows a sector offers, and `EXTRACTOR_SECTORS` names the one sector the Extractor reaches.

A third mode, Grid, is retired. Its test returns False without looking at a widget, and the branch that reads it is marked unreachable in the source.

Layer	Row the page draws	Second row
Crypto	Crypto Scrumming	Base Currency Extractor (Multi-Target)
Stock	Stock Scrumming	none
Commodities	Commodity Scrumming	none
Forex	Forex Scrumming	none
Indices	Index Scrumming	none
Futures / Perps	Futures Scrumming	none

`src/gui/bot_wizard.py` — `ModeSelectionPage.variant` answers the name the page offers, and `src/trading/scrumming/sizing.py`, in `sector_variant`, names the same variant for the sector the built bot carries, so the page and the engine cannot drift.

A layer whose variant the program does not hold draws no row at all. It draws why instead, worded as a refused market is worded, and the page offers no Next. `bot_wizard_surface.py`, in `mode_refusal`, writes that line through `sizing.UNTRADEABLE_REASON_FORMAT`. All six layers name a built variant today, so the count of layers drawing that line is zero.

`src/gui/bot_wizard.py` — `BotCreationWizard.nextId`

```
def nextId(self):
    current = self.currentId()
    if current == PAGE_MODE:
        if self._mode_page.is_extractor():
            return PAGE_EXTRACTOR_POOL
        return PAGE_ASSET
```

Functional. In the React build the wizard is drawn by `bot_wizard.js` into a space the exchange screen keeps for it. The screen keeps that space only after + New Bot is pressed. The press asks the surface, and the surface answers with the name of the module that draws the wizard.

`src/gui/main_tabs/exchange_tab_surface.py` — `react_bot_wizard`

```
def react_bot_wizard(exchange_id: str) -> str:
    """The module that draws the Create Auto Trader wizard for ``exchange_id``.

    ``ExchangeTabModel`` calls this as its ``on_new_bot``, so
    ``on_new_bot_clicked`` puts the module name in ``bot_wizard``.
    """
    return BOT_WIZARD_MODULE
```

Functional. Cancel closes the wizard. Finish closes it from the last page and refuses from any other. Either close drops the module the screen holds, so the space goes with the wizard and the exchange screen underneath it is whole again.

`src/gui/main_tabs/exchange_tab_surface.py` — `ExchangeTabModel.close_bot_wizard`

```
def close_bot_wizard(self) -> None:
    """Drop the wizard, so the screen keeps no space for it."""
    if self.bot_wizard is None:
        return
    self.bot_wizard = None
    self.calls.append([BOT_WIZARD_CLOSED, self.exchange_id])
```

Functional. Every control on the page sends the steps taken so far, not the one just pressed. The surface lays out fresh pages on each call and keeps nothing between them, so the page holds the walk and sends the whole of it. The answer replaces the payload the page holds and the wizard draws again.

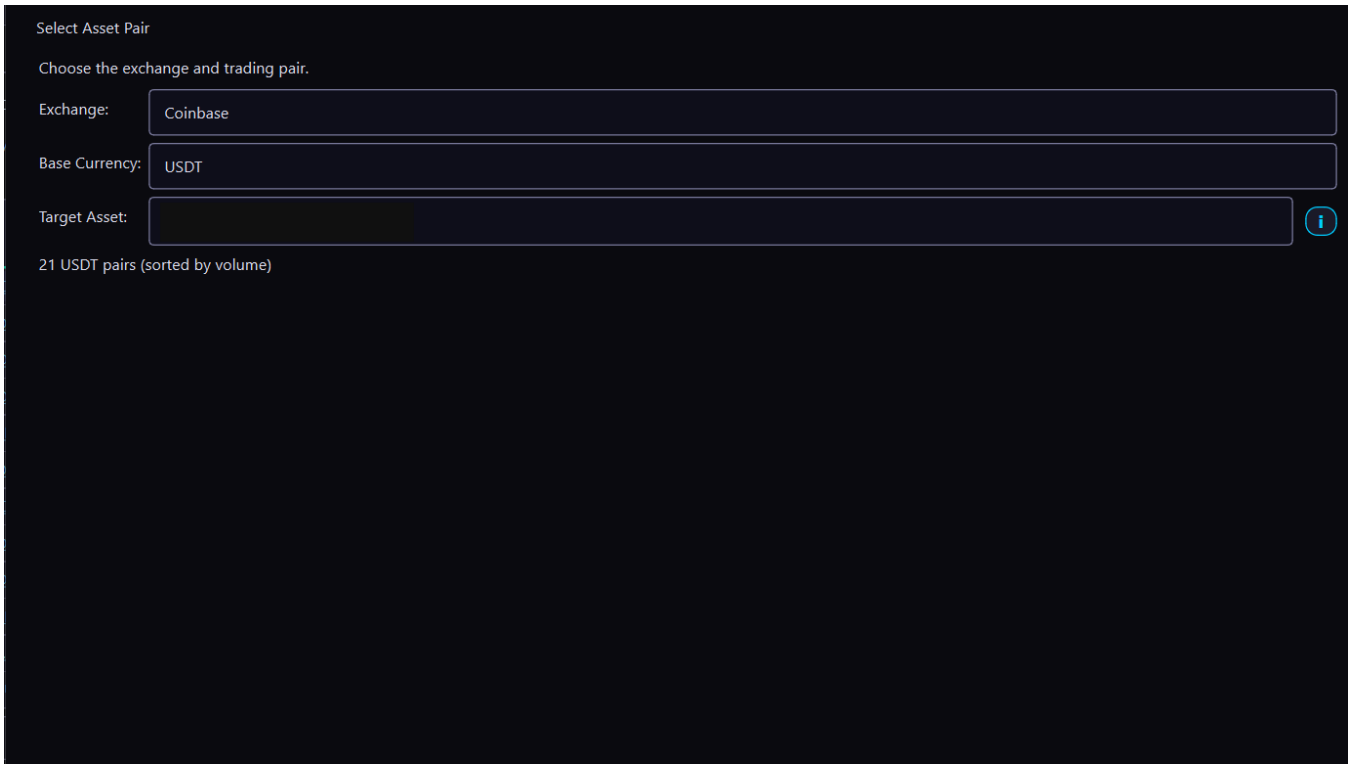
`src/gui/web/bot_wizard.js` — `press`

```
function press(name, step) {
  lastPress = { name: name, step: step };
  dispatched.push(lastPress);
  remember(step);
  lastAnswer = hasBridge()
    ? global.acervator.call(METHOD, copyOf(heldSteps)).then(take)
    : Promise.resolve(null);
  return lastPress;
}
```

What the fleet runs. The mode is the one setting this page writes, and it decides which engine class the bot becomes and which pages the wizard shows next. Every live record carries the Scrumming mode, so Accumulation Trading is both the choice the page opens on and the only kind of bot the live run has used.

Nothing is lost by taking the opening choice here. The Extractor pages later in this part describe a bot that has never run, and the Extractor's own description marks it partially built and untested. A reader following this walkthrough should take the Scrumming path and read the Extractor sections as a design.

```
mode   the page opens on   Accumulation Trading
       the fleet runs      scrumming on 38 of 38
```



The wizard's asset page: exchange, base currency and target asset.

After Scrumming / Accumulation is selected, next we are presented with Exchange, Base Currency, and Target Asset options.

Exchange - A privately and federally licensed financial platform that allows API interfacing for remote or automated trade execution.

The row lists the venues already connected. Picking one re-scans that venue and keeps only the spot markets the venue itself marks active.

`src/gui/bot_wizard.py` — the market filter inside `_fetch_markets`

```
for sym, info in exch.markets.items():
    if (
        not info.get("active", True)
        or info.get("type", "spot") != "spot"
    ):
        continue
```

Base Currency - This will be a National Currency, Stable Coin, or High Volume Crypto (A15, A14, BNB) for which multiple trading pairs are available on the selected exchange.

The page offers a fixed list of seven, and it holds exactly the national currencies, stable coins and high-volume crypto named above.

`src/gui/bot_wizard.py` — `AssetSelectionPage`, the base list

```
self._base.addItems(["USDT", "USDC", "A15", "A14", "BNB", "EUR", "USD"])
```

Target Asset - This is the asset in which the scrumming bot bases its position. If $50 and A05 are selected, it will strive to maintain and compound against a balance of $50 in A05 until it is shut down or some other adverse market condition occurs.

The list holds every asset trading against the base you chose, each with a cached coin icon and, where the venue reported one, a 24-hour volume figure. The round information button opens a written description of the selected asset, and the line under the rows counts the pairs found. Sorting by volume puts the tradeable pairs at the top of a list that runs to several hundred entries on a large venue, and the sort reads a cached figure rather than fetching one.

`src/gui/bot_wizard.py` — the volume sort

`src/gui/bot_wizard.py` — the volume sort

```
filtered.sort(
    key=lambda m: as_finite_float(m.get("volume")) or VOLUME_REFUSED_USD,
    reverse=True,
)
```

The sort reads through the one admission rule the trading package owns rather than a reading rule of the wizard's own, and a refused volume sorts as `VOLUME_REFUSED_USD` and labels as nothing. A saved number too large to be a float therefore costs that one pair its volume figure instead of losing the whole pair list.

The coin icon on a row. The icon is the file the logo library already keeps for that asset. The wizard reads the kept directory and fetches nothing, so picking an asset never waits on a network read. A row with no kept file draws a lettered disc in the asset's own colour, so the row still names what the bot would accumulate. The bot tables draw their marks from the same library, and Where a mark comes from covers the library itself.

`src/gui/bot_wizard.py` — `_get_coin_icon` reads the kept file

```
path = KEPT_LOGOS.kept_path(symbol)
if path and path.exists():
    px = QPixmap(str(path))
```

The address a crypto mark is filled from. A coin picture on the coin data source is addressed by that coin's own image id, and the id follows allocation order, so no id is derived from a ticker or from a slug. The crypto catalogue carries a confirmed id for ten assets. Every other entry sets no primary address and carries the ticker fallback alone, which is the icon address keyed on the ticker and needs no id.

```
assets in the crypto catalogue          48
carrying a confirmed coin image id      10
carrying the ticker fallback only       38
```

`src/exchange/crypto_assets.py` — `AssetManager.logo_candidates`, best address first

```
addresses = [indexed] if indexed else []
addresses += [asset.logo_url, asset.logo_fallback_url] if asset else []
addresses.append(SYMBOL_ICON_URL.format(symbol=name))
```

An unconfirmed id would still build an address, and the picture that address answered would be another coin's. An entry with no confirmed id therefore carries no primary address at all, and the reader falls through to the fallback.

Diagram: flowchart, left to right

- library fill asks one asset -> **coin index holds an address**
- B -- yes -> **read that address**
- B -- no -> **entry carries a confirmed id**
- D -- yes -> **read the coin picture address**
- D -- no -> **read the ticker fallback**
- read that address -> **the body is an image**
- read the coin picture address -> **the body is an image**
- read the ticker fallback -> **the body is an image**
- G -- yes -> **keep the file**
- G -- no -> **record the asset unresolved by name**

A body that is not an image is never kept, and the asset is written down by name with the reason, so the wizard draws the lettered disc rather than a broken picture. The asset logo library covers the fill and its read budget.

What the fleet runs. The three rows on this page decide more than the pair. The venue picked here rebuilds the TA Timeframe list two pages on, so a venue that does not carry a timeframe removes it from the choice, and the Trading Fee row exists because the venue sets what a round trip costs.

All 38 live bots sit on one venue. Every timeframe this walkthrough names below is therefore a timeframe that venue serves, and the fee split described further on is not a difference between venues.

The base currency and the target asset are each bot's own identity and have no fleet value to quote. What is worth knowing is the shape: one bot holds one target asset against one base, and the pool sigil an Extractor hands on instead is the only departure from that rule anywhere in the wizard.

```
exchange         the fleet runs   one venue across 38 of 38
base currency    the fleet runs   per bot
target asset     the fleet runs   one asset per bot
```

OVERTAKEN, and the paragraphs and the block above are kept as written. The wizard opens on the sector pressed on the Live tab, and the three rows on this page are that sector's. The target list is the products the recording holds under that sector, so a Stocks bot is never offered a crypto pair and a Crypto bot is never offered a ticker.

`src/gui/main_tabs/bot_wizard_surface.py` — the sector the wizard holds

```
def normalise_sector(name: Any) -> str:
    from .asset_class_surface import normalise

    held = normalise(name)
    return held or SECTOR_DEFAULT
```

OVERTAKEN, and the sentences and the two blocks above are kept as written. The Base Currency row is not a fixed list of seven. The picked venue's own market rows refill it: the seven keep their declared order and every further money leg the rows hold follows, in name order. On the live Coinbase recording that draws seven entries on four sectors, nine on Crypto and thirteen on Forex.

`src/gui/main_tabs/bot_wizard_surface.py, in base_items_for` — the entries the row offers

The row names the money a bot funds its position in and folds its proceeds back into. A venue records three symbol shapes for that money. A spot pair records it alone. A perpetual joins the leg it settles on. A dated contract joins that leg and its expiry date. The money is the same name in all three, so one reading takes all three.

`src/exchange/market_rules_store.py, in quote_currency` — the money leg

```
rest = str(symbol or "").partition(SYMBOL_SEPARATOR)[2]
held = rest.partition(SETTLE_SEPARATOR)[0]
return held.partition(EXPIRY_SEPARATOR)[0]
```

The store owns that reading, and two places read it: this row, through `src/gui/main_tabs/bot_wizard_surface.py, in quote_currency`, and the sale's fee check, through `src/trading/scrumming/execution.py, in _venue_quote_currency`. The venue reports its fee in the currency alone, so the check compares the same name the venue used and a contract's sale is not refused its fee.

Every sector now offers every market its recording holds. Driving both builds over the live recording reads 2147 markets offered against 2147 recorded, where the same drive read 1885 before: 896 of 896 on Crypto, 1032 of 1032 on Stocks, 25 of 25 on Commodities, 20 of 20 on Forex, 6 of 6 on Indices and 168 of 168 on Futures / Perps.

One base asset carries several contracts, so the Target Asset row names a contract by the symbol the venue recorded rather than by the asset alone. A spot pair is still named by its asset. The row that reads `A05` for the spot pair reads `A05/USD:USD-261030` for the December contract on the same asset, and the three contracts on one asset are three rows that read apart.

`src/gui/main_tabs/bot_wizard_surface.py, in market_name` — what a row is called

A market the account cannot clear on keeps its place in the list and carries the reason beside it, naming the clearing venue the venue published and the clearing reach this platform has. Finish refuses that market. The live recording carries no clearing venue on any of its 2147 rows yet, so no row draws that reason today.

`src/trading/scrumming/sizing.py, in clearing_refusal` — the refusal the row carries

The picked market travels to the bot as the symbol the venue recorded.

`src/gui/main_tabs/bot_wizard_surface.py, in asset_config` — what the asset page hands on

```
"base_currency": self.combo_text(BASE_COMBO_NAME).strip().upper(),
"target_asset": self.target_data() or EMPTY_TEXT,
"symbol": self.target_symbol(),
```

`src/trading/container/config.py, in BotConfig.__post_init__` joins the two legs only where that symbol is empty, so a contract reaches the engine as the market the operator picked and not as the spot pair of the same name.

The sector also decides the smallest order the bot may place. The venue publishes a size step for each market, and that step answers first; the sector answers only where the venue published none. Coinbase lists a tokenised metal and a dated contract under the same Commodities tab, and one divides while the other does not, so the sector alone cannot say.

`src/trading/scrumming/sizing.py` — which rule governs one market

```
def market_unit_rule(
    recorded: Any, asset_class: str = "", venue: str = "", moment_s: Any = None
) -> Optional[str]:
    demanded = session_unit_rule(getattr(recorded, "session", None), moment_s)
    if demanded is not None:
        return demanded
    declared = recorded_unit_rule(recorded)
    if declared is not None:
        return declared
    return unit_rule(asset_class, venue)
```

The wizard names no moment, so it reads no session and the step answers first. The order path in `guarded_place_order` names the moment, so a market whose own session takes a whole share alone reads whole outside its normal hours.

Driven with every outbound connection refused, on a recording carrying one market per sector, the wizard answers this for the market picked:

sector	market	the rule the market demands
Crypto	a spot pair	fractional
Stocks	a share	whole
Commodities	a tokenised metal	fractional
Commodities	a dated contract	whole
Forex	a tokenised fiat pair	whole
Forex	a pair the venue divides	fractional
Indices	an index contract	whole
Futures / Perps	a perpetual contract	whole

The two Commodities rows and the two Forex rows are the point. Both sectors carry markets of both kinds, and the row the sector cites never overrides the step the venue published.

A sector with no configured venue opens no wizard. Pressing New Bot there writes one line to the status log instead. All six sectors have a venue today, so all six open.

`src/gui/main_window.py` — what New Bot answers with no venue for the sector

```
exchanges = self._matching_exchanges()
if not exchanges:
    self._status_log.log(NO_MATCHING_VENUE_LOG, "warning")
    return
```

A sealed fleet opens no wizard either. While the saved fleet is sealed, pressing New Bot puts one box over the window and one Console line saying to unlock first, and no bot is made. The words and the measurement are on the Console page.

`src/gui/main_window.py` — what New Bot answers while the fleet is sealed

```
if self._refuse_while_sealed(exchange_id):
    return
```

Trading Parameters

Next we come to the combined Scrumming Bot configuration page which has several sections for fine tuning how the specific instance behaves. Given that Acervator, at the time of this writing, is still in active development the description for each setting should be seen as a design intention should any issues be encountered.



The Trading Parameters group of the wizard's parameter page.

OVERTAKEN, and the paragraph and the picture above are kept as written. Four lines sit under Scrum Fold Ratio in the Scrumming Settings group, and a market that divides draws only the first of them.

`src/gui/main_tabs/bot_wizard_surface.py` — the four lines

```
UNIT_RULE_NOTE = "unit_rule_note"
UNIT_MINIMUM_NOTE = "unit_minimum_note"
UNIT_INTERVAL_NOTE = "unit_interval_note"
UNIT_COARSE_NOTE = "unit_coarse_note"
```

The first line names the smallest move. The second names the two units a whole-unit position opens at and what they cost at one unit's price. The third names the position at which one unit equals one Opposing Trade

Interval, which is `100` divided by the interval: at a 5% interval that is 20 units. The fourth is drawn only below that size, and it says what the coarseness costs.

A smaller position is accepted. The wizard says the cost and Finish still creates the bot. Being priced out of a fine position is the market's nature, the same way being priced out of a market is.

`src/gui/main_tabs/bot_wizard_surface.py` — the position at which one unit is one interval

```
def interval_unit_count(interval_pct: Any) -> Optional[float]:
    if type(interval_pct) not in (int, float):
        return None
    held = float(interval_pct)
    if not math.isfinite(held) or held <= 0.0:
        return None
    return 100.0 / held
```

Driven on a share priced at $250 with a 5% interval, the four lines read:

```
This market does not divide, so the bot sizes in whole units.
A whole-unit position opens at 2 units, which cost $500.00 at $250.00000000 a unit.
One unit equals one 5% interval at 20 units.
$1,000.00 buys 4 units, so one unit moves 25% of the position against a 5%
interval. The bot still trades; every move is coarser than the interval it was
given.
```

Raising the Target Balance to $5,000 drops the fourth line. 20 units at $250 is $5,000, which is the size at which one unit and one interval are the same move.

When the Whole-Unit Market group shows

The group holds one box, Open Position At, and it shows for a market the engine ever sizes in whole units. Two kinds of market qualify.

The first is a market whose own published step is a whole unit. The second is a market on a venue that publishes a US equity session: such a venue takes a whole unit alone outside regular hours, so the same market divides at 10am and does not at 5pm. `src/trading/scrumming/sizing.py`, in `session_unit_rule`, is where the hour decides it, and `BotContainer.guarded_place_order` asks that question at the moment of every order.

`CITED_VENUE_SESSIONS` publishes twelve such rows, each read from the venue's own hours page: stock on Alpaca, E*TRADE, Fidelity, Interactive Brokers, Schwab, tastytrade and Webull; indices on Alpaca, E*TRADE and Fidelity; crypto and commodities on E*TRADE.

`src/gui/main_tabs/bot_wizard_surface.py`, in `session_takes_whole_units`, is where the wizard reads it, so Open Position At is offered wherever the engine applies it.

Personal Hold still follows the market's own step alone. A share that divides during the trading day can be held in fractions, so that box keeps its decimal places even where Open Position At shows.

That box is gone. Personal Hold was removed from every wizard and from every running-bot page, so no hold box takes a step from the market any more. Open Position At is the only row on this page that reads the market's own unit rule. `src/gui/main_tabs/bot_wizard_surface.py`, in

`session_takes_whole_units`, is where the wizard reads that rule.

Both builds draw the same four lines. The React page reads them from the same payload the Qt page reads, so neither can carry a word the other does not.

`src/gui/web/bot_wizard.js` — where the React page draws them

```
if (name === PARAMS_PAGE_NAME) {
  return rows.concat(groups).concat(unitNotes());
}
```

Order Visibility - Trades are listed on the order books or tracked internally to the platform.

Two entries, Order Book (Visible) and Internal (Invisible). `ScrummingBot` reads the choice once at construction and holds it as its invisible flag. The label a reader sees and the value the bot stores are two different strings on the same row.

`src/gui/bot_wizard.py` — the Order Visibility row

```
self._visibility = QComboBox()
self._visibility.addItem("Order Book (Visible)", "orderbook")
self._visibility.addItem("Internal (Invisible)", "internal")
self._visibility.setToolTip("How orders appear on the exchange.")
self._visibility.currentIndexChanged.connect(self._on_visibility_changed)
mf.addRow("Order Visibility:", self._visibility)
```

Current reading, 2026-10-29. The words are the same. The box takes them from the one wizard declaration, through the one description helper.

`src/gui/bot_wizard.py` — `TradingParamsPage`

```
set_description(self._visibility, BOT_WIZARD_DESCRIPTIONS["visibility"])
```

Driven on a bot rebuilt from a stored record, the two choices take different branches on the same tick: Internal activated all three stored Stack tranches, and Order Book activated none of them. The order type an engine trade carries is chosen off the same flag, on the sell side and the buy side alike.

`src/trading/scrumming/execution.py` — the sell side's order type

```
if self._invisible:
    ot = OrderType.MARKET
    exec_price = None
else:
    ot = OrderType.LIMIT
    exec_price = vh_fp
```

Aggressive Trading - Trades are priced so that they fill immediately. Trading like this is a bit like guerilla warfare. In and out before anyone notices.

A checkbox, off at the start. Every engine-initiated order then leaves as an immediate-or-cancel limit priced through the spread, so it pays the taker fee for an immediate fill. Manual Fire is unaffected.

`src/gui/bot_wizard.py` — the Aggressive Trading row

```
self._aggressive = QCheckBox("Aggressive Trading (force IOC-limit takers)")
```

The box's value reaches the bot and reaches no order. One place in the trading package builds an immediate-or-cancel order, inside the method that opens a Stack from a SCRUM, and that method raises before it gets there. The two places that do choose an order type read Order Visibility and never this flag.

`src/trading/scrumming_bot.py` — the only reader that would change an order

```
_aggressive = bool(getattr(self, "_aggressive", False))
_order_type_for_visible = (
    OrderType.IOC_LIMIT if _aggressive else OrderType.LIMIT
)
```

Stack Mode - Stack Mode enables Stack Tranches which operate on the Sell or Scrum side. This forms the “upside” of the organic ladder structure whereas Fold Tranches form its “downside”.

The box takes its start state from the engine rather than from a second literal typed onto the page.

`src/gui/bot_wizard.py` — `TradingParamsPage`, the Stack Mode default

```
from ..trading.bot_container import STACK_MODE_DEFAULT

self._stack_mode = QCheckBox(
    "Stack Mode (split SCRUM across upward tranches)"
)
# One declaration, so the box and the config cannot disagree.
self._stack_mode.setChecked(STACK_MODE_DEFAULT)
```

The box's value arrives on the bot and gates four places: the branch that turns a SCRUM into a Stack, and the three reconciliation stages. Driven, the guard discriminates — on, all three stored tranches activated; off, none did. No tranche can be created today, because the one method that appends one raises on an attribute no object in the tree carries. That raise is the reason the four rows below report a value that arrives and a behaviour that waits.

`src/trading/scrumming_bot.py` — the refusal, read from a restored bot

```
AttributeError: 'ScrummingBot' object has no attribute 'exchange_interface'
```

What the fleet runs. This is the row to read twice. The box opens **ticked**, because it takes its start state from the engine's own declaration, and all 38 live bots run Stack Mode **off**. A reader who walks the wizard and leaves this page alone builds a bot configured differently from every bot on the fleet.

The three rows below it only matter while this box is ticked. Split Distance, Tranche Count and Spacing shape the ladder a Stack builds, and no live bot has built one, so the figures those three carry on the fleet are the page's own opening values and not tuned choices. There is nothing to suggest about them, because nothing has run them.

The refusal above is why. One method appends a Stack tranche and it raises before it gets there, so a ticked box today changes what the record says and not what the bot does. That is a different thing from the box being off on purpose, and both are true of the fleet at once.

Current reading. The method that appends a Stack tranche no longer raises. Driven on a stand-in venue in a scratch home, a fold and a distribution each opened a three-rung ladder. The box gates two more places. A fold opens a ladder for every unit it bought, and the distribution sale opens one ladder for each set of a fold's extra units that no ladder holds, anchored no lower than that fold's buy price.

`src/trading/scrumming/tick_phases.py` — `TickPhaseMixin._tick_distribute`

```
if getattr(self.config, "stack_mode", False):
    await self._ladder_dist_sets(price, summary)
```

Split Distance - This setting determines the spacing between tranches if Tranche Spread (not available yet) is being used.

A percentage from 0.10 to 20.00, at 1.00 % to start. The bot hands it to the stack maths as the gap between one tranche and the next, and the Spacing row below decides how that gap grows across the ladder.

`src/gui/bot_wizard.py` — the Split Distance row

```
self._split_distance = QDoubleSpinBox()
self._split_distance.setRange(0.1, 20.0)
self._split_distance.setDecimals(2)
self._split_distance.setSuffix(" %")
self._split_distance.setValue(1.0)
```

Driven at the maths the bot hands it to, 1.00 % priced three rungs a gap of 0.9901 and 0.9804 per cent apart, and 2.50 % priced the same three 2.439 and 2.381 per cent apart. The figure reaches that maths only through the method named under Stack Mode above, so the value is correct and the ladder is not yet built.

`src/trading/scrumming_bot.py` — the hand-off

```
split_dist = float(getattr(self.config, "split_distance", 1.0) or 1.0)
```

Tranche Spread - This allows a given Scrum or Fold to divide its result X# of times across multiple incremented (as dictated by Split Distance) positions.

The page carries no control of that name, and no setting named `tranche_spread` reaches the engine.

The same sweep run for `split_distance` returns a declaration, a restore entry and a reader, so the sweep itself finds a setting when one is there.

In development.

The sentence that opens this row is retired. Counted over the whole tracked tree, the name appears once, and that once is this page describing its own absence. Spacing is the control that exists and is read, and it names the four models the engine carries. Tranche Count decides how many pieces a Scrum becomes, and Spacing decides where each piece sits.

`src/trading/stack_math.py` — the models Spacing chooses from

```
SPACING_MODES = ("quadratic", "fibonacci", "linear", "exponential")
```

Tranche Count - This can also be referred to as Spread Count. It determines how pieces a given Scrum or Fold is split into and distributed across incremented tranches as opposed to just one.

A whole number from 2 to 20, at 3 to start, written out as `stack_tranche_count_target`. It is a target rather than a promise: the runtime count drops when a tranche would fall under the venue minimum, or when two computed prices land within a tenth of a percent of each other and merge.

The sentence above about the runtime count is overtaken in one clause. It is not edited and not removed. It read:

> *It is a target rather than a promise: the runtime count drops when a tranche would fall under the venue minimum, or when two computed prices land within a tenth of a percent of each other and merge.*

The merge clause stands. The venue-minimum clause now reads: the count drops while the split can still give every piece the venue's smallest order size, and the split is **refused** where it cannot.

Three cases, driven on a hand-built ladder of one unit over four rungs:

```
floor 0.200000   four pieces of 0.250000      the count holds at four
floor 0.400000   two pieces of 0.500000       the count drops to two
floor 2.000000   refused                      no Stack opens
no floor cited   refused                      no Stack opens
```

`src/trading/stack_math.py`, in `_apply_min_order_size`, reads the smallest piece the ladder actually holds rather than a count of pieces, because a merge leaves pieces of unequal size and a count can fit while one piece sits under the floor. `src/trading/stack_math.py`, in `split_scrum_into_tranches`, refuses a floor that is not a positive finite number.

A venue that published neither a minimum amount nor an amount step has an unknown floor, and `src/trading/scrumming_bot.py`, in `_open_stack_from_scrum`, passes that absence through rather than reading it as zero. The scrum still sells as one undivided order; only the split is refused.

`src/gui/bot_wizard.py` — the Tranche Count row

```
self._stack_count = QSpinBox()
self._stack_count.setRange(2, 20)
self._stack_count.setValue(3)
```

Driven at the same maths, a target of 3 built three rungs and a target of 7 built seven, each rung's size falling from 0.4 to 0.171429 units out of one 1.2-unit Scrum. The count reaches that maths through the method named under Stack Mode, so it too is a correct value waiting on a ladder.

`src/trading/scrumming_bot.py` — the hand-off

```
n_target = int(getattr(self.config, "stack_tranche_count_target", 3) or 3)
```

Spacing - This adds a scaling factor Split Distance and works in conjunction with Tranche Spread and Tranche Count to induce curves and more aggressive growth within the ladder structure.

Three entries: Linear, Quadratic and Exponential. The sequence beside each name is the cumulative distance from the anchor in units of Split Distance.

`src/gui/bot_wizard.py` — the Spacing row

```
self._stack_spacing = QComboBox()
self._stack_spacing.addItem("Linear (1, 2, 3, 4…)", "linear")
self._stack_spacing.addItem("Quadratic (1, 2, 4, 7…)", "quadratic")
self._stack_spacing.addItem("Exponential (1, 2, 4, 8…)", "exponential")
```

The Quadratic entry read 1, 2, 4, 7 and the engine prices that model at n squared, so the row named a ladder the engine has never built. Both the wizard row and the live bot window now read 1, 4, 9, 16, which is what the engine publishes. The engine also carries a fourth model, Fibonacci at 1, 2, 3, 5, that neither control offers.

`src/trading/stack_math.py` — `level_multipliers`, driven for four levels

```
linear       [1.0, 2.0, 3.0, 4.0]
quadratic    [1.0, 4.0, 9.0, 16.0]
exponential  [1.0, 2.0, 4.0, 8.0]
fibonacci    [1.0, 2.0, 3.0, 5.0]
```

Personal Hold - Setting to be removed.

It is removed. The box is off both New Bot wizards and off the running-bot page, on Live, on Paper and in the Simulator, and the React pages for all three. The field is off the bot declaration and both engine readers are gone with it. Everything written below this line records what the control did while it was there. Read it as history, not as the page you see now.

The reservation stays, and it is automatic. A bot still works out which units on an asset are its own, so two bots on one asset cannot eat each other. Nothing on any screen configures it. `src/trading/capital_reservation.py`, in `effective_available`, is what bounds a bot to the units no other bot has claimed.

What changed, driven on records built by hand. A bot holding nothing back is unaffected: every figure reads the same before and after — 30 units at the venue, 30 claimable, 30 it may sell, a claim of 2.2 units at a price of 100, and 27.8 units left available to a bot sharing the asset. A bot holding five units back now sells the larger amount: 25 units before and 30 after, with its claim falling from 7.2 units to 2.2 and a co-tenant's available rising from 22.8 to 27.8. All 38 stored bots hold nothing back, so no bot on the fleet sells a different amount.

A bot saved with the old setting still loads. The retired name is dropped before a bot is built. `src/trading/container/config.py`, in `_sanitize_deprecated_kwargs`, is where it is dropped, and the record beside it lists every other setting retired the same way.

The control is still on the page. `TradingParamsPage.get_config` still emits `personal_hold_qty`, and the capital reservation mixin still reads it, adding it to the units the bot claims against its siblings. A removal has that reader to retire with it.

`src/trading/scrumming/capital_reservation_mixin.py` — `_compute_reservation_qty`

```
_hold = float(getattr(self.config, "personal_hold_qty", 0.0) or 0.0)
```

`src/gui/bot_wizard.py` — `TradingParamsPage.get_config`, the settings this group emits, in the order of the rows above

```
"stack_mode": self._stack_mode.isChecked(),
"split_distance": self._split_distance.value(),
"stack_tranche_count_target": int(self._stack_count.value()),
"stack_spacing_mode": self._stack_spacing.currentData(),
"personal_hold_qty": float(self._personal_hold_qty.value()),
```

The same method writes `visibility` and `aggressive_trading` before it branches on the kind of bot, so an Extractor carries those two as well.

Five sites read the number, not one. Two of them change what the engine decides. The other three carry the value to those two, or draw the row the operator types into.

```
src/trading/container/restore.py, in restore_bots_from_state
    stored key into the kwarg
src/trading/scrumming/capital_reservation_mixin.py, in _compute_reservation_qty
    the size of the claim
src/trading/scrumming/capital_reservation_mixin.py, in _ensure_capital_reservation
    the text of the claim
src/trading/scrumming/reconciliation.py, in _claimable_exchange_units
    the adoption ceiling
src/gui/live_settings/settings_tab.py, in _create_settings_tab
    seeds the live row
```

Both deciding readers are reached on every run. The claim is sized at the top of every tick. The adoption ceiling is read once when a bot starts, and again every reconcile interval after that.

```
src/trading/scrumming_bot.py, in tick                    the claim, once per tick
src/trading/bot_container.py, in _dispatch_bootstrap     the ceiling, on bot start
src/trading/scrumming_bot.py, in tick                    the ceiling, every reconcile interval
```

The ceiling is read on two more occasions, and one of them is not new. Sizing a sale reads it, which was true before any fill recorded anything, so the figure a detonation sells against is already net of the hold. Each fill on a path that is already holding a balance reading reads it as well, because the recorder that stores the reading nets it. The detonation's own fill is the exception: it moves the stored figure by the units it sold and reads no ceiling.

```
src/trading/scrumming/reconciliation.py, in _venue_claimable_units
    the ceiling, when a sale is sized
src/trading/scrumming/reconciliation.py, in _record_audited_venue_units
    the ceiling, on every reading it stores
src/trading/scrumming/execution.py, in _execute_buy
    a fill that reaches that recorder
src/trading/scrumming/execution.py, in _execute_manual_rebalance
    a fill that reaches that recorder, on every intent
```

The adoption ceiling is the reader that bears on a sale. A bot whose venue balance stands above its own book adopts the difference as a lot, and the hold is subtracted before that adoption, so the held units never enter the book the sell pre-check is measured against.

```
src/trading/scrumming/execution.py, in _execute_detonation — the sell pre-check reads the book
```

Driven on a restored record carrying a five-unit hold, with twenty units in the book, thirty at the venue and a price of one hundred, against the same code with all five readers taken out:

```
with the readers    claimable 25.0   adopted 5.0    book 25.0
readers removed     claimable 30.0   adopted 10.0   book 30.0
```

That bot may sell 25 units as the code stands and 30 with the readers gone. The five it holds back are the hold itself. The claim a bot places against its siblings moves the same way, 16.0 units down to 11.0, which raises a co-tenant bot's own permitted sale from 14.0 units to 19.0.

```
src/trading/capital_reservation.py, in effective_available — the co-tenant's cap
```

The removal is not taken. Stored state does load without the value — three records of three restored, 26 of 27 stored fields identical, none different, and nothing raised — but a bot carrying a hold would be free to sell the units the hold keeps back. The removal is safe only for a bot whose hold reads zero.

What the fleet runs. Personal Hold reads zero on all 38 live bots, so the five readers listed above subtract nothing from any claim today, and the removal above is safe for every bot on the fleet.

Order Visibility is the row in this group that changes a live trade. At Order Book the engine rests limit orders and the trade waits for the market; at Internal it sends market orders and takes the price on offer. 37 bots run Order Book and one runs Internal, so a reader following the page is following what nearly the whole fleet does.

Aggressive Trading is clear on all 38, which matches the box. Leaving it clear costs nothing today either way: the flag reaches the bot and reaches no order, as the two paragraphs above measure.

Stack Mode and the three rows under it carry their own note above, because the box opens the other way from the fleet.

```
setting              the page opens at   the fleet runs
Order Visibility     Order Book          Order Book 37, Internal 1
Aggressive Trading   clear               off on 38
Stack Mode           ticked              off on 38
Split Distance       1.00 %              1.0 on 38
Tranche Count        3                   3 on 38
Spacing              Linear              linear on 38
Personal Hold        0                   0.0 on 38
```

The last row in that table names a control that no longer exists. Six rows draw in this group now, not seven, and the picture below was taken before the removal.



The Scrumming Settings group.

Scrolling down we next find the first block of Scrumming Settings. These are the core or basic metrics for a Scrumming Bot.

Opposing Trade Interval - Establishes the minimum travel distance required by price action from the point a given trade in order the next trade of the opposite type to occur.

A percentage from 0.10 to 20.00, at 1.00 % to start. The Trading Fee below is added to it, so the real distance a reversal must travel is the two together.

`src/gui/bot_wizard.py` — the Opposing Trade Interval row

```
self._scrumming_interval = QDoubleSpinBox()
self._scrumming_interval.setRange(0.1, 20.0)
self._scrumming_interval.setDecimals(2)
self._scrumming_interval.setSuffix(" %")
self._scrumming_interval.setValue(1.0)
```

Driven on a bot restored from a stored configuration, the percentage sets two thresholds. One is the dollar drift the position must show before the tick looks at anything else. The other is the price a reversal must reach after an opposite trade, and the Trading Fee is inside both of them.

```
target $200.00
  interval 1.00 %   drift threshold $2.00    the tick reads the gates
  interval 5.00 %   drift threshold $10.00   the tick holds and says so

pivot $100.00000000, one tick later at $101.00000000
  interval 0.10 %   required $100.70000000   cleared, no hysteresis blocker
  interval 1.00 %   required $101.60000000   hysteresis blocker raised
  interval 5.00 %   required $105.60000000   hysteresis blocker raised
```

Asked for a figure under its floor the box answers the floor: asked for 0.00 it answers 0.10.

What the fleet runs. The box opens at 1.00 % and all 38 live bots run 5.00 %. On a $200 target that is a $10 drift before the tick reads any gate, where the opening figure would start reading gates at $2. The Trading Fee is added on top, so the real reversal distance on the fleet is 5 % plus that bot's own fee.

The sentence above about the reversal distance is overtaken by a third term. It is not edited and not removed. It read:

> *The Trading Fee is added on top, so the real reversal distance on the fleet is 5 % plus that bot's own fee.*

A third term now joins those two. `src/trading/otd_math.py`, in `minimum_opposing_trade_distance_pct_from_config`, adds the band extension, and `band_extension_pct_from_config` reads it off `BotConfig.band_travel_pct`. The operator's placement rule names that term: the minimum opposing trade distance, the trading fee, and the Bollinger Band extension where one is present.

Driven on one tape whose band span sat at 7.7331 % of its own midline, with the fleet's 5.00 % interval and a 1.60 % fee:

```
Band Travel   distance        guide on a tranche sold at $100.00
   0 %         6.600000 %      $93.40000000
  70 %        12.013138 %      $87.98686200
```

Zero switches the term off, so a bot with Band Travel at zero reads the same 6.600000 % it read before.

Current reading. The sum keeps a floor of two fees. Where the interval and the band extension together sit under one fee, the distance is two fees. A 5.00 % interval sits above every fee the box offers, so both rows above read the same figures.

`src/trading/otd_math.py` — `distance_settings`

```
return DistanceSettings(
    interval_pct=setting_pct(config, "scrumming_interval_pct"),
    fee_pct=trading_fee_pct_from_config(config, venue_fee_pct),
    band_travel_pct=setting_pct(config, "band_travel_pct", BAND_TRAVEL_OFF_PCT),
)
```

This is the row that decides how often a bot trades at all. A smaller figure makes the bot act on smaller moves and pay the fee more often; a larger one waits for a move worth acting on. Five per cent is the figure the live run has used throughout, and it is also the floor the cartridge threshold is clamped to, so raising it raises that floor as well.

BB Tolerance - Determines the minimum distance of the Bollinger Band extent price action must be in order for a trade action to occur.

A percentage from 0.25 to 5.00, at 1.00 % to start. The band-proximity detector takes it as its tolerance. This is the narrowest range on the page, and it cannot be set to zero.

`src/gui/bot_wizard.py` — the BB Tolerance row

```
self._bb_tolerance = QDoubleSpinBox()
self._bb_tolerance.setRange(0.25, 5.0)
self._bb_tolerance.setDecimals(2)
self._bb_tolerance.setSuffix(" %")
self._bb_tolerance.setValue(1.0)
```

Driven on one tape whose last close sat at band position 0.976123, the tolerance decides whether that close counts as sitting at the upper band, and the landing strip follows that answer. Asked for 0.00 the box answers 0.25, so it cannot be set to zero.

```
band position 0.976123, one tape, one tick
  tolerance 0.25 %   near upper False   landing strip False
  tolerance 5.00 %   near upper True    landing strip True, upper side
```

Landing Strip Candles - Determines the strictness of Landing Strip detection. Minimum is three candles. Longer Landing Strips are historically more likely to indicate an impending market reversal than shorter ones assuming the taper remains intact or grows tighter.

A whole number of candles from 2 to 10, at 3 to start. The three the description names is the opening value and not the floor: the box's own floor is 2, asked for 0 or 1 it answers 2, and asked for 11 it answers 10.

`src/gui/bot_wizard.py` — the Landing Strip Candles row

```
self._ls_candles = QSpinBox()
self._ls_candles.setRange(2, 10)
self._ls_candles.setValue(3)
self._ls_candles.setSuffix(" candles")
```

The tick runs two detectors over the same candles and this number reaches one of them. Band proximity takes the stored figure as its minimum pattern length. Tightening keeps its own minimum consecutive run, a platform constant of three, because a run of shrinking bodies is not a count of tight bodies at a band. At a stored 2 a trailing run of two tight bodies reads as a landing strip and lifts the confidence boost to 0.1900; at a stored 3 the same tape reads no landing strip and no boost.

```
tightening = detect_landing_strip_v2(
    candles,
    min_consecutive=TIGHTENING_MIN_CONSECUTIVE,
    shrink_threshold=TIGHTENING_SHRINK_THRESHOLD,
    bb_tolerance_pct=TIGHTENING_TOLERANCE_PCT,
)
```

Raising the box's floor to three is not taken. A box holding a stored 2 answers 3 the moment its range starts at three, and putting the range back leaves the 3 behind, so the next Save would write three into a bot whose record says two.

The description now reaches every screen that shows this row. The two wizards and the three Settings groups each attach it to their own box. Both row lists the shell draws from carry it in their wording tables, so a reader moving between a Qt screen and the shell meets one sentence. Read off the built screens, all eight answer that sentence and the stored figure stays 3 on each.

`src/gui/main_tabs/live_settings_tab_surface.py` — the Settings row's wording

```
"landing": "The bot confirms a Landing Strip when this many of the newest Heikin Ashi candles have small bodies and price sits near
a Bollinger Band. This count reaches the band-proximity check only. The separate shrinking-body check keeps its own fixed count.",
```

Current reading, 2026-10-29. The Settings row's wording table is the shared declaration itself, so every screen draws the shorter sentence below.

`src/gui/descriptions.py` — `SETTINGS_TAB_DESCRIPTIONS`

```
"landing": "Landing Strip: newest Heikin Ashi candles checked for Bollinger Band closeness. The shrinking-body check keeps its own
fixed count.",
```

TA Timeframe - This is the timeframe at which the bot operates and denotes the price chart it will monitor for trade decisions.

Seven entries to start, with 1h chosen. Pick an exchange and the list is rebuilt from the timeframes that venue carries, so a venue without 4h does not offer it. The starting choice is an index into the list rather than a named timeframe.

`src/gui/bot_wizard.py` — the TA Timeframe row

```
self._ta_timeframe = QComboBox()
```

```
self._ta_timeframe.setCurrentIndex(4)  # Default 1h
```

Driven on a bot restored from a stored configuration, the name decides which chart the bot fetches, and every indicator vote is then computed on that chart.

```
one bot, one tick each, two stored names
  1h   chart asked for 1h   consensus BULLISH 0.0219   band position 0.500000
  1d   chart asked for 1d   consensus BEARISH 0.0863   band position 0.976123
```

What the fleet runs. The box opens on 1h because its starting choice is index four of the seven entries. No live bot runs 1h. All 38 run 5m, the second entry, and they have run it for the whole live test. A reader who walks this page and leaves the row alone builds a bot on a chart the operator has never traded.

Pick 5m to match the fleet, or pick a slower chart deliberately and know what it costs. A slower chart smooths every indicator vote, so the bot sees fewer moves and fires less often; a faster one sees more moves and pays more fees to act on them. This row and the Read Rate row further on decide that together: the chart sets what a candle means, and the read rate sets how often the bot looks at it.

The choice also reaches two rows that are counted in candles rather than in minutes. The Soft CB Cooldown and the Extractor's watch-list refresh both scale with this pick, so changing the chart changes how long those two last without either figure moving.

When a venue drops the pick, the row says so. No venue carries every chart. Change the venue after picking a chart, or open Bot Settings on a bot whose saved chart its venue stopped carrying, and the list is rebuilt without that chart. The box then holds the hourly chart, or the first one on the list. One amber line under the box names the venue, the chart that was picked, and the chart the box holds instead.

`src/exchange/timeframes.py, in unserved_timeframe_notice`

```
UNSERVED_TIMEFRAME_FORMAT = (
    "{exchange} does not serve the {chosen} candle. This picker now shows {used}."
)
```

Driven on the wizard page and on all three Bot Settings dialogs, with the sockets refused and no venue contacted:

```
picked   venue        box holds   the line under the box
4h       coinbase     1h          coinbase does not serve the 4h candle. This picker now shows 1h.
2h       coinbase     2h          nothing, and the row is not drawn
4h       notavenue    4h          nothing, and the row is not drawn
```

The middle row is the control. A chart the venue does carry is kept and nothing is said, so the line is reading the venue's own list and not firing on every open. The last row is the second control. A venue the table does not list is read as carrying every chart, so a venue that names nothing has nothing refused.

The line is drawn in one place for every screen that holds a chart picker, so the wording cannot drift between them.

```
src/gui/bot_wizard.py, in set_exchange_id
src/gui/live_settings/settings_tab.py, in _create_settings_tab
src/gui/paper/paper_settings_tab.py, in _create_settings_tab
src/gui/simulator/sim_settings_tab.py, in _create_settings_tab
```

What is still silent, and it is not this row. The line reports to a screen he opens. A bot already running on a chart its venue does not carry asks for that chart on every tick, reads nothing back, and takes no indicator reading from the answer. Nothing is written to its log and nothing is drawn on its card. Where that warning belongs on a running bot is an open question for the author.

The bot's own chart now changes with it. The amber line above reports a picker. The bot kept the chart the venue does not carry and went on asking for it. A bot built or restored on such a chart now has its own stored chart replaced by the longest shorter chart that venue does carry, and runs on that.

`src/exchange/timeframes.py, in snapped_timeframe`

```
below = ALL_TIMEFRAMES[: ALL_TIMEFRAMES.index(chosen)]
for shorter in reversed(below):
    if shorter in served:
        return shorter
return ""
```

The replacement happens once, where the bot's settings are built, so every reader of the setting sees the same chart.

`src/trading/container/config.py, in BotConfig._snap_ta_timeframe`

```
unserved = self.ta_timeframe
self.ta_timeframe = shorter
self.ta_timeframe_unserved = unserved
```

Driven on a built bot with the sockets refused and no venue contacted, reading the chart the bot asks for rather than the chart a box displays:

```
stored   venue       bot runs on   bot asks the venue for
4h       coinbase    2h            2h
2h       coinbase    2h            2h
4h       notavenue   4h            4h
```

The middle row is the control. A chart the venue does carry is left alone, so the replacement is reading the venue's own list. The last row is the second control. A venue the table does not list is read as carrying every chart, so a venue that names nothing has nothing replaced.

Every venue and every chart, driven as a stored setting. The table holds 19 venues and the application offers 11 charts, which is 209 pairs. Twenty-six of those pairs are a chart the venue does not carry, and all 26 are replaced.

```
bitfinex     2h->1h
cryptocom    2h->1h
coinbase     4h->2h   12h->6h   1w->1d
gateio       6h->4h   12h->4h
gemini       2h->1h   4h->1h    12h->6h   1w->1d
huobi        2h->1h   6h->4h    12h->4h
kraken       2h->1h   6h->4h    12h->4h
mexc         2h->1h   6h->4h    12h->4h
```

The two Coinbase aliases carry the same three rows as Coinbase itself.

A chart with nothing shorter on the venue's list. No chart is ever replaced by a longer one. Where the venue carries nothing shorter, the stored chart is left exactly as it is and the bot is not refused. Measured across all 209 pairs, that case arises **0** times, because every one of the 19 venues carries the one-minute chart, the shortest the application offers.

The warning and the log line. One amber line under the box names the venue, the chart that was dropped, and the chart the bot runs on instead. It is drawn on the live, Paper and Simulator Bot Settings dialogs and appended to the box's tool tip on each.

`src/exchange/timeframes.py, in snapped_timeframe_notice`

```
SNAPPED_TIMEFRAME_FORMAT = (
    "{exchange} does not serve the {chosen} candle. This bot operates on "
    "{used} instead, as a forced condition."
)
```

The same replacement writes one warning line to the console log, read back out of the file:

```
2026-10-24 [WARNING] coinbase does not serve the 4h candle; ta_timeframe forced to 2h on AAA/BBB
```

Driven on the same run, a bot on a chart Coinbase does carry wrote **0** such lines and a bot on a venue the table does not list wrote **0**, so the line follows the venue's list and not every start-up.

Picking a chart yourself clears the line. A chart chosen in the dialog is the operator's own, so the forced-condition line stops being true. Choosing one clears the recorded dropped chart in the same press, and the line is gone the next time the dialog opens.

`src/gui/live_settings/settings_tab.py, in SettingsTabMixin._ta_timeframe_picked`

```
self._mark_changed("ta_timeframe", timeframe)
self._mark_changed("ta_timeframe_unserved", "")
```

All three wizards now say it too. The Paper wizard and the Simulator wizard rebuilt their chart list from the venue and said nothing. All three wizard pages and all three Bot Settings dialogs now draw a line.

```
src/gui/bot_wizard.py, in set_exchange_id
src/gui/paper/paper_bot_wizard.py, in set_exchange_id
src/gui/simulator/sim_bot_wizard.py, in set_exchange_id
src/gui/live_settings/settings_tab.py, in _create_settings_tab
src/gui/paper/paper_settings_tab.py, in _create_settings_tab
src/gui/simulator/sim_settings_tab.py, in _create_settings_tab
```

What a bot restored on a carried chart looks like. Fifty-nine settings before, sixty after, every one of the fifty-nine identical to the byte. The one setting added is the record of a dropped chart, and it is empty.

Target Balance - This is the intended starting and locked value for the investment position that the Scrumming Bot is controlling.

From $1.00 to $1,000,000.00. Its start value is whatever default the wizard was handed, which is the figure the Settings Trading page stores.

`src/gui/bot_wizard.py` — the Target Balance row

```
self._target_balance = QDoubleSpinBox()
self._target_balance.setRange(1.0, 1000000.0)
self._target_balance.setDecimals(2)
self._target_balance.setPrefix("$ ")
self._target_balance.setValue(defaults.get("default_target_balance", 200.0))
```

Driven on one position worth $203.00, the figure is the line the excess is measured from, and the Opposing Trade Interval above is a percentage of it. A dollar and a half of target turns the same position from one the tick evaluates into one it holds.

```
one position of $203.00, one tick each
  target $200.00   excess +$3.00   scrum side   interval $2.0000   gates read
  target $201.50   excess +$1.50   scrum side   interval $2.0150   tick holds
```

The start value does come from the figure the wizard is handed: handed 200.00 the row opens at 200.00, and handed 777.50 it opens at 777.50. Asked for 0.00 the box answers 1.00.

Max Entry Price - If the new bot does not detect the requisite amount (as dictated by Target Balance) of the Target Asset, this price threshold sets a limit at which it will attempt to perform the initiating Fold.

Eight decimal places, at $0.00000000, where zero means no ceiling. The buy path is the only reader, and it refuses a buy above the ceiling. Manual Fire goes around it.

`src/gui/bot_wizard.py` — the Max Entry Price row

```
self._max_entry_px = QDoubleSpinBox()
self._max_entry_px.setRange(0.0, 10_000_000.0)
self._max_entry_px.setDecimals(8)
self._max_entry_px.setPrefix("$ ")
self._max_entry_px.setValue(0.0)
```

Min Entry (Should Be Exit) Price - If the new bot detects a requisite amount (as dictated by the Target Balance) of the Target Asset, this price threshold sets a limit at which it will attempt to perform the initiating Scrum.

The same shape, where zero means no floor. The buy path is the only reader here too, so the number refuses a buy below the floor and never reaches a sell. The row is labelled Min Entry Price on the page, without the parenthesis his line carries.

`src/gui/bot_wizard.py` — the Min Entry Price row

```
self._min_entry_px = QDoubleSpinBox()
self._min_entry_px.setRange(0.0, 10_000_000.0)
self._min_entry_px.setDecimals(8)
self._min_entry_px.setPrefix("$ ")
self._min_entry_px.setValue(0.0)
```

Trading Fee % - This allows the trading fee for the target exchange to be set. This is added to Minimum Opposing Trade Distance to further ensure buys / sells are properly distant and that a given bot is not losing an excessive amount to fee chop in volatile but overly tight market regimes.

A percentage from 0.00 to 5.00, at 0.60 % to start, which is the Coinbase Advanced Trade maximum tier. It steps in twentieths of a percent, and it is a per-side figure.

`src/gui/bot_wizard.py` — the Trading Fee row

```
self._trading_fee = QDoubleSpinBox()
self._trading_fee.setRange(0.0, 5.0)
self._trading_fee.setSuffix(" %")
self._trading_fee.setDecimals(2)
self._trading_fee.setSingleStep(0.05)
self._trading_fee.setValue(0.6)
```

Current reading. The distance reads the larger of this box and the highest fee the venue has reported on the bot's fills, so a venue that charges more than the box is never read as the box. A box at zero reads 0.60 % in every check, the figure the Settings tab shows for it.

`src/trading/otd_math.py` — `trading_fee_pct_from_config`

```
fee = setting_pct(config, "trading_fee_pct", DEFAULT_TRADING_FEE_PCT)
if fee < 0.0:
    raise DistanceSettingUnreadable(
        "trading_fee_pct", getattr(config, "trading_fee_pct", None), BELOW_ZERO
    )
venue = float(venue_fee_pct or 0.0)
if math.isfinite(venue) and venue > fee:
    return venue
return fee
```

What the fleet runs. The box opens at 0.60 % and the live fleet is split: 24 bots run 1.6 % and 14 run 0.6 %. All 38 sit on one venue, so the split is not a venue difference, and nothing else in the stored record separates the two groups.

The higher figure is the more conservative one, and that is worth understanding before changing it. The fee is added to the Opposing Trade Interval, so a reversal on a 1.6 % bot has to travel further before it may fire. A reader setting this row is not only recording what the venue charges; they are also setting how far price must move between a sell and the next buy.

Max Target Growth % - This determines the maximum amount of growth the Target Balance can increase in a given Market Cycle with a cycle being a Fold / Scrum / Fold sequence. Essentially any Fold preceded by a Scrum will be allowed to Fold an amount of profit back in and, if Surplus remains after the Target Delta is re-zero’d, it can be used to increase Target Balance up to this hard limit for that cycle. This is the organic compounding mechanic.

Now: only a fold spends wire credits, under this cap, as it spends a sale's own dollars; a cartridge or Manual Fire buy leaves them in the wallet (`src/trading/scrumming/sizing.py, in rebalance_held_usd`).

A percentage from 0.00 to 100.00, at 1.00 % to start. Setting it to zero freezes Target Balance. It steps a quarter of a percent at a time, and it is the only mechanism allowed to raise the figure.

`src/gui/bot_wizard.py` — the Max Target Growth row

```
self._max_target_growth_pct = QDoubleSpinBox()
self._max_target_growth_pct.setRange(0.0, 100.0)
self._max_target_growth_pct.setSuffix(" %")
self._max_target_growth_pct.setDecimals(2)
self._max_target_growth_pct.setSingleStep(0.25)
self._max_target_growth_pct.setValue(1.0)
```

Scrum Fold Ratio - This precedes Surplus calculation as described under Max Target Growth %. It determines how much of a given trade's profits will be redistributed to directly contribute to its own organic compounding. Note that this does not interfere with normal Target Delta re-zeroing and is intended to only serve as a "cushion" to slow runaway compounding when Wire Credits are being received from multiple sources.

A whole percentage from 1 to 100, at 100 % to start.

`src/gui/bot_wizard.py` — `TradingParamsPage.get_config`, the settings this group emits, in the order of the rows above

```
"scrumming_interval_pct": self._scrumming_interval.value(),
"bb_tolerance_pct": self._bb_tolerance.value(),
"bb_landing_strip_candles": self._ls_candles.value(),
"ta_timeframe": self._ta_timeframe.currentData(),
"target_balance": self._target_balance.value(),
"max_entry_price": (float(_max_ep) if _max_ep > 0 else None),
"min_entry_price": (float(_min_ep) if _min_ep > 0 else None),
"trading_fee_pct": self._trading_fee.value(),
"max_target_growth_pct": self._max_target_growth_pct.value(),
"scrum_fold_pct": self._scrum_fold_pct.value(),
```

Where the code departs. Three of these rows read differently in the engine than the entries above say.

Min Entry Price is the first. The buy path is the only place either entry-price field is read; it refuses a buy above the ceiling and refuses a buy below the floor. Neither number reaches the sell path, so a bot that holds the asset and reaches that price sells nothing. Manual Fire runs its own rebalance and reads neither.

```
ceiling $90 against a price of $100    buy refused at the gate
ceiling $110 against the same price    gate silent
floor $110 against a price of $100     buy refused at the gate
the same floor, the same price, sell   no refusal of any kind
```

`src/trading/scrumming/execution.py` — `_execute_buy`

```
_max_ep = getattr(self.config, "max_entry_price", None)
_min_ep = getattr(self.config, "min_entry_price", None)
try:
    _px = float(price) if price is not None else 0.0
except (TypeError, ValueError):
    _px = 0.0
if _max_ep is not None and _px > 0 and _px > float(_max_ep):
```

The sell path already takes the price, so the floor check has somewhere to attach.

Proposed, not present, in `_execute_sell`*:*

```
_min_ep = getattr(self.config, "min_entry_price", None)
if _min_ep is not None and price > 0 and price < float(_min_ep):
    self._bus.emit(
        "bot.log",
        bot_id=self.bot_id,
        message=(
            f"SELL REFUSED (min_entry_price floor): current price "
            f"${price:.8f} < ${float(_min_ep):.8f}."
        ),
    )
    return None
```

`min_entry_price` is declared in `src/trading/container/config.py` and reaches the bot through the wizard block above, so the proposal adds no new setting.

A zero in either entry-price box is not the same thing in both. Both rows say zero means no ceiling and no floor, and both controls honour that: a box left at zero emits an absent value rather than a number. The engine's own guard is narrower. It treats only the absent value as off, so a record carrying a literal zero in the ceiling refuses every auto-buy for as long as it stands. The same zero in the floor is harmless, because no price is below it.

`src/trading/scrumming/execution.py` — the two guards

```
if _max_ep is not None and _px > 0 and _px > float(_max_ep):
if _min_ep is not None and _px > 0 and _px < float(_min_ep):
```

No control can write that zero. The three places that collect the value each write an absent value in its place, so a record carrying a literal zero came from an older build or from an edit outside the application.

```
src/gui/bot_wizard.py                          absent unless the box reads above 0
src/gui/live_settings/settings_tab.py          absent unless the box reads above 0
src/gui/main_tabs/bot_wizard_surface.py        absent unless the box reads above 0
```

Proposed, not present, in both guards:

```
if _max_ep and float(_max_ep) > 0 and _px > 0 and _px > float(_max_ep):
```

It is proposed rather than taken because a bot standing on such a record is refusing every buy today, and a build that read the zero as off would have it buying on its next tick.

Trading Fee is the second. The fee is added to the Minimum Opposing Trade Distance, and at every figure the box offers except one every reader agrees. At a stored zero they split: ten reads inside the trading package treat a zero as six tenths of a percent, and two inside the same executor honour the zero.

```
stored 0.60 %   distance filter 1.60 %   buy and sell hysteresis 1.60 %
stored 2.50 %   distance filter 3.50 %   buy and sell hysteresis 3.50 %
stored 0.00 %   distance filter 1.60 %   buy and sell hysteresis 1.00 %
```

`src/trading/otd_math.py` — the read, and its own note on the split

```
return minimum_opposing_trade_distance_pct(
    getattr(config, "scrumming_interval_pct", 0) or 0,
    getattr(config, "trading_fee_pct", 0.6) or 0.6,
)
```

That module's docstring names the split and says it is named there and not repaired there. Closing it changes the distance a reversal must travel for any bot whose fee is zero, which is a live figure on a live tick.

Current reading. The split is closed. Every read takes a stored zero as 0.60 %, and the distance keeps a floor of two fees. At a 1.00 % interval the three stored figures read:

```
stored 0.60 %   distance filter 1.60 %   buy and sell hysteresis 1.60 %
stored 2.50 %   distance filter 5.00 %   buy and sell hysteresis 5.00 %
stored 0.00 %   distance filter 1.60 %   buy and sell hysteresis 1.60 %
```

Max Target Growth % is the third, and it is closed. Nine read sites take the figure, and every one of them names the declared default of 1.0. No bot can reach that fallback: every construction path builds the config through one factory and the class declares the field, so the value is never absent. A tenth site was an expression whose value was discarded and it is gone.

`src/trading/scrumming_bot.py` — the shape all nine share

```
_growth = float(getattr(self.config, "max_target_growth_pct", 1.0) or 0.0)
```

Three of the nine used to fall back to zero, which would have made a bot compound or freeze depending on which site read first. They are the manual rebalance, the compounding snapshot and the Smart Wire inputs.

```
src/trading/scrumming/execution.py   _execute_manual_rebalance
src/trading/scrumming/snapshots.py   _compounding_snapshot
src/trading/scrumming_bot.py         get_swos_inputs
```

Where the Scrum Fold Ratio is read. One method reads it, once per sale, and it runs on the slice of tranches that sale appended rather than on the whole queue.

Now: a sale on a bot with an outgoing wire reads it once more, for the share of the sale's profit that its wire surplus comes from (`src/trading/scrumming/wire_routing.py, in _sale_surplus_usd`).

`src/trading/scrumming/fold_tranches.py` — the read and the clamp

```
_fold_pct = max(0, min(100, int(getattr(self.config, "scrum_fold_pct", 100))))
_new_tranches = self._fold_tranches[_tranche_count_before:]
if _fold_pct < 100 and _new_tranches:
```

At a hundred the branch does not run and the sale's tranches keep every dollar. Below a hundred each tranche keeps the ratio's share of units and of the sale's own proceeds, and the remainder is retired as cash with a realised profit line. Money in a tranche that did not come from this sale is wired-in credit and keeps its full value.

```
100 % on two $100 tranches   queued $200.00, 2.0000 units
 40 % on the same two        queued  $80.00, 0.8000 units
  1 % on the same two        queued   $2.00, 0.0200 units
```

The clamp's lower bound is zero and both controls start at one, so a record carrying a zero empties the sale's whole fold queue and retires the cash instead. Raising the clamp to match the controls would change what such a bot rebuys on its next sale.

What the fleet runs. Three of the ten rows in this group are set away from what the page opens at, and the two most important are the rows a reader is most likely to leave alone. Those two carry their own notes above. The other eight match the page, and that match is the suggestion.

Target Balance is the one row with no fleet value to quote. It is set per bot: ten distinct figures across the 38, chosen against each position rather than against a rule. Both entry-price rows are unset on all 38, which also settles the literal-zero case described above. No live record carries the zero that would refuse every automatic buy, so that hazard is on the page and not on the fleet.

Max Target Growth sits at 1.00 % on every bot, the same figure the operator's own line earlier in this part names as the conservative global setting for the live run. Scrum Fold Ratio reads 100 on 30 bots and 50 on eight; at 100 the branch that trims a sale's tranches never runs at all, so those 30 bots requeue every dollar a sale produced.

The two tolerance rows match the page and work on the same candles from different sides. BB Tolerance decides whether a close counts as sitting at a band, and Landing Strip Candles decides how long a tight run has to be before it counts as a pattern. Loosening the first finds more band touches; raising the second finds fewer patterns.

```
setting                  the page opens at     the fleet runs
Opposing Trade Interval   1.00 %               5.0 on 38
BB Tolerance              1.00 %               1.0 on 38
Landing Strip Candles     3 candles            3 on 38
TA Timeframe              1h                   5m on 38
Target Balance            the Settings figure  ten distinct figures across 38
Max Entry Price           $0.00000000          unset on 38
Min Entry Price           $0.00000000          unset on 38
Trading Fee %             0.60 %               1.6 on 24, 0.6 on 14
Max Target Growth %       1.00 %               1.0 on 38
Scrum Fold Ratio          100 %                100 on 30, 50 on 8
```



The Advanced Scrumming and Hedge Rebalance groups.

Next are the “advanced” Scrumming settings which primarily affect when the bot is allowed to fire a trade. These can be thought of as “calibrating the scope”.

Detect Threshold - Intended as the point at which the bot “takes the safety off” and starts looking for a shot. To be re-evaluated.

A whole percentage from 10 to 90, at 75 % to start. The engine reads 75 as a lower mark of 0.125 and an upper mark of 0.875, measured across the band rather than in dollars.

`src/trading/scrumming/circuit_breakers.py` — `_bb_detect_thresholds`

```
  detect_frac = max(0.0, min(1.0, detect_pct / 100.0))
  half = detect_frac * 0.5
  return (0.5 - half, 0.5 + half)
```

Driven on one 40-candle tape, both marks move with the figure: 10 gave 0.45 and 0.55, 50 gave 0.25 and 0.75, 75 gave 0.125 and 0.875, and 90 gave 0.05 and 0.95. The tick reads the pair twice, and the upper mark is a hard gate. On a tape at band position 0.75 the figure at 75 held the scrum and named two blockers, and the same tape at 40 cleared both and armed it.

`src/trading/scrumming_bot.py` — the two readings in one tick

```
  _bb_lower_dt_pre, _bb_upper_dt_pre = self._bb_detect_thresholds()
```

Fire Threshold - The final Bollinger Band approach metric. Once satisfied, the bot can fire a trade.

A percentage from 0.10 to 10.00, at 0.50 % to start. It measures distance from the band, where the Detect Threshold above it measures distance from the midline.

`src/gui/bot_wizard.py` — the Fire Threshold row

```
self._scrum_fire_pct = QDoubleSpinBox()
self._scrum_fire_pct.setRange(0.1, 10.0)
self._scrum_fire_pct.setDecimals(2)
self._scrum_fire_pct.setSuffix(" %")
self._scrum_fire_pct.setValue(0.5)
```

The ramp's near-band test is the only place this figure lands. Driven on one tape at band position 0.75, 0.50 % and 2.00 % each left the ramp short of FIRE and the scrum blocked; 10.00 % took the ramp from SEARCH to FIRE in one tick and removed that blocker.

`src/trading/scrumming/tick_phases.py` — the near-band test

```
fire_pct_frac = self.config.scrum_fire_pct / 100.0
_near_band = abs(_price - _bb_upper) <= fire_pct_frac * _bb_upper
```

BB Midline Gate - This is another, perhaps redundant layer, of Bollinger Band travel protection. It is different in that it is concerned with distance from the midline instead of the entire local width.

A checkbox, on at the start. While it is on, a scrum fires only above the midline and a fold only below it.

`src/gui/bot_wizard.py` — the BB Midline Gate row

```
self._bb_midline_gate = QCheckBox("BB Midline Gate")
self._bb_midline_gate.setChecked(True)
```

Driven on one tape at band position 0.27 with the position above target: on, the tick refused the scrum and named the band gate in its blocker list; off, that blocker was gone. The fold half of the same reading stayed open both ways, because the price sat below the midline.

`src/trading/scrumming_bot.py` — the midline block

```
if self.config.bb_midline_gate:
    scrum_ok = (bb_pos > 0.50) and not self._phantom_locked
    fold_ok_midline = bb_pos < 0.50
else:
    scrum_ok = not self._phantom_locked
    fold_ok_midline = True
```

One sentence above is overtaken. It is not edited and not removed. It reads:

> *The fold half of the same reading stayed open both ways, because the price sat below the midline.*

The current reading. That holds while no phantom lock is held. The fold half now carries the phantom lock as well as the band, so with phantom bots switched on and a lock held, the fold half is refused below the midline too. With phantom bots switched off the fold half reads the band alone, exactly as the sentence describes.

`src/trading/scrumming_bot.py` — the midline block, as it stands

```
if self.config.bb_midline_gate:
    scrum_ok = (bb_pos > 0.50) and not self._phantom_locked
    fold_ok_midline = (bb_pos < 0.50) and not self._phantom_locked
else:
    scrum_ok = not self._phantom_locked
    fold_ok_midline = not self._phantom_locked
```

The lock the fold half reads, who writes it and the swept figures are on Part 7, Indicators.

Read Rate - To be re-evaluated.

From 1 to 60 minutes, at 5 minutes to start. It sets the search-mode read rate; track mode reads ten times faster. Fire mode reads at that same faster rate.

The wizard writes it as `scrum_read_rate_min`, bot creation passes it through, and the bot reads it at the top of every tick. The range starts at 1, so this control cannot switch the throttle off.

`src/trading/scrumming_bot.py` — `ScrummingBot.tick`, the throttle

```
if self.config.scrum_read_rate_min > 0 and not self._manual_fire_pending:
    _tick_sec = max(self.tick_interval, 0.1)
    _base_skip = max(1, int((self.config.scrum_read_rate_min * 60) / _tick_sec))
    self._tick_skip_search = _base_skip
    if self._scrum_target_mode in ("track", "fire"):
        self._tick_skip = max(1, _base_skip // 10)
    else:
        self._tick_skip = _base_skip
```

The tick itself runs every five seconds, so the figure in minutes becomes a count of skipped ticks. Driven over eighty ticks: 1 minute skipped 12 and worked 6, 5 minutes skipped 60 and worked 1, and 60 minutes skipped 720 and worked none. A skipped tick fetches no price and runs no gate.

`src/trading/scrumming_bot.py` — the tick period the count divides

```
@property
def tick_interval(self) -> float:
    return 5.0
```

Band Travel - Previously described. To be re-evaluated.

A whole percentage from 0 to 100, at 70 % to start. Zero switches it off. The wizard writes it as `band_travel_pct` and bot creation passes it through.

The engine reads it once per evaluation. It measures how far price has moved since the last trade, as a share of the current band width, and raises a second trigger once price covers that share. The same trigger overrides a trend hold, so Band Travel can release a trade the trend gates were holding.

`src/trading/scrumming_bot.py` — the band-travel trigger

```
_bb_width = max(bb_result.upper - bb_result.lower, 1e-12)
band_travel_frac = abs(ticker.last - self._last_trade_price) / _bb_width
if (
    abs(ticker.last - self._last_trade_price)
    >= self.config.band_travel_pct / 100.0 * _bb_width
    and delta > 0
):
    band_travel_triggered = True
```

Driven on one rising tape where 95 % of the last twenty candles closed up, with price 37 % of the band width above the last trade: 0 left the trend hold in place and suppressed the scrum, 30 raised the trigger and the override released it, and 70 left the hold in place again, because 37 is under 70. The figure decides, not an on and an off.

`src/trading/scrumming_bot.py` — the override the trigger feeds

```
trend_override = abs(delta) >= _interval_usd * 2.0 or band_travel_triggered
if trend_hold and trend_override:
```

BB Bullseye Check - If current price and Bollinger Band thresholds are equal, the user can opt to perform a double-sized trade.

A checkbox, on at the start.

`src/gui/bot_wizard.py` — the BB Bullseye row

```
self._bb_bullseye = QCheckBox("BB Bullseye Check")
self._bb_bullseye.setChecked(True)
```

A double size is not in the engine. Nothing sizes a trade off this box. Its one decision reader counts a band touch within 0.5 %, or a candle wick within 0.2 %, as band proximity, and proximity with the delta at or over the interval arms the BB priority skew. Driven on one tape with price at the upper band, on emitted the skew and took the confidence floor from 0.25 to 0.1923; off emitted nothing and left the floor where it was.

`src/trading/scrumming_bot.py` — the band touch the box admits

```
_bb_proximity_upper = (
    (bb_pos >= _bb_upper_dt_pre) or _be_upper or _be_upper_wick
)
```

It does not bypass the Fire Threshold. Driven with price 0.3 % under the upper band and the Fire Threshold at its lowest 0.10 %, the ramp stayed short of FIRE with the box on and with it off, and the scrum carried the same blocker both ways. What changes is the floor the TA confidence must clear.

`src/trading/scrumming_bot.py` — the floor the skew divides

```
_ta_conf_skew = position_boost + bb_confidence_boost
if _bb_priority_arm:
    _ta_conf_skew += _BB_PRIORITY_SKEW
_eff_conf_floor = _skewed_confidence_floor(_ta_conf_skew)
```

The second reader writes the two BULLSEYE notices onto the Activity Log and changes no decision.

`src/trading/scrumming/tick_phases.py` — the notice reader

```
if bullseye_upper or bullseye_upper_wick:
    _ramp = self._scrum_target_mode.upper()
    _fire_gate = "passes" if _ramp == "FIRE" else "holds"
```

Current reading, how far the skew takes the floor. The 0.1923 above holds only while two indicators reach less. The floor now stays above what the two heaviest voters reach at full confidence, at the bot's weights on that tick, and never rises above 0.25. Driven with the skew alone, the floor went to 0.1923 at equal weights, just above 0.1966 at the published weights, and stayed at 0.25 where two indicators reach 0.3362.

`src/trading/scrumming_bot.py` — `ScrummingBot.tick`

```
_eff_conf_floor = _skewed_confidence_floor(_ta_conf_skew, summary.signals)
```

Current reading, the floor over 0.25. The floor no longer stops at 0.25. With the five lightest published weights abstaining, two indicators reach 0.3151. The floor read 0.25 there before this change, with or without a favour. It now reads just above 0.3151. Driven on a stand-in exchange with four of twelve equal weights abstaining, two agreeing indicators read 0.2500 and the fold waited. The code before this change, taken from git, folded. Three read 0.3750 and folded on both.

`src/trading/scrumming_bot.py` — `_skewed_confidence_floor`, the last two lines

```
eased = floor if math.isnan(denominator) else floor / denominator
return three_indicator_floor(eased, signals)
```

Wire Inflow Stack - To be re-evaluated.

A percentage from 0.00 to 100.00, at 1.00 % to start. The wizard writes it as `wire_inflow_stack_pct`, and one method in the engine reads it. Above zero, wire income arriving while the position sits within that band of its target and within that band of its entry price goes onto the target and queues an aggressive buy, rather than spreading over the fold queue.

Bot creation passes it, and so does a restart. One declaration serves both paths, so every field the config declares reaches a new bot and a restored one. Bot Settings can set it on a bot that is already running.

`src/trading/scrumming/wire_routing.py` — `WireRoutingMixin.apply_wire_income`

```
try:
    stack_pct = float(getattr(self.config, "wire_inflow_stack_pct", 1.0) or 0)
except (TypeError, ValueError):
    stack_pct = 0.0
_stack_eligible = False
_stack_reason = ""
_target = 0.0
_entry_px = 0.0
if stack_pct > 0:
```

`src/trading/container/config.py` — the one carried set

```
carried = {f.name for f in fields(BotConfig)} - foreign - {"mode"}
kwargs = {
    key: value
    for key, value in _sanitize_deprecated_kwargs(collected).items()
    if key in carried
}
```

The caller is live. A fold routes its compounded profit through Smart Wire, which hands the share to the target bot, and that is where this figure decides. Driven on one bot sitting at its target and at its entry price, 0.00 parked ten dollars as pending and left the target at 200.00, while 1.00 stacked it, took the target to 210.00 and queued ten dollars for the next tick.

`src/trading/scrumming/tick_phases.py` — the live caller

```
mgr.distribute_fold_profit(
    source_id=self.bot_id,
    profit_usd=float(_growth_applied),
    ref=f"fold-compound@{buy_fill:.8f}",
)
```

The row is removed. Your ruling on Wire Credits retires the percentage: a share that sizes a stack is grid behaviour, and the requirement is that an arriving credit reaches an active tranche. No screen offers the row now, and no figure is read in its place. A credit takes one of two routes and no other.

`src/trading/scrumming/wire_routing.py` — what is left of `apply_wire_income`

```
if self._fold_tranches:
    share = self._spread_wire_usd_over_fold_queue(u, [credit])
    ...
# Case 2 — no tranches yet; park in pending
self._pending_wire_credits += u
```

Driven on the same bot, at its target and at its entry price, with a ten-dollar credit. The tranche figure is the one that matters: the stack took the credit away from an open tranche, and it no longer does.

```
the bot holds one $5.00 tranche    before      after
what happened to the credit        stacked     added to the tranche
the tranche                        $5.00       $15.00
Target Balance                     $210.00     $200.00
queued for an aggressive buy       $10.00      $0.00

the bot holds no tranche           before      after
what happened to the credit        stacked     parked
the parked pool                    $0.00       $10.00
Target Balance                     $210.00     $200.00
```

A bot saved with the figure still loads. The name joins the retirement record in `src/trading/container/config.py`, which drops it before the bot is built.

```
"personal_hold_qty",
"wire_inflow_stack_pct",
```

Hedge Rebalance Active - Determines if Current Price drifting below Initial Entry Price will have a limited amount of funds that can be used to keep re-zeroing the Target Delta at key bearish thresholds or areas of possible reversal.

A checkbox, on at the start. It opens the one group on the page that holds a reserve outside Target Balance.

`src/gui/bot_wizard.py` — the Hedge Rebalance row

```
self._hedge_rebalance = QCheckBox("Hedge Rebalance Active")
self._hedge_rebalance.setChecked(True)
```

Hedge Balance - Sets a limit on the amount of additional liquidity a given bot is allowed to absorb when Current Price drifts below Initial Entry Price.

At $200.00 to start, a reserve the bot holds apart from Target Balance. It is the figure a hedge buy is checked against and the ceiling a refill stops at, and the bot reads it off its configuration rather than keeping a second copy, so a restart cannot leave the two disagreeing. One hedge buy spends half of what the reserve holds, bounded by the gap to target.

`src/trading/scrumming/tick_phases.py` — the one ceiling

```
@property
def _hedge_balance_initial(self) -> float:
    if not self.config.hedge_rebalance_active:
        return 0.0
    return float(self.config.hedge_balance)
```

`src/trading/scrumming_bot.py` — the figure that sizes one hedge buy

```
_use = min(self._hedge_bal * 0.5, _gap)
```

The switch arms the reserve. Ticking Hedge Rebalance Active on a running bot raises the reserve to the Hedge Balance figure, and it never lowers a reserve the bot already holds, so the box cannot destroy one. Unticking keeps the reserve and freezes it, because the arm test and the refill test both read the switch first. A word instead of a tick is refused and the switch does not move.

Diagram: flowchart, left to right

- Hedge Rebalance Active -> **Apply**
- Apply -> **set_hedge_rebalance_active_live**
- set_hedge_rebalance_active_live -> **the reserve a hedge buy reads**
- Hedge Balance -> **Apply**
- Apply -> **set_hedge_balance_live**
- set_hedge_balance_live -> **the ceiling a refill stops at**

The reserve itself stays on the saved record, because it is a balance and not a setting, and it refills from compound growth at eight hundredths of each fold's profit.

A Hedge Balance of zero says what it is. Zero is a figure the box accepts and it is not an off switch. It is an empty reserve that never refills, and a reserve the bot already holds stays spendable until it drains. Both tooltips say so, on both builds, and the Activity Log says so as the figure is typed.

```
HEDGE BALANCE A11 LIVE UPDATE: $200.00 -> $0.00. A $0.00 Hedge Balance is an
empty reserve that never refills, not an off switch. Untick Hedge Rebalance
Active to turn the hedge off. Reserve $200.00 stays spendable until it drains.
```

The running bot's row is routed through a method rather than written straight onto the config, and that method refuses a figure it cannot use.

```
set $500.00  applied   cap $500.00  reserve unchanged
set   $0.00  applied   cap   $0.00  reserve unchanged
set  -$5.00  refused   hedge_balance must be >= 0
set   "abc"  refused   hedge_balance must be numeric
set  $75.00  applied   cap  $75.00  reserve unchanged
```

A cap of zero shuts the refill gate for good while leaving the old reserve drainable, and raising the cap above zero re-opens it, so the state is recoverable by the same control that caused it.

`src/gui/bot_wizard.py` — `TradingParamsPage.get_config`, the settings these two groups emit, in the order of the rows above

```
"scrum_detect_pct": self._scrum_detect_pct.value(),
"scrum_fire_pct": self._scrum_fire_pct.value(),
"bb_midline_gate": self._bb_midline_gate.isChecked(),
"scrum_read_rate_min": self._scrum_read_rate.value(),
"band_travel_pct": self._band_travel_pct.value(),
"bb_bullseye_check": self._bb_bullseye.isChecked(),
"wire_inflow_stack_pct": self._wire_inflow_stack_pct.value(),
"hedge_rebalance_active": self._hedge_rebalance.isChecked(),
"hedge_balance": self._hedge_amount.value(),
```

The group title on screen carries an internal release identifier after the words Advanced Scrumming. Issue #420 carries that, and four more group titles with it.

The issue #420 clause above is overtaken. It is not edited and not removed. It read:

> *The group title on screen carries an internal release identifier after the words Advanced Scrumming. Issue #420 carries that, and four more group titles with it.*

Issue #420 closed on 2026-10-02. Every group title the wizard draws is plain words. Read off the eleven titles one declaration holds, none carries a release number or an internal note number.

```
src/gui/main_tabs/bot_wizard_surface.py, in GROUP_TITLES

mode_group        Trading Parameters       scrum_group       Scrumming Settings
adv_group         Advanced Scrumming       hedge_group       Hedge Rebalance
cb_group          Circuit Breakers         risk_group        Risk Controls
gates_group       Strategy Gate Flags      whole_unit_group  Whole-Unit Market
expiry_group      Contract Expiry          lock_group        Higher-TF Lock Duration
extractor_group   Extractor — Pool & Artillery
```

What the fleet runs. Read Rate is the row that differs most from the page. The box opens at five minutes, and 30 of the 38 live bots run it at one minute, with eight at five. At one minute the search throttle skips twelve ticks and works one; once the bot reaches track or fire it reads ten times faster, which is every tick. That is the difference between a bot that notices a move inside a five-minute candle and one that waits out the candle first.

Hedge Rebalance Active opens ticked and is off on 32 of the 38, on for six. With it off the bot holds no reserve outside Target Balance, so the Hedge Balance row below has nothing to size and the hedge path never runs. A reader who wants the hedge must tick the switch and give the reserve a figure; one without the other does nothing.

The rest of the group matches the page, and that uniformity is the suggestion. Detect Threshold at 75 puts the two marks at 0.125 and 0.875 of the band, Fire Threshold is the page's own 0.5 % on 37 bots and tighter on one, and Band Travel is 70 on 34 bots and 75 on four. Nothing in the stored record separates the smaller groups from the larger ones. The whole fleet sits on one venue, and each split is a per-bot choice rather than a venue or a class difference.

Read the three band rows together, because they gate in sequence. Detect Threshold decides when the bot starts looking, Fire Threshold decides when it may shoot, and BB Midline Gate refuses a sell below the midline and a buy above it whatever the other two say. Band Travel sits outside that chain and can release a trade the trend gates were holding, so it is the one row that loosens rather than tightens.

The sentence above about Band Travel loosening only is overtaken. It is not edited and not removed. It read:

> *Band Travel sits outside that chain and can release a trade the trend gates were holding, so it is the one row that loosens rather than tightens.*

The override it names is unchanged. The row now tightens as well. `src/trading/otd_math.py`, in `band_extension_pct`, makes Band Travel the third term of the placement distance, and `src/trading/scrumming_bot.py`, in `tick`, hands that reader the band the tick already computed. The Fold-Tranche panel in `src/gui/live_settings/fold_tranches_tab.py` hands it the same band, so the

gate and the guide answer one distance.

```
_otd_pct_for_gate = minimum_opposing_trade_distance_pct_from_config(
    self.config,
    band_upper=float(getattr(bb_result, "upper", 0.0) or 0.0),
    band_lower=float(getattr(bb_result, "lower", 0.0) or 0.0),
    reference_price=float(getattr(bb_result, "middle", 0.0) or 0.0),
)
```

Current reading. The tick reads the three settings and the venue fee once, through `opposing_distance_settings`, and takes the fold factor from that reading at the tick's own band.

`src/trading/scrumming_bot.py` — `ScrummingBot.tick`

```
_otd_pct_for_gate = _distance.distance_pct(
    float(getattr(bb_result, "upper", 0.0) or 0.0),
    float(getattr(bb_result, "lower", 0.0) or 0.0),
    float(getattr(bb_result, "middle", 0.0) or 0.0),
)
_otd_factor = fold_rebuy_factor_from_pct(_otd_pct_for_gate)
```

```
setting               the page opens at   the fleet runs
Detect Threshold      75 %                75 on 38
Fire Threshold        0.50 %              0.5 on 37, 0.1 on 1
BB Midline Gate       ticked              on on 38
Read Rate             5 min               1 on 30, 5 on 8
Band Travel           70 %                70 on 34, 75 on 4
BB Bullseye Check     ticked              on on 38
Wire Inflow Stack     1.00 %              1.0 on 38
Hedge Rebalance Active ticked             off on 32, on on 6
```

Wire Inflow Stack leaves that list. All 38 ran the opening figure, which is why nothing in the stored record separates one bot from another on it. The row is gone from all six screens and no figure replaces it.



The Circuit Breakers group.

Now we arrive at some safety controls. Circuit Breakers are designed to fully inhibit trade actions for a given period should an extreme volatility (pump and dump) event occur. Soft Circuit Breakers have a candle-count based timer whereas Hard Circuit Breakers require the user to clear the bot to continue trading.

Soft CB Threshold - The amount of instantaneous, single-candle price action required for the bot to pause trading for a number of candles denoted by Soft CB Cooldown.

A percentage from 0.0 to 100.0, at 25.0 % to start. Zero switches it off. It interrupts only the side of the market that moved, so an upward candle stops scrums and a downward one stops folds.

`src/gui/bot_wizard.py` — the Soft CB Threshold row

```
self._cb_soft_pct.setSuffix(" %")
self._cb_soft_pct.setValue(25.0)
```

The move the figure is compared against is the candle's own span, high minus low over open, as a percentage. The figure decides: on one candle spanning 24.9 %, a stored 10.0 % tripped the breaker and a stored 30.0 % did not, and on one spanning exactly 25.0 % a stored 25.0 % tripped. A stored 0.0 % left a 50 % candle alone, which is the off position the paragraph above describes.

The side follows the close against the open. A candle closing up shut the SCRUM side, a candle closing down shut the FOLD side, and a shut side refuses the trade at `circuit_breaker_scrum` or `circuit_breaker_fold` in the gate chain while the other side's chain is untouched. The breaker is read once per worked tick, and only a candle timestamp the bot has not seen can trip it, so one candle never trips twice.

Hard CB Threshold - The amount of instantaneous, single-candle price action required for the bot to be hard stopped at which point the user must re-authorize trading.

The same range, at 35.0 % to start. Zero switches it off. Where the soft breaker interrupts one side, this one pauses the bot outright and the pause survives a restart.

`src/gui/bot_wizard.py` — the Hard CB Threshold row

```
self._cb_hard_pct = QDoubleSpinBox()
self._cb_hard_pct.setRange(0.0, 100.0)
self._cb_hard_pct.setDecimals(1)
self._cb_hard_pct.setSuffix(" %")
self._cb_hard_pct.setValue(35.0)
```

Driven on the same span reading: at a stored 35.0 % a candle spanning 35.0 % tripped the hard breaker and paused the bot, one spanning 34.9 % did not, and a stored 40.0 % left the 35.0 % candle alone. A stored 0.0 % left a 90 % candle alone. Once tripped it holds every later candle, quiet ones included, and only the Reset All Breakers button clears it — the reset reports the trip percentage it cleared and returns the bot from paused to running. The pause is written into the bot's stored record, so a bot restored from a hard trip comes back paused.

The pause reaches the scrum and the fold gate chain. It does not reach the five trade paths the tick reads before it: Manual Fire, the Wire Stack fire, Max Cartridge, Detonation and the initial entry. Measured on a hard-tripped, paused bot with a position $30.00 past a $20.00 Max Cartridge threshold: the bot still wrote MAX

CARTRIDGE FIRE and still sent the cartridge trade notification, while the same bot with Max Cartridge at 0.0 % wrote nothing. Nothing is changed here for that, because raising the hard breaker above the cartridge would stop a rebalance that fires on live bots today.

Now: the Wire Stack fire is gone, and a pending stack that a saved state carries joins the wire-credit pool on restore (`src/trading/scrumming/state_io.py, in import_scrumming_state`).

Soft CB Cooldown - This is the number of candles that must close before the Soft Circuit Breaker opens again.

From 1 to 100 candles, at 3 to start. It is counted in closed candles, so its real length follows the bot's own timeframe.

`src/gui/bot_wizard.py` — the Soft CB Cooldown row

```
self._cb_cooldown = QSpinBox()
self._cb_cooldown.setRange(1, 100)
self._cb_cooldown.setValue(3)
```

The figure is the count of fresh candles the shut side waits out, and it is the count the code uses: a stored 1 re-opened on the first fresh candle, a stored 3 on the third, and a stored 10 on the tenth. The count only moves on a candle timestamp the bot has not seen, so a repeated candle left the remaining count where it was. A record carrying a cooldown of 0 is read as 3, the declared default. A record carrying a word stops the breaker check for that tick with a warning, which leaves the soft breaker unable to trip.

A stored word now leaves both breakers armed

The sentence above describes what the reader did before this unit. Every breaker figure is now read through one declaration, `breaker_figures`, and a stored value that is not a number reads the declared figure instead of stopping the check.

Driven on one candle spanning 30.0 % of its open with the soft threshold at 25.0 %:

```
stored cooldown   before                       after
a word            no decision, breaker idle   3 candles, SCRUM side shut
7                 7 candles, SCRUM side shut  7 candles, SCRUM side shut
empty             3 candles, SCRUM side shut  3 candles, SCRUM side shut
missing           3 candles, SCRUM side shut  3 candles, SCRUM side shut
```

The three readable rows do not move, which is what separates this from a change of behaviour. The Console names the bad value ahead of the trip line:

```
CIRCUIT BREAKER FIGURE UNREADABLE: circuit_breaker_cooldown_candles='three'.
The declared figures are used — soft 25.00%, hard 35.00%, cooldown 3 candles —
and both breakers stay armed.
```

That line repeats only when the set of unreadable fields changes, so one bad record writes it once rather than on every tick. The same declaration reads the two thresholds, so a word stored in Soft CB Threshold or Hard CB Threshold no longer stops the check either. A sweep of 63,168 candle and figure combinations reads the same decision, the same side, the same cooldown and the same Console wording before and after on every numeric figure.

While a cooldown holds, the shut side is refused at the breaker gate and the bot keeps running — the cooldown pauses nothing. It ends on its own when the count reaches zero, with a SOFT CIRCUIT BREAKER RESET line naming the side that re-opens, and the Reset All Breakers button ends it early.

Max Cartridge Size - This is the maximum amount of deviation allowed for the Target Delta. At this threshold the bot is actively and aggressively looking for a trade opportunity.

Now: a cartridge buy pays back only the tranches whose own sale price, less the distance, sits at or above the fill, and it spends none of the dollars the unpaid tranches, the wire credits or the pool hold (`src/trading/scrumming/execution.py, in _execute_manual_rebalance`).

A percentage from 0.0 to 200.0, at 10.0 % to start. Crossing it fires an aggressive rebalance that bypasses the detection, hysteresis and soft-breaker checks. It is the one figure on the page whose range runs past 100.

`src/gui/bot_wizard.py` — the Max Cartridge Size row

```
self._max_cartridge_pct = QDoubleSpinBox()
self._max_cartridge_pct.setRange(0.0, 200.0)
self._max_cartridge_pct.setDecimals(1)
self._max_cartridge_pct.setSuffix(" %")
self._max_cartridge_pct.setValue(10.0)
```

The threshold is Target Balance times the figure over 100, and the fire is on absolute Target Delta. Driven on a $200.00 target: at 10.0 % the threshold was $20.00, a position $20.00 past target fired and $19.99 past target did not; at 5.0 % the threshold was $10.00, $11.00 fired and $9.99 did not; at 0.0 % a position $100.00 past target fired nothing, which is the off position.

A correction to the sentence above it: the hysteresis check is not bypassed. The tick tests the opposing-trade pivot before the fire and writes MAX CARTRIDGE BLOCKED instead, naming the price the pivot still needs. What the fire does bypass is the band detection, the higher-timeframe bias and the soft breaker — and, because the tick reads this figure before it reads either breaker, the hard breaker as well.

Smart Cartridge - This allows the Max Cartridge Size to organically resize in response to current price range as defined by the current-candle Bollinger Band reading.

One checkbox labelled Calibrate to BB range, off at the start. While it is on, the cartridge size comes from the current band range instead of the fixed percentage above it.

`src/gui/bot_wizard.py` — the Smart Cartridge row

```
self._cartridge_smart_chk = QCheckBox("Calibrate to BB range")
self._cartridge_smart_chk.setChecked(False)
```

The band range is upper minus lower over the midline, as a percentage, and the box swaps it in for the fixed figure. It is held between two bounds: never below the Opposing Trade Interval, never above Smart Ceiling. Driven on a $200.00 target with the fixed figure at 10.0 %: off, the threshold stayed $20.00; on, with a band spanning 20.0 % of its midline, the threshold became $40.00. On the same tick the bot read a band spanning 80.0 %, the clamp gave 30.0 % and the threshold $60.00, and a position $30.00 past target that fired with the box off fired nothing with it on.

The band the box reads is the one the previous worked tick stored, so the first worked tick after a start has no band and keeps the fixed figure.

The band is now the one this tick read

The sentence above describes what the tick did before this unit. The band is written by `_refresh_market_reading`, which the tick calls above every early return, so the figure the cartridge reads belongs to the candle the tick has just fetched. What the band is has not changed: the same detection runs over the same candles with the same two settings.

Driven from a cold start on a $200.00 target with the fixed figure at 10.0 % and a band spanning 7.48 % of its midline:

```
tick   before             after
one    10.00 %, $20.00    7.48 %, $14.97
two     7.48 %, $14.97    7.48 %, $14.97
```

The first worked tick after a start used the fixed figure and now uses the band of that tick. The second tick does not move. The Console names the figure on the first tick now instead of on the second:

```
SMART CARTRIDGE calibrated to 7.48% (BB range 7.48%, floor=1.00%,
ceiling=30.00%). Effective threshold = $14.97.
```

Smart Ceiling - To be re-evaluated.

A percentage from 1.0 to 100.0, at 30.0 % to start. It caps the cartridge threshold while Smart Cartridge is on.

It is the upper of the two bounds on the band range, and it binds only when the band is wider than it. Driven on a $200.00 target with Smart Cartridge on: a band spanning 120.0 % under a 30.0 % ceiling gave 30.0 % and a $60.00 threshold, the same band under a 15.0 % ceiling gave 15.0 % and $30.00, and a band spanning 29.0 % under the 30.0 % ceiling gave 29.0 % and $58.00 — the ceiling did not bind. A band spanning 0.5 % gave 1.0 %, the Opposing Trade Interval, because the lower bound binds there instead.

The control cannot emit a zero. A record hand-edited to carry one is read as 30.0 %, the declared default, not as no ceiling.

A stored zero now means no ceiling

The two sentences above describe what the reader did before this unit. A stored zero is now read as off, and the band range passes through held only by the Opposing Trade Interval below it. The box still cannot emit a zero, so only a hand-edited record reaches this reading.

Driven on a $200.00 target with Smart Cartridge on, a band spanning 100.0 % of its midline and the interval at 1.0 %:

```
stored ceiling   answer      threshold
0.0              100.00 %    $200.00
missing          30.00 %      $60.00
-5.0              1.00 %       $2.00
15.0             15.00 %      $30.00
30.0             30.00 %      $60.00
```

A missing field and a negative figure read exactly as they read before. The Console names the reading in words rather than printing a zero per cent:

```
SMART CARTRIDGE calibrated to 100.00% (BB range 100.00%, floor=1.00%,
ceiling=off). Effective threshold = $200.00.
```

The opening figure is declared once, in `src/trading/container/config.py`, and `smart_cartridge_ceiling_pct` in that module is the one reader.

```
MAX_CARTRIDGE_SMART_CEILING_PCT_DEFAULT: float = 30.0
SMART_CARTRIDGE_CEILING_OFF: float = 0.0
SMART_CARTRIDGE_CEILING_OFF_TEXT: str = "off"
```

`src/gui/bot_wizard.py` — `TradingParamsPage.get_config`, the settings this group emits, in the order of the rows above

```
"circuit_breaker_soft_pct": self._cb_soft_pct.value(),
"circuit_breaker_hard_pct": self._cb_hard_pct.value(),
"circuit_breaker_cooldown_candles": self._cb_cooldown.value(),
"max_cartridge_size_pct": self._max_cartridge_pct.value(),
"max_cartridge_smart": self._cartridge_smart_chk.isChecked(),
"max_cartridge_smart_ceiling_pct": self._cartridge_smart_ceiling.value(),
```

A trip stops a trade at one gate on each side of the chain, and 07-indicators.md lists both chains in full.

`src/trading/gate_chain.py` — the two breaker gates in the built chains

```
CircuitBreakerGate(side="scrum"),
```

`src/trading/gate_chain.py` — and the fold chain

```
CircuitBreakerGate(side="fold"),
```

What the fleet runs. Two rows in this group are set away from what the page opens at, and together they change what the Max Cartridge row means. Max Cartridge Size runs at 5.0 % against an opening 10.0 %, and Smart Cartridge is ticked on every bot although the box opens clear.

With Smart Cartridge on, the threshold comes from the live band range rather than from the fixed percentage, held between the Opposing Trade Interval below it and Smart Ceiling above it. On the fleet those two bounds are 5 % and 30 % of target, so the cartridge fires somewhere in that span and follows the market's own width. The fixed 5.0 % is then used only on the first worked tick after a start, before the bot has read a band. A reader who leaves Smart Cartridge clear gets the fixed figure on every tick instead, which is a fixed dollar trigger in a market whose width moves.

The three breaker rows match the page. Read the cooldown against the bot's own chart rather than against the clock: the fleet runs five-minute candles, so three candles is fifteen minutes with one side shut, and the same 3 on an hourly bot would be three hours.

```
setting              the page opens at   the fleet runs
Soft CB Threshold    25.0 %              25.0 on 38
Hard CB Threshold    35.0 %              35.0 on 38
Soft CB Cooldown     3 candles           3 on 38, fifteen minutes at 5m
Max Cartridge Size   10.0 %              5.0 on 38
Smart Cartridge      clear               ticked on 38
Smart Ceiling        30.0 %              30.0 on 38
```



The Risk Controls and Strategy Gate Flags groups.

After Circuit Breakers, which help defend against extreme volatility, we come to Risk Controls. These are designed to cap the amount of profit or growth a given bot can earn before performing a full position exit.

Enable Position Ceiling - Enables a growth cap for a given position.

A checkbox, off at the start. Nothing in the Risk Controls group acts until it is on.

`src/gui/bot_wizard.py` — the Position Ceiling row

```
self._position_ceiling_enabled = QCheckBox("Enable Position Ceiling")
self._position_ceiling_enabled.setChecked(False)
```

Functional. The box gates six places in the engine. Five refuse or shrink a buy: the ceiling figure itself, the pre-buy check, the buy executor, the first entry of a brand-new bot, and the fold hold. The sixth runs on restore and pulls a stored target back down to the ceiling. The box is read at the moment of each decision and never copied aside, so unticking it releases the brake on the very next read. Driven on a two hundred dollar anchor at three times: with the box off, a fold rebuy of fifty dollars onto a seven hundred dollar position passed; with it on, the same buy was refused; untick, and it passed again.

`src/trading/scrumming_bot.py` — `ScrummingBot.position_ceiling_usd`

```
if not getattr(self.config, "position_ceiling_enabled", False):
    return None
```

Where the code departs. Two of the three Detonation rows below do not wait for this box. The detonation check reads its own box and the anchor, and a search of the engine for a detonation site that reads the ceiling box returns nothing. Driven: with the ceiling box off and Detonation on, the hourly check ran and asked the venue for candles. The sentence above holds for the ceiling and the fold taper, and Detonation is the exception.

`src/trading/scrumming_bot.py` — `ScrummingBot._check_detonation_trigger`, its first two conditions

```
if not getattr(self.config, "detonation_enabled", False):
    return False
...
if current_value <= self._anchor_target_balance:
    return False
```

Ceiling Multiple - This setting caps the maximum amount of growth a position at a multiple of the Target Balance (anchor) and, once reached (and under higher timeframe bullish conditions with Detonation enabled) will allow the entire position to be sold and the corresponding bot will pause all further operations. Without Detonation enabled, this becomes a user notification.

From 1.0 to 10.0, at 5.0x anchor to start. The anchor is the target balance the bot was created with, not the balance it has grown to. It steps half a multiple at a time and carries its unit in the box.

`src/gui/bot_wizard.py` — the Ceiling Multiple row

```
self._position_ceiling_multiple = QDoubleSpinBox()
self._position_ceiling_multiple.setRange(1.0, 10.0)
self._position_ceiling_multiple.setDecimals(1)
self._position_ceiling_multiple.setSingleStep(0.5)
self._position_ceiling_multiple.setSuffix("x anchor")
self._position_ceiling_multiple.setValue(5.0)
```

Functional. The figure becomes a dollar ceiling, the anchor times the multiple. On a two hundred dollar anchor the four corners of the range read: one gives two hundred dollars, three gives six hundred, five gives one thousand, ten gives two thousand. The refusal is strictly above, so a projected position of exactly six hundred dollars is allowed against a six hundred dollar ceiling and six hundred dollars and one cent is refused. The same seven hundred and fifty dollar projection is refused at three and allowed at six, which is the figure changing the answer.

`src/trading/scrumming_bot.py` — `ScrummingBot._pre_buy_allowed`, layer two

```
_smart_mult = max(1.0, min(10.0, _smart_mult))
_smart_ceiling_usd = _anchor * _smart_mult
if _projected > _smart_ceiling_usd:
```

The clamp and the multiplication above now sit in one function that the pre-buy check, the ceiling property and the tick's fold evaluation all call, so the three ceilings the bot reads are one number.

`src/trading/scrumming/sizing.py` — `position_ceiling`

```
def position_ceiling(anchor_target_balance: float, multiple: float) -> float:
    mult = max(CEILING_MULTIPLE_MIN, min(CEILING_MULTIPLE_MAX, multiple))
    return anchor_target_balance * mult
```

Functional. While the ceiling binds, the bot still sells and stops buying. The fold side shrinks first and then stops: the fold's dollar size is multiplied by a taper that is full below half the ceiling, falls in a straight line to a tenth across the upper half, and reaches zero at the ceiling. Measured against a six hundred dollar ceiling: two hundred and ninety dollars held full size, five hundred and ninety dollars cut the fold to thirteen per cent, and six hundred dollars stopped it. Three things release the brake — price falling back under the ceiling, a detonation resetting the target to the anchor, and unticking the box.

`src/trading/scrumming/tick_phases.py` — `_tick_execute_fold` spends the taper

```
_taper = self.fold_rate_taper
...
buy_cost = _fusd * _taper
```

The multiplication is now `fold_spend_usd`, and the taper schedule is `fold_rate_taper`, both in the shared sizing module; the executor's line reads `buy_cost = fold_spend_usd(_fusd, _taper)` and the Simulator's fold reads the same two functions over its own tranches.

`src/trading/scrumming/sizing.py` — the taper schedule

```
def fold_rate_taper(ratio: float) -> float:
    if ratio >= 1.0:
        return 0.0
    if ratio < TAPER_START_RATIO:
        return 1.0
    return 1.0 - (ratio - TAPER_START_RATIO) / TAPER_START_RATIO * TAPER_DROP
```

Where the code departs. The sentence above says the bot pauses all further operations once the ceiling is reached and the position is sold. The executor does not pause it. It resets the target balance to the anchor, clears the fold queue and the tranches, reseeds the lots at the fill price, and logs that the bot will re-accumulate from scratch on the next dip. A search of the detonation executor for a pause call returns nothing. The second half of the sentence does hold: with Detonation off, the ceiling only brakes, and the operator is told through the Console line and through the Fire button's own tooltip on the Bot Swarm row.

`src/trading/scrumming/execution.py` — `_execute_detonation`, what it resets

```
prior_target = self._target_balance
self._target_balance = self._anchor_target_balance
...
self._fold_tranches.clear()
self._fold_queue_usd = 0.0
```

Functional. One name serves two unrelated mechanisms, and the engine's own wording has been corrected in this unit. The percentage box in Circuit Breakers labelled Smart Ceiling caps the cartridge threshold, and the

engine reports it inside its cartridge line as a percentage. This dollar ceiling is what every ceiling refusal in the engine names, and those lines now read Position Ceiling and Ceiling Multiple, which are the labels on this screen. A reader can now tell the two apart: one is a percentage under a cartridge heading, the other is a dollar figure under the name of the control that set it.

`src/trading/scrumming/tick_phases.py` — the cartridge line, for contrast

```
  f"SMART CARTRIDGE calibrated to "
  f"{_smart_pct:.2f}% "
  f"(BB range {_bb_range_pct:.2f}%, "
  f"floor={_interval_floor:.2f}%, "
  f"ceiling={_smart_ceiling:.2f}%). "
```

The block above is the line as it read before this unit. The ceiling now prints through a name, so a stored zero reads as a word instead of as nought per cent.

```
  f"floor={_interval_floor:.2f}%, "
  f"ceiling={_ceiling_text}). "
```

One declaration holds the Ceiling Multiple opening figure

Four readers in the engine each wrote their own figure to fall back on when a bot's record carries no Ceiling Multiple, and all four named one anchor multiple against the 5.0 this page states. On a $200.00 anchor that is a $200.00 ceiling in place of a $1,000.00 one, so a restored target of $700.00 was snapped to $200.00 and a buy was refused at a fifth of the figure you set. No real record reaches those figures, because every bot's configuration declares the field.

All four now read one declaration. `src/trading/container/config.py` holds the figure and `ceiling_multiple` in that module answers for it.

```
  POSITION_CEILING_MULTIPLE_DEFAULT: float = 5.0
```

Driven on a record carrying no Ceiling Multiple, with a stored target of $700.00 against a $200.00 anchor: the restore left the target at $700.00, where before it cut it to $200.00. A record that does carry the figure restored to $700.00 both times, and the same restore with Enable Position Ceiling clear left $700.00 in every reading.

The two readers inside `src/trading/scrumming_bot.py` read the same declaration, so the pre-buy refusal, the first entry of a brand-new bot, the fold hold and the restore all answer one figure.

One declaration holds the Ceiling Multiple range

Eleven places held the one-to-ten span. Two engine readers wrote the pair out by hand, and all six Ceiling Multiple boxes typed it into their own control. Nothing reported a disagreement, so a change in one place would have left the other ten behind.

Eight of those copies now read one declaration, in `src/trading/container/config.py`.

```
  CEILING_MULTIPLE_MIN: float = 1.0
  CEILING_MULTIPLE_MAX: float = 10.0
```

The two Electron rows still carry their own pair. They sit in a directory another change holds open, and they are named rather than touched.

Driven on a $100.00 anchor, every reading is what it was. A stored multiple of twenty still caps at ten, a restored target of $1,500.00 still snaps to $1,000.00, and all six boxes still open at 5.0x and span 1.0 to 10.0. Moving the declaration to a span of three to seventy-seven moved every one of those readings together. The same move against the old code moved only the sizing function that reads the declaration and left all eight copies where they were.

Enable Detonation - Enables an entire remaining position to be sold after the Ceiling Multiple growth threshold is crossed.

A checkbox, off at the start. Detonation needs the box ticked, a position worth more than its anchor, and a bullish reading at or above the confidence you set. It also needs the reading to have just turned bullish, so a tape that was already bullish last time fires nothing.

`src/trading/scrumming_bot.py` — `ScrummingBot._check_detonation_trigger`

```
conf_min = float(getattr(self.config, "detonation_confidence_min", 0.75))
is_bullish = (
    summary.consensus_direction == SignalDirection.BULLISH
    and summary.consensus_confidence >= conf_min
)

fired = is_bullish and not self._detonation_last_signal_bullish
```

Where the code departs. The sentence above says the sale happens after the Ceiling Multiple threshold is crossed. The engine's bar is the anchor, not the ceiling. Driven on a two hundred dollar anchor: a position worth two hundred dollars and one thousandth of a cent passed the bar and the hourly check fetched candles; a position worth exactly two hundred dollars was refused before any fetch. A bot with Detonation on and the ceiling left off can therefore harvest at any value above its starting target, well below any multiple.

The bar reads the larger of the count attributed to the bot and `_audited_venue_units`, the last balance an audit recorded, so a venue figure above the attributed count clears the bar on its own.

`src/trading/scrumming_bot.py` — the bar `_check_detonation_trigger` actually uses

```
current_value = (
    self._current_holdings * price * float(self._quote_to_usd or 1.0)
)
if current_value <= self._anchor_target_balance:
    return False
```

The multiplication is now `priced_usd`, the one position-value function every site in the bot calls: the tick, the ceiling ratio, the detonation bar above, the manual rebalance, the reconciler and the Simulator's position.

`src/trading/scrumming_bot.py` — the bar as it reads now

```
audited_units = (
    as_finite_float(getattr(self, "_audited_venue_units", 0.0)) or 0.0
)
current_value = priced_usd(
    max(self._current_holdings, max(0.0, audited_units)),
    price,
    float(self._quote_to_usd or 1.0),
)
if current_value <= self._anchor_target_balance:
    return False
```

Functional. The box is what starts the whole path. Driven with the box off, the detonation phase returned at once and asked the venue for nothing; with it on, the check ran and asked for one candle series. Behind the box sits an hourly rate limit, measured: a gap of three thousand five hundred and ninety-nine seconds since the last check fetched nothing and a gap of three thousand six hundred fetched once.

`src/trading/scrumming/tick_phases.py` — `_tick_detonation`

```
if getattr(self.config, "detonation_enabled", False):
    try:
        fired = await self._check_detonation_trigger(ticker)
```

Detonation TF - Selects the timeframe for the chart that is being evaluated for bullish conditions that will allow the detonation to occur.

Two entries, 1d and 1w. The label and the stored value are the same string on both rows, which is not true of every picker in the wizard.

`src/gui/bot_wizard.py` — the Detonation TF row

```
self._detonation_timeframe = QComboBox()
self._detonation_timeframe.addItem("1d", "1d")
self._detonation_timeframe.addItem("1w", "1w")
```

Functional. The pick does two jobs. It names the candle series the vote reads, and it sets how long the one-shot bullish latch survives between checks. One day keeps the latch eighty-six thousand four hundred seconds and one week keeps it six hundred and four thousand eight hundred. Driven on one bullish tape with the latch already set and exactly eighty-six thousand four hundred seconds since the last check: the daily pick retired the latch and fired, the weekly pick kept the latch and did not. One second earlier, neither fired. That is the pick changing the answer on identical candles.

`src/trading/scrumming_bot.py` — the latch life

```
latch_ttl = max(
    float(TIMEFRAME_SECONDS.get(tf, TIMEFRAME_SECONDS["1d"])),
    _DETONATION_LATCH_MIN_TTL_S,
)
if elapsed >= latch_ttl:
    self._detonation_last_signal_bullish = False
```

Min Confidence - This is the minimum technical analysis confidence index (via the Indicator Voting Panel) that will allow the detonation to occur.

Now: a detonation also needs three agreeing indicators, so a vote that only two indicators could reach waits at any bar, `src/trading/scrumming_bot.py, in ScrummingBot._check_detonation_trigger`.

From 0.50 to 1.00, at 0.75 to start.

Since 2026-10-28 every Min Confidence box runs from 0.00 to 1.00 and still opens at 0.75, both read from `src/trading/container/config.py`. The detonation check votes at the bot's own twelve weights. The dated entry for that day at the end of this page carries the readings.

Functional. The reader works, and it is exact. On one tape whose consensus read zero point two three three four, a bar of zero point two three three three fired and a bar of zero point two three three five did not, on the same candles with the same state. The comparison is at or above, so a bar set to the reading itself fires.

`src/trading/scrumming_bot.py` — the comparison

```
conf_min = float(getattr(self.config, "detonation_confidence_min", 0.75))
is_bullish = (
    summary.consensus_direction == SignalDirection.BULLISH
    and summary.consensus_confidence >= conf_min
)
```

Where the code departs. No figure this box can emit was reached. The box starts at zero point five, and across one hundred and seventy-one tapes — nine shapes, three lengths, three noise levels — the highest bullish consensus the voting panel produced was zero point two five three four. The engine's own floors on the same quantity sit lower still: a quarter for an ordinary trade, and nineteen hundredths on the band-priority arm. A bar of zero point five refused every tape driven, including the strongest. Nothing in this reading is impossible in principle: with every voter agreed at its own best reading the panel would reach eighty-four hundredths, and the twelve voters simply cancel each other long before that. The gap is between the range the box offers and the range the panel produces.

`src/trading/scrumming_bot.py` — the quantity, and the two floors the rest of the engine uses against it

```
_TA_CONFIDENCE_FLOOR = 0.25
_BB_PRIORITY_CONFIDENCE_FLOOR = _TA_CONFIDENCE_FLOOR / (1.0 + _BB_PRIORITY_SKEW)
```

Current reading, the band-priority floor. Nineteen hundredths holds on the band-priority arm only while two indicators reach less. The arm's floor now stays above what the two heaviest voters reach at full confidence, so at the published weights it sits just above 0.1966.

`src/trading/scrumming_bot.py` — `_skewed_confidence_floor`

```
past_two = math.nextafter(two_indicator_ceiling(signals), math.inf)
return min(floor, max(floor / denominator, past_two))
```

Current reading, the band-priority floor over 0.25. At the published weights with all twelve voting, the arm's floor still sits just above 0.1966. Where two indicators reach 0.25 or more, the floor now sits just above what they reach, not at 0.25.

`src/trading/ta_engine.py` — `three_indicator_floor`

```
return max(floor, math.nextafter(two_indicator_ceiling(signals), math.inf))
```

Design intention. Two repairs are possible and both change what a live bot does on its next tick, so neither is shipped here. The box's floor could move down to meet the panel, or the detonation could compare against the same floor the rest of the engine uses. Either one arms a control that is quiet today, on bots holding real money, which is the operator's decision and not a wiring job.

PROPOSED — `src/gui/main_tabs/live_settings_tab_surface.py`, the row's range

```
"range": (0.10, 1.00),
```

Functional. Two things the bar does not reach. It is absent from the record the Bot Swarm row reads, so that tooltip can report the timeframe a bot is watching and never the bar it must clear. And the detonation check builds a fresh voting panel with no weights, so a bot carrying the operator's own indicator weights is judged for detonation by the default weights instead.

`src/trading/scrumming_bot.py` — the panel the detonation check builds

```
engine = VotingEngine()
parsed = candles_from_raw(candles)
summary = engine.compute_all(parsed, tf)
```

`src/gui/bot_wizard.py` — `TradingParamsPage.get_config`, the settings this group emits, in the order of the rows above

```
"position_ceiling_enabled": self._position_ceiling_enabled.isChecked(),
"position_ceiling_multiple": self._position_ceiling_multiple.value(),
"detonation_enabled": self._detonation_enabled.isChecked(),
"detonation_timeframe": self._detonation_timeframe.currentData(),
"detonation_confidence_min": self._detonation_confidence_min.value(),
```

Next we use the Strategy Gate Flags which allows top-level trade restrictions to be enabled or disabled thus relaxing or restricting the conditions under which a trade action can occur. The settings, in this case, are self-descriptive.

Now: on a running Scrumming bot this group also holds a Hunger tick box and a Hunger Pace box, which ease the scrum and fold vote floor over idle candles (`src/gui/live_settings/settings_tab.py, in SettingsTabMixin._add_hunger_rows`).

Functional. The box draws five checkboxes, all ticked at the start: SCRUM requires bullish TA, SCRUM holds in sustained uptrend, SCRUM defers to higher-TF bullish, FOLD requires bearish TA, and FOLD defers to higher-TF bearish.

`src/gui/bot_wizard.py` — `TradingParamsPage.get_config`, the five flags this group emits

```
"scrum_require_ta_bullish": self._gate_scrum_ta_chk.isChecked(),
"scrum_hold_in_uptrend": self._gate_scrum_uptrend_chk.isChecked(),
"scrum_defer_to_htf": self._gate_scrum_htf_chk.isChecked(),
"fold_require_ta_bearish": self._gate_fold_ta_chk.isChecked(),
"fold_defer_to_htf": self._gate_fold_htf_chk.isChecked(),
```

Design intention. Five flags are declared and the box draws five checkboxes. A sixth, the fold-side twin of the scrum gate that holds a sell during a sustained uptrend, is gone: it is off the bot config, off the restore round-trip and off both wizard surfaces. It had no reader that moved a decision, so nothing the engine does changed with it, and a bot record written before the removal still loads because creation keeps only the fields the config declares.

A fold-side hold needs a measurement before it needs a flag. The engine holds one trend figure, a count of the last twenty candles that close above their open, and it holds no downtrend figure for a gate to read.

Proposed, not present, in `src/trading/gate_chain.py`*:*

```
class FoldTrendHoldGate(Gate):
    """FOLD-only: hold the buy while a sustained downtrend runs."""

    name = "trend_hold_fold"
    side = "fold"

    def evaluate(self, ctx: GateContext) -> GateResult:
        if not ctx.eff_trend_hold:
            return GateResult(passed=True)
        return GateResult(
            passed=False,
            blocker_message=f"trend_hold_fold({ctx.trend_strength:.0%})",
        )
```

That gate reads the bullish count after the scrum checkbox has been applied to it, so as written it would hold a buy during an uptrend. A downtrend figure and a checkbox have to arrive with it.

What the fleet runs. Both switches in Risk Controls are off on all 38 live bots, which is the state the page opens in. With the ceiling off a position grows without a brake and the fold never tapers. With Detonation off no bot sells a whole position. The three figures beside those switches therefore sit unread on every record, at the values the page opens at.

That pairing is the thing to understand before changing either switch. The ceiling brakes buying and the detonation sells the position, and the detonation does not wait for the ceiling: its own bar is the anchor, which any grown position is above. Ticking Detonation alone arms a full exit well below any multiple, so a reader who wants the ceiling behaviour described above has to tick both.

The five Strategy Gate Flags are the opposite case, and the fleet's uniformity is the whole suggestion. All five are ticked on all 38, which is the strictest reading the page offers: a sell needs a bullish vote, holds through a sustained uptrend and defers to the higher timeframe, and a buy needs a bearish vote and defers as well. Unticking one lets the bot trade more often on weaker evidence. None has been relaxed in the live run.

```
setting                     the page opens at   the fleet runs
Enable Position Ceiling     clear               off on 38
Ceiling Multiple            5.0x anchor         5.0 on 38, unread
Enable Detonation           clear               off on 38
Detonation TF               1d                  1d on 38, unread
Min Confidence              0.75                0.75 on 38, unread
SCRUM requires bullish TA   ticked              on on 38
SCRUM holds in uptrend      ticked              on on 38
SCRUM defers to higher-TF   ticked              on on 38
FOLD requires bearish TA    ticked              on on 38
FOLD defers to higher-TF    ticked              on on 38
```



The Profit Routing group.

Moving onto the final section, we have Profit Routing which was intended to allow profits to be routed differently during initial set-up. This will be re-evaluated and potentially removed.

The group and both of its rows are removed, from the wizard and from Bot Settings. `_route_scrum_proceeds_via_wires` moves the scrum proceeds and never asks what a route says. The operator's ruling on a setting that reaches no trade: "I do not want dangling settings fixed that do not or have not affected what is now almost 6000 trades or data points. The trading mechanisms are valid and sound." Profit reaches accumulation through harvest-fold and the fold tranches, and a wire drawn on the Bot Swarm tab carries any cross-bot share, so neither row had a destination left to name.

Route - Destination for profits.

Target bot ID - Field for manually a bot ID which was intended to create a Smart Wire under the Bot Swarm tab.

`src/trading/scrumming/wire_routing.py` — the cross-bot destination the engine does read, taken from the drawn wires and never from a stored bot id

```
_wires = _wire_mgr.get_outgoing_wires(self.bot_id)
```

A bot saved before the removal still loads. The restore path keeps only the keys that name a field the config declares, so a stored route is dropped at that filter and every other field arrives at the figure it was saved with.

`src/trading/container/restore.py` — the whole selection

```
_kwargs = bot_config_kwargs(mode, cfg, exchange_id=cfg["exchange_id"])
if "stack_mode" not in cfg:
    _kwargs["stack_mode"] = STACK_MODE_DEFAULT
```

Two further settings are removed alongside them, and neither ever had a row on any screen. Investment amount was saved and restored and read by nothing. Spacing style was saved and restored and read by

nothing, and the placement the engine derives from Opposing Trade Distance and the band extension is the method that replaced it.

The sentence above names two terms behind a tranche's placement and the engine reads one of them. `src/trading/otd_math.py`, in `minimum_opposing_trade_distance_pct_from_config`, adds the scrumming interval and the trading fee and nothing else, and that sum is what the fold gate and the Fold Tranches row both use. `band_extension_pct` in the same module states the band term and no caller hands it a band, so the figure it would contribute is 0.000% everywhere today. Section 13 of 05-novel-concepts.md carries the rule and the figures.

Current reading. The fold gate and the Fold Tranches row read one distance with three terms: the interval, the fee and the band extension the tick's band asks for. It never sits under two fees, and its fee is the larger of the setting and the highest fee the venue has reported.

`src/gui/main_tabs/fold_tranches_tab_surface.py` — `fold_distance_pct`

```
reader = getattr(bot, "opposing_trade_distance_pct", None)
try:
    if callable(reader):
        return float(reader(getattr(bot, "_last_bb", None)))
    return minimum_opposing_trade_distance_pct_from_config(bot.config)
except (AttributeError, TypeError, ValueError):
    return None
```

What the fleet runs. Neither removed row has a figure to report, because neither ever reached a saved record. What runs in their place is profit folding, which is on for all 38 live bots, and the Smart Wires drawn on the Bot Swarm tab, which carry every share that crosses from one bot to another.

A reader asking where a bot's profit goes should follow two paths and not a route. Profit from a fold stays with the bot, through the fold tranches and the target growth the Max Target Growth row caps. Profit that leaves a bot travels along a wire the operator drew, and the Wire Inflow Stack row decides what the receiving bot does with it.

```
profit_folding_active   the fleet runs    true on 38 of 38
a stored route          the fleet holds   none; the key is dropped on restore
```

The receiving bot no longer carries that row. Your ruling retired it, so a share arriving along a wire has one destination and no choice of its own. Nothing on a screen sizes the arrival, and no per-cent figure is read for it.

```
an arriving share   the bot holds tranches   added to them, evenly
an arriving share   the bot holds none       parked until a sell opens one
```



The Phantom Bots page.

Lastly we have the selection for the Phantom (Balance) Bots. These are intended to provide trade action overrides from higher timeframe charts and indicator sets which, in turn, may result in an improved trade or prevent a premature one.

Functional. This is the last page on the scrumming path:

- Enable Phantom Bots, a checkbox, off at the start.
- Active Timeframe, eleven checkboxes: 1m, 5m, 15m, 30m, 1h, 2h, 4h, 6h, 12h, 1d and 1w. All start clear. The eleven names come from `ALL_TIMEFRAMES` in `src/exchange/timeframes.py`, so the page holds no copy of its own.
- Candles to lock, 1 to 10, at 2, inside the Higher-TF Lock Duration group.

A bot runs one phantom. Ticking a box clears every other box, and the heading states the rule: "Active Timeframe — pick one, above this bot's TA Timeframe:". A request ticking one timeframe and then another hands back one name.

`src/gui/main_tabs/bot_wizard_surface.py` — `BotWizardModel.set_phantom_timeframe`

```
for found in PHANTOM_TIMEFRAMES:
    if found != timeframe and self.phantom_checks[found]:
        self.phantom_checks[found] = False
        self.calls.append([CHECK_SET_CHECKED, found, False])
self.phantom_checks[timeframe] = True
```

A timeframe that cannot run is refused. A phantom must outrank the bot's own TA Timeframe. A box at or below it stays clear and its tooltip says why, and a box the venue does not carry behaves the same way. The Qt page and the Electron page share one rule, so on a one-hour bot a five-minute tick hands back no name and records the refusal `phantom_not_higher`.

step	module	symbol
state the rule	`src/gui/main_tabs/bot_wizard_surface.py`	`BotWizardModel.phantom_refusal`
apply it on a tick	`src/gui/main_tabs/bot_wizard_surface.py`	`BotWizardModel.set_phantom_timeframe`
apply it on the Qt page	`src/gui/bot_wizard.py`	`PhantomConfigPage._on_timeframe_toggled`
hand on the choice	`src/gui/bot_wizard.py`	`PhantomConfigPage.get_config`

The page hands on only the box that is both ticked and available. Before it lets you leave, it asks the API load monitor whether that many phantoms would breach the safety threshold for the venue, and offers you Back to adjust or Continue anyway.

`src/gui/bot_wizard.py` — `PhantomConfigPage.get_config`

```
  def get_config(self):
      checked = [
          tf
          for tf, cb in self._tf_checks.items()
          if cb.isChecked() and not self._timeframe_refusal(tf)
      ]
      return {
          "enable_phantoms": self._enable.isChecked(),
          "phantom_timeframes": checked,
          "lock_candle_count": self._lock_candles.value(),
      }
```

`enable_phantoms` and `phantom_timeframes` reach the bot as arguments to `ScrummingBot.__init__` rather than through the bot config, so the box and the timeframe on this page are what the new bot opens on. A stored timeframe the venue does not serve reaches the bot as an empty list, and the bot writes its own note naming the entry it dropped.

`lock_candle_count` is the one figure on this page a new bot does not receive. It is no field on `BotConfig` and no argument of that constructor: the phantom coordinator holds an attribute of that name, and Bot Settings writes it there on a running bot, which opens on the coordinator's own default of two. The attribute changes nothing either way. The lock guard that read it has been retired, so the coordinator keeps the figure and no method on it reads it.

What the page delivers to a new bot. All three figures on this page reach the bot that is built. One helper selects them and the constructor call carries them, so the lock count arrives as the coordinator's own count.

the page writes	the constructor argument	read on the new bot at
`enable_phantoms`	`enable_phantoms`	`_phantoms_enabled`
`phantom_timeframes`	`phantom_timeframes`	`_phantom_timeframes`
`lock_candle_count`	`lock_candle_count`	`_coordinator.lock_candle_count`

step	module	symbol
select the three	`src/trading/container/config.py`	`phantom_init_kwargs`
hand them over	`src/gui/main_window.py`	the `ScrummingBot` call
hold the count	`src/trading/scrumming_bot.py`	`ScrummingBot.__init__`

Driven on a one-hour Coinbase bot with the box ticked, `1d` picked and the spin box at seven, against the page's own defaults of clear, nothing and two:

```
                        page   new bot
enable_phantoms         True   True
phantom_timeframes      1d     1d
lock_candle_count       7      7
```

A config carrying no count leaves the argument out, so the coordinator keeps its own figure of two. An Extractor holds no coordinator and takes no count.

The engine now holds the same one-phantom rule as this page. The wizard let one box stay ticked, and the engine kept whatever a saved record named. A bot restored with several names built one reader for each of them. It keeps the highest-ranked name as the one it runs and the rest as names set aside, through the one declaration `one_active_phantom_timeframe`, and so does every path that writes the record.

`src/paper/fleet_source.py` — `wizard_phantom_timeframes`

```
  served = [one for one in chosen if one in offered]
  active, _set_aside = one_active_phantom_timeframe(served)
  return active
```

Driven on a bot given 6h and 1d over 801 band positions in steps of 0.03 of a price unit: one reader is built where two were, and the lower-ranked reading no longer reaches either gate chain. A bot given one name answers identically on every row.

The count creates no lock, because the coordinator has no method that makes one. What it reaches is the attribute the state file saves, the restore path writes back and the Bot Settings phantom page shows, so each of those reads the figure the page was set to rather than two.

Design intention. The engine behind this page is unfinished. The Indicator Voting Panel section of this manual records that phantom bots are still in active development and that related features do not yet work, so nothing is proposed for the page.

In development.

What the fleet runs. Phantom Bots are off on all 38 live bots, which is the state the page opens in. Every record still carries one stored timeframe, 15m, and no bot reads it while the box is clear. The lock count reads two candles on all 38, the figure the page opens at.

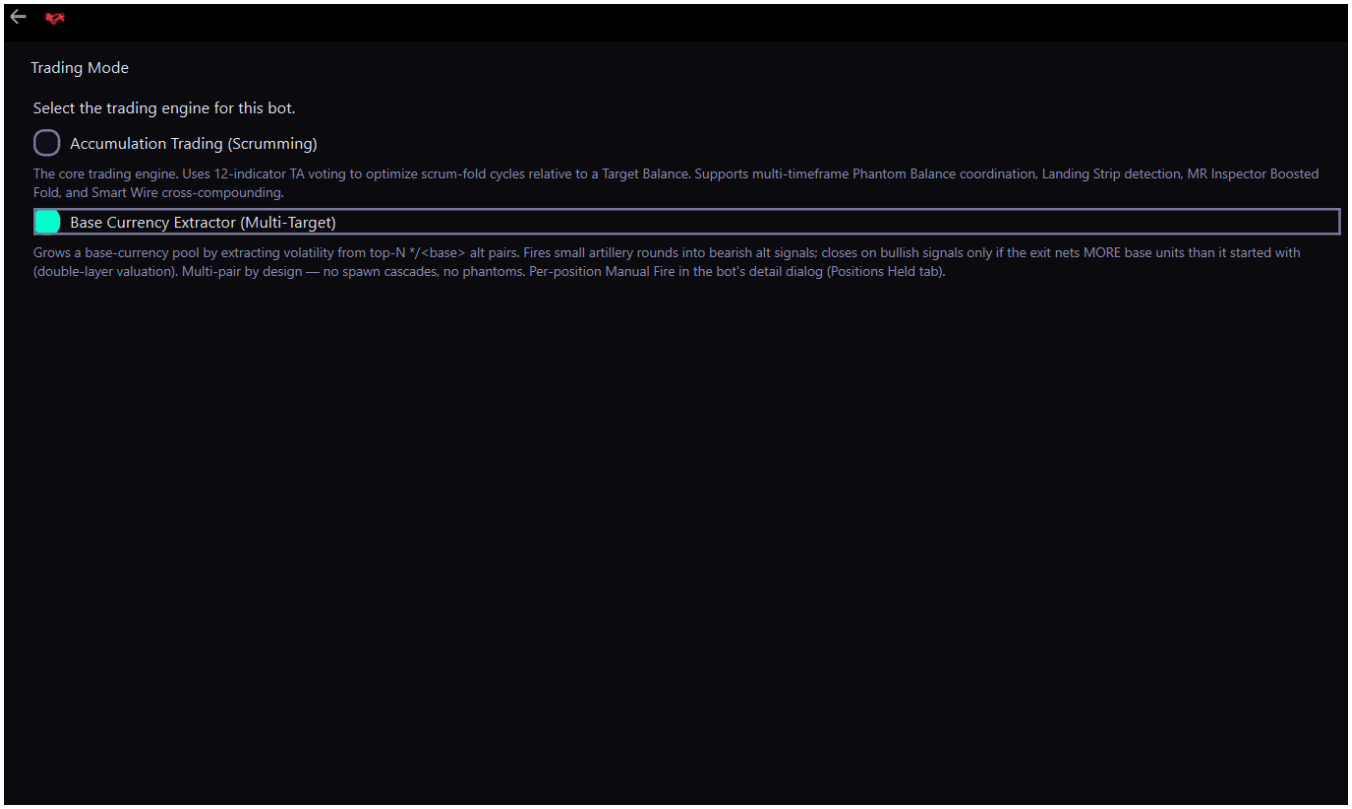
The stored 15m shows the page's own rule at work. A phantom must outrank the bot's own TA Timeframe, and the fleet runs 5m, so 15m is the next step the page allows. A reader building a bot on a slower chart has to pick a higher step again, and a box at or below the bot's chart stays clear and says why in its tooltip.

Nothing is suggested about turning this on. The engine behind the page is unfinished, so the only honest reading of the clear box is that the feature waits on the build rather than on a setting.

```
setting               the page opens at   the fleet runs
Enable Phantom Bots   clear               off on 38
Active Timeframe      all clear           15m stored on 38, unread
Candles to lock       2                   2 on 38
```

Extractor Bot (Partially Built; Untested)

The second type of bot offered within Acervator is the Extractor. These operate quite differently from Scrumming Bots and actually operate as Siblings of them. In fact, an Extractor Bot cannot even be called unless a corresponding Base Currency Scrumming Bot (i.e. A15:USD or A14:USD) is already active. This is due to the core operating principle of the Extractor bot to acquire more of these base currencies by performing trades against available alternate currency pairings. It does this by using and blocking off a portion of the Parent's position within an Extractor Tranche that represents an active position taken against one of the available alternate pairs. The Extractor Tranche remains open until its opposing accumulating (or Short Position if preferred by the user) or profit taking trade is filled. Extractor Tranches can be of any size but should generally be a relatively small fraction of the Parent's total position which will allow the Extractor to take multiple positions if available and allowed by the specific user.



The Trading Mode page with the Extractor selected.

Functional. The same first page as before, with the other radio chosen. From here the wizard goes to the Extractor Pool page instead of the asset page. Phantom bots and profit folding are both switched off for this bot as it is built, and an Extractor never sees the Phantom page at all.

OVERTAKEN, quoted whole:

> ***Functional.*** *The same first page as before, with the other radio chosen.*

True today: that radio is on the crypto layer alone. Five readings settle it, and every one is in the shipped code. `src/gui/bot_wizard.py`, in `ExtractorPoolPage`, offers five pool currencies and each is a crypto asset. `src/gui/main_tabs/bot_wizard_surface.py`, in `pool_config`, writes the venue, the pool currency and the pool sigil, and writes no sector at all. `src/trading/bot_container.py`, in `_asset_class`, answers crypto for a bot naming no sector. `src/trading/extractor_bot.py`, in

`_pair_filter_matches`, keeps a market whose quote side is the pool currency. The same file, in `_hand_base_currency_to_parent`, needs a parent accumulation bot holding that same crypto currency. So the Extractor row draws on the crypto layer and on no other, and `EXTRACTOR_SECTORS` in the surface is where that one sector is named.

`src/gui/bot_wizard.py` — `BotCreationWizard.get_bot_config`

```
config.update(self._params_page.get_config())
if config["mode"] == "extractor":
    config["enable_phantoms"] = False
    config["profit_folding_active"] = False
else:
    config.update(self._phantom_page.get_config())
```

Design intention. Phantom overrides and profit folding belong to the parent, and the two hard False values above are that decision written down. The mode is stamped onto the config at the very top of the same method, so nothing downstream has to guess which kind of bot it received.

`src/gui/bot_wizard.py` — the mode stamp

```
if self._mode_page.is_extractor():
    config["mode"] = "extractor"
    config.update(self._extractor_pool_page.get_config())
else:
    config["mode"] = "scrumming"
    config.update(self._asset_page.get_config())
```

What the fleet runs. The mode is the one choice on this page that cannot be changed afterwards. It is stamped onto the configuration, the two kinds of bot are different engine classes, and nothing downstream re-reads the radio. Every live record carries the Scrumming mode, so the pages that follow describe a build and not a running configuration.

The two hard False values above are why an Extractor never shows a Phantom page and never shows a profit-folding row. On the Scrumming side both of those run the other way, and that contrast is the quickest way to see what the Extractor gives up: it takes no higher-timeframe override and it compounds nothing of its own.

```
setting                on a Scrumming Bot    on an Extractor
mode                   scrumming on 38       no live bot
profit_folding_active  true on 38            forced false
enable_phantoms        off on 38             forced false
```



The Extractor Pool page.

Functional. This page draws:

- Exchange, the connected venues. Changing it re-scans that venue.
- Pool Base Currency, a fixed list of five. It names the asset the pool accumulates.
- A line counting the pairs available against that base, as a reading of which pairs trade there. Its tick boxes, its Select all and Clear buttons and its stored list are gone: the bot ranks those pairs by the last day's volume and keeps the busiest, re-ranking on each refresh. His ruling of 13 September 2026 reads *"Not needed. List of compatible alt pairs is scanned and piped through the Trading IVP along with the chart being read. Same as Scrumming except multiple markets are being checked by one bot for entries and exit opportunities."*

The page hands on a single asterisk where a Scrumming Bot would hand on one target asset. Its own docstring calls that the pool sigil. An Extractor holds a pool of alts rather than one target, and the sigil keeps the field's shape for the code downstream that builds a trading symbol.

`src/gui/bot_wizard.py` — `ExtractorPoolPage._POOL_BASES`

```
_POOL_BASES = ["A15", "A14", "USDT", "USDC", "BNB"]
```

`src/gui/bot_wizard.py` — `ExtractorPoolPage.get_config`

```
return {
    "exchange_id": self._exchange.currentData(),
    "base_currency": base,
    "target_asset": "*",  # pool sigil — multi-pair indicator
    "extractor_alt_targets": checked,
}
```

Design intention. The five pool bases are exactly the assets named as base currencies above. Leaving the list empty hands the choice back to the bot, and only a box that is both ticked and available reaches the config.

`src/gui/bot_wizard.py` — how a ticked alt is collected

```
for i in range(self._alt_list.count()):
    item = self._alt_list.item(i)
    if item.checkState() == Qt.Checked:
        sym = item.data(Qt.UserRole)
        if sym:
            checked.append(sym)
```

The typed pair list the two blocks above collect left the product under his ruling of 13 September 2026, and its name reads **zero** occurrences across the source, the Electron build, the tooling and the entry point. The pool page still asks for the exchange and the base currency, and the bot ranks the pairs itself.

`src/trading/extractor_bot.py` — `ExtractorBot._refresh_watch_list`, how the pairs are chosen now

```
ranked.sort(key=lambda x: x[1], reverse=True)
top_n = int(self.config.extractor_scan_top_n)
top_n = max(5, min(10, top_n))
new_watch = [sym for sym, _ in ranked[:top_n]]
```



The Extractor group of the parameter page, first nine rows.

Functional. Choosing the Extractor hides the eight scrumming groups and shows this one. The two sides never appear together.

`src/gui/bot_wizard.py` — `TradingParamsPage.set_mode`, the group swap

```
scrum_visible = not is_grid and not is_extractor
for g in (
    self._mode_group,
    self._scrum_group,
    self._adv_group,
    self._hedge_group,
    self._cb_group,
    self._risk_group,
    self._gates_group,
    self._routing_group,
):
    g.setVisible(scrum_visible)
self._extractor_group.setVisible(is_extractor)
```

The group heading reads Extractor, an em dash, Pool, an ampersand, then Artillery. Qt reads a single ampersand in a group heading as the marker that names a keyboard shortcut, so the source writes two. The pair draws as one ampersand and sets no shortcut.

Chunk size (USD) - This determines the maximum amount of the parent’s pool that the Extractor can use.

From $10.00 to $10,000,000.00, at $100.00 to start. It is the pool every artillery round below is drawn from.

`src/gui/bot_wizard.py` — the Chunk size row

```
self._ext_chunk_size_usd = QDoubleSpinBox()
self._ext_chunk_size_usd.setRange(10.0, 10_000_000.0)
self._ext_chunk_size_usd.setPrefix("$")
self._ext_chunk_size_usd.setDecimals(2)
self._ext_chunk_size_usd.setValue(100.0)
```

The pool is now turned into base-currency units on the bot's first watch-list refresh. The Extractor reads its own base currency's dollar price off its exchange and hands it to the method that rebases the pool, so the pool stops being counted one dollar to one coin. A dollar-pegged base takes a rate of one and reads no price at all.

Driven on a $250.00 pool against a base priced at $4,000.00, the bot read a pool of 0.0625 base units where it used to read 250.

`src/trading/extractor_bot.py` — `ExtractorBot._acquire_usd_per_base_rate`

```
if base in DOLLAR_PEGGED_CURRENCIES:
    rate = 1.0
else:
    try:
        ticker = await self.exchange.get_ticker(f"{base}/USD")
        rate = float(ticker.last)
```

Both screens call this row Chunk size (USD), the name the sentence above gives it. The running-bot window used to call it Pool size (USD) and the Bot Swarm pool tooltip used to send the operator to that name. Four sites carried the second name and all four now carry the first. The stored key is unchanged, so a saved bot still loads.

`src/gui/main_tabs/live_settings_tab_surface.py` — the row the running window and the Electron page both draw

```
{
    "name": "ext_chunk_size",
    "kind": DOUBLE_SPIN,
    "group": EXTRACTOR_GROUP,
    "row_label": "Chunk size (USD):",
    "field": "extractor_chunk_size_usd",
}
```

An edit to this row on a running bot reaches the bot and not only the stored configuration. Apply looks the field up in one routing map and sends it to the bot's own method, which writes the configuration field and the base-unit pool figures together. Driven on a $250.00 pool against a base priced at $4,000.00, an edit to $800.00 moved the bot's own pool from 0.0625 base units to 0.2 in the same run.

`src/gui/main_tabs/bot_live_settings_surface.py` — `RUNTIME_ROUTED`, the one map both Apply paths read

```
RUNTIME_ROUTED = {
    "extractor_chunk_size_usd": "set_chunk_size_usd",
}
```

Artillery size (USD) - This determines the individual size of Extractor Tranches.

From $0.50 to $100,000.00, at $5.00 to start. The default is set small enough to fire often and still clear a venue's minimum order cost.

`src/gui/bot_wizard.py` — the Artillery size row

```
self._ext_artillery_size_usd = QDoubleSpinBox()
self._ext_artillery_size_usd.setRange(0.5, 100_000.0)
self._ext_artillery_size_usd.setPrefix("$")
self._ext_artillery_size_usd.setDecimals(2)
self._ext_artillery_size_usd.setValue(5.0)
```

One round converts at that same rate, every time it fires. Driven at $7.50 against a base priced at $4,000.00, a round sized 0.001875 base units where it used to size 7.5.

`src/trading/extractor_bot.py` — `ExtractorBot._fire_artillery`, the sizing

```
artillery_usd = float(self.config.extractor_artillery_size_usd)
artillery_base = self._usd_to_base(artillery_usd)
```

Watch list top-N - The determines the number of Alternate Currency pairs the bot will scan for potential extraction.

From 5 to 10, at 8 to start. The pairs are ranked by twenty-four hour volume, so the list holds the most liquid alternates against the chosen base.

`src/gui/bot_wizard.py` — the watch list size row

```
self._ext_scan_top_n = QSpinBox()
self._ext_scan_top_n.setRange(5, 10)
self._ext_scan_top_n.setValue(8)
```

Both screens call this row Watch list top-N. The running-bot window used to call it Auto-scan top-N. Its own hint used to end *"Ignored when manual alt-targets are set."*, and the typed pair list that sentence named left the product under his ruling of 13 September 2026, so the condition could no longer happen. The hint now names the clamp and the one pair that survives the cut.

`src/gui/main_tabs/live_settings_tab_surface.py` — the hint the row carries

```
"ext_scan_top_n": (
    "Top-N */<base> pairs by 24h volume to keep on the watch list. "
    "Clamped to 5 to 10. A pair holding an open position stays on "
    "the list below the cut."
),
```

An edit to this row on a running bot reaches the bot. The engine reads the figure where it ranks the list, so the write onto the configuration is the write the bot trades on. Driven, a running bot ranked twelve offered pairs and kept 8 before the edit and 5 after it, in the same run.

Watch list refresh - This determines the rate at which the Extractor will scan its watched markets. This is the equivalent of a Timeframe for the Extractor but covers multiple pairs.

From 10 to 240 candles, at 60 to start, which is once an hour at a one-minute cadence. It re-ranks the list rather than re-reading one pair.

`src/gui/bot_wizard.py` — the watch list refresh row

```
self._ext_scan_refresh = QSpinBox()
self._ext_scan_refresh.setRange(10, 240)
self._ext_scan_refresh.setValue(60)
self._ext_scan_refresh.setSuffix(" candles")
```

The count is now candles of the bot's own timeframe. It used to be ticks, and the loop ticks every five seconds whatever the timeframe says, so 60 came due after five minutes. Driven on a one-minute bot, 60 now comes due after 3,600 seconds and not after 300.

The running-bot window used to offer 10 to 600 and call them ticks. It offers the same 10 to 240 candles the wizard does, so both screens and the sentence above agree.

They now agree on the name as well. The running-bot window used to call this row Scan refresh and now calls it Watch list refresh, the name the sentence above gives it.

An edit to this row on a running bot reaches the bot. Driven on a one-hour bot, 60 candles answered 216,000 seconds before the edit and 120 candles answered 432,000 seconds after it, in the same run.

One method turns the setting into seconds, and both the refresh check and the base-currency price read ask it, so the two share one cadence and one reading of the number.

`src/trading/extractor_bot.py` — `ExtractorBot._refresh_interval_seconds`

```
candles = int(self.config.extractor_scan_refresh_candles)
candle_seconds = TIMEFRAME_SECONDS.get(
    self._timeframe, TIMEFRAME_SECONDS[self.DEFAULT_TIMEFRAME]
)
return float(candles * candle_seconds)
```

Pool Reserve - To be re-evaluated.

Removed. His ruling of 13 September 2026 reads *"Not needed. Conceptually its another name for Chunk Size."* The row is off the creation wizard and off the running-bot window, and the field is off the engine's declaration. The capacity check asks only whether the free pool covers one round.

```
def _has_chunk_capacity(self, artillery_base: float) -> bool:
    return self._chunk_free_base >= artillery_base
```

Exit % - To be re-evaluated.

A percentage from 10.0 to 100.0, at 100.0 % to start. Bot creation passes it through as `extractor_exit_pct`. It sets the share of a position's alt units an exit sells, and the Extractor reads it nine times across three methods: the profitability test, the bullish exit, and the per-position Manual Fire.

OVERTAKEN, and the sentences above are kept as written. The Extractor reads the field three times, in two methods: twice in the profitability test and once in the bullish exit. The Manual Fire neither reads it nor writes it. It passes the full share to the exit as an argument, so a save taken while the order is in flight writes the share you set rather than a temporary 100.0.

`src/trading/extractor_bot.py` — `ExtractorBot.manual_fire_position`, the share it passes

```
try:
    await self._execute_bullish_exit(
        pos, alt_price_in_base, exit_pct=FULL_EXIT_PCT
    )
```

A press that sells nothing now says so. A full exit always removes the pair from the held positions, so a pair still held after the press means the venue refused the order and nothing was sold. The press then reports a refusal, names the units still held, and points at the exit's own line for the reason. Before this, the same press reported a completed close and a unit count it had not sold, while the position stayed open.

`src/trading/extractor_bot.py` — `ExtractorBot.manual_fire_position`, the refusal

```
if pair in self._positions:
    msg = (
        f"EXTRACTOR MANUAL FIRE {pair}: nothing sold. The position is "
        f"still open and still holds "
        f"{self._positions[pair].alt_units:.6f} units. The exit's own "
        f"refusal line carries the reason."
    )
```

The profitability test is the one that can refuse. It prices the proportional sell in base units, takes the trading fee off, and compares the result against the same share of the cost basis. An exit that would gain dollars but lose base units does not happen.

`src/trading/extractor_bot.py` — `ExtractorBot._exit_is_profitable_in_base`

```
units_to_sell = pos.alt_units * (self.config.extractor_exit_pct / 100.0)
base_back = units_to_sell * alt_price_in_base
fee_pct = float(getattr(self.config, "trading_fee_pct", 0.6))
base_back_after_fee = base_back * (1.0 - fee_pct / 100.0)
base_in_proportional = pos.cost_basis_base * (
    self.config.extractor_exit_pct / 100.0
)
return base_back_after_fee > base_in_proportional
```

Max compounding tier - Allows the Extractor to attempt a number of compounding Swing Trades with a given Extractor Tranche with subsequent re-entries based upon the Parent Scrumming Bot’s Minimum Opposing Trade Distance + Trade Fee + Bollinger Band extension settings.

Removed. His ruling of 13 September 2026 reads *"Not needed."* The row is off both screens, the tier counter is off the position record, and the re-entry pricing that read it is gone with it. The two pricing helpers it borrowed are shared with the Scrumming side and stay where they are, in `otd_math.py` and `stack_math.py`. A realised gain always locks to the pool.

```
# The gain locks to chunk_free_base; no roll re-enters the pair.
self._chunk_free_base += base_received
log_kind = "LOCK_TO_POOL"
```

Max cost-basis multiple - To be re-evaluated.

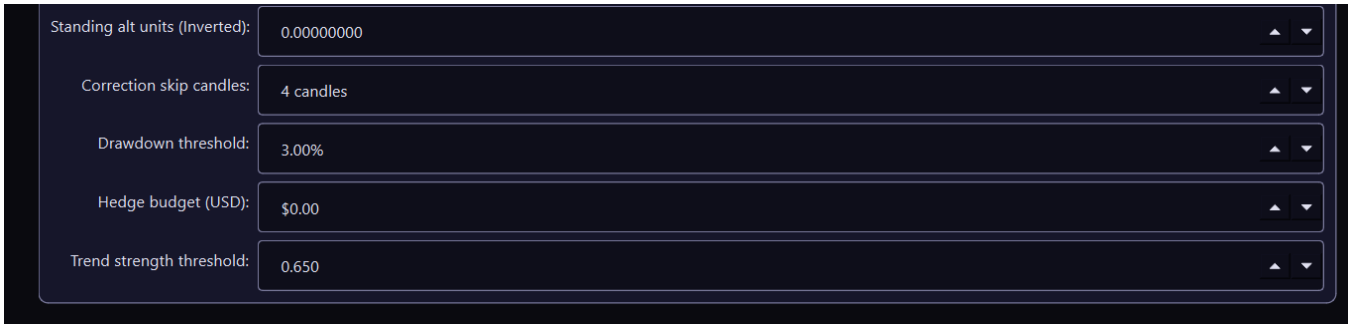
Removed. His ruling of 13 September 2026 reads *"Not needed. Trade action is not based on assumed price limits. Its based on market structure."* It was the ceiling on averaging down, and averaging down went with it, so the row, the ceiling and the log line that quoted it are all gone.

Direction - To be re-evaluated.

Removed. His ruling of 13 September 2026 puts it among the settings that are *"hallucinated nonsense that does not comply with the spec."* The Inverted mode went with the row: the pair filter keeps markets quoted in the base, and the entry side is always a buy.

```
def _entry_order_side(self):
    """OrderSide for artillery entry: BUY."""
    from ..exchange.base import OrderSide

    return OrderSide.BUY
```



The Extractor group, remaining five rows.

Standing alt units (inverted) - To be re-evaluated.

Removed. His same ruling covers it. It served the Inverted direction above, so the pool rebases from the dollar figure on every Extractor.

```
self._chunk_size_base = self._chunk_size_usd / usd_per_base
self._chunk_free_base = self._chunk_size_base
```

Correction skip candles - To be re-evaluated.

Removed. His same ruling covers it. It throttled averaging down, and averaging down went with it, so the throttle and the method that lengthened it into seconds are both gone. The watch-list refresh still counts candles through the same helper.

```
def _refresh_interval_seconds(self) -> float:
    candles = int(self.config.extractor_scan_refresh_candles)
    return float(candles) * self._candle_seconds()
```

Drawdown threshold - To be re-evaluated.

Removed. His same ruling covers it. The operator no longer sets when a position counts as down; a position is down while its dollar value sits below the value snapshotted at firing. The pool light still turns red on that reading.

```
return (
    self._position_value_usd(pos, alt_price_in_base)
    < pos.artillery_size_usd_at_entry
)
```

Trend Strength Threshold - To be re-evaluated.

Removed. His same ruling covers it. The signal provider carries the same threshold as its own published default, so the bot is built without the argument and the reading does not move.

```
self._ta_provider = TASignalProvider(
    exchange,
    timeframe=self._timeframe,
)
```

Hedge budget (USD) - no entry above names this row.

Removed. His same ruling covers it. It funded averaging down, which went with it, so the budget, the reserve it converted into and both of their saved keys are gone. The capital claim reserves the pool alone.

```
total_reserved_base = self._chunk_size_base
```

`src/gui/bot_wizard.py` — `TradingParamsPage.get_config`, the five Extractor settings the wizard emits

```
"extractor_chunk_size_usd": self._ext_chunk_size_usd.value(),
"extractor_artillery_size_usd": self._ext_artillery_size_usd.value(),
"extractor_scan_top_n": int(self._ext_scan_top_n.value()),
"extractor_scan_refresh_candles": int(
    self._ext_scan_refresh.value()
),
"extractor_exit_pct": self._ext_exit_pct.value(),
```

The same method writes `visibility` and `aggressive_trading` before it branches on the kind of bot, so an Extractor carries those two as well.

The five Extractor-only settings the engine declares

Functional. One declaration names every setting that belongs to an Extractor and to no Scrumming Bot. A Scrumming config carrying any of the five is refused when it is built, and so is an Extractor config carrying a Scrumming-only name. Creation filters the wizard's dictionary against that one declaration, and the restore path calls the same helper, so a name added to it reaches a new bot and a restored bot without a second list being edited.

`src/trading/container/config.py` — `bot_config_kwargs`, how the five are carried

```
carried = {f.name for f in fields(BotConfig)} - foreign - {"mode"}
kwargs = {
    key: value
    for key, value in _sanitize_deprecated_kwargs(collected).items()
    if key in carried
}
```

setting	what its read decides
Chunk size (USD)	the pool every round draws from
Artillery size (USD)	the size of one round

setting	what its read decides
Watch list top-N	how many ranked pairs are kept
Watch list refresh	when the list is re-ranked
Exit %	the share an exit sells

One of the five is read once, at construction, and never again: the pool size. The other four are read off the config as the bot ticks, so an edit to a running bot reaches them.

The sentence above is overtaken on its first half. The pool size is still read once at construction, and it is read again whenever the operator edits the row on a running bot: Apply sends that one field to the bot's own setter, which writes the configuration field and the base-unit pool figures in one step. So all five settings reach a running bot, four by being read where the decision is taken and the fifth by being routed.

Driven once per setting, with the configuration value and the bot's own figure read in the same run:

setting	configuration, before and after	the bot's own figure, before and after
Chunk size (USD)	250.00 to 800.00	0.0625 to 0.2 base units
Artillery size (USD)	5.00 to 20.00	0.00125 to 0.005 base units
Watch list top-N	8 to 5	8 pairs to 5 pairs
Watch list refresh	60 to 120 candles	216,000 to 432,000 seconds
Exit %	100.0 to 40.0	10.0 to 4.0 alt units

Two settings that were read once per process have left the product. The hedge budget and the standing alt units each converted to base units inside the first-tick rebase and never converted again. Both left with their own rows under his ruling of 13 September 2026, and neither name appears anywhere in the source today.

`src/trading/extractor_bot.py` — `ExtractorBot.set_chunk_size_usd`, the routed re-derivation

```
self._chunk_size_usd = new_value
self.config.extractor_chunk_size_usd = new_value
if rate > 0:
    new_chunk_base = new_value / rate
else:
    new_chunk_base = new_value
```

The dollar-to-base rate. The Extractor reads its own base currency's dollar price on every watch-list refresh, which is the moment it already goes to the venue. The first reading rebases the pool into base-currency units and writes the capital claim; every reading after that goes through the spike-protected update, so the rate a round is sized at stays current. A dollar-pegged base takes a rate of one and reads no price at all.

`src/trading/extractor_bot.py` — `ExtractorBot.tick`, the refresh step

```
if self._watch_list_due_for_refresh():
    await self._acquire_usd_per_base_rate()
    await self._refresh_watch_list()
```

The rebase runs once, on a bot's first tick with no position open, so it cannot be charged twice and cannot throw away base units a running pool has already earned. One case is open: a bot restored from a state file saved

before the rebase existed keeps the base-unit figures that file holds, because a restore carries them. Every new round is still sized at the real rate; the pool total is the part that stays stale.

The three phantom values a restored bot takes

Functional. Three phantom values are not settings the engine declares: the enable flag, the timeframe list and the lock count. One declaration names all three, and both paths read it. The wizard path reads it when a bot is created, and the restore path reads it when a saved bot comes back, so neither path names a key of its own.

`src/trading/container/config.py` — `PHANTOM_ARGUMENTS`, the one declaration

```
PHANTOM_ARGUMENTS: tuple = (
    PhantomArgument("enable_phantoms", False, _restored_phantom_flag),
    PhantomArgument("phantom_timeframes", (), _restored_phantom_timeframes),
    PhantomArgument("lock_candle_count", None, _restored_lock_count),
)
```

A record naming none of the three leaves the engine's own figures in place: the flag reads on, the timeframe list is the length above the bot's own chart, and the lock count reads two. A saved lock count that is not a number keeps the count of two and the bot still loads, with one log line naming the figure it refused.

What a restored watch list holds

Functional. A saved watch list comes back as the pairs it holds. A record whose watch list holds a single pair as one word now restores as that one pair. The same reading serves the phantom timeframe list on all three data sources, so a stored timeframe of one word restores as that one length.

`src/trading/container/config.py` — `stored_list`, the one reading

```
def stored_list(value: Any) -> list:
    if isinstance(value, str):
        return [value] if value else []
    if isinstance(value, (list, tuple)):
        return list(value)
    return []
```

a saved record holds	the restored bot holds
one pair written as a word	that one pair
a list of pairs	the same list
an empty value	no pairs
no key at all	no pairs

What the watch list holds

Functional. The bot ranks every alt that trades against its base by the last day's volume and keeps the busiest, re-ranking on each refresh. The count is the Watch list top-N figure, clamped to between five and ten. A pair holding an open position stays on the list, and a pair the venue has dropped is removed with a log line that names it.

`src/trading/extractor_bot.py` — `ExtractorBot._refresh_watch_list`, the ranking

```
ranked.sort(key=lambda x: x[1], reverse=True)
top_n = int(self.config.extractor_scan_top_n)
top_n = max(5, min(10, top_n))  # clamp to 5-10
new_watch = [sym for sym, _ in ranked[:top_n]]
```

A refresh is due after the Watch list refresh figure's worth of candles of the bot's own timeframe, so nine candles is 2,700 seconds on a five-minute bot, 32,400 on an hourly one, 129,600 on a four-hour one and 777,600 on a daily one. A timeframe the engine's table does not carry falls back to the hourly length.

`src/trading/extractor_bot.py` — `ExtractorBot._candle_seconds`

```
return float(
    TIMEFRAME_SECONDS.get(
        self._timeframe, TIMEFRAME_SECONDS[self.DEFAULT_TIMEFRAME]
    )
)
```

Who may close an Extractor Tranche

The operator's own description of the two sides:

> *"...these tranches remain until the acquired Alternate Currency stack is sold by either the Extractor Bot (Sibling) or its corresponding Scrumming Bot (Parent). All Tranches will persist under a bot's Details > Tranches Tab."*

One of the two sides runs. The Extractor closes its own tranche on a bullish exit, at the share the exit setting names, and that is the Sibling side. The Parent side names a force-sell by the base-currency Scrumming Bot at a percentage of growth, and nothing in the engine carries that percentage. It is not a field on a bot's configuration, it is not one of the five Extractor settings, and every mention of a force-sell in the source sits inside a label or a tooltip. What the toggle does is store a word, and its own tooltip says the Parent side is not acted on.

`src/trading/extractor_bot.py` — `ExtractorBot.set_tranche_arbiter`, the write

```
position.arbiter = normalize_arbiter(arbiter)
```

A built Parent side would read a growth percentage the engine does not declare. The shape it would take, as a proposal rather than a build:

PROPOSED

```
# PROPOSED. The growth percentage arrives as an argument because no
# field carries one.
def force_sell_extractor_tranche(self, tranche_id: str, growth_pct: float) -> bool:
    row = self._extractor_tranche_row(tranche_id)
    if row["arbiter"] != ARBITER_PARENT:
        return False
    cost = row["cost_basis_base"]
    growth = (row["mark_value_base"] - cost) / cost
    return growth >= growth_pct / 100.0
```

Where that percentage comes from is the operator's decision: it is a control nobody has named, and the toggle waits on it.

No bot on the saved fleet is an Extractor. All thirty-eight records carry the Scrumming mode, and all thirty-eight live claims carry the Scrumming kind.

What the fleet runs. All five Extractor settings sit on every saved record, because one declaration carries every field to every bot. A Scrumming Bot reads none of them. All 38 records hold exactly the figures this page opens at, and those are starting values rather than tuned choices, because no Extractor has run.

Read the two size rows as the pair that bounds the whole bot. Chunk size is the pool. Artillery size is one round drawn from it. The pool divided by the round is how many positions the bot can hold at once, so a reader who wants more open positions raises the pool or lowers the round. Both figures are converted into base-currency units on the first watch-list refresh, so the dollar amount you type is read once and then held as coin.

Watch list refresh interacts with the bot's own timeframe rather than standing on its own. The count is candles, so the same 60 is one hour on a one-minute bot and two and a half days on an hourly one. A reader who wants an hourly re-rank must set the count against the timeframe the bot is on.

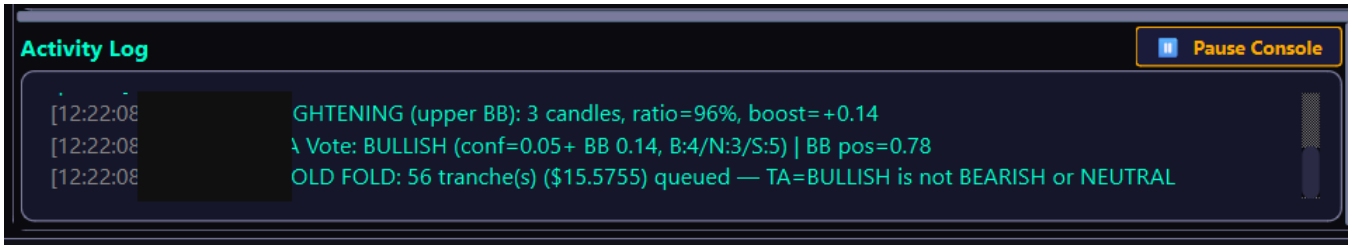
setting	the page opens at	the fleet holds	a live bot reads it
Chunk size (USD)	$100.00	the opening figure	no
Artillery size (USD)	$5.00	the opening figure	no
Watch list top-N	8	8 on 38	no
Watch list refresh	60 candles	60 on 38	no
Exit %	100.0 %	100.0 on 38	no

The eight rows marked Removed above have no figure to report. They are off both screens and off the engine's declaration, so nothing saves a value for them.

Additional Main Window > Trading Tab Features

Trade Logic and Gate Activity

This spool displays the trading logic and gate activity.



The Activity Log pane.

Functional. The pane is read-only. Every line opens with a timestamp in a muted colour, and the message takes its colour from its level: one colour for info, one for success, one for warning, one for error. Three kinds of message get a shape of their own. A trade notification draws larger and bold and takes its colour from the stage of the trade. A wire flow or wire income line draws in magenta behind a bolt character. A wire stack line draws in the pending colour behind the same character. The pane holds 5,000 lines and drops the oldest past that.

The parts one line draws

A line is a timestamp, then the bot's own tag, then the glyph, then the event, the side, the market and the numbers:

```
[hh:mm:ss] [TICKER/last4] <glyph> <EVENT> [<side>] <market> — <numbers>

[16:46:30] [A15/c7a2] ▼ FILLED [SCRUM] A15-USD — 0.001842 @ $61234.50000000
[16:46:30] [A15/c7a2] ▲ FILLED [FOLD] A15-USD — 0.003000 @ $60100.00000000
[16:46:30] [A15/c7a2] ■ WIRE INCOME: +$1.2345 from a1b2c3d4 distributed evenly
[16:46:30] [A15/c7a2] RISK GATE [SCRUM] blocked by hysteresis_scrum. Price $0.00000317
```

Each part, and what sets it:

part	what sets it
the timestamp	taken when the message arrives, drawn in a muted colour, in a column of its own so a wrapped message continues where the message starts
the bot's tag	the window adds it so you can tell which bot spoke
the glyph	a scrum sells, so it draws the chart's own sell glyph; a fold buys, so it draws the chart's buy glyph
the bolt	a wire line carries the bolt, behind the tag
the event	the stage of the trade, in capitals, which is the same word the line's colour comes from
the side	the trade's own role, in square brackets, the way a gate line writes its side
the market and the numbers	the pair traded and the writer's own figures, to the places they were always drawn

A role the chart sets no side for draws no glyph, which is what a wire-stack acquisition and a self-destruct sale get. Nothing is drawn for where a trade leaves the position, because no trade message carries a holdings figure. That figure sits on the plain line a fold writes beside its trade line.

The field that names a line's shape

The writer names the kind of line it is writing, and the pane draws the shape that kind carries. A trade writer names a trade, a wire flow or wire income writer names a wire flow line, a wire stack writer names a wire stack line, and every other writer names nothing and its line draws in its level colour, through `level_color`. The kind decides and the words do not, so a line that names a trade later in its text still draws in the trade shape. The stage word inside a trade message chooses that line's colour — green for `FILLED`, amber for `PLACED`, the primary colour for `SENT`, and red for a message naming no stage, which is what a cancellation gets — and `trade_text` cuts the drawn text and the role out of it. A `WIRE FLOW` or `WIRE INCOME` line takes magenta and a `WIRE STACK` line takes the pending colour. The bot's tag opens a line a bot wrote.

Sixteen writers name a kind, all of them in the bot brain: one trade writer and fifteen wire writers.

```
LINE_KIND_TRADE = "trade"
LINE_KIND_WIRE_FLOW = "wire_flow"
LINE_KIND_WIRE_STACK = "wire_stack"
```

`src/core/event_bus.py` — the three kinds a writer names on its own bot line

One function decides the shape for both builds, so the Qt pane and the React page cannot drift apart. It takes the kind beside the message and the level, and it takes the gate lights where a line carries them.

```
def line_style(
    message: str,
    level: Any = DEFAULT_LOG_LEVEL,
    kind: Optional[str] = None,
    lights: Optional[list] = None,
) -> dict:
```

`src/gui/main_tabs/status_log_surface.py` — `line_style`

The glyph is read from the declaration the Asset Charts readout draws its fills with, so the two screens cannot disagree about which way a trade went:

```
ROLE_GLYPHS = {
    SIDE_ROLES[SELL_SIDE]: READOUT_SELL_GLYPH,
    SIDE_ROLES[BUY_SIDE]: READOUT_BUY_GLYPH,
}
```

`src/gui/main_tabs/trade_charts_tab_surface.py` — `SIDE_ROLES`

The Simulator and the Paper Trader draw their own Activity Log through the same function, so a change to the shape reaches all three panes at once.

`src/gui/simulator/sim_status_log.py` — `SimStatusLog._render_safe`

```
def _render_safe(self, ts: str, message: str, level: str = "info") -> None:
    self.append(surface.line_html(ts, surface.line_style(message, level)))
    self.verticalScrollBar().setValue(self.verticalScrollBar().maximum())
```

What the pane reports about a line whose shape is in doubt

A line whose writer names no kind still draws, at the plain size in its level colour, and nothing is lost from the pane. The pane reports it when that line's own words ask for a shape, so a writer that should name a kind and does not is visible on the System Status tab instead of quietly drawing the wrong shape. The report carries the words the line opens with, which name the writer, beside the kind the pane drew.

```
_log_emit(
    "trading.12.008.postcondition.line_kind_named_by_writer",
    actual=kind or surface.NO_KIND,
    expected=asked,
    context={
        "writer": surface.writer_mark(message),
        "drawn_kind": style["kind"],
        "font_size_px": style["font_size_px"],
        "bold": style["bold"],
    },
)
```

`src/gui/widgets/status_log.py` — `StatusLog._report_named_kind`

The Live pane also reports every line it draws whose text names a trade, so a trade line that drew plain shows as a fault. The report reads the two apart: the drawn shape comes from the text after the bot's tag, and the asked-for shape from those words being anywhere in the message at all. A line that names no trade gets no report, so the report follows fills rather than every line.

`src/gui/widgets/status_log.py` — `StatusLog._report_trade_shape`

```
if surface.TRADE_PREFIX not in message:
    return
with contextlib.suppress(Exception):
    from src.core.signal_contract import emit as _log_emit

    _log_emit(
        "trading.12.007.postcondition.trade_line_drawn_as_trade",
        actual=style["kind"],
        expected=surface.KIND_TRADE,
        context={
            "tag": style["tag"],
            "font_size_px": style["font_size_px"],
            "bold": style["bold"],
            "color": style["color"],
        },
    )
```

Pause Console is a toggle. While it is down, each new line goes into a buffer of 2,000 instead of the screen, and a full buffer drops the newest rather than the oldest. Resume replays the buffer with the original timestamps and adds a line counting what it replayed. One writer goes through the pause regardless, for a message you must not miss. A timer checks the pane's health every sixty seconds and writes a warning into the pane itself when the render-error count rises, or when nothing has rendered for ten minutes while bots are running.

`src/gui/widgets/status_log.py` — `StatusLog.log`, the pause branch

```
if len(self._pause_buffer) < self._pause_buffer_cap:
    self._pause_buffer.append((ts, message, level))
return
```

Design intention. Two decisions follow from the pane's job. The buffer drops the newest line rather than the oldest, so the lines around the moment you hit pause are the ones that survive. And a pane that has gone quiet says so in the pane, because silence otherwise reads as calm.

`src/gui/main_tabs/trading_tab.py` — the silence check

```
bots_active = self._bot_manager and any(
    b.state.value == "running"
    for b in getattr(self._bot_manager, "_bots", {}).values()
)
if bots_active and age > 600:
```

What the lines say. Every tick message follows one shape: the event in capitals, the side in square brackets where the message has one, then a short sentence and the numbers behind it. A message names the gate that refused and the reading that made it refuse. It never names a direction or a strength the data behind it cannot give.

A hold line names the half of the gate that failed. Two conditions must both hold before the TA gate lets a scrum through: the direction has to be bullish or neutral, and the confidence has to clear the floor. The line says which one refused, so a bullish reading held back by its confidence is never reported as a wait for a bullish reading.

`src/trading/scrumming_bot.py` — the scrum hold reason

```
if eff_direction not in (
    SignalDirection.BULLISH,
    SignalDirection.NEUTRAL,
):
    _scrum_why = f"TA={eff_direction.name} is not BULLISH or NEUTRAL"
else:
    _scrum_why = (
        f"TA={eff_direction.name} but confidence "
        f"{eff_confidence:.4f} < {_eff_conf_floor:.4f} floor"
    )
```

Design intention. A confidence and the floor it is measured against print at four decimals wherever a message compares them. At two decimals they collided, and the pane showed a comparison of a number against itself, which is not a statement anybody can act on.

One gate indicator line for each position tick

A position tick draws one line, and that line ends in the same gate lights the History tab's Gates column draws. The line names each bank, the price and the panel's three counts, and the lights carry the rest. No tick writes a paragraph of voter readings to the pane.

```
[hh:mm:ss] GATES Scrum <state>. Fold <state>. Price $<price>. Panel <counts>. <lights>

[16:46:30] GATES Scrum blocked TGT, BB, OTD. Fold blocked OTD. Price $0.00003210.
           Panel 4 bullish, 5 bearish, 3 neutral. Latched hysteresis_fold.
[16:46:30] GATES Scrum armed. Fold armed. Price $0.00003210.
           Panel 4 bullish, 5 bearish, 3 neutral.
```

Nineteen lights close the line: ten for the scrum bank and nine for the fold bank, drawn in that order. Each light is its own label above its own colour, and the colour says what that gate did on this tick. Each light carries the bank it belongs to as a field rather than as a light of its own, so no marker takes one of the nineteen places.

`src/trading/gate_vocabulary.py` — the two banks' labels, in draw order

```
scrum   TGT  INT  BB  FIRE  TA  LS  TRND  HTF  CB  OTD
fold    BB   MID  TA  LS  TRNQ  CEIL  HTF  CB  OTD
```

The chains behind those labels hold more gates than the row has lights: fourteen on the scrum side and ten on the fold side, because one label can stand for more than one gate and each chain ends in an override gate that runs in its own pass.

light	what it says
green	the gate passed and its bank is armed
red	the gate is blocking its bank
amber	the gate was read and something else is blocking
grey	the gate was not read on this tick
cyan	the landing strip is overriding that bank

Current reading, the cyan LS light. The LS light now reads cyan only while a strip sits on that side and the TA light on that bank shows neither a refusal nor an override. A strip that waits for a third agreeing indicator leaves the LS light grey.

`src/trading/gate_vocabulary.py` — `gate_light_row`

```
  scrum_ta_held = _GATE_TA in scrum_blocked | scrum_replaced
  fold_ta_held = _GATE_TA in fold_blocked | fold_replaced
```

A tick draws its line when the lights are not the same as the last line's. A tick whose gate state has not moved spends no line, which is what stops the spool. A latched gate is named in words as well, because that block holds until its own condition clears rather than turning over each tick.

```
  def _emit_gate_light_line(
      self,
      scrum_result,
      fold_result,
      summary,
      ticker_last: float,
  ) -> None:
```

`src/trading/scrumming/snapshots.py` — the one per-tick writer

The whole voter panel and every blocker phrase still exist. They go to the Console tab's own log on each line the pane draws, so a reading a tick produced is never lost, and the History tab keeps the full blocker text in its Gates column. The panel reads as a count and three groups, strongest confidence first, with an indicator's own raw reading in brackets where it publishes one; a neutral vote carries no confidence, so only its name prints.

```
  RISK GATE [SCRUM] blocked by hysteresis_scrum. Price $0.00000317.
  Panel 6 bullish, 3 bearish, 3 neutral.
  bullish vortex 1.00, kaufman_er 0.70 (er 1.0), macd 0.60, adx 0.36 (adx 25.52),
  volume 0.35, supertrend 0.26.
  bearish bollinger_bands 0.82, stochastic_rsi 0.50, slingshot 0.35.
  neutral ichimoku, rsi, zscore (z 0.143).
```

Both snapshot lines read one renderer, so the blocked half and the fired half of a decision cannot drift into two shapes.

`src/trading/scrumming/snapshots.py` — the one panel renderer

```
def _panel_line(summary: Optional[VotingSummary]) -> str:
    panel = _build_panel_snapshot(summary)
    if not panel:
        return PANEL_ABSENT_TEXT
```

API Interaction Log

This spool displays API handshake information and data transfer speeds for these messages.



The API Interaction Log pane.

Functional. The pane is read-only, holds 2,000 lines, and never wraps. One entry is written per API call. A call that arrives on any thread but the GUI thread is refused outright and written to a thread-violation file under the runtime log directory instead, because touching a widget from a background thread ends the process. An entry carries a timestamp, the exchange name, the action and a Reason line, then Endpoint, Result, Response time and Data usage wherever the record holds them. Pause API Log buffers up to 2,000 lines and flushes them on resume with a count.

An order the venue refuses writes its own entry. The entry reads `ORDER_REFUSED`, carries the venue's own message on its Reason and Result lines, and is marked as an error. Every entry marked error or warning is also appended to `api_failures.ndjson` in the `api/` bucket of the runtime log directory, so a refusal is readable after a restart. The file rotates at 5 MB and keeps five backups, which bounds the set at six files. Every other entry lives in memory for the run and goes when the process goes.

`src/gui/main_window.py` — `_on_api_event`, the thread check

```
current = _threading.current_thread().name
origin = entry.get("_thread_name", "unknown")
if current != "MainThread":
```

Design intention. The pane should tell you what the platform did with the bytes it just paid for. The candle fetch writes a Data usage note naming how many voters the engine builds and naming every one of them, taken from the engine's own declaration, so the note agrees with the Voting Panel beside it.

`src/exchange/ccxt_connector.py` — `ohlcv_data_usage`, what the fetch writes

```
def ohlcv_data_usage() -> str:
    voters = ", ".join(DEFAULT_WEIGHTS)
    return f"Fed into {len(DEFAULT_WEIGHTS)}-indicator TA engine ({voters}) for voting"
```

The pane draws this today:

```
  Data usage: Fed into 12-indicator TA engine (bollinger_bands, vortex, macd,
  stochastic_rsi, ichimoku, volume, slingshot, adx, kaufman_er, supertrend,
  zscore, rsi) for voting
```

The pane draws it on one unwrapped line; it is broken over three here to fit the page.

`DEFAULT_WEIGHTS` in `src/trading/ta_engine.py` is the one declaration of the voter set, so a count taken from it cannot drift again. A thirteenth voter reaches the note with no edit to the connector.

Two mechanisms with no control on this tab

Two more mechanisms sit inside the engine and the wizard offers no switch for either. Neither appears anywhere else in this manual, and their state differs: one runs, one is declared and dormant.

Fair Value Gap. Three candles that leave a gap between the first and the third mark a price band the market tends to come back to. The indicator scans the last 24 candles for those bands, reports whether price sits inside one or is approaching one, counts how many are open, and gives the bounds of the nearest. It is not one of the twelve voters. It adjusts the confidence of a side that already has a case: a fold gains ten points near a bullish band, a scrum gains eight inside a bearish one.

`src/trading/indicators/fvg.py` — the four constants that set its reach

```
FVG_LOOKBACK = 24  # candles to scan for 3-candle gap patterns

FVG_PROXIMITY_PCT = 0.005  # "approaching from above": within 0.5% of FVG top

FVG_BULL_BOOST = 0.10  # fold confidence boost when in/near bullish FVG

FVG_BEAR_BOOST = 0.08  # scrum confidence boost when inside bearish FVG
```

The engine imports the indicator and its four constants together, so the adjustment travels with the reading rather than being applied somewhere else.

`src/trading/ta_engine.py` — what it takes from that module

```
from .indicators.fvg import (
    FVG_BEAR_BOOST,
    FVG_BULL_BOOST,
    FVG_LOOKBACK,
    FVG_PROXIMITY_PCT,
    FVGIndicator,
)
```

Boost Fold. The intent is a pair: sell a slice of holdings when the mean reversion reading goes extreme, then buy that slice back when price returns to its moving average, and let the difference raise the target. The Market Inspector states the pair at the head of its own module.

`src/trading/mr_inspector.py` — the pair, in the module's own words

```
# ■ When all 3 gates pass → Boost Sell (sell 2.5% of holdings) ■
# ■ When price returns to SMA → Boost Fold (buy back cheaper)  ■
# ■ Profit from boost fold increments target (compound growth)  ■
```

Where it runs is the stock accumulation bot, which carries the queued amount, the reference price, the moving average and the two counters, and works all of them. The crypto Scrumming Bot declares the same four fields, sets them to zero and never reads them again: across the whole trading package the only lines naming them are those four assignments. A crypto bot therefore takes no boost fold, whatever the Market Inspector reads.

`src/trading/scrumming_bot.py` — the four fields, written once and never read

```
        self._boost_fold_q: float = 0.0
        self._boost_fold_ref: float = 0.0
        self._boost_fold_sma: float = 0.0

        self._boost_folds: int = 0
```

Proposed, not present, in `src/trading/scrumming_bot.py`*:* the crypto side needs the read half of the pair, taking the queued amount back at the moving average the way the stock bot does, guarded so a bot with an empty queue takes no action.

```
if self._boost_fold_q > 0 and self._boost_fold_sma > 0:
    if price <= self._boost_fold_sma and price < self._boost_fold_ref:
        buy_usd = self._boost_fold_q
        self._boost_folds += 1
        self._boost_fold_q = 0
```

Add Crypto Exchange Button (to be changed - Add Exchange)

While the initial set up of Acervator requires at least one valid exchange API, additional exchanges can be added and have their own bot swarms. The current upper operational limit of Acervator is unknown. Multi-exchange testing has yet to be attempted as of 8/25/26 with API compatibility work pending. This button also currently opens the Settings Panel but on the incorrect ‘User’ Tab when it should be ‘Exchanges’.



The Add Crypto Exchange button.

Functional. The button sits at the right of the exchange sub-tab row, as the corner widget of that row. Each layer builds its own and labels it for that layer: Add Crypto Exchange on the crypto page, Add Stock Exchange on the stock page. Both run the same handler as the button on the empty card a layer shows while it holds no exchange at all.

`src/gui/main_tabs/trading_tab.py` — `_make_layer`

```
tab_w = QTabWidget()
add_btn = QPushButton(f"■ Add {label_text} Exchange")
add_btn.setMinimumWidth(140)
add_btn.setToolTip(f"Add a {label_text} exchange connection")
add_btn.clicked.connect(self._add_exchange)
tab_w.setCornerWidget(add_btn)
```

Design intention. The handler already knows which wing you are in and passes that to the dialog, so it has somewhere to say which tab to open.

`src/gui/main_window.py` — `_add_exchange` today

```
dlg = SettingsDialog(self._settings, self._status_log, self, wing=_wing)
dlg.exec()
```

Proposed, not present:

```
dlg = SettingsDialog(self._settings, self._status_log, self, wing=_wing)
dlg.show_exchanges_tab()
dlg.exec()
```

The dialog in `src/gui/settings_dialog.py` has no such method yet. The proposal adds one rather than changing the call shape, so the wing argument already passed here keeps working exactly as it does.

Add Exchange opens the Settings dialog on the Exchanges tab

Functional. The press opens Settings with the Exchanges tab already showing. The tab is named at the point of opening, so the tab order is untouched and the File menu's Settings still opens where it always did.

`src/gui/main_window.py` — `_add_exchange`

```
dlg = _cls(self._settings, self._status_log, self, wing=_wing)
dlg.show_tab(EXCHANGE_TAB)
dlg.exec()
```

The tab is found by its title, never by its position. A tab added in front of Exchanges does not move which tab the press lands on.

`src/gui/settings_dialog.py` — `show_tab`

```
for at in range(book.count()):
    if book.tabText(at) == title:
        book.setCurrentIndex(at)
        return
```

Both builds answer the same title on every route. Driven in a live window:

route	asset class	the tab shown
Add Exchange, corner button	Crypto	Exchanges
Add Exchange, corner button	Stock	Exchanges

route	asset class	the tab shown
Exchange menu, Add Exchange	Crypto	Exchanges
File menu, Settings	Crypto	User

Commodities and Forex carry no Add Exchange button of their own, and a press from the Exchange menu under either class is refused before a dialog is built.

OVERTAKEN, and the sentences above are kept as written. Commodities and Forex each carry two Add Exchange buttons of their own, on their own layer: the corner button on the tab bar and the one on the empty-state card. Both read the class name and both are dead while no venue serves the class, so a press under either class is still refused before a dialog is built. Driven in a live window with a venue serving Commodities, the layer's own button opened the dialog on the Exchanges tab.

`src/gui/main_window.py` — the press the guard refuses

```
_wing = normalise(getattr(self, "_asset_class", None))
if not add_exchange_enabled(_wing):
    self._status_log.log(class_state(_wing)["note"], "warning")
    return
```

The exchange row in the Electron shell. The shell draws one tab button per configured exchange, and the screen under the button is drawn by `exchange_tab.js`. A layer shows its empty card only while it holds no exchange, which is what the Qt tab row does once it drops the Get Started tab.

`src/gui/main_tabs/trading_tab_surface.py` — `layer_exchanges`

```
for entry in entries or []:
    holder = entry if isinstance(entry, dict) else {}
    exchange_id = str(holder.get("exchange_id") or "")
    if not exchange_id:
        continue
    layer = "stock" if is_equity_exchange(exchange_id) else "crypto"
    split[layer][exchange_id] = exchange_display_name(holder)
```

OVERTAKEN, and the block above is kept as written. The shell's layer names come from the same taxonomy the Qt stack reads, so an equity venue routes to the stocks layer under that name and the shell draws one layer per asset class.

`src/gui/main_tabs/trading_tab_surface.py` — the layer an equity venue routes to

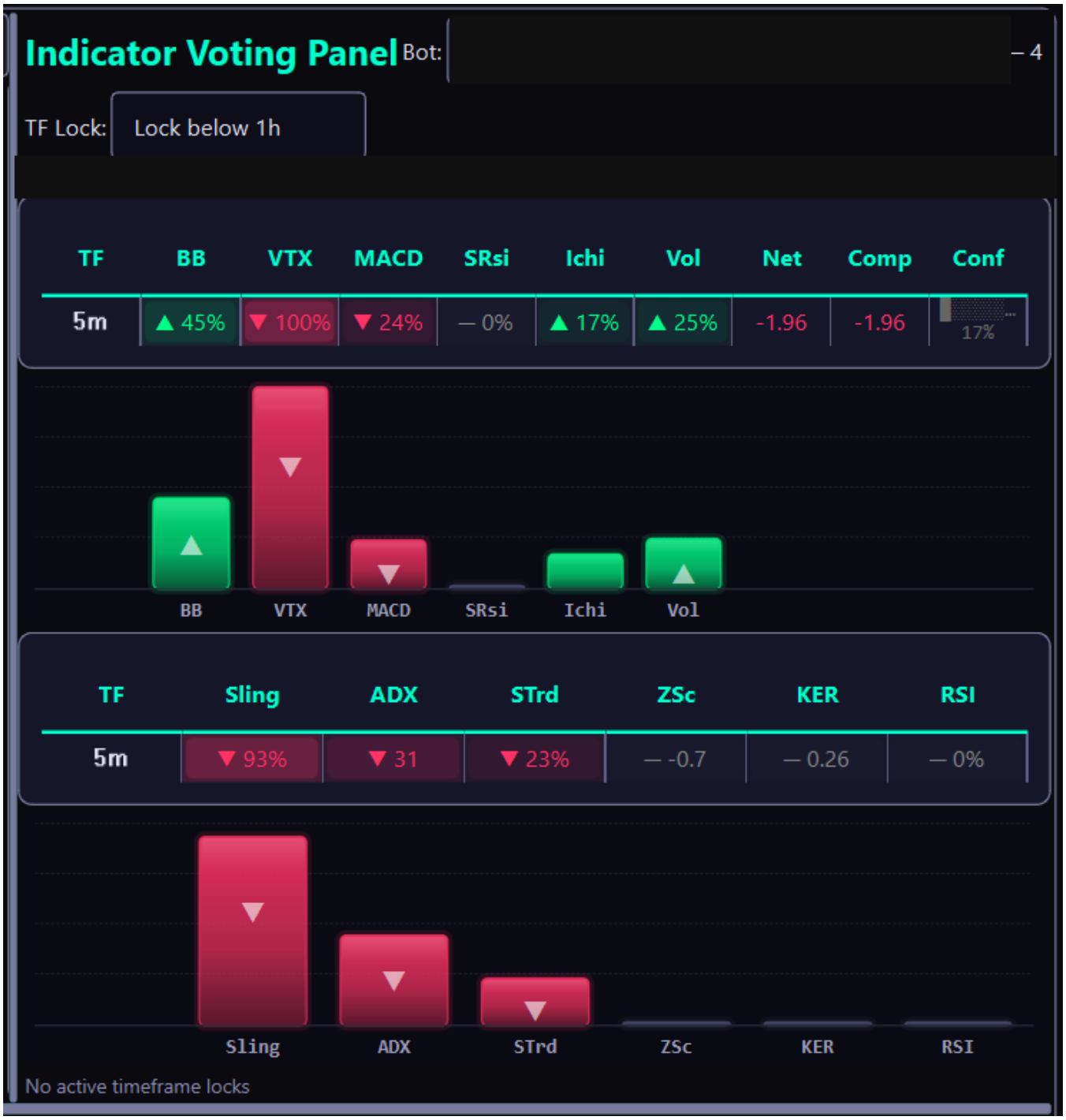
```
layer = "stocks" if is_equity_exchange(exchange_id) else "crypto"
```

One caption for one exchange. The caption on a tab comes from a single helper, so the start-up path and the settings path cannot label the same exchange two different ways. An exchange saved with no name is captioned from its own id, and an entry with no id gets no tab at all.

`src/gui/main_tabs/trading_tab_surface.py` — `exchange_display_name`

```
def exchange_display_name(entry: Any) -> str:
    holder = entry if isinstance(entry, dict) else {}
    exchange_id = str(holder.get("exchange_id") or "")
    named = str(holder.get("display_name") or "")
    return named or exchange_id.capitalize()
```

Indicator Voting Panel



The Indicator Voting Panel, at the right of the Trading tab.

Functional. The panel fills the right half of the Trading tab.

- The row at the top holds, from the left: the panel's title, a blue left arrow, the bot dropdown, a blue right arrow and the privacy control. A stretch sits on each side of the four controls, so the dropdown sits between the two arrows and the group sits in the middle of the space the title leaves. The dropdown names the bot whose votes the panel draws, and its list refills every tick.
- The rate line under it shows the A15 and A14 prices with their satoshi and gwei equivalents, and the venue name at the end.
- Two tables of six voters each. A green up triangle is a bullish vote, a red down triangle a bearish one, an em dash a neutral one. ADX, ZSc and KER print a raw value; the other nine print a percentage.
- Each voter belongs to one of three groups, trend, momentum or structure, and the group decides its header colour.
- Net, Comp and Conf close the first table. Conf draws as a small filled bar with its percentage beside it.
- A bar chart under each table draws that table's six confidences at the width of the column above.
- The line at the foot names any timeframe lock that is active. It is a readout; no control on the panel sets a lock.

The arrows. Each arrow moves the dropdown one place along the list it already holds. The list does not wrap: a press at the first bot leaves the panel on the first bot, and a press at the last leaves it on the last. An arrow with nothing left to step to is greyed. The two glyphs, their size and their family are the Charts tab's own, imported rather than copied, so the two control rows cannot drift apart: each arrow is 34 by 26 pixels and draws U+25C0 or U+25B6 in Segoe UI Symbol at 16 pixels.

`src/gui/indicator_panel.py` — `IndicatorVotingPanel._step_bot`

```
def _step_bot(self, by: int) -> int:
    """Move the dropdown ``by`` places and answer the index it lands on.

    The list does not wrap: a press at either end leaves the
    selection where it is.
    """
```

`src/gui/main_tabs/indicator_panel_surface.py` — the arrows take the Charts tab's numbers

```
from .native_chart_surface import (
    ARROW_GLYPH_FAMILY,
    ARROW_GLYPH_PX,
    CONTROL_HEIGHT_PX,
)
```

The privacy control. The panel draws the universal control, `PrivacyDot` in `src/gui/widgets/privacy_dot.py`. The glyph is a filled circle when the dropdown is readable and an empty circle when it is masked, and the control sizes to its own glyph rather than to a fixed square. The Charts tab's own dot takes its colour from the same token.

A register read that raises draws neither circle. The dot takes the register's own absent mark, the em dash, and its tooltip reads `STATE UNREADABLE`. `src/gui/main_tabs/privacy_dot_surface.py, in glyph` answers all three marks and both builds read it, and `src/gui/widgets/privacy_dot.py, in refresh`

writes the failed read to the log.

`src/gui/design_system.py` — the token the dot paints in

```
PRIMARY_BRIGHT = "#00ffee"  # Brighter cyan accent: privacy dot, Sim tab
```

The page answers a press. An ask that names an action goes to the window and moves the one Qt panel the window holds, so the page and the window never show two selections.

`src/gui/main_tabs/trading_tab.py` — `TradingTabMixin._wire_live_feeds` binds it

```
bind = getattr(getattr(self, "_trading_tab", None), "set_votes_handler", None)
answer = getattr(self, "_answer_votes", None)
if callable(bind) and callable(answer):
    bind(answer)
```

Functional. The dropdown is not the only way to pick the bot. A press on a row of the Scrumming Bots table draws that bot here, and the dropdown moves to it. A press on the row of the bot already drawn empties the panel. Whichever control the operator uses, `select_bot` is the one thing that writes the bot the panel holds.

`src/gui/indicator_panel.py` — `IndicatorVotingPanel.select_bot`

```
wanted = str(bot_id or "")
if not wanted:
    self._selection_cleared = True
    self._bot_selector.setCurrentIndex(-1)
    self._refresh_bot_arrows()
    return self._selected_bot_id
```

Two empty states, and each names its cause. A selection cleared by a second press reports the cause `no_selection` and reads *"no bot is selected — press a bot row, or pick one from the dropdown above."* A bot the fleet no longer carries reports `bot_missing` and reads *"bot … is selected but no longer present in the fleet."* A bot the dropdown does not carry leaves the panel where it is, so a stale ask cannot blank it. The cause clears when a summary arrives, so a panel drawing twelve voters never reports the reason it had been blank.

`src/gui/indicator_panel.py` — `IndicatorVotingPanel.update_data`

```
if multi_tf_summary:
    self._no_data_cause = ""
    self._no_data_message = ""
```

HIS.

> *"Live - Bug - IVP - Some bots IVP panels are not loading when selecting. Observed with A19, A24, A11, and A35. Most are working. Cycled to through the bot list a few times and some are consistently failing to load and also giving the candle cache notice in the console but the panels never populate."*

Two causes a panel with no votes names. A panel holding no votes prints one sentence saying why, on the panel rather than only in the Console. Too few candles means the slot holds fewer bars than the engine needs. Cold start means the bot is running and has computed no reading since the platform launched. A market well

over the floor can still read cold start, so a full panel is not the only healthy reading.

market	candles the slot held	the panel reads
A24	6 to 9	too few candles
A38	10 to 12	too few candles
A35	13 to 16	too few candles
A19	11 to 18	too few candles
A11	95 to 96	cold start
a market that votes	100	twelve voters

`src/gui/indicator_panel.py` — the sentence each cause prints

```
    "cold_start": "cold start — this bot is running and has computed no TA since "
    "the platform launched. Its first read lands on the next TA "
    "evaluation.",
    "too_few_candles": "too few candles — {candles} cached for {symbol} {timeframe}, "
    "and the TA engine needs {floor}.",
```

The floor is written once. The engine needs thirty candles before any indicator votes, and that figure has one home beside the engine that needs it. The three panels read it from there and print it as a field, so no panel carries the number in its own prose. A market with a shorter history is refused a vote, and that refusal is correct rather than a fault, because an indicator computed on too little history is a wrong number.

`src/trading/ta_engine.py` — the one place the floor is written

```
#: The candles ``VotingEngine.compute_all`` needs before any indicator votes.
MIN_CANDLES_FOR_TA = 30
```

Every running bot reads its market on every tick. A bot writes its voter reading above every early exit its tick can take, so a bot parked at its target, a bot firing a cartridge and a bot answering a manual fire each hold a current reading for the panel to draw. The shared price pool keeps one candle slot per market and timeframe and that slot lives one timeframe, so a tick that finds the slot fresh makes no venue call. A tick that sees the same newest candle as the last one skips the vote count entirely, so a bot parked at its target counts its votes once per candle rather than once per tick. The voter reading is drawn and recorded and nothing in the trading engine reads it, so no gate and no trade changes.

`src/trading/scrumming_bot.py` — `ScrummingBot._refresh_market_reading`, called from `ScrummingBot.tick` above its early exits

```
        ticker = await self._get_ticker(symbol)
        self.stats.current_price = ticker.last
        await self._refresh_market_reading(symbol)
```

The panel can compute from the pool. Where a bot holds no reading of its own, the panel asks the shared pool for that bot's candles and computes the votes itself. No venue call is made and no reading is invented, because the candles are the ones the platform already holds. A bot that is stopped, idle or stopped on an error is refused, because a bot that is not running reads no market and its own sentence says so.

`src/gui/main_window.py` — `MainWindow._ivp_summary_from_pool`, read by the dashboard refresh

```
                            summary = getattr(bot, "_last_summary", None)
                            if bot is not None and summary is None:
                                summary = self._ivp_summary_from_pool(bot)
```

Cold start measures its claim. The cold start sentence says the bot has read no market since the platform started. The last price the bot read is what decides it: a price above zero means the bot reached the market, and cold start is refused. A running bot that has read a price is given the candle count the panel measured instead.

`src/gui/main_window.py` — `MainWindow._ivp_last_read_price`, read by `MainWindow._ivp_empty_state_cause`

```
            price = self._ivp_last_read_price(bot)
            detail["price"] = price
            if price <= 0:
                return "cold_start", detail
```

One sentence this page carried is overtaken. It is quoted whole, and the sentence under it is true today.

OVERTAKEN, quoted whole:

> *"A market well over the floor can still read cold start, so a full panel is not the only healthy reading."*

True today: a market over the floor draws its twelve voters, because the panel computes from the pool's own candles when the bot holds no reading. A reading over the floor and a blank panel together are a fault, and the panel names it `ta_compute_failed`.

A stored reading draws with its age. When a bot has a reading saved from an earlier evaluation, the panel draws that reading rather than emptying, with the time it was taken, how old it is, and the cause sentence after it. The window build and the page build draw the same sentence in the same place.

`src/gui/indicator_panel.py` — the banner over a stored reading

```
            self._staleness_label.setText(
                f"■ LAST TA READ, NOT CURRENT — taken {when}, "
                f"{age_phrase(age_s)}. {message}"
            )
            self._staleness_label.show()
```

`src/gui/indicator_panel.py` — `IndicatorVotingPanel.INDICATOR_COLS`

```
INDICATOR_COLS = [
    ("bollinger_bands", "BB", "S"),
    ("vortex", "VTX", "T"),
    ("macd", "MACD", "M"),
    ("stochastic_rsi", "SRsi", "M"),
    ("ichimoku", "Ichi", "T"),
    ("volume", "Vol", "S"),
    ("slingshot", "Sling", "S"),
    ("adx", "ADX", "T"),
    ("supertrend", "STrd", "T"),
    ("zscore", "ZSc", "M"),
    ("kaufman_er", "KER", "M"),
    ("rsi", "RSI", "M"),
]
```

Design intention. One place that shows what every voter thinks and how far they agree. The names, the split into two rows and the three colour groups all come out of the one list above, so the panel cannot fall out of step with itself as voters are added or reordered.

`src/gui/indicator_panel.py` — the two rows, both cut from that list

```
_ROW_A_INDICATOR_COLS = INDICATOR_COLS[:6]  # BB VTX MACD SRsi Ichi Vol
_ROW_B_INDICATOR_COLS = INDICATOR_COLS[6:]  # Sling ADX STrd ZSc KER RSI
```

The operator's own reading of each column follows in the next section.

Where the maths lives. 07-indicators.md is the single home for all twelve. Each voter is described once there and once only: the operator's own line, then what the cell prints and what makes the vote bullish, bearish or neutral, then the published formula with the code that computes it. Nine voters print a direction arrow and a confidence percentage. Three print a raw value instead, and those three are ADX, Z-Score and Kaufman ER.

Voter	Its formula and its vote
BB	Bollinger Bands
VTX	Vortex Indicator
MACD	MACD
SRsi	Stochastic RSI
Ichi	Ichimoku Cloud
Vol	Volume
Sling	Slingshot
ADX	ADX and DMI — raw value
STrd	Supertrend
ZSc	Z-Score — raw value
KER	Kaufman Efficiency Ratio — raw value
RSI	RSI

One decision picks the text a cell carries. Three voters are named in it and every other voter falls through to the percentage.

`src/gui/indicator_panel.py` — `_populate_indicator_cell`

```
if ind_key == "adx":
    adx_v = details.get("adx", 0)
    if details.get("ranging"):
        cell_text = f"Rng {adx_v:.0f}"
    else:
        cell_text = f"{sym} {adx_v:.0f}"
elif ind_key == "zscore":
    cell_text = f"{sym} {details.get('z', 0):+.1f}"
elif ind_key == "kaufman_er":
    cell_text = f"{sym} {details.get('er', 0):.2f}"
else:
    cell_text = f"{sym} {confidence:.0%}"
```

The same file carries the gate logic chain that turns those twelve votes into a trade decision.

Net, Comp and Conf

The three columns that close the first table each read one field. Each one draws a number in its cell and a bar behind both tables, and both the number and the bar follow that column's own field.

Column	Its field	What it holds	Its range
Net	`net_score`	the bullish weight of the twelve voters, less their bearish weight	a signed tally, zero when the voters cancel out
Comp	`composite_net`	Net joined with the Net of each higher timeframe a phantom bot watches	the same units as Net; a dash when the row carries no composite
Conf	`confidence`	how far the voters agree with each other	0 to 1

`src/gui/main_tabs/indicator_panel_surface.py` — one definition, read by the window and by the page

```
drawn = [
    (sign_direction(net), net_fraction(net), BAR_COLORS[sign_direction(net)]),
    (sign_direction(comp), net_fraction(comp), BAR_COLORS[sign_direction(comp)]),
    (band, conf, _channels(CONF_BAND_COLORS[band])),
]
```

Bar height. Conf's bar is its own percentage. Net's and Comp's bar stands full at `NET_PILLAR_FULL_SCALE`, which is 3.00 either way; a reading past that draws a full bar. The twelve voters could together produce a much larger tally than 3.00, and a bar scaled to that ceiling would never leave the floor.

Bar colour. The hue names the kind of reading: green above zero and red below it for Net and Comp, and for Conf grey under 30 per cent, amber at 30 and over, green at 60 and over. The brightness names the size of the reading, from `PILLAR_SHADE_FLOOR` of the hue at zero to the whole hue at full scale. The cell's text takes the same hue and brightens toward white by `CELL_LIFT_SPAN` instead of dimming, so a weak reading is never harder to read than a strong one.

The descriptions. Each of the three carries the same text on its column heading and on its cell, so a hover over either one answers.

> ***Net.*** *Net vote. The bullish weight of the twelve voters above, less their bearish weight. Above zero the panel leans up. Below zero it leans down. A voter with no opinion adds nothing. The bar below stands full at 3.00 either way, and a bigger vote draws a brighter bar.*
>
> ***Comp.*** *Comp is short for composite Net. It is this bot's Net vote, joined with the Net vote of each higher timeframe a phantom bot watches. A higher timeframe pulls the reading its own way. With no phantom bot running, Comp reads the same as Net. A dash means this row carries no composite of its own.*
>
> ***Conf.*** *Conf is short for confidence. It measures how far the voters agree with each other, and nothing else. It is the Net vote divided by the weight of every voter that cast one. 0% means the voters cancel out or none has an opinion. 100% means every voter that cast one agrees. A high reading does not say the trade is a good one, only that the panel is of one mind. Grey is under 30%, amber is 30% and over, green is 60% and over.*

The maths behind Net and Conf lives with the voters, in 07-indicators.md.

The Indicator Voting Panel's layout

The panel beside the exchange stack carries seven columns on each row, aligned under each other: TF and six indicators above, TF and six more below. The three collated columns — Net, Comp and Conf — close the first row and stand as one pillar each behind both rows. Both rows share one grid, and both the window and the page head their columns through one function, so neither can head a column the other leaves blank. The two rows draw seventeen column headings between them.

`src/gui/main_tabs/indicator_panel_surface.py` — `column_titles`

```
def column_titles(subset: list, *, include_aggregates: bool = False) -> list:
    titles = [TF_COLUMN_TITLE] + [short for _, short, _ in subset]
    pad = AGGREGATE_TITLES if include_aggregates else []
    titles = titles + list(pad)
    return titles + [EMPTY_TITLE] * (PANEL_COLUMN_COUNT - len(titles))
```

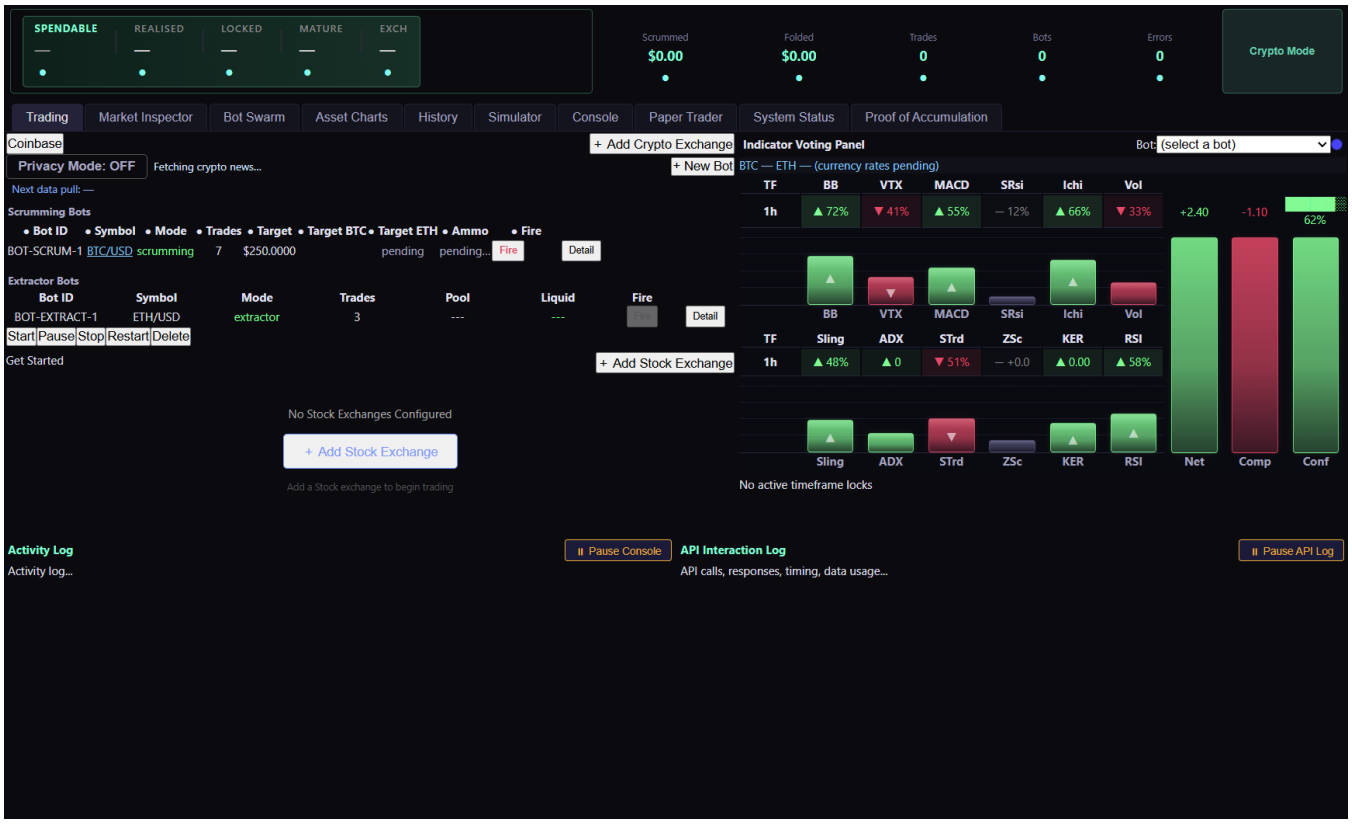
The header row draws no ground of its own, in either build, so each pillar runs behind it rather than being cut by it. The window's theme normally gives a header section a ground and an underline, and the panel overrides both.

`src/gui/indicator_panel.py` — the container and the table draw no ground

```
container.setStyleSheet("QWidget { background: transparent; }")
table.setStyleSheet(
    "QTableWidget { background: transparent; border: none; } "
    "QTableView { background: transparent; border: none; } "
    "QHeaderView { background: transparent; border: none; } "
    "QHeaderView::section { background: transparent; border: none; }"
)
```

A pillar stands on the floor of the space it occupies, at its own fraction of that space, so it reaches the upper graph only at a full reading. The column heading takes the theme's accent colour in both builds, and each mini-table holds its header plus two timeframe rows, with the confidence bars below taking the rest of the height. A summary carrying more timeframes is cut at two rows.

07-indicators.md carries the panel's own entry.



The Trading tab in the Electron shell

The asset class group

HIS.

> *"the large Crypto / Stock Wing button in the upper right of the GUI which will now become a segmented group featuring one layer button per available market sector"*
>
> *"This should be one square segmented into rectangular buttons. You stated you had inferred the meaning. This is not segmenting. This is copy / paste / repurpose. OCIR. Will need to be fixed."*
>
> *"Get these mode buttons corrected. I gave you a properly elaborated spec for how this segmented of the ONE square was supposed to be done and yet, here I am, in the latest build still seeing a STUPID design that the GUI Archetype should have flagged as pure grade shit."*

Functional. The header strip ends in one square at the upper right of the window. Four asset classes divide it into four rectangles, two across and two down, by one vertical line and one horizontal line. The square is 68 pixels on each side and each segment is 34 by 34. Exactly one segment is active, and pressing a segment makes that class active, retitles the Add Exchange button and stores the choice, so the next launch opens on the same class.

A segment past the first column drops the border it shares with the segment to its left, and a segment past the first row drops the border it shares with the segment above, so one line draws between two neighbours rather than two. Each segment rounds exactly one corner, and only the corner it shares with the square; every corner inside the square is square. The active segment fills with its class accent and keeps its own outline, which is what marks it at a glance.

`src/gui/main_tabs/asset_class_surface.py` — `segment_box`

```
said = [f"border: 1px solid {ds.OUTLINE}"]
if column > 0:
    said.append("border-left: none")
if row > 0:
    said.append("border-top: none")
said.append("border-radius: 0px")
if top and left:
    said.append(f"border-top-left-radius: {corner}")
```

Design intention. A model that treats its members as a line cannot produce a square, so a segment carries a row and a column rather than a place on a line. `segment_cell` answers that place and `segment_box` reads it, and both builds take the same answer. The side is one declared number, `group_side_px`, so neither build measures its own row height and neither can draw a different square. The segments sit in a `QButtonGroup` set exclusive, so a second active segment is refused before any code of ours runs, and `_build_class_group` builds them from the taxonomy and reads the stored class back.

`src/gui/main_tabs/asset_class_surface.py` — `grid_shape`

```
columns = math.isqrt(held)
if columns * columns < held:
    columns += 1
rows = held // columns + (1 if held % columns else 0)
```

`src/gui/web/header_strip.js` — the square's own style

```
var side = length(model[SIDE]);
var groupStyle = {
  display: "grid",
  gridTemplateColumns: tracks(cell(model[GRID_COLUMNS], 1)),
  gridTemplateRows: tracks(cell(model[GRID_ROWS], 1)),
  width: side,
  height: side
};
```

Nothing is written as four. The count comes from the taxonomy when the group is built, so a class added there gets a segment with no further edit. The columns are the integer square root of the count rounded up, and the rows follow, so the grid is the one nearest to square. The last row's segments share the columns the full rows use, so every row fills the width.

```
count   columns x rows   the last row
1       1 x 1            one segment, rounding all four corners
2       2 x 1            full
3       2 x 2            one segment spanning both columns
4       2 x 2            full
5       3 x 2            two segments spanning 2 and 1 columns
6       3 x 2            full
9       3 x 3            full
12      4 x 3            full
```

`src/trading/ata_spm.py` — the taxonomy the group reads

```
ASSET_CLASSES = (
    CLASS_CRYPTO,
    CLASS_STOCKS,
    CLASS_COMMODITIES,
    CLASS_FOREX,
)
```

OVERTAKEN, and the block above is kept as written. The taxonomy holds six sectors, so the square divides into six rectangles, three across and two down, which is the count table's six row.

OVERTAKEN, and the count table above is kept as written. Every row holds two segments, so the columns no longer follow the count and only the rows do:

```
count   columns x rows   the last row
1       1 x 1            one segment, rounding all four corners
2       2 x 1            full
3       2 x 2            one segment spanning both columns
4       2 x 2            full
5       2 x 3            one segment spanning both columns
6       2 x 3            full
9       2 x 5            one segment spanning both columns
12      2 x 6            full
```

`src/trading/ata_spm.py` — the taxonomy the group reads

```
ASSET_CLASSES = (
    CLASS_CRYPTO,
    CLASS_STOCKS,
    CLASS_COMMODITIES,
    CLASS_FOREX,
    CLASS_INDICES,
    CLASS_FUTURES_PERPS,
)
```

OVERTAKEN, and the Functional paragraph above is kept as written. The side is not 68 pixels and is not a typed figure. It is the column count times the widest sector name's room plus its padding, with a floor of 34 pixels a segment.

Measured offscreen with 172 font families loaded, at ten pixels bold:

sectors	widest name	its room	segment	square side
four	Commodities	44 px	56 px	112 px
six	Futures / Perps	56 px	68 px	204 px

OVERTAKEN, and both tables above are kept as written. The square's side no longer follows the sector count. `GRID_COLUMNS` is two, so every row holds two segments and a further sector adds a row instead of a column; the side is two segments of the room the first four sector names need. Six sectors therefore draw two across and three down, in the side four sectors draw.

The operator set this:

> *"I would prefer if we keep all six Sector buttons in the same square area but just have it divided into six equal rectangles. The current updated version for 2205 makes the area much wider and I do not want height or width of the square area to change."*

Measured on the Windows platform plugin, which is the one the application runs, at ten pixels bold, with the window drawn at 700 and at 900 pixels:

sectors	columns x rows	segment	square side
four	2 x 2	74 x 74 px	148 px
six	2 x 3	74 x 49 px	148 px

The offscreen host reads different widths for the same names, so the table above it is that host's figures and not the application's.

`src/gui/main_tabs/asset_class_surface.py` — the side and the segment

```
def segment_size_px(count: Any = None) -> tuple:
    rows, columns = grid_shape(count)
    if rows <= 0 or columns <= 0:
        return (0, 0)
    side = group_side_px(count)
    width = (side - (columns - 1) * GROUP_SPACING_PX) // columns
    height = (side - (rows - 1) * GROUP_SPACING_PX) // rows
    return (width, height)
```

Three rows of 49 fill 147 of the 148, so `grid_margins_px` carries the pixel left over as a bottom margin and all six rectangles stay the same size.

A sector name too wide for its segment breaks before it is shortened. A name holding a space breaks on the last space that leaves both lines inside the segment, and a name holding none is shortened with an ellipsis as before. Six sectors draw `Futures / Perps` over two lines and `Commodities` as `Commoditi…`, which is what four sectors already draw.

`src/gui/main_tabs/header_strip.py` — `ClassGroupBar._fitted`

```
if whole(full):
    return full
for at in range(len(full) - 1, 0, -1):
    if full[at] != " ":
        continue
    head = full[:at]
    tail = full[at + 1 :]
    if whole(head) and whole(tail):
        return head + "\n" + tail
return metrics.elidedText(full, Qt.ElideRight, room)
```

The React page reads the same figures and applies the same rule: its tracks are `repeat(2, 74px)` by `repeat(3, 49px)` in a 148 pixel box, and a name holding a space takes `white-space: normal` while a name holding none keeps its ellipsis.

`src/gui/web/header_strip.js` — the square's own tracks

```
  gridTemplateColumns: tracks(
    cell(model[GRID_COLUMNS], 1),
    cell(model[MINIMUM_WIDTH])
  ),
  gridTemplateRows: tracks(cell(model[GRID_ROWS], 1), cell(model[SEGMENT_HEIGHT])),
```

The square takes no spare width. Its side is a declared number, so no slot of the header row takes the width the figures leave. Each part draws at the width its own text asks for, down to its floor, and the row inserts empty space before the square, which sits at the row's right end.

`src/gui/main_tabs/header_strip.py` — the Qt insertion

```
  top_row.insertStretch(
      surface.TOP_ROW_ORDER.index(surface.TOP_ROW_SPACER_BEFORE),
      surface.TOP_ROW_SPACER_STRETCH,
  )
```

Design intention. A group that the GUI archetype could not judge is how a separated design survived two builds, so the archetype now carries a rule for it. GUI007 reads every function that names a segment and writes its own border box, and it refuses three shapes: a group that suppresses no shared edge, a branch that rounds more than one corner, and a spacing or gap constant that is not zero. Its fixture pair is `harness_fixtures/gui_archetype/known_good_segmented.py`, which exits zero, and `harness_fixtures/gui_archetype/known_bad_separated.py`, which exits one.

`dev_harness/harness/gui_archetype.py` — `_scan_segmented_group_skin`

```
  shared = any(f"border-{edge}: none" in joined for edge in _SEGMENT_EDGES)
  if not shared:
      findings.append(Finding(..., rule_id="GUI007", ...))
```

The window keeps a model of the group as well as the widgets. The bridge answers `main_window.state` from that model, and the model publishes the same group the strip draws, built from the same function, so one press names any class and nothing carries two answers. Reading the class list at call time removes the class the model could refuse.

`src/gui/main_tabs/main_window_surface.py` — `MainWindowModel.select_asset_class`

```
  key = asset_class_surface.normalise(name)
  self.trading_mode = key
  self.class_buttons = asset_class_surface.class_buttons(key)
  self.trading_stack_index = asset_class_surface.layer_page(key)
  self.window_title = asset_class_surface.window_title(key)
```

The title is one format for every class, and the stack page comes from the taxonomy.

`src/gui/main_tabs/asset_class_surface.py` — `window_title`

```
  return f"Acervator — {display_name(name).upper()} LAYER"
```

class	stack page	window title	what the page draws
Crypto	0	`Acervator — CRYPTO LAYER`	the crypto trading layer
Stock	1	`Acervator — STOCK LAYER`	the equity trading layer
Commodities	2	`Acervator — COMMODITIES LAYER`	the card, reading `Commodities has no trading layer yet.`
Forex	2	`Acervator — FOREX LAYER`	the card, reading `Forex has no trading layer yet.`

A class with no trading layer still takes a layer title, because the title names the active class and not the page. The sentence about the missing layer is the card's own note.

OVERTAKEN, and the table and the paragraph above are kept as written. Commodities draws stack page 2 and Forex page 3, each its own trading layer with its own empty state. The card page sits last at page 4 and no live class reaches it. Read in a live window by pressing each segment:

class	stack page	window title	what the page draws
Crypto	0	`Acervator — CRYPTO LAYER`	`No Crypto Exchanges Configured`
Stock	1	`Acervator — STOCK LAYER`	`No Stock Exchanges Configured`
Commodities	2	`Acervator — COMMODITIES LAYER`	`No Commodities Exchanges Configured`
Forex	3	`Acervator — FOREX LAYER`	`No Forex Exchanges Configured`

A retired class name reaches the class that holds its markets. The taxonomy names the pair, and the group reads that name rather than carrying its own copy, so a selection stored under an older taxonomy still opens on the right class.

`src/trading/ata_spm.py` — the retired names

```
RETIRED_CLASSES = {
    "metals": CLASS_COMMODITIES,
    "energy": CLASS_COMMODITIES,
    RETIRED_CLASS_DERIVATIVES: CLASS_CRYPTO,
}
```

`src/gui/main_tabs/asset_class_surface.py` — `normalise`

```
asked = LEGACY_CLASS_WORDS.get(asked, asked)
asked = RETIRED_CLASSES.get(asked, asked)
```

stored name	the class it opens on	the button reads	its accent
`metals`	commodities	Commodities	`#ffaa00`
`energy`	commodities	Commodities	`#ffaa00`
`derivatives`	crypto	Crypto	`#00ccaa`
`stock`	stocks	Stock	`#6699ff`
`equities`	stocks	Stock	`#6699ff`
a name no class holds	crypto	Crypto	`#00ccaa`

The `derivatives` row and the block above it are overtaken, and both are kept as written. A stored `derivatives` name now opens on Futures / Perps, the sector the contract rule files a contract on a crypto underlying under.

```
# src/trading/ata_spm.py, in RETIRED_CLASSES
    RETIRED_CLASS_DERIVATIVES: contract_class(CLASS_CRYPTO),
```

Design intention. Derivatives names no sector. A futures contract is a form a contract takes, and the class belongs to the thing underneath it, so a future on an index sits with equities and a future on ether sits with crypto. A venue's dated futures and perpetuals are still read, from the same public product list, and each product takes the class of its own underlying.

`src/trading/ata_asset_maps.py` — the placement one futures product takes

```
def futures_placement(product: dict) -> tuple[str, str, str]:
    """The underlying, the asset class and the sector one futures product takes.
```

The design intention above is overtaken in two places, and it is kept as written. A future on a crypto asset now sits with Futures / Perps, and a future on an index sits with Indices. The placement passes each class it reads through `src/trading/ata_spm.py, in contract_class`, the rule every other placement runs, so the scan and the recording file one product under one sector.

```
# src/trading/ata_asset_maps.py, in futures_placement
    root = _root_of(product)
    if not root:
        return "", NO_ASSET_CLASS, ""
    if not _non_crypto(product) and (root in CRYPTO_SYMBOLS or _crypto_sector(root)):
        return root, contract_class(CLASS_CRYPTO), _crypto_sector(root)
    held = NON_CRYPTO_ROOTS.get(root)
    if held is None:
        return root, NO_ASSET_CLASS, ""
    return held[2], contract_class(held[0]), held[1]
```

The naming. The operator's words are "market sector". Under the published standards the top level is the **asset class**, and a sector is the tier below it. GICS names sectors inside equities. S&P GSCI names sectors inside commodities. The group therefore selects an asset class. Sectors stay the tier below, where the ATA-SMP scanner uses them.

OVERTAKEN, and the paragraph above is kept as written. Sector is his word for the top level, and it is his invention rather than a published term.

> *"really mean that as investment product categories...its my invention or renaming"*

Every screen and every page of this manual calls the top level a sector. The code's own names keep the asset-class spelling, because `sector` already names the tier below in the scanner's maps and one word cannot name both tiers.

The choice survives a restart. The class is written into settings on every press and read back when the group is built.

`src/core/settings.py` — the stored field

```
active_asset_class: str = "crypto"
```

The Add Exchange button follows the class

Functional. The button's text, its tooltip and whether it can act all come from the active class, and each is read from one place. A class with no configured venue says so and refuses the press; it does not offer another class's venue list.

`src/gui/main_tabs/asset_class_surface.py` — `add_exchange_label`

```
state = class_state(name)
if not state["served"]:
    return EMPTY_LABEL.format(name=state["name"])
return f"{ADD_PREFIX} {state['name']} {state['noun']}"
```

class	the button reads	it can act
Crypto	■ `Add Crypto Exchange`	yes
Stock	■ `Add Stock Broker`	yes
Commodities	`Commodities — no venue yet`	no
Forex	`Forex — no venue yet`	no

`src/gui/main_tabs/asset_class_surface.py` — the notes a class draws

```
NO_VENUE_NOTE = "No configured venue serves {name} yet."
NO_LAYER_NOTE = "{name} has no trading layer yet."
```

A venue may serve more than one class. The active class filters the venue list; it never owns it.

`src/gui/main_tabs/asset_class_surface.py` — `venue_classes`

```
found = set(EXTRA_VENUE_CLASSES.get(asked, ()))
if asked in EQUITY_VENUES:
    found.add("stocks")
elif asked in crypto_venues():
    found.add("crypto")
```

OVERTAKEN, and the block above is kept as written. Two lines of it changed. A venue id now takes every class it serves rather than stopping at the first, and a class name an earlier taxonomy used is resolved onto the live class holding it instead of being dropped.

`src/gui/main_tabs/asset_class_surface.py` — `venue_classes` today

```
found = {retired_onto(one) for one in EXTRA_VENUE_CLASSES.get(asked, ())}
if asked in EQUITY_VENUES:
    found.add("stocks")
if asked in crypto_venues():
    found.add("crypto")
```

The block above is overtaken, and it is kept as written. A venue is now listed only under a sector where it trades a market. Stocks comes from the equity list only for an id a broker connector answers, and a broker that names its own sectors bounds what the rest adds.

```
# src/gui/main_tabs/asset_class_surface.py, in venue_classes
    found = {retired_onto(one) for one in EXTRA_VENUE_CLASSES.get(asked, ())}
    served = broker_served(asked)
    if asked in EQUITY_VENUES and served is not None:
        found.add("stocks")
    held = recorded if isinstance(recorded, dict) else recorded_venue_classes()
    found |= set(held.get(asked, frozenset()))
    if served:
        found &= set(served)
    if asked in crypto_venues():
        found.add("crypto")
```

OVERTAKEN, and the table above is kept as written. Coinbase serves three classes, so the Commodities button can act and the table's last two rows read differently.

class	the button reads	it can act	venues
Crypto	■ `Add Crypto Exchange`	yes	15
Stock	■ `Add Stock Broker`	yes	10
Commodities	■ `Add Commodities Exchange`	yes	1
Forex	`Forex — no venue yet`	no	0

Coinbase is the first venue on three layers at once. Its tab is seated on the Crypto layer, the Stock layer and the Commodities layer, and each layer drops its Get Started card when the tab arrives. Pressing a class moves the Live stack to that class's own layer.

`src/gui/main_window.py` — the layers one venue's tab is seated on

```
wanted = [
    name
    for name in self._served_layers(exchange_id)
    if name in layers and exchange_id not in layers[name]["exchange_tabs"]
]
```

Forex draws its empty card still. No connected venue serves a currency pair, so the Forex layer keeps `No Forex Exchanges Configured` and its button refuses the press.

One venue list, read from one place. Nine equity venue ids are declared once and eight files name them. Order matters to the pages and membership matters to the layers: three view models publish the list to a page, so it is sorted where the Live view model reads it and every launch gives the pages the same nine in the same

order. Which layer a venue is put on is decided by membership alone.

`src/gui/main_tabs/asset_class_surface.py` — the one declaration

```
EQUITY_VENUES = frozenset(
    {
        "alpaca",
        "ibkr",
        "schwab",
        "tdameritrade",
        "webull",
        "tastytrade",
        "fidelity",
        "etrade",
        "interactivebrokers",
    }
)
```

OVERTAKEN, and the code block above is kept as written. `EQUITY_VENUES` no longer names `fidelity`, and Fidelity is offered on no tab.

`src/gui/main_tabs/trading_tab_surface.py` — the order the pages are given

```
EQUITY_EXCHANGE_IDS = tuple(sorted(EQUITY_VENUES))
```

what reads the list	what it decides
the Live view model	the layer each venue is published under
the Paper and Simulator view models	the same, on their own pages
the Qt Live tab	the layer a venue tab is seated on
`src/gui/simulator/sim_trading_tab.py`	the layer a venue page is seated on
the Settings dialog	whether a venue's row is an equity row

Two of the nine trade nowhere today. One closed its interface on 10 May 2024 and one publishes none, both recorded on 15-venue-compatibility.md. Both stay on the list, because this section records where the list lives and not what is on it.

The paragraph above is overtaken in part, and it is kept as written. Both ids stay on the list. The one whose interface closed is now listed under no sector, because no broker connector answers it, and the second Interactive Brokers id reads the same. The firm that publishes no interface stays under its four sectors, and whether a firm nothing can trade through belongs on any tab is the operator's decision.

OVERTAKEN, and the paragraph above is kept as written. Fidelity, the firm that publishes no interface, has left the list and is offered on no tab.

A stored venue that no sector lists now sits on no layer, on the Live, Paper and Simulator tabs in both builds. That covers an entry stored for Fidelity, for TD Ameritrade's id or for the second Interactive Brokers id. The Settings Exchanges page still draws each such entry, so Remove Selected can delete it.

```
# src/gui/main_tabs/trading_tab_surface.py, in layer_exchanges
        for layer in exchange_layers(exchange_id):
            split[layer][exchange_id] = exchange_display_name(holder)
```

```
# src/gui/main_window.py, in MainWindow._served_layers
            return exchange_layers(exchange_id)
```

```
# src/gui/paper/paper_trading_tab.py, in PaperTradingTab.add_exchange_tab
        if not exchange_layers(exchange_id):
            return
```

Read with the home redirected and no venue contacted, one stored entry each:

Stored venue	Live, React Paper and React Simulator, before	Qt Paper and Qt Simulator, before	Now, in every build
fidelity	Stocks, Crypto, Commodities, Indices	Stock	no layer
tdameritrade	Crypto	Stock	no layer
interactivebrokers	Crypto	Stock	no layer
alpaca, coinbase, kraken	each venue's own layers	each venue's own layer	unchanged

Every tab but Status and Console follows the active asset class

Functional. Pressing a class segment filters the whole window. The bar carries nine tabs, and seven of them show only the exchanges, the bots, the charts and the rows of the class on show. Status and Console are left alone.

```
Sim   Paper   Live   Charts   Inspector   Swarm   History   Status   Console
```

tab	what it shows for the active class
Sim	the simulated fleet and the venue pages whose venue serves the class
Paper	the paper fleet and its venue pages, on the same rule
Live	the exchange sub-tabs whose venue serves the class, and inside each the bots trading a market of that class
Charts	one chart per bot of the class, so the asset list holds only those
Inspector	the ATA-SPM zone's own class follows the group, and the active-symbol list holds only the class's bots
Swarm	one node per bot of the class
History	one row per fill on a market of the class
Status	nothing an asset class owns. It reports emitters and the process, and it never filters
Console	nothing an asset class owns. It is a log tail, and it never filters

`src/gui/main_tabs/class_filter_tab.py` — the one method every press reaches

```
def select_asset_class(self, name: Any) -> None:
    super().select_asset_class(name)
    self._apply_asset_class()
```

A bot belongs to the class of the market it trades. A status carries the venue it trades on and the market it trades, and those two answer the class.

`src/gui/main_tabs/class_filter_surface.py` — `market_class`

```
served = venue_classes(venue_id)
if len(served) == 1:
    return next(iter(served))
if DERIVATIVES in served and is_derivative(symbol):
    return DERIVATIVES
```

A sector never replaces a tab. Every filtered tab holds its own page under every sector. A sector with no bot draws the same tab, with its lists empty. `src/gui/main_tabs/class_filter_tab.py, in _refilter_class_tabs` narrows the Live sub-tabs and the rows the History, the Sim and the Paper tab hold. It replaces no page, hides no page and disables no page.

Read on the running window, with one bot standing in for a fleet on Crypto and none on the other five sectors. Each cell counts the controls on the page that tab holds, every one of them enabled and none of them hidden.

sector	Live	Sim	Paper	Charts	Inspector	Swarm	History
Crypto	216	21	20	3	33	13	10
Stock	216	21	20	2	33	13	10
Commodities	216	21	20	2	33	13	10
Forex	216	21	20	2	33	13	10
Indices	216	21	20	2	33	13	10
Futures / Perps	216	21	20	2	33	13	10

Six of the seven columns do not move. The Charts column moves by one, which is the one chart panel the one Crypto bot earns. That is the rows inside a list being filtered, which is the only thing a sector changes.

OVERTAKEN, quoted whole:

> ***A tab with nothing for the class says so.*** *It draws a heading, the thing it has none of, and the way back. Both builds draw that note from the one model* `note_model` *writes, the Qt build in* `EmptyTabQtPanel` *and the React build with* `src/gui/web/class_note.js`*, so neither side holds a word of its own.*
>
> `src/gui/main_tabs/class_filter_surface.py` *— the sentences*

```
EMPTY_HEADING = "{name} — nothing to show"
EMPTY_NOTE = "No {name} {subject} to show."
EMPTY_HINT = (
    "{name} is the active asset class. Press another class button in the "
    "header strip to see that class."
)
```

> ***Two tabs keep a card of their own instead of the note.*** *The Live tab draws* `No Stock Exchanges Configured` *with its Add button on screen while no venue serves the class, and the Paper tab draws its own card the same way, with its Add Exchange button naming the class. The note replaces a tab only when a venue does serve the class and no bot trades it.*

True today: no tab draws a class note. `note_model`, `EMPTY_HEADING`, `empty_hint`, `build_label` and `tab_state` are gone from `src/gui/main_tabs/class_filter_surface.py`, and `EMPTY_HINT` named no symbol in that file even while the quoted passage stood. `empty_note` and `EMPTY_NOTE` remain for one caller only, `tools/capture_screen_figure.py, in class_note`, which looks for a label no tab draws any more and refuses the capture when it finds none. The Live tab still draws its own `No Stock Exchanges Configured` card with its Add button on it while no venue serves the sector, and the Paper tab still draws its own card the same way. Both are pages those tabs own, and neither is a note that replaces a tab.

OVERTAKEN, quoted whole:

> *True today: no tab draws a class note.*

True today: every one of the seven class-following tabs draws its own note. `src/gui/main_tabs/class_filter_tab.py, in ClassFilterTabMixin._draw_class_notes` seats one label on each tab's own page and shows it only while that tab keeps no row of the active sector. `class_filter_surface.empty_note` still writes the sentence, so the Sim tab reads "No Crypto simulated bot to show." and the Inspector reads "No Crypto scanned market to show.". Status and Console carry no subject in `TAB_SUBJECTS` and get no note.

A tab with no note is now refused before any label is read. The sentence above says which tabs get no note. The figure helper asked for the sentence, received an empty string for those tabs, and then matched a visible label against it, so any blank label on the page answered as the note. The helper now refuses the capture as soon as the sentence comes back empty, naming the tab.

`tools/capture_screen_figure.py`, in `class_note`

```
    if not wanted.strip():
        raise CaptureRefused(
```

Driven offscreen on a page carrying one blank label. Asked for the Trading tab, which carries no subject, the helper returned that blank label before and refuses now. The two controls read the same on both sides: the real note on a class tab is still found, and a class tab drawing no note is still refused for drawing none.

What the page held	Tab asked for	Before	Now
one blank label	Trading	returned the blank label	refused, naming the tab
the real note and a blank label	Live	returned the note	returned the note
no note	Live	refused	refused

The note does not replace a page. It is one line at the top of the page the tab already owns, so the Live and Paper cards named above still draw beside it.

Driven offscreen with three records built by hand on one venue, in both builds, under each of the six sectors:

```
                  crypto   the other five sectors
Live, Charts,      content  note
Inspector, Swarm
Sim, Paper,        note     note
History
Status, Console    none     none
```

The Qt build and the React build answer that table cell for cell. Under crypto the four fleet-fed tabs draw the records and no note, which is what makes the reading a measurement rather than a tab that failed to draw.

A filter hides rows and changes nothing else. No bot is stopped, no position is closed and no state is written by a press, and the class survives a restart: a fresh window with a class stored draws every one of the seven tabs for that class at first draw, with no button pressed.

Which build draws the Live tab

The Live tab picks its page from the variant seam. `src/gui/variant_surface.py` registers the screen `TRADING` and holds two loaders for it: `_qt_trading` returns the Qt page and `_react_trading` returns `TradingTabReact`. `src/gui/main_tabs/trading_tab.py` builds the Qt page on every start, and `draws_react` then decides which page the tab shows. Under the Qt build the tab shows that page. Under the React build `_react_trading_page` makes the Qt page a hidden child of the React page and shows the React page.

The Qt page keeps every widget the main window writes to. The window writes to its status log and to `_indicator_panel` under both builds, so neither build may skip the Qt build step.

`src/gui/react_trading_tab.py` — the page the React build shows

```
def models(live: Any = None) -> dict:
    tab = (
        trading_tab_surface.bind_live(live)
        if live is not None
        else trading_tab_surface.view_model
    )
    return {
        trading_tab_surface.METHOD: tab({}),
        status_log_surface.METHOD: status_log_surface.view_model({}),
        indicator_panel_surface.METHOD: indicator_panel_surface.view_model({}),
    }
```

`TradingTabReact` reads the same bridge method the Qt tab reads, which is `trading.tab`, and `bind_live` serves it from the running program's own exchange list. The page fetches nothing over the network: `panel_html` inlines every module and every stylesheet into the page head, and because a module cannot read its own file name off an inlined script tag, `marker_script` names the module whose tag comes next and `namer_script` hands that name to `panel_host.js`. `react_trading_tab.STYLE_ASSETS` names the two stylesheets, `trading_tab.css` and `exchange_tab.css`, and `react_trading_tab.roster` puts the four style-source modules ahead of the panel modules.

The two builds paint different grounds. The Qt page paints `#2d2d2d` and the React page paints `#0a0a0f`.

What the React Live page draws

The React Live page carries seven child modules and two stylesheets, all inlined.

`src/gui/react_trading_tab.py` — the modules the page carries

```
CHILD_MODULES: tuple[str, ...] = (
    "status_log.js",
    "indicator_panel.js",
    "table_cells.js",
    "bot_status_table.js",
    "extractor_bot_table.js",
    "crypto_news_ticker.js",
    "exchange_tab.js",
)
```

`src/gui/web/trading_tab.css` gives the page its chrome: colour, border and type. `trading_tab.js` writes every layout value on the element's own style attribute from the payload, and it mounts the voting panel itself, keeping a slot for the panel and handing the slot to the panel host. The variant seam therefore registers no screen for the voting panel, and none is needed.

The page also carries four style-source modules ahead of the panel modules — `design_tokens.js`, `theme_engine.js`, `shared_widgets.js` and `header_strip.js`. `trading_tab.js` parses every Qt style sheet in its payload with `styleOf`, which lives in the last of those, so a page without them paints no colour the payload asks for. `checkedStyle` reads the active fill out of the state blocks of that sheet, which a reader taking the plain block alone would leave behind.

One exchange layer shows at a time. A stack page carries a flex display on its own style attribute, and the stylesheet marks a hidden page `display:none`, which is what keeps the Crypto layer and the Stock layer from drawing one above the other. The empty-state card takes its ground and its edge from the stylesheet, because Qt counts the alpha of both in bytes and `keptSheet` drops a byte alpha from the payload, so only the card's corner radius survives the trip. Two buttons add an exchange: the corner button beside the tabs is plain chrome and the card button below it carries the layer accent, which `placeholder_add_style` publishes, and the stylesheet paints the corner button only, so the two read as different controls. Each log pane is one bordered ground, and each splitter handle carries the border colour so the pointer can see what it takes hold of.

The voting panel's cells. The panel draws one row per timeframe and one column per indicator, and each cell prints its vote direction as an arrow and its confidence as a percentage. Green is bullish and red is bearish. `indicator_panel_surface.indicator_cell_colors` publishes each cell's text colour and its tint and the page writes both on the cell, so no rule in the stylesheet sets a colour on a cell. The stylesheet gives the head cells a ground of their own and paints their text with the page variable `--accent`, which carries the theme's accent for all five themes, so the two builds draw a column heading in one colour. It clips a cell wider than its column, so a narrow panel shows an ellipsis instead of running one vote over the column beside it. The Net, Comp and Conf cells wrap instead of clipping, because the Conf cell holds a ten-block bar and a percentage: the bar takes the first line and the percentage takes the second.

`src/gui/web/trading_tab.css` — the cells that wrap

```
[data-part="indicator-body-cell"][data-state="net"],
[data-part="indicator-body-cell"][data-state="comp"],
[data-part="indicator-body-cell"][data-state="conf"] {
  white-space: normal;
  overflow-wrap: anywhere;
  text-overflow: clip;
  line-height: 1.1;
}
```

The bars arrive settled. The bar model starts every bar at zero and steps it toward its target. The Qt widget runs its own frame timer; the page draws the frame it is handed and runs no timer, so the window hands it a settled one. The two published factors answer forty-nine frames, and the panel pushes that many steps before it publishes.

`src/gui/react_trading_tab.py` — `settled_frames`

```
    return math.ceil(
        math.log(surface.BARS_SETTLE_DELTA) / math.log(1.0 - surface.BARS_LERP_FACTOR)
    )
```

An ornament decides from the drawn height. `ConfidenceBarsWidget.paintEvent` tests the bar it painted, in pixels. The page measures the graph after it draws, draws again at the height it found, and watches for a resize, so a taller graph keeps the arrows the window keeps. Each mini-table keeps the height Qt fixes it to with `setFixedHeight`, which is its header plus two rows of slack, so a table holding one row is the same height as one holding three.

`src/gui/web/indicator_panel.js` — `renderPanel`

Eleven published values reach the page and set the ornaments and the type:

```
arrow_min_height_px  the bar height an arrow needs
shine_min_height_px  the bar height a highlight needs
shine_limit_px       the tallest a highlight may grow
glow_inset_px        the halo width beside a bar and beside a pillar
column_min_pad_px    the narrowest pad beside a bar
label_font           the face a bar name and a pillar name take
arrow_font           the face and the size a bar arrow takes
empty_font           the face the empty-graph note takes
label_height_px      the box a bar name centres in
label_offset_px      how far that box sits under the baseline
margin_left_px       where the grid and the baseline start
```

What each engine still sizes for itself. No published value names the header row's height or a mini-table's height, so each engine sizes those from its own font. The panel publishes ten heights and none of them names the header row.

The venue page inside the React Live page

The React build draws the venue page inside the Live page rather than in a window of its own. `trading_tab.js` draws one empty box per layer and marks it as the exchange pane; `mountExchanges` finds that box and calls `renderExchangePane`, and `exchange_tab.js` draws the whole venue into it. The pane takes the height of the tab body, the two bot tables scroll inside it, and the command bar keeps its own height and stays in view, which is what the Qt build does. The Qt build instead builds one `ExchangeTab` widget per venue and puts it in a tab strip.

`src/gui/web/trading_tab.css` — the boxes that take the pane's height

```
[data-part="stack"],
[data-part="page"],
[data-part="tab-widget"],
[data-part="tab-body"],
[data-part="exchange-pane"] {
  min-height: 0;
}
```

One fleet load reaches the page once. The window builds one `ExchangeTabReact` per venue and hands every fleet load to it. That object rebuilds five payloads and announces them; `TradingTabReact.hold_venue` takes the object, follows that announcement, rebuilds the venue payload and pushes it into the page. One fleet load, one source, one draw. A venue module names its exchange in its request and the page answers from the bag it holds for that exchange, so two venues on two layers read two different fleets from one page.

`src/gui/react_trading_tab.py` — the Live page follows one venue

```
def hold_venue(self, venue: Any) -> bool:
    """Draw ``venue`` in this tab and follow every payload it publishes."""
    exchange_id = str(getattr(venue, "exchange_id", "") or "")
    published = getattr(venue, "published", None)
    if not exchange_id or published is None:
        return False
    self._venues[exchange_id] = venue
    published.connect(self._venue_published)
    self._venue_published()
    return True
```

The header strip's own pages paint their own ground. The five stat cards, the money strip and the tab bar carry no style sheet. `page_html` writes six chrome colours on the root element and paints nothing, and each of the three pages paints its own ground from them. A stat card paints its face from the button ground, every page paints its body from the theme ground, and no page scrolls, so no scroll bar draws.

`src/gui/react_dashboard_stat_card.py` — one card's ground

```
PAGE_STYLE = (
    "*{margin:0;padding:0;box-sizing:border-box}"
    "html,body{height:100%;overflow:hidden}"
    "body{background:var(--bg);color:var(--text)}"
    f"#{CARD_ROOT_ID}{{height:100%}}"
    '[data-part="card"]{height:100%;background:var(--btn-bg)}'
)
```

Every Live tab feed reaches the React page

Six modules write to the Activity Log and each calls the Qt pane directly. The pane reports every call to one listener, and the window hands that listener to the React tab when the tab is built, so a pause, a resume and a watchdog line all arrive on both panes. The Qt pane paints through the same surface the React pane reads, asking it for the shape and for the line rather than building either itself, so the two panes cannot disagree about what a message looks like.

`src/gui/widgets/status_log.py` — `set_relay`

```
        def set_relay(self, relay) -> None:
            """Take the callable every ``log``, ``pause``, ``resume`` and
            ``notice`` call is reported to."""
            self._relay = relay
```

The API Interaction Log takes the same route: `MainWindow._on_api_event` builds one text block per API call and hands the block to the React tab first, so a paused pane holds it and a running pane paints it. Both panes keep the same capped block buffer, so both drop the same oldest block.

`src/gui/main_window.py` — `_on_api_event`

```
            block_text = "\n".join(plain_lines)
            self._push_live_tab({"api_lines": [block_text]})
```

The data-pool line under the New Bot button is refreshed by a one-second timer on both venue pages, and the publish carries the fresh payload to the venue's own page and to the Live page beside it.

`src/gui/react_exchange_tab.py` — `_update_pull_rate_label`

```
        def _update_pull_rate_label(self) -> None:
            if self._stopped:
                return
            self._screen.update_pull_rate_label()
            self._publish()
```

The voting panel's two header lines travel with the cells. One says the reading on screen is the last one taken and how old it is; the other prints the A15 and A14 spot rates. The dashboard tick calls `_publish_votes` with the fleet, the selected bot and the reading it has just given the Qt panel, and `react_trading_tab.votes_payload` turns that reading into the panel payload, so both panels show one bot and one set of cells. The Qt panel formats the age once and hands the result over, so the two banners cannot drift apart.

`src/gui/indicator_panel.py` — `panel_reading`

```
            if self._showing_stored and self._shown_stored:
                read.update(self._shown_stored)
            if self._rates_seen:
                read["rates"] = self._rate_snapshot
```

The panes and the tables divide the page's height from one declaration. The main splitter's four sizes were four literals in the Qt page and are now read from the surface, so one declaration sets both builds. The two log panes take under a quarter of the page and the tables take the rest.

`src/gui/main_tabs/trading_tab_surface.py` — `MAIN_SPLITTER`

```
MAIN_SPLITTER = {
    "orientation": "vertical",
    "handle_width_px": HANDLE_WIDTH_PX,
    "children_collapsible": False,
    "children": ["top_splitter", "bottom_splitter"],
    "sizes_px": [660, 190],
}
```

The voting panel's two graphs divide whatever height is left after the two tables, and the host the tab keeps for the panel is a flex column with no minimum, so the panel measures itself against the space rather than against its own content.

`src/gui/web/trading_tab.js` — the panel host

```
    var panelProps = {
      style: {
        flex: AUTO,
        overflow: AUTO,
        display: FLEX,
        flexDirection: COLUMN,
        minHeight: ZERO
      }
    };
```

The shared candle cache behind the panel

HIS.

> *"A19 stuck at 13...A24 stuck at 11...probably others...need to isolate the issue."*

The shared candle cache holds one slot for each exchange, symbol and timeframe. A slot answers a request while it is fresh and holds as many bars as the request asked for, or while it holds every row the venue has. A slot records how many bars the venue supplied whenever that is fewer than the number asked for, so a market that cannot supply the count is not re-fetched on every call.

`src/exchange/data_pool.py` — the one test both call sites share

```
    def can_serve(self, limit: int) -> bool:
        """True when this slot is fresh and holds ``limit`` rows, or holds every row
        ``venue_row_cap`` says the venue has."""
        if self.is_stale or not self.candles:
            return False
        if len(self.candles) >= limit:
            return True
        return (
            self.venue_row_cap is not None and len(self.candles) >= self.venue_row_cap
        )
```

A fetch grows the series instead of replacing it. A fetch merges what came back into the bars already held, keeps one row for each bar time, orders them oldest first, and keeps the newest rows up to the number the pool asked for. A venue that answers newest first, and a venue that repeats a bar time inside one answer, both land in time order with no repeat.

`src/exchange/data_pool.py` — the merge

```
def _merge_candle_rows(stored: list, fetched: list) -> Optional[list]:
    """One row per timestamp from ``stored`` then ``fetched``, oldest first, None
    when a row carries no finite ``row[0]``."""
    by_time: dict[float, list] = {}
    for row in list(stored) + list(fetched):
        at = _row_timestamp(row)
        if at is None:
            return None
        by_time[at] = row
    return [by_time[at] for at in sorted(by_time)]
```

One venue call for each slot life. A slot lives for the number of seconds its own timeframe names, which is three hundred at five minutes. A market that stays short is called once inside that window rather than once for each request, the same rate every other market already had. Nothing new sets this rate; it is the slot's own life.

`src/exchange/data_pool.py` — where the rate comes from

```
    @property
    def ttl_seconds(self) -> float:
        return TF_SECONDS.get(self.timeframe, 3600)

    @property
    def is_stale(self) -> bool:
        return self.age_seconds > self.ttl_seconds
```

A short answer is not a short history. The venue answers with the periods that traded inside a window ending now, so a market that trades rarely returns a handful however long it has been listed. A slot the venue cannot fill therefore asks for the window immediately before the oldest bar it holds. Each refresh adds older bars, the merge folds them in on bar time, and the stored series grows until it holds what the engine needs.

`src/exchange/data_pool.py` — the window a short slot asks for next

```
    def next_reach_back_ms(self, requested: int) -> Optional[int]:
        """Epoch milliseconds of the window before the oldest row or last
        ``reach_back_since``, recorded, None at ``MIN_CANDLES_FOR_TA`` rows or past
        ``REACH_BACK_PERIODS``."""
```

What bounds the reach back. Two figures already in the tree set the span and no third figure was chosen: the engine's floor, and the length of one period from the pool's own table of timeframes. The pool reaches back thirty periods for each of the thirty bars the engine needs, which is nine hundred periods, or seventy-five hours at five minutes. A market trading in fewer than one period in thirty cannot reach the floor inside that span; at that point the pool stops reaching back, keeps the bars it has, and the panel prints the sentence it printed before.

`src/exchange/data_pool.py` — the bound, built from the floor and the period

```
REACH_BACK_PERIODS = MIN_CANDLES_FOR_TA * MIN_CANDLES_FOR_TA
```

A market that fills the count carries no short-answer record, so no start time is computed for it and none is sent.

Bot creation and a restart read one declaration

Functional. One function selects the settings a new bot is built with, and it reads the dataclass that declares them rather than a list of names. It keeps every key that names a field the config declares and drops the keys that belong to the other mode, which is the same rule the factory applies when it refuses one. The restart path calls the same function, so a field added to the declaration reaches both paths with no second edit.

`src/trading/container/config.py` — `bot_config_kwargs`, the selection

```
foreign = (
    _BOT_CONFIG_SCRUMMING_ONLY_FIELDS
    if mode == BotMode.EXTRACTOR
    else _BOT_CONFIG_EXTRACTOR_ONLY_FIELDS
)
# make_bot_config sets mode itself and raises on a foreign field.
carried = {f.name for f in fields(BotConfig)} - foreign - {"mode"}
```

`src/trading/container/restore.py` — the whole selection

```
_kwargs = bot_config_kwargs(mode, cfg, exchange_id=cfg["exchange_id"])
if "stack_mode" not in cfg:
    _kwargs["stack_mode"] = STACK_MODE_DEFAULT
```

Every place the field set is declared. The three sets partition the dataclass exactly: their union holds every name, no field sits outside them, and no name in them is absent from the dataclass.

```
BotConfig                              54 fields
_BOT_CONFIG_SHARED_FIELDS              16
_BOT_CONFIG_SCRUMMING_ONLY_FIELDS      33
_BOT_CONFIG_EXTRACTOR_ONLY_FIELDS       5
```

Two defaults the restart path keeps as its own. Stack Mode has two defaults, and they answer two different questions. Creation reads a retired Grid checkbox and treats its absence as off. A saved record never holds

that key, so an absent Stack Mode on a restart takes the declared default instead.

```
creation   bulk_trading absent   stack_mode False
restore    stack_mode absent     stack_mode True, STACK_MODE_DEFAULT
```

An Extractor whose record holds a pool figure and no Target Balance restores to that pool figure, which is what creation gives such a bot. Two writers reach the saved fleet file and neither can write a configuration section short of a field — one writes the whole dataclass and the other loads the file, edits a wire route and writes the file back — so a record the program wrote always carries Target Balance.

```
src/trading/bot_container.py     "config": asdict(self.config)
src/gui/bot_visualizer.py        loads, edits one route, writes back
```

A stored null on a quantity or on a list reaches a number and an empty list rather than a null.

```
if "inverted_extractor_standing_alt_units" in _kwargs:
    # A stored null reaches float() as 0.0.
    _kwargs["inverted_extractor_standing_alt_units"] = float(
        _kwargs["inverted_extractor_standing_alt_units"] or 0.0
    )
```

The block above is overtaken. The setting it coerces left the product under his ruling of 13 September 2026, and the name reads **zero** occurrences across the source, the Electron build, the tooling and the entry point. The sanitiser the restore path calls today drops every retired key and promotes one retired spacing word, and it coerces no quantity.

`src/trading/container/config.py` — `_sanitize_deprecated_kwargs`

```
out = {k: v for k, v in kwargs.items() if k not in _DEPRECATED_KWARGS}
if out.get("stack_spacing_mode") == "logarithmic":
    out["stack_spacing_mode"] = "quadratic"
return out
```

One bad stored value stops one bot. The build of a bot's configuration sits inside the handler written for it. A record holding a boolean where a list belongs, or a word where a quantity belongs, is one skipped bot, named in the restart ledger, and the fleet around it comes back.

Two settings the wizard collects that a new bot cannot read.

setting	why
Lock duration (candles)	`lock_candle_count` is no field on `BotConfig` and no argument of `ScrummingBot.__init__`. The phantom coordinator holds an attribute of that name, and Bot Settings writes it there on a running bot. The coordinator has no method that reads that attribute, so the figure changes no decision on a running bot either.
Profit Folding, on an Extractor	`BotCreationWizard.get_bot_config` writes the flag False for an Extractor. The field is Scrumming-only, so the factory refuses it on an Extractor config, and no control types it.

The lock row reaches a new bot as well. `lock_candle_count` is a keyword argument of `ScrummingBot.__init__`, and `phantom_init_kwargs` in `src/trading/container/config.py` carries it beside the other two, so the figure the phantom page was set to becomes the new bot's coordinator count. The Profit Folding row above stands as written: the field is Scrumming-only and no control types it.

Two the wizard collects that a new bot does read. `enable_phantoms` and `phantom_timeframes` reach `ScrummingBot.__init__` as arguments from `src/gui/main_window.py`, so the figure on the phantom page is the figure the new bot opens on. The selector drops both, and the constructor call carries them instead.

The Profit Folding Active row on a running bot

The creation wizard's own group does not carry this row. The running bot's Scrumming Settings group does, and so does the row list the shell draws from.

	the control
Where	`src/gui/live_settings/settings_tab.py`
React row	`src/gui/main_tabs/live_settings_tab_surface.py`
Kind	checkbox, on at the start
What it writes	the bot config key the engine reads seven times: three decide a fold, three write a log line, one fills a snapshot

```
on   surplus $10.00 applied $2.00   target $200.00 became $202.00   preview $2.00
off  surplus $10.00 applied $0.00   target stayed $200.00           preview $0.00
```

Profit Folding Active - When it is on, a fold's surplus may raise Target Balance up to the Max Target Growth % cap. When it is off, the surplus is not applied, the preview of a prospective fold reads zero, and the bot writes a skipped line naming the flag. A bot restored from its own record takes its own flag and reads no application setting.

`src/trading/scrumming_bot.py` — the line the off state emits

```
f"[COMPOUND SKIPPED] ({source}): "
f"profit_folding_active=False — the "
f"compound-growth feature is off for "
f"this bot. No target bump."
```

No wizard holds a checkbox for this flag. Each of the four wizard builds names `profit_folding_active` once, and every one of those writes is the Extractor branch switching it off. A new Scrumming bot therefore opens on the declared default, which is on, and this row is the only control over it.

The Tranche Despawn Timer

The timer is the one row of the Hedge Rebalance group that a new bot cannot be given: neither wizard build carries it, so a new bot opens at the declared default of zero, which is off. The label on both screens reads Tranche Despawn Timer, and the control runs from 0 to 365 days with zero drawn as "Off".

```
src/gui/live_settings/settings_tab.py            the one control, 0 to 365 days
src/gui/main_tabs/live_settings_tab_surface.py   the React row for it
```

The engine reads the stored figure exactly once, through one shared helper, and the sweep that uses it runs once per tick outside every exception handler. Every shape the control cannot type reads as off rather than as a number: a negative figure, a boolean, a string, an absent field, a not-a-number and an infinity all read zero, and a fractional figure truncates.

`src/trading/container/config.py` — the one reading

```
def despawn_threshold_days(config) -> int:
    days = as_finite_float(getattr(config, "tranche_despawn_days", 0))
    if days is None:
        return 0
    return min(DESPAWN_MAX_DAYS, max(0, int(days)))
```

What the tooltip promises, and what the sweep does. The threshold is inclusive, so a record at exactly the threshold goes and a record one second younger stays.

The claim	What holds
Despawn is not a trade	No call reaches the exchange object, at any threshold
No order is placed or cancelled	The sweep calls none of the order methods
Holdings and cost basis are untouched	Holdings and both cost-basis lots are identical before and after
A tranche with no timestamp is never despawned	An ageless record survives at a one-day threshold
A stack tranche holding a resting order is kept until it settles	A pending record with an order id is kept at four hundred days, and removed once filled or cancelled

The target balance, the anchor and the hedge reserve are unchanged too, which the line the operator reads already claims. The queue total is recomputed to match what is left rather than being decremented.

The word a removal uses. Merge, despawn and clear are the only three things that collapse or remove a tranche, and despawn removes rather than delists. The sweep's own report and the line the operator reads both say removed, and the sweep's key set is a subset of the preview's, so either report can be read by one consumer. 08-tabs/bot-swarm.md carries the block.

```
sweep     ageless_kept  fold_removed  stack_kept_live_order  stack_removed
          threshold_days  usd_removed
preview   the same six, plus fold_open, stack_open and units_removed
```

The predicate is written three times: once in the sweep that removes, once in the shared preview the Qt panel reads, and once inside the React surface. All three answer the same counts and the same dollars on every row. A third copy of a predicate is a drift hazard rather than a present fault, and collapsing it would reach files other rows own.

The accumulation ceiling and the capital claim

The adoption ceiling comes from Target Balance. A bot that has never scrummed adopts the holding already sitting on the venue as its opening position, and a dollar ceiling bounds what it may take. That ceiling is Target Balance, the figure the operator types, so it is set from two screens: the creation wizard's row and the live Bot Settings page.

`src/trading/scrumming/tick_phases.py` — the ceiling, in `_tick_initialise`

```
_cap_usd = float(self._target_balance or 0.0)
```

When the ceiling holds a bot back, the Activity Log says so and names the control to raise:

```
... units stay unmanaged. Raise this bot's Target Balance to change this.
```

The claim has no off switch. A bot claims the funds it is allowed to work with, which is what stops two bots on one asset from taking each other's money. The claim is unconditional and runs once per tick.

`src/trading/scrumming/capital_reservation_mixin.py` — the claim

```
async def _ensure_capital_reservation(self, current_price: float) -> None:
    if current_price is None or current_price <= 0:
        return
```

One reader of another bot's claim decides a sale. Nothing else in the platform reads a claim, and no screen reads one. With two bots on one asset, thirty units at the venue and the first bot holding eleven of them spoken for, the second bot's permitted sale falls to nineteen units, and a twenty-unit sell is refused with the reservation named.

`src/trading/scrumming/execution.py` — the sell pre-check

```
_crr_effective = _crr_reg.effective_available(
    asset=self.config.target_asset,
    bot_id=self.bot_id,
    total_holdings=float(self._current_holdings or 0),
)
if amount > _crr_effective + 1e-12:
```

A claim leaves the table five ways: its owner calls `release`, a bot drops its own with `release_for`, the expiry sweep calls `prune_expired`, the fleet sweep calls `sweep_unknown_bots`, and a silent bot loses its claim on the heartbeat schedule.

Five places let a bot through with no claim behind it. Each one is a choice to keep trading rather than to stop, and each now writes a line naming what was lost.

where	what fails there	what happens
`src/trading/container/registry.py`, the admission branch	no registry is attached	the bot is admitted, nothing is claimed
the same file, the rate branch	no price for the base currency	the bot is admitted, nothing is claimed
the same file, the error branch	the consult raises	the bot is admitted, nothing is claimed
`src/trading/extractor_bot.py`, the chunk-rate claim	the claim raises	the Extractor runs unclaimed
the Scrumming claim in the reservation mixin	the claim raises	the bot ticks on, its token cleared

The Extractor reads its holdings before it claims. `ExtractorBot._read_base_holdings` asks the venue for the free balance of the base currency and answers a figure, or answers nothing when the read fails. Nothing is not zero, and it is not room to claim: `ExtractorBot.set_initial_chunk_rate` still rebases the chunk, places no claim, and writes one warning naming the bot and the asset. A figure goes to the registry with the claim, so the registry can compare the request against what the bot owns, which is the check the Scrumming bot already passes. `ExtractorBot._has_chunk_capacity` then decides whether the free pool covers one round.

Where the reservation lives. `src/trading/capital_reservation.py` declares `CapitalReservationRegistry`, and six modules read it: the bot, its reservation mixin, the scrumming package, the container restore path, the Extractor and the emitter contract. A bot reads another bot's claim through `reservations_for`, and a sale is decided by `effective_available`. The dollar-denominated registry that once sat beside the asset-unit one is gone, with the Qt table, the view model and the renderer page that drew it; it was built zero times, which is why its removal changed no behaviour. That registry's two operator overrides went with it, so the claim table takes no force-release of any kind.

One connector per venue, shared by every bot on it

A fleet builds one exchange connector for each venue it trades on, not one for each bot. The first bot to start on a venue opens that connection, and every later bot on the same venue is handed the same object. Ten bots across two venues hold two connectors between them. A bot is never handed the connector of a venue it does not trade on.

`src/gui/main_window.py` — `MainWindow._live_connector`

```
held = self._exchange_connectors.get(eid)
if held is None:
    return None
if getattr(held, "is_connected", False):
    return held
```

What one connector is responsible for. It holds the session for its own venue and nothing else. It paces every request against that venue's rate limit, keeps the market list the venue loaded, answers the balance and ticker reads the tab draws, and runs the trade-history scan for each symbol registered on it. The symbols registered on a venue's connector are the symbols of the bots on that venue, and each bot is registered once.

`src/trading/bot_container.py` — `BotManager.set_connector`

```
for bot in self._bots.values():
    if eid is not None and getattr(bot.config, "exchange_id", eid) != eid:
        continue
    candidates += 1
    if bot.bot_id in registered:
        continue
```

A dropped connection is released before a replacement opens. A held connector that no longer reports itself connected is dropped from the venue dictionary and released. Its scan set is cleared, both venue handles are dropped, and its worker pool is shut, so no retired connector keeps scanning behind the live one. The next bot to start on that venue then opens a fresh session.

`src/exchange/ccxt_connector.py` — `CCXTConnector.release`

```
  cleared = len(self._scan_symbols)
  self._scan_symbols.clear()
  self._injected_ex = None
  self._ccxt_sync = None
  self._ccxt = None
  self._connected = False
```

A fleet of ten bots, eight on one venue and two on a second, started in full:

```
  connector objects built                      2
  bots holding their own venue's connector    10
  bots holding another venue's connector       0
  bot registrations performed on a connector  10
  retired connectors still scanning            0
```

One worker serves a connector, so one answer must fill every slot

HIS.

> *"CoinBase API is responding very slow and I am seeing the entire fleet go stale. Want to make sure this is an exchange-side issue and not a bug that has shown up all of a sudden."*
>
> *"No bot should be skipping it. All bots should have current data in order to properly execute trades."*

A connector runs every venue call on one worker thread. The queue in front of that worker holds eight calls, and a ninth caller is refused rather than made to wait. Eight is therefore the size of a waiting room and not how many calls run at once, and the calls a connector can serve in a minute is set by how long the venue takes to answer, not by the request floor the rate meter's ceiling is derived from.

`src/exchange/ccxt_connector.py` — one worker, and the queue in front of it

```
  SYNC_QUEUE_CAP: int = 8

  SYNC_CALL_CONCURRENCY: int = 1
```

One account read names every currency. A venue answers one balance request with the whole account. A cache that stores one currency in a slot and fills one slot per request therefore asks for the same answer once per currency, so a fleet of thirty-nine assets made thirty-nine identical requests for one page of data. The read now writes every slot the exchange holds, and gives a slot to each currency the answer names, so the next reader of any currency the venue holds answers from cache.

`src/exchange/data_pool.py` — one read, every slot

```
    def _fill_balance_entries(self, exchange_id: str, reported: dict) -> int:
        """Write every ``_balances`` slot this exchange holds from one account read.

        Returns how many slots the read filled. A currency the read names gets a
        slot it did not have, so the next reader of any currency the venue holds
        answers from cache instead of making its own account read.
        """
```

A currency the venue reports as empty costs one read, not two. The funded map drops every total at or below zero, so asking it alone cannot tell "the venue said zero" from "the venue did not list it". The account read answers every currency the venue lists, zeros included, and the funded map is now derived from it rather than read separately.

`src/exchange/ccxt_connector.py` — `CCXTConnector.get_account_balances`

```
        for currency, info in (raw.get("total") or {}).items():
            result[currency] = Balance(
                currency=currency,
                free=float((raw.get("free") or {}).get(currency, 0) or 0),
                used=float((raw.get("used") or {}).get(currency, 0) or 0),
                total=float(info or 0),
                absent=False,
            )
```

The refresher lands before the slot it fills expires. A ticker slot lives five seconds and a bot ticks every five seconds, so a slot refilled on a five-second sleep was always expired when a bot asked, and each bot then made its own call. The refresher's period is now half the shortest cache lifetime it fills, and a cycle sleeps the remainder of that period rather than the whole of it, so a slow read does not push the period past the lifetime.

`src/trading/bot_container.py` — the period, derived from the lifetimes

```
TICKER_REFRESH_INTERVAL_S = min(TICKER_TTL_SECONDS, BALANCE_TTL_SECONDS) / 2.0
```

Driven against a stub venue holding each bulk read for 1.6 seconds, with the same thirty-nine symbols a saved fleet carries:

```
                                  before   after
refresher period, observed         7.02s   2.35s
ticker slot reads                   1092    1482
slot reads that came back stale      234       0
worst slot age                     6.40s   2.41s
```

A reported call time is the venue's answer, and the wait is reported beside it. The clock used to start before a call entered the queue, so a call that waited behind seven others reported their time as its own. Driven against a stub answering every call in 150 milliseconds, eight callers at once reported 151, 300, 451, 602, 752, 902, 1052 and 1203 milliseconds — a staircase in which each call carried every call ahead of it. The worker now times the venue's answer alone, and the wait for the worker is carried in its own field.

`src/exchange/ccxt_connector.py` — `SyncCallTiming.resolve`

```
    def resolve(self, submitted_at: float) -> None:
        """Set ``queued_ms`` and ``served_ms`` from the worker's own timestamps."""
        if self.started_at <= 0.0:
            return
        self.queued_ms = max(0.0, (self.started_at - submitted_at) * 1000.0)
        if self.finished_at > 0.0:
            self.served_ms = max(0.0, (self.finished_at - self.started_at) * 1000.0)
```

A refused call is counted where it can be read. A refused call reaches no API log, so a count of logged calls falls when the queue is at its worst and the rate meter read that fall as idleness. The status-line pill now names the call rate, the share of the window a worker was busy, and the calls the queue refused, and its colour follows the worse of the two loads.

`src/gui/main_tabs/main_window_surface.py` — what the pill says

```
API_PILL_TEXT_FORMAT = (
    "API {exchange}: {calls_per_minute:.0f} CPM, worker {worker_busy_pct:.0f} %"
)
API_PILL_REFUSED_FORMAT = ", {queue_refusals} refused"
```

What the stale marker measures. A price cell is marked stale when the shared ticker slot behind it is more than twenty seconds old. It is a per-bot age on a shared slot, so the whole fleet reads stale together whenever the one read that refills every slot is itself refused.

`src/gui/main_tabs/table_cells_surface.py` — the threshold and the marker

```
STALE_MARKER = "(stale)"

PRICE_STALE_AFTER_S = 20.0
```

Driven serially against a counting stub, so the queue refused nothing and the count measures what the fleet asks for:

```
                                       before   after
one cycle, 39 currencies                   39       1
a later cycle, same 39 currencies          39       1
one cycle, 5 of 44 currencies unlisted     49       6
a later cycle, same 44 currencies          49       1
one cycle, 39 symbols on one timeframe     39      39
a later cycle inside the slot's lifetime    0       0
```

A refresh with no slot to refill makes no venue call. The refresher runs on its own period whether or not any bot is registered, so a fleet with nothing running still paid for a bulk read. Both refreshes now answer zero without a call when the exchange holds no slot. The skip reads the slot count and never a slot's age: a cycle skipped as not yet due let a slot expire while a slow read was still in flight, which put thirty-nine stale reads back on the measurement.

`src/exchange/data_pool.py` — `MarketDataPool._slots_held`

```
    def _slots_held(self, slots: dict, exchange_id: str) -> int:
        """How many of ``slots`` belong to ``exchange_id``.

        A refresh with no slot to refill makes no venue call, so an idle fleet
        costs nothing. The refresh is not skipped on a slot's age: the loop's
        period already sits inside the slot lifetime, and skipping a cycle let
        a slot expire while a slow read was still in flight.
        """
```

Driven with thirty-nine symbols and thirty-nine bots ticking every five seconds, over a twenty-second window:

```
                                    before    after
single-currency balance reads           78        0
single-symbol ticker reads               39        0
bulk ticker reads                         3        8
account reads                             0        8
total venue calls             120 (360/min)   16 (48/min)
the same window with nothing running      3        0
```

One worker answers 330 calls a minute at the 182 milliseconds the venue was measured at. The fleet asked for 360 and now asks for 48, and every running bot still reads its market every tick.

A dollar box draws a dash rather than its own bound

Target Balance and Hedge Balance on the Live Bot Settings window, and Target Balance in the creation wizard, each read their stored figure through `src/trading/container/config.py, in as_box_figure` before the box sees it. That admission refuses a stored flag, a text number, a not-a-number, an endless number, an integer too large for a decimal, and a finite figure outside the box's own span.

A refused figure leaves the box at the bottom of its span drawing the dash the indicator panel draws for a cell it cannot read, names the field in the log, and adds one sentence to the box's hover text. The dash clears the first time the operator moves the box, and from then on the box draws the figure he set.

Live Bot Settings, both boxes, drawn and driven

The window was built under a shown parent against a bot carrying each stored kind in turn, and both boxes were read back off the real control.

Stored value	Target Balance draws	Hedge Balance draws
`250.0`	`$ 250.00`	`$ 250.00`
`"250"`	the dash	the dash
`"abc"`	the dash	the dash
`True`	the dash	the dash
an integer of 400 digits	the dash	the dash
a not-a-number	the dash	the dash
an endless number	the dash	the dash

Three of those kinds used to cost the whole Settings tab: a stored text number, a stored word and a stored huge integer each raised out of the tab's build. The tab now builds on all nine kinds, and the React page draws

the same figure for every one.

Apply writes nothing the operator did not enter

The window was opened on each bad kind, nothing was touched, and Apply was pressed. The bot's stored figure was unchanged on every kind, including the text number, the word, the flag, the not-a-number and the endless number. Stepping the box once and pressing Apply writes the figure the step landed on, which is the control that proves the unchanged readings could have moved.

Two rows on the same screen read the same field and are admitted with it. The Target-BTC and Target-ETH rows draw nothing for a figure the box draws as a dash, and the Apply announcement no longer raises on a stored integer too large for its own data slot.

The creation wizard refuses rather than choosing a figure

`src/gui/bot_wizard.py, in TradingParamsPage.isComplete` answers False while the Target Balance box still draws its dash, so Finish refuses a bot whose target nobody entered. Typing a figure clears the dash, `isComplete` answers True, and the figure typed is what the bot is built on. The wizard was driven on all nine kinds: Finish was refused on the six refused kinds and allowed on the three admitted ones.

The pair list beside it admits each market's volatility the same way. A market whose swing cannot be read draws no volatility and keeps its row, and the three kinds that used to abort the list part way now leave every other market in place.

Where each piece sits

What	Where
the admission	`src/trading/container/config.py, in as_box_figure`
the two dollar boxes	`src/gui/live_settings/settings_tab.py, in _show_admitted_figure`
the dash cleared on the first move	`src/gui/live_settings/settings_tab.py, in _clear_refused_dash`
the same reading for the React page	`src/gui/main_tabs/live_settings_tab_surface.py, in admitted_figure`
the wizard's stored default	`src/gui/bot_wizard.py, in _show_stored_target`
the Finish refusal	`src/gui/bot_wizard.py, in TradingParamsPage.isComplete`
the pair list's volatility	`src/gui/main_tabs/bot_wizard_surface.py, in bag_number`

Where the wizard's phantom defaults come from

Two boxes on the wizard's phantom page open on a figure nobody typed: the tick box that turns phantom bots on, and the lock duration beside it. Each of those figures is now declared in one place, and every surface that draws the box reads that one place.

`src/core/settings.py` — the two declarations every wizard reads

```
#: The phantom tick box every new bot opens with.
DEFAULT_ENABLE_PHANTOMS = True
#: Candles a higher-timeframe lock holds an opposing trade back for.
DEFAULT_LOCK_CANDLE_COUNT = 2
```

The readers are the Settings page, the Live wizard, the Paper wizard, the Simulator wizard, the shell wizard model, and the running-bot phantom page on all three data sources. The bot itself and the restore path read the tick box too, so a saved bot with no stored answer starts the same way a new one does.

A figure you have set always wins. A stored figure in Settings beats the declaration, and a saved bot's own record beats both, so a bot created before this change keeps the figure it was created with.

Before this, the tick box had its answer typed in nine places and four of them said off while five said on, and the lock figure was typed in seven places that happened to agree. Nothing compared any of them. Moving the one declaration to off cleared all four wizard surfaces and the Settings page together; moving the lock figure to seven moved fourteen readings across eight screens at once. Both were put back to the values the screens have always shown.

Counted on the shipped tree, as files that name the declaration besides the file that declares it:

Box	Declared value	Files reading it	Places it used to be typed
Enable Phantom Bots	on	8	9, four of them disagreeing
Candles to lock	2	10	7, all agreeing by luck

2026-10-25 - #665 - the lock figure reaches a new bot on all three builds

The table above records that the lock figure is no argument of the bot's own constructor. That is no longer true of any of the three builds.

On the live build the figure travels as a keyword argument, because it belongs to the bot's timeframe coordinator rather than to the bot's configuration.

`src/trading/container/config.py, in phantom_init_kwargs` passes it, and `src/trading/scrumming_bot.py, in __init__` hands it to the coordinator clamped at 1. An absent figure is left out, which keeps the coordinator's own declared count.

On the Paper Trader and the Simulator the figure travels in the saved record, because a bot on those builds reads its lock holder out of that record. The two record builders are `src/paper/fleet_source.py, in wizard_record` and `src/simulator/fleet_source.py, in wizard_record`.

Driven on each build, with the wizard box set to 7, left on the declared 2, and with no figure sent at all:

```
Live       7 -> coordinator 7     2 -> coordinator 2     none -> coordinator 2
Paper      7 -> tab box 7         2 -> tab box 2         none -> tab box 2
Simulator  7 -> tab box 7         2 -> tab box 2         none -> tab box 2
```

Nothing creates a higher-timeframe lock yet, so the figure still sizes a lock that is never held. Row 222 of issue #665 carries that half.

2026-10-25 - #665 - one read seeds every row on the Detail page

Row 277. Opening Detail on a running bot fills each row from the bot's own stored record. Reading one field off that record was written twice in the page's own module: once for the numeric boxes and once, separately, for the tick boxes, the drop-downs and the text rows.

The two copies decided the same thing. Each chose between reading the field with a fallback and reading it with none, and a row that names no fallback must raise rather than draw a figure nobody stored.

One read now serves every row.

`src/gui/main_tabs/live_settings_tab_surface.py, in read_config_field`

```
def read_config_field(config: Any, field: str, default: Any, no_default: bool) -> Any:
    return getattr(config, field) if no_default else getattr(config, field, default)
```

`read_number` seeds the numeric boxes through it, and `LiveSettingsTabModel._read_control` seeds every other kind of row through the same call. The Paper page and the Simulator page both inherit that model, so three Detail pages read a row one way.

Every row draws what it drew before, and a row whose field the record does not carry still refuses rather than inventing a figure.

```
stored record carrying the field
  numeric box       2.5
  tick box          True
  drop-down         both

record missing the field, row naming no fallback
  numeric box       refused
  tick box          refused
```

Driven with the one read moved to a figure nothing uses, every kind of row followed it together. Before the change the same move reached the numeric boxes alone and the tick box and the drop-down kept their own values. The file is byte identical after the move was put back.

2026-10-25 - #665 - the despawn rule is declared once

The section "The Tranche Despawn Timer" above records the rule being written three times and defers collapsing it. It is collapsed now.

One function turns a stored figure into whole despawn days. The sweep that removes records, the shared preview the Qt panel reads, the Fold Tranches screen and the Bot Settings despawn row all call it. The cap it saturates at and the four candidate windows the panel offers are each declared once beside it.

`src/trading/container/config.py` — the one rule

```
def whole_despawn_days(value) -> int:
    days = as_finite_float(value)
    if days is None:
        return 0
    return min(DESPAWN_MAX_DAYS, max(0, int(days)))
```

Live, Paper and the Simulator read it through the panel chrome they share, and the Electron build reads it from its two screen modules. That is one rule on all four surfaces.

Thirteen stored figures were read at five sites before and after, and every site gave the same answer on every figure, including a flag, a word and the two endless numbers. Moving the cap to a figure nothing uses moved all five together; before the change the same move reached the shared site alone and both screens kept the old cap. The file is byte identical after the move was put back.

Stored figure	Whole days
0	0, which is off
7	7
365	365
the cap itself	the cap
one past the cap	the cap
7.9	7
a negative figure	0
a flag, a word, nothing at all	0
a not-a-number, an endless number	0
a whole number too large for a figure	0

2026-10-25 - #665 - the Apply routing map is declared once

Apply decides, per edited row, whether the figure goes through one of the bot's own live setters or is written straight onto its stored record. Six rows take a setter. That list was written twice, once for the Qt window and once for the Electron surface, with nothing comparing them.

`src/gui/main_tabs/bot_live_settings_surface.py` — the one list

```
RUNTIME_ROUTED = {
    "target_balance": "set_target_balance_live",
    "visibility": "set_visibility_live",
    "aggressive_trading": "set_aggressive_live",
    "hedge_balance": "set_hedge_balance_live",
    # setattr alone moves the flag and leaves _hedge_bal at zero.
    "hedge_rebalance_active": "set_hedge_rebalance_active_live",
    # set_chunk_size_usd also rescales _chunk_free_base and the registry claim.
    "extractor_chunk_size_usd": "set_chunk_size_usd",
}
```

The Qt window, the Electron surface and the Simulator's Detail page all read it, and the Electron build is handed it in the same payload it draws from. The Paper window routes nothing on purpose: a paper bot is a stored record with no running object, so every row there goes to one record writer.

Driven by pressing Apply on a bot that records each setter it is handed. Both Apply paths routed the same six, and a seventh row that carries no entry was written onto the record by both, which is the reading that shows a difference would show.

Adding a seventh entry to the one list made both paths route seven. Before the change the same addition reached the Electron surface alone and the Qt window still routed six. The file is byte identical after the entry was removed.

2026-10-25 - #665 - the Hedge Balance opening figure is declared once

The Hedge Balance box on the creation wizard opens on a figure nobody typed. That figure was written in five places: the bot's own declaration, and once in each of the four wizards the operator can reach.

`src/trading/container/config.py` — the one figure

```
hedge_balance: float = 200.0
```

The Live wizard, the Paper wizard, the Simulator wizard and the Electron wizard all read it now. A bot you created earlier keeps the figure it was created with, because a saved record carries its own.

Read at all five places before and after, every one answered $200.00. Moving the declaration to a figure nothing uses moved all five together; before the change the same move reached the declaration alone and all four wizard boxes still opened at $200.00. The max-cartridge box beside it was read the same way each time and never moved, which is the reading that shows the boxes answer for themselves.

2026-10-26 - #665 - both Electron Ceiling Multiple rows read the one declaration

The two rows the Electron build draws for the Ceiling Multiple still typed their own span and their own opening figure. Both read the one declaration now, so all eight screens on this page answer from the same place.

`src/gui/main_tabs/bot_wizard_surface.py` — the Electron wizard row

```
"position_ceiling_multiple": {
    "kind": SPIN_DOUBLE,
    "minimum": CEILING_MULTIPLE_MIN,
    "maximum": CEILING_MULTIPLE_MAX,
    "decimals": 1,
    "step": 0.5,
    "suffix": "x anchor",
    "value": POSITION_CEILING_MULTIPLE_DEFAULT,
},
```

The opening figure was written in thirteen places, not nine. The six boxes on this page, the status every bot row is read from, the two Electron rows, and four more in the Paper and Simulator record readers. All thirteen read one declaration now.

`src/trading/container/config.py` — the one opening figure and the one span

```
POSITION_CEILING_MULTIPLE_DEFAULT: float = 5.0
CEILING_MULTIPLE_MIN: float = 1.0
CEILING_MULTIPLE_MAX: float = 10.0
```

Every box was built on its real screen and read back off the real control. All six span 1.0 to 10.0, open at 5.0x and step half a multiple, before and after. Both Electron rows answer the same span and the same figure through the real bridge, and the dollar ceiling on a $100.00 anchor is unchanged at every stored multiple.

Moving the declaration to a span of three to seventy-seven and an opening figure of 6.25 moved every one of those readings together. The same move on the code before this change left both Electron rows at one to ten and 5.0, and left all six opening fallbacks at 5.0. The Smart Ceiling box beside it was read the same way each

time and never moved, which is the reading that shows each box answers for itself.

The Fold Tranches surface no longer claims that a figure written out by hand is reported when it changes on one side alone. Nothing did that reporting, and the module it named imports the old interface library, which a surface may not reach. The docstring now says where its two imported figures come from.

`src/gui/main_tabs/fold_tranches_tab_surface.py` — what the module reads in

```
from ...trading.container.config import DESPAWN_MAX_DAYS, DESPAWN_PREVIEW_WINDOWS
```

Every value that surface hands out was read back before and after the sentence went. The payload is the same 28,046 bytes and its 409 declared figures are unchanged, so no figure left with the sentence.

2026-10-26 - #665 - a stored zero fold ratio reads as the declared ratio

The Scrum Fold Ratio box runs from 1 to 100 and opens at 100. The engine clamp accepted anything from zero upward, and a stored zero emptied a whole sale's fold queue. Driven on a sale of two tranches worth $200.00 and two units, a stored zero queued $0.0000 and no units at all, and retired the whole $200.00 as cash. The sale still wrote its usual line, naming a ratio of nought per cent. A record like that is not something the screen can write.

The clamp now admits only a figure a box offers, and a record outside that span reads the declared ratio of 100. The same sale with a stored zero queues the full $200.00 and two units. This is the choice between two answers, and the declared ratio is the one the page already states: a whole percentage from 1 to 100, at 100 % to start. The ratio is a cushion that holds proceeds back, so an unreadable record means no cushion, not the largest one a box can hold. It is also the rule every other stored figure on this screen already follows, where a figure outside a box's span is no figure at all.

`src/trading/container/config.py` — the one reader every surface calls

```
def fold_ratio_pct(config) -> int:
    read = as_box_figure(
        getattr(config, "scrum_fold_pct", FOLD_RATIO_PCT_DEFAULT),
        FOLD_RATIO_MIN_PCT,
        FOLD_RATIO_MAX_PCT,
    )
    if read is None:
        return FOLD_RATIO_PCT_DEFAULT
    return int(read)
```

The span was written in nine places and the clamp was a tenth that disagreed with all nine at its bottom end. The three settings pages, the three New Bot wizards and the two Electron rows each typed one to a hundred, and the engine accepted zero. All ten read one declaration now.

`src/trading/container/config.py` — the one span and the one opening ratio

```
FOLD_RATIO_MIN_PCT: int = 1
FOLD_RATIO_MAX_PCT: int = 100
FOLD_RATIO_PCT_DEFAULT: int = int(
    next(one.default for one in fields(BotConfig) if one.name == "scrum_fold_pct")
)
```

The Max Target Growth figure had the same shape in a quieter way. Thirteen places read it and every one of them named its own standby figure of one per cent for a record that carries no such figure. None of those standby figures could ever answer, because every bot record and both fleet records declare the field, so the read always finds a value. Measured against a record built without the field, with a null, with a zero, with a text number, with a flag and with a not-a-number: the field is present in every case and the standby figure answers in none.

`src/trading/container/config.py` — the one growth reader

```
def target_growth_pct(config):
    return getattr(config, "max_target_growth_pct", TARGET_GROWTH_PCT_DEFAULT)
```

Each of the thirteen calls that reader now and names no figure of its own. Every one keeps its own handling of what it reads, so a record holding a null still reads as nought where it did before and still raises where it did before. The Paper and Simulator record readers carried the figure twice more each; those four read the one declaration too.

The two blocker lines the gate writes when a reversal is too close spelled the distance themselves, by adding the interval to the fee. The distance has one definition and the market scan beside it already called it.

`src/trading/gate_chain.py` — the distance both lines print

```
eff_pct = minimum_opposing_trade_distance_pct(
    ctx.scrumming_interval_pct, ctx.trading_fee_pct
)
```

Both lines read the same figures as before. At an interval of 1.00 % and a fee of 0.60 % the scrum line still reads a required $101.60000000 against a pivot of $100.00000000 and the fold line still reads $98.40000000, and across all 42 pairs the two boxes can hold the readings are identical. The definition bounds the distance at fifty per cent, and the widest pair the boxes offer reaches 25.00 %, so the bound never binds on anything a reader can set. The same two lines the cartridge writes, and the figure the stack ladder opens on, read the one definition now as well.

The two refusals the buy and sell executors write still read the fee for themselves, because they read a fee set to zero as zero where every other place reads it as 0.60 %. Routing them would change a refusal a bot makes today, so they are named and left.

Current reading. Both refusals take `opposing_distance_settings`, the one reading the gate takes. A fee set to zero reads 0.60 % there too, and a setting that holds no number refuses the order and names the setting.

`src/trading/scrumming/execution.py` — `ExecutionEngineMixin._execute_buy`

```
try:
    _held = self.opposing_distance_settings()
except DistanceSettingUnreadable as _unread:
    self._refuse_unreadable_distance(_unread, "FOLD", _px_check)
    return None
_eff_pct = _held.distance_pct()
```

The ratio was swept from minus five to a hundred and five in steps of a hundredth, which is 11,001 readings, and every integer boundary falls on a step. 600 readings moved and all 600 sit below one, where the old clamp read nought. The 10,401 readings from one upward are identical, so no bot carrying a ratio a box can hold folds any differently. Three record classes that used to stop the sale with an error — a null, a not-a-number and an endless number — now read the declared ratio and the sale finishes. A flag and a text number moved as well, and no box can write either.

Moving the declared ratio to seventy-seven and the declared growth to 7.77 moved every reading together: the zero-ratio sale queued $154.0000 and the growth cap on a record with no figure read $15.5400. The same move on the code before this change left the zero-ratio sale at $0.0000 and the cap at $2.0000, which is the reading that shows the copies did not follow. A sale at a legitimate ratio of 40 queued $80.0000 and 0.8000 units in all four readings, which is the figure read the same way each time to show the readings report at all.

Where the earlier fold-ratio entry now reads differently

Three passages above described the clamp and the standby figures as they were. They stand as written, and these are the readings that replaced them.

written above	what it reads now
"`_fold_pct = max(0, min(100, int(getattr(self.config, \"scrum_fold_pct\", 100))))`" under Where the Scrum Fold Ratio is read	`_fold_pct = fold_ratio_pct(self.config)`, the one reader, admitting 1 to 100
"The clamp's lower bound is zero and both controls start at one, so a record carrying a zero empties the sale's whole fold queue and retires the cash instead."	The clamp admits only a figure a box offers. A record carrying a zero reads the declared 100 and the sale queues its whole proceeds
"Raising the clamp to match the controls would change what such a bot rebuys on its next sale."	It did, and only for a record below one. Every ratio from 1 to 100 rebuys exactly as before

2026-10-26 - #665 - the despawn tooltip is declared once

The Tranche Despawn Timer box carries the longest words on the Bot Settings page. Live, Paper and the Simulator each spelled those words out, and the Electron row list held a fourth copy. One entry answers for all four now.

`src/gui/live_settings/settings_tab.py, in _create_settings_tab` — the read the three Qt tabs share

```
from ..main_tabs.live_settings_tab_surface import DESPAWN_FIELD, TOOLTIPS

self._tranche_despawn_days.setToolTip(TOOLTIPS[DESPAWN_FIELD])
```

Each box was built for real and asked what words it carries. All four report the same length, and so does the Scrum Fold Ratio row beside it.

```
surface     the despawn words it reports   the Scrum Fold Ratio row
Live        1,086 characters               183 characters
Paper       1,086 characters               183 characters
Simulator   1,086 characters               183 characters
Electron    1,086 characters               183 characters
```

Moving the one declaration to wording nothing uses moved all four together, to 1,095 characters each. The same move before this change reached the Electron row alone and the three Qt boxes kept the old words. The file is byte identical after the move was put back. The neighbouring row read 183 characters in every one of those readings, which is the figure that shows a reading happened at all.

Where the earlier despawn timer entry now reads differently

Three statements in "The Tranche Despawn Timer" above describe the control. They stand as written, and these are the readings that replaced them.

written above	what it reads now
"The timer is the one row of the Hedge Rebalance group that a new bot cannot be given"	The box is built into the Advanced Scrumming group. The Hedge Rebalance group is the next group down and holds no despawn row. A new bot still cannot be given the timer
"neither wizard build carries it"	Four wizard builds exist and none carries the row. Each of the fifteen wizard files names the field no times, against 7, 5, 7 and 7 for the neighbouring ratio field, so the zero reads the tree
"the one control, 0 to 365 days"	Four screens build that box: the Live, Paper and Simulator settings tabs and the Electron row list. The span is unchanged

2026-10-26 - #665 - the grid conversion method is gone

The engine held one method that turned remembered trades into buy and sell pairs for the retired mode. Nothing called it, and no page of this manual named it.

`src/trading/scrumming_bot.py, in memorize_to_grid` — removed

```
                          call sites   definitions   occurrences
memorize_to_grid, before           0             1             1
memorize_to_grid, after            0             0             0
_main_lots_invariant_ok            1             1             2
a name never declared              0             0             0
```

The count walks the parsed tree of all 624 tracked Python files. A method a few lines below it in the same class reads one caller, and a name that was never declared reads the same figures as the removed one, so the zero is a reading rather than a search that matched nothing. The engine was then imported for real: the class no longer carries the name and three neighbouring methods are still there.

What the record it read is used for now. Nothing reads it. One entry joins the remembered-trade list for each filled sell and each filled buy, and this method was the only thing that had ever read that list.

```
the record   src/trading/scrumming/execution.py, class MemorisedTrade
the writers  src/trading/scrumming/execution.py, in _execute_sell
             src/trading/scrumming/execution.py, in _execute_buy
the readers  none
```

Both writers sit inside the paths that place an order, so they are named and left.

2026-10-26 - #665 - the ratio sits in one group, and the fill writes no spare record

Five names said something the code did not. A ratio box filed under two different group titles, two controls a stored-key search could not find, one fact worded two ways, a way in nobody used, and a record written on every fill and read by nothing.

The Scrum Fold Ratio box sits in Scrumming Settings on every screen

Eight screens build the box. Four put it under Scrumming Settings and four under Advanced Scrumming. Each box was built for real and asked which group box holds it.

```
screen                   before               after
Live settings            Advanced Scrumming   Scrumming Settings
Paper settings           Advanced Scrumming   Scrumming Settings
Simulator settings       Advanced Scrumming   Scrumming Settings
Electron settings rows   Advanced Scrumming   Scrumming Settings
Live wizard              Scrumming Settings   Scrumming Settings
Paper wizard             Scrumming Settings   Scrumming Settings
Simulator wizard         Scrumming Settings   Scrumming Settings
Electron wizard rows     Scrumming Settings   Scrumming Settings
```

Scrumming Settings is the group, because the manual already says so. The Trading Parameters section states that four unit-rule lines sit under Scrum Fold Ratio in the Scrumming Settings group, and the control's own description ties it to Max Target Growth %, which reads Scrumming Settings on all eight screens. No page of this manual puts the ratio in Advanced Scrumming.

`src/gui/live_settings/settings_tab.py, in _create_settings_tab` — the row's new home

```
sf.addRow("Max Target Growth %:", self._max_target_growth)
sf.addRow("Scrum Fold Ratio:", self._scrum_fold_pct)
```

The box itself is untouched. Every screen reports a value of 100 and a tooltip of 183 characters, before and after, and Max Target Growth % beside it reads Scrumming Settings in all sixteen readings.

The read-rate and bullseye controls carry the name of the key they store

A search of a wizard for a stored key found the read-rate control and the bullseye control once each, against seven for the two rows beside them. The one hit was the line that hands the value out; the control itself was named for neither key. The three settings screens and the Electron row list carried the same two short names.

```
file                          read rate        bullseye         ratio    travel
                              before  after    before  after    control  control
bot_wizard.py                      1      7         1      5          7        7
paper_bot_wizard.py                1      7         1      5          7        7
sim_bot_wizard.py                  1      7         1      5          7        7
bot_wizard_surface.py              1      5         1      5          5        5
settings_tab.py                    2      8         2      6          8        2
paper_settings_tab.py              2      8         2      6          8        2
sim_settings_tab.py                2      8         2      6          8        2
live_settings_tab_surface.py       1      3         1      3          3        1
```

`src/gui/bot_wizard.py, class TradingParamsPage` — the control now carries the key's own name

```
self._scrum_read_rate_min = QSpinBox()
af.addRow("Read Rate:", self._scrum_read_rate_min)
```

The bullseye reads fewer lines than the read rate because a tick box takes fewer setter calls; the tick box beside it, the midline gate, reads the same figure.

No stored key changed. All four wizard builds hand out the same keys and the same values: 50 keys on the Live build, 47 on Paper, 47 on the Simulator and 50 on the Electron build, with none of the 194 key readings different. A record carrying the two keys was written by hand and loaded: a stored read rate of 17 and a stored bullseye of off reach the box and the tick box on the running screen.

A further 24 stored keys in the Live wizard are still found on one line each. The same short-name shape covers them, and the settings screens abbreviate throughout. Those are not this entry's two controls and are recorded on the issue.

One declaration holds the passphrase prompt

The install wizard and the Settings page each wrote their own words for one fact. Both screens now read one declaration.

```
screen           before                                          after
install wizard   Passphrase you chose when creating the API key    unchanged
Settings page    Passphrase set when creating API key             the same 46 characters
```

`src/gui/main_tabs/init_wizard_surface.py` — the one declaration

```
PHRASE_FIELD_PROMPT = "Passphrase you chose when creating the API key"
```

Which venues ask for a passphrase did not change. Both screens already read `PASSPHRASE_EXCHANGES`, so only the words differed.

Moving the one declaration to wording nothing uses moved both screens together. The same move before this change reached the install wizard alone and the Settings page kept the old words. Both files are byte identical after the move was put back, by sha256. The API key prompt beside it held its own words in every reading.

The signal provider drops a way in that nobody used

The provider took a trend-strength figure from its caller. The Extractor builds the only provider in the tree and passes a venue and a timeframe alone.

```
name                         before   after
trend_strength_threshold          0       0    callers that pass it
trend_strength                    8       8    callers that pass it, the control
a name never declared             0       0
```

The count walks the parsed tree of all 624 tracked Python files. The control name reads eight, so the count reports; a name that was never declared reads zero everywhere. The figure stays at 0.65 and is now one declaration.

`src/trading/ta_signal_provider.py` — the figure, and the one line that reads it

```
TREND_HOLD_MIN_SHARE: float = 0.65

trend_hold = trend_strength > TREND_HOLD_MIN_SHARE
```

Every fill stops writing the spare record

One record joined a list on each filled sell and each filled buy. Nothing read the list once the retired mode's method went.

A hand-built venue answered every read from records written by hand and settled an order as filled. No venue was contacted: the socket counter read zero on every run and its control moved it to one on a single refused call.

```
one fill                records   holdings   sells   buys   volume   scrummed   folded   fill
sell, before the change       1        8.0       1      0    200.0      200.0      0.0    100
sell, after the change     none        8.0       1      0    200.0      200.0      0.0    100
buy, before the change        1      10.01       0      1      1.0        0.0      1.0    100
buy, after the change      none      10.01       0      1      1.0        0.0      1.0    100
```

Nothing else about either fill moved. The record type, the field and both writers are gone, and the list no longer exists to be written.

Where the two earlier entries now read differently

Two statements above describe what this unit changed. They stand as written, and these are the readings that replaced them.

written above	what it reads now
"All four report the same length, and so does the Scrum Fold Ratio row beside it"	The despawn reading is unchanged at 1,086 characters on all four screens. The Scrum Fold Ratio row is no longer the row beside it: the ratio sits in Scrumming Settings and the despawn box stays in Advanced Scrumming. The row beside the despawn box is now Band Travel
"Both writers sit inside the paths that place an order, so they are named and left"	Both writers are gone. A hand-built venue settled an order as filled with no network, which drove each writer for real, so the removal was proved rather than named

written above	what it reads now
"the record `src/trading/scrumming/execution.py, class MemorisedTrade`"	That class is gone, with the field that held its list and the two lines that appended to it. The file is still there and still holds `SettledFillFee`, the fee record each settled fill writes

2026-10-26 - #665 - the cartridge cap, the taper's name, and the bar on the Bot Swarm row

The cartridge cap carries its own words in the log

Two unrelated controls wrote the word ceiling. One is the percentage cap on the cartridge threshold. The other is the dollar ceiling on the position. A line in the bot log could therefore send a reader to either control. The cartridge line now names the cartridge cap. The position-ceiling lines keep the words row 118 gave them.

Both controls were driven on one bot in one run. The calibration wrote its line, then the fold refused on the position ceiling.

```
cartridge, before   SMART CARTRIDGE calibrated to 24.00% (BB range 24.00%, floor=5.00%, ceiling=30.00%). Effective threshold = $48.
00.
cartridge, after    SMART CARTRIDGE calibrated to 24.00% (BB range 24.00%, floor=5.00%, cartridge cap=30.00%). Effective threshold =
$48.00.
ceiling, both       FOLD BLOCKED by position ceiling: current $1000.00 >= ceiling $600.00 (166.7% of ceiling). Scrum still allowed.
```

The words the two lines share read `ceiling` before the change and nothing after it. The ceiling line is the same in both runs, which is the reading that shows only the cartridge line moved.

`src/trading/scrumming/tick_phases.py, in _tick_smart_cartridge_pct` — the line

```
f"floor={_interval_floor:.2f}%, "
f"cartridge cap={_cap_text}). "
```

The fold taper is named for the size it scales

The taper multiplies the dollars a fold spends. It does not change how often a fold happens. Its name said rate. The engine's figure, the status record's key and the wording on both screens now say size.

site	kind	before	after
`src/trading/scrumming/sizing.py`	the schedule	`fold_rate_taper`	`fold_size_taper`
`src/trading/bot_container.py, in get_status`	write	`fold_rate_taper`	`fold_size_taper`
`src/gui/main_tabs/bot_status_table_surface.py, in risk_suffix`	read	fold rate tapered to	fold size tapered to
`src/gui/widgets/bot_status_table.py, in update_bots`	read	fold rate tapered to	fold size tapered to
`src/gui/web/bot_status_table.js`	the page read	default_fold_taper	unchanged, it never said rate
the Enable Position Ceiling tooltip, eight files	write	Fold rate tapers	Fold size tapers

A record saved under the old key still reads. The reader takes the new key first, then the old one, then the declared figure of one.

```
record carries                Live page   Paper page   Sim page   Live widget   Paper widget   Sim widget
fold_size_taper 0.37              37%          37%        37%           37%            37%         37%
fold_rate_taper 0.42              42%          42%        42%           42%            42%         42%
neither key                      100%         100%       100%          100%           100%        100%
```

`src/gui/main_tabs/bot_status_table_surface.py, in fold_taper_of` — the reader

```
for field in (FOLD_TAPER_FIELD, FOLD_TAPER_FIELD_RETIRED):
    if field in status and status[field] is not None:
```

Moving the one declaration to wording nothing writes sent all six readings to 100 per cent for the new key and left the old-key reading at 42 per cent. The surface file is byte identical after the move was put back, by sha256.

The Bot Swarm row shows the bar a detonation must clear

Min Confidence reached no status record. The Fire tooltip on a Bot Swarm row could name the timeframe a bot watches and never the figure a vote has to reach. The record now carries the bar and the tooltip names it.

```
record carries                          the tooltip's detonation line
0.60, before the change    Detonation armed: monitoring 1W for BULLISH auto-harvest.
0.75, before the change    Detonation armed: monitoring 1W for BULLISH auto-harvest.
0.60, after the change     Detonation armed: monitoring 1W for BULLISH auto-harvest at confidence 0.60 or above.
0.75, after the change     Detonation armed: monitoring 1W for BULLISH auto-harvest at confidence 0.75 or above.
```

Current reading, 2026-10-29. The detonation line is shorter, and a Fire description keeps it only while the whole stays within twenty words. A Fire button that already names its blocking gates and its ceiling leaves this line out.

`src/gui/main_tabs/bot_status_table_surface.py` — `DETONATION_FORMAT`

```
DETONATION_FORMAT = "\nDetonation armed: {timeframe}, confidence {confidence:.2f} or above."
```

Before the change the two records read the same, so no screen could tell a set figure from the opening one. After it they differ. The timeframe is the same in all four readings, which is the reading that shows only the bar was added.

`src/trading/bot_container.py, in get_status` — the key

```
"detonation_confidence_min": float(
    getattr(
        self.config,
        "detonation_confidence_min",
        DETONATION_CONFIDENCE_MIN_DEFAULT,
    )
),
```

The opening figure of 0.75 is one declaration, read by the record and by the box.

`src/trading/container/config.py` — the figure

```
DETONATION_CONFIDENCE_MIN_DEFAULT: float = 0.75
```

What the three repairs reach, and what they do not move

The Paper Trader's table and the Simulator's table are the Live table with their own price feed, so one repair in the Live surface and one in the Live widget reach all three. Every one of the six readings above was taken on the real model or the real widget of its own build.

Nothing detonates differently, no ceiling moves and no bar changes. The taper, the ceiling, the ceiling multiple, the ratio and the calibrated cartridge percentage were each driven over a swept set of 269 readings on the tree before the change and the tree after it. The two sets are byte identical by sha256, and changing one reading by hand makes the comparison report one difference.

```
area                    readings   differing
the taper schedule            21           0
the position ceiling          60           0
the ceiling multiple          10           0
the ratio to ceiling          20           0
the cartridge figure         150           0
the detonation bar             8           0
```

No venue was contacted on any run. The socket counter read zero every time and a single deliberate call moved it to one.

Where three earlier entries now read differently

Three statements above describe what this unit changed. They stand as written, and these are the readings that replaced them.

written above	what it reads now
"the engine reports it inside its cartridge line as a percentage"	It still does, and the percentage is now labelled cartridge cap. The bare word ceiling no longer appears in the cartridge line, so a search of the log for ceiling returns the position-ceiling lines alone
"the taper schedule is `fold_rate_taper`, both in the shared sizing module"	The schedule is `fold_size_taper`, in the same module. The multiplication is still `fold_spend_usd`
"It is absent from the record the Bot Swarm row reads, so that tooltip can report the timeframe a bot is watching and never the bar it must clear"	The record carries `detonation_confidence_min` and the tooltip names it. The reading for a stored 0.60 and the reading for the opening 0.75 now differ

2026-10-27 - the holdings audit, and what a buy books

The three figures the holdings audit compares

The audit reads three counts for one bot on one asset. Two are the bot's own: the total of its lot book, and its single holdings figure. The third is the exchange's own balance for that asset. The audit takes the larger of the bot's two counts and compares it against the exchange's count, inside a band of half a per cent.

`src/trading/scrumming/reconciliation.py, in _reconcile_holdings`

```
  exchange_units, _scalar_units, _lot_units = _readings
  self._record_audited_venue_units(exchange_units)
  internal_units = _lot_units if _lot_units > _scalar_units else _scalar_units
```

Read today: the larger count still decides the direction. Where the exchange sits at or below it, the half-per-cent band applies to each of the two counts, so the audit no longer reads aligned while the lot book sits below the exchange. Where the exchange sits above it, nothing changed.

`src/trading/scrumming/reconciliation.py, in _reconcile_holdings`

```
  _drift_down = exchange_units <= internal_units
  _widest_units = (
      max(abs(exchange_units - _lot_units), abs(exchange_units - _scalar_units))
      if _drift_down
      else abs(drift_units)
  )
```

What the audit does for each ordering

Each ordering below was driven on constructed counts. No exchange was contacted.

the three counts	what the audit did	the lot book after
exchange 50.1, lots 50.0, holdings 50.0	inside the band, so nothing was written	50.0
exchange 40.0, lots 50.0, holdings 50.0	scaled every lot by the exchange ratio and set the holdings figure to the exchange	40.0
exchange 40.0, lots 45.0, holdings 50.0	scaled every lot by the exchange over the lot total and set the holdings figure to the exchange	40.0
exchange 45.0, lots 40.0, holdings 50.0, a usable price	booked the five units the lot book lacked as one reconciliation lot and set the holdings figure to the exchange	45.0
exchange 45.0, lots 40.0, holdings 50.0, no usable price	set the holdings figure to the exchange, booked nothing, and logged the lot book five short	40.0
exchange 45.0, lots 40.0, holdings 50.0, sibling bots holding five	set the holdings figure to the exchange, booked nothing, and logged the five units the sibling bots hold	40.0
exchange 40.0, lots 36.0, holdings 40.1	the holdings figure sat inside the band and the lot book outside it, so booked four units as one reconciliation lot	40.0
exchange 40.0, lots 40.0, holdings 30.0	set the holdings figure to the exchange and left the lot book where it was	40.0
a lot whose units are text, not finite, or not a lot	refused before any write and logged `RECONCILE REFUSED`; the lot book, the holdings figure and the stored exchange count stay as they were	unchanged
a holdings figure that is not finite	refused before any write and logged `RECONCILE REFUSED`; nothing is repaired and the stored exchange count stays as it was	unchanged
an exchange total that is not finite, below zero, or not a number	refused before any write and logged `RECONCILE REFUSED`; nothing is repaired and the stored exchange count stays as it was	unchanged

the three counts	what the audit did	the lot book after
the balance read raised	refused before any write and logged `RECONCILE REFUSED` with the error; nothing is repaired and the stored exchange count stays as it was	unchanged
exchange 40.0, lots 30.0, holdings 30.0, a usable price	booked the ten missing units as one reconciliation lot	40.0
exchange 40.0, lots 30.0, holdings 30.0, no usable price	deferred the adoption and wrote nothing	30.0
exchange 40.0, lots 30.0, holdings 30.0, sibling bots holding ten	claimed nothing, because no unit on the asset was this bot's	30.0
the exchange omitted the asset from its answer	refused, and kept the last exchange count	30.0

A lot book below the exchange stays below it in the last three rows. The startup read repairs it when a price is usable and no sibling bot holds the surplus, and leaves it short otherwise, so the gap can survive a restart.

Read today: the table holds sixteen rows, and the three that sentence names are still its last three. Two rows above them also end with the lot book below the exchange: a holdings figure above the exchange with no usable price, and one with sibling bots holding the gap. Both retry on the next audit. The four refusal rows added above them leave every figure where it was.

The reading only the operator can take

Those six rows say what the audit does with each ordering. They do not say which ordering any live bot is in. That needs the lot total, the holdings figure and the exchange's balance for one bot, read beside each other in one pass, and the third of them can only come from the exchange. The reading is the operator's to take on the running platform.

Read today: the reading sits on the Trading tab. Rest the pointer on a bot's Ammo cell, and its mouse-over names the stored exchange count, the holdings figure and the lot-book total together. The exchange count is net of any sibling bot on the same asset, as the Ammo column section above describes.

A buy books the units the exchange reported

A fold buy, an initial entry, a hedge buy and a manual tranche fire all book the count the exchange reported bought, never the count that was asked for. An order resting on the book reports no filled amount, and the bot answers that press as no buy: no lot is added, the holdings figure does not move, and the log reads `BUY SENT, NO FILL REPORTED`.

`src/trading/scrumming/execution.py, in _execute_buy`

```
filled_units, actual_fill, _ = await self._settled_fill(
    order, self.config.symbol, amount, price, label="FOLD"
)
```

The four places that add the bought units to the lot book read one figure the buy leaves behind, so none of them can size a lot from the ask.

`src/trading/scrumming/execution.py, in ExecutionEngineMixin`

```
  _last_buy_filled_units: float = 0.0
```

Diagram: flowchart, left to right

- _execute_buy reads the settled fill -> **_last_buy_filled_units**
- _last_buy_filled_units -> **initial entry lot**
- _last_buy_filled_units -> **fold rebuy lots**
- _last_buy_filled_units -> **manual tranche lot**
- _last_buy_filled_units -> **hedge lot**

What that repair stops

A press the exchange never filled used to credit the position anyway. The next audit then read the position above the exchange, took the first ordering in the table above, and scaled every lot down to match. Units left the lot book that the exchange had never sold.

the exchange bought	what was booked before	what is booked now
ten of ten asked	ten	ten
five of ten asked	ten	five
none of ten asked	ten	nothing, and the press reads as no buy

Driven on a two-lot book of 6.0 and 4.0 units against an exchange holding 10.0: a buy of ten units that filled nothing left the book at 3.0 and 2.0 units before the repair, and at 6.0 and 4.0 units after it. The drift line the audit writes appears in the first run and not in the second.

The hedge reserve falls by what the fill cost

A hedge buy debits the reserve by the money the fill cost, capped at the amount it asked for, and the log names both.

`src/trading/scrumming_bot.py, in tick`

```
  _hedge_spent = min(_use, _hedge_asset * hedge_fill)
```

Which count the Ammo cell priced

The cell's rule is unchanged and was read off the composed cell on four inputs. It prices the exchange's count where the platform holds one, falls back to the bot's attributed count where it does not, and names the count it used in its tooltip either way.

what the cell was given	what it drew	what the tooltip said
exchange 40.0, attributed 30.0	$100.0000	measured on the exchange's count
no exchange count	$50.0000	no exchange count held, measured on the attributed count
an exchange count of zero	$50.0000	no exchange count held, measured on the attributed count
exchange 40.0, the last read refused	$100.0000 (held)	the held reading, with the reason

Read today: the drawn figures are unchanged, and the tooltip column reads differently. Each row now names the lot-book total beside the exchange count and the holdings figure, and the held row ends `Exchange read refused.` without the reason.

2026-10-28 - the holdings audit lands the lot book on the exchange, and every fill moves the priced figures

When the exchange sits at or below the larger count

The audit still takes the larger of the bot's two counts to decide the direction. When the exchange sits at or below it, the bot adopts the exchange count for both of its own: the holdings figure becomes the exchange count, and the lot book moves to it. A lot book above the exchange scales down by the exchange over the lot total. A lot book below the exchange gains one reconciliation lot for the gap, at the current price and under the sibling ceiling, the two rules a surplus already meets on the way up. Each repair writes one `DRIFT DOWN` line to the bot log with both counts before and after.

`src/trading/scrumming/reconciliation.py, in _adopt_drift_down`

```
if _lots_before > exchange_units:
    _ratio = exchange_units / _lots_before
    for _lot, _units in zip(self._main_lots, lot_each, strict=True):
        _lot["units"] = _units * _ratio
    self._main_lots = [l for l in self._main_lots if l["units"] > 1e-12]
elif exchange_units - _lots_before > 1e-9:
    _claimable, _sib_units = self._claimable_exchange_units(exchange_units)
    _adopt = min(exchange_units, _claimable)
```

Where the gap cannot be booked, the line says why and the next audit tries again. No usable price books nothing, because a lot with no basis reads as profit against every price. Units the sibling bots hold stay theirs. A gap of 1e-9 units or less reads as aligned, the same floor the earlier drift-down branch kept.

What the worked example holds before and after

One bot holds lots of 30 and 15 units, a holdings figure of 50, and an exchange count of 40. Both versions of the audit ran in one process, the earlier one read from git at the commit this change starts from.

```
                       lots after    each lot          holdings after   gap to the exchange
before this change       36.0        24.0, 12.0          40.0             4.0 below
this change              40.0        26.67, 13.33        40.0             none
```

Every ordering, swept

Thirteen values were set for each of the three counts, from 0 to 1,000,000, so 2,197 combinations. Each ran on the tree before this change and on this change, with a usable price and no sibling bot.

```
the exchange at or below the larger count      1,547 combinations
                                               before   after
  lot book left below the exchange                371       0
  lot book landed on the exchange                       1,479
  inside the band, nothing written                         68
the exchange above the larger count              650 combinations
  read identical on both trees                    650 of 650
```

Changing one digit of one reading in the second set makes the comparison report one difference, so the identical count can report a change. The same 650 also read identical with no usable price and with sibling bots holding five units. In those two runs, the 338 combinations that needed units added stop short and name the reason.

The refusal exits write to the bot log

A refused input used to leave one line in the program's own log alone, and a balance read that raised left only a debug line. Each refusal now also writes one `RECONCILE REFUSED` line to the bot log, naming the input. Nothing else changed: no lot is scaled, the holdings figure stays, and the stored exchange count is not updated.

`src/trading/scrumming/reconciliation.py, in _emit_reconcile_refused`

```
message=(
    f"RECONCILE REFUSED ({reason}): {why}. No lot was rescaled and "
    f"no holdings were reset. Retrying next interval."
),
```

What the Ammo mouse-over reads

The tooltip names the exchange count, the holdings figure and the lot-book total in one sentence, on the Qt table and on the page alike. The bot's status record carries the lot total as `lot_book_units`, read through the same lot reader the audit compares. The Paper Trader and Simulator tables carry the same sentence. Their records build the holdings figure from the stored lots, so there the two figures read the same.

`src/gui/main_tabs/table_cells_surface.py` — the sentence the tooltip carries

```
VENUE_UNITS_NOTE_FORMAT = (
    "Measured on exchange {units:.6f}; holdings {holdings:.6f}, lot book {lots}."
)
ATTRIBUTED_UNITS_NOTE_FORMAT = (
    "NO EXCHANGE COUNT; measured on holdings {holdings:.6f}, lot book {lots}."
)
```

Current reading, 2026-10-29. The second sentence drops two words, so the Ammo description stays within twenty at its longest. The first is unchanged.

`src/gui/main_tabs/table_cells_surface.py` — `ATTRIBUTED_UNITS_NOTE_FORMAT`

```
ATTRIBUTED_UNITS_NOTE_FORMAT = "NO EXCHANGE COUNT; holdings {holdings:.6f}, lot book {lots}."
```

Read off the built cell of a real bot, its status record passed through both tables, before and after one audit pass on the worked example:

```
before the audit   Within dust band — no action pending NO EXCHANGE COUNT; measured on holdings 50.000000, lot book 45.000000.
after the audit    Fold territory — buy deficit on bearish Measured on exchange 40.000000; holdings 40.000000, lot book 40.000000.
```

A row whose record carries no lot total reads `lot book unread`, and a reading that looks for the lot figure reports it missing there.

Every state now fits twenty words. 2,880 combinations of the cell's inputs were composed on both trees.

	before this change	this change
states over twenty words	1,908	0
the longest tooltip, in words	149	20
Qt tooltip equal to the page's	2,880	2,880
every other field of the cell identical	2,880 of 2,880	
the Current Position Value cell identical	72 of 72	

Three clauses were cut to fit. The held reading no longer names the reason, which the bot log keeps. The two-reads note is four words, `Position Value reads holdings.`, because the three figures already show the two counts. The Manual Fire note no longer names the dust band, which the Fire button's own tooltip still names.

Every fill path moves the two priced figures at the fill

A completed trade updates every figure it affects when the exchange confirms the fill, by the count the exchange reports filled. Current Position Value and the Ammo cell now move at that moment on every fill path, and no path reads the exchange more often than it did.

Each path was driven on a bot built in a throwaway home, over a stand-in exchange that fills every order in full, one path at a time. The column on the right counts the balance reads each fill makes, which this change leaves as they were.

fill path	the commit before this change	this change	reads
autonomous scrum	at the fill	at the fill	1
distribution sell	at the fill	at the fill	1
invisible stack rung	at the fill	at the fill	1
visible stack rung	both figures wait for the audit	at the fill	0
initial entry	at the fill	at the fill	1
fold rebuy	at the fill	at the fill	1
hedge buy	at the fill	at the fill	1
Manual Fire, scrum	at the fill	at the fill	1
Manual Fire, fold	at the fill	at the fill	2
wire stack	at the fill	at the fill	1
cartridge	at the fill	at the fill	1
detonation	at the fill	at the fill	1
expiry close	the Ammo cell waits for the audit	at the fill	0
self-destruct	the Ammo cell waits for the audit	at the fill	1
Extractor tranche return	the Ammo cell waits for the audit	at the fill	0

The audit those four waited for runs every 20 ticks. A tick is 5 seconds plus its own run time, so the wait reached 100 seconds, and longer while ticks ran slow. The bot's settings default to stack mode on with the

order book visible, and there an autonomous scrum rests its rungs on the book. Its fills reach the bot through the visible stack rung, the path that moved neither figure.

`src/trading/scrumming_bot.py, in _reconcile_stack_tranches_visible`

```
self._current_holdings = max(0.0, self._current_holdings - _filled)
self._record_venue_units_after_fill(-_filled)
```

The four repaired paths ran on the commit before this change and on this change in one run, the earlier files read from git. Each waited for the audit before and settles at the fill now. The eleven other paths read identical on both. A returned tranche on a bot with no stored exchange count leaves the count at zero, so the Ammo cell keeps pricing the held count instead of the returned units alone.

2026-10-28 - #665 - Min Confidence reaches down to zero, and the detonation vote reads the bot's weights

The span every Min Confidence box reads

Every Min Confidence box now runs from 0.00 to 1.00 and still opens at 0.75. That is the three Qt creation wizards, the three Qt Bot Settings tabs, the React wizard field and the React Bot Settings row. Each reads one span and one opening figure, so no box can offer a different floor.

`src/trading/container/config.py` — the opening figure and the span

```
DETONATION_CONFIDENCE_MIN_DEFAULT: float = 0.75

#: The span every Min Confidence box offers, the span ``VotingEngine`` gives a
#: consensus confidence.
DETONATION_BAR_MIN: float = 0.0
DETONATION_BAR_MAX: float = 1.0
```

The floor comes from the vote

The floor is the lowest reading the vote can give. The engine sets a confidence to the net score over the weight that voted, so it never reads below zero. It calls a vote bullish only above a net score of 0.1, so a bullish confidence can be as small as 0.1 over the weight that voted: 0.0085 at the published weights. No two-decimal bar above 0.00 lets every bullish reading through.

`src/trading/ta_engine.py, in VotingEngine._aggregate`

```
voted_weight = sum(s.weight for s in signals if not s.abstained)
consensus_conf = abs(net) / voted_weight if voted_weight > 0.0 else 0.0
```

Driven on 162 constructed tapes at the published weights, read in every 100-candle window the detonation check fetches:

```
windows of 100 candles   3,078
bullish readings         1,744, from 0.0088 to 0.3578, median 0.1167
at a bar of 0.50             0
at a bar of 0.25            49
at a bar of 0.10         1,026
at a bar of 0.01         1,728
at a bar of 0.00         1,744
```

The detonation vote reads the bot's own twelve

The detonation check now votes at the same twelve weights as the bot's own vote. A new bot holds all twelve at zero, so its vote reads neutral and its detonation cannot fire until Save on the Settings window hands it the TA Indicators figures. The Settings page entry for this day carries the zero start.

`src/trading/scrumming_bot.py, in ScrummingBot._check_detonation_trigger`

```
engine = VotingEngine(weights=self._ta_weights)
```

What each box held, and what the check did

Each tree ran in its own redirected home, the earlier one checked out from git at the commit this change starts from. A figure of 0.10 went into every box. One 100-candle tape read BULLISH at 0.1837.

```
                                  before this change    this change
Qt creation boxes, live paper sim 0.50 to 1.00          0.00 to 1.00
Qt Bot Settings boxes, live paper sim  0.50 to 1.00     0.00 to 1.00
React wizard field, as drawn      min 0.5, max 1        min 0, max 1
React Bot Settings row, as drawn  min 0.5, max 1        min 0, max 1
0.10 put into a box               holds 0.50            holds 0.10
Detonation on, the bar the box held  not fired          fired
Detonation off                    not fired, no fetch   not fired, no fetch
a saved bot holding 0.75          loads 0.75            loads 0.75
a new bot, Detonation on, bar 0.10  -                   not fired; fired after Save
```

The engine holds no floor of its own: a bar of 0.10 set on the config fired on both trees. With the drawn React wizard field taken out of the comparison, the Qt and React readings name that field on one side only, so the comparison can report a missing field.

Where four earlier entries now read differently

They stand as written, and these are the readings that replaced them.

written above	what it reads now
"From 0.50 to 1.00, at 0.75 to start."	From 0.00 to 1.00, at 0.75 to start, on every Min Confidence box
"No figure this box can emit was reached."	A figure the vote reaches can be set: 1,744 of 1,744 bullish readings clear a bar of 0.00
"PROPOSED — `src/gui/main_tabs/live_settings_tab_surface.py`, the row's range"	Not built as proposed. The row reads the one span, 0.00 to 1.00

written above	what it reads now
"And the detonation check builds a fresh voting panel with no weights"	The check votes at the bot's own twelve weights

2026-10-29 - every fill is booked once, from the count the exchange settled

A completed trade changes what the bot holds, what it counts and what it queues. Each of those now changes once for each fill, by the count the exchange reports filled, on every path an order can take. A count the exchange did not report changes nothing.

A resting order is booked when the exchange fills it

At Order Book a scrum or a fold rests as a limit order at the last price. When the re-reads see no fill, the bot keeps the order on its list of held orders, `_resting_orders`, and the activity log says so. The open-order check still refuses a second order on the same side while it rests.

Each worked tick reads the exchange's open orders once and books every part of a held order the exchange filled since the last read. The holdings audit and the start-up read do the same before they compare anything, so no fill is booked twice. An order the exchange closes with nothing filled leaves the list and books nothing.

The read costs one exchange call on each worked tick while an order is held, a second on the tick the holdings audit runs, and one more for each held order that leaves the open list. A bot with no held order makes no added call. A part booked later reads the exchange no more often than a fill inside the re-reads does.

```
# src/trading/bot_container.py, in BotContainer._reconcile_resting_orders
            open_by_id = {str(getattr(o, "id", "") or ""): o for o in (listed or [])}
            for record in [
                r for r in self._resting_orders if r.get("symbol") == symbol
            ]:
                body = open_by_id.get(record["order_id"])
                if body is None:
                    try:
                        body = await self.exchange.get_order(record["order_id"], symbol)
```

Each path books through one function

A later part goes through the same booking function as a fill inside the re-reads. The held count, the lot book, the fold queue, the counts and the announcements therefore move exactly as they do for an immediate fill.

```
fill path                              booked by
scrum                                  _book_sale_fill, then _book_scrum_fill
distribution sell                      _book_sale_fill, then _book_dist_fill
stack spend                            _book_sale_fill, then _book_stack_spend
fold rebuy                             _book_purchase_fill, then _book_fold_fill
initial entry                          _book_purchase_fill, then _book_entry_fill
hedge buy                              _book_purchase_fill, then _book_hedge_fill
manual tranche fire                    _book_purchase_fill, then _book_manual_tranche_fill
Manual Fire, cartridge, wire stack     _book_manual_sale or _book_manual_purchase
detonation                             _book_detonation
self-destruct                          _book_self_destruct
expiry close                           _book_expiry_close
Extractor round and exit               _book_artillery and _book_exit
```

A fold settles the tranches it planned by the share of its order that filled. A part-filled fold drains that share of each planned tranche, and the rest stays queued for the next fold.

Driven, a scrum's limit order filling after the re-reads. The bot held 9.5 units of a stand-in market, $50 over its $900 target. The order for 0.5 units rested, then the stand-in exchange filled it.

```
                           the commit before this change    this change
order placed, held         9.5                              9.5
exchange fills it, held    9.5                              9.5
first worked tick, held    9.5                              9.0, one $49.70 tranche queued, one trade
the next scrum decision    sells 0.5 again                  sizes $0.00 and sells nothing
after the holdings audit   held 8.5, $50 under target       nothing to correct
```

Driven, a fold's limit order filling after the re-reads. The order bought back one $50 tranche.

```
                           the commit before this change    this change
first worked tick          the tranche still queued         the tranche settled, 0.526 units booked
the next fold decision     buys the same $50 again          no tranche is eligible
after the holdings audit   0.526 units adopted as a lot,    nothing to correct
                           $52.50 over target
```

A part-filled scrum booked 0.2 of its 0.5 units at once and the other 0.3 at the next worked tick, one trade for the one order. A restart while a fold rested carried the held order in the saved state and booked it during start-up, so the next fold found nothing to buy a second time.

The detonation, self-destruct, expiry close and Extractor read the settled count

These paths booked from the order's first reply alone. Each now reads the order the way the scrum does, through `_read_settled_order`, and a count of zero books nothing at all.

```
# src/trading/scrumming_bot.py, in ScrummingBot.self_destruct
        fill_amount, fill_price, _ = await self._settled_fill(
            order, self.config.symbol, units, ref_price, label="SELF-DESTRUCT"
        )
```

```
the first reply says 0 filled    the commit before this change        this change
detonation, exchange fills       nothing booked                       3 units booked, queue cleared
self-destruct, exchange fills    0 units booked, bot paused           9 units booked, bot paused
self-destruct, no fill           queue cleared, a trade counted,      nothing changes, the order is held
                                 bot paused, a 0-unit fill announced
expiry close, exchange fills     9 held against the exchange's 0      0 held, no lot left, one trade
Extractor round, no fill         a 0.05-unit position opened          no position, the pool unchanged
Extractor exit, no fill          the position closed, a gain booked   the position and the pool unchanged
```

An expiry close now also takes each settled fill off the lot book, highest buy price first, and counts the order once as a trade. The lot book no longer waits for the holdings audit after an expiry close.

```
# src/trading/scrumming/tick_phases.py, in TickPhaseMixin._book_expiry_close
        self._take_filled_units_off_lots(filled)
        self._current_holdings = max(0.0, self._current_holdings - filled)
```

Trade volume counts each fill once

`BotContainer._count_trade_volume` is the one writer of the trade volume. Each fill adds its units times its price, in dollars, once.

```
# src/trading/bot_container.py, in BotContainer._count_trade_volume
        self.stats.trade_volume += notional
```

```
trade volume over the fill's value    the commit before this change    this change
scrum, distribution sell              1.99                             1.00
fold, hedge buy                       2.00                             1.00
entry, manual tranche, stack spend    1.00                             1.00
the eleven other paths                0.00                             1.00
```

The realised figure the P&L milestone reads

The P&L milestone alert reads the fleet's realised total, which adds each bot's `stats.realised_pnl`. Nothing wrote that field, so the total read zero. The exchange walk now writes it with the figure the header strip's realised column already draws.

```
# src/trading/scrumming/reconciliation.py, in refresh_exchange_position_health
                self.stats.realized_pnl_exchange = self.stats.realised_pnl = float(
                    _ph.realized_pnl_usd
                )
```

The walk runs on a tick at most once in 300 seconds, and the start-up read forces one. The figure therefore moves at the walk, up to 300 seconds after the fill. A fill still makes no walk.

```
# src/trading/scrumming/reconciliation.py, in ReconciliationEngineMixin
    EXCHANGE_HEALTH_REFRESH_COOLDOWN_SEC = 300.0
```

Driven, one scrum and one fold, then one exchange walk, with the alert threshold at $1.

```
                                    the commit before this change    this change
fleet realised total after the walk  $0.00                            $5.00
P&L milestone alerts                 none                             one, P/L Milestone: $5.00
the risk monitor's total P/L         $0.00                            $5.00
```

Driven, the tick's walk call inside the 300 seconds, around the same scrum and fold. Neither commit walked, before or after either fill, so the change adds no walk to a fill.

Every reader of `stats.realised_pnl` now reads the exchange's figure where it read zero. The Swarm card prints it as the bot's P/L. The risk monitor adds it to its total P/L and checks each bot against its per-bot loss limit.

```
# src/gui/visualizer/bot_node.py, in BotNodeWidget.paintEvent
            pnl = stats.get("realised_pnl", 0)

# src/trading/risk_manager.py, in RiskManager._build_snapshot
            pnl = stats.get("realised_pnl", 0) + stats.get("unrealised_pnl", 0)
```

What a held order stops

A Manual Fire, a cartridge or a wire stack whose order is still held answers the next press with MANUAL FIRE HELD and sends nothing. The expiry close sends nothing more while its order is held. The Extractor sends no round and no exit on a pair while an order on that pair is held.

Now: the wire stack is gone, and a Manual Fire or cartridge buy also sends nothing while the venue lists an open buy on the market, one with no order id included (`src/trading/scrumming/execution.py, in _refuses_stacked_buy`).

```
# src/trading/scrumming/execution.py, in _execute_manual_rebalance
        _held = self._resting_order_for("_book_manual_sale", "_book_manual_purchase")
```

The Product Manual PDF rebuild is owed for this page.

2026-10-29 - every verified fill is announced once, in one shape

A fill is announced once, when its booking ends, in one shape. The trade log, the fill alert, the fill sounds and the Charts mark read that one announcement, and the desktop shell receives it as well. No table and no figure on a screen moves because of it yet.

The shape is declared once

Every announcement carries ten fields and one fee field. The fee field holds the dollars the exchange charged, or the reason no fee could be read. The topic is still `trade.filled`, because every reader subscribes by that name and a second topic beside it would announce each fill twice.

```
# src/core/emit_contracts.py, in SettledFill
    bot_id: str
    side: str
    type: str
    amount: float
    price: float
    usd: float
    profit: float
    operator_initiated: bool
    symbol: str
    exchange: str
    fee_usd: Optional[float] = None
    fee_refusal: Optional[str] = None
```

One function sends it

Each booking function sends its fill through one call. The bot, its exchange and its symbol come from the bot itself, so no path can name them differently.

```
# src/trading/bot_container.py, in BotContainer._announce_fill
        fill = SettledFill(
            bot_id=str(self.bot_id),
            side=side,
            type=kind,
            amount=float(units),
            price=float(price),
            usd=float(usd),
            profit=float(profit),
            operator_initiated=bool(operator_initiated),
            symbol=str(symbol or self.config.symbol),
            exchange=str(self.config.exchange_id),
            **fee,
        )
        self._bus.emit(FILL_TOPIC, **fill.payload())
```

The call is the last step of every booking. A reader that looks at the bot when the announcement arrives finds every figure the fill moved already written.

```
# src/trading/scrumming/tick_phases.py, in TickPhaseMixin._book_scrum_fill
        self._last_trade_price = sell_fill
        self._announce_fill(
            kind="SCRUM",
            side=FILL_SELL,
            units=_sold_units,
            price=sell_fill,
            usd=scrum_usd,
            profit=0.0,
            operator_initiated=False,
            fee=self._fill_fee_fields(sell_fill),
        )
```

Five kinds of fill now announce

These five booked their fills and told no reader. Each now announces under its own kind, and a part that fills later is announced by the same booking.

```
kind                  booked by
STACK_SPEND           ScrummingBot._book_stack_spend, each part of the order
STACK_RUNG            ScrummingBot._reconcile_stack_tranches_visible
EXPIRY_CLOSE          TickPhaseMixin._book_expiry_close
EXTRACTOR_ARTILLERY   ExtractorBot._book_artillery
EXTRACTOR_EXIT        ExtractorBot._book_exit
```

An Extractor exit pressed from Positions Held is marked as started by the operator. The Extractor reads the fee off its own order, in the currency the pair is priced in.

```
# src/trading/extractor_bot.py, in ExtractorBot.manual_fire_position
            await self._execute_bullish_exit(
                pos, alt_price_in_base, exit_pct=FULL_EXIT_PCT, operator_initiated=True
            )
```

The readers read the one shape

The four readers and the desktop shell read each announcement through one function. A missing field, a value of the wrong kind, or a fee other than exactly one is refused and named. No field takes a default.

```
# src/core/emit_contracts.py, in read_settled_fill
    faults = _fill_faults(event_data)
    if faults:
        raise FillShapeError("; ".join(faults))
```

Each reader calls it before it acts.

```
reader                 reads the fill in
the trade log          LogManager._on_trade_filled_bus
the fill alert         NotificationManager._on_trade_filled
the fill sounds        fill_fields, for MainWindow._on_trade_filled_sfx
the Charts mark        fill_record, for both Charts tabs
the desktop shell      follow_fills
```

The announcement reaches the desktop shell

The bridge publishes each announcement as the `fill` push section. The shell hands the frame to the window, and the panel host keeps the newest one for any panel that follows that section. No panel follows it yet.

```
# src/core/desktop_bridge.py, in follow_fills
        live.publish(FILL_SECTION, **fill.payload())
```

What the readings showed

Every completion point was driven on the commit before this change and on this change. A real bot traded against a stand-in exchange, in a home folder made for the run, with the real readers attached and no socket leaving the machine.

```
                              the commit before this change     this change
shapes an announcement took   6, two flat and four nested        1
kinds announced, settled fill 13 of 18                           18 of 18
kinds announced, a later fill 13 of 17                           17 of 17
a part, then the rest later   2 announcements on 13 kinds,       2 announcements on 17 kinds,
                              none on 4                          units summing to the fill
the desktop shell             no frame                           one fill frame per fill
```

On the 13 kinds that announced before, the alert, the sounds and both Charts marks read the same content on both commits. The trade log row differs in one field only.

```
the trade log row of a scrum     the commit before this change     this change
exchange                         empty                             the bot's exchange
every other field                the same                          the same
```

Before this change a reader filled a missing field with zero. With the units taken out, the trade log wrote 0 units and the alert read 0.000000 units. Now each of the ten fields taken out in turn is refused by name, and so are both fee fields at once and no fee field at all. No row, alert, sound, mark or frame follows a refused announcement.

```
taken out              the reader answers
amount                 FillShapeError: amount absent
side                   FillShapeError: side absent
both fee fields        FillShapeError: 2 of fee_usd, fee_refusal present, not 1
nothing                read, and every reader acts on it
```

Each of the 17 held-order kinds filled 40 per cent with its order and the rest at the next held-order read. Each order was announced twice, once for each part, and the units summed to the fill. The same count with one announcement sent a second time read 3 announcements and more units than the exchange filled.

```
a scrum of 1.25 units          announcements    units announced
40 per cent, then the rest     2                1.25
one announcement sent twice    3                1.75
```

Where earlier sentences now read differently

They stand as written. These are the readings that replaced them.

written above	what it reads now
"The Product Manual PDF rebuild is owed for this page."	`docs/Acervator-Product-Manual.pdf` is rebuilt with this section and carries this page

2026-10-29 - every figure a fill settles is written when the fill is announced

When a fill is announced, the bot first writes every figure that fill settles. These are the Trades count, the year-to-date dollars, the realised profit, the fees, the average entry and cost basis, the open-order counts, the stored position value, and the buy and sell counts. The bot tables pull each bot's status every two seconds, so a row shows the new figures at its next pull. The exchange walk still reads the exchange once in 300 seconds. Where its figure differs from the one written at the fill, the walk writes the exchange's figure.

One function writes the figures

The announcement writes the figures before it sends the fill. A figure that cannot be written is logged, and the fill is still announced.

```
# src/trading/bot_container.py, in _announce_fill
        try:
            self._write_fill_figures(fill)
        except Exception:
            logger.exception(
                "Bot %s %s fill of %.8f: writing its figures raised, and the "
                "fill is still announced",
                self.bot_id,
                kind,
                fill.amount,
            )
        self._bus.emit(FILL_TOPIC, **fill.payload())
```

The trade count, the buy or sell count and the last trade time count each order once, on the first piece the exchange settles. The order body that settled the piece shows whether it is the first.

```
# src/trading/bot_container.py, in BotContainer._write_fill_figures
        if filled - units <= FILL_DUST_UNITS:
            self.stats.total_trades += 1
            if fill.side == FILL_BUY:
                self.stats.total_buys += 1
            else:
                self.stats.total_sells += 1
        self.stats.last_trade_time = time.time()
```

The walk keeps what it read

Each walk keeps three readings. They are the exchange's fill history for the symbol, the year-to-date tally with the units of every order, and the open orders. A fill joins each reading through the walk's own arithmetic, so the figure written at the fill is the figure the walk computes once the exchange lists that fill.

```
# src/trading/scrumming/reconciliation.py, in ReconciliationEngineMixin._write_clock_figures
        price = float(fill.price)
        if math.isfinite(price) and price > 0.0:
            self.stats.position_value = priced_usd(
                self._current_holdings, price, float(self._quote_to_usd or 1.0)
            )
        if fill.symbol != self.config.symbol:
            return
        self._join_ytd_tally(fill, order_id, filled)
        self._join_fill_history(fill, order_id, filled)
        self._join_open_orders(fill, order_id, rests)
```

A fill adds only the units of its order that a reading does not already hold. The walk runs first in a worked tick, so it can read a resting order that the exchange filled before the held-order read books it. That order is then in the reading already, and it is not counted a second time.

```
# src/trading/scrumming/reconciliation.py, in unread_units
    return max(0.0, min(float(units), float(filled) - float(held)))
```

The exchange's figure is the one that stands

Each walk replaces the three readings and drops the fills that joined them since the last walk. Where the exchange lists a fill at another price or with another fee, or lists an order the platform did not place, the walk writes the exchange's figure.

```
# src/trading/scrumming/reconciliation.py, in ReconciliationEngineMixin.refresh_exchange_position_health
            _trades = await self.fetch_fill_history()
            if _trades is None:
                return False
            self._fills_since_walk = ()
```

Other bots on the same exchange and symbol

A bot's walk reads every fill the exchange holds for its symbol, so a fill by another bot on the same exchange and symbol moves that bot's walk figures too. The announcing bot adds its fill to the readings of each such bot. An Extractor round or exit on a pair that a Scrumming Bot trades reaches that bot the same way. The other bot's own position value does not move.

```
# src/trading/container/registry.py, in BotRegistryMixin.bots_on_symbol
        return [
            bot
            for bot in list(self._bots.values())
            if getattr(getattr(bot, "config", None), "exchange_id", None) == exchange_id
            and getattr(getattr(bot, "config", None), "symbol", None) == symbol
        ]
```

What stays on the walk

Four figures still move at the walk alone. Each needs a reading only the exchange can give.

```
figure                                      what the walk reads for it
cash                                        the exchange's balance of USD and USDC
unrealised profit                           the exchange's spot position
fill count                                  the pieces the exchange filled each order in
average entry and cost basis, on an         the exchange's portfolio breakdown
exchange that answers a spot position
```

The profit an announcement carries

A distribution sale announces the change in the target that its booking made, and a manual tranche fire announces the growth it added to the target, as a fold does. A distribution sale grows nothing, so it announces $0.00. The fill sounds play the profit sample on a profit above zero. A distribution sale therefore no longer plays it, and a manual tranche fire that grows the target does.

```
# src/trading/scrumming/tick_phases.py, in TickPhaseMixin._book_dist_fill
            profit=float(self._target_balance) - _target_before,

# src/trading/scrumming_bot.py, in ScrummingBot._book_manual_tranche_fill
            profit=_mf_growth,
```

What the driven fills showed

Every completion point was driven on the commit before this change and on this change, one kind of fill at a time. A real bot traded against a stand-in exchange that keeps a fill history, in a home folder made for each run, with no socket leaving the machine.

```
                                    the commit before this change      this change
the row's Trades cell after a scrum 3 at the next pull, 4 at the       4 at the next pull
                                    walk 300.5 s after the fill
figures written at the fill         the fleet trade total on 15 of     every figure but cash and the
                                    18 kinds, the year-to-date         fill count, on all 16 kinds of
                                    dollars on 7                       the Scrumming Bot
figures the walk moved after it     every figure the walk owns         cash and the fill count only
the figure at the fill against the  -                                  the same on all 16 kinds
exchange's own arithmetic
a resting scrum the walk read first fleet total 5 and $352.00 sold,    4 and $227.00, the exchange's
                                    the exchange's 4 and $227.00       figures
exchange calls during each fill     2 to 7, by kind                    the same calls in the same order,
                                                                       on all 18 kinds
```

The Extractor holds no walk. Its round and its exit now add to its trade count and its buy or sell count, which no Extractor fill moved before.

Two Scrumming Bots on one exchange and symbol were registered on a real bot manager, with an Extractor beside them. A scrum by the first bot, and then an Extractor round on the same symbol, each moved the second bot's Trades count, year-to-date dollars, fees and basis at the fill. Its next walk moved none of them. On the commit before this change, each waited for the second bot's own walk.

With the exchange listing the scrum 1 per cent lower and at twice the fee, and listing an order the platform did not place, the walk wrote the exchange's figures over the ones written at the fill.

```
the walk after the scrum     written at the fill    the walk, the exchange's figure
the row's Trades cell        4                      5
realised profit              $24.50                 $23.25
fees                         $6.882                 $7.926
cost basis                   $713.00                $762.00
```

Where two earlier sentences now read differently

They stand as written. These are the readings that replace them.

written above	what it reads now
"The figure therefore moves at the walk, up to 300 seconds after the fill."	The realised figure moves when the fill is announced, while the bot holds a complete fill history from its last walk. The walk then writes the exchange's figure over it
"No table and no figure on a screen moves because of it yet."	The announcement writes every figure the fill settles before it is sent, and the bot tables draw them at their next two-second pull

The Product Manual PDF rebuild is owed for this page and for `docs/manual/08-tabs/settings.md`.

Indicator Voting Panel

This panel shows the readings for each of the 12 as well the collated indices and confidence score.

TF Lock - To be re-evaluated.

The control is removed. `indicator_panel.py` builds no `_tf_lock_combo`, and `indicator_panel_surface.py` publishes no `tf_lock` block, so nothing puts a timeframe on `indicator.tf_lock_changed` any more. `main_window.py` still subscribes to that topic; with no publisher, `_lock_timeframe` keeps whatever the coordinator sets for itself.

`src/gui/indicator_panel.py` — the header, after the removal

```
header.addStretch()
header.addWidget(QLabel("Bot:"))
self._bot_selector = QComboBox()
```

The bot selector and its privacy dot close the header on the right. The pair symbol and the vote tally that stood there are gone: `_symbol_label` and `_summary_label` are not built, and the payload carries no `symbol_text` or `summary_text`.

A drop-down of eleven entries: None, then Lock below each of ten timeframes from 5m to 1w. It opens on the fifth entry, Lock below 1h. Choosing one writes a status line beside the box and puts the timeframe on the event bus.

The main window takes that event and sets `_lock_timeframe` on the coordinator of every running Scrumming Bot. Nothing declares or reads that name, so the choice reaches the coordinator and stops there. The tooltip on the box says the lock feeds directly into active Scrumming Bots. Issue #155 carries this.

The issue #155 clause above is overtaken. It is not edited and not removed. It read:

The tooltip on the box says the lock feeds directly into active Scrumming Bots. Issue #155 carries this.

Issue #155 closed on 2026-10-22, so the pointer names no open work. The reading above it still holds: the write reaches no reader. The name the window sets on the coordinator and the name the Phantom Bots pages read are two different names, and only the second is declared on the bot.

```
src/gui/main_window.py, in _on_tf_lock_changed
    writes   bot._coordinator._lock_timeframe
    readers  none

src/trading/scrumming_bot.py        declares _phantom_lock_timeframe on the bot
src/gui/main_tabs/phantom_bots_tab_surface.py   LOCK_TIMEFRAME_ATTRIBUTE names that one
src/gui/live_settings/phantom_bots_tab.py       reads it off the bot
src/gui/paper/paper_phantom_bots_tab.py         reads it off the bot
src/gui/simulator/sim_phantom_bots_tab.py       reads it off the bot
```

Which of the two names the drop-down should set is a product choice, so nothing here proposes one.

`src/gui/main_window.py` — `_on_tf_lock_changed`

```
for bot in self._bot_manager._bots.values():
    if isinstance(bot, ScrummingBot) and hasattr(bot, "_coordinator"):
        bot._coordinator._lock_timeframe = tf
```

The two clauses above are overtaken. Neither is edited and neither is removed. They read:

> *The reading above it still holds: the write reaches no reader. The name the window sets on the coordinator and the name the Phantom Bots pages read are two different names, and only the second is declared on the bot.*
>
> *Which of the two names the drop-down should set is a product choice, so nothing here proposes one.*

The choice is settled, and it is an implementation one. The name the three Phantom Bots pages already read is the one that survives. The handler sets that name on the bot and keeps setting the older name on the coordinator, so a reader of the older spelling still finds its value.

`src/gui/main_window.py` — `_on_tf_lock_changed`, as it stands

```
for bot in self._bot_manager._bots.values():
    if not isinstance(bot, ScrummingBot):
        continue
    setattr(bot, LOCK_TIMEFRAME_ATTRIBUTE, tf)
    if hasattr(bot, "_coordinator"):
        bot._coordinator._lock_timeframe = tf
```

Driven on a hand-built record, with two different settings in one run, both names hold the chosen timeframe and the lock row on the page carries it.

lock set to	what the pages read	the page's SCRUM lock state row
nothing yet	empty	LOCKED — SCRUM suppressed by TF phantom
4h	4h	LOCKED — SCRUM suppressed by 4h TF phantom

lock set to	what the pages read	the page's SCRUM lock state row
1d	1d	LOCKED — SCRUM suppressed by 1d TF phantom

The first row is what the page drew for every setting before this change: a gap where the timeframe belongs.

Live, Paper and the Simulator read the one name, so all three draw the timeframe once a value arrives. Only the Live window subscribes to the topic, and nothing publishes it, so no press reaches any of the three today. Parity across the three surfaces is not closed while the control is absent.

Three clauses above are overtaken. None is edited and none is removed. They read:

> `main_window.py` *still subscribes to that topic; with no publisher,* `_lock_timeframe` *keeps whatever the coordinator sets for itself.*
>
> *The handler sets that name on the bot and keeps setting the older name on the coordinator, so a reader of the older spelling still finds its value.*
>
> *Only the Live window subscribes to the topic, and nothing publishes it, so no press reaches any of the three today.*

The current reading. The operator has ruled that the control is not coming back to this panel and that the setting stays with the Phantom Bots page. The handler and its subscription are therefore removed, and so is its write onto the coordinator. The window subscribes to nothing on that topic, and the register in `main_window_surface.py` no longer names either the topic or the handler.

The bot now reads the lock from the coordinator instead. One method on the bot writes both names every tick, from the coordinator's own active lock list, and clears both where the operator has phantom bots switched off.

Which ticks reach it, measured. The call sits on the main gate path, so it runs on every tick that reaches the scrum flag and on no tick that returns above it. A hand-built bot driven through the real tick ran eight full passes and reached the call on none of them, because its position value equals its target balance, which leaves no delta and returns at the interval guard. Twelve returns sit above the call. Calling the method directly on the same bot moved the flag to locked and the timeframe to the lock's own, so the zero is a fact about the path that tick took and not about the reading.

`src/trading/scrumming_bot.py` — `ScrummingBot._refresh_phantom_lock`

```
if not self._phantoms_enabled:
    self._phantom_locked = False
    self._phantom_lock_timeframe = ""
    return
timeframe = str(coordinator.active_lock_timeframe() or "")
self._phantom_locked = bool(timeframe)
self._phantom_lock_timeframe = timeframe
```

The timeframe comes from the lock itself, so the page names the timeframe that suppressed the scrum rather than leaving the gap the first table row shows.

`src/trading/phantom_balance.py` — `TimeframeCoordinator.active_lock_timeframe`

```
held = [str(one.get("source_tf") or "") for one in self.get_active_locks()]
if not held:
    return ""
return max(held, key=tf_rank)
```

Driven. A bot was built by hand with a stand-in venue that refuses every order, and a lock was placed on its coordinator for a timeframe above the parent. Four arms ran in one pass, and a second pair ran on a ripe row.

```
a lock on the coordinator, read at the bot and at the page

phantoms off, lock active      flag False   page UNLOCKED — SCRUM allowed
phantoms on, lock active       flag True    page LOCKED — suppressed by 1d
phantoms on, lock expired      flag False   page UNLOCKED — SCRUM allowed
phantoms on, no lock           flag False   page UNLOCKED — SCRUM allowed
```

The locking arm and the non-locking arm answer differently at the bot, at the scrum flag and on the page, so the reading can see the lock. Before this change all four arms answered alike, with the coordinator reporting the same active lock.

Swept across the band from -0.3 to 1.3 in steps of 0.0005, which is 3,201 rows, with phantom bots on and a lock held: the scrum flag changes on 1,601 rows and the midline refusal appears on 801 more, between 0.5 and 0.9. The final scrum decision changes on none of the 3,201 rows.

```
3,201 rows, phantoms on, a held lock against an expired one

scrum flag differs                     1,601
midline refusal differs                  801
final scrum decision differs               0
fold decision differs, as a control        0
```

Why no trade changes. On a ripe row the ripe-harvest override removes the midline refusal, and a row below the upper detect threshold is already refused by band proximity. Driven on a ripe row at 0.95 with the lock held, the scrum flag reads false, the blocked list is empty and the scrum fires. Whether the band should hold a ripe scrum is the open decision on issue #1359.

With phantom bots switched off, every one of the 3,201 rows is identical before and after the change, compared row by row on the scrum flag, the midline refusal, the final scrum decision and the fold decision.

Live, Paper and the Simulator draw from one page model, and all three were driven with the same locked bot. All three answer the identical sentence, and all three answer the unlocked sentence when the lock expires, so parity across the three is closed on that row. A Paper or Simulator page fed a stored record rather than a running bot draws the unlocked sentence, because a saved record carries the phantom settings and never a lock.

Nothing creates a lock on the coordinator yet, so the flag reads clear on every running bot today. What should create one changes which trades fire and is the operator's decision.

The operator's ruling, 2026-10-27. Asked what should create a phantom lock, he answered:

This is a strange question. A phantom bot is simply a higher timeframe read that is piped through a parallel trading gate logic chain. It is intended to improve rather than restrict ripe scrums or folds so it must work on both sides. It should be thought of as an optional guardian angel that may optimize trades.

On precedence he answered that where the phantom does hold, it holds ahead of the ripe-harvest override.

Three readings above are overtaken. None is edited and none is removed. They read:

1d	1d	LOCKED — SCRUM suppressed by 1d TF phantom

fold decision differs, as a control 0

Nothing creates a lock on the coordinator yet, so the flag reads clear on every running bot today. What should create one changes which trades fire and is the operator's decision.

The phantom read reaches both sides. The fold flag now carries the same lock term the scrum flag carries, written by the same method on the bot, taken from the same active lock list, and read by the same gate on the other chain. The term is optional per bot through the tick box the operator already has: with phantom bots switched off, that method clears the flag and neither side reads anything.

`src/trading/scrumming_bot.py` — `ScrummingBot.tick`, both flags

```
self._refresh_phantom_lock()
if self.config.bb_midline_gate:
    scrum_ok = (bb_pos > 0.50) and not self._phantom_locked
    fold_ok_midline = (bb_pos < 0.50) and not self._phantom_locked
else:
    scrum_ok = not self._phantom_locked
    fold_ok_midline = not self._phantom_locked
```

Driven through the real tick. A bot was built by hand with a stand-in venue that refuses every order, one tranche queued and a position above target, so the tick reached both gate chains rather than returning above them. Four arms ran, each of them twice: once with the fold's own technical-analysis tick box on, and once with it cleared, which is a box the operator sets.

```
band position 0.11, the fold gate's own verdict, the TA box cleared

phantoms off, lock held        fold gate passes      fold fires
phantoms on,  lock held        fold gate refuses     fold held
phantoms on,  lock expired     fold gate passes      fold fires
phantoms on,  no lock          fold gate passes      fold fires
```

Swept, before against after. Each arm ran over 801 window levels from 0.800 to 1.200 in steps of 0.0005. That moved the real band position from -0.558 to 1.559 and put 398 of the 801 rows below the midline. Every row was compared on eleven recorded verdicts. A second sweep of 2,001 rows at a step of 0.000001 covered the midline crossing on its own, reading band positions from 0.4737 to 0.5181, and the fold refusal appeared on exactly the 1,185 rows below 0.50 and on none of the 816 at or above it.

```
801 rows an arm, eleven verdicts a row, two trees

phantoms off, lock held          every verdict identical
phantoms on,  no lock            every verdict identical
phantoms on,  lock expired       every verdict identical
phantoms on,  lock held          fold flag differs on            398
                                 fold gate refusal differs on    398
                                 fold decision differs on        362
                                 every scrum reading differs on    0
```

The 398 rows are the rows below the midline, where the band used to pass the fold on its own. The 362 are the rows where no other gate was holding the fold, so the lock is what changed the trade.

The control on that reading. Measured on the same rows after the change, a held lock against no lock differs on 801 lock readings, 403 scrum flag readings, 398 fold flag readings and 362 fold decisions, so the comparison can report a difference. The scrum side and three neighbouring fold gates were read in every arm the same way and all of them are identical before and after. No venue socket was attempted in either run, and the counter reached one on a single refused attempt.

Which surfaces it reaches. Live reads the lock on both sides. Paper and the Simulator compose both flags from the band alone and read no lock, before this change and after it, so neither moved. The Simulator's replay of a Live record does recover the lock, and it now applies it to both sides, so a replayed record answers what Live answered.

`src/simulator/validation.py` — `rerun_context`, the replay's two flags

```
if bot.bb_midline_gate:
    scrum_ok = (bb_pos > BB_MIDLINE) and not locked
    fold_ok_midline = (bb_pos < BB_MIDLINE) and not locked
else:
    scrum_ok = not locked
    fold_ok_midline = not locked
```

Still the operator's. What should create a lock on the coordinator. Nothing creates one today, so the flag reads clear on every running bot. The naming has not moved either: the flag, the page row quoted above and the log line all still describe restriction rather than the guardian his ruling names.

Bot - Drop down menu for selecting which bot's TA signals are displayed.

The list refills every tick and carries one entry per running bot, opening on a placeholder until one is chosen. Choosing a bot is what makes the twelve columns below read anything at all.

`src/gui/indicator_panel.py` — the selector

```
self._bot_selector = QComboBox()
self._bot_selector.setMinimumWidth(180)
self._bot_selector.addItem("(select a bot)", "")
self._bot_selector.currentIndexChanged.connect(self._on_bot_selected)
```

TF - Timeframe for the selected bot.

One row per timeframe, each row a separate verdict from the same twelve voters. The rows are the bot's own timeframes and any phantom timeframes it runs.

In development.

BB - Bollinger Bands - https://en.wikipedia.org/wiki/Bollinger_Bands

In Acervator, Bollinger Bands are used as the thresholds or boundaries at which trades are allowed to fire. Multiple market structure pieces are intertwined with our readings of the bands such as Landing Strip Detection (Heikin-Aishii Candle Consolidation Pattern) and Minimum Opposing Trade Distance. From a tactical standpoint, these represent the area or zone through which an investment position is passing.

The cell prints a direction arrow and a confidence percentage. Direction comes from the band position alone: under 0.15 or under 0.35 votes bullish, over 0.85 or over 0.65 votes bearish, and anything between the two middle figures votes neutral. Bollinger Bands carries the formula.

`src/trading/indicators/bollinger.py` — the vote

```
if bb_pos < 0.15:
    direction = SignalDirection.BULLISH
    confidence = max(0.0, min(1.0, (0.15 - bb_pos) / 0.15 * 0.8 + 0.3))
elif bb_pos > 0.85:
    direction = SignalDirection.BEARISH
    confidence = max(0.0, min(1.0, (bb_pos - 0.85) / 0.15 * 0.8 + 0.3))
elif bb_pos < 0.35:
    direction = SignalDirection.BULLISH
    confidence = 0.2
elif bb_pos > 0.65:
    direction = SignalDirection.BEARISH
    confidence = 0.2
```

VTX - Vortex Indicator - https://en.wikipedia.org/wiki/Vortex_indicator

In my experience (which is entirely subjective), this particular indicator is unusually strong at flagging early trend reversals. For Acervator, we are generally looking for signal saturation or near-saturation in either polarity which, under optimum conditions, will align with Bollinger Band thresholds being hit.

The cell prints an arrow and a percentage. The crossover of the two lines decides the vote, and then Acervator adds one reading of its own: the direction is overwritten to bullish whenever the downward line reaches 1.30, whatever the crossover said. Vortex Indicator carries the formula.

`src/trading/indicators/vortex.py` — the ceiling that overwrites the vote

```
VX_CEILING_PCT = 130.0
```

MACD - Moving Average Convergence Divergence - https://en.wikipedia.org/wiki/MACD

This is one of the oscillators used and, with it, we are looking for “top of the hill” formations in either polarity and these, under optimum conditions, will align with Bollinger Band thresholds being hit. Keep in mind that I do not approach any of these indicators from a mathematical standpoint as I am a purely visual trader without a quant or certified technical analysis background.

The cell prints an arrow and a percentage. A signal-line crossover decides the vote first. Failing that, a histogram growing away from zero votes with its own sign, and a histogram merely on one side of zero votes the same way with less conviction. MACD carries the formula.

`src/trading/indicators/macd.py` — the crossover, tested first

```
if prev_macd <= prev_signal and curr_macd > curr_signal:
    direction = SignalDirection.BULLISH
```

SRsi - Stochastic RSI - https://en.wikipedia.org/wiki/Stochastic_oscillator

Our second oscillator and, like the vortex indicator, we are looking at initial maximizing or prolonged periods of saturation of the signal at either polarity and this will, under optimum conditions, align with other indicators to trigger a trade.

The cell prints an arrow and a percentage. The vote needs a crossover of the two smoothed lines, and how deep in the range that crossover happens sets the confidence: a bullish cross under 30 scores 0.8, under 50 scores 0.5, and higher still scores less. Stochastic RSI carries the formula.

`src/trading/indicators/stochastic_rsi.py` — the bullish crossover

```
if prev_k <= prev_d and k > d:
    crossover = "bullish"
    if k < 30:
        direction = SignalDirection.BULLISH
        confidence = 0.8
```

Ichi - Ichimoku Cloud - https://en.wikipedia.org/wiki/Ichimoku_Kink%C5%8D_Hy%C5%8D

This particular indicator is one of the few ‘predicative’ indicators. In Acervator, we are focused on its additional ability to strengthen market reversal zone detection which is where we prefer to shave off or fold in funds.

The cell prints an arrow and a percentage. Several cloud readings add into one score, and only a score past 0.08 either way casts a vote. The score doubles as the confidence. Ichimoku Cloud carries the formula.

`src/trading/indicators/ichimoku.py` — the vote

```
if score > 0.08:
    direction = SignalDirection.BULLISH
    confidence = max(0.0, min(1.0, score))
elif score < -0.08:
    direction = SignalDirection.BEARISH
    confidence = max(0.0, min(1.0, abs(score)))
else:
    direction = SignalDirection.NEUTRAL
    confidence = 0.0
```

Net - Primary TF panel index collated from all 12 indicators.

The cell prints a signed figure to two decimals, green above zero and red below. It is the weighted bullish total less the weighted bearish total, and an abstaining voter adds nothing to either side. Net, Comp and Conf carries

the arithmetic.

`src/gui/indicator_panel.py` — the Net cell

```
net = tf_data.get("net_score", 0)
item = QTableWidgetItem(f"{net:+.2f}")
```

Comp - Composite TF panel index provided by the highest TF Phantom Bot that is active. Phantom bots are still in active development so any related features are not yet working.

The cell prints an em dash when no composite figure is present, which is what a fleet with no active phantom shows. Net, Comp and Conf carries the arithmetic.

In development.

Conf - Confidence reading for the panel.

This is the ‘breadth’ of agreement between all 12 indicators.

The cell draws a ten-block bar and the percentage beside it. The bar fills one block per tenth. Green from 0.6, amber from 0.3, grey below that.

`src/gui/indicator_panel.py` — the Conf cell

```
conf = tf_data.get("confidence", 0)
conf_bar = "■" * int(conf * 10) + "■" * (10 - int(conf * 10))
item = QTableWidgetItem(f"{conf_bar} {conf:.0%}")
```

Sling - CM (Chris Moody) Slingshot -

https://www.tradingview.com/script/GE7tSQK1-CM-Sling-Shot-System/

Another excellent indicator for reversal detection and here it adds to our laser precision detection of such zones and structures.

The cell prints an arrow and a percentage. A squeeze breaking either way votes first, and a band snapback votes when no squeeze is breaking. Slingshot carries the formula and the attribution, which is shared between three authors rather than one.

`src/trading/indicators/slingshot.py` — the squeeze, tested first

```
if squeeze_bull:
    direction = SignalDirection.BULLISH
    confidence = squeeze_conf
    active_type = "squeeze_bull"
```

ADX - https://en.wikipedia.org/wiki/Average_directional_movement_index

This is a trend strength indicator but, as with the other indicators, we are using it to detect reversal by way of signal decreasing.

The cell prints the raw ADX value, 0 to 100, not a percentage. Below 20 the cell reads Rng and the figure, with no arrow. The vote itself is decided by which directional line is on top, at any ADX, so a ranging market still

casts a directional vote even though the cell shows no arrow. ADX and DMI carries the formula.

`src/trading/indicators/adx.py` — the vote

```
if bull_dominant and strong_trend:
    direction = SignalDirection.BULLISH
    confidence = max(0.0, min(1.0, (adx - 35) / 30 + 0.5))
elif bear_dominant and strong_trend:
    direction = SignalDirection.BEARISH
    confidence = max(0.0, min(1.0, (adx - 35) / 30 + 0.5))
elif bull_dominant:
    direction = SignalDirection.BULLISH
    confidence = max(0.0, min(0.5, adx / 70))
elif bear_dominant:
    direction = SignalDirection.BEARISH
    confidence = max(0.0, min(0.5, adx / 70))
else:
    direction = SignalDirection.NEUTRAL
    confidence = 0.0
```

STrd - https://www.tradingview.com/support/solutions/43000634738-supertrend/

Supertrend is based upon Average True Range and, unsurprisingly further assists in detecting reversals.

The cell prints an arrow and a percentage. The vote is simply which side of the sticky band the price closed on, so this voter is never neutral once it reads at all. A band that has fallen to zero makes it abstain instead. Supertrend carries the formula.

`src/trading/indicators/supertrend.py` — the vote

```
direction = SignalDirection.BULLISH if curr_bull else SignalDirection.BEARISH
```

ZSc - Z Score - https://www.tradingview.com/script/KSMvIkvh-Z-Score-Predictive-Zones-AlgoPoint/

Our current indicator is the basic or classic Z Score indicator but this will be upgraded. But, as before, we are after more data to confirm reversals.

The cell prints a signed z-value to one decimal, not a percentage, and issue #316 made that value the smoothed one. Past the level the indicator remembers the vote is strong and contrarian: high votes bearish, low votes bullish. Until a turn is remembered that level is two deviations either way. Between 1.5 and 2 it votes the same way at a fixed 0.25 confidence, and inside 1.5 it casts no vote. Z-Score carries the formula.

`src/trading/indicators/zscore.py` — the thresholds

```
extreme_high = z > 3.0
strong_high = z > 2.0
mild_high = z > 1.5
extreme_low = z < -3.0
strong_low = z < -2.0
mild_low = z < -1.5
```

The cell now prints a smoothed z, and the vote compares it against levels the market itself set. The indicator remembers where the z-score last turned around, averages those turning points, and uses each average as the level that decides a vote. Until it has remembered a turn, the fixed level of two deviations decides, exactly as before. Each average is also projected back into a price, which is the upper and lower zone the hover text carries.

`src/trading/indicators/zscore.py` — the level that decides, and its price

```
target_z_high = self.reversal_threshold if peak_z is None else peak_z
target_z_low = -self.reversal_threshold if trough_z is None else trough_z

resistance_price = sma + target_z_high * std
support_price = sma + target_z_low * std
```

KER -

Kaufman Efficiency Ratio. It measures how much of the distance the price travelled became progress in one direction. A reading near 1.0 is a straight move and a reading near 0.0 is noise. A low ratio marks the market Acervator prefers. The Efficiency Ratio Regime gate refuses a SCRUM at 0.70 and above, and refuses one again at 0.05 and below.

RSI -

Relative Strength Index. It reports the share of recent movement that ran upward, on a 0 to 100 scale. Above 70 votes bearish, below 30 votes bullish, and between the two it casts no vote. It runs the same published Wilder recursion as the Stochastic RSI above, computed separately inside its own module. Its weight is 0.8, the lightest in the panel, which it shares with Volume.

The twelve voters

Reference. The operator's own reading of each column opens this part. What follows is the code behind those columns.

Functional. One weight map names the twelve indicators that vote, and one method builds exactly those twelve from it. The engine builds the list once and runs it in order on every tick. "Candles needed" in the table below is the tape length each indicator wants before it will read anything. Give it less and it abstains outright rather than guessing.

`src/trading/ta_engine.py` — `VotingEngine._create_indicators`

```
def _create_indicators(self) -> list:
    """Instantiate all indicator instances with configured weights."""
    return [
        BollingerBands(weight=self.weights.get("bollinger_bands", 1.0)),
        VortexIndicator(weight=self.weights.get("vortex", 0.9)),
        MACD(weight=self.weights.get("macd", 1.2)),
        StochasticRSI(weight=self.weights.get("stochastic_rsi", 1.0)),
        IchimokuCloud(weight=self.weights.get("ichimoku", 1.1)),
        VolumeAnalysis(weight=self.weights.get("volume", 0.8)),
        SlingshotIndicator(weight=self.weights.get("slingshot", 1.0)),
        ADXIndicator(weight=self.weights.get("adx", 1.0)),
        KaufmanERIndicator(weight=self.weights.get("kaufman_er", 1.0)),
        SupertrendIndicator(weight=self.weights.get("supertrend", 1.0)),
        ZScoreIndicator(weight=self.weights.get("zscore", 0.9)),
        RSIIndicator(weight=self.weights.get("rsi", 0.8)),
    ]
```

Design intention. The weights rank the voters by how much they are trusted. MACD is heaviest at 1.2 and Ichimoku next at 1.1. RSI and Volume are lightest at 0.8, RSI because it overlaps the Stochastic RSI and Volume because it is meant to confirm rather than lead. Hand the engine a different map and the panel retunes without an indicator being touched.

`src/trading/ta_engine.py` — `VotingEngine.__init__`

```
self.weights = weights or DEFAULT_WEIGHTS.copy()
self._indicators = self._create_indicators()
```

Panel column	Voter	Weight	Module under `src/trading/indicators/`	Candles needed
BB	`bollinger_bands`	1.0	`bollinger.py`	20
VTX	`vortex`	0.9	`vortex.py`	15
MACD	`macd`	1.2	`macd.py`	35
SRsi	`stochastic_rsi`	1.0	`stochastic_rsi.py`	36
Ichi	`ichimoku`	1.1	`ichimoku.py`	79
Vol	`volume`	0.8	`volume.py`	25
Sling	`slingshot`	1.0	`slingshot.py`	52
ADX	`adx`	1.0	`adx.py`	30
STrd	`supertrend`	1.0	`supertrend.py`	12
ZSc	`zscore`	0.9	`zscore.py`	51
KER	`kaufman_er`	1.0	`kaufman_er.py`	12
RSI	`rsi`	0.8	`rsi.py`	15

Functional. Count the files in the indicator folder and you get 23, not 12. Eleven of them cast no vote. Two carry the shared maths and the signal vocabulary. Three are measurements other code reads. Six are structural detectors that feed the bot's own pattern reads rather than the panel. Twelve is the count of voters, and twelve is the right count.

```
shared        helpers.py, types.py
measurements  atr.py, bb_proximity.py, heikin_ashi.py
detectors     fvg.py, landing_strip.py, m_top.py, w_bottom.py,
              spring.py, macd_taper.py
```

Design intention. One module holds one indicator, and a module that casts no vote still lives here so the voters can read it.

The per-module rule sentences, overtaken

True today: no check in the tree holds the rule. The repository tracks one Python module under `tests/`, its own conftest, so the per-module check and its two assertions are gone.

The package layout carries the rule instead. Each indicator is one module, and `src/trading/ta_engine.py` imports each indicator class or detector function by name from its own module.

Two imports cross between the indicator modules, and both reach a module in the measurements row above rather than a voter. `bb_proximity.py` and `landing_strip.py` each read the Heikin-Ashi measurement. Beside those, every module reads the shared helpers and types. No voter imports another voter.

Indicator formulae

Each block below carries the published formula for reference, then the code that computes it, then what the code does today and what the reading is meant to give the platform. The operator's own reading of each indicator opens this part and is not repeated here.

Every indicator computes its own maths from candles alone.

True today: no voter borrows another voter's arithmetic, and no check in the tree holds the rule. The twelve voters above names the module layout that holds it instead.

Where the code departs from the published formula, the code is the defect. Departures lists the ones found.

Shared terms

Every formula below uses these. All five live in `src/trading/indicators/helpers.py`.

```
SMA(v, n)     mean of the last n values                          _sma_tail
sigma(v, n)   population deviation of the last n values          _stdev_tail
EMA(v, n)     seed = SMA(v, n) at index n-1, then                _ema
              EMA_t = (v_t - EMA_prev) * k + EMA_prev
              k = 2 / (n + 1)
              no value exists before the seed
TR_t          max(H_t - L_t, |H_t - C_prev|, |L_t - C_prev|)     _true_range
              the first bar's TR is H - L
Wilder(v, n)  seed = mean of the first n, then                   (per module)
              W_t = (W_prev * (n - 1) + v_t) / n
```

Functional. True Range gives one bar per candle. The first bar is simply high minus low, because the other two ways of measuring it need a previous close and the first bar has none. Each indicator then takes the slice its own formula covers. ATR and Supertrend take the whole series. ADX and Vortex drop the first bar, because

directional movement needs a bar before it. The moving average helper answers nothing at all below its seed, rather than zero, so reading it too early fails loudly instead of returning a plausible wrong number.

`src/trading/indicators/helpers.py` — `_true_range`

```
tr = [candles[0].high - candles[0].low]
for i in range(1, len(candles)):
    c = candles[i]
    prev_close = candles[i - 1].close
    tr.append(
        max(c.high - c.low, abs(c.high - prev_close), abs(c.low - prev_close))
    )
return tr
```

Design intention. One definition per shared term, held in one place, so five indicators cannot quietly drift apart on the same arithmetic. Slingshot is the one deliberate exception. It computes its own true range, and its first bar agrees with this one.

`src/trading/indicators/slingshot.py` — the exception, written out

```
# Carter's Keltner leg needs True Range; `tr_all` computes it
# without `ATRIndicator`.
tr_all = [candles[0].high - candles[0].low]
```

Bollinger Bands

Published formula, for reference:

```
Middle    = SMA(close, 20)
Upper     = Middle + 2.0 * sigma(close, 20)
Lower     = Middle - 2.0 * sigma(close, 20)
%B        = (close - Lower) / (Upper - Lower)
BandWidth = (Upper - Lower) / Middle
```

Functional. The vote is a reading of where the last close sits between the two bands. Zero is the lower band, one is the upper, and the panel and the gates both call that number the band position. A second method returns all three bands for every candle so a chart can draw them, and answers nothing for the first nineteen. Give it fewer than twenty closes, or a window with no range at all, and it abstains.

`src/trading/indicators/bollinger.py` — `BollingerBands.compute`

```
sma = _sma_tail(closes, self.period, tail=self.period)
std = _stdev_tail(closes, self.period, tail=self.period)

mid = sma[-1]
upper = mid + self.std_dev * std[-1]
lower = mid - self.std_dev * std[-1]
price = closes[-1]
# BandWidth = (Upper - Lower) / Middle. `candles_from_raw` admits
# only positive closes, so `mid` cannot be zero.
band_width = (upper - lower) / mid
```

Design intention. The bands are the boundary a trade fires at. That is the operator's own reading, and the band position is the one number that carries it into the engine. Four gates read it directly and both overrides arm on it, so this reading has more reach into a trade decision than any other voter.

`src/trading/gate_chain.py` — two of the four, in the scrum chain

```
MidlineGate(side="scrum"),
TargetFiresGate(),
BBProximityGate(side="scrum"),
```

`src/trading/gate_chain.py` — and the other two, in the fold chain

```
MidlineGate(side="fold"),
SmartCeilingGate(),
BBProximityGate(side="fold"),
```

Vortex Indicator

Published formula, for reference:

```
VM+_t = |H_t - L_(t-1)|
VM-_t = |L_t - H_(t-1)|
VI+   = sum(VM+, 14) / sum(TR, 14)
VI-   = sum(VM-, 14) / sum(TR, 14)
```

Functional. Three totals are taken over each fourteen-bar window: upward movement, downward movement, and true range. A window whose true range totals zero, which is what a halted market gives, is skipped rather than divided by. The two movement totals are then divided by the true-range total, so both lines are dimensionless and 1.0 is dead parity. The vote reads the crossover of the two lines.

`src/trading/indicators/vortex.py` — `VortexIndicator.window_sums`

```
for i in range(1, n_c):
    vm_plus.append(abs(candles[i].high - candles[i - 1].low))
    vm_minus.append(abs(candles[i].low - candles[i - 1].high))
```

`src/trading/indicators/vortex.py` — `VortexIndicator.lines`

```
plus_series[i] = sum_vp / sum_tr_window
minus_series[i] = sum_vm / sum_tr_window
```

Design intention. The operator uses the Vortex to catch a reversal early, and watches for the signal to saturate at one polarity or the other. The vote follows the two published lines, and nothing overwrites it once the crossover has decided. A saturated upward line caps the vote's confidence at 0.45 and leaves the direction alone. The saturated downward line is published in the reading's own detail, where nothing reads it, so Acervator adds no reading of its own on top of the published lines here, and Departures no longer carries one.

`src/trading/indicators/vortex.py` — `VortexIndicator.compute`, the one cap the vote carries

```
if vip_at_ceiling:
    confidence = max(0.0, min(confidence, 0.45))
```

MACD

Published formula, for reference:

```
MACD Line = EMA(close, 12) - EMA(close, 26)
Signal    = EMA(MACD Line, 9)
Histogram = MACD Line - Signal
```

Functional. One method builds all three series and the vote reads the last two bars of them. What you are looking at is the gap between a fast and a slow average of the price, and how quickly that gap is changing. The signal line is an average of the MACD line rather than of the candles, so its first real value does not arrive until 33 bars into the tape. A series with a hole in the middle raises rather than being handed on.

`src/trading/indicators/macd.py` — `MACD._lines`

```
ema_fast = _ema(closes, self.fast)
ema_slow = _ema(closes, self.slow)
macd_line: _Line = [
    None if (f is None or s is None) else f - s
    for f, s in zip(ema_fast, ema_slow)
]
```

`src/trading/indicators/macd.py` — the signal line and the histogram

```
signal_line: _Line = [None] * first
signal_line.extend(_ema(real, self.signal_period))
histogram: _Line = [
    None if (m is None or g is None) else m - g
    for m, g in zip(macd_line, signal_line)
]
```

Design intention. The operator watches for a top-of-the-hill shape in either polarity, timed against a Bollinger band threshold. The histogram is the series that shape reads from, and the vote reads the last two of its bars, so a turn is caught on the bar it happens rather than a bar later.

`src/trading/indicators/macd.py` — the two bars the vote reads

```
  macd_line, signal_line, histogram = self._lines(closes)

  curr_hist = histogram[-1]
  prev_hist = histogram[-2]
  curr_macd = macd_line[-1]
```

The confidence divides the histogram by a share of the last close, so the same crossover scores the same on a four-figure asset and on one worth a few millionths of a dollar.

Stochastic RSI

Published formula, for reference:

```
  StochRSI = (RSI - min(RSI, 14)) / (max(RSI, 14) - min(RSI, 14))
  %K       = SMA(StochRSI * 100, 3)
  %D       = SMA(%K, 3)
```

Functional. The module runs Wilder's recursion over the closes to build its own RSI series, then places each RSI value inside its own fourteen-value range. The vote reads the crossover of the two smoothed lines. The result saturates at 0 and at 100 far more often than plain RSI does, and that saturation is what the panel is watching for. A window whose RSI never moved is caught before the division, and the vote abstains on it rather than dividing by that spread.

`src/trading/indicators/stochastic_rsi.py` — `StochasticRSI.rsi_values`

```
  avg_gain = (avg_gain * (self.rsi_period - 1) + gains[i]) / self.rsi_period
  avg_loss = (avg_loss * (self.rsi_period - 1) + losses[i]) / self.rsi_period
```

`src/trading/indicators/stochastic_rsi.py` — `StochasticRSI.stoch_ratios`

```
  window = rsi_values[i - self.stoch_period + 1 : i + 1]
  low = min(window)
  ratios.append((rsi_values[i] - low) / (max(window) - low))
```

Design intention. This is the operator's second oscillator, and he looks for a long run at one polarity to line up with the other voters. Rescaling RSI against its own recent range is what makes those runs stand out at all. A window that never moved is reported as flat rather than hidden, so the caller can tell a real zero from a market that stood still.

`src/trading/indicators/stochastic_rsi.py` — what the caller is handed

```
  return start, ratios, flat
```

Ichimoku Cloud

Published formula, for reference:

```
Tenkan   = (highest high 9  + lowest low 9)  / 2
Kijun    = (highest high 26 + lowest low 26) / 2
Senkou A = (Tenkan + Kijun) / 2        plotted 26 bars forward
Senkou B = (highest high 52 + lowest low 52) / 2   plotted 26 forward
Chikou   = close                       plotted 26 bars back
```

Functional. Every line here is a midpoint: the highest high and the lowest low of a window, averaged. A window that has not closed yet answers nothing rather than zero. Five values come back for each candle. Two of them draw the cloud twenty-six bars ahead of the last candle, and that is the only forward-looking reading anywhere in the panel. The lines are returned at their own bar, so the twenty-six bar shift belongs to whatever draws them.

`src/trading/indicators/ichimoku.py` — `IchimokuCloud.lines`

```
tenkan = self._span_mid(candles, i, self.tenkan)
kijun = self._span_mid(candles, i, self.kijun)
senkou_b = self._span_mid(candles, i, self.senkou_b)
senkou_a = (
    (tenkan + kijun) / 2.0
    if (tenkan is not None and kijun is not None)
    else None
)
out.append((tenkan, kijun, senkou_a, senkou_b, candles[i].close))
```

Design intention. The operator calls this one of the few predictive indicators and uses it to sharpen reversal-zone detection. The forward cloud is the half of the indicator that does that work. A window that has not closed answers nothing rather than zero, so an unfinished cloud can never be mistaken for a flat one.

`src/trading/indicators/ichimoku.py` — `IchimokuCloud._span_mid`

```
if end < period - 1:
    return None
return self._mid(candles, end - period + 1, period)
```

The cloud thickness divides by the price bare. Every candle reaching this point has already been refused unless its price is above zero, so the division needs no guard of its own.

Volume

Published formulae, for reference:

```
OBV_t  = OBV_prev + V_t   if C_t > C_(t-1)
         OBV_prev - V_t   if C_t < C_(t-1)
         OBV_prev         otherwise
TP_t   = (H_t + L_t + C_t) / 3
MF_t   = TP_t * V_t
MFI    = 100 - 100 / (1 + sum(MF where TP rose, 14)
                        / sum(MF where TP fell, 14))
CLV_t  = ((C_t - L_t) - (H_t - C_t)) / (H_t - L_t)
CMF    = sum(CLV * V, 20) / sum(V, 20)
A/D_t  = A/D_prev + CLV_t * V_t
```

Functional. The Volume column is the one voter the operator does not describe in his list above, so this is its only description in the manual. Three published measures are computed separately and scored together. The vote answers one question: does the volume traded agree with the direction the price moved. A price low that On-Balance Volume refuses to match is the divergence this module scores highest. Where neither money-flow bucket was ever credited, the Money Flow Index answers 50 rather than 100, so a market that never moved does not read as maximum overbought.

`src/trading/indicators/volume.py` — `VolumeAnalysis._obv`

```
obv = [0.0]
for i in range(1, len(candles)):
    if candles[i].close > candles[i - 1].close:
        obv.append(obv[-1] + candles[i].volume)
    elif candles[i].close < candles[i - 1].close:
        obv.append(obv[-1] - candles[i].volume)
    else:
        obv.append(obv[-1])
```

`src/trading/indicators/volume.py` — `VolumeAnalysis._mfi`

```
if pos_mf <= 0.0 and neg_mf <= 0.0:
    return 50.0
if neg_mf <= 0.0:
    return 100.0
return 100.0 - 100.0 / (1.0 + pos_mf / neg_mf)
```

Design intention. Volume confirms or contradicts the other eleven. It does not lead them. Its weight is 0.8, the lightest in the panel alongside RSI, and that number is the intention written down.

`src/trading/ta_engine.py` — the weight that says so

```
"volume": 0.8,
```

Both tests in the block above compare a money flow against zero. Money flow is a price times a volume and carries no fixed scale, so only an exact zero can stand as the test.

Slingshot

Published formula, for reference:

```
UpperBB = SMA(close, 20) + 2.0 * sigma(close, 20)
LowerBB = SMA(close, 20) - 2.0 * sigma(close, 20)
UpperKC = SMA(close, 20) + 1.5 * SMA(TR, 20)
LowerKC = SMA(close, 20) - 1.5 * SMA(TR, 20)
sqzOn   = (LowerBB > LowerKC) and (UpperBB < UpperKC)
delta_t = C_t - (donchian_mid_20 + SMA(close, 20)) / 2
val     = linreg(delta, 20, 0)
```

Functional. The squeeze is on while the Bollinger band sits entirely inside the Keltner Channel. When it opens again, a fitted momentum line gives the sign of the release, and that sign is the vote. The module computes its own true range rather than calling the shared helper, and its first bar agrees with the shared one anyway.

`src/trading/indicators/slingshot.py` — `SlingshotIndicator.compute`

```
rangema = trma_v[i] if trma_v[i] is not None else 0.0
up_kc = mid + rangema * self.kc_mult
lo_kc = mid - rangema * self.kc_mult
sqz_on = (lo > lo_kc) and (up < up_kc)
seg = [
    deltas[k] for k in range(max(delta_lo, i - self.bb_period + 1), i + 1)
]
val = self._linreg_endpoint(seg)
```

Design intention. The operator wants a precise reading of a reversal zone, and he names Chris Moody's Slingshot system as the source. The code does not implement that system. What is in the module is John Carter's TTM Squeeze, with Bollinger's own rules 6 and 8 for the snapback. Moody's method is an EMA trend and pullback with no band, no channel and no squeeze at all. The module says so itself, in its own docstring. The column keeps the name he gave it, and the name and the code are two different indicators.

`src/trading/indicators/slingshot.py` — the module stating it

```
# LazyBear's KC shares the BB basis; rangema is the SMA of
# True Range over the same period.
```

In development. Which of the two indicators the column should hold is the operator's decision, not a coding one, so nothing is proposed here.

ADX and DMI

Published formula, for reference:

```
up_t   = H_t - H_(t-1)
down_t = L_(t-1) - L_t
+DM_t  = up_t   if up_t > down_t and up_t > 0     else 0
-DM_t  = down_t if down_t > up_t and down_t > 0   else 0
+DI    = 100 * Wilder(+DM, 14) / Wilder(TR, 14)
-DI    = 100 * Wilder(-DM, 14) / Wilder(TR, 14)
DX     = 100 * |+DI - -DI| / (+DI + -DI)
ADX    = Wilder(DX, 14)
```

Functional. One smoothing recursion carries every line in this indicator. The reading tells you how committed a move is, and says nothing about which way it points. Below 20 the market is ranging. Above 35 it is trending hard. The indicator abstains when its smoothed true range is not above zero, so a market that moved at all still gets a vote however little the asset costs.

`src/trading/indicators/adx.py` — `ADXIndicator._wilder_smooth`

```
    if len(values) < period:
        return [0.0] * len(values)
    result = [0.0] * (period - 1)
    result.append(sum(values[:period]) / period)
    for v in values[period:]:
        result.append((result[-1] * (period - 1) + v) / period)
    return result
```

Design intention. The operator reads ADX for trend strength, and he watches it fall rather than rise, because a falling reading marks a trend running out of road. The engine carries the other half of that: a strong trend holds a scrum back rather than letting it fire into a move that is still running.

`src/trading/gate_chain.py` — `ADXTrendSuppressionGate.evaluate`

```
    if ctx.adx < self.adx_threshold:
        return GateResult(passed=True)
```

Issue #414 owned the abstain threshold, the correction landed, and the issue is closed.

Supertrend

Published formula, for reference:

```
ATR     = Wilder(TR, 10)
hl2_t   = (H_t + L_t) / 2
rawUp   = hl2 + 3.0 * ATR
rawLow  = hl2 - 3.0 * ATR
Upper_t = rawUp  if rawUp < Upper_prev  or C_(t-1) > Upper_prev  else Upper_prev
Lower_t = rawLow if rawLow > Lower_prev or C_(t-1) < Lower_prev  else Lower_prev
bullish while C_t >= Lower_t; turns bullish again when C_t > Upper_t
```

Functional. A band here is sticky. It moves toward the price and refuses to move away from it, unless the price closed past it last bar. What comes out is a line that trails under the price in an uptrend and over it in a downtrend, and the bar where price crosses that line is the flip.

`src/trading/indicators/supertrend.py` — `SupertrendIndicator.compute`

```
    final_ub = (
        raw_ub
        if raw_ub < prev_ub or candles[i - 1].close > prev_ub
        else prev_ub
    )
    final_lb = (
        raw_lb
        if raw_lb > prev_lb or candles[i - 1].close < prev_lb
        else prev_lb
    )
```

Design intention. Supertrend rests on Average True Range and helps the operator spot a reversal. The flip is the event that does that work, and the line either side of it is context. The bullish test and the bearish test differ on purpose, so a trend has to be broken outright before the line turns.

`src/trading/indicators/supertrend.py` — the asymmetric flip test

```
prev_bull = st[-1]
if prev_bull:
    curr_bull = candles[i].close >= final_lb
else:
    curr_bull = candles[i].close > final_ub
```

The band this indicator measures distance from is derived rather than read off the tape, and it can genuinely reach zero once the average true range passes the midpoint over the multiplier. The indicator abstains outright on that bar instead of dividing by it.

`src/trading/indicators/supertrend.py` — the abstention

```
if st_line <= 0.0:
    return Signal(
        "supertrend",
        timeframe,
        SignalDirection.NEUTRAL,
        0.0,
        self.weight,
        abstained=True,
    )
```

Z-Score

Published formula, for reference:

```
mean  = SMA(close, 50)
sigma = population deviation of the same 50 closes
Z     = (close - mean) / sigma
```

Functional. The reading is how many standard deviations the price sits from its own fifty-bar mean, and the vote reads the extremes of that. Fifty bars is longer than the twenty the Bollinger Bands use, so a stretch those bands have already absorbed still shows up here. A window with almost no spread at all abstains rather than dividing by it.

`src/trading/indicators/zscore.py` — `ZScoreIndicator.compute`

```
closes = [c.close for c in candles[-self.period :]]
sma = sum(closes) / self.period
variance = sum((c - sma) ** 2 for c in closes) / self.period
std = variance**0.5
```

Design intention. The operator calls this the classic Z-Score, asked for the upgrade issue #316 landed, and uses it to confirm a reversal. The longer window is why it earns a column of its own beside the Bollinger Bands.

One formula, written once, for both bars. The current bar and the previous bar each divide by their own deviation bare, and a guard above each refuses a deviation under the same threshold before the division runs.

`src/trading/indicators/zscore.py` — both divisions

```
z = (candles[-1].close - sma) / std
```

```
z_prev = (candles[-2].close - s2) / std2
```

Published method. The source the operator named publishes two formulae. The first is the classic score above. The second turns a score back into a price, so a level the market has reversed at can be drawn as a zone that widens and narrows with the deviation.

```
Projected Price = Mean + (Target Z * Standard Deviation)
```

The same page names the noise reduction: the raw score is smoothed by a volume weighted average before anything reads it. Turning points are then found in that smoothed series, kept only when they reach a minimum score, and the last few of each side are averaged into the two levels the vote uses.

Both bars now read one window through one expression, so the two spellings that disagreed cannot come back. A window whose prices never moved returns nothing at all, which is how the reading abstains rather than shifting.

`src/trading/indicators/zscore.py` — one window, one expression, every reader

```
def _window_zscore(closes: list, close: float) -> Optional[tuple]:
    if _window_has_no_range(closes):
        return None
    count = len(closes)
    mean = sum(closes) / count
    deviation = (sum((c - mean) ** 2 for c in closes) / count) ** 0.5
    return mean, deviation, (close - mean) / deviation
```

What the page does not publish. It names four settings and gives a default value for none of them. Three take a number this file already used or the page itself quotes, and one is chosen here and marked as such.

Setting	Value	Where the value comes from
Z-Score Length	50	the period this file already used
Reversal Threshold	2.0	the strong level this file already used, and the level the page quotes as the classic one
Pivot bars	1	a turn against the bar each side; the page gives noise reduction to the smoothing instead
Lookback Depth	5	chosen here, because the page publishes no value and this file had none

Kaufman Efficiency Ratio

Published formula, for reference:

```
ER = |C_t - C_(t-10)| / sum(|C_i - C_(i-1)|, 10)
```

Functional. Net displacement divided by the distance travelled to get there. 1.0 is a straight line, 0.0 is pure noise, and both halves read the same eleven closes. A window where the price never moved travels exactly

zero distance, and the indicator abstains on that rather than reporting a regime it never measured.

`src/trading/indicators/kaufman_er.py` — `KaufmanERIndicator.compute`

```
closes = [c.close for c in candles[-(self.period + 1) :]]
net_change = abs(closes[-1] - closes[0])
price_travel = sum(
    abs(closes[i] - closes[i - 1]) for i in range(1, len(closes))
)
```

Design intention. A low ratio marks the market Acervator prefers, in the operator's own words above. The engine acts on that at both ends: a scrum is refused when the market is running too straight to mean-revert, and refused again when it is too noisy to hold an edge.

The division that produces the ratio is bare. A guard four lines above returns early when the distance travelled is not above zero, so the denominator is already proved positive by the time the division runs.

`src/trading/indicators/kaufman_er.py` — the division

```
er = net_change / price_travel
```

RSI

Published formula, for reference:

```
gain_t  = max(C_t - C_(t-1), 0)
loss_t  = max(C_(t-1) - C_t, 0)
AvgGain = Wilder(gain, 14)
AvgLoss = Wilder(loss, 14)
RS      = AvgGain / AvgLoss
RSI     = 100 - 100 / (1 + RS)

AvgLoss = 0 and AvgGain > 0   ->  RSI 100
AvgLoss = 0 and AvgGain = 0   ->  no reading, the indicator abstains
```

Functional. The reading is the share of recent movement that ran upward, on a 0 to 100 scale. Above 70 votes bearish, below 30 votes bullish, and anything between casts no vote. A bar with neither a gain nor a loss abstains outright.

`src/trading/indicators/rsi.py` — `RSIIndicator.compute`

```
if rsi > 70:
    direction = SignalDirection.BEARISH
elif rsi < 30:
    direction = SignalDirection.BULLISH
else:
    direction = SignalDirection.NEUTRAL
```

Design intention. RSI is the plainest voter in the panel, and it overlaps the Stochastic RSI on purpose. Its weight is 0.8, which stops the same evidence being counted twice at full strength.

One number is read one way here. The vote tests it, the overbought and oversold flags in the same dictionary test it, and nothing cuts it before either of them, so an RSI of 70.003 sets overbought and votes bearish on the same bar.

`src/trading/indicators/rsi.py` — the one index the dictionary carries

```
return {
    "rsi": rsi,
    "rs_indeterminate": rs_indeterminate,
    "overbought": rsi > 70,
    "oversold": rsi < 30,
```

Rounding belongs to display, not to a comparison, and it is done where a figure is drawn. Issue #414 carried this and is closed.

Departures from the published maths

Functional. No departure from the published maths remains. Every voter now reads the same on an asset worth four figures and on one worth a few millionths of a dollar, because no division and no threshold in the twelve carries an absolute constant against a quantity measured in the asset's own price units. Where a denominator can genuinely reach zero the indicator abstains on that bar rather than adding a constant to it.

- The Vortex direction follows the upward line against the downward one, and nothing overwrites it after the crossover has decided. The two lines themselves match Botes and Siepman exactly, and no reading of Acervator's own is laid on top of them.
- The RSI vote and the overbought and oversold flags beside it read one number. The index is not cut before any of them tests it, so the label and the vote cannot disagree about the same bar.

Supertrend is the one place an abstention stands in for a division. Its lower band is derived from the average true range rather than read off the tape, so it can reach zero on a real market, and the indicator refuses to vote on that bar.

Design intention. An indicator has to read the same on every asset, because one threshold serves the whole fleet.

The four scale-invariance files, overtaken

OVERTAKEN, quoted whole:

> *"Four test files hold that property for the twelve, one shape at a time, by running the same tape at several price scales and comparing the readings."*

```
test_bollinger_and_slingshot_are_scale_invariant
test_vortex_and_landing_strip_are_scale_invariant
test_macd_and_kaufman_er_are_scale_invariant
test_the_last_eight_indicators_are_scale_invariant
```

True today: no check in the tree holds the property. None of the four files is there, and the repository tracks one Python module under `tests/`, its own conftest. The runs recorded in the entries further down this page hold the property instead, each naming the scales it drove and the readings it compared. The bounds the readings must stay inside are in `src/trading/ta_invariants.py`.

Issue #414 owned the two readings above and is closed. Each one now reports the published maths with nothing added, so neither needs a replacement chosen for it. The entries dated below record what each repair changed and what it was measured on.

`src/trading/indicators/vortex.py` — `VortexIndicator.compute`, the crossover the direction follows

```
bullish_cross = prev_sep <= 0 and separation > 0
bearish_cross = prev_sep >= 0 and separation < 0
```

2026-09-08 15:12 - #414 - the ratio the gate reads

The published Efficiency Ratio is Kaufman's, unchanged here: the distance between the first and last close of the window, divided by the distance the price travelled to cover it. The Kaufman entry above states it and the code computes it.

```
ER = |C_t - C_(t-10)| / sum(|C_i - C_(i-1)|, 10)
```

The indicator gave three other parts of the platform that ratio rounded to four decimal places. One of them is the Efficiency Ratio Regime gate, which refuses a scrum at 0.05 and below and at 0.70 and above, so the gate compared a shortened number against its two thresholds. The indicator now publishes the ratio it computed.

`src/trading/indicators/kaufman_er.py` — what the reading carries

```
details={
    "er": er,
    "er_prev": er_prev,
```

Three parts of the platform read that number and each builds the same gate input from it: the live bot's tick, the Simulator's back test, and the ATA gate scan. Nothing on any screen changes, because the panel cell, the hover text and the ATA-SMP sentence each cut the number to two, three and four decimals of their own.

`src/trading/scrumming_bot.py` — the gate input, one of the three

```
efficiency_ratio=_extract_signal_detail(summary, "kaufman_er", "er", 0.0),
```

Measured over 71 recorded Coinbase tapes, run at a price of three millionths of a dollar and again at a four-figure price. The run drove 37,380 readings through the real voting engine and the real scrum chain. The gate changed its mind on five of them, in both directions: three where the rounding had pulled a ratio just above 0.05 down onto the threshold, and two where it had pulled a ratio near zero onto the sentinel that tells the gate no reading arrived.

```
A22 2025, bar 264, both price scales
    before   gate reads 0.05                  scrum blocked
    after    gate reads 0.05001050640890935   scrum allowed

A22 2025, bar 263, both price scales
    before   gate reads 0.2257                scrum allowed
    after    gate reads 0.2256565656565654    scrum allowed

A14 2025, bar 233, both price scales
    before   gate reads 0.0                   scrum allowed
    after    gate reads 2.983774235682565e-05 scrum blocked
```

The same run read the other eleven voters back over one tape at 88 bars of each price scale, and every reading is identical, which is what a repair to one indicator has to look like.

2026-09-08 15:54 - #414 - when ADX first has something to say

Wilder's ADX needs two runs of bars before it exists. The first is fourteen bars of true range and directional movement, which seed the smoothing. The second is fourteen DX values, which the ADX is the average of. Twenty-eight bars, and not one more. The ADX and DMI entry above states the formula and the code computes it.

```
+DI    = 100 * Wilder(+DM, 14) / Wilder(TR, 14)
-DI    = 100 * Wilder(-DM, 14) / Wilder(TR, 14)
DX     = 100 * |+DI - -DI| / (+DI + -DI)
ADX    = Wilder(DX, 14)          first value at bar 28
```

The indicator refused to vote until thirty bars had arrived. The extra two came from the first upload and never had a reason. On those two bars the market had moved, the ADX existed, and the panel counted eleven voters instead of twelve.

`src/trading/indicators/adx.py` — when the indicator speaks

```
n = len(candles)
# period bars seed the smoothed TR, and period DX values seed the ADX.
if n < self.period * 2:
```

The bar the ADX first exists on has no earlier ADX to compare against. The module read a zero from the padding the smoothing writes ahead of its first real value, so the rising flag on that bar was decided against a number that was never a reading. The readings now start where the padding ends.

`src/trading/indicators/adx.py` — the previous reading

```
adx_values = s_dx[self.period - 1 :]
adx = adx_values[-1] if adx_values else 0.0
p_adx = adx_values[-2] if len(adx_values) >= 2 else adx
```

The indicator also gave every reader its ADX cut to two decimal places, and the directional lines with it. The ADX Trend Suppression gate is one of those readers and compares the number against 30.0. The indicator

now publishes what it computed.

`src/trading/indicators/adx.py` — what the reading carries

```
  details={
      "adx": adx,
      "di_plus": di_plus,
      "di_minus": di_minus,
```

Eight parts of the platform read those three numbers. Three build the same gate input: the live bot's tick, the Simulator's back test and the ATA gate scan. Five display it and each cuts it themselves, so the panel cell still prints a whole number and the ATA-SMP sentence still prints two decimals.

reader	what it shows
`scrumming_bot.py` gate input	the full reading, `23.968874676972366`
`back_test.py` gate input	the full reading
`ata_gate_scan.py` gate input	the full reading
`ata_spm.py` sentence	`trend strength ADX 23.97`, unchanged
`indicator_panel.py` cell	`24`, unchanged
`indicator_panel_surface.py` cell	`24`, unchanged
`indicator_panel_surface.py` hover	`23.969`, was `23.970`
`snapshots.py` bot log line	`(adx 23.9689)`, was `(adx 23.97)`

The bot log printed whatever the indicator published, so it now cuts a decimal reading to four places of its own.

`src/trading/scrumming/snapshots.py` — the bot log cell

```
  reading = cell.get("detail")
  if reading is not None:
      shown = f"{reading:.4f}" if isinstance(reading, float) else reading
```

Measured over 71 recorded Coinbase tapes, run at a price of three millionths of a dollar and again at a four-figure price. The run drove 42,140 readings through the real voting engine and the real scrum chain, from a window of twenty-six bars upward.

```
  abstentions   before 280 per price scale    after 140 per price scale
  readings that moved
      the two recovered bars           280
      the published decimal places  41,580
  scrum verdict changed
      the two recovered bars           136
      the published decimal places       0
```

Every abstention that remains is a window under twenty-eight bars, where Wilder's ADX does not exist. Every one that ended is a window of twenty-eight or twenty-nine bars, where it did.

```
A34 2021, window 27 bars, both price scales
    before   abstained     13 DX values, one short of the fourteen
    after    abstained     the same reason

A34 2021, window 28 bars, both price scales
    before   abstained     the guard asked for 30 bars
    after    ADX 21.847870255087297  bullish at 0.3121

A34 2023, window 28 bars, both price scales
    before   gate reads 0.0                 scrum allowed
    after    gate reads 74.20924187088121   scrum blocked
```

The last row is what the repair changes about a live reading. A bot that has just started, or one whose exchange returned a short window, spent two bars firing scrums into a trend the ADX could already measure. The panel felt it too: on the A34 2021 bar the twelfth voter arriving moved the net score from 0.7437 to 1.0558 and the consensus confidence from 0.1352 to 0.1624, and across the sweep the consensus direction itself changed on 58 readings.

The reading carries no price scale. Every number above is the same at three millionths of a dollar and at a four-figure price, to the last few bits.

The same run read the other eleven voters back over one tape at 88 bars of each price scale, and every reading is identical.

2026-09-08 16:22 - #414 - the MACD reading at a millionth of a dollar

Appel's three numbers are unchanged and the MACD entry above states them. This repair touches what the indicator hands its readers, not how the lines are built.

```
MACD Line = EMA(close, 12) - EMA(close, 26)
Signal    = EMA(MACD Line, 9)
Histogram = MACD Line - Signal
```

All three are prices. The indicator published them cut to six decimal places, which is a fixed step of one millionth of a dollar. A market worth three millionths of a dollar has a whole MACD smaller than that step, so every reader was handed a zero. The same market shape at a four-figure price published real numbers. One threshold serves the whole fleet, so one shape has to read the same on both.

`src/trading/indicators/macd.py` — what the reading carries

```
details={
    "macd_line": curr_macd,
    "signal_line": curr_signal,
    "histogram": curr_hist,
```

Measured over 71 recorded Coinbase tapes, run at a price of three millionths of a dollar and again at a four-figure price. The run drove 3,750 readings through the real voting engine and both real gate chains, from a window of thirty-six bars upward.

```
readings publishing three zeros
    three millionths   before 1,834 of 1,875   after 0 of 1,875
    four figures       before     0 of 1,875   after 0 of 1,875
the MACD line over the histogram, the two scales agreeing
    before        0 of 1,875 pairs
    after     1,875 of 1,875 pairs
readings that moved             3,750 of 3,750
scrum and fold verdicts changed     0 of 3,750
```

One bar, at both prices, before and after:

```
A34 2021, bar 234, a bullish crossover
    before   three millionths   histogram  0.0
             four figures       histogram  1.651648
    after    three millionths   histogram  4.149197409777354e-09
             four figures       histogram  1.6516482592467057
the MACD line over the histogram
    before   three millionths   no value, the division is by zero
             four figures       -20.5541919344
    after    three millionths   -20.5541889248
             four figures       -20.5541889248
```

The confidence never carried a price scale and does not change here. It divides the histogram by a share of the last close, a price over a price, and the two shares now have names of their own.

`src/trading/indicators/macd.py` — the two shares the confidence divides by

```
crossover_scale = abs(closes[-1]) * 0.001
momentum_scale = abs(closes[-1]) * 0.002
```

A close of zero leaves nothing to divide by. The indicator used to raise on such a bar and take the whole voting run down with it, so the other eleven voters never answered either. It now says it has no reading, which is what every other voter does when its own denominator is empty.

`src/trading/indicators/macd.py` — the abstention that replaces the crash

```
if crossover_scale == 0.0 or momentum_scale == 0.0:
    # No scale to divide by, so macd publishes no confidence here.
    return Signal(
        "macd",
        timeframe,
        SignalDirection.NEUTRAL,
        0.0,
        self.weight,
        abstained=True,
    )
```

The bound that checks the reading against itself allowed a slack of two millionths. At three millionths of a dollar that slack is wider than any real MACD, so the bound passed a histogram with the wrong sign and a histogram of zero. It is exact now, and it holds on all 3,750 readings.

`src/trading/ta_invariants.py` — the bound on the reading

```
  lambda d: float(d["histogram"])
  == float(d["macd_line"]) - float(d["signal_line"]),
```

Nine parts of the platform read this indicator. Two read the vote and its strength rather than the numbers, and one builds its own series for the chart.

reader	at three millionths of a dollar
`ta_invariants.py` bound	refuses a wrong histogram, where it passed one
`ta_engine.py` recorder	records the reading, where it recorded three zeros
`ata_spm.py` sentence	`histogram +4.1491974e-09`, was `histogram +0.000000`
`scrumming_v3_sim.py` flags	flip on 1,006 of 1,875 readings, 0 of 1,875 at four figures
`indicator_panel.py` hover	`macd_line: -0.000`, unchanged
`indicator_panel_surface.py` hover	`macd_line: -0.000`, unchanged
`scrumming_bot.py` position boost	reads the vote, unchanged
`ata_gate_scan.py` favour	reads the vote, unchanged
`native_chart.py` chart series	builds its own series, unchanged

The ATA-SMP sentence printed six decimals of a price, so on a micro-priced market it printed nothing but zeros. It now prints eight significant figures, which reads the same at every price.

`src/trading/ata_spm.py` — the sentence one voter writes

```
  "macd": ("MACD", ("histogram",), "histogram {value:+.8g}"),
```

The two hover cells still show three zeros, because the panel cuts every voter's numbers to three decimal places of its own. That format is shared by all twelve and is not this repair.

No gate changes its mind. Neither chain reads these numbers; both read the panel's consensus, and the consensus, the vote, its strength, the abstention flag and the other eleven voters are identical on all 3,750 readings.

2026-09-08 16:52 - #414 - the money flow a halted market never traded

Quong and Soudack's index is published in two forms, and this repair needs both of them.

```
  TP_t = (H_t + L_t + C_t) / 3
  MF_t = TP_t * V_t

  positive money flow = sum of MF over the 14 bars where TP rose
  negative money flow = sum of MF over the 14 bars where TP fell

  MFI = 100 - 100 / (1 + positive / negative)
  MFI = 100 * positive / (positive + negative)
```

The two lines give the same number. The second is defined on one input more than the first. Where every falling bar in the window traded nothing, the negative flow is zero: the first line divides by that zero, and the second answers 100. One hundred is the right answer there, because every dollar that moved was buying.

One input is left over, and neither published line answers it. A window that credited neither bucket gives nothing over nothing. The index does not exist on that bar.

Functional. The module answered 50 on that bar. Fifty is the middle of the scale and it is a number, published to the voting panel, to the ATA-SMP post and to the emitter recorder, about a market the indicator had measured nothing on. Driven through the voting engine on a constructed seventy-minute halt at three millionths of a dollar, the Volume column voted NEUTRAL and still counted in the panel's denominator. On a window whose last fourteen bars traded nothing at all, it voted BULLISH at 0.37 strength.

The index now reports that it has no reading, and the column abstains on that bar. Abstaining is what this module already does when the window's average volume is zero, and what Supertrend, ADX, the Efficiency Ratio and the MACD each do at their own missing denominator.

`src/trading/indicators/volume.py` — `VolumeAnalysis._mfi`

```
    total_mf = pos_mf + neg_mf
    if total_mf <= 0.0:
        return math.nan
    return 100.0 * (pos_mf / total_mf)
```

`src/trading/indicators/volume.py` — `VolumeAnalysis.compute`

```
    mfi = self._mfi(candles, self.mfi_period)
    if math.isnan(mfi):
        # `_mfi` has no money ratio; `mfi_ob` and `mfi_os` read False on nan.
        return Signal(
            "volume",
            timeframe,
            SignalDirection.NEUTRAL,
            0.0,
            self.weight,
            abstained=True,
        )
```

The item cites a money flow below 1e-09 read as zero, on two tests that answered 100 on it. Commit `b27c6cdb` removed both tests on 2026-09-04. Driven through the voting engine on a window at three millionths of a dollar whose falling bars carry a volume of 1e-05, the tree that still holds those tests answers 100.0 and this tree answers 99.999001008.

A money flow can also reach zero by running out of double precision, and that is the one route by which an absent reading could still print as the top of the scale. Measured: at a price of three millionths of a dollar the product of price and volume is exactly zero for any volume at or below 7.9688e-319. The smallest volume in the 411 recorded tablets is 100. No order size the venue can express comes within three hundred orders of magnitude of that threshold, and where the falling side does underflow, the ratio puts the index at 100 to every digit a double carries, which is the correct answer rather than an absent one.

The window with no money flow is not in the recorded data either. All 411 tablets hold daily bars; across 99,582 fourteen-bar windows not one credited neither bucket, the typical price repeated on 127 of 104,925 bars, and the longest run of repeats is five. The fleet computes on five-minute bars, which the tablets do not contain, so the recorded data says nothing about whether a seventy-minute flat window occurs live.

The reading was also published cut to one decimal place. The vote itself always tested the computed number, so no gate ever read the short one, but the panel's hover cell prints three decimals and was handed a number carrying one. It now carries what the module computed.

`src/trading/indicators/volume.py` — the reading

```
  "mfi": mfi,
```

Six parts of the platform read this indicator.

reader	at three millionths of a dollar
`ta_invariants.py` bound	bounds the computed number; no bound applies on an abstention
`ta_engine.py` recorder	records the computed number, and an empty reading on an abstention
`ata_spm.py` sentence	`Money Flow Index 56.5`, unchanged on all 3,750 readings
`indicator_panel.py` hover	`mfi: 56.450`, was `mfi: 56.500`
`indicator_panel_surface.py` hover	`mfi: 56.450`, was `mfi: 56.500`
the panel's consensus	loses one voter from its denominator on an abstention

Neither gate chain is in that list. Both read the panel's consensus rather than any voter's numbers.

No gate changes its mind. Over 71 recorded tapes at two price scales, 3,750 readings and 15,000 gate verdicts, the vote, its strength, the abstention flag, the panel's consensus and the other eleven voters are identical, and both chains fire the same way on every one. Swapping the published form of the ratio moved the index on 1,907 of the 3,750 by at most 2.2e-14, which is the last bits of double arithmetic and vanishes at one decimal place. Publishing it uncut moved the hover cell on 3,710 of 3,750 and the ATA-SMP sentence on none of them.

2026-09-08 17:45 - #414 - the Vortex reports the Vortex

Botes and Siepman published two lines and one reading of them. The lines measure upward and downward movement over the same fourteen bars, each divided by the true range of those bars. The reading is the crossover: the upward line above the downward one is a rising market, and the other way round is a falling one. A downward line far above parity is the strongest falling reading the indicator has.

```
VM+_t = |H_t - L_(t-1)|
VM-_t = |L_t - H_(t-1)|
VI+   = sum(VM+, 14) / sum(TR, 14)
VI-   = sum(VM-, 14) / sum(TR, 14)
```

Functional. The two lines above are exactly as published, and this repair does not touch them. What came out is the four lines that ran after the vote had already been decided. They read the downward line reaching 1.30, called it a coming reversal, lifted the strength to at least 0.60 and rewrote the vote to bullish.

`src/trading/indicators/vortex.py` — `VortexIndicator.compute`, removed

```
if vim_at_ceiling:
    confidence = min(1.0, max(confidence, 0.60))
    direction = SignalDirection.BULLISH
```

Seventy-one recorded Coinbase tapes were run through the voting engine at three millionths of a dollar and again at 1234.0, 3,750 readings in all. The downward line reached 1.30 on 78 of them, spread over 30 tapes. On every one of those 78 the published lines said falling and the overwrite published rising. The strength was already at or above 0.60 on all 78, so the 0.60 lift changed nothing and only the vote moved.

```
3,750 readings, 71 tapes, two price scales
    VI- at or above 1.30                          78
    vote before the repair, all 78            BULLISH
    vote after the repair, all 78             BEARISH
    strength changed                                0
    votes moved the other way                       0
    VI- range on those rows          1.3014 to 1.4859
```

Design intention. An indicator carries one published set of sums and one published reading of them, and nothing else rides on its name. The ceiling that remains touches strength only and never the vote: an upward line at or above 1.30 caps the strength at 0.45. Both ceiling readings and both floor readings are still published beside the vote, so the panel can show that a line is saturated without the vote claiming the opposite of what the lines say.

`src/trading/indicators/vortex.py` — the ceiling that stayed

```
VX_CEILING_PCT = 130.0

if vip_at_ceiling:
    confidence = max(0.0, min(confidence, 0.45))
```

Seven parts of the platform read this indicator.

reader	on a VI- at or above 1.30
the panel's consensus	one vote moves from rising to falling; moved on 78
`indicator_panel.py` cell	the arrow turns from up to down
`indicator_panel_surface.py` cell	the arrow turns from up to down
`ata_spm.py` sentence	prints the gap between the lines; unchanged on all 78
`ta_invariants.py` bound	bounds both lines at or above zero; unchanged
`ta_engine.py` recorder	records the vote it now casts
`scrumming_bot.py` position boost	pays only at the upper band or the middle, and never reached these rows

`native_chart.py` is not in that list. It draws the two lines from `VortexIndicator.lines`, which this repair does not touch.

The position boost is the one reader the overwrite could have reached and never did. It pays a bullish Vortex 0.12 at the upper band and takes 0.05 back in the middle. A downward line saturating means the price has

fallen hard, and on all 78 rows the band position sat between -0.134 and 0.224, below both zones. The boost paid nothing before the repair and nothing after it.

The gate changes its mind on four windows. Over 71 tapes at two price scales, 3,750 readings and 30,000 gate verdicts, eight verdicts move and every one of them moves the same way: a fold that was refused now fires. No scrum verdict moves. Both windows sit at or below the lower band, where the price is at the bottom and the buy-back belongs.

```
30,000 gate verdicts, both chains, four flag states
    fold refused -> fold fires                      8
    fold fires -> fold refused                      0
    scrum verdicts changed                          0
    windows                                         4
        A22 2022, band position 0.1168, VI- 1.3276
        A23  2026, band position -0.1158, VI- 1.3014
    both windows identical at both price scales
```

Both moves need the fold's own bearish requirement switched on. With that switch off the fold does not consult the panel at all and nothing moves.

The reading was also published cut to four decimal places. No vote and no gate ever read the cut copy, because every branch inside the indicator tests the number it computed. The panel's hover cell prints three decimals and was handed a number carrying four, and the cell printed a different third decimal on 726 of 15,000 readings. It now carries what the indicator computed.

`src/trading/indicators/vortex.py` — the reading

```
"vi_plus": vi_plus,
"vi_minus": vi_minus,
"separation": separation,
"sep_acceleration": sep_acceleration,
```

One example, from A34 2021: the downward line computed 1.0835408442150014 and published 1.0835, so the cell printed `1.083` where the number is `1.084`. The ATA-SMP sentence prints four decimals and matched the cut copy exactly, so it reads the same on all 3,750.

2026-09-08 19:05 - #414 - the RSI label and the RSI vote read one number

Wilder publishes one quantity. It is the share of the last fourteen bars of movement that ran upward, on a nought to a hundred scale. The same number decides the label a reader sees and the vote the panel casts: above seventy is overbought and below thirty is oversold.

```
gain_t  = max(C_t - C_(t-1), 0)
loss_t  = max(C_(t-1) - C_t, 0)
AvgGain = Wilder(gain, 14)
AvgLoss = Wilder(loss, 14)
RS      = AvgGain / AvgLoss
RSI     = 100 - 100 / (1 + RS)
```

Functional. The indicator held two copies of that quantity. It cut one to two decimal places and published it, and it kept the whole one for the overbought and oversold labels. The vote read the cut copy. A reading of 70.003 cuts to seventy exactly, and seventy is not above seventy, so the label said overbought while the vote said neutral, and the two left the indicator together.

`src/trading/indicators/rsi.py` — the cut, removed

```
return {
    "rsi": round(rsi, 2),
    "overbought": rsi > 70,
    "oversold": rsi < 30,
```

One number now leaves the indicator, and the label, the vote, the strength and the published reading all read it. The code block in the RSI section above is the code in this tree. The rounding has gone back to the two places that display a number: the confirming sentence prints two decimals and the panel's hover cell prints three.

`src/trading/indicators/rsi.py` — the reading

```
return {
    "rsi": rsi,
    "rs_indeterminate": rs_indeterminate,
    "overbought": rsi > 70,
    "oversold": rsi < 30,
```

The gap the two copies can disagree in is five thousandths of a point wide, just above seventy and again just below thirty. Seventy-one recorded Coinbase tapes were run through the voting engine at three millionths of a dollar and again at 1234.0, 3,750 readings in all, and not one landed inside it. The closest reading to seventy was 70.06, which was already voting bearish.

```
3,750 readings, 71 tapes, two price scales
    readings inside 70.000 to 70.005                0
    readings inside 29.995 to 30.000                0
    closest reading to seventy                  70.06
    closest reading to thirty                   29.97
    the index ran from                 12.6 to 89.45
```

One bar puts a market inside it. Take any recorded run of forty bars or more and ask what final close lands the index on 70.003. Every one of the 2,901 runs admits such a close, because the index rises with the last price. Most of those bars are large, because most windows sit a long way from seventy: the middle one needs a move of nearly twenty-six per cent. Seventy-two of the runs need a bar of one per cent or less, and the smallest was half a basis point after 271 real days of A21.

```
2,901 recorded runs, the final bar each one needs
    smallest                                  +0.0049%
    middle                                      25.70%
    largest                                    334.53%
    runs needing one per cent or less               72
    runs needing five per cent or less             272
```

The smallest is the one to read. Half a basis point is an ordinary tick on a market that has already run up to the seventy line.

```
A21 2023, 271 recorded bars, then one bar of +0.0049%
    reading published, before                  70.00
    reading published, after                  70.003
    label, before and after                overbought
    vote, before                              NEUTRAL
    vote, after                               BEARISH
    identical at both price scales
```

Design intention. A number that decides something is the number that gets published, and a display rounds it at the moment it is shown. Two copies of one reading is the shape that lets a label and a vote say different things about the same bar while both look correct on their own.

Nothing about the vote moved on the recorded tapes. The reading published now differs from the cut copy on all 3,750, by at most five thousandths of a point; the direction is the same on every one; the strength moves on 180 of them, by at most one ten-thousandth. The panel's consensus moves on 24 readings and the other eleven voters do not move at all.

```
3,750 readings, both price scales
    reading published differs from computed     3,750
    vote direction moved                            0
    vote strength moved                           180
    overbought or oversold moved                    0
    panel consensus moved                          24
    the other eleven voters                         0
```

No gate changes its mind. Over 30,000 verdicts — both chains, four states of the two direction switches — 1,322 fire before the repair and the same 1,322 fire after it. The repair only bites inside a window five thousandths wide, the recorded tapes never enter it, and the largest strength move anywhere else is one ten-thousandth against a confidence floor near a quarter.

Inside the window the gate does change its mind. Every one of the 2,901 runs was walked in with its own final bar and put through both chains at both price scales: 5,802 readings, and on every one of them the label said overbought while the vote said neutral. After the repair not one of them disagrees. Of the 46,416 verdicts, 60 fire before the repair and six of those stop firing, all on the buy side and all the same way.

```
5,802 constructed readings, 46,416 gate verdicts
    label disagreeing with the vote, before     5,802
    label disagreeing with the vote, after          0
    vote moved from neutral to bearish          5,802
    verdicts that fire before                      60
    scrum fires -> scrum refused                    6
    scrum refused -> scrum fires                    0
    fold verdicts changed                           0
    windows                                         3
        DOT 2023, consensus 0.2002 -> 0.1728, floor 0.1786
        A14 2022, consensus 0.2087 -> 0.1813, floor 0.1923
        MATIC 2022, consensus 0.2286 -> 0.1910, floor 0.1923
```

All six need the scrum's own bullish requirement switched on. The mechanism is the panel: a bearish vote at 0.4001 in place of a neutral one at nought pulls the bullish consensus below the confidence floor, and the sell of the surplus is refused on a bar the RSI calls overbought.

Ten places read this indicator. Four read the vote and six read the published number.

reader	what its reading becomes
the panel's consensus	the vote and its strength, decided by the number the labels use; direction unchanged on all 3,750
`ta_engine.py` recorder	records the computed number in place of the cut copy
the trading gate chain	reads the consensus the vote feeds; no verdict moved
`indicator_panel.py` cell	the arrow and the strength percentage; unchanged on all 3,750
`indicator_panel.py` hover text	prints three decimals; moved on 3,356 of 3,750
`indicator_panel_surface.py` hover text	moved on the same 3,356
`ata_spm.py` sentence	prints two decimals; unchanged on all 3,750
`ta_invariants.py` bound	holds the reading between nought and a hundred; unchanged
`phantom_balance.py` record	stores what the indicator published
`snapshots.py` bot.log line	its headline readings are ADX, Z-Score and Kaufman, so it prints the RSI name and strength only

The bot's own position boost and the back test both ask a signal for a number, and both ask only for ADX, Kaufman and Z-Score. Neither has ever asked this indicator for one.

The reading was also published cut to two decimal places, and the panel's hover cell prints three. On 3,356 of the 3,750 readings the third decimal the cell printed was a nought that the indicator had not computed. The confirming sentence prints two decimals, which is exactly what the cut copy carried, so it reads the same on every one.

2026-09-09 19:37 - #414 - the band a millionth of a dollar wide

Item 6 of the issue names two epsilons, one in the Ichimoku and one in the Supertrend. Neither is in this tree. Both came out on 2026-09-04, in the commit that dropped the last eight absolute constants from price-scaled denominators.

```
b27c6cdb  fix(ta): drop the last eight absolute epsilons from scaled denominators
```

That makes eight items out of eight where the expression the issue quotes had already been repaired, and where the defect still standing underneath it was a rounded copy of the reading reaching a consumer. This entry closes that pattern for the whole folder.

John Bollinger's bands are the worst of the set, because their width is what the scrum ramp measures against. The published definition fixes no number of decimal places, and a band is an ordinary price in the asset's own units.

```
Middle Band = SMA(period)
Upper Band  = Middle + std_dev * sigma(period)
Lower Band  = Middle - std_dev * sigma(period)
BandWidth   = (Upper - Lower) / Middle
```

The indicator published all three cut to six decimal places, a step of one millionth of a dollar. A10 trades at three millionths, so the whole channel is narrower than the step and the three bands land on one number.

`src/trading/indicators/bollinger.py` — the reading, before and after

```
"upper": round(upper, 6),          "upper": upper,
"middle": round(mid, 6),           "middle": mid,
"lower": round(lower, 6),          "lower": lower,
```

Measured over 71 recorded Coinbase tapes, run at a price of three millionths of a dollar and again at a four-figure price. Every window from 36 bars in steps of 11 drove the real voting engine, and each reading built a gate context through the shipped scan. Over 1,875 windows at the small price the distance from the midline to the band read exactly zero on 1,014 of them, and on a further 20 the lower band reached zero and the scan refused the channel outright. At the four-figure price it read zero on none.

```
1,875 windows at 3.1e-06        1,875 windows at 1234.0
    half-span exactly 0.0           half-span exactly 0.0
        before  1,014                   before  0
        after   0                       after   0
    band refused outright           smallest half-span
        before  20                      before  7.76315
        after   0                       after   7.76315
```

The consequence is a decision, not a display. The ramp that arms a scrum divides the price's travel by that half-span, and a floor of a millionth of a millionth stands in when the span is zero. The ramp then walked a different path on a cheap asset than on an expensive one holding the same shape.

`src/trading/ata_gate_scan.py` — the step that decides whether a scrum is armed

```
span = max(abs(edge - middle), BAND_SPAN_FLOOR)
travel_frac = abs(price - middle) / span
```

The two price scales disagreed about that state on 162 of the 1,875 windows. They now agree on all of them, which is the property the fleet needs: one detect threshold serves every bot, whatever its asset costs.

```
target_fires compared between the two price scales
    before   agree 1,713   disagree 162
    after    agree 1,875   disagree 0
```

Both gate chains ran on every reading, in all four states of the two direction switches. Of 30,000 verdicts, 1,322 fire before the repair and 58 change. All 58 are on the buy side, all 58 are at the small price scale, and every one sits on a window where the ramp state moved. Nothing changed at four figures.

```
30,000 gate verdicts
    fire before the repair                1,322
    changed                                  58
        scrum refused -> scrum fires         50
        scrum fires -> scrum refused          8
        fold verdicts changed                 0
    changed at 1234.0                         0
    changed at 3.1e-06                       58
    rows carrying them                       27
        rows where the ramp state moved      27
```

The same cut sat on every other indicator in the folder. Fifty-three calls across thirteen files cut a reading where it was computed rather than where it was shown, and all fifty-three are gone. None was kept.

```
zscore 7   ichimoku 6   bollinger 5   landing_strip 5   fvg 4   macd_taper 4
slingshot 9   supertrend 3   bb_proximity 3   atr 2   stochastic_rsi 2
volume 2   rsi 1
```

No display loses its shape, because every one of them already carries its own format. The panel hover cell prints three decimals, the bot log line prints four, and each confirming sentence in the ATA-SMP names its own width.

`src/trading/ata_spm.py` — one voter's sentence, formatted where it is shown

```
"bollinger_bands": (
    "Bollinger Bands",
    (BAND_POSITION_KEY,),
    "band position {value:.4f}",
),
```

Item 6 also names two statements that compute a value and bind it to nothing. The Ichimoku one is the third cloud state. The code named the other two and then derived the third a second time where the reading is published, so one expression now decides it.

```
inside_cloud = not above_cloud and not below_cloud
...
"price_vs_cloud": (
    "inside" if inside_cloud else "above" if above_cloud else "below"
),
```

The volume one is the money-flow slope. It is published beside the balance-volume slope the same indicator already publishes, so the value it computes now reaches a reader.

```
mfi_rising = mfi > mfi_prev
```

Across the 3,750 readings, 339,170 non-numeric readings were compared. None moved and none was dropped. The cloud state is identical on every window, which is what a pure rebinding has to look like, and its inside branch runs on 499 of them rather than never. The money-flow slope takes both values.

```
price_vs_cloud   below 1,579   above 1,112   inside 499   absent 560
mfi_rising       True 2,025    False 1,725
```

Twelve places read these numbers, and the table below says what each one's reading becomes.

reader	what its reading becomes
the scan's scrum ramp	a real band width at any price; 58 of 30,000 verdicts move, all on the buy side
the live bot's scrum ramp	unchanged; it reads the proximity result, whose three band prices were never cut
the Z-Score gate	the computed z; it moved on 3,470 readings and crossed the two-deviation line on none
the ADX gate	unchanged; that reading was uncut already
the Efficiency Ratio gate	unchanged; that reading was uncut already
the confidence floor	the landing-strip boost, uncut; no consensus direction moved
the panel's twelve cells	direction and strength unchanged on all 3,750; strength moved on 358 votes
`indicator_panel.py` hover text	the computed number, printed to three decimals as before
`ata_spm.py` sentences	each keeps its own width, so every sentence reads as it did
`snapshots.py` bot log line	prints four decimals of the computed number
`phantom_balance.py` record	stores what the indicator published
`ta_invariants.py` bounds	no breach on any reading, before or after

Three figures the page already carries need restating against this run. The departures section above says two departures remain, the Vortex direction overwrite and the RSI rounding. Both are repaired: the Vortex on 2026-09-08 and the RSI on the same day, each in its own entry above.

The debug-report citation, overtaken

OVERTAKEN, quoted whole:

> *"The section also lists four scale-invariance test files. None is in the tree, which now holds its conftest and these debug reports and nothing else, so the property they held is held by the runs recorded in each entry instead."*

```
the 2026-09-09 indicator-rounding debug report
```

True today: the four files are still absent, and so is the report named above. The repository tracks one Python module under `tests/`, its own conftest, and no directory of debug reports beside it. The runs recorded in the entries on this page are the whole record of the property.

What changes for an operator: a bot on a market priced in millionths of a dollar now sees a Bollinger channel with a real width instead of a flat line, so its scrum arms on the same bars a bot on an expensive market would.

Trading gate logic chain

The epigraph on the title page reads *dissolvendus reformandusque*, dissolved and reformed. The chain below does exactly that, once per tick: twelve separate readings dissolve into one signed number, and the gates re-form that number into an order or a refusal.

The chain

Diagram: flowchart, top to bottom

- OHLCV candles -> **The 12 indicator modules**
- The 12 indicator modules -> **VotingEngine.compute_all** *(Signal: direction, confidence, weight)*
- VotingEngine.compute_all -> **VotingEngine._aggregate**
- VotingEngine._aggregate -> **VotingSummary.net_score**
- VotingEngine._aggregate -> **VotingSummary.consensus_confidence**
- VotingSummary.net_score -> **consensus_direction: deadband +/- 0.1**
- consensus_direction: deadband +/- 0.1 -> **BULLISH / BEARISH / NEUTRAL**
- BULLISH / BEARISH / NEUTRAL -> **ScrummingBot.tick: is_bullish, is_bearish**
- VotingSummary.consensus_confidence -> **ScrummingBot.tick: is_bullish, is_bearish**
- _skewed_confidence_floor: 0.25 relaxed by the skew -> **ScrummingBot.tick: is_bullish, is_bearish**
- OHLCV candles -> **detect_bb_proximity**
- detect_bb_proximity -> **bb_position**
- bb_position -> **bb_pos against _bb_detect_thresholds**
- ScrummingBot.tick: is_bullish, is_bearish -> **build_scrumming_scrum_chain**
- bb_pos against _bb_detect_thresholds -> **build_scrumming_scrum_chain**
- adx signal detail 'adx' -> **build_scrumming_scrum_chain**
- kaufman_er signal detail 'er' -> **build_scrumming_scrum_chain**
- zscore signal detail 'z' -> **build_scrumming_scrum_chain**
- build_scrumming_scrum_chain -> **ChainResult.should_fire**
- ChainResult.should_fire -> **_tick_execute_scrum, then _execute_sell** *(true)*
- ChainResult.should_fire -> **ChainResult.blocked, one message per gate** *(false)*
- _tick_execute_scrum, then _execute_sell -> **guarded_place_order**

Net, Comp and Conf

`VotingEngine._aggregate` in `src/trading/ta_engine.py` produces all three panel aggregates from the twelve signals.

```
weighted_score = direction * confidence * weight        one per voter
bull_score     = sum of |weighted_score| over the BULLISH voters
bear_score     = sum of |weighted_score| over the BEARISH voters
net_score      = bull_score - bear_score                 the Net column
voted_weight   = sum of weight over the voters that did not abstain
confidence     = min(1.0, |net_score| / voted_weight)    the Conf column
```

Functional. Net is a direction bias, not a reversal signal. Positive means the bullish voters outweighed the bearish ones. Conf is the breadth of that agreement, not its force. Six voters agreeing strongly while six abstain reads lower than twelve agreeing mildly, and the panel says the same thing in the Conf column's own tooltip. An abstaining indicator is left out of the divisor, so a short tape cannot water down the reading of the voters that did

read. Net is rounded to four decimals and Conf is capped at 1.0.

`src/trading/ta_engine.py` — `VotingEngine._aggregate`

```
net = bull_score - bear_score
```

On the panel. Net, Comp and Conf are drawn as one pillar each, standing in the last three columns of the shared grid. A pillar runs from the row-B plot floor to the row-A plot ceiling, so it passes both tables and both bar graphs and tops out on the line a voter at full confidence reaches. `collated_pillars` in `indicator_panel_surface.py` names each pillar and takes its direction from its own metric's sign, so Net follows `net_score`, Comp follows `composite_net` and Conf follows the summary's direction.

`src/gui/main_tabs/indicator_panel_surface.py` — `collated_pillars`

```
first = PANEL_COLUMN_COUNT - len(AGGREGATE_TITLES)
return [
    {"name": title, "direction": way, "column": first + at}
    for at, (title, way) in enumerate(zip(AGGREGATE_TITLES, directions))
]
```

A pillar carries no height reading. The value stays in the table cell above it, and `AGGREGATE_TITLES` names the pillar once, at its base. `RULED_COLUMNS` keeps the row partition, the plot baseline and the dotted increments on TF and the twelve, so none of them crosses a pillar.

When a cell cannot read its figure. `net_cell`, `comp_cell` and `conf_cell` admit a reading through `as_finite_float` before printing it. A reading that is not a finite number draws `ABSENT_CELL_TEXT` in `ABSENT_TEXT_COLOR`: the em dash the Comp column has always drawn where there is no composite. An integer too large to hold as a float is one such reading, and so is a NaN. No cell prints a figure it did not read, and no cell carries a notice in place of one.

`src/gui/main_tabs/indicator_panel_surface.py` — `absent_cell`

```
return {
    "text": ABSENT_CELL_TEXT,
    "text_color": ABSENT_TEXT_COLOR,
    "tooltip": tooltip,
}
```

The grid holds one reading. `timeframe_rows` settles every row — its timeframe, its reading and the signals keyed by indicator — before the row count is cut. The grid therefore holds the timeframes the fed reading holds and no others. A value a row cannot read draws the em dash in that row's own cell, so a refusal never leaves a row carrying the previous reading's numbers. `vote_totals` sums the three vote counts through the same admission step, and `lock_line` draws the em dash for a lock field it cannot read. The Paper and Simulator panels draw their rows and their aggregate cells through the same functions.

`src/gui/indicator_panel.py` — `IndicatorVotingPanel.update_data`

```
rows = ivp.timeframe_rows(multi_tf_summary)
self._vote_totals = ivp.vote_totals(multi_tf_summary)
self._table_a.setRowCount(len(rows))
```



The restyled panel

`src/trading/ta_engine.py` — and the divisor beneath it

```
voted_weight = sum(s.weight for s in signals if not s.abstained)
consensus_conf = abs(net) / voted_weight if voted_weight > 0.0 else 0.0
```

Design intention. Conf answers how many of the voters that could read agree, not how loud they were. Dividing by the weight of the voters that actually read, rather than by all twelve, is the arithmetic that says so. The panel repeats the same rule to the reader in its own tooltip, so the screen and the engine cannot tell two different stories.

`src/trading/ta_engine.py` — the rule, stated where it is applied

```
# A weighted mean divides by the weight of the points included, and an
# abstention is not one; it is NEUTRAL, so it adds nothing to `net`.
```

The deadband

`VotingSummary.consensus_direction` in `src/trading/indicators/types.py` turns Net into a direction, and it holds a deadband of plus and minus 0.1.

```
net_score > +0.1   ->  BULLISH
net_score < -0.1   ->  BEARISH
otherwise          ->  NEUTRAL
```

`ScrummingBot.tick` then folds the direction and the Conf reading into one boolean per side.

```
is_bullish = direction in (BULLISH, NEUTRAL) and confidence >= floor
is_bearish = direction in (BEARISH, NEUTRAL) and confidence >= floor
```

Functional. The confidence floor starts at 0.25 and the bot relaxes it when the vote is skewed. A landing strip at the upper band sets the bullish flag outright, and one at the lower band sets the bearish flag. A neutral direction passes both sides, so a panel with no consensus does not on its own block a trade. The floor and the gates below decide that.

`src/trading/indicators/types.py` — `VotingSummary.consensus_direction`

```
def consensus_direction(self) -> SignalDirection:
    if self.net_score > 0.1:
        return SignalDirection.BULLISH
    elif self.net_score < -0.1:
        return SignalDirection.BEARISH
    return SignalDirection.NEUTRAL
```

Design intention. The deadband stops a near-zero Net reading as a direction. Plus or minus 0.1 against a full voted weight of 11.7 is a very small bias, and the band treats it as none at all. The bot then folds the direction and the confidence into one boolean per side, and a neutral panel passes both.

`src/trading/scrumming_bot.py` — the two booleans, in plain terms

```
is_bullish = direction in (BULLISH, NEUTRAL) and confidence >= floor
is_bearish = direction in (BEARISH, NEUTRAL) and confidence >= floor
```

Current reading, how low the floor goes. The bot relaxes the floor no further than what two indicators reach. The engine runs the vote again with the two heaviest voters at full confidence and the rest at none, at the bot's weights on that tick. A relaxed floor stays above that figure and never rises above 0.25, so a vote needs a third agreeing indicator wherever the plain floor needs one. The DIST sale and the hedge top-up read a second floor, the same one without hunger.

`src/trading/scrumming_bot.py` — `_skewed_confidence_floor`

```
if denominator <= 1.0:
    return floor / denominator
past_two = math.nextafter(two_indicator_ceiling(signals), math.inf)
return min(floor, max(floor / denominator, past_two))
```

Current reading, three agreeing at the floor and at a strip. The floor now stays above the two-indicator line at every skew, so it can rise over 0.25. A landing strip sets its flag only when the vote tops the same line. A bot with every weight at zero keeps its strip as before, because its vote has no line. Live, the Market Inspector scan, Paper and the Simulator read one function for both booleans.

`src/trading/ta_engine.py` — `vote_sides`, the strip and the floor

```
if strip and not three_agree(confidence, signals):
    strip = ""
clears = confidence >= floor
```

The band position

Functional. The band position is %B: zero at the lower band, one at the upper. The two thresholds a scrum and a fold must clear come back in the same terms, so both sides of every comparison are positions and neither side is a price. The fraction is clamped to the 0 to 1 range, so a stored setting outside 0 to 100 cannot push a threshold outside the band. A missing setting falls back to 75 through an explicit test for absence, so a stored zero stays zero.

The relationship in plain terms:

```
detect_frac  = scrum_detect_pct / 100          75% by default
upper_detect = 0.5 + detect_frac * 0.5         0.875
lower_detect = 0.5 - detect_frac * 0.5         0.125

SCRUM requires bb_pos >= upper_detect
FOLD  requires bb_pos <= lower_detect
```

`src/trading/scrumming/circuit_breakers.py` — `_bb_detect_thresholds`

```
detect_frac = max(0.0, min(1.0, detect_pct / 100.0))
half = detect_frac * 0.5
return (0.5 - half, 0.5 + half)
```

Design intention. Working in band positions rather than in prices is what lets one threshold serve the whole fleet. A bot on a four-figure asset and a bot on an asset priced in millionths both read 0.875 and mean the same thing by it. This is the pattern the constants listed above break. A missing setting is tested for absence rather than for truthiness, so a deliberate zero survives instead of being turned back into 75.

`src/trading/scrumming/circuit_breakers.py` — the absence test

```
# None-sentinel default so a legitimate 0 isn't coerced to 75 by
# `or` truthiness.
_raw = getattr(self.config, "scrum_detect_pct", None)
```

The scrum chain gates

`src/trading/gate_chain.py` — `build_scrumming_scrum_chain`

```
return GateChain(
    gates=[
        DeltaPositiveGate(),
        IntervalGate(),
        TADirectionGate(side="scrum"),
        TrendHoldGate(),
        MidlineGate(side="scrum"),
        TargetFiresGate(),
        BBProximityGate(side="scrum"),
        CircuitBreakerGate(side="scrum"),
        HTFDeferGate(side="scrum"),
        HysteresisGate(side="scrum"),
        ADXTrendSuppressionGate(),
        EfficiencyRatioRegimeGate(),
        ZScoreExtremityGate(side="scrum"),
        RipeHarvestScrumOverride(),  # runs in pass 2 regardless of list position
    ],
    side="scrum",
)
```

Functional. Every gate in the list runs, and every refusal is collected. The trade fires only when the refusal list is empty, so one blocked gate is enough to hold it. The order of the list fixes the order the refusals appear in, which is the order the log prints them.

`src/trading/gate_chain.py` — the verdict

```
return ChainResult(
    should_fire=(not final_blocked),
    passed=passed,
    blocked=final_blocked,
    overrides_applied=overrides_applied,
)
```

Design intention. One gate holds one question, and each writes its own refusal in its own words. Reading back a held trade is then a matter of reading which name refused, rather than re-deriving the whole decision.

`src/trading/gate_chain.py` — `DeltaPositiveGate.evaluate`, the pattern every gate follows

```
def evaluate(self, ctx: GateContext) -> GateResult:
    if ctx.delta > 0:
        return GateResult(passed=True)
    return GateResult(passed=False, blocker_message="delta≤0")
```

Gate	Passes when	Reads	Overridden by
`delta_positive`	the holdings sit above target	the target delta	none
`interval`	the delta clears the scrumming interval	the delta	none
`ta_bullish`	`is_bullish`	Net, Conf and the floor	`ripe_harvest_override`

Gate	Passes when	Reads	Overridden by
`trend_hold`	no sustained uptrend	more than 13 of the last 20 candles closing above their open	`ripe_harvest_override`
`midline_scrum`	`bb_pos` above 0.50 and no phantom lock	`bb_position`	`ripe_harvest_override`
`target_fires`	the detect and fire ramp is in FIRE	the target ramp	`ripe_harvest_override`
`bb_proximity_scrum`	`bb_pos >= upper_detect`	`bb_position`	none
`circuit_breaker_scrum`	no soft trip on the scrum side	the breaker state	none
`htf_defer_scrum`	the higher timeframe is not bullish	the coordinator	none
`hysteresis_scrum`	the price cleared the last pivot plus fees	the pivot	none
`adx_trend_suppression`	ADX under 30.0	the `adx` signal detail	none
`efficiency_ratio_regime`	ER between 0.05 and 0.70	the `er` signal detail	none
`zscore_extremity`	Z above -2.0	the `z` signal detail	none

Functional. Three of those gates skip the collated vote and read one indicator's own number: ADX, the Efficiency Ratio and the Z-Score. The bot fills each field from that indicator's signal on the tick. A reading of zero or below is the not-populated marker, and all three pass on it.

`src/trading/gate_chain.py` — `EfficiencyRatioRegimeGate.evaluate`, one of the three

```
def evaluate(self, ctx: GateContext) -> GateResult:
    if ctx.efficiency_ratio <= 0.0:
        return GateResult(passed=True)
    if ctx.efficiency_ratio >= self.upper_threshold:
```

Design intention. These three describe the market regime rather than its direction, which is why they reach the chain on their own instead of through the vote. Passing on the marker keeps a bot that never filled the field from being frozen by it. The cost is real and worth stating: a genuine zero reads exactly the same as a missing one, and the gate cannot tell them apart.

In development. Separating the two would need a marker the field cannot produce, and no shape for that has been chosen.

The fold chain gates

`src/trading/gate_chain.py` — `build_scrumming_fold_chain`

```
return GateChain(
    gates=[
        TranchesQueuedGate(),
        TADirectionGate(side="fold"),
        MidlineGate(side="fold"),
        SmartCeilingGate(),
        BBProximityGate(side="fold"),
        CircuitBreakerGate(side="fold"),
        HTFDeferGate(side="fold"),
        HysteresisGate(side="fold"),
        ZScoreExtremityGate(side="fold"),
        DeepFoldOverride(),
    ],
    side="fold",
)
```

Functional. The buy side runs nine gates against the sell side's thirteen. Six of the sell-side gates have no twin here, and the buy side adds two of its own.

```
absent on the fold side   delta_positive, interval, trend_hold,
                          target_fires, adx_trend_suppression,
                          efficiency_ratio_regime
present only here         tranches_queued, smart_ceiling
```

Design intention. The two chains mirror each other, and two of the six absences carry a written reason. The interval gate says in its own source that it needs no fold twin, because the bot returns early on the same condition. The trend-hold gate is declared sell-side only, and the config field that would arm a buy-side twin is declared and unread. 06-trading-tab.md carries the proposal for that one, at the Strategy Gate Flags group.

Functional. That config field is now removed. The buy side holds no flag for a trend hold and no gate for one, so the asymmetry is a plain absence rather than a flag waiting on a gate. The sell-side gate is unchanged. On one tick with twenty candles all closing above their open, the sell side reported `trend_hold` and the buy side reported nothing.

`src/trading/gate_chain.py` — `IntervalGate`, stating its own case

```
# No FOLD twin; ScrummingBot.tick returns early on below_interval when both
# its fold queue and distribution accumulator are empty.
```

The other four absences carry no stated reason in the code at all. That is worth knowing before anyone assumes the asymmetry is deliberate.

Gate	Passes when	Overridden by
`tranches_queued`	at least one fold tranche is waiting	none
`ta_bearish`	`is_bearish`	`deep_fold_override`
`midline_fold`	`bb_pos` below 0.50	`deep_fold_override`
`smart_ceiling`	the position sits under the smart ceiling	none
`bb_proximity_fold`	`bb_pos <= lower_detect`	none

Gate	Passes when	Overridden by
`circuit_breaker_fold`	no soft trip on the fold side	none
`htf_defer_fold`	the higher timeframe is not bearish	none
`hysteresis_fold`	the price cleared the last pivot plus fees	none
`zscore_extremity`	Z below +2.0	none

The two overrides

Functional. An override runs in a second pass and force-passes gates that already refused. Its own verdict never appears in the pass list or the block list. Each refusal it lifts has its message rewritten to name the override, and the name is recorded separately, so the log tells you which gate was overridden and by what.

`src/trading/gate_chain.py` — `GateChain.evaluate`, the second pass

```
overrides_applied: list[str] = []
for gate in self._override_gates:
    result = gate.evaluate(ctx)
    if not result.passed:
        continue
    for target_name in result.override_gates:
        if target_name in blocked_index:
            idx = blocked_index.pop(target_name)
            _name, _msg = blocked[idx]
            blocked[idx] = (_name, f"OVERRIDDEN_BY:{gate.name}({_msg})")
            overrides_applied.append(
                OVERRIDE_APPLIED_FORMAT.format(
                    gate=target_name, override=gate.name, refusal=_msg
                )
            )
```

Functional. The record separates a refusal that stood from one an override replaced, and it names the override in both cases. A replaced refusal is one line naming the gate, the override that replaced it and the refusal text it replaced. A refusal that stood from a gate an override covers says which override did not apply, and a refusal from a gate no override covers names none.

`src/trading/gate_chain.py` — the two forms a reader reads

```
OVERRIDE_APPLIED_FORMAT = "{gate} overridden by {override}: {refusal}"
OVERRIDE_NOT_APPLIED_FORMAT = "{message} [override not applied: {overrides}]"
```

Override	Arms on	Force-passes
`ripe_harvest_override`	positive delta, interval cleared, `bb_pos >= upper_detect`	`midline_scrum`, `target_fires`, `trend_hold`, `ta_bullish`
`deep_fold_override`	negative delta, interval cleared, `bb_pos <= lower_detect`	`midline_fold`, `ta_bearish`

Design intention. At the band extreme, the band position outranks the vote. That comes straight from the operator's own reading of the Bollinger Bands: they are the boundary a trade fires at, and a position sitting on one is the condition he wants to trade. Neither override touches the circuit breaker, the hysteresis or the three

indicator gates, so the safety gates keep their veto.

`src/trading/gate_chain.py` — `RipeHarvestScrumOverride`, and what it may lift

```
name = "ripe_harvest_override"
side = "scrum"
overrides = ("midline_scrum", "target_fires", "trend_hold", "ta_bullish")
```

The trend-hold override

Functional. A third override clears the uptrend hold, and it is not a gate in either chain. It runs in `ScrummingBot.tick` before the chain is built, and it fires on either of two conditions: the position is at least twice the scrumming interval away from target in dollars, or Band Travel has raised its own trigger. When it fires, the bot sells into a sustained uptrend that the SCRUM holds in sustained uptrend box was set to hold through.

`src/trading/scrumming_bot.py` — the override in `ScrummingBot.tick`

```
trend_override = abs(delta) >= _interval_usd * 2.0 or band_travel_triggered
if trend_hold and trend_override:
```

Override	Arms on	What it clears
`trend_hold_override`	the delta at twice the scrumming interval in dollars, or the Band Travel trigger	the uptrend hold, before the chain is built

Functional. The operator sees it in three places. The bot log carries a TREND-HOLD OVERRIDE line naming the trend share, the condition that cleared it and the dollar figure behind that condition. The per-tick GATES line names TRND under its overridden clause, so the TRND light paints the landing strip's cyan rather than the passed green. The gate record carries one entry under `overrides_applied`, written only when the box was set, so a tick with the box clear reads apart from a tick the override acted on.

`src/trading/scrumming_bot.py` — the record entry, in `ScrummingBot.tick`

```
if _trend_hold_cleared_by and _flag_hold_in_uptrend:
    _scrum_replaced.append(
        OVERRIDE_APPLIED_FORMAT.format(
            gate=_GATE_TREND_HOLD,
            override=TREND_HOLD_OVERRIDE,
            refusal=(
                f"trend_hold({trend_strength:.0%}) cleared by "
                f"{_trend_hold_cleared_by}"
            ),
        )
    )
```

Design intention. The override stays as it is. A move of twice the interval is the profit the whole cycle exists to take, and holding it back to wait out a trend forfeits it. What was missing was the record, not a different rule: the hold was cleared and no screen said so.

Functional. The override is in the live engine alone. The back test and the paper fleet build their gate inputs from the raw trend reading and never clear it, so a Simulator tick and a Paper tick hold where a live tick sells. The checkbox tooltip says so on all three screens.

`src/trading/scrumming_bot.py` — the only module the override is written in

```
trend_override = abs(delta) >= _interval_usd * 2.0 or band_travel_triggered
```

Where the armed flag comes from

Functional. The gate row's two armed flags are the chain's own verdict. One chain decides per side, and the row carries that decision and the refusals that produced it, so a row and a decision cannot disagree. Every refusal the gate row names is a refusal a gate wrote, including the three indicator gates, which the row reports under their own names.

`src/trading/scrumming_bot.py` — the row, written in `ScrummingBot.tick`

```
self._last_gate_state["scrum_armed"] = bool(
    _scrum_chain_result.should_fire
)
self._last_gate_state["scrum_blockers"] = list(_scrum_blockers)
```

The one effective-flag mapping

Functional. Five raw readings become five effective readings through one function. A cleared require box passes its direction outright; a cleared hold or defer box drops its block. Four readers call it: the live tick, the Market Inspector's gate scan, the back test and the paper fleet.

`src/trading/gate_chain.py` — `effective_gate_flags`, the one mapping

```
return EffectiveGateFlags(
    eff_is_bullish=bool(is_bullish) if require_bullish else True,
    eff_is_bearish=bool(is_bearish) if require_bearish else True,
    eff_trend_hold=bool(trend_hold) if hold_in_uptrend else False,
    eff_htf_blocks_scrum=bool(htf_blocks_scrum) if defer_scrum else False,
    eff_htf_blocks_fold=bool(htf_blocks_fold) if defer_fold else False,
)
```

Reader	Where it calls the mapping
Live	`src/trading/scrumming_bot.py`, in `ScrummingBot.tick`
Market Inspector	`src/trading/ata_gate_scan.py`, in `build_context`
Simulator	`src/simulator/back_test.py`, in `tape_context`
Paper Trader	`src/paper/paper_run.py`, in `paper_tape_context`

Which records name the override

Functional. Three records carry the name of the override that replaced a refusal, one for each data source, and all three write it under the same key. The key is spelled once, in `OVERRIDES_APPLIED_KEY`.

Data source	The record	Where the name is written
Live	the gate log row `ScrummingBot.tick` writes	`scrum_fixture.overrides_applied`
Paper	the gate row `paper_log.paper_row` writes	`scrum_fixture.overrides_applied`
Simulator	the `bot.gate_decision` event `SimBus` emits	`scrum_fixture.overrides_applied`

Functional. The Paper Trader and the Simulator both read one record, the one the shipped chains are evaluated into. A row where no override acted carries an empty list; a row where one acted carries one entry per refusal it replaced. A row that passed outright and a row an override passed therefore read apart.

`src/simulator/validation.py` — `latch`, the record both of them read

```
return {
    "scrum_armed": bool(scrum.should_fire),
    "fold_armed": bool(fold.should_fire),
    "scrum_blockers": [message for _name, message in scrum.blocked],
    "fold_blockers": [message for _name, message in fold.blocked],
    "scrum_overrides_applied": [str(one) for one in scrum.overrides_applied],
    "fold_overrides_applied": [str(one) for one in fold.overrides_applied],
}
```

Not available. The nineteen gate lights the Simulator's parity comparison draws read the armed flag and the blocker list alone, so a replaced refusal still reads there as a plain pass. Its two sides are a recorded row and a rerun of the same inputs, and a row recorded before the key existed carries none, so reading the override into those lights would move the parity verdict on past rows.

`src/simulator/validation.py` — `light_states`, the four inputs it takes

```
def light_states(
    scrum_armed: bool,
    fold_armed: bool,
    scrum_blockers: Sequence[str],
    fold_blockers: Sequence[str],
) -> list[dict]:
```

Reading the chain after the fact

Not available. A trade already on the books keeps the reading that was recorded beside it. Nothing recomputes the direction, the Net, the Conf, the vote counts, an indicator's own vote or the band position for a past trade, so a gate row that was never written stays missing.

One writer. A gate row reaches `gate.log` only when `LogManager.log_gate_decision` writes it at the moment the gate decides.

`src/core/logging_engine.py` — the one call that records a decision

```
def log_gate_decision(
    self,
    exchange: str,
    bot_id: str,
    symbol: str,
    scrum_armed: bool,
    fold_armed: bool,
    scrum_blockers: Optional[list] = None,
    fold_blockers: Optional[list] = None,
```

2026-09-24 22:50 - #202 - one admission rule for every panel reading

Four rules read a stored number and they disagreed. A saved flag read as the number one on the panel and as nothing in the trading package. A 400-digit saved number ended the panel's refresh, which left one timeframe's label above the previous reading's numbers.

One rule reads every number now, and it is the trading package's own.

`src/trading/container/config.py` — the one admission rule

```
def as_finite_float(value) -> Optional[float]:
    if type(value) is float:
        return value if math.isfinite(value) else None
    if type(value) is int and -_FLOAT_SAFE_INT <= value <= _FLOAT_SAFE_INT:
        return float(value)
    return None
```

It answers nothing for a saved flag, for a not-a-number, for either infinity, for a number written as text and for an integer too large to be a float. It never raises, so a refusal cannot end a refresh.

What a refused reading draws. Each cell says for itself. The panel draws zero, so a refused Net reads `+0.00`, a refused Conf reads an empty bar at `0%`, and a refused indicator cell reads its direction beside `0%`.

`src/gui/main_tabs/indicator_panel_surface.py` — the figure a refusal draws

```
#: What a cell, bar, pillar or tint draws when as_finite_float refuses its reading.
REFUSED_READING: float = 0.0
```

The Qt panel and the React panel now build one cell from one pair of builders, `indicator_cell_text` and `indicator_cell_colors`, so the two draw the same text and the same tint from the same reading.

Three sentences the retired rules carried. Each is quoted whole, and each is overtaken by the sentence written under it.

> *"One published reading as a float, or* `fallback` *when it is not one."*

Overtaken. That rule read a saved flag as the number one and raised on a saved integer too large to be a float. `as_finite_float` answers nothing for either, and `REFUSED_READING` is the figure the panel draws in its place.

> *"*`value` *as a float, or* `fallback` *when it is not one."*

Overtaken, and this sentence stood in two files. Both read either infinity as a dollar figure. `as_finite_float` answers nothing for either, and `REFUSED_FIGURE` is the figure a Paper record and a Simulator record draw.

> *"The number one market row carries, nothing where it carries no number."*

Overtaken. That rule refused a saved flag and still raised on a saved number too large to be a float. `as_finite_float` refuses both, and `VOLUME_REFUSED_USD` is the volume a refused pair row sorts by.

2026-10-24 - #665 - a gate row carries the panel it was decided on

The gate log is how a past decision is read back. A row held the decision and none of the numbers behind it, so the reasoning was unrecoverable. A row now carries one cell per voter, and each cell holds that indicator's direction, its confidence, its weight and its headline reading.

`src/trading/scrumming/snapshots.py` — the panel a row carries

```
def _indicator_values(self) -> Optional[dict]:
    summary = getattr(self, "_last_summary", None)
    if summary is None:
        return None
    return _build_panel_snapshot(summary)
```

Both gate emitters pass it, so a fired trade and a refused tick are recorded the same way. `src/trading/scrumming/snapshots.py, in _emit_gate_decision_at_fire` writes the fired row, and `_emit_gate_decision_at_quiet_tick` writes the refused one.

An absent panel is not a row of zeros. A tick that computed no panel leaves the key off its row, so a reader can tell a panel that was never computed from a panel whose indicators all read zero. `src/core/logging_engine.py, in log_gate_decision` writes the key only when it is given one.

Three rows read back off a written gate log — one fired, one refused, one with no panel:

```
trade_action SCRUM    adx BULLISH 0.71 weight 1.4 adx 27.5
                      z_score BEARISH 0.33 weight 0.9 z -1.82
                      kaufman_er NEUTRAL 0.00 weight 1.0 er 0.18
tick_exit gate-chain  the same three cells
tick_exit gate-chain  no indicator key at all, because no panel was computed
```

No gate decision moved. Both scrumming chains were driven over 1024 gate input sets before and after the change: 32 scrum fires and 32 fold fires on each side, and 0 of 1024 whole decisions differ.

2026-10-25 - #665 - the consolidation figure is declared once

Row 280. Landing Strip detection asks whether a Heikin-Ashi body is tight. The figure it compares against is three percentage points of the candle's own high-low range, and that figure was written in five places: the detector's own parameter, a number typed at the live tick, a second number typed at initial entry, a named copy in the gate scan, and a third named copy in the Simulator.

No control writes this figure. It is not an operator setting, and the Product Manual named it under no spelling before this entry.

One declaration now serves every caller, and it sits beside the detector that reads it.

`src/trading/indicators/bb_proximity.py, in detect_bb_proximity`

```
CONSOLIDATION_THRESHOLD_PCT: float = 3.0


def detect_bb_proximity(
    candles: list[Candle],
    tolerance_pct: float = 1.0,
    consolidation_threshold: float = CONSOLIDATION_THRESHOLD_PCT,
    min_pattern_candles: int = 2,
```

Four callers read that one name. The live tick and the initial entry are the two that reach real money.

```
src/trading/scrumming_bot.py, in tick
src/trading/scrumming/tick_phases.py, in _tick_initial_entry
src/trading/ata_spm.py, in proximity_of
src/simulator/validation.py, in bb_reading
```

The last of the four is read by the Paper build, the Simulator and the back test, so Live, Paper and the Simulator now ask the same question of a candle.

Driven on one 222-candle tape, the reading is unchanged. The declaration was then moved to a figure nothing uses and all four callers followed it together, while the detector's other two opening figures did not move.

```
figure 3.0, the declaration
  gate scan reading     landing strip upper, 10 candles, strength 1.0000
  Simulator reading     landing strip upper, 10 candles, strength 1.0000

the one declaration moved 3.0 -> 0.77
  gate scan reading     no landing strip, 0 candles, strength 0.0000
  Simulator reading     no landing strip, 0 candles, strength 0.0000
  pattern candles       2      unmoved
  band tolerance        1.0    unmoved
```

Before the change the same move reached the detector's own parameter alone, and all four callers kept reading three, because each passed its own copy. The file is byte identical after the move was put back.

Subsystem Tabs

Reference. Main Window, Market Inspector and Asset Charts each have a heading below, and the manual's own text for the screen comes first with what the code does today under it. Every other screen has one section of its own, one file per screen in 08-tabs/, and this part points at it rather than repeating it.

Detail: 08-tabs/simulator.md. Detail: 08-tabs/paper-trader.md. Detail: 08-tabs/proof-of-accumulation.md. Detail: 08-tabs/bot-swarm.md. Detail: 08-tabs/history.md. Detail: 08-tabs/console.md. Detail: 08-tabs/system-status.md. Detail: 08-tabs/settings.md.

One list names the ten tabs the window builds, and every screen below that the window does not build says as much in its own entry. Two are skeletons now: Status and Accumulation each draw a name, one sentence saying the tab is not built, and the issue that carries the build-out. Sim and Paper were skeletons until issues #117 and #19 built them, and both are full screens today.

`src/gui/main_tabs/main_window_surface.py` — the order before issue #450

```
CANONICAL_TAB_ORDER = (
    TRADING_TAB,
    MARKET_INSPECTOR_TAB,
    BOT_SWARM_TAB,
    ASSET_CHARTS_TAB,
    HISTORY_TAB,
    SIMULATOR_TAB,
    CONSOLE_TAB,
    PAPER_TRADER_TAB,
    SYSTEM_STATUS_TAB,
    PROOF_OF_ACCUMULATION_TAB,
)
```

Each of those names is a constant holding the label the tab bar shows. The skeletons take their label from their own surface, so the bar and the screen cannot carry two spellings of one name. Issue #450 renamed every one of those constants and reordered the tuple, and the section under this one holds what the file carries today.

`src/gui/main_tabs/main_window_surface.py` — the labels before issue #450

```
TRADING_TAB = "Trading"
MARKET_INSPECTOR_TAB = "Market Inspector"
BOT_SWARM_TAB = "Bot Swarm"
ASSET_CHARTS_TAB = "Asset Charts"
HISTORY_TAB = "History"
SIMULATOR_TAB = "Simulator"
CONSOLE_TAB = "Console"

PAPER_TRADER_TAB = paper_trader_tab_surface.HEADING
SYSTEM_STATUS_TAB = system_status_tab_surface.HEADING
PROOF_OF_ACCUMULATION_TAB = proof_of_accumulation_tab_surface.HEADING
```

The module `paper_trader_tab_surface` named in that block is gone. `src/gui/paper/paper_trading_tab_surface.py, in HEADING` carries the Paper label today.

The order, the names and the colours today

The bar now opens on Sim and every tab carries a single word. The order below is what the window ends with, and it is decided in the one place named above.

`src/gui/main_tabs/main_window_surface.py` — `CANONICAL_TAB_ORDER`

```
CANONICAL_TAB_ORDER = (
    SIM_TAB,
    PAPER_TAB,
    LIVE_TAB,
    CHARTS_TAB,
    INSPECTOR_TAB,
    SWARM_TAB,
    ACCUMULATION_TAB,
    HISTORY_TAB,
    STATUS_TAB,
    CONSOLE_TAB,
)
```

Eight of the ten labels are new words for screens that already existed. History and Console were already one word and did not change. Each constant was renamed to match the word it now holds.

`src/gui/main_tabs/main_window_surface.py` — the labels

```
LIVE_TAB = "Live"
INSPECTOR_TAB = "Inspector"
SWARM_TAB = "Swarm"
CHARTS_TAB = "Charts"
HISTORY_TAB = "History"
SIM_TAB = "Sim"
CONSOLE_TAB = "Console"

PAPER_TAB = paper_trading_tab_surface.HEADING
STATUS_TAB = system_status_tab_surface.HEADING
ACCUMULATION_TAB = proof_of_accumulation_tab_surface.HEADING
```

The eight renames, in the order the bar reads them:

was	is now
Simulator	Sim
Paper Trader	Paper
Trading	Live
Asset Charts	Charts
Market Inspector	Inspector
Bot Swarm	Swarm
Proof of Accumulation	Accumulation
System Status	Status

Sim, Paper and Live carry the promotion pipeline, and three different grounds separate them at a glance. The other seven tabs share the gold ground.

`src/gui/main_tabs/main_window_surface.py` — `TAB_GROUNDS`

```
TAB_GROUNDS = {
    SIM_TAB: BLACK_GROUND,
    PAPER_TAB: WHITE_GROUND,
    LIVE_TAB: GOLD_GROUND,
    CHARTS_TAB: GOLD_GROUND,
    INSPECTOR_TAB: GOLD_GROUND,
    SWARM_TAB: GOLD_GROUND,
    ACCUMULATION_TAB: GOLD_GROUND,
    HISTORY_TAB: GOLD_GROUND,
    STATUS_TAB: GOLD_GROUND,
    CONSOLE_TAB: GOLD_GROUND,
}
```

No tab holds a colour of its own. A ground names two theme tokens, and every theme fills those tokens with its own values, so gold is the yellow that suits each theme rather than one hex value everywhere.

`src/gui/theme_engine.py` — the six tab tokens each theme carries

```
theme             tab_gold_bg tab_gold_text tab_black_bg tab_black_text tab_white_bg tab_white_text
cyberpunk_dark    #fcee0a     #8c0018       #0a0a0f      #ff5577        #f5f5fa      #0a0a0f
neon_light        #f0cf1f     #99001f       #1a1a2e      #ff6b8a        #ffffff      #1a1a2e
classic_terminal  #ffff00     #990000       #0a0a0a      #ff3333        #e8e8e8      #0a0a0a
minimal_modern    #eab308     #7f1d1d       #1a1a1a      #ff6b6b        #ffffff      #1a1a1a
glass_metal       #e8b34a     #6b1020       #1c1c24      #ff6688        #e8e8f0      #1c1c24
```

Every pair above clears WCAG 2.2 AA at 4.5 to 1. The narrowest margin is red on gold under Minimal Modern, at 5.22 to 1.

The Qt bar paints those colours itself, because a Qt style sheet cannot colour one tab by its position. The theme's own style sheet sets the six colours on the bar, so switching theme repaints it.

`main_tab_bar` — `MainWindowTabBar.paintEvent` The file is deleted and no module paints a tab ground by its position now. The bar takes its look from the theme's own Qt style sheet, in `src/gui/theme_engine.py` under its `QTabBar::tab` rules, with the selected-tab rule in `src/gui/main_tabs/main_window_surface.py`. The block below records what the bar did while the file was in the tree.

```
painter.fillRect(self.tabRect(index), ground)
painter.setPen(text)
painter.drawText(self.tabRect(index), int(Qt.AlignCenter), self.tabText(index))
```

The React bar draws the same ten buttons with the same two colours each. The window publishes the colours beside the labels, so the page holds no palette of its own.

`src/gui/main_tabs/main_window_surface.py` — what the frontend reads

```
"tab_labels": list(self.tab_labels),
"tab_methods": dict(self.tab_methods),
"tab_colours": {tab: dict(pair) for tab, pair in self.tab_colours.items()},
"theme": self.theme,
```

The Live tab, which the manual's part list calls the Trading Tab, has its own section, 06-trading-tab.md, and the Indicator Voting Panel has 07-indicators.md.

A strategy reaches real money through four steps: Market Inspector, Simulator, Paper Trader, Live. Each step is a gate, and all four screens are built. 08-tabs/promotion-pipeline.md draws the chain and marks where a step still hands nothing to the next.

The sections below run in the order 04-manual-parts.md lists the tabs.

Main Window

Portfolio Information Panels

Found at the top of the Main Window at all times, this provides metrics for total performance and activity of the platform.

The window is built in three parts: the menu bar, the header strip, and the tab row. The panels above fill that strip, and each tab below sits in that row.

The strip is one row. Five labelled columns run down the left — SPENDABLE, REALISED, LOCKED, MATURE and EXCH — and five counter cards close it on the right.

`src/gui/main_tabs/header_strip.py` — `HeaderStripMixin._build_header_strip`

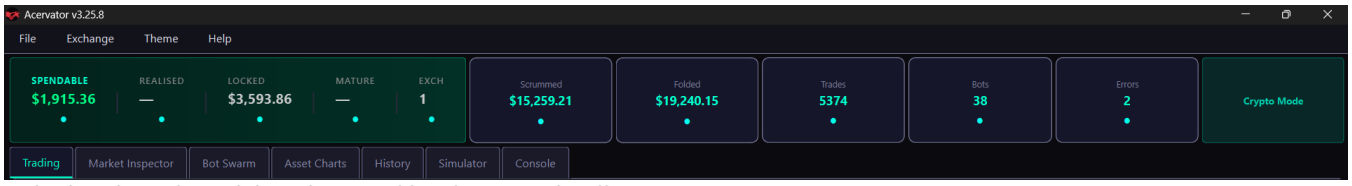
```
self._spendable_widget = SpendableProfitsWidget()
top_row.addWidget(self._spendable_widget, stretch=3)
```

The five cards are Scrummed, Folded, Trades, Bots and Errors. One aggregate call fills every field on the strip once a tick, which keeps two cards from disagreeing about the same fleet.

`src/gui/main_window.py` — `_refresh_dashboard`

```
agg = self._bot_manager.get_aggregate_stats()
_scr = float(agg.get("total_scrummed_usd", 0.0) or 0.0)
_fld = float(agg.get("total_folded_usd", 0.0) or 0.0)
self._stat_scrummed.set_value(f"${_scr:,.2f}")
self._stat_folded.set_value(f"${_fld:,.2f}")
```

Each field carries a privacy dot that masks the value through the registry the Swarm tab shares. The strip hides itself while the Sim tab or the Paper tab is active, and the Paper tab draws four figures of its own in place of it. The absent live strip is itself the signal that the screen is not live trading.



The header strip and the tab row, with Privacy Mode off.

The chrome above the strip carries four menus. File holds Settings, Reset All Settings and Exit. Exchange holds Add Exchange. Theme lists the five display names the theme engine declares, and Help holds About.

`src/gui/main_window.py` — the menu bar

```
file_menu = menu_bar.addMenu("&File")
file_menu.addAction("&Settings", self._open_settings)
file_menu.addAction("&Reset All Settings", self._reset_settings)
```

What the Theme menu does

The Theme menu paints and it stores. One press applies the theme to the Qt window, repaints the tab labels, repaints every React page already open, and writes the name under the `theme` key, so the choice is still there at the next start.

`src/gui/main_window.py` — `_switch_theme`

```
painted = repaint_pages(self, name)
self._remember_theme(name)
```

A page already up takes the new palette without being rebuilt. One statement per web view rewrites the six chrome colours on the document root, so every drawn row and the scroll position stay where they were.

```
the Market Inspector page, 900x600, 92,250 sampled pixels of ground

before the press   #0a0a0f
after the press    #f5f5fa    184,500 sampled pixels differing
pressed back       #0a0a0f    identical to before
the Activity Log   1 React pages repainted in neon_light
```

A page built later opens on the same theme. Every React page host named one theme in its own source; each now leaves the name unset, and the page reads back the theme the window is painted in.

`src/gui/react_history_panel.py` — `page_theme`

```
return stored_theme(applied_theme() if theme is None else theme)
```

Driven with nothing passed to the builder, the Alerts, Console, History and Market Inspector pages each painted a different ground under a stored Cyberpunk Dark and a stored Neon Light, at 184,500 to 187,500 sampled pixels differing per page. All four were identical before.

The window's theme table and the chart chrome table are one declaration now. The chart table is built from the names the theme table declares, so a name cannot sit in one and not the other, and the five colour bodies it holds are unchanged.

`src/gui/tradingview_chart.py` — `CHART_THEMES`

```
CHART_THEMES: dict[str, dict[str, str]] = {
    name: _CHART_CHROME.get(name, _CHART_CHROME[DEFAULT_THEME_NAME])
    for name in THEMES
}
```

The theme does not reach the contents of a page. A page's rows, panes and buttons arrive on the payload as a Qt stylesheet string built from the one dark token table, which carries no theme dimension. The Console tab, driven under three themes, painted the same five grounds every time, and each of the five is a token from that table.

`src/gui/design_system.py` — the grounds the Console tab painted under all three

```
SURFACE_CHART = "#0a0a12"  # Chart, console and group-box ground
SURFACE_CONSOLE = "#05050a"  # Console ground
SURFACE_CONSOLE_HEADER = "#0a0a14"  # Console header ground
```

Privacy Mode is off in the figure, so each field draws its own value rather than four asterisks. SPENDABLE and LOCKED carry dollar figures. EXCH counts the open exchange sub-tabs. Crypto Mode at the right swaps the window between the crypto layer and the stock layer.

REALISED and MATURE draw an em dash. Both take a literal absence at the one call site that fills them, on every tick.

`src/gui/main_window.py` — `_refresh_dashboard`

```
self._spendable_widget.update_profits(
    {
        "spendable": _wallet_cash,
        "total_realised": None,
        "locked": _crypto_value,
        "mature": None,
        "exchange_count": exchanges,
    }
)
```

Turning Privacy Mode on masks that em dash to asterisks, which reads as a hidden number rather than an empty column. 06-trading-tab.md carries the proposal for the first of the two. Issue #428 carries a second disagreement on this strip, between what the counter tooltips promise and what the cards draw.

The issue #428 clause above is overtaken. It is not edited and not removed. It read:

> *Issue #428 carries a second disagreement on this strip, between what the counter tooltips promise and what the cards draw.*

Issue #428 closed on 2026-10-03. The two counter tooltips name the same source the cards draw from, and neither promises a reset.

```
src/gui/main_tabs/header_strip_surface.py

SCRUMMED_TOOLTIP   "the dollars every bot has sold so far this year ...
                    comes from the exchange's own trade history, so the venue
                    and the card agree ... It survives a restart."
FOLDED_TOOLTIP     the same sentence for buys
```

Current reading, 2026-10-29. Both counter descriptions are shorter, within twenty words. They keep the source and the restart.

`src/gui/main_tabs/header_strip_surface.py` — `SCRUMMED_TOOLTIP`

```
SCRUMMED_TOOLTIP = (
    "Dollars every bot has sold this year, read from the exchange's own trade "
    "history. It survives a restart."
)
```

Both columns now carry an exchange figure. The window hands the strip the same payload the React strip receives, built by one function, so the two hosts cannot show different numbers.

`src/gui/main_window.py` — `_refresh_dashboard`

```
self._spendable_widget.update_profits(
    header_strip_surface.profits_payload(agg, exchanges)
)
```

REALISED is the venue's matched profit and loss across the fleet. MATURE is the profit held in positions worth more than three times what they cost. Where the venue has answered for no bot, both stay empty, and the mask now leaves an empty column empty instead of drawing four asterisks over it.

`src/gui/main_tabs/header_strip_surface.py` — `exchange_amount`

```
answered = int(data.get(EXCHANGE_FRESHNESS_KEY, 0) or 0)
if answered <= 0:
    return None
```

The venue's portfolio breakdown answers each position's cost basis, average entry price and unrealised profit, and each bot reads those three from it. The breakdown carries no lifetime realised figure for a spot position, so REALISED is the platform's first-in, first-out match over every fill the venue holds for each bot's symbol, one figure per bot, summed across the fleet; a bot pages the whole history once and then one page per refresh, and logs the venue's cost basis against the cost of the buys left open.

`src/trading/scrumming/reconciliation.py` — `fetch_fill_history`

```
if self._fill_history is None:
    self._fill_history = FillHistory(self.config.symbol)
return await self._fill_history.refresh(self.exchange)
```

Detail: 08-tabs/portfolio-panels.md.

2026-09-09 07:23 - #128 - the header strip's spendable panel is a panel, its dot is not

The header strip is one registered panel that draws two other modules inside itself. The shell mounts `header_strip` as chrome, above the tab bar and on show for every tab. Inside it, the spendable panel and the five counter cards each fill a space the strip leaves for them, and each of those spaces is a mount of its own.

`src/gui/web/header_strip.js` — the spendable space, as the strip draws it

```
  function renderSpendable(target, model) {
    var host = global.acervatorPanelHost;
    var space = spaceNamed(target, SPENDABLE);
    if (!host || space === null) {
      return null;
    }
    space.setAttribute(CHILD_ATTR, SPENDABLE_MODULE);
    return host.mount(SPENDABLE_MODULE, space, model) ? space : null;
  }
```

`spendable_profits.js` now registers, so the shell knows it by name and the strip draws it the way the Live tab draws its Activity Log and the Charts tab draws its chart. The registration names no bridge method, so the tab bar gives it no tab. Nineteen panels register where eighteen did before.

The privacy dot is a different kind of thing. It has no space and no host: two other modules render it as an element inside their own trees, so the panel host never draws it and a registration would name a panel with nowhere to go. Its Registers in Electron cell now reads `-`, the mark this table already uses for a column that does not apply to a row.

`src/gui/web/spendable_profits.js` — where the dot comes from

```
  function dotSpan() {
    var own = global.acervatorDot;
    if (own && own.Dot) {
      return own.Dot;
    }
    var api = global.acervatorHeader;
    return api && api.PrivacyDot ? api.PrivacyDot : null;
  }
```

Both builds were read on one fleet snapshot and one privacy state, with LOCKED and Folded masked and every other field revealed. Nothing was written into the page: the shell fetched every figure over the bridge. Every value, label, glyph, colour and tooltip on the strip agrees between them.

The strip's slots now share the row the way the Qt layout shares it. A Qt stretch divides the whole width; a CSS `flex-grow` divides only what is left over, so the counters drew at five different widths. With a zero flex basis the five cards measure 148 pixels each against Qt's 153, and the spendable panel takes 34.6 per cent of the row against Qt's 34.8.

Qt file	React module	Uses React	Bridge	Manifest	Registers in Electron	Ships in the build	RENDERS	Scope
`src/gui/widgets/spendable_profits.py`	`spendable_profits.js`	yes	yes	yes	yes	yes	yes	in scope
`src/gui/widgets/privacy_dot.py`	`privacy_dot.js`	yes	yes	yes	-	yes	yes	in scope

The strip fetches its own figures. The shell asks for the strip with an empty request, so both surfaces now read the fleet the bridge is bound to, the way the History, Charts, Trading and Inspector surfaces already read theirs. The aggregate is `BotManager.get_aggregate_stats`, the same call the Qt window's own tick makes, and the exchange count is the length of `SettingsManager.list_exchanges`, which is the list the window builds

one sub-tab from. Neither figure is derived in the surface or in the page.

`src/gui/main_tabs/header_strip_surface.py` — the request fills itself

```
def live_view_model(params: dict, live: Any) -> dict:
    asked = dict(params or {})
    if asked.get(STATS_PARAM) is None:
        aggregate = fleet_aggregate(live)
        if aggregate is not None:
            asked[STATS_PARAM] = aggregate
```

The spendable columns take the same aggregate through `header_strip_surface.profits_payload`, the one builder the Qt tick already calls, so the two hosts read one snapshot and cannot disagree.

Each privacy dot in the strip's columns now carries the pointing hand the Qt dot sets on itself. `privacy_dot_surface.dot_view` publishes `cursor_shape`, and the two surfaces that build a dot for this strip now keep it. `dashboard_stat_card_surface` still cuts it away, so the five counter dots draw with the ordinary pointer; that card is its own row.

Qt draws a one pixel panel edge around each counter card where the page draws none, because `dashboard_stat_card_surface.FRAME_STYLE` is empty and the `StyledPanel` shape reaches the page as an attribute no rule reads. That is the same row.

2026-09-09 08:42 - #128 - the counter cards and the bot table are components

The five counter cards and the Scrumming bot table are drawn inside a panel that has already registered, and neither registers in its own right. The strip leaves five spaces and builds a request for each one, carrying that counter's caption, amount, privacy field and whether it is pressable. The bot table is drawn once for the exchange the Trading tab is showing, from a payload the module keeps under that exchange's own name.

`src/gui/web/header_strip.js` — one request per counter

```
  function cardRequest(card) {
    var asked = {};
    asked[CARD_LABEL_PARAM] = text(card[LABEL]);
    asked[CARD_VALUE_PARAM] = text(card[TEXT]);
    if (text(card[FIELD_ID]) !== undefined) {
      asked[CARD_FIELD_ID_PARAM] = text(card[FIELD_ID]);
    }
    asked[CARD_CLICKABLE_PARAM] = card[CLICKABLE] === true;
    return asked;
  }
```

The panel host keeps one host element per panel name and asks each panel with one request of its own. Registering either module was driven in the running shell to see what it would give: the card drew an empty caption and the default amount, and the table drew its ten headers over no rows, each into a host of its own under the panel area, while the strip's five counters kept their figures. Both cells now read `-`, the mark this table already uses for a column that does not apply to a row, and the mark the privacy dot took for the same reason.

Each counter card now draws the one pixel edge, the rounded corner and the padding that Qt paints around it. Qt takes all three from the theme rule for a framed panel; the shell selects no theme, so the page had nothing to

read and drew a plain box. The card's own skin carries them, built from the design tokens rather than from a colour written into the page.

`src/gui/main_tabs/dashboard_stat_card_surface.py` — the card's skin

```
FRAME_STYLE = (
    "border-style: solid; "
    f"border-width: {FRAME_LINE_WIDTH_PX}px; "
    f"border-color: {ds.OUTLINE}; "
    f"border-radius: {ds.RADIUS_SM}px; "
    f"padding: {ds.SPACE_CARD_PAD}px;"
)
```

The five counter dots now carry the pointing hand as well. The card cut its dot down to five fields and left out the cursor the dot publishes; it keeps all eight now, which is what the spendable columns already do.

Both builds were read on one fleet snapshot with every field revealed. Nothing was written into the page: the shell fetched the aggregate and the bot list over the bridge, and breaking each source in turn emptied the figures it feeds and left the other side drawing. Every label, amount, colour, glyph, tooltip, column header, row order and cell text agrees between the two.

Two differences remain, and both belong to other rows. Qt fills each card with the theme's card ground, which no design token carries. And the counter space is 141 pixels wide on all five while the card inside takes its content width, which is how the strip has placed its counters since before this change.

Qt file	React module	Uses React	Bridge	Manifest	Registers in Electron	Ships in the build	RENDERS	Scope
`src/gui/widgets/dashboard_stat_card.py`	`dashboard_stat_card.js`	yes	yes	yes	-	yes	yes	in scope
`src/gui/widgets/bot_status_table.py`	`bot_status_table.js`	yes	yes	yes	-	yes	yes	in scope

Inspector

Market Inspector Tab

The concept with the Market Inspector Tab is evaluate markets from a higher point of view and provide strategy proposals in three forms: Oppositional Trading Pairs (Trading Pairs w/ Opposing Trends), Bot Swarm Topologies (Bot Swarm Network Proposals), and Exchange Comparison Arbitrage.

The tab is now called Inspector. It sits fifth on the bar, on the gold ground with red text.

The tab splits in two. The left half holds the HTF Signals table and the Opposing Pairs table, both scored by one analyzer. One method drives the whole pipeline and keeps the results on that analyzer.

`src/trading/market_inspector.py` — `MarketInspector.scan_universe`

```
def scan_universe(
    self,
    candles_by_symbol_by_tf: dict,
    active_symbols: set,
    closes_by_symbol: dict,
) -> None:
```

The right half holds proposal cards from four detectors: momentum funnel, mean-reversion pair, sector cluster and distance to band. The engine takes a plain dictionary, unions the detectors, drops overlapping proposals by asset and caps the result.

`src/trading/topology_proposals.py` — `detect_all_topologies`

```
def detect_all_topologies(
    context: dict[str, Any],
    cap: int = PROPOSAL_CAP,
) -> list[dict[str, Any]]:
```

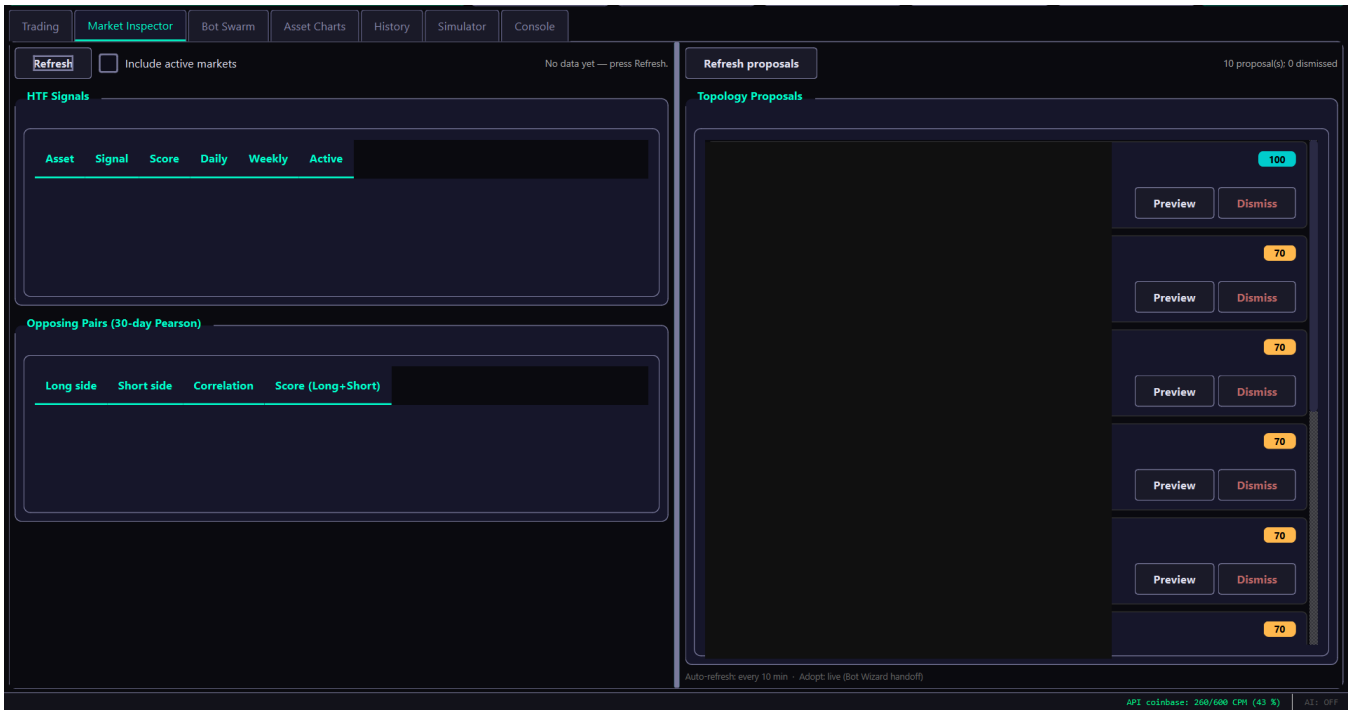
Adopting a card names the new bots, their combined budget and every existing wire the adopt would change before it creates anything.

`src/gui/main_window.py` — `_adopt_topology_proposal`

```
new_bots = [
    b for b in proposal.get("bots", []) if not b.get("existing_bot_id")
]
wires = list(proposal.get("wires", []))
new_count = len(new_bots)
```

Two of the three proposal forms reach the screen. Exchange comparison arbitrage has a module, `src/trading/arbitrage.py`, and no importer under `src/`; the tab draws no arbitrage panel.

In development.



The Market Inspector tab, before the first scan.

Refresh and the Include active markets checkbox sit above the two tables, and the status line reads `No data yet — press Refresh.` until a scan lands, which is the state the figure captures. Leaving the checkbox clear hides the markets a bot already holds.

HTF Signals carries six columns: Asset, Signal, Score, Daily, Weekly and Active. Opposing Pairs carries four: Long side, Short side, Correlation and Score. The pairing enumerates every long against every short and keeps the ones whose thirty-day return correlation sits in the configured negative window.

`src/trading/market_inspector.py` — `MarketInspector._find_opposing_pairs`

```
def _find_opposing_pairs(self, signals: list, closes_by_symbol: dict) -> list:
    """Enumerate long × short candidates; keep pairs whose 30-day
    return correlation sits in the configured negative window."""
    longs = [s for s in signals if s.direction == "long" and s.score >= 0.3]
    shorts = [s for s in signals if s.direction == "short" and s.score >= 0.3]
```

Refresh proposals runs the detectors, and the line beside it counts the proposals held and the cards dismissed. Each card names its archetype and the assets in it, then a line reading the score, the asset count, the wire count and the number of bots an adopt would create. The badge takes its colour from the score: teal at the high threshold and above, amber at the middle one, grey below. Every card in the figure comes from the sector-cluster detector. Preview lists the wires as Source, Target, Pct and Rationale.

Dismiss hides a card for a day, and the dismissal survives a restart. The pane writes the held ids into the operator's settings under `topology_dismissed_proposals` and reads them back on the next launch, so the count beside Refresh names what is held rather than zero.

`src/gui/main_tabs/market_inspector_topologies_surface.py` — `persist_dismissed`

```
def persist_dismissed(self) -> None:
    """Write the held dismissals to the store, recording a refusal."""
    if self.dismiss_store is None:
        return
    try:
        self.dismiss_store.set(DISMISS_SETTINGS_KEY, dict(self.dismissed))
```

A refused write puts `Dismissal NOT saved:` and the cause on that same line in place of the count, and the card stays hidden for the session.

The footer names the auto-refresh period and the adopt route. The status bar under it carries the API load pill, drawn green below half load, amber above it, and red past the monitor's safety percentage, then the `AI:` state label.

React draws this screen. Both halves are one web page inside a single view, and the build decides which class the tab builder makes. `src/gui/main_tabs/market_inspector_tab.py` — `_build_market_inspector_tab`

```
from ..variant_surface import MARKET_INSPECTOR, surface_class

self._market_inspector = surface_class(MARKET_INSPECTOR)()
```

`MarketInspectorReactTab` subclasses the Qt tab, so the scan cycle, the filtering and the analyzer writes are the same code on both sides. Five accessors are what the two sides answer differently: `_set_status`, `_set_refresh_enabled`, `_shown_signals`, `_fill_signal_rows` and `_fill_pair_rows`. The Qt tab writes them into its widgets and the React tab writes them into the page.

`src/gui/react_market_inspector_tab.py` — `MarketInspectorReactTab._build_ui`

```
self._web = QWebEngineView(self)
self._web_page = MarketInspectorPage(self)
self._web.setPage(self._web_page)
self._web.loadFinished.connect(self._on_load_finished)
self._web.setHtml(tab_html())
```

Refresh and the Include active markets checkbox report their press back to Python, and the right pane reports Refresh proposals, Preview, Dismiss, Cancel and Adopt the same way. Setting `ACERVATOR_VARIANT` to `qt` builds the Qt widgets instead, unchanged.

Detail: 08-tabs/market-inspector.md.

2026-09-09 09:42 - #128 - the HTF Signals and Opposing Pairs tables come out

Neither screen drew the two tables. The Qt tab built both group boxes and put neither into a layout, so no table ever reached the window. In the running shell, the React screen drew no table part at all, while its payload carried six signal columns and seven pair columns.

`src/gui/market_inspector.py` — `_fill_pair_rows`

```
def _fill_pair_rows(self, pairs: list) -> None:
    """Hold the opposing pairs the Opposing Trades stepper draws."""
    self._pairs = list(pairs)
    self._render_empty_notes()
```

The screen holds neither table by design. The six-zone layout moved HTF Signals into the reserved Phantom Bot zone, and it hands each opposing pair to the Opposing Trades stepper, one entry at a time. Both sides now drop the renderer, the column sets and the two empty sentences, and the tab keeps only the rows the zones read.

Nothing on the screen moved. The Qt picture and the React picture each match byte for byte before and after the change, and the drawn markup of the panel matches at 21,194 characters.

The earlier comparison that closed these two rows read its table items out of the payload. `signalRow` and `pairRow` handed back cells from the model, which agreed on both sides whatever the page drew. This change drops both.

OVERTAKEN, quoted whole:

> *"The run is in 2026-09-09_market_inspector_tables.md."*

True today: the run record is not in the tree. The repository tracks one Python module under `tests/`, its own conftest, and no directory of run records beside it. The two figures the run produced are the ones in this entry: both pictures match byte for byte, and the drawn markup matches at 21,194 characters.

2026-09-09 23:05 - #128 - the Electron shell draws the Inspector tab and its proposals pane

The Inspector tab draws in the Electron shell. The shell asks the application which tabs it builds, matches `market_inspector.state` to the module that serves it, and gives that module the Inspector tab.

`desktop/renderer/panel_host.js` — `register`

```
    panels[name] = spec;
    dropFault(name);
    kinds[name] = declaredKind(name, spec);
```

The panel host now draws the proposals pane into the right-hand slot, under the name `market_inspector_topologies.js` registers. The slot used to be filled by a call straight to the module, so the registration drew nothing and removing it changed no pixel.

`src/gui/web/market_inspector.js` — `fillSlot`

```
    host.mount(TOPOLOGY_PANEL, slot, null);
```

`desktop/renderer/index.html` links `market_inspector.css`, the stylesheet that carries the colour, border and type of every part the two modules stamp. The file shipped and nothing loaded it, so every button on the screen drew in the browser default. The button rule reads the design token that holds the ground the theme gives a Qt push button.

`src/gui/web/market_inspector.css` — the button ground

```
  background: var(--SURFACE_2, var(--btn-bg));
```

The check boxes on the tab drew as the browser's own control, white and 13 pixels, where Qt paints an 18 pixel box with a `#7a7a9c` edge on `#0e0e1a`. That ground was in no token, so the table gained one under the name of the role it fills, and the stylesheet names the token rather than the value.

`src/gui/main_tabs/design_system_surface.py` — the check-box ground

```
SURFACE_INPUT = "#0e0e1a"
```

Both screens were driven on one payload of 54 markets, 300 daily candles each and three ranked proposals. Twenty-seven of twenty-seven picture items match, so rows 878 and 879 read `yes` under Registers in Electron. The Opposing Pairs rows match at nothing on either side, so that item proves the column set and not the rows.

OVERTAKEN, quoted whole:

> *"The run is in 2026-09-09_market_inspector_register.md."*

True today: the run record is not in the tree. The figures it produced are the ones in this entry: one payload of 54 markets, 300 daily candles each, three ranked proposals, and twenty-seven of twenty-seven picture items matching.

Charts

Under the Asset Charts Tab, you will find our active bot (position) chart display. This will be upgraded to display only one chart at a time and will be able to display all indicators found in the Indicator Voting Panel.

The tab is now called Charts. It sits fourth on the bar, straight after Live, on the gold ground with red text.

The tab scrolls one panel per active bot, each painted with QPainter and no browser, carrying trade markers, position markers, grid lines, the standing tranche floors and the target balance line.

`src/gui/main_tabs/charts_tab.py` — `ChartsTabMixin._build_charts_tab`

```
def _build_charts_tab(self) -> None:
    """Build the Asset Charts tab and add it to the main tab widget."""
    # --- Tab 2: Charts ---
    self._charts_tab = TradeChartsTab()
    self._main_tabs.addTab(self._charts_tab, "Asset Charts")
```

The chart reads its overlays from the engine rather than computing any of them.

`src/gui/native_chart.py` — `CandlestickChart._compute_indicators`

```
candles = self._candles
self._bb_data = BollingerBands(20, 2.0).bands(candles)
vi_plus, vi_minus = VortexIndicator(14).lines(candles)
```

Candles arrive through a fetcher with three sources in order: exchange OHLCV first, a public API second, the last good fetch third.

`src/exchange/chart_data.py` — `ChartDataFetcher.fetch`

```
# --- Source 1: Exchange OHLCV via CCXT ---
if exchange is not None:
    try:
        candles, source = await self._fetch_exchange(
            exchange, symbol, timeframe, limit
        )
        if candles:
            self._set_cache(symbol, timeframe, candles)
            return candles, source
```



The Asset Charts tab, one panel per running bot.

The panel set rebuilds from the current bot roster, and the tab scrolls them.

The header line of a panel names the symbol, the last price, the bot state, the timeframe, the feed and the candle count. The line under it carries the open, high, low and close of the current candle, the change in price and in percent, and the volume. Rising candles draw teal and falling candles red.

The dashed lines across the plot are the standing fold-tranche floors, each labelled with its price. `TB-Ceiling` is the dollar-target line. The price axis runs down the right edge with the last price boxed on it, and the volume bars run along the foot.

The toolbar under each panel holds the TF picker and eight indicator toggles.

`src/gui/main_tabs/native_chart_surface.py` — `INDICATOR_TOGGLES`

```
INDICATOR_TOGGLES = (
    ("bb", "BB", "#50a0f0"),
    ("vortex", "Vortex", "#00ff88"),
    ("macd", "MACD", "#ffcc00"),
    ("stochrsi", "SRsi", "#ff9060"),
    ("ichimoku", "Ichi", "#c080ff"),
    ("volume", "Vol", "#80ffcc"),
    ("slingshot", "Sling", "#ff4488"),
    ("bbullseye", "BBull", "#ff00aa"),
)
```

Volume starts on and the other seven start off, which is the state the figure shows. Sling and BBull paint a placeholder shape rather than the indicator, and each says as much in its own tooltip. Issue #430 carries a tooltip on this toolbar that names a colour the chart does not paint.

The issue #430 clause above is overtaken. It is not edited and not removed. It read:

> *Issue #430 carries a tooltip on this toolbar that names a colour the chart does not paint.*

Issue #430 closed on 2026-09-09. No tooltip on this toolbar names a colour. The Vortex entry names its two lines, its period and its pane and stops there.

```
src/gui/native_chart.py
    tooltip="Vortex Indicator (VI+ / VI-, period 14, sub-pane)"

tooltips declared in that module   14
tooltips naming a colour            0
```

The legend at the right names the two position markers, Invisible and On Book, and the feed label closes the row.

One chart, chosen with the arrows

The tab now draws one chart at a time. A left arrow, a centre ticker list and a right arrow choose which traded asset it shows, and a readout beside them says which of how many is on screen. The list wraps, so neither arrow dead-ends, and the ticker readout is itself a drop-down. With no bots running the readout says zero of zero and both arrows are switched off.

`src/gui/widgets/trade_charts_tab.py` — the readout beside the arrows

```
POSITION_FORMAT = "{at} of {total}"
EMPTY_TICKER_TEXT = "No asset"
EMPTY_POSITION_TEXT = "0 of 0"
```

The toggles, and the two that start off

Every indicator the chart can draw now starts switched on, and a check box for each sits along the bottom of the panel. Two start off, because each paints a filled shape over the price where the Bollinger bands and the Ichimoku cloud are drawn. Slingshot paints solid marks at a squeeze release and a snapback. BB Bullseye paints four shaded envelopes directly on the bands themselves.

`src/gui/native_chart.py` — one entry per drawn indicator

```
class ChartOverlay:
    key: str
    label: str
    colour_field: str
    pane: str
    occludes: bool
    draw: str
    tooltip: str
```

The renderer is built to grow

`CHART_OVERLAYS` declares each indicator once, and the paint routine walks that list rather than naming any of them. Another overlay is one entry in the list and one draw method; no existing overlay is touched to add it.

The renderer holds no colour of its own. Every painted value resolves through `PALETTE_ROLES`, which names the theme field and the transparency each role reads, so a theme sets the look and the renderer does not.

2026-09-08 23:40 - #128 - the chart registers a panel in the shell

The chart now joins the Electron shell's roster of panels. Before this change the shell listed fifteen panels and said of this one that it registered nothing to draw. It now lists sixteen and says nothing.

`src/gui/web/native_chart.js` — the chart joins the roster

```
  if (global.acervatorPanelHost) {
    global.acervatorPanelHost.register({
      render: renderChart,
      load: loadChart,
      loadError: loadError
    });
  }
```

The chart takes no tab of its own. It names no bridge method the application serves a tab from, so the tab bar passes over it and the chart draws where it belongs, inside the Charts tab. The ten tabs the application reports are the same ten before and after.

The chart in the shell still draws its waiting line rather than a price. The Charts tab asks it for a symbol and a timeframe and sends none of the 180 candles the tab itself holds, and the symbol it sends is empty. Both values belong to the Charts tab file, not to the chart, so the row below keeps `no` under Registers in Electron until they are carried across.

2026-09-09 01:20 - #128 - the Charts tab hands the chart its data

The chart in the shell draws the price now. The tab passes it the asset it is following, the feed name, the candles it is holding and the size of the space the chart has, and the shell's panel host does the drawing rather than the tab reaching past it.

`src/gui/web/trade_charts_tab.js` — what the chart slot asks for

```
          return global.acervator.call(api.method, {
            reset: true,
            symbol: symbol,
            timeframe: mount.getAttribute(CHART_TF_ATTR),
            source: text(panel[SOURCE]),
            candles: candles,
            width: mount.clientWidth,
            height: mount.clientHeight
          });
```

The panel carries the asset it follows. It was built once with a blank name and renamed only its header line, so the name the chart was asked for was always blank.

`src/gui/main_tabs/trade_charts_tab_surface.py` — the panel's own asset

```
    def set_symbol_property(self, symbol: Any) -> None:
        """Rename the chart through its public setter, which repaints."""
        self.symbol = symbol
        self.label = symbol
```

Both builds now draw the same header, the same 180 candles, the same last price of 78,623.00, the same three axis prices and the same time axis. The two pictures still differ on the indicator lines. The Bollinger bands, the Ichimoku cloud and the Vortex, MACD and Stochastic RSI panes draw in the Qt build and in neither place in the shell, because nothing hands the chart those five readings yet. The eight indicator switches under the Qt chart are absent in the shell for the same reason. The row below keeps `no` under Registers in Electron until the readings arrive.

2026-09-09 03:05 - #128 - the indicators draw in both builds

The chart in the shell draws every indicator the Qt chart draws. The five readings now reach it: the Bollinger bands with their cloud, the Ichimoku cloud with its four lines and the lagging line, and the Vortex, MACD and Stochastic RSI panes under the price. The eight switches sit under the chart with Slingshot and BB Bullseye off, and the two position markers sit on the toolbar beside the feed name.

Each reading is the one the indicator itself published. The chart asks the five indicator classes for their values and paints what they answer.

`src/gui/main_tabs/native_chart_surface.py` — the chart takes its readings

```
        self.set_indicator_series(
            bb=BollingerBands(BOLLINGER_PERIOD, BOLLINGER_STD).bands(candles),
```

Both builds were driven from the same 38 running bots and the same 180 daily A15/USD candles. Both draw the same header, the same candles, the same last price of 78,623.00, the same four axis prices, the same time axis, and the same three panes reading Vortex 0.8761, MACD -245.82 and Stoch RSI 0.2318. The row below now reads `yes` under Registers in Electron.

Two figures moved to make that true. The Qt chart asked for 492 pixels and was given 250 until an indicator switch was pressed, so its MACD pane, its Stochastic RSI pane and its time axis were cut off; it now takes its full height as soon as the candles arrive. The panes were also listed as MACD, Vortex, Stochastic RSI in one

place and Vortex, MACD, Stochastic RSI in the other; the switch order decides, so both now read Vortex, MACD, Stochastic RSI.

The volume bars are empty in both pictures. The public feed serves no volume with its daily candles, so that one row of the comparison rests on two empty strips.

`src/gui/theme_engine.py` — the chart tokens each theme carries

```
chart_bg_top: str = "#08080e"
chart_up: str = "#00e5a0"
chart_down: str = "#ff2d6f"
chart_zone_fold: str = "#fcee0a"
```

Detail: 08-tabs/asset-charts.md.

Live Tab

The Live tab is where the bots trade real money on a real exchange. The exchange connections fill the left, the Indicator Voting Panel fills the right, and the Activity Log and the API Interaction Log close the foot.

The tab is now called Live. It sits third on the bar, straight after Paper, on the gold ground with red text.

2026-09-09 06:35 - #128 - the Live tab's two widgets register a panel

The Indicator Voting Panel and the Activity Log now register with the shell's panel host, and the Live tab draws both through it. Before the change the shell listed sixteen registered panels and named these two as modules that draw nothing; it lists eighteen now and names neither.

`src/gui/web/status_log.js` — what each of the two modules adds

```
  if (global.acervatorPanelHost) {
    global.acervatorPanelHost.register({
      render: renderLog,
      load: loadLog,
      loadError: loadError
    });
  }
```

Neither registration names a bridge method, so the tab bar gives neither a tab of its own. The Live tab keeps a slot for each and hands the slot to the host, which is how the Charts tab already draws the price chart.

`src/gui/web/trading_tab.js` — the Live tab's slot for the Activity Log

```
    slot.setAttribute(CHILD_ATTR, STATUS_LOG_MODULE);
    return host.mount(STATUS_LOG_MODULE, slot, model) ? slot : null;
```

The host is the draw path now, not a second name for it. With both registrations taken out and the same reading driven in, each slot fell from 43,264 and 4,532 characters of markup to 82 and 72, and each carried the sentence naming the panel that did not draw.

Both builds were driven with one reading: 180 daily A15/USD candles from CoinGecko through the voting engine, at 15m, 1h and 1d, and eight Activity Log messages covering a placed, a filled and a sent trade, a wire

flow, a wire stack, a warning and an error. The Qt tab was built at the size the shell gives the React tab, 1386 by 708.

The twelve indicator columns carry the same name, direction and bar in both. Each bar fills its share of a 70-pixel plot.

column	direction	confidence	Qt height	React height
BB	NEUTRAL	0.000	2.00	2.00
VTX	BEARISH	0.328	22.96	22.69
MACD	BEARISH	0.150	10.50	10.38
SRsi	BEARISH	0.200	14.00	13.84
Ichi	BEARISH	0.210	14.70	14.53
Vol	NEUTRAL	0.000	2.00	2.00
Sling	NEUTRAL	0.000	2.00	2.00
ADX	BEARISH	0.267	18.69	18.48
STrd	BULLISH	0.261	18.27	18.05
ZSc	NEUTRAL	0.000	2.00	2.00
KER	NEUTRAL	0.000	2.00	2.00
RSI	NEUTRAL	0.000	2.00	2.00

Net, Comp and Conf stand in the last three of the ten columns in both, take the same three directions, and run from the ceiling of the top row of bars to the floor of the bottom row. Only their bases carry a label, in a strip 22 pixels deep under that floor.

No horizontal line crosses the three. Every element in the React panel was read for a top border, a bottom border or a background gradient: ninety-three carry one, and every box ends at or before 1185.93, where the Net column starts. The 1 pixel each indicator column carries is that column's own base, which Qt paints as a line from the left margin to the x of the Net column. The dotted increments sit on those same cells and on no other.

	Qt	React
ceiling	171	267.2
floor	428	523.2
label strip	428 to 450	523.2 to 545.2
columns	8, 9, 10 of 10	8, 9, 10 of 10
rules end at	437, the Net column	1185.9, the Net column

The Activity Log draws the same eight lines in the same order with the same text. Each line takes the colour of its kind, and a trade line is raised to 14 pixels and bold.

line	colour	size	weight
Bot a15_core started	rgb(0,255,204)	12	normal
SCRUM PLACED	rgb(255,170,0)	14	bold
SCRUM FILLED	rgb(0,255,136)	14	bold
WIRE FLOW	rgb(255,102,221)	12	bold
WIRE STACK	rgb(255,204,68)	12	bold
Coinbase rate limit	rgb(255,170,0)	12	normal
Reconciliation refused	rgb(255,51,102)	12	normal
FOLD SENT	rgb(0,255,204)	14	bold

Each mini-panel table shows its header and two rows in both builds, and both cut the 1d row off the bottom. Qt fixes the table to that height and turns both scrollbars off; the React table read neither of the two numbers the payload publishes for it, stood two rows taller, and pushed the three pillar labels and the six lower bar labels below the panel. It reads them now.

`src/gui/web/indicator_panel.js` — the height the table body takes

```
    var bodyStyle = {
      display: BLOCK,
      maxHeight: length(slackHeight(table)),
      overflow: HIDDEN
    };
```

One line is drawn in one picture and not the other. The panel's locks line, `No active timeframe locks`, sits inside the Qt panel and two pixels below the bottom of the React one, which reaches it by scrolling. The Live tab's own splitter is what differs: Qt gives its top section 472 pixels of the 708 the tab has, the React tab gives it 411, and the panel cannot draw shorter than 415 because each bar graph carries a 100-pixel minimum in both.

Detail: 06-trading-tab.md. Detail: 07-indicators.md.

Converting the Interface to React

Every screen in this application is drawn by Qt. Issue #128 replaces them with React and keeps the backend in Python. This section is the running record of that work. One row per screen, one column per step, and a mark only where the step is finished.

One screen draws React in the application today: the History table. It runs inside the Chromium that the Qt toolkit already ships, not inside Electron.

`src/gui/react_history_panel.py` — `HistoryWebTable._setup_ui`

```
self._web = QWebEngineView()
self._web.loadFinished.connect(self._on_load_finished)
self._web.setHtml(panel_html(theme))
layout.addWidget(self._web, 1)
```

What the tree holds

Measured on 5 September 2026 over the tracked files.

```
React modules             70 files, 80,292 lines, 0 stub markers
                          65 of them call React.createElement
                          React and ReactDOM are vendored on disk
View-model modules        74 files, 74,207 lines
Parity tests              72 files, 170,244 lines
Electron shell             8 code files, 791 lines, plus a package manifest
Renderer manifest         70 names
Qt modules under src/gui  69, every one importing PySide6
Tabs drawn by React        0
```

No file in this repository has ever carried a JavaScript extension other than plain `.js`. A history search over every branch returns no commit that added, deleted or renamed one, and the same search for the plain extension returns 81 paths, so the search does find a file that existed.

```
git log --all --diff-filter=ADR --name-only -- "*.jsx" "*.tsx" "*.ts"   no commits
git log --all --diff-filter=ADR --name-only -- "*.js"                   81 paths
```

What the Electron shell drew

The shell has been started once, and what it drew is measured. All 70 renderer modules loaded and none failed. Two of them register a panel for the page to draw, so the page had nothing to ask the other 68 for. The window showed an empty History table, with no tab bar and no navigation.

```
React modules the page list names   70
modules that failed to load          0
modules that register a panel        2
panels the page asked to draw        2
panels that failed to draw           0
```

The two that register are the Console tab panel and the header strip. The full per-file record is in the 2026-09-05 file-verification audit, removed with the audit folders.

The tab bar the shell draws now

The shell builds the same ten tabs the application builds, with the same labels and in the same order. It holds no list of its own. `main_window.state` reports `tab_labels` and `tab_methods`; the bar keeps one tab per label, filled by the registered panel that calls that label's bridge method.

```
Sim  Paper  Live  Charts  Inspector
Swarm  Accumulation  History  Status  Console
```

A module that draws a screen inside a dialog registers no bridge method and takes no tab. `market_inspector_tab.js` and `bot_swarm_tab.js` are the two: both belong to the Live Bot Settings dialog.

Measured in one Electron launch, with the backend serving a two-bot fleet:

```
tabs on the bar                    10
panels that drew text              10
panels that failed to draw          0
child panels drawn inside a tab     7
```

The header strip stays above the bar for every tab.

How to read the table

Each row is a Qt module the running window builds. The columns are the steps of the conversion, in the order the work happens.

Column	What the mark means
React module	A renderer module in `src/gui/web` serves this screen
Surface	A Python view model of the screen exists, with no toolkit in it
Bridge	The view model is registered as a method the frontend may call
Manifest	The renderer module is named in the shell's module list
Ships in the build	The built React application carries the module inside it
RENDERS	What the operator sees when he opens that screen
Scope	Whether the row is work this item must do

The last column is the item. A row is finished only when it reads `yes`. Every other column can be marked while the operator sees no change, and that is the difference between wiring written and a screen replaced.

The RENDERS column takes three values, and none of them is derived from the bridge or the manifest. `yes` means React draws where Qt used to. `shell` means the module draws in the Electron shell and nowhere else. A dash means the Qt widget is still the screen.

The Scope column names the module that builds a screen the item has still to convert. Header strip and window chrome are the parts of the Main Window that sit outside the tab row.

The table carries nine columns and no tab column. The screen a module draws is named in that screen's own section above. Header strip and window chrome are the parts of the Main Window that sit outside the tab row.

The Scope column takes five values.

Value	What it means
in scope	The running window builds this screen, and the cell names what builds it
React side	The file exists because of this conversion, so a React module for it would be a module for a module
not a screen	The module defines no view
builds the shell	The module fills the tab bar rather than sitting in it
shelved	Nothing the window builds reaches it

The walk that answers the column starts at the main window. It follows an imported name only where the module calls that name, subclasses it or reads an attribute of it, so a re-export block reaches nothing. A row is in scope when some module on that walk builds one of the classes the row defines.

`tools/conversion_scope.py` writes the column, so the answers are re-measured rather than typed.

The conversion table

Qt file	React module	Uses React	Bridge	Manifest	Registers in Electron	Ships in the build	RENDERS	Scope
splash_screen.py	no	-	yes	no	no	-	no	shelved
src/gui/alerts_tab.py	alerts_tab.js	yes	yes	yes	-	yes	yes	in scope
src/gui/analytics_tab.py	analytics_tab.js	yes	yes	yes	no	yes	no	shelved
src/gui/audio_suite.py	audio_suite.js	yes	yes	yes	no	yes	no	shelved
src/gui/bot_live_settings.py	bot_live_settings.js	yes	yes	yes	yes	yes	yes	in scope
bot_swarm_list	removed	-	-	-	-	-	-	deleted
src/gui/bot_visualizer.py	bot_visualizer.js	yes	yes	yes	yes	yes	yes	in scope
src/gui/bot_wizard.py	bot_wizard.js	yes	yes	yes	-	yes	yes	in scope
src/gui/buy_confirmation_dialog.py	buy_confirmation.js	yes	yes	yes	-	yes	yes	in scope
src/gui/competition_tab.py	competition_tab.js	yes	yes	yes	no	yes	no	shelved
src/gui/crypto_news_ticker.py	crypto_news_ticker.js	yes	yes	yes	-	yes	shell	in scope
src/gui/history_qt_table.py	no	-	no	no	no	-	no	React side
src/gui/history_tab.py	history_tab.js	yes	yes	yes	yes	yes	yes	in scope
src/gui/indicator_panel.py	indicator_panel.js	yes	yes	yes	yes	yes	shell	in scope
src/gui/init_wizard.py	init_wizard.js	yes	yes	yes	no	yes	no	shelved
src/gui/instance_consent_dialog.py	instance_consent_dialog.js	yes	no	yes	no	yes	no	shelved
src/gui/journal_tab.py	journal_tab.js	yes	yes	yes	no	yes	no	shelved
src/gui/launcher.py	launcher.js	yes	yes	yes	no	yes	no	shelved
src/gui/live_bot_window.py	no	-	yes	no	no	-	no	shelved
src/gui/live_settings/bot_swarm_tab.py	bot_swarm_tab.js	yes	yes	yes	yes	yes	yes	in scope

Qt file	React module	Uses React	Bridge	Manifest	Registers in Electron	Ships in the build	RENDERS	Scope
src/gui/live_settings/fold_chrome.py	fold_chrome.js	yes	yes	yes	-	yes	yes	in scope
src/gui/live_settings/fold_tokens.py	fold_tokens.js	yes	yes	yes	no	yes	no	not a screen
src/gui/live_settings/fold_tranches_tab.py	fold_tranches_tab.js	yes	yes	yes	-	yes	yes	in scope
src/gui/live_settings/market_inspector_tab.py	market_inspector_tab.js	yes	yes	yes	yes	yes	yes	in scope
src/gui/live_settings/phantom_bots_tab.py	phantom_bots_tab.js	yes	yes	yes	-	yes	yes	in scope
src/gui/live_settings/positions_held_tab.py	no	-	no	no	no	-	-	in scope
src/gui/live_settings/settings_tab.py	live_settings_tab.js	yes	yes	yes	no	yes	yes	in scope
src/gui/live_settings/stack_tranches_tab.py	stack_tranches_tab.js	yes	yes	yes	-	yes	yes	in scope
src/gui/live_settings/status_tab.py	no	-	yes	no	no	-	yes	in scope
src/gui/main_tabs/audio_suite_surface.py	no	-	yes	no	no	-	no	React side
src/gui/main_tabs/buy_confirmation_surface.py	no	-	yes	no	no	-	no	React side
src/gui/main_tabs/console_log_handler.py	no	-	no	no	yes	-	-	in scope
src/gui/main_tabs/console_tab.py	console_tab.js	yes	yes	yes	yes	yes	yes	in scope
src/gui/main_tabs/empty_tabs.py	no	-	no	no	no	-	-	shelved
src/gui/main_tabs/header_strip.py	header_strip.js	yes	yes	yes	yes	yes	yes	in scope
src/gui/main_tabs/proof_of_accumulation_tab.py	proof_of_accumulation_tab.js	yes	yes	yes	yes	no	shell	in scope
src/gui/main_tabs/stock_main_window_surface.py	no	-	yes	no	no	-	no	React side
src/gui/main_tabs/system_status_tab.py	system_status_tab.js	yes	yes	yes	yes	no	yes	in scope
src/gui/main_tabs/trading_tab.py	trading_tab.js	yes	yes	yes	yes	yes	-	in scope
src/gui/main_tabs/tradingview_chart_surface.py	no	-	yes	no	no	-	no	React side
src/gui/main_window.py	main_window.js	yes	yes	yes	no	no	yes	in scope

Qt file	React module	Uses React	Bridge	Manifest	Registers in Electron	Ships in the build	RENDERS	Scope
src/gui/market_inspector.py	market_inspector.js	yes	yes	yes	yes	yes	yes	in scope
src/gui/market_inspector_topologies.py	market_inspector_topologies.js	yes	yes	yes	yes	yes	yes	in scope
src/gui/native_chart.py	native_chart.js	yes	yes	yes	yes	yes	yes	in scope
PaperTraderTabQt	paper_trader_tab.js	yes	yes	yes	yes	no	yes	in scope
src/gui/qt_safe_events.py	no	-	yes	no	no	-	no	not a screen
src/gui/react_history_panel.py	no	-	yes	no	no	-	no	React side
src/gui/react_history_tab.py	no	-	no	no	no	-	no	React side
react_paper_trader_tab	no	-	no	no	no	-	no	React side
src/gui/react_proof_of_accumulation_tab.py	no	-	no	no	no	-	no	React side
src/gui/react_simulator_tab.py	no	-	no	no	no	-	no	React side
src/gui/react_system_status_tab.py	no	-	no	no	no	-	no	React side
src/gui/settings_dialog.py	settings_dialog.js	yes	yes	yes	-	yes	yes	in scope
src/gui/shared_testnet.py	shared_testnet.js	no	yes	yes	no	yes	no	shelved
src/gui/simulator_tab.py	simulator_tab.js	yes	yes	yes	yes	yes	yes	in scope
fleet_replay_panel	fleet_replay_panel.js	yes	yes	yes	no	yes	yes	shelved
sim_visuals	sim_visuals.js	yes	yes	yes	no	yes	yes	shelved
nuclear_mode_panel	nuclear_mode_panel.js	yes	yes	yes	no	yes	yes	shelved
sim_stat_strip	sim_stat_strip.js	yes	yes	yes	no	yes	yes	shelved
SimulatorTab	simulator_tab.js	yes	yes	yes	yes	yes	yes	shelved
src/gui/start_all_progress_dialog.py	start_all_progress.js	yes	yes	yes	-	yes	yes	in scope
src/gui/stock_main_window.py	no	-	yes	no	no	-	no	shelved

Qt file	React module	Uses React	Bridge	Manifest	Registers in Electron	Ships in the build	RENDERS	Scope
src/gui/testnet_tab.py	testnet_tab.js	yes	yes	yes	no	yes	no	shelved
src/gui/tradingview_chart.py	tradingview_chart.js	yes	yes	yes	no	yes	no	shelved
src/gui/usb_auth_widget.py	usb_auth_widget.js	yes	yes	yes	no	yes	no	shelved
src/gui/visualizer/bot_node.py	bot_node.js	yes	yes	yes	yes	yes	yes	in scope
src/gui/visualizer/quick_routing.py	quick_routing.js	yes	yes	yes	yes	yes	yes	in scope
src/gui/visualizer/themes.py	visualizer_themes.js	-	yes	yes	yes	yes	yes	in scope
src/gui/visualizer/wire_canvas.py	wire_canvas.js	yes	yes	yes	yes	yes	yes	in scope
src/gui/widgets/__init__.py	no	-	yes	no	no	-	no	not a screen
src/gui/widgets/api_tester_tab.py	api_tester_tab.js	yes	yes	yes	no	yes	no	shelved
src/gui/widgets/bot_selection.py	bot_selection.js	yes	yes	yes	no	yes	no	not a screen
src/gui/widgets/bot_status_table.py	bot_status_table.js	yes	yes	yes	-	yes	yes	in scope
capital_registry_panel	no	-	no	no	no	-	no	shelved
src/gui/widgets/dashboard_stat_card.py	dashboard_stat_card.js	yes	yes	yes	-	yes	yes	in scope
src/gui/widgets/exchange_tab.py	exchange_tab.js	yes	yes	yes	no	yes	-	in scope
src/gui/widgets/extractor_bot_table.py	extractor_bot_table.js	yes	yes	yes	no	yes	yes	in scope
src/gui/widgets/notification_spool.py	notification_spool.js	yes	yes	yes	no	yes	no	shelved
src/gui/widgets/privacy_dot.py	privacy_dot.js	yes	yes	yes	-	yes	yes	in scope
src/gui/widgets/pulse_manager.py	pulse_manager.js	yes	yes	yes	no	yes	no	shelved
src/gui/widgets/spendable_profits.py	spendable_profits.js	yes	yes	yes	yes	yes	yes	in scope
src/gui/widgets/status_log.py	status_log.js	yes	yes	yes	yes	yes	shell	in scope

Qt file	React module	Uses React	Bridge	Manifest	Registers in Electron	Ships in the build	RENDERS	Scope
`src/gui/widgets/trade_charts_tab.py`	`trade_charts_tab.js`	yes	yes	yes	yes	yes	yes	in scope

The Simulator rebuild removed the files above; they are not in the tree.

The Capital Registry row names a file the tree no longer holds. The registry it drew was built nowhere, so the table could draw no row. Four modules went together: `capital_registry_panel`, its view model, its renderer page, and the registry behind them. The bridge no longer offers the method, and the renderer manifest no longer names the page.

Totals across the 76 rows above, measured on 5 September 2026:

```
React module            63
Uses React              60
Bridge                  72
Manifest                62
Registers in Electron   25
Ships in the build      58
RENDERS                 44
RENDERS, in scope       35 of 43
out of scope            41
```

Four columns are all but complete. RENDERS, the column that is the item, is not.

Re-measured on 6 September 2026, counted off the 76 rows above:

```
React module            55
Uses React              53
Bridge                  67
Manifest                55
Registers in Electron   15
Ships in the build      55
RENDERS                 29
```

Only the last column moved. The Settings row already carried a React module, a bridge method and a manifest entry; what it gained is the screen.

Re-measured later on 6 September 2026, counted off the same 76 rows:

```
React module            55
Uses React              53
Bridge                  67
Manifest                55
Registers in Electron   15
Ships in the build      55
RENDERS                 30
RENDERS, in scope       30 of 43
out of scope            33
```

The Create Auto Trader wizard is the row that moved. `bot_wizard.js` draws it into a space the React exchange screen keeps once + New Bot is pressed, so the wizard is a screen and no longer a module with nowhere to draw.

Counted once more the same day, after the Live Bot Settings window went over:

```
React module            55
Uses React              53
Bridge                  67
Manifest                55
Registers in Electron   15
Ships in the build      55
RENDERS                 38
RENDERS, in scope       38 of 43
out of scope            33
```

Eight rows moved together: the window a bot row opens, six of the files its tabs are built from, and the delegate that draws the fold-tranche row borders. `fold_tokens.py` keeps its dash. Its colours are on the drawn rows, and the file names no screen of its own.

When that count was taken the Paper Trader, Proof of Accumulation and System Status tabs had no Qt module to replace and no React module to replace it with, so the table carried no row for them. `empty_tabs.py` drew all three empty, and its own row read `builds the shell`. Issue #19 has since built the Paper tab, which now has a Qt module and a React module of its own.

In development.

That is no longer the state. Each of the three has a renderer module in `src/gui/web`, a bridge method and a manifest entry, and `empty_tabs.py` carries the row for all three. Its Scope cell reads `in scope` and its RENDERS cell reads `yes`. The section below says what each tab draws.

Counted once more on 6 September 2026, after the header strip and the Start All dialog went over:

```
React module            58
Uses React              56
Bridge                  69
Manifest                58
Registers in Electron   15
Ships in the build      58
RENDERS                 42
RENDERS, in scope       42 of 43
out of scope            33
```

Six rows moved. Three are the header strip parts. One is the Start All progress dialog. One is the Live Bot Settings screen tab, where the React module cell was the wrong one and now names the module that draws it. The last is the buy confirmation dialog, which gained a module, a bridge method and a build entry and still reads `no` under RENDERS.

Counted again on 6 September 2026, after the buy confirmation dialog went over:

```
React module             58
Uses React               56
Bridge                   69
Manifest                 58
Registers in Electron    15
Ships in the build       58
RENDERS                  43
RENDERS, in scope        43 of 43
out of scope             33
```

One row moved. It is the buy confirmation dialog, and it was the last in-scope row reading `no`. The count comes from `tools/conversion_table.py`, which reads the cells of the table above.

Counted again on 6 September 2026, after the window's own tab bar went over:

```
React module             59
Uses React               57
Bridge                   69
Manifest                 59
Registers in Electron    15
Ships in the build       58
RENDERS                  44
RENDERS, in scope        44 of 44
out of scope             32
```

One row moved, and it is `main_window.py`. The row read `builds the shell` because the module filled the tab bar rather than sitting in it; the bar it fills is now React, so the row is in scope and it renders. `Ships in the build` stays `no`: the newest bundle under `dist` carries 70 renderer modules and `main_window.js` is not one of them, because the module is newer than that build.

Counted again on 6 September 2026, after the three unbuilt tabs went over:

```
React module             60
Uses React               58
Bridge                   70
Manifest                 60
Registers in Electron    16
Ships in the build       58
RENDERS                  45
RENDERS, in scope        45 of 45
out of scope             31
```

One row moved, and it is `empty_tabs.py`. It is the second row that read `builds the shell`, and it is the only row naming three React modules: the Paper Trader, System Status and Proof of Accumulation skeletons each have a renderer module, a bridge method and a manifest entry, and all three register a panel with the Electron shell. `Ships in the build` reads `no` for the same reason as the row above: the newest bundle under `dist` carries 70 renderer modules and these three are not among them.

No row now reads `builds the shell`.

Counted again on 6 September 2026, after the Notifications and Alerts tab went over:

```
React module             60
Uses React               58
Bridge                   70
Manifest                 60
Registers in Electron    16
Ships in the build       58
RENDERS                  46
RENDERS, in scope        45 of 45
out of scope             31
```

One row moved, and it is `alerts_tab.py`. Its Scope cell still reads `shelved`, because nothing the running window builds reaches the screen. `variant_surface` now holds the row under the name `ALERTS`, so the React build makes `AlertsReactTab` and the Qt build makes `AlertsTab` unchanged. The React tab draws the whole screen in one web view from `src/gui/web/alerts_tab.js`, and `AlertsTab.refresh`, `AlertsTab._save_config`, `AlertsTab._test_telegram` and `AlertsTab._acknowledge_all` are the same methods on both sides.

2026-09-09 10:31 - #128 - the Live Bot Settings window fills its tab pages

The window a bot row opens already registered with the shell's panel host. Its cell read `no`, and the running shell named it among nineteen registered panels, so the cell was stale.

```
registered  bot_live_settings bot_swarm_tab bot_visualizer console_tab
            header_strip history_tab indicator_panel market_inspector
            market_inspector_tab market_inspector_topologies native_chart
            paper_trader_tab proof_of_accumulation_tab simulator_tab
            spendable_profits status_log system_status_tab trade_charts_tab
            trading_tab
```

Registering was never the whole of it. The window drew its title, its tab strip and its two footer buttons, and every one of its seven tab pages was an empty box. Read off the drawn page, the window held 28 elements and each page held none.

```
part                   children  characters
status-page                   0           0
settings-page                 0           0
fold-tranches-page            0           0
stack-tranches-page           0           0
bot-swarm-page                0           0
market-inspector-page         0           0
phantom-bots-page             0           0
```

Each tab module already carried a filler that looks for the empty space this window leaves. Nothing called it. The window now asks each module's own loader for its view model and hands it the page.

```
    var wait =
      typeof loader === "function"
        ? loader(tabRequest(model))
        : Promise.resolve(null);
    return Promise.resolve(wait).then(function (found) {
      return api.fill(space, found) === null ? null : space;
    });
```

A tab is asked for the bot the window built, never for one of its own. The window's answer now names that bot, and the request carries it under the name the window's own surface publishes.

```
    config = model.bot.config
    return {
        "bot_id": model.bot.bot_id,
        "symbol": config.symbol,
        "mode": config.mode,
        "state": model.bot.state,
    }
```

Three pages fill from it. Fold Tranches draws eleven health rows, three Clear buttons, an order picker of five choices and a filter box. Stack Tranches draws seven summary rows and two Clear buttons. Phantom Bots draws four summary rows, twelve timeframe boxes and the candles-to-lock number. Take the bot out of the window's answer and every one of those rows goes.

```
page                 rows  order choices  elements   with the bot removed
fold-tranches-page     11              5        69    0,  5, 33
stack-tranches-page     7              0        33    0,  0, 12
phantom-bots-page       4              0        92    0,  0, 46
```

The order picker survives the removal because the fold chrome is asked for its own controls rather than for a bot, which is what proves the row counter was reading the page and not returning zero everywhere.

Every page drew at once before this. A page not on show was marked hidden and still carried an inline display, which beats the browser's own rule for a hidden element. The page not on show now takes `display: none`, so one tab is on screen at a time.

```
    var style = {
      display: props.current === true ? FLEX : DISPLAY_NONE,
```

The window's own stylesheet had never reached the shell page, so its tab buttons painted white on grey. The shell now links it. Its rules were written for a page holding this window alone, and the shell page holds every panel: 17 of the 20 tables on that page belong to other panels, as do 5 of the 12 tab buttons. Each rule is scoped to the window before the link, and five of six other panels render byte-identical to the run before it.

```
[data-part="bot-live-settings"] table {
  border-collapse: collapse;
  width: 100%;
}
```

The Settings page stays empty and its row stays `no`. Its surface builds from the bot's own settings values, and it stops on the first one the window cannot supply.

```
AttributeError: 'BotConfigSource' object has no attribute 'visibility'
```

The window answers a stand-in bot, not a running one. Giving it a running one would also point Apply Changes at that bot, which is a change to what a control writes and is not this unit's to make.

The exchange screen and the Extractor table are drawn inside the Live tab, not by the shell. Both rows stay `no`: the Live tab names an exchange only when the shell's backend is the trading program itself, and the backend reachable here names none, so neither child is built.

```
    slot.setAttribute(CHILD_ATTR, EXCHANGE_MODULE);
    return api.renderTab(slot, shown.model);
```

The Qt window could not be built beside the React one. It reads a mode and a state that carry a `value`, then dozens of settings fields with no fallback, so it needs a bot the running program holds.

```
AttributeError: 'str' object has no attribute 'value'
```

The window's own tab bar

The row of tab names across the top of the window is drawn by React under the React build. `MainWindow._setup_ui` no longer builds a `QTabWidget` by name. It asks the variant seam for the tab book, so one builder gives the operator the Qt bar he runs today or the React one.

```
            self._main_tabs = _main_tab_book_class()()
            self._main_tabs.setMovable(True)
```

Under the Qt build the seam answers with the widget the operator runs today and nothing changes. Under the React build it answers with a book that keeps that widget as the stack of pages, hides its own bar, and puts a web view above it. Every call the window makes is answered over the widget behind the bar, so the tab order and the pages are the same objects on both sides.

```
        def addTab(self, widget, label: str) -> int:
            """Add ``widget`` under ``label`` and redraw the bar."""
            index = self._book.addTab(widget, label)
            self._push_tabs()
            return index
```

The bar itself is the Electron shell's navigation, run inside the window. The page is built from the shell's own scripts, read off disk, and the buttons are drawn by one React module both sides reach. Neither host holds a bar of its own, so the two cannot draw a different one.

```
#: The shell's own navigation, read from ``desktop/renderer`` and inlined.
SHELL_SCRIPTS: tuple[str, ...] = ("panel_host.js", "tab_bar.js")
```

The shell's navigation asks that React module for the buttons and passes it the tab names, the selected one, and what to do with a press and a drop. It holds no button of its own.

```
drawn.renderTabBar(bar, {
  names: found,
  selected: current,
  onSelect: select,
  onMove: onMove
});
```

A press on a button is reported back to Python, which moves the real tab book to that tab. Dragging a tab onto another reports the two slots the same way, and the same handler hands them to the hidden bar, so a drag reorders the pages as it did before.

```
            move = request.get("move") or {}
            if "from" in move and "to" in move:
                self._book.tabBar().moveTab(int(move["from"]), int(move["to"]))
```

The menu bar, the window title, the window icon and the status bar are still Qt. `MainWindow._setup_menu` builds the four menus and `_setup_status_bar` builds the status line; neither was changed, and every menu action the window offered before it is on it now.

The three tabs with nothing behind them yet

Paper Trader, System Status and Proof of Accumulation were named on the tab bar and were not built when that count was taken. Each one drew a heading, a sentence saying the tab is not built, and the issue that carries the build-out. Every word of it came from that tab's own Python surface, so the page invented nothing. Two of the three are still empty; issue #19 built the Paper tab.

```
return {
    "accessible_name": HEADING,
    "built": BUILT,
    "heading": HEADING,
    "issue": ISSUE,
    "issue_text": ISSUE_TEXT,
    "method": METHOD,
    "state_text": STATE_TEXT,
}
```

Under the Qt build the three sentences are Qt labels, as before. Under the React build the same view model goes to the tab's renderer module, which draws it in a web view. The builder asks the variant seam which of the two to make, and it hands both the same values.

```
        model = surface.view_model({})
        panel = _empty_tab_class()(model)
```

Neither side holds a colour or a size of its own. `SKIN` names the ground, the two text colours, the two text sizes, the padding and the gap; the Qt style sheets are built from it and the page reads it as custom properties.

```
SKIN = {
    "--empty-tab-ground": ds.SURFACE_0,
    "--empty-tab-heading-colour": ds.TEXT_MAX,
    "--empty-tab-body-colour": ds.TEXT_EMPTY_STATE,
```

The page runs the Electron shell's `panel_host.js`, so the module is drawn the same way the shell draws it: by name, into one host element, with the reason written onto the page when it does not draw.

The header strip

The strip and the five counters above the tab row are no longer built by name. `HeaderStripMixin._build_header_strip` asks the variant seam for each class, so one builder draws the Qt widgets or the React pages.

```
        self._spendable_widget = _spendable_profits_class()()
        top_row.addWidget(self._spendable_widget, stretch=3)

        card_class = _stat_card_class()
        self._stat_scrummed = card_class("Scrummed", "$0.00")
```

The spendable strip is drawn by `SpendableProfitsReact`, which hosts the strip module in one web view. It answers the two calls the main window makes, and writes both into the same view model the Qt strip is described by.

```
        def update_profits(self, data: dict) -> None:
            self._model.update_profits(data)
            self._push()
```

Each column carries a privacy dot drawn by the dot module. A press on one runs the model in `privacy_dot_surface`, which flips the field in the process privacy registry and repaints the amount as four stars.

```
    def clicked(self) -> None:
        self.calls.append(CLICKED)
        try:
            registry = get_privacy_mask_registry()
            registry.set_masked(self._field_id, not registry.is_masked(self._field_id))
        except Exception:
            self.calls.append(TOGGLE_FAILED)
        self.refresh()
```

One card is drawn by `StatCardReact`. A press on the card emits the card's one signal, and only while the header strip has armed it.

The glyph, the tooltip and the skin of a dot now have one home. Both header surfaces return what the dot surface paints, so the card and the strip cannot drift apart.

```
def privacy_dot(field_id: str, masked: Optional[bool] = None) -> dict:
    painted = privacy_dot_surface.dot_view(field_id, masked)
    return {name: painted[name] for name in _DOT_FIELDS}
```

The Start All progress dialog

The dialog the Start All button opens is drawn by `StartAllProgressReactDialog`. It replaces only the method that builds the controls. The progress handling, the Cancel press and the unsubscribe on close are the same code the Qt dialog runs.

```
        def store(self) -> surface.StartAllProgressModel:
            return self._model
```

Every control it needs is a small holder over that store. The list answers `addItem` and `count`; the buttons answer `setEnabled`. Both build sites, in the main window and in the launcher, now ask the variant seam for the class.

The buy confirmation dialog

This is the one in-scope row that still reads `no`, and the reason is reachability rather than missing work.

The main window builds the broker in this file. Nothing calls the broker's request method, which is the only sender of the signal that opens the dialog, so the running window has no path to the screen.

```
        async def request_confirmation(
            self,
            *,
            bot_id: str,
            symbol: str,
            reason: str,
```

Measured by walking every call in `src` and `main.py`: zero call sites, and no import of that name under another. The module, the bridge method and the build entry are all present, so the screen is one caller away.

Re-measured on 6 September 2026, by the same walk over 354 modules: still zero call sites. That count decides what the operator can reach, and it did not move. The row's RENDERS cell moved for a different measurement — the class the broker raises.

The dialog the broker raises is now `BuyConfirmationReactDialog`. The broker asks the variant seam for that class, so the React build opens the page and `ACERVATOR_VARIANT=qt` opens the same Qt widgets as before.

```
def dialog_class() -> type:
    """The confirmation dialog class the running build variant draws."""
    from .variant_surface import BUY_CONFIRMATION, surface_class

    return surface_class(BUY_CONFIRMATION)
```

Every figure the page shows is written in Python. `details_text` in `buy_confirmation_surface` writes the seven detail rows. The Qt label is set from it, and the browser draws the same seven lines sliced into their

values. No arithmetic runs in the page.

```
def after_buy_usd(holdings_before: float, price: float, cost_usd: float) -> float:
    """Dollar value the position reaches once this buy fills."""
    return holdings_usd(holdings_before, price) + cost_usd
```

A press on the page names a button. Python turns that name into an answer and accepts the dialog through the same `_answer` the Qt buttons call. The page places no order and decides nothing.

```
        def run_action(self, name: str) -> bool:
            answer = surface.BUTTON_ANSWERS.get(name)
            if answer is None:
                return False
            self._answer(answer)
            return True
```

A window closed without a press is still no. A request nobody answers inside sixty seconds is refused too.

The gap

Issue #128 says this item replaces the Qt screens with React. Measured against that sentence:

```
screens the running window builds     52
screens that draw React                1
screens that draw in the shell only    2
tabs drawn by React                    0
```

The renderer manifest names 70 modules. The application the operator runs hosts no manifest, and loads exactly one of those modules: the History table's, which one Qt widget puts on a page of its own. The shell that does host the manifest draws two panels and offers no way to reach a third.

One call site carries the whole difference between the React build and the Qt build. `src/gui/history_table_variant.py` holds the only call to the variant resolver in the tree, and every other tab builds the same Qt widget either way.

`src/gui/history_table_variant.py` — `history_table_class`

```
chosen = resolve_variant() if variant is None else variant
if chosen == QT:
    from .history_qt_table import HistoryQtTable

    return HistoryQtTable
from .react_history_panel import HistoryWebTable

return HistoryWebTable
```

Why the count read as near-complete

Two instruments report on this work, and both answer a narrower question than the item asks.

The progress tool marks a screen paired when a bridge method reaches a renderer module the manifest names. A manifest entry declares that the shell would load the file. It does not put a screen in front of the operator.

`tools/conversion_state.py` — `served_methods`, the pairing rule

```
def served_methods(root: pathlib.Path) -> dict[str, str]:
    """Bridge method to the renderer module that speaks it, loaded ones only.

    A method absent from `registered_surfaces` is left out of the result.
    """
    loaded = renderer_modules(root)
```

The parity tests compare a Qt view model against the React view model of the same screen. Both sides are plain Python data and neither has to be displayed. A passing test proves the two descriptions agree with each other. It does not prove either one is on screen.

`parity_pins` reads only a parity test that imports exactly one surface module and exactly one Qt module. It collects the names on both sides and pairs them.

`tools/conversion_state.py` — `parity_pins`, what a pin reads

```
def parity_pins(root: pathlib.Path) -> dict[str, set[str]]:
    qt_by_name = {dotted_name(root, path) for path in qt_modules(root)}
    surfaces = surface_methods(root)
    pins: dict[str, set[str]] = {}
```

Both readings were true. Neither measured the item, and the RENDERS column is the one that does.

Not reached from a live tab

These Qt modules are not built by the live window or by any of its tab builders. They are outside this conversion. One row is a package marker rather than a screen, and it stays where it is.

Qt module	Why it is out of scope
`alerts_tab.py`	Not reachable from a live tab
`analytics_tab.py`	Not reachable from a live tab
`audio_suite.py`	Not reachable from a live tab
`competition_tab.py`	Not reachable from a live tab
`init_wizard.py`	Not reachable from a live tab
`journal_tab.py`	Not reachable from a live tab
`launcher.py`	Not reachable from a live tab
`live_bot_window.py`	Not reachable from a live tab
`stock_main_window.py`	Not reachable from a live tab
`testnet_tab.py`	Not reachable from a live tab
`usb_auth_widget.py`	Not reachable from a live tab
`widgets/__init__.py`	Package marker, no screen

Qt module	Why it is out of scope
`widgets/api_tester_tab.py`	Not reachable from a live tab
`widgets/capital_registry_panel.py`	Not reachable from a live tab
`widgets/notification_spool.py`	Not reachable from a live tab
`widgets/pulse_manager.py`	Not reachable from a live tab

One row in that table names a file the tree no longer holds. The Capital Registry panel is removed with the registry that fed it, so no conversion is owed for it.

The rows in that table are inside this item. The operator's 6 September 2026 directive puts the whole interface in scope, so a screen the window does not build is converted like any other. A converted row keeps `shelved` in the Scope cell of the table above, because that cell answers reachability. Its RENDERS cell reads `yes` once `variant_surface` holds the row and the React class draws the screen.

The reachability walk is rooted at the window and its tab builders, and an edge is a use rather than an import: a call, a base class, or a returned class. An import alone is not an edge, so the re-export block in the main window leaves a widget unreached. Ten screens known to be built and eleven known to be dead were run through the walk as a control, and all twenty-one came back on the expected side.

Detail on each live screen sits one file down, in 08-tabs/README.md.

2026-09-09 11:34 - #128 - the four modal dialogs are opened, not mounted

Four rows read no under Registers in Electron: the bot wizard, the buy confirmation, the Settings dialog and the Start All progress dialog. None of the four is a panel the shell should hold, so all four now take the dash the table already uses for a module that does not register in its own right.

The running shell was asked about each name before anything was changed. All four modules load and none registers a panel.

```
manifest names        69
panels registered     19
bot_wizard            bot_wizard.js registered no panel to draw
buy_confirmation      buy_confirmation.js registered no panel to draw
settings_dialog       settings_dialog.js registered no panel to draw
start_all_progress    start_all_progress.js registered no panel to draw
```

Each was then registered by hand and opened into a host of its own, which is the test that decides the cell. All four draw, so none is an empty shell; what decides them is that nothing in the shell opens them and each one's look belongs to the page its own host builds.

```
name                 elements  characters  where it belongs
bot_wizard                 27         906  a space exchange_tab.js keeps
buy_confirmation           29         254  a window the broker raises
settings_dialog           431        2465  a window the menu raises
start_all_progress         10         390  a window Start All raises
```

The wizard is the clearest of the four. The exchange screen keeps a space for it and fills that space once + New Bot is pressed, so its content reaches the shell through the tab above it.

```
    space.setAttribute(CHILD_ATTR, moduleName);
    return api.fill(space, null, { onClosed: closeBotWizard });
```

The Settings dialog drew all eleven tab pages at once. A page not on show was marked hidden and still carried an inline display, which beats the browser's own rule for a hidden element. Read off the drawn page, 320 parts sat outside the window; with the page not on show taking `none`, none do.

```
      display: props.current === true ? FLEX : DISPLAY_NONE,
```

Every drop-down on that dialog drew one letter per choice. The dialog turns each choice into plain data before it reaches the page, and listing a word gives its letters, so ten carriers drew as ten letters. A choice that is one word now stays that word.

```
before   sms_carrier   A  T  V  S  U  C  B  M  G  O
after    sms_carrier   AT&T  T-Mobile  Verizon  Sprint  US Cellular  Cricket
```

Thirty-eight tick boxes and eight number boxes drew with nothing beside them. The words were already in the payload under `text`, and the units under `suffix`, and neither was drawn. Both are drawn now, so the Sound page reads the same on both builds.

```
part            before   after
row-label           31      31
check-text           0      38
value-prefix         0       2
value-suffix         0       6
```

The Start All progress page did not fit the window it is drawn in. It is sized from the dialog's own width and height and then given padding, and the page carried a margin of its own, so the box measured 548 by 388 inside a window of 520 by 360 and both buttons fell below the edge. A stylesheet of two rules puts the padding and the margin inside that size.

```
before   window 520x360   page 548x388   parts outside the window 10
after    window 520x360   page 520x360   parts outside the window  0
```

The wizard's own walk row is not finished. It draws the four step names as words, where the Qt wizard draws Back, Next, Commit, Finish and Cancel and turns Back and Finish off on the first page. The payload names the four steps and carries no words and no on-or-off state for them.

```
WALK_STEPS = (WALK_NEXT, WALK_BACK, WALK_CANCEL, WALK_FINISH)
```

The RENDERS column asks what the operator sees when he opens the screen, so each of the four was asked of the running build rather than of the table. The variant seam holds sixteen screens and answers a class per build.

```
screen                      qt build                  react build
Settings dialog             SettingsDialog            SettingsDialogReact
Start All progress dialog   StartAllProgressDialog    StartAllProgressReactDialog
Buy confirmation dialog     BuyConfirmationDialog     BuyConfirmationReactDialog
bot wizard                  no entry: both builds open BotCreationWizard
```

Three of the four keep `yes`, and each was drawn under both builds and pictured. The wizard takes the dash. It has no entry on the seam, so + New Bot opens the Qt wizard whichever build is running, and its React module draws only inside the exchange screen the shell builds when its backend names an exchange.

The buy confirmation keeps `yes` for the class the broker raises, which is the React one. Nothing calls the broker's request method, so the operator reaches neither build of that screen; the section above records that count and it has not moved.

The wizard's walk row is a gap in what the payload carries, and the Python surface owns it. Qt's wizard supplies its own five buttons and works out which are live from the page it is on. The React module is handed four step names and nothing else, so it has no words to draw and no state to grey.

```
WALK_STEPS = (WALK_NEXT, WALK_BACK, WALK_CANCEL, WALK_FINISH)
```

Two things are missing from `bot_wizard_surface`: the words each step draws, and whether each step is live on the page now showing. Back is dead on the first page and Finish is dead until the last, and both follow from the step order the surface already holds.

```
PROPOSED, in bot_wizard_surface.py

WALK_WORDS = {"back": "< Back", "next": "Next >", "commit": "Commit",
              "finish": "Finish", "cancel": "Cancel"}

walk_live(page) -> {"back": page is not the first, "next": a page follows,
                    "finish": no page follows, "cancel": True}
```

2026-09-09 12:45 - #128 - the alerts tab is opened, the news strip is drawn inside the exchange screen

Two rows read `no` under Registers in Electron: the Notifications and Alerts tab and the crypto news strip. Neither is a panel the shell should hold, so both now take the dash the table already uses for a module that does not register in its own right.

The running shell was asked about each name before anything was changed. Both modules load and neither registers a panel.

```
manifest names        69
panels registered     19
alerts_tab            alerts_tab.js registered no panel to draw
crypto_news_ticker    crypto_news_ticker.js registered no panel to draw
module load errors    []      faults []
```

Each was then registered by hand and opened into a host of its own, which is the test that decides the cell. Both draw, so neither is an empty shell; what decides them is that the shell opens neither, and that a panel host has no way to give either one the values it needs.

```
name                  opened  elements  characters  what it drew
alerts_tab              true        54         233  both tables empty
crypto_news_ticker      true         2          21  Fetching crypto news
strips on the page                                  2, for one exchange space
```

The alerts tab is raised by a window rather than by a tab. The variant seam holds it under the name ALERTS, so the Qt build opens `AlertsTab` and the React build opens `AlertsReactTab`, which is one browser view filling the whole tab. Opened as a panel instead, it draws every control and no row, because the host asks with an empty request and no notification manager reaches it.

The news strip belongs to the exchange screen. That screen keeps a space for it and fills that space, once per exchange, the way it fills the two bot tables. A registration holds one host per name, so registering the strip put a second copy of it on the page.

```
    space.setAttribute(CHILD_ATTR, moduleName);
    return api.mount(target);
```

The two RENDERS cells part company. The variant seam answers a class per build for the alerts tab, so that cell keeps `yes`. It holds no entry for the news strip, so the exchange header builds the Qt strip whichever build is running, and the React module draws in the Electron shell and nowhere else. That cell now reads `shell`, the value this table already defines for exactly that.

```
screen                      qt build            react build
Notifications and Alerts    AlertsTab           AlertsReactTab
crypto news strip           no entry: both builds build CryptoNewsTicker
```

The strip was never fed. The exchange screen asked the backend for the strip's state with an empty request, so the strip was built and never started, and the line read "Fetching crypto news" for as long as the shell ran. Qt starts the strip as it builds it and the screen already reports that it did, so the request now carries the start it reports.

```
  function tickerRequest(model) {
    var started = model[NEWS_TICKER_STARTED];
    return typeof started === "number" && started > ZERO ? { start: true } : {};
  }
```

The backend had no fetch to run either. The strip's model takes its fetch as an argument and the bridge never gave it one, so the worker answered with no stories. The bridge now hands in the same transport the Qt strip uses, and runs the fetch away from the request that asked for it, so the answer arrives the way it arrives in Qt.

```
    crypto_news_ticker_surface.use_fetch(SafeRequest, safe_urlopen, time.monotonic)
```

With that in place the shell fetched 43 stories from the ten feeds and drew them, one at a time, in the exchange header. Refusing every feed drops them again, which is what proves the line carries what the fetch found.

```
live      [2/43] The Defiant - SEC Crypto Custody Rewrite Enters White House Review
broken    (no crypto news feeds reachable)
no start  Fetching crypto news
```

Both builds of each screen were then driven on one reading and compared item by item. The alerts tab was read on one notification manager filled the way the main window fills it: the risk rules over a three-bot fleet, the profit and loss milestone, one bot error and one bot start. The news strip was read on one answer from the shipped fetch, given to each side through its own receiver.

```
alerts tab       13 routing rows, 4 history rows      differences 0
news strip       6 stories, 8 steps, wraps 6 to 1     differences 0
```

The routing table now draws **14** rows. `AlertEvent.TRADE_EXECUTED` in `src/core/notifications.py` became `BUY_FILLED` and `SELL_FILLED`, one event for each of the two fill boxes on `Settings > SMS`. The table reads the rows off `NotificationManager.get_config`, so it draws whatever that answers and no count is written on the page. Eleven of the fourteen rows name the SMS channel, which `notifications.message_channels_for` gives every event `notifications.EVENT_MESSAGE_KIND` pairs with a switch.

Every alert row matches: its time, its priority word, its title, its message, its channels, and the colour of each. The four priorities draw in four colours, low grey through critical red, and the rows run newest first on both sides. Emptied of notifications, both sides draw no rows and both read `Unread: 0`.

The strip shows the same story at each place, moves one place forward on each step, and wraps from the last story to the first on both sides. Its words are `#cfe6ff` on both and its pointer is the hand on both.

Two colours were wrong on the page and are now right. The tab's group boxes carry a Qt style sheet whose colour paints the group's title alone; the page was painting the whole box with it, so the routing table's Event and Priority columns and the three field labels drew in the accent where Qt draws them in the body colour. The page also carried no stylesheet of its own, so with the group colour moved to the title it fell back to the browser's black.

```
                        before        after         qt
routing Event column    #00ffcc       #e0e0f0       #e0e0f0
field labels            #00ffcc       #e0e0f0       #e0e0f0
```

The two strips take the fetch at different moments. The Qt strip shows a story as soon as the fetch answers, because the worker reports to it. The page has no such report, so it shows the answer on its next 15 second step. Closing that needs a route from the backend to a module the shell holds by name, and the strip is drawn by its parent rather than held by name.

The Qt tables are drawn by the theme, not by the tab, so the page draws no grid lines, no alternating row bands and no header underline. Those live in the Qt style sheet for every table in the application and reach no payload.

2026-09-09 13:08 - #128 - the RENDERS column reads what the running build draws

The running program answers this column, not the table. The table marks 42 rows in scope. The reading covered 38 of them. 26 agree with the cell they carried. 12 do not, and every one of the 12 claims more than the operator gets. No row claims less. Four rows carry no reading, and each keeps the cell it carried.

```
rows marked in scope       42
measured                   38
  agree                    26
  disagree                 12
could not measure           4    bot_wizard, header_strip,
                                 market_inspector_topologies, visualizer/themes
```

Ten cells change here. Six drop to a dash and four drop to `shell`.

Qt file	before	after	what the running build draws
`src/gui/main_tabs/trading_tab.py`	yes	-	`_build_trading_tab` gives the Live tab a page with no web view
`src/gui/widgets/exchange_tab.py`	yes	-	`ExchangeTab` is that page
`src/gui/widgets/bot_status_table.py`	yes	-	`BotStatusTable` sits inside that page
`src/gui/widgets/extractor_bot_table.py`	yes	-	`ExtractorBotTable` sits inside that page
`src/gui/live_settings/positions_held_tab.py`	yes	-	`BotLiveSettingsReactDialog` carries seven tab labels and none of them is Positions Held
`src/gui/main_tabs/console_log_handler.py`	yes	-	the window installs `_QtLogHandler` on the root logger
`src/gui/widgets/trade_charts_tab.py`	yes	shell	`TradeChartsTab` is the Charts tab, and `trade_charts_tab.js` draws in the Electron shell alone
`src/gui/native_chart.py`	yes	shell	`ChartPanel` and `CandlestickChart` draw inside that tab, and `native_chart.js` draws in the shell alone
`src/gui/indicator_panel.py`	yes	shell	`IndicatorVotingPanel` is the voting panel, and `indicator_panel.js` draws in the shell alone
`src/gui/widgets/status_log.py`	yes	shell	`StatusLog` is the log pane, and `status_log.js` draws in the shell alone

Two of the twelve keep the cell they carry. The news strip cell already reads `shell`. The Main Window cell keeps `yes`, because none of the three values fits what that window draws.

The Main Window is a React tab book inside a Qt frame, with a Qt menu bar and a Qt status bar, and `MainWindow` in `src/gui/main_window.py` builds all three.

```
MainWindow._setup_ui           builds the tab book from the variant seam
MainWindow._setup_menu         builds the Qt menu bar
MainWindow._setup_status_bar   builds the Qt status bar
```

The 42 in-scope cells now read:

```
yes      30
shell     5
-         7
```

2026-09-09 14:30 - #34 - the Status tab leaves the empty-tab pair

Status is no longer a skeleton, so it leaves `empty_tabs.py` and takes a row of its own. Accumulation stays where it was and still draws its empty state.

`src/gui/main_tabs/system_status_tab.py` — what the window builds for the tab

```
    def _build_system_status_tab(self) -> None:
        """Append the Status tab, and hold None when its page cannot be built."""
        try:
            from ..react_system_status_tab import SystemStatusReactPanel

            self._system_status_tab = SystemStatusReactPanel()
            self._main_tabs.addTab(self._system_status_tab, HEADING)
        except Exception as exc:  # noqa: BLE001 - a missing tab is not a crash
            logger.warning("Status tab unavailable: %s", exc)
            self._system_status_tab = None
```

Three cells move. The `empty_tabs.py` row drops `system_status_tab.js` and now names one module. A new row carries the screen itself, and a second names the Qt-side file the conversion added.

Qt file	React module	RENDERS	Scope
`src/gui/main_tabs/empty_tabs.py`	`proof_of_accumulation_tab.js`	yes	in scope
`src/gui/main_tabs/system_status_tab.py`	`system_status_tab.js`	shell	in scope
`src/gui/react_system_status_tab.py`	no	-	React side

The new cell reads `shell`, and the reason is the seam. `variant_surface.py` records a Qt loader and a React loader for every screen that has both, and this screen has no entry there, because no Qt widget ever drew it. `yes` states that React draws where Qt used to, and nothing here used to. The module was read drawing under its own tab in the Electron shell, so `shell` is what was measured.

```
registered           system_status_tab is in acervatorPanelHost.registered()
tab bar              system_status_tab, ninth of ten
select               returns system_status_tab
drawn                17 subsystem panels, 11 tab groups, 78 emitter rows
```

The detail is in 08-tabs/system-status.md.

2026-09-09 15:12 - #435 - Start All waits 3.5 seconds and retries a bot once

The Start All dialog used to promise a pause of about 2.5 seconds with a two-second minimum. The engine waited 0.6. The pause is now 3.5 seconds, it is written down once, and the dialog quotes that figure rather than a number of its own.

```
#: Seconds `BotManager.start_all` waits between bots, and the figure the Start
#: All dialog quotes. 0.6 drew HTTP 429 on three of 38 starts.
START_ALL_GAP_SECONDS: float = 3.5
```

The boot of 8 September 2026 is in the operator's own log. Thirty-eight bots were brought up one at a time and three of them came back with the same refusal from Coinbase: too many requests, on the call that lists the accounts. Those three did not trade until they were started again eight minutes later. A wider pause is exactly what the dialog always said the gap was for.

```
2026-09-08 11:10:43,649 BOT_CONNECT_FAILED | RateLimitExceeded: coinbase GET
https://api.coinbase.com/api/v3/brokerage/accounts?limit=250
429 Too Many Requests | DIAGNOSIS: Rate limited.
```

A bot that does not report itself running inside ten seconds is now stopped and started one more time, and then the sequence moves on whatever happened. The stop is what makes the second attempt real, because a bot still in its starting state refuses a plain second start and the retry would do nothing.

```
            if not verified and not getattr(self, "_start_all_cancel", False):
                logger.warning(
                    "Bot %s did not reach RUNNING in %.1fs; retrying once",
                    bot.bot_id,
                    verify_timeout_seconds,
                )
                await asyncio.sleep(min_gap_seconds)
                await bot.stop()
                await bot.start()
                verified = await self._await_running(bot, verify_timeout_seconds)
```

The platform now writes a line for each of those events, so a boot that loses a bot says so in the log instead of leaving a silent hole. A bot that comes up on the retry gets one warning; a bot that fails both attempts gets two.

```
Bot LATEBOT starting on LATEBOT/USD
Bot LATEBOT did not reach RUNNING in 10.0s; retrying once
Bot LATEBOT starting on LATEBOT/USD
Bot DEADBOT did not reach RUNNING in 10.0s; retrying once
Bot DEADBOT is not RUNNING after 10.0s; start_all moved on
```

The cost is real and it is worth knowing before the next launch. Thirty-eight bots mean thirty-seven waits. The 8 September pass took 3 minutes 33 seconds for its 35 bots at a mean of 5.3 seconds each, so the same fleet at the new pause reaches about 5 minutes, and three retries add about 40 seconds on top.

```
gaps alone, 37 waits       0.6 s -> 22.2 s        3.5 s -> 129.5 s
measured pass, per bot     mean 5.318 s           mean 8.218 s
38 bots, three retried     3 min 17 s             about 5 min 45 s
```

2026-09-09 21:37 - #147 - the Accumulation tab leaves the empty-tab pair

Accumulation is no longer a skeleton, so it leaves `empty_tabs.py` and takes a row of its own. It draws three zones: the player window on the left, the square enemy screen upper right, and the party window across the lower half. Every zone draws a placeholder and no art, class or mode is built.

`src/gui/main_tabs/proof_of_accumulation_tab.py` — what the window builds

```
    def _build_proof_of_accumulation_tab(self) -> None:
        """Append the Accumulation tab, and hold None when its page cannot be built."""
        try:
            from ..react_proof_of_accumulation_tab import ProofOfAccumulationReactPanel

            self._proof_of_accumulation_tab = ProofOfAccumulationReactPanel()
            self._main_tabs.addTab(self._proof_of_accumulation_tab, HEADING)
        except Exception as exc:  # noqa: BLE001 - a missing tab is not a crash
            logger.warning("Accumulation tab unavailable: %s", exc)
            self._proof_of_accumulation_tab = None
```

Three cells move and two rows are new. The empty-tab row names no module now, so nothing the window builds reaches it.

Qt file	React module	RENDERS	Scope
`src/gui/main_tabs/empty_tabs.py`	no	-	shelved
`src/gui/main_tabs/proof_of_accumulation_tab.py`	`proof_of_accumulation_tab.js`	shell	in scope
`src/gui/react_proof_of_accumulation_tab.py`	no	-	React side

The new cell reads `shell`, and the reason is the seam. Sixteen screens are registered there and none of them is this one, because no Qt widget ever drew a proof-of-accumulation screen. The two shelved Qt classes are set aside by the operator's directive of 21 April 2026 and this tab is built on neither.

```
seam screens             16
a PoA, competition or testnet entry among them   none
registered               proof_of_accumulation_tab, with no fault
tab bar                  Accumulation, seventh of ten
zones drawn              player window 2, enemy screen 2, party window 56
party window             8 groups of 5, 40 slots on page 1 of 3
wallet                   0 panels closed, 1 panel and 3 sections open
```

Removing the registration is the control. The panel name leaves the register, the Accumulation tab leaves the bar, and every zone count falls to nought while the same counter still reads 154 elements elsewhere on the page.

```
registered               proof_of_accumulation_tab absent, 18 names
reasonFor                proof_of_accumulation_tab.js registered no panel to draw
tab bar                  nine tabs, Accumulation gone
zones drawn              0, 0, 0
whole page               154 elements, so the noughts are the zones
```

The chain the shell draws is a field of the payload, so a demo run on the TestNet asks the same bridge method with a different chain and reaches the same page. Nothing else in the request changes.

```
def chain_of(params: dict) -> str:
    """The chain ``params`` names, or ``LIVE_CHAIN`` when it names none in ``CHAINS``."""
    asked = params.get(CHAIN_FIELD) if isinstance(params, dict) else None
    return asked if asked in CHAINS else LIVE_CHAIN
```

One tooling defect surfaced and is fixed here. The page carries three tables under the same header, and the Scope writer took the first, which is a two-row excerpt inside an earlier entry. It wrote a totals block into that excerpt. The span now picks the widest table.

```
    spans = []
    for start, line in enumerate(lines):
        if line.startswith(TABLE_HEAD):
            end = start + 2
            while end < len(lines) and lines[end].startswith("|"):
                end += 1
            spans.append((start, end - 1))
    if not spans:
        raise LookupError("no line starts with " + TABLE_HEAD)
    return max(spans, key=lambda span: span[1] - span[0])
```

Pointed at the real table the writer disagrees with the committed Scope column on twenty rows and moves five totals. That disagreement is not this entry's to resolve, so the column was not rewritten and the three cells above were set by the rule the column states.

2026-09-12 - #665 - the Phantom Bots page takes one timeframe and keeps it

The page holds three settings. All three now reach the bot they are drawn for, and two of them survive a restart.

One phantom, not a set. The eleven timeframe boxes are one picker. It offers all eleven names, greys the ones the bot's venue does not serve, and opens on the timeframe the bot holds. A bot holding several, because it was restored before this, opens on the highest of them.

setting	label on the page	what it reaches
enable	Enable Phantom Bots	the tick gate that builds the phantom set
timeframe	Timeframe:	the timeframe list the phantom set is built from
lock	Candles to lock:	the coordinator's candle count

The constraint is the one the Comp field already stated. A timeframe at or below the bot's own TA Timeframe is refused, and the reason is written beside the picker. A timeframe the venue does not serve carries a second message. Each pick below was made on a page freshly opened on 1d, on a Coinbase bot whose TA Timeframe is 1h:

```
picked  kept   told to the window       reason written beside the picker
1d      1d     phantom_timeframes 1d    -
6h      6h     phantom_timeframes 6h    -
1h      1d     nothing                  1h is not above this bot's TA Timeframe 1h
15m     1d     nothing                  15m is not above this bot's TA Timeframe 1h
4h      1d     nothing                  4h is not offered by coinbase
```

The refusal is not new work for the engine. A phantom at or below the parent's own timeframe is already dropped twice over: the higher-timeframe bias keeps only a higher rank, and the Comp calculation skips a rank at or below the parent's. On the bot above, four of the five phantoms a restart used to build reached neither.

Two of the three now survive a restart. The saved record carries the one timeframe and the candle count beside the enable flag it already carried. Driven through the fleet-load path on a record written for the run:

```
record carries                 restored bot holds                    lock
phantom_timeframe 1d           1d                                       2
lock_candle_count 7            5m 15m 30m 1h 1d                         7
neither key                    5m 15m 30m 1h 1d                         2
```

A bot nobody has chosen for is left exactly as it was. The record takes a timeframe only when the bot holds one. A bot still on the engine's own six, which a Coinbase filter cuts to five, writes no timeframe key and restores to the same five. The narrowing to one happens when the operator picks one, and not before.

```
bot                saved keys                                  restored holds  lock
untouched          phantoms_enabled lock_candle_count                 5 names       2
one timeframe set  phantoms_enabled lock_candle_count                 1 name        7
                   phantom_timeframe
```

The window now publishes what the page reads. The answer a tab is asked for carried four values, and the page read five others off it that were never there, so every phantom reading on the React build drew at its own default. The answer carries them now.

```
published before   bot_id symbol mode state
published now      bot_id symbol mode state exchange_id ta_timeframe
                   enabled timeframes lock_candle_count
```

The candle count the page sets still sizes nothing. `TimeframeCoordinator` defines the one function that would read it and nothing calls that function, so the figure persists and waits.

2026-09-13 - #23 - the venue page draws in the second build

The venue page now has a host. `src/gui/react_exchange_tab.py` builds one web view for each venue and draws `exchange_tab.js` inside it.

The host answers the four calls the main window already makes on the Qt venue tab. Each call drives a surface model and pushes the answer into the page.

```
call the window makes             what the host runs
exchange_id                       the attribute the constructor sets
update_bots(statuses)             the screen model, then both bot tables
_refresh_privacy_mode_btn_style   the screen model, then the header dots
stop_feeds                        the page script that takes the bridge away
```

The page carries nine modules and one style sheet. It fetches nothing, because every script and every rule is written into the document.

```
design_tokens.js         theme_engine.js          shared_widgets.js
header_strip.js          table_cells.js           bot_status_table.js
extractor_bot_table.js   crypto_news_ticker.js    exchange_tab.js
one style sheet          exchange_tab.css         26 rules
```

The screen draws a header row, a data-pool line, two bot sections and a command bar of five buttons. `bot_status_table.js` draws the Scrumming rows and `extractor_bot_table.js` draws the Extractor rows, each into a space the screen keeps for it.

The page names what it loaded. A venue holding seven bots reads like this.

```
page modules       all nine, exchange_tab.js last
page style sheets  exchange_tab.css, 26 rules
scrum-table rows   5
extractor rows     2
```

The style sheet paints what the payload leaves unpainted. The Privacy Mode button, the data-pool line and the two section labels carry their own colours from the backend, so no rule touches them. The five command buttons and the New Bot button carry none, so the sheet gives each one a ground, an edge and a text colour.

An emptied style sheet reads as zero rules, and a withheld module drops out of the module list and draws no row. Both readings fall when the thing they measure is taken away.

```
run              module list       style sheet rules   scrum rows
whole page       nine modules                     26            5
style emptied    nine modules                      0            5
table withheld   eight modules                    26            0
```

The page reports every control press back to Python. `ExchangePage` reads the console line the page writes and hands it to `run_action`, which drives the surface that owns the method and then pushes the new payload.

```
pressed           reached Python             the screen recorded
Start on row 1    bot-a14-02, start          command.sent start scrumming
nothing pressed   no call                    no entry
```

The host starts no bot and places no order while it builds or draws. It reaches three names under the trading package, and none of them is a bot, a container or an exchange client.

```
src.trading                    the package
src.trading.gate_vocabulary    the gate names
src.trading.target_bands       the Target A15 and Target A14 arithmetic
```

The news strip is fed. `bind_news_transport` gives the strip the same reader the desktop bridge gives it, and the first ask carries the start the Qt strip performs as it builds.

2026-09-13 - #23 - the bot wizard gets a host and a stylesheet

`react_bot_wizard.py` builds the wizard page. `bot_wizard.css` paints it. The host answers the calls the window already makes on the Qt wizard, so one set of calls drives either build.

the window calls	the host answers with
the class, with the venue list and the stored defaults	`BotWizardReactDialog.__init__`
exec	`QDialog.exec`
DialogCode.Accepted	`QDialog.DialogCode`
get_bot_config	`BotWizardReactDialog.get_bot_config`

The page carries every asset it needs. `dialog_html` inlines React, the four style sources, the table cells module, the wizard module and the sheet. The page fetches nothing over the network. `MODULE_GLOBALS` names the global each asset defines once its script tag has run.

The page says what it loaded. `LOADED_MODULES_JS` reads each of those globals back. `LOADED_STYLES_JS` counts the sheets the browser parsed and the rules whose selector names the wizard class. One run of the built host read this:

```
modules   react  react-dom  design_tokens  theme_engine  shared_widgets
          header_strip  table_cells  bot_wizard        all true
styles    sheets 1    rules 56    wizard rules 50
rail      asset  mode  params  phantom  extractor_pool   params current
rows      47 field rows drawn
```

The same run with the sheet emptied on disk read `wizard rules 0`, and the browser drew all 47 rows in its own plain type. The same run with the module emptied on disk read `bot_wizard.js false`, no rail stops and no rows.

One model draws every page. `BotWizardReactDialog.model` asks `bot_wizard_surface.view_model` for the whole wizard as one payload. The venue list, the stored defaults, the market list and the offered timeframes go in around the steps the page sends.

A press on the page comes back to Python. The page writes one console line. `BotWizardPage` hands that line to `answer_call`, which builds a fresh payload and hands it back. A finished walk accepts the dialog, and a cancelled walk rejects it.

The sheet paints the parts the module names. Every rule sits under the `acervator-bot-wizard` class and selects one `data-part` value.

part	what the sheet paints
wizard-title	the window name, in the accent colour, over a rule

part	what the sheet paints
page-rail-stop	one pill per page, and the current page takes the accent
page-title, page-subtitle	the page heading and its line of help
field-group	a bordered block with its title in the accent
field-row-label, field	a right-aligned label against a growing control
target-combo, info-button	the pair box, its circle and the info button, on one line
timeframe-row	one chip per timeframe, and a chip the venue refuses dims
alt-list, alt-row	the pair list the pool page fills
warning-box	the call-budget message the phantom page raises
walk-step	the Next, Back, Cancel and Finish buttons

The host reaches no venue and creates no bot. The venue list and the market list are arguments, so the dialog opens no connection of its own. A run that built the host and drew all five pages at three widths imported no module under `src/trading` and wrote no file under the runtime tree.

The table row above still reads a dash. `variant_surface.py` records no loader pair for this screen, so the running window builds the Qt wizard. The row changes when that pair is recorded.

2026-09-13 - #23 - the window chooses the three new hosts

`variant_surface.py` records three more loader pairs. `register` holds one Qt loader and one React loader for each screen. The running build selects one side of each pair.

screen name	the Qt loader returns	the React loader returns
Charts	`TradeChartsTab`, in `widgets/trade_charts_tab.py`	`ChartsTabReact`, in `react_charts_tab.py`
Exchange page	`ExchangeTab`, in `widgets/exchange_tab.py`	`ExchangeTabReact`, in `react_exchange_tab.py`
Bot creation wizard	`BotCreationWizard`, in `bot_wizard.py`	`BotWizardReactDialog`, in `react_bot_wizard.py`

Three call sites ask `surface_class` for the class. `ChartsTabMixin` builds the Charts tab in `_build_charts_tab`. `MainWindow` builds one venue page in `add_exchange_tab` and opens the wizard in `_create_bot`. Each call site keeps the Qt class as its fallback, so a screen with no recorded pair still builds the original.

The window names what it built. One window ran under each build, and each class came off the live object.

```
screen          qt build            react build
Charts tab      TradeChartsTab      ChartsTabReact
venue page      ExchangeTab         ExchangeTabReact
wizard          BotCreationWizard   BotWizardReactDialog
```

Each React page names what it loaded, once the window reaches it.

```
Charts tab   header_strip, native_chart, trade_charts_tab, no faults
venue page   nine modules, exchange_tab.js last, exchange_tab.css, 26 rules
wizard       eight module globals, all true, one sheet, 56 rules
```

The reading falls when one pair goes away. A run with the Charts pair deleted built the Qt tab and logged the refusal. The venue page and the wizard still answered with their hosts.

```
recorded pairs        Charts tab       venue page         wizard
all three             ChartsTabReact   ExchangeTabReact   BotWizardReactDialog
Charts pair deleted   TradeChartsTab   ExchangeTabReact   BotWizardReactDialog
```

The venue page keeps a dash in the table above. `TradingTabMixin` holds the Qt layer page as a hidden child in `_react_trading_page`, and that page holds the venue tabs. The React venue page draws its nine modules as soon as a run shows that layer page, so the host is built and reachable, and the Live tab hides it.

2026-09-13 - #23 - the two strips get a host and a stylesheet

`react_status_log.py` builds the Activity Log page. `status_log.css` paints it. `react_crypto_news_ticker.py` builds the news strip page. `crypto_news_ticker.css` paints it. Each host answers the calls the window already makes on the Qt widget, so one set of calls drives either build.

the window calls on `StatusLog`	`StatusLogReact` **answers with**
log	`StatusLogReact.log`
force_log	`StatusLogReact.force_log`
pause, resume	`StatusLogReact.pause`, `StatusLogReact.resume`
is_paused, toggle_pause	`StatusLogReact.is_paused`, `StatusLogReact.toggle_pause`
health_stats	`StatusLogReact.health_stats`
setMaximumHeight	`QWidget.setMaximumHeight`

the header calls on `CryptoNewsTicker`	`CryptoNewsTickerReact` **answers with**
the class	`CryptoNewsTickerReact.__init__`
start	`CryptoNewsTickerReact.start`
stop	`CryptoNewsTickerReact.stop`
force_refresh	`CryptoNewsTickerReact.force_refresh`
current_headlines	`CryptoNewsTickerReact.current_headlines`

Each page carries every asset it needs. `panel_html` inlines React, the four style sources, the strip's own module and its sheet. Neither page fetches over the network. `MODULE_GLOBALS` names the global each asset defines once its script tag has run.

Each page says what it loaded. `LOADED_MODULES_JS` reads those globals back. `LOADED_STYLES_JS` names each sheet and counts the rules the browser parsed from it. One run of the built hosts read this:

```
status log   modules  design_tokens theme_engine shared_widgets header_strip status_log
             styles   status_log.css 12 rules             lines drawn 8
news strip   modules  design_tokens theme_engine shared_widgets header_strip crypto_news_ticker
             styles   crypto_news_ticker.css 8 rules      headlines drawn 1
```

The readings fall when an asset goes. The same run with each sheet emptied read no rules, and the browser drew both strips on white. The same run with each module emptied dropped that module from the roster and drew no line and no headline.

```
run         status log            news strip
live        12 rules, 8 lines     8 rules, 1 headline
no sheet     0 rules, 8 lines     0 rules, 1 headline
no module   12 rules, 0 lines     8 rules, 0 headlines
```

The pane paints a line by its kind. `status_log_surface.line_style` reads the message prefix first and the level second. A pixel census of each saved picture read these counts.

```
live        0a0a0f 238504   0e0e1a 193353   7a7a9c 2594   00ff88 478   ffaa00 458
no sheet    ffffff 430648   ffaa00 1759     00ff88 1365   ff66dd 531   ffcc44 461
no module   0a0a0f 450000
```

The sheet carries the page ground, the pane ground and the border. The payload carries every line colour, so a line keeps its colour with no sheet at all.

One payload draws the whole pane. `pane_payload` asks `status_log_surface.build_view_model` for the pane state, then puts every line the model holds under `document`. `build_view_model` drains `take_painted`, so its own `document` carries one call's batch. `document_blocks` counts the whole document, and the two agree once `pane_payload` has run.

The strip shows the story it holds. `CryptoNewsTickerReact._run_fetch` runs the strip's worker only when the strip carries a `fetcher`. `CryptoNewsTickerModel.on_headlines` writes `NO_FEEDS_TEXT` for an empty answer, so a strip with no reader would report no feeds over the stories it holds.

The strip reached no feed. A run built the host with four recorded stories and no `fetcher`. The strip drew the first story with its position and its feed name. The run refused every connect away from loopback and counted none.

```
fetcher              None
fetches started      1
headlines held       4
connects attempted   0
label                [1/4] The Defiant - SEC crypto custody rewrite enters White House review
```

Each sheet paints the parts its module names.

part	what the sheet paints
log	the read-only pane, its ground, its border and its scrollbar
line	one block, wrapped rather than widened

part	what the sheet paints
stamp	the leading time, in tabular figures
body	the message span, which takes its colour from the payload
placeholder	the line an empty pane shows
ticker	the frame around the strip
headline	one story on one line, clipped at the strip's edge

Both table rows above still read `shell`**.** `variant_surface.py` records no loader pair for either strip, so the running window builds the Qt pane and the Qt strip. Each row changes when its pair is recorded.

2026-09-13 - #23 - four modules already had a host, and the tables get their skin

Four rows read `-` or `shell`. All four already draw in the window the operator launches. No new host was written. The measurement is below, and four cells move.

The venue page host mounts both bot tables inside itself. The Charts host mounts the chart module inside itself. The Status panel is its own host.

```
react_exchange_tab.CHILD_MODULES   table_cells, bot_status_table,
                                   extractor_bot_table, crypto_news_ticker
react_charts_tab.CHILD_MODULES     native_chart
react_system_status_tab            PANEL_MODULE system_status_tab.js
```

The window reaches all three. `main_window.py` asks the seam for the venue page class, and `charts_tab.py` asks it for the Charts class. `SystemStatusTabMixin` builds the Status panel with no seam, because no Qt widget ever drew that tab.

```
main_window.py, in add_exchange_tab            page_class = surface_class(EXCHANGE)
main_tabs/charts_tab.py, in _build_charts_tab  built = surface_class(CHARTS)()
main_tabs/system_status_tab.py                 SystemStatusReactPanel()
```

Each page names itself off the live object. A run built each screen, drove content into it and read the class and the accessible name back.

```
ExchangeTabReact           React Exchange Tab   widget and web view
ChartsTabReact             React Charts Tab     widget and web view
SystemStatusReactPanel     Status               widget and web view
```

Each page reports what it loaded. The venue page already answered this. The Charts page and the Status page now answer it too, because `page_html` gives every style sheet a `data-asset` name of its own.

```
venue page   9 modules, exchange_tab.css 31 rules
Charts tab   3 modules, trade_charts_tab.css 20 rules, 0 panel faults
Status tab   1 module, system_status_tab.css 25 rules
```

Each page drew the content driven into it. Six bots went into the venue page, forty candles into the chart, and the emitter read-out into Status.

```
venue page   4 Scrumming rows, 2 Extractor rows, 56 cells, 6 coin badges
Charts tab   40 candles, 40 wicks, 40 volume bars, 3 indicator panels
Status tab   17 subsystems, 11 tab groups, 78 emitter rows
```

The readings can fail. Each page was built again with its style sheet emptied and again with its module absent. Every reading reported the loss.

```
sheet emptied    rule count 0, page paints white
venue module out modules drop both tables, rows 0 and 0, no cells
chart module out modules drop native_chart, fault "registered no panel to draw"
status module out modules empty, emitter rows 0
```

Both variants drew the same content, and five colours did not match. Each screen was built under the qt variant and the React variant with the same bots and the same candles. Every table cell agreed, header for header and row for row. Five colours read off the saved pictures did not.

element	qt variant	React before	React now
column header text	`#00ffcc`	`#e0e0f0`	`#00ffcc`
header underline	2px accent	1px grey	2px accent
Detail button face	`#1a1a28`	`#efefef`	`#1a1a28`
Fire button face	`#1a1a28`	`#efefef`	`#1a1a28`
coin badge	a coloured disc	nothing drawn	a coloured disc

The badge is the disc `_get_coin_icon` paints when no logo file is cached. Its colour comes from the characters of the name. One function now answers for both sides, so the two cannot drift.

```
def coin_disc_color(symbol: str) -> str:
    hue = sum(ord(char) for char in str(symbol)) % HUE_WHEEL
    red, green, blue = colorsys.hsv_to_rgb(
        hue / HUE_WHEEL, DISC_SATURATION / ALPHA_HIGHEST, DISC_VALUE / ALPHA_HIGHEST
    )
```

Four discs were read off the qt picture and the same four off the React picture. They agree to the digit.

```
A15-USD   #5fb479      A23-USD   #5fb496
A14-USD   #5fb484      A21      #b45fb1
```

Two differences stay, and each has a reason. The header ground reads `#16162a` under Qt and `#1a1a28` under React. No design token holds the first value, and the sheet may name no colour the design system does not hold. The column rule reads a lighter grey under Qt for the same reason.

The Charts tab chrome differs above the chart, and that belongs to another row. The qt arrows draw no glyph, the qt timeframe box carries a `TF:` label, and the qt indicator toggles paint filled squares where React draws check boxes. Every one of those sits in `trade_charts_tab.js`, whose row already reads `yes`.

Status draws the same under both variants. It has no Qt original, so the window builds the React panel either way. The two pictures hold the same colours in the same counts at all three widths.

```
status qt      #141420 60397   #0a0a0f 28812   #ffffff 4080
status react   #141420 60397   #0a0a0f 28812   #ffffff 4080
```

Four cells move to `yes`. The Status cell moves because the column reports what the operator sees, and he sees the React panel in the window.

Qt file	React module	before	after
`src/gui/widgets/bot_status_table.py`	`bot_status_table.js`	-	yes
`src/gui/widgets/extractor_bot_table.py`	`extractor_bot_table.js`	-	yes
`src/gui/native_chart.py`	`native_chart.js`	shell	yes
`src/gui/main_tabs/system_status_tab.py`	`system_status_tab.js`	shell	yes

2026-09-13 - #23 - the tab bar paints from the theme again

The ten grounds are gone. The bar paints no ground of its own. Every tab now takes the colours the running theme's own style sheet sets, which is what the bar did before the grounds were added.

`src/gui/variant_surface.py` — `_qt_main_tab_book`

```
def _qt_main_tab_book() -> type:
    """Import and return the Qt main tab book."""
    from PySide6.QtWidgets import QTabWidget

    return QTabWidget
```

Six theme tokens go with them. Each of the five themes carried a ground colour and a text colour for black, white and gold. None of the thirty values is declared now, and no style sheet names one.

CITED AS ABSENT. The file `main_tab_bar` is deleted, so the paint block earlier on this page records what the bar did between the seventh and the thirteenth of September 2026.

Read off the rendered bar, five themes, both builds. A bright pixel here is one reading at least 200 red, at least 150 green and at most 120 blue. The gold ground filled most of the bar before and none of it now.

```
theme             qt before  qt after   react before  react after
cyberpunk_dark    20,880     0          14,262        0
neon_light        20,771     0          14,178        0
classic_terminal  22,027     0          14,262        0
minimal_modern    20,643     0          14,091        0
glass_metal       20,604     0          14,078        0
```

2026-09-13 - #23 - Accumulation leaves the bar in one place

One tuple names every tab the window does not build. The window skips the builder for a name in it, and the tab list the frontend reads leaves that name out. The bar now reads Sim, Paper, Live, Charts, Inspector, Swarm, History, Status, Console.

`src/gui/main_tabs/main_window_surface.py` — `UNBUILT_TABS`

```
UNBUILT_TABS = (ACCUMULATION_TAB,)

BAR_TAB_ORDER = tuple(name for name in CANONICAL_TAB_ORDER if name not in UNBUILT_TABS)
```

Removing the name from that tuple puts the tab back. The Accumulation screen is unfinished, not unwanted. Its builder, its panel, its bridge method and its manual page are all unchanged.

2026-09-13 - #23 - a tab that fails to build no longer moves the tabs after it

The reorder advances its slot only over a tab the bar carries. A name the bar does not carry took a slot before, so every tab after a missing one stayed where the builders left it. The Qt build fails to build Sim and Paper, and its bar now reads the canonical order of the seven tabs it does have.

`src/gui/main_tabs/main_window_surface.py` — `reordered_tabs`

```
    order = list(labels)
    target_index = 0
    for name in desired:
        if target_index >= len(order):
            break
        for current_index in range(target_index, len(order)):
            if order[current_index] == name:
                if current_index != target_index:
                    order.insert(target_index, order.pop(current_index))
                target_index += 1
                break
```

Read off the two running bars. The React build carries all nine tabs it builds. The Qt build carries seven, in the same order.

```
react   Sim, Paper, Live, Charts, Inspector, Swarm, History, Status, Console
qt      Live, Charts, Inspector, Swarm, History, Status, Console
qt was  Live, Charts, Inspector, Console, Swarm, Accumulation, Status, History
```

The two missing tabs have one cause. The Qt Sim page and the Qt Paper page each read a panel key their own surface no longer publishes, so each builder raises and the window logs the tab as unavailable. That belongs to the Simulator rebuild and to the Paper screen, not to the bar.

2026-10-16 - #158 - the splitter edges take the theme and seven theme values go

All four handle edges were one cyan. A horizontal handle draws a left and a right edge, a vertical handle a top and a bottom. All four named the same fixed cyan at a tenth of full opacity, so both light themes drew that cyan over a pale ground. The four edges now read `border_accent`, which every palette already declared and no rule read.

`src/gui/theme_engine.py` — `_theme_qss`, the horizontal handle

```
QSplitter::handle:horizontal {{
    width: 5px;
    background-color: {t.border_primary};
    border-left:  1px solid {t.border_accent};
    border-right: 1px solid {t.border_accent};
}}
```

A pressed handle asked for an opacity Qt does not have. The rule set `opacity: 0.85` beside the accent. Qt drops a property it does not know, so a pressed handle drew the full accent a hovered one already drew. The rule now paints the accent under the alpha byte Qt does read, at the same 0.85 share. The colour comes from the palette's own accent, so an accent the operator sets reaches the pressed handle too.

`src/gui/theme_engine.py` — `SPLITTER_PRESSED_ALPHA` and the pressed handle

```
SPLITTER_PRESSED_ALPHA = 217

QSplitter::handle:pressed {{
    background-color: {rgba(t.accent_primary, SPLITTER_PRESSED_ALPHA)};
}}
```

Read off the rendered handle, five themes, before and after. The reading is the left edge at the handle's mid-height, and the handle's middle while the pointer holds it. Every edge now carries its own palette's accent hue, and every pressed handle now sits between that accent and the ground it stands on.

```
theme             edge before   edge after    pressed before  pressed after
cyberpunk_dark    47,72,84      38,106,104    0,255,204       1,218,176
neon_light        207,232,239   196,169,229   102,0,204       124,37,211
classic_terminal  5,48,28       4,86,4        0,255,0         1,218,1
minimal_modern    216,241,239   199,212,239   37,99,235       68,121,237
glass_metal       35,60,70      65,83,106     136,204,255     120,178,222
```

Seven palette values are gone. Each of the five palettes set `accent_success`, `accent_warning`, `accent_info` and `glow_color`, and the declared default held `font_mono`, `font_size_large` and `radius_lg`. No rule in the style-sheet text named one of the seven. A Qt style sheet has no glow property at all, and no widget marks itself as a success, a warning, an info, a monospace or a large thing for a rule to match. A palette carries 48 named values where it carried 55. `glow_color` also names a local colour inside three indicator panels, which is a different thing under the same name and stays.

One corner radius and one grey are repaired in the widget table. `RADIUS_FULL` held 9999 for a pill. Nothing imported it, and the drawing engine refuses a corner roundness above half the shape, so the value could draw no pill. The widget table holds 195 names where it held 196. `CARD_METRIC_LABEL` held a three-digit grey, the only one in the table, which is why a helper widens it before any comparison. It holds the same grey in six digits now, and the pixel does not change.

`src/gui/design_system.py` — the corner radii today

```
RADIUS_NONE = 0
RADIUS_XS = 4  # Chips, tight inputs
RADIUS_CARD = 6  # Analytics metric cards
RADIUS_SM = 8  # Default — buttons, inputs, cards
RADIUS_MD = 12  # Elevated cards, dialogs
RADIUS_LG = 16  # Large surfaces, bottom sheets (rare in desktop)
```

Both builds still write the same style sheet. The second build keeps its own copy of the palette table and of the template, in `src/gui/main_tabs/theme_engine_surface.py`. Each build's five style sheets match the other's byte for byte.

```
theme             characters before  characters after
cyberpunk_dark    7419               7404
neon_light        7415               7400
classic_terminal  7427               7402
minimal_modern    7422               7407
glass_metal       7415               7410
```

2026-10-26 - #1556 - the React tab book answers both tab-visibility questions

The Qt tab widget can hide a tab, and the React book could not be asked. The React book stands in for the Qt tab widget and forwards the calls the window makes over the hidden widget it holds. Two calls were missing: the one that asks whether a tab is on show, and the one that hides or shows it. Each raised an attribute error, so anything hiding a tab on the main book would have stopped the window.

`src/gui/react_main_window.py` — `MainTabBookReact.setTabVisible`

```
        def setTabVisible(self, index: int, visible: bool) -> None:
            """Show or hide the tab at ``index`` and redraw the bar without it."""
            self._book.setTabVisible(index, visible)
            self._push_tabs()
```

A hidden tab also leaves the bar. The bar is drawn from a list of labels the book sends it. That list was every label the book held, so a hidden tab would have stayed on the bar with no page behind it. The book now sends the visible labels alone, and keeps the full list for mapping a press back to a page, so a press still reaches the right tab.

Driven offscreen on a book holding three tabs. Nothing on this machine draws text offscreen, so every figure below is a value the widget reports or the list the page is sent, never a picture.

```
before   isTabVisible(1)         raised 'MainTabBookReact' object has no attribute
         setTabVisible(1, False) raised 'MainTabBookReact' object has no attribute
now      isTabVisible(1)         True, then False once hidden
         isTabVisible(0)         True throughout
         labels                  Trading, Charts, History, throughout
         the bar is sent         Trading, History
         shown again             Trading, Charts, History
```

The neighbouring tab is the control. Tab 0 reads True on every reading, and the full label list is unchanged, so the False on tab 1 is a reading and not a book that lost its tabs.

A press still reaches the page it names. The book maps a pressed label back to a page through the full list, so hiding a tab does not shift the pages after it. Driven on a book of four tabs with the second hidden, each press landing on the page whose label it carried.

```
hidden            Charts
pressed Trading   page 0, reads Trading
pressed History   page 2, reads History
pressed Console   page 3, reads Console
shown again       pressed Charts, page 1, reads Charts
pressed Nowhere   the page does not move
```

The last two rows are the controls. A label the book carries again is reachable again, and a label the book does not carry moves nothing.

2026-10-27 - #53 - one declaration carries the mouse-over box's size and its wait

A control's mouse-over description draws in a box, and four figures decide that box. Three are its size and one is its wait. All four now sit together in `src/gui/theme_engine.py`, and every rule that draws the box reads them from there.

`src/gui/theme_engine.py` — the one declaration

```
TOOLTIP_MAX_WIDTH_PX = 560
TOOLTIP_FONT_SIZE_PX = 12
TOOLTIP_PADDING_PX = 8
TOOLTIP_DELAY_MS = 3000
```

Four style sheets draw the box, and two of them used to carry their own numbers. `src/gui/main_tabs/theme_engine_surface.py` typed 560px, 12px and 8px into its template. `src/gui/main_window.py` read the width and set no text size and no spacing at all. Both read the declaration now.

```
rule set                                          width  size  padding
src/gui/theme_engine.py                             560    12        8
src/gui/main_tabs/theme_engine_surface.py           560    12        8
src/gui/main_window.py                              560    12        8
src/gui/main_tabs/main_window_surface.py            560    12        8
```

The wait is a style hint, not a timer. `src/gui/tooltips.py, in WakeDelayStyle` answers Qt's own wake-up hint with `TOOLTIP_DELAY_MS`, and `main.py` puts that style on the application at launch. Qt restarts the wait on every pointer movement, however small, so the pointer has to hold still for the full three seconds.

```
application without the style   the toolkit's own 700 ms
application with the style      3000 ms
```

The React History page carries its own declaration, read by its own rule and its own timer. `src/gui/web/history_panel.css` declares the two figures on `:root`. The `.tooltip` rule reads the width

and `src/gui/web/history_panel.js` reads the wait, which it used to hold as a number of its own.

`src/gui/web/history_panel.css` — the React declaration

```
:root {
  --tooltip-max-width: 560px;
  --tooltip-delay-ms: 3000;
}
```

Read off the running page. The page was asked what wait it will use and what width its box draws at, with a neighbouring element as the control.

```
--tooltip-delay-ms           3000
--tooltip-max-width          560px
the wait the page will use   3000
.tooltip max-width           560px
.pager max-width             none
```

The pager is the control. The rule reaches the box and not the pager, so the 560px is a reading of the rule rather than of every element on the page.

Move the declaration and every box follows it together. Set the four figures to numbers nothing else uses and read all four Qt rule sets and the running React page again.

```
                                          branch        before this change
src/gui/theme_engine.py                   377 17 3      377 17 3
src/gui/main_tabs/theme_engine_surface.py 377 17 3      560 12 8
src/gui/main_window.py                    377 17 3      377 -- --
src/gui/main_tabs/main_window_surface.py  377 17 3      377 17 3
the wake hint                             4321          4321
the React page's wait                     4747          not reachable
the React box's width                     393px         393px
```

What this does not change. No description's words change. 597 controls across the Live, Paper and Simulator screens set one, 365 of those sentences run over ten words, and rewriting them waits on the definition of `unavoidable` that requirement 3 asks for.

The rule sits on the application, so all three Qt screens draw from it. No `QToolTip` rule exists under `src/gui/paper/` or `src/gui/simulator/`. The Simulator's darker tone restates the whole block under its own selector, built from the same four figures, so a box on a Simulator screen reads the same.

```
plain tone      maximum width 578   text size 12   margin 9
Simulator tone  maximum width 578   text size 12   margin 9
```

The 578 is the 560 plus the box's own 9 pixel margin on each side.

Nothing else on a screen moves. All ten style sheets, five themes on each build, are byte-identical to the ones before this change.

```
theme             characters   unchanged
cyberpunk_dark    15605        yes
neon_light        15601        yes
classic_terminal  15599        yes
minimal_modern    15611        yes
glass_metal       15618        yes
```

One box no rule can reach. 124 React controls pass a `title` property, which draws the browser's own box. That box takes no stylesheet and no wait figure, so neither declaration reaches it. Changing those sites to the page's own box is a separate job, one screen at a time.

2026-10-27 - #53 - every mouse-over description fits twenty words

The operator set the limit:

> *A longer flat limit, no exceptions.*

The figure is twenty words. Every mouse-over description meets it, and no class of control is exempt. That figure replaces `ten words unless unavoidable`, which had no definition of `unavoidable` and so gave no sentence a verdict.

The same screen was written three times. The settings tab and the creation wizard each exist as a Live file, a Paper file and a Simulator file, and 113 of their controls carried a byte-identical description in all three copies. Each copy was its own sentence, so a repair to one left the other two behind.

```
                                on origin/current   on this change
controls with a description            597 calls          597 calls
descriptions over twenty words             228                  0
copies of a shared sentence                  3                  1
shared declarations                          0                112
```

`src/gui/descriptions.py` now holds one dict per screen, keyed by the control's own name, and the Live, Paper and Simulator copies of that screen all read it. One control is left out: the indicator panel sets its own window description twice, so it has no single declaration to carry.

```
SETTINGS_TAB_DESCRIPTIONS        50 controls
BOT_WIZARD_DESCRIPTIONS          49
FOLD_TRANCHES_TAB_DESCRIPTIONS    6
TRADING_TAB_DESCRIPTIONS          3
STACK_TRANCHES_TAB_DESCRIPTIONS   2
INDICATOR_PANEL_DESCRIPTIONS      1
PHANTOM_BOTS_TAB_DESCRIPTIONS     1
```

One control keeps its own sentence on each build. The indicator panel's staleness banner says more on Live than on Paper and the Simulator, because the Live panel also draws it when a bot has no stored reading at all. Forcing one sentence on all three would lose that, so the three stay separate.

Move the one declaration and all three screens follow. The Aggressive Trading box was set to wording nothing else uses, the three settings tabs were built, and each was asked what its box reports.

```
                    this change                       on origin/current
Live                the moved wording                 the moved wording
Paper               the moved wording                 its own old sentence
Simulator           the moved wording                 its own old sentence
control, the box's label    unchanged on all three  unchanged on all three
```

On `origin/current` there is no shared declaration, so the same move reaches the Live file alone. Both files were restored and read byte-identical by sha256.

Four descriptions were wrong about the code, and each is repaired.

```
control               where it sat                    what it said
Split Distance        the three settings tabs         a Logarithmic spacing mode
Standing Surplus      the three settings tabs         the pool never drains
Profit Folding        the three settings tabs         only this control turns it off
Privacy Mode          two exchange tabs               18 masked fields
```

The spacing modes are `linear`, `quadratic`, `fibonacci` and `exponential` in `src/trading/stack_math.py`, and `src/trading/container/config.py` migrates the old `logarithmic` value to `quadratic`. The pool does drain: `src/trading/scrumming_bot.py` reduces it by whatever Max Target Growth allows and names the drained figure in its own log line. The creation wizard turns Profit Folding off for every Extractor bot, so the settings control is not the only place it is set. `src/core/privacy_mask_registry.py` holds 21 fields, not 18, so the new sentence names no count and cannot drift again.

What a shorter sentence costs. A description capped at twenty words cannot carry every clause the long one did. Each rewrite keeps the fact the control is for and drops the rest: a worked example, a figure for a default that the box already shows, or a second mechanism named in passing. The longest loss is the data pool freshness line, which fell from 79 words and no longer spells out the cache-hit arithmetic or the per-entry lifetimes.

What is still open on this item. A description on every control that has none is unbuilt, and `src/gui/tooltips.py, in set_description` still has no callers. 178 of the 597 calls build their text at runtime, so no static count reaches them.

Eight descriptions are built while the screen runs, and their fixed words ran over too. A word count of a stored sentence cannot see them, so they were counted a second way: the fixed text inside the template, with the values left out. Eight came out over twenty words, and all eight are now under.

```
                                    fixed words before   after
the Fire button, five states                22 to 26     15 to 19
the minimum rebuy guide, three tabs               41          17
```

The Fire button's five sentences lived twice — once in `src/gui/main_tabs/bot_status_table_surface.py` as a named value the React build reads, and once typed again inside `src/gui/widgets/bot_status_table.py`. The Qt table now reads the named value, so both builds draw one sentence.

```
                         this change        on origin/current
the five sentences       15 to 19 words     22 to 26 words
one declaration          yes                no, each build its own copy
```

Two runtime lines are appended to a Fire description and are not part of the sentence: a ceiling warning, and the list of gates blocking a fire.

2026-10-28 - #53 - the four live screens route every description through one helper

The operator ranks four screens as the ones he works: Trading, Charts, Bot Swarm and Market Inspector. On those screens every control now carries a description, every description fits the twenty-word figure, and every one of them is set by `src/gui/tooltips.py, in set_description`.

The one way in had no callers, so no rule could reach a single box. The helper existed and 84 calls on these four screens went straight to the toolkit instead. Each of those 84 is now a `set_description` call, so the next rule about a description is one change rather than 84.

```
                                 before   after
set_description calls in src          1     100
setToolTip calls in src/gui         603     520
on the four screens                  84       0
```

The twenty was prose, and prose reaches no screen. No module held the figure: `src/gui/theme_engine.py` declares the box's width, padding, text size and wait, and none of the four is a word count. `src/gui/tooltips.py, in DESCRIPTION_WORD_LIMIT` now holds it, `set_description` reads it on every call, and a description over it is still shown and is named in the GUI log with its word count. The screen never loses a sentence to the figure, and an over-long one stops being invisible.

What the built screens report. Each of the four was built with no window on screen, its widget tree walked, and `toolTip()` read off every control. A scrollbar and the toolkit's own tab arrows are left out; they are the toolkit's, not the product's.

```
screen                        controls   no description   over twenty words
                                         before  after    before   after
Trading                             44      10      3         7       0
Trading, one exchange layer         37       6      0        10       0
Charts                              19       1      0         3       0
Bot Swarm                           17       7      0         0       0
Market Inspector                    73      36      0         3       0
Market Inspector, topologies         3       0      0         0       0
Market Inspector, topology preview  14       9      0         1       0
total                              207      69      3        24       0
```

The three left are the indicator panel's blank padding columns. `src/gui/main_tabs/indicator_panel_surface.py, in column_titles` pads each table to a fixed column count with an empty title, so those three head nothing and describe nothing.

Where the 66 new descriptions went.

```
the Market Inspector's credential boxes      20, one format naming the field and venue
its venue buttons                            10, each naming its credential state
its ATA settings boxes                        6, one per setting a phase reads
the Bot Swarm's swarm buttons and theme menu  7
one exchange layer's fleet commands           6, each naming what SHIFT adds
the topology preview's two column sets        8
the six Add Exchange buttons on the empty-state cards
the indicator panel's bot menu, the Charts timeframe menu, New Bot, Cancel
```

Two sentence sets lived in several files, and a screen read the wrong copy. The Extractor table's eight column descriptions were declared four times — in `src/gui/main_tabs/extractor_bot_table_surface.py` and again in the Live, Paper and Simulator tables — and the topology preview typed four of its own beside the surface that already held them. Each now reads the one declaration, so the Qt screen and the browser screen draw one sentence.

```
                                        declarations before   after
Extractor column descriptions                            4       1
topology preview's bots, wires and adopt                 2       1
```

The three Extractor tables were built and each asked what its eight column headings report: all three read the same, and all three are under the figure. Control: changing one column in the surface alone makes that comparison report a difference.

The copies had already drifted. `src/gui/main_tabs/market_inspector_topologies_surface.py` still held a 31-word bots sentence and a 21-word adopt sentence while the Qt dialog beside it held the short ones, so the browser screen was over the figure and the Qt screen was not.

What a shorter sentence costs here. The indicator panel's column headings lost the most. Their long form named the author of each indicator, the raw value range, and the second threshold a gate reads where it differs from the voter's. Each heading keeps what the column is and the one threshold that acts:

```
heading   words before   after
Conf                86      19
KER                107      17
ADX                 78      19
ZSc                 73      19
Comp                56      17
Net                 49      19
```

A column heading also carries the line about pressing the label to sort, which is nine of the twenty. `src/gui/main_tabs/bot_status_table_surface.py, in COLUMN_TOOLTIPS` is therefore held to eleven words a column, and the browser build appends a privacy line of ten instead.

One defect found in the same files. `src/gui/widgets/exchange_tab.py, in ExchangeTab` installs an event filter on the application and never took it off. A layer taken off the bar stopped its feeds and left the filter in place, so every later key press reached a freed widget. `stop_feeds` now removes it, and the same teardown that raised in that filter ends clean.

The rebuild is owed. `docs/Acervator-Product-Manual.pdf` is not rebuilt here; another branch holds that file. This page, `docs/manual/08-tabs.md`, is the one that needs it.

2026-10-29 - every other screen describes every control in twenty words

Every screen the window builds now gives each control a description of twenty words or fewer. No description names a version, an issue or a pull request. The section above covers four screens in the Qt build. This section covers the rest of the window, and the browser build of every screen.

What the built screens report. Each screen was built with no window on screen, once in the Qt build and once in the browser build. Every control, table heading and tab was asked for its description. Each figure adds the two builds together. A planted description of twenty-one words reads as over in both builds, and a planted one of twenty reads as within.

screen	controls	no description		over twenty		a number	
		before	after	before	after	before	after
Window header strip	30	6	0	21	0	0	0
Window status bar	0	0	0	2	0	0	0
Main tab bar	18	18	0	0	0	0	0
Live tab	270	34	0	27	0	0	0
Charts tab	38	15	0	0	0	0	0
Inspector tab	101	0	0	3	0	0	0
Swarm tab	36	13	0	0	0	0	0
History tab	22	10	0	2	0	0	0
Status tab	80	80	0	0	0	0	0
Console tab	4	4	0	0	0	0	0
Paper tab	126	62	0	11	0	0	0
Sim tab	121	81	0	11	0	0	0
Settings, crypto	212	137	0	4	0	0	0
Settings, stocks	212	137	0	4	0	0	0
Live Bot Settings, Scrumming	194	83	0	53	0	1	0
Live Bot Settings, Extractor	118	28	0	32	0	1	0
Paper Bot Settings, Scrumming	163	54	0	53	0	1	0
Paper Bot Settings, Extractor	107	19	0	32	0	1	0
Sim Bot Settings, Scrumming	163	54	0	53	0	1	0
Sim Bot Settings, Extractor	107	19	0	32	0	1	0
Live bot wizard	151	29	0	34	0	0	0
Paper bot wizard	145	30	0	30	0	0	0
Sim bot wizard	145	30	0	30	0	0	0
Error Log	16	14	0	0	0	0	0
Start All	4	4	0	0	0	0	0
Buy confirmation	6	6	0	0	0	0	0
Fleet unlock	6	0	0	0	0	0	0
Instance consent	4	4	0	0	0	0	0
every screen	2599	971	0	434	0	6	0

The Qt build draws the Status tab's column heads as labels, so that build counts no control there. Those heads now carry the sentences the browser build draws.

One sentence, both builds. A sentence that each build typed for itself now sits once, in the screen's surface module or in `src/gui/descriptions.py`. Both builds read that one copy. The Bot Settings tables, the phantom and lock tables, the Positions Held headings, the Settings dialog and the creation wizard hold most of them.

```
declaration in src/gui/descriptions.py   on origin/current   this change
SETTINGS_TAB_DESCRIPTIONS                                50            59
BOT_WIZARD_DESCRIPTIONS                                  49            61
PHANTOM_BOTS_TAB_DESCRIPTIONS                             1             2
FOLD_TRANCHES_TAB_DESCRIPTIONS                            6             6
TRADING_TAB_DESCRIPTIONS                                  3             3
STACK_TRANCHES_TAB_DESCRIPTIONS                           2             2
INDICATOR_PANEL_DESCRIPTIONS                              2             2
```

Every description goes through one helper. The screens the window builds set each description with `src/gui/tooltips.py, in set_description`. The Settings dialog sets each control's sentence from that control's declaration, in `src/gui/settings_dialog.py, in _describe_controls`.

```
                                on origin/current   this change
set_description calls in src                  112           717
setToolTip calls in src/gui                   512            19
```

Two of the 19 are inside the helper itself, and one is the header strip card that the helper calls through. The other 16 are on screens the window does not build.

Descriptions built while the screen runs. Each description built from live values was built with the longest values its inputs can take. All 186 of them fit, and the longest is twenty words. Where a description adds lines to a sentence, `src/gui/descriptions.py, in fitted` keeps each line only while the whole stays within twenty words.

```
description                                       words before   after
Fire, held, with gates, ceiling and detonation              68      17
Fire, armed, with ceiling and detonation                    42      20
Fire, nothing to do                                         74      16
Fire, the exchange count disagrees                          82      18
bot table heading, browser build                            30      20
asset mark, no logo and no web address                      24      18
Bot Swarm node                                              21      20
Extractor tranche row                                       92      17
Extractor pool                                              27      14
Position Value, two reads                                   59      18
creation wizard information button                          71       6
```

What a shorter description costs.

- The Fire button names the gates that hold it by their gate label. It names three and counts the rest. When the gates fill the twenty words, the ceiling and detonation lines are left out. When there is room for one line, the ceiling line comes first.
- The Position Value cell says that Ammo uses the exchange's count, and gives that count. It no longer explains why the two dollar figures on the row differ.
- The Extractor tranche row names the Extractor that holds the lease and the amount leased. The alt units and the last mark follow only while they fit. The paragraph about why the row is a record and not inventory is gone.
- The Bot Swarm node reads `Growth` where it read `Realised growth`.

- The creation wizard's information button says what pressing it does. The box it opens still holds the asset's full description.

Where the readings above now differ

The readings above stand as written. This table gives what each one reads now.

written above	what it reads now
"Two runtime lines are appended to a Fire description and are not part of the sentence: a ceiling warning, and the list of gates blocking a fire."	Three lines can follow the sentence: the gates that hold it, a ceiling line and a detonation line. Each is kept only while the whole stays within twenty words.
"the five sentences 15 to 19 words"	The five sentences are 11 to 13 words, so a line can follow each one.
"A description on every control that has none is unbuilt"	Every control the window builds carries a description.
"178 of the 597 calls build their text at runtime, so no static count reaches them."	The 186 descriptions built at runtime were driven at their longest values, and none is over twenty words.
"`src/gui/tooltips.py, in DESCRIPTION_WORD_LIMIT` now holds it"	`src/gui/descriptions.py` holds the figure, and the helper imports it from there.
"The three left are the indicator panel's blank padding columns."	Each blank head sits over a pillar the other table titles, and carries that pillar's sentence.
"the browser build appends a privacy line of ten instead."	No heading carries a privacy line. The dot under the label carries the privacy state.
"SETTINGS_TAB_DESCRIPTIONS 50 controls" and the rows under it	The declaration table in this section gives the current counts.
"set_description calls in src 1 100"	717 calls, and 19 direct calls are left in `src/gui`.

The padded heads take their sentence from `src/gui/main_tabs/indicator_panel_surface.py, in column_tooltips`. A heading's sentence comes from `src/gui/main_tabs/bot_status_table_surface.py, in header_tooltip`.

The rebuild is owed. `docs/Acervator-Product-Manual.pdf` is not rebuilt here. This page, `docs/manual/08-tabs.md`, needs it.

2026-10-28 - #1374 - where the owed rebuild now stands

Where the sentence about the rebuild now reads differently

One reading above describes the state before this branch. It stands as written, and this is the reading that replaced it.

written above	what it reads now
"**The rebuild is owed.** `docs/Acervator-Product-Manual.pdf` is not rebuilt here; another branch holds that file. This page, `docs/manual/08-tabs.md`, is the one that needs it."	`docs/Acervator-Product-Manual.pdf` is rebuilt in this branch from the merged pages, and it carries this page

`tools/build_product_manual.py` is what builds it, and the page order it reads is the order of `docs/manual`.

Subsystem Detail

Reference. One file per subsystem tab, written from the source. Each file names the module that implements the screen, the symbols inside it, the bridge method that serves its renderer, and the screen's current state.

The parent part is 08-tabs.md, which points here for every screen but Main Window, Market Inspector and Asset Charts, and carries the manual's own text for those three. The rows below run in the order the tab list in 04-manual-parts.md sets, which is the order 08-tabs.md runs its sections in.

How an update is written down

Each tab has one section, and that section is its file below. An update to a tab takes a dated subsection under that one heading and never a second section named for the tab.

```
    ## 2026-09-08 14:20 - #407, #442 - the six zones and their arrows
       date       time    issues addressed   what the update did
```

The date leads so the subsections sort, and they run forward down the page. An update with no issue behind it writes `no issue recorded` in that field rather than a number nothing supports. `.claude/rules/documentation.md` bans an issue number in documentation, and the operator's directive of 2026-09-08 amends it for this header alone; version names stay banned everywhere.

`dev_harness/harness/docs_archetype.py` refuses both halves

```
rule_id="DOC011"   # two contents sections naming one tab
rule_id="DOC012"   # a dated update that runs backwards
```

Contents

File	Covers	State
portfolio-panels.md	The header strip: spendable columns, five counter cards, privacy dots	Live
simulator.md	Live's tab forked over Stone Tablets: Validation, Back Test, Portfolio Battery, the gate-latch criterion, the replay layer	Live, rebuilt under issue #117
paper-trader.md	The live feed, the fake balance, the paper log	Live — issue #19
proof-of-accumulation.md	Identity, Merkle log, competitions, ACRV, the local chain	Skeleton tab, engine only — issue #147
market-inspector.md	Higher-timeframe scanner, opposing pairs, topology proposals, adopt	Live
bot-swarm.md	Nodes, Smart Wires, wire credits, the fold-tranche book	Live
asset-charts.md	One candlestick panel per traded symbol	Live
history.md	Venue trade history, grading, gate analysis	Live
console.md	Python log tail and the emitter signal stream	Live
system-status.md	Emitter Network and Watchdog, the two halves that run today	Skeleton tab — issue #34
settings.md	The User page, the Exchanges page, and what each of the eleven pages persists	Live
alerts.md	The two channel forms, the Event Routing table, the notification history	Both builds complete, the window opens neither
promotion-pipeline.md	How a strategy earns its way to real money	Four of four steps built

The live tab set

One list names the ten tabs the window builds, and each one is moved into that position after the builders run. Two of them are empty: Accumulation and Status.

`src/gui/main_tabs/main_window_surface.py` — `CANONICAL_TAB_ORDER`

```
CANONICAL_TAB_ORDER = (
    SIM_TAB,
    PAPER_TAB,
    LIVE_TAB,
    CHARTS_TAB,
    INSPECTOR_TAB,
    SWARM_TAB,
    ACCUMULATION_TAB,
    HISTORY_TAB,
    STATUS_TAB,
    CONSOLE_TAB,
)
```

The Live tab has its own section: 06-trading-tab.md. The Indicator Voting Panel has 07-indicators.md.

Every screen the window does not build gets a sentinel instead of a widget, so a legacy reader gets that sentinel rather than an error. The list covers the Competition and Local Testnet tabs, the analytics, risk, journal, alerts and audio pages, and the retired Paper Trader widget, which is not the Paper tab issue #19 built.

`src/gui/main_tabs/retired_tabs.py` — `RetiredTabsMixin._install_retired_tab_sentinels`

```
  def _install_retired_tab_sentinels(self) -> None:
      """Assign None to ``_paper_trader``, ``_competition_tab`` and the rest.

      ``main_window`` tests ``_analytics_tab``, ``_risk_tab``,
      ``_journal_tab`` and ``_alerts_tab`` before refreshing them.
      """
```

The underlying engines behind several of those keep running; only the surface is gone.

Read on the current tree, the engines that ran for nothing are stopped. Of the engines behind those pages, the window builds two: the risk engine and the notification manager. The risk engine still runs every 2 seconds, and its drawdown alerts still reach the Console and SMS. The audio suite's chime still plays from the Market Inspector.

`src/gui/main_window.py` — `MainWindow.__init__`, the two engines the window builds

```
              from ..trading.risk_manager import RiskManager
              from ..core.notifications import get_notification_manager

              self._risk_manager = RiskManager(bot_manager)
              self._notif_manager = get_notification_manager()
```

engine	before	now
analytics	built at launch, a snapshot about every 10 s	not built
testnet bridge	installed at launch, a 250 ms timer for the whole session	not installed
trade journal	built at launch, three day files read	built only when an AI note is written
reconciliation engine	built at launch, never called	not built
crash snapshot	a whole-fleet file written every 30 s	not built
risk engine	two fleet sweeps every 2 s	one sweep

When a risk rule raises its alert

Each of the seven risk rules raises its alert only for a reading strictly beyond its limit. A reading exactly on the limit raises none. Every alert text says the reading "exceeds" the limit, and every alert's reading does. The one test sits on the rule, and every rule calls it.

`src/trading/risk_manager.py` — `RiskRule.exceeded_by`

```
    def exceeded_by(self, reading: float) -> bool:
        """True only for a ``reading`` strictly above ``threshold``; ``evaluate``
        applies this one test to every rule.
        """
        return reading > self.threshold
```

Every alert writes one warning line to the Console and to `system.log`. A critical alert also goes to the notification manager, which sends it to the alert sound, Telegram and SMS. SMS sends only while the "P/L threshold alerts" box on `Settings > SMS` is ticked.

rule	limit	what it reads	severity
`max_drawdown`	10 %	the fall of total P/L from its peak	warning
`critical_drawdown`	25 %	the fall of total P/L from its peak	critical
`per_bot_loss`	$50	one bot's realised loss	warning
`asset_concentration`	40 %	one asset's share of the running target balance	warning
`exchange_concentration`	60 %	one venue's share of the running target balance	warning
`correlated_exposure`	50 %	the combined share of a pair at a correlation of 0.7 or more	warning
`rapid_loss`	5 %	the fall of total P/L inside 300 s	critical

Driven on constructed fleets in a redirected home, with every send stopped at its import. One step is 0.0001 of P/L, the place each bot's status rounds to, or one cent of target balance.

```
rule                    one step below   on the limit   one step above
max_drawdown            no alert         no alert       warning
critical_drawdown       no alert         no alert       critical, to SMS
per_bot_loss            no alert         no alert       warning
asset_concentration     no alert         no alert       warning
exchange_concentration  no alert         no alert       warning
correlated_exposure     no alert         no alert       warning
rapid_loss              no alert         no alert       critical, to SMS
```

A rule that alerts waits out its cooldown before it alerts again: 300 s, and 600 s for `rapid_loss`. A reading on the limit raises nothing, so it starts no cooldown. Another asset beyond the limit in the same sweep then raises the alert for that rule.

How a screen reaches its renderer

One registry maps a method name to a view-model function. A surface is a plain function that takes the request's parameters and returns a serialisable result, and a screen is reachable exactly when its method appears in that registry.

`src/core/desktop_bridge.py` — `build_registry`

```
def build_registry() -> Dict[str, Handler]:
    """Return the methods the frontend may call, keyed by method name.

    This is the wiring point for the whole frontend: a surface is
    reachable exactly when it appears here. Surfaces are imported inside
    the function so that the transport above carries no dependency on any
    one domain package.
    """
```

The seven surfaces behind the retired tabs are in that registry, and building the registry imports none of them. Each one is imported on the first request for its own method, so a session that never asks for a retired page never loads its surface.

`src/gui/main_tabs/retired_surfaces.py` — `on_demand_handlers`

```
def on_demand_handlers() -> dict[str, Callable[[dict], dict]]:
    """Return one handler per ``SURFACE_MODULES`` method, unimported until called."""
    return {method: _handler(module) for method, module in SURFACE_MODULES.items()}
```

Read on the current tree, a packaged build carries none of the seven surfaces. `src/gui/main_tabs/retired_surfaces.py, in on_demand_handlers` registers a page only when the running program can find its surface, so a packaged build's registry holds none of the seven methods. A request for one gets the bridge's `UnknownMethod` refusal, and no import runs. A run from the source folder still answers all seven.

`src/gui/main_tabs/retired_surfaces.py` — `on_demand_handlers` and `_is_carried`, as they read now

```
def on_demand_handlers() -> dict[str, Callable[[dict], dict]]:
    """Return one handler per ``SURFACE_MODULES`` method, unimported until called.

    A method whose module ``_is_carried`` cannot find gets no handler.
    """
    return {
        method: _handler(module)
        for method, module in SURFACE_MODULES.items()
        if _is_carried(module)
    }


def _is_carried(module_name: str) -> bool:
    """Return True when ``find_spec`` finds ``module_name`` beside this module."""
    return importlib.util.find_spec("." + module_name, __package__) is not None
```

The rule that leaves the seven out of the build is on 17-what-the-build-includes.md, under "A screen the window never builds".

The transport is one JSON object per line over the pipe the parent process already owns: no socket and no port.

Each surface module holds the screen as data. Nothing in one imports Qt, which is why the same view model serves the Qt tab and the renderer without a second description of the screen existing.

Thirty-three surface modules wrote their values out a second time rather than reading them from the module that declares them. Four of them now read that one declaration. The design system table and the theme engine import no Qt, so a surface that imports none can read either.

`src/gui/main_tabs/design_system_surface.py`, in the module body

```
TOKENS: dict[str, Any] = {
    name: getattr(design_system, name) for name in TOKEN_NAMES
}
```

The rest keep their own copy. Each names a Qt source, and a surface reaches no Qt source, so no read of the one declaration is available to them. Their docstrings state what the module does instead: nothing compares the two copies.

Back to the manual index.

Portfolio Information Panels

Reference. The strip above the tab bar, present on every tab except the two isolated ones.

What builds it

One method builds the strip. It makes the window's central widget, lays a single row across the top of it, and hands back the layout the tab row goes into. That row holds the five spendable columns on the left and the five counter cards on the right, and it does not move as you change tabs.

`src/gui/main_tabs/header_strip.py` — `HeaderStripMixin._build_header_strip`

```
self._spendable_widget = SpendableProfitsWidget()
top_row.addWidget(self._spendable_widget, stretch=3)
```

The row draws seven slots: the money strip, the five counter cards, and the asset class square that ends it. The first six divide the whole row between them, at three shares for the money strip and one for each card. The square takes no share, so its slot shrinks to the square and the square draws hard against the row's right edge. No background is left anywhere in the row.

`src/gui/main_tabs/header_strip_surface.py`

```
TOP_ROW_STRETCH = [3, 1, 1, 1, 1, 1, 0]
```

Overtaken, quoted whole: "TOP_ROW_STRETCH = [3, 1, 1, 1, 1, 1, 0]".

True today: `TOP_ROW_STRETCH` holds eight shares, `[6, 1, 1, 1, 1, 1, 1, 0]`, and is not written out. `header_strip_surface.py` builds it from `KPI_COLUMNS` and `COUNTER_CARDS` — one share per money column the strip holds, `COUNTER_STRETCH` for each card, and `CLASS_GROUP_STRETCH` for the class square. Six columns and six cards answer the six, the six ones and the nought, so the list carries one share per

TOP_ROW_ORDER slot by construction. The ACCUMULATED card is the sixth, added after this page was written.

Each of the six also carries a floor, the width its own whole amounts need, so a long figure is never cut.

`src/gui/main_tabs/header_strip.py` — `HeaderStripMixin._build_header_strip`

```
  card.setMinimumWidth(surface.slot_natural_w(slot))
```

Overtaken, quoted whole: "The first six divide the whole row between them, at three shares for the money strip and one for each card."

True today: the money strip's three shares come out below its own floor at every window up to about 1950 pixels wide, so it draws at that floor and the five cards take the whole surplus between them.

Both builds draw the row at the same geometry. The floor the Qt row sets on a widget is the number the React page sets on the same slot, and the surface serves that number to both.

`src/gui/main_tabs/header_strip_surface.py`

```
  "slots": {slot: slot_natural_w(slot) for slot in TOP_ROW_ORDER}
```

`src/gui/web/header_strip.js`

```
  style.minWidth = length(floor);
```

Measured at a 1920 pixel window, in both builds: the money strip 686, each card 217, the square 112.

Overtaken, quoted whole: the two code blocks above, which name `slot_natural_w` at both sites, and "Measured at a 1920 pixel window, in both builds: the money strip 686, each card 217, the square 112."

True today: both sites read `slot_floor_w`, which answers `slot_natural_w` for the money strip and `slot_min_w` for every other slot. `top_row_min_w` adds up the same `slot_floor_w`, so the narrowest window the budget names is the narrowest the row can draw at — 1384 pixels, which is what Qt reports for the real window.

`src/gui/main_tabs/header_strip.py` — `HeaderStripMixin._build_header_strip`

```
  card.setMinimumWidth(surface.slot_floor_w(model["key"]))
```

`src/gui/main_tabs/header_strip_surface.py` — `width_budget`

```
  "slots": {slot: slot_floor_w(slot) for slot in TOP_ROW_ORDER}
```

Read off the real row on the Qt build, with the machine's own fonts, at a 1920 pixel window: the money strip 866, each card 144 or 145, the square 148. At 1400 the strip reads 668, a card 90 or 91, and the square 148.

Left: the spendable columns

Five labelled columns fill the left half, divided by thin vertical rules.

Column	Meaning
SPENDABLE	Cash the exchange reports, across the bots sharing one wallet
REALISED	Realised profit and loss
LOCKED	Value the open positions hold
MATURE	Wire profit old enough to move
EXCH	Connected exchange count

The widget builds four of the five from one list, and each entry carries the label the operator reads and the payload key that fills it.

`src/gui/widgets/spendable_profits.py` — `SpendableProfitsWidget.__init__`

```
  for label_text, key in [
      ("REALISED", "total_realised"),
      ("LOCKED", "locked"),
      ("MATURE", "mature"),
      ("EXCH", "exchanges"),
  ]:
```

Overtaken, quoted whole: the five-row table above, and the four labels in the block above it.

True today: the strip draws six columns and REALISED is not one of them. `header_strip_surface.py, in KPI_COLUMNS` declares SPENDABLE, P/L, LOCKED, MATURE, EXCH and AMMO. `spendable_profits_surface.py, in COLUMNS` declares the same six, and `spendable_profits.py, in SpendableProfitsWidget._setup_ui` builds one column per entry of that list in place of the typed four. The name REALISED is spelled in neither file. P/L carries the only profit and loss figure the strip draws, and it is the unrealised figure the exchange answers.

One payload draws all five. A value the payload leaves out draws an em dash rather than a zero, which keeps "no reading" and "a reading of nothing" apart. That is the difference between a column with nothing behind it and a column reporting a genuine zero, and on this strip it matters: two of the five have nothing behind them today.

`src/gui/widgets/spendable_profits.py` — `SpendableProfitsWidget._money_text`

```
  @staticmethod
  def _money_text(value) -> str:
      """Render one amount as money text, or as the empty marker."""
      amount = SpendableProfitsWidget._amount_of(value)
      if amount is None:
          return "—"
      return f"${amount:,.2f}"
```

REALISED and MATURE are the two. Both are handed a literal absence at the one call site that fills the strip, so both draw the marker on every tick. The Trading tab section carries the proposal for the first of them: 06-trading-tab.md.

Both now carry a figure, and the figure comes from the exchange. Realised is the venue's own matched profit and loss across the fleet. Mature is the profit held in positions worth more than three times what they cost,

which is the growth threshold the platform applies everywhere it decides maturity.

Overtaken: "Realised is the venue's own matched profit and loss across the fleet."

The venue answers a cost basis, an average entry price and an unrealised profit per spot position, and no lifetime realised figure. Realised is the platform's own first in, first out match over the venue's own fills, one figure per bot, added across the fleet.

Overtaken: "Mature is the profit held in positions worth more than three times what they cost, which is the growth threshold the platform applies everywhere it decides maturity."

Mature is the part of a position's value that sits over three times its cost. A position worth $350 on a $100 cost basis holds $50 of mature profit. Three times the cost is still the threshold the platform applies everywhere it decides maturity.

`src/gui/main_tabs/header_strip_surface.py` — `profits_payload`

```
  "total_realised": exchange_amount(data, "total_realized_exchange"),
  "mature": exchange_amount(data, "total_mature_exchange"),
```

Absence is still a reading. When the venue has answered for no bot, both columns take nothing and draw the marker, and the mask leaves that marker alone. A hidden figure shows four asterisks; an empty column stays empty under Privacy Mode.

`src/core/privacy_mask_registry.py` — `mask_or`

```
  text = str(value)
  if field_id not in ALL_FIELD_IDS or text == ABSENT_TEXT:
      return text
```

What the venue answers, and what the platform derives

The venue's portfolio breakdown answers three figures per open spot position: the cost basis, the average entry price and the unrealised profit. Each bot reads those three from the venue on every refresh and holds them as its own. The breakdown carries no lifetime realised figure for a spot position, so Realised is derived: every fill the venue holds for the bot's symbol, matched first in, first out, one figure per bot, summed across the fleet.

`src/exchange/ccxt_connector.py` — `get_spot_positions`

```
  portfolios = await self._call_sync(self._ex.fetch_portfolios)
  rows = await self._call_sync(self._ex.fetch_portfolio_details, uuid)
  basis = float(row.get("cost_basis", 0) or 0)
```

The venue's cost basis is the check on the derivation. After the fills are matched, the bot logs the venue's cost basis beside the cost of the buys FIFO left open and beside the average-cost basis, with each gap in per cent, and names the closer of the two. A venue whose account uses another matching method shows up there as a gap on both.

`src/trading/scrumming/reconciliation.py` — `_log_basis_check`

```
_fifo_gap = abs(_fifo - float(venue_basis)) / _scale
_avg_gap = abs(_avg - float(venue_basis)) / _scale
_closer = "fifo" if _fifo_gap <= _avg_gap else "average"
```

A venue without a portfolio breakdown leaves the three figures to the same derivation, and the check does not run.

`src/exchange/position_health.py` — `compute_position_health`

```
while sell_remaining > 1e-12 and buy_queue:
    buy_lot = buy_queue[0]
    take = min(buy_lot[0], sell_remaining)
    realized += (t.price - buy_lot[1]) * take
```

A sell is matched against the oldest open buy, so a sell placed above the latest fold and below the first buy of a market that has fallen since lowers the figure. Over the 2026 fill export, 1,302 of 2,653 sells lowered it and 66 were priced under the most recent buy.

Each bot holds the complete fill history for its symbol. The first refresh pages the venue newest to oldest until a page comes back short; every later refresh fetches one page and stops at the first fill it already holds.

`src/exchange/fill_history.py` — `FillHistory.refresh`

```
if len(fills) < FILL_PAGE_LIMIT:
    ended_short = True
    break
if joined:
    break
```

Before this, one call fetched the newest 500 fills and the figure was FIFO over that window alone. On the two symbols past 500 fills the window read $-137.38 and $119.91 where the complete history reads $-390.53 and $60.27.

`src/trading/scrumming/reconciliation.py` — `refresh_exchange_position_health`

```
_trades = await self.fetch_fill_history()
if _trades is None:
    return False
_asset_base = self.config.symbol.split("/")[0]
_ph = compute_position_health(_trades, _asset_base)
```

Mature applies one constant. A position counts once its value reaches its cost plus two hundred per cent of its cost, and the column sums the profit on the positions that qualify.

`src/trading/smart_wire.py` — `mature_profit_usd`

```
MATURE_GROWTH_PCT: float = 200.0
if value < basis * (1.0 + MATURE_GROWTH_PCT / 100.0):
    return 0.0
return value - basis
```

Overtaken: "the column sums the profit on the positions that qualify."

The column adds up what each qualifying position holds over the threshold. `mature_threshold_usd` is the cost plus two hundred per cent of the cost, and `mature_profit_usd` answers the value above it.

`src/trading/smart_wire.py` — `mature_profit_usd`

```
if not is_mature(cost_basis_usd, current_value_usd):
    return 0.0
return float(current_value_usd) - mature_threshold_usd(cost_basis_usd)
```

A position at three times its cost exactly is mature and holds nothing over the threshold. `is_mature` carries that test on its own, so the count of mature positions reads the position while the money column adds nothing for it.

`src/trading/smart_wire.py` — `is_mature`

```
threshold = mature_threshold_usd(cost_basis_usd)
if threshold <= 0.0:
    return False
return math.isfinite(value) and value >= threshold
```

Realised reads the same on every start

A bot writes its realised figure only when the walk reached the end of the venue's history. A walk that stops short keeps the last whole reading, writes `stats.fill_history_complete` False and says so in the log, so the same fills always answer the same figure.

`src/trading/scrumming/reconciliation.py` — `refresh_exchange_position_health`

```
_complete = bool(getattr(self._fill_history, "complete", False))
self.stats.fill_history_complete = _complete
if _complete:
    self.stats.realized_pnl_exchange = float(_ph.realized_pnl_usd)
```

The fleet aggregate answers `realised_history_complete` False while any bot the venue answered for reads False, and the column then draws the empty marker. An absent figure never looks like a present one.

`src/gui/main_tabs/header_strip_surface.py` — `realised_amount`

```
if not bool(data.get(REALISED_COMPLETE_KEY, True)):
    return None
```

The Paper and Simulator fleets walk no venue history, so their aggregates omit the key and their realised figure passes through whole.

Right: the five counter cards

Five cards close the row, and each one counts a single thing.

Card	What it counts
Scrummed	Cumulative USD sold across every bot since the process started
Folded	Cumulative USD bought across every bot since the process started
Trades	Executed buys and sells across the active bots
Bots	Bots in the RUNNING state
Errors	Errors across every bot since the last reset

Errors is the one card you can click. Arming it takes one call, and the click opens the rolling error buffer with its Reset button.

`src/gui/main_tabs/header_strip.py` — `HeaderStripMixin._build_header_strip`

```
self._stat_errors.set_clickable(True, "Click to open the error log.")
self._stat_errors.clicked.connect(self._show_error_log_dialog)
```

Current reading, 2026-10-29. The card takes no words of its own when it is made clickable. Its description and its click line are declared beside it.

`src/gui/main_tabs/header_strip_surface.py` — `ERRORS_CLICK_TOOLTIP`

```
ERRORS_TOOLTIP = "Errors across all bots since the last reset."
ERRORS_CLICK_TOOLTIP = "Press to open the Error Log, where Reset clears them."
```

Issue #428 carries a disagreement between what the tooltips on these cards promise and what the cards draw.

The issue #428 clause above is overtaken. It is not edited and not removed. It read:

> *Issue #428 carries a disagreement between what the tooltips on these cards promise and what the cards draw.*

Issue #428 closed on 2026-10-03. The two counter tooltips name the exchange's own trade history, which is the source the cards draw from, and both say the figure survives a restart.

```
src/gui/main_tabs/header_strip_surface.py

SCRUMMED_TOOLTIP   "the dollars every bot has sold so far this year ...
                    comes from the exchange's own trade history, so the venue
                    and the card agree ... It survives a restart."
FOLDED_TOOLTIP     the same sentence for buys
```

Current reading, 2026-10-29. Both counter descriptions are shorter, within twenty words. They keep the source and the restart.

`src/gui/main_tabs/header_strip_surface.py` — `SCRUMMED_TOOLTIP`

```
SCRUMMED_TOOLTIP = (
    "Dollars every bot has sold this year, read from the exchange's own trade "
    "history. It survives a restart."
)
```

A sixth card, P/L, is constructed and then hidden. Code that still writes to it keeps working and nothing appears on screen.

`src/gui/main_tabs/header_strip.py` — `HeaderStripMixin._build_header_strip`

```
self._stat_pnl = StatCard("P/L", "$0.00")
self._stat_pnl.setVisible(False)
```

The mode button on the right ends the row. `_toggle_trading_mode` swaps the window between the crypto and stock layers.

Overtaken, quoted whole: "The mode button on the right ends the row. `_toggle_trading_mode` swaps the window between the crypto and stock layers."

True today: one square ends the row. It is segmented two by two into one button per asset class, and a press makes that class active. The square keeps its own size, never stretches, and sits in the row's right corner.

`src/gui/main_tabs/header_strip.py` — `HeaderStripMixin._build_header_strip`

```
top_row.addWidget(
    class_group,
    surface.slot_stretch("mode_button"),
    Qt.AlignVCenter | Qt.AlignRight,
)
```

Privacy

Every field on the strip carries its own privacy dot. Attaching one registers the field, and from then on the card draws its value through the mask.

`src/gui/main_tabs/header_strip.py` — `HeaderStripMixin._build_header_strip`

```
self._stat_scrummed.attach_privacy_dot("counter.scrummed")
self._stat_folded.attach_privacy_dot("counter.folded")
self._stat_trades.attach_privacy_dot("counter.trades")
self._stat_bots.attach_privacy_dot("counter.bots")
self._stat_errors.attach_privacy_dot("counter.errors")
```

The registry behind those field ids is shared. Hide a field here and the same field hides on the Bot Swarm tab, because both surfaces ask the one registry.

`src/core/privacy_mask_registry.py` — `mask_or`

```
def mask_or(value, field_id: str, mask: str = "****") -> str:
    """Return ``mask`` when ``field_id`` is masked, else ``str(value)``.

    A ``field_id`` outside ``ALL_FIELD_IDS`` never masks, which covers every
    ``TA_FIELD_IDS_EXCLUDED`` entry.
    """
```

Where the numbers come from

One aggregate call fills the whole strip, once a tick. Every card and every column is written from that single answer, so two cards cannot disagree about the same fleet.

`src/gui/main_window.py` — `_refresh_dashboard`

```
agg = self._bot_manager.get_aggregate_stats()
_scr = float(agg.get("total_scrummed_usd", 0.0) or 0.0)
_fld = float(agg.get("total_folded_usd", 0.0) or 0.0)
self._stat_scrummed.set_value(f"${_scr:,.2f}")
self._stat_folded.set_value(f"${_fld:,.2f}")
```

The aggregate itself is `FleetAggregationMixin.get_aggregate_stats` in `src/trading/container/aggregation.py`.

Hiding

Two screens are isolated from live trading, and on those two the strip hides itself. The absent strip is the signal: no live numbers are on this screen.

`src/gui/main_window.py` — `_on_main_tab_changed`

```
isolated_tabs = set(ISOLATED_TABS)
container = getattr(self, "_header_strip_container", None)
if container is not None:
    container.setVisible(tab_name not in isolated_tabs)
```

The window surface carries the same pair as a constant, and issue #450 renamed both labels.

`src/gui/main_tabs/main_window_surface.py` — `ISOLATED_TABS`

```
ISOLATED_TABS = (SIM_TAB, PAPER_TAB)
```

The Simulator draws its own strip in place of this one. `SimStatStrip` in `sim_stat_strip` mirrors these ten fields against sim balances. The Simulator rebuild removed this file; it is not in the tree.

The strip stays on Sim and reads the sim fleet

The Simulator rebuild took the Sim tab out of the isolated pair, so the strip hides on Paper alone. On Sim the same ten fields carry the Simulator's figures, read from the Sim tab's fleet source, and the live fleet's numbers do not reach the strip while Sim is in front. The window picks the fleet on every tick and on every tab change.

`src/gui/main_tabs/main_window_surface.py` — the two tuples

```
ISOLATED_TABS = (PAPER_TAB,)

#: The tabs the header strip reads the Simulator's fleet on, not the live one.
SIM_FED_TABS = (SIM_TAB,)
```

`src/gui/main_window.py` — `_refresh_header_strip`

```
if self._header_strip_reads_sim():
    sim_tab = self._simulator_tab
    self._write_header_strip(
        sim_tab.fleet_source().aggregate(), sim_tab.exchange_count()
    )
    return
```

simulator.md covers what each field reads on Sim.

Bridge

The strip answers one bridge method, and one renderer module draws it.

`src/gui/main_tabs/header_strip_surface.py` — `METHOD`

```
METHOD = "header.strip"
```

The renderer module is `header_strip.js`.

Back to the subsystem index.

Simulator Tab

Reference. The second step of the promotion pipeline: the live fleet, replayed against stored history. Issue #117 rebuilt the screen, and it carries three modes: Validation, Back Test and Portfolio Battery.

What the tab holds today

The Sim tab is the Live tab's code, forked and modified, reading Stone Tablets and the YTD trade files instead of a venue. It sits first on the bar. The Qt build draws `SimTradingTab` and the React build draws `SimTradingTabReact`, which hosts one page module and six child modules forked from Live's. This section describes the tab in the order it draws, read off the running program in both builds on the current commit. The sections under "How each part was built" are each unit's own record, and the sections under "The widget the rebuild replaced" describe the screens that are gone.

`src/gui/variant_surface.py` — the two loaders

```
def _qt_simulator() -> type:
    """Import and return the Qt Sim tab, ``SimTradingTab``."""
    from .simulator.sim_trading_tab import SimTradingTab

def _react_simulator() -> type:
    """Import and return the React Sim tab, ``SimTradingTabReact``."""
    from .simulator.sim_react_trading_tab import SimTradingTabReact
```

The header strip over the tab

The window's header strip stays in view with Sim in front, and it reads the Simulator's fleet rather than the live one. Only Paper is isolated. The five columns and the six cards are Live's: with nothing held the columns read an em dash and the cards read zero; with a fleet held, the cards read Live's arithmetic over the held records, and EXCH counts the seated venues.

`src/gui/main_tabs/main_window_surface.py` — the tabs the strip reads from

```
ISOLATED_TABS = (PAPER_TAB,)

#: The tabs the header strip reads the Simulator's fleet on, not the live one.
SIM_FED_TABS = (SIM_TAB,)
```

Spendable follows the run mode. In Validation and Portfolio Battery it is the run's budget, the sum of the held Target Balances, because no buy is refused for cash in those two modes. In Back Test it is Live's wallet arithmetic over the held bots' own cash.

`src/gui/main_tabs/simulator_tab_surface.py` — the wallet by mode

```
MODE_FUNDING = {
    MODE_VALIDATION: back_test.FUNDED_BY_TARGETS,
    MODE_BACK_TEST: back_test.FUNDED_BY_PROCEEDS,
    MODE_PORTFOLIO_BATTERY: back_test.FUNDED_BY_TARGETS,
}
```

The venue stack and the Get Started card

The Simulator starts empty. At open no venue is seated and the Get Started card shows in the venue pane. The card asks for a first fleet, says a fleet is what begins a run, and offers the two ways in of the active mode, which is Validation at open. Import Live Fleet copies the stored fleet's records for one exchange, and the venue stack then seats one sub-tab per exchange the held fleet names. Every fleet change seats the stack again.

`src/gui/simulator/sim_trading_tab.py` — the seating

```
    def _sync_exchange_tabs(self) -> None:
        wanted = [str(eid) for eid in self._fleet_source.exchanges() if eid]
        for eid in wanted:
            self.add_exchange_tab(eid, exchange_display_name({"exchange_id": eid}))
        self._drop_unlisted_exchange_tabs(wanted)
        self.refresh_bots()
        self.refresh_votes()
        self._refresh_replay()
```

Since one fleet per mode landed, the fleet the stack seats is the active mode's. The tab's fleet source holds three fleets, one for Validation, one for Back Test and one for Portfolio Battery, and every reader and every act works on the fleet of the mode in force. A mode press moves the source's mode and fires the same fleet change, so the venues, the rows, the strip and the way-in row redraw from that mode's fleet; a venue that only one mode's fleet names unseats when another mode is shown and seats again on return. A mode whose fleet is empty shows the Get Started card, and the card holds the three mode buttons above its two ways in, so the

operator can leave an empty mode from the card. The section "One fleet per mode" under "How each part was built" describes the change.

`src/simulator/fleet_source.py` — the fleet in force

```
    @property
    def _records(self) -> dict[str, dict]:
        """The records map of the fleet in force, ``_fleets`` under ``_mode``."""
        return self._fleets[self._mode]

    def set_mode(self, mode: str) -> str:
        """Make ``mode`` the fleet every reader and act works on; a name
        outside ``MODES`` changes nothing. Answers the mode in force."""
        if mode in MODES:
            self._mode = str(mode)
        return self._mode
```

The mode row and the corner

The row above the bot list holds the three mode buttons, Validation, Back Test and Portfolio Battery, then `+ New Bot`. The active mode's button wears Live's Privacy Mode ON sheet and the other two the OFF sheet. The data-pool row under it holds nothing. The venue tab bar's corner holds the active mode's two ways in, and the Get Started card holds the same two. Validation offers Import Live Fleet and Generate From YTD. Back Test offers Import Live Fleet and Create New Bots. Portfolio Battery offers Run Portfolio and Run Every Portfolio. The mode row lives on the venue page, so before a fleet is held the card offers Validation's two.

`src/gui/main_tabs/simulator_tab_surface.py` — the modes and their ways in

```
MODES = (MODE_VALIDATION, MODE_BACK_TEST, MODE_PORTFOLIO_BATTERY)

ROWS_FOR_MODE = {
    MODE_VALIDATION: RESERVED_ROWS,
    MODE_BACK_TEST: BACK_TEST_ROWS,
    MODE_PORTFOLIO_BATTERY: BATTERY_ROWS,
}
```

The three mode names and their tuple now live in the fleet source, because the sim fleet file is keyed by them; the surface imports the same four names, so the block above reads the same values from `src/simulator/fleet_source.py`. A mode press does more than restyle the row: the host hands the mode to the source, then fires the fleet change, and the tab draws that mode's fleet. A press while a run or a Battery is in flight writes the in-flight line and changes nothing. The card under the mode row holds the same three buttons.

`src/gui/simulator/sim_trading_tab.py` — the press

```
        self._fleet_source.set_mode(mode)
        for venue in list(self._crypto_exchange_tabs.values()) + list(
            self._stock_exchange_tabs.values()
        ):
            venue.show_mode(mode)
        self.fleet_changed.emit()
```

The corner's first button, in every mode, is Clear Fleet. It sits to the left of the mode's two ways in, at their size. A press empties the held fleet: every record on every venue leaves, the venues unseat, the Get Started card returns, and the sim fleet file is written with no record. The card holds Clear Fleet above its two ways in only while a fleet is held; with nothing held the card offers the two ways in alone. The section "Clear Fleet empties the held fleet" under "How each part was built" describes the press.

Since the way-in row landed, Clear Fleet and the two ways in draw on a row above the exchange tab row, not in the corner, and only while a fleet is held; the paragraph after the next block describes that row.

`src/gui/simulator/sim_trading_tab_surface.py` — the corner's list

```
def way_in_buttons(mode: str = sim.MODES[0]) -> list:
    """The corner's buttons as the page draws them: ``clear_fleet_button``
    first, then the two ways in ``mode`` offers from ``sim.reserved_rows``."""
    return [clear_fleet_button()] + [
        {
            "action": row["action"],
            "text": row["text"],
            "accessible_name": row["button_name"],
            "minimum_width_px": live.ADD_BUTTON_MIN_WIDTH_PX,
            "minimum_height_px": WAY_IN_BUTTON_HEIGHT_PX,
        }
        for row in sim.reserved_rows(mode)
    ]
```

The buttons the corner held draw on their own row above the exchange tab row: Clear Fleet, the mode's two ways in, then Start Run, in that order from the left, each at the corner button's size. The corner itself holds nothing, so the exchange tabs take the full row and a third venue's tab has room. The row draws only while a fleet is held. With no fleet held the row is absent and the Get Started card offers the mode's two ways in. A press on Start Run starts the active mode's run over the fleet on the venue shown: the Validation rerun, the Back Test walk, or the Portfolio Battery chooser and its run. The card never holds Start Run. The section "Start Run starts the active mode's run from the way-in row" under "How each part was built" describes the row and the press.

`src/gui/simulator/sim_trading_tab.py` — the row above the tab bar

```
            way_in_row_widget = QWidget()
            way_in_row = QHBoxLayout(way_in_row_widget)
            way_in_row.setContentsMargins(*tab_surface.WAY_IN_ROW_LAYOUT["margins_px"])
            way_in_row.setSpacing(tab_surface.WAY_IN_ROW_LAYOUT["spacing_px"])
            way_in_row.addStretch(1)
            way_in_row_widget.setVisible(False)
            page_layout.addWidget(way_in_row_widget)
```

`src/gui/simulator/sim_trading_tab_surface.py` — the row's last button

```
def start_run_button() -> dict:
    """The corner's Start Run button, at the way-in buttons' size."""
    return {
        "action": sim.START_RUN_ACTION,
        "text": sim.START_RUN_TEXT,
        "accessible_name": sim.button_name(sim.START_RUN_ACTION),
        "minimum_width_px": live.ADD_BUTTON_MIN_WIDTH_PX,
        "minimum_height_px": WAY_IN_BUTTON_HEIGHT_PX,
    }
```

The row's list takes the held count and is empty at zero, so the row is not drawn; above zero it is Clear Fleet, the mode's two and Start Run.

```
    if int(held or 0) <= 0:
        return []
    return (
        [clear_fleet_button()]
        + [
            {
                "action": row["action"],
                "text": row["text"],
                "accessible_name": row["button_name"],
                "minimum_width_px": live.ADD_BUTTON_MIN_WIDTH_PX,
                "minimum_height_px": WAY_IN_BUTTON_HEIGHT_PX,
            }
            for row in sim.reserved_rows(mode)
        ]
        + [start_run_button()]
    )
```

The two tables and the command bar

The Scrumming Bots table is the forked `SimBotStatusTable`. It hides until a row arrives, then draws one row per held scrumming bot with Live's ten columns, the Bot ID cell coloured by the bot's state. The Extractor Bots table sits under it, forked the same way, and hides until an extractor record is held. Under the tables the command bar holds Start, Pause, Stop, Restart and Delete. Pause, Restart and Stop move the selected bot's state through the Simulator's bot manager, and the row reads it back. Delete asks Live's confirmation and removes the record. Start is the run's button in Validation and Back Test.

Since Start Run landed on the way-in row, Start on the command bar moves the selected bot's state in every mode, as Pause, Stop and Restart do, and starts no run. The run's button is Start Run on the way-in row. While a run is in flight, Stop on one of its rows ends it and every other bar press writes the in-flight line.

`src/gui/simulator/sim_trading_tab.py` — the bar's Start, every mode

```
        if command == "start":
            try:
                moved = self._bot_manager.start(bot_id) != bot.state
                self._status_log.log(f"✓ Bot {bot_id} RUNNING.", "success")
```

`src/simulator/sim_bot_manager.py` — the states the bar moves

```
    def pause(self, bot_id: str) -> str:
        """``BotContainer.pause``'s rule: any state lands on ``paused``."""
        self._require(bot_id)
        return self._fleet.set_state(bot_id, BotState.PAUSED.value)

    def stop(self, bot_id: str) -> str:
        """``BotContainer.stop``'s rule: any state lands on ``stopped``."""
        self._require(bot_id)
        return self._fleet.set_state(bot_id, BotState.STOPPED.value)
```

Each row's Fire and Detail buttons are Live's. Detail opens the Simulator's Bot Settings window over the bot's record with Live's seven tabs for a scrumming bot: Status, Settings, Fold Tranches, Stack Tranches, Bot Swarm, Market Inspector and Phantom Bots. The window reads the record; opening and closing it writes nothing to the fleet file. `+ New Bot` opens the Simulator's Bot Wizard, titled Create Auto Trader, whose Finish holds one new record.

`src/gui/simulator/sim_bot_detail.py` — the tabs a scrumming bot's window carries

```
        tabs.addTab(
            self._wrap_scrollable(self._create_status_tab()), surface.TAB_STATUS
        )
        tabs.addTab(
            self._wrap_scrollable(self._create_settings_tab()), surface.TAB_SETTINGS
        )
        if mode == surface.MODE_SCRUMMING:
            self._install_fold_tranches_tab(tabs)
            self._install_stack_tranches_tab(tabs)
```

The held fleet is the Simulator's own. Every fleet change writes it to `sim_fleet.json` under the sim bucket of the log root, and the tab reads it back at the next build, so a created or imported bot survives a restart. The live `bot_state.json` is never written. The Simulator receives and asks; it never sends.

`src/core/log_paths.py` — the bucket

```
def get_sim_dir() -> Path:
    """``sim/`` bucket — every file the Simulator writes.

    ``src.simulator.fleet_source.FleetSource`` keeps the sim fleet file
    here; ``~/.acervator/bot_state.json`` is never written from the
    Simulator.
    """
    p = resolve_log_root() / "sim"
```

The Indicator Voting Panel and the replay layer behind it

The panel beside the venue pane is the forked `SimIndicatorVotingPanel`. It carries Live's title, Bot selector, currency rate strip, two pillar tables, two confidence bar graphs, timeframe-lock line and staleness banner. Its selector holds the held fleet and follows each bot's state. While the selected bot runs, the pillar tables draw its reading from its Stone Tablet and the banner names the tablet's last candle. A bot that is not running draws Live's cause instead.

`src/gui/simulator/sim_trading_tab.py` — the feed

```
        feed = tab_surface.ivp_feed(
            self._tablet_source, self._fleet_source.bot_for(chosen)
        )
        if feed.get("summary"):
            panel.show_stored(
                feed["stored"], feed["when"], feed["age"], feed["message"]
            )
        else:
            panel.show_no_data(
```

One flip button sits after the panel title and reads Replay. A press shows the layer behind the panel: the VWAP window over the Stone Tablet playback window, each drawn from the chosen tablet's last hundred candles. The layer's header holds the flip, now reading Indicators, a Tablet chooser 240 px wide that lists every tablet on disk, and one button reading Retrieve Tablet or Update Tablet. A press runs the retrieval or update through the Simulator's read-only connector on a worker thread, records each venue call on the API Interaction Log, and writes the started line and the finished or refused line on the Activity Log. The connector answers `get_ohlcv` over the public candle endpoint and raises `SendRefused` for every other name. This layer is the first of the three permitted differences from Live.

`src/simulator/read_only_connector.py` — the refusal

```
    def __getattr__(self, name: str) -> Any:
        """Refuse every name outside ``READ_NAMES``."""
        raise SendRefused(
            f"ReadOnlyConnector answers {READ_NAMES} and cannot {name!r}. "
            "The Simulator receives and asks; it sends nothing."
        )
```

The flip button now sits first in the header, at the panel's top-left corner, before the title, and the replay layer's header seats it first at the same corner. It draws one width on both layers, the width of the wider word, so Replay and Indicators occupy one rect and the way back is where the way in was. Each press emits `sim.layer.flipped` through `signal_contract` with the pressed rect against the previous press's rect.

`src/gui/simulator/sim_trading_tab.py` — the seat

```
        if self._layer == surface.LAYER_PLAYBACK:
            self._indicator_panel.header_row().removeWidget(self._flip_button)
            self._chart_header.insertWidget(FLIP_SEAT_INDEX, self._flip_button)
        else:
            self._chart_header.removeWidget(self._flip_button)
            self._indicator_panel.header_row().insertWidget(
                FLIP_SEAT_INDEX, self._flip_button
            )
```

The two spools

The Activity Log sits at the foot left with Pause Console, and the API Interaction Log at the foot right with Pause API Log, Live's titles at Live's positions. Pause holds new lines and Resume releases them in order.

`src/gui/simulator/sim_trading_tab.py` — the pause

```
        def _on_activity_pause_toggled(checked: bool):
            if checked:
                self._status_log.pause()
                self._activity_pause_btn.setText("■  Resume Console")
            else:
                self._status_log.resume()
                self._activity_pause_btn.setText("■  Pause Console")
```

The Activity Log shows the trades of Validation, Back Test and Portfolio Battery runs, one line per simulated trade in Live's fill line shape, stamped with the candle's time rather than the clock. That is the second permitted difference. The API Interaction Log records Stone Tablet and YTD retrieval activity only: its log is a `SimApiLog` that accepts two action words and refuses every other by name, so a venue order line cannot reach it. That is the third. Generate From YTD writes one block per press.

`src/simulator/sim_api_log.py` — the allowed actions

```
TABLET_ACTION = "FETCH_TABLET"

#: The word the Generate From YTD read's action opens with.
YTD_ACTION = "FETCH_YTD"

#: Every action word ``SimApiLog.record`` accepts. Any other raises ``SendRefused``.
ALLOWED_ACTIONS = (TABLET_ACTION, YTD_ACTION)
```

The three run modes and what each writes

Every mode's run starts from Start Run on the way-in row. Under Validation and Back Test the press starts the run the two paragraphs below describe, which Start on the command bar started before the row's button landed. Under Portfolio Battery the press opens Run Portfolio's chooser, one portfolio and one span, and runs it. One handler on each host, `_start_run_pressed`, reads the mode and the venue shown.

`src/gui/simulator/sim_trading_tab.py` — the press by mode

```
        elif mode in tab_surface.RUN_MODES:
            outcome = self._start_run(venue, mode)
        else:
            outcome = self._run_battery(surface.RUN_PORTFOLIO_ACTION)
```

Validation. With Validation active, Start on the command bar moves every scrumming bot on the page to running, reruns the gate chain on the candle each recorded fill fell in, and puts the rerun lights beside the recorded ones. The run passes when they latch identically, never on profit and never on a trade count. Each rerun fill reaches the Activity Log as a trade line, and the bots land on stopped when the run ends.

`src/simulator/validation.py` — the criterion

```
    @property
    def latches_identically(self) -> bool:
        """True while every light and both armed flags read the same."""
        return (
            not self.disagreed
            and self.recorded_scrum_armed == self.rerun_scrum_armed
            and self.recorded_fold_armed == self.rerun_fold_armed
        )
```

Back Test. With Back Test active, Start walks each bot along its Stone Tablet with the Live bot's own sizing arithmetic, shared as pure code, and each scrum and fold reaches the Activity Log as it fills.

`src/simulator/back_test.py` — the shared arithmetic

```
from ..trading.scrumming.sizing import (
```

Portfolio Battery. With Portfolio Battery active, Run Portfolio opens a chooser of one portfolio over the archive's thirty-five and one span over seven, and Run Every Portfolio opens it with the span alone. The run holds one bot per symbol at $500 times the portfolio's mix share, walks the RA-StoneTablets on a worker thread, and writes one reading per timeframe.

`src/simulator/portfolio_battery.py` — the default Target Balance

```
DEFAULT_TARGET_USD = 500.0
```

Since the Battery walked every candle, each bot evaluates every bar of its tablet from the thirtieth on, the Activity Log carries a line at each walk's start, every 5,000 bars and at its end, Run Every Portfolio holds one portfolio's bots, walks them, writes their report, clears them and holds the next, every reading is HODL end, Harvest-Fold end and the difference, and Stop on a Battery row ends the walk at the bar reached.

`src/simulator/portfolio_battery.py` — the evaluations a walk makes

```
def evaluations_expected(bars: int) -> int:
    """How many gate-chain evaluations a walk over ``bars`` makes: one per bar
    from the ``MIN_CANDLES``th on, none under ``MIN_CANDLES``."""
    return max(0, int(bars) - (MIN_CANDLES - 1))
```

Stop on a row of the run in flight ends the run where it is.

`src/gui/simulator/sim_trading_tab.py` — the stop

```
    def _stop_run(self, bot_id: str) -> None:
        self._status_log.log(f"Stopping bot {bot_id}...", "info")
        self._run["stopper"] = bot_id
        self._status_log.log(
            tab_surface.run_stopping_line(self._run.get("mode", ""), bot_id), "warning"
        )
        self._run_stop.set()
```

Every run writes one trade parity report, a Markdown file and a JSON sidecar, under the simulator directory of the reports directory, named for the mode, the subject, the span and the time. A stopped run's report reads stopped and partial.

`src/simulator/parity_report.py` — the file stem

```
def report_name(mode: str, subject: str, span: str, stamp: str) -> str:
    """The file stem: mode, subject, span and stamp joined by ``__``."""
    return "__".join(
        (safe_name(mode), safe_name(subject)[:SUBJECT_LIMIT], safe_name(span), stamp)
    )
```

Every run also emits Live's topics on the Simulator's private event bus, and Live's log manager writes them under the sim bucket as four files: one row per gate evaluation, per fill and per voting summary, each gate and voting row carrying the run id the report carries. The live bucket's files are untouched.

`src/simulator/sim_bus.py` — the files

```
#: The files ``LogManager.attach_to_bus`` writes the four topics into.
SIM_LOG_FILES = ("trade.log", "gate.log", "voting.log", "diagnostics.log")
```

One sentence and the block above are overtaken. They are quoted whole, and what replaces them follows.

> *Live's log manager writes them under the sim bucket as four files: one row per gate evaluation, per fill and per voting summary, each gate and voting row carrying the run id the report carries.*
>
> *SIM_LOG_FILES = ("trade.log", "gate.log", "voting.log", "diagnostics.log")*

The gate rows no longer share one file. Live's log manager writes them under the sim bucket as three fixed files plus one gate log per exchange and sector, at `gate/<exchange>/<sector>/gate.log` relative to that bucket. `src/simulator/sim_bus.py, in sim_asset_class` answers the sector per market through `src/simulator/portfolios.py, in asset_class`, so a sim decision is filed under the sector its live twin is filed under, and `src/simulator/sim_bus.py, in sim_log_paths` lists the fixed three with one key per gate log present. The live bucket's files are still untouched. The layout and the carry of the records written before the split are in the live trade history reference.

```
#: The files ``LogManager.attach_to_bus`` writes the four topics into. The gate
#: topic reaches one file per exchange and sector, which ``sim_log_paths`` lists.
SIM_LOG_FILES = ("trade.log", "voting.log", "diagnostics.log")
```

Every emit a run raises on its worker thread, the TA engine's postcondition pins among them, lands in the Simulator's own signal sink, `signals/session.jsonl` under the sim bucket, and none in the process sink under `signals/` of the log root. The Activity Log names the sim sink's path at each run's start, and the process sink gains one `sim.sink.routed` row at each run's end.

`src/simulator/sim_bus.py` — the line at a run's start

```
#: The Activity Log line ``routed_run`` writes at each run's start.
SINK_LINE_FORMAT = "{run} signals: {path}"
```

The colour

Every Sim ground is the theme's nigredo tone. The widget tree carries the tone property, and the theme restates each ground token a quarter of the way to black under that selector, so the Sim's grounds sample darker than Live's under the same theme while the text and accent colours stay the theme's.

`src/gui/theme_engine.py` — the tone

```
TONE_PROPERTY = "tone"
NIGREDO = "nigredo"

#: The share of the distance to black every ``NIGREDO_GROUNDS`` token moves.
NIGREDO_FRACTION = 0.25
```

Where the tablets live

The tablet archive is in the repository. A clone carries all 407 files, so the history arrives with the code and no venue is called to fetch it. 406 are candle tablets and one is the index they are read through.

`src/trading/stone_tablets/storage.py` — the root a source checkout resolves

```
def _tablet_root() -> Path:
    if is_frozen():
        return HOME_TABLETS_DIR
    return project_root() / "stone_tablets"
```

A tablet is never modified. It is recorded venue history, and a back test over a tablet that drifted proves nothing. Tracking makes that rule checkable rather than merely stated: the archive is under version control, so any later change to any file in it shows up as a difference against the committed copy.

The line-ending rule the repository applies to text does not reach the archive. The index file holds 5,283 carriage returns, and normalising them would change its digest and break the checksum every row carries.

`.gitattributes` — the archive's exemption

```
stone_tablets/** -text
```

A built executable carries no archive, so it reads the home tree instead, at `~/.acervator/stone_tablets`. A source run never reads that copy.

What is not built

Privacy Mode, the news line and the data-pool line are not on the tab; their row holds the mode buttons, by ruling. A column map exists for the Coinbase export alone, so an export from another exchange is refused at import by name until a sample of it exists. Portfolio Battery walks an asset class only under a unit rule cited from its venue, and a class with no cited rule is not simulated.

In development.

How each part was built

Every section from here to "The widget the rebuild replaced" is one build's own record, kept as it was written when that build merged, in build order: the Stone Tablet and RA-StoneTablet sections, the three mode sections, the YTD trade files and the tab's builder from before the fork, then units 7 through 31a. Where a later unit changed what a sentence describes, the sentence now says so and names that unit's report on issue #117. The section above describes the tab on the current commit.

The candles

Stone Tablets are the history and a replay only reads them. The registry stores one timeframe and derives every other from it.

`src/trading/stone_tablets/registry.py` — `NATIVE_TIMEFRAME`

```
NATIVE_TIMEFRAME: str = "5m"
"""Timeframe every tablet stores; ``_rollup`` derives all the others."""
```

Three functions in `src/trading/stone_tablets/storage.py` move tablets between disk and memory.

Function	What it does
`read_tablet`	Loads one tablet from disk
`write_tablet`	Writes one back
`read_manifest`	Carries the index of what exists

The tape the removed Fleet Replay handed its bots did not roll up: it asked the registry for the native timeframe only, and the tape refused any other unless the caller supplied that series when the tape was built. The rebuild removed Fleet Replay (pull request 454); the forked tab reads tablets through its tablet source, and the section "What the tab holds today" describes it.

`src/exchange/tablet_backend.py` — `TabletBackend.fetch_ohlcv`

```
        key = (str(symbol), tf)
        rows = self._tf_rows.get(key)
        if rows is None:
            raise ValueError(
                f"no {tf} series for {symbol!r}. Serving the native "
                f"{NATIVE_TIMEFRAME} series instead would make every "
                f"timeframe agree with itself; supply tf_rows[{key!r}]."
            )
```

Portfolio Battery history

The thirty-five portfolios from the historical archive are now named in code, and their sixty-three symbols are what the new mode runs over. Each one carries its symbols, an equal weight for every symbol, and the archive it was read out of.

`src/simulator/portfolios.py` — one portfolio

```
    "CRYPTO_BLUE": Portfolio(
        name="CRYPTO_BLUE",
        symbols=("A15", "A14", "BNB"),
        description="Large-cap crypto — institutional grade",
    ),
```

Their price history is real, and it is kept apart from the live fleet's. The tablets a replay reads sit under one root; the battery's sit under a second one, and a battery build never opens the first.

`src/trading/stone_tablets/ra_paths.py` — the second root

```
_RA_ROOT: Path = Path.home() / ".acervator_ra_tablets"
```

Two sources fill it and neither needs a key. Crypto arrives through the adapter the fleet already uses, driven by the venue's own public candle endpoint. Everything else arrives through a second adapter beside it.

`src/trading/stone_tablets/ra_fetcher.py` — the non-crypto adapter

```
class YahooChartAdapter(ExchangeAdapter):
    exchange_id = "yahoo"
    chunk_limit = RA_CHUNK_DAYS
    BASE_URL: str = "https://query1.finance.yahoo.com/v8/finance/chart"
    SOURCE: str = "yahoo_chart_v8_ONE_DAY_SPLIT_ADJUSTED"
```

Every tablet records where its numbers came from and when they were fetched, and the builder refuses to write one without both. A price with no source is what made the archive's own figures worthless.

`src/trading/stone_tablets/ra_fetcher.py` — the refusal

```
        resolved = source or str(getattr(adapter, "SOURCE", ""))
        if not resolved:
            raise ValueError(
                f"{type(adapter).__name__} carries no SOURCE; pass source= naming "
                f"the endpoint the candles come from. {adapter.exchange_id!r} is "
                f"an exchange id, not provenance."
            )
```

Where a source has nothing, nothing is written in its place. The missing days are recorded as missing, in their own file beside the tablets.

`src/trading/stone_tablets/ra_fetcher.py` — what a missing period records

```
@dataclass
class TabletGap:
    """One requested period a source returned no rows for."""

    asset: str
    exchange_id: str
    timeframe: str
    year: int
    since_ms: int
    until_ms: int
    reason: str
    checked_at: str
```

At the RA coverage build nothing ran the bot logic over these tablets and no screen showed them; unit 24's Portfolio Battery walks them from the corner (comment 5725991788).

`src/simulator/portfolio_battery.py` — the run over these tablets

```
def plan_run(
    names: Sequence[str],
    tablets: Any,
    held: Sequence[SimBot] = (),
```

Portfolio Battery coverage

Every symbol in every portfolio has real daily prices on disk. No portfolio names a period of its own, so all six archive periods apply to all thirty-five of them, and the span asked for is 2020 to 2026.

`src/trading/stone_tablets/ra_import.py` — the years asked for

```
def _archive_years() -> tuple[int, ...]:
    """Return every calendar year ``PERIODS`` touches, ascending."""
    years: set[int] = set()
    for start, end in PERIODS.values():
        years.update(range(int(start[:4]), int(end[:4]) + 1))
    return tuple(sorted(years))
```

A symbol reaches one source or the other by what it is. A coin the crypto venue never listed falls to the second source instead of being left empty, and every tablet records which one served it.

`src/trading/stone_tablets/ra_import.py` — the routing

```
def route_for(symbol: str) -> SymbolRoute:
    """Return ``symbol``'s route, crypto to Coinbase and everything else to Yahoo."""
    upper = symbol.upper()
    if is_crypto(upper):
        return SymbolRoute(upper, COINBASE_SOURCE, YAHOO_CRYPTO_SOURCE)
    return SymbolRoute(upper, YAHOO_SOURCE)
```

The import reads its own result back off disk and says what it holds. A count on its own would not be an answer, so the statement carries every source with its fetch times and every period no source served.

`python -m src.trading.stone_tablets.ra_import coverage` — the headline

```
SYMBOLS
  asked for ......... 63
  returned data ..... 60
  tablets ........... 411
  candles ........... 105336

SOURCES
  coinbase_exchange_candles_ONE_DAY: 71 tablets
  yahoo_chart_v8_ONE_DAY_SPLIT_ADJUSTED: 340 tablets
```

A share year is not a calendar year. The market shuts at weekends and on public holidays, and those days are recorded as missing rather than filled, so a share year holds about 250 days where a coin year holds every one.

`python -m src.trading.stone_tablets.ra_import coverage` — a share and a coin

```
  SPY
    2022    251 rows  2022-01-03..2022-12-30
    2023    250 rows  2023-01-03..2023-12-29
  A15
    2022    365 rows  2022-01-01..2022-12-31
    2023    365 rows  2023-01-01..2023-12-31
```

A ticker that no longer reaches the company the archive meant is recorded as a finding, not as a failure. Three of the sixty-three return nothing at all, and what the endpoint answered is written down in place of a price.

`~/.acervator_ra_tablets/GAPS.json` — a ticker that no longer trades

```
  CCIV   2021 yahoo  [2021-01-01..2021-12-31] HTTPError: HTTP Error 404: Not Found
  EXPR   2021 yahoo  [2021-01-01..2021-12-31] HTTPError: HTTP Error 404: Not Found
  IPOF   2021 yahoo  [2021-01-01..2021-12-31] HTTPError: HTTP Error 404: Not Found
```

Three more symbols are short at one end. BBBY reaches a company first traded in July 2026 rather than the one the archive ran, and `MATIC` stops on 14 October 2025 because the coin was renamed.

`python -m src.trading.stone_tablets.ra_import coverage` — a symbol cut short

```
  MATIC  missing: 2026
    2024    366 rows  2024-01-01..2024-12-31
    2025    287 rows  2025-01-01..2025-10-14
```

At the RA coverage build nothing ran the bot logic over these prices and no screen showed them; unit 24's Portfolio Battery walks them from the corner (comment 5725991788).

`src/simulator/portfolio_battery.py` — the default Target Balance each symbol runs at

```
DEFAULT_TARGET_USD = 500.0
```

Validation Mode

Validation takes each YTD trade, finds the historical candle its timestamp falls in, and reruns the gates on that candle. It then puts the gate row it latched beside the gate row the log recorded, one light at a time, over the nineteen lights the shared vocabulary names. A run passes when every light reads the same. It is never judged on profit and never on a trade count.

`src/simulator/validation.py` — the criterion

```
    @property
    def latches_identically(self) -> bool:
        """True while every light and both armed flags read the same."""
        return (
            not self.disagreed
            and self.recorded_scrum_armed == self.rerun_scrum_armed
            and self.recorded_fold_armed == self.rerun_fold_armed
        )
```

The two ways in

Import Live Fleet clones the live running fleet from the bot_state load and keeps each bot's own id, so a recorded gate row belongs to a bot by that id. Generate From YTD scans the trade files and creates one new bot per pair that was traded. Those bots are new, they carry new ids, and they never wrote a gate row, so a recorded row belongs to one of them by exchange, symbol and the time window instead.

```
Import Live Fleet    38 bots, matched on bot id and time window
Generate From YTD    39 bots, matched on exchange, symbol and time window
```

The two counts differ, and that is a fact about the two sources rather than a fault. A pair traded earlier in the year can have no live bot now, and a live bot can have traded nothing yet. Neither path invents a bot to close the gap.

When more than one exchange is active the tab asks which one to use before it runs anything. One chooser serves both buttons.

`src/simulator/fleet_source.py` — the chooser

```
def exchange_choice(exchanges: list[str], chosen: str = "") -> dict:
    """Whether the operator must pick an exchange, and which one is in force.

    ``prompt`` is True while more than one exchange is active and ``chosen``
    names none of them.
    """
```

It reports what it could not verify

The tablets end on 1 August 2026 and the trade export runs to 7 September, so about five weeks of trades have no candle to snap to. Validation names that period and counts the entries on both sides of it. A count of verified trades with no denominator is exactly what this replaces.

```
3942 of 4904 YTD entries snapped to a candle; 962 could not be.
after_last_candle: 905
inside_gap: 57
uncovered span 2026-08-01T18:52:58Z to 2026-09-07T21:46:48Z; the newest
candle is 2026-08-01T18:35:00Z
```

Which half of the rerun moves

A candle supplies the price and the Bollinger reading. It cannot supply the bot's own state, so the delta, the tranche counts, the circuit breaker and every flag come from the row the log recorded. Each light says which half drove it, so a disagreement can be read back to its cause.

```
S/BB   F/BB   F/MID     the tablet candle
S/LS   F/LS             the landing-strip override
every other light       the recorded row
```

What the first full run measured

Every recorded reading the run checked was reproduced from the tablet exactly, but from a window sitting four hours behind the trade's own candle. The tablet prices are right: measured over one asset, 9,869 of 10,038 recorded prices sit inside the tablet candle covering their own moment. The tape is therefore sound, and the live reading it is compared against was taken from older candles than the ones that moment held.

```
38 bots, matched to a recorded gate row on bot id and time window.
62 of 227 reruns latched every gate identically.
3572 of 4313 gate lights agreed.
60 of 60 recorded readings were reproduced exactly from the tablet.
45 of them came from a window 50 candles (250 minutes) behind the trade's
own candle.
```

Back Test Mode

Back Test runs the bot logic over a recorded tape. It walks each bot along its own Stone Tablet, feeds every window into the gates that run live, and records where a scrum and a fold latch. A latch that clears every gate then trades: the scrum sells the excess above the dollar target and the fold buys it back with the cash that scrum put aside.

`src/simulator/back_test.py` — the one gate chain, shared with Validation

```
from .validation import (
    BB_MIDLINE,
    MIN_RERUN_CANDLES,
    RERUN_WINDOW_CANDLES,
    bb_reading,
    candle_interval_ms,
    iso_stamp,
    latch,
    tablet_for,
)
```

At the first clone a mode selector sat above the two button rows and a result pane below the Replay Log changed with it; unit 20 put the mode buttons on the venue page's header row, and the corner holds the active mode's two ways in (comment 5720691825).

```
Mode: Validation    Import Live Fleet   Generate From YTD
Mode: Back Test     Import Live Fleet   Create New Bots
```

The two ways into a back test

Import Live Fleet clones the live running fleet from the bot_state load, the same reader Validation uses. At the first clone Create New Bots made one simulated bot on the Stone Tablet the selector showed, at the Bot Wizard's defaults; since unit 23 it opens the forked wizard and holds one record from the wizard's values (comment 5723334396).

`src/gui/main_tabs/simulator_tab_surface.py` — the new bot the selector defines

```
def new_bot_specs(entry) -> list[dict]:
    """One new-bot spec for ``entry``'s asset, on the Bot Wizard's own
    defaults."""
```

A third way in: a push from the Market Inspector

Press Push to Sim on a swarm topology proposal or on an opposing-trade pair and the Simulator holds one bot per market the candidate names, under Back Test, and leaves Back Test the run mode. Open the Sim tab afterwards and it is already in that mode with those bots in its table. Nothing starts: the bots are held idle and Start Run is the press that walks them.

`src/gui/simulator/sim_trading_tab.py` — where a push lands

```
def push_to_back_test(self, candidate: Any) -> dict:
    """Hold one sim bot per pushed market under Back Test, register the
    wires between them, and leave Back Test the run mode.
    """
```

Each market takes the venue and the timeframe of its own Stone Tablet, so a pushed bot walks the tape the Simulator already holds for it. A market with no tablet refuses the whole push and moves no mode, which the Market Inspector page describes at the button.

A Target Balance the Simulator cannot trade against refuses the push the same way. Each row's dollar target is read through the one admission Push to Paper reads, so an endless figure, a figure that is not a number, a stored flag, a negative figure and text no dollar can be read from are refused on both destinations alike; the Activity Log names every market refused, nothing is held and the run mode is unchanged. A row naming no target, or naming zero, takes the `target_balance` default `BotConfig` declares. A market named twice in one candidate holds one bot, never two.

`src/gui/simulator/sim_trading_tab_surface.py, in push_configs` — the three lists a push reads

```
def push_configs(candidates: Any, entries: Any) -> tuple:
    """``(configs, missing, refused)`` for one pushed candidate's symbols."""
```

A swarm topology also names the wires between its bots, and those travel with them. Each pushed bot gets its own `SimWireLedger`, each wire is registered between two of those bots at the rate the proposal names, and the sim fleet file carries both under Back Test. A fold on a pushed source bot then routes its share to the pushed target, which is the routing A fold's growth travels the wires to another sim bot describes and Start Run carries the fleet's wires, and the run's end saves them drives.

One pair holds one rate, so a topology naming the same source and target twice refuses the whole push, names the pair and holds nothing.

The Activity Log reports both halves:

```
PUSH_HELD_FORMAT = "Pushed {count} bot(s) to Back Test: {assets}."
PUSH_WIRED_FORMAT = "Wired {count} of them: {wires}."
```

A wire the push cannot deliver refuses the whole push, exactly as a missing tablet does: no bot is held, no wire is registered and the run mode does not move.

The push's mode choice reaches the file

The bots, the wires and the mode land in one write: `push_to_back_test` holds each bot, registers each wire, then fires `fleet_changed`, which runs `FleetSource.save`, so the sim fleet file carries all three at once. The mode it carries is the one The mode choice is wired and the way-ins follow it reads back, so a push sets the mode for the next launch and not only for the rest of the session.

Driven on the real pane over a real Stone Tablet manifest, on a topology naming two markets and one wire between them:

```
answered        mode back_test, held 2, wires 1, refused none
mode on disk    back_test
wires on disk   1
Activity Log    Pushed 2 bot(s) to the Simulator; its run mode is back_test.
```

A refused push writes nothing. With one market no tablet names, the file read back byte-identical before and after the press, the mode in force did not change, and the fleet held the same two bots an earlier push had landed.

One clock

Every bot ticks on one cadence. The step is sized once from the longest tape in the run, so a short tablet and a long one are read on the same clock rather than each on its own.

```
def shared_step(entries: Sequence[Any], ticks_per_bot: int, max_candles: int) -> int:
    """One step in candles, sized off the longest tape so every bot ticks
    together.

    A ``ticks_per_bot`` of zero, or no entry, answers one.
    """
```

What one back test measured

The imported fleet, read on 8 September 2026 from the live Stone Tablets.

```
38 of 38 bots ran over 1610472 tablet candles.
2026-01-01T00:00:00Z to 2026-08-01T18:35:00Z, 5272 gate-chain evaluations.
58 scrum latches and 55 fold latches.
58 scrum sells and 55 fold buys filled, $38.02 in fees.
```

Each row names the bot, its tablet, how much tape it read, how often the gates latched, how many trades filled, the coin it gained or gave up and the cash it now holds. Units alone do not answer whether a cycle accumulated, because a bot that sold into a rise holds fewer coins and more cash.

```
Bot ID | Symbol | Tablet | Candles | Ticks | Scrum / Fold latched |
Scrum / Fold filled | Units gained | Cash held
```

A missing tablet is named, not fetched silently

A bot whose asset has no tablet is listed with no tape and counted in the run's own lines. Filling one is a call for information and goes through the same gap filler the tablet build uses, into whichever tablet root the caller names.

`src/simulator/back_test.py` — the fetch, and the answer with no connector

```
async def download_missing(
    pairs: Sequence[tuple[str, str]],
    connector: Any,
    since_ms: int,
    until_ms: Optional[int] = None,
    quote_currency: str = "USD",
    registry: Any = None,
) -> list[Any]:
```

Driven into a throwaway root with a recorded tape, it wrote one tablet of 701 candles and a manifest, and the live tablet root kept its 407 files.

```
connector calls: 3
report: ZZZTEST coinbase chunks_ok 3 errors 0 candles 701
entry: ZZZTEST coinbase 5m 701 candles read back: 701
live root unchanged: True 407
```

Portfolio Battery Mode

The operator's words, 2026-09-18: *"Honest framing should be to show our strategy's ability to identify opportunities for partial exists and re-entries so as to improve portfolio end states. [...] I only seek to compare HODL against Harvest-Fold across as many historical markets and asset classes as possible."* The sentence left out says the mode is not a contest. The Battery reads one comparison, HODL end against Harvest-Fold end and the difference, per portfolio, per span, per timeframe; the section "The Battery walks every candle, one portfolio at a time, and reads HODL against Harvest-Fold" says how.

The third mode runs the thirty-five archive portfolios over their own price history. Every symbol streams its daily bars through the same gate chain a live bot ticks, at three timeframes, and each result is set against what holding that symbol untraded would have paid over the same span.

`src/simulator/portfolio_battery.py` — the three timeframes

```
#: The timeframes a daily tablet builds, spelled as ``HTF_TIMEFRAMES`` spells
#: them.
TIMEFRAMES = ("1d", "1w", "1M")
```

The baseline comes off the same walk

Buy and hold is not a figure carried over from the archive. It is the capital the symbol opened with, carried from the opening price to the closing price of the same walk, so the two sides cannot read different prices.

`src/simulator/portfolio_battery.py` — the baseline

```
    @property
    def baseline_usd(self) -> float:
        """``capital_usd`` carried from ``start_price`` to ``end_price``,
        untraded."""
        if self.start_price <= 0.0:
            return 0.0
        return self.capital_usd * self.end_price / self.start_price
```

One gate chain, fed a different source

The battery defines no gate of its own. It hands its bars to the Back Test walker, which builds the shipped scrum and fold chains and reads their answer.

`src/simulator/portfolio_battery.py` — the walk

```
    bot = battery_bot(asset, exchange_id, timeframe, capital_usd)
    result = walk(bot, candles_from_raw(bars), walk_step(len(bars), ticks))
```

A weekly and a monthly bar are folded, never invented

The tablets hold one bar a day. A coarser bar is the calendar bucket its days fall in: the first open, the highest high, the lowest low, the last close and the summed volume. A trailing bucket is kept as it stands rather than padded out.

`src/simulator/portfolio_battery.py` — one folded bar

```
def fold_bucket(rows: Sequence[Sequence[float]]) -> list[float]:
    """One bar from ``rows``: first open, highest high, lowest low, last close,
    summed volume."""
    return [
        float(rows[0][0]),
        float(rows[0][1]),
        max(float(one[2]) for one in rows),
        min(float(one[3]) for one in rows),
        float(rows[-1][4]),
        sum(float(one[5]) for one in rows),
    ]
```

Three symbols hold no tablet

CCIV, IPOF and EXPR stopped trading and no source served them. A portfolio holding one runs on the symbols that exist and reports the share of its capital that reached no tape. That share is a column on every row, beside the count of symbols that ran.

`src/simulator/portfolio_battery.py` — the missing share

```
    @property
    def missing_weight(self) -> float:
        """``missing_usd`` as a share of every symbol's ``capital_usd``."""
        whole = sum(one.capital_usd for one in self.runs)
        return self.missing_usd / whole if whole > 0.0 else 0.0
```

The two ways into a battery

At the first clone the two rows the removed strips left carried one portfolio and all of them, with a portfolio selector and a span selector under the mode selector; since unit 24 Run Portfolio opens a chooser of the portfolio and the span, and Run Every Portfolio of the span alone (comment 5725991788). The span list is the archive's six periods followed by the whole tape.

`src/gui/main_tabs/simulator_tab_surface.py` — the two ways in

```
BATTERY_ROWS: tuple[dict[str, Any], ...] = (
    {
        "name": NEWS_TICKER_ROW,
        "height_px": 24,
        "action": RUN_PORTFOLIO_ACTION,
        "text": RUN_PORTFOLIO_TEXT,
        "button_name": button_name(RUN_PORTFOLIO_ACTION),
    },
    {
        "name": DATA_POOL_ROW,
        "height_px": 18,
        "action": RUN_EVERY_PORTFOLIO_ACTION,
        "text": RUN_EVERY_PORTFOLIO_TEXT,
        "button_name": button_name(RUN_EVERY_PORTFOLIO_ACTION),
    },
)
```

What one battery measured

SPAC_BUST over the archive's Apr24-Apr25 period, driven through the Qt tab. Three of its five symbols have a tablet. The accumulation logic ended ahead of buy and hold at the daily bar and at the weekly bar. The monthly bar holds twelve bars, under the thirty the Bollinger window and the voting engine need, so it reports no run rather than a result.

```
SPAC_BUST  1d  3 of 5  756 bars  669 ticks  35 trades
           buy and hold $390.75   accumulation $415.60   +24.85 (+6.36%)
SPAC_BUST  1w  3 of 5  159 bars   72 ticks   6 trades
           buy and hold $458.26   accumulation $478.74   +20.48 (+4.47%)
SPAC_BUST  1M  0 of 5    0 bars    0 ticks   0 trades   missing weight 100%

missing: IPOF (no_tablet), CCIV (no_tablet)
7 recorded gaps fall inside the span
```

Every portfolio over the whole tape reads 189 symbol runs in 26 seconds. Twelve of the thirty-five ended ahead of their own buy and hold at one timeframe or more.

The run from the corner

Run Portfolio opens a chooser of the same shape as the exchange chooser, with a portfolio row listing the thirty-five names and a span row listing the seven spans; Run Every Portfolio opens it with the span row alone. On Ok the tab gives each portfolio its bots. Where the held fleet carries one scrumming bot per symbol of the portfolio, on the venue the run reads that symbol from, with a Target Balance above zero, those bots run at their own Target Balances and gate settings. Otherwise one bot is generated per symbol at the default Target Balance of $500 times the portfolio's mix share; every archive portfolio's mix is equal, so each bot reads $500. A portfolio that records the size of each position would run at those sizes; no archive entry records one.

`src/simulator/portfolio_battery.py` — each bot's Target Balance

```
def symbol_targets(
    portfolio: Portfolio, default_usd: float = DEFAULT_TARGET_USD
) -> dict[str, float]:
    """Each symbol's Target Balance: its ``positions_usd`` size when
    ``sizes_known``, else its ``weights`` share of ``default_usd`` per symbol,
    to the cent."""
    if portfolio.sizes_known:
        sizes = portfolio.positions_usd or {}
        return {symbol: round(float(sizes[symbol]), 2) for symbol in portfolio.symbols}
    if portfolio.mix is None:
        return {symbol: float(default_usd) for symbol in portfolio.symbols}
    whole = float(default_usd) * len(portfolio.symbols)
    weights = portfolio.weights
    return {symbol: round(whole * weights[symbol], 2) for symbol in portfolio.symbols}
```

The run's bots become the held fleet: a bot already held under its id at the same symbol, venue and Target Balance keeps its record, a generated bot is held under the origin `battery`, and every other held record is dropped. The Scrumming Bots table then draws the run's bots and the strip's Spendable reads the run's budget, the sum of their Target Balances, before the run starts.

`src/simulator/fleet_source.py` — the run's bots become the held fleet

```
    def hold_battery_fleet(self, bots: Sequence[SimBot]) -> list[SimBot]:
```

The run itself walks on a worker thread, as the Market Inspector's Scan Now does, so the window keeps answering; each portfolio's reading reaches the Activity Log as it lands, then the run's own lines and the report line. A second press while a run is in flight writes one line and starts nothing. A symbol with no tablet or no cited unit rule is missing weight and named in the report, and the run proceeds over the rest.

`src/gui/simulator/sim_trading_tab.py` — the worker thread

```
        self._battery_thread = threading.Thread(
            target=self._compute_battery,
            args=(plan, span),
            name="sim-portfolio-battery",
            daemon=True,
        )
        self._battery_thread.start()
```

What the Battery flow measured

Before, in both builds over a scratch home holding a stored fleet of two coinbase bots and five 2022 tablets, a corner press on Run Portfolio opened no chooser and wrote one refusal line, and the runner driven directly walked every symbol at $200 with the outcome line reading `not computed`. After, Run Portfolio on CRYPTO_BLUE over 2022 opened the chooser, held three bots at $500 each with Spendable reading $1,500.00, ran on the worker thread, wrote one report whose Battery section read `1d improved, 1w defended, 1M not run` with BNB named as `no_tablet` and its $500 as missing weight, and wrote the report line; BOGLEHEAD over the same span read the same words with all three symbols run; Run Every Portfolio held sixty-three bots at $31,500.00 and wrote one report. A planted portfolio with a 60/30/10 mix read Targets of $900, $450 and $150; a stored fleet of A15, A14 and BNB on coinbase at $1,000, $750 and $250 ran at those Targets with the report naming the bots loaded from the held fleet. Over a copy of the whole tape, Run Every Portfolio took 24.3 seconds in Qt and 24.9 in React while a 30 ms timer kept ticking with a longest gap of 0.342 and 0.125 seconds; the same run on the GUI thread read a gap of 1.035 seconds over a 1.32 second run. Every scratch file hashed equal before and after every press, no bot was constructed and no socket left loopback.

It is not a Monte Carlo

A Monte Carlo samples many paths and reports the spread of what they pay. This mode reads one recorded path per symbol, so it reports one outcome per timeframe and no distribution. Nothing on the screen carries a percentile.

`src/simulator/portfolio_battery.py` — the spans, one recorded path each

```
#: Every span a run may be asked for: the archive's six, and the whole tape.
SPANS = (FULL_SPAN,) + tuple(PERIODS)
```

The YTD trade files and the import

The Simulator's second data source, kept as the first clone's record wrote it; unit 22 reads these files through Generate From YTD and unit 22a gave the importer one column map per exchange export.

The YTD trade files

The Simulator's second data source is the operator's own trade record, kept on disk under the exchange history bucket. One file holds one exchange, one symbol and one year. Each file names its exchange inside itself, not only in its filename, so a file that is moved or renamed still says where its trades came from.

`src/exchange/ytd_trade_store.py` — what one row holds

```
@dataclass
class YtdTrade:
    """One fill: ``id``, ``ts_ms``, ``side``, ``amount``, ``price``, ``cost``
    and ``fee``."""

    id: str
    ts_ms: int
    side: str
    amount: float
    price: float
    cost: float
    fee: float
    fee_currency: str
```

Beside the rows, each file carries a schema version, the time of the import, and one entry per export that contributed rows. That entry names the export file and its digest, so a file that grew over two imports records both. A MANIFEST beside the files indexes every one of them with its row count and its date span.

`src/exchange/ytd_trade_store.py` — the provenance on each file

```
@dataclass
class ImportSource:
    """One import that contributed rows: ``file``, ``sha256`` and ``rows_added``."""

    file: str
    sha256: str
    imported_at: str
    rows_added: int
```

The columns the import requires

The record arrives as a transactions CSV exported from the exchange. The import reads nine columns and refuses a file missing any one of them, naming which. Notes, sender address and recipient address are never carried in.

`src/exchange/ytd_csv_import.py` — the required columns

```
REQUIRED_COLUMNS: tuple[str, ...] = (
    COL_ID,
    COL_TIMESTAMP,
    COL_TYPE,
    COL_ASSET,
    COL_QUANTITY,
    COL_PRICE_CURRENCY,
    COL_PRICE,
    COL_SUBTOTAL,
    COL_FEES,
)
```

The transaction type decides the side. Buys and sells are kept; reward income, deposits and withdrawals are counted and dropped, because they are not trades and must never reach a validation run as though they were.

The asset and the price currency together give the pair. The quantity is negative on a sell in the export, so its sign is checked against the type and then the magnitude is stored.

A second import of an overlapping export adds only the rows whose id is new. A file that gains nothing is not rewritten at all.

One column map per exchange export

The nine columns above are Coinbase's. Each exchange writes its own export, so the import carries one column map per exchange, chosen by the exchange id the import is given. A map names the export's column for each field of a trade, its buy and sell type strings, the number of rows above its header, the form of its timestamps and the currency marks on its money cells. Every map lands on the same trade fields and the same file, so Generate From YTD reads one format whichever exchange wrote the export.

`src/exchange/ytd_csv_import.py` — the map

```
@dataclass(frozen=True)
class ExportColumnMap:
    """The columns, type strings and text forms of one exchange's CSV export,
    each column named for the ``YtdTrade`` field it fills."""

    exchange_id: str
    export_name: str
    id_column: str
    timestamp_column: str
    type_column: str
    asset_column: str
    quote_column: str
    quantity_column: str
    price_column: str
    cost_column: str
    fee_column: str
    buy_types: frozenset[str]
    sell_types: frozenset[str]
    header_scan_lines: int
    timestamp_suffix_utc: str
    timestamp_format: str
    money_prefixes: str
    dropped_columns: tuple[str, ...] = ()
```

Coinbase's map holds the nine columns, the two type sets, the twenty header rows, the timestamp suffix and the currency marks the import read before the maps existed, so a Coinbase export writes the same files it always did. A map is written from a sample export on disk and never from documentation. An exchange with no map is refused at import by name, with a line saying a sample export is needed, and nothing is written.

`src/exchange/ytd_csv_import.py` — the registry and the refusal

```
EXPORT_MAPS: dict[str, ExportColumnMap] = {COINBASE_MAP.exchange_id: COINBASE_MAP}

def export_map_for(exchange_id: str) -> ExportColumnMap:
    """Return ``EXPORT_MAPS[exchange_id]``, refusing an ``exchange_id`` with no
    map by name."""
```

```
no column map for exchange 'kraken': a sample kraken export is needed before its map is written. Maps exist for: ['coinbase'].
```

The exchanges and their sample exports

One row per exchange Acervator connects to. A map exists for Coinbase alone, written from the operator's own transactions export. No other sample export is on hand, and the export each of the others would map is not named until its sample arrives, because a name taken from documentation is a guess.

```
exchange     map       sample export on hand                          export it maps
coinbase     present   yes, the operator's own transactions export   Coinbase transactions export
binance      none      no                                             not named until a sample arrives
kraken       none      no                                             not named until a sample arrives
kucoin       none      no                                             not named until a sample arrives
bybit        none      no                                             not named until a sample arrives
okx          none      no                                             not named until a sample arrives
gateio       none      no                                             not named until a sample arrives
bitget       none      no                                             not named until a sample arrives
huobi        none      no                                             not named until a sample arrives
mexc         none      no                                             not named until a sample arrives
bitfinex     none      no                                             not named until a sample arrives
gemini       none      no                                             not named until a sample arrives
poloniex     none      no                                             not named until a sample arrives
bitstamp     none      no                                             not named until a sample arrives
cryptocom    none      no                                             not named until a sample arrives
```

A period with no rows is a gap

Where the export carries no trade for a symbol in a period the export covers, that period is written to a gap record. Nothing is invented to fill it.

`src/exchange/ytd_trade_store.py` — the gap record

```
@dataclass
class TradeGap:
    """One period between ``since_ms`` and ``until_ms`` that no row covers."""

    exchange_id: str
    symbol: str
    year: int
    since_ms: int
    until_ms: int
    reason: str
    checked_at: str
```

Reading the trade files

The Simulator reads these files through one class, on the same rule as the tablet reader. It answered six names at the first clone and answers eight since unit 22 (comment 5722467005), refusing every other name itself.

`src/simulator/ytd_trade_source.py` — the refusal

```
    def __getattr__(self, name: str):
        """Refuse every name outside ``READ_NAMES``."""
        raise SendRefused(
            f"YtdTradeSource answers {READ_NAMES} and cannot {name!r}. "
            "The Simulator receives and asks; it sends nothing."
        )
```

What the import run measured

The operator's own export was read into a throwaway directory. The figures below come from that run.

```
export read     5,798 rows, 13 columns
kept            5,766 trades - 3,229 buys, 2,537 sells
dropped         32 - reward income 16, deposits 15, withdrawals 1
written         39 files, one per symbol, all of them 2026
range           2026-04-12 to 2026-09-07
gaps recorded   76
second import   0 rows added, 39 files byte-identical
```

How the Sim tab reaches the bar

`SimulatorTabMixin` builds the tab and inserts it into the window's tab book. The builder catches every error, writes one warning line, and leaves the tab off the bar. A tab that asks the panel for a value the panel does not publish is therefore a missing tab, not a crash the operator can see.

`src/gui/main_tabs/simulator_tab.py` — the builder

```
def _build_simulator_tab(self) -> None:
    """Insert the Sim tab at ``SIMULATOR_BUILD_INDEX``."""
    try:
        from ..variant_surface import SIMULATOR, surface_class

        self._simulator_tab = surface_class(SIMULATOR)()
        self._main_tabs.insertTab(
            SIMULATOR_BUILD_INDEX, self._simulator_tab, HEADING
        )
    except Exception as exc:  # noqa: BLE001 - a missing tab is not a crash
        logger.warning(UNAVAILABLE_LOG, exc)
        self._simulator_tab = None
        self._seat_simulator_build_note(failure_text(exc))
```

The window builds nine tabs. Sim takes the first slot and Paper the second. The Accumulation tab stays off the bar until the operator asks for it.

```
Sim  Paper  Live  Charts  Inspector  Swarm  History  Status  Console
```

A caught build keeps the slot and names what went wrong

The Sim tab no longer drops off the bar. A caught build seats `src/gui/main_tabs/simulator_tab.py, in BuildFailureNote` in the same slot, under the same heading. The page draws three lines: the tab did not build, what the build reported, and that every other tab is working. The warning line is still written, so the log keeps its record as well.

The window attribute stays unset. Every reader of it — the header strip, the Inspector's push and the sector filter — still reads the tab as absent, and the tab that follows no longer slides into the Sim slot.

```
def _seat_simulator_build_note(self, text: str) -> None:
    """Insert ``BuildFailureNote`` carrying ``text`` at ``SIMULATOR_BUILD_INDEX``.

    A tab book that refuses this insert leaves ``NOTE_REFUSED_LOG`` and no
    page, keeping a caught build clear of the window.
    """
    try:
        self._main_tabs.insertTab(
            SIMULATOR_BUILD_INDEX, BuildFailureNote(text), HEADING
        )
    except Exception as note_exc:
        logger.warning(NOTE_REFUSED_LOG, note_exc)
```

Driven with the Sim class refused, beside the same drive accepting as the control:

the build	tabs seated	heading at the Sim slot	lines drawn	warnings
refused	4	Sim	3	1
accepted	4	Sim	the tab's own	0
the real registered class	4	Sim	the tab's own	0

Before this change the refused drive seated 3 tabs, no heading read Sim, and not one of the 8 strings in the tab book named the tab or the failure.

The Qt tab is Live's tab code, forked

The Qt build of the Sim tab is `SimTradingTab`, a fork of the Live tab's own code under the Simulator's names. The seam's Qt loader answers it; the widget written from scratch, `SimulatorTabQt`, stays in `src/gui/simulator_tab.py` and nothing loads it.

`src/gui/variant_surface.py` — the Qt loader

```
def _qt_simulator() -> type:
    """Import and return the Qt Sim tab, ``SimTradingTab``."""
    from .simulator.sim_trading_tab import SimTradingTab

    return SimTradingTab
```

Each module under `src/gui/simulator/` is one Live module copied and renamed. The copy keeps Live's layout, titles, sizes and design tokens; it changes the name, the feed and the send side, and nothing else.

Simulator module	forked from
`sim_trading_tab.py` `SimTradingTab`	`src/gui/main_tabs/trading_tab.py` `_build_trading_tab`, and the window's `add_exchange_tab`
`sim_exchange_tab.py` `SimExchangeTab`	`src/gui/widgets/exchange_tab.py` `ExchangeTab`
`sim_bot_status_table.py` `SimBotStatusTable`	`src/gui/widgets/bot_status_table.py` `BotStatusTable`
`sim_extractor_bot_table.py` `SimExtractorBotTable`	`src/gui/widgets/extractor_bot_table.py` `ExtractorBotTable`
`sim_indicator_panel.py` `SimIndicatorVotingPanel`	`src/gui/indicator_panel.py` `IndicatorVotingPanel`
`sim_status_log.py` `SimStatusLog`	`src/gui/widgets/status_log.py` `StatusLog`

Diagram: flowchart, left to right

- variant_surface SIMULATOR -> **SimTradingTab**
- SimTradingTab -> **exchange layer stack**
- exchange layer stack -> **SimExchangeTab**
- SimExchangeTab -> **SimBotStatusTable**
- SimExchangeTab -> **SimExtractorBotTable**
- SimTradingTab -> **QStackedWidget**
- QStackedWidget -> **SimIndicatorVotingPanel**
- QStackedWidget -> **LineView over PlaybackView**
- SimTradingTab -> **SimStatusLog**
- SimTradingTab -> **API Interaction Log**

What the fork draws

The tab is Live's four splitters at Live's sizes: the exchange layer stack beside the panel on top, the Activity Log and the API Interaction Log side by side below. Every module carries Live's title. The venue page holds the Scrumming Bots table and the Extractor Bots table, both hidden until a row arrives, and the command bar of Start, Pause, Stop, Restart and Delete. The panel holds its title, the Bot selector and its privacy dot, the currency rate strip, both indicator tables, both confidence bar graphs, the timeframe-lock line and the staleness banner, hidden until raised.

Three positions differ from Live by ruling. The corner Live gives ■ `Add Crypto Exchange` holds Import Live Fleet, Generate From YTD and Create New Bots, each at Live's corner-button width and height. The row above the bot list, where Live draws Privacy Mode and the news line, holds Validation, Back Test and Portfolio Battery, then `+ New Bot` where Live draws it. The data-pool row keeps Live's height and holds nothing. Privacy Mode, the news line and the data-pool line are not forked.

`src/gui/simulator/sim_trading_tab.py` — the corner buttons

```
WAY_IN_BUTTONS = (
    (surface.IMPORT_LIVE_FLEET_ACTION, surface.IMPORT_LIVE_FLEET_TEXT),
    (surface.GENERATE_FROM_YTD_ACTION, surface.GENERATE_FROM_YTD_TEXT),
    (surface.CREATE_NEW_BOTS_ACTION, surface.CREATE_NEW_BOTS_TEXT),
)
```

The replay layer sits behind the panel in one stack. The flip button seats itself on whichever layer is showing: after the panel title, or at the head of the replay layer, so the way back is never hidden with the panel.

`src/gui/simulator/sim_trading_tab.py` — the flip

```
        if self._layer == surface.LAYER_PLAYBACK:
            self._indicator_panel.header_row().removeWidget(self._flip_button)
            self._chart_header.insertWidget(0, self._flip_button)
        else:
            self._chart_header.removeWidget(self._flip_button)
            self._indicator_panel.header_row().insertWidget(1, self._flip_button)
```

The seat later moved to the head of both rows, `FLIP_SEAT_INDEX`, and the chart header reads its margins and spacing off the panel's header row, so the button lands at one corner under both layers.

`src/gui/simulator/sim_trading_tab.py` — the chart header's margins

```
        panel_header = self._indicator_panel.header_row()
        self._chart_header = QHBoxLayout()
        self._chart_header.setContentsMargins(panel_header.contentsMargins())
        self._chart_header.setSpacing(panel_header.spacing())
```

What the fork does not carry

The copies hold no bot manager, no connector and no event bus. Every send the Live code makes is cut, not stubbed: the signal-contract emits that write under `~/.acervator_logs/signals/`, the event-bus emit behind the bot selector, the API-log listener, the activity-log watchdog that read the live bot manager, the TA snapshot store that read and wrote `~/.acervator/ta_snapshots/`, the data-pool read behind the display price, and the demo readings the Live panel invents from a random walk when no bot is loaded. Zero occurrences of `ScrummingBot`, `BotContainer`, `ExchangeInterface`, `EventBus` and `PhantomBalance` under `src/gui/simulator/`.

At unit 7 the way-in buttons, the mode buttons, `+ New Bot`, the Fire buttons and the command bar's handler were wired to nothing, and the tab held two sources that nothing read. Units 10 to 31 wired each of them (comments 5706487452, 5707202420, 5708384371, 5714936418, 5716410892, 5718837037, 5721499839, 5728708856, 5733262015); the section "What the tab holds today" describes what each does now.

`src/gui/simulator/sim_trading_tab.py` — the two sources

```
        self._tablet_source = (
            tablet_source
            if tablet_source is not None
            else TabletSource(surface.TABLET_ROOT)
        )
        self._fleet_source = fleet_source if fleet_source is not None else FleetSource()
```

Asked for a send by name, each raises `SendRefused`.

At unit 7 the window's header strip hid while Sim was in front, as `ISOLATED_TABS` then named it. Unit 9 took Sim out of that tuple and fed the strip from the sim fleet (comment 5705741677); the section "The header strip shows on Sim" describes the strip as it draws.

The React page is Live's page modules, forked

The React build of the Sim tab is `SimTradingTabReact`, a fork of the Live tab's React host under the Simulator's name, and it loads seven page modules that are Live's seven copied under Simulator names. The seam's React loader answers it; the page written from scratch, `SimulatorTabReact` in `src/gui/react_simulator_tab.py` with `simulator_tab.js`, stays in the tree and nothing loads it.

`src/gui/variant_surface.py` — the React loader

```
  def _react_simulator() -> type:
      """Import and return the React Sim tab, ``SimTradingTabReact``."""
      from .simulator.sim_react_trading_tab import SimTradingTabReact

      return SimTradingTabReact
```

Each forked module sits beside its source under `src/gui/web/`. The copy keeps Live's layout, part names, sizes and design tokens; it changes the bridge method it asks, the globals it defines, the feed and the send side, and nothing else.

Simulator module	forked from
`sim_trading_tab.js`	`trading_tab.js`
`sim_exchange_tab.js`	`exchange_tab.js`
`sim_indicator_panel.js`	`indicator_panel.js`
`sim_status_log.js`	`status_log.js`
`sim_bot_status_table.js`	`bot_status_table.js`
`sim_extractor_bot_table.js`	`extractor_bot_table.js`
`sim_table_cells.js`	`table_cells.js`
`src/gui/simulator/sim_react_trading_tab.py` `SimTradingTabReact`	`src/gui/react_trading_tab.py` `TradingTabReact`
`src/gui/simulator/sim_trading_tab_surface.py`	`src/gui/main_tabs/trading_tab_surface.py`, the payload builder
`src/gui/simulator/sim_exchange_tab_surface.py`	`src/gui/main_tabs/exchange_tab_surface.py`, the payload builder

Diagram: flowchart, left to right

- variant_surface SIMULATOR -> **SimTradingTabReact**
- SimTradingTabReact -> **sim_trading_tab.js**
- sim_trading_tab.js -> **sim_exchange_tab.js**
- sim_exchange_tab.js -> **sim_bot_status_table.js**
- sim_exchange_tab.js -> **sim_extractor_bot_table.js**
- sim_bot_status_table.js -> **sim_table_cells.js**
- sim_extractor_bot_table.js -> **sim_table_cells.js**
- sim_trading_tab.js -> **sim_indicator_panel.js**
- sim_trading_tab.js -> **replay layer: VWAP over playback**
- sim_trading_tab.js -> **sim_status_log.js**
- sim_trading_tab.js -> **API Interaction Log pane**

What the page draws

The page is Live's four splitters at Live's sizes, read off Live's own `trading_tab_surface`: the exchange layer stack beside the panel on top, the Activity Log and the API Interaction Log side by side below. Every module carries Live's title and Live's part name, so the page and the Qt fork enumerate the same modules in the same order. The venue page holds the Scrumming Bots table and the Extractor Bots table, both hidden until a row arrives, and the command bar. The panel holds its title, the Bot selector and its privacy dot, the currency rate strip, both indicator tables, both confidence bar graphs, the timeframe-lock line and the staleness banner.

The three ruled positions are the Qt fork's. The corner Live gives ■ `Add Crypto Exchange` holds Import Live Fleet, Generate From YTD and Create New Bots, at Live's corner-button width and height, in Live's corner-button chrome. The venue header holds Validation, Back Test and Portfolio Battery in Live's Privacy-Mode sheet, then `+ New Bot` where Live draws it. The data-pool row keeps Live's style and height and holds a blank line.

`src/gui/simulator/sim_trading_tab_surface.py` — the corner buttons

```
WAY_IN_BUTTONS = (
    (sim.IMPORT_LIVE_FLEET_ACTION, sim.IMPORT_LIVE_FLEET_TEXT),
    (sim.GENERATE_FROM_YTD_ACTION, sim.GENERATE_FROM_YTD_TEXT),
    (sim.CREATE_NEW_BOTS_ACTION, sim.CREATE_NEW_BOTS_TEXT),
)
```

The replay layer sits behind the panel in one stack the page draws: the panel slot over a layer holding the VWAP window above the Stone Tablet playback window in one vertical splitter. The flip button sits after the panel title, seated by `sim_indicator_panel.js`, and at the head of the replay layer, seated by `sim_trading_tab.js`, so the way back is never hidden with the panel. A press writes the page's ask on the console line the host reads, and the host redraws the tab with the other layer showing.

`src/gui/simulator/sim_react_trading_tab.py` — the flip

```
        def flip_layer(self) -> str:
            """Swap the page between the panel layer and the replay layer."""
            other = (
                sim.LAYER_PLAYBACK
                if self._state.replay_layer == sim.LAYER_INDICATORS
                else sim.LAYER_INDICATORS
            )
            return self.show_layer(other)
```

The panel module later moved the flip before its title, so both modules draw it first in their header at the panel's top-left corner. Each module's ask carries the pressed button's rect under `flip_rect`, read by the tab module's `flipRect` in the layer stack's coordinates, and `show_layer` emits `sim.layer.flipped` with it on every change of layer.

`src/gui/web/sim_indicator_panel.js` — the header's order

```
    return element(DIV_TAG, headProps, [
      flip,
      element(Title, { key: TITLE_PART, model: model }),
      spacer,
```

What feeds the page

The host builds one payload per bridge method from state it owns: the tab payload from its own `SimTradingTabState`, the Activity Log from its own `StatusLogModel`, the panel from its own `IndicatorPanelModel`, and each seated venue's three payloads from that venue's own `ExchangeTabModel`, its `SimBotStatusTableModel`, the fork unit 13 made of Live's scrum model (comment 5707202420), and Live's `ExtractorBotTableModel`. Live's module-level models are never read. The page asks each method once on mount and the host answers from those payloads; every push, `show_tab`, `show_votes`, `show_log_call` and `add_exchange_tab`, hands the page a fresh payload and redraws it.

`src/gui/simulator/sim_react_trading_tab.py` — the payloads

```
def models(
    state: tab_surface.SimTradingTabState,
    log: status_log_surface.StatusLogModel,
    panel: indicator_panel_surface.IndicatorPanelModel,
) -> dict:
    """The view model of every bridge method the page's modules ask for."""
    return {
        token_surface.METHOD: token_surface.view_model({}),
        tab_surface.METHOD: state.view_model({}),
        LOG_METHOD: log_payload(log, {"whole": True}),
        PANEL_METHOD: panel_payload(panel),
    }
```

What the page does not carry

The host holds the same two sources the Qt fork holds, `TabletSource` and `FleetSource`, and no bot manager, no connector and no bus. The news ticker module, which reaches its feeds over the network, is not in the page's roster. The data-pool line's text and its one-second timer, and Privacy Mode, are not forked. The Activity-Log watchdog and the API-log listener were not forked at unit 8; unit 17 forked the watchdog over the sim bot manager and unit 18 the listener over the sim log (comments 5717588355, 5718837037). Every ask the page makes is answered from the payloads the host holds and written to the console for the host to read. At unit 8 the host answered the flip and held every other press, so the way-in buttons, the mode buttons, `+ New Bot`, the Fire buttons and the command bar changed nothing; units 12 to 31 wired each press (comments 5708384371, 5707202420, 5714936418, 5716410892, 5717588355, 5728708856, 5733262015), and the section "What the tab holds today" describes them.

The header strip shows on Sim

The window's header strip, the five columns and the five counter cards above the tab row, stays on screen while the Sim tab is in front. `ISOLATED_TABS` names the Paper tab alone, so the strip hides on Paper and on nothing else.

`src/gui/main_tabs/main_window_surface.py` — the two tuples

```
ISOLATED_TABS = (PAPER_TAB,)

#: The tabs the header strip reads the Simulator's fleet on, not the live one.
SIM_FED_TABS = (SIM_TAB,)
```

While Sim is in front the ten fields carry the Simulator's figures, read from the Sim tab's fleet source; on every other tab they carry the live fleet's, as before. The window decides on every dashboard tick and again the moment the operator changes tab, so a switch never leaves the other fleet's figures on the strip. The live fleet's numbers do not appear while Sim is in front.

`src/gui/main_window.py` — `_refresh_header_strip`

```
if self._header_strip_reads_sim():
    sim_tab = self._simulator_tab
    self._write_header_strip(
        sim_tab.fleet_source().aggregate(), sim_tab.exchange_count()
    )
    return
if live_stats is None:
    if not self._bot_manager:
        return
    live_stats = self._bot_manager.get_aggregate_stats()
self._write_header_strip(live_stats, len(self._exchange_tabs))
```

Each field keeps the definition the Trading tab page gives it, applied to the sim fleet. SPENDABLE is the sim fleet's wallet cash and LOCKED the value its positions hold. REALISED is the sim fleet's fills matched first-in first-out, one figure per bot, summed. MATURE is the profit on sim positions past two hundred per cent of their cost. EXCH counts the Sim tab's own venue sub-tabs. The five cards are the sim fleet's sold and bought dollars, its trades, its running bots and its errors. The fleet source answers them through one call, in the keys the live

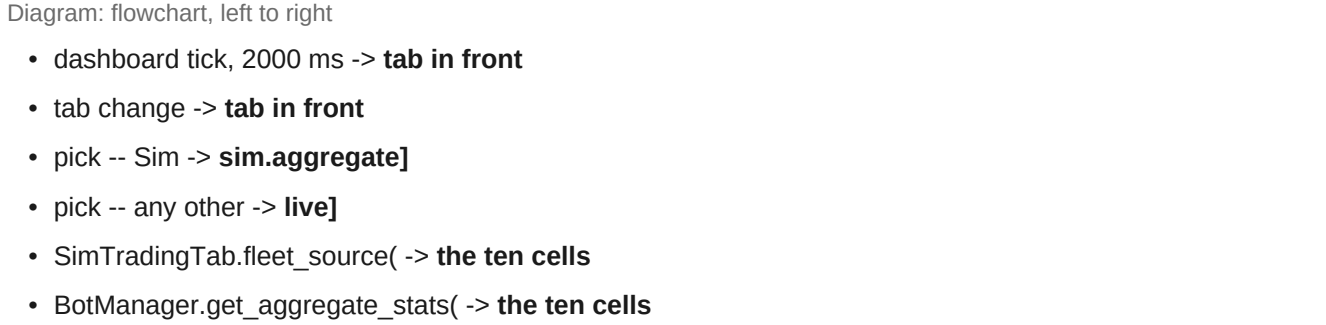
aggregate uses.

`src/simulator/fleet_source.py` — `FleetSource.aggregate`

```
def aggregate(self) -> dict:
    """The header strip's figures for the fleet the Simulator holds.

    No bot is loaded into the Simulator, so the answer is
    ``EMPTY_AGGREGATE``.
    """
    return dict(EMPTY_AGGREGATE)
```

At unit 9 no bot was loaded into the Simulator, so the strip read on Sim as it reads on Live with no bots: the four money columns an em dash, EXCH the count of seated venues, the five cards zero. Unit 21 made `aggregate` answer Live's arithmetic over the held records (comment 5721499839), so the figures arrive with the way-ins that load a fleet and the runs that trade it.



AMMO reads the sim fleet's own total

AMMO on the strip is every sim bot's Target Delta added together, in whole dollars and with no sign, so a bot above its target cannot cancel one below. It draws green while more sim bots hold more than their target and red while more hold less, which is the rule the Trading tab page gives the column. The fleet source publishes the total and the two bot counts the colour reads.

`src/simulator/fleet_source.py` — `aggregate_stats`

```
ammo_readings.append((position_value, bot.live_target_usd, bot.target_usd))
...
"total_target_delta_usd": ammo["total_target_delta_usd"],
"bots_scrum_territory": ammo["bots_scrum_territory"],
"bots_fold_territory": ammo["bots_fold_territory"],
```

One call sums those three figures for every fleet source, so the Simulator's total and Live's cannot be worked out differently.

`src/trading/target_bands.py` — `fleet_ammo`

```
target_usd = ammo_target(live_target, configured_target)
if target_usd > 0:
    total_usd += abs(target_delta(position_value, target_usd))
    where = target_territory(position_value, target_usd)
```

A reading takes the grown target wherever a sim bot carries one, which is the target that bot's own Ammo cell is measured against. The strip's total is therefore the sum of the column above it. A sim bot with no target contributes nothing to either figure.

The venue stack seats the fleet's exchanges

The Sim tab seats one venue sub-tab per exchange the sim fleet names, in both builds. At unit 10 the fleet was the saved bot record the tab's fleet source read, and the exchange set the distinct exchange ids of the stored bots; since unit 21 the fleet is the held records and the Simulator starts empty (comment 5721499839). Each host seats its venues when it is built and again whenever its `fleet_changed` signal fires, which every way-in does. A sub-tab is captioned as Live captions an exchange saved with no name, the capitalised id. Live's venue set is the live configuration; the Simulator never reads it.

`src/gui/simulator/sim_trading_tab.py` — the seating

```
    def _sync_exchange_tabs(self) -> None:
        """Seat a sub-tab for each exchange the fleet names and drop the rest.

        The caption is ``exchange_display_name`` over the id, as Live captions
        an exchange saved with no name.
        """
        wanted = [str(eid) for eid in self._fleet_source.exchanges() if eid]
        for eid in wanted:
            self.add_exchange_tab(eid, exchange_display_name({"exchange_id": eid}))
        self._drop_unlisted_exchange_tabs(wanted)
```

When the fleet names an exchange the bar does not hold, that venue page is seated beside the others. When the bar holds an exchange the fleet no longer names, its sub-tab is taken off, and the Get Started card comes back once a layer's bar is empty. EXCH on the header strip counts the sub-tabs on the crypto layer, as it counts Live's. A seated venue with no bots draws the forked venue page: the mode buttons, `+ New Bot`, the empty data-pool row, the command bar, and both tables hidden until a row arrives.

Diagram: flowchart, left to right

- bot_state.json, through Import Live Fleet -> **FleetSource.exchanges**
- tab build -> **_sync_exchange_tabs**
- fleet_changed -> **_sync_exchange_tabs**
- FleetSource.exchanges -> **_sync_exchange_tabs**
- sync -- not on the bar -> **add_exchange_tab**
- sync -- no longer named -> **_drop_unlisted_exchange_tabs**
- add_exchange_tab -> **the venue sub-tab bar**
- _drop_unlisted_exchange_tabs -> **the venue sub-tab bar**

- the venue sub-tab bar -> **EXCH**

The Get Started card asks for a first fleet

With no exchange seated the layer shows the Get Started card. Its frame, its geometry and the order of its parts are Live's; its contents are the Simulator's. The heading reads `No Crypto Fleet Loaded`, the hint reads `Load a Crypto fleet to begin a run`, and the button position holds Import Live Fleet, Generate From YTD and Create New Bots, one under the other, each at Live's card-button size and in the layer's accent. At unit 10 they were wired to nothing; units 21, 22 and 23 then wired the corner button and the card button of each action together (comments 5721499839, 5722113817, 5723035277).

`src/gui/simulator/sim_trading_tab_surface.py` — the card's texts

```
def placeholder_title_text(label: Any) -> str:
    """The Get Started card's heading for one layer: the first step is a fleet."""
    return f"No {label} Fleet Loaded"

def placeholder_hint_text(label: Any) -> str:
    """The line under the Get Started card's buttons for one layer."""
    return f"Load a {label} fleet to begin a run"
```

The Qt card and the React card read those two functions, so the two builds cannot title the card differently. The venue stack is the first module of the tab to read one of its two sources; the tables, the panel and the two spools still draw empty.

The Scrumming Bots table draws the sim fleet

The Scrumming Bots table on each seated venue draws one row per sim bot on that exchange, in both builds. The rows come from the tab's fleet source: each held record, imported, generated or created (unit 21, comment 5721499839), is one read-only record, and the record answers the same status keys a live bot answers, so the forked table draws it with Live's own cell code. The table stays hidden until a row arrives and hides again when the last row leaves, as Live's does.

`src/gui/simulator/sim_trading_tab.py` — the feed

```
    def refresh_bots(self) -> int:
        """Hand every seated venue its rows from ``FleetSource.statuses``.

        Answers how many rows were handed out. A venue whose list is empty
        hides its tables, as Live's does.
        """
        handed = 0
        for store in (self._crypto_exchange_tabs, self._stock_exchange_tabs):
            for eid, tab in list(store.items()):
                statuses = self._fleet_source.statuses(eid)
                tab.update_bots(statuses)
                handed += len(statuses)
        return handed
```

Both hosts call it after the venues are seated, at build and on every `fleet_changed`. The Simulator runs no refresh timer: a sim figure moves only when a load or a run moves it, and the signal that fires then is what redraws the rows. Live's rows follow the bot manager every two seconds; the Simulator's follow the fleet it holds.

Diagram: flowchart, left to right

- bot_state.json record, through Import Live Fleet -> **SimBot**
- SimBot -> **row_status**
- row_status -> **FleetSource.statuses**
- FleetSource.statuses -> **refresh_bots**
- refresh_bots -> **SimBotStatusTable.update_bots**
- refresh_bots -> **SimVenue.update_bots**
- SimVenue.update_bots -> **sim_bot_status_table.js**

What one record carries

The stored record is read in four parts. The config gives the ids, the symbol, the mode and the gate fields. The stats give the price, the position value, the trade count, the scrummed and folded dollars, the realised profit and the error count. The saved scrumming state gives the grown target the engine re-zeroes to, the quote rate, the phase and the lots. The saved state gives the state the Bot ID cell is coloured by. Holdings are the units summed across the lots, which is how the live bot sets its own holdings.

the cell	reads
Bot ID, its colour	the record's id and its saved state
Symbol	the config symbol and exchange
Current Position Value	holdings, the record's price, the quote rate
Trades	the stats trade count
Target	the grown target, or the config target
Target A15, Target A14	the target over the fleet's own A15 and A14 rows
Ammo	the position value and the target
Fire	the saved phase; the armed action and the gate state read as a live bot's do before its first tick

The runtime values a record does not hold, the armed action, the ceiling ratio and the gate state, read as they do on a live bot before its first tick.

The row's own price, and the fleet's own rates

A sim row is priced from its own record and never from the live data pool. The price age is zero, the sim's own reading for this tick, so the Position Value cell and the Ammo cell price from the same figure, as the Trading tab page requires of them. The Target A15 and Target A14 cells denominate through the fleet's own A15 and A14 rows on that exchange, not through the live currency monitor, and carry no drift suffix because the Simulator has no 24-hour figure to draw one from; with no such row the cell reads `pending`.

`src/gui/simulator/sim_bot_status_table_surface.py` — the price

```
    def _price_reading(self, status, stats) -> tuple:
        """The row's own ``current_price`` at ``SIM_PRICE_AGE_S``, and its ``quote_to_usd``."""
        price = float(stats.get("current_price", live.NO_PRICE))
        quote_rate = float(
            status.get("quote_to_usd", live.DEFAULT_QUOTE_TO_USD)
            or live.DEFAULT_QUOTE_TO_USD
        )
        return price, SIM_PRICE_AGE_S, quote_rate
```

The Qt fork reads the same constant and the same denomination helper, so the two builds price a row identically.

Fire and Detail

Fire is present on every row with Live's styling and tooltips. A press asks the fleet source to fire, and the fleet source refuses every send by name, so the Activity Log carries Live's own failure line, `Fire on <id> failed`, with the refusal's words. No venue and no live bot is reached. The sim's own scrum and fold arithmetic is a later unit; until it lands every Fire on the Simulator is refused this way.

Detail is present on every row. A press opens the Simulator's bot detail window: the frame of Live's Bot Settings window, with the title, the pair and mode header, the state badge, Prev and Next over the venue's sim fleet, the Status tab over the row's figures, and Close. It edits nothing. At unit 13 the six tabs that read a live bot's runtime state, and the React-drawn window, were not forked; unit 13a forked them (comment 5712057220).

`src/gui/simulator/sim_trading_tab.py` — the two presses

```
    def _on_bot_fire(self, bot_id: str) -> None:
        """Manual Fire on a sim bot: ask ``FleetSource`` to ``fire`` and log the
        refusal to the Activity Log, as the window logs a failed Fire."""
        try:
            self._fleet_source.fire(bot_id)
        except SendRefused as exc:
            self._status_log.log(f"Fire on {bot_id[:8]} failed: {exc}", "error")
```

Under React the page's Fire and Detail asks reach the host through the console line, the host applies the ask to that venue's own table model, and the same two handlers run.

Until the way-ins landed

At unit 13 the fleet source read the saved bot record as it lay on disk, so a build with a saved fleet drew that fleet's rows at start. Unit 21 gated both the venues and the rows behind Import Live Fleet, so the tab starts empty (comment 5721499839).

New Bot opens the Simulator's wizard

`+ New Bot` on a seated Sim venue opens the Simulator's Create Auto Trader wizard, a fork of Live's under the Simulator's names, in both builds. It carries Live's five pages in Live's order, Trading Mode, then Select Asset Pair or Extractor Pool, then Trading Parameters, then Phantom Bots, with the same fields, the same ranges, the same walk buttons and the same look, and it opens on the operator's stored defaults as Live's does. Finish creates one sim bot and its row appears in the Scrumming Bots table. Cancel creates nothing.

`src/gui/simulator/sim_trading_tab.py` — the press reaches the fork

```
        tab = SimExchangeTab(
            exchange_id,
            display_name,
            on_new_bot=self._create_bot,
            on_bot_clicked=self._on_bot_detail,
            on_bot_fire=self._on_bot_fire,
            status_log=self._status_log,
        )
```

Diagram: flowchart, left to right

- + New Bot -> **_create_bot**
- _create_bot -> **SimBotCreationWizard or SimBotWizardReactDialog**
- SimBotCreationWizard or SimBotWizardReactDialog -> **get_bot_config** *(Finish)*
- get_bot_config -> **FleetSource.create**
- FleetSource.create -> **fleet_changed**
- fleet_changed -> **refresh_bots**
- SimBotCreationWizard or SimBotWizardReactDialog -> **nothing changes** *(Cancel)*

What the Simulator's wizard reads

The venue list is the seated sub-tabs. The stored defaults come through the same settings reader Live's window holds, so the target balance, the visibility, the aggressive box, the phantom enable and the lock candles open on the operator's own figures; nothing is written. The Target Asset list is the assets the Simulator holds a tablet for on the picked exchange. A tablet is filed by asset and exchange and names no quote, so its asset is offered under every base currency, with no volume figure and no volume sort. Live's wizard asks the venue for its markets; the Simulator's asks the tablets.

`src/gui/simulator/sim_bot_wizard_surface.py` — the market table

```
    for entry in tablet_source.entries():
        exchange_id = str(entry.exchange_id)
        asset = str(entry.asset).upper()
        key = (exchange_id, asset)
        if key in seen or not asset:
            continue
        seen.add(key)
        rows = out.setdefault(exchange_id, [])
        for base in live.BASE_CURRENCIES:
```

What Finish creates

The wizard's config is shaped by the same call Live shapes it with, so a mode-foreign key or a bad shape is refused with Live's own Activity Log line. The shaped config takes the stored-record shape the live process writes and is read through the same record reader the imported fleet uses, so a created bot holds exactly the fields a stored bot's record gives: the ids, the symbol, the target, the timeframe, the interval, the fee and the gate fields. Its id is eight characters, drawn as a live bot draws its own. Its state is idle, so its Bot ID cell is grey. Its price, holdings and trades are zero, as on a live bot before its first tick. An Extractor is refused when no single Scrumming Bot on the venue holds its pool base, with Live's own message.

`src/simulator/fleet_source.py` — the add path

```
    def create(self, config: dict) -> SimBot:
        record = wizard_record(config)
        bot_id = str(uuid.uuid4())[:BOT_ID_LENGTH]
        bot = _sim_bot_from_record(bot_id, record, origin=NEW_ORIGIN)
        if bot is None:
            raise ValueError("the wizard config names no symbol")
        self._created.append(bot)
        return bot
```

The fleet source then fires `fleet_changed`, the venues re-seat, the tables redraw, and the Activity Log carries `Bot <id> created: <symbol> (scrumming) - IDLE`.

What is not forked

Live's pre-flight symbol check contacts the venue; the Simulator offers only the assets it holds a tablet for. The phantom page's API-load warning reads Live's venue call rate; the Simulator makes no venue call, so the page answers True, as Live's React wizard does with no load reading.

A created bot was not persisted at unit 12

At unit 12 the Simulator kept no fleet file of its own, and a created bot lived in the fleet source until the tab closed; unit 12a gave it the sim fleet file (comment 5710961369). Nothing writes `bot_state.json`.

The React host opens the wizard one turn later

Under React the venue page's press reaches the host through the console line. A web page opened inside another page's console callback never finishes loading, so the host opens the wizard on the next event-loop turn, and its page loads.

`src/gui/simulator/sim_react_trading_tab.py` — the deferred open

```
            QTimer.singleShot(
                0, lambda: self._open_bot_wizard(exchange_id, defaults_override)
            )
```

The Extractor Bots table draws the sim extractors

The Extractor Bots table on a seated Sim venue is the forked `SimExtractorBotTable` under Qt and `sim_extractor_bot_table.js` under React, fed from the same `FleetSource.statuses` list the Scrumming Bots table reads, kept to the records whose mode is extractor. It carries Live's eight columns, Bot ID, Symbol, Mode, Trades, Pool, Liquid, Fire and the blank Detail column, with Live's tooltips and Live's two buttons. It is hidden with its label until the venue's fleet holds an extractor, shown when one arrives, and hidden again when the last leaves, while the Scrumming Bots table keeps its own rows.

`src/simulator/fleet_source.py` — the keys an extractor row reads

```
def _extractor_status(bot: SimBot) -> dict:
    """The keys ``ExtractorBot.get_status`` adds over the base status, which
    the Extractor Bots table reads: the base currency, the three pool figures,
    the two position counts and ``extractor_pool_color`` over them."""
    return {
        "base_currency": bot.base_currency,
        "chunk_size_usd": bot.chunk_size_usd,
        "chunk_size_base": bot.chunk_size_base,
        "chunk_free_base": bot.chunk_free_base,
        "n_positions_open": bot.n_positions_open,
        "n_positions_drawdown": bot.n_positions_drawdown,
        "pool_color": extractor_pool_color(
            bot.n_positions_open, bot.n_positions_drawdown
        ),
    }
```

Diagram: flowchart, left to right

- bot_state.json record, through Import Live Fleet -> **SimBot with the pool figures**
- SimBot with the pool figures -> **row_status, the extractor keys added**
- row_status, the extractor keys added -> **FleetSource.statuses**
- FleetSource.statuses -> **refresh_bots**
- refresh_bots -> **SimExtractorBotTable.update_bots**
- refresh_bots -> **ExtractorBotTableModel through SimVenue**
- ExtractorBotTableModel through SimVenue -> **sim_extractor_bot_table.js**

What an extractor record carries

A stored extractor record is read in the same four parts as a scrumming one, plus the extractor block the live bot saves: the dollar size of its pool, the pool in base units, the base units not deployed to any position, and the list of open positions with each one's state. A record the wizard just created holds no such block, so it reads as a live extractor reads before its first rate arrives: the base size and the free base both equal to the dollar size, and no position.

the cell	reads
Bot ID	the record's id
Symbol, its coin icon	the base currency the extractor accumulates

the cell	reads
Mode, its colour and tooltip	the saved state
Trades	the stats trade count
Pool	the pool's dollar size
Liquid, its tooltip	the free base priced at the pool's own dollars per base unit, with the position counts and the base name
Liquid's colour	green with no position open, red with one in drawdown, yellow otherwise
Fire	present and disabled, as on Live
Detail	present

The Pool and Liquid figures are composed by Live's own row code in both builds; the Simulator adds no arithmetic of its own. The pool colour is the live extractor's rule, written once as pure code over the two position counts.

`src/simulator/fleet_source.py` — the pool colour

```
def extractor_pool_color(n_positions_open: int, n_positions_drawdown: int) -> str:
    """The Liquid cell's colour name for one extractor, the rule of
    ``ExtractorBot.pool_color``: ``POOL_GREEN`` with no position open,
    ``POOL_RED`` with one in drawdown, ``POOL_YELLOW`` otherwise."""
    if n_positions_open <= 0:
        return POOL_GREEN
    if n_positions_drawdown > 0:
        return POOL_RED
    return POOL_YELLOW
```

The parent

An Extractor works against a parent base-currency Scrumming Bot. Live's row draws no parent id; what it shows of the parent is the base currency the extractor accumulates, in the Symbol cell and in the Liquid tooltip. The sim row shows the same, from the record's base currency, which is the base the parent scrumming bot on that venue holds.

Detail and Fire on an extractor row

Detail is present on every extractor row in both builds. A press opens the Simulator's bot detail window over that extractor, with its symbol, its mode, its state and its trade count on the Status tab and, since unit 13a, the Settings and Positions Held tabs beside it (comment 5712057220), and selects the row, as a press on the Scrumming Bots table does. Under React the page's ask reaches the host through the console line, the host applies it to that venue's own extractor table model, and the same handler runs.

Fire is present and disabled on every extractor row, as it is on Live's, because Manual Fire is per position for an Extractor. A press reaches nothing: no name is asked of the fleet source and no line is written.

`src/gui/simulator/sim_react_trading_tab.py` — the extractor table's asks

```
        if method == EXTRACTOR_METHOD:
            if (
                params.get(extractor_surface.ACTION_PARAM)
                not in EXTRACTOR_PRESS_ACTIONS
            ):
                return False
            extractor_surface.drive(self.extractor, params)
            return True
```

The Simulator keeps its own fleet file

A bot created through the Simulator's wizard survives a restart, in both builds. The tab's fleet source writes one file of its own, `sim_fleet.json`, under the `sim` bucket of the log root; the parity reports of unit 25 land under the reports bucket instead, and the four log files of unit 31a beside the fleet file (comments 5725784001, 5734233752). Every `fleet_changed` writes it first, before the venues re-seat and the tables redraw, and the tab build reads it once. The write goes through the same helper the live process writes `bot_state.json` with, so a crash mid-write leaves the previous file whole. The section "A created bot is not persisted" above describes the tab before this file existed; its last sentence, that nothing writes `bot_state.json`, stays true.

`src/core/log_paths.py` — the bucket

```
def get_sim_dir() -> Path:
    """``sim/`` bucket — every file the Simulator writes.

    ``src.simulator.fleet_source.FleetSource`` keeps the sim fleet file
    here; ``~/.acervator/bot_state.json`` is never written from the
    Simulator.
    """
    p = resolve_log_root() / "sim"
    p.mkdir(parents=True, exist_ok=True)
    return p
```

`src/gui/simulator/sim_trading_tab.py` — the write hangs on the signal

```
        self.fleet_changed.connect(self._fleet_source.save)
        self.fleet_changed.connect(self._sync_exchange_tabs)
        self._sync_exchange_tabs()
```

Diagram: flowchart, left to right

- Finish in the wizard -> **FleetSource.create**
- FleetSource.create -> **fleet_changed**
- fleet_changed -> **FleetSource.save**
- FleetSource.save -> **sim_fleet.json under the sim bucket**
- fleet_changed -> **refresh_bots**
- the next launch -> **tab build**
- tab build -> **FleetSource reads sim_fleet.json**

- FleetSource reads sim_fleet.json -> **FleetSource.bots**
- FleetSource.bots -> **refresh_bots**

What the file holds

The file carries `saved_at`, `saved_at_human`, `bot_count` and `bots`, the same keys the live state file carries. Each entry under `bots` is the per-bot record the live process writes, `config`, `stats`, `scrumming_state`, `state_when_saved` and `bot_id`, so the one record reader that maps a stored live bot to a row maps a stored sim bot the same way. At unit 12a the file held the wizard's bots and nothing of the live fleet, with the live read beside it; since unit 21 it holds every held record, imported ones included, and the live read runs only through Import Live Fleet (comment 5721499839).

`src/simulator/fleet_source.py` — the write

```
    def save(self) -> Optional[Path]:
        payload = {
            "saved_at": time.time(),
            "saved_at_human": datetime.now().strftime(SAVED_AT_HUMAN_FORMAT),
            "bot_count": len(self._records),
            "bots": dict(self._records),
        }
        path = self.sim_path()
        try:
            atomic_write_json(path, payload, indent=2, default=str)
        except (OSError, TypeError, ValueError) as exc:
            logger.error("sim fleet save failed: %s: %s", path, exc)
            return None
```

Since one fleet per mode landed, the file holds all three fleets. The top carries `saved_at`, `saved_at_human`, `bot_count` summed over the three, and `fleets`, one entry per mode; each entry carries the four keys the block above wrote for the one fleet, so a mode's entry is a file of the old shape. Every fleet change writes all three, so an import under Validation, a bot created under Back Test and a Battery run's generated fleet land in one file and survive a restart under their own modes.

Since the mode memory landed, the top carries one key more: `mode`, named by `MODE_KEY`, holding the mode in force at the write. That is the key `FleetSource.__init__` reads back, so the write that saves a fleet also saves which mode the tab opens in.

`src/simulator/fleet_source.py` — the write, one entry per mode

```
        payload = {
            "saved_at": saved_at,
            "saved_at_human": saved_at_human,
            "bot_count": sum(len(records) for records in self._fleets.values()),
            FLEETS_KEY: {
                mode: {
                    "saved_at": saved_at,
                    "saved_at_human": saved_at_human,
                    "bot_count": len(records),
                    "bots": dict(records),
                }
                for mode, records in self._fleets.items()
            },
        }
```

A mode's entry carries its wires

Each mode's entry also carries `smart_wires` and `smart_wire_ledgers`, the two keys `bot_state.json` carries for the live fleet. A wire belongs to one mode's fleet, so Back Test's wires are saved and read back under Back Test and Validation's under Validation. `FleetSource.hold_wire_manager` takes a `SimWireManager`'s rows onto the mode in force and `FleetSource.wire_manager` builds the manager back at the next launch, so a simulated wire survives a restart the way a live one does.

A file written before these keys existed reads as a fleet with no wire, and a build that does not know the keys reads the same bots, the same mode and the same rows it read before. Both were driven: the unchanged tree's reader and this one gave an identical reading of a file carrying both keys, and the same comparison reported a difference the moment one value was changed by hand.

`src/simulator/fleet_source.py` — the write, per mode

```
            FLEETS_KEY: {
                mode: {
                    "saved_at": saved_at,
                    "saved_at_human": saved_at_human,
                    "bot_count": len(records),
                    "bots": dict(records),
                    WIRES_KEY: [dict(one) for one in self._wires[mode]],
                    WIRE_LEDGERS_KEY: [
                        dict(one) for one in self._wire_ledgers[mode]
                    ],
                }
                for mode, records in self._fleets.items()
            },
```

Absent, empty, malformed

An absent file, or one holding no bytes, gives no sim bots and no log line. A file that is not a JSON object holding a `bots` object gives no sim bots and one warning line naming the file, and the tab builds. A record that names no symbol is kept in the file, is not drawn, and is named in one warning line at build, as the live state file carries forward a record it did not load.

`src/simulator/fleet_source.py` — the read

```
    try:
        loaded = json.loads(text)
    except json.JSONDecodeError as exc:
        logger.warning("sim fleet file %s malformed: %s", path, exc)
        return {}
    stored = loaded.get("bots") if isinstance(loaded, dict) else None
    if not isinstance(stored, dict):
        logger.warning("sim fleet file %s malformed: no bots object", path)
        return {}
```

Since one fleet per mode landed, the read answers three maps. A file holding `fleets` reads each mode's entry through the reading above, one warning per mode for a record that names no symbol; a mode name under `fleets` that is not one of the three is not read and is named in one warning line. A file in the old shape, one `bots` map at the top and no `fleets`, is read as the Validation fleet with the other two empty, and one line says so, so the fleet the operator held before the change is drawn at the next launch under Validation, the mode the tab opens in. The first fleet change after that launch writes the file in the new shape.

`src/simulator/fleet_source.py` — the old shape and the unknown mode

```
    stored_fleets = loaded.get(FLEETS_KEY)
    if isinstance(stored_fleets, dict):
        unknown = [str(name) for name in stored_fleets if name not in MODES]
        if unknown:
            logger.warning(
                "sim fleet file %s names %d mode(s) outside %s and does not "
                "read them: %s",
                path,
                len(unknown),
                MODES,
                ", ".join(unknown),
            )
```

```
    fleets[MODE_VALIDATION] = _records_of(stored, path, MODE_VALIDATION)
    logger.info(
        "sim fleet file %s holds one fleet; read as the %s fleet",
        path,
        MODE_VALIDATION,
    )
```

Driven in the real window in both builds with a scratch home: a bot created through the wizard was read off the file with the typed symbol, target, timeframe and interval, and the second process drew its row with those values; `bot_state.json` hashed the same before and after every step, and a write planted into its path from the sim side moved the hash, so the comparison can report. The file removed drew the live row alone with no warning; the file overwritten with text that is not JSON drew the live row alone with one warning naming it; a file holding one readable record and one that names no symbol drew the readable one and named the other.

Detail opens the Simulator's Bot Settings window with all seven tabs

Detail on a Simulator row now opens Live's Bot Settings window, forked, with every tab Live gives the bot's mode, in both builds. A scrumming bot gets Status, Settings, Fold Tranches, Stack Tranches, Bot Swarm, Market Inspector and Phantom Bots; an extractor gets Status, Settings and Positions Held. The window carries Live's title, Live's header, Prev and Next over the venue's sim fleet, Apply Changes and Close. The section "Fire and Detail" above describes the window before the six tabs were forked; the sentence that they were not forked yet no longer holds, and the sentence that the window edits nothing stays true.

Each tab is fed from the stored record, read as the bot the window reads. `SimBotView` holds one `SimBot` and its raw record and answers the attribute names Live's tabs read off a live bot: the config rebuilt as a restart rebuilds it, the stats, the tranches, the counters and the parked credits from `scrumming_state`, the phantom flag, the phantom timeframes and the lock count from the record's own keys, and the positions and the pool figures from `extractor_state`. Every name Live writes a bot through is refused.

`src/simulator/sim_bot_view.py` — the refusal

```
    def __getattr__(self, name: str) -> Any:
        """Refuse every send in ``SEND_NAMES`` and every runtime name not held."""
        if name.startswith("__"):
            raise AttributeError(name)
        if name in SEND_NAMES:
            return _refusal(name)
        reason = (
            f"SimBotView holds no {name!r}: the stored record carries no such "
            "reading. The Simulator receives and asks; it sends nothing."
        )
        raise SendRefused(reason)
```

Diagram: flowchart, left to right

- Detail on a sim row -> **_on_bot_detail**
- _on_bot_detail -> **SimBotView over the SimBot and its record**
- _on_bot_detail -> **surface_class SIM_BOT_DETAIL**
- surface_class SIM_BOT_DETAIL -> **SimBotDetailDialog, seven forked mixins**
- surface_class SIM_BOT_DETAIL -> **SimBotDetailReactDialog, sim_bot_live_settings.js and eight forked tab modules**
- SimBotDetailDialog, seven forked mixins -> **every press asks the view and is refused**
- SimBotDetailReactDialog, sim_bot_live_settings.js and eight forked tab modules -> **every press asks the view and is refused**

What each tab reads

tab	read from the record	drawn when the record holds nothing
Status	`stats`	the row's figures, as the table draws them
Settings	`config`, the live target, the anchor, the surplus, the consumed budget	Target A15 and Target A14 from the fleet's own A15 and A14 rows; the budget row reads an unreadable dash
Fold Tranches	`fold_tranches`, the parked credits, the ledger, the four counters	no Extractor Tranche row, Live's own answer with no bot manager
Stack Tranches	`stack_tranches`, the two counters	—
Bot Swarm	the `SimWireManager` the view was built with	Live's own line, Bot Swarm not active for this bot
Market Inspector	nothing; a scan is not a record	Live's own no-scan screen; the shared analyzer is never asked
Phantom Bots	`phantoms_enabled`, `phantom_timeframes`, `lock_candle_count`	the runtime rows as a bot before its first tick
Positions Held	`extractor_state.positions`, the four chunk figures	—

The budget row is the one figure the window does not draw. Its arithmetic is a live bot's own, and the unit that shares the trading arithmetic as pure code carries it; until then both builds read `— (unreadable)`, the reading Live gives a cap it cannot read.

Every press is refused

Apply Changes, Reset All Breakers, SELF-DESTRUCT, the three Clear buttons on Fold Tranches, Fire on a fold row, the two Clear buttons on Stack Tranches and Fire on a position are all present, drawn as Live draws them. In the Qt build a press runs Live's own confirmation, then asks the view, and the view's refusal lands in Live's own failure box: `Nothing was cleared — the call failed`, `Manual Fire refused`, `Schedule failed`. In the React build the press reaches the window through the page's console line and the refusal is drawn on the pending-change line. Apply Changes records each edited field as Live does, asks the view for every one on the press, and the line reads `Refused N change(s) — the Simulator sends nothing`. No file is written: `bot_state.json` and the sim fleet file hash the same before and after every press.

`src/gui/simulator/sim_bot_detail.py` — Apply Changes

```
    def _apply_changes(self) -> None:
        """Ask the view for every pending field; each is refused and logged."""
        if not self._changes:
            return
        refused = []
        for field, value in list(self._changes.items()):
            try:
                self._route_change(field, value)
            except SendRefused as exc:
                logger.warning(
                    surface.ROUTE_REFUSED_LOG, self._bot.bot_id[:8], field, exc
                )
                refused.append(
                    surface.APPLIED_REFUSED_FORMAT.format(
                        field=field, value=value, reason=exc
                    )
                )
```

Both builds, resolved as Live resolves

The hosts open the window through the variant seam, beside Live's own pair, so the Qt host opens the Qt fork and the React host opens the React fork. The React fork is Live's React window forked under the Simulator's names: its own page module, `sim_bot_live_settings.js`, eight forked tab modules and one sim surface per tab, each answering its own method name. The React host opens the window one event-loop turn after the page's console line that carried the press, as the wizard is opened, because a web view opened inside another page's console callback never finishes loading.

`src/gui/variant_surface.py` — the pair

```
register(BOT_LIVE_SETTINGS, _qt_bot_live_settings, _react_bot_live_settings)
register(SIM_BOT_DETAIL, _qt_sim_bot_detail, _react_sim_bot_detail)
```

Driven in the real window in both builds with a scratch home and every socket but loopback refused: Detail on the scrumming row read seven tabs off the widget tree and off the page, titled and ordered as Live's window over the same record shape; Detail on the extractor row read three. The Fold Tranches tab drew three rows from a record holding three tranches and two from a record holding two. Next opened the next bot's window on the same tab. Every press above reached the refusal, and `bot_state.json` hashed the same after each one.

The Sim venue pane is Live's width

The Sim tab's top splitter now settles at the same pair as Live's at every width the window can take. Before, the Qt Sim's venue pane was 48 px narrower than Live's at the window's floor and 66 px narrower at 1920 px, in the built program with its theme. The cause was the flip button seated after the panel title: under the theme it asks 134 px, so the forked panel asked 575 px where Live's panel asks 439, and a splitter given a size below a pane's minimum raises that size and keeps the raised figure as the pane's share on every later resize. The Sim divided 600:575 where Live divides 600:500. The replay layer behind the panel asks 8 px and never decided the pair.

The flip button now reports a minimum width of 0 to the header row, and its size policy lets the row shrink it. The panel asks 441 px, under the 500 the splitter is given, so the Sim keeps 600:500 as Live does. At the floor the

header has 185 px of room and the button draws at its full width; dragged narrower than its content, the button gives up width before the pane does, as the React page's flex header already does.

`src/gui/simulator/sim_trading_tab.py` — the flip button

```
class FlipButton(QPushButton):
    """A ``QPushButton`` named ``FLIP_BUTTON_NAME`` whose layout may shrink it to width 0."""

    def __init__(self, text: str, parent: Optional[QWidget] = None) -> None:
        super().__init__(text, parent)
        self.setObjectName(FLIP_BUTTON_NAME)
        self.setAccessibleName(FLIP_BUTTON_NAME)
        self.setSizePolicy(QSizePolicy.Preferred, self.sizePolicy().verticalPolicy())

    def minimumSizeHint(self) -> QSize:  # noqa: N802
        """The base hint's height over a width of 0."""
        return QSize(0, super().minimumSizeHint().height())
```

Read off one window holding both tabs, with the Windows fonts loaded and the theme applied, before and after:

```
window        Live          Sim before    Sim after
1400 x 900    [751, 626]    [703, 674]    [751, 626]
1550 x 974    [833, 694]    [780, 747]    [833, 694]
1856 x 1076   [1000, 833]   [936, 897]    [1000, 833]
1920 x 1080   [1035, 862]   [969, 928]    [1035, 862]
```

The window's floor is its own minimum size, 1400 by 900; a smaller size asked of it settles there. The React page divides its top splitter by the same two figures as flex weights with no floor of its own, so its pair read equal to Live's before this change and after: `[750, 625]` at 1400 and `[999, 832]` at 1856. After the flip, at the settled width, the VWAP window and the Stone Tablet playback window each draw 626 px wide and 248 px high in the Qt build and 625 by 264 in the React build, both inside the pane; a 900 px minimum planted on the replay layer at runtime moves the pair in both builds, and removing it restores the pair.

The command bar acts on the sim fleet

Pause, Stop, Restart and Delete on the Simulator's venue page each change one sim bot's state, and the table reads it back. Start did the same at unit 15; since unit 31 it starts the run in Validation and Back Test mode and moves every scrumming row on the page (comment 5733262015). The bar itself is Live's: it takes its bot from the table you clicked last, tries the other table with no selection there, and with none anywhere says `Select a bot first.` and does nothing, in both builds. The press then reaches the Sim host's `_on_bot_command`, the window's handler forked, which asks the Simulator's bot manager instead of the live one.

`SimBotManager` in `src/simulator/sim_bot_manager.py` is the fork of the parts of `BotManager` that hold and move bots. It holds one `FleetSource` and no venue, no connector, no event bus and no coroutine. Its registry is the sim fleet file's records: `get_bot` answers a held record's `SimBot` and None for an unknown id, so a command on such an id logs Live's `Bot <id> not found.` line and moves nothing; since unit 21 an imported row is a held record and no row is read from `bot_state.json` directly (comment 5721499839). Each verb moves the record's `state_when_saved` under the live bot's own rule for that verb.

`src/simulator/sim_bot_manager.py` — the verbs

```
def start(self, bot_id: str) -> str:
    bot = self._require(bot_id)
    if bot.state in ALREADY_RUNNING_STATES:
        logger.warning("Bot %s already running", bot_id)
        return bot.state
    return self._fleet.set_state(bot_id, BotState.RUNNING.value, clear_error=True)

def pause(self, bot_id: str) -> str:
    self._require(bot_id)
    return self._fleet.set_state(bot_id, BotState.PAUSED.value)

def stop(self, bot_id: str) -> str:
    self._require(bot_id)
    return self._fleet.set_state(bot_id, BotState.STOPPED.value)

def restart(self, bot_id: str) -> str:
    self.stop(bot_id)
    return self.start(bot_id)

def unregister(self, bot_id: str) -> bool:
    return self._fleet.remove(bot_id)
```

The rule per command, read off `BotContainer`, and the Activity Log lines, which are Live's with the venue step absent:

```
Start     running or starting: unchanged, one logger warning; any other
          state: last_error cleared, running          ✓ Bot <id> RUNNING.
Pause     any state: paused                            Pausing bot <id>... / Bot <id> paused.
Stop      any state: stopped                           Stopping bot <id>... / Bot <id> stopped.
Restart   stopped, then running                        ✓ Bot <id> restarted.
Delete    Live's box, "Delete bot <id>? This cannot be undone."; Yes removes the
          record; No changes nothing                   Bot <id> deleted.
```

Start lands on `running` and not `starting`: Live's `starting` lasts until the bot's loop task takes its first step, and the Simulator has no task. Pause has no state guard, as the live bot's `pause` has none. Each success writes the window notification's line, `[notification] Bot <id> RUNNING | success`, through `notification_line` in `sim_trading_tab_surface.py`, and plays Live's state-change sound; a failed verb logs Live's `Failed to <verb> bot <id>` line and plays Live's error sound. The lines Live prints for connecting to the venue, the REAL MONEY warning among them, are absent, because they read a connect result and the Simulator makes no connect.

`FleetSource` gains three names for the held records: `sim_bot_for`, `set_state` and `remove`. They read and move the records the sim fleet file holds and nothing else; `save` stays the one writer, and every other name still raises `SendRefused`.

After every state move, and after a confirmed Delete, the host fires `fleet_changed`: `FleetSource.save` writes the sim fleet file with the moved state, or without the deleted record, and `refresh_bots` hands each venue its rows again, so the Bot ID cell's colour and tooltip in the Scrumming Bots table, and the Mode cell's in the Extractor Bots table, read the new state. `bot_state.json` is not written.

In the React build the venue page's command press is the `command` ask, which the host now answers beside `+ New Bot`; the ask reaches Live's own `ExchangeTabModel.cmd`, whose `Select a bot first.` line reaches the page through the host. The row highlight on the React page comes from the Detail press, as on Live's React page, which has no row click.

Read off the real window in both builds over a scratch sim fleet file holding one scrumming record and one extractor record, both `idle`, Start, Pause, Start, Stop, Restart, Delete on each row:

```
press      Bot ID cell tooltip      colour      sim fleet file
Start      State: RUNNING           #00ff88     running
Pause      State: PAUSED            #ffaa00     paused
Start      State: RUNNING           #00ff88     running
Stop       State: STOPPED           #666666     stopped
Restart    State: RUNNING           #00ff88     running
Delete No  State: RUNNING           #00ff88     running
Delete Yes row gone                             record gone, bot_count one lower
```

A record planted with `state_when_saved` `error` and a `last_error` reads `State: ERROR` in `#ff3366`; Start moves it to `running` and empties `last_error`. Start on a record already `running` leaves the file's bytes unchanged. The live-origin row logs `Bot <id> not found.`. The scratch `bot_state.json` hashes identical after every press, and a write planted into it from outside the program moves the hash. `ScrummingBot`, `ExtractorBot`, `BotManager.register`, `BotManager.unregister`, `BotContainer.start`, `BotContainer.stop`, `BotContainer.pause` and `MainWindow._on_bot_command` were watched for the whole run and saw zero calls; no socket left loopback. `SimBotManager` read for every attribute naming an exchange, a connector, a session, a socket, a client or a network holds zero, beside `BotManager`'s four.

The voting panel reads the Stone Tablet

The Indicator Voting Panel on the Sim tab is fed, in both builds, from the tablets on disk. The Bot selector lists the sim fleet, one entry per accumulation bot in Live's own `symbol [id] (state)` form, and opens on the first. Choosing a bot draws the twelve voters, the Net, Comp and Conf columns, both bar graphs and the lock line for that bot's market on that bot's timeframe, computed by the real voting engine over the last hundred rows of the tablet filed under the bot's asset, exchange and timeframe. The rate strip prices A15 and A14 from the fleet's own rows. Nothing is invented; a bot with no tablet says so.

The reading is one function both hosts call, the window's dashboard feed forked over the tablet reader. It finds the tablet, reads the window, runs the engine, merges each phantom timeframe that has a tablet of its own, and weighs the Comp column over those rows with the same function the live feed weighs by.

`src/gui/simulator/sim_trading_tab_surface.py` — the feed

```
def ivp_feed(source: Any, bot: Any, now: Optional[float] = None) -> dict:
    """The voting panel's reading for ``bot`` over the tablet ``tablet_for``
    finds, with each phantom timeframe merged and ``composite_net`` over the
    phantom rows, or the one empty-state cause; a reading carries ``summary``,
    ``stored``, ``when``, ``age`` and ``message`` for ``show_stored``."""
```

What feeds each part

The Simulator has no venue tick. The panel is fed at tab build, on every `fleet_changed`, and on every change of the Bot selector. A fleet change re-lists the selector, re-prices the rate strip and redraws the chosen bot; a selector change redraws the chosen bot alone. On the Qt build the selector's change is a signal the tab connects; on the React build the page asks the host for the chosen bot, the way the flip button asks for the replay layer.

part	fed from
Bot selector	the fleet's statuses, accumulation bots only
rate strip	the fleet's A15 and A14 rows, priced by the currency monitor's own derivation
both indicator tables	the engine over the tablet window, one row per timeframe
both bar graphs	the same reading; the Qt bars move on their frame timer, the React bars arrive settled as Live's do
Net, Comp and Conf	the engine's net score, the composite over the phantom rows, the consensus confidence
lock line	`No active timeframe locks`; a sim bot holds no coordinator and no lock
staleness banner	the newest candle's time and age, and the day the tablet ends

`src/gui/simulator/sim_trading_tab.py` — the Qt host's feed

```
    def refresh_votes(self) -> dict:
        statuses = self._fleet_source.statuses()
        panel = self._indicator_panel
        held = panel.blockSignals(True)
        try:
            panel.update_bot_list(statuses)
        finally:
            panel.blockSignals(held)
        panel.update_currency_rates(tab_surface.rate_snapshot(statuses))
        return self._feed_votes(panel.selected_bot_id)
```

The banner over a tablet reading

A tablet ends where it ends, and a reading computed on candles that closed weeks ago is not a current one. The panel says so in Live's own words: the amber banner Live raises over a stored reading is raised over every tablet reading, naming the newest candle's time, how long ago that was, and the day the tablet ends. The banner is hidden on every empty state.

```
■ LAST TA READ, NOT CURRENT — taken 11:35:00, 46d 20h ago. Stone Tablet ends 2026-08-01.
```

The weighting behind these votes

A vote is a weighted total. Twelve indicators each cast a direction, and each one's say is scaled by the weight you set on the Settings TA Indicators page. The panel reads those same twelve figures, so the direction and the confidence it draws are the ones a live bot on that market would reach.

`src/gui/main_tabs/simulator_tab_surface.py` — `multi_tf_summary`

```
summary = VotingEngine(weights=weights_in_force()).compute_all(
    parsed, timeframe
)
```

No recorded decision carries the weighting it was made under. The gate log holds the armed flags, the blockers and the figures behind them, and it holds no weight. A reading over past candles is therefore today's weighting over an old tape, and the banner says that beside the figures rather than leaving them to look settled.

`src/gui/main_tabs/simulator_tab_surface.py` — `WEIGHTING_NOT_RECORDED_TEXT`

```
WEIGHTING_NOT_RECORDED_TEXT = (
    "Votes computed at the indicator weights your settings hold now. No "
    "recorded decision carries the weighting it was made under."
)
```

A run says the same thing once, in the Activity Log, and counts how many of the twelve sit away from the built-in figures. A count of zero means your sliders are at the figures the program ships with.

`src/simulator/back_test.py` — `weighting_line`

```
Indicator weighting: this rerun votes at the 12 weights your settings hold
now, 12 of them away from the built-in figures. No recorded gate decision
carries the weighting it was made under, so a weight you have changed since
then moves this reading.
```

Driven on a hand-built tape of 240 candles, with twelve weights written by hand and every one away from the built-in figure, the walk's own recorded votes moved on 211 of 211 ticks, and the ticks whose recorded decision read bearish moved from 5 to 14. The Validation rerun is a separate matter: it reads the recorded direction off the gate row itself and computes no vote, so its reading of a decision is the decision's own.

A bot with nothing to read

An idle or stopped sim bot draws Live's empty state for a bot that is not running, and the reading appears the moment Start moves it, because Start fires `fleet_changed`. A bot in error draws Live's error line. A bot whose asset, exchange and timeframe name no tablet on disk draws the sentence this page already names, and a bot whose tablet holds fewer rows than the engine needs is named with its count.

```
No TA data — bot is idle — a bot that is not running evaluates no TA.
No TA data — No Stone Tablet on disk.
```

A phantom timeframe with no tablet of its own is skipped, and one line names it in the program's log. On the operator's machine every tablet is 5m, so every phantom is skipped today and the Comp column reads the Net column.

One phantom timeframe reaches the Comp column. Two sentences above are overtaken in part. Neither is edited and neither is removed. They read:

> *It finds the tablet, reads the window, runs the engine, merges each phantom timeframe that has a tablet of its own, and weighs the Comp column over those rows with the same function the live feed weighs by.*

The voting panel's reading for `bot` *over the tablet* `tablet_for` *finds, with each phantom timeframe merged and* `composite_net` *over the phantom rows, or the one empty-state cause; a reading carries* `summary`, `stored`, `when`, `age` *and* `message` *for* `show_stored`.

The current reading. The feed still merges a phantom timeframe that has a tablet, and still weighs the Comp column with the live feed's own function. The record's list is reduced to one name first, by the declaration Live reads, so a record naming several reaches the column with the highest-ranked of them alone.

`src/gui/simulator/sim_trading_tab_surface.py` — the feed's phantom read

```
phantom_tfs, _set_aside = one_active_phantom_timeframe(
    list(bot.phantom_timeframes)
)
phantom_tfs = phantom_tfs or default_phantom_timeframes(timeframe)
```

The back test reads the same declaration, in `src/simulator/back_test.py, in phantom_timeframes_for`, so a walk builds one phantom tape where a record names several.

What the reading measured

The real window, both builds, a scratch tablet root holding copies of the A15 and A14 5m tablets, and a scratch sim fleet of five bots. Selecting the A15 bot drew every cell of both tables and every bar equal to the engine's own answer over the same hundred rows, in both builds, and Live's own panel handed the same summary drew the same cells, banner, rate line and lock line. A copy of the A15 tablet with its last twenty closes raised five per cent, filed under another asset, moved the reading: BB from 99 to 69 per cent, Net from –0.79 to +2.11, the consensus from bearish to bullish. The Qt bars read at two frames sixty milliseconds apart differed and settled on their targets; the React bars read equal at two moments, as Live's React bars do. The A23 bot, with no tablet, read the no-tablet sentence; the idle bot read the idle sentence and, after Start on the command bar, the A15 reading. The scratch fleet file and every tablet hashed identical after every reading, and a byte planted into two of them moved the hash. No socket left loopback.

The selector's entry follows the state

Every command on the bar that moves a bot fires `fleet_changed`, and the panel re-reads the fleet on it. On the Qt build the selector is rebuilt whenever any entry's id or text changed, so the entry names the state the bot is in now, in Live's own form; the React page rebuilds its entries on the same signal. After Start, Pause, Stop and Restart the two builds read the same entry, and a bot whose record carries `cooldown` reads `(cooldown)` in both.

`src/gui/simulator/sim_indicator_panel.py` — the entry, one function

```
def _selector_entry_text(status: dict) -> str:
    bot_id = str(status.get("bot_id", ""))
    return ivp.SELECTOR_ITEM_FORMAT.format(
        symbol=status.get("symbol", ivp.DEFAULT_SYMBOL_TEXT),
        short_id=bot_id[: ivp.SELECTOR_ID_PREFIX_LEN],
        state=status.get("state", ivp.DEFAULT_STATE_TEXT),
    )
```

What a fed panel with nothing to draw looks like

A panel fed a bot with no reading draws two empty tables and the words `Awaiting TA signals...` in both bar graphs, on Live and on the Sim tab, in both builds. That picture is the same whether the panel has been fed or not; the cause is held in the panel's own reading and the page's payload, not on the screen. An idle bot at open, or a running bot with no tablet on disk, draws the same pixels as a panel nobody has fed, with the cause under them.

```
No TA data — bot is idle — a bot that is not running evaluates no TA.
No TA data — No Stone Tablet on disk.
```

With a tablet on disk and a bot that is paused, running or in cooldown, the tablet reading draws at open with no press on the selector, in both builds.

The Activity Log spool holds and resumes

The Activity Log at the foot left of the Sim tab is `SimStatusLog`, Live's `StatusLog` under the Simulator's name, seated under Live's title with Live's ■ `Pause Console` toggle in both builds. Every member of Live's pane is on the fork with Live's value: read-only, the placeholder `Activity log...`, 5,000 lines with the oldest dropped past that, a pause buffer of 2,000 that drops the newest when full, a resume that replays the held lines with their original stamps under one line counting them, and `notice` and `force_log`, which draw through a pause. Each line is `[hh:mm:ss] message`, the stamp in the muted colour and the message in its level's colour, with the trade, wire-flow and wire-stack shapes Live raises; since unit 30 a simulated trade's line carries the tape's stamp instead (comment 5731181590). The React page draws the same lines from the host's own `StatusLogModel` through `sim_status_log.js`, the fork of `status_log.js`.

`src/gui/simulator/sim_trading_tab.py` — the toggle's handler

```
        def _on_activity_pause_toggled(checked: bool):
            if checked:
                self._status_log.pause()
                self._activity_pause_btn.setText("■  Resume Console")
            else:
                self._status_log.resume()
                self._activity_pause_btn.setText("■  Pause Console")
```

The page's own press

On the React page the Pause Console press makes two asks, `paused` on the log method and `activity_paused` on the tab method, and the host answers both through one call. It pauses or resumes its log, pushes the batch, and sets the caption, the way the Qt toggle's handler does; a second ask for the state already held changes nothing. The page's own bridge answers a log ask from the document the page holds, so the press keeps every line drawn since the page was built.

`src/gui/simulator/sim_react_trading_tab.py` — the press

```
        def set_activity_paused(self, paused: bool) -> bool:
            wanted = bool(paused)
            if wanted != self._log.paused:
                self.show_log_call("pause" if wanted else "resume")
            if wanted != self._state.activity_paused:
                self.show_tab({tab_surface.ACTIVITY_PAUSED_PARAM: wanted})
            return self._log.paused
```

The watchdog, forked over the sim fleet

Both hosts run Live's Activity-Log watchdog every sixty seconds over the spool's own health and the sim bots the sim bot manager holds. It writes Live's line when the render-error count rises, and Live's silence line when nothing has rendered for ten minutes while a sim bot is running, once per ten minutes, with the critical line at thirty. Each line is force-logged, so a pause cannot hide it. The tick is Live's `watchdog_tick`, one definition, over `SimBotManager.bots`; a row read from `bot_state.json` is not held and is not counted.

`src/gui/simulator/sim_trading_tab_surface.py` — the tick

```
def watchdog_lines(
    state: live.WatchdogState, stats: Any, bots: Any, now: float
) -> list:
    running = live.running_bots(bot.state for bot in bots)
    return live.watchdog_tick(state, stats, now, running, running > 0)
```

What the spool reading measured

The real window, both builds, a scratch sim fleet of one idle bot. The label, the toggle and the pane read the same three rectangles as Live's at 1400 and at the window's floor. Start on the bar drew ✓ `Bot <id> RUNNING.` and its notification; Pause Console then held the four lines Pause and Stop write and drew their two notifications, and Resume Console flushed the four in order under `(resumed — 4 buffered message(s) above)`, in both builds. Ten lines held under a pause flushed in order; 2,011 lines under a pause held 2,000 and dropped the eleven newest; 5,001 lines left 5,000 with the second first. The nine sample lines read the same colour, weight and size on the Sim's pane as on Live's in the same process, both builds. The watchdog's quiet tick wrote nothing; aged past ten minutes with one sim bot running it wrote Live's silence line through a pause. On the base commit the React page's press left the log running and dropped the page to the lines it was built with; on the branch it holds, flips the caption and keeps every line. `bot_state.json` hashed identical after every press, and a byte planted into it moved the hash. No socket left loopback.

The API Interaction Log spool holds and resumes

The API Interaction Log at the foot right of the Sim tab is Live's pane under Live's title with Live's ■ `Pause API Log` toggle in both builds: read-only, the placeholder `API calls, responses, timing, data usage...`, no wrap, 2,000 lines with the oldest dropped past that, and a pause buffer of 2,000. Each host holds an API log of its own, a `SimApiLog` since unit 29 (comment 5730130103) and an `APIInteractionLog` at unit 18, built beside the two sources and read back through `api_log()`, and never the process-wide log the venue connectors write. The writer is Live's `_on_api_event`, forked over that log: one entry becomes Live's block, a stamped head line naming the exchange and the action, a Reason line, then Endpoint, Result, Response and Data usage where the entry holds them. A block arriving while Pause API Log is down is held, and Resume API Log draws the held blocks in the order they arrived under one line counting them. An entry recorded on any thread but the GUI thread is refused and written to the day's thread-violation file under the runtime log directory, as Live's writer does.

`src/gui/simulator/sim_trading_tab.py` — the writer

```
    def _on_api_event(self, entry: dict) -> None:
        current = threading.current_thread().name
        if tab_surface.api_event_off_thread(
            entry, "SimTradingTab._on_api_event", current
        ):
            return
        block_text = tab_surface.api_block(entry)
        if self._api_log_paused:
            buf = self._api_log_pause_buffer
            buf.append(block_text)
            cap = self._api_log_pause_buffer_cap
            if len(buf) > cap:
                del buf[: len(buf) - cap]
            return
        self._api_log_view.appendPlainText(block_text)
```

The two logs

The Sim's log and the venue log are two objects. A venue call the Live bots make is recorded on the process-wide log, whose one listener is the Live tab's writer, so it draws on Live's pane and cannot reach the Sim's. An entry recorded on `api_log()` reaches the Sim's writer alone. At unit 18 nothing recorded into the Sim's log; a Stone Tablet retrieval or update and the YTD trade-file read are the two recorders the directive names, and unit 28 landed the first and unit 29 the second (comments 5728708856, 5730130103).

`src/gui/simulator/sim_trading_tab.py` — the log

```
        self._api_log = api_log if api_log is not None else APIInteractionLog()
        self._bot_manager = SimBotManager(self._fleet_source)
        self._layer = surface.LAYER_INDICATORS
        self._build()
        self._api_log.add_listener(self._on_api_event)
```

Unit 29 changed that constructor to `SimApiLog()` (comment 5730130103).

The block on the React page

The React host's writer pushes each block to the page as one of `api_lines` on the tab method, and the host's own state appends it to its capped pane or holds it in its pause buffer, the way Live's page model does. The page draws the pane's blocks one line each and follows the newest while scrolled to the bottom. A block recorded before the page is built is drawn when the page opens, because the page's tab payload is built from the same state.

`src/gui/simulator/sim_react_trading_tab.py` — the writer

```
        def _on_api_event(self, entry: dict) -> None:
            current = threading.current_thread().name
            if tab_surface.api_event_off_thread(
                entry, "SimTradingTabReact._on_api_event", current
            ):
                return
            self.show_tab({tab_surface.API_LINES_PARAM: [tab_surface.api_block(entry)]})
```

The page's own Pause API Log press

On the React page the Pause API Log press asks `api_paused` on the tab method, and the host answers through one call that pauses or resumes its buffer and sets the caption, the way the Qt toggle's handler does; a resume flushes the held blocks in order under the marker in the same push, and a second ask for the state already held changes nothing.

`src/gui/simulator/sim_react_trading_tab.py` — the press

```
        def set_api_paused(self, paused: bool) -> bool:
            wanted = bool(paused)
            if wanted != self._state.api_buffer.paused:
                self.show_tab({tab_surface.API_PAUSED_PARAM: wanted})
            return self._state.api_buffer.paused
```

Past the pause buffer's cap

Past 2,000 held blocks the two builds part, and each follows its own side of Live. The Qt pane drops the oldest held block, so the flush ends on the newest entry; the React page keeps the first 2,000 and drops the newest, because the page model's buffer is Live's own `ApiPauseBuffer`, imported. The two rules are Live's two rules, and the Sim inherits whichever Live settles on.

`src/gui/main_tabs/trading_tab_surface.py` — the page model's hold

```
    def hold(self, line: Any) -> None:
        """Keep one line for the flush, up to the cap."""
        if len(self.lines) < self.cap:
            self.lines.append("" if line is None else str(line))
```

What the API spool reading measured

The real window, both builds, a scratch sim fleet of one idle bot. The label, the toggle and the pane read the same three rectangles as Live's at 1400 and at the window's floor. One tablet-retrieval entry recorded on the Sim's log drew Live's six-line block on the Sim pane and nothing on Live's; Pause API Log then held two entries and drew nothing, and Resume API Log drew both in order under `--- (resumed; 2 buffered line(s) above) ---`, in both builds. One entry recorded on the process-wide log drew five lines on Live's pane and nothing on the Sim's; the same entry on the Sim's log drew on the Sim's. An entry recorded from a worker thread drew nothing and wrote one line naming the Sim's writer to the thread-violation file under the scratch home. Ten entries held under a pause flushed in order; 2,001 held under a pause kept 2,000, the Qt pane dropping the first and the React page dropping the last; 1,001 more two-line entries left the view at 2,000 blocks with the newest last. On the base commit no log existed for the Sim, nothing drew on its pane from any log, and the React page's press left the buffer running; on the branch it holds, flips the caption and flushes. `bot_state.json` hashed identical after every reading, and a byte planted into it moved the hash. No socket left loopback.

The two strip rows keep Live's heights

The two rows above the bot list on a Sim venue page are Live's rows at Live's heights in both builds. The first is the row Live gives Privacy Mode and the news line: on the Sim it holds Validation, Back Test and Portfolio Battery in Live's Privacy-Mode sheet, then a stretch, then `+ New Bot` where Live draws it, and nothing else. The second is the row Live gives the data-pool line: on the Sim it is Live's label at Live's style holding no text in the Qt build and one no-break space in the React build, so the line keeps its height. No timer runs on a Sim venue page, so nothing rewrites either row after it is built; Live's venue page runs three, the ticker's 15 s step, its hourly refresh and the data-pool line's one-second rewrite. A press on a mode button sends `mode_clicked` with the mode's key and reaches no handler yet.

Neither row takes its height from a constant. Under the theme the header row is as tall as `+ New Bot`, which carries the theme's accent button padding, and the data-pool row is as tall as one line of its 11 px text plus its 2 px padding. The `height_px` figures in `RESERVED_ROWS`, 24 and 18, were read with no fonts and no theme; nothing that runs reads them.

`src/gui/simulator/sim_exchange_tab.py` — the two rows

```
            header = QHBoxLayout()
            self._mode_buttons: dict[str, QPushButton] = {}
            for mode in MODES:
                mode_btn = QPushButton(MODE_TEXT[mode])
                mode_btn.setAccessibleName(MODE_TEXT[mode])
                mode_btn.setFocusPolicy(Qt.NoFocus)
                mode_btn.setStyleSheet(MODE_BUTTON_STYLE)
                header.addWidget(mode_btn)
                self._mode_buttons[mode] = mode_btn
            header.addStretch()
            self._add_bot_btn = QPushButton("+ New Bot")
            self._add_bot_btn.setProperty("accent", True)
            if on_new_bot:
                self._add_bot_btn.clicked.connect(lambda: on_new_bot(exchange_id))
            header.addWidget(self._add_bot_btn)
            layout.addLayout(header)

            self._data_pool_row = QLabel("")
            self._data_pool_row.setAccessibleName(DATA_POOL_ROW_NAME)
            self._data_pool_row.setStyleSheet(
                f"color:{ds.MAIN_BADGE_TEXT}; font-size:11px; padding:2px 6px;"
            )
            layout.addWidget(self._data_pool_row)
```

Read off one window holding both tabs, with the Windows fonts loaded and the theme applied, one venue on Live and the same venue on the Sim, both venue pages handed the same one-bot status list through `update_bots`:

```
row                          Qt Live   Qt Sim   React Live   React Sim
header row                   33        33       33.2         33.2
data-pool row, one bot row   19        19       20.2         20.2
data-pool row, no bot row    407       407      20.2         20.2
```

The Qt data-pool label takes the leftover height on both pages when the bot tables hide, because both `QVBoxLayouts` are Live's; the React line is `flex: none` and keeps its height. The window's floor is 1400 by 900, its own minimum size, so the floor reading is the 1400 reading. The React figures are taken with both web views at device-pixel ratio 1.25; opened Live-first the two views hold 1.25 and 1.0 and each 1 px border snaps to 0.8 px on one side only, which reads as 33.2 against 33.8 and is a fact about the reading, not the page. The header's child list on the Sim reads Validation, Back Test, Portfolio Battery, a stretch and `+ New Bot` in both builds; Live's reads Privacy Mode, the news line and `+ New Bot`. A fourth button planted into the Sim header at runtime reads as a sixth child in both builds and the list returns to five when it is removed. `reserved_rows` is called by nothing in either build, no button carrying a way-in text sits inside a venue page, and `bot_state.json` hashed identical after every reading, while a byte planted into it moved the hash.

The mode choice is wired and the way-ins follow it

The tab holds one run mode, in both builds. The sim fleet file names the mode in force, and the tab opens in the mode that file names. A file naming no mode, or naming one the Simulator does not have, opens the tab in Validation, the first of `MODES`, which is what a first launch reads.

src/simulator/fleet_source.py — the key the file carries

```
#: The sim fleet file's key naming the mode in force, read back by ``__init__``.
MODE_KEY = "mode"
```

A press on Validation, Back Test or Portfolio Battery in a venue page's header makes that mode the run mode; its button carries Live's Privacy-Mode ON sheet and the two others carry the OFF sheet, on every seated venue page at once. The corner Live gives ■ Add Crypto Exchange and the Get Started card's button position each hold the two ways in the run mode offers, and the third is not drawn:

```
run mode            corner and card offer
Validation          Import Live Fleet, Generate From YTD
Back Test           Import Live Fleet, Create New Bots
Portfolio Battery   Run Portfolio, Run Every Portfolio
```

The pairs are ROWS_FOR_MODE, read through reserved_rows, and both builds draw the corner and the card from the same function. Each corner button keeps Live's corner-button minimum width and the 24 px height the fork names, and each card button Live's 180 by 36; the accessible names stay sim-<action> at the corner and sim-<action>-card on the card.

src/gui/simulator/sim_trading_tab_surface.py — the corner buttons per mode

```
def way_in_buttons(mode: str = sim.MODES[0]) -> list:
    """The two corner buttons ``mode`` offers, from ``sim.reserved_rows``, as
    the page draws them."""
    return [
        {
            "action": row["action"],
            "text": row["text"],
            "accessible_name": row["button_name"],
            "minimum_width_px": live.ADD_BUTTON_MIN_WIDTH_PX,
            "minimum_height_px": WAY_IN_BUTTON_HEIGHT_PX,
        }
        for row in sim.reserved_rows(mode)
    ]
```

The sheet each mode button carries comes from one function too, so the Qt header and the React header cannot disagree: mode_style answers Live's PRIVACY_STYLE_ON for the run mode and PRIVACY_STYLE_OFF for the rest. The React mode button also carries aria-pressed, as Live's Privacy Mode button does.

src/gui/simulator/sim_exchange_tab_surface.py — the sheet per button

```
def mode_style(mode: str, active: str) -> str:
    """The sheet the button for ``mode`` carries while ``active`` is the run
    mode: Live's Privacy-Mode ON sheet on the active mode, OFF on the rest."""
    return MODE_BUTTON_STYLE_ACTIVE if mode == active else MODE_BUTTON_STYLE
```

Held by the tab, not by a venue page

Live's Privacy Mode is a per-page button because each page's button mirrors the process-wide mask registry. The run mode is the tab's: the corner and the card belong to the tab's layer, not to a venue page, the card draws when no venue page exists, and one run runs over the whole sim fleet. Two seated venues show the same active button. In the Qt build the tab holds it as `SimTradingTab.mode` and a venue page's press reaches `set_mode`; in the React build the tab state holds it as `SimTradingTabState.mode`, a press sends `mode_clicked` with the mode's key on the venue page's method, and the host's `run_action` hands it to `set_mode`.

`src/gui/simulator/sim_trading_tab.py` — the Qt press

```
    def set_mode(self, mode: str) -> str:
        """Make ``mode`` the run mode: the active sheet on every seated venue
        page's header and the mode's two ways in at each corner and card.
        A name outside ``MODES`` changes nothing; answers the mode in force."""
        if mode not in surface.MODES:
            return self._mode
        self._mode = mode
        for venue in list(self._crypto_exchange_tabs.values()) + list(
            self._stock_exchange_tabs.values()
        ):
            venue.show_mode(mode)
        self._draw_way_ins()
        return self._mode
```

`src/gui/simulator/sim_react_trading_tab.py` — the React press

```
        def set_mode(self, mode: str) -> str:
            """Make ``mode`` the run mode: redraw the tab so each corner and
            card offers that mode's two ways in, and re-publish every venue so
            its header carries the active sheet; answers the mode in force."""
            if self._state.set_mode(mode) == mode:
                self.show_tab({})
                self._venue_published()
            return self._state.mode
```

With no venue seated there is no header row, so nothing on the screen changes the run mode; the Get Started card offers the open mode's two ways in, Import Live Fleet and Generate From YTD, until a fleet is loaded and a venue page draws.

Diagram: flowchart, left to right

- mode button press -> **set_mode**
- set_mode -> **show_mode on every venue page**
- set_mode -> **way_in_buttons of the run mode**
- way_in_buttons of the run mode -> **the corner**
- way_in_buttons of the run mode -> **the Get Started card**
- the corner -> **_way_in**
- the Get Started card -> **_way_in**

- way_in -- Create New Bots -> **the Simulator's wizard**
- way_in -- any other -> **SendRefused on the Activity Log**

Where a way-in press lands

Create New Bots, at the corner or on the card, opens the Simulator's Create Auto Trader wizard through `_create_bot`, the same path `+ New Bot` takes, in both builds. At unit 20 Import Live Fleet, Generate From YTD, Run Portfolio and Run Every Portfolio asked the fleet source for the run by name; the fleet source answered its read set and nothing else, so the ask raised `SendRefused`, and the tab wrote one line to the Activity Log naming the button. Units 21, 22 and 24 built the four runs (comments 5721499839, 5722113817, 5725991788).

`src/gui/simulator/sim_trading_tab.py` — the way-in handler

```
    def _way_in(self, action: str) -> None:
        """One way-in pressed at the corner or on the card: Create New Bots
        opens the wizard through ``_create_bot``; every other action asks
        ``FleetSource`` for it by name, which raises ``SendRefused`` until that
        run lands, and the refusal is logged to the Activity Log."""
        if action == surface.CREATE_NEW_BOTS_ACTION:
            self._create_bot(self._current_venue_id())
            return
        try:
            getattr(self._fleet_source, action)
        except SendRefused as exc:
            self._status_log.log(tab_surface.way_in_refused_line(action, exc), "error")
```

The line reads `Import Live Fleet refused: FleetSource answers (...) and cannot 'import_live_fleet'. The Simulator receives and asks; it sends nothing.`, at the error level, with the button's own text in front.

What the mode reading measured

Read off the real window over a scratch home with every socket but loopback refused, the theme applied and the Windows fonts loaded, once with an empty sim fleet and once with one idle scrumming bot, in both builds:

```
reading                                  Qt          React
run mode at open                         validation  validation
sheets at open                           ON OFF OFF  ON OFF OFF
after Back Test                          OFF ON OFF  OFF ON OFF
after Portfolio Battery                  OFF OFF ON  OFF OFF ON
after Validation                         ON OFF OFF  ON OFF OFF
corner in Validation                     Import Live Fleet, Generate From YTD
corner in Back Test                      Import Live Fleet, Create New Bots
corner in Portfolio Battery              Run Portfolio, Run Every Portfolio
widgets or nodes carrying a mode or way-in
  text, one bot seated                   11          9
  the same, empty fleet                  8           8
```

The Qt count with one bot is the header's three, the two layers' corners at two each, and the two layers' cards at two each; the crypto card is held but not listed once a venue is seated. The React count is the header's three,

the two corners at two each and the hidden stock layer's card at two; the React page draws no card on a layer that holds a venue. With an empty fleet neither build has a header, and the count is the two corners and the two cards. A button planted under the tab with a mode text moved each count by one and the count returned when it was removed. Every corner and card press landed where this section says: four refusal lines on the Activity Log, each naming its button, and the wizard for Create New Bots. `reserved_rows` is called on every draw of the corner and the card, in both builds, and `bot_state.json` hashed identical after every press while a byte planted into it moved the hash.

Import Live Fleet fills the tables and the strip

Import Live Fleet copies the stored bots of one exchange out of the live fleet file into the Simulator's own fleet file, in both builds. A press at the corner or on the Get Started card reads which exchanges the live file names. With more than one it opens the chooser; with exactly one it takes that one and opens nothing; with none it writes one Activity Log line naming the file and moves nothing. Each stored bot on the chosen exchange is copied into the sim fleet under its own bot id, marked as imported, and the fleet signal fires: the sim fleet file is written, the venue seats, and the Scrumming Bots table and the Extractor Bots table draw one row per imported bot, matched by bot id. The Activity Log reads `Imported 3 bot(s) from bot_state.json on coinbase.` The live fleet file is only read, never written.

`src/simulator/fleet_source.py` — the copy

```
    def import_live_fleet(self, exchange_id: str) -> list[SimBot]:
        wanted = str(exchange_id or "")
        if not wanted:
            return []
        imported: list[SimBot] = []
        for bot_id, record in self.stored_records().items():
            bot = _sim_bot_from_record(bot_id, record, origin=LIVE_ORIGIN)
            if bot is None or bot.exchange_id != wanted:
                continue
            record["bot_id"] = bot_id
            record["origin"] = LIVE_ORIGIN
            self._records[bot_id] = record
            imported.append(bot)
        return sorted(imported, key=lambda one: (one.symbol, one.bot_id))
```

A second import of the same exchange replaces each held copy with the file's current record, under the same bot id; a held record the file no longer names stays until Delete removes it. A bot created through the wizard is kept beside the imported ones, so one sim fleet can hold both kinds. The command bar acts on an imported row as it acts on a wizard's: Start on an imported idle bot writes Live's `✓ Bot <id> RUNNING.` line and colours its Bot ID cell.

Diagram: flowchart, left to right

- Import Live Fleet, corner or card -> **_way_in**
- _way_in -> **FleetSource.stored_exchanges**
- options -- more than one -> **SimExchangeChoiceDialog**
- options -- exactly one -> **the exchange**
- chooser -- Ok -> **the exchange**

- chooser -- Cancel -> **one Activity Log line**
- the exchange -> **FleetSource.import_live_fleet**
- FleetSource.import_live_fleet -> **fleet_changed**
- fleet_changed -> **sim_fleet.json**
- fleet_changed -> **the venue seats**
- the venue seats -> **one row per imported bot**

The Simulator starts empty

The tab opens on the Get Started card with no venue seated, in both builds, whatever the live fleet file holds. The fleet source answers the records the sim fleet file holds and nothing else: its bots, its exchanges, its row statuses and its strip figures all come from those records, so with an empty sim fleet there is no venue, no row and no figure until a way in loads one. The live fleet file is opened only when Import Live Fleet is pressed, and then only to read it.

`src/simulator/fleet_source.py` — the held records alone

```
    def exchanges(self) -> list[str]:
        """Every distinct ``exchange_id`` the held bots name, sorted; empty
        until a way in has loaded a fleet."""
        return sorted({bot.exchange_id for bot in self.bots() if bot.exchange_id})
```

On the next launch the tab reads the sim fleet file once and draws the imported rows from it; the live fleet file is not opened. The Bot Swarm tab keeps its own read of the live file, which is Live's and unchanged.

The exchange chooser

The chooser is Live's one-question dialog, the Bot Swarm tab's Configure Profit Wire, holding the bot wizard's `Exchange:` row: the title reads Import Live Fleet, one line names the exchanges, the combo lists each exchange under the caption its venue sub-tab carries, and Ok and Cancel close it. Both builds open the same dialog, as both open Live's Delete box. Cancel writes `Import Live Fleet cancelled.` to the Activity Log and moves nothing.

`src/gui/simulator/sim_exchange_choice.py` — the row

```
            row = QFormLayout()
            self._exchange = QComboBox()
            self._exchange.setAccessibleName(EXCHANGE_CHOICE_ROW_LABEL)
            for entry in exchange_choice_options(ids):
                self._exchange.addItem(entry["display_name"], entry["exchange_id"])
            row.addRow(EXCHANGE_CHOICE_ROW_LABEL, self._exchange)
```

The ten fields on Sim

While Sim is in front the header strip reads Live's ten fields over every record the Simulator holds, imported or wizard-created, with Live's own arithmetic: the wallet is the largest cash balance any held bot carries, a position is holdings times price times the quote rate when both are known, maturity is read only where the venue has answered for that bot, and each year-to-date sum falls back to the lifetime sum at zero. A record that carries no figure for a field reads zero there, as a live bot's fresh statistics do, so a wizard-created bot adds nothing to the money columns until a run trades it, and SPENDABLE and LOCKED keep the em dash until a held record carries cash or a position.

`src/simulator/fleet_source.py` — the strip over the held records

```
    def aggregate(self) -> dict:
        """The header strip's figures over every held record,
        ``aggregate_stats`` of ``bots``."""
        return aggregate_stats(self.bots())
```

The arithmetic is a fork of the live fleet's, not a shared call, because the live version is a method of the bot manager reading live bot objects and the trading package does not change for the Simulator; the maturity threshold is imported from the live module, so the two cannot disagree on it. The strip re-reads on the window's dashboard tick and on the next tab change, as it does on Live after a bot is created.

What the import reading measured

Read off the real window in both builds over a scratch home holding a live fleet file with three scrumming bots and one extractor across `coinbase` and `kraken`, and no sim fleet file:

```
reading                                  Qt                       React
at open: venues seated                   none                     none
at open: Get Started card                listed, visible          drawn, visible
Import Live Fleet: chooser               Import Live Fleet, coinbase and kraken listed, both builds
coinbase chosen: venue seated            coinbase                 coinbase
rows by bot id, Scrumming Bots           u21scr01, u21scr02       u21scr01, u21scr02
rows by bot id, Extractor Bots           u21ext01                 u21ext01
strip after one tick                     $165.50 $155.25 11 1 2 | $1,500.75 $4.20 $367.49 $92.47 1
kraken chosen: venues seated             coinbase, kraken         coinbase, kraken
strip after one tick                     $235.50 $185.25 16 1 3 | $1,500.75 $5.30 $658.49 $92.47 2
aggregate against Live's over the four   21 of 21 keys equal      21 of 21 keys equal
Start on u21scr02                        RUNNING, Bots 2          RUNNING, Bots 2
second process, rows from the sim file   four rows, live file not opened by the Simulator
one exchange in the live file            no chooser, three imported, both builds
no bot in the live file                  one line, nothing moves, both builds
Cancel on the chooser                    one line, nothing moves, both builds
```

The scratch live fleet file hashed the same after every step in every run, and a byte planted into it from outside the program moved the hash. No live bot, bot manager or bot container was constructed, and no socket left loopback.

Generate From YTD fills the tables from the YTD trade files

Generate From YTD builds one simulated bot per pair the operator's own trade record names on one exchange, in both builds. A press at the corner or on the Get Started card first reads the exchange history directory itself: a directory that is missing, empty or without its manifest writes one Activity Log line naming the path and holds nothing. With a manifest, the press reads which exchanges it names; with more than one it opens the chooser under the title Generate From YTD, with exactly one it takes that one, and with none it writes one line naming the manifest. Each traded pair on the chosen exchange becomes one record in the sim fleet under the pair's own id, the symbol at the exchange, marked as generated, and the fleet signal fires: the sim fleet file is written, the venue seats, and the Scrumming Bots table draws one row per pair. The Activity Log reads `Generated 3 bot(s) from 4 YTD trade file(s) on coinbase.` The trade files are only read, never written.

`src/simulator/fleet_source.py` — the generation

```
    def generate_from_ytd(self, source: Any, exchange_id: str) -> YtdGeneration:
        wanted = str(exchange_id or "")
        if not wanted:
            return YtdGeneration()
        entries = [one for one in source.entries() if one.exchange_id == wanted]
        held: list[SimBot] = []
        missing: list[YtdFileEntry] = []
        files_read = 0
        for bot in ytd_fleet(source, wanted):
            own = [one for one in entries if one.symbol == bot.symbol]
            absent = [one for one in own if source.trade_path(one) is None]
            if absent:
                missing.extend(absent)
                continue
            trades: list[YtdTrade] = []
            for entry in own:
                trades.extend(source.trades(entry))
            files_read += len(own)
            target_usd = ytd_target_usd(trades)
            record = ytd_record(bot, target_usd)
            self._records[bot.bot_id] = record
```

Diagram: flowchart, left to right

- Generate From YTD, corner or card -> **YtdTradeSource.root_state**
- state -- missing, empty, no manifest -> **one Activity Log line naming the path**
- state -- ready -> **the exchanges the manifest names**
- options -- none -> **one line naming the manifest**
- options -- more than one -> **SimExchangeChoiceDialog**
- options -- exactly one -> **the exchange**
- chooser -- Ok -> **the exchange**
- chooser -- Cancel -> **one line, nothing moves**
- the exchange -> **FleetSource.generate_from_ytd**
- FleetSource.generate_from_ytd -> **fleet_changed**

- fleet_changed -> **sim_fleet.json**
- fleet_changed -> **the venue seats**
- the venue seats -> **one row per traded pair**

The Target Balance a pair's fills establish

A YTD file names no bot and no target. On Live the target is the dollar value the bot keeps its position at, and the bot reads its position as holdings times price. The pair's own fills establish the position they built, so the generated record's Target Balance is read off them: over every fill of the pair across its year files, oldest first, the units the buys bought less the units the sells sold, valued at the last fill's price, rounded to the cent.

```
units_bought = the amount of every BUY fill, summed
units_sold   = the amount of every SELL fill, summed
units_held   = units_bought - units_sold
last_price   = the price of the last fill
Target       = units_held x last_price, to the cent      when units_held is positive
Target       = none                                      when it is not
```

The fill's cost is its quote leg, the amount times the price before the fee; its net over the fills is the quote spent, not the position's value, so the derivation reads the amount and the price and does not read the cost. When the fills sold more than they bought, the position was opened before the file's first row and the file gives no basis: the record holds no target, the Target cell draws `---`, and the Activity Log says how many records read so. On the scratch files the reading was taken over, A15/USD bought 0.007 and sold 0.002 across two year files, holding 0.005 at a last price of 61,000, so its Target is $305.00; A23/USD bought 1.0 and sold 3.0, so it holds no target.

`src/simulator/fleet_source.py` — the derivation

```
def ytd_target_usd(trades: Sequence[YtdTrade]) -> Optional[float]:
    ordered = sorted(trades, key=lambda one: one.sort_key())
    if not ordered:
        return None
    bought = sum(float(one.amount) for one in ordered if one.side == SIDE_BUY)
    sold = sum(float(one.amount) for one in ordered if one.side == SIDE_SELL)
    held = bought - sold
    if held <= 0:
        return None
    return round(held * float(ordered[-1].price), 2)
```

Under the funding directive, the spendable budget of a Validation or Portfolio Battery run is unbounded and always equals the total of the Target Balances of every simulated bot in the run. A generated bot with no target therefore adds nothing to that budget; the run funds the pairs whose fills established a position and the others stand in the fleet with no target until one is set.

What a generated record carries

The record is the wizard's record shape with the pair's ids and the derived target: the exchange, the quote currency as the base currency, the base asset as the target asset, the symbol, and the Target Balance, or `null` where the fills gave no basis. Every other config field is the bot config's own default, the same a wizard-created bot carries when nothing on the wizard is changed, so a generated bot reads the one-hour timeframe and the default gates. The record carries no statistics, no lots and no phantom set, because the fills are the pair's year of history and not the bot's own figures: the strip's money fields read zero over generated records, Bots counts one when it is started, and a run is what trades it.

`src/simulator/fleet_source.py` — the record

```
def ytd_record(bot: SimBot, target_usd: Optional[float]) -> dict:
    record = wizard_record(
        {
            "exchange_id": bot.exchange_id,
            "base_currency": bot.base_currency,
            "target_asset": bot.asset,
            "symbol": bot.symbol,
            "target_balance": 0.0 if target_usd is None else float(target_usd),
        }
    )
    record["config"]["target_balance"] = target_usd
    record["bot_id"] = bot.bot_id
    record["origin"] = YTD_ORIGIN
    return record
```

A second generation of the same exchange replaces each held copy under the same id with a record re-derived from the files as they lie, so a pair whose files grew reads its new target and a started bot returns to idle; a bot created through the wizard or imported from the live fleet is kept beside the generated ones. The command bar acts on a generated row as it acts on any other: Start on a generated idle bot writes Live's `✓ Bot <id> RUNNING.` line and colours its Bot ID cell.

The empty cases say which

Each case writes one Activity Log line naming the path or the file, in the shape the import's lines take, and holds nothing. The directory is read before any trade file, so a missing directory is named and is not created by the press. A manifest row whose file is gone names that file and leaves that pair out; the other pairs on the exchange are generated.

`src/gui/simulator/sim_trading_tab_surface.py` — the lines

```
GENERATED_FORMAT = (
    "Generated {count} bot(s) from {files} YTD trade file(s) on {exchange}."
)
NO_TARGET_FORMAT = (
    "{count} of them hold no Target Balance: the fills sold more than they bought."
)
GENERATE_CANCELLED_TEXT = "Generate From YTD cancelled."
YTD_ROOT_MISSING_FORMAT = "No YTD trade directory at {path}."
YTD_ROOT_EMPTY_FORMAT = "{path} holds no YTD trade file."
YTD_ROOT_NO_MANIFEST_FORMAT = "{path} holds no {manifest}."
YTD_NO_PAIR_FORMAT = "{manifest} under {path} names no traded pair."
YTD_FILE_MISSING_FORMAT = (
    "{file} named by {manifest} is missing; {symbol} on {exchange} not generated."
)
```

The reader answers eight names now: the state of its directory and the presence of one entry's file joined the six, and the directory it reads is resolved without being created.

`src/simulator/ytd_trade_source.py` — the directory's state

```
    def root_state(self) -> str:
        if not self._root.is_dir():
            return ROOT_MISSING
        if not any(self._root.iterdir()):
            return ROOT_EMPTY
        if not (self._root / MANIFEST_NAME).is_file():
            return ROOT_NO_MANIFEST
        return ROOT_READY
```

What the generation reading measured

Read off the real window in both builds over a scratch home holding three pairs on `coinbase` (A15/USD over two year files, A14/USD, A23/USD) and one on `kraken` (A18/USD), 15 fills in all, a scratch live fleet file that is only hashed, and no sim fleet file:

```
reading                                 Qt                      React
at open: venues seated                  none                    none
Generate From YTD: chooser              Generate From YTD, coinbase and kraken listed, both builds
coinbase chosen: venue seated           coinbase                coinbase
rows by id, Scrumming Bots              A15/USD@coinbase, A14/USD@coinbase, A23/USD@coinbase, both builds
Target cells                            $305.0000, $345.0000, ---, both builds
Activity Log                            Generated 3 bot(s) from 4 YTD trade file(s) on coinbase., both builds
                                        1 of them hold no Target Balance: the fills sold more than they bought.
strip after one tick                    $0.00 $0.00 0 0 0 | — — — — 1, both builds
kraken chosen at the corner             one row, A18/USD@kraken, $36.0000; two venues, both builds
Start on A15/USD@coinbase               ✓ Bot A15/USD@coinbase RUNNING., Bots 1, both builds
a fourth pair planted, coinbase again   four rows, A22/USD@coinbase at $200.0000, both builds
second process, rows from the sim file  five rows; no trade file opened, both builds
no exchange_history directory           No YTD trade directory at <path>., not created, both builds
an empty directory                      <path> holds no YTD trade file., both builds
files without MANIFEST.json             <path> holds no MANIFEST.json., both builds
a manifest naming no pair               MANIFEST.json under <path> names no traded pair., both builds
one named file removed                  <file> named by MANIFEST.json is missing; A14/USD on coinbase not generated.; two rows,
both builds
Cancel on the chooser                   Generate From YTD cancelled.; nothing moves, both builds
```

Every trade file, the manifest and the scratch live fleet file hashed the same after every step in every run, and a byte planted into the live fleet file and into the manifest from outside the program moved the hash. No live bot,

bot manager or bot container was constructed, and no socket left loopback.

Create New Bots fills one row through the forked wizard

Create New Bots is Back Test's second way in, in both builds. It opens the Simulator's Create Auto Trader wizard from the Back Test corner, from the Get Started card when the mode is Back Test, and `+ New Bot` on a seated venue opens the same wizard. The wizard carries Live's five pages in Live's order with Live's fields, and Finish creates one sim bot: its record is held, the sim fleet file is written, the venue the bot names seats if it is new, and the row draws in that venue's Scrumming Bots table or Extractor Bots table. The row carries the wizard's target in the Target cell and the wizard's interval in the status the row is drawn from, and the Bot Settings window opened from the row's Detail shows every value the wizard took.

`src/gui/simulator/sim_trading_tab.py` — the corner and the card reach the wizard

```
        if action == surface.CREATE_NEW_BOTS_ACTION:
            self._create_bot(self._current_venue_id())
            return
```

Diagram: flowchart, left to right

- Create New Bots, corner or card -> **_way_in**
- + New Bot on a venue -> **_create_bot**
- _way_in -> **_create_bot**
- _create_bot -> **SimBotCreationWizard or SimBotWizardReactDialog**
- SimBotCreationWizard or SimBotWizardReactDialog -> **get_bot_config, 49 keys or 13** *(Finish)*
- get_bot_config, 49 keys or 13 -> **FleetSource.create, wizard_record**
- FleetSource.create, wizard_record -> **fleet_changed**
- fleet_changed -> **sim_fleet.json**
- fleet_changed -> **the venue seats, the row draws**
- the venue seats, the row draws -> **Detail: Bot Settings over SimBotView**

Every field the wizard sets, and where it lands

A scrumming walk hands 49 keys and an extractor walk 13, the counts Live's wizard hands from the same window. The record's config holds every field a bot's configuration declares, so a typed value on the Trading Parameters page reaches the Settings tab's control for that field, one control per field.

page	what it sets	the record	the row	the Settings window
Trading Mode	the mode	`config.mode`	which table the row lands in; the Mode cell of an extractor row	seven tabs for a scrumming bot, three for an extractor
Select Asset Pair	exchange, base currency, target asset	the three ids and the symbol	the venue sub-tab; the Symbol cell	the title and the pair
Extractor Pool	exchange, pool base	the same, the target asset *	the Symbol cell reads the pool base	the same

page	what it sets	the record	the row	the Settings window
Trading Parameters	44 settings for a scrumming bot, 7 for an extractor	every one on the config	Target from the target balance; Pool from the chunk size; the Fire tip from the timeframe, the ceiling and the detonation	the Settings tab, one control per field
Phantom Bots	enable, one timeframe, the lock count	the enable flag and the timeframes beside the config	—	the Phantom Bots tab's enable box and timeframe

The three Phantom Bots keys are stored beside the config, as the live process stores them for a running bot: the enable flag as typed, and the timeframes as the bot would hold them, the ticked one, or the one timeframe above the bot's own when none is ticked, kept to the timeframes the venue offers. An extractor stores the flag off and no timeframe. The lock count is the one field the wizard collects that reaches no new bot on Live: the phantom coordinator holds it and a new bot's coordinator holds 2, so the Simulator writes none and its Phantom Bots tab reads 2, as Live's does for a new bot.

`src/simulator/fleet_source.py` — the phantom keys

```
    scrumming = mode == BotMode.SCRUMMING
    record = {
        "config": stored,
        "stats": {},
        "scrumming_state": {},
        "state_when_saved": BotState.IDLE.value,
        "phantoms_enabled": scrumming and bool(collected.get("enable_phantoms", False)),
    }
    timeframes = (
        wizard_phantom_timeframes(
            collected.get("phantom_timeframes"), built.ta_timeframe, built.exchange_id
        )
        if scrumming
        else []
    )
    if timeframes:
        record["phantom_timeframes"] = timeframes
```

The venues the wizard offers

Live's wizard lists the venues connected in settings. The Simulator's markets are its Stone Tablets, so its wizard lists the seated venues first and then every exchange the tablet manifest names, and the Get Started card can create a bot before any fleet is loaded. The section "What the Simulator's wizard reads" above describes the list before the tablets' exchanges were added.

`src/gui/simulator/sim_bot_wizard_surface.py` — the venue list

```
def wizard_exchanges(exchange_ids: Iterable[str], tablet_source: Any) -> list[dict]:
    seated = [str(eid) for eid in exchange_ids if str(eid)]
    filed = sorted(
        {str(entry.exchange_id) for entry in tablet_source.entries()} - set(seated)
    )
    return seated_exchanges([*seated, *filed])
```

The React page narrows its timeframes as the Qt screen does

The Qt wizard lists the picked venue's own timeframes in the TA Timeframe combo and greys the phantom timeframes the venue does not offer. The React wizard takes one list per venue and narrows both pages from it; the React host hands it that list, so both builds show the same timeframes for the same venue. Live's window hands its own React wizard no list, so Live's React page lists the wizard's fixed seven and greys nothing; that is Live's and is not changed here.

`src/gui/simulator/sim_react_trading_tab.py` — the list handed over

```
            wizard = wizard_class(
                exchanges,
                defaults,
                self,
                theme=self._theme,
                markets=markets,
                timeframes=wizard_surface.venue_timeframes(exchanges),
            )
```

The old runner's bot builder is not on the path

`new_bots` in `src/simulator/back_test.py` builds bots from six-key specs for the old tab's Back Test runner, which the window no longer builds. Create New Bots reaches it nowhere: watched across every reading below it counted 0 calls, and a direct call counted 1. It is left as it is.

The route on a fresh Simulator

With no fleet loaded the mode is Validation, the card and the corner offer Import Live Fleet and Generate From YTD, and no header row exists to press Back Test on. The route to Create New Bots is: load a fleet, press Back Test on the header, press Create New Bots at the corner, or `+ New Bot` on the venue. With the mode on Back Test and every bot deleted, the card returns offering Create New Bots.

What the creation reading measured

Driven in the real window in both builds with a scratch home, every socket but loopback refused, a scratch live state file holding one bot on each of two exchanges and a scratch tablet manifest naming four assets on both. Before the change, the three Phantom Bots keys stopped at the wizard: the record carried none, the Phantom Bots tab read the enable box ticked with the wizard's box unticked and the timeframe 1d with 4h ticked, and the card's wizard opened with no venue and Finish was refused for an empty target asset.

```
reading                                              Qt                       React
Back Test pressed: the corner                        Import Live Fleet, Create New Bots     the same
the wizard, every page walked, every field typed     49 keys                  49 keys
record config values against the typed values        47 of 47                 47 of 47
Settings tab controls against the typed values       42 of 42                 42 of 42
Phantom Bots tab: enable, timeframe, lock            off, 4h, 2               off, 4h, 2
the row: Bot ID, Symbol, Trades, Target              id, A15/USD, 0, $1,234.5600 (both)
the row against a target of 200                      the Target cell differs (both)
an extractor, pool base A15 on the same venue        13 keys; 12 of 12; Pool $250.50 (both)
an extractor with no holder                          refused with Live's own box (both)
+ New Bot on the other venue                         one row, $333.3300 (both)
Start on the created row: the strip's Bots           0 to 1                   0 to 1
Spendable                                            a dash before and after (both)
a second process over the same home                  every created row again, every value (both)
every bot deleted under Back Test: the card          Import Live Fleet, Create New Bots (both)
Create New Bots on the card: the venues offered      coinbase, kraken from the tablets (both)
bot_state.json                                       unchanged after every step (both)
ScrummingBot, ExtractorBot, BotManager               0 calls (both)
```

The strip's Spendable reads the fleet's cash balance, which a created bot has none of; the funding rule that makes it the sum of the held Target Balances belongs to the unit that shares the trading arithmetic, for Validation and Portfolio Battery. The venue page's rows in the React build carry no click that selects a row for the command bar, on Live's page and on the Simulator's; Detail selects the row as it opens.

One trading logic: the Simulator sizes its trades with the Live bot's own arithmetic

The Live bot and the Simulator now share one definition of every sizing figure. The position value, the Target Delta, the delta percent, the scrumming interval in USD, the scrum's units, a sale's proceeds net of its fee, the fold's eligible tranches, the per-cycle growth cap, what a fold takes from each tranche, the fold's spend and its units, the position ceiling, the fold taper and the cartridge threshold each live as one function in a module beside the bot's mixins. The bot calls them at the lines that computed them inline, and the Simulator's walk calls the same functions over a simulated position. The module holds no state and imports no bot, no venue and no bus.

`src/trading/scrumming/sizing.py` — three of the figures

```
def scrum_units(delta_usd: float, price: float) -> float:
    return abs(delta_usd) / price

def fold_spend_usd(eligible_usd: float, taper: float) -> float:
    return eligible_usd * taper

def fold_units(spend_usd: float, price: float) -> float:
    return spend_usd / price
```

Unit 26a gave `scrum_units` and `fold_units` a third argument, the unit rule; the section "The unit rule" quotes the three-argument forms (comment 5725310316).

What the tick calls

`ScrummingBot.tick` values the position through `priced_usd`, takes the delta through `target_delta_usd` and its percent through `target_delta_pct`, sizes the interval through `scrumming_interval_usd` and asks `delta_below_interval` whether to hold. The scrum sells `scrum_units` of the delta at the ticker and books the proceeds through `sale_proceeds_usd` less the fee the venue reports. The fold reads `eligible_fold_tranches` under the opposing-trade rebuy factor, plans what it takes from each tranche under `cycle_growth_cap_usd` and `fold_cap_remaining_usd` through `plan_fold_consumption`, spends `fold_spend_usd` of that under the taper, books `fold_units` at the fill and settles the tranches through `settle_fold_plan`.

`src/trading/scrumming_bot.py` — the tick's four figures as they read now

```
delta = target_delta_usd(current_value, self._target_balance)
delta_pct = target_delta_pct(delta, self._target_balance)
_interval_usd = scrumming_interval_usd(
    self._target_balance, self.config.scrumming_interval_pct
)
```

The bot's behaviour was read unchanged off the operator's own logs without building a bot: over 270 scrum fills paired with the gate row the bot wrote at each fire, `scrum_units` over the row's delta and ticker answered the row's filled amount on 270 of 270; over 61 fold fills, `fold_units` over the row's spend and fill answered the filled amount on 61 of 61. One row with its amount moved by one unit in the eighth place was read as a disagreement.

What the Simulator's walk calls

`apply_scrum` sells `scrum_units` of the delta, values them through `priced_usd`, estimates the fee through `estimated_fee_usd` from the bot's Trading Fee, since a walk has no venue to report one, and queues the net proceeds as one fold tranche in the shape the tick builds. `apply_fold` is the tick's fold over the position's tranches: eligibility, the cap plan, the taper when the bot's ceiling is on, the spend and the units, then the settlement. The Target Delta no longer sizes a fold, as it never sized one in the tick.

`src/simulator/back_test.py` — the fold's spend

```
spend = fold_spend_usd(sum(one["usd"] for one in slices), taper)
if funding == FUNDED_BY_PROCEEDS:
    spend = wallet_capped_spend_usd(spend, position.cash_usd)
```

Before this the Simulator sized a fold on the Target Delta and capped it at the cash its scrums had left. Walked over one scratch tape, that sized 46 folds where the Live arithmetic sizes 187, each at most the per-cycle growth cap of its target; the 47 scrums latched on the same candles both ways, and 12 of them sold the same units, the two before the first fold and those where the folds between had brought both positions back to the same holding. A count of arithmetic on a price, units, a target, an interval, a delta, a spend or a fee across `src/simulator` reads zero sizing expressions; the twenty-one that remain value a held position for the strip, a portfolio's baseline and its improvement, an Extractor position's profit, and a candle interval in time.

The funding rule

A Validation or Portfolio Battery run has no venue wallet. Its spendable budget is unbounded and always equals the sum of the held bots' Target Balances; a bot with no Target Balance adds nothing. The run reads that sum once as it starts and carries it as `budget_usd`, and no fold in these two modes is held to the cash the scrums left. Back Test keeps its own rule: each bot's fold spends its own scrum proceeds and no more, through `wallet_capped_spend_usd`, the cap Manual Fire applies at the wallet. A fold sized by the Live arithmetic spends at most the tranche dollars its own scrums queued, so on the same tape the two fundings walked the same 234 trades, one of them apart by two femto-dollars of rounding.

`src/simulator/back_test.py` — the budget

```
def run_budget_usd(bots: Sequence[SimBot]) -> float:
    return sum(float(bot.target_usd) for bot in bots if bot.target_usd is not None)
```

The header strip reads the rule. With the Sim tab in front in Validation or Portfolio Battery, Spendable reads the sum of the held Target Balances; in Back Test it reads what it read before, the largest cash balance the held records carry. Driven with four held bots whose targets summed to twelve hundred dollars, one of them with no target, the cell drew `$1,200.00` in both modes and `$0.00` in Back Test, in both builds, and the walk over the tab's own fleet and tablets called eighteen of the twenty shared functions, every call from `walk`; the two never reached are the cartridge threshold, which only the tick fires, and the wallet cap, which only Back Test applies.

`src/gui/main_tabs/simulator_tab_surface.py` — the mode's funding

```
MODE_FUNDING = {
    MODE_VALIDATION: back_test.FUNDED_BY_TARGETS,
    MODE_BACK_TEST: back_test.FUNDED_BY_PROCEEDS,
    MODE_PORTFOLIO_BATTERY: back_test.FUNDED_BY_TARGETS,
}
```

One drawdown

The state an Extractor writes on a position below its entry value was spelled twice, once in the Extractor and once in the Simulator's fleet reader. Both names now read `DRAWDOWN_STATE` from the sizing module; a planted change to that one definition moved both.

`src/trading/scrumming/sizing.py` — the state

```
DRAWDOWN_STATE = "drawdown"
```

The unit rule: fractional or whole, per asset class, cited from the venue

A Portfolio Battery run walks shares and funds as well as coins, and a share can only be bought the way its venue sells it. Before any non-crypto symbol is simulated, one question is answered per asset class from the venue's own published page: does the class trade in fractions of a unit, or in whole units only. The answer is a row in a table the sizing module carries, and a class with no row is not simulated.

The portfolios name sixty-three symbols and the RA-StoneTablet import asks for the same sixty-three. Thirteen are crypto: A34, ATOM, AVAX, BNB, A15, A22, DOT, A14, A21, MATIC, A23, TRX and A18. The other fifty are

US exchange-listed shares and funds, every one read off the Yahoo route as a share: AAPL, AGG, AMC, AMD, AMZN, ARKF, ARKG, ARKK, ARKW, BABA, BBBY, BIDU, BND, CCIV, COIN, CVNA, DKNG, EXPR, GLD, GME, GOOGL, IEF, IPOF, IWM, JD, KOSS, MCHI, META, MSFT, NIO, NVDA, OPEN, PLTR, PRNT, PTON, QQQ, ROKU, SLV, SNDL, SOFI, SPY, TDOC, TIP, TLT, TSLA, USO, UWMC, VNQ, XLU and ZM. The classes are named with the words the ATA-SPM scanner spells: crypto and stocks. GLD, SLV and USO are the funds the scanner charts for the metals and energy classes; on the venue they are exchange-listed fund shares, so the stocks row binds them. No portfolio names a spot, forward or contract instrument of the metals, energy, forex or derivatives classes, and no connected venue lists one.

The determination

Each page below was opened as a document on 2026-09-17 and its sentences quoted. No endpoint was called, no credential was sent, and no account page was opened. A rule a page does not state is not cited.

class	venue Acervator connects to	fractional	conditions
crypto	Coinbase, the one verified exchange and the venue every held live bot names	yes	the size is a multiple of the product's `base_increment` and at least its `base_min_size`; the connector reads both off the market as the amount precision and the minimum amount
stocks	Alpaca, the one broker connector in the tree, which sends an order's `qty`; no screen constructs it today	yes	exchange-listed common stocks and ETFs, generally over one dollar with a daily volume over ten thousand shares; a per-asset `fractionable` flag read only through the authenticated assets endpoint, so it stays a condition and is not read; market, limit, stop or stop-limit orders; time in force Day; no short sales
metals, energy, forex, derivatives	none	not cited	no connected venue lists the class; not simulated

The crypto row, quoted from the venue's pages

```
https://docs.cdp.coinbase.com/api-reference/exchange-api/rest-api/orders/create-new-order
  titled "Create a new order", no date shown, read 2026-09-17
  "The size can be in any increment of the base currency (e.g. A15 for the
   A15-USD product)."
  "The size can be in incremented in units of base_increment."
  "The size must be no less than the base_min_size and no larger than the
   base_max_size for the product."
https://docs.cdp.coinbase.com/api-reference/advanced-trade-api/rest-api/products/get-product
  titled "Get Product", no date shown, read 2026-09-17
  base_increment: "Minimum amount base value can be increased or decreased
   at once."
```

The stocks row, quoted from the venue's pages

```
https://alpaca.markets/support/fractional-shares
  titled "Does Alpaca support fractional shares?", dated February 2024,
  read 2026-09-17
  "Yes! Alpaca supports fractional trading via API and the dashboard."
  "Fractional Securities include: US Exchange listed Securities (Common
   Stocks and Exchange Traded Funds) Selected Over the Counter American
   Depository Receipts"
  "We generally will allow securities to be fractionable under the below
   scenarios: Exchange Listed Over $1 Average Daily Trading Volume over
   10,000 shares"
  "To see if an asset is fractional, check the field fractionable = true
   from the GET/v2/assets endpoint."
https://docs.alpaca.markets/docs/fractional-trading
  titled "Fractional Trading", updated 2025-09-24, read 2026-09-17
  "Alpaca currently supports fractional trading for market, limit, stop &
   stop limit orders with a time in force=Day, accommodating both fractional
   quantities and notional values."
  "Both notional and qty fields can take up to 9 decimal point values."
  "Not all assets are fractionable yet so please make sure you query assets
   details to check for the parameter fractionable = true."
  "We do not support short sales in fractional orders. All fractional sell
   orders are marked long."
```

Both classes the portfolios name trade fractionally on the venue Acervator connects to for them. No class a Battery run walks today reads whole-unit, so today every walk sizes as it did before; the whole-unit variant below is the switch the table throws the day a row reads whole.

`src/trading/scrumming/sizing.py` — the table and the function that reads it

```
FRACTIONAL_UNITS = "fractional"
WHOLE_UNITS = "whole"

CITED_UNIT_RULES: dict[tuple[str, str], str] = {
    (CLASS_CRYPTO, "coinbase"): FRACTIONAL_UNITS,
    (CLASS_STOCKS, "alpaca"): FRACTIONAL_UNITS,
}

def unit_rule(asset_class: str, venue: str) -> Optional[str]:
    return CITED_UNIT_RULES.get((str(asset_class), str(venue)))
```

A bot's class comes from its symbol and its venue: a name in the crypto list is crypto, every other archive name is stocks, and a name outside the archive on a crypto exchange is crypto. The venue a class trades on is the bot's own exchange for crypto and Alpaca for stocks. TRX has RA tablets from the Yahoo source alone, because Coinbase served it no candle, so its pair is crypto on `yahoo`, which no row cites, and TRX is not simulated.

`src/simulator/portfolios.py` — the class and the venue

```
def asset_class(symbol: str, exchange_id: str = "") -> Optional[str]:
    name = str(symbol or "").upper()
    if name in CRYPTO_SYMBOLS:
        return CLASS_CRYPTO
    if name in SYMBOLS:
        return CLASS_STOCKS
    if str(exchange_id or "") in crypto_venues():
        return CLASS_CRYPTO
    return None

def trading_venue(class_name: Optional[str], exchange_id: str = "") -> str:
    if class_name == CLASS_CRYPTO:
        return str(exchange_id or "")
    if class_name == CLASS_STOCKS:
        return STOCKS_VENUE
    return ""
```

The whole-unit variant

The two sizing functions now take the rule. Under the fractional rule they answer the expression they answered before, unchanged. Under the whole rule they floor to a whole number of units, and a count within one billionth of a whole number reads as that number: a bare floor read one unit under on 11,593 of 200,000 exact multiples of a price in the pre-mortem sweep, and none with the grain, which is the ninth decimal an Alpaca quantity carries at most. A rule that is neither word raises, so no walk sizes under an unnamed rule.

`src/trading/scrumming/sizing.py` — sizing under the rule

```
WHOLE_UNIT_GRAIN = 1e-9

def sized_units(units: float, rule: str) -> float:
    if rule == FRACTIONAL_UNITS:
        return units
    if rule == WHOLE_UNITS:
        return float(math.floor(units + WHOLE_UNIT_GRAIN))
    raise ValueError(f"unit rule {rule!r} is not one of {UNIT_RULES}")

def scrum_units(delta_usd: float, price: float, rule: str) -> float:
    return sized_units(abs(delta_usd) / price, rule)

def fold_units(spend_usd: float, price: float, rule: str) -> float:
    return sized_units(spend_usd / price, rule)
```

The Live bot passes the fractional rule at each of its nine call sites and changes nothing else. The Simulator resolves the rule once per bot as a run reaches it and carries it through the walk: the opening position, every scrum and every fold size under it. A whole-unit scrum whose Target Delta buys fewer than one unit fills nothing and logs why. A whole-unit fold spends the whole units' price; what it could not spend is taken back off the last entries of the fold plan, so that remainder stays in the tranche, and a fold whose spend buys fewer than one unit fills nothing, settles nothing, and logs why.

`src/simulator/back_test.py` — the whole-unit fold

```
    units = fold_units(spend, float(price), rule)
    if units <= 0.0:
        if rule == WHOLE_UNITS:
            logger.info(
                "%s: a fold of $%.2f at %.8f is %s; nothing fills",
                bot.bot_id,
                spend,
                float(price),
                BELOW_ONE_UNIT,
            )
        return None
    if rule == WHOLE_UNITS:
        bought_usd = priced_usd(units, float(price))
        plan = trim_fold_plan(plan, spend - bought_usd)
        spend = bought_usd
```

A whole-unit fold needs tranche dollars of at least one unit's price. At the one percent default growth cap on a five hundred dollar target the cap is five dollars, so a share priced above five dollars would never fold in whole units. This binds the Battery's default Target Balance only if a row ever reads whole; today none does.

Not cited, not simulated

A run resolves each bot's class and venue before it opens a position. A pair with no row in the table is refused: the bot's outcome reads `uncited_rule`, nothing is walked for it, and the run's lines carry one line naming the bot, its class, its venue and that the rule is not cited. The Battery does the same per symbol and counts the symbol's capital as missing weight, and its lines name every refused symbol.

`src/simulator/back_test.py` — the refusal in the runner

```
    for bot in bots:
        class_name, venue, rule = cited_rule_for(bot.asset, bot.exchange_id)
        if rule is None:
            outcomes[UNCITED_RULE] += 1
            results.append(
                BotResult(
                    bot_id=bot.bot_id,
                    symbol=bot.symbol,
                    tablet_key="",
                    outcome=UNCITED_RULE,
                    asset_class=class_name,
                    venue=venue,
                )
            )
            continue
```

`src/simulator/back_test.py` — the line the Activity Log draws

```
def uncited_rule_line(result: BotResult) -> str:
    return (
        f"{result.bot_id}: {result.symbol} is class {result.asset_class or 'none'} "
        f"on venue {result.venue or 'none'}, which has no cited unit rule; "
        "not simulated."
    )
```

What the unit-rule reading measured

Before this, over the unit 26 scratch tape with four scratch bots, an AAPL bot filled 47 scrums and 187 folds in fractional units, and a bot of no class and a A15 bot on an uncited venue each walked the same 234 trades with nothing refused. After, on the same tape: the AAPL bot walks as stocks on Alpaca under the fractional rule, 234 trades, and the A15 bot on Coinbase walks 234 trades identical to unit 26's reading over the same tape, trade for trade; the bot of no class, the A15 bot on Gemini and a TRX bot on the Yahoo route are each refused with their line. With the stocks row planted whole for one run, the AAPL bot walked 233 trades, every one a whole number of units between one and ten, every dollar figure equal to the units times the price, two scrum latches and two fold latches unfilled below one unit, and the A15 bot unchanged; with the crypto row planted whole instead, the A15 bot at a one thousand dollar target in a hundred dollar asset opened ten units and filled nothing, because its one percent interval never reached one unit, and the AAPL bot was unchanged. Each plant was removed and the table read as before.

On the running Sim tab in both builds, one A15 bot and one AAPL bot imported through Import Live Fleet in Validation over scratch tablets, `back_test.run` under the debugger with a breakpoint on every pure sizing function: the rule was read twice, once per bot, from the runner; 470 sizing calls, 94 scrum sizes and 376 fold sizes, every one with `walk` on the stack; the A15 trades equal to unit 26's reading; under the planted stocks row the AAPL trades all whole and the plan trimmed on all 185 folds. Two bots imported on Gemini were refused and their two lines, drawn through the tab's own log method, read back off the Qt widget and off the React page. No bot was constructed and no socket left loopback. The Live replay over unit 26's copied logs with the fractional rule answered 270 of 270 scrum fills and 61 of 61 fold fills, and the same over a fresh copy taken today; one row with its amount moved by one unit in the eighth place read as a disagreement.

The trade parity report every run writes

Every Simulator run writes one report file when it ends: a Validation run, a Back Test run, a Portfolio Battery run. The gate-light comparison lives in that file and nowhere on the tab. The report names what the run verified and what it could not, and it carries the run's budget under the funding rule and every symbol the run refused. Nothing on the Sim tab draws a light, a table or a verdict for a run; when a run ends the Activity Log carries one line naming the report's path, and the operator opens the file from that path.

`src/simulator/parity_report.py` — the writer

```
def write_report(
    mode: str, run: object, tablets: Optional[TabletSource]
) -> ParityReport:
    """Write the report of a finished ``run`` in ``mode`` and answer where it
    landed."""
    stamp = utc_stamp()
    return write_figures(FIGURES[mode](run, tablets, stamp), stamp)
```

Diagram: flowchart, left to right

- validation.run, back_test.run or portfolio_battery.run_battery -> **the pass over the tablets**
- the pass over the tablets -> **one dict of figures**
- one dict of figures -> **the Markdown file**
- one dict of figures -> **the JSON sidecar**
- the Markdown file -> **reports/simulator under the log root**
- the JSON sidecar -> **reports/simulator under the log root**
- validation.run, back_test.run or portfolio_battery.run_battery -> **the run object carries ParityReport**
- the run object carries ParityReport -> **log_report on the Sim tab: one Activity Log line naming the path**

Where the report lands

The file lands under the `reports` bucket of the log root, in a `simulator` directory of its own, beside the version-sweep reports the bucket already holds. One run writes one Markdown file and one JSON sidecar with the same name. The name is the mode, the subject, the span and a UTC stamp to the microsecond, joined by two underscores, so two runs never share a name and no report is ever overwritten: the Markdown is created exclusively, and a name already taken moves to the next suffix.

```
validation__coinbase-live__2023-11-14_2023-11-17__20260918T052817474607Z.md
validation__coinbase-live__2023-11-14_2023-11-17__20260918T052817474607Z.json
back_test__coinbase-live__2023-11-14_2023-11-17__20260918T052820763009Z.md
portfolio_battery__DEGEN__2022__20260918T052822879951Z.md
```

The subject is the exchange and the fleet's origin for Validation and Back Test, and the portfolio's name for a Battery, or `every-portfolio` when every portfolio ran. The span is the first and last day of the tape the run read, or the Battery's own span label.

`src/simulator/parity_report.py` — no report is overwritten

```
def create_pair(directory: Path, stem: str) -> tuple[Path, Path]:
    """The first ``stem`` under ``directory`` whose Markdown and JSON paths both
    do not exist, the Markdown created exclusively so no report is overwritten."""
```

What the report holds

Both files carry the same figures. The Markdown is for a person; the JSON is for a program, and a later unit reads the four comparison counts from it. The two cannot disagree on a number, because the Markdown is rendered from the same dictionary the JSON stores.

Every mode carries these sections:

- **Run.** The mode, the UTC stamp, the build, the fleet's origin, the exchange, the span, the bot count, and the funding with the budget under it. Validation and Portfolio Battery fund from the held Target Balances: the budget is their sum and no fold is capped for cash. Back Test funds each bot's fold from its own scrum proceeds and carries no run budget. A Validation run states that it reruns gates and sizes no trade, so no buy was refused for cash.

- **Tablets.** Every Stone Tablet the run read: its key, asset, exchange, timeframe, year, candle count, first and last candle, and the checksum the MANIFEST row carries.
- **Bots.** One row per bot: symbol, Target Balance, origin, class, venue, the unit rule the pair trades under and whether the pair has a cited row, the outcome, and what the run produced for it. Validation carries the snapped and unsnapped trade counts and the rows compared and latching; Back Test carries the trades, scrums, folds, units gained, cash and fees; the Battery carries each symbol's capital and its weight of the portfolio, the trades, the units gained, and the buy-and-hold baseline beside the accumulation.
- **Not verified.** One line per thing the run could not verify. The section is never empty: a run that verified everything says so in one line.
- **Lines.** The run's own Activity Log lines, verbatim.

The gate-light comparison

A Validation report carries every rerun row: the trade, the bot, the trade's stamp, the candle's and the gate row's, whether the row latches identically, how many of the nineteen lights agreed, the armed flags on both sides, every disagreeing light with its recorded state, its rerun state and which side drove it, and any blocker phrase that maps to no light. Above the rows sit the agreement counts and four counts under the words the module comparison uses, applied to the two sides of one gate row: the record the live bot wrote, and the rerun the Simulator latched.

count	what it reads
missing	a blocker the recorded row names that maps to no gate light
extra	a blocker the rerun raised that maps to no gate light
differs	a light in both whose recorded and rerun states differ
variant differs	a row whose nineteen lights all agree and whose armed flags differ

Each count is written beside its definition, in both files.

`src/simulator/parity_report.py` — the four counts

```
def comparison_counts(rows: Sequence[RowComparison]) -> dict:
    """``summarise`` over ``rows`` and the four ``COUNT_NAMES``."""
    counts = dict(summarise(rows))
    counts[MISSING] = sum(len(one.unknown_recorded_blockers) for one in rows)
    counts[EXTRA] = sum(len(one.unknown_rerun_blockers) for one in rows)
    counts[DIFFERS] = sum(len(one.disagreed) for one in rows)
    counts[VARIANT_DIFFERS] = sum(
        1 for one in rows if not one.disagreed and not one.latches_identically
    )
    return counts
```

The Portfolio Battery section

A Battery report carries, per portfolio and timeframe, how many symbols ran, the capital committed, the capital missing and its share of the weight with the symbols that carry it, the arithmetic of accumulation less baseline with the verdict word, the units gained, the trades and the fees; then every refused symbol with the line the run wrote for it. At unit 25 the defend, improve or reverse outcome against the portfolio's historical path read `not computed`, with why: the design intent's default Target Balance, mix share and reading were the next Battery unit's, and the run walked every symbol at the same capital with equal weights. Unit 24 wrote the reading (comment 5725991788); the section "Portfolio Battery Mode" describes it.

Since the Battery flow landed, that outcome is computed. The historical path is buy and hold over the span: its start is the Target Balances committed, its trough is the sum of each symbol's lowest untraded value on a walked bar, and its end is the baseline. The scrummed path ends at the accumulation. The reading is the highest of the three marks the scrummed end clears, and the report carries it per portfolio and timeframe with the four figures written out, then counts the readings per timeframe.

`src/simulator/portfolio_battery.py` — the reading

```
def reading_for(
    ran: bool, start_usd: float, trough_usd: float, end_usd: float, scrummed_usd: float
) -> str:
    """The highest historical mark ``scrummed_usd`` clears: ``REVERSED`` at or
    above ``start_usd`` when ``end_usd`` fell below it, ``IMPROVED`` above
    ``end_usd``, ``DEFENDED`` above ``trough_usd``, else ``UNDEFENDED``;
    ``NOT_RUN`` while ``ran`` is False."""
    if not ran:
        return NOT_RUN
    if end_usd < start_usd and scrummed_usd >= start_usd:
        return REVERSED
    if scrummed_usd > end_usd:
        return IMPROVED
    if scrummed_usd > trough_usd:
        return DEFENDED
    return UNDEFENDED
```

The report's header names the run's budget as the sum of the run's bots' Target Balances, the origins those bots carry, and, per portfolio, whether the bots were generated or loaded from the held fleet.

A run that raises

A run that raises part-way still writes a file. It carries the header the run had, every bot it was handed, the tablets if the source still answers, the exception's type and message, and a Not verified section saying nothing was verified. Then the exception propagates as it did before. A report that cannot be written raises to the caller, so a run whose report is missing is never reported as done.

`src/simulator/validation.py` — the wrap every runner carries

```
    try:
        outcome = _validate(bots, tablets, ytd, gates, exchange_id, limit, lag_sample)
    except Exception as exc:
        write_partial(
            VALIDATION, exc, bots=bots, exchange_id=exchange_id, tablets=tablets
        )
        raise
    return replace(outcome, report=write_report(VALIDATION, outcome, tablets))
```

The line on the tab

Each Sim host answers `log_report`, which writes one line through the tab's own Activity Log at the level Import Live Fleet uses. At unit 25 no button started a run on the tab; unit 24's Run Portfolio and unit 31's Start call it with the run's report (comments 5725991788, 5733262015).

Since the Battery flow landed, Run Portfolio and Run Every Portfolio start a run from the corner and the card and call it with the run's report when the run ends.

```
Portfolio Battery report written: <log root>/reports/simulator/portfolio_battery__CRYPTO_BLUE__2022__20260918T063630643184Z.md
```

```
Validation report written: <log root>/reports/simulator/validation__coinbase-live__2023-11-14_2023-11-17__20260918T052817474607Z.md
```

What the report reading measured

Before this, over a scratch home holding four stored bots, three Stone Tablets, three RA-StoneTablets, three YTD trade files and a gate.log of six rows, the three runners answered their lines and the reports bucket held no file after all three. After, each run wrote one Markdown file and one JSON sidecar under the simulator directory of the bucket, and running each mode twice wrote twelve files with none overwritten. The Validation sidecar's four counts read equal to the Markdown's. With a raise planted inside the tablet read, each mode wrote a partial file naming the planted exception and the exception still propagated. With one rerun row rewritten to disagree on one light, another rewritten to agree on every light with its scrum flag flipped, and a third given one recorded and two rerun blocker phrases that map to no light, the counts moved from missing 0, extra 0, differs 40, variant differs 0 to 1, 2, 18 and 1, in both files. Every scratch file hashed equal before and after every run, and a second copy with one byte flipped read as changed.

On the running Sim tab in both builds, the four bots imported through Import Live Fleet, each runner driven over the tab's own fleet source and tablet source under the debugger with a breakpoint on the writer: the writer was reached once per run with the runner's own `run` on the stack, the file appeared under the scratch reports directory, the sidecar's counts read equal to the Markdown's, and the Activity Log line naming the path was drawn through the tab's own log and read back off the Qt widget and off the React page. No bot was constructed and no socket left loopback.

The Simulator colour distinction is the theme's nigredo tone

The Sim tab paints every ground darker than Live's, under the same theme. The tone is the theme's own ground tokens moved a quarter of the way to black; the accent, the text and the borders are the theme's. The derivation, its one fraction and the contrast arithmetic per theme are on the Settings page under the Simulator's tone, nigredo.

Nothing on the tab names a colour for this. The Qt build marks the tab with one property, and the theme's stylesheet paints the tree it marks; the windows the tab opens are its children, so the chooser and the Bot Settings window paint in the tone too.

`src/gui/simulator/sim_trading_tab.py` — the mark

```
# The theme's nigredo_qss paints this tree, and the windows it parents.
# A QWidget subclass paints its stylesheet ground only with this attribute.
self.setProperty(TONE_PROPERTY, NIGREDO)
self.setAttribute(Qt.WA_StyledBackground, True)
```

The React build marks its host the same way for the Qt windows it parents, and its page takes the tone twice: once at build, when `panel_html` hands the tone to the page chrome, and once per theme switch, when the window's repaint reads the tone off the web view. The design tokens the page embeds go through the same function: every token whose name, or whose alias target, starts with `SURFACE_` is a ground the page paints, and the host darkens it before the page reads it.

`src/gui/simulator/sim_react_trading_tab.py` — the page side

```
GROUND_NAME_PREFIX = "SURFACE_"

def nigredo_design(model: dict) -> dict:
    aliases = model.get("alias_targets", {})

    def is_ground(name: str) -> bool:
        return name.startswith(GROUND_NAME_PREFIX) or str(
            aliases.get(name, "")
        ).startswith(GROUND_NAME_PREFIX)
```

```
return page_html(
    STYLE_ASSETS, (), page_body(), theme, (host_script(built, venues),), NIGREDO
)
```

The two Sim windows drawn by React, the Bot Settings window and the wizard, hand the same tone to their page chrome and carry it on their web views.

What the tone reading measured

The real window was built in each build over a scratch home with every socket but loopback refused, the theme switched five times through the Theme menu's own path, and the picture of the whole window grabbed with Live in front and again with Sim in front. The ground is the tab's own pixel at its top-left corner; the heading is the most common colour inside the Activity Log label, and the label's second colour is its text. At 1400 by 900, the window's floor:

```
theme             point            Live      Sim       Qt         Live      Sim       React
cyberpunk_dark    ground           #0a0a0f   #08080b   differs    #0a0a0f   #08080b   differs
cyberpunk_dark    heading ground   #0a0a0f   #08080b   differs    #0a0a0f   #08080b   differs
cyberpunk_dark    heading text     #00ffcc   #00ffcc   same       #00ffcc   #00ffcc   same
neon_light        ground           #f5f5fa   #b8b8bc   differs    #0a0a0f   #08080b   differs
neon_light        heading ground   #f5f5fa   #b8b8bc   differs    #f5f5fa   #b8b8bc   differs
classic_terminal  ground           #0a0a0a   #080808   differs    #0a0a0f   #08080b   differs
classic_terminal  heading ground   #0a0a0a   #080808   differs    #0a0a0a   #080808   differs
minimal_modern    ground           #fafafa   #bcbcbc   differs    #0a0a0f   #08080b   differs
minimal_modern    heading ground   #fafafa   #bcbcbc   differs    #fafafa   #bcbcbc   differs
glass_metal       ground           #1c1c24   #15151b   differs    #0a0a0f   #08080b   differs
glass_metal       heading ground   #1c1c24   #15151b   differs    #1c1c24   #15151b   differs
```

Every Sim value is the Live value beside it through `toward_black` at 0.25. The same twenty rows read the same at 1920 by 1080. The heading text reads `#00ffcc` on every row in both builds, so the tone moved no text.

The React ground column is the venue page's own ground, which is the design token `SURFACE_0` under every theme on Live and its darkened value on Sim; the page's outer ground, read at the web view's corner, follows the theme as the Qt ground does and darkens the same way. That split between the page's two grounds is Live's own, stated on the tabs page under the theme menu, and the tone keeps it.

The header strip sits above every tab and takes no tone. Its ground read the theme's own value with Live in front and with Sim in front, on every theme, in both builds:

```
cyberpunk_dark  #0a0a0f   neon_light  #f5f5fa   classic_terminal  #0a0a0a
minimal_modern  #fafafa   glass_metal #1c1c24
```

The windows the Sim opens read the tone at their own corner, both builds, every theme; the React Bot Settings window's page and the React wizard's page each read the same value as their frame:

```
theme             chooser   Bot Settings   wizard
cyberpunk_dark    #08080b   #08080b        #08080b
neon_light        #b8b8bc   #b8b8bc        #b8b8bc
classic_terminal  #080808   #080808        #080808
minimal_modern    #bcbcbc   #bcbcbc        #bcbcbc
glass_metal       #15151b   #15151b        #15151b
```

The built bundles read the same way. Each variant was built, launched over an empty scratch home with every HTTP route pointed at a refused loopback port and Chromium's resolver mapped away, its window found by process id and sized to the floor, and Live then Sim brought to the front by a posted click on the tab bar. Off a capture of that window, under the stored default theme:

```
bundle   Live ground   Sim ground   Live heading text   Sim heading text   strip card, Live and Sim in front
qt       #0a0a0f       #08080b      #00ffcc             #00ffcc            #16162a  #16162a
react    #0a0a0f       #08080b      #00ffcc             #00ffcc            #12121a  #12121a
```

Two plants proved the reading can fail. With the fraction set to 0, every Sim value read equal to Live's, twenty rows per build. With the fraction set to 1, every Sim ground and every window read `#000000`. The plant was removed and the module compared byte for byte with its pre-plant digest. The scratch copy of the fleet file read

the same digest before and after every one of the ten readings; a copy with one byte appended read a different digest, so the comparison reports a change.

Before this change the same reading gave every Sim value equal to Live's, under every theme, in both builds: no distinction reached a pixel.

The skin dictionary carries the tone too

The `SKIN` dictionary the surface serves puts its two ground entries through the same function, so a reader of the served payload gets the tone the tab paints. The block under the skin tokens the sheet reads shows the ground entry as `ds.SURFACE_0`; it reads the darkened value now.

`src/gui/main_tabs/simulator_tab_surface.py` — the two ground entries

```
SKIN = {
    "--sim-ground": toward_black(ds.SURFACE_0, NIGREDO_FRACTION),
    "--sim-chart-ground": toward_black(ds.SURFACE_CHART, NIGREDO_FRACTION),
```

The renders

One pair per theme, Live beside Sim, in each build, at 1400 by 900, under `artifacts/u117/u27/` in the repository's ignored artifacts directory. The fraction each was taken at is 0.25, and it is one number in the theme engine.

Back to the subsystem index. ## The replay layer draws the chosen tablet and retrieves through a read-only connector

The second layer behind the Indicator Voting Panel holds the VWAP window over the Stone Tablet Playback window, and both draw the tablet the chooser names. The layer's header row, where the flip button sits, carries `Tablet:`, a chooser and one retrieval button, in both builds. The flip reads `Replay` on the panel's header and `Indicators` on the layer's: under the Classic Terminal theme the panel title leaves the flip 90 px at the window's floor, and `Replay` draws whole there under every theme. The chooser lists every tablet on disk by its file key and every held market that has no tablet, and starts on the selected bot's tablet by the rule the panel follows. The two windows draw the last hundred candles of the chosen tablet: the close line and the VWAP line above, one wick and one body per candle below. With no tablet chosen, or a tablet under thirty candles, both windows draw empty.

`src/gui/main_tabs/simulator_tab_surface.py` — the chooser's items

```
def tablet_choices(source: TabletSource, bots: Sequence[Any] = ()) -> list[dict]:
    """The tablet chooser's items: one per MANIFEST row, keyed by
    ``tablet_key`` and shown as it, then one per held market in ``bots`` with
    no tablet at ``NATIVE_TIMEFRAME``, keyed by ``market_key`` and shown with
    ``NO_TABLET_TEXT``."""
```

`src/gui/main_tabs/simulator_tab_surface.py` — the two windows' payloads

```
def replay_feed(source: TabletSource, key: str) -> dict:
    """The two windows' payloads for the item ``key`` names: ``vwap_payload``
    and ``playback_payload`` over ``window_of`` the tablet, empty with no
    entry or under ``MIN_CANDLES``, with ``entry`` and ``refusal`` beside them."""
```

The Qt tab draws the feed on `LineView` and `PlaybackView` through `set_payload`; the React host pushes it as the tab's `replay` and the page draws two SVG windows from the same points and shapes. Both paint the same five colours, read off `SKIN` through `replay_colours`.

When the windows draw

The feed runs at build, on every `fleet_changed`, on the flip to the layer, on the chooser's change, when the panel's bot changes, and when a retrieval ends. No timer feeds it: a tablet on disk does not move.

`src/gui/simulator/sim_trading_tab.py` — the feed

```
    def _feed_replay(self) -> dict:
        feed = surface.replay_feed(self._tablet_source, self._tablet_key)
        self._replay = feed
        self._vwap_view.set_payload(feed["vwap"])
        self._playback_view.set_payload(feed["playback"])
        self._retrieve_button.setText(feed["button_text"])
```

The retrieval button

The button reads `Retrieve Tablet` while the chosen item has no tablet on disk and `Update Tablet` while it has one. A press retrieves the chosen market at the native five-minute timeframe: from the fetcher's own start, `YTD_START_MS`, when no tablet exists, or from the tablet's last candle when one does, up to the newest closed candle. The candle now forming is left out, because the registry never replaces a timestamp it already holds.

`src/simulator/tablet_retrieval.py` — the span

```
def retrieval_span(entry: Any, now_ms: Optional[int] = None) -> tuple[int, int]:
    """``(since_ms, until_ms)`` for one press: ``YTD_START_MS`` with no
    ``entry``, else one step past ``entry.last_ts_ms``; up to
    ``closed_until_ms``."""
```

The walk runs on a daemon thread, as the Portfolio Battery's does, and never on the GUI thread. It goes one adapter chunk at a time, each chunk one `download_missing` call over a registry built on the tab's tablet root, and stops at the first chunk the venue refuses. Every write lands under that root and nowhere else.

`src/simulator/tablet_retrieval.py` — the walk

```
    registry = StoneTabletsRegistry(root)
    span = int(adapter.chunk_span_ms)
    cursor = int(since_ms)
    while cursor <= int(until_ms):
        chunk_end = min(cursor + span - STEP_5M_MS, int(until_ms))
        reports = await download_missing(
            [(outcome.asset, outcome.exchange_id)],
            connector,
            cursor,
            chunk_end,
            registry=registry,
        )
```

The read-only connector

The connector the walk reads through is the Simulator's one venue path. It holds `CoinbasePublicCandles`, the shipped reader of the venue's public candle endpoint, which carries no key and reaches no account or order endpoint. It answers `get_ohlcv` and nothing else; every other name raises `SendRefused`, the shape `TabletSource` and `FleetSource` have. The Live connector is not used: it requires a key to connect, carries `place_order`, and records on the process-wide API log.

`src/simulator/read_only_connector.py` — the refusal

```
    def __getattr__(self, name: str) -> Any:
        """Refuse every name outside ``READ_NAMES``."""
        raise SendRefused(
            f"ReadOnlyConnector answers {READ_NAMES} and cannot {name!r}. "
            "The Simulator receives and asks; it sends nothing."
        )
```

`get_ohlcv` pages the reader by the venue's three-hundred-row page so a chunk of three hundred and fifty candles comes back whole.

What the two spools show

The Activity Log carries the started line naming the tablet and the span, one progress line per venue call naming the rows received and their span, and the finished line naming the candles appended, the chunks walked and the file. A refused venue writes one line naming the error and that nothing more was written. A second press while a walk is in flight writes one line and starts nothing.

The API Interaction Log carries one block per venue call, in Live's block: the exchange and `FETCH_TABLET` with the tablet's key, a Reason line naming the candles asked and their start, the public endpoint path, the rows received and the latest close, the response time, and the file the rows were appended to. A refused call carries the error as its result. Each call crosses from the worker thread on a Qt signal and is recorded on the GUI thread, which is what the API log writer's thread rule requires.

`src/gui/simulator/sim_trading_tab_surface.py` — the block's fields

```
RETRIEVAL_ACTION_FORMAT = "FETCH_TABLET {key}"
RETRIEVAL_REASON_FORMAT = (
    "Get {limit} candles ({timeframe}) for {symbol} since {since} - "
    "Stone Tablet {verb}"
)
RETRIEVAL_RESULT_FORMAT = "{rows} candles received, latest close={close}"
```

What the replay reading measured

Both builds, the real window with a scratch home, every socket but loopback refused, a scratch tablet root holding the A15 and A14 5m 2026 coinbase tablets and a held fleet of the A15, A14 and A23 coinbase bots. With the A15 bot selected and `Replay` pressed, the chooser listed the two tablets and `A23_5m_coinbase — No Stone Tablet on disk.`, started on the A15 tablet, and both windows' payloads hashed equal to `vwap_payload` and `playback_payload` over the same hundred candles; moved to A14, both redrew. A retrieval of A23 against a loopback candle server, the reader's endpoint pointed at it, wrote `A23_5m_2026_coinbase.json` and its MANIFEST row under the scratch root, drew the windows from it, and recorded one API block per call; a second press updated from the last candle; the endpoint restored to the venue and the socket refused, the press wrote the refusal line and the root's files hashed identical. `bot_state.json` and the two copied tablets read byte-identical after every step. No bot was constructed and no socket left loopback.

The API spool shows Stone Tablet and YTD retrieval activity only

The Sim tab's API Interaction Log carries two kinds of block and refuses every other, in both builds. The log each host builds beside its two sources is the Simulator's own kind, `SimApiLog`, Live's log with one rule added: an entry is accepted when its action opens with one of two words, and any other action is refused before the entry exists, before the file line is written and before the pane is told. The two words are the two recorders the directive names. A Stone Tablet retrieval or update records one block per venue call, and Generate From YTD records one block per read of the trade files. Nothing else records on the Sim's log, and a venue order cannot reach the pane.

`src/simulator/sim_api_log.py` — the allowed set

```
#: The word a Stone Tablet retrieval's action opens with; the tablet key follows.
TABLET_ACTION = "FETCH_TABLET"

#: The word the Generate From YTD read's action opens with.
YTD_ACTION = "FETCH_YTD"

#: Every action word ``SimApiLog.record`` accepts. Any other raises ``SendRefused``.
ALLOWED_ACTIONS = (TABLET_ACTION, YTD_ACTION)
```

`src/simulator/sim_api_log.py` — the refusal

```
class SimApiLog(APIInteractionLog):
    """``APIInteractionLog`` whose ``record`` accepts ``ALLOWED_ACTIONS`` only."""

    def record(
        self,
        exchange: str,
        action: str,
        reason: str,
        endpoint: str = "",
        params: dict = None,
        result: str = "",
        elapsed_ms: float = 0.0,
        level: str = "info",
        data_usage: str = "",
    ) -> dict:
        if not action_allowed(action):
            message = REFUSED_FORMAT.format(action=action, allowed=ALLOWED_ACTIONS)
            logger.warning(message)
            raise SendRefused(message)
        return super().record(
```

Three ways a venue order cannot reach the pane

The object refuses. A record whose action is not a tablet retrieval or a YTD read raises the same refusal the tab's sources raise, appends nothing, writes no file line and calls no listener, so nothing is drawn on either build; one warning line names the action refused. The process-wide log is a different object. The venue connectors record on the log the Live tab listens to, and the Sim's writer listens to the Sim's log alone, so an entry recorded on the venue log draws on Live's pane and not the Sim's. The callers are the allowed set. Four places under the Simulator's tab code record on the Sim's log, two per build, and they carry the two allowed words and nothing else.

Diagram: flowchart, left to right

- Update Tablet or Retrieve Tablet, one venue call -> **FETCH_TABLET key**
- Generate From YTD, one read -> **FETCH_YTD**
- FETCH_TABLET key -> **SimApiLog.record**
- FETCH_YTD -> **SimApiLog.record**
- any other action -> **SimApiLog.record**
- log -- allowed -> **the Sim API Interaction Log pane**
- log -- refused -> **SendRefused, nothing drawn**

The Generate From YTD block

A Generate From YTD press that reads the trade files records one block on the Sim's API Interaction Log, in Live's block, after the read and before the Activity Log's generation line. The head line carries the exchange chosen and the action. The Reason line names the read. The Endpoint line is the exchange history directory the files were read from. The Result line counts the fills and the files and names every file read. The Response line is the time the read took. The Data usage line counts the bots the read generated, and adds the manifest rows skipped when a row's file is missing. A press that stops before a read, because the directory is missing, empty or without its manifest, because the manifest names no pair, or because the chooser was cancelled, records no block and writes its Activity Log line alone.

```
[05:51:06] COINBASE FETCH_YTD
  Reason: Read the YTD trade files for coinbase - Generate From YTD
  Endpoint: <the exchange history directory>
  Result: 10 fills read from 4 file(s): A15-USD_2025_coinbase.json, A15-USD_2026_coinbase.json, A14-USD_2026_coinbase.json, A23-
USD_2026_coinbase.json
  Response: 2.8ms
  Data usage: Generated 3 bot(s) on the Scrumming Bots table
```

`src/gui/simulator/sim_trading_tab_surface.py` — the block's fields

```
YTD_REASON_FORMAT = "Read the YTD trade files for {exchange} - Generate From YTD"
YTD_RESULT_FORMAT = "{fills} fills read from {files} file(s): {names}"
YTD_DATA_USAGE_FORMAT = "Generated {bots} bot(s) on the Scrumming Bots table"
YTD_NOTHING_GENERATED_TEXT = "Nothing generated"
YTD_MISSING_USAGE_FORMAT = "; {missing} manifest row(s) skipped, file missing"
```

`src/gui/simulator/sim_trading_tab.py` — the record

```
        started = time.perf_counter()
        made = self._fleet_source.generate_from_ytd(source, chosen)
        elapsed_ms = (time.perf_counter() - started) * 1000.0
        self._api_log.record(
            **tab_surface.ytd_api_entry(made, chosen, source.root(), elapsed_ms)
        )
```

The file names and the fill count come off the generation itself, which now answers the names of the files it read and the fills it read beside the bots it held and the rows it skipped.

`src/simulator/fleet_source.py` — what one generation answers

```
@dataclass(frozen=True)
class YtdGeneration:
    bots: tuple[SimBot, ...] = ()
    files_read: int = 0
    missing: tuple[YtdFileEntry, ...] = ()
    files: tuple[str, ...] = ()
    trades_read: int = 0
```

The Stone Tablet block keeps its text

A retrieval's block is the block the replay layer's section above describes, unchanged in every field; its action now opens with the shared word rather than a copy of it, so the word has one definition.

`src/gui/simulator/sim_trading_tab_surface.py` — the action

```
RETRIEVAL_ACTION_FORMAT = TABLET_ACTION + " {key}"
```

What the API spool refusal reading measured

Both builds, the real window with a scratch home, every socket but loopback refused, scratch YTD trade files in the shape the generation reading above drove, copies of the A15, A14 and A22 tablets, copies of the SPY and GLD RA tablets, and a copy of `bot_state.json`. One session: Generate From YTD with coinbase chosen drew one block whose Result line named the four files and the ten fills, whose Response line carried the read's time, and whose lines read the same on both pages apart from the stamp and the path; Update Tablet on A22 against a loopback candle server drew forty tablet blocks; Import Live Fleet drew none; Portfolio Battery over EQUITY_MACRO drew none. After the session the pane held 41 blocks, and 41 of them opened with an allowed word. A venue order recorded on the Sim's log raised the refusal and drew nothing, on either build; the same entry on the process-wide log drew on Live's pane and not the Sim's. Live's 37 action names recorded on the Sim's log were refused 37 of 37; with the order word planted into the allowed set the count read 36, and with the plant removed it read 37 again. One tablet entry recorded directly drew. On the base commit the same session drew no block for Generate From YTD, the order record drew, and the 37 names drew 37 of 37. Every copied file and every trade file read byte-identical after every step, and a byte planted into a copy of `bot_state.json` moved its hash. No socket left loopback.

The Activity spool shows the trades of Validation, Back Test and Portfolio Battery runs

Every simulated fill writes one line on the Sim tab's Activity Log, in both builds, as the walk produces it. The line is Live's fill line as Live's pane receives it: the bot prefix Live's window puts on every bot line, then `SELL FILLED` for a scrum or `BUY  FILLED` for a fold, the units to six places, the asset, the price to eight places. The Simulator has no venue, so it has no intended price and no slippage to report; in their place the line carries what the walk knows about the fill, the dollars and the fee to five places, in the shape Live's proceeds line writes them. A scrum's dollars read `gross`, the notional sold; a fold's read `spent`. The line is written at Live's `info` level and paints at the pane's plain size, because that is what Live's fill line paints at.

`src/gui/simulator/sim_trading_tab_surface.py` — the line one `SimTrade` writes

```
TRADE_LINE_LEVEL = "info"
BOT_PREFIX_FORMAT = "[{asset}/{tail}] "
BOT_ID_TAIL_CHARS = 4
TRADE_LINE_FORMAT = (
    "{prefix}{word}: {units:.6f} {asset} @ ${price:.8f} "
    "({usd_word} ${usd:.5f}, fee ${fee:.5f})"
)
SCRUM_FILL_WORD = "SELL FILLED"
FOLD_FILL_WORD = "BUY FILLED"
SCRUM_USD_WORD = "gross"
FOLD_USD_WORD = "spent"
```

Unit 31a moved the fill words to `src/simulator/sim_bus.py` (comment 5734916119).

```
[2021-02-05T00:00:00Z] [SPY/ahoo] SELL FILLED: 0.193792 SPY @ $387.70999146 (gross $75.13498, fee $0.45081)
[2026-07-29T15:35:00Z] [A03/0bda] BUY FILLED: 155.417407 A03 @ $0.02252000 (spent $3.50000, fee $0.05600)
```

The stamp is the tape's time

Every other line on the pane is stamped with the clock at the moment it is written, as Live's pane stamps it. A simulated fill happened at the tape's candle, and a daily tablet's candles all fall at midnight, so a clock stamp would read the same on every line of a daily walk and say nothing about when the fill fell. A trade line is stamped with the candle's time instead, the same UTC stamp the report writes for a run's first and last candle. Both forks of the pane gain one call for it, a write under a stamp the caller gives, which holds the line under a pause exactly as an ordinary write does and renders it otherwise.

`src/gui/simulator/sim_status_log.py` — the write under a given stamp

```
        def log_at(self, stamp: str, message: str, level: str = "info") -> None:
            """``log`` under ``stamp`` instead of the clock: held under a pause, else rendered."""
            self._tell("log", message, level)
            if self._paused:
                if len(self._pause_buffer) < self._pause_buffer_cap:
                    self._pause_buffer.append((stamp, message, level))
                return
            self._render(stamp, message, level)
```

The seam the walk hands each fill to

The walk that sizes every simulated trade gains one callback, handed each `SimTrade` the moment it fills, the way the Battery hands its progress callback each portfolio's line. Every runner threads it through to the walk: Back Test's run, the Battery's symbol, portfolio and battery runs, and Validation's run. A symbol two portfolios share on one bot is walked once, so its fills reach the callback once.

`src/simulator/back_test.py` — the seam

```
#: The seam ``walk`` hands each ``SimTrade`` to as it fills, as ``run_battery``
#: hands ``progress`` each portfolio's line.
TradeSink = Callable[[SimTrade], None]
```

```
        if filled is not None:
            trades.append(filled)
            fees += filled.fee_usd
            if on_trade is not None:
                on_trade(filled)
```

A Validation run's trades are the recorded fills it reruns

Validation sizes no trade of its own; it reruns the recorded fills against the gate rows the live bot wrote. Its trade line is that recorded fill, one per rerun: the bot, its symbol, a scrum for a sell and a fold for a buy, the fill's time, price, amount, cost and fee. A fill the run does not rerun writes no line; the report names it under what could not be verified.

`src/simulator/validation.py` — the recorded fill as a trade

```
def rerun_trade(bot: SimBot, fill: Any) -> "SimTrade":
    """The ``SimTrade`` one rerun YTD fill reads as: ``SCRUM`` for ``SIDE_SELL``
    and ``FOLD`` otherwise, ``amount`` as the units, ``cost`` as the USD."""
```

Off the worker thread, as the portfolio lines travel

A Battery run walks on its worker thread. Each fill crosses to the GUI thread through a signal beside the one that carries each portfolio's line, and the host writes it there. On the Qt tab the line is appended to the pane as it arrives. On the React page the host paints it into its own log at once and pushes every line painted since the last push once per strip tick, two seconds, so a walk of thousands of fills does not push thousands of scripts; a push carries the pending lines ahead of anything else it carries.

`src/gui/simulator/sim_trading_tab.py` — the signal and the writer

```
    #: One ``SimTrade`` the Battery's walk filled on the worker thread.
    battery_trade = Signal(object)
```

```
    def log_trade(self, trade: SimTrade) -> None:
        """One Activity Log line per ``SimTrade``: ``trade_line`` under
        ``trade_stamp`` through ``SimStatusLog.log_at`` at ``TRADE_LINE_LEVEL``."""
        self._status_log.log_at(
            tab_surface.trade_stamp(trade),
            tab_surface.trade_line(trade),
            tab_surface.TRADE_LINE_LEVEL,
        )
```

`src/gui/simulator/sim_react_trading_tab.py` — the push cadence

```
#: One push per strip tick carries every trade line ``log_trade`` painted since the
#: last push, so a walk of thousands of fills does not push thousands of scripts.
TRADE_PUSH_INTERVAL_MS = DASHBOARD_TICK_MS
```

Nothing else reaches the spool during a run

During a run the Activity Log's writers are the run's own: the started line when the press is taken, one line per portfolio as it lands, one line per fill as it fills, and when the run ends its summary lines and the report line. The watchdog writes only when the render-error count rises or when nothing has rendered for ten minutes while a sim bot runs, and a run renders continuously; the command bar's notices are reached only by its own presses; the window's notifications reach Live's pane. The pane holds 5,000 lines and drops the oldest past that; a pause holds 2,000 and drops the newest, and Resume replays the held lines in order under one line counting them.

Back Test and Validation, through the run

What the trade-line reading measured

Both builds, the real window with a scratch home holding a copy of `bot_state.json` with 38 coinbase bots, the A03, A11 and A13 tablets, a copy of the whole RA-StoneTablet root, a copy of the gate log and YTD trade files built from the SCRUM and FOLD rows three of those bots wrote; every socket but loopback refused. Import Live Fleet held 38 bots. Back Test through its run walked 3 bots, produced 1,670 fills and wrote 1,670 trade lines and the report line; every line's stamp, prefix, word, units, asset, price, dollars and fee read equal to its fill. One bot at 1,300 candles, a walk of exactly three fills, wrote three lines, each with its fee. Validation through its run rerun 36 recorded fills and wrote 36 lines and the report line. Run Portfolio on BOGLEHEAD over every span, pressed at the corner, wrote 204 trade lines for the 204 fills its nine walks produced, beside the started line, one portfolio line, eight summary lines and the report line, and nothing else; the report's rows counted the same 204. Run Portfolio on FULL wrote its first fills within three seconds of the press with the run still in flight; Pause Console pressed then held every later line, 819 on the Qt tab and 719 on the React page, and Resume flushed them in their held order under the line counting them. Run Every Portfolio over the whole root produced 2,066 fills in 189 walks and wrote 2,066 trade lines beside 47 of the run's own; on the Qt tab the pane read 681 lines at 9.2 seconds of a 27-second run and 2,054 at 27.3, and on the React page 1,573 at 21.5 seconds against 1,587 fills produced. After that run both panes held 4,936 lines under the cap of 5,000. On the base commit the same sessions wrote no trade line in either build. Every copied file read byte-identical after every step, a byte planted into a copy of `bot_state.json` moved its hash, no bot was constructed and no socket left loopback.

Start runs Validation and Back Test from the command bar

The command bar's Start is the press that starts a run. Live starts a bot with Start on the selected row, and no other press on the Trading tab starts anything, so the Simulator's run control is that same Start. What it starts follows the run mode: in Validation mode it starts the Validation run, in Back Test mode the Back Test walk, and in Portfolio Battery mode it stays the state move the command bar section above describes, because the Battery's run is the corner's. Stop on a row of the run ends it. No new button is drawn.

`src/gui/simulator/sim_trading_tab.py` — the dispatch in `_on_bot_command`

```
        if self.run_running():
            if command == "stop" and bot_id in self._run.get("bot_ids", []):
                self._stop_run(bot_id)
                return
            self._status_log.log(
                tab_surface.run_in_flight_line(
                    self._run.get("mode", ""),
                    len(self._run.get("bot_ids", [])),
                    command,
                ),
                "warning",
            )
            return
        if command == "start" and self._mode in tab_surface.RUN_MODES:
            self._start_run(bot, self._mode)
            return
```

Diagram: flowchart, left to right

- Start on the command bar -> **_on_bot_command**
- _on_bot_command -> **_start_run: every scrumming bot on the page to running, fleet_changed, the started line** *(Validation or Back Test mode)*
- _start_run: every scrumming bot on the page to running, fleet_changed, the started line -> **_compute_run on a daemon thread**
- _compute_run on a daemon thread -> **validation.run over TabletSource, YtdTradeSource, GateLogSource** *(Validation)*
- _compute_run on a daemon thread -> **back_test.run over TabletSource** *(Back Test)*
- validation.run over TabletSource, YtdTradeSource, GateLogSource -> **run_trade: one line per rerun fill**
- back_test.run over TabletSource -> **run_trade: one line per rerun fill**
- validation.run over TabletSource, YtdTradeSource, GateLogSource -> **write_report**
- back_test.run over TabletSource -> **write_report**
- write_report -> **run_finished: _take_run moves the bots to stopped, writes the lines and the report line**
- Stop on a run row -> **_stop_run sets the event the runner reads**

The run is the page's scrumming fleet

A Validation pass is one pass over a fleet: `validation.run` takes the bots, measures one coverage, one tape lag and one match key over them, and writes one report. Start therefore runs the mode over every held scrumming bot on the pressed page's exchange, `_run_bots` in both hosts, and every one of those rows reads `running` while the run is in flight and `stopped` when it ends. The selected row is the bar's own precondition, `Select a bot first.`, and is not changed. Extractor rows are not run: the gate chains the rerun evaluates are the scrumming bot's, and the walk sizes a scrum and a fold.

`src/gui/simulator/sim_trading_tab.py` — the fleet a run covers

```
    def _run_bots(self, exchange_id: str) -> list[SimBot]:
        """Every held scrumming bot on ``exchange_id``, the fleet a run covers."""
        return [
            one
            for one in self._fleet_source.bots()
            if one.exchange_id == exchange_id and one.mode == SCRUMMING_MODE
        ]
```

`_start_run` moves each of those bots to `running` through `SimBotManager.start`, the same rule Start applies to one bot, and fires `fleet_changed`, so the sim fleet file is saved and every venue page draws its rows again before the walk begins: the Bot ID cell reads `State: RUNNING` in its running colour. Then the started line is written and `_compute_run` starts on a daemon thread named `sim-mode-run`, the shape the Battery and the Market Inspector's Scan Now use, so the window keeps answering. When the worker ends, `_take_run` on the GUI thread moves every run bot to `stopped` through `SimBotManager.stop`, fires `fleet_changed` again, and writes the run's own lines and the report line through `log_report`.

```
Validation started on coinbase over 24 bot(s); budget $16,860.00, the sum of the held Target Balances; no rerun fill is refused for
cash.
Back Test started on coinbase over 24 bot(s); each fold spends its own scrum proceeds.
```

The Validation line names the run's budget under the funding rule: the sum of the held Target Balances, `run_budget_usd`, which is also what the strip's Spendable reads in that mode. Validation reruns gates and sizes no trade, so no fill is refused for cash, and the report's header says so. Back Test keeps the proceeds funding, unchanged.

What the worker reads

The Validation worker reads the tab's `TabletSource` over the Stone Tablets, a `YtdTradeSource` over the YTD trade files, and a `GateLogSource` over `gate.log` and its rotations under the trade directory of the log root. Each answers reads alone and refuses every other name. The Back Test worker reads the `TabletSource` alone. Neither touches a widget, the fleet source or a venue; the worker's only write is the report.

`src/gui/simulator/sim_trading_tab.py` — the two runners, called as unit 30's seam left them, with the stop event added

```
                outcome = validation.run(
                    bots,
                    self._tablet_source,
                    YtdTradeSource(),
                    GateLogSource(),
                    exchange_id=exchange_id,
                    limit=surface.VALIDATION_RERUN_LIMIT,
                    on_trade=self.run_trade.emit,
                    stop=self._run_stop.is_set,
                )
```

Each rerun fill crosses to the GUI thread on `run_trade` and is written by `log_trade` in the trade-line shape the Activity spool section describes; the run's lines and the failed line cross on `run_line`; the outcome crosses on `run_finished`. The React host carries the same three signals, the same `_start_run`, `_compute_run`,

_stop_run and _take_run, and reaches them from the page's command bar through Live's own ExchangeTabModel.cmd.

Stop ends the run with a partial report

Stop on a row of the run writes Live's Stopping bot <id>..., the run's stopping line, and sets a threading.Event. Both runners take stop, a callable read before each bot and before each row or tick: _validate reads it before each bot's snap and before each row's rerun, _walk_fleet before each bot and walk before each tick. When it answers True the pass ends where it is, with stopped set on the run, and the same write_report writes the report: partial reads yes with the stop named, the header carries stopped and the bots reached, every unreached bot's row reads not reached, and the Not verified section leads with the stop line. A stopped walk answers the candles it read, the last close it ticked and the trades it filled. When the worker ends, _take_run writes Live's Bot <id> stopped. for the row Stop was pressed on, moves every run bot to stopped, and writes the lines and the report line as for a whole run.

```
Stopped by the operator: 19 of 24 bots walked, 1 cut short at its last bar ticked; 5 bot(s) not reached: <ids>.
Stopped by the operator: 1 of 24 bots reached, 0 rows rerun; 23 bot(s) not reached: <ids>.
```

While a run is in flight, Start, Pause, Restart and Delete on any row, and Stop on a row outside the run, write one line and move nothing:

```
A Validation run is in flight over 24 bot(s); Stop on one of its rows ends it, and start waits for it.
```

A run that raises writes the failed line, moves its bots to stopped, and its partial file is on disk from write_partial, as the report section describes.

Each differing light carries its cause

A non-zero count is a finding, and the report says what drove it. Every disagreeing light already carries which side of the rerun moves it; it now carries a cause as well, one of three words read off the row's own fields.

cause	what it reads
tape	the tablet candle the rerun read differs from the reading the bot recorded: the recorded bb_pos sits more than the lag match gap from the rerun's, or the bank's recorded band flag differs from the rerun's
fixture	a recorded field the light reads is absent from the row's fixture, or the phantom lock could not be recovered from the recorded row
chain	the same inputs on both sides and a different light: the gate chain

src/simulator/validation.py — the cause of one light that is blocked on one side

```
def light_cause(
    light: LabelComparison,
    row: Any,
    context: GateContext,
    locked_known: bool,
    absent: Sequence[str],
) -> str:
    key = (light.bank, light.label)
    if light.driven_by == TAPE:
        if recorded_bb_pos(row) is None:
            return FIXTURE_CAUSE
        if tape_moved(row, context, light.bank):
            return TAPE_CAUSE
        if key == ("S", "BB") and not locked_known:
            return FIXTURE_CAUSE
        return CHAIN_CAUSE
    if light.driven_by == RECORD:
        for fixture, field_name in LABEL_FIELDS.get(key, ()):
            if f"{fixture}.{field_name}" in absent:
                return FIXTURE_CAUSE
        if tape_moved(row, context, light.bank):
            return TAPE_CAUSE
        return CHAIN_CAUSE
    return CHAIN_CAUSE
```

`LABEL_FIELDS` names, per light, the recorded fixture fields `rerun_context` reads for it. A light that is blocked on exactly one side is classified by the rules above. A light blocked on neither side, `passed` against `not_the_blocker`, moved only because its bank's armed flag moved, so it takes the cause of its bank's blocked lights: `tape` before `fixture` before `chain` where they mix, because the tape is the data the criterion is about, and `chain` where no light in the bank is blocked. The report writes the cause beside each disagreeing light, a per-row count of causes, and under the four counts a line `differs by cause: tape N, fixture N, chain N` with the three definitions, in both files. No tolerance is widened and no gate is changed to make a light agree; the chain is Live's.

Two lights read the phantom lock, not one. The lock used to reach only the scrum band term, so an unrecoverable lock was a fixture cause on the scrum band light alone. The fold midline term now carries the lock as well, so the same unrecoverable lock is a fixture cause on the fold midline light too. One declaration names both lights and the classifier reads it.

`src/simulator/validation.py` — the two lights, and the clause that reads them

```
LOCK_READING_LIGHTS = (("S", "BB"), ("F", "MID"))

if key in LOCK_READING_LIGHTS and not locked_known:
    return FIXTURE_CAUSE
```

The empty cases name the missing source

The run still runs when a source is empty, and the report's Not verified section names what could not be rerun. With no YTD trade directory, the Start press writes the YTD root line before the started line, every bot reads `no YTD trade file; nothing snapped.` and the comparison table holds no row. With no `gate.log`, every bot's snapped fills match nothing, and the bot's own line says so. With no Stone Tablet for a pair, that bot reads `no Stone Tablet; nothing snapped.` and the rest of the fleet is rerun.

```
<id> (AAA/USD): no YTD trade file; nothing snapped.
<id> (AAA/USD): no gate row recorded for this bot; 4 snapped trades matched nothing.
<id> (BBB/USD): no Stone Tablet; nothing snapped.
```

`src/simulator/parity_report.py` — the per-bot line for a bot with no recorded gate row, read off the `gate_rows` count `_validate` fills

```
        elif row["outcome"] == VALIDATED and not row["gate_rows"]:
            out.append(
                f"{row['bot_id']} ({row['symbol']}): no gate row recorded for this "
                f"bot; {row['snapped']} snapped trades matched nothing."
            )
```

What the run from the bar measured

Before, in both builds over a scratch home holding 24 scrumming bots on coinbase, one 5m Stone Tablet of 2,000 candles per pair, one YTD trade file per pair with four fills, and a `gate.log` of one fired row per fill in the operator's row shape, each row latched by the shipped chain over the tablet candle that holds its fill: Import Live Fleet held 24 rows, and Start in Validation mode wrote `✓ Bot <id> RUNNING.`, moved the one selected row, wrote no trade line and no report; Back Test the same; the two runners had no caller under the tab.

After, in both builds over the same home: Start in Validation mode moved all 24 rows to `RUNNING` within 40 ms on the Qt tab and within 0.6 seconds on the React page, wrote the started line with the budget of $16,860.00, wrote 96 trade lines as the 96 recorded fills were rerun, wrote the run's six lines and the report line, and moved the rows to `STOPPED`; the report's comparison table read missing 0, extra 0, differs 0, variant differs 0 over 96 rows compared, the sidecar's four counts equal to the Markdown's. Start in Back Test mode moved the 24 rows the same way, walked 48,000 candles in about seven seconds, wrote 17 trade lines for the 17 fills, and wrote its report. Stop pressed on the last row one second into a Back Test wrote the stopping lines and a partial report reading 19 of 24 bots walked, one cut short and five not reached on the Qt tab, 11 of 24 and 13 not reached on the React page; a Start pressed while that run was in flight wrote the in-flight line and started nothing; Stop pressed the moment a Validation run started wrote a partial report reading 1 of 24 bots reached and 0 rows rerun on the Qt tab, 23 of 24 and 0 rows rerun on the React page, and every run bot read `STOPPED` after each.

Over a scratch home whose first bot's rows carried one recorded S/TA light flipped to blocked with its scrum disarmed, one recorded `target_fires` removed, and one recorded reading moved to the opposite band with its lights latched there, plus one fill a day past the tablet's last candle, the table read differs 21 by cause tape 11, fixture 1 and chain 9: the removed field's row read S/FIRE `fixture`; the flipped row read S/TA `chain` with its eight armed-flag lights `chain`; the moved row read S/BB `tape` with its ten dependent lights `tape`; and the fill past the tablet read `after_last_candle` under Not verified with the uncovered span. Both builds read the same 21.

Over three more scratch homes, one with no YTD directory, one with no `gate.log` and one with no tablet for the second pair, the report named each case per bot as the lines above show, the YTD root line preceded the started line in the first, and the third rerun the first pair's four rows to four zeros.

Over scratch copies of the operator's own `bot_state.json`, his `gate.log` and its five rotations, and the 38 Stone Tablets his 38 scrumming bots' assets and timeframes name, with no YTD trade file on this machine: Import Live Fleet held 38 rows and Start in Validation mode wrote the YTD root line, ran, and wrote a report whose 38 bot rows each read `no YTD trade file; nothing snapped.`, whose comparison table read four zeros over 0 rows compared, and whose header named the budget as the sum of the 38 Target Balances. Without the exchange's YTD export nothing can be snapped, so nothing can be rerun, and the report says so rather than reading a pass. With YTD trade files built from the fired rows his own gate log holds, one fill per SCRUM or FOLD row inside its tablet's span, the run rerun 55 recorded fills and the table read missing 0, extra 0, differs 275, variant differs 0, with 23 of the 55 rows latching identically and 770 of 1,045 lights agreeing; every one of the 275 differing lights read `tape`, none `fixture` and none `chain`, because 50 of the 55 recorded readings were reproduced from a window 50 candles, 250 minutes, behind the fill's own candle, the lag the first full run measured, so the bot read a different candle from the one the tablet holds at the fill's moment and the chain, fed the tablet's candle, lit the lights differently; fourteen bots carried a fired row inside their tablet's span and the other 24 read `no YTD trade file`. Every copied file read byte-identical after every press, a byte planted into a copy of `bot_state.json` moved its hash, no bot was constructed and no socket left loopback.

The Simulator writes Live's log rows under the sim bucket

Every run writes the rows a live bot writes, into the Simulator's own files, so a run can be troubleshot and tuned from its files as a live bot can from its logs. The operator, 2026-09-18: *"Simulator Logs and Emitter Network must be able to provide sufficient data for troubleshooting and optimization."* Each Sim host builds one private bus at tab build and one second copy of Live's log writer over the sim bucket, attached to that bus. The bus is never the process-wide one, and nothing under the Simulator subscribes to the live bus. The runners emit Live's topics on the private bus, the writer's own listeners write them, and the four files land beside the sim fleet file.

`src/simulator/sim_bus.py` — the bus and the writer one host builds

```
def new_sim_bus() -> EventBus:
    """A private ``EventBus`` for one Sim host, never ``get_event_bus``."""
    return EventBus()

def sim_log_manager(
    bus: Any, symbol_of: Optional[Callable[[str], str]] = None
) -> LogManager:
    """A ``LogManager`` over ``get_sim_dir`` attached to ``bus``, with
    ``symbol_of`` as its symbol resolver when given."""
    manager = LogManager(log_dir=get_sim_dir())
    if symbol_of is not None:
        manager.set_symbol_resolver(symbol_of)
    manager.attach_to_bus(bus)
    return manager
```

`src/gui/simulator/sim_trading_tab.py` — built once, in the tab's `__init__`

```
        self._bus = new_sim_bus()
        self._log_manager = sim_log_manager(self._bus, self._symbol_of)
```

The writer's own attach subscribes the five topics it subscribes on Live, so the sim files carry Live's names and Live's row shapes. One writer, one definition, two buckets.

topic on the private bus	file under the sim bucket	who emits it
`trade.filled`	`sim/trade.log`	the walk on every fill, the rerun on every rerun fill
`bot.gate_decision`	`sim/gate.log`	the walk on every evaluated candle, the rerun on every rerun row
`bot.voting_panel_snapshot`	`sim/voting.log`	the walk on every evaluated candle
`bot.log`	`sim/diagnostics.log`	the walk on every fill and for every bot it refuses, the rerun for every bot it cannot rerun
`pnl.event`	`sim/pnl/daily/`	nothing; the directory is made at build and stays empty

The writer's console handler is not attached a second time: `main.py` builds Live's writer before the window, so the application logger already holds Live's handler when the tab builds, and the sim writer attaches none. No `sim/system.log` exists, and the live `system.log` is not doubled.

Diagram: flowchart, left to right

- Start on the command bar -> **_compute_run on a daemon thread**
- _compute_run on a daemon thread -> **back_test.run or validation.run with bus=self._bus**
- back_test.run or validation.run with bus=self._bus -> **RunEmitter: the bus and one run id**
- RunEmitter: the bus and one run id -> **walk: one candle evaluated**
- walk: one candle evaluated -> **bus.emit on the private bus**
- bus.emit on the private bus -> **LogManager listener**
- LogManager listener -> **one row appended to sim/gate.log**

Every row carries its run id and its candle

Each run takes one id before its first row, `<mode>-<stamp>`, and every gate row, voting row and diagnostics row of the run carries it. The candle's time is on every row as well. A row's top-level `timestamp` is the wall clock at the write, as it is on Live, because the writer sets it and no call takes one; the candle's time sits under `data` as `candle_at` and `candle_ts_ms`, the stamp the Activity Log's trade line carries. A reader finds a candle by `data.candle_at`, never by the top-level stamp.

`src/simulator/sim_bus.py` — the four keys every gate and voting row adds

```
    def stamp(self, candle_ts_ms: int) -> dict:
        """The four keys every row adds: ``run_id``, ``mode``,
        ``candle_ts_ms`` and ``candle_at``."""
        return {
            "run_id": self.run_id,
            "mode": self.mode,
            "candle_ts_ms": int(candle_ts_ms),
            "candle_at": iso_stamp(candle_ts_ms),
        }
```

A trade row carries no run id and no candle: Live's trade listener writes a closed set of fields and drops every other key, and that listener is not changed. A trade row is found through the gate row written in the same moment for the same bot with the same action, the pairing the sizing replay already makes. A diagnostics row carries the run id and the candle inside its message, because Live's diagnostics listener writes `symbol` and `message` only.

`src/simulator/sim_bus.py` — the diagnostics message

```
BOT_LINE_FORMAT = "{message} (run {run_id}, candle {candle_at})"
```

What a walk's gate row holds

A walk's gate row is Live's gate row: the two armed flags and the two blocker lists the chain latched, the scrum and fold fixtures with the keys the live bot writes at fire time, the tranche snapshot and the compounding snapshot in the live shapes, `evaluated_at_tick` as the walk's tick count, and the trade action and side on a row where a fill happened. Live's fixture shape has no function of its own, two dict literals inside the live bot's tick; the Simulator spells the same keys from the walk's gate context.

`src/simulator/sim_bus.py` — the scrum fixture's first keys

```
def scrum_fixture(context: GateContext, reading: Any = None) -> dict:
    """The ``scrum_fixture`` keys ``ScrummingBot.tick`` writes, read off
    ``context`` and ``reading``."""
    return {
        "delta": float(context.delta),
        "below_interval": bool(context.below_interval),
        "ticker_last": float(context.ticker_last),
        "bb_pos": float(context.bb_pos),
        **_landing(reading),
```

Functional. The builder takes a third input, and the fixture now opens with it: each refusal an override replaced on that side, under the key the live gate log already writes. A row where no override acted carries an empty list, so a row an override passed and a row that passed on its own read apart.

`src/simulator/sim_bus.py` — the key the fixture opens with

```
OVERRIDES_APPLIED_KEY: [str(one) for one in overrides_applied],
```

One row per evaluated candle, not one per fill as Live writes, because a blocked candle is the one a person troubleshoots: the blocker list on that row names the light that held, and the fixture names the reading behind it. The voting row beside it carries the whole vote of that candle, every indicator's direction, confidence and details, so a pillar that voted wrong is found by candle.

`src/simulator/back_test.py` — the walk's emit order, Live's fire order

```
        if emitter is not None:
            action = fill_action(filled)
            if filled is not None:
                emitter.trade_filled(filled, bot.exchange_id)
                emitter.bot_line(bot.bot_id, fill_line(filled), stamp)
            emitter.voting_snapshot(bot, summary, stamp, action, fill_side(filled))
            emitter.gate_decision(
                bot,
                context,
                armed,
                stamp,
                reading=reading,
                position=position,
                tick=ticks,
                trade_action=action,
                side=fill_side(filled),
            )
```

A scrum row's `usd` is the settled proceeds net of the estimated fee, as Live's scrum row carries it; a fold row's `usd` is the spend. The Activity Log's trade line keeps printing the gross beside the fee. The fill words that line uses moved into the bus module, so the surface's format now holds the prefix and the line the diagnostics row carries.

`src/simulator/sim_bus.py` — Live's fill line, without the window's prefix

```
FILL_LINE_FORMAT = (
    "{word}: {units:.6f} {asset} @ ${price:.8f} ({usd_word} ${usd:.5f}, fee ${fee:.5f})"
)
SCRUM_FILL_WORD = "SELL FILLED"
FOLD_FILL_WORD = "BUY FILLED"
SCRUM_USD_WORD = "gross"
FOLD_USD_WORD = "spent"
```

A Validation rerun row carries the recorded row and every light

A Validation run emits one gate row per rerun, with the rerun's armed flags, blockers and fixtures where Live's fields sit, and two more fields under `data`: the recorded row as the gate-log reader read it, and the comparison with all nineteen lights. Each light names its bank and label, the recorded state, the rerun state, whether they agree, which side drives it and, where they differ, the cause. A differing light is found from the file alone, by candle, with no report open: read the rows whose comparison holds a light that does not agree, and each such row names its candle.

`src/simulator/validation.py` — the fields a rerun row adds

```
def rerun_row_fields(row: Any, seen: RowComparison) -> dict:
    """The fields a rerun's gate row adds beside Live's: ``recorded`` from
    ``recorded_row_fields`` and ``comparison`` from ``comparison_row`` with
    all nineteen ``lights`` and their ``agrees``."""
    from .parity_report import comparison_row

    comparison = comparison_row(seen)
    comparison["lights"] = [
        {
            "bank": one.bank,
            "label": one.label,
            "recorded": one.recorded,
            "rerun": one.rerun,
            "agrees": bool(one.agrees),
            "driven_by": one.driven_by,
            "cause": one.cause,
        }
        for one in seen.labels
    ]
    return {"recorded": recorded_row_fields(row), "comparison": comparison}
```

Validation computes no voting summary, because the rerun reads the vote the bot recorded, so a Validation run writes no voting row. A bot with no YTD file or no tablet, and a bot whose snapped trades matched no recorded gate row, is named in the diagnostics file with the same line the report's Not verified section carries.

The observer reads every run and the report names its rows

For the length of one run, Live's emit observer is subscribed to the private bus over the two Live contracts the Simulator emits, the fill and the gate decision. The run's emitter counts what it emits per topic; the observer counts what arrived and reports a required field that is absent, a value outside its vocabulary, and a declared topic that never fired. At the run's end the observer is finished, its subscriptions are removed, and its reading goes into the run object and the report beside the run id and the four file paths. A run that emits no fill reports the fill topic as never emitted, so an empty run reads as empty rather than as clean.

`src/simulator/sim_bus.py` — the observer's reading at the run's end

```
    def close(self) -> dict:
        """Finish the ``EmitObserver``, remove its subscriptions and answer
        ``to_dict`` with ``emitted`` beside it; a second call answers the same."""
        if self._read is not None:
            return self._read
        for remove in self._unsubscribe:
            remove()
        self._unsubscribe = []
        self._observer.finish()
        self._read = {"emitted": dict(self.emitted), "observer": self._observer.to_dict()}
        return self._read
```

The report's Run section gains the run id, the four file paths, the rows emitted per topic and the observer's line; the sidecar carries the same under `header.rows`, with each violation listed.

```
- run id: back_test-20260918T183818245464Z
- row files: trade.log <sim bucket>/trade.log, gate.log <sim bucket>/gate.log, voting.log <sim bucket>/voting.log, diagnostics.log
<sim bucket>/diagnostics.log
- rows emitted: bot.voting_panel_snapshot 4752, bot.gate_decision 4752, trade.filled 17, bot.log 17
- observer: 2 of 2 declared topic(s) seen; bot.gate_decision 4752, trade.filled 17; 0 violation(s)
```

Live's contract module declares a voting topic nothing emits and no contract for the voting snapshot topic the live bot does emit, so the voting row is written under no contract and the observer does not read it. Live's trade rows carry an empty exchange, because the live fire sites pass none; the Simulator's carry the bot's exchange in that field.

The current reading, 2026-10-29. Live's trade rows now carry the bot's exchange too, because every Live fill is sent in one shape that names it. The Simulator's emitter builds that same shape, so its dollar figure arrives as `usd`, the name the trade log reads. One scrum of 1.25 units at $100 with a $0.75 fee wrote $125.00 to the trade log on the commit before this change, the units times the price, and writes $124.25 now. A fold row wrote $123.50 on both.

`src/simulator/sim_bus.py, in RunEmitter.trade_filled` — the one shape, with the run's stamp beside it

```
            self._send(TRADE_TOPIC, **fill.payload(), **self.stamp(trade.ts_ms))
```

What the bucket holds and how large it grows

After a run the sim bucket holds the four files beside the sim fleet file and the empty daily directory. Each file rotates at 50 MiB with five backups, the writer's own bound. Measured over the scratch fleet of 24 bots at 200 ticks each, a Back Test wrote 4,752 gate rows in 11.9 MB, about 2.5 KB a row, and 4,752 voting rows in 27.1 MB, about 5.7 KB a row, the vote's twelve signals and their details making the difference. A whole-root Portfolio Battery of 189 walks at 120 ticks each writes about 22,700 rows a file, so its voting rows cross the rotation bound within one run and the older ones survive in the numbered backups.

What the emitter reading measured

Before, in both builds over a scratch home holding 24 scrumming bots, one 2,000-candle 5m tablet per pair, four fills per pair, one fired gate row per fill, the live bucket's five files each holding one line, and a copy of the 411 RA-StoneTablets: after a Validation run, a Back Test run and a Run Portfolio, the sim bucket held the sim fleet file and nothing else, no host held a bus, and no report named a run id.

After, in both builds over the same home: the Validation run wrote 96 gate rows and 96 trade rows under its run id, the Back Test 4,752 gate rows, 4,752 voting rows, 17 trade rows and 17 diagnostics rows, and the Battery 1,017 gate rows, 1,017 voting rows, 204 trade rows and 204 diagnostics rows; every gate and voting row carried the run id and the candle, every row of each file carried its contract's required fields, and the observer read 2 of 2 declared topics seen and 0 violations on each run, its counts equal to the rows on disk. The host's bus was not the process-wide bus, read by identity; the process-wide bus's subscriptions read the same after every run as after the window was built; the live trade bucket's four files, the 24 tablets, the YTD files and the 411 RA tablets hashed identical after every run, and a byte appended to a copy of the fleet file moved its hash. The live `system.log` changes during a run on the base tree already, from the window's own panel timer and the Simulator's own loggers, and is the process log rather than a trade-bucket file.

Off the tab, one Back Test walk over the same fleet with the bus and the writer, under the debugger with `amount` removed from one fill payload at the emit line, read one observer violation naming the absent field,

carried into the report; the same walk untouched read zero. The sizing replay of the one trading logic, run over that walk's `sim/trade.log` and `sim/gate.log`, paired 17 of 17 fills with their gate rows and reproduced 4 of 4 scrum unit counts, 13 of 13 fold unit counts, 4 of 4 interval readings and 4 of 4 scrum proceeds; its own planted row read one disagreement. Over the scratch home whose recorded rows carry one flipped S/TA light, one removed `target_fires` and one reading moved to the opposite band, `sim/gate.log` alone, with no report open, named three rows with a differing light by candle: the removed field's candle with S/FIRE `fixture`, the flipped candle with S/TA `chain` and its eight armed-flag lights, and the moved candle with S/BB `tape` and its ten dependents, in both builds.

Clear Fleet empties the held fleet

Clear Fleet is the corner's first button in every mode and, while a fleet is held, the Get Started card's first button. A press empties the whole held fleet, every record on every venue, after one confirmation in Live's Delete Bot box shape. The venues unseat, the card returns, the header strip reads zeros and the sim fleet file is written with no record. `bot_state.json` is not touched. Both builds draw the button from one surface list and both hosts run one handler, `_clear_fleet`.

The button at the corner and on the card

The corner's list is `way_in_buttons` in `src/gui/simulator/sim_trading_tab_surface.py`: `clear_fleet_button` first, then the run mode's two ways in, each at Live's corner-button size, 140 px wide and 24 px tall. The card's list is `placeholder_way_in_buttons`, which puts Clear Fleet above the two ways in only while `held`, the count of held records, is above zero. The Qt host draws both lists in `SimTradingTab._draw_way_ins` and draws them again at the end of every `_sync_exchange_tabs`; the React host writes `held` into `SimTradingTabState` on every `show_tab` and calls `show_tab` at the end of every `_sync_exchange_tabs`, so the page's `WayInButtons` and `PlaceholderWayIns` components draw the same two lists with no change to `src/gui/web/sim_trading_tab.js`. The button's accessible name is `sim-clear-fleet` at the corner and `sim-clear-fleet-card` on the card.

Since the way-in row landed, that list draws on a row above the exchange tab bar in both builds, not in the tab bar's corner, and it is empty with no fleet held, so the row is absent and the card alone offers the ways in. The Qt host builds the row in `SimTradingTab._make_layer` and the React page draws it in `WayInButtons` above the tab bar. The corner holds nothing. The section "Start Run starts the active mode's run from the way-in row" describes the row.

`src/gui/simulator/sim_trading_tab_surface.py` — the card's list

```
def placeholder_way_in_buttons(
    accent: Any, mode: str = sim.MODES[0], held: int = 0
) -> list:
    """The card's buttons at Live's card-button size and sheet: Clear Fleet
    first while ``held`` records are held, then the two ways in ``mode`` offers."""
    rows = list(sim.reserved_rows(mode))
    if int(held or 0) > 0:
        rows.insert(0, {"action": sim.CLEAR_FLEET_ACTION, "text": sim.CLEAR_FLEET_TEXT})
```

The card draws only on a layer that seats no venue. Every venue the Simulator seats today, `coinbase`, `kraken` and the Battery's `yahoo`, sits on the crypto layer, so the crypto card and a held crypto fleet are never on screen together. The card's Clear Fleet is the way to empty a fleet held on the other layer, such as an

imported `bot_state.json` fleet on an equity venue, which the stack does not show.

Which layer a Simulator venue is seated on

The tab asks one list whether a venue trades equities. The list is declared once, in the module the header's asset class group reads, and the Simulator tab reads that same name.

`src/gui/simulator/sim_trading_tab.py` — the read, in the tab's own build

```
self._equity_exchange_ids = acs.EQUITY_VENUES
```

A venue on the list is seated on the stock layer and every other venue on the crypto layer. The id is matched in lower case, so a venue seated under a capitalised name reaches the same layer. Where the list lives, and what reads it, is on 06-trading-tab.md.

The confirmation and the clear

A press reaches `_way_in` on either host and then `_clear_fleet`. The box is `QMessageBox.question` under the title `Clear Fleet`, with Yes and No, as the command bar's Delete opens its `Delete Bot` box. The question names the count and the venues. Yes runs `FleetSource.clear`, which drops every held record and answers how many were held, then writes the cleared line, the notification `Fleet cleared`, plays Live's state-change sound and fires `fleet_changed` once. `FleetSource.save` writes the sim fleet file with `bot_count` 0. `_sync_exchange_tabs` reads no exchange, takes every venue off, adds the Get Started page back, and re-reads the tables, the panel and the replay layer. The header strip reads `aggregate_stats` over no bot on its next tick, every field zero, and EXCH 0. No is the cancelled line and nothing moves.

`src/simulator/fleet_source.py` — the clear

```
    def clear(self) -> int:
        """Drop every held record, on every exchange; answers how many were
        held. The sim fleet file loses them on the next ``save``."""
        count = len(self._records)
        self._records = {}
        return count
```

Clear Fleet empties every venue, not the shown venue alone. Live's Delete acts on one bot id. The fleet is one map: `FleetSource` holds every venue's records together, `save` writes them as one file, and the strip reads them together. A per-venue clear would be a different control with a different name.

`src/gui/simulator/sim_trading_tab_surface.py` — the box and the lines

```
CLEAR_FLEET_BOX_TITLE = sim.CLEAR_FLEET_TEXT
CLEAR_FLEET_QUESTION_FORMAT = (
    "Clear the Simulator fleet of {count} bot(s) on {venues}? This cannot be undone."
)
CLEARED_FORMAT = "Cleared {count} bot(s) on {venues}; the Simulator fleet is empty."
CLEAR_CANCELLED_TEXT = "Clear Fleet cancelled."
NOTHING_HELD_TEXT = "No fleet is held; nothing to clear."
FLEET_CLEARED_NOTICE = "Fleet cleared"
```

A run in flight, nothing held, cancelled

While a Validation or Back Test run is in flight, the press writes the same in-flight line the command bar writes, naming Clear Fleet as the command that waits, and opens no box. While a Portfolio Battery run is in flight, the press writes the Battery's in-progress line. With no record held, the press writes `No fleet is held; nothing to clear.` and opens no box. No on the box writes `Clear Fleet cancelled.` and removes nothing. In each of these cases `fleet_changed` does not fire and the sim fleet file is not written.

The two signals a press emits

Every press emits `sim.fleet.clear_pressed` through `signal_contract.emit`, bound in each host as `_pin_emit`, and a confirmed clear emits `sim.fleet.cleared` after it. The press signal carries the outcome and the held count after the press, against the count the outcome allows: unchanged for a cancel, a refusal or nothing held, zero for a clear. The cleared signal carries the held count and the seated venue count after the clear, both expected zero, with the count removed and the venues unseated in its context. Each host flushes the process sink after the emit, so the handler's file under the runtime log directory's `signals` folder holds the rows right after the press.

`src/gui/simulator/sim_trading_tab.py` — the cleared signal

```
        _pin_emit(
            tab_surface.CLEARED_SIGNAL,
            actual={
                "held_after": len(self._fleet_source.bots()),
                "venues_after": self.exchange_count(),
            },
            expected={"held_after": 0, "venues_after": 0},
            context={"removed": removed, "venues_unseated": list(venues)},
        )
```

What the clear reading measured

Before, in both builds over a scratch home holding a `bot_state.json` with four records on `coinbase` and one on `kraken`, a tablet manifest naming four assets per venue and no sim fleet file: Import Live Fleet on both venues seated both and held five records; no button named `sim-clear-fleet` existed at the corner or on any card in any mode, the five records stayed held through every press, and the only route to an empty fleet was Delete, one row and one box at a time.

After, in both builds over the same home: the corner read `sim-clear-fleet`, `sim-import-live-fleet` and `sim-generate-from-ytd` in that order from the left, each 140 by 24, Clear Fleet's left edge 142 px before Import Live Fleet's. With five records held, the stock layer's card held `sim-clear-fleet-card` first and the crypto card was not drawn. The corner press opened the box `Clear Fleet` reading `Clear the Simulator fleet of 5 bot(s) on coinbase, kraken? This cannot be undone.` with Yes and No; No wrote the cancelled line and left five held. Yes wrote `Cleared 5 bot(s) on coinbase, kraken; the Simulator fleet is empty.` and the notification, unseated both venues, drew the crypto card with two buttons and no Clear Fleet, zeroed the five cards and read EXCH 0 on the strip's next tick, wrote the sim fleet file with `bot_count` 0 and an empty `bots` map, and left two signal rows in `session.jsonl`, both `ok` true. Back Test then Create New Bots on the card walked the wizard to one bot on `coinbase`, which seated the venue and put Clear Fleet on the stock card; that card's press opened the box naming one bot and Yes emptied the fleet again. The corner press with nothing held wrote `No fleet is held; nothing to`

`clear.` and opened no box. A Back Test started with its worker held open read `run_running` true; the corner press wrote the in-flight line naming Clear Fleet, opened no box and left five held. `bot_state.json` hashed identical after every step in every run, and a byte appended to a copy of it moved the hash. The Watchdog Archetype read every emit in both hosts wired to `signal_contract`.

Start Run starts the active mode's run from the way-in row

Start Run is one button on the way-in row above the exchange tab bar, drawn under every mode while a fleet is held, to the right of the mode's two ways in. A press starts the active mode's run over the fleet on the venue shown. Start on the command bar no longer starts a run in any mode; it moves the selected bot's state, as Pause, Stop and Restart do. Both builds draw the button from one surface list and both hosts run one handler, `_start_run_pressed`.

The button on the way-in row

The row's list is `way_in_buttons` in `src/gui/simulator/sim_trading_tab_surface.py`, which takes the held count: empty at zero, otherwise Clear Fleet first, then the run mode's two ways in, then Start Run, each at Live's corner-button size, 140 px wide and 24 px tall. The Qt host hands the count in `SimTradingTab._draw_way_ins`, which runs again at the end of every `_sync_exchange_tabs`; the React host hands it through `layer_card`, which the tab payload carries on every `show_tab`. The button's accessible name is `sim-start-run`. The Get Started card never holds it: the card draws on a layer with no venue, and a held fleet seats its venue.

`src/gui/simulator/sim_trading_tab_surface.py` — the row's list

```
def way_in_buttons(mode: str = sim.MODES[0], held: int = 0) -> list:
    """The way-in row's buttons as the page draws them, while ``held``
    records are held: ``clear_fleet_button`` first, then the two ways in
    ``mode`` offers from ``sim.reserved_rows``, then ``start_run_button``;
    an empty list with nothing held, so the row is absent and the card shows."""
```

The way-in row above the exchange tabs

The row is a widget of its own above each layer's exchange tab bar, so the tabs and the buttons never share a line and a third or fourth venue's tab has room. The Qt host builds it in `SimTradingTab._make_layer` as a `QWidget` with a horizontal layout at margins 0 and spacing 2, added to the page's column before the tab widget, hidden while the list is empty; the tab widget gets no corner widget. The React page draws it in `WayInButtons` as the first child of the tab widget's column, above the tab bar, wrapping to a second line where the pane is narrower than the buttons, and draws nothing while the list is empty; the tab bar keeps its tabs and the stretch after them. Both hosts read the row's margins and spacing from one place.

`src/gui/simulator/sim_trading_tab_surface.py` — the row's layout

```
WAY_IN_ROW_LAYOUT = {"margins_px": [0, 0, 0, 0], "spacing_px": 2}
```

`src/gui/web/sim_trading_tab.js` — the row above the bar

```
      element(
        DIV_TAG,
        widgetProps,
        element(WayInButtons, { layer: layer, actions: props.actions }),
        element(DIV_TAG, barProps, barTabs, element(DIV_TAG, spacerProps, null)),
        element(DIV_TAG, bodyProps, body)
      )
```

Read with three venues seated, `coinbase`, `yahoo` and `kraken`, in both builds: at a 1400 px window every tab and every button had its own rect, no two overlapping, the row's bottom edge above the bar's top edge, and the tab bar's corner empty, under all three modes. The Qt window's floor is its layout minimum, 1145 px wide in the reading, and the row fits on one line there. The React window shrinks to 900 px, where the pane is 476 px wide and Start Run wraps to a second line; every tab and button stayed inside the pane with no overlap.

The press by mode

A press reaches `_way_in` on either host and then `_start_run_pressed`. The handler reads the tab's run mode and the venue on show. Under Validation and Back Test it runs `_start_run` over every held scrumming bot on that venue, the run the section "Start runs Validation and Back Test from the command bar" describes: each bot to running, the started line, the worker thread, the trade lines, the report and the report line, every bot to stopped. Under Portfolio Battery it runs `_run_battery` under Run Portfolio's action, so the chooser opens with the portfolio and span rows, and the run follows as the section "The run from the corner" describes. Run Portfolio and Run Every Portfolio keep their own buttons; Start Run under the Battery is a third way to the same chooser, because one button means one thing in every mode.

`src/gui/simulator/sim_trading_tab.py` — the handler

```
    def _start_run_pressed(self) -> None:
        venue = self._current_venue_id()
        mode = self._mode
        run_bots = self._run_bots(venue)
        running_before = tab_surface.rows_running(run_bots)
```

`_start_run` takes the venue id where it took a selected bot, because the corner has no selected row; the run it starts is unchanged.

```
    def _start_run(self, exchange_id: str, mode: str) -> str:
```

Start on the bar moves one bot

The dispatch unit 31 put at the top of `_on_bot_command`, which read the mode and started the run on Start, is removed from both hosts. Start on the bar now reaches the state move in every mode: `SimBotManager.start` on the selected bot, the running line, the notification and `fleet_changed`. The in-flight guard above it stays: while a run is in flight, Stop on one of its rows ends the run and every other bar press writes the in-flight line.

A second press in flight

Start Run is never disabled, because Live's command bar never disables Start. A press while a Validation or Back Test run is in flight writes the in-flight line naming Start Run as the command that waits, and starts nothing. A press while a Portfolio Battery run is in flight writes the Battery's in-progress line. A press on a venue holding no scrumming bot writes the no-bot line.

```
A Back Test run is in flight over 24 bot(s); Stop on one of its rows ends it, and Start Run waits for it.
```

The signal a Start Run press emits

Every press emits `sim.run.start_pressed` through `signal_contract.emit`, bound in each host as `_pin_emit`, and flushes the process sink after it. The signal carries the outcome, one of started, refused in flight, no bot or cancelled, and the count of the venue's scrumming rows reading running after the press, against the count the outcome implies: every run bot after a started Validation or Back Test, the count before the press otherwise. The mode, the venue, the held count and the run's bot count travel in its context.

`src/gui/simulator/sim_trading_tab.py` — the signal

```
        _pin_emit(
            tab_surface.START_PRESSED_SIGNAL,
            actual={"outcome": outcome, "rows_running": running_after},
            expected={"outcome": outcome, "rows_running": expected_running},
            context=context,
        )
```

What the Start Run reading measured

Before, in both builds over a scratch home holding a `bot_state.json` of 24 scrumming bots on `coinbase`, one 2,000-candle 5m tablet per pair, four fills per pair in YTD files, one fired gate row per fill, and a scratch RA root holding five 2022 tablets: Import Live Fleet held 24 rows; the corner read Clear Fleet then the mode's two under every mode; no button named `sim-start-run` existed under any mode, and a press asked for it read no button in Qt and no element in React.

After, in both builds over the same home: the way-in row read Clear Fleet, the mode's two, then `sim-start-run` under every mode, each 140 by 24, and was absent with no fleet held at open and after Clear Fleet. Start on the bar with the first row selected, under each of the three modes, wrote the running line, moved that one row to RUNNING and no other, started no thread and wrote no report; Stop moved it to STOPPED. Start Run under Validation moved all 24 rows to RUNNING, wrote 96 trade lines and six run lines, wrote the Validation report and named it on the Activity Log, and the report read missing 0, extra 0, differs 0, variant differs 0 over 96 rows. Start Run under Back Test pressed twice fast wrote the started line and then the in-flight line naming Start Run, moved all 24 rows to RUNNING, wrote 17 trade lines and the Back Test report. Start Run under Portfolio Battery opened the chooser titled Run Portfolio with 35 portfolios and seven spans; CRYPTO_BLUE over 2022 wrote 45 trade lines, the Battery's eleven lines and the Battery report. Four signal rows landed in `session.jsonl`, every one `ok` true: started under Validation with 24 rows running, started and refused in flight under Back Test, started under Portfolio Battery with 0 rows running. Every watched file, 59 of 59, hashed identical after every step in every run, and a byte appended to a copy of `bot_state.json` moved the hash. No socket left loopback. The Watchdog Archetype read every emit in both hosts wired to `signal_contract`.

The flip sits at the panel's top-left corner under both layers

The operator's item: *"'Replay' button in the IVP panel needs to move to the IVP's upper left corner where the 'Indicators' button appears after 'Replay' is click. Both of these buttons should appear in the same location."* One button flips the layer stack. It now sits first in whichever header row is showing, at the row's left edge, and draws one width on both layers, so the rect that reads Replay on the panel is the rect that reads Indicators on the replay layer.

One seat under both layers

The Qt host seats its one `FlipButton` at `FLIP_SEAT_INDEX`, the first item, in the panel's header row at build and on every flip, and at the same index in the replay layer's header. The replay layer's header takes its margins and spacing off the panel's header row instead of restating them. The page does the same from one declaration: `REPLAY_HEADER_LAYOUT` reads the panel header's margins and the panel column's spacing from `indicator_panel_surface`, `sim_indicator_panel.js` draws the flip before the title, and `sim_trading_tab.js` draws it first on the replay header.

`src/gui/simulator/sim_trading_tab_surface.py` — the replay header's layout

```
REPLAY_HEADER_LAYOUT = {
    "margins_px": list(cast(list, panel.HEADER["margins_px"])),
    "spacing_px": int(cast(int, panel.CONTAINER["spacing_px"])),
}
```

One width for both words

Replay and Indicators differ in width, so one seat alone gives two rects. The Qt `FlipButton` answers `sizeHint` with the wider of the two words' hints, each asked of the style the way `QPushButton.sizeHint` asks it; its `minimumSizeHint` keeps a width of 0, so the pane pair the section "The Sim venue pane is Live's width" records does not move. The page's `flip_button` payload carries `other_text`, the word the button reads on the other layer, and the sheet reserves that word's width as a hidden zero-height line after the text. The sheet also pins the button to the top of its row, and the replay header wraps its chooser and retrieval button to a second line when the pane is narrower than its controls, so the button keeps its width and its corner at every pane width.

`src/gui/simulator/sim_trading_tab.py` — the width

```
    def sizeHint(self) -> QSize:  # noqa: N802
        """The base hint, widened to the widest word in ``texts``."""
        base = super().sizeHint()
        widest = max(self.hint_for(text).width() for text in self._texts)
        return QSize(max(base.width(), widest), base.height())
```

`src/gui/web/sim_trading_tab.css` — the reservation

```
[data-part="sim-flip-button"]::after {
  content: attr(data-other-text);
  display: block;
  height: 0;
  overflow: hidden;
  visibility: hidden;
}
```

The flip's signal

Each press emits `sim.layer.flipped` through `signal_contract`: `actual` is the pressed button's rect in the layer stack's coordinates, `expected` the rect of the previous press, `ok` their equality or None on the first press, and `context` names the layer left and the layer shown. The Qt host reads the rect in `flip_layer` before it moves the button; the page reads it at the press through the tab module's `flipRect` and sends it under `flip_rect` with either flip ask, and the React host's `show_layer` emits on every change of layer. One function builds the row for both hosts.

`src/gui/simulator/sim_trading_tab_surface.py` — the row

```
def flipped_pin(leaving: str, shown: str, pressed: Any, previous: Any) -> dict:
    """The ``LAYER_FLIPPED_SIGNAL`` row for one press: ``actual`` the pressed
    rect, ``expected`` the previous press's rect, ``ok`` their equality or
    None on the first press, ``context`` the layer left and the layer shown."""
    return {
        "actual": pressed,
        "expected": previous,
        "ok": None if previous is None or pressed is None else pressed == previous,
        "context": {"from": leaving, "to": shown},
    }
```

What the flip reading measured

Read off the widget tree and the page in the layer stack's coordinates, over a scratch home holding a `bot_state.json` of two scrumming bots on `coinbase` and two 5m tablets, the live fleet imported. Before, at a 1400 by 900 window, the Qt button read x 123, y 5, 47 by 20 on the panel and x 4, y 5, 64 by 20 on the replay layer; the React button read x 122, y 2, 57 by 22 and x 4, y 2, 74 by 22. At the Qt floor, 1145 by 1124, the same two rects; at the React reading of 900 by 700 the panel's title wrapped and the button read x 116, y 8, 57 by 22 against x 4, y 8, 54 by 22 on the layer, where the row squeezed it. No `sim.layer.` row existed.

After, in both builds at 1400 and at the floor, the button read one rect on the panel, on the layer after a press, and on the panel after a second press: Qt x 4, y 5, 64 by 20; React x 4, y 2, 74 by 22. The title sat to the button's right at every reading. Seating the panel's button back after the title moved it to x 123 in Qt and x 122 in React, and seating it first again restored the rect, so the reading can fail. At the React floor the replay header wrapped its retrieval button to a second line at x 4, y 26, inside the pane. Four `sim.layer.flipped` rows landed per variant, the second and every later one `ok` true with `expected` equal to `actual`. The Qt top splitter read the same pair before and after. Every watched file, 8 of 8, hashed identical after every step, and a byte appended to a copy of `bot_state.json` moved the hash. No socket left loopback.

One fleet per mode

The Simulator holds one fleet for each run mode: a Validation fleet, a Back Test fleet and a Portfolio Battery fleet. The tab shows the fleet of the mode in force. Pressing a mode swaps the tables, the header strip, the venue stack and the way-in row to that mode's fleet. Import Live Fleet, Generate From YTD, Create New Bots, Clear Fleet and the Battery's generated fleet act on the fleet of the mode in force alone, so the Battery's generated fleet and its Yahoo venue live under Portfolio Battery and do not appear under Validation. One file holds all three fleets. Both builds draw the same screens and both hosts run the same source.

The source holds three fleets

The fleet source keeps one records map per mode and one mode in force. Every reader and every act it answers works on the map of that mode, so no caller changed: the tables, the strip, the venue stack, the way-in row, the bot manager and the three runners all read the fleet in force through the calls they made before. The mode is set on the source by the hosts and read back by them; the Qt host's mode is that read, and the React host writes it into its tab state before each draw.

`src/simulator/fleet_source.py` — the three modes and the fleet in force

```
MODE_VALIDATION = "validation"
MODE_BACK_TEST = "back_test"
MODE_PORTFOLIO_BATTERY = "portfolio_battery"
MODES = (MODE_VALIDATION, MODE_BACK_TEST, MODE_PORTFOLIO_BATTERY)

#: The sim fleet file's key holding one entry per mode of ``MODES``.
FLEETS_KEY = "fleets"
```

```
    def held_by_mode(self) -> dict[str, dict]:
        """Per mode of ``MODES``, ``held`` (how many records draw as a
        ``SimBot``) and ``venues`` (their distinct ``exchange_id`` values,
        sorted)."""
```

The press swaps the fleet

A mode press on the venue header or on the card reaches the host's `set_mode`. The host hands the mode to the source, restyles every seated venue header, then fires the fleet change: the file is written, the venues the new fleet names seat and the rest unseat, the rows, the panel and the replay re-read, and the way-in row and the card redraw. The strip reads the new fleet on the window's next tick. A press while a run or a Battery is in flight writes the in-flight line and changes nothing, because the run's end stops its bots by id on the fleet in force.

Diagram: flowchart, left to right

- mode button, header or card -> **host.set_mode**
- host.set_mode -> **FleetSource.set_mode**
- FleetSource.set_mode -> **fleet_changed**
- fleet_changed -> **FleetSource.save, three fleets**
- fleet_changed -> **_sync_exchange_tabs**
- _sync_exchange_tabs -> **venues seat and unseat**

- _sync_exchange_tabs -> **rows, panel, replay**
- _sync_exchange_tabs -> **way-in row and card**
- window tick -> **header strip**

The card holds the mode row

A mode whose fleet is empty shows the Get Started card and no venue, and the venue header is the only place the mode buttons were drawn. The card now holds the three mode buttons under its title, from the same list the venue header draws, with the same sheets: the active mode wears Live's Privacy Mode ON sheet and the other two the OFF sheet. The Qt host builds them once per layer and restyles them on every fleet change; the React page draws them from the card payload's `mode_buttons`, and a press sends `mode` on the tab's own ask, which the host hands to `set_mode` as it hands the venue header's press. The card's buttons are named `sim-mode-<mode>-card`.

`src/gui/simulator/sim_trading_tab_surface.py` — the card's mode buttons

```
def placeholder_mode_buttons(active: str = sim.MODES[0]) -> list:
    """The card's three mode buttons, ``venue.mode_buttons`` over ``active``
    under ``card_mode_name``, so the card offers the mode choice the venue
    header offers."""
    return [
        {**button, "accessible_name": card_mode_name(button["key"])}
        for button in venue.mode_buttons(active)
    ]
```

Clear Fleet names the mode

Clear Fleet empties the fleet of the mode in force and keeps the other two. Its box and its cleared line name the mode.

`src/gui/simulator/sim_trading_tab_surface.py` — the box and the line

```
CLEAR_FLEET_QUESTION_FORMAT = (
    "Clear the {mode} fleet of {count} bot(s) on {venues}? This cannot be undone."
)
CLEARED_FORMAT = "Cleared {count} bot(s) on {venues}; the {mode} fleet is empty."
```

The signal a mode press emits

Every mode press emits one row through the emitter network. The row's `actual` is the mode in force with its held count and venues after the press; its `expected` is the pressed mode with the count and venues the source held for it before the press; its context names the outcome, the mode left and every mode's count. The outcomes are `shown`, `refused_in_flight` and `unknown_mode`. The two Clear Fleet rows carry the mode in their context.

`src/gui/simulator/sim_trading_tab_surface.py` — the signal

```
MODE_SHOWN_SIGNAL = "sim.fleet.mode_shown"
MODE_OUTCOME_SHOWN = "shown"
MODE_OUTCOME_IN_FLIGHT = "refused_in_flight"
MODE_OUTCOME_UNKNOWN = "unknown_mode"
MODE_SHOWN_FORMAT = "{mode} fleet shown: {count} bot(s) on {venues}."
```

The React wizard is deleted on the GUI thread

Driving the flow on the React build found a crash: a bot created through the React wizard, then a Battery run, ended the process with a Chromium thread check, because the wizard's web view was left to Python's garbage collector, which ran on the Battery's worker thread. The React host now deletes the wizard through the event loop as soon as it closes, on the GUI thread, in both the accepted and the cancelled path. The same flow then ran to its end.

`src/gui/simulator/sim_react_trading_tab.py` — the disposal

```
            try:
                accepted = wizard.exec() == wizard.DialogCode.Accepted
                config = wizard.get_bot_config() if accepted else {}
            finally:
                # The dialog's QWebEngineView is deleted on this thread by the
                # event loop; a worker thread's garbage collection would abort.
                wizard.deleteLater()
```

What the fleet-per-mode reading measured

Read off the running program in both builds, over a scratch home holding a `bot_state.json` of 24 scrumming bots on `coinbase`, one 5m tablet per pair, four fills per pair in YTD files, one fired gate row per fill, and a scratch RA root holding the 2022 tablets of A15, A14, SPY, QQQ, IWM, GLD and SLV.

Before, one fleet: Import Live Fleet under Validation drew 24 rows, and the same 24 rows drew under Back Test and Portfolio Battery; Create New Bots under Back Test made 25 under every mode; Run Portfolio over DIGITAL_GOLD 2022 replaced them with 4 rows on `coinbase` and `yahoo` under every mode; Clear Fleet under Back Test left every mode empty. The file held one `bots` map. No card held a mode button, and no `sim.fleet.mode_shown` row existed.

After, three fleets, the same readings in both builds: at open the card with three mode buttons, Validation active. Import under Validation, 24 rows on `coinbase`, the way-in row Clear Fleet, Import Live Fleet, Generate From YTD, Start Run, Spendable $15,470.00, EXCH 1. Back Test pressed on the header: no venue, no row, the card with Back Test active and Import Live Fleet and Create New Bots, Spendable an em dash, EXCH 0, one signal row `ok` true, the line `Back Test fleet shown: 0 bot(s) on no venue.` Create New Bots on the card through the wizard: one row on `coinbase`. Portfolio Battery pressed: the card again; Run Portfolio on the card over DIGITAL_GOLD 2022: 4 rows, 2 on `coinbase` and 2 on `yahoo`, both tabs seated, Spendable $2,000.00, EXCH 2. Each mode pressed in turn read its own fleet: 24 rows, 1 row, 4 rows, each with its way-in row, its strip and its venue tabs, and each press one signal row with `expected` equal to `actual`. The file held `fleets` with `bot_count` 24, 1 and 4. Clear Fleet under Back Test: the box read `Clear the Back Test fleet of 1 bot(s) on coinbase? This cannot be undone.`, the line `Cleared 1 bot(s) on coinbase; the Back Test fleet is empty.`, the file 24, 0 and 4, the card back with Back Test active; the card's Validation button pressed drew the 24 rows again, and Portfolio Battery the 4.

A file planted in the old shape, one `bots` map holding one record, drew that record under Validation and nothing under the other two, with the line `holds one fleet; read as the validation fleet`, and the first mode press rewrote it under `fleets`. A file planted with `fleets` naming `validation`, `back_test` and `paper` drew one row under Validation, one under Back Test and none under Portfolio Battery, with one warning naming `paper`, and the first press wrote the three modes without it. Every watched file, 61 of 61, hashed identical after every step, and a byte appended to a copy of `bot_state.json` moved the hash. No socket left loopback.

Six-step counts for the React page: contract members 4 of 4 (the card's mode buttons, the card's mode row layout, the placeholder order, the tab ask's `mode`), feeds 2 of 2 (the card payload on every tab draw, the venue payload on every fleet change), driven and read back 3 of 3 (each of the three card buttons pressed, and the active sheet, the rows and the file read after).

Both bundles were built and launched in isolation, with no network and a scratch home holding two live bots, and driven through Windows UI Automation. In each: the card at open with its three mode buttons; Import Live Fleet, two rows, the way-in row; Back Test on the venue header, the card with Back Test active and its two ways in, the line `Back Test fleet shown: 0 bot(s) on no venue.`; Validation on the card's own button, the two rows and Validation's row again; Back Test on the header again, the card. The file held `fleets` with 2, 0 and 0, and three signal rows landed per build. The Qt bundle also walked Create New Bots through the wizard's Next and Finish buttons under Back Test, and the file then held 2, 1 and 0 with the new bot under Back Test; the React wizard's asset pair is a page control UI Automation does not drive, so Create under Back Test on the React build is the source-tree reading above. On the bundles, the live application's own periodic save rewrote `bot_state.json` sixty seconds after launch with the same two live bots and no sim bot, as it does on every launch; in the source-tree readings the file hashed identical after every step.

The Battery walks every candle, one portfolio at a time, and reads HODL against Harvest-Fold

The operator's words, 2026-09-18: *"Porfolio Battery - This is scaffolded. It does not appear to be loading any historical data. I run it twice and generated what are undoubtedly bogus but revealing reports. I love the Yahoo exchange appearing and loading the stocks but when 'running all portfolios' should load one, run, clear, load next, run, clear, repeat. These must be real runs against real data using the scrumming bots and IVP trading logic."* And the framing, the same day: *"I only seek to compare HODL against Harvest-Fold across as many historical markets and asset classes as possible."*

Before this, a Battery bot evaluated about one bar in every fourteen of a daily tablet over six years, because the walk stepped so that each bot and timeframe spent about 120 evaluations; Run Every Portfolio held every portfolio's bots at once and walked them in one loop; and the report read each portfolio as `reversed`, `improved`, `defended` or `undefended` against its historical path. Now every bar is evaluated, one portfolio is held and walked at a time, and the report reads three figures.

Every candle

The walk hands the Back Test walker no step. The walker evaluates every bar from the thirtieth on, so a walk over N bars makes N less 29 evaluations, and the report states both counts side by side. The walk is timed around the walker alone, and the report states the seconds per 1,000 evaluations it measured and, at that rate, how long a crypto bot's 5m walk takes over `2026 test run` and over `All`.

`src/simulator/portfolio_battery.py` — the evaluations a walk makes

```
def evaluations_expected(bars: int) -> int:
    """How many gate-chain evaluations a walk over ``bars`` makes: one per bar
    from the ``MIN_CANDLES``th on, none under ``MIN_CANDLES``."""
    return max(0, int(bars) - (MIN_CANDLES - 1))
```

`src/simulator/portfolio_battery.py` — the walk, every bar

```
    result = walk(
        walked,
        candles_from_raw(bars),
        funding=FUNDED_BY_TARGETS,
        rule=rule,
        on_trade=took,
        stop=stop,
        emitter=emitter,
        on_tick=on_tick,
    )
```

The walk reads as moving

One Activity Log line opens each bot's walk at each timeframe, naming the bars and the evaluations to make; one line lands every 5,000 bars, naming the bar reached, the evaluations so far, the partial exits and re-entries so far and the seconds; one line closes it with the evaluations, the seconds, the rate and the three figures. At the measured rate a line lands about every eight seconds on a 5m walk, and a daily walk ends before its first one.

`src/simulator/portfolio_battery.py` — the cadence

```
#: How many bars a walk covers between two progress lines. At the measured
#: rate of about 1.7 ms an evaluation a line lands about every 8.5 seconds on
#: a 5m walk, and a daily walk ends before its first one.
PROGRESS_EVERY_BARS = 5_000
```

`src/simulator/back_test.py` — the walker's tick callback

```
        if on_tick is not None:
            on_tick(index + 1, len(candles), ticks)
```

Stop ends the walk at the bar reached

Stop on a Battery row sets an event the walk reads before each portfolio, each walk and each tick. The walk in progress ends at the bar reached, the walks and portfolios not reached are named, and the partial report carries the bar, its candle stamp and the evaluations made. The remaining portfolios are not run, and the summary reads `partial`.

`src/gui/simulator/sim_trading_tab.py` — the stop

```
    def _stop_battery(self, bot_id: str) -> None:
        """Stop on ``bot_id``, a row of the Battery in flight: Live's stopping
        line, ``battery_stopping_line``, and the event the walk reads before
        each tick."""
        self._status_log.log(f"Stopping bot {bot_id}...", "info")
        self._battery["stopper"] = bot_id
        self._status_log.log(tab_surface.battery_stopping_line(bot_id), "warning")
        self._battery_stop.set()
```

`src/simulator/portfolio_battery.py` — the stopped line

```
WALK_STOPPED_LINE_FORMAT = (
    "{asset} {timeframe} stopped at bar {bar:,} of {bars:,} ({candle_at}): "
    "{evaluations:,} evaluation(s) made."
)
```

One portfolio at a time

The runner walks the named portfolios in order. Before each, it hands the host that portfolio's bots; the host holds them as the Portfolio Battery fleet, fires the fleet change, and the venues seat, the rows and the strip draw them, and the Yahoo venue seats for a stock portfolio. The portfolio's bots walk on a fresh set of walks, so a symbol two portfolios share is walked in each; its report is written; the host is handed the report, writes its line and, under Run Every Portfolio, holds an empty fleet, so every venue unseats and the Get Started card returns before the next portfolio loads. A single Run Portfolio keeps its fleet held at the end. When the press names more than one portfolio the runner writes one more report, the summary, and the host writes its line last.

`src/simulator/portfolio_battery.py` — the loop

```
    for name in chosen.names:
        if halted or (stop is not None and stop()):
            halted = True
            not_reached.append(name)
            continue
        bots = chosen.by_portfolio.get(name, {})
        seated = [bots[asset] for asset in PORTFOLIOS[name].symbols if asset in bots]
        if on_portfolio_started is not None:
            on_portfolio_started(name, list(seated))
        emit_portfolio_started(name, seated, str(span))
        emitter = RunEmitter(bus, new_run_id(PORTFOLIO_BATTERY), PORTFOLIO_BATTERY)
```

`src/gui/simulator/sim_trading_tab.py` — the two slots

```
    def _battery_portfolio_started(self, name: str, bots) -> None:
        """Hold ``bots``, one portfolio's, as the fleet through
        ``FleetSource.hold_battery_fleet``, fire ``fleet_changed`` so the
        venues seat and the rows and the strip draw them, and write
        ``battery_loaded_line``."""
        self._fleet_source.hold_battery_fleet(list(bots))
        self.fleet_changed.emit()
        self._status_log.log(tab_surface.battery_loaded_line(name, bots), "info")

    def _battery_portfolio_finished(self, name: str, run) -> None:
        """Write ``run``'s report line through ``log_report``; when the press
        names more than one portfolio, hold an empty fleet through
        ``FleetSource.hold_battery_fleet``, fire ``fleet_changed`` so every
        venue unseats, and write ``battery_cleared_line``."""
        if run.report is not None:
            self.log_report(run.report)
        if not self._battery.get("every"):
            return
        self._fleet_source.hold_battery_fleet(())
        self.fleet_changed.emit()
        self._status_log.log(tab_surface.battery_cleared_line(name), "info")
```

Under Run Every Portfolio the press holds an empty fleet, so nothing is drawn while the crypto tablets are retrieved and the first fleet the tab draws is portfolio 1's. Under Run Portfolio the press holds that one portfolio's bots, as the section "The run from the corner" says, and the first portfolio's start re-holds the same records. Each portfolio's start also empties the held fills, so a bot two portfolios share marks one portfolio's fills on the replay layer, and its loaded line names the portfolio's bot count, venues and budget.

`src/gui/simulator/sim_trading_tab.py` — the press

```
        self._fleet_source.hold_battery_fleet(() if every else plan.bots)
        self.fleet_changed.emit()
```

The three figures

HODL end is the Target Balance bought at the walk's opening bar and held to its last bar. Harvest-Fold end is the same capital walked by the scrumming bot on the IVP's logic, its units at the last close plus the cash its partial exits left. The difference is Harvest-Fold end less HODL end, as a figure and as a percentage of HODL end. The historical trough is the lowest the untraded holding read on a walked bar, a figure on the path. The partial exits and re-entries are counted beside them. The report's Battery section, the Activity Log's per-portfolio line and the summary read these per timeframe per portfolio; the summary totals them per timeframe over the portfolios that ran and lists each portfolio with its run id, its report file and its three figures. Nothing ranks, scores or declares a winner: the four words of the earlier reading, its rule, the verdict word, the best timeframe and the improved timeframes are gone from the module and the report.

`src/simulator/portfolio_battery.py` — the comparison in the report's words

```
COMPARISON_RULE = (
    "HODL end is the Target Balance bought at the walk's opening bar and held to "
    "its last bar; Harvest-Fold end is the same capital walked by the scrumming "
    "bot on the IVP's logic, its units at the last close plus the cash its "
    "partial exits left; difference is Harvest-Fold end less HODL end, as a "
    "figure and as a percentage of HODL end; historical trough is the lowest the "
    "untraded holding read on a walked bar. Nothing ranks or declares a winner."
)
```

`src/simulator/portfolio_battery.py` — the three figures of one timeframe

```
    @property
    def difference_usd(self) -> float:
        """``accumulation_usd`` less ``baseline_usd``."""
        return self.accumulation_usd - self.baseline_usd

    @property
    def difference_pct(self) -> float:
        """``difference_usd`` as a percentage of ``baseline_usd``."""
        return difference_pct_of(self.baseline_usd, self.accumulation_usd)
```

`src/simulator/parity_report.py` — the section per timeframe

```
                f"- HODL end: {money(frame['baseline_usd'])} (bought at the "
                "opening bar, held to the last)",
                f"- Harvest-Fold end: {money(frame['accumulation_usd'])} "
                f"({frame['partial_exits']} partial exit(s), {frame['re_entries']} "
                "re-entr"
                + ("y" if frame["re_entries"] == 1 else "ies")
                + ")",
                f"- difference: {frame['difference_usd']:+,.2f} "
                f"({frame['difference_pct']:+.2f}% of HODL end); {frame['arithmetic']}",
                f"- historical trough: {money(frame['trough_usd'])}",
```

The two emits

Each portfolio's start emits `sim.battery.portfolio_started` with the bots seated against the portfolio's symbol count, and its end emits `sim.battery.portfolio_finished` with the three figures, the evaluations made against the evaluations expected and the fills per timeframe; the row reads `ok` when every walk that ran made every evaluation expected of it and Stop did not end it.

`src/simulator/portfolio_battery.py` — the two signals

```
#: The signals one portfolio's start and end emit through ``signal_contract``.
PORTFOLIO_STARTED_SIGNAL = "sim.battery.portfolio_started"
PORTFOLIO_FINISHED_SIGNAL = "sim.battery.portfolio_finished"
```

What the every-candle reading measured

Both builds, the real window over a scratch home, every socket but loopback refused, a scratch RA root holding the operator's daily files for SPY, GLD, A15, A14, BNB and SLV, and a loopback stand-in for the public candle endpoint serving 5m for A15 and A14 from 2026-04-01 and refusing every other product. Before, Run Portfolio over CRYPTO_BLUE on `2026 test run` read A15 at 5m 49,328 bars and 121 ticks, A15 at 1d 172 bars and 143 ticks, the Battery section `5m defended, 1d defended`, and no line between the retrieved line and the portfolio line; Run Every Portfolio held sixty-three bots from the press to the end and wrote one report. After, the same press read A15 at 5m 49,336 bars and 49,307 evaluations, A14 the same, A15 at 1d 172 bars and 143 evaluations, each the bars less 29, in 223 and 203 seconds at 4.5 and 4.1 seconds per 1,000 evaluations in Qt and 174 and 144 seconds at 3.5 and 2.9 in React; a line at each walk's start, every 5,000 bars and at its end; the Battery section reading, at 5m, HODL end $1,313.13, Harvest-Fold end $2,183.19, difference +870.06 (+66.26% of HODL end) with 609 partial exits and 2,769 re-entries on the stand-in's candles, and at 1d HODL end $1,093.45, Harvest-Fold end $1,096.17, difference +2.71 (+0.25%). Run Every Portfolio on the same span held no fleet at the press, no venue seated and the Get Started card drawn, then AGGRESSIVE's ten on Coinbase and Yahoo with the loaded line naming its budget of $5,000.00, then ALL_WEATHER's five on Yahoo at $2,500.00, then ARK_SUITE's five, then BALANCED's ten, and so on through thirty-five portfolios, the held fleet read empty between two of them and at the end and the fleet file's Portfolio Battery map holding no bot after the last clear (a first reading, before the press held nothing, had shown sixty-three bots at the press); thirty-six reports, one per portfolio and the summary, whose per-portfolio table lists thirty-five run ids; 844,289 evaluations of 844,289 expected in both builds, 80 minutes in Qt beside a React run of 51 minutes; the summary's totals at 5m over nine portfolios and at 1d over twenty-three read the same figures in both builds. Stop pressed on BNB's row a hundred seconds into the crypto walk ended it at bar 5,008 of 49,340 in Qt and 5,886 of 49,358 in React, the partial report naming the bar, its candle stamp and the evaluations made, and A14 and BNB as not walked. With `portfolio_battery.walk` wrapped in the driver process to step fourteen bars, A15 at 5m read 3,523 evaluations against 49,312 expected and the finished emit read `ok` false, so the reading can fail. Every copied daily file and `bot_state.json` hashed identical after every press, with a planted byte moving the hash; the operator's RA root listed 414 entries before and after; no socket left loopback. Both bundles, launched in isolation over a scratch home, ran Portfolio over EQUITY_MACRO on `All` and read SPY and GLD at 1d 1,678 bars and 1,649 evaluations each, at 1w 349 and 320, at 1M 81 and 52, and at 1d HODL end $2,513.22, Harvest-Fold end $2,001.64, difference -511.57 (-20.36% of HODL end) on the operator's own Yahoo files.

The Simulator's run emits go to the Simulator's own signal sink

Every emit a run raises on its worker thread lands in the Simulator's own signal sink, `signals/session.jsonl` under the sim bucket, and none in the process sink under `signals/` of the log root. The operator, 2026-09-18: *"Entire idea for the Emitter Network is to provide us signals for verifying proper software function resulting from all user actions or automated sequences."* Before this, one Portfolio Battery press over one crypto portfolio at 5m emitted 1,287,112 rows into the process sink, its ladder of six files rotated through five backups, and the press row, the live pins and every row from before the press rotated out with them. The emit sites are not changed, `src/trading/ta_engine.py` is not opened, and the shared trading code does not know which side it runs on: the wire itself learned a routing by thread.

`src/core/signal_contract.py` — the routing seam

```
def route_thread(sink: SignalSink) -> None:
    """Send every `emit` raised on the calling thread to `sink` until `unroute_thread`."""
    with _ROUTES_LOCK:
        _ROUTES[threading.get_ident()] = sink


def unroute_thread() -> None:
    """Return the calling thread's emits to the process sink `set_sink` installed."""
    with _ROUTES_LOCK:
        _ROUTES.pop(threading.get_ident(), None)


def get_sink() -> Optional[SignalSink]:
    """The sink the calling thread emits into: its `route_thread` sink when one
    is set, else the process sink."""
    if _ROUTES:
        with _ROUTES_LOCK:
            routed = _ROUTES.get(threading.get_ident())
        if routed is not None:
            return routed
    with _ACTIVE_LOCK:
        return _ACTIVE
```

`emit` reads `get_sink()` and hands the record to the sink it answers, as it did before. With no route set the dict is empty, and the wire is one truth test on an empty dict, then the lock and the process sink, as before. A thread routes itself, by its own ident, and clears its own route, so a route is never left under an ident the thread no longer holds.

The sim sink is the process sink's class over the sim bucket

The sim sink is one `SignalSink` per process, the class the process sink is, at `sim/signals/session.jsonl`, opened on the first run of the process and answered on every run after. It rotates at 50 MiB into `.1` to `.5`, keeps 350,000 rows in memory and flushes every 500 rows, the process sink's own figures, and thins its own `session.digest.jsonl` beside it as the process sink does. Each row has the shape `Signal.to_json` writes on both sides.

`src/simulator/sim_bus.py` — the sim sink

```
def sim_signal_sink() -> SignalSink:
    """The one ``SignalSink`` per process at ``SIGNALS_FILE`` under
    ``SIGNALS_DIR`` of ``get_sim_dir``, built as ``install_process_sink``
    builds the process sink and opened on the first call."""
    global _SIGNAL_SINK
    with _SIGNAL_SINK_LOCK:
        if _SIGNAL_SINK is None:
            sink = SignalSink(flush_every=PROCESS_SINK_FLUSH_EVERY)
            sink.path = get_sim_dir() / SIGNALS_DIR / SIGNALS_FILE
            _SIGNAL_SINK = sink
        return _SIGNAL_SINK
```

Each worker marks itself for the length of its run

Every run's worker thread routes itself on its first statement and clears the route when the run returns or raises. One context, `routed_run`, does both: on entry it opens the sim sink, routes the calling thread and writes the Activity Log line; on exit it reads whether the thread's sink is still the sim sink, clears the route, flushes the sim sink and emits the routed row on the process sink. The six worker entries wrap their bodies in it: `_compute_battery`, `_compute_run` and `_compute_retrieval` on each host.

`src/simulator/sim_bus.py` — the run context

```
@contextmanager
def routed_run(run: str, line: Callable[[str], None]) -> Iterator[SignalSink]:
    """Route the calling thread's emits to ``sim_signal_sink`` for the block,
    hand ``line`` the ``SINK_LINE_FORMAT`` line on entry, and on exit unroute,
    flush the sim sink and emit ``SINK_ROUTED_SIGNAL`` on the process sink."""
    sink = sim_signal_sink()
    thread = threading.get_ident()
    rows_before = int(sink.health()["emitted"])
    route_thread(sink)
    try:
        line(SINK_LINE_FORMAT.format(run=run, path=sink.path))
        yield sink
    finally:
        routed = get_sink() is sink
        unroute_thread()
        sink.flush()
        restored = get_sink() is not sink
        emit(
            SINK_ROUTED_SIGNAL,
            actual={
                "thread": thread,
                "run": run,
                "path": str(sink.path),
                "routed": routed,
                "restored": restored,
                "rows": int(sink.health()["emitted"]) - rows_before,
            },
            expected={"routed": True, "restored": True},
            ok=routed and restored,
        )
        process_sink = get_sink()
        if process_sink is not None:
            process_sink.flush()
```

`src/gui/simulator/sim_trading_tab.py` — the Battery's worker under the context

```
        with routed_run(
            surface.MODE_PORTFOLIO_BATTERY,
            lambda line: self.battery_line.emit(line, "info"),
        ):
```

A Validation or Back Test run routes under its mode name, and the replay layer's tablet retrieval under `retrieval`. No run spawns a thread of its own: `run_battery`, `back_test.run` and `validation.run`

walk on the calling thread, and the retrieval runs its event loop on the worker thread. A route does not pass to a thread a routed thread starts.

The tab's own presses stay on the process sink

The six pins the tab emits on the GUI thread, `sim.fleet.mode_shown`, `sim.fleet.clear_pressed`, `sim.fleet.cleared`, `sim.run.start_pressed`, `sim.replay.marks_drawn` and `sim.layer.flipped`, are the operator's actions on the running program, and they land on the process sink as before, beside the live bots' pins and the Console's own. The Indicator Voting Panel's feed, `ivp_feed`, computes the selected bot's stored reading on the GUI thread, so the thirteen postcondition pins of that one evaluation stay on the process sink too. The Console's lower pane reads the process sink on the GUI thread, so it draws those and never the walk.

`src/gui/simulator/sim_trading_tab.py` — a GUI-thread pin, unchanged

```
        _pin_emit(
            tab_surface.LAYER_FLIPPED_SIGNAL,
            **tab_surface.flipped_pin(
                leaving, self._layer, pressed, self._last_flip_rect
            ),
        )
```

The routed row and the Activity Log line

At each run's end one `sim.sink.routed` row lands on the process sink. It carries the worker's thread ident, the run name, the sim sink's path, whether the thread's sink still read the sim sink when the run ended, whether it reads the process sink again after the unroute, and the rows the sim sink took over the run; it reads `ok` when both verdicts hold. A seam that stops consulting the route reads `routed` false, and one that does not clear it reads `restored` false. At each run's start the Activity Log names the sim sink's path, after the started line and before the first walk line.

`src/simulator/sim_bus.py` — the row and the line

```
#: The row ``routed_run`` emits on the process sink at each run's end.
SINK_ROUTED_SIGNAL = "sim.sink.routed"

#: The Activity Log line ``routed_run`` writes at each run's start.
SINK_LINE_FORMAT = "{run} signals: {path}"
```

The Watchdog Archetype reads source, not a sink: a pin is wired when its name resolves to `signal_contract.emit`, and the routing lives inside that wire, so a sim emit site reads wired on the same rule as a live one and the archetype needs no sink named.

Diagram: flowchart, left to right

- Run Portfolio on the GUI thread -> **the started line, the press row on the process sink**
- the started line, the press row on the process sink -> **_compute_battery on sim-portfolio-battery**
- _compute_battery on sim-portfolio-battery -> **routed_run: route_thread, the signals line**
- routed_run: route_thread, the signals line -> **run_battery, walk, compute_all**
- run_battery, walk, compute_all -> **_ta_emit through signal_contract.emit**

- _ta_emit through signal_contract.emit -> **get_sink**
- get_sink -> **sim/signals/session.jsonl** *(routed thread)*
- get_sink -> **signals/session.jsonl** *(any other thread)*
- run_battery, walk, compute_all -> **finally: unroute_thread, flush**
- finally: unroute_thread, flush -> **sim.sink.routed on the process sink**

What the sink reading measured

Both builds, the real window over a scratch home, every socket but loopback refused, the process sink installed as `main` installs it, unit 31's wide seed and a loopback stand-in for the public candle endpoint. Before, a Back Test over the 24 seeded bots took the process sink from 23 rows and 12,464 bytes at the press to 61,824 rows and 42,183,886 bytes at the end, 61,802 of the new rows the TA engine's, with a planted emit on the worker thread beside them; Run Portfolio over CRYPTO_BLUE on `2026 test run` emitted 1,287,112 rows, the ladder held 445,060 rows and 305,926,618 bytes in six files when the run ended, the sink's own health read 937,112 evicted, and the press row, the planted worker row and the planted GUI row had rotated out; no sim sink existed. After, the same Back Test left the process sink at 45 rows in Qt and 42 in React, the rows above the press watermark the press row, the panel's one evaluation, the Console's own pins, the planted GUI row and the routed row, while the sim sink took 61,777 rows and 42,157,581 bytes, 61,776 of them the TA engine's and one the planted worker row; the same Battery press left the process sink at 60 rows in Qt and 57 in React with zero evicted, no `ta.*` row and no `sim.battery.*` row above the press watermark, while the sim sink took 1,287,136 rows in Qt and 1,287,162 in React, its ladder holding 445,136 and 445,162 rows across six files at the end, its last row `sim.battery.portfolio_finished`. The routed row read `ok` with `routed` and `restored` true and `rows` 61,777 for the Back Test, 1,287,136 and 1,287,162 for the Battery, and 0 for Retrieve Tablet, whose connector emits nothing. The Activity Log named the sim sink's path after each started line. A planted emit on the GUI thread landed on the process sink and not the sim sink; the GUI thread routed on purpose for one emit landed that emit on the sim sink, so the reading can fail; a helper thread that routed itself and exited left the next GUI emit on the process sink; the flip's row landed on the process sink. The scratch `bot_state.json` and every copied daily file hashed identical after every press, with a planted byte moving the hash; no socket left loopback.

The widget the rebuild replaced

Every section from here to the end of the page describes a screen that is gone: the empty Sim tab, the original Simulator widget with Fleet Replay, Nuclear Mode and the Bridge, and the first clone units 7 and 8 replaced. Each is kept whole as the record of what the rebuild replaced, and nothing in them describes the tab that draws.

The empty Sim tab before the fork

The Simulator is removed. The tab is named Sim, it opens first on the bar, and it draws an empty panel with a heading and two sentences. Nothing behind it runs: no fleet, no replay, no practice venue and no Nuclear Mode. Every section below this one describes the screen that was removed, and is kept as the record of what the rebuild replaces.

The panel is the one the Paper, Status and Accumulation tabs already draw. Its words come from a view model, so the Qt build and the React build say the same thing.

```
METHOD = "simulator_tab.state"

HEADING = "Sim"
ISSUE = 117
BUILT = False
STATE_TEXT = "This tab is not built."
ISSUE_TEXT = f"Issue #{ISSUE} carries the build-out."
```

What builds it

The window constructs the tab and inserts it beside Trading. Three of the four wiring calls take a lambda, so the exchange connectors, the swarm and the Market Inspector proposals resolve when they are asked for rather than when the tab is built. The bot manager is handed over directly.

`src/gui/main_tabs/simulator_tab.py` — `SimulatorTabMixin._build_simulator_tab` The Simulator rebuild removed this file; it is not in the tree.

```
        self._simulator = SimulatorTab()
        # _reorder_main_tabs runs after every builder, so index 1 is not the final slot.
        self._main_tabs.insertTab(1, self._simulator, "Simulator")
        if hasattr(self._simulator, "set_connectors_getter"):
            self._simulator.set_connectors_getter(
                lambda: getattr(self, "_exchange_connectors", {})
            )
        if hasattr(self._simulator, "set_bot_manager"):
            self._simulator.set_bot_manager(self._bot_manager)
```

The tab is isolated. The window's header strip hides while the Simulator is in front, and the tab draws its own strip of the same ten fields against sim balances.

`sim_stat_strip` — `SimStatStrip.FIELDS` The Simulator rebuild removed this file; it is not in the tree.

```
    FIELDS = (
        "Spendable",
        "Realised",
        "Locked",
        "Mature",
        "Exch",
        "Scrummed",
        "Folded",
        "Trades",
        "Bots",
        "Errors",
    )
```

The Mode picker names three modes and drives two panels. Nuclear raises its own page. Validation and Looping Back Test both raise Fleet Replay.

`SimulatorTab` — `SimulatorTab._on_sim_mode_changed` The Simulator rebuild removed this file; it is not in the tree.

```
        key = self.sim_mode()
        stack = getattr(self, "_stack", None)
        if stack is not None:
            # Validation and Looping both drive the fleet page.
            stack.setCurrentIndex(1 if key == "nuclear" else 0)
```

The mode goes no further than the picker. The handler passes the key on only to a panel that carries a matching setter, and Fleet Replay carries none, so the two fleet modes run the same replay and the choice changes the hint line alone.

`SimulatorTab` — the mode hand-off The Simulator rebuild removed this file; it is not in the tree.

```
        panel = getattr(self, "fleet_replay", None)
        if panel is not None and hasattr(panel, "set_sim_mode"):
            try:
                panel.set_sim_mode(key)
```

Fleet Replay

`FleetReplayPanel` in `fleet_replay_panel` is the panel. Four steps run it. The Simulator rebuild removed this file; it is not in the tree.

Load. The panel builds one real bot for every saved config whose symbol has a Stone Tablet, and names the symbols it had to skip. The class body is live's own, so the sim runs live code against a stored tape rather than a second implementation; the single sim-only branch gives each bot a private event bus, which is why a sim fill never reaches the live sound engine or the live logs.

`fleet_replay_panel` — `_spawn_sim_fleet` The Simulator rebuild removed this file; it is not in the tree.

```
        def _spawn_sim_fleet(self) -> int:
            """Construct a real sim bot for every loaded config.

            Returns the number spawned. Reads the Stone Tablet registry
            through the same call Start Replay uses, rather than adding a
            second candle path.

            The controller is built but NOT started: the bots exist, hold
            their imported state and can be inspected, and the tape only
            advances when the operator presses Start Replay.
            """
```

Configs. The loader reads the operator's state file read-only and returns each bot's saved config. `load_smart_wires_from_state` does the same for the topology, and `summarize_loaded_configs` reports what loaded. The fleet is never fabricated; it is whatever that file holds.

`bot_state_loader` — `load_bot_configs_from_state` The Simulator rebuild removed this file; it is not in the tree.

```
def load_bot_configs_from_state(
    path: Optional[Path] = None,
    mode_filter: Optional[str] = "scrumming",
) -> list[dict[str, Any]]:
```

Fetch. Fetch YTD calls the same history function the History tab's Refresh calls, so the two return the same trades. A replay still runs without it, and the panel then says on screen that the run is synthetic.

`fleet_replay_panel` — `_on_fetch_ytd_clicked` The Simulator rebuild removed this file; it is not in the tree.

```
        def _on_fetch_ytd_clicked(self) -> None:
            """Call the same function the History tab calls.

            The History tab's Refresh dispatches
            ``fetch_all_history_chunked(bot_manager, since_ts)``. No custom
            enumeration, no exchange matching, no connector picking: if the
            History tab returns N trades, Fleet Replay returns the same N.
            """
```

Run. `FleetReplayController` in `fleet_replay_controller` drives it. The Simulator rebuild removed this file; it is not in the tree.

Piece	What it does
`_build_sim`	Assembles the bots and the tape
`sim_bot_id`, `live_bot_id`	Keep the two id spaces apart
`_on_sim_trade`	Records each fill

One assertion runs once, at the point where the bot set is final and before any of them can trade.

`fleet_replay_controller` — `_assert_capital_isolation` The Simulator rebuild removed this file; it is not in the tree.

```
    def _assert_capital_isolation(self) -> None:
        """Every constructed sim bot must be off the live registry.

        The guards upstream -- the factory raising, `_instantiate_bot`
        refusing, `_crr()` returning None in sim mode -- each protect
        one path. This checks the outcome they exist to produce, once,
        at the point where the bot set is final and before any of them
        can trade. Aborts the run rather than let a replay write sim
        reservations into the operator's live capital state.
        """
```

The bots trade against a tape, not against a venue. `TabletBackend` implements the ccxt surface beneath a real connector, holds the seeded balances, sweeps resting orders, and owns the one clock every symbol advances on.

`fleet_replay_controller` — the tape the bots are handed The Simulator rebuild removed this file; it is not in the tree.

```
        # TabletBackend implements the ccxt surface beneath a real CCXTConnector.
        self._tape = TabletBackend(
            {
                sym: [list(r) for r in rows]
                for sym, rows in self._candles_by_symbol.items()
            },
            balances=dict(seed_by_quote),
            fee_rate_by_symbol=_fees,
        )
        _conn = CCXTConnector("coinbase")
        _conn.attach_backend(self._tape)
        self._exchange = _conn
```

Three more modules sit beside the controller and none of them runs a replay. The older fake exchange is built only by tests, the union clock is constructed only inside that exchange, and the candle series a replay builds is thrown away once its symbol names have been read.

`sim_exchange` — `FleetSimExchange` The Simulator rebuild removed this file; it is not in the tree.

```
"""Candle-driven fake exchange for Fleet Replay, over real symbols.

Serves per-symbol ``CandleSeries`` through the ``ExchangeInterface`` API,
so a bot trades A15/USD or A14/USD against stored candles instead of a
venue. ``NuclearSimExchange`` (``nuclear_sim_exchange``)
is the other simulated venue and serves synthetic TAPEA/TAPEB symbols.
The Simulator rebuild removed this file; it is not in the tree.

Reachability: nothing under ``src/`` constructs ``FleetSimExchange``.
``FleetReplayController`` imports only ``make_symbol_series_map`` from
this module and runs against ``TabletBackend``
(``src/exchange/tablet_backend.py``). The class itself is built by tests.
```

`ReplayProgress` carries candles played, trades fired and tick failures back to the panel's progress timer, and a finished run is written under the log root in the live schema.

`sim_run_log` — `SimRunLog` The Simulator rebuild removed this file; it is not in the tree.

```
"""Persist a Fleet Replay run under ~/.acervator_logs/sim/ in the live schema."""
```

Two bot tables draw at once. The Trading tab's own table is mounted into the Simulator, and the fleet panel's three-column table is still built and still filled. Collapsing the two is the first of the seven pieces of issue #117.

`fleet_replay_panel` — `FleetReplayPanel.COLUMNS` The Simulator rebuild removed this file; it is not in the tree.

```
        # Three columns. The gate row lives in GateStatusPanel, not here.
        COLUMNS = ("Symbol", "Target USD", "Sim Trades")
```

The validation criterion

The Simulator is validated when the gates latch identically on the same data. Not profit and loss, and not the trade count.

No module measures that. Nothing in the tree reads a sim gate latch and a live gate latch and compares the two.

In development. Validation Mode measures it now; the section at the end of this page states how.

What the screen does show is each bot's arm state while the tape runs. On every refresh tick the panel reads each bot's last gate state and paints a double-stacked row of lights, ten scrum then nine fold.

`sim_visuals` — `GateLightsCell` The Simulator rebuild removed this file; it is not in the tree.

```
    class GateLightsCell(QWidget):
        """One linear labelled row of trading gates.

        ``_draw_bank`` paints the ten ``_GATE_ORDER_SCRUM`` lights then the
        nine ``_GATE_ORDER_FOLD`` lights, each below its own label.
        ``gate_light_color`` picks every colour, and ``update_gates`` sets the
        arm and blocker state ``paintEvent`` reads.
        """
```

That cell and the History table's gate cell read one vocabulary, so the two surfaces cannot drift apart on what a gate is called or on what blocked it.

`src/trading/gate_vocabulary.py` — the shared labels

```
"""gate_vocabulary.py -- the bot's gate labels, blocker map and light states.

Qt-free. Two renderers read it: the Simulator's ``GateLightsCell``
(``sim_visuals``) and the History table's
gate cell (``src/exchange/history_read_contract.py``). One vocabulary,
so a gate added on one surface cannot be missing from the other.
```

The Simulator rebuild removed this file; it is not in the tree.

After a run the panel matches each live trade to the nearest sim fill on the same symbol and side, and prints the result into the Performance log. That is a report on trade counts and timing. It is not the criterion above.

`src/trading/stone_tablets/parity_harness.py` — `compare_trades`

```
def compare_trades(
    live_trades: list[dict],
    sim_trades: list[Any],
    tolerance_s: float = DEFAULT_TOLERANCE_S,
    window_since_ts: float = 0.0,
    window_until_ts: float = 0.0,
) -> ParityReport:
```

The panel puts the honest label on screen. A run with no live trades loaded is comparable to nothing, and the panel says so rather than looking like a parity run.

`fleet_replay_panel` — `_note_parity_state` The Simulator rebuild removed this file; it is not in the tree.

```
        def _note_parity_state(self) -> None:
            """Say so when the run cannot be compared to live.

            ``_ytd_trades`` being empty is the NORMAL condition, and the panel
            said nothing about it, so a synthetic run looked exactly like a
            parity run on screen.
            """
```

`SimValidationGuard` names five conditions under which a run cannot be trusted. The last two halt immediately whatever their count. The guard is defined and covered by tests, and no module under `src/` calls it.

`sim_validation_guard` — `ValidationIssueType` The Simulator rebuild removed this file; it is not in the tree.

```
class ValidationIssueType:
    NO_TABLET = "no_tablet"
    PRE_LISTING = "pre_listing"
    UNRESOLVABLE_TS = "unresolvable_timestamp"
    SCHEMA_DRIFT = "schema_drift"
    ADDRESS_MISMATCH = "address_mismatch"
```

The voting readout beside the fleet can fill itself with invented numbers. Three seconds after the tab is built, with no bot yet in its selector, it runs the real voting engine over a seeded random walk and draws the vote. On the Simulator that is the normal state until a fleet loads, so the panel can read as a result when it is a placeholder.

`src/gui/indicator_panel.py` — `IndicatorVotingPanel._generate_demo_ta`

```
        def _generate_demo_ta(self):
            """Generate demo TA from synthetic candles.

            Gated by ``_may_fabricate()``. Produces a deterministic
            random walk (``seed = md5(bot_id)``) and runs the real
            VotingEngine over it, labelled with the real bot's symbol
            so the panel stays stable across refreshes.
            """
```

Nuclear Mode

The operator cancelled Nuclear Mode. The rebuild deleted its panel, its two fleet controllers and its candle source, so no file this section once walked through is in the tree and no Nuclear run starts. The three modes the Sim tab carries today — Validation, Back Test and Portfolio Battery — stand in its place, and the record of what Nuclear did stands below, under what the closed issues landed.

One module outlived it. `src/core/system_load_oscillator.py` holds `SystemLoadOscillator`, which supplied the load pulse that paced a run, and nothing in the tree constructs it.

Bridge

Five methods are registered for this screen, and the renderer modules carry the matching names. They serve the Electron frontend; the tab the operator runs today is drawn in Qt.

Bridge method	Serves
`simulator_tab.state`	The tab, its mode switcher and its table
`fleet_replay_panel.state`	Fleet Replay
`nuclear_mode_panel.state`	Nuclear Mode
`sim_stat_strip.state`	The tab's own ten-field strip
`sim_visuals.state`	The gate lights and the sim charts

`src/core/desktop_bridge.py` — `build_registry`

```
  def build_registry() -> Dict[str, Handler]:
      """Return the methods the frontend may call, keyed by method name.

      This is the wiring point for the whole frontend: a surface is
      reachable exactly when it appears here. Surfaces are imported inside
      the function so that the transport above carries no dependency on any
      one domain package.
      """
```

2026-09-08 08:17 - #117 - what the closed issues landed

The old Simulator was removed and the rebuild replaced it. The tab is named Sim, it opens first on the bar on a black ground, and it draws a clone of the Live tab reading Stone Tablets. It carries three modes — Validation, Back Test and Portfolio Battery — and Nuclear Mode is cancelled. The rest of this section describes the screen that was removed and is kept as the record of what the rebuild replaces.

```
  def _build_simulator_tab(self) -> None:
      """Insert the Sim tab at ``SIMULATOR_BUILD_INDEX``."""
      self._add_empty_tab(simulator, index=SIMULATOR_BUILD_INDEX)
```

The tab stacks two panels. Fleet Replay loads every bot from the operator's own state file, builds one real bot per config, and plays Stone Tablet candles through them against a fake exchange. The sim uses the bot class body unchanged, which is the parity guarantee: it runs live's code against a fake exchange rather than a second implementation.

The tab is now called Sim. It sits first on the bar, on a black ground with red text.

`fleet_replay_panel` — `_spawn_sim_fleet` The Simulator rebuild removed this file; it is not in the tree.

```
def _spawn_sim_fleet(self) -> int:
    """Construct a real sim bot for every loaded config.

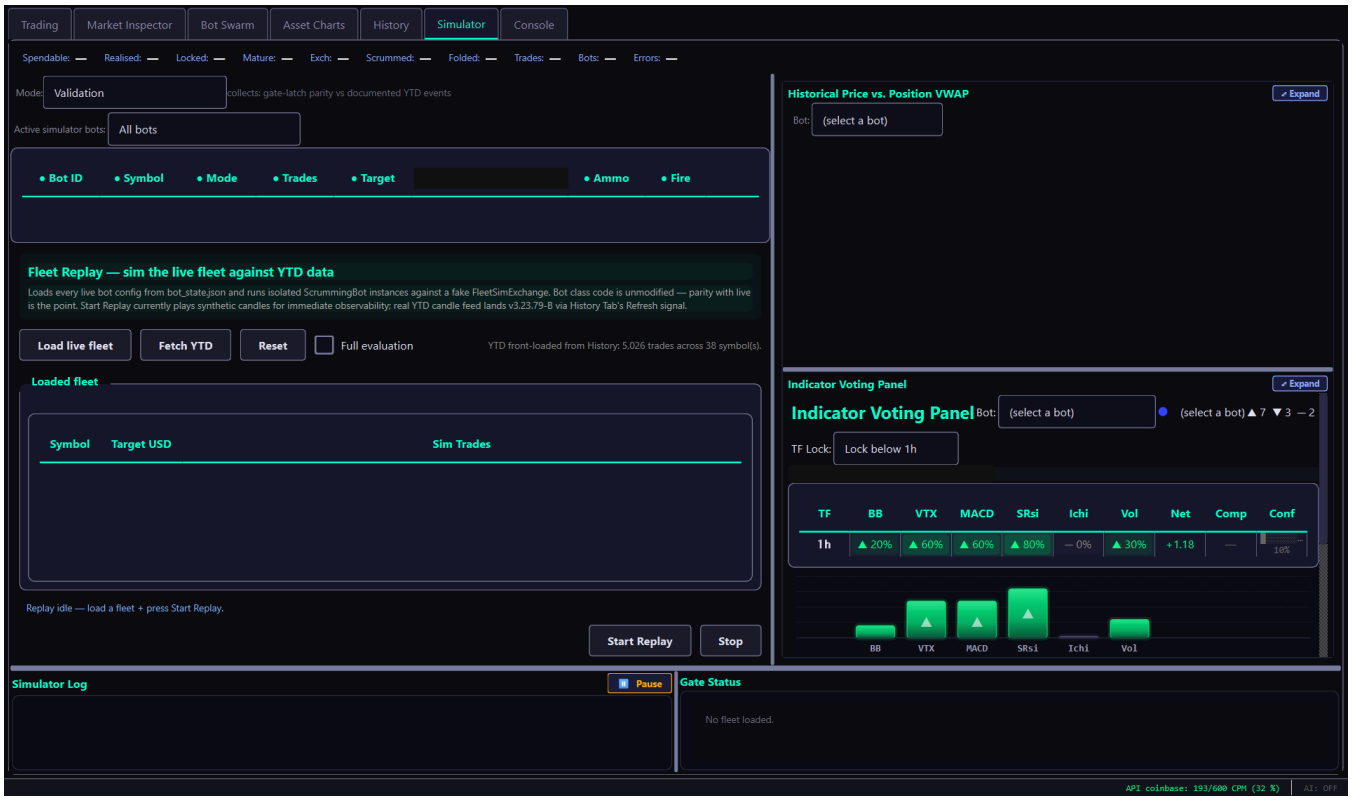
    Returns the number spawned. Reads the Stone Tablet registry
    through the same call Start Replay uses, rather than adding a
    second candle path.
```

The criterion is the gates latching identically on the same data. Not profit and loss, and not the trade count. The sim's gate cell reads the same vocabulary the History table reads, which stops the two surfaces drifting apart.

`sim_visuals` — `GateLightsCell` The Simulator rebuild removed this file; it is not in the tree.

```
class GateLightsCell(QWidget):
    """One linear labelled row of trading gates.
```

Nuclear Mode looped the same fleet over the tablet window with per-cycle market noise and a load pulse, writing over no tablet. It stood as a soak test, judged on coverage and survival, and it compared nothing to live. The operator cancelled it, its code is deleted, and the Sim tab has three modes, not four.



The Simulator tab, with no fleet loaded.

The strip along the top replaces the window's own while this tab is active. It names the same ten readings against sim balances, and every one draws an em dash in the figure, because no fleet is loaded.

React draws that strip. The tab asks `variant_surface` which of the two strips to build, and both answer the same `set` and `clear` calls, so the fleet panel and Nuclear Mode write to either without knowing which they hold.

`SimulatorTab` — `SimulatorTab._build_stat_strip` The Simulator rebuild removed this file; it is not in the tree.

```
from ..variant_surface import SIM_STAT_STRIP, surface_class

return surface_class(SIM_STAT_STRIP)(self)
```

`SimStatStripWebStrip` holds `SimStatStripModel` where the Qt strip held ten label pairs, and draws it with `sim_stat_strip.js` in one `QWebEngineView`. A field name the strip does not carry is still ignored, and an empty value still falls back to the em dash. Setting `ACERVATOR_VARIANT` to `qt` builds the Qt strip instead, unchanged.

React drew the Nuclear Mode page too, before the rebuild deleted both. The tab asked the same seam for the panel class that it asked for the strip.

`SimulatorTab` — `SimulatorTab._nuclear_panel_class` The Simulator rebuild removed this file; it is not in the tree.

```
from ..variant_surface import NUCLEAR_MODE, surface_class

return surface_class(NUCLEAR_MODE)
```

`NuclearModeReactPanel` inherits the Qt panel, so the fleet preview, Start, Stop and the half-second status tick are one piece of Python on both sides. Every widget write moved behind a named accessor. The Qt panel answers those with labels, spin boxes and tick boxes; the React panel answers them by writing into its view model and redrawing the page.

`react_nuclear_mode_panel` — `NuclearModeReactPanel._set_status_text` The Simulator rebuild removed this file; it is not in the tree.

```
def _set_status_text(self, key: str, text: str) -> None:
    """Show ``text`` on the live-status row ``key`` names."""
    if key in self._panel.status_text:
        self._panel.status_text[key] = text
        self.push()
```

The page draws the header card, the fleet readout, the four run settings, the Start and Stop buttons and the seventeen live-status rows. A press on the page runs the inherited Python and comes back as a redraw. Setting the variant to `qt` builds the Qt panel instead, unchanged.

The two drawn panels follow the same route. `SimPriceVwapChartReact` inherits the price and VWAP chart, so the ticks, the thinning and the trade markers stay one piece of Python, and `GateStatusPanelReact` inherits the gate pane.

`SimulatorTab` — `SimulatorTab._price_chart_class` The Simulator rebuild removed this file; it is not in the tree.

```
from ..variant_surface import SIM_PRICE_CHART, surface_class

return surface_class(SIM_PRICE_CHART)
```

Neither panel is redrawn in JavaScript. Python builds the whole draw program — every line, box, dot and label, with its colour and its position — and `sim_visuals.js` runs that program on one canvas. The picture is decided on the Python side, so the two builds cannot draw different charts from the same ticks.

`sim_visuals_surface` — `chart_program` The Simulator rebuild removed this file; it is not in the tree.

```
if not model.symbols:
    return []
if model.focus:
    return focused_program(model, width_px, height_px)
return band_program(model, width_px)
```

The gate pane draws one labelled row per bot and nineteen lights on each, and a gate reading written from Python changes what those lights show. An empty pane still says "No fleet loaded."

The Fleet Replay page is the last of the four. `FleetReplayReactPanel` inherits the Qt panel, so Load live fleet, Fetch YTD, Reset, Start Replay, Stop and both timers are the same Python. Every widget write the panel used to make moved behind a named accessor, and the React panel answers those by writing into its view model.

`SimulatorTab` — `SimulatorTab._fleet_replay_class` The Simulator rebuild removed this file; it is not in the tree.

```
from ..variant_surface import FLEET_REPLAY, surface_class

return surface_class(FLEET_REPLAY)
```

A press on the page names its own action. The panel maps that name to the method the Qt button was wired to, looked up on the panel itself, so the two sides run one piece of code.

`react_fleet_replay_panel` — `FleetReplayReactPanel.run_action` The Simulator rebuild removed this file; it is not in the tree.

```
step = surface.ACTIONS.get(str(request.get(ACTION_KEY) or ""))
named = self.STEP_RUNNERS.get(step)
if named is not None:
    getattr(self, named)()
```

Everything else on this tab is still drawn by Qt.

Mode is the picker beside it. Three modes, each with its own line saying what it collects.

`SimulatorTab` — `SimulatorTab.SIM_MODES` The Simulator rebuild removed this file; it is not in the tree.

```
SIM_MODES = (
    (
        "Validation",
        "validation",
        "Stone Tablets paired with YTD data. Verifies trade-gate "
        "parity at documented events.",
    ),
    (
        "Looping Back Test",
        "looping",
        "Loops the tablets with market-restructuring noise at a fixed "
        "rate. Strategy development and calibration.",
    ),
    (
        "Nuclear",
        "nuclear",
        "Load oscillation, swarm injection and high-traffic smart "
        "wire. Measures performance, stability and reliability — not "
        "trade validity.",
    ),
)
```

The table under it is the same class the Trading tab uses, with the same ten columns and a privacy dot on each header.

Column 2 is now Current Position Value, priced from the exchange, and the state colour it used to carry sits on the Bot ID cell. The Trading tab page describes both — see The bot tables.

`src/gui/widgets/bot_status_table.py` — `BotStatusTable.SCRUMMING_COLUMNS`

```
SCRUMMING_COLUMNS = ColumnSpec(
    labels=(
        "Bot ID",
        "Symbol",
        "Mode",
        "Trades",
        "Target",
        "Target A15",
        "Target A14",
        "Ammo",
        "Fire",
        "",
    ),
```

Fleet Replay fills the left column. Load live fleet builds the bots, Fetch YTD pulls the year of live trades the run is compared against, and Reset clears both. Full evaluation widens the run. The line beside the buttons counts the trades and symbols the History tab front-loaded. Loaded fleet carries three columns: Symbol, Target USD and Sim Trades, and the last of those increments during a run. The progress label reads `Replay idle` until Start Replay runs, and Stop drains the current tick and exits.

The panel says so out loud when a run cannot be compared to live, rather than letting a synthetic run look like a parity run.

`fleet_replay_panel` — `_note_parity_state` The Simulator rebuild removed this file; it is not in the tree.

```
def _note_parity_state(self) -> None:
    """Say so when the run cannot be compared to live.
```

The right column stacks two panels, each with its own Expand button. The first charts historical price against position VWAP for the bot its picker names, drawn from that bot's Stone Tablet candles. The second is the same Indicator Voting Panel the Trading tab carries: a bot picker, the vote count badge, the TF Lock, the currency rate line, then the first six voters — BB, VTX, MACD, SRsi, Ichi and Vol — with Net, Comp and Conf, and the confidence bars under them. The remaining six, Sling, ADX, STrd, ZSc, KER and RSI, sit below the area the figure shows.

Simulator Log and Gate Status close the tab. Gate Status draws the same nineteen lights the History table draws, and `No fleet loaded.` stands in its place until a fleet loads.

What is on the tab now

The rebuild's first tab is on the bar. It is a clone of the Trading tab reading Stone Tablets: the bot list with the live table's own ten columns, the Indicator Voting Panel, and a second layer holding the VWAP window over the tablet playback window, with one button between them. The crypto news ticker and the data pool line are not copied, and the rows they held stay empty for Import Live Fleet and Generate From YTD.

`src/gui/main_tabs/simulator_tab_surface.py` — the candle window

```
#: The candle window live reads. ``ScrummingBot`` asks ``get_ohlcv`` for 100,
#: and the Simulator reads the same count off the tablet.
WINDOW_CANDLES = 100
```

The Simulator receives and asks; it never sends. Its one data path reads tablet files, holds no venue, and refuses by name anything that is not a read.

The buttons that start a Validation run

Those two rows now carry the buttons that start a run. Validation snaps each YTD trade to the historical candle its timestamp falls in, reruns the gates on that candle, and puts the gate row it latched beside the gate row the log recorded. It passes when the gates latch identically, and it is never judged on profit or on a trade count.

```
Import Live Fleet    38 bots from the bot_state load, matched by bot id
Generate From YTD    39 new bots, one per traded pair, matched by pair
```

A run always reports what it could not verify. The tablets end on 1 August 2026 and the trade export runs to 7 September, so about five weeks of trades have no candle to snap to. The pane names that period and counts the entries on both sides of it.

```
3942 of 4904 YTD entries snapped to a candle; 962 could not be.
uncovered span 2026-08-01T18:52:58Z to 2026-09-07T21:46:48Z
```

The first clone, before the fork

The Sim tab is a clone of the Trading tab, and its data source is the Stone Tablets on disk. It carries the bot list, the Indicator Voting Panel, and a second layer holding the VWAP window over the tablet playback window. One button flips the panel area between those two layers.

`src/gui/main_tabs/simulator_tab_surface.py` — the two layers and the button

```
LAYER_INDICATORS = "indicators"
LAYER_PLAYBACK = "playback"
LAYERS = (LAYER_INDICATORS, LAYER_PLAYBACK)

FLIP_BUTTON_TEXT = {
    LAYER_INDICATORS: "Show Playback",
    LAYER_PLAYBACK: "Show Indicators",
}
```

One model is built from the tablet, and both builds draw it.

Diagram: flowchart, left to right

- RA-StoneTablet files -> **TabletSource**
- TabletSource -> **simulator_tab_surface**
- simulator_tab_surface -> **SimulatorTabQt**
- simulator_tab_surface -> **simulator_tab.js**

It receives and asks. It never sends.

The Simulator's whole data path is one class that reads tablet files. It holds no venue and defines no write. It answers five names and refuses every other name itself, so a send cannot be expressed through it.

`src/simulator/tablet_source.py` — the refusal

```
    def __getattr__(self, name: str):
        """Refuse every name outside ``READ_NAMES``."""
        raise SendRefused(
            f"TabletSource answers {READ_NAMES} and cannot {name!r}. "
            "The Simulator receives and asks; it sends nothing."
        )
```

Asked for nine venue calls — an order, a market buy, a cancellation, an edit, a withdrawal, a leverage change, a transfer, a tablet write and a save — it refused all nine and answered every read.

The columns are the Trading tab's own

The bot list draws the ten columns the live table draws, and the same two fixed widths, because the surface imports them rather than restating them. A column added to the live table appears here with no second edit.

```
from .bot_status_table_surface import COLUMN_LABELS, FIXED_WIDTHS
```

No simulated fleet exists yet, so the list holds no rows and says why. Import Live Fleet and Generate From YTD are what fill it, and both are later units.

One candle window, on both sides

The live engine asks the venue for one hundred candles. The Simulator reads the last hundred rows off the tablet, so the indicator window is one number on both sides rather than two.

```
#: The candle window live reads. ``ScrummingBot`` asks ``get_ohlcv`` for 100,
#: and the Simulator reads the same count off the tablet.
WINDOW_CANDLES = 100
```

The live engine now receives that hundred, and the newest row in it is the bar that just closed. The connector sends the count in the exchange library's count slot. Until this change it sent the count in the start-time slot and left the count slot empty, so the venue answered with a page of its own and the library kept the oldest 300 rows of it. Every indicator then read a window ending 50 five-minute bars behind the price the gate compared it against. That is four hours and ten minutes.

`src/exchange/ccxt_connector.py` — four arguments, each in its own slot

```
data = await self._call_sync(
    self._ex.fetch_ohlcv,
    symbol,
    timeframe,
    None if since is None else int(since),
    int(limit),
)
```

The library's own slice was driven over 1,215 recorded five-minute pages. The window ended 50 bars early on every page before the change and on none after it. On 162 rows of the recorded gate log, the band latch differs between the stale window and the current one on 88.

The VWAP window

VWAP is the published cumulative figure: typical price times volume, running, divided by running volume, where typical price is the average of the high, the low and the close. A point with no volume behind it yet carries nothing rather than a number.

```
def vwap_series(candles: Sequence[Sequence[float]]) -> list[Optional[float]]:
    """Cumulative ``sum(typical_price * volume) / sum(volume)`` per row.

    A row whose cumulative volume is still zero carries None.
    """
```

The two strips that are not copied

The crypto news ticker and the data pool line are not on this tab, and their code is not carried into it. A live news feed and a live cache-health line describe nothing a tablet reader does. The rows they held keep their height and hold nothing, and that space is where Import Live Fleet and Generate From YTD go.

```
#: The rows the two strips held on the Trading tab, and the height each keeps.
RESERVED_ROWS: tuple[dict[str, Any], ...] = (
    {"name": "news_ticker_row", "height_px": 24},
    {"name": "data_pool_row", "height_px": 18},
)
```

With no tablet on disk

The tab draws an empty state and names what is missing. The panel reads `No TA data — No Stone Tablet on disk.`, the selector holds nothing, and both windows hold zero points. Nothing is invented in place of the missing tape.

What one run measured

Both builds were opened and read off the drawn window and the drawn page. The figures below come from that run.

```
tablet          A18_1d_2026_coinbase, 250 candles, 2026-01-01 to 2026-09-07
window          100 candles
tablets listed  411
votes           5 bullish, 2 bearish, 5 neutral
Qt to React     32 of 32 values matched, with a tablet and again with none
```

The line beside the voting panel title

One label sits beside the Indicator Voting Panel title. It holds the panel's empty state and nothing else. `no_data_text` builds that line from the panel model and answers an empty string while the model holds a reading, so the label is hidden whenever the panel has values to draw.

`src/gui/main_tabs/indicator_panel_surface.py` — the line the label draws

```
def no_data_text(multi_tf_summary: dict, message: str) -> str:
    """``message`` through ``NO_DATA_FORMAT``, or empty while a summary exists."""
    summary = multi_tf_summary if isinstance(multi_tf_summary, dict) else {}
    if summary or not message:
        return EMPTY_TEXT
    return NO_DATA_FORMAT.format(message=message)
```

Both builds draw the same line. The Qt clone sets the label and hides it when the line is empty. The page module writes the same value into the element that carries the same name.

`src/gui/simulator_tab.py` — the Qt clone draws the line

```
def _draw_indicators(self, panel: dict) -> None:
    self._indicator_title.setText(panel["title_text"])
    # no_data text is empty while the panel holds a reading, so the
    # summary label shows only the one reason there is nothing to draw.
    reason = panel["no_data"]["text"]
    self._indicator_summary.setText(reason)
    self._indicator_summary.setVisible(bool(reason))
    for table, spec in zip(self._indicator_tables, panel["tables"], strict=True):
        self._fill_indicator_table(table, spec)
```

With no tablet on disk the label reads the sentence this page already names, under both builds.

```
No TA data — No Stone Tablet on disk.
```

The style sheet the Sim page loads

The page carries one style sheet. `TAB_STYLE_ASSETS` names it, and `page_html` reads the file and writes it into the page's head.

`src/gui/react_simulator_tab.py` — the sheet the page carries

```
TAB_STYLE_ASSETS: tuple[str, ...] = ("simulator_tab.css",)
```

The sheet sets the page ground, the type, the pane grid and the table rules. The grid repeats the three pane sizes the Qt clone gives its splitters, so the bot list, the layer pane and the bottom pane take the same share of the window in both builds.

`src/gui/web/simulator_tab.css` — the pane grid

```
.acervator-simulator-tab {
  display: grid;
  grid-template-columns: 600fr 500fr;
  grid-template-rows: 500fr 350fr;
  grid-template-areas:
    "fleet layer"
    "bottom bottom";
}
```

Without the sheet the page draws on the browser's own defaults: a white ground, serif type, one column and no grid.

The skin tokens the sheet reads

The sheet names no colour of its own. Every colour arrives as a custom property on the tab element, written from the `SKIN` dictionary the surface builds.

`src/gui/main_tabs/simulator_tab_surface.py` — three of the skin entries

```
SKIN = {
    "--sim-ground": ds.SURFACE_0,
    "--sim-better-colour": ds.SUCCESS,
    "--sim-agrees-colour": ds.SUCCESS,
}
```

A better portfolio row and an agreeing light take the same colours the Qt clone paints them, because both builds read one dictionary.

The two battery selectors reach the page

The portfolio and the span the operator chooses now reach the page before a battery has run. `build_view_model` takes both and hands them to `empty_battery`, which carries them into the payload the selectors draw from.

`src/gui/main_tabs/simulator_tab_surface.py` — the idle battery payload

```
charged = battery_payload_held or empty_battery(portfolio, span)
```

Both builds pass the two values. Before this the idle payload named the defaults, so a chosen portfolio was lost until a run wrote it back.

Crypto tablets are 5m and each bot walks the timeframe its tablet carries

The RA store holds two kinds of tablet beside each other under one MANIFEST. A non-crypto asset keeps its daily file, `GLD_1d_2025_yahoo.json`, written by the daily builder as before. A crypto asset gains a native five-minute file, `A15_5m_2026_coinbase.json`, the same shape the live fleet's store keeps, keyed on asset, timeframe, year and exchange by the same `tablet_filename`. The Battery reads a crypto bot off the 5m file and a non-crypto bot off the daily file, and the report names which one each bot read, with its timeframe, candle count and checksum. The operator's words, 2026-09-18: *"Crypto Tablets being daily is a failure. Must be 5m resolution. Cannot match trading logic for 5m against 1d charts."*

`src/simulator/portfolio_battery.py` — the timeframe each asset's tablet carries

```
def tablet_timeframe_for(asset: str) -> str:
    """The timeframe ``asset``'s RA tablet carries: ``CRYPTO_TABLET_TIMEFRAME``
    for a crypto asset, else ``TABLET_TIMEFRAME``."""
    return CRYPTO_TABLET_TIMEFRAME if is_crypto(asset) else TABLET_TIMEFRAME
```

One registry shape, one rollup

The RA store reuses the live registry. `StoneTabletsRegistry` opened on the RA root reads the RA MANIFEST as it stands, indexes every daily row and every 5m row, writes a 5m tablet through the same `ingest_candles` the live store uses, and rolls 5m candles up to any higher timeframe it supports through its one `_rollup`. Nothing in the registry changed; the Battery calls it on a second root. The daily side keeps its own fold: `resample` still builds a weekly and a monthly bar from daily rows, because the registry does not roll up to a week or a month.

`src/simulator/portfolio_battery.py` — the per-bot read

```
    def bars(self, bot: SimBot, timeframe: str) -> list[list[float]]:
        """``bot``'s bars at ``timeframe`` inside the span: a refused asset
        answers none, a crypto bot reads ``registry().get_candles`` on
        ``CRYPTO_EXCHANGE``, and a non-crypto bot ``resample`` over ``rows``."""
        asset = bot.asset
        if self.refusal(asset):
            return []
        if not is_crypto(asset):
            return resample(self.rows(asset), timeframe)
        if str(timeframe) not in SUPPORTED_TIMEFRAMES:
            return []
        if self._end_ms <= self._start_ms:
            return []
        return self.registry().get_candles(
            asset,
            self._start_ms,
            self._end_ms - 1,
            timeframe=str(timeframe),
            exchange_id=CRYPTO_EXCHANGE,
        )
```

The timeframes a bot walks

A crypto bot walks its own timeframe first, on the native 5m candles, then each of the Battery's three timeframes the registry can roll 5m up to. That is `1d`; `1w` and `1M` are outside the registry's supported set, so a crypto bot's walk is `5m` then `1d`. A non-crypto bot walks `1d`, `1w` and `1M` folded from its daily file, as before. A mixed portfolio reports four timeframes and each row names which bots reached it. Read against Live: the live bot's gate chain ticks its own timeframe, 5m on every one of the operator's thirty-eight bots, and its phantoms read higher timeframes from the venue; the Simulator's rollup is the registry's, so the Simulator and Live read the same hour from two sources, and the Back Test's phantom timeframes are unit 37's.

`src/simulator/portfolio_battery.py` — the set

```
def bot_timeframes(bot: SimBot, timeframes: Sequence[str] = TIMEFRAMES) -> tuple:
    """The timeframes the walk evaluates ``bot`` at: a crypto bot its own
    ``ta_timeframe`` on the native ``5m`` tablet and then each of
    ``timeframes`` in ``SUPPORTED_TIMEFRAMES``, the ones the registry rolls
    ``5m`` up to; a non-crypto bot ``timeframes`` as given, folded from its
    daily tablet."""
```

The chooser states the cost before the press

Run Portfolio's chooser carries one more line under the portfolio and span combos. It reads the preflight for the choice in force: for each crypto asset of the portfolio, the 5m candles the RA root does not hold over the span, the chunks and venue calls the fetch will make, and the bytes at the measured 63 bytes a stored candle. The line changes with every combo change. OK is the consent and nothing else asks. Over three crypto assets on the `2026 test run` span the line reads about 49,000 candles an asset, 846 calls and 9.3 MB; over `All` it reads 703,296 candles an asset, and the operator decides.

`src/simulator/tablet_retrieval.py` — the cost of one asset

```
def retrieval_cost(
    registry: StoneTabletsRegistry,
    asset: str,
    exchange_id: str,
    since_ms: int,
    until_ms: int,
) -> RetrievalCost:
    """The ``RetrievalCost`` of ``asset`` on ``exchange_id`` over
    ``[since_ms, until_ms]``: every ``TIMEFRAME`` step ``registry.missing_ranges``
    reports, through ``adapter_for``'s ``chunk_limit`` and ``PAGE_ROWS``; an
    exchange with no adapter costs nothing and is refused at the press."""
```

src/gui/simulator/sim_exchange_choice.py — the line on the chooser

```
def battery_cost_text(parent: Any, names: Any, span: str) -> str:
    """``portfolio_battery.cost_line`` over ``plan_costs`` on the parent's
    ``battery_tablet_source`` for ``names`` and ``span``; ``COST_UNREAD_TEXT``
    when the parent has no such source or the read raises."""
```

The Battery's own thread retrieves, then walks

After OK the Battery's daemon thread runs the retrieval before the first walk, through the same path the replay layer's Retrieve Tablet press uses: the read-only connector over the public candle endpoint, one chunk at a time through the gap filler, each chunk written into the year file and the MANIFEST as it lands, so a second press over the same span finds no gap and fetches nothing. One Activity Log line opens each asset's fetch and one closes it, naming the candles appended, the chunks, the calls and the file. A venue refusal closes the asset by name, and that bot reads `no_tablet` at 5m; the daily file never stands in for it. Each landed asset emits `sim.tablet.retrieved` and each refusal `sim.tablet.refused` through the signal contract, the prediction from the chooser's preflight beside the observed count.

src/simulator/portfolio_battery.py — the retrieval before the walk

```
    chosen = plan if plan is not None else plan_run(names, tablets)
    start_ms, end_ms = span_bounds(span, tablets.entries())
    costs = battery_costs(tablets, chosen.bots, start_ms, end_ms)
    retrievals = retrieve_tablets(tablets, costs, str(span), connector, progress)
    tape = TapeCache(tablets, start_ms, end_ms)
    for row in retrievals:
        if row["refused"]:
            tape.refuse(
```

The Sim API pane records a FETCH_TABLET block per connector call, the same block the replay layer's Retrieve Tablet press writes, because both hosts hand `run_battery` the tab's own connector and that connector crosses each call to the GUI thread, where the pane's writer accepts it. The recorder names the tablet off the call itself, through `call_tablet`: the symbol's base, the timeframe and the year of the first candle asked, so a Battery call reads `FETCH_TABLET A15_5m_2026_coinbase` and a replay-layer call keeps its own key. One block is one connector call of 350 candles; the public endpoint answers it in two pages of 300 and 50, which is why the chooser's call count is twice the block count. The Battery's per-asset lines stay on the Activity Log and the per-call progress line is the replay layer's alone.

`src/simulator/tablet_retrieval.py` — the tablet one call names

```
def call_tablet(call: Any, exchange_id: str = EXCHANGE_ID) -> tuple[str, str]:
    """``(key, file)`` of the tablet one ``VenueCall`` fills: ``tablet_filename``
    over the call's ``symbol`` base, its ``timeframe`` and the UTC year of its
    ``since_ms``, on ``exchange_id``; ``key`` is ``file`` without its suffix."""
```

`src/gui/simulator/sim_trading_tab.py` — the Battery on the tab's connector

```
                progress=lambda line: self.battery_line.emit(line, "info"),
                on_trade=self.battery_trade.emit,
                bus=self._bus,
                connector=self._connector,
```

The 2026 test run span

The live fleet's test run began 2026-04-01 with every bot on 5m. The chooser offers that span beside the archive's six, `2026 test run`, from 2026-04-01 to the current UTC day, read at call time so a window left open over midnight still ends today.

`src/simulator/portfolios.py` — the span

```
def period_window(span: str) -> Optional[tuple[str, str]]:
    """``PERIODS[span]`` with ``TEST_RUN_SPAN``'s end read as ``today_utc``
    now, or None for a label ``PERIODS`` does not hold."""
```

The report's Tablets section names each bot's file

Under the MANIFEST table the Battery report carries a per-bot table: the bot, its own timeframe, the timeframe its tablet carries, the file or files it read, the candle count, the checksum, the timeframes it walked with each outcome, and the refusal where it read no bar. A retrieval table follows it, one row per crypto asset the press checked: the candles asked, the candles appended, the chunks, the calls and the refusal.

`src/simulator/parity_report.py` — the per-bot rows

```
def battery_tablets_by_bot(run: BatteryRun) -> list[dict]:
    """One row per bot of ``run``: the bot's own ``ta_timeframe``, the
    ``tablet_timeframe`` its walk read, every ``TabletRead`` file with its
    candle count and checksum, the timeframes it walked with each outcome,
    and the ``refusal`` where it read no bar."""
```

What the 5m reading measured

Both builds, the real window over a scratch home, every socket but loopback refused, a scratch RA root holding the operator's daily files for A15, A14, BNB, GLD and SLV, and a loopback stand-in for the public candle endpoint serving 5m from 2026-04-01 and refusing BNB. Under Portfolio Battery, Run Portfolio opened the chooser with the cost line under the combos; the line changed on a control span; CRYPTO_BLUE on `2026 test run` read three assets, 49,313 candles each, 846 calls, 9.3 MB. OK: the Activity Log carried the retrieving line and the retrieved line for A15 and for A14, and the refusal by name for BNB; `A15_5m_2026_coinbase.json` and `A14_5m_2026_coinbase.json` landed under the scratch root with their MANIFEST rows and candle counts near 288 a day; two `sim.tablet.retrieved` rows and one `sim.tablet.refused` row reached the signal sink; the Sim API pane held 290 `FETCH_TABLET` blocks after two presses, 141 for A15, 141 for A14 and 8 for BNB, read off the Qt pane's text and the React page's API log element, while the stand-in counted two pages for each; the walk read A15 and A14 at `5m` and at `1d`, and BNB `no_tablet` at 5m; the report's Tablets section named each bot's file, timeframe, candle count and checksum. A second press over the same span stated nothing to retrieve and fetched nothing, and the two 5m files hashed identical before and after it. On a daily-only root with the stand-in absent, DIGITAL_GOLD refused A15 and A14 by name and walked GLD and SLV on their daily files at `1d`, `1w` and `1M`. Every copied daily file and `bot_state.json` hashed identical after every press, with a planted byte moving the hash; the operator's RA root listed 414 entries before and after. No bot was constructed and no socket left loopback.

The playback marks each fill on its candle and the header reads the opposing-trade distance

After a run, the Stone Tablet Playback window draws one glyph on the candle each fill landed on, at the fill's price, for the bot the chooser shows. A scrum draws `dissolve`, a triangle pointing down as a gold outline. A fold draws `reform`, a triangle pointing up filled blue. The layer's header carries a second line that reads the shown bot, the count of fold-to-scrum pairs, the mean and the median opposing-trade distance, and the estimated improvement. The operator's words, 2026-09-18: *"Just need to make sure these play back in the window with it marking where trades could have occurred. Not sure if relevant to even draw a VWAP chart but total estimated improvement in terms of % would be useful and can measure distances between opposing trades without having to know amounts that would have been traded."*

Diagram: flowchart, left to right

- back_test.walk -> **battery_trade / run_trade** *(SimTrade with scrum_price, timeframe)*
- battery_trade / run_trade -> **log_trade: the Activity line, and _fills**
- battery_finished / run_finished -> **_refresh_replay follow_run**
- _refresh_replay follow_run -> **replay_feed: shown_bot, fills_for, playback_payload, replay_figures**
- replay_feed: shown_bot, fills_for, playback_payload, replay_figures -> **PlaybackView marks and the figures label**
- replay_feed: shown_bot, fills_for, playback_payload, replay_figures -> **PlaybackWindow polygons and the figures span**
- replay_feed: shown_bot, fills_for, playback_payload, replay_figures -> **sim.replay.marks_drawn**

The fills the host keeps

Each host holds every `SimTrade` a run hands it, in fill order, on `_fills`. `log_trade` appends the trade beside the Activity line it writes, so a Battery fill and a Back Test fill reach the list by the one path both already took. The list empties when a run starts, in `_run_battery` after the chooser accepts and in `_start_run`, so a Back Test's marks never sit on a Battery's tape. The Battery's finished `BatteryRun` is held beside the list for the improvement figure. `fills` and `battery_outcome` answer both to the window.

`src/gui/simulator/sim_trading_tab.py` — the fill kept

```
    def log_trade(self, trade: SimTrade) -> None:
        """One Activity Log line per ``SimTrade``: ``trade_line`` under
        ``trade_stamp`` through ``SimStatusLog.log_at`` at ``TRADE_LINE_LEVEL``,
        and the trade appended to ``_fills`` for the replay layer's marks."""
        self._fills.append(trade)
```

The scrum a fold re-entered against

`SimTrade` carries two more fields. `scrum_price` is the scrum's own price on a scrum. On a fold it is `plan_source_price` over the fold's plan: each source tranche's `ref`, the price its scrum sold at, weighted by the dollars the plan takes from it. One fold that consumes two tranches reads their weighted centre, and the pair count is one per fold. Zero names no scrum, which is what a Validation rerun's fill carries, so a rerun forms no pair. `timeframe` is the `ta_timeframe` the walk ran at. The Battery walks a stock bot at `1d`, `1w` and `1M` under one bot id, and a weekly fill's stamp is a Monday's daily candle at the week's close; the daily tape marks the `1d` walk alone.

`src/simulator/back_test.py` — the record

```
@dataclass(frozen=True)
class SimTrade:
    """One scrum sell or fold buy the tape produced, with ``scrum_price`` the
    scrum's own ``price`` on a scrum and ``plan_source_price`` over the
    tranches consumed on a fold, zero naming no scrum. ``timeframe`` is the
    ``ta_timeframe`` the walk ran at, empty when unknown."""

    bot_id: str
    symbol: str
    side: str
    ts_ms: int
    price: float
    units: float
    usd: float
    fee_usd: float
    scrum_price: float = 0.0
    timeframe: str = ""
```

`src/trading/scrumming/sizing.py` — the fold's scrum price

```
def plan_source_price(plan: list) -> float:
    """The scrum price a fold under ``plan`` re-enters against: each source
    tranche's ``ref`` weighted by the USD the plan takes from it; zero for an
    empty ``plan`` or one taking no USD."""
```

The distance arithmetic

The opposing-trade distance of one pair is the scrum price less the fold price, as a percentage of the scrum price. A fold that re-entered below its scrum reads positive; one above reads negative. Over a run the header reads the count of pairs, the mean and the median. The two functions sit in `sizing.py` beside the arithmetic the same fills were sized under, and read no record: the pairs come from `trade_pairs` in the surface, each fold whose `scrum_price` is above zero. The improvement is the Harvest-Fold end less the HODL end as a percentage of the HODL end, `baseline_usd` and `accumulation_usd` of the shown bot's `SymbolRun` at the tape's timeframe, the figure the report's Battery section carries. A run with no Battery outcome reads an em dash there.

`src/trading/scrumming/sizing.py` — one pair, and the run's figures

```
def opposing_trade_distance_pct(scrum_price: float, fold_price: float) -> float:
    """``scrum_price`` less ``fold_price`` as a percentage of ``scrum_price``,
    positive for a fold that re-entered below its scrum."""
    return 100.0 * (float(scrum_price) - float(fold_price)) / float(scrum_price)


def opposing_trade_distances(pairs: Sequence[Sequence[float]]) -> dict:
    """``opposing_trade_distance_pct`` over each ``(scrum_price, fold_price)``
    in ``pairs`` whose scrum price is above zero: the ``count``, the
    ``mean_pct``, the ``median_pct`` and the ``distances_pct``, the two figures
    None when ``count`` is zero."""
```

`src/gui/main_tabs/simulator_tab_surface.py` — the improvement

```
def improvement_pct(hodl_usd: float, harvest_fold_usd: float) -> Optional[float]:
    """``harvest_fold_usd`` less ``hodl_usd`` as a percentage of ``hodl_usd``;
    None when ``hodl_usd`` is not above zero."""
```

The two glyphs

One definition, `MARK_GLYPHS`, names both marks, and both hosts read it: the Qt view in Python, the page off the playback payload. Each glyph is three corners in a unit mark, y down, centred on the fill. `dissolve` is the scrum: the corners `(-0.5, -0.5)`, `(0.5, -0.5)`, `(0, 0.5)`, an outline. `reform` is the fold: `(-0.5, 0.5)`, `(0.5, 0.5)`, `(0, -0.5)`, filled. A mark is one candle column wide and a twentieth of the pane tall: 6 px by 13 px on the 624 px pane at 1400, 4 px by 12 px on the 378 px pane at the floor. The colours are `--sim-mark-scrum`, Live's gold, and `--sim-mark-fold`, Live's blue, the two tokens the Charts tab colours its scrum and fold markers with.

`src/gui/main_tabs/simulator_tab_surface.py` — the glyphs

```
DISSOLVE_GLYPH = "dissolve"
REFORM_GLYPH = "reform"
MARK_GLYPHS = {
    back_test.SCRUM: {
        "name": DISSOLVE_GLYPH,
        "points": [[-0.5, -0.5], [0.5, -0.5], [0.0, 0.5]],
        "filled": False,
    },
    back_test.FOLD: {
        "name": REFORM_GLYPH,
        "points": [[-0.5, 0.5], [0.5, 0.5], [0.0, -0.5]],
        "filled": True,
    },
}
MARK_WIDTH_RATIO = 1.0
MARK_HEIGHT_FRACTION = 0.05
MARK_OUTLINE_PX = 1.5
```

The two glyphs are hermetic characters

Two sentences above are overtaken. Each is quoted whole, with the true sentence under it.

Overtaken: *"Each glyph is three corners in a unit mark, y down, centred on the fill."*

True now: each glyph is one character, centred on the fill.

Overtaken: *"*`dissolve` *is the scrum: the corners* `(-0.5, -0.5)`, `(0.5, -0.5)`, `(0, 0.5)`, *an outline.* `reform` *is the fold:* `(-0.5, 0.5)`, `(0.5, 0.5)`, `(0, -0.5)`, *filled."*

True now: the scrum is U+1F761, named ALCHEMICAL SYMBOL FOR DISSOLVE. The fold is U+1F75F, named ALCHEMICAL SYMBOL FOR PRECIPITATE. A mark is set at a twentieth of the pane in the symbol family, which resolves to Segoe UI Symbol, and both characters resolve on it. The corner lists, the fill flags and the outline width are gone with the triangles they described.

The definition lives in the chart's surface and the Simulator imports it, so both tabs draw the same two characters from the same object.

```
DISSOLVE_GLYPH = "\U0001f761"
REFORM_GLYPH = "\U0001f75f"
MARK_GLYPHS = {
    SCRUM_SIDE: {"name": DISSOLVE_GLYPH},
    FOLD_SIDE: {"name": REFORM_GLYPH},
}
```

The marks on the payload

`playback_payload` takes the shown bot's fills beside the candles and answers `marks`: one per fill whose stamp is a candle's stamp in the window, carrying the candle's index and x, the price's y over the window's bounds, the side, the scrum price and the glyph name. A fill on a candle outside the window is counted in `fills` and not in `mark_count`. The Qt view draws each mark as a polygon after the candles; the page draws one `polygon` element per mark with `data-side`, `data-glyph`, `data-price` and `data-candle-index`, its outline kept at pixel width under the window's non-uniform scale.

The last sentence above is overtaken. It is quoted whole, with the true sentence under it.

Overtaken: *"The Qt view draws each mark as a polygon after the candles; the page draws one* `polygon` *element per mark with* `data-side`*,* `data-glyph`*,* `data-price` *and* `data-candle-index`*, its outline kept at pixel width under the window's non-uniform scale."*

True now: the Qt view draws each mark's character after the candles. The page draws one text element per mark, carrying the same four data attributes, centred on the fill's point across and down. Neither takes an outline, and the payload no longer carries one.

`src/gui/main_tabs/simulator_tab_surface.py` — one mark

```
def mark_shapes(
    candles: Sequence[Sequence[float]],
    fills: Sequence[Any],
    low: float,
    high: float,
) -> list[dict]:
    """One mark per fill in ``fills`` whose ``ts_ms`` is a candle's stamp in
    ``candles``: the candle's ``index`` and ``x``, the fill's price as ``y``
    over ``low`` and ``high``, its ``side`` and the ``MARK_GLYPHS`` name."""
```

`src/gui/simulator_tab.py` — the Qt polygon

```
            polygon = QPolygonF(
                [
                    QPointF(centre_x + dx * mark_w, centre_y + dy * mark_h)
                    for dx, dy in glyph["points"]
                ]
            )
            painter.setPen(QPen(colour, outline))
            painter.setBrush(QBrush(colour) if glyph["filled"] else Qt.NoBrush)
            painter.drawPolygon(polygon)
```

The chooser reads the mode's tablet root

The replay layer lists and draws the tablets of the mode in force. `replay_source` answers the Battery's RA reader under Portfolio Battery and the live-fleet reader otherwise, and `_refresh_replay`, `_feed_replay` and the retrieval press read it, so after a Battery run over a stock portfolio the chooser lists each bot's daily file by its key, `SPY_1d_2024_yahoo`, and a retrieval press writes under the root the chooser was read from. Before this change the layer read the live-fleet root in every mode and listed `SPY_5m_yahoo — no tablet` after the same run.

`src/gui/simulator/sim_trading_tab.py` — the source by mode

```
    def replay_source(self) -> TabletSource:
        """The tablet reader the replay layer lists and draws from:
        ``battery_tablet_source`` under ``MODE_PORTFOLIO_BATTERY``,
        ``tablet_source`` otherwise."""
        if self._mode == surface.MODE_PORTFOLIO_BATTERY:
            return self._battery_tablet_source
        return self._tablet_source
```

The chooser names a tablet, and `shown_bot` names the bot whose fills the tablet marks: of the held bots on the tablet's market, the run's last-fill bot, else the panel's selected bot, else the lowest bot id. At a run's end both hosts refresh the layer with `follow_run`, so the chooser defaults to the last fill's bot; a chooser change draws the other bot's marks and figures.

`src/gui/main_tabs/simulator_tab_surface.py` — the shown bot

```
def shown_bot(
    bots: Sequence[Any],
    asset: str,
    exchange_id: str,
    last_bot_id: str = "",
    selected_bot_id: str = "",
) -> Any:
    """The bot whose fills the playback marks for a tablet on ``asset`` and
    ``exchange_id``: of the ``bots`` on that market, the one ``last_bot_id``
    names, else the one ``selected_bot_id`` names, else the lowest ``bot_id``;
    None with no bot on the market."""
```

The header's figures

The figures sit on the header's second line in both builds. The first line's fixed-width chooser and the retrieval button leave 180 px at 1400, measured, and the line needs about 470. The Qt label wraps at the floor; the page's span takes the header's full width under its wrapping row. With no run held the line reads `no run`.

`src/gui/simulator/sim_trading_tab_surface.py` — the line

```
REPLAY_FIGURES_FORMAT = (
    "{bot_id} · {pairs} pair{plural} · distance mean {mean} · median {median} · "
    "improvement {improvement}"
)
```

When the marks draw

The feed runs where it ran before: at build, on every `fleet_changed`, on the flip, on the chooser's change, when the panel's bot changes and after a retrieval. Two moments are added: the run's end, `_take_battery` and `_take_run`, with the chooser moved to the run's last bot. Every draw emits `sim.replay.marks_drawn` through `signal_contract` with the view's payload read back against the feed's: the tablet, the bot, the marks drawn, the fills handed, the pairs and the three percentages, on both sides.

`src/gui/simulator/sim_trading_tab_surface.py` — the emit row's figures

```
def marks_reading(key: str, playback: Any, figures: Any) -> dict:
    """The eight figures ``MARKS_DRAWN_SIGNAL`` carries on each side: the
    tablet ``key``, the bot, the marks drawn, the fills handed, the pairs and
    the three percentages, read off one ``playback`` payload and its
    ``figures``."""
```

What the marks reading measured

Read off the running program in both builds, scratch home, every socket but loopback refused, a scratch RA root holding EQUITY_MACRO's two daily files trimmed to 100 candles each, SPY's first hundred of 2024 and GLD's from row 125. On the base commit the same Battery run put 14 fills on the spool, SPY 8 and GLD 6, and the layer drew 0 candles, no mark, no figures line and no emit row, its chooser listing `GLD_5m_yahoo` and `SPY_5m_yahoo` with no tablet. After: Run Portfolio over EQUITY_MACRO on All, 14 fills on the spool at `1d`, the report's rows SPY 8 trades and GLD 6 at 1d and `short_tape` at 1w and 1M; the flip; the chooser on `GLD_1d_2024_yahoo`, the last fill's bot; 6 marks, four `dissolve` and two `reform`, each painted at its candle's x and its price's y, the Qt painter's polygons and the page's polygons both within a thousandth of a pixel of the payload's point; the header `GLD@yahoo · 2 pairs · distance mean +2.02% · median +2.02% · improvement -0.18%`, equal to the hand recomputation over the held fills (distances 1.89% and 2.15%) and to `(535.32 − 536.31) / 536.31` off the report's row. The chooser moved to `SPY_1d_2024_yahoo`, on the page through its own select: 8 marks, `SPY@yahoo · 4 pairs · distance mean +2.26% · median +2.14% · improvement -0.08%`, equal to the hand figures (1.85%, 1.64%, 3.13%, 2.43%) and the row's `(531.84 − 532.26) / 532.26`. A planted fill on a stamp outside the window raised `fills` to 9 and left `marks` at 8; a planted fold at 200 against a scrum at 100 moved the mean to –18.19% and the pairs to 5; the fills restored, the line read as before. At the floor, the pane at 378 px on Qt and 441 px on the page, the marks drew at 4 px by 12 px and the line wrapped whole. Nine `sim.replay.marks_drawn` rows per build, every one `ok`. `bot_state.json` and both tablets hashed identical after every step, a planted byte moving the hash; the operator's RA root listed 414 entries before and after; no socket left loopback; no bot was constructed.

The simulated bot acts like its live equivalent

A loaded bot reads `IDLE` until a run starts. While its walk runs its row reads `RUNNING` and moves as the walk fills: the trade count, the current position value, the target and the unrealised figure are written to the held record on every tick and every fill, in the fields the live bot writes, and the tables draw them. Scrums form fold tranches on the record in the live shape, folds consume them and grow the target by the surplus at the configured rate, and the Bot Settings window's Fold Tranches and Stack Tranches tabs draw the walk's tranches when opened mid-run. Every tick reads the higher-timeframe bias from the walk's own candles rolled up to the bot's phantom timeframes, through the registry's one rollup and the live coordinator's weighing, so no gate reads a hard-coded `None`. The operator's words, 2026-09-18: *"Loaded bots are showing running despite there not being a replay active. Should 'turn on' and begin updating trade counts, current position value, and target balance. Once running, bot metrics must update as trade actions are simulated ... It is imperative for simulated bots to be acting just like their live equivalent. Scrums and Folds form Tranches. Surplus profits compound at the configured rate."* And: *"Any TF higher than 5m was supposed to be getting calculated using the 5m data."*

Diagram: flowchart, left to right

- back_test.walk -> **the host's sim bus** *(BotStatsSnapshot on bot.stats)*
- the host's sim bus -> **_on_bus_stats** *(worker thread)*

- _on_bus_stats -> **_take_bot_stats on the GUI thread** *(bot_stats signal, queued)*
- _take_bot_stats on the GUI thread -> **FleetSource.write_stats: stats, scrumming_state**
- _take_bot_stats on the GUI thread -> **SimBotManager.start / stop** *(running / stopped)*
- _take_bot_stats on the GUI thread -> **refresh_bots: the rows** *(every 2 s)*
- FleetSource.write_stats: stats, scrumming_state -> **Detail: SimBotView over the record, the tranche tabs**
- back_test.walk -> **tape_context: the HTF fields** *(phantom_tapes, _rollup, weigh_higher_tf_bias)*
- back_test.walk -> **position.target_usd, target_path** *(fold surplus, cycle cap)*

A loaded bot reads IDLE

Import Live Fleet copies each stored record and writes its state `idle`, whatever the live file saved, and the sim fleet file's records read `idle` at construction. That is the live restore's own rule: `restore_bots_from_state` recreates every persisted bot in IDLE and it stays there until the operator starts it. Before this an imported bot read the live file's `running` with no run in flight. Generate From YTD and Create already wrote `idle`.

`src/simulator/fleet_source.py` — the one write

```
def loaded_idle(record: dict) -> dict:
    """``record`` with its ``state_when_saved`` written ``BotState.IDLE``, as
    ``restore_bots_from_state`` recreates every persisted bot in IDLE until
    the operator starts it; a loaded bot never reads ``running`` with no run."""
    record["state_when_saved"] = BotState.IDLE.value
    return record
```

The row moves with the walk

The walk emits a `BotStatsSnapshot` on the run's bus under `bot.stats` at its opening, on every tick, on every fill and at its end. Each host subscribes to that topic on its own bus at build; the subscriber runs on the walk's thread and re-emits the snapshot through the `bot_stats` signal, so the record is written on the GUI thread through `FleetSource.write_stats`. A tick writes what the live tick writes: the current price, the position value and the unrealised figure, with the lots the holdings are summed from. A fill writes what the live fill writes: the trade count, the buys and sells, the scrummed and folded sums, the trade volume, and the scrumming state the record holds, the grown target, the fold tranches, the lots, the cycle's consumed cap, the standing surplus and the growth side, the last trade's price and side, and the lifetime tranche counters. The Scrumming Bots table reads the position value off the holdings and the price, Trades off the trade count and Target off the grown target, all three from the record.

The opening snapshot carries the `running` mark and the closing one `stopped`, so a row reads `RUNNING` while its own walk runs and `STOPPED` when it ends; a fleet of many bots walked one at a time reads one green row at a time. The Start Run press moves no Back Test row itself; under Validation, which reruns recorded rows in one pass and walks nothing, the press still moves every run bot to `running` as before. The rows redraw at once on a mark and every two seconds otherwise, the live dashboard's own interval, through `refresh_bots` alone rather than `fleet_changed`, which also writes the fleet file and re-seats the venues. A 5m walk ticks about two hundred times a second, so the record is always a few ticks ahead of the cells between two redraws, and equal to them at the end.

`src/simulator/back_test.py` — the snapshot

```
@dataclass(frozen=True)
class BotStatsSnapshot:
    """What one walk wrote at one moment, in the two shapes the held record
    holds: ``stats`` in ``BotStats``'s keys and ``scrumming_state`` in the
    keys ``SimBotView`` reads; ``state`` is the mark the walk's start and end
    carry, empty on a tick or a fill."""

    bot_id: str
    symbol: str
    timeframe: str
    ts_ms: int
    tick: int
    event: str
    state: str
    stats: dict
    scrumming_state: dict
```

`src/gui/simulator/sim_trading_tab.py` — the write on the GUI thread

```
        bot_id = str(getattr(snapshot, "bot_id", ""))
        try:
            self._fleet_source.write_stats(
                bot_id, snapshot.stats, snapshot.scrumming_state
            )
            if snapshot.state == back_test.RUNNING_STATE:
                self._bot_manager.start(bot_id)
                self._status_log.log(f"✓ Bot {bot_id} RUNNING.", "success")
```

The live bot's `stats.realised_pnl` is written by nothing on its scrum and fold path, so the walk writes it no more than the live bot does; the realised surplus of a completed cycle reaches the row where the live bot books it, in the target's growth, the folded sum and the standing surplus.

True today: the live bot's `stats.realised_pnl` is written at its exchange walk, from the exchange's own fill history. The snapshot this walk sends still carries no realised figure.

```
# src/trading/scrumming/reconciliation.py, in refresh_exchange_position_health
                self.stats.realized_pnl_exchange = self.stats.realised_pnl = float(
                    _ph.realized_pnl_usd
                )

# src/simulator/back_test.py, in stats_snapshot
        "unrealised_pnl": position.unrealised_pnl_usd(price),
```

The Product Manual PDF rebuild is owed for this page.

The sentence above stands as written. This is the reading that replaced it.

written above	what it reads now
"The Product Manual PDF rebuild is owed for this page."	`docs/Acervator-Product-Manual.pdf` is rebuilt with this page

Scrums and folds form tranches on the record

The position keeps its lots as the live bot keeps `_main_lots`: the opening entry is one lot at the opening price. A scrum sorts the lots by their original cost, highest first, sells from them in that order and queues one fold tranche per lot sold from, each carrying that lot's original cost as `initial_buy_price`, the sell price as `ref`, the proceeds' share as `usd` and the units taken. A fold rebuys the eligible tranches under unit 26's plan and settle, books the bought units as lots per consumed tranche's share, and closes the tranches it drained. The snapshot carries the tranches and the lots, so the Fold Tranches tab opened mid-run draws the walk's tranches, as the live window draws a bot's at open; it does not refresh while open, and the live window does not either. A tranche's age on the tab is measured from the wall clock, so a walk's tranche created on the tape's April candle reads months old in September. The live stack is Stack Mode's ladder of resting venue orders opened off a scrum; the walk places no order, so the Stack Tranches tab reads `No stack tranches yet.` for a walked bot. Merge, despawn and clear are not walked.

`src/simulator/back_test.py` — one tranche per lot

```
    position.main_lots.sort(
        key=lambda lot: float(lot.get("initial_buy_price", 0) or 0), reverse=True
    )
    remaining = units
    for lot in list(position.main_lots):
        if remaining <= 1e-12:
            break
        take = min(float(lot.get("units", 0) or 0), remaining)
        if take <= 1e-12:
            continue
        position.fold_tranches.append(
            {
                "usd": (take / units) * proceeds,
                "units": take,
                "ref": float(price),
                "initial_buy_price": float(lot.get("initial_buy_price", 0) or 0),
                "created_ts": float(ts_ms) / 1000.0,
            }
        )
```

The surplus compounds into the target

Each fold's surplus is the units bought less the units its consumed tranches sold at their sell price, priced at the fill, the live fold's own figure. The cycle cap is the growth rate of the cycle-open target, and the fold grows the target by the surplus plus any standing surplus, held to what the cap leaves; the rest stays standing. The growth side is set to `lower`, and the cycle resets when the tick's Bollinger position reaches the opposite extreme, 0.75 after a lower-side growth, so the consumed cap zeroes and the next fold may grow the full cap again. The gates read the grown target for the Target Delta, as the live tick reads its own. Three pure functions carry the arithmetic beside unit 26's, and the live bot keeps its inline copy.

`src/trading/scrumming/sizing.py` — the growth and the cycle

```
def target_growth_applied(
    surplus_usd: float, standing_usd: float, cap_remaining_usd: float
) -> tuple[float, float]:
    """What ``_apply_fold_target_growth`` adds to the target and what it parks:
    ``(applied, standing_after)`` where the applied growth is the surplus plus
    the standing pool held to ``cap_remaining_usd``, and the rest stays
    standing; nothing is applied when the cap is consumed."""
    if cap_remaining_usd <= 1e-9:
        return 0.0, max(0.0, float(standing_usd) + max(0.0, float(surplus_usd)))
    available = max(0.0, float(surplus_usd)) + max(0.0, float(standing_usd))
    applied = min(available, float(cap_remaining_usd))
    return applied, max(0.0, available - applied)
```

A fold's growth travels the wires to another sim bot

A simulated bot can send part of a fold's growth to another simulated bot. A wire names a source, a target and a percentage. The moment a fold compounds its growth into the source's target balance, the wire's share of that same figure is credited to the other bot. The source keeps its own growth whole and the share is new money at the destination, which is how the live fold route behaves.

The share lands in the target's standing fold tranches, split evenly, the way wire credits land on a live bot. A sim bot holds no position between walks, so a share arriving while the target is not walking is parked on its ledger and spread over the first tranche its own walk builds. The ledger carries the lifetime sent, the lifetime received, the provenance per funder and the parked pool.

`SimWireManager` in `src/simulator/sim_wire.py` owns the topology and one ledger per sim bot.

verb	what it does
`register_wire`	sets the source-to-target percentage, from 0 up to 100
`unregister_wire`	drops one route
`attach_bot`	holds the position of a walk in progress a share can reach
`release_bot`	gives that position back when the walk ends
`distribute_fold_profit`	moves each wire's share of one fold's growth
`land_parked_credits`	spreads a parked pool over standing tranches
`export_wires`, `import_wires`	the routes the sim fleet file carries
`export_ledgers`, `import_ledgers`	the ledgers the sim fleet file carries

`src/simulator/back_test.py` — the fold hands its growth on

```
    growth = grow_target(bot, position, units, slices, float(price), int(ts_ms))
    if wires is not None and growth > 0.0:
        route_fold_growth(wires, bot.bot_id, growth, float(price))
```

One fold driven over an empty scratch home, two bots at a $200.0000 target, a scrum at $110.00 and a fold at $95.00, with one wire at 40 per cent between them:

```
the fold's growth on the source       7.4550
the wire's rate                         40.0 per cent
the share that leaves                    2.9820
the target's fold queue before          54.6700
the target's fold queue after           57.6520
the source's lifetime sent               2.9820
the target's lifetime received           2.9820
```

The same pair walked over a whole tape sends four times. The source's own walk is unchanged by the wire, 5 scrums and 4 folds either way, and the target ends holding more of its asset because the wired money bought more on its folds.

```
                        no wire       a wire at 40 per cent
source scrums, folds    5, 4          5, 4
source end target       295.9112      295.9112
target end units        3.2066        3.2334
target lifetime received  0.0000       38.3645
```

Start Run carries the fleet's wires, and the run's end saves them

Start Run in Back Test hands the run the mode's own `SimWireManager`, built from the sim fleet file by `FleetSource.wire_manager`. A fold then routes its share while the tape walks, which is what makes a pushed topology's wiring mean anything. Validation is handed no manager, because a wire belongs to the fleet of the mode it was pushed into.

`src/gui/simulator/sim_trading_tab.py` — Start Run builds the manager

```
            wires = (
                self._fleet_source.wire_manager()
                if mode == surface.MODE_BACK_TEST
                else None
            )
```

When the run ends, `_hold_run_wires` takes the manager back onto the fleet through `FleetSource.hold_wire_manager` and saves the file, so what a wire carried during a run is on disk before the tab is closed. A run that raises saves the same way, so a partial pass keeps what it booked. Both builds do this: the React host's `_start_run`, `_compute_run` and `_take_run` carry the manager the same way.

One pair driven over a synthetic tape, two bots at a $200.0000 target and one wire at 40 per cent between them, 15 scrums and 128 folds each:

```
                          no wire     a wire at 40 per cent
source scrums, folds       15, 128     15, 128
source end target         204.9836    204.9836
source end units            1.7574      1.7574
target fold queue         102.9692    104.8879
source lifetime sent        0.0000      1.9187
target lifetime received    0.0000      1.9187
transfers booked                 0           49
```

The source's own walk is identical either way. The target's fold queue carries the wired dollars: `104.8879 - 102.9692 = 1.9187`, which is the figure both ledgers book.

The whole of the source's compounded growth passes the wire. It grew `204.9836 - 200.0000 = 4.9836` over the run and 40 per cent of that is `1.9934`; `1.9187` of it cleared `MIN_WIRE_USD` and the remaining `0.0747`, over 79 folds, was dust the floor refused. One fold's own arithmetic:

```
the fold's growth on the source       0.1467
the wire's rate                         40.0 per cent
the per-wire safety cap                100.0 per cent
the share that leaves                   0.0587
```

A sim bot walks its whole tape before the next bot starts, so a share leaving the source is parked on the target's ledger and lands on the first tranches the target's own walk builds. In this pair all 49 shares parked, and the target's ledger ends with `parked_usd` at zero and 49 wire-credit rows standing on its 15 tranches.

The higher timeframes come from the walk's own candles

For each phantom timeframe the record names, or the one above the walk's timeframe when it names none, the walk rolls its own candles up through the registry's `_rollup`: the completed buckets once, and the bucket holding the current candle from its rows up to that candle, so no later candle reaches a window. The window is the last hundred rolled candles, the live phantom's own fetch, and under thirty of them the phantom reads no summary, as the live phantom sets none. The voting engine votes on the window, and the summaries are weighed through `weigh_higher_tf_bias`, the loop lifted out of the live coordinator's `get_higher_tf_bias`, which now calls it: each higher summary at or above the confidence floor contributes its rank times its confidence to its side, both sides empty read `NEUTRAL`, the heavier side wins. The bias name and the two block flags then reach the gate context as the live tick sets them, a scrum deferred on `BULLISH` and a fold on `BEARISH` under the two defer flags. A record with phantoms off reads no bias, as the live bot builds no phantom for it. A timeframe the walk's candles cannot roll up to, one at or under the walk's own or outside the registry's table (`1w` among them), is refused by name on the report and reads no summary.

`src/trading/phantom_balance.py` — the one weighing

```
        return weigh_higher_tf_bias(
            [(p.timeframe, p.rank, p.last_summary) for p in higher],
            below_parent,
            min_confidence,
        )
```

`src/simulator/back_test.py` — the window without look-ahead

```
    def window(self, index: int) -> list[list[float]]:
        """The last ``PHANTOM_WINDOW_CANDLES`` rolled candles up to and
        including the walk's candle at ``index``."""
        key = self.keys[index]
        partial = _rollup(
            self.rows[self.starts[key] : index + 1], self.factor, self.bucket_ms
        )
```

The report gains the target's path and the bias per fill

The Back Test report's bot rows carry the target at the walk's end, the snapshot's stats block, the target path and the HTF reading. A `Compounding` section lists each bot's target at open and at end, the growth applied, the steps, the standing surplus and the stats figures, then each bot's target path step by step: the candle, the surplus, the cycle cap, the consumed figure, the applied growth and the target before and after. A `Higher timeframes` section lists each bot's named, evaluated and refused timeframes with the bias counts over the ticks, then every fill with the bias its gates read, the target after it and the growth it applied. The sidecar holds the same dicts under `trades`, `compounding` and `htf`.

The stats emit and the bias emit

Every fill and every walk's end emit `sim.bot.stats_written` through `signal_contract`: the bot, the tick, the fills and the fields moved, against the fill count off the trades, `ok` when the record's trade count equals it. Every tick emits `sim.bot.htf_bias`: the bot, the candle, the bias, the two block flags and each phantom's reading or refusal, against the timeframes the bot names, `ok` when each was evaluated or refused by name; a fill's row lands unthrottled and the rest once per five seconds per bot.

What the live-equivalent reading measured

Read off the running program in both builds, scratch home, every socket but loopback refused, a `bot_state.json` of three scrumming bots whose saved state read `running`, each with one live fold tranche, the first with a 15m phantom, the second naming `1w`, the third with phantoms off on a rising tape, and one 5m tablet of 1,500 candles per pair. On the base commit Import Live Fleet read three `RUNNING` rows with no run; during the Back Test the first row read `$720.0000`, `7`, `$805.0000` from before the press to the end while the spool carried 22 of its fills; the Fold Tranches tab drew the imported tranche; the report carried no target path and no bias; no emit row landed. After: Import Live Fleet read three `IDLE` rows in both builds. Start Run: the first row `RUNNING` and the other two `IDLE`, its record moving at a tenth of a second, 15 trades and a position of $721.84 against a target of $807.49, then 19 trades, $758.15 and $809.46 with one tranche queued, the cells holding the opening `$800.0000`, `0`, `$800.0000` until the next redraw; Detail pressed while it ran drew one open tranche, `7.622694` units, `$99.3034` parked, sell ref `$13.07965900`, original cost `$11.36281900`, and the Stack Tranches tab `No stack tranches yet.`, in both builds; then the second row `RUNNING` alone, then the third. At the end every row `STOPPED` and equal to the report: the first 23 trades, position $783.6261, target $809.7497, unrealised $198.24, scrummed $395.90, folded $114.64, four tranches queued; the second 11 trades, target $704.15; the third 7 scrums, no fold, seven tranches queued, target $600.00. The first bot's 17 growth steps recomputed by hand from the first: surplus $0.3030 against a cycle cap of $8.00 with nothing consumed, applied $0.3030, target $800.00 to $800.3030. The first bot's 15m bias over 211 ticks: 9 `None` before thirty rolled candles, 201 `NEUTRAL`, 1 `BEARISH`; its first fill's `NEUTRAL` recomputed by hand from 48 rolled candles, the 15m summary `BEARISH` at 0.2049 under the 0.30 floor. The second bot's `1w` refused as `cannot roll up from the walk's timeframe`, the third's `15m` as `phantoms off`, both reading no bias. Forty-four `sim.bot.stats_written` and forty-four `sim.bot.htf_bias` rows per build, every one `ok`. The scratch `bot_state.json`, the three tablets and the manifest hashed identical after every press, a planted byte moving a copy's hash; no socket left loopback; no bot was constructed. The Back Test still steps its tape at two hundred evaluations a bot, so a three-bot run over 1,500 candles lasts about three seconds; the row's movement is read off the record between two redraws and off the cells at the marks and at the end.

Back Test walks the full Stone Tablet

Back Test should be able to 'Load Live Fleet' but runs full Stone Tablet for a given target asset or all tablets corresponding to the loaded / configured fleet.

Back Test walks every candle of every year file the tablet store holds for each bot's asset on the bot's venue, in time order, at the bot's own timeframe, with no stepping. Before this landed a bot read one year file, the oldest, and evaluated about two hundred of its bars. Now a bot with two year files of 3,000 candles evaluates 5,971 bars: 6,000 less the 29 the first window needs. A bot whose timeframe is not the store's 5m reads the same files rolled up through the registry's one rollup, so a 1h bot over the same two files evaluates 471 of 500 bars. A bot whose asset has no file, or whose timeframe the registry cannot roll 5m up to, is named on the Activity Log, in the started line and in the report, and walks nothing.

`src/simulator/back_test.py` — the files and the tape one bot walks

```
def native_entries(entries: Sequence[Any], asset: str, exchange_id: str) -> list:
    """Every entry in ``entries`` for ``asset`` on ``exchange_id`` at
    ``NATIVE_TIMEFRAME``, by year."""


def bot_tape(registry: StoneTabletsRegistry, files: Sequence[Any], bot: SimBot) -> list:
    """Every bar across ``files`` at ``bot_timeframe`` for ``bot``'s asset on
    its venue, in time order, through ``registry.get_candles``; no file or a
    timeframe outside ``SUPPORTED_TIMEFRAMES`` answers none."""
    timeframe = bot_timeframe(bot)
    if not files or timeframe not in SUPPORTED_TIMEFRAMES:
        return []
    since = min(int(one.first_ts_ms) for one in files)
    until = max(int(one.last_ts_ms) for one in files)
    return registry.get_candles(
        bot.asset, since, until, timeframe=timeframe, exchange_id=bot.exchange_id
    )
```

The runner reads the tape through a registry built on the tablet source's root for the run's length, the way the Portfolio Battery reads a crypto bot, and drops it when the run ends. The registry writes nothing. The stepping constant, the shared step and the two runner arguments that carried them are gone from the runner and from both hosts; the walk is called with its default step of one.

`src/simulator/back_test.py` — the runner, per bot

```
        files = native_entries(entries, bot.asset, bot.exchange_id)
        raw = bot_tape(registry, files, bot)
        if not raw:
            outcomes[NO_TABLET] += 1
```

The started line states the cost

Start Run under Back Test writes one line before the walk begins. The line names the exchange, the bot count and the funding as before, then how many bots walk, the candles and files they read, the evaluations they make and about how many minutes that takes at the stated rate, and then the bots with no tablet at their timeframe, by name. The counts come from the MANIFEST rows alone, so the line lands at the press and reads no candle body.

`src/gui/simulator/sim_trading_tab_surface.py` — the sentence the line adds

```
BACK_TEST_COST_FORMAT = (
    " {walking} bot(s) walk {candles:,} candle(s) in {files} file(s), "
    "{evaluations:,} evaluation(s), about {minutes:.1f} min at "
    "{per_thousand:.2f} s per 1,000{absent}."
)
BACK_TEST_ABSENT_FORMAT = "; {count} bot(s) with no tablet at its timeframe: {names}"
```

The rate the line uses is one constant, read off the walked lines on this machine with the sim bus live and phantoms on, and the report states the rate each walk measured beside it.

`src/simulator/back_test.py` — the rate the started line assumes

```
WALK_SECONDS_PER_THOUSAND = 5.4
```

Both hosts build the cost in the same place, inside the start of the run, and hand it to the started line.

`src/gui/simulator/sim_trading_tab.py` — the cost at the press

```
        else:
            cost = back_test.fleet_cost(bots, self._tablet_source)
        self._status_log.log(
            tab_surface.run_started_line(
                mode, exchange_id, len(bots), back_test.run_budget_usd(bots), cost
            ),
            "success",
        )
```

Each bot's walk reads as moving on the Activity Log

Each bot's walk writes one line as it starts, naming the files, the bars and the evaluations to make; one line every 5,000 bars, naming the bar reached, the evaluations so far, the scrum sells and fold buys so far and the seconds; and one line as it ends, naming the evaluations, the bars, the files, the seconds and the rate. The lines cross to the Activity Log through the same signal the run's other lines use, on both hosts.

`src/simulator/back_test.py` — the three lines

```
WALK_STARTED_LINE_FORMAT = (
    "Walking {bot_id} ({symbol}) at {timeframe} on {exchange_id}: {files} "
    "file(s), {bars:,} bar(s), {evaluations:,} evaluation(s) to make."
)
WALK_PROGRESS_LINE_FORMAT = (
    "{bot_id}: bar {bar:,} of {bars:,}, {evaluations:,} evaluation(s), "
    "{scrums} scrum sell(s), {folds} fold buy(s), {seconds:.1f} s."
)
WALK_ENDED_LINE_FORMAT = (
    "{bot_id} walked: {evaluations:,} evaluation(s) over {bars:,} bar(s) in "
    "{files} file(s), {seconds:.1f} s ({per_thousand:.2f} s per 1,000); "
    "{scrums} scrum sell(s), {folds} fold buy(s)."
)
```

The progress cadence, the evaluations a walk makes and the rate arithmetic are one definition each, in the Back Test module, and the Portfolio Battery imports them from there.

`src/simulator/portfolio_battery.py` — the four names imported

```
from .back_test import (
    FOLD,
    FUNDED_BY_TARGETS,
    MIN_CANDLES,
    PROGRESS_EVERY_BARS,
    RATE_LINE_FORMAT,
```

Stop ends a Back Test walk at the bar reached

Stop on a row of the run in flight ends the walk where it is. The walk in progress writes a stopped line naming the bar reached, its candle stamp and the evaluations made; the partial report carries that line on the bot's row, the evaluations made against the evaluations expected, and the bots not reached by name.

`src/simulator/back_test.py` — the stopped line

```
    @property
    def stopped_at(self) -> str:
        """``WALK_STOPPED_LINE_FORMAT`` when ``stopped``, else empty."""
        if not self.stopped:
            return ""
        return WALK_STOPPED_LINE_FORMAT.format(
            bot_id=self.bot_id,
            bar=self.candles_read,
            bars=self.bars,
            candle_at=iso_stamp(self.last_ts_ms) or "none",
            evaluations=self.ticks,
        )
```

The report counts the walk per bot

The report's Bots table gains the timeframe, the files, the bars, the evaluations expected beside the evaluations made, the seconds, the rate and the stopped line; the tablet column names every file walked. The Run section carries the files walked, the evaluations, the evaluations expected, the seconds and the rate over the whole fleet, and the run's lines on the Activity Log carry the same totals in one rate line.

`src/simulator/parity_report.py` — the columns added

```
        ("tablet_key", "tablet"),
        ("timeframe", "timeframe"),
        ("files", "files"),
        ("bars", "bars"),
        ("ticks", "ticks"),
        ("evaluations_expected", "expected"),
        ("walk_seconds", "seconds"),
        ("seconds_per_thousand", "s per 1,000"),
```

One row per walked bot in the sim sink

Each walked bot emits one row through the signal contract when its walk ends: the files, the bars, the evaluations, the fills, the seconds and whether Stop cut it short, against the evaluations expected. The row is `ok` only when the walk made every evaluation and was not stopped. The run's worker thread is routed to the Simulator's own signal sink, so the row lands there, and a long walk rotates that sink, so the row of an earlier bot sits in a rotated file beside the live one.

`src/simulator/back_test.py` — the emit

```
    pin_emit(
        BOT_WALKED_SIGNAL,
        actual={
            "bot": bot.bot_id,
            "files": list(keys),
            "bars": result.bars,
            "evaluations": result.ticks,
            "fills": len(result.trades),
            "seconds": round(seconds, 3),
            "stopped": bool(result.stopped),
        },
        expected={"evaluations": expected, "stopped": False},
        ok=result.ticks == expected and not result.stopped,
```

The path from the press to the report

Diagram: flowchart, top to bottom

- Start Run under Back Test -> **_start_run: fleet_cost over the tab's tablets, the started line with candles, files, evaluations, minutes**
- _start_run: fleet_cost over the tab's tablets, the started line with candles, files, evaluations, minutes -> **_compute_run on the worker thread, routed to the sim sink**
- _compute_run on the worker thread, routed to the sim sink -> **_walk_fleet: a registry on the tablet root**

- _walk_fleet: a registry on the tablet root -> **per bot: native_entries, bot_tape at the bot's timeframe**
- per bot: native_entries, bot_tape at the bot's timeframe -> **_walk_bot: the walking line, walk over every bar, a line every 5,000 bars, the walked or stopped line, one bot_walked row**
- _walk_bot: the walking line, walk over every bar, a line every 5,000 bars, the walked or stopped line, one bot_walked row -> **write_report: files, bars, evaluations, expected, seconds, rate per bot**
- write_report: files, bars, evaluations, expected, seconds, rate per bot -> **run_finished: the run's lines and the report line on the Activity Log**

What was measured

Two scrumming bots on coinbase at 5m, each asset with two year files of 3,000 candles, the first ending 2025-12-31T23:55Z and the second starting 2026-01-01T00:00Z, on a scratch home with every socket but loopback refused, read off the running program in both builds. Before: each bot read 3,000 candles of one file and made 199 evaluations; the started line named no candle count and no time; no progress line landed. After: the started line read `2 bot(s) walk 12,000 candle(s) in 4 file(s), 11,942 evaluation(s), about 1.1 min at 5.40 s per 1,000`; each bot's walking line read 2 files, 6,000 bars and 5,971 evaluations to make; one progress line landed at bar 5,000 of 6,000; each walked line read 5,971 evaluations over 6,000 bars in 2 files, 32.8 and 31.5 seconds in Qt, 32.7 and 32.2 in React, at 5.49, 5.27, 5.48 and 5.39 seconds per 1,000; the report's rows read files 2, bars 6,000, ticks 5,971, expected 5,971, and 314 and 237 fills, equal to the spool's FILLED lines per bot; two rows in the sim sink, both `ok`. A bot whose asset had no file read `no Stone Tablet for CCC on coinbase at 5m; nothing walked.` on the Activity Log, `no_tablet` in the report and its name on the started line's absent clause. A bot at 1h over the same two files read 500 bars and 471 evaluations, walked in 2.3 seconds. Stop pressed nine seconds into a walk read `stopped at bar 1,724 of 6,000 (2025-12-27T13:35:00Z): 1,695 evaluation(s) made.` in Qt and bar 1,985 in React, the report `partial`, the second bot not reached, the sink row not `ok`. The walk wrapped to step ten bars read 598 evaluations against 5,971 expected, so the evaluation reading can fail. The scratch fleet file, the four tablets and the manifest hashed identical after every press, a planted byte moving a copy's hash; no socket left loopback; no bot was constructed.

Three passages above describe what this section changes and are kept as written: the "One clock" passage under Back Test Mode, which quotes the shared step that is gone; the sentence under the simulated-bot section that the Back Test still steps its tape at two hundred evaluations a bot; and the three code-block headers that name the Battery module as the home of the evaluations-expected and progress-cadence definitions, which now live in the Back Test module and reach the Battery by import.

2026-09-19 17:40 - #55 - the glyph definition moves to the chart's surface

His words on the Charts tab, 2026-09-08: *"Want charts tab to only display one richly detailed and Acervator-annonated chart at a time"*. The Charts painter now draws every fill as the two hermetic glyphs the playback draws, and the one definition both read lives in the chart's surface. The Simulator's surface imports it from there; the playback view, the page and the payload read the same names they read before, so nothing else on this tab changed.

`src/gui/main_tabs/simulator_tab_surface.py` - the import and the key check

```
from .native_chart_surface import (
    MARK_GLYPHS,
    MARK_HEIGHT_FRACTION,
    MARK_OUTLINE_PX,
    MARK_WIDTH_RATIO,
)
```

```
#: ``MARK_GLYPHS`` is keyed by ``SCRUM_SIDE`` and ``FOLD_SIDE``, the strings
#: ``back_test.SCRUM`` and ``back_test.FOLD`` carry.
if MARK_GLYPHS.keys() != {back_test.SCRUM, back_test.FOLD}:
    raise ImportError("MARK_GLYPHS keys differ from back_test.SCRUM and FOLD")
```

Read off the running program in both builds: the object the Charts painter draws its triangles from is the object the Simulator's surface holds, and a scrum on the Charts tab has the corners the playback gives a scrum.

One sentence this entry overtakes

Not reworded, quoted here. `docs/manual/08-tabs/simulator.md, under 'The two glyphs'` - "One definition, `MARK_GLYPHS`, names both marks, and both hosts read it: the Qt view in Python, the page off the playback payload." Three readers now, the Charts painter the third, and the definition sits in `src/gui/main_tabs/native_chart_surface.py`.

2026-09-24 - #858 - the bot list upgrades reach the Simulator

His line closes the bot-list list on that item: "Transpose these upgrades to Sim and Paper once completed." Three upgrades are in it. Columns that sort. Labels that wrap, with the privacy button centred beneath each one. A row press that moves the Indicator Voting Panel to that bot. All three now draw on the Simulator's own Scrumming Bots table, in both builds, over the Simulator's own figures.

Every column sorts, over the Simulator's own price

A press on a column heading orders the rows by that column. A second press on the same heading reverses the order. The order is recomputed on every rewrite, so a figure that keeps moving does not unsort the list. The Simulator orders by the row's own reading and never by the live price pool, because the table and the page model each answer their own lookups.

`src/gui/simulator/sim_bot_status_table.py` - the Simulator's own sort readings

```
        def _sort_lookups(self) -> SortLookups:
            """``order_statuses``' two readings, over the Simulator's own figures."""

            def price(status) -> tuple:
                stats = status.get("stats", {}) or {}
                return self._display_price(status, stats)

            def denom_text(
                quote: str, base: str, exchange_id: str, target: float
            ) -> str:
                return self._denom_cell(quote, base, exchange_id, target)[0]

            return SortLookups(price, denom_text)
```

Read off the running program, both builds, at 700, 900 and 1400 pixels wide, over a fleet of 38 rows:

reading	window	page
columns that move on a press	9 of 10	9 of 10
a second press reverses	7 of 9 exactly	yes
the order survives a fresh fleet	yes	yes

The Detail column holds one identical button a row, so it carries no value to order by and is the one column a press does not sort. Target A15 and Target A14 do not reverse to an exact mirror, because a row that draws nothing in those two columns stays beneath the figures whichever way the sort runs.

The labels wrap and the privacy dot sits beneath each one

Each of the ten labels sits inside its own column, over as many lines as it needs, with one blue dot centred under the nine that carry a mask. The label steps down one size when its longest word will not fit the column. A press on the dot masks that column; a press on the words sorts it.

width	Current Position Value, window	the page	dots	largest offset
700	3 lines in 67 px, size 10	46.2 px over a 15.4 px line	9	0.5 px
900	3 lines in 92 px, size 11	30.8 px over a 15.4 px line	9	0.5 px
1400	3 lines in 154 px, size 11	15.4 px over a 15.4 px line	9	0.008 px

No label is cut at any of the three widths, in either build. The tenth column draws no dot and keeps the same empty dot row, so every label sits on one line.

A row press moves the Simulator's own Voting Panel

A press on a row draws that bot on the Simulator's panel. A second press on the same row takes the panel to its empty state. A press on another row moves the panel to that bot. The panel's own dropdown still picks a bot, and the row it names highlights itself in the list. The panel is the Simulator's own, never Live's.

`src/gui/simulator/sim_trading_tab.py` - the one bot both sides read

```
        self._bot_list_link = BotListPanelLink(
            self._indicator_panel, self._bot_list_hosts
        )
        self._indicator_panel.bot_selected.connect(self._bot_list_link.panel_moved)
```

Read on the running program, both builds, at all three widths: a first press puts the pressed row's bot on the panel, a second press on that row answers no bot, and a press on another row answers that row's bot.

The table is Live's table with three readings of its own

The Simulator's window table was a copy of Live's, taken before the header was rebuilt. It is now Live's table with the Simulator's three readings over it: the display price, the Target-denomination cell and the sort lookups. The page module carries the same relationship to Live's module, and the model already read the shared surface.

`src/gui/simulator/sim_bot_status_table.py` - the row's own price

```
        def _display_price(self, status: dict, stats: dict) -> tuple:
            """The row's own ``current_price``, at ``SIM_PRICE_AGE_S``."""
            del status
            return float(stats.get("current_price", 0.0) or 0.0), SIM_PRICE_AGE_S
```

An empty Simulator fleet still draws the header

The Simulator's own fleet file holds no bot on this machine, which is the normal case. With no row at all the header still draws its ten labels and its nine dots, a press on a heading changes nothing and raises nothing, and a press where a row would be finds no row.

reading, no rows	window	page
labels drawn	10	10
dots drawn	9	9
a heading press	no error, still no row	no error, still no row
a row press	answers no bot	finds no row

2026-09-25 - a back test's orders are formatted to the venue

A Back Test now sizes every order to the rules the execution venue publishes for that market. The rules come from a recording the live connector writes when it reads its markets, so a run that reaches no venue still knows what the venue would accept. `src/exchange/market_rules_store.py` holds the recording under the runtime home.

The bar source and the execution venue are two different things. A tablet names both, and the rules are read against the venue that would execute the order.

```
# src/simulator/back_test.py, in venue_rules_for
def venue_rules_for(asset: str, exchange_id: str, symbol: str) -> MarketRules:
```

Two cells the Back Test report gains

Each fill row carries a **sized by** cell, reading `recorded venue rules` when the market's own record answered and `cited unit rule` when nothing was recorded. Each bot row carries an **orders refused** count, and the Activity Log carries one line per bot naming the reason behind that count.

```
u881l1-fractional: A34/USD sizes on coinbase's recorded rules, minimum 0.1 on a
size step of 0.01.
u881l1-fractional: A34/USD refused 710 order(s): 710 below the venue's minimum
size.
```

A refusal costs no tranche

A fold refused under the venue's minimum settles no plan and spends no cash, so its tranches stay queued. They accumulate until a later fold buys a legal size. On the reading above, 19 of 730 latched folds filled and the rest waited.

A stored figure no box can hold draws a dash

The Simulator's Bot Settings tab and its creation wizard read every stored dollar figure through one admission before the control sees it. `src/trading/container/config.py, in as_box_figure` is that admission. It refuses six kinds: a stored flag, a text number, a not-a-number, an endless number, an integer too large for a decimal, and a finite figure outside the control's own span.

A refused figure leaves the box at the bottom of its span drawing the dash the indicator panel draws for a cell it cannot read. The log names the field. The dash clears the first time the operator moves the box. Never a zero.

`src/gui/simulator/sim_settings_tab.py, in SimSettingsTabMixin` takes `_show_admitted_figure` and `_clear_refused_dash` from Live's own `src/gui/live_settings/settings_tab.py, in SettingsTabMixin`, so Target Balance and Hedge Balance draw what the Live boxes draw. Three of the six kinds used to stop the whole tab from building; all six now build it.

`src/gui/simulator/sim_settings_tab.py, in _refresh_target_denom_rows` reads the same admission, so a target the box draws as a dash reaches neither the Target-BTC row nor the Target-ETH row as money.

`src/gui/simulator/sim_bot_wizard.py, in SimTradingParamsPage` calls `src/gui/bot_wizard.py, in _show_stored_target` for the stored default target, and its `isComplete` answers False while that dash is drawn. Finish stays dark until the operator enters a figure, so no sim bot is built on a target nobody chose.

`src/gui/simulator/sim_bot_wizard.py, in SimAssetSelectionPage` admits each market's volatility before it compares it, so one market's unreadable swing costs that row its volatility and no other market its place in the pair list.

Apply writes nothing. `src/simulator/sim_bot_view.py, in SimBotView` declares no `set_config_field`, so every field the window sends is refused and the pending line reads the refused count.

The React build draws the same dash. `src/gui/web/sim_live_settings_tab.js, in NumberControl` and `src/gui/web/sim_bot_wizard.js, in NumberField` print the words inside the box rather than beside it, as Qt prints them, and `src/gui/web/sim_bot_wizard.js, in numberSpec` prefers the spec the payload carries under `values.number_specs` so the field sees the dash at all.

2026-10-20 - #937 - the Scrumming table's state sits on the Symbol cell

The Simulator's Scrumming Bots table is a subclass of the window's own bot list, so it draws that list's ten columns. The first column is Asset. It draws the target asset's own mark, and the asset's ticker where no mark is kept. It carries no bot id, no state colour and no state tooltip. The state colour and the mode-and-state tooltip sit on the Symbol cell.

The Extractor Bots table keeps its own Bot ID column, so every sentence on this page about the Extractor's Bot ID column stays true.

Read off the drawn table, four invented bots, the viewport grabbed at 1600 by 420:

```
column 0, "Asset"     a 32 by 32 mark, or the ticker where none is kept
column 1, "Symbol"    the pair in the state colour, the mode and state in its tooltip
row height            36 pixels
running  #00ff88      paused  #ffaa00      error  #ff3366
```

Eight sentences above are overtaken

Each is quoted whole, with the true sentence under it.

Overtaken: *"It hides until a row arrives, then draws one row per held scrumming bot with Live's ten columns, the Bot ID cell coloured by the bot's state."*

True now: it hides until a row arrives, then draws one row per held scrumming bot with Live's ten columns, the Symbol cell coloured by the bot's state.

Overtaken: *"The saved state gives the state the Bot ID cell is coloured by."*

True now: the saved state gives the state the Symbol cell is coloured by.

Overtaken: *"Its state is idle, so its Bot ID cell is grey."*

True now: its state is idle, so its Symbol cell is grey.

Overtaken: *"*`FleetSource.save` *writes the sim fleet file with the moved state, or without the deleted record, and* `refresh_bots` *hands each venue its rows again, so the Bot ID cell's colour and tooltip in the Scrumming Bots table, and the Mode cell's in the Extractor Bots table, read the new state."*

True now: the same save and the same hand-back happen, and the colour and tooltip that read the new state are the Symbol cell's in the Scrumming Bots table and the Mode cell's in the Extractor Bots table.

Overtaken: *"Start on an imported idle bot writes Live's* ✓ `Bot <id> RUNNING.` *line and colours its Bot ID cell."*

True now: Start on an imported idle bot writes that same line and colours its Symbol cell.

Overtaken: *"Start on a generated idle bot writes Live's* ✓ `Bot <id> RUNNING.` *line and colours its Bot ID cell."*

True now: Start on a generated idle bot writes that same line and colours its Symbol cell.

Overtaken: *"*`_start_run` *moves each of those bots to* `running` *through* `SimBotManager.start`*, the same rule Start applies to one bot, and fires* `fleet_changed`*, so the sim fleet file is saved and every venue page draws its rows again before the walk begins: the Bot ID cell reads* `State: RUNNING` *in its running colour."*

True now: the same move, the same save and the same redraw happen, and the cell that reads `State: RUNNING` in its running colour is the Symbol cell.

Overtaken: *"Column 2 is now Current Position Value, priced from the exchange, and the state colour it used to carry sits on the Bot ID cell."*

True now: column 2 is Current Position Value, priced from the exchange, and the state colour it used to carry sits on the Symbol cell.

Two blocks above read the first column by its old name

Neither is a sentence, so both stay exactly where they stand.

The cell table under the saved-record passage holds the row `| Bot ID, its colour | the record's id and its saved state |`. The first column reads the target asset's kept mark, and the colour that row names is the Symbol cell's.

The reading block under the imported-row walk holds `the row: Bot ID, Symbol, Trades, Target`. The first of those four is the Asset column, and the figure beneath it is the asset's mark or its ticker.

The code block listing the ten column labels names `"Bot ID"` first. `src/gui/main_tabs/bot_status_table_surface.py, in COLUMN_LABELS` names `"Asset"` first today.

Both tabs build their sector pages from one declared list

The Simulator tab and the Paper tab each hold a stack of sector pages. Both stacks come from the same declared list, read at the moment the page is asked for rather than at import.

`src/gui/main_tabs/asset_class_surface.py, in LAYERED_CLASSES` declares the six sectors a trading layer stands behind.

```
LAYERED_CLASSES = frozenset(
    {"crypto", "stocks", "commodities", "forex", "indices", "futures_perps"}
)
```

`src/gui/main_tabs/asset_class_surface.py, in layered_classes` puts those names in taxonomy order. `src/gui/main_tabs/trading_tab_surface.py, in layer_order` answers that order to every tab that draws a layer stack.

`src/gui/simulator/sim_trading_tab_surface.py, in build_view_model` reads that one call and names the page on show out of the same list.

```
key = live.layer_named(layer)
order = live.layer_order()
```

Two fields of the tab payload carry the result: the page list and the position of the page on show inside it.

```
"trading_stack": {
    "pages": list(order),
    "current_index": order.index(key),
},
```

`src/gui/paper/paper_trading_tab_surface.py, in build_view_model` writes those same two fields from the same call, so neither tab holds a sector list of its own.

What each tab answers, per sector

Each tab's own state object was asked for all six sectors, and the page list was read off the payload the tab serves.

sector asked	Sim pages	Sim page on show	Paper pages	Paper page on show
Crypto	6	0	6	0
Stock	6	1	6	1
Commodities	6	2	6	2
Forex	6	3	6	3
Indices	6	4	6	4
Futures / Perps	6	5	6	5
a name no layer carries	6	0	6	0

Both tabs answer the same list, in the same order, on every ask.

```
crypto, stocks, commodities, forex, indices, futures_perps
```

A sector name no layer carries draws page 0, which is the crypto layer. The page on show moves across all six asks, so the figure is read from the sector asked and is not a fixed answer.

The added-layer reading

A seventh sector was added to the declaration and both tabs were asked again. Both gained the page together.

the declaration holds	Sim pages	Paper pages	highest page on show
six sectors	6	6	5
seven sectors	7	7	6

The seventh name was last in both page lists and last in both card lists. The declaration was then put back, and both declaring files read byte for byte as they did before.

The sector press reaches every sector page on both tabs

Pressing a sector on the header strip now moves the Simulator tab and the Paper tab to that sector's page, in the Qt build and in the React build. Before this change both tabs stayed on the Crypto page for every press, so five of the six pages could not be opened at all.

`src/gui/main_tabs/class_filter_tab.py, in _ask_tab_for_class` is the one caller that hands a sector to these two tabs. It calls each tab's own `set_asset_class`, which now shows that sector's page as well as narrowing the rows.

```
self._asset_class = class_filter_surface.normalise(name)
self._show_class_layer(self._asset_class)
```

`src/gui/simulator/sim_trading_tab.py, in SimTradingTab._show_class_layer` and `src/gui/paper/paper_trading_tab.py, in PaperTradingTab._show_class_layer` each read the page number from the declared sector list and put the layer stack on it.

```
stack.setCurrentIndex(acs.layer_page(key))
```

The React tabs send the sector as the layer, on the ask their own page already answers. `src/gui/simulator/sim_react_trading_tab.py, in SimTradingTabReact.set_asset_class` and `src/gui/paper/paper_react_trading_tab.py, in PaperTradingTabReact.set_asset_class` each carry it into `show_tab`.

```
self.show_tab({"layer": self._asset_class})
```

No new button, panel or chooser was added. The sector buttons on the header strip are the way in, and they were already there.

The Qt stacks hold one page per sector

`src/gui/simulator/sim_trading_tab.py, in SimTradingTab._build` and `src/gui/paper/paper_trading_tab.py, in PaperTradingTab._build` each built one Crypto page and one Stock page by hand. Both now build one page per declared sector, in the loop `src/gui/main_tabs/trading_tab.py, in TradingTabMixin._build_trading_tab` already uses for the Live tab.

```
self._class_layers: dict = {}
for layered in acs.layered_classes():
    page, bar, store, card = _make_layer(
        acs.display_name(layered), acs.accent(layered)
    )
    self._class_layers[layered] = {
        "page": page,
        "tabs": bar,
        "exchange_tabs": store,
        "placeholder": card,
    }
    self._trading_stack.addWidget(page)
```

Every name the two tabs used before is kept, and each one points at the same page it pointed at before. The six names below are read off the new list instead of being built one at a time, so nothing that reads them changed.

```
_crypto_tab_widget      _stock_tab_widget
_crypto_placeholder     _stock_placeholder
_crypto_exchange_tabs   _stock_exchange_tabs
```

Add Exchange still seats a crypto venue on the Crypto page and an equity venue on the Stock page. That routing is untouched.

What the sector press shows, before and after

Each tab was built and asked for every sector through `ClassFilterTabMixin.select_asset_class`, which is the call the header strip's own press makes. The Qt figure is the page the layer stack shows, named by the heading drawn on that page. A React page's pixels cannot be read back, so the React figure is the page the tab payload names.

sector pressed	Sim before	Sim after	Paper before	Paper after	Live
Crypto	Crypto	Crypto	Crypto	Crypto	Crypto
Stock	Crypto	Stock	Crypto	Stock	Stock
Commodities	Crypto	Commodities	Crypto	Commodities	Commodities
Forex	Crypto	Forex	Crypto	Forex	Forex
Indices	Crypto	Indices	Crypto	Indices	Indices
Futures / Perps	Crypto	Futures / Perps	Crypto	Futures / Perps	Futures / Perps

The table reads the same in both builds. The Live column did not move, before or after, because the Live tab already answered this way.

The page count on the two tabs moved from 2 to 6 in the Qt build. In the React build it stayed at 6, where the six pages were already built and only the page on show was stuck.

An unknown sector draws the first page

A sector name no layer carries draws the Crypto page on both tabs, in both builds, and on the Live tab as well. Nothing fails and nothing draws blank.

sector pressed	Sim	Paper	Live
a name no layer carries	Crypto	Crypto	Crypto

The page on show moves across the six declared sectors and falls back to the first on an unknown one, so the figure is read from the sector pressed.

The declared list is the only list

One extra sector was added to the declaration and all four tabs were asked again. Every one of them gained the page, and every one of them showed it.

the declaration holds	Sim pages	Paper pages	highest page on show
six sectors	6	6	5
seven sectors	7	7	6

The declaration was then put back, and both declaring files read byte for byte as they did before.

The sentence this entry overtakes

From `docs/manual/08-tabs/paper-trader.md`, in the section titled "The Paper tab keeps its own card while no venue serves the class":

> `src/gui/main_tabs/class_filter_tab.py, in _refilter_class_tabs` *hands the sector to the tab's own* `set_asset_class`*, which narrows the rows the tab lists and nothing else.*

True today: the sector reaches `set_asset_class`, which narrows the rows and shows that sector's page. The tab still keeps its own card under every sector, served or not, and no sector replaces it, hides it or disables it.

2026-10-25 - #937 - the Simulator page draws the Asset column's mark

The Simulator draws its Scrumming Bots table twice. The window draws one, and the page inside the window's browser draws the other. Both now draw the first column the same way. Where the logo library has put a file on disk for that asset, the cell draws that image at 32 pixels and holds no text. Where none is kept, the cell draws the asset's ticker, so the row still names what the bot accumulates. Where the asset database knows that asset's own organisation, the address rides on the cell and a press opens it.

`src/gui/web/sim_bot_status_table.js` — `AssetLogo`

```
  logoProps[PART_ATTR] = LOGO_PART;
  logoProps[LOGO_PATH_ATTR] = text(found[LOGO_PATH]);
  logoProps[LOGO_SIZE_ATTR] = text(found[LOGO_SIZE]);
```

Three invented bots on the Simulator's page

One asset has a kept file. One has none and is named by the asset database. One is named nowhere. Each reads differently from the other two.

```
a file is kept             no text, one 32-pixel image, no address
no file, the base is known the ticker, no image, the address on the cell
the base is known nowhere  the ticker, no image, no address
row height                 38 pixels, the same on the Live and Paper pages
```

What the Simulator's page reports about the payload

The page checks the payload it was handed and lists what is missing. It names the mark's four cell fields, so a payload that stops carrying one is reported rather than drawn as a blank square.

`src/gui/web/sim_bot_status_table.js` — the audited cell fields

```
   var CELL_FIELDS = [
     TEXT,
     COLOR,
     TOOLTIP,
     ALIGNMENT,
     ALIGNMENT_VALUE,
     LOGO_PATH,
     LOGO_IMAGE,
     LOGO_SIZE,
     LINK_URL,
     CHART_URL,
     UNDERLINE
   ];
```

Driven with every outward connection refused, the reading made no connection attempt and the page made no request. A control attempt afterwards was counted and refused, so the counter was able to report one.

The Simulator sentence this entry completes

It is quoted whole, and the sentence under it is true today.

Quoted whole:

> *"The Simulator's Scrumming Bots table is a subclass of the window's own bot list, so it draws that list's ten columns. The first column is Asset. It draws the target asset's own mark, and the asset's ticker where no mark is kept."*

True today: that holds for the window's table and for the page, which draws the same mark from the same payload. The Extractor Bots table keeps its own Bot ID column on both surfaces.

2026-10-25 - #665 - the simulated record carries the wizard's lock figure

The simulator wizard collected the Candles-to-lock figure and the record it saved held no key for it, so the bot built from that record had no lock holder and the Phantom Bots tab fell back to the declared 2.

`src/simulator/fleet_source.py, in wizard_record` now writes the figure beside the two phantom keys it already wrote, clamped at 1, which is the clamp the live bot's own constructor applies.

```
    lock_candles = collected.get("lock_candle_count") if scrumming else None
    if isinstance(lock_candles, (int, float)) and not isinstance(lock_candles, bool):
        record["lock_candle_count"] = max(1, int(lock_candles))
```

`src/simulator/sim_bot_view.py, in SimBotView` reads that key to build the bot's lock holder, and the Candles-to-lock box on the Phantom Bots tab reads the holder.

```
wizard 7     record holds the key     tab box 7
wizard 2     record holds the key     tab box 2
no key       record holds no key      tab box 2
```

A simulated record written before this change holds no such key, loads, and leaves the tab box on the declared 2. The Simulator and the Paper Trader take the same route, so the two cannot answer this differently.

2026-10-25 - #665 - the Simulator wizard reads the one Hedge Balance figure

The Hedge Balance box on the Simulator wizard used to open on a figure typed into the Simulator wizard itself. It reads the bot's own declaration now, the same declaration the Live wizard, the Paper wizard and the Electron wizard read.

`src/gui/simulator/sim_bot_wizard.py, in SimTradingParamsPage` — the opening figure

```
self._hedge_amount.setValue(BotConfig.hedge_balance)
```

Two more readings on this build join one declaration. The whole-days despawn rule is the one rule in `src/trading/container/config.py, in whole_despawn_days`, read by the Simulator Fold Tranches panel through the chrome it shares with Live. The Apply routing map the Simulator's Detail page reads is the one list in `src/gui/main_tabs/bot_live_settings_surface.py`, which the Qt window now reads too.

Read before and after, the box opened on $200.00 both times. Moving the declaration to a figure nothing uses moved the Simulator box with the other three; before the change the same move left all four at $200.00.

Paper Trader Tab

Reference. The third step of the promotion pipeline. Issue #19 built the screen. The tab row carried an empty tab until that build landed, and The screen as it stands now describes what draws there today.

The empty tab it replaced

The tab drew three lines: its name, one sentence saying it was not built, and the issue that owned it. It read no bot, no price and no trade.

`paper_trader_tab_surface` — the empty state it carried

```
HEADING = "Paper Trader"
ISSUE = 19
BUILT = False
STATE_TEXT = "This tab is not built."
ISSUE_TEXT = f"Issue #{ISSUE} carries the build-out."
```

The module `paper_trader_tab_surface` is gone, and the empty state above went with it. No module on disk holds `BUILT` or `STATE_TEXT` for this tab.

Two frontends draw that one view model. `EmptyTabsMixin` in `src/gui/main_tabs/empty_tabs.py` builds the Qt tab, and `src/gui/web/paper_trader_tab.js` registers a panel with the Electron shell's panel host. `src.core.desktop_bridge` serves the model under `paper_trader_tab.state`.

What the step is

Real time is its defining property. The Simulator replays stored candles as fast as the machine allows. Paper takes the live feed at the speed the market delivers it, and that is the whole difference between them.

Live, Paper and the Simulator differ in one thing only: where the data comes from. The trading logic stays one body of pure code that all three call. Only the stateful shells fork, which is what keeps a paper run from holding a live object.

The paper budget is twice the dollar target, so a bot has room to fold without the exercise ending on the first dip.

`src/trading/container/config.py` — `BotConfig`, the field it doubles

```
target_balance: float = 200.0  # Balance the bot trades relative to
```

Paper sat behind the Simulator, which had to reproduce the gate latches first. Nuclear Mode was to be a second gate; the operator cancelled it, so one gate remains and issue #19 built the tab after the Simulator rebuild landed.

Where the platform once offered it

Two live surfaces named the step. The first is the Swarm tab's third sub-tab, labelled Paper Swarm. Its Start button flips a flag and relabels itself. No bot is constructed, no feed is attached and no order is recorded. Its caption is an empty string now, so the sub-tab promises nothing.

`src/gui/bot_visualizer.py` — the Start handler inside `_create_paper_bot_row`

```
def _toggle():
    if bot_data["running"]:
        bot_data["running"] = False
        start_btn.setText("■ Start")
```

The second is the wing toggle on the header strip. Pressing it swaps the whole wing. The Qt window no longer names a Paper Trader in the line it writes, and the React mirror still does, so the two hosts print different text.

`src/gui/main_tabs/main_window_surface.py` — the line the React mirror writes

```
"→ STOCK WING: equity exchanges + equity Paper Trader. (Crypto wing paused.)"
```

Issues #422 and #426 both folded into issue #19, which built the tab.

The screen as it stands now

The tab is built. It sits second on the bar and carries the Trading tab's panes over the live exchange feed. `PaperTraderTabQt` draws them in Qt and `src/gui/web/paper_trader_tab.js` draws them in React, from one view model.

`paper_trader_tab_surface` — the fields both hosts read That module is gone and no module on disk holds `DECLARED_FIELDS` for this tab. The Paper tab's surface is `src/gui/paper/paper_trading_tab_surface.py`, `PaperTradingTab` in `src/gui/paper/paper_trading_tab.py` draws it in Qt, and `PaperTradingTabReact` in `src/gui/paper/paper_react_trading_tab.py` draws `src/gui/web/paper_trading_tab.js` in React. The names `PaperTraderTabQt` and `src/gui/web/paper_trader_tab.js` are on no file.

```
DECLARED_FIELDS = (
    "accessible_name",
    "balance",
    "built",
    "feed",
    "fleet",
    "heading",
    "indicators",
    "issue",
    "method",
    "panes",
    "privacy_button",
    "reserved_rows",
    "run",
    "skin",
    "symbol",
    "symbols",
)
```

The module `paper_trader_tab_surface` is gone, and no module on disk holds `DECLARED_FIELDS` for this tab. `src/gui/paper/paper_trading_tab_surface.py, in view_model` answers the tab today.

The left pane holds the Privacy Mode button, the market selector, the two button rows and the bot list. The bot list uses the Trading tab's own columns. The right pane holds the Indicator Voting Panel, computed from the live window. An empty selector draws the empty state and invents no reading.

`paper_trader_tab_surface` — the empty state That module is gone, and `indicator_payload` is declared once, in `src/gui/main_tabs/simulator_tab_surface.py`, with its only caller in that same module. Nothing on the Paper side reads it. The Paper pane is `src/gui/paper/paper_indicator_panel.py` in Qt and `src/gui/web/paper_indicator_panel.js` in React.

```
def indicator_payload(
    candles: Sequence[Any], timeframe: str, symbol: str, refusal: str
) -> dict:
    model = ivp.IndicatorPanelModel()
    if refusal:
        model.show_no_data(refusal)
        return ivp.build_payload(model)
    summary = multi_tf_summary(candles, timeframe, symbol)
    if not summary:
        model.show_no_data(NO_SYMBOL_TEXT)
        return ivp.build_payload(model)
    model.set_summary(summary, symbol)
    return ivp.build_payload(model)
```

The module `paper_trader_tab_surface` is gone, and no module on disk holds `indicator_payload` for this tab. `src/gui/paper/paper_trading_tab_surface.py, in ivp_feed` reads the panel today.

The two presses

Import Live Fleet reads the stored bot record and lists what it finds. Start Paper Run opens the run and begins ticking. The same button then reads Stop Paper Run and ends it, leaving the balances and the trades on screen.

`paper_trader_tab_surface` — the second row's two faces That module is gone. The two faces are declared in `src/gui/paper/paper_trading_tab_surface.py`, which holds `START_RUN_ACTION`, `STOP_RUN_ACTION`, `START_RUN_TEXT`, `STOP_RUN_TEXT`, `in button_name` and `in run_buttons`.

```
{
    "name": DATA_POOL_ROW,
    "height_px": 18,
    "action": STOP_RUN_ACTION if running else START_RUN_ACTION,
    "text": STOP_RUN_TEXT if running else START_RUN_TEXT,
    "button_name": button_name(
        STOP_RUN_ACTION if running else START_RUN_ACTION
    ),
}
```

The module `paper_trader_tab_surface` is gone, and no module on disk holds `DATA_POOL_ROW`. `src/gui/paper/paper_trading_tab_surface.py, in run_buttons` carries the row's two faces today.

Downstream only

The feed reader answers six names and refuses every other one, so a send has no spelling. It calls the venue's public market endpoints, which carry no key and no signature.

`src/paper/live_feed_source.py` — the refusal

```
def __getattr__(self, name: str):
    """Refuse every name outside ``READ_NAMES``."""
    raise SendRefused(
        f"LiveFeedSource answers {READ_NAMES} and cannot {name!r}. "
        "The Paper Trader receives and asks; it sends nothing."
    )
```

One gate chain

A tick builds the same `GateContext` a live bot builds and evaluates the shipped scrum and fold chains on it. The paper side adds no gate and changes none.

`src/paper/paper_run.py` — what a tick evaluates

```
window = list(candles[-WINDOW_CANDLES:])
reading = bb_reading(window, bot)
summary = VotingEngine().compute_all(window, bot.ta_timeframe, symbol=bot.symbol)
context = tape_context(bot, balance, window, reading, summary)
armed = latch(context)
```

The gate row names the override that replaced a refusal

A tick's gate row now says which refusals an override lifted, one entry per refusal, per side. A row that passed outright and a row an override passed used to read the same: both showed armed with an empty refusal list. They read apart now, and the refusal's own text is kept beside the override's name.

`src/paper/paper_log.py` — the entry written on the Paper Trader's gate row

```
def scrum_fixture(context: Any, overrides_applied: Sequence[str] = ()) -> dict:
    """The scrum half of a gate row, with each refusal an override replaced
    under ``OVERRIDES_APPLIED_KEY``."""
```

The Paper Trader reads the same record the Simulator reads, and the key is spelled once for all three data sources. Live's gate log, the Paper Trader's gate row and the Simulator's gate event therefore record an override the same way. Which gate each override covers, and the sentence that says what the overrides are for, are on the indicators page under The two overrides.

Two overrides exist and they cover six of the twenty-four gates the two shipped chains hold. Both chains were built and each gate asked which override covers it. The control is the same ask answering both ways in one run: it named an override for these six and nothing for the other eighteen.

Override	Side	Gates it covers
`ripe_harvest_override`	scrum	midline_scrum, ta_bullish, target_fires, trend_hold
`deep_fold_override`	fold	midline_fold, ta_bearish

Nothing about which trades fire changed. The armed verdict and the refusal list are untouched, and the row simply carries one more field.

The fake balance

Each bot opens a budget of twice its dollar target: units worth the target, and the rest in cash to fund a fold. The balances live in memory and are dropped when the tab goes away.

`src/paper/fake_balance.py` — the opening

```
def opening_balance(target_usd: float, price: float) -> FakeBalance:
    whole = budget_usd(target_usd)
    held_usd = min(whole / BUDGET_MULTIPLE, whole)
    units = held_usd / float(price) if float(price) > 0.0 else 0.0
    return FakeBalance(
        units=units,
        cash_usd=whole - units * float(price),
        tranches=0,
        budget_usd=whole,
    )
```

The fleet ledger

The fake balance is one figure for the whole fleet, not one figure per bot. Five paper bots with a $200 target open Paper Spendable at $1000 and Paper Locked at $1000. Each variable holds the full fleet sum, which is twice the fleet's dollar target in total.

`src/paper/fake_balance.py` — the opening

```
def opening_ledger(bots: Sequence[Any]) -> PaperLedger:
    total = fleet_target_usd(bots)
    return PaperLedger(
        fleet_target_usd=total,
        opening_spendable_usd=total,
        opening_locked_usd=total,
    )
```

The fleet changes when a bot is added or removed. Every press of Start Paper Run rebuilds the ledger from the fleet it is handed, so both opening figures move on the next press and never inside a run. Six $200 bots open $1200 in each figure and four open $800.

Four figures are tracked. Paper Spendable is the cash the fleet holds. Paper Locked is what its units are worth at the last price the feed answered. Paper Realized Profits moves when a fold buy closes the tranche a scrum sell opened. Paper Mature Profits is the profit on positions past 200 percent growth, the figure the live wing already reads.

`src/trading/smart_wire.py` — the maturity rule both wings call

```
MATURE_GROWTH_PCT: float = 200.0
"""A position is mature once its value exceeds its cost basis by this percent."""
```

The four figures are held apart from the live ones. They live in memory on the run, they are never written into the stored bot record, and no paper figure reaches the header strip's real-money columns.

`src/paper/fake_balance.py` — the four labels the strip prints

```
FIGURE_LABELS = (
    ("spendable_usd", SPENDABLE_LABEL),
    ("locked_usd", LOCKED_LABEL),
    ("realized_profit_usd", REALIZED_LABEL),
    ("mature_profit_usd", MATURE_LABEL),
)
```

The Qt window and the React page print one line from one view model, so the two hosts cannot report different money.

`paper_trader_tab_surface` — the one line

```
"text": LEDGER_SEPARATOR.join(f"{one['label']} {one['text']}" for one in cells),
```

The module `paper_trader_tab_surface` is gone, and no module on disk holds `LEDGER_SEPARATOR`.

The Paper Trader log

Every paper action is written to one file. Each line carries two halves: the gate decision that produced the action and, when one filled, the pretend trade in the shape a Coinbase year-to-date row takes. Validation reads both of those shapes already, so a paper run is checked by the reader that checks the live wing.

`src/paper/paper_log.py` — the row

```
return {
    "timestamp": iso_stamp(tick.wall_ms),
    "category": CATEGORY,
    "exchange": str(exchange_id),
    "bot_id": str(tick.bot_id),
    "data": gate_half(tick),
    "trade": trade_half(tick.filled) if tick.filled is not None else None,
    "ledger": dict(figures or {}),
}
```

The log has a home of its own. It is a sibling of the live folders and never sits inside one, so nothing a paper run writes can land in a live tree.

`src/paper/paper_paths.py` — the root

```
PAPER_ROOT: Path = Path.home() / ".acervator_paper"
```

Gate names come from one place. The row records the blocker phrases the chain produced and the labels those phrases map to, and it spells no gate name of its own.

`src/trading/gate_vocabulary.py` — the mapping the log calls

```
def gate_for_blocker(blocker: str) -> str:
    """Map one blocker string to its gate label, or "" if unknown."""
```

A trade is never invented. A line carries a trade half only because the gate chain latched and the fill was applied to a fake balance.

`src/paper/paper_run.py` — the only writer

```
paper_log.append_row(
    paper_log.paper_row(seen, bot.exchange_id, run.figures())
)
```

Real time

A run ticks on the wall clock and asks the feed for one bot per fire, so a fleet of many bots never blocks the window on a single pass. One bar is shared across the open bots.

`PaperTraderTabQt` — the round robin

```
def advance_once(self) -> list:
    """Tick the next open bot against its newest live window."""
    if self._run is None or not self._run.running or not self._run.bots:
        return []
    chosen = self._run.bots[self._cursor % len(self._run.bots)].bot_id
    self._cursor += 1
    made = surface.advance_run(self._run, self._feed, chosen)
    self.refresh()
    return made
```

Paper Trade History

The manual's part list names a second History tab reading a paper trade log. No such module exists. `HistoryTab` in `src/gui/history_tab.py` reads live venue history alone.

The ledger such a tab would read now exists. Every paper action lands in the file the Paper Trader log names, and no screen reads that file yet.

In development.

2026-09-08 08:17 - #19 - what the closed issues landed

Real-time, API-fed trades against a fake budget. This is designed as the second tier of strategy validation within the platform.

The tab is now called Paper. It sits second on the bar, on a white ground with black text, and it is the only white tab.

The screen was empty until issue #19 built it. The tab row carried a Paper Trader placeholder, which drew its name, one sentence saying it was not built, and the issue that owned it. The section under this one holds what the tab is now.

`paper_trader_tab_surface` — the empty state it carried

```
HEADING = "Paper Trader"
ISSUE = 19
BUILT = False
STATE_TEXT = "This tab is not built."
ISSUE_TEXT = f"Issue #{ISSUE} carries the build-out."
```

The module `paper_trader_tab_surface` is gone, and the empty state above went with it.

The Qt tab and the `src/gui/web/paper_trader_tab.js` panel both draw that one view model, so neither can drift from the other.

Live, Paper and the Simulator differ in one thing only, where the data comes from. The trading logic stays one body of pure code all three call, and only the stateful shells fork. Real time is Paper's defining property, and its budget is twice the dollar target.

`src/trading/container/config.py` — `BotConfig`, the field that budget doubles

```
target_balance: float = 200.0  # Balance the bot trades relative to
```

Two live surfaces once offered the step. The Swarm tab's Paper Swarm sub-tab is chrome: its Start button flips a flag and relabels itself, and no bot is constructed. Its caption is now an empty string, so it promises nothing. The wing toggle no longer names a Paper Trader in the Qt window, while the React mirror still does. Issues #422 and #426 folded into issue #19.

The tab as it stands now

The screen is built. It is the Trading tab's panes over the live exchange feed: the Privacy Mode row, the market selector, the two button rows, the bot list, the Indicator Voting Panel, the Live Feed pane, the Fake Balance table and the Paper Run table. The Qt window and the React page draw one view model.

`paper_trader_tab_surface` — what the tab answers That module is gone and no bridge serves `paper_trader_tab.state`. The surface is `src/gui/paper/paper_trading_tab_surface.py`, which declares `METHOD = "paper_trading.tab"`, `HEADING = "Paper"` and `ISSUE = 19`. The block below records the older names.

```
METHOD = "paper_trader_tab.state"

HEADING = "Paper"
ISSUE = 19
BUILT = True
```

The module paper_trader_tab_surface is gone. src/gui/paper/paper_trading_tab_surface.py, in METHOD answers paper_trading.tab today, and that file holds HEADING and ISSUE. No module on disk holds BUILT for this tab.

The data path is the venue's public market feed, and it goes one way. The reader names the six things it answers, and every other name raises. An order, a cancellation or a venue write cannot be written through it.

src/paper/live_feed_source.py — the read set and the refusal

```
READ_NAMES = (
    "venue",
    "product_id",
    "granularity",
    "candles",
    "ticker",
    "asked_at",
)


class SendRefused(AttributeError):
    """Raised when a name outside ``READ_NAMES`` is asked of ``LiveFeedSource``."""
```

Live candles enter the same gate chain a live bot runs. Nothing on the paper side defines a gate of its own.

src/paper/paper_run.py — the tick

```
context = tape_context(bot, balance, window, reading, summary)
armed = latch(context)
```

The money is fake and lives in memory alone. A run writes no file, and no figure it produces reaches the stored bot record or the header strip.

src/paper/fake_balance.py — the budget

```
BUDGET_MULTIPLE = 2.0
```

The two strips the Trading tab carries are not copied, and neither is their supporting code. Their rows carry Import Live Fleet and the Start or Stop press.

The fake balance is a fleet figure. Five paper bots with a $200 target open Paper Spendable at $1000 and Paper Locked at $1000, each holding the full fleet sum. Adding or removing a bot moves both openings on the next press of Start.

src/paper/fake_balance.py — the fleet total both openings take

```
def fleet_target_usd(bots: Sequence[Any]) -> float:
    """The sum of every bot's ``target_usd``, the figure both openings take."""
```

Two more figures are tracked beside them. Paper Realized Profits moves when a fold buy closes the tranche a scrum sell opened, and Paper Mature Profits is the profit on positions past 200 percent growth, which is the

figure the live wing already reads. No paper figure is written into the stored bot record or the header strip.

`src/paper/fake_balance.py` — the strip's four figures

```
FIGURE_LABELS = (
    ("spendable_usd", SPENDABLE_LABEL),
    ("locked_usd", LOCKED_LABEL),
    ("realized_profit_usd", REALIZED_LABEL),
    ("mature_profit_usd", MATURE_LABEL),
)
```

Every paper action is recorded in one file of its own. Each line carries the gate decision and, when one filled, the pretend trade in the shape a Coinbase year-to-date row takes. The file sits beside the live folders and never inside one.

`src/paper/paper_paths.py` — where the log lives

```
PAPER_ROOT: Path = Path.home() / ".acervator_paper"
```

How the Paper tab reaches the bar

`PaperTraderTabMixin` builds the tab and inserts it into the window's tab book. The builder catches every error, writes one warning line, and leaves the tab off the bar. A tab that asks the panel for a value the panel does not publish is therefore a missing tab, not a crash the operator can see.

`src/gui/main_tabs/paper_trader_tab.py` — the builder

```
def _build_paper_trader_tab(self) -> None:
    """Insert the Paper tab at ``PAPER_BUILD_INDEX``."""
    try:
        from ..variant_surface import PAPER_TRADER, surface_class

        self._paper_trader_tab = surface_class(PAPER_TRADER)()
        self._main_tabs.insertTab(
            PAPER_BUILD_INDEX, self._paper_trader_tab, HEADING
        )
    except Exception as exc:  # noqa: BLE001 - a missing tab is not a crash
        logger.warning("Paper tab unavailable: %s", exc)
        self._paper_trader_tab = None
```

The window builds nine tabs. Paper takes the second slot, after Sim.

```
Sim  Paper  Live  Charts  Inspector  Swarm  History  Status  Console
```

The line beside the voting panel title

One label sits beside the Indicator Voting Panel title. It holds the panel's empty state and nothing else. The shared function `no_data_text` builds that line, and the Simulator page shows it. The label is hidden whenever the panel has values to draw.

`PaperTraderTabQt` — the Qt clone draws the line

```
def _draw_indicators(self, panel: dict) -> None:
    self._indicator_title.setText(panel["title_text"])
    # no_data text is empty while the panel holds a reading, so the
    # summary label shows only the one reason there is nothing to draw.
    reason = panel["no_data"]["text"]
    self._indicator_summary.setText(reason)
    self._indicator_summary.setVisible(bool(reason))
    for table, spec in zip(self._indicator_tables, panel["tables"], strict=True):
        self._fill_indicator_table(table, spec)
```

With no paper fleet the label names the press that builds one, under both builds.

```
No TA data — No paper fleet yet. Press Import Live Fleet.
```

The style sheet the Paper page loads

The page carries one style sheet. `TAB_STYLE_ASSETS` names it, and `page_html` reads the file and writes it into the page's head.

`react_paper_trader_tab` — the sheet the page carries That module is gone, no module declares `TAB_STYLE_ASSETS` for this page, and there is no `paper_trader_tab.css` on disk. The Paper page carries Live's own two sheets, declared in `src/gui/paper/paper_react_trading_tab.py` as `STYLE_ASSETS`, and `in page_html` from `src/gui/react_history_panel.py` writes them into the page's head.

```
TAB_STYLE_ASSETS: tuple[str, ...] = ("paper_trader_tab.css",)
```

`src/gui/paper/paper_react_trading_tab.py` — the sheets the page carries today

```
STYLE_ASSETS: tuple[str, ...] = ("trading_tab.css", "exchange_tab.css")
```

The sheet sets the page ground, the type, the pane grid and the table rules. The grid repeats the two pane sizes the Qt clone gives its splitters, so the bot list, the voting panel, the feed pane and the run pane take the same share of the window in both builds.

`src/gui/web/paper_trader_tab.css` — the pane grid

```
.paper-top,
.paper-bottom {
  display: grid;
  grid-template-columns: 600fr 500fr;
}
```

Without the sheet the page draws on the browser's own defaults: a white ground, serif type, one column and no grid.

The skin tokens the Paper sheet reads

The sheet names no colour of its own. Every colour arrives as a custom property on the tab element, written from the `SKIN` dictionary the surface builds. A scrum row and a fold row in the Paper Run table take the two colours `SIDE_COLOURS` gives them in the Qt clone.

`src/gui/web/paper_trader_tab.css` — the two trade rows

```
.paper-tab tr[data-side="scrum"] td {
  color: var(--paper-scrum-colour, var(--text));
}

.paper-tab tr[data-side="fold"] td {
  color: var(--paper-fold-colour, var(--text));
}
```

Start Paper Run redraws the pane

`start_run` draws the pane after it opens the run. With no paper fleet the first tick returns nothing, so without that draw the Paper Run pane kept the word it held before the press while the run was open.

`PaperTraderTabQt` — the press that opens a run

```
    def start_run(self) -> str:
        """Open the run and tick one bot per timer fire from now."""
        if not self._bots:
            self._bots = surface.live_fleet()
        self._run = surface.start_run(self._bots)
        self._cursor = 0
        self.advance_once()
        self.refresh()
        self._tick_timer.start(surface.tick_interval_ms(self._run, self._symbol))
        return self._run.state
```

Both builds carry the same draw.

2026-09-19 - #19 - Live's tab code cloned under Paper names

Unit Q1 of issue #19. The Paper tab is now Live's tab code, forked under Paper names in both builds, with Import Live Fleet as the fleet's one way in and the header strip reading the paper ledger while Paper is in front. The two hosts the sections above describe, `PaperTraderTabQt` and `react_paper_trader_tab`, stay on disk and nothing loads them; a later unit removes them once a paper bot ticks on the clone.

The budget rule today overtakes four sentences

Four sentences on this page carry a figure the operator's words of 2026-09-19 overtake. Each is quoted here and kept above as it stands.

- "The paper budget is twice the dollar target, so a bot has room to fold without the exercise ending on the first dip." (What the step is)
- "Each bot opens a budget of twice its dollar target: units worth the target, and the rest in cash to fund a fold." (The fake balance)

- "Each variable holds the full fleet sum, which is twice the fleet's dollar target in total." (The fleet ledger)
- "Real time is Paper's defining property, and its budget is twice the dollar target." (2026-09-08 08:17)

The rule now: the paper budget is unbounded, and Paper Spendable opens at the sum of the fleet's Target Balances. His words: *"a fake, unbounded (starts equal to bots total target balance) budget."* No fold is refused for cash. `BUDGET_MULTIPLE` in `src/paper/fake_balance.py` still reads `2.0`; the unit that starts a paper run retires it, and no clone code reads it.

`src/paper/fake_balance.py` — the opening figure the rule keeps

```
def fleet_target_usd(bots: Sequence[Any]) -> float:
    """The sum of every bot's ``target_usd``, the figure both openings take."""
```

The Qt tab is Live's tab code, forked

The Qt build of the Paper tab is `PaperTradingTab`, a fork of the Live tab's own code under the Paper Trader's name. The seam's Qt loader answers it.

`src/gui/variant_surface.py` — the Qt loader

```
def _qt_paper_trader() -> type:
    """Import and return the Qt Paper tab, ``PaperTradingTab``."""
    from .paper.paper_trading_tab import PaperTradingTab

    return PaperTradingTab
```

Each module under `src/gui/paper/` is one Live module copied and renamed, or one of the Simulator's forks copied once more where that fork's only change was the feed. The copy keeps Live's layout, titles, sizes and design tokens.

Paper module	forked from
`paper_trading_tab.py` `PaperTradingTab`	`src/gui/main_tabs/trading_tab.py` `_build_trading_tab`, and the window's `add_exchange_tab`
`paper_exchange_tab.py` `PaperExchangeTab`	`src/gui/widgets/exchange_tab.py` `ExchangeTab`
`paper_bot_status_table.py` `PaperBotStatusTable`	`src/gui/simulator/sim_bot_status_table.py` `SimBotStatusTable`
`paper_extractor_bot_table.py` `PaperExtractorBotTable`	`src/gui/simulator/sim_extractor_bot_table.py` `SimExtractorBotTable`
`paper_indicator_panel.py` `PaperIndicatorVotingPanel`	`src/gui/simulator/sim_indicator_panel.py` `SimIndicatorVotingPanel`
`paper_status_log.py` `PaperStatusLog`	`src/gui/simulator/sim_status_log.py` `SimStatusLog`
`paper_bot_detail.py` `PaperBotDetailDialog` and its seven tab mixins	`src/gui/simulator/sim_bot_detail.py` `SimBotDetailDialog` and its seven
`paper_exchange_choice.py` `PaperExchangeChoiceDialog`	`src/gui/simulator/sim_exchange_choice.py` `SimExchangeChoiceDialog`
`src/paper/fleet_source.py` `PaperFleetSource`, `PaperBot`	`src/simulator/fleet_source.py` `FleetSource`, `SimBot`, one fleet and no run mode

Paper module	forked from
`src/paper/paper_bot_manager.py` `PaperBotManager`	`src/simulator/sim_bot_manager.py` `SimBotManager`
`src/paper/paper_bot_view.py` `PaperBotView`	`src/simulator/sim_bot_view.py` `SimBotView`

Diagram: flowchart, left to right

- variant_surface PAPER_TRADER -> **PaperTradingTab**
- PaperTradingTab -> **exchange layer stack**
- exchange layer stack -> **PaperExchangeTab**
- PaperExchangeTab -> **PaperBotStatusTable**
- PaperExchangeTab -> **PaperExtractorBotTable**
- PaperTradingTab -> **PaperIndicatorVotingPanel**
- PaperTradingTab -> **PaperStatusLog**
- PaperTradingTab -> **API Interaction Log**
- PaperTradingTab -> **PaperFleetSource**
- PaperFleetSource -> **paper_fleet.json under PAPER_ROOT**

What the fork draws

The tab is Live's four splitters at Live's sizes: the exchange layer stack beside the panel on top, the Activity Log and the API Interaction Log side by side below. Every module carries Live's title. The venue page holds Privacy Mode where Live draws it, `+ New Bot` where Live draws it, the Scrumming Bots table and the Extractor Bots table, both hidden until a row arrives, and the command bar of Start, Pause, Stop, Restart and Delete. The panel holds its title, the Bot selector and its privacy dot, the currency rate strip, both indicator tables, both confidence bar graphs, the timeframe-lock line and the staleness banner.

Three positions differ from Live by ruling, the ruling unit 7 of issue #117 made for the Simulator. The corner Live gives ■ `Add Crypto Exchange` holds Import Live Fleet and Start Paper Run, each at Live's corner-button width. The Get Started card holds the same two where Live's card holds its add button. The news line and the data-pool line are not forked; the data-pool row keeps Live's height and holds nothing.

`src/gui/paper/paper_trading_tab_surface.py` — the corner buttons

```
CORNER_BUTTONS = (
    (paper.IMPORT_LIVE_FLEET_ACTION, paper.IMPORT_LIVE_FLEET_TEXT),
    (paper.START_RUN_ACTION, paper.START_RUN_TEXT),
)
```

A press on `+ New Bot` writes one Activity Log line naming Import Live Fleet as the fleet's way in; the paper bot wizard is a later unit's. A press on Start Paper Run writes one line saying no paper run is built; the run is a later unit's too.

`src/gui/paper/paper_trading_tab_surface.py` — the two lines

```
NEW_BOT_FORMAT = (
    "+ New Bot on {exchange}: the Paper fleet is loaded through "
    "{way_in}; the paper bot wizard is not built."
)

START_RUN_TEXT = "Start Paper Run: no paper run is built; nothing started."
```

THE FORM THE WIZARD REPLACED. The block above stays here, because a reader who saw that line needs to find it. `NEW_BOT_FORMAT` left the tree, and the press opens the Bot Creation Wizard instead of writing a line. Its neighbour in that module now holds the button's caption rather than a refusal.

`src/gui/paper/paper_trading_tab_surface.py` — what stands there today

```
IMPORT_LIVE_FLEET_TEXT = "Import Live Fleet"
START_RUN_TEXT = "Start Paper Run"
STOP_RUN_TEXT = "Stop Paper Run"
```

What the fork does not carry

The copies hold no live bot manager, no connector and no event bus. Every send the Live code makes is cut, not stubbed: the signal-contract emits, the event-bus emit behind the bot selector, the API-log listener on the process-wide log, the watchdog over the live bot manager, the TA snapshot store, the data-pool read behind the display price, and the demo readings the Live panel invents from a random walk. Zero occurrences of `ScrummingBot`, `BotContainer`, `BotManager`, `EventBus` and `PhantomBalance` under `src/gui/paper/` and `src/paper/`.

The Simulator's run modes, its way-in row, its replay layer, its Stone Tablet source, its Portfolio Battery, its retrieval and its nigredo tone are not carried. A Paper module forked from a Simulator fork has those parts cut.

Asked for a send by name, `PaperFleetSource` raises `SendRefused`. The Fire button on a row reaches it and the refusal lands in the Activity Log.

`src/paper/fleet_source.py` — the refusal

```
    def __getattr__(self, name: str):
        """Refuse every name outside ``READ_NAMES``."""
        raise SendRefused(
            f"PaperFleetSource answers {READ_NAMES} and cannot {name!r}. "
            "The Paper Trader receives and asks; it sends nothing."
        )
```

The React page is Live's page modules, forked

The React build of the Paper tab is `PaperTradingTabReact`, a fork of the Live tab's React host under the Paper Trader's name, and it loads seven page modules that are Live's seven copied under Paper names. The seam's React loader answers it.

`src/gui/variant_surface.py` — the React loader

```
  def _react_paper_trader() -> type:
      """Import and return the React Paper tab, ``PaperTradingTabReact``."""
      from .paper.paper_react_trading_tab import PaperTradingTabReact

      return PaperTradingTabReact
```

Paper module	forked from
src/gui/web/paper_trading_tab.js	trading_tab.js
src/gui/web/paper_exchange_tab.js	exchange_tab.js
src/gui/web/paper_indicator_panel.js	sim_indicator_panel.js, the flip button cut
src/gui/web/paper_status_log.js	sim_status_log.js
src/gui/web/paper_bot_status_table.js	sim_bot_status_table.js
src/gui/web/paper_extractor_bot_table.js	sim_extractor_bot_table.js
src/gui/web/paper_table_cells.js	sim_table_cells.js
src/gui/paper/paper_react_trading_tab.py PaperTradingTabReact	src/gui/react_trading_tab.py TradingTabReact
src/gui/paper/paper_trading_tab_surface.py	src/gui/main_tabs/trading_tab_surface.py, the payload builder
src/gui/paper/paper_exchange_tab_surface.py	src/gui/main_tabs/exchange_tab_surface.py, the payload builder
src/gui/paper/paper_react_bot_detail.py PaperBotDetailReactDialog	src/gui/simulator/sim_react_bot_detail.py, over Live's nine window bundles

Diagram: flowchart, left to right

- variant_surface PAPER_TRADER -> **PaperTradingTabReact**
- PaperTradingTabReact -> **paper_trading_tab.js**
- paper_trading_tab.js -> **paper_exchange_tab.js**
- paper_exchange_tab.js -> **paper_bot_status_table.js**
- paper_exchange_tab.js -> **paper_extractor_bot_table.js**
- paper_bot_status_table.js -> **paper_table_cells.js**
- paper_extractor_bot_table.js -> **paper_table_cells.js**
- paper_trading_tab.js -> **paper_indicator_panel.js**
- paper_trading_tab.js -> **paper_status_log.js**
- paper_trading_tab.js -> **API Interaction Log pane**

What the page draws

The page is Live's four splitters at Live's sizes, read off Live's own `trading_tab_surface`. Every module carries Live's title and Live's part name, so the page and the Qt fork enumerate the same modules in the same order. The corner holds Import Live Fleet and Start Paper Run in Live's corner-button chrome, and the Get Started card holds the same two at Live's card-button size. The venue page holds Privacy Mode, plain space where Live draws the news line, `+ New Bot`, the data-pool row at Live's height holding a blank line, the two tables and the command bar.

`src/gui/web/paper_trading_tab.js` — the corner

```
  // The corner Live gives its add button: the payload's buttons in one row.
  function CornerButtons(props) {
    var listed = listField(props.layer, CORNER_BUTTONS);
```

The host builds one payload per bridge method from state it owns: the tab payload from its own `PaperTradingTabState`, the Activity Log from its own `PaperStatusLogModel`, the panel from its own `IndicatorPanelModel`, and each seated venue's three payloads from that venue's own `ExchangeTabModel`, its `PaperBotStatusTableModel` and Live's `ExtractorBotTableModel`. Every ask the page makes is written to the console line the host reads, and `run_action` answers it; nothing is sent anywhere else.

`src/gui/paper/paper_react_trading_tab.py` — the console line

```
ACTION_PREFIX = "acervator-paper:"
```

The header strip shows on Paper

The window's header strip stays on screen while the Paper tab is in front. `ISOLATED_TABS` names no tab now, so the strip hides on nothing. While Paper is in front the ten fields carry the Paper Trader's figures: SPENDABLE, REALISED, LOCKED and MATURE from the paper ledger, EXCH from the seated venues, and the five cards from the held records. On every other tab the strip carries what it carried before; the live fleet's figures do not appear while Paper is in front, and no paper figure appears on Live.

`src/gui/main_tabs/main_window_surface.py` — the tuples and the reader

```
ISOLATED_TABS: tuple = ()

#: The tabs the header strip reads the Paper Trader's ledger and records on.
PAPER_FED_TABS = (PAPER_TAB,)


def header_strip_reads_paper(tab_name: Any) -> bool:
    """Whether the stat strip reads the Paper Trader's ledger and records on the tab named."""
    return tab_name in PAPER_FED_TABS
```

`src/gui/main_window.py` — `_refresh_header_strip`, the branch beside the Sim's

```
            if self._header_strip_reads_paper():
                paper_tab = self._paper_trader_tab
                self._write_header_strip(
                    paper_tab.aggregate(), paper_tab.exchange_count()
                )
                return
```

The tab's `aggregate` is `strip_aggregate`: Live's arithmetic over the held records, with the ledger's four money figures laid over the four money keys. Before a run the ledger holds zeros, so the four money columns read an em dash, as Live's read with no bot.

`src/paper/fleet_source.py` — the ledger's keys over the aggregate's

```
LEDGER_TO_AGGREGATE = (
    ("spendable_usd", "wallet_cash_usd"),
    ("locked_usd", "crypto_position_value_usd"),
    ("realized_profit_usd", "total_realized_exchange"),
    ("mature_profit_usd", "total_mature_exchange"),
)
```

Diagram: flowchart, left to right

- dashboard tick, 2000 ms -> **tab in front**
- tab change -> **tab in front**
- pick -- Sim -> **SimTradingTab.aggregate**
- pick -- Paper -> **PaperTradingTab.aggregate**
- pick -- any other -> **BotManager.get_aggregate_stats**
- SimTradingTab.aggregate -> **the ten cells**
- PaperTradingTab.aggregate -> **the ten cells**
- BotManager.get_aggregate_stats -> **the ten cells**

AMMO reads the paper fleet's own total

AMMO on the strip is every paper bot's Target Delta added together, in whole dollars and with no sign, so a bot above its target cannot cancel one below. It draws green while more paper bots hold more than their target and red while more hold less, which is the rule the Trading tab page gives the column. The fleet source publishes the total and the two bot counts the colour reads.

`src/paper/fleet_source.py` — `aggregate_stats`

```
ammo_readings.append((position_value, bot.live_target_usd, bot.target_usd))
...
"total_target_delta_usd": ammo["total_target_delta_usd"],
"bots_scrum_territory": ammo["bots_scrum_territory"],
"bots_fold_territory": ammo["bots_fold_territory"],
```

One call sums those three figures for every fleet source, so the Paper Trader's total, the Simulator's and Live's cannot be worked out differently.

`src/trading/target_bands.py` — `fleet_ammo`

```
target_usd = ammo_target(live_target, configured_target)
if target_usd > 0:
    total_usd += abs(target_delta(position_value, target_usd))
    where = target_territory(position_value, target_usd)
```

A reading takes the grown target wherever a paper bot carries one, which is the target that bot's own Ammo cell is measured against. The strip's total is therefore the sum of the column above it. A paper bot with no target contributes nothing to either figure.

Import Live Fleet fills the tables and the strip

Import Live Fleet copies the stored bots of one exchange out of the live fleet file into the Paper Trader's own fleet file, in both builds. A press at the corner or on the Get Started card reads which exchanges the live file names. With more than one it opens the chooser; with exactly one it takes that one and opens nothing; with none it writes one Activity Log line naming the file and moves nothing. Each stored bot on the chosen exchange is copied whole, every key of the record, into the paper fleet under its own bot id, and the fleet signal fires: the paper fleet file is written, the venue seats, and the two tables draw one row per imported bot. The Activity Log reads `Imported 38 bot(s) from bot_state.json on coinbase.` on a fleet of thirty-eight. The live fleet file is only read, never written.

`src/paper/fleet_source.py` — the copy

```
    def import_live_fleet(self, exchange_id: str) -> list[PaperBot]:
```

`src/paper/paper_paths.py` — where the paper fleet file lives

```
#: The paper fleet file under ``PAPER_ROOT``, in ``bot_state.json``'s shape.
PAPER_FLEET_NAME = "paper_fleet.json"
```

The tab starts empty. The fleet source answers the records the paper fleet file holds and nothing else, so with no paper fleet there is no venue, no row and no figure until Import Live Fleet loads one. On the next launch the tab reads the paper fleet file once and draws the imported rows from it.

The file's top level carries the wires

Beside `bots`, the paper fleet file carries `smart_wires` and `smart_wire_ledgers`, the two keys `bot_state.json` carries for the live fleet. The Paper tab holds one fleet and no run mode, so both keys sit at the file's top level rather than inside a mode's entry the way the Simulator nests them.

`PaperFleetSource.hold_wire_manager` takes a `PaperWireManager`'s rows onto the fleet and `PaperFleetSource.wire_manager` builds the manager back at the next launch, so a paper wire survives a restart the way a live one does. Stop Paper Run holds the run's manager and saves the file, so what a wire carried during a run is on disk before the tab is closed.

A file written before these keys existed reads as a fleet with no wire, and a build that does not know the keys reads the same bots, the same venues and the same rows it read before. Both were driven: the unchanged tree's reader and this one gave an identical reading of a file carrying both keys, and the same comparison reported a difference the moment one value was changed by hand.

`src/paper/fleet_source.py` — the write

```
        payload = {
            "saved_at": time.time(),
            "saved_at_human": datetime.now().strftime(SAVED_AT_HUMAN_FORMAT),
            "bot_count": len(self._records),
            "bots": dict(self._records),
            WIRES_KEY: [dict(one) for one in self._wires],
            WIRE_LEDGERS_KEY: [dict(one) for one in self._wire_ledgers],
        }
```

Deleting a paper bot drops its ledger row and every wire row naming it, the way the live state file drops a removed bot's, so no route is left pointing at a bot the fleet no longer holds.

A fold's growth travels the wires to another paper bot

A paper bot can send part of a fold's growth to another paper bot. A wire names a source, a target and a percentage. The moment a paper fold compounds its growth into the source's target balance, the wire's share of that same figure is credited to the other bot. The source keeps its own growth whole and the share is new money at the destination, which is how the live fold route behaves.

The share lands in the target's standing fold tranches, split evenly, the way wire credits land on a live bot. A paper balance opens on the target's first worked tick, so a share arriving before that is parked on its ledger and spread over the tranches the target holds when it opens, or over the first tranche its own scrum builds. The ledger carries the lifetime sent, the lifetime received, the provenance per funder and the parked pool.

`PaperWireManager` in `src/paper/paper_wire.py` owns the topology and one ledger per paper bot.

verb	what it does
`register_wire`	sets the source-to-target percentage, from 0 up to 100
`unregister_wire`	drops one route
`attach_bot`	holds the fake balance of an open paper bot a share can reach
`release_bot`	gives that balance back when the run ends
`distribute_fold_profit`	moves each wire's share of one fold's growth
`land_parked_credits`	spreads a parked pool over standing tranches
`export_wires`, `import_wires`	the routes the paper fleet file carries
`export_ledgers`, `import_ledgers`	the ledgers the paper fleet file carries

`src/paper/paper_run.py` — the fold hands its growth on

```
    growth = grow_target(bot, balance, units, slices, float(price), int(candle_ts_ms))
    if wires is not None and growth > 0.0:
        route_fold_growth(wires, bot.bot_id, growth, float(price))
```

One fold driven over an empty scratch home, two bots at a $200.0000 target, a scrum at $110.00 and a fold at $95.00, with one wire at 40 per cent between them:

```
the fold's growth on the source       6.7773
the wire's rate                         40.0 per cent
the share that leaves                    2.7109
the target's fold queue before          49.7000
the target's fold queue after           52.4109
the source's lifetime sent               2.7109
the target's lifetime received           2.7109
```

The same pair ticked over a whole price series sends four times. The source's own walk is unchanged by the wire, 5 scrums and 4 folds either way, and the target ends holding more of its asset because the wired money bought more on its folds.

```
                          no wire       a wire at 40 per cent
source scrums, folds      5, 4          5, 4
source end target       249.3815      249.3815
target end units          2.8997        3.1297
target lifetime received  0.0000       19.7526
```

A paper wire reaches paper money only. The manager holds one ledger per paper bot and delivers nothing to an id it has no ledger for, so a wire pointed at a live bot or at a simulated bot books no transfer and reports that the target has no ledger. That was driven against a real live bot id and a real simulated bot id: both routes answered `target bot has no ledger`, no transfer was booked, and both state files read the same digest before and after.

A loaded bot reads IDLE

Import Live Fleet copies each stored record and writes its state `idle`, whatever the live file saved, and the paper fleet file's records read `idle` at construction. That is the live restore's own rule, and the Simulator's. A row's state is the Bot ID cell's colour and its tooltip's `State:` line; Live's ten columns hold no State column.

`src/paper/fleet_source.py` — the one write

```
def loaded_idle(record: dict) -> dict:
    """``record`` with its ``state_when_saved`` written ``BotState.IDLE``, as
    ``restore_bots_from_state`` recreates every persisted bot in IDLE until
    the operator starts it; a loaded bot never reads ``running`` with no run."""
    record["state_when_saved"] = BotState.IDLE.value
    return record
```

The command bar moves a record's state through `PaperBotManager`: Start writes `running` and Live's ✓ `Bot <id> RUNNING.` line, Pause `paused`, Stop `stopped`, Restart `running`, and Delete opens Live's Delete box. No paper bot ticks yet; the state is the record's. Detail opens the seven tabs over the record through `PaperBotDetailDialog`, and Apply is refused: the pending line reads `Refused 1 change(s) — the Paper Trader sends nothing`.

How the Paper tab reaches the bar today

The builder in `src/gui/main_tabs/paper_trader_tab.py` is unchanged. It asks the seam for the Paper class, and the seam now answers the clone in both builds. The window builds nine tabs and Paper takes the second slot, after Sim, in both.

`src/gui/variant_surface.py` — the registration

```
register(PAPER_TRADER, _qt_paper_trader, _react_paper_trader)
register(PAPER_BOT_DETAIL, _qt_paper_bot_detail, _react_paper_bot_detail)
```

The window's floor is 1400 by 900 pixels, set in `src/gui/main_window.py`, so no tab is reached narrower than 1400 through the window. Constructed alone, the Qt tab draws at 806 pixels wide and no narrower after a fleet loads, the sum of Live's own minimum sizes; the React page draws at 700.

Overtaken, quoted whole: "The window's floor is 1400 by 900 pixels, set in `src/gui/main_window.py`".

True today: 1400 by 900 is still the figure, and the width is no longer typed out beside it. `main_window.py, in window_min_w_px` answers the wider of `WINDOW_MIN_W_PX`, the window's own figure, and `header_strip_surface.window_min_w`, which is what the header strip's own slots need. Both are read in one place, so the two cannot drift. Six counter cards answer 1400 px and a seventh answers 1476 px. The height stays `WINDOW_MIN_H_PX`, 900 px.

What the clone reading measured

Read off the real window in both builds over a scratch home holding a stand-in `bot_state.json` of the operator's record shape, thirty-eight scrumming records on coinbase at 5m, every socket but loopback refused:

```
reading                                  Qt                      React
Paper on the bar                         second of nine          second of nine
Live modules on the tab before the clone 0 of 6                  0 of 10
Paper modules on the tab after the clone 6 of 6                  10 of 10
strip on Paper at open                   — — — — EXCH 0          — — — — EXCH 0
Import Live Fleet: rows                  38, State: IDLE         38, State: IDLE
Import Live Fleet: Bot ID cell colour    #888888                 rgb(136, 136, 136)
venue seated, EXCH                       Coinbase, 1             Coinbase, 1
paper fleet file                         38 records, all idle    38 records, all idle
bot_state.json hash after every press    unchanged               unchanged
a planted byte on a copy                 hash moved              hash moved
a planted live figure on the Paper strip did not appear          did not appear
a planted paper figure on the Paper strip appeared               appeared
the same paper figure on the Live strip  did not appear          did not appear
Detail                                   7 tabs, Apply refused   7 tabs, Apply refused
old hosts in the import closure          neither                 neither
```

2026-09-20 - #19 - The paper exchange adapter feeds the tab

Unit Q2 of issue #19. The Paper tab now holds one paper exchange adapter, `PaperExchange` in `src/paper/paper_exchange.py`, beside the feed reader the sections above describe. It answers every read a paper bot asks of the venue from the Coinbase Advanced Trade public market endpoints, and no write. Import Live Fleet reads the venue's product list once and each record's ticker once, and every one of those reads is a block on the API Interaction Log in both builds.

The feed reader sentences today overtake two passages

Two passages on this page describe the feed reader `LiveFeedSource`. Each is quoted here and kept above as it stands.

- "The feed reader answers six names and refuses every other one, so a send has no spelling." (Downstream only)
- "The data path is the venue's public market feed, and it goes one way. The reader names the six things it answers, and every other name raises." (The tab as it stands now)

The rule now: the tab's feed reader is `PaperExchange`, which answers the eleven names in its `READ_NAMES` and refuses every other public name. `LiveFeedSource` stays on disk for the two old hosts, which nothing loads, and its `GRANULARITY` and `BAR_SECONDS` tables grew from eight rows to nine, the `4h` row added, because the venue's candles page lists nine granularity names. A `1w` ask of `LiveFeedSource` sent `FIVE_MINUTE` to the venue and answered 5m bars under the weekly name; the adapter rolls `1w` up from daily bars instead.

`src/paper/paper_exchange.py` — the read set

```
READ_NAMES = (
    "venue",
    "product_id",
    "granularity",
    "timeframes",
    "products",
    "ticker",
    "candles",
    "windows",
    "quote_rate",
    "asked_at",
    "calls",
)
```

What the adapter answers, and from which endpoint

`products` reads `GET market/products` and keeps the products the venue trades, by the rule the Market Inspector's `_product_trades` states: `status` online and `trading_disabled` not set. `ticker` reads `GET market/products/{id}/ticker?limit=1` and answers the last trade's price as `last`, with the book's `best_bid` and `best_ask` from the same answer; `last` is None when the venue answers no trade. `candles` reads `GET market/products/{id}/candles` at any of the nine granularity names, at most 350 bars a call, and answers `[ts_ms, open, high, low, close, volume]` rows oldest first. `windows` answers one `candles` window per higher timeframe a record's phantoms name. `quote_rate` answers 1.0 for USD with no call, and otherwise the `{QUOTE}/USD` ticker's `last`, the book's mid when the venue answers no trade; the thirteen USDC records share one `USDC/USD` read.

`src/paper/paper_exchange.py` — the ticker's answer

```
    def _ticker_view(self, symbol: str, entry: TickerEntry) -> dict:
        return {
            "symbol": symbol,
            "product_id": entry.symbol,
            "last": entry.last or None,
            "best_bid": entry.bid or None,
            "best_ask": entry.ask or None,
            "timestamp": entry.timestamp,
        }
```

One rollup, and only what the venue serves

The venue lists no weekly granularity. A `1w` ask reads daily bars in pages of 350, newest page first, until seven days per week asked are held, and rolls them through the stone tablets' `_rollup`, the one rollup every derived tablet timeframe reads through. Each timestamp is moved back by `MONDAY_OFFSET_MS` before the rollup and forward after it, the shift the Market Inspector's `weekly_from_daily` makes, so every weekly bucket starts on Monday 00:00 UTC. A 100-bar weekly window needs 700 daily bars, two calls. A granularity outside the nine is sent as spelled, so the venue's own 400 answers it and lands on the pane as an error block.

`src/paper/paper_exchange.py` — the rollup

```
def weekly_rows(daily: Sequence[Sequence[float]]) -> list[list[float]]:
    shifted = [[row[0] - MONDAY_OFFSET_MS, *row[1:6]] for row in daily]
    return [
        [float(int(row[0]) + MONDAY_OFFSET_MS), *row[1:6]]
        for row in _rollup(shifted, DAYS_PER_WEEK, WEEK_MS)
    ]
```

Cached with Live's two windows, paced at the connector's interval

Every ticker answer is held in a `TickerEntry` and every candle window in a `CacheEntry`, the two slot types Live's `MarketDataPool` holds in `src/exchange/data_pool.py`. A ticker asked again within 5 s is served from the slot with no call and no block. A candle window asked again within one bar of its timeframe, `TF_SECONDS`, is served the same way while the slot holds enough rows. Every venue call waits `PUBLIC_MIN_INTERVAL_S` after the call before it, the figure `src/exchange/market_inspector_fetcher.py` states for the public route and the exchange connector keeps as `_min_request_interval`, 0.1 s; one lock holds the calls in order, so two threads asking at once are spaced. A 429 is recorded as a warning block, waited out for the venue's `Retry-After` seconds when the answer carries one and one interval otherwise, and asked once more.

`src/paper/paper_exchange.py` — the cache and the pace

```
from ..exchange.data_pool import CacheEntry, TickerEntry
from ..exchange.market_inspector_fetcher import (
    API_LEVEL_ERROR,
    API_LEVEL_SUCCESS,
    API_LEVEL_WARNING,
    DAYS_PER_WEEK,
    MONDAY_OFFSET_MS,
    PUBLIC_MIN_INTERVAL_S,
    WEEK_MS,
    _product_trades,
)
```

Every read is a block on the API Interaction Log

The adapter records one entry per venue call on the `APIInteractionLog` its host hands it, the tab's own log, never the process-wide one Live's connector records on. The entry carries Live's fields: the exchange, the action (`FETCH_MARKETS`, `FETCH_TICKER` or `FETCH_OHLCV`, the words Live's connector records), the reason, the venue path as the endpoint, the params, the result, the response time, the level and the data usage. Each host now carries one signal, `apiEntryLogged`, and one receiver beside its `_on_api_event`, `_cross_api_event`, the crossing the main window carries for Live; the host registers the receiver as the log's listener, so an entry recorded on a worker thread lands in `_on_api_event` on the GUI thread through Qt's queued connection and draws as Live's block. The Live tab's pane is not written: the adapter never records on `get_api_log`.

`src/gui/paper/paper_trading_tab.py` — the crossing

```
    def _cross_api_event(self, entry: dict) -> None:
        """The ``api_log()`` listener: emit ``apiEntryLogged``, which Qt queues
        onto the GUI thread for ``_on_api_event`` from any other thread."""
        self.apiEntryLogged.emit(entry)
```

Import Live Fleet reads the venue once

After Import Live Fleet copies the records and fires `fleet_changed`, each host starts one worker thread, `paper-feed-import`, which runs `read_fleet` over the adapter: one `products` read, then one `ticker` read per imported record. Each read lands on the pane as it completes. When the worker ends, its summary crosses a second signal, `feedRead`, and one line lands on the Activity Log naming how many products the venue trades, how many of the fleet's products are among them, how many tickers answered, and how many calls it took; a fleet product the venue does not trade is named on the line, and a product list the venue did not answer is said so, naming no product. The GUI thread never waits on a read.

`src/gui/paper/paper_trading_tab_surface.py` — the line

```
FEED_LINE_FORMAT = (
    "Feed: {products} product(s) trade on {exchange}; {traded} of {records} fleet "
    "product(s) among them; {answered} of {records} ticker(s) answered in {calls} call(s)."
)
FEED_NO_PRODUCTS_FORMAT = (
    "Feed: {exchange} answered no product list; {answered} of {records} ticker(s) "
    "answered in {calls} call(s)."
)
FEED_UNTRADED_FORMAT = " Not traded: {symbols}."
FEED_THREAD_NAME = "paper-feed-import"
```

Every write name is refused

`PaperExchange.__getattribute__` raises `SendRefused` for every public name outside `READ_NAMES`, before any attribute is read, and `__getattr__` raises it for a name the class does not define. `place_order`, `cancel_order`, `create_order` and every other write name have no spelling on the adapter, and a method planted on the class without its name in `READ_NAMES` is refused too. No file under `src/paper/` or `src/gui/paper/` imports a credential loader, `ccxt`, `ScrummingBot`, `BotContainer` or `BotManager`.

`src/paper/paper_exchange.py` — the refusal

```
    def __getattribute__(self, name: str):
        """Answer a private name or one of ``READ_NAMES``; refuse the rest."""
        if not name.startswith("_") and name not in READ_NAMES:
            raise SendRefused(REFUSED_FORMAT.format(read_names=READ_NAMES, name=name))
        return object.__getattribute__(self, name)
```

What the feed reading measured

Read off the real window in both builds over a scratch home holding a copy of the operator's `bot_state.json`, thirty-eight scrumming records on coinbase at 5m, every socket but loopback refused, and a loopback stand-in shaped from the venue's pages: the four endpoints with their fields, 400 for a granularity outside the nine names, empty candles for a listed product with `trading_disabled` set, 350 candles a call at most, a ticker carrying `best_bid` and `best_ask`, and a 429 on a plant. On the commit before this unit, Import Live Fleet put nothing on the pane in either build, the feed reader's ticker answered one price with no bid and no ask, and a `1w` ask sent `FIVE_MINUTE` to the stand-in.

```
reading                                          Qt                       React
blocks on the pane after Import Live Fleet       39                       39 entries, 234 lines
actions                                          1 FETCH_MARKETS,         1 FETCH_MARKETS,
                                                 38 FETCH_TICKER          38 FETCH_TICKER
thread the reads ran on                          paper-feed-import        paper-feed-import
thread violations written                        0                        0
least gap between two stamps                     0.112 s                  0.112 s
gaps under 0.1 s                                 0                        0
the Activity Log line                            38 of 38 tickers,        38 of 38 tickers,
                                                 39 calls                 39 calls
a ticker asked twice within 5 s                  two asks, one call       two asks, one call
a 1w window against its daily rows               equal, Monday buckets    equal, Monday buckets
a 12h ask                                        HTTP 400 error block     HTTP 400 error block
a listed-not-traded product                      [] and a No data block   [] and a No data block
a planted 429                                    one wait of 1000 ms,     one wait of 1000 ms,
                                                 then answered            then answered
place_order, cancel_order, create_order          SendRefused              SendRefused
a method planted without its name                SendRefused              SendRefused
a method planted with its name in READ_NAMES     answered                 answered
bot_state.json hash across the press             unchanged                unchanged
a planted byte on a copy                         hash moved               hash moved
sockets opened outside loopback                  0                        0
```

The stand-in serves the 4h name and a `USDC-USD` ticker with trades because the venue's pages list them; whether the real venue answers `FOUR_HOUR` and whether `USDC-USD` carries trades are read at the real venue, not here.

One real reading followed, on both built bundles against the public host, no credential file present and the adapter's own pace: Import Live Fleet made 39 calls in each build, the product list answering 917 trading products and 8 not trading, all 38 fleet products among them, 38 of 38 tickers answered with their bid and ask, no 429, the least gap between two stamps 0.191 s in the Qt build and 0.168 s in the React build, the 39 calls spanning 11.4 s and 10.7 s. Launched in a Windows AppContainer with no network, the same press made 39 calls in each build, every one refused by the container and every one a block on the pane naming its cause, and the Activity Log line read that the venue answered no product list and 0 of 38 tickers.

One more real reading settled the ninth name against the source. Through the built bundle's own copy of `PaperExchange`, against the public host, no credential file present and the adapter's own pace, one candles ask per name in `GRANULARITY` on `A15-USD`, nine calls, ten bars each:

```
name             status  rows  bar spacing
ONE_MINUTE       200     10    60 s
FIVE_MINUTE      200     10    300 s
FIFTEEN_MINUTE   200     10    900 s
THIRTY_MINUTE    200     10    1,800 s
ONE_HOUR         200     10    3,600 s
TWO_HOUR         200     10    7,200 s
FOUR_HOUR        200     10    14,400 s
SIX_HOUR         200     10    21,600 s
ONE_DAY          200     10    86,400 s
```

The nine calls were 0.271 s apart at least and spanned 2.3 s; no 400 and no 429. The venue answers `FOUR_HOUR`, so the 4h row stays. `src/exchange/timeframes.py` still says the venue ships eight

granularities and no 4h; that file is Live's and this unit does not touch it.

2026-09-20 - #19 - The paper bot ticks and fills on the clone

Unit Q3 of issue #19. Start Paper Run now starts one worker thread that ticks every running paper bot at Live's cadence over PaperExchange, evaluates the shipped gate chains on each worked tick, sizes every scrum and fold with the pure functions Live calls, fills a scrum at the book's best bid and a fold at the best ask with the venue's taker fee, and moves the paper ledger the header strip reads. The old hosts left the tree.

The apply_scrum and apply_fold the earlier sections describe sized a fill with their own arithmetic; the two below are the Simulator's, forked over src/trading/scrumming/sizing.py.

Who runs a paper bot

PaperRunner in src/paper/paper_run.py is the forked shape of the Simulator's _walk_bot: one threading.Thread named RUN_THREAD_NAME (paper-run) per Start Paper Run. Its loop calls advance once, waits TICK_INTERVAL_S (5.0 s, the value ScrummingBot.tick_interval answers) through the one clock seam WallClock.wait, and reads the stop event before each pass. The tape the Simulator walks is replaced by the adapter's live reads.

src/paper/paper_run.py — the loop

```
    def _loop(self) -> None:
        try:
            while not self._stop.is_set():
                self.advance()
                if self._stop.is_set():
                    break
                self._clock.wait(self._stop, self._tick_interval_s)
```

Each pass reads every held record's state through the host's _states map, which the host rebuilds on the GUI thread at every fleet_changed. A record whose state reads running counts one tick; a paused, stopped or idle record is skipped that pass and picked up the pass after the command bar moves it. A running bot is worked every tick_skip ticks, the rule ScrummingBot.tick applies: scrum_read_rate_min minutes over the 5 s tick, a tenth of that in track or fire, and the first tick after a bot starts running always worked, as Live's init tick is. PaperBot in src/paper/fleet_source.py carries scrum_read_rate_min from the record's config, 5 when the config names none.

src/paper/paper_run.py — the skip

```
def tick_skip(scrum_read_rate_min: int, scrum_target_mode: Optional[str]) -> int:
    rate = max(0, int(scrum_read_rate_min or 0))
    if rate <= 0:
        return 1
    base = max(1, int((rate * 60) / max(TICK_INTERVAL_S, 0.1)))
    if str(scrum_target_mode or "") in TRACK_MODES:
        return max(1, base // 10)
    return base
```

A worked tick reads ticker and candles on the runner's thread, never on the GUI thread. The ticker is cached 5 s and the candle window one bar, the two windows src/exchange/data_pool.py gives Live, so a

5m bot's gates read a new window every 300 s while its fill price follows the book every 5 s.

Who evaluates and who sizes

`tick` builds the `GateContext` through the Simulator's `tape_context` over the window's newest close, `bb_reading` and `VotingEngine.compute_all`, and evaluates it with `latch`, unchanged from the sections above. The higher-timeframe bias is None in this unit, so the two defer flags block nothing; unit Q5 widens it.

`apply_scrum` and `apply_fold` are the Simulator's two fills forked over a `FakeBalance`, and every figure comes from `src/trading/scrumming/sizing.py` by name: `scrum_units`, `priced_usd`, `estimated_fee_usd` and `sale_proceeds_usd` on a scrum; `fold_rebuy_factor`, `eligible_fold_tranches`, `cycle_growth_cap_usd`, `fold_cap_remaining_usd`, `plan_fold_consumption`, `fold_rate_taper`, `fold_spend_usd`, `fold_units`, `settle_fold_plan`, `plan_source_price` and `fold_surplus_usd` on a fold. A scrum sells from the highest-priced lot first and queues one fold tranche per lot sold from, in the dict shape Live's `_tick_execute_scrum` builds. A fold is planned under the cycle growth cap, as Live's `tick_phases` plans it, and the target's growth from the surplus is unit Q4's.

Now: the fold reads each tranche against the bot's distance settings, as Live's fold does, and the factor helper named above is gone (`src/paper/paper_run.py, in apply_fold`).

`src/paper/paper_run.py` — the scrum's size and proceeds

```
    units = scrum_units(float(delta), float(price), rule)
    ...
    notional = priced_usd(units, float(price))
    fee = estimated_fee_usd(notional, taker_fee_pct(bot))
    proceeds = sale_proceeds_usd(notional, fee)
```

The unit rule is read once per run through `run_rule`, which is `unit_rule(CLASS_CRYPTO, venue)`: `fractional` on coinbase. A venue with no cited rule starts nothing and writes `run_no_rule_line`.

Who fills, at what price and fee

A scrum fills at the tick's `best_bid` and a fold at the tick's `best_ask`, both from the one `PaperExchange.ticker` read of that tick; never at `last`. A fold's tranche eligibility is read at the tick's `last`, where Live's `fold_tranches.py` reads it. The fee is `estimated_fee_usd` of the notional at `taker_fee_pct`: the record's `trading_fee_pct` when set, else `TAKER_FEE_PCT`, the venue's taker fee at the default tier, 0.6 percent, the rule the Simulator's `DEFAULT_TRADING_FEE_PCT` applies. A scrum's cash gains the proceeds net of the fee; a fold's cash loses the spend plus the fee. Each `PaperTrade` carries the `bid`, `ask` and `last` of its read and the inputs its size came from, `delta_usd` on a scrum, `eligible_usd` and `taper` on a fold, so the paper log can be replayed through the same functions. The fill's stamp is the clock's seconds.

`src/paper/paper_run.py` — the two fills at the book

```
    if armed["scrum_armed"]:
        filled = apply_scrum(bot, balance, bid, now_s, stamp, context.delta, rule)
    elif armed["fold_armed"]:
        filled = apply_fold(bot, balance, ask, last, now_s, stamp, rule)
```

Who holds the money

`FakeBalance` in `src/paper/fake_balance.py` is the fork of the Simulator's `SimPosition`: `units`, `cash_usd`, `fold_tranches` as a list of Live's tranche dicts, `main_lots`, `target_usd`, `anchor_target_usd`, the growth cycle's figures and the trade counters, with `cost_basis_usd` over the lots and `mature_usd` through `mature_profit_usd`. It opens on the bot's first worked tick as the Simulator's `opening_position` opens: `fold_units` of the target at the read's `last` as one lot, plus `cash_usd` of the target, the bot's share of the unbounded budget.

`src/paper/paper_run.py` — the opening

```
def opening_balance(bot: PaperBot, price: float, rule: str) -> FakeBalance:
    target_usd = float(bot.target_usd or 0.0)
    units = fold_units(target_usd, float(price), rule) if price > 0.0 else 0.0
    lots = [{"units": units, "initial_buy_price": float(price)}] if units > 0 else []
    return FakeBalance(
        units=units,
        cash_usd=target_usd,
        opening_price=float(price),
        main_lots=lots,
        target_usd=target_usd,
        anchor_target_usd=target_usd,
    )
```

The ledger opens at the fleet total. `opening_ledger` sets Paper Spendable and Paper Locked to `fleet_target_usd` over every held record; on the operator's 38 records that is $3,698.46. `PaperLedger.figures` keeps both figures whole: each open balance replaces its `anchor_target_usd` share of the opening with its `cash_usd` for Spendable and its `value_usd` for Locked, so a record the runner has not opened still counts its target. No fold is refused for cash; the spend is not capped by the wallet, the `FUNDED_BY_TARGETS` rule the Simulator's Validation runs under, and Spendable reads negative rather than refuse. Paper Realized Profits is the sum of `fold_surplus_usd` over the run's folds, added through `record_close` at each fold.

`src/paper/fake_balance.py` — the two figures over the open balances

```
        for bot_id, balance in held.items():
            price = float(at.get(bot_id, 0.0) or 0.0)
            spendable += balance.cash_usd - balance.anchor_target_usd
            locked += balance.value_usd(price) - balance.anchor_target_usd
```

What crosses to the GUI thread

Both hosts, `PaperTradingTab` in `src/gui/paper/paper_trading_tab.py` and `PaperTradingTabReact` in `src/gui/paper/paper_react_trading_tab.py`, carry four queued signals in the Simulator's shape: `run_line` for each runner line, `run_trade` for each `PaperTrade`, `run_finished` for the run at the thread's end, and `run_figures` for the ledger's figures, which the runner computes on its thread after each worked pass and each fill and the host holds as `_figures`. `aggregate` lays that snapshot over `strip_aggregate`, so the header strip's four money cells read a dict the GUI thread owns and never the runner's balances. The strip repaints from it on the window's 2 s dashboard tick.

src/gui/paper/paper_trading_tab.py — the crossing

```
        self._runner = PaperRunner(
            run,
            self._exchange,
            self._read_states,
            on_trade=self.run_trade.emit,
            on_figures=self.run_figures.emit,
            on_finished=self.run_finished.emit,
            say=lambda line: self.run_line.emit(line, "info"),
        )
```

Start Paper Run and Stop Paper Run

_start_paper_run is the Simulator's _start_run forked. A press with no run up opens paper_run.start over the held fleet under the run rule, starts the runner, writes run_started_line and turns the run button's face to STOP_RUN_BUTTON, "Stop Paper Run", on the corner and on the Get Started card. A fleet holding no record writes RUN_NO_BOT_TEXT and starts nothing. While the run is up the same seat's press reaches _stop_paper_run, which writes RUN_STOPPING_TEXT and sets the event the runner reads; the loop ends at the tick reached, run_finished crosses, and _take_paper_run writes run_ended_line and turns the button back to "Start Paper Run". The balances, the ledger and the trades stay readable on run() after the end. The Qt host relabels its four run buttons; the React host redraws the page through show_tab with run_running on PaperTradingTabState, and the page draws the seat's text and action from the payload as it did before.

src/gui/paper/paper_trading_tab_surface.py — the seat's two faces

```
def run_buttons(running: bool = False) -> tuple:
    if not running:
        return CORNER_BUTTONS
    return (CORNER_BUTTONS[0], STOP_RUN_BUTTON)
```

The command bar is unchanged: Start, Pause and Stop move one record's state through PaperBotManager, and the runner reads that state each pass. Start Paper Run starts the runner whatever the records read, so a bot started on the bar during a run ticks from the next pass.

The old hosts are gone

PaperTraderTabQt, react_paper_trader_tab, paper_trader_tab.js, paper_trader_tab.css and paper_trader_tab_surface are removed. The six names the clone read off the old surface, HEADING, IMPORT_LIVE_FLEET_ACTION, IMPORT_LIVE_FLEET_TEXT, START_RUN_ACTION, START_RUN_TEXT and button_name, live in src/gui/paper/paper_trading_tab_surface.py with STOP_RUN_ACTION and STOP_RUN_TEXT. src/gui/main_tabs/main_window_surface.py reads the tab's name and method there, src/gui/main_tabs/paper_trader_tab.py reads the name there, and src/core/desktop_bridge.py serves the clone's view_model under paper_trading.tab for the Electron shell, one call holding no host. tools/sync_renderer_modules.py rewrote desktop/renderer/module_manifest.js, which now lists the seven paper page modules and no old page.

src/gui/main_tabs/main_window_surface.py — the tab's name

```
from ..paper import paper_trading_tab_surface
...
PAPER_TAB = paper_trading_tab_surface.HEADING
```

The paper log row gains the book and the fill

`paper_row` in `src/paper/paper_log.py` carries a `book` block, the `last`, `bid` and `ask` of the tick's read, and a `fill` block, every field of the `PaperTrade`, beside the gate half and the trade half. The log stays the only file a run writes, under `PAPER_ROOT`. Unit Q6 rebuilds the log on the bus.

What the run reading measured

Read off the real window in both builds, under `pdb`, over a scratch home holding a copy of the operator's `bot_state.json`, thirty-eight scrumming records on coinbase at 5m, every socket but loopback refused, and Q2's loopback stand-in extended with a tape: every product's 5m candles follow one recorded shape whose newest bar advances one bar per stand-in minute from the first candles read, the ticker's last is that bar's close with the bid 0.05 percent under it and the ask 0.05 percent over it, and a 15 s delay can be planted on one ticker answer. The two records started on the bar read `scrumming_interval_pct` 5.0 and `scrum_read_rate_min` 1, so the tape rose about eight percent over five bars in a zig-zag and fell about fourteen over the four after. On the commit before this unit, in both builds, Start Paper Run wrote *"Start Paper Run: no paper run is built; nothing started."*, no thread started, `BUDGET_MULTIPLE` read 2.0, the fleet budget read $7,396.91, twice the target, and `apply_scrum` filled at the price it was handed with no bid and no ask in its signature.

```
reading                                      Qt                          React
Import Live Fleet, rows                      38                          38
Start on two rows through the command bar    both running                both running
thread after Start Paper Run                 paper-run                   paper-run
the started line                             2 of 38 running,            2 of 38 running,
                                             target $3,698.46            target $3,698.46
ledger at the press, Spendable and Locked    3,698.4572 each             3,698.4572 each
the run button while up                      Stop Paper Run, 4 seats     Stop Paper Run, 3 seats
ticks in the first minute, two bots          24 (one per bot per 5 s)    24
worked ticks in the first minute             4 (one per bot per 60 s)    4
first scrum, A34/USDC, bar 51                at 725.294446 = the bid;    the same
                                             last 725.657275
first scrum, A32/USDC                       at 929.390286 = the bid     the same
scrum units against scrum_units on the       0.00241917, equal;          equal, equal
log row's delta, bid and rule                0.00188792, equal
scrum fee against 0.6% of the notional       0.01052768, equal, twice    equal, twice
strip after the scrums                       $3,701.95 / $0.00 /         the same
                                             $3,698.46 / $0.00 / 1
first fold, A34/USDC, bar 57                 at 654.668976 = the ask;    the same
                                             last 654.341805
fold units against fold_units on the row's   0.00038187, equal;          equal, equal
usd, ask and rule                            0.00029801, equal
fold fee against 0.6% of the spend           0.0015, equal, twice        equal, twice
realized per fold, fold_surplus_usd          0.02434372, twice           the same
Realized on the ledger against the sum       0.04868745 = 0.04868745     the same
strip after the folds                        $3,701.44 / $0.05 /         the same
                                             $3,694.04 / $0.00 / 1
fills at last                                0 of 4                      0 of 4
a 15 s delay on one ticker answer            the answer waited 15.0 s;   15.0 s; 378 paints
                                             803 paints over 40 s        over 40 s
the control, the GUI thread held 3 s         0 paints                    0 paints
thread-violation lines                       0                           0
Stop Paper Run                               thread ended; 218 ticks,    the same
                                             20 worked, 2 scrums,
                                             2 folds, fees $0.0241,
                                             realized $0.0487
the run button after the stop                Start Paper Run             Start Paper Run
a planted zero-cash balance, a fold          filled $0.25; cash -0.2515  the same
a fill's price planted to last               read as not at the book     the same
bot_state.json across the four presses       unchanged                   unchanged
a planted byte on a copy                     hash moved                  hash moved
stand-in calls over the run                  79: 1 products, 58 ticker,  79, 0.139 a second
                                             20 candles; 0.14 a second
sockets opened outside loopback              0                           0
```

The stand-in's tape holds fewer than one hundred bars, so the adapter's candle cache never held enough rows and every worked tick read a fresh window; on the real venue a 5m window holds one hundred bars and is served from the cache for 300 s. The scratch copy of `bot_state.json` was rewritten during the run by `main.py`'s `periodic_save`, the live application's own sixty-second state save under the scratch home; no file under `src/paper/` or `src/gui/paper/` writes it.

One line for another item: no element on the React page, Live's `src/gui/web/exchange_tab.js` or the forks, calls the page's own `selectScrumRow`, so a row press selects nothing for the command bar and only a Detail or Fire press selects the row on the model; the reading selected the row through the venue model's `select_scrum_row` and pressed the page's own Start.

What the bundle reading measured

Both builds were made from this unit's commit and launched twice each, `HOME` and the profile variables on a scratch home holding a copy of the operator's `bot_state.json` and no credential file, each press an accessibility Invoke or a button message posted to the launched tree's own window, the window captured by its handle. In a Windows AppContainer with no network, Import Live Fleet made 39 calls in each build, every one refused by the container; a Detail press on two rows, the window it opened closed, and Start on the bar moved the two records to running; Start Paper Run started the runner and turned the seat to Stop Paper Run; over about 97 s the runner made 8 refused reads, 4 tickers and 4 candle windows, and wrote 4 paper rows each carrying *"The live feed answered no candle."*; the strip read $3,698.46 for Spendable and Locked; Stop Paper Run ended the runner and the seat read Start Paper Run again.

The one real reading, both bundles against the public host alone, no credential file, the adapter's own pace, the two same records running:

reading	Qt bundle	React bundle
Import Live Fleet	39 calls, 917 products, 38 of 38 tickers	the same
ticks over the run	52 in 128 s	the same shape
worked ticks	6	6
venue calls during the run	8: 6 FETCH_TICKER, 2 FETCH_OHLCV of 100	8: 6 FETCH_TICKER, 2 FETCH_OHLCV of 100
least gap between two calls	0.258 s	0.257 s
calls a second over 122 s	0.066	0.066
the A34/USDC book on one read	last 0.221450, bid 0.221420, ask 0.221430	last 0.221840, bid 0.221760, ask 0.221840
paper rows written	6, 3 per bot	6, 3 per bot
fills	0, none expected	0
strip while running	$3,698.46 / $0.00 / $3,698.52 / $0.00 / 1	$3,698.46 / $0.00 / $3,698.46 / $0.00 / 1
Stop Paper Run	52 ticks, 6 worked, the seat read Start	the seat read Start Paper Run again
thread-violation lines	0	0

The 100-bar window the venue answers is served from the adapter's cache for 300 s, so the second worked tick of each bot read a fresh ticker and no candles. Locked moved with the venue's last price on the Qt run, from $3,698.46 to $3,698.52, and read $3,698.46 on the React run, whose two prices sat within a cent of their openings. The Activity Log's lines are not readable through UI Automation's text pattern on the React page, so the bundle's own `system.log` and the paper log carry the React reading. ## 2026-09-23 - #19 - The paper bot acts like its live equivalent

Unit Q4 of issue #19. A paper run now writes what it does into the held record: a scrum's tranches reach the record and Detail's Fold Tranches tab, a fold's surplus compounds into the target under the cycle cap, the Target Delta reads the ticker's last trade instead of the newest bar's close, and the row's Trades, Current Position Value and Target move within two seconds of a fill. A bot reads `RUNNING` while the runner holds it and `STOPPED` after Stop Paper Run.

The sentences unit Q4 overtakes

Three passages above describe the run as it was before this unit. Each is quoted here and kept above as it stands.

- "`tick` builds the `GateContext` through the Simulator's `tape_context` over the window's newest close" (Who evaluates and who sizes). `tick` now builds it through `paper_tape_context` in `src/paper/paper_run.py`, which prices the position at the ticker's last trade and leaves the candle window to the indicators, as Live's tick does.
- "A fold is planned under the cycle growth cap, as Live's `tick_phases` plans it, and the target's growth from the surplus is unit Q4's." (Who evaluates and who sizes). This is that unit: `apply_fold` calls `grow_target` after it settles the plan.
- "Q4 draws the tranches; this unit stores them" (What crosses to the GUI thread, and the Q3 comment on the issue). The tranches now reach the record on every snapshot, so Detail draws them mid-run.

The Target Delta reads the ticker

The Simulator's `tape_context` in `src/simulator/back_test.py` prices a position at `window[-1].close`, the only price a stone tablet carries. Live's `ScrummingBot.tick` prices it at the ticker's last trade every five seconds and reads the candle window for the indicators alone. Paper has both, so the Simulator's function is left untouched and `src/paper/paper_run.py` carries its own `paper_tape_context`.

`paper_tape_context` takes the tick's last trade for three fields only: `ticker_last`, the `delta` against the balance's grown `target_usd`, and `mem253_current_pos`. Every other field is unchanged: `bb_pos` comes from `bb_reading` over the window, the indicators from `VotingEngine.compute_all`, and `trend_reading` from the window itself. The same shared helpers size the figures, so only the price source differs.

`src/paper/paper_run.py` — the delta at the ticker

```
    last = float(ticker_last)
    ...
    position_usd = balance.value_usd(last)
    delta = target_delta_usd(position_usd, target_usd)
```

This matters on the real venue. `PaperExchange` caches a ticker for five seconds and a 5m candle window for three hundred, so before this unit the Target Delta could be five minutes old while the book moved. In one stand-in reading the gate priced a position at a bar close of 100.61803399 while the same tick's last trade read 101.83.

A fold compounds its surplus into the target

`apply_fold` settles its plan, books the bought units, and then calls `grow_target`, the Simulator's own function in `src/simulator/back_test.py`. It is imported, not forked: the growth step does not differ between a simulated bot and a paper bot by one figure, and the operator's rule for this issue is that trading logic is shared as pure code and only stateful shells are forked.

`grow_target` reads `fold_surplus_usd` over the consumed slices, holds it to what `cycle_growth_cap_usd` and `fold_cap_remaining_usd` leave of the cycle, applies `target_growth_applied`, adds the applied figure to `target_usd` and to `cycle_cap_consumed_usd`, parks the rest as standing surplus, sets the growth side to `lower`, and records the step on `target_path`. A record with `profit_folding_active` off grows nothing. The cap is `max_target_growth_pct` of the target per cycle, one per cent on the operator's records.

`reset_growth_cycle` runs at the top of every worked tick, before the context is built, so the growth cycle and the gates read the same bar. It is `growth_cycle_side` over the bar's band position: a cycle opened on the lower side zeroes its consumed cap when the band position reaches 0.75, and one opened on the upper side when it reaches 0.25.

`src/paper/paper_run.py` — the growth step inside the fold

```
    realized = fold_surplus_usd(units, slices, float(price))
    grow_target(bot, balance, units, slices, float(price), int(candle_ts_ms))
```

`FakeBalance` in `src/paper/fake_balance.py` gains the two fields `grow_target` writes, `growth_applied_usd` and `target_path`. Every `PaperTrade` now carries `target_usd_after`, the balance's target once the growth has been taken.

Current reading. The surplus a fold books is the units it bought less the units its scrum sold, priced at the fold. Live, Paper and the Simulator read it through one function, so all three book the same figure. A scrum of 0.1 units at $100 and a fold at $98, at a 0.6 per cent fee, book $0.14 of surplus in Paper and in the Simulator alike.

`src/trading/scrumming/sizing.py` — `fold_surplus_usd`

```
asset_at_scrum = sum(units_at_scrum(one) for one in slices)
return max(0.0, priced_usd(float(units_bought) - asset_at_scrum, float(fill_price)))
```

The run writes the record, and the row follows

`PaperRunner` gains one seam, `on_stats`, beside `on_trade` and `on_figures`. `post_stats` builds a `BotStatsSnapshot` through `stats_snapshot`, the Simulator's own builder, so a paper row and a simulated row carry the same keys. The snapshot goes out at three moments: once with `SNAPSHOT_START` and the state `running` on the tick a bot's balance opens, once per worked tick with `SNAPSHOT_TICK` or `SNAPSHOT_FILL`, and once per opened balance with `SNAPSHOT_END` and the state `stopped` when the loop ends.

Both hosts pass `on_stats=self.bot_stats.emit`, a fifth queued signal, so the write lands on the GUI thread in `_take_bot_stats`. That method calls `PaperFleetSource.write_stats`, which merges the snapshot's `stats` into the record's `stats` and its `scrumming_state` into the record's, then arms `_stats_redraw_timer`, a single-shot timer at `STATS_REDRAW_MS`, two thousand milliseconds, Live's own dashboard interval. A snapshot naming a record no longer held is dropped.

The row reads the record, so one write moves every cell that reads it: Current Position Value from `stats.position_value` and the lots, Trades from `stats.total_trades`, and Target from `scrumming_state.target_balance`, the grown target. The five cards on the header strip read the same records through `aggregate_stats`, so Trades, Scrummed and Folded move with the row.

`src/gui/paper/paper_trading_tab.py` — the record write

```
            self._fleet_source.write_stats(
                bot_id, snapshot.stats, snapshot.scrumming_state
            )
```

RUNNING while the runner holds the bot

A snapshot carrying the state `running` moves the record through `PaperBotManager.start` and redraws the rows at once; one carrying `stopped` moves it through `PaperBotManager.stop` and redraws at once. Each writes two Activity Log lines: Live's own ✓ `Bot <id> RUNNING.` or `Bot <id> stopped.`, and then `bot_opened_line` or `bot_ended_line` naming the units, the price, the fills and the target's two figures. The command bar keeps its own Start, Pause and Stop, and `PaperFleetSource.set_state` stays the only writer of a record's state underneath.

Detail draws the run's tranches

Detail's Fold Tranches tab is unchanged code. It draws `PaperBotView`, which reads `scrumming_state.fold_tranches` off the record, and the record now carries the tranches the run's scrums queued, so a Detail window opened during a run draws the open tranche. Live refreshes no open Bot Settings window on a timer and this adds none: the window draws what the record held when it opened.

What the live-equivalent reading measured

Both variants, the real `MainWindow` under `pdb` on a scratch home, every socket but loopback refused, against a loopback stand-in serving 1m bars and a ticker that moves inside each bar. Two records, targets $100 and $200, a 0.5 per cent scrumming interval, phantoms off.

reading	Qt	React
fills in one 215 s run	10	10
row Trades at +1 s after a fill	3	3
row Trades at +2 s after a fill	4	4
row Target before the first fold	$100.0000 / $200.0000	the same
row Target after two folds	$100.0454 / $200.0910	$100.0470 / $200.0942
Current Position Value after a fold	$97.7949 / $195.5608	$97.7313 / $195.4367
growth steps recomputed by hand	4 of 4 equal to 10 decimal places	4 of 4 equal
cycle cap on a $100 target	$1.00	$1.00
Detail Fold Tranches opened mid-run	2 rows	2 tranches
bar close held across two ticks	100.61803399	100.61803399
gate ticker_last across those two ticks	100.00780971 -> 101.90722988	100.00746202 -> 101.88644039
delta across those two ticks	0.00000000 -> 0.15219235	0.00000000 -> 0.10227326
cards: Trades, Scrummed, Folded	10, $5.1931, $5.1620	10, $5.2814, $5.2497
state while the runner held the bot	RUNNING	RUNNING
state after Stop Paper Run	STOPPED	STOPPED
venue calls, all loopback	93	93
non-loopback connections refused	0	0

The same run before this unit filled ten times and moved nothing: Trades stayed `0`, Target stayed `$100.0000`, the Target Delta read the bar's close while the ticker stood 1.8 per cent away, Detail drew no tranche, the cards read zero, and both rows still read `running` after Stop.

Two plants, each of which made a reading fail. With the redraw interval planted to sixty seconds the row held its old cells at +1, +2, +3 and +4 seconds after a fill, and a forced `_redraw_stats` then moved Trades from 3 to 5, so the instrument would have seen the move. With `profit_folding_active` planted off, four folds realised $0.022 to $0.047 of surplus each and the target stayed exactly $100.00000000 and $200.00000000 with an empty `target_path`.

What the bundle reading measured, against the venue

The Qt bundle, built from this branch and launched in isolation: a scratch home with no `bot_state.json` at launch and no credential file at any point, the window driven through the .NET UIAutomationClient assembly, every press an `InvokePattern` invoke or a `SelectionItemPattern` select on an element found under a window matched by its own process id. Two records on pairs the venue lists, `A15/USD` at a $100 target and `A14/USD` at $200, written to the scratch file after launch so no bot is ever restored.

```
moment                                    reading
-------------------------------------------------------------------
Import Live Fleet                         915 product(s) trade on coinbase;
                                          2 of 2 fleet products among them;
                                          2 of 2 tickers answered in 3 calls
row after Import                          paperlive1 A15/USD, Trades 0,
                                          Target $100.0000
Start on the first row                    "✓ Bot paperlive1 RUNNING."
Start Paper Run                           the runner thread up, Bots card 1
Current Position Value at 52 s            $99.9882
                       72 s               $99.9868
                       92 s               $99.9628
                      112 s               $99.9428
                      132 s               $99.9645
                      152 s               $99.9623
Ammo across the same marks                $0.0118 to $0.0377
LOCKED on the header strip                $299.99 down to $299.94, back to $299.97
SPENDABLE / REALISED / MATURE             $300.00 / $0.00 / $0.00
cards: Scrummed, Folded, Trades, Errors   $0.00, $0.00, 0, 0
the A14 row's Target A15 cell             0.002371 then 0.002372
after Stop Paper Run                      $99.9694, Trades 0
bot_state.json across every press         575DED46ED9CB7B7, unchanged
the planted-byte control                  B7821D9854A0EB0F, a different digest
```

No fill was expected inside two minutes and none came; the position value is the live book moving under a fixed holding, which is what a two-minute reading shows.

The call ledger, read off the API Interaction Log pane, carries 28 calls over 150 seconds:

```
13:03:23  FETCH_MARKETS                     Import Live Fleet
13:03:23  FETCH_TICKER  A15/USD             the same press
13:03:23  FETCH_TICKER  A14/USD             the same press
13:03:54  FETCH_TICKER                      the run's first worked tick
13:03:55  FETCH_OHLCV   100 bars at 5m      the same tick, the only one
13:04:00  FETCH_TICKER                      then one every five seconds
   ...                                      to 13:05:53, 24 of them
```

One candle read serves the whole run, because the adapter holds a 5m window for three hundred seconds; the ticker is read once per five-second tick, the cadence `ScrummingBot.tick_interval` sets. That is 0.19 calls a second, and the adapter's own `PUBLIC_MIN_INTERVAL_S` paces them.

Two controls could not be pressed, and the routes are named. The command bar starts the first row and no other: `PaperBotStatusTable.get_selected_bot_id` reads both `selectedItems()` and `currentRow()`, and only the first row's cell answers `HasKeyboardFocus=True`. Four attempts on the second row each reported `IsSelected=True HasKeyboardFocus=False`, and the Activity Log answered `Select a bot first.` or repeated the first bot's line. On the React bundle the Paper panel never reaches the accessibility tree at all: the Paper tab item answers `invoked` to an `InvokePattern` and `selected` to a `SelectionItemPattern`, and no child window contains the tab's own rect, so there is nothing to post a button message to. The window's descendant count stays at 225 through all three routes. Unit T1b of issue 34 read the same wall on the same bundle, and the React readings on this page stand on the source-tree React build.

2026-09-24 - #858 - the bot list upgrades reach the Paper Trader

His line closes the bot-list list on that item: "Transpose these upgrades to Sim and Paper once completed." Three upgrades are in it. Columns that sort. Labels that wrap, with the privacy button centred beneath each one. A row press that moves the Indicator Voting Panel to that bot. All three now draw on the Paper Trader's own Scrumming Bots table, in both builds, over the paper fleet's own figures.

The paper columns sort over the paper fleet's own price

A press on a column heading orders the rows by that column. A second press on the same heading reverses the order. The order is recomputed on every rewrite, so a figure that keeps moving does not unsort the list. The Paper Trader orders by the row's own reading and never by the live price pool, because the table and the page model each answer their own lookups.

`src/gui/paper/paper_bot_status_table.py` - the paper fleet's own sort readings

```
        def _sort_lookups(self) -> SortLookups:
            """``order_statuses``' two readings, over the Paper Trader's figures."""

            def price(status) -> tuple:
                stats = status.get("stats", {}) or {}
                return self._display_price(status, stats)

            def denom_text(
                quote: str, base: str, exchange_id: str, target: float
            ) -> str:
                return self._denom_cell(quote, base, exchange_id, target)[0]

            return SortLookups(price, denom_text)
```

Read off the running program, both builds, at 700, 900 and 1400 pixels wide, over the 38 rows the paper fleet file holds:

reading	window	page
columns that move on a press	9 of 10	9 of 10

reading	window	page
a second press reverses	7 of 9 exactly	yes
the order survives a fresh fleet	yes	yes

The Detail column holds one identical button a row, so it carries no value to order by and is the one column a press does not sort.

The paper labels wrap and the privacy dot sits beneath each one

Each of the ten labels sits inside its own column, over as many lines as it needs, with one blue dot centred under the nine that carry a mask. The label steps down one size when its longest word will not fit the column. A press on the dot masks that column; a press on the words sorts it.

width	Current Position Value, window	the page	dots	largest offset
700	3 lines in 67 px, size 10	46.2 px over a 15.4 px line	9	0.5 px
900	3 lines in 92 px, size 11	30.8 px over a 15.4 px line	9	0.5 px
1400	3 lines in 154 px, size 11	15.4 px over a 15.4 px line	9	0.008 px

No label is cut at any of the three widths, in either build. The tenth column draws no dot and keeps the same empty dot row, so every label sits on one line.

A row press moves the Paper Trader's own Voting Panel

A press on a row draws that bot on the Paper Trader's panel. A second press on the same row takes the panel to its empty state. A press on another row moves the panel to that bot. The panel's own dropdown still picks a bot, and the row it names highlights itself in the list. The panel is the Paper Trader's own, never Live's.

`src/gui/paper/paper_trading_tab.py` - the one bot both sides read

```
        self._bot_list_link = BotListPanelLink(
            self._indicator_panel, self._bot_list_hosts
        )
        self._indicator_panel.bot_selected.connect(self._bot_list_link.panel_moved)
```

Read on the running program, both builds, at all three widths: a first press puts the pressed row's bot on the panel, a second press on that row answers no bot, and a press on another row answers that row's bot.

The paper table is Live's table with three readings of its own

The Paper Trader's window table was a copy of Live's, taken before the header was rebuilt. It is now Live's table with the Paper Trader's three readings over it: the display price, the Target-denomination cell and the sort lookups. The page module carries the same relationship to Live's module, and the model already read the shared surface.

`src/gui/paper/paper_bot_status_table.py` - the row's own price

```
        def _display_price(self, status: dict, stats: dict) -> tuple:
            """The row's own ``current_price``, at ``PAPER_PRICE_AGE_S``."""
            del status
            return float(stats.get("current_price", 0.0) or 0.0), PAPER_PRICE_AGE_S
```

An empty paper fleet still draws the header

With no row at all the header still draws its ten labels and its nine dots, a press on a heading changes nothing and raises nothing, and a press where a row would be finds no row.

reading, no rows	window	page
labels drawn	10	10
dots drawn	9	9
a heading press	no error, still no row	no error, still no row
a row press	answers no bot	finds no row

2026-09-24 - #571 - the asset class group and Add Exchange reach the Paper Trader

The class group is the window's, and the Paper tab draws it

Functional. The segmented asset class group is part of the header strip at the top of the window, not part of any one tab. It is on screen while the Paper tab is in front, and a press on a class button reaches the Paper tab like every other.

Read off the running window with the Paper tab in front, in both builds, at 700, 900 and 1400 pixels wide:

```
Crypto   Stock   Commodities   Forex
```

All four buttons sit inside the window at every one of those three widths. The group holds 142 pixels whatever the window gives it, and each button shortens its own name to the room it has, so a narrow window elides a label and never pushes a button off the right edge.

One class for the whole window. The Paper tab holds no second class of its own. The window keeps the active class, the settings file stores it for the next launch, and the window tells the Paper tab which class that is.

`src/gui/main_tabs/class_filter_tab.py` — the list that reaches Paper

```
SELF_COUNTING_TABS = ("_history_tab", "_simulator_tab", "_paper_trader_tab")
```

OVERTAKEN, quoted whole:

`src/gui/main_tabs/class_filter_tab.py` — *the table that reaches Paper*

```
SELF_COUNTING_TABS = {
    "History": "_history_tab",
    "Sim": "_simulator_tab",
    "Paper": "_paper_trader_tab",
}
```

True today: the tab titles were keys only the removed note needed, so the table is a plain list of the three window attributes. `_refilter_class_tabs` reads it and hands the sector to each tab's own `set_asset_class`.

Paper's Add Exchange button follows the active class

Functional. The Paper tab now carries the Add Exchange button in the two seats Live gives it: the right of the venue sub-tab row, and the Get Started card. It reads the class the header strip has active, and it sits before Import Live Fleet and Start Paper Run in both seats.

`src/gui/paper/paper_trading_tab_surface.py` — the seat both builds read

```
def add_exchange_seat(asset_class: Any = None) -> dict:
    from ..main_tabs import asset_class_surface as acs

    key = acs.normalise(asset_class)
    return {
        "action": ADD_EXCHANGE_ACTION,
        "text": acs.add_exchange_label(key),
        "tooltip": acs.add_exchange_tooltip(key),
        "enabled": acs.add_exchange_enabled(key),
    }
```

Read for each class in turn, on the running program, in both builds:

class	the button reads	it can act
Crypto	■ Add Crypto Exchange	yes
Stock	■ Add Stock Broker	yes
Commodities	Commodities — no venue yet	no
Forex	Forex — no venue yet	no

A class with no venue says so here too. The button states the class, its tooltip names what is missing, and the press is refused. It never offers another class's venue list.

Both builds draw the same button. The words, the tooltip and whether the button can act come from one place, so the Qt widget and the page cannot drift. Read at 700, 900 and 1400, four classes, two seats: 24 readings a build, and every value the same on both sides.

What the Add Exchange press does on the Paper tab

Functional. Paper never opens the live venue settings. A venue page exists on Paper once the paper fleet holds a bot on that venue, so the press offers the venues of the active class that the live fleet file names, and seats the one chosen.

`src/gui/paper/paper_trading_tab.py` — the handler

```
key = acs.normalise(self._asset_class)
if not acs.add_exchange_enabled(key):
    self._status_log.log(acs.class_state(key)["note"], "warning")
    return
self._import_live_fleet(key)
```

Driven once per class on the running program, with the live fleet file naming Coinbase and nothing else. The Activity Log line and the venue count after the press:

class pressed	the Activity Log line	venues seated
Crypto	`Imported 38 bot(s) from bot_state.json on coinbase.`	1
Stock	`bot_state.json names no Stock venue to add.`	unchanged
Commodities	`No configured venue serves Commodities yet.`	unchanged
Forex	`No configured venue serves Forex yet.`	unchanged

Both builds write the same four lines. No venue was contacted: every non-loopback connection was refused during the run, and the live fleet file hashed the same before and after.

The Paper tab keeps its own card while no venue serves the class

Functional. The Paper tab keeps its own page under every sector, served or not. No sector replaces it, hides it or disables it. `src/gui/main_tabs/class_filter_tab.py, in _refilter_class_tabs` hands the sector to the tab's own `set_asset_class`, which narrows the rows the tab lists and nothing else. A sector Paper holds no venue for draws the same page with its Get Started card on it, and the Add Exchange button on that card names the sector.

Read on the running window, one bot standing in for a fleet on Crypto and none on the other five sectors. The cell counts the controls on the page the Paper tab holds, every one enabled and none hidden:

sector	controls on the Paper page
Crypto	20
Stock	20
Commodities	20
Forex	20
Indices	20
Futures / Perps	20

OVERTAKEN, quoted whole:

> ***Functional.*** *The class filter replaces a tab with a note when the tab holds nothing for the class. The Paper tab is left its own card instead while it holds no venue of that class, so the Add Exchange button on that card stays on screen and names the state. That is the rule the Live tab already follows.*
>
> `src/gui/main_tabs/class_filter_tab.py` — *the gate*

```
tab = getattr(self, attr, None)
ask = getattr(tab, "class_venues", None)
if not callable(ask):
    return False
return not ask(key)
```

> *Read on the running window, before and after one Coinbase venue is seated:*

class	venues Paper holds, none seated	keeps its card	after Coinbase	keeps its card
Crypto	none	yes	Coinbase	no
Stock	none	yes	none	yes
Commodities	none	yes	none	yes

class	venues Paper holds, none seated	keeps its card	after Coinbase	keeps its card
Forex	none	yes	none	yes

> *Once the class has a venue on Paper, the note takes over again. A tab that does not answer which venues it holds is untouched by this: the History tab and the Simulator tab draw their notes exactly as before, read on the running window at all three widths under all four classes, 134 values compared and none of them moved but Paper's own.*

True today: the gate is gone. `_tab_keeps_card` is removed from `src/gui/main_tabs/class_filter_tab.py`, and no tab needs an exemption, because no tab is ever replaced. Paper is no longer the one tab that keeps its page — the History tab and the Simulator tab keep theirs on the same rule, and so does every other filtered tab.

One venue list, read by every consumer

The nine broker ids that route a venue to the stock layer were written out four times. The Paper tab held the fourth copy; it now reads the one name the venue map declares, as the Live tab does.

`src/gui/paper/paper_trading_tab.py` — the venue ids

```
self._equity_exchange_ids = acs.EQUITY_VENUES
```

Three sentences this entry overtakes

The manual carries these sentences about the Paper Trader's corner. Each is quoted as it stands, with the sentence that is true today beneath it.

> *"The corner Live gives* ■ `Add Crypto Exchange` *holds Import Live Fleet and Start Paper Run, each at Live's corner-button width."*

The corner holds three buttons: Add Exchange, Import Live Fleet and Start Paper Run, each at Live's corner-button width.

> *"The Get Started card holds the same two where Live's card holds its add button."*

The Get Started card holds the same three, and Live's add button is among them.

> *"The corner holds Import Live Fleet and Start Paper Run in Live's corner-button chrome, and the Get Started card holds the same two at Live's card-button size."*

Both seats hold Add Exchange first, then those two, in the same chrome and at the same sizes.

2026-09-25 - #19 - a paper bot's settings reach its stored record

Apply Changes writes the paper record

The Paper Trader's Bot Settings window drew every field a live bot's window draws, and Apply Changes refused each one. A paper bot could therefore only ever be a copy of a live bot, which left no way to prove a variant before real money touched it. Apply Changes now writes each pending field into the bot's own stored record, and the paper fleet file keeps it.

`src/paper/fleet_source.py` — the write

```
def set_config_field(self, bot_id: str, field: str, value: Any) -> Any:
    wanted = str(bot_id)
    record = self._records.get(wanted)
    if not isinstance(record, dict):
        raise KeyError(f"no paper record is held under {wanted!r}")
    name = str(field)
    if name in RECORD_FIELDS:
        record[RECORD_FIELDS[name]] = value
        return value
```

A field the config does not declare is still refused

The write accepts the fifty-four fields the bot config declares, plus the three the record holds outside it — the phantom switch, its timeframes and the lock count. Any other name is refused, and the window says so on the same line it uses for an applied change.

`src/gui/paper/paper_bot_live_settings_surface.py` — the two lines

```
CHANGE_REFUSED_FORMAT = "Refused {count} change(s) — the Paper Trader sends nothing"
CHANGE_APPLIED_FORMAT = "Applied {count} change(s) to the paper record"
```

What the driven window reported

Both readings came from the real window, with the venue's market read stood in for by a recorded answer. The target balance was changed from twenty-five dollars to seventy-seven dollars fifty, and the row, the record and the file all followed it.

```
no writer   Refused 1 change(s) — the Paper Trader sends nothing
            record 25.0   row 25.0
writer      Applied 1 change(s) to the paper record
            record 77.5   row 77.5   paper fleet file 77.5
React page  Applied 1 change(s)   bot rebuilt at 77.5
```

Creating a paper bot is still the one way in

The venue page's new-bot button writes a line naming the way in, because no paper bot wizard is built. A variant is made by importing the live fleet and changing the imported bot's settings.

`src/gui/paper/paper_trading_tab_surface.py` — the line that button writes

```
NEW_BOT_FORMAT = (
    "+ New Bot on {exchange}: the Paper fleet is loaded through "
    "{way_in}; the paper bot wizard is not built."
)
```

THE FORM THE WIZARD REPLACED. The block above stays here, because a reader who saw that line needs to find it. That constant left the tree. The press now reaches a handler that opens the wizard and hands its settings to the paper fleet source.

`src/gui/paper/paper_trading_tab.py` — the handler that replaced it

```
        wizard_class = surface_class(PAPER_BOT_WIZARD)
        exchanges = wizard_surface.wizard_exchanges(self._exchange_tabs, self._exchange)
        defaults = wizard_surface.stored_defaults()
        markets = wizard_surface.venue_markets(self._exchange)
        wizard = wizard_class(exchanges, defaults, self, markets=markets)
```

2026-09-26 - #19 - a paper fill draws its own Activity Log line

A paper run held each fill and drew none of them. The pane beside the bot table carried the run's start and end and each bot's mark, and a reader watching a run fill ten times read nothing about any of the ten. A fill now draws its own line on the Paper Activity Log, on both hosts.

A fill draws the moment it happens

The line takes the same trade shape the Live Activity Log uses, and it carries the market, the side, the size and the price. The size reads to six decimals and the price to eight, which is what Live's own trade line reads.

`src/gui/paper/paper_trading_tab_surface.py` — the line one fill writes

```
FILL_LINE_FORMAT = (
    "{prefix} {role}{split}{symbol}{split}{stage}"
    "{extra_split}{units:.6f} @ ${price:.8f}"
)
```

The line carries the fill's own time

The stamp is the fill's own moment, never the moment the pane drew it. The runner stamps each fill on its own thread, and the line reads that stamp back instead of asking the clock.

`src/gui/paper/paper_trading_tab_surface.py` — the stamp

```
def fill_stamp(trade: Any) -> str:
    """The stamp a fill's Activity Log line carries: ``TIMESTAMP_FORMAT`` over the
    trade's ``wall_ms``, the fill's own time rather than the clock."""
```

Each record names the side in its own words

One fill leaves two records, and each names the side in its own vocabulary. The pane line says SCRUM or FOLD. The stored row says SELL or BUY. Neither is a drift, and the reason is a different reader at each end. The pane line has to match Live's trade line, because the glyph table keys on Live's two role words. The stored row has to match the reader that reads a Coinbase fill back, because Validation reads a paper row through the reader it reads a live row through.

The paper log module records that reason itself. Its own docstring says the two sides come from the stored-trade module, and the paper log spells neither of them.

`src/paper/paper_log.py` — the two sides, in the stored reader's words

```
#: A scrum sells and a fold buys, in the vocabulary ``YtdTrade.side`` uses.
SIDE_FOR = {SCRUM: SIDE_SELL, FOLD: SIDE_BUY}
```

The pane's two role words come from the table the glyph reads, so a line's role and its glyph cannot disagree.

`src/gui/paper/paper_trading_tab_surface.py` — the pane's two role words

```
FILL_ROLES = {SCRUM: SIDE_ROLES[SELL_SIDE], FOLD: SIDE_ROLES[BUY_SIDE]}
```

What the driven composer answered

One pass drove both functions over two paper trades, a scrum and a fold, with the two fill times five minutes and thirty-three seconds apart.

```
08:22:03 TRADE NOTIFICATION: SCRUM: A15: FILLED — 0.000153 @ $65432.10000000
08:27:36 TRADE NOTIFICATION: FOLD: A15: FILLED — 0.000154 @ $64980.55000000
```

The two stamps hold that gap although one pass drew them both. A stamp read off the clock would have put both lines on the same second, which is the reading that tells the two apart.

2026-09-26 - #19 - a paper bot is created from scratch

The Paper tab could hold a fleet only by copying the live one. Its new-bot button wrote one Activity Log line naming Import Live Fleet and opened nothing. That button now opens the Paper Trader's Bot Creation Wizard on both hosts, and the settings it collects reach `PaperFleetSource.create`, which had been in the tree with no caller at all.

The wizard's shape is the Simulator's, which already had one on both hosts against the same contract. One module holds Paper's own readings, and both hosts read it.

`src/gui/paper/paper_bot_wizard_surface.py` — the request the React fork asks

```
METHOD = "paper_bot_wizard.state"
```

The press that opens the wizard

The route is the same on both hosts and starts on the tab bar. Press Paper, press the venue sub-tab, then press the button beside the bot tables. The Qt tab opens the wizard at once. The React tab opens it one turn later, because a web view opened inside another page's own callback never finishes loading.

`src/gui/paper/paper_exchange_tab.py` — the button the route starts at

```
self._add_bot_btn = QPushButton("+ New Bot")
```

Each host draws the wizard from one registry line

One screen name carries both loaders, as every other forked screen does. The Qt loader answers a five-page window and the React loader answers the dialog that draws those pages in a browser.

`src/gui/variant_surface.py` — the screen and its two loaders

```
  register(PAPER_BOT_WIZARD, _qt_paper_bot_wizard, _react_paper_bot_wizard)
```

The pair list carries the venue's own last price

The venue's product list is what the asset page offers. A product record holds a last price and carries neither a volume nor a volatility, and the page orders the list by name rather than by size. Each row carries that price in the currency the pair prices in, because a pair against Bitcoin prices in Bitcoin and not in dollars. A price at one unit or more reads to two decimal places and a smaller one reads to eight with its trailing zeros cut. A row the venue priced at nothing carries no price at all.

`src/gui/paper/paper_bot_wizard_surface.py` — the label one row carries

```
def price_label(price: Any, quote: str) -> str:
    """The last-price tail an asset row carries, empty when ``price`` is not a
    finite number.

    A price of ``WHOLE_UNIT_PRICE`` or more prints to two places and a smaller
    one to ``SMALL_PRICE_PLACES``, with the trailing zeros cut; the quote is
    named because a row on ``A14/A15`` prices in A15, not in dollars.
    """
```

The wizard's settings become the paper record

A finished wizard hands one settings map to the paper fleet source. The record built from it takes the shape a stored live record has, its state idle, and its own mark naming the wizard as its source rather than a copy. Every reader of that mark answers for a made bot as it answers for a copied one, and the command bar starts and stops it the same way.

`src/paper/fleet_source.py` — the call the wizard's Finish reaches

```
    def create(self, config: dict) -> PaperBot:
        """Hold one record built from the bot wizard's config through
        ``wizard_record`` and answer its ``PaperBot``, its ``bot_id`` the first
        ``BOT_ID_LENGTH`` characters of a ``uuid4`` as a live bot draws its own.
        """
```

What the driven wizard answered

One pass drove each host at three pane widths, with the venue's product read stood in for by a recorded answer, and read the made bot back off the real fleet source beside a copied bot the same pass had loaded.

```
Qt      5 pages    A14/USD made, marked as made here
                   A23/USD held, marked as copied
React   8 of 8 page scripts, 5 page stops, 2 mode rows
                   A14/USD made, marked as made here
both    49 of 49 settings agree, none differs
labels  A15  (64,231.07 USD)   A14  (2,410.55 USD)   A18  (0.00004312 USD)
status  3 USD pairs, 3 carrying a last price
widths  700   900   1400   every control inside the pane
```

Three sentences and one heading this entry overtakes

Each is quoted here and kept above as it stands, with the sentence that is true today beneath it.

> *"A press on + `New Bot` writes one Activity Log line naming Import Live Fleet as the fleet's way in; the paper bot wizard is a later unit's."*

A press on that button opens the Bot Creation Wizard, and its Finish creates the bot.

> *"The venue page's new-bot button writes a line naming the way in, because no paper bot wizard is built."*

That button opens the wizard, and the Activity Log carries the opening line and then the created bot's own line.

> *"A variant is made by importing the live fleet and changing the imported bot's settings."*

A variant is made either way: by changing an imported bot's settings, or by the wizard on a pair no live bot trades.

> *"Creating a paper bot is still the one way in"*

Creating a paper bot is a way in of its own, standing beside Import Live Fleet.

Two code blocks this entry retires

Both blocks name a constant the tree no longer holds. Each stays where it stands, marked as the form the wizard replaced, with today's code beneath it. The two sites are the corner-button passage and the settings-record entry above.

`src/gui/paper/paper_trading_tab_surface.py` — what neither block names any more

```
CORNER_BUTTONS = (
    (IMPORT_LIVE_FLEET_ACTION, IMPORT_LIVE_FLEET_TEXT),
    (START_RUN_ACTION, START_RUN_TEXT),
)
```

2026-10-04 - #1103 - a pushed candidate spawns paper bots

The Market Inspector draws a Push to Paper button beside Push to Sim, on an open Opposing Trades entry and in the topology preview. One press spawns one paper scrumming bot per market the candidate names and registers the wires between them.

The operator's ruling:

> *"For Paper Trader, the pushed topology must spawn the corresponding paper scrumming bots. Paper tab does not have multiple run modes. It pretends to trade against actual exchange data and records the result."*

"If the wiring does not travel then its not a complete or valid topology push."

Paper has no run mode, and none is added

The Simulator holds a candidate under its Back Test mode and leaves that mode in force. Paper has no mode to land a candidate under and gains none here. A pushed bot is simply held, idle, the way a bot the wizard created is held, and Start Paper Run works it against the venue's own data.

`src/paper/fleet_source.py` — the push, on the fleet source itself

```
def push_candidate(
    self,
    candidate: Any,
    exchange_id: str = "",
    ta_timeframe: str = "",
) -> dict:
```

The venue is Paper's own, and so is the timeframe

The Simulator resolves each pushed market to the venue its Stone Tablet was recorded on. Paper reads the venue live, so it needs no tablet: every pushed bot takes the venue `PaperExchange.venue` answers and the timeframe `BotConfig` declares as its own default.

`src/paper/fleet_source.py` — the timeframe is read off the dataclass

```
PUSHED_TIMEFRAME_DEFAULT = str(
    next(
        one.default
        for one in dataclass_fields(BotConfig)
        if one.name == "ta_timeframe"
    )
)
```

A pushed market keeps the quote its symbol names. `BTC/USD` becomes a bot on `BTC/USD` and `BTC/USDC` becomes a bot on `BTC/USDC`. A symbol naming no pair spawns nothing, and one market named twice spawns one bot.

The Target Balance is checked before a bot is held

A topology proposal that names no dollar figure reaches the push as a zero, and a zero takes the same 200 the Bot Wizard offers, read off `BotConfig`. Every other unusable figure refuses the whole push instead of holding a bot that cannot trade: a true or false in place of a dollar amount, an infinity, a not-a-number, a negative figure, and any value that is not a number at all.

`src/paper/fleet_source.py` — what one pushed market's Target Balance may be

```
def pushed_target_usd(value: Any) -> Optional[float]:
```

Driven on the real push, each of those five answered its own refusal with no bot held and no wire registered. Without the check an infinity and a not-a-number each became a bot whose Target Balance read zero, and a true became a bot trading against one dollar.

The wires travel with the bots

Each wire is registered on the fleet's own `PaperWireManager` between the bot ids the push created, and `hold_wire_manager` puts the rows on the fleet so the next `save` carries them in the paper fleet file.

`src/paper/fleet_source.py` — the wires that applied

```
def register_pushed_wires(manager: Any, wires: Any, bot_ids: Any) -> list:
```

A wire that cannot travel refuses the whole push. Five gaps do it: a wire end naming no market, both ends naming the one market, an end naming a market the push did not create, the one pair named twice, and a rate outside the range `register_wire` takes. Each refusal spawns nothing and registers nothing.

A wire row the push cannot read at all counts as naming no market, so it is refused rather than dropped. One pair named twice is refused for the same reason: a pair holds one rate, so keeping the second row would report two wires where the fleet holds one.

`src/paper/fleet_source.py` — the refusal for one pair named twice

```
PUSH_WIRE_TWICE_FORMAT = (
    "{source} to {target} is named twice, and one pair holds one rate"
)
```

The count the push reports is therefore the count the fleet holds.

`src/paper/fleet_source.py` — the four wire refusals

```
PUSH_WIRE_UNNAMED_TEXT = "a wire names no market at one of its ends"
PUSH_WIRE_UNHELD_FORMAT = "{source} to {target}: nothing was pushed for {asset}"
```

What the Activity Log carries

A push that lands writes the spawned line and, when the candidate carried wires, the wired line. A refusal writes its own line and nothing else moves. No line names a run mode.

`src/paper/fleet_source.py` — the line a landed push writes

```
PUSH_SPAWNED_FORMAT = (
    "Pushed {count} paper bot(s) on {venue}, trading the venue's own data: "
    "{assets}."
)
```

A press that reaches no Paper tab is refused by the window, which names the missing tab and spawns nothing.

`src/gui/paper/paper_trading_tab_surface.py` — the refusal with no tab

```
PUSH_NO_PAPER_TAB_TEXT = (
    "Push to Paper refused: the Paper tab did not build, so no paper bot was "
    "spawned."
)
```

A pushed bot is a paper record and nothing else

It reaches the paper fleet file alone. No live bot is created, no order is placed and the live engine and the live logs see nothing of it. A pushed wire is the same: it is registered on `PaperWireManager` and never on the live `SmartWireManager`.

What the driven push answered

The push was driven from the Inspector press through the window's handler to the Paper tab, with the venue read blocked at its edge. A two-market candidate carrying one wire between the two was the input.

read at the Paper tab	before this entry	after it
paper bots held afterwards	0	2
wire rows held afterwards	0	1

Both bots read `coinbase`, `1h`, `scrumming` and `idle`, and both were reached again through `PaperBotManager.get_bot`. The before column is the same script run against the tree as it stood, where the Inspector offers no Push to Paper at all.

One sentence this entry overtakes

It is quoted whole and kept where it stands, on the Market Inspector page, with the sentence that is true today beneath it.

> *"The Paper Trader has no mode a candidate could land under, so no push targets it."*

The Paper Trader still has no mode a candidate could land under, and a push targets it anyway: the candidate's bots are spawned on Paper's own venue rather than held under a mode.

A stored figure no box can hold draws a dash

The Paper Trader's Bot Settings tab and its creation wizard read every stored dollar figure through one admission before the control sees it. `src/trading/container/config.py, in as_box_figure` is that admission. It refuses six kinds: a stored flag, a text number, a not-a-number, an endless number, an integer too large for a decimal, and a finite figure outside the control's own span.

A refused figure leaves the box at the bottom of its span drawing the dash the indicator panel draws for a cell it cannot read. The log names the field. The dash clears the first time the operator moves the box. Never a zero.

`src/gui/paper/paper_settings_tab.py, in PaperSettingsTabMixin` takes `_show_admitted_figure` and `_clear_refused_dash` from Live's own `src/gui/live_settings/settings_tab.py, in SettingsTabMixin`, so Target Balance and Hedge Balance draw what the Live boxes draw. Three of the six kinds used to stop the whole tab from building; all six now build it.

`src/gui/paper/paper_settings_tab.py, in _refresh_target_denom_rows` reads the same admission, so a target the box draws as a dash reaches neither the Target-BTC row nor the Target-ETH row as money.

`src/gui/paper/paper_bot_wizard.py, in PaperTradingParamsPage` calls `src/gui/bot_wizard.py, in _show_stored_target` for the stored default target, and its `isComplete` answers False while that dash is drawn. Finish stays dark until the operator enters a figure, so no paper bot is built on a target nobody chose.

Apply writes nothing for a refused box. `src/paper/paper_bot_view.py, in set_config_field` is the only write path, and it is reached only by a control the operator moved.

The React build reads the same figures. `src/gui/paper/paper_react_bot_wizard.py` loads Live's own `src/gui/web/bot_wizard.js` and `src/gui/paper/paper_react_bot_detail.py` loads Live's own `src/gui/web/live_settings_tab.js`, and `src/gui/paper/paper_live_settings_tab_surface.py, in PaperLiveSettingsTabModel` is Live's model, so the page draws the dash inside the box as Qt draws it.

2026-10-18 - #1203 - one resolver answers a market's sector

Functional. Every tab that follows the asset class reads one chain, and the venue's own recording is the last word in it. A sector whose fleet holds a bot draws that bot. A sector whose fleet holds none draws the same tab with empty lists, and no tab replaces itself with a note.

Diagram: flowchart, left to right

- the class group press -> **set_active**
- set_active -> **bots_of_class**
- bots_of_class -> **bot_class**
- bot_class -> **market_class**
- market_class -> **symbol_class**
- symbol_class -> **the venue recording's asset_class**

`src/gui/main_tabs/class_filter_tab.py, in _apply_asset_class` starts the chain. `src/gui/main_tabs/class_filter_surface.py, in bots_of_class` narrows the fleet, and `src/gui/main_tabs/asset_class_surface.py, in symbol_class` answers the sector the recording holds for the symbol. A symbol the recording holds no sector for answers crypto. An unknown venue answers an empty string.

The sector each market answers

Driven on one venue serving all six sectors, with a recording holding one market per sector:

the recording's sector	`market_class` answers
crypto	crypto
stocks	stocks
commodities	commodities
forex	forex
indices	indices
futures_perps	futures_perps
no sector recorded	crypto

`bots_of_class` keeps 1 of 6 under each of the six sectors, both when the bot status declares its own `asset_class` and when the sector is resolved from the venue and the symbol.

What each tab draws under each sector

Driven through the real `src/gui/main_tabs/class_filter_tab.py, in _apply_asset_class`, with a fleet of six bots, one per sector:

tab	Crypto	Stock	Commodities	Forex	Indices	Futures / Perps
Sim	content	content	content	content	content	content
Paper	content	content	content	content	content	content
Live	content	content	content	content	content	content
Charts	content	content	content	content	content	content
Inspector	content	content	content	content	content	content
Swarm	content	content	content	content	content	content
History	content	content	content	content	content	content
Status	unfiltered	unfiltered	unfiltered	unfiltered	unfiltered	unfiltered
Console	unfiltered	unfiltered	unfiltered	unfiltered	unfiltered	unfiltered

The same six runs against an empty fleet draw nothing in every sector, so a `content` cell is a read and not a fixed answer.

The second resolver is gone

`src/gui/main_tabs/class_filter_surface.py, in market_class` carried a branch for a derivatives sector beside the recording. That branch could not run: `src/gui/main_tabs/asset_class_surface.py, in venue_classes` intersects its answer with the declared sectors, and `src/gui/main_tabs/asset_class_surface.py, in retired_onto` resolves `derivatives` onto crypto, so the name entered neither set. The branch, the `is_derivative` test behind it and its suffix list are removed, and `symbol_class` is the one resolver left.

The sentence above naming crypto is overtaken, and it is kept as written. `retired_onto` now resolves `derivatives` onto futures_perps.

A contract market still answers its own sector. The recording carries `futures_perps` on a perpetual row, which is the sector the taxonomy declares and the sector the Futures / Perps button filters on.

The Sim and the Paper tab build their pages per sector

`src/gui/paper/paper_trading_tab_surface.py, in build_view_model` and `src/gui/simulator/sim_trading_tab_surface.py, in build_view_model` each read `src/gui/main_tabs/trading_tab_surface.py, in layer_order` for their stack pages. Asked for all six sectors, both answer six pages, with the page on show running 0 through 5 as the sector moves. Both tabs are seated in both builds: the React build answers `SimTradingTabReact` and `PaperTradingTabReact`, and the Qt build answers `SimTradingTab` and `PaperTradingTab`. The bar holds nine tabs.

The Paper tab holds no sector list of its own, in either build. The browser build reads the one declared list at the moment the page is asked for, and writes two fields of its payload from that call: the page list, and the position of the page on show inside it.

`src/gui/paper/paper_trading_tab_surface.py` — both fields from one call

```
    key = live.layer_named(layer)
    order = live.layer_order()
```

The Qt build adds one stack page per name on the same list while the tab is built, and numbers the page on show from the same place, so the two builds cannot hold different sets of pages.

`src/gui/paper/paper_trading_tab.py` — one page per declared sector

```
        for layered in acs.layered_classes():
            page, bar, store, card = _make_layer(
                acs.display_name(layered), acs.accent(layered)
            )
```

The declared list is six sectors long, so the Paper tab answers six pages. Its own state object was asked for every sector in turn and the page list was read off the payload the tab serves. The page on show moves across all six asks, which is the control: the figure follows the sector asked and is not a fixed answer. A sector name no layer carries draws page 0, which is the crypto layer.

Sector asked	Pages	Page on show
Crypto	6	0
Stock	6	1
Commodities	6	2
Forex	6	3
Indices	6	4
Futures / Perps	6	5
a name no layer carries	6	0

The Simulator tab reads the same call and answered the same seven rows, figure for figure. The declaration itself and the Simulator's own reading are on the Simulator page under Both tabs build their sector pages from one declared list.

What a failed tab build still does

A tab whose build raises writes one warning line and leaves its window attribute unset. `src/gui/main_tabs/simulator_tab.py, in _build_simulator_tab` and `src/gui/main_tabs/paper_trader_tab.py, in _build_paper_trader_tab` each catch the refusal. Driven against a tab book that refuses the insert, beside the same book accepting as the control:

the tab book	warnings logged	tabs seated
refuses the insert	2	0
accepts the insert	0	2

Nothing reaches the screen for a refused build. The bar comes up short with no mark on it.

The Sim tab's own slot now carries the reason

The Sim tab answers differently today. `src/gui/main_tabs/simulator_tab.py, in SimulatorTabMixin._build_simulator_tab` seats `src/gui/main_tabs/simulator_tab.py, in BuildFailureNote` in the Sim slot when it catches, so the bar keeps nine tabs and the Sim page names the tab and what the build reported. The window attribute still stays unset, so the header strip, the Inspector's push and the sector filter all still read the tab as absent.

The Paper tab is unchanged. A refused Paper build still writes one warning line and leaves its slot empty.

2026-10-20 - #937 - a Paper row's state sits on its Symbol cell

The Paper Trader's Scrumming Bots table is a subclass of the window's own bot list, so its first column is Asset. It draws the target asset's own mark, and the asset's ticker where no mark is kept. It carries no bot id, no state colour and no state tooltip. The state colour and the mode-and-state tooltip sit on the Symbol cell. The Paper Extractor table keeps its own Bot ID column.

One sentence this page carried is overtaken

It is quoted whole, and the sentence under it is true today.

OVERTAKEN, quoted whole:

> *"A row's state is the Bot ID cell's colour and its tooltip's `State:` line; Live's ten columns hold no State column."*

True today: a row's state is the Symbol cell's colour and that cell's tooltip's `State:` line. Live's ten columns still hold no State column.

One reading above names the first column by its old name

The Import Live Fleet reading table holds the row `Import Live Fleet: Bot ID cell colour`, with `#888888` under Qt and `rgb(136, 136, 136)` under React. It is a recorded reading, not a sentence, so it stays exactly where it stands. The cell that carries that grey today is the Symbol cell.

Where the paper wizard's phantom defaults come from

The paper wizard's phantom page opens its tick box and its lock duration from the same two declarations the Live wizard reads. Neither figure is typed into the paper host.

`src/gui/paper/paper_bot_wizard.py` — the paper page falls back to the store's own declared figure

```
            self._stored_lock_candles = int(
                (defaults or {}).get("default_lock_candle_count")
                or AppSettings().default_lock_candle_count
            )
```

A figure set in Settings still wins, and a paper bot's own saved record wins over that, so an existing paper bot keeps the figure it was created with. The declarations themselves, the full reader list and what the counts were before are on the Trading tab page under Where the wizard's phantom defaults come from.

All three wizard phantom pages were built and both boxes read off the built widgets. With nothing stored, each falls back to the declaration and opens on and 2. With off and 9 stored, each opens off and 9, so the boxes follow what is handed to them rather than holding a fixed answer.

Host	What was stored	Tick box	Lock box
Live	nothing	on	2
Paper	nothing	on	2
Simulator	nothing	on	2
Live	off and 9	off	9
Paper	off and 9	off	9
Simulator	off and 9	off	9

The three hosts therefore no longer need comparing.

2026-10-25 - #937 - the Paper page draws the Asset column's mark

The Paper Trader draws its Scrumming Bots table twice. The window draws one, and the page inside the window's browser draws the other. Both now draw the first column the same way. Where the logo library has put a file on disk for that asset, the cell draws that image at 32 pixels and holds no text. Where none is kept, the cell draws the asset's ticker, so the row still names what the bot accumulates. Where the asset database knows that asset's own organisation, the address rides on the cell and a press opens it.

`src/gui/web/paper_bot_status_table.js` — `AssetLogo`

```
  logoProps[PART_ATTR] = LOGO_PART;
  logoProps[LOGO_PATH_ATTR] = text(found[LOGO_PATH]);
  logoProps[LOGO_SIZE_ATTR] = text(found[LOGO_SIZE]);
```

Three invented bots, read off the drawn page

One asset has a kept file. One has none and is named by the asset database. One is named nowhere. Each reads differently from the other two.

```
  a file is kept             no text, one 32-pixel image, no address
  no file, the base is known the ticker, no image, the address on the cell
  the base is known nowhere  the ticker, no image, no address
  row height                 38 pixels, the same on all three pages
```

The page's own report of the payload

Each page checks the payload it was handed and lists what is missing. The three pages name the mark's four cell fields, so a payload that stops carrying one is reported rather than drawn as a blank square.

`src/gui/web/paper_bot_status_table.js` — the audited cell fields

```
  var CELL_FIELDS = [
    TEXT,
    COLOR,
    TOOLTIP,
    ALIGNMENT,
    ALIGNMENT_VALUE,
    LOGO_PATH,
    LOGO_IMAGE,
    LOGO_SIZE,
    LINK_URL,
    CHART_URL,
    UNDERLINE
  ];
```

Driven with every outward connection refused, the reading made no connection attempt and the page made no request. A control attempt afterwards was counted and refused, so the counter was able to report one.

The sentence above that this entry completes

It is quoted whole, and the sentence under it is true today.

Quoted whole:

> *"The Paper Trader's Scrumming Bots table is a subclass of the window's own bot list, so its first column is Asset. It draws the target asset's own mark, and the asset's ticker where no mark is kept."*

True today: that holds for the window's table and for the page, which draws the same mark from the same payload. Neither surface fetches anything while a row draws.

2026-10-25 - #665 - the paper record carries the wizard's lock figure

The paper wizard collected the Candles-to-lock figure and the record it saved held no key for it. The bot built from that record therefore had no lock holder at all, and the Phantom Bots tab fell back to the declared 2 whatever the operator typed.

`src/paper/fleet_source.py, in wizard_record` now writes the figure beside the two phantom keys it already wrote, clamped at 1, which is the clamp the live bot's own constructor applies.

```
    lock_candles = collected.get("lock_candle_count") if scrumming else None
    if isinstance(lock_candles, (int, float)) and not isinstance(lock_candles, bool):
        record["lock_candle_count"] = max(1, int(lock_candles))
```

`src/paper/paper_bot_view.py, in PaperBotView` reads that key to build the bot's lock holder, and the Candles-to-lock box on the Phantom Bots tab reads the holder.

```
wizard 7     record holds the key     tab box 7
wizard 2     record holds the key     tab box 2
no key       record holds no key      tab box 2
```

A paper record written before this change holds no such key, loads, and leaves the tab box on the declared 2.

2026-10-25 - #665 - the Paper wizard reads the one Hedge Balance figure

The Hedge Balance box on the Paper wizard used to open on a figure typed into the Paper wizard itself. It reads the bot's own declaration now, the same declaration the Live wizard, the Simulator wizard and the Electron wizard read.

`src/gui/paper/paper_bot_wizard.py, in PaperTradingParamsPage` — the opening figure

```
self._hedge_amount.setValue(BotConfig.hedge_balance)
```

The Paper despawn box takes the same route: the whole-days rule it reads is the one rule in `src/trading/container/config.py, in whole_despawn_days`, which the Paper Fold Tranches panel reads through the chrome it shares with Live.

Read before and after, the box opened on $200.00 both times. Moving the declaration to a figure nothing uses moved the Paper box with the other three; before the change the same move left all four at $200.00.

2026-10-26 - #665 - the Paper row names the fold taper by size and shows the detonation bar

The Paper fold reads the same taper schedule as Live, and that schedule's name changed. One statement above carries the old name. It stands as written, and this is the reading that replaced it.

written above	what it reads now
"`plan_fold_consumption`, `fold_rate_taper`, `fold_spend_usd`, `fold_units`"	The same four, with the schedule now named `fold_size_taper`. It multiplies the dollars a fold spends, which is what the Paper fold always used it for

`src/paper/paper_run.py, in fold_taper` — the Paper reading

```
return fold_size_taper(ratio_to_ceiling(value, ceiling))
```

The Paper row's status record also carries the detonation confidence minimum now, so the Fire tooltip on a Paper row names the figure a vote has to clear. The Paper table is the Live table with the Paper price feed, so one repair in the Live surface reaches both.

`src/paper/fleet_source.py, in row_status` — the key the Paper row carries

```
"detonation_confidence_min": bot.detonation_confidence_min,
```

Read on the real Paper page model and the real Paper widget, a record carrying 0.60 shows "at confidence 0.60 or above" and a record carrying the opening 0.75 shows 0.75. Before the change both records read the same line, with no figure in it.

2026-10-27 - #1125 - a pair with no recorded row answers no sector at all

The chain above ends at the venue's own recording, and its last step named a sector where the recording named none.

OVERTAKEN, and the sentence and the table row above are kept as written:

A symbol the recording holds no sector for answers crypto.

no sector recorded	crypto

Current reading: `src/gui/main_tabs/asset_class_surface.py, in symbol_class` answers the recorded sector, then the one sector the venue serves where it serves one, and an empty string where the venue serves several and the recording holds no row. A pair whose sector cannot be read is filed under no sector rather than under crypto.

the recording's sector	`market_class` answers
crypto	crypto
stocks	stocks
commodities	commodities
forex	forex
futures_perps	futures_perps
no row at all	nothing

Driven on one venue serving six sectors, with its own recording holding 2029 rows. Every recorded sector answers itself, before and after. The last row is what moved. A venue serving crypto alone and a venue serving stocks alone are the controls: each answers its own sector, and the two answers differ.

A Paper row carries no declared sector of its own, so `src/gui/main_tabs/class_filter_surface.py, in bot_class` resolves it from the venue and the symbol. A Paper bot on a venue whose recording holds its pair answers that pair's sector. A Paper bot on a pair no recording holds now appears under no sector button.

CITED AS ABSENT

The two paths below are named on this page only to record what the Paper tab drew before its rebuild. Neither opens on disk.

- `src/gui/web/paper_trader_tab.js` — the React panel the empty tab registered. The Paper page is `src/gui/web/paper_trading_tab.js` now, hosted by `src/gui/paper/paper_react_trading_tab.py`.
- `src/gui/web/paper_trader_tab.css` — the sheet that page loaded. The Paper page carries `trading_tab.css` and `exchange_tab.css`, named by STYLE_ASSETS in `src/gui/paper/paper_react_trading_tab.py`.

Back to the subsystem index.

Proof of Accumulation

Reference. The design is an on-chain competition layer where bots compete anonymously and the winners take a token on Base, which is Coinbase's layer two. Every instance holds a cryptographic identity, signs each trade into an append-only log, and publishes only a commitment to that log. The strategy stays private and the proof is public.

This is currently proposed as a concept but will likely require the building of a supporting blockchain team for proper / full implementation. This system is designed to enable users of Acervator to compete against each

anonymously via our own Proof of Accumulation blockchain. The idea is to convert trades executed into videogame metrics such as damage to a coliseum style monster or a fellow trader in a 1v1 face off. This further positions the platform as a surgical tool that can be finely tuned and customized to produce intense competition scenarios between entire groups of traders. This, of course, opens the door for actual tokenized Trading Guilds who may require their members to have a certain number of PoA tokens under their belt to join. There will be much more to follow on this as I do intend to scaffold it out for internal testing.

The tab the window does not build

The main window builds no Accumulation tab. The tab name sits in the canonical order and then in the skip list, so the reorder step drops it and the bar carries nine tabs, not ten. Dropping one name from that list is the one edit that puts the tab on the bar.

`src/gui/main_tabs/main_window_surface.py` — the skip list and what it leaves

```
UNBUILT_TABS = (ACCUMULATION_TAB,)

BAR_TAB_ORDER = tuple(name for name in CANONICAL_TAB_ORDER if name not in UNBUILT_TABS)
```

Read at runtime, `BAR_TAB_ORDER` answers Sim, Paper, Live, Charts, Inspector, Swarm, History, Status and Console. The window's own build step is guarded on the same list, so the method that would append the panel is never called.

`src/gui/main_window.py` — the guard, and the method it guards

```
            if ACCUMULATION_TAB not in UNBUILT_TABS:
                self._build_proof_of_accumulation_tab()
```

Read on the current tree, the guard hands the builder to the step timer, and the build reads the same guard. The window calls the builder only under it, so the build leaves the panel module the builder imports, `src.gui.react_proof_of_accumulation_tab`, out of a packaged build. The tab's own surface still ships, because the window's model imports it for the heading.

`src/gui/main_window.py, in MainWindow._setup_ui` — the guard as it reads now

```
            if ACCUMULATION_TAB not in UNBUILT_TABS:
                self._timed_build_step(
                    ACCUMULATION_TAB, self._build_proof_of_accumulation_tab
                )
```

Dropping the name from the skip list puts the panel back in the build as well. The rule is on 17-what-the-build-includes.md.

The Competition tab and the Local Testnet tab are not built either. One mixin assigns `None` to both attributes on every launch.

`src/gui/main_tabs/retired_tabs.py` — the two sentinels

```
        self._competition_tab = None

        self._testnet_tab = None
```

Three tab modules therefore exist and no tab draws. `src/competition/` is the engine behind all three: 47 modules, and the sections below describe what each one does today.

The package entry imports every module and lists its names, so each form is reachable by the package name alone. A name one module repeats from a module bound above it stays unexported, and reaching that one means importing its own module directly. Each module says in its own text which case it is in.

What the screen declares

One module declares the whole screen as data. It is registered in the bridge, so a renderer can ask for the shell by name even with no tab in front of it.

`src/gui/main_tabs/proof_of_accumulation_tab_surface.py` — the shell it declares

```
METHOD = "proof_of_accumulation_tab.state"

HEADING = "Accumulation"
BUILT = True
STATE_TEXT = (
    "The top row is halved: the current Vessel on the left, the Map or the "
    "Encounter on the right. The party window takes the lower half. No pixel art "
    "is drawn."
)
```

That `BUILT` flag reads true and the window still skips the tab, so the flag records an intention and the skip list records the behaviour. The skip list is what runs.

The declared shell halves its top row and takes the lower half for the party. Three zones, seven subtabs, two chains, eight event types and fifteen mechanism controls are declared. `src/gui/web/proof_of_accumulation_tab.js` is the renderer module for those zones.

Declared	Count	Names
Zones	3	Current Vessel, Enemy Screen, Party Window
Subtabs	7	Maps, Character Details, Gear, Resources, Quint, Skills, Guild
Subtab keys	7	Ctrl+1 to Ctrl+7, in that order
Chains	2	Live, Demo TestNet
Event types	8	four modes, each with a Standard and an Elite flag
Mechanism controls	15	identity and its confirm, distil, train, transfer, spend, grade, payout, close, drop, exclusion, reset and its confirm, world and its confirm
Wallet sections	4	Quintessence, Trophies, Loot, Vessels
Party rows	120	40 a page, 3 pages, 8 groups of 5 a page

A player picks a chain, Live or Demo TestNet, then picks one of the eight event types, and every mechanism control runs against that pair. The identity control, the reset control and the world control each ask first and act

on a second press. Every control prints what its mechanism did or the refusal that mechanism raised, and eleven exception types are caught as a refusal rather than a crash. Until this node writes its identity file, every other control refuses for want of a participant.

Each subtab draws off one or two engine modules, and the counts below are what each one puts on the screen.

Subtab	What it draws	Module behind it
Maps	5 grid bounds: 9 squares a side, 81 a layer, 20 layers, 79 participants a layer, 1 square in view	`world_grid`
Character Details	5 stats with their principle and effect, the 10 bands a stat climbs, and the 27 trading metrics	`entity_stats`, `rpg_metrics`
Gear	the loot the wallet holds, 4 item classes with their cohesion, 8 mechanisms	`loot_drop`, `items`
Resources	7 materials, each with the Quintessence band one unit embeds and where that band came from	`materials`
Quint	the chain, its two files, the smallest unit, the supply cap, the minted total	`quintessence_ledger`
Skills	two named pages and the one skill on the ladder, over its 10 levels	`skill_ladder`
Guild	0 guilds, 0 members, the two ranks and the treasury address shape	`guild_roster`

Clicking the current Vessel opens the Character Details subtab, which is also what its shortcut does. The Maps subtab opens only in an event whose mode carries a map. Two of the four modes do — the dungeon crawl and the raid — so four of the eight event types open it and the other four refuse and say why. A request that names no subtab, or names one that cannot open, opens Character Details.

`src/gui/main_tabs/proof_of_accumulation_tab_surface.py` — the refusals caught

```
CONTROL_FAULTS = (
    ActionSpendError,
    LootError,
    OSError,
    OverflowError,
    QuintessenceLedgerError,
    RedistributionError,
    RotationRefusedError,
    SkillLadderError,
    TypeError,
    ValueError,
    WorldGridError,
)
```

Identity

Each instance gets an Ed25519 keypair. The private key signs every trade in the log. The public key is the identity another party verifies against. Strategy parameters are never signed and never published.

`src/competition/bot_identity.py` — `BotIdentity`

```
class BotIdentity:
    """
    Manages a bot's Ed25519 keypair.  The private key never leaves this object
    unencrypted.  The public key is the bot's network-visible identity.
```

The trade log

Each signed trade appends to a Merkle tree. Leaves are the SHA-256 of a trade's canonical bytes, the root is the commitment, and a proof lets a third party confirm one trade sits in the log without receiving the log.

`src/competition/merkle_log.py` — `verify_proof`

```
def verify_proof(leaf_hash: str, proof: List[dict], root: str) -> bool:
    """Verify a Merkle inclusion proof."""
    current = leaf_hash
    for step in proof:
        if step["position"] == "left":
            current = _node_hash(step["hash"], current)
        else:
            current = _node_hash(current, step["hash"])
    return current == root
```

`merkle_root` returns the commitment and `merkle_proof` builds the inclusion proof one trade at a time. No subtab reads that root.

The competition lifecycle

Four phases run in order. A competition runs either locally across instances on one machine and one price feed, or between two machines exchanging signed submissions.

`src/competition/competition_engine.py` — the module's own summary

```
  1. REGISTRATION  — bots register with capital commitment + config hash
  2. ACTIVE        — bots trade; each trade appended to their Merkle log
  3. SUBMISSION    — trading closes; bots submit Merkle root + performance claim
  4. ADJUDICATION  — arbiter verifies submissions, ranks bots, awards tokens
```

The token ledger

The ledger is append-only and idempotent: settling the same result twice writes one record. Balances are replayed from the log and no operation edits one. Ten public methods answer, and the method reporting what is still mintable is `remaining_ever`.

`src/competition/token_ledger.py` — the ten methods `TokenLedger` declares

```
    def award(
    def balance(self, bot_id: str) -> int:
    def awards(self, bot_id: str) -> List[AwardRecord]:
    def total_minted(self) -> int:
    def remaining_ever(self) -> int:
    def season_minted(self, season: int) -> int:
    def leaderboard(self, top_n: int = 20) -> List[dict]:
    def supply_summary(self) -> dict:
    def save(self):
    def load(self) -> "TokenLedger":
```

The hard cap is ten million and the ledger refuses an award that would pass it. Each season awards seventeen twentieths of the season before, which is 85 per cent held as an exact ratio rather than a decimal, with a floor of a hundred.

`src/competition/season_schedule.py` — the supply constants

```
TOTAL_SUPPLY_CAP = 10_000_000  # Hard cap — immutable
GENESIS_SEASON = 1
INITIAL_REWARD = 500_000  # season 1 pool, in ACRV tokens
DECAY_NUMERATOR = 17
DECAY_DENOMINATOR = 20
MIN_SEASON_REWARD = 100
```

Nothing advances the season. The counter lives behind a gated registry function on the chain and no module under `src` reaches it.

The five rarity tiers

Five tiers are awarded on rank within the field. Four carry a lifetime ceiling on how many can ever exist and Harvest carries none, held in the field `max_ever`. Each tier is named for a stage of the alchemical path and pays a fixed number of tokens.

Tier	Stage	Rank	ACRV paid	Ever minted, at most
Harvest	NIGREDO	top 50%	10	no cap
Gold Fold	ALBEDO	top 10%	50	100,000
Bear Slayer	CITRINITAS	top 25% in a verified bear market	100	10,000
Grand Accumulator	RUBEDO	top 1% across three consecutive seasons	500	1,000
Ekthelius	UNIO MYSTICA	perfect score across every metric	10,000	21

The first tier and the last show both cases of the ceiling field.

`src/competition/season_schedule.py` — the uncapped tier, then the capped one

```
    RarityTier(
        name="Harvest",
        emoji="■",
        description="Top 50% of competition field",
        rank_pct_max=0.50,
        condition="Win rate > 50% of competing bots",
        max_ever=None,
        base_value=10,
    ),
RarityTier(
    name="Ekthelius",
    emoji="∞",
    description="Perfect score across all metrics, any season",
    rank_pct_max=0.001,
    condition="100% win rate + top Sharpe + max capital efficiency",
    max_ever=21,
    base_value=10_000,
),
```

The token contract

The ledger above is the platform's own record. On the chain the token is an ERC-20 called Acervator Token. Three properties are fixed at deployment and cannot change afterwards: the supply cap, the one address allowed to mint, and the absence of any way to burn.

Property	Value
Standard	ERC-20, named Acervator Token
Chain	Base, chain id 8453
Hard cap	10,000,000 ACRV, held on the chain
Minting	the CompetitionRegistry contract only
Burning	never

The token takes no address when it is built. One call afterwards names the registry as the minter, a second call to it is refused, and any target that is not itself a contract is refused, so a wallet cannot be named the minter by mistake.

`contracts/ACRV.sol` — the one wiring call

```
    function setRegistry(address registryAddress) external onlyOwner {
        require(registry == address(0),          "ACRV: registry already set");
        require(registryAddress.code.length > 0, "ACRV: registry not a contract");
        registry = registryAddress;
        emit RegistrySet(registryAddress);
    }
```

The contracts are not deployed. Five are deployed together — the token, the competition registry, the trophy, Quintessence and Governance. The registry and the trophy each need the other's address, and `setTrophy` is what cuts that cycle: the registry deploys first and points back at the trophy afterwards. Two npm packages

supply the libraries they build on, and the deploy script takes the network and the signing key. Its own instructions name the Base test network first.

`contracts/deploy.py` — how it is called

```
npm install @openzeppelin/contracts @chainlink/contracts
export ACERVATOR_PRIVATE_KEY=0x...
python deploy.py --network sepolia
```

Nothing needs an address that does not yet exist. The token takes no constructor argument, the registry takes the token's address, the trophy and Quintessence take the registry's, and Governance takes all four. Five wiring calls follow, each running once.

Diagram: flowchart, top to bottom

- 1. ACRV(-> **2. CompetitionRegistry(token, feeds**
- 2. CompetitionRegistry(token, feeds -> **3. ACRV.setRegistry(registry**
- 3. ACRV.setRegistry(registry -> **4. AcervatorTrophy(registry**
- 4. AcervatorTrophy(registry -> **5. CompetitionRegistry.setTrophy(trophy**
- 5. CompetitionRegistry.setTrophy(trophy -> **6. Governance(the four**
- registry

Computing an address before the contract exists does not solve the cycle. A computed address commits to the values handed to the constructor, so computing the registry's address still needs the token's address first, and the circle closes again.

```
address   = keccak256(0xff ++ deployer ++ salt ++ keccak256(init_code))[12:]
init_code = creation bytecode ++ the encoded constructor arguments
```

The build settings are fixed in `foundry.toml`, so the deployed bytecode is the bytecode the analyzers read. The deploy script refuses to run on an artifact `forge build` has not written.

The registry's award function writes every record and both log entries before it pays, so a token contract that called back during the payment cannot find the competition still marked open.

The Quintessence contract

A second contract holds Quintessence on the chain. It divides one unit into ten to the eighteenth parts, has no owner, no pause, no upgrade hook and no burn, and names its minter once when it is built.

Term	Value
Smallest unit	one Quintessence divided into 10^18 parts
Hard cap	33,000,000 Quintessence, as 33 followed by 24 zeros of those parts
Owner	none
Pause	none
Upgrade hook	none
Burn	never

Term	Value
Minting	the Proof-of-Accumulation registry only, named once when built

`contracts/Quintessence.sol` refuses a transfer too small for the recipient to receive one whole smallest unit, rather than paying it as nothing. `tests/contracts/QuintessenceConservation.t.sol` drives the conservation rule.

The buckets move on the chain as they do in the platform's ledger, with one difference: a wallet cannot pay another wallet out of the held bucket. It authorises the payment and the registry runs it.

Governance, the franchise and the halt council

A fifth contract governs the other four. A vote sits at one of four levels, and each level sets its own holding to propose, its own quorum, its own approval share and its own delay before the change takes effect.

Level	Holding to propose	Quorum	Approval	Delay
Informational	1 Quintessence	10%	50%	none
Patch	25 Quintessence	20%	50%	2 days
Interface	75 Quintessence	30%	60%	7 days
Core	150 Quintessence	40%	67%	30 days

The franchise is a second number beside the balance, and it moves no Quintessence. A balance that rises carries the franchise up with it in the same block. A balance that falls leaves the franchise at its remembered ceiling, and that ceiling stands for ninety days after the address's last action before it begins to fall back toward the balance, closing the gap at a ninetieth a day over the next ninety. The ramp is continuous rather than stepped, so the rate holds between days too.

`contracts/Governance.sol` — the franchise rule, in its own words

```
//   franchiseOf(a) >= Quintessence.balance(a), for every address, always
```

The halt council is five elected addresses, three of which halt one named mechanism for seven days. No release call exists: the halted state reads a timestamp, so a halt ends by itself.

`contracts/Governance.sol` — the council and the halt

```
    uint256 public constant COUNCIL_SIZE = 5;
    uint256 public constant HALT_SIGNALS_REQUIRED = 3;
    uint256 public constant HALT_SECONDS = 7 days;
    uint256 public constant HOLD_SECONDS = 90 days;
    uint256 public constant RESYNC_SECONDS = 90 days;
    uint256 public constant BPS_DENOMINATOR = 10_000;
```

Every deployment stays immutable. A change no vote can make is made by deploying a new contract and pointing migration at it on a Core vote. Holders then spend their own units into the migration held address, where Quintessence retires them. An address that never spends keeps its units on this contract and reaches nothing the new one governs.

Trophies

One SVG per tier, picked by name. An unknown tier raises rather than returning an empty drawing, and `generate_preview_html` lays the whole set out on one page.

`src/competition/trophy_generator.py` — `generate_trophy`

```
def generate_trophy(tier: str, data: TrophyData) -> str:
    fn = GENERATORS.get(tier)
    if not fn:
        raise ValueError(f"Unknown tier: {tier!r}")
    return fn(data)
```

Inside `harvest_svg` a text path letters the epigraph's own instruction around the trophy ring, in its usual form: SOLVE ET COAGULA. Each drawing also letters its own alchemical stage across the face, NIGREDO for Harvest through RUBEDO for Grand Accumulator.

Where a trophy's artwork is kept

The artwork and the metadata are held on the chain itself. The token's metadata call returns the JSON inline, and the picture inside that JSON is the tier's drawing, also inline. Nothing points at IPFS, the file-sharing network most NFT projects park their pictures on, and nothing points at any other host, so a trophy lasts as long as the chain does.

`contracts/AcervatorTrophy.sol` — the metadata call

```
            '","image":"', svgB64,

            "data:application/json; base64,",
```

The trophy contract holds the four ceilings, so a tier cannot be over-minted by a caller. `tests/contracts/TrophyTierCaps.t.sol` drives them and `contracts/MetadataLib.sol` builds the inline JSON.

Tier	Trophies ever	Where the ceiling is held
Harvest	no limit	nowhere, by design
Gold Fold	100,000	the trophy contract
Bear Slayer	10,000	the trophy contract
Grand Accumulator	1,000	the trophy contract
Ekthelius	21	the trophy contract

The chain

`local_testnet.py` simulates the whole Base environment in memory, with no wallet and no network. Seven classes answer: four are the chain and its contracts, three are the records those four store.

Class	Holds
`LocalChain`	The blocks

Class	Holds
`LocalACRV`	The ERC-20 balances and the mint history
`LocalRegistry`	Competitions, submissions and adjudications
`LocalTestnet`	The three above, plus a mock oracle and transaction receipts
`Block`	One block's number, hash, parent hash, timestamp and transactions
`TxRecord`	One transaction's hash, block, sender, function, arguments and gas
`ChainEvent`	One event a contract emitted, with its block and its arguments

A block and a transaction are each named by their own contents, so no two records of different content share a name. The two names read different fields.

value	names a transaction	names a block
sender and recipient	yes	no
the call and its arguments	yes	no
gas	yes	no
success flag	yes	no
block number	no	yes
timestamp	no	yes
parent block's name	no	yes
the transactions held	no	yes

One call runs a whole competition and returns the result table, writing nothing outside a temporary directory.

`src/competition/local_testnet.py` — a demo run

```
  from src.competition.local_testnet import LocalTestnet

  testnet = LocalTestnet()
  result = testnet.run_demo_competition(n_bots=3, season=1)
  print(result["results_table"])
```

The token carries eighteen decimals on that chain, the demo price series is drawn from a fixed seed so two runs agree, and a load that finds another schema reports at most five altered records rather than every one.

`src/competition/local_testnet.py` — the chain constants

```
  TOKEN_DECIMALS = 18
  TX_SUCCESS = 1
  GENESIS_BLOCK_ID = "0x" + "0" * 64
  TIMESTAMP_DIGITS = 3
  ALTERED_LOG_LIMIT = 5
  DEMO_PRICE_SEED = 42
```

A checkpoint file and an append log hold the chain between launches. The chain adds what changed rather than rewriting itself, and a checkpoint is written every fifty thousand records so a load does not replay the whole log. The reset control names both files and the bytes they hold before it deletes either.

The real targets sit ready for the day it deploys. The two ABIs are complete and the four address fields are empty, so the calls are already described the day an address arrives.

`src/competition/base_config.py` — the two chains and what they answer

```
Base Mainnet: chain_id=8453  — production
Base Sepolia: chain_id=84532 — testnet (deploy here first)

BASE_MAINNET   acrv_address = ''   registry_address = ''
BASE_SEPOLIA   acrv_address = ''   registry_address = ''
ACRV_ABI       12 entries
REGISTRY_ABI   16 entries
```

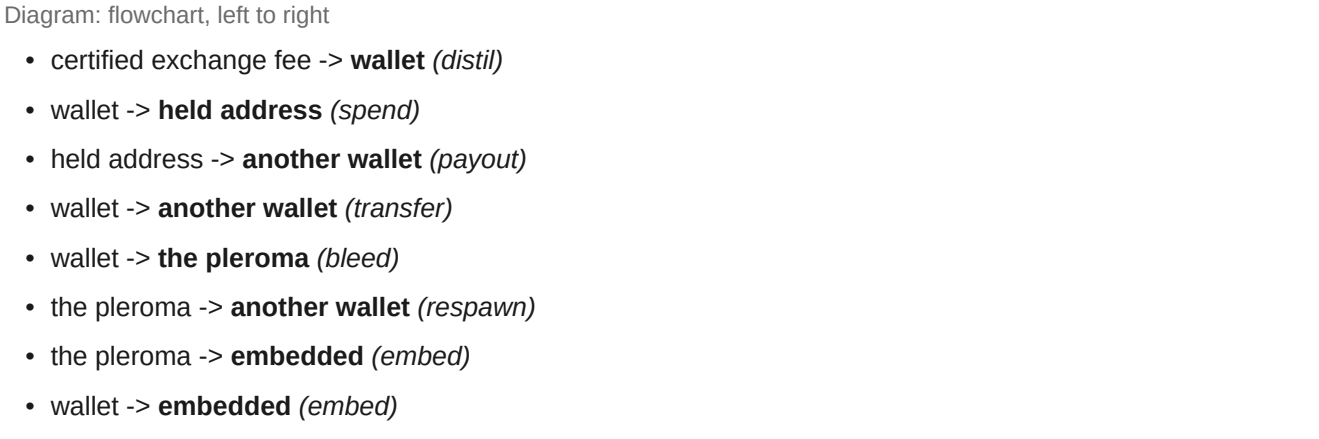
Quintessence

A certified trade distils the platform's second asset. Entering an event spends it, and nothing destroys it. It keeps its own ledger, apart from the token ledger above, because the two obey opposite rules: the token only ever moves outward into a balance, while Quintessence circulates.

`src/competition/quintessence_ledger.py` — the nine operations

```
def distil(self, address: str, fee_usd: object, trade_grade: object) -> Decimal:
def spend(self, address: str, amount: object, held_address: str) -> Decimal:
def transfer(self, sender, recipient, amount, skill_level) -> QuintessenceTransfer:
def payout(self, held_address: str, credits: dict) -> Decimal:
def respawn(self, address: str, amount: object) -> Decimal:
def embed_from_pleroma(self, amount: object) -> Decimal:
def embed_from_wallet(self, address, amount, embedded_amount) -> QuintessenceEmbed:
def release_from_embedded(self, address, amount, recovered_amount) -> QuintessenceRelease:
def release_all_to_pleroma(self, amount: object) -> Decimal:
```

Quintessence can be in exactly four places, and the four always add up to everything ever distilled. A wallet holds what a participant can spend. A held address holds what they have already spent, which rests there and funds later awards. The payout operation moves an award from the held address into the winner's wallet. The pleroma holds what bled out of a transfer and the ledger respawns that to other participants. The embedded bucket holds what a thing in the world carries in itself, drawn out of the pleroma and returned there when the thing is broken.



- embedded -> **another wallet** *(release)*
- embedded -> **the pleroma** *(release)*

Every operation checks that sum before it writes, and refuses the write when it does not balance.

`src/competition/quintessence_ledger.py` — the two conditions the check reads

```
  is_balanced=delta == 0 and not negatives,
  is_within_cap=self._total_ever_minted <= QUINTESSENCE_SUPPLY_CAP,
```

Term	Value
Hard cap	33,000,000 Quintessence, total ever distilled
Pre-ownership	none, for anyone
Distil rate	one Quintessence for one dollar of certified exchange fee
Grade curve	the trade's grade, zero to one, multiplies the award
Destruction	never
Transfer bleed	8% at skill level one, falling to 4% at level ten
Resolution	one Quintessence divides into 100,000,000 minimum units of 0.00000001

The module holds the cap and the rate as constants, and the write path refuses a mint past the cap rather than reporting it afterwards.

`src/competition/quintessence_ledger.py` — the recorded numbers

```
  QUINTESSENCE_SUPPLY_CAP = Decimal(33_000_000)
  QUINTESSENCE_MINIMUM_UNIT = Decimal("0.00000001")
  QUINTESSENCE_UNITS_PER_WHOLE = 100_000_000
  QUINTESSENCE_PER_FEE_USD = Decimal(1)
  BLEED_FRACTION_AT_LEVEL_1 = Decimal("0.08")
  BLEED_FRACTION_AT_LEVEL_10 = Decimal("0.04")
  MIN_TRANSFER_SKILL_LEVEL = 1
  MAX_TRANSFER_SKILL_LEVEL = 10
```

Ten movement kinds are named, one per operation plus the bleed, so every write carries the name of the movement that made it.

One whole Quintessence divides into a hundred million minimum units

The minimum unit is the smallest amount that can exist. A divine essence is potent at a minute amount, so one whole unit carries a hundred million places to hold power in, at the same resolution as the first cryptocurrency.

Every amount a bucket receives is a whole number of minimum units. The amount is rounded down onto that figure as it is written, so no wallet, held address, pleroma or embedded balance can carry a fraction the currency cannot express.

`src/competition/quintessence_ledger.py` — the rounding and the refusal

```
def quantize_quintessence(amount: Decimal) -> Decimal:
    return amount.quantize(QUINTESSENCE_MINIMUM_UNIT, rounding=ROUND_DOWN)


def is_on_quintessence_grid(amount: Decimal) -> bool:
    return amount == quantize_quintessence(amount)
```

The rounding leftover returns to the pleroma, because nothing is destroyed

A bleed of eight per cent down to four per cent does not divide evenly at eight of the ten skill levels. The amount the recipient receives is rounded down and the bleed takes the rest, so the fraction that cannot be paid joins the pleroma rather than vanishing. That keeps the four buckets equal to everything ever distilled, to the unit.

`src/competition/quintessence_ledger.py` — the transfer split

```
received = quantize_quintessence(sent - sent * fraction)
bled = sent - received
```

Skill level	Bleed on 100 Quintessence	Received
1	8	92
2	7.55555556	92.44444444
3	7.11111112	92.88888888
4	6.66666667	93.33333333
5	6.22222223	93.77777777
6	5.77777778	94.22222222
7	5.33333334	94.66666666
8	4.88888889	95.11111111
9	4.44444445	95.55555555
10	4	96

The five stat requirements are whole Quintessence already — one stat measures 550 at level 100 and five of them measure 2,750 — so the resolution changes nothing about the stat scale.

Where the ledger is built

Every launch builds the ledger and attaches it to the window beside the local chain, so a later panel finds it where it finds the chain. The same launch hands that ledger to the certification socket, so the socket and the panels move one ledger and not two.

`src/gui/shared_testnet.py` — the ledger, and the socket that takes it

```
        ledger = cls.install_quint_ledger(quint_ledger_path)
        main_win._quint_ledger = ledger
        main_win._market_rotation = bridge.install_market_rotation(rotation_path)
        main_win._capture_bounds = bridge.install_capture_bounds(
            main_win._market_rotation, bounds_path
        )
        main_win._certification_socket = bridge.install_certification_socket(
            ledger, socket_path, main_win._capture_bounds
        )
```

Fluor is the name Quintessence takes when it pays for a movement on the chain. Every game fee is Quintessence; only the chain movement is called Fluor.

> *"Everything is Quint and we can invent our version of gas using an alchemical term for 'flow' or 'move' or 'fuel'"*

The certified fill

A trade certifies through one method, which signs one fill, logs it, posts it and distils the venue's fee. It refuses a fill already certified.

`src/competition/certification_socket.py` — what the method contracts to do

```
    def certify(self, identity: BotIdentity, fill: CertifiedFill) -> CertificationReceipt:
        """Sign, log, post and distil one fill, and return its receipt.
```

Certification is assembled from parts that already existed and five the socket added.

Part of certification	Where it comes from
Signing one fill	`BotIdentity.sign_trade`
Refusing a wrong competition, a wrong bot, a bad signature	`MerkleTradeLog.append`
The commitment over every certified fill	`MerkleTradeLog.root`
Proof of one fill without the log	`MerkleTradeLog.proof_for`
A summary revealing no fill	`MerkleTradeLog.submission_summary`
The transaction and the event on chain	`LocalChain.send_tx`, `LocalChain.emit`
Minting the award	`QuintessenceLedger.distil`
One chain and one ledger per launch	`SharedTestnetBridge.install_on`
A fill offered for certification	`CertifiedFill`
What one certification produced	`CertificationReceipt`
The replay, stranger and cap refusals	`CertificationSocket.certify`
The lifetime fee total	`lifetime_certified_fee_usd`
The fill subscriber	`attach_to_bus`

Read on the current tree, no launch builds that chain or that ledger. The main window no longer calls the install, so the row above names the factory and not a launch step. Nothing in the running window calls it.

`src/gui/shared_testnet.py` — `SharedTestnetBridge.install_on`, the factory the row names

```
    @classmethod
    def install_on(
        cls,
        main_win,
```

Four refusals guard the mint, and the second column names what each one prevents.

Refused	Without its guard
A fill already certified	the fee total doubles, $4.00 to $8.00
A bot that has certified nothing	it enters an event
A forged signature	500 Quintessence mints for a bot that signed nothing
An award past the 33,000,000 cap	the log keeps a fill that never distilled

The fill event carries the venue's own fee, read back off the settled order rather than estimated from a configured percentage, so a distil mints on what the exchange actually charged. `src/trading/scrumming/execution.py` places the order and `src/exchange/ccxt_connector.py` reads it back; `src/trading/trade_grader.py` supplies the grade that multiplies the award.

The capture bounds on one award

An award is bounded four ways: a market must be in the rotating reward set, the award must name an activation, it must wait out a candle cooldown, and no participant may take more than a fixed share of a market's allotment.

`src/competition/capture_bounds.py` — the bounds and the cooldown

```
COOLDOWN_CANDLES = 3
COOLDOWN_FLOOR_S = 900
MIN_SCORED_AXES = 1
EMISSION_PER_ACTIVATION = None
EXECUTION_READABLE_BPS = 100.0
```

The cooldown is three candles of the bot's own timeframe with a floor of nine hundred seconds, so a fast bot waits the floor and a slow one waits its candles.

Bot timeframe	Three candles	The wait	What decides it
1m	180 s	900 s	the floor
3m	540 s	900 s	the floor
5m	900 s	900 s	they agree
15m	2,700 s	2,700 s	three candles
1h	10,800 s	10,800 s	three candles
1d	259,200 s	259,200 s	three candles

No emission figure exists. `EMISSION_PER_ACTIVATION` holds nothing, so an activation refuses for want of a figure nobody has set. Eleven refusal reasons are named, the missing emission and the missing grade

among them.

The rotating reward set

Which markets may reward Quintessence rotates. The set is drawn from the markets ranked by the dollar value traded in twenty-four hours, and the draw is committed before it is revealed, so no participant can learn the set while the window is open.

`src/competition/market_rotation.py` — the bounds on the draw

```
TOP_N_BY_VOLUME = 20
MIN_ELIGIBLE_POOL = 12
DRAW_DENOMINATOR = 4
MAX_PARTICIPANT_SHARE_PCT = Decimal("5")
AGE_LOOKUP_INTERVAL_S = 13.0
ROTATION_SALT_BYTES = 16
```

One market in four is drawn, the pool refuses to draw below twelve eligible markets, and no participant may take above five per cent of a market's allotment. The ranking reads the dollar value traded, not the coin count.

What the ranking reads	Where it comes from
The dollar value traded in 24 hours	`quoteVolume`, served by the venue
The same figure where a venue omits it	coins traded times the last price
The coin count, still recorded	`volume_24h`, read by the topology proposals

While the window is open, neither a participant, another node, nor the exchange can learn which markets pay; only the node that drew the set holds the answer, on its own disk. Seven states are named for one market: eligible, in rotation, outside the top volume, excluded by the exchange, pool below floor, not drawn, and no open window. An exclusion filed in a season binds at the next season boundary.

The six-month project age rule

A market rewards Quintessence only if its project is at least six calendar months old. The lookup reads the project's start date from the market data service, caches it outside the repository, and answers one of five verdicts.

`src/competition/project_age.py` — the rule and its verdicts

```
MIN_PROJECT_AGE_MONTHS = 6
HTTP_TIMEOUT_S = 15.0
GENESIS_CACHE_FILE_VERSION = 1
OLD_ENOUGH = "old_enough"
TOO_YOUNG = "too_young"
NO_COINGECKO_ID = "no_coingecko_id"
NO_GENESIS_DATE = "no_genesis_date"
LOOKUP_FAILED = "lookup_failed"
```

Six months means six calendar months, not a count of days. Four of the five verdicts are refusals: the project is under six months old, the catalogue has no identifier for it, the service answers with no date, or the call did not answer. The lookup is assembled from parts the platform already had.

Part of the lookup	Where it comes from
The catalogue identifier for an asset	`crypto_assets.ASSETS`
The project's own start date	the market service's coin detail address
A request that refuses any address but http and https	`safe_url.SafeRequest`
Opening that request	`safe_url.safe_urlopen`
Writing the cache file without a torn write	`io_utils.atomic_write_json`
Runtime data outside the repository	the same home folder the rest of the runtime uses
The six-month test	`months_after`
One market's answer	`ProjectAgeVerdict`
The four refusals	`ProjectAgeLookup.verdict_for`

The event modes and the turn

Four modes pair with one Elite flag, so there are eight event types and no mode is written twice. The code is what every other module keys on; the label is what the screen draws beside the Standard or Elite word.

Mode	Label on screen	Shape
`monster_smash`	Fixation	One participant against low to midlevel creatures
`team_monster_smash`	Coagulation	A certified guild, up to 120 against 1
`dungeon_crawl`	Descension	One participant, or a group of six, through a dungeon
`raid`	Cementation	A group of sixty, the most challenging and the most rewarding

The four labels are the names of alchemical operations, which he asked for.

> *"Please rename events to match Hermetic tests or similar..."*

One table indexed by the Elite flag carries every difference between a Standard and an Elite event, so the two cannot drift apart.

`src/competition/poa_modes.py` — the one table the Elite flag indexes

```
STANDARD_RULES = VariantRules(
    STANDARD_SUFFIX, STANDARD_LABEL, STANDARD_TIMEFRAME, 0, 0, 0, 0
)
ELITE_RULES = VariantRules(ELITE_SUFFIX, ELITE_LABEL, ELITE_TIMEFRAME, 1, -1, 1, 1)

VARIANT_RULES: dict[bool, VariantRules] = {False: STANDARD_RULES, True: ELITE_RULES}
```

An Elite event runs on the one-minute candle and a Standard event on the five-minute candle. Elite raises difficulty, loot rarity and drop rate by one step each and lowers the entry fee by one.

Event type	Turn	Difficulty	Entry fee	Loot rarity	Loot drop
`monster_smash`	5m	1	1	1	1
`monster_smash_elite`	1m	2	1	2	2
`team_monster_smash`	5m	2	2	2	2

Event type	Turn	Difficulty	Entry fee	Loot rarity	Loot drop
`team_monster_smash_elite`	1m	3	1	3	3
`dungeon_crawl`	5m	3	3	3	3
`dungeon_crawl_elite`	1m	4	2	4	4
`raid`	5m	4	4	4	4
`raid_elite`	1m	5	3	5	5

A turn is one candle of the market clock, and `turn_at` names the turn that clock is in. An Impetus pool holds the points one turn grants and loses what that turn does not spend.

`src/competition/poa_modes.py` — the Impetus curve

```
  WORLD_TURN_TIMEFRAME = "1h"
  WORLD_TURN_SECONDS = TF_SECONDS[WORLD_TURN_TIMEFRAME]
  IMPETUS_AT_FIRST_LEVEL = 4
  IMPETUS_LEVELS_PER_STEP = 20
  IMPETUS_SPEED_CAP_FACTOR = 2
  IMPETUS_FLOOR = 1
```

One world turn is one hour, read off the shared timeframe table, which is 3,600 seconds. A participant gains one Impetus point a turn every twenty levels.

Level	Impetus a turn
1	4
21	5
40	6
60	7
80	8
100	9

The action bands and the record store

Five bands price every action from the cheapest to the dearest, a hundred to one across the range. A charge may span turns, with a floor of one turn.

`src/competition/action_spend.py` — the bands and what each buys

```
BANDS: dict[str, Decimal] = {
    BAND_X1: Decimal("0.001"),
    BAND_X3: Decimal("0.003"),
    BAND_X10: Decimal("0.010"),
    BAND_X30: Decimal("0.030"),
    BAND_X100: Decimal("0.100"),
}

BAND_ACTIONS: dict[str, str] = {
    BAND_X1: "move, switch weapon, take an item from a bag",
    BAND_X3: "a basic attack, a basic heal",
    BAND_X10: "a class ability on a cooldown",
    BAND_X30: "a group-wide ability, a threat move across the field",
    BAND_X100: "a multi-turn spell, and the decisive tactics beside it",
}
```

A spend rests its cost at one address, the event pot, and a record store holds every action a participant paid for. The store's own file carries a version, so a load that finds another schema says so.

The pot divides on performance

An Elite event's pot returns three quarters of itself to the participants, divided on a normalised performance score. The remainder rests on the chain rather than being rounded away, and a participant with no scored axis takes no share.

`src/competition/event_redistribution.py` — the return share

```
RETURN_PERCENT = 75
SCORED = "scored"
NO_SCORE = "no_score"
```

The division reads performance and never spend: scaling one participant's spend by any factor leaves every payout identical, while halving that participant's score moves them all.

Head to head

`challenge_protocol.py` carries the Elo ladder. A challenger sends a signed challenge, the target accepts or declines, both trade the agreed asset for the agreed duration, and the shared engine adjudicates. The stake flows from loser to winner and both ratings move.

`src/competition/challenge_protocol.py` — the ladder constants

```
DEFAULT_ELO = 1200
ELO_K_FACTOR = 32
MIN_ELO = 100
```

The world grid

A world is a stack of layers, each a square grid. The grid side is not chosen: it is derived from how many records one layer's turn can hold on the chain, so the world is exactly as large as the chain can carry.

`src/competition/world_grid.py` — the derivation, and the figures it fixes

```
WORLD_RECORD_BYTES = 1473
TURN_BYTE_CAPACITY = 1_048_576
TREE_SPHERES = 10
SEPHIROT_LAYERS = TREE_SPHERES * 2
ARENA_LAYER = 0
BASE_VIEWRANGE_SQUARES = 1
SQUARE_STEPS = 100
WORLD_SEED_BYTES = 16
```

Read at runtime, one turn holds 711 records a layer, one participant writes 9 of them, so 79 participants fit a layer and the grid is 9 squares a side holding 81 squares. Twenty layers are declared, two for each of the ten spheres, and layer zero is the arena every participant starts on. One square is a hundred steps across.

Figure	Value
Bytes one world record writes	1,473
Bytes one layer's turn may hold	1,048,576
Records one layer's turn holds	711
Records one participant writes a turn	9
Participants a layer	79
Squares a side	9
Squares a layer	81
Layers declared	20
Squares in view	1
Steps across one square	100

Eight chain functions and seven events are named for the world: create, breach a layer, discover a zone, discover an asset, learn an asset, extract Quintessence and publish the seed. The seed is committed when the world is created and published later, so no participant can read the world ahead of discovering it. Discovery runs at three levels: the square, the zone region on it, and the fact found there.

A terrain grid is separate from the world grid and need not line up with it.

> *"Also, terrain grids do not need to overlap perfectly with the world grid. These grids are varied in size and shape while be linked together at their own borders."*

A zone region carries a boundary of at least three vertices and the participant who discovered it. It carries no terrain kind, so no terrain has a map mark.

Movement, and the world turn's pool

Movement is derived rather than stored. One record opens a journey leg, and where a participant stands is computed from that leg and the clock. A participant who has not moved stands at the arena.

`src/competition/world_movement.py` — what a leg declares

```
MOVEMENT_PRECISION = 28
BASE_STEPS_PER_TURN = SQUARE_STEPS
STANDING_FROM_LEG = "journey leg"
STANDING_FROM_ARENA = "arena"
JOURNEY_FILE_VERSION = 1
```

Where an address stands is answered one of two ways, and the answer names which one it came from. Both end in the locator text every place comparison reads.

Diagram: flowchart, top to bottom

- a consumer asks where an address stands -> **does a covering leg exist?**
- legs -- yes -> **progress_at derives the square and the two steps**
- legs -- no -> **PoaWorld.arena_position**
- progress_at derives the square and the two steps -> **MoverStanding source = journey leg**
- PoaWorld.arena_position -> **MoverStanding source = arena**
- MoverStanding source = journey leg -> **locator text every place comparison reads**
- MoverStanding source = arena -> **locator text every place comparison reads**

A turn grants a pool of steps, and the pool is counted in steps rather than in squares, so a part of a square is expressible.

> *"Just need to be able to say that character A traversed x% of a given square in a given turn and this will make it relatively easy to simulate varied terrain within one or across multiple squares. Should be simple enough to make this dynamic and per character or army or group with encumberance playing a role."*

Encumbrance is the rule that would spend more of the pool for a heavier load.

> *"Its a function of Strength (max weight) and Constitution (turn point penalty while carrying)."*
>
> *"Just need to be able to say that character A traversed x% of a given square in a given turn... Should be simple enough to make this dynamic and per character or army or group with encumberance playing a role."*

`src/competition/world_turn.py` — the pool's bounds

```
STEP_UNIT = "step"
MIN_GRANT_STEPS = 0
MIN_SPEND_STEPS = 1
FIRST_TURN_INDEX = 0
```

Nothing computes an encumbrance penalty. The strength and constitution stats exist and no module reads them for a movement cost.

Dungeons and the second clock

A participant on the world map is on the one-hour world turn. Entering a dungeon moves them to the event turn, which is one to five minutes, and leaving returns them. A dungeon entry refuses a party whose members are not all standing at the entrance.

> *"PoA - Dungeons - World Map - Another example of needed inference. What happens after a player enters a dungeon? These are the sorts of design questions I want you to ask while proceeding. Who. What Where. When. Why. How."*

`src/competition/dungeon_entry.py` — the clock it moves a participant to

```
EVENT_CLOCK = "event turn"
DUNGEON_STORE_VERSION = 1
```

A dungeon is addressed by a whole locator, not by a square alone, so two dungeons on different layers of one square are different places.

The Vessel

A Vessel is the body a participant occupies. Every Vessel a participant holds reads one wallet balance, so the wallet is the only bound on how many a participant may have. A Vessel is named by a 32-byte identity, and every record about a Vessel keys on that identity.

`src/competition/vessels.py` — what a Vessel's standing answers

```
VESSEL_ID_BYTES = 32
MERGED_WITH_PLEROMA = "fully merged with the pleroma"
HOLDS_A_BALANCE = "holds a Quintessence balance"
NEVER_HELD_A_BALANCE = "has never held a Quintessence balance"
VESSEL_COUNT_UNCAPPED = "the wallet balance is the only bound on Vessel count"
```

A level gained keeps the Vessel it was gained on, so a per-class level record is held for each. Nothing gives a Vessel an assignment, gear, equipping, destruction or permadeath.

Stats, and the metrics they convert from

Five stats are declared in one table, each measured in Quintessence, each climbing ten bands. A bot's trading profile converts into twenty-seven RPG metrics.

Declared	Count	Names
Stats	5	strength, dexterity, constitution, intelligence, wisdom
Bands a stat climbs	10	one a sphere
Trading metrics	27	read off the bot's own record
Metrics nothing holds	7	experience, level, character class, gear, enemy, threat, guild

Those seven read nothing because no field under `src` holds them. The stat table carries a principle and an effect for each stat.

`src/competition/entity_stats.py` — the potential scale a stat is read on

```
FIRST_SPHERE = 1
NO_POTENTIAL = Decimal(0)
FULL_POTENTIAL = Decimal(1)
POTENTIAL_PRECISION = 28
```

The seven character classes

Seven classes are declared. Each takes one of three principles and one of four roles, and five assignments cover the role combinations a healer or a support can take.

`src/competition/rpg_classes.py` — the principles, the roles and the arc

```
PRINCIPLES: tuple[str, ...] = (SALT, SULPHUR, MERCURY)
ROLES: tuple[str, ...] = (TANK, DAMAGE, HEALER, SUPPORT)
ARC_LEVELS = 100
FIRST_LEVEL = 1
```

The seven are Lead Ward, Tin Bulwark, Iron Edge, Solar Lance, Quicksilver Draught, Copper Conduit and Silver Mirror, each named for the metal of its planet. A class arc runs a hundred levels from level one.

The skill ladder and the transfer skill

One ladder of ten levels carries every skill. A level costs two and a half times the level below it, starting at one, and each level adds a tenth to the skill's effect. A use is weighted by its quality, and a use of quality zero advances nothing.

> *"All skills have levels that grow through use. Skills should be intelligently designed with growth curves similar to Eve Online."*

`src/competition/skill_ladder.py` — the cost and effect curves

```
UNTRAINED_LEVEL = 0
FIRST_LEVEL_COST = Decimal(1)
LEVEL_COST_MULTIPLIER = Decimal("2.5")
UNTRAINED_EFFECT = Decimal(1)
EFFECT_STEP_PER_LEVEL = Decimal("0.1")
TRANSFER_SKILL_NAME = "Quintessence Transfer"
TRANSFER_HOURS_DIVISOR = Decimal(10)
MIN_TRANSFER_HOURS = Decimal(1)
```

Exactly one skill sits on the ladder, the transfer skill, and it is the skill a Quintessence transfer is gated on.

> *"Quint is transferable between players via a specific skill isolated to common Guild members, takes significant time to complete based on amount and skill level, and has a negative effect of 'bleeding' quint back into the 'platonic' where it can be respawned and redistributed to other PoA participants."*

Three of the four properties he named are built and the fourth is not.

Term	State
gated by a skill	built - an untrained skill is refused

Term	State
lossy	built - 8 per cent at level one, 4 per cent at level ten
slow	the program answers the hours; no clock holds a transfer open
guild members only	nothing exists to check

A thousand Quintessence takes a hundred hours at level one and ten hours at level ten. No in-flight record is kept, so nothing holds a transfer in a queue. Two skill pages are named, one for a Vessel and one for a Reincarnate, and only the ladder carries a skill.

Materials, items and crafting

Seven materials are declared, all metal ores, each embedding a band of Quintessence in one unit. Four item classes are declared, and an item is a list of materials held together by cohesion.

Declared	Count	Names
Materials	7	lead, tin, iron, gold, quicksilver, copper and silver ore
Quality grades	5	Calx, Cauda Pavonis, Flores, Elixir, Magisterium
Item classes	4	armour, weapons, accessories, consumables
Storage classes	3	gear in counted slots, consumables and resources in stacks

`src/competition/conversion_rates.py` — the band one unit of the reference ore embeds

```
  IRON_ORE_QUINTESSENCE_LOW_QUALITY = Decimal("0.00000001")
  IRON_ORE_QUINTESSENCE_HIGH_QUALITY = Decimal("0.00000005")
```

An item's cohesion is the Quintessence that holds it together, and equipping is allowed only while the total Quintessence held covers that cohesion. A craft runs over several world turns, moves Quintessence out of a wallet and destroys none of it, and a finished craft puts its item in the crafting Vessel's store. A craft consumes the material list it names.

`src/competition/items.py` — the two equip answers

```
  EQUIP_ALLOWED = "the total Quintessence covers the item's cohesion"
  EQUIP_REFUSED = "the item's cohesion is above the total Quintessence held"
```

Eight mechanisms an item needs do not exist: a crafting recipe, salvage, equipping, inventory, slot count, encumbrance, loot generation and named instances. `src/competition/items.py` names all eight and says what each waits on. `src/competition/foraging.py` gathers a material from the square a Vessel stands on, and `src/competition/inventory.py` holds what one Vessel carries.

Loot

Five loot tiers share the quality grade names, and their weights sum to exactly a hundred. A qualifying market makes a drop, and the roll is drawn from a fixed seed so a run repeats.

Tier	Weight
Calx	60

Tier	Weight
Cauda Pavonis	25
Flores	11
Elixir	3.5
Magisterium	0.5

`src/competition/loot_drop.py` — the weights and the roll

```
WEIGHT_TOTAL_PCT = Decimal(100)
PERCENT_SCALE = Decimal(100)
LOOT_DROP_SEED = 1155
LOOT_FILE_VERSION = 2
UNITS_PER_DROP = 1
SHORT_FORM_CHARS = 12
```

A trimmed chart, with a tier cut, renormalises as exact fractions rather than decimals, so the remaining bands still total exactly a hundred and keep the odds they had. `tests/contracts/AcervatorLoot.t.sol` drives the loot contract and `contracts/AcervatorLoot.sol` holds it. A drop names the thing that dropped, so a loot record identifies its item rather than a tier alone.

No source names the item type a loot tier yields. A drop therefore takes that mapping from its caller and refuses a drop without one. A drop also names its holder by wallet address and names no Vessel, so the store is handed to the delivery rather than read off the drop.

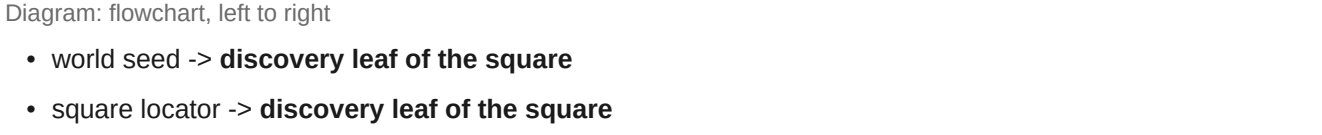
Monsters

Twelve tiers run by signed depth, six ranks below the participant's level and six above. Every tier's own level is unset, so the Quintessence a tier embeds cannot be computed until a level is named.

`src/competition/monster_table.py` — the ranks and the art brief's counts

```
RANKS_PER_SIDE = 6
BRIEF_FRAMES_WITH_CAST = 26
BRIEF_FRAMES_WITHOUT_CAST = 21
BRIEF_ROSTER_FRAMES = 531
BRIEF_DESIGN_COUNT = 21
BRIEF_ATTESTED_NAMES = 21
BRIEF_TOTAL_NAMES = 23
```

Twenty-one designs are attested and twenty-three names are held in all. Nothing ties a tier to a world layer, because twelve does not divide the twenty declared layers. A monster's identity is derived from the committed world seed, the layer and the locator, so two nodes on one seed spawn the same monster in the same place. One sixty-four-digit leaf per square carries five draws, each read out of its own window of that leaf: three of sixteen digits and two of eight.



- discovery leaf of the square -> **digits 0 to 16: committed Quintessence**
- discovery leaf of the square -> **digits 16 to 32: monster tier**
- discovery leaf of the square -> **digits 32 to 48: monster design**
- discovery leaf of the square -> **digits 48 to 56: the material**
- discovery leaf of the square -> **digits 56 to 64: the grade**
- digits 48 to 56: the material -> **the Vessel store**
- digits 56 to 64: the grade -> **the Vessel store**

A world's tier sets the depth it reaches and the Quintessence a slaying imports into the world. `src/competition/world_tier.py` declares the first tier as one and holds no figure for the import.

Alignment, prayer and consecrated places

Alignment runs on one scale from full destruction through neutral to full creation, and it rolls up over three levels.

`src/competition/alignment.py` — the scale

```
CREATION = "creation"
DESTRUCTION = "destruction"
NEUTRAL = Decimal(0)
FULL_CREATION = Decimal(1)
FULL_DESTRUCTION = Decimal(-1)
```

A priest prays, and prayer is the first thing that writes another player's alignment. No record holds a target's consent and the prayer call asks for none. A priest also consecrates a place for a guild, and prayer holds that place blessed or lets it fall cursed. The consecration clock is the world turn.

`src/competition/consecration.py` — the two statuses and the clock

```
BLESSED = "blessed"
CURSED = "cursed"
FULL_CONSECRATION = Decimal(1)
LOST_CONSECRATION = Decimal(0)
CLOCK = "world turn"
```

Domination, and the two forms a taking takes

A Vessel must be incapacitated before it can be taken. Incapacitation comes from health at zero or from a mental effect, and a taking is either temporary control of another player's Vessel or theft of it.

`src/competition/domination.py` — the condition and the two forms

```
CONDITION_ABLE = "able"
CONDITION_INCAPACITATED = "incapacitated"
KIND_PHYSICAL = "physical"
KIND_MENTAL = "mental"
TAKING_CONTROL = "control"
TAKING_THEFT = "theft"
ACTION_CONTROL = "temporary control of another player's Vessel"
ACTION_THEFT = "theft of another player's Vessel"
```

Temporary control ends at a named world turn, so a driven Vessel returns to its owner rather than being held for ever.

The PvP vote

A world enters PvP mode on a player vote, and only in PvP mode may one player destroy another's Vessels.

> *"There will be PvP Worlds and Events. A world entering PvP mode is determined by an active Player Vote and that can put forth once every 24hrs. The voting window persists for 15m or three 5m candles. Only while in PvP mode or participating in PvP events can one player destroy another's Vessels. This will allow players to have specific Vessels they are willing to fight to the death with..."*
>
> *"51% or higher. Proper Democracy over here..."*
>
> *"Three successive pro-PVP votes over 72hrs will lock the World in PvP mode for an entire week starting from the third vote."*

`src/competition/pvp_vote.py` — the three figures he set

```
CASTING_WINDOW_TURNS = 3
VOTE_CADENCE_WORLD_TURNS = 24
RESOLUTION_WORLD_TURNS = 1
CARRY_NUMERATOR = 51
CARRY_DENOMINATOR = 100
CARRIES_TO_LOCK = 3
LOCK_LOOKBACK_WORLD_TURNS = 72
PVP_LOCK_WORLD_TURNS = 7 * 24
```

The casting window is three candles, a vote may be called once every twenty-four world turns, and a vote carries at fifty-one parts in a hundred. Three carries inside seventy-two world turns lock the world in PvP mode for seven days.

Guilds and the treasury

A guild roster holds each guild's members, its officers and the treasury it spends. Two ranks are declared, officer and member, and the treasury is a Quintessence address built from a fixed prefix and the guild's own name.

> *"only those holding PoA tokens can form a guild... guild membership locks and stakes PoA tokens, by guild rank... a guild may hold any number of members"*
>
> *"A guild holds a Quintessence treasury, filled two ways and spent one way. Filled by the transfer skill, from a member, paying the bleed. Filled by the guild's own event awards. Spent on action costs for guild members,*

inside an event. Never paid out to a member's personal wallet."

`src/competition/guild_roster.py` — the ranks and the treasury address

```
ROSTER_FILE_VERSION = 1
TREASURY_ADDRESS_PREFIX = "poa_guild_treasury_"
TREASURY_LEDGER_BUCKET = "wallets"
RANK_OFFICER = "officer"
RANK_MEMBER = "member"
```

The treasury is filled two ways and spent one way, and never pays into a member's personal wallet.

Diagram: graph, left to right

- a member's wallet -> **the guild treasury, wallets bucket** *(transfer skill)*
- a member's wallet -> **the pleroma** *(bleed)*
- the guild treasury, wallets bucket -> **the event pot, held bucket** *(an action's cost)*
- the event pot, held bucket -> **participants' wallets** *(payout by performance)*

The roster saves to a file and the Guild subtab reads it back, so the roster loads empty until something writes that file. **No control founds a guild.** `src/competition/army_command.py` links Raid parties into an army and makes a party leader a General, needing at least two linked Raids.

Linking two nodes

Two instances link over loopback so both chains hold the same records. The link refuses a message above eight mebibytes and gives up on a connection after five seconds.

`src/competition/node_link.py` — the link's bounds

```
LOOPBACK_HOST = "127.0.0.1"
DEFAULT_NETWORK = "acervator-poa"
MAX_MESSAGE_BYTES = 8 * 1024 * 1024
CONNECT_TIMEOUT_S = 5.0
```

A participant is one node holding a Quintessence wallet the ledger knows, and a bot is a source that feeds one. `src/competition/participant_node.py` holds that pairing.

Map marks

Every kind a map draws carries one glyph and one colour token. Three families carry marks and three carry none. The glyphs are stand-ins: no pixel art is drawn.

Family	Marked	Why
Vessel	yes	one planet glyph a class
Loot tier	yes	one glyph a tier
Event mode	yes	one glyph a mode
Monster kind	no	six of the twelve tiers carry no name, and no glyph set covers twelve positions
World fact kind	no	the kind is text a caller passes in, and no module under `src` names one

Family	Marked	Why
Zone terrain	no	a zone region carries no terrain kind

Each colour is a design style token rather than a literal colour, so the marks follow the application's own theme.

Every rate, and its provenance

One table carries every rate that turns one quantity into another, and each rate says where its figure came from. Thirty-eight rates are declared.

Provenance	Rates	What it means
measured	8	the figure came from running something and reading it
decided	3	he named the figure
working	27	a stand-in, and no source sets it

`src/competition/conversion_rates.py` — the three provenances

```
MEASURED = "measured"
DECIDED = "decided"
WORKING = "working"
RATE_ABSENT = None
```

Twenty-seven of the thirty-eight are working figures, so most of the conversion layer awaits a decision or a measurement.

The party row's mark

A party row carries one mark, ranked so the worst state shows. Six marks are ranked and four of the six have no source under `src`, so `participant_mark` answers none of them.

`src/gui/main_tabs/proof_of_accumulation_tab_surface.py` — the six ranked marks

```
MARK_RANKS: tuple[str, ...] = (
    MARK_DEAD,
    MARK_MISSED_WINDOW,
    MARK_OUT_OF_IMPETUS,
    MARK_AFFLICTED,
    MARK_NO_CLASS,
    MARK_ALIGNMENT_SKEW,
)
```

Dead and no class picked are answerable today. Missed the window, out of Impetus, afflicted and alignment skew wait on an action record and on the art brief's own figures.

The two meters, and the reset

Two meters sit beside each other on the screen: how full the current block is, and how much of the turn is left. Together they answer a question neither answers alone.

> *"A block fill and real time turn completion meter next to each other."*

"PoA - Demo Mode - ... Must be able to reset the testnet."

The reset control names both chain files and the bytes they hold, asks first, and acts on a second press. It refuses on the Live chain, as the world control does.

In development.

What the engine does on the Demo chain today

The controls work against the Demo TestNet chain. Pick that chain, press the identity control twice, then the distil control and the spend control. Each prints what its mechanism did, and each writes a file under the runtime directory.

```
the identity control writes   bot_identity.json
the distil control writes     quintessence_ledger_testnet.json
the spend control writes      poa_record_store_testnet.json
```

The wallet over the party window draws what the spend left.

```
distil   Distilled 0.1 Quint.
spend    Band x1 cost 0.001 Quint, resting at poa_elite_event_pot.
wallet   Quint reads -- before the distil and 0.099 after the spend
```

On a chain that has distilled, the Quint subtab names the file as held and reports the figure minted on it.

```
Ledger file      quintessence_ledger_testnet.json - held
Minted so far    0.1
Smallest unit    0.00000001
Supply cap       33000000
```

The skills subtab states the duration and the bleed, the train control records one use and reports the level it reached, and the Guild subtab reports the roster.

```
duration      1000 Quint takes 100 hours at level 1 and 10 hours at level 10.
train         Quintessence Transfer stands at level 1 on 1.0 weighted uses.
standing      level 1 - effect 1.1x - bleed 8.00% - transfers sent 0
guild         Guild, Ctrl+7 - 0 guilds, 0 members, ranks officer and member
return pool   Return pool, 75%
```

The Quint subtab also states what no subtab draws.

```
No block height and no merkle root is drawn. merkle_log writes the trade log
and no subtab reads its root.
```

What is not built

No tab draws any of it. The window skips the Accumulation tab, builds neither the Competition tab nor the Local Testnet tab, and nothing draws a block explorer.

Absent	Count	What is missing
Tabs	3	Accumulation, Competition, Local Testnet
Pixel art	all	every class, loot tier and event mode shows a stand-in glyph
Vessel mechanisms	8	assignments, lifeskilling, crafting, notifications, gear, equipping, destruction, permadeath
Item mechanisms	8	a crafting recipe, salvage, equipping, inventory, slot count, encumbrance, loot generation, named instances
Metrics with no field	7	experience, level, character class, gear, enemy, threat, guild
Party marks with no source	4 of 6	missed the window, out of Impetus, afflicted, alignment skew
Map mark families	3 of 6	monster kind, world fact kind, zone terrain
Unset figures	2	the emission an activation awards, the level each monster tier sits at
Working conversion rates	27 of 38	no source sets the figure

Nothing founds a guild, nothing advances the season, and no contract is deployed on either network. The emission figure is the sharpest gap: an activation refuses for want of it, so no Quintessence award can pass the capture bounds until it is set.

Market Inspector Tab

Reference. The higher-timeframe scanner, the ATA-SMP publisher and the topology proposal pane. First step of the promotion pipeline.

What builds it

`MarketInspectorTabMixin._build_market_inspector_tab` in `src/gui/main_tabs/market_inspector_tab.py` constructs the tab and wires its three injection points before adding it to the row.

The tab splits horizontally. The left half holds the scanner and the ATA-SMP publisher. The right half holds the approval bucket, the proposal cards and the Phantom Bot zone.

The six zones

The screen draws six zones, three down each side. ATA-SMP, Opposing Trades and Multi-Exchange Arbitrage run down the left. ATA-SMP Ready to Send, Bot Swarm Topologies and Phantom Bot HTF Signals run down the right. One function names each side in screen order, so a zone cannot appear on one host and go missing on the other.

`src/gui/main_tabs/market_inspector_surface.py` — the rows each side draws

```
def left_module_rows(
    run: Any,
    scan_state: Any,
    pair_count: Any,
    connectors: Any,
    sector_count: Any = 0,
) -> list:
    """The three left-side regions as key, title and status, in screen order."""
```

The left pane divides its height by a fixed share. ATA-SMP takes two shares and the other two zones take one each.

`src/gui/main_tabs/market_inspector_surface.py` — the heights the zones take

```
LEFT_MODULE_SHARES = (2, 1, 1)
```

Every zone works the same way. It shows one entry at a time, with a left arrow, a right arrow and a line saying which entry is on screen out of how many the zone holds. An arrow press wraps at both ends, so the last entry steps forward to the first.

`src/gui/main_tabs/market_inspector_surface.py` — what one arrow press does

```
def step_to(at: Any, total: Any, by: Any) -> int:
    """The entry index one arrow press moves to, wrapping at both ends."""
```

Clicking the entry opens it. The four expanded lines are the test, the window, the result, and why that answer let the entry through. Every zone's open entry takes one height whatever buttons it carries.

`src/gui/main_tabs/market_inspector_surface.py` — the four names an open entry carries

```
DETAIL_TEST_NAME = "Test"
DETAIL_WINDOW_NAME = "Window"
DETAIL_RESULT_NAME = "Result"
DETAIL_REASON_NAME = "Why it is here"
```

None of the four ever draws a name with nothing beside it. "Why it is here" reads `MethodResult.gate_text`, which names the threshold a verdict cleared, or, for one that ran and did not clear it, the one `failed_reason` names: the p-value above the significance, the trace statistic short of its critical value, or both. "Result" reads `statistic_text`. A test that did not run states no statistic and no reason to state, and both cells draw the dash this tab draws for every figure it cannot state.

`src/trading/pair_selection.py` — what a verdict that did not pass reads

```
GATE_P_ABOVE_FORMAT = "p {p_value:.4f} is above the {significance} required."
GATE_TRACE_SHORT_FORMAT = (
    "The trace statistic {trace:.1f} did not clear the {critical:.1f} required."
)
GATE_NO_REASON_TEXT = "—"
```

A zone holding nothing keeps its waiting sentence as the headline and reports a position of "0 of 0". An empty zone reads as a state rather than as a zone that failed to draw.

Zone	What its line says while it holds nothing
ATA-SMP	"No sector added. Name one and press Scan Now."
Opposing Trades	The scan state: not asked, running, or finished with its three counts
Multi-Exchange Arbitrage	The venues in reach, or that no exchange source is wired
ATA-SMP Ready to Send	"The last scan left no post. Nothing to approve."
Bot Swarm Topologies	One of three sentences, by scan state
Phantom Bot HTF Signals	"Phantom Bot source not wired."

The Phantom Bot HTF Signals row above is overtaken, and it is kept as written. The line now names the state of its source. With no bot lookup wired, it says the source is not wired. With the lookup wired and no bot running a phantom timeframe, it counts the bots it read.

`src/gui/main_tabs/market_inspector_surface.py` — `phantom_zone_text`

```
if bots is None:
    return PHANTOM_HTF_UNWIRED_TEXT
return PHANTOM_HTF_NONE_FORMAT.format(count=len(list(bots)))
```

The Qt build gives every zone a `ProposalStepper`. The two web hosts give every zone a `ZoneStepper`. Both are written from one description of the zone, so a zone's lines are decided in one place.

`src/gui/market_inspector.py` — the Qt widget every zone uses

```
class ProposalStepper(QWidget):
    """One zone's entries shown one at a time, with arrows and a expansion.
```

`src/gui/main_tabs/market_inspector_surface.py` — the rest of the zone description

```
LEFT_MODULE_KEYS = (ATA_SPM_MODULE, OPPOSING_TRADES_MODULE, ARBITRAGE_MODULE)

def right_zone_rows(run: Any, bucket: Any = None) -> list:
    """The three right-side zones as key, title and status, in screen order."""

def zone_view(run: Any, bucket: Any, board: Any) -> dict:
    """One zone's headline, position line, counter, entry rows and actions."""
```

The window sizes each group from `LEFT_MODULE_SHARES`, and the page turns the same tuple into a flex value, reading an equal share for anything that is not a positive number.

`src/gui/web/market_inspector.js` — the share one zone takes on the page

```
const EQUAL_SHARE = 1;
const SHARE_FLEX_TAIL = 0;
function shareFlex(share) { ... }
```

Left half: the scanner

The signal tables

A filter row carries a Refresh button and an "Include active markets" checkbox. The default hides the markets already under a bot, which keeps the table on entry opportunities.

`HTF Signals` lists one row per market: Asset, Signal, Score, Daily, Weekly, Active.

`Opposing Pairs (cointegration, Engle-Granger + Johansen, p<=0.05)` lists Long side, Short side, Method, Window, Statistic, Correlation and the combined score.

The ATA-SMP control row

The zone draws three lines. The ticker field sits on the first line with the sector menu beside it. The four timeframe buttons sit under their own heading on the second. Scan Now, Scan All and Settings sit on the third. Every control sits inside the pane from 700 pixels of tab width upward, with no scroll bar.

`src/gui/main_tabs/market_inspector_surface.py` — the three line names

```
TIMEFRAME_TITLE = "Timeframe"
SECTOR_ROW_PART = "sector-row"
SCAN_ROW_PART = "scan-row"
```

Each group of buttons breaks at the count the pane's own width holds. The window asks the same function the page asks, so both hosts wrap at one count.

`src/gui/main_tabs/market_inspector_surface.py` — how many cells fit a line

```
def columns_for(
    available: int, cell: int, spacing: int = SETTINGS_ROW_SPACING_PX
) -> int:
```

Both hosts read the same pane width, because the window sets the splitter handle to the theme's own value rather than to the toolkit's default, and the page leaves the same gap.

Tab width	Venue and category buttons on a line	Settings rows on a line
700	2	1
900	3	1
1400	6	2
1960	8	2

`src/gui/main_tabs/market_inspector_surface.py` — the gap between the two panes

```
SPLITTER_HANDLE_PX = 7
BUTTON_COLUMNS = 4
```

The timeframe grid wraps after `BUTTON_COLUMNS` cells, as the Level 1 groups do. A ticked timeframe, a held venue credential and the chosen asset class are one state and take one look, and the field's own lines take one size in both hosts.

`src/gui/main_tabs/market_inspector_surface.py` — the look a pressed control takes

```
CHECKED_BUTTON_STYLE = (
    f"background:{ds.PRIMARY}; color:{ds.ON_PRIMARY}; border-color:{ds.PRIMARY};"
)
DETAIL_STYLE = "color: #ccc; font-size: 11px;"
```

Every row of the field takes that one size, in both hosts.

The asset classes and their sectors

Four classes ship. The published taxonomies name them: the S&P GSCI methodology makes commodities an asset class with sectors under it, and ISO 10962, the CFI standard, sorts each product by its own form rather than by a class of its own.

> *S&P GSCI groups energy as petroleum and natural gas*

`src/trading/ata_spm.py` — the class set every button, menu and scan reads

```
ASSET_CLASSES = (CLASS_CRYPTO, CLASS_STOCKS, CLASS_COMMODITIES, CLASS_FOREX)
```

OVERTAKEN, and the sentence and block above are kept as written. Six sectors ship. The two added are Indices and Futures / Perps, and the class box, the class table and every scan read all six.

`src/trading/ata_spm.py` — the sector set every button, menu and scan reads

```
ASSET_CLASSES = (
    CLASS_CRYPTO,
    CLASS_STOCKS,
    CLASS_COMMODITIES,
    CLASS_FOREX,
    CLASS_INDICES,
    CLASS_FUTURES_PERPS,
)
```

Neither added sector carries a listed market yet, so a scan of either walks no sector at all. The map store answers an empty set for a class it holds no entry for, which is what it already does for crypto.

The two sentences above are overtaken for Futures / Perps, and both are kept as written. A Futures / Perps scan now walks the venue's futures list, because every product on a crypto root is placed there. An Indices scan walks the same list for a product on an index root.

The operator asked for the published names rather than this project's own.

> *I just want to adopt the standard categorization and language so that the implementation is professional and accurate...*

A retired class name still resolves.

The name given	The class it answers
`metals`	commodities
`energy`	commodities

The name given	The class it answers
`derivatives`	crypto, because a futures product takes the class of its underlying
a name neither taxonomy holds	none; the press is refused

The `derivatives` row above is overtaken, and it is kept as written. The name now answers futures_perps, the class the contract rule files a contract on a crypto underlying under.

A refused name runs no scan, so no class is scanned with no timeframes behind it.

`src/trading/ata_spm.py` — the class one name resolves to

```
def asset_class_named(name: Any) -> str:
```

`src/trading/ata_asset_maps.py` — the sector one name resolves to

```
def sector_named(typed: Any) -> str:
```

The sectors each class holds, and how many markets the shipped map lists in each. A crypto list and a stocks list are read at press time instead, from the venue's own products list and the screener, so those two counts move with the venue.

Class	Sector	Markets the map lists
crypto	by sector tag	read at press time
stocks	portfolio	47
commodities	energy	4
commodities	industrial metals	2
commodities	precious metals	8
commodities	agriculture	0
commodities	livestock	0
forex	major	7
forex	minor	21
forex	exotic	0

A sector holding no market is not drawn as a sector the scan can walk. Its count is drawn under the ticker field with the reason it holds nothing.

`src/gui/main_tabs/market_inspector_surface.py` — the line an empty sector takes

```
TICKER_EMPTY_SECTORS_FORMAT = "{sectors}: 0 market(s) listed, {reason}."
```

Commodities reads "agriculture, livestock: 0 market(s) listed, no fund listed in dollars holds them, and this map carries no futures." Forex reads "exotic: 0 market(s) listed, no pair is named for this tier yet."

One function builds that line for any class, from the sectors the map declares and lists nothing for.

`src/gui/main_tabs/market_inspector_surface.py` — the line, and what it reads

```
  def empty_sector_line(asset_class: Any) -> str:
      """``TICKER_EMPTY_SECTORS_FORMAT`` over ``ata_asset_maps.unlisted_sectors``."""
```

`src/trading/ata_asset_maps.py` — the sectors a class holds, and the ones it lists nothing for

```
  def sectors_for(asset_class: Any) -> tuple:
  def unlisted_sectors(asset_class: Any) -> tuple:
```

`src/trading/ata_spm.py` — the name a typed sector's own rows carry

```
  def sector_key_of(rows: Any, typed: Any) -> str:
```

A typed sector takes the name its own rows carry, so a retired sector name draws the headline of the sector holding its markets.

What each class can reach, and what it can trade

Every class answers two questions apart. The first is whether a retail trader can reach the market from home. The second is whether a venue is wired here that can place an order on it. ATA-SMP scans a market either way, for its charts.

Class	Reachable from home	Venue wired
crypto	yes, a retail account on a crypto venue takes orders over its API	yes, the connector the live bots trade on
stocks	yes, a retail broker account takes equity orders over its API	no, no broker connector is reachable
commodities	yes, every listed row is an ETF share a retail broker account buys	no, the same missing broker connector
forex	yes, a retail account at a registered dealer takes spot orders	no, no dealer connector exists

The four commodity spot pairs carry no venue at all. A futures row in any class is reachable and unwired: the venue takes orders on its US futures products, and the futures adapter reads candles and places none.

The table above gains a row, and it is kept as written. Futures / Perps answers what a futures row answers, because `CLASS_DEPLOYABILITY` holds the futures form's own record for it.

Class	Reachable from home	Venue wired
futures_perps	yes, the venue takes orders on its US futures products over the same API	no, the futures adapter reads candles and places none

The form each listing carries

Every row draws the form its product takes. The form rides on the row; no control was added and no menu gained an entry.

Listing	Form drawn
a crypto pair on the exchange	`spot`
a commodity spot pair	`spot`
a fund share	`ETF`
a portfolio equity or a screener quote	`equity`

Listing	Form drawn
a dated venue contract	`future`
a perpetual venue contract	`perpetual`

`src/trading/ata_asset_maps.py` — where a futures product is placed

```
def futures_placement(product: Any) -> tuple[str, str, str]:
    """The underlying, the class and the sector one futures product takes."""
```

A root neither the record nor the root table resolves is held back, named on the order line and counted. Nothing is guessed.

The placement above now ends in the contract rule, and the block is kept as written. A crypto root answers Futures / Perps, and an index root answers Indices with no sector.

```
# src/trading/ata_asset_maps.py, in futures_placement
    root = _root_of(product)
    if not root:
        return "", NO_ASSET_CLASS, ""
    if not _non_crypto(product) and (root in CRYPTO_SYMBOLS or _crypto_sector(root)):
        return root, contract_class(CLASS_CRYPTO), _crypto_sector(root)
    held = NON_CRYPTO_ROOTS.get(root)
    if held is None:
        return root, NO_ASSET_CLASS, ""
    return held[2], contract_class(held[0]), held[1]
```

The ticker field

The field names one market, read on demand. Typing narrows a list the program already holds. Nothing is fetched on a keystroke and no venue is asked.

`src/gui/main_tabs/market_inspector_surface.py` — the field and what it says

```
TICKER_FIELD_PLACEHOLDER = "Ticker"
TICKER_FIELD_TOOLTIP = (
    "Name one market to read on demand. Typing offers the tickers the "
    "sector menu beside it holds."
)
```

The operator named the field, and said what it is for.

> *Sector Field - Incorrect naming... This entry field is for reading specific, individual markets on demand and will require TICKER recognition...*

Each offer names its class, its form, whether a bot can deploy there, and whether a bot can size a scrum on it. Offers come prefix matches first, the chosen class first inside each, then matches holding the text later in the name.

`src/gui/main_tabs/market_inspector_surface.py` — one offered row

```
TICKER_OFFER_FORMAT = "{symbol}  ({asset_class} · {form} · {deploy} · {tradeable})"
TICKER_MATCH_LIMIT = 8
```

Typed text is folded to upper case, outer spaces are stripped, and a hyphen, an underscore or an inner space reads as a slash. The folded text is matched against each class's names, the chosen class first.

`src/gui/main_tabs/market_inspector_surface.py` — the separators one name may carry

```
TICKER_SEPARATORS = ("-", "_", " ")
TICKER_JOIN = "/"
```

Typed	Resolves to	Class
`a15`, `A15/USD`, `a15-usd`, `A15USD`	A15 on the exchange	crypto
`gld`, `GLD/USD`	GLD on the chart venue	commodities
`eur/usd`, `eurusd`, `eur usd`	EUR/USD on the chart venue	forex
`A15/EUR`	nothing; that quote is not one the connector route reads	—
`ZZZQ`	nothing; no class lists it	—

A name the chosen class does not hold but another class does moves the class box to the class holding it, redraws the four timeframe buttons for that class, and writes one line to the Activity Log.

`src/trading/ata_spm.py` — the line a move writes

```
CLASS_MOVED_TEXT = (
    "ATA-SPM ticker {ticker} is listed under {placed}; "
    "the class box moves from {chosen} to {placed}"
)
```

`src/gui/main_tabs/market_inspector_surface.py` — the rows one keystroke offers

```
def ticker_offers(typed: Any, asset_class: Any, connectors: Any = None) -> TickerOffers:
    """The offered rows and the names left off them, together."""

def class_tickers(asset_class: Any) -> list:
def class_walk(asset_class: Any) -> list:
```

`class_walk` answers `ata_spm.ASSET_CLASSES` with the chosen class first, so a typed name is matched against that class before any other.

`src/gui/main_tabs/market_inspector_surface.py` — where one typed name lands

```
def market_listing(ticker: Any, asset_class: Any) -> Optional[Any]:
    """The ``ata_spm.TickerPlacement`` a typed ticker names, across every class."""

def placements_of(typed: Any, asset_class: Any) -> list:
```

`src/trading/ata_spm.py` — what a placement carries

```
@dataclass(frozen=True)
class TickerPlacement:
    """One typed name, the listing it resolved to, and the class holding it."""

    listing: Any
    asset_class: str
    typed: str = ""
```

A name no class holds is refused under the field, nothing runs, and no venue is asked.

`src/trading/ata_spm.py` — the refusal a name no class holds takes

```
TICKER_UNHELD_FORMAT = "No class lists ticker {ticker}. Pick one the field offers."
```

`src/gui/main_tabs/market_inspector_surface.py` — the line a sector with no list takes

```
TICKER_NO_LIST_FORMAT = "No ticker list for {sector}. A typed name still scans."
```

A sector whose list is read at press time says so instead, and a typed name still scans.

`src/gui/main_tabs/market_inspector_surface.py` — the line a read-at-press list carries

```
TICKER_PRESS_LIST_FORMAT = (
    "Scan Now reads the {sector} list from {source}. A typed name still scans."
)

def ticker_note(asset_class: Any, connectors: Any = None) -> str:
    """The one line under the field: the empty sectors, the press list, or a refusal."""

def ticker_offer(listing: Any, asset_class: Any, connectors: Any = None) -> str:
```

The Qt completer shows the offered row and writes the bare symbol into the field. The page's option carries the offer as its label and the symbol as its value.

Whether a bot can size a scrum

One function answers whether a bot can size a scrum on a market. A scrum sells the excess above a dollar target, and a venue accepts an order only at or above its own published minimum, stepped onto its own increment.

`src/trading/scrumming/sizing.py` — the three answers

```
def tradeable_answer(
    rules: Any, price: Any = None, excess_usd: float = REFERENCE_SCRUM_EXCESS_USD
) -> str:
```

Answer	Drawn as	What it means
yes	`can size a scrum`	the smallest order the venue accepts costs no more than the excess
no	`cannot size a scrum`	it costs more, so a scrum would have nothing to submit

Answer	Drawn as	What it means
unknown	`size rules not read`	no market record was obtained

An unknown is never read as either other, and such a market stays on the list. A market answering no is kept off the offered list, counted and named.

`src/gui/main_tabs/market_inspector_surface.py` — the line a refused market takes

```
TICKER_UNTRADEABLE_FORMAT = (
    "{count} market(s) left off, the smallest order costs more "
    "than a scrum's excess: {names}"
)
```

The excess the answer measures against is the largest one the saved fleet can produce. The highest Target Balance in the fleet is 350 dollars and every bot carries the same 5 per cent scrumming interval, so the largest excess is 17 dollars 50.

`src/trading/scrumming/sizing.py` — the reference excess

```
LARGEST_FLEET_TARGET_USD = 350.0
FLEET_SCRUMMING_INTERVAL_PCT = 5.0
REFERENCE_SCRUM_EXCESS_USD = scrumming_interval_usd(
    LARGEST_FLEET_TARGET_USD, FLEET_SCRUMMING_INTERVAL_PCT
)
```

OVERTAKEN, quoted whole:

> *The excess the answer measures against is the largest one the saved fleet can produce. The highest Target Balance in the fleet is 350 dollars and every bot carries the same 5 per cent scrumming interval, so the largest excess is 17 dollars 50.*

True today: the screen reads the fleet each time it labels a row, and it says on the screen which figure it used. The two constants were renamed, because neither one reads the fleet.

`src/gui/main_tabs/market_inspector_surface.py, in fleet_scrum_excess` reads `bot_state.json` read-only and answers the largest excess the saved records produce, with a count of how many gave one. It never writes that file. `src/gui/main_tabs/market_inspector_surface.py, in excess_note` then names the figure: the fleet sentence where the count is one or more, and the assumed sentence where the count is zero.

`src/gui/main_tabs/market_inspector_surface.py` — the two sentences the note carries

```
TICKER_UNTRADEABLE_MEASURED_FORMAT = (
    "{count} market(s) left off, the smallest order costs more than a scrum's "
    "excess, measured {note}: {names}"
)
```

A saved fleet reads, for example, `against $20.00, the largest scrum excess the 2 saved bot(s) produce`. No saved fleet reads `against an assumed scrum excess of $17.50, from a $350.00 Target Balance at a 5% interval`, which names the assumption as an assumption.

`src/trading/scrumming/sizing.py` — the assumed pair, which reads no fleet

```
ASSUMED_TARGET_BALANCE_USD = 350.0
ASSUMED_SCRUMMING_INTERVAL_PCT = 5.0
REFERENCE_SCRUM_EXCESS_USD = scrumming_interval_usd(
    ASSUMED_TARGET_BALANCE_USD, ASSUMED_SCRUMMING_INTERVAL_PCT
)
```

A row the ticker field labels is labelled before any bot exists, so the fleet's largest excess is the widest figure the screen can honestly use. A bot about to place an order is a different case: `src/trading/bot_container.py, in guarded_place_order` passes that bot's own excess, through `src/trading/scrumming/sizing.py, in bot_scrum_excess_usd`.

The venue's published minimum is ceiled onto its own increment, valued at the venue's own last price, and never falls under the venue's minimum order cost.

`src/trading/scrumming/sizing.py` — the smallest order the venue accepts

```
def smallest_order_usd(rules: Any, price: Any) -> Optional[float]:
```

`src/exchange/market_inspector_fetcher.py` — where its rules come from

```
def trading_rules(
    exchange_connectors: Any, accepted_quotes: Any = DEFAULT_QUOTES
) -> dict[str, tuple]:
```

The rules and the price come off the market table each connector has already loaded, as one `MarketRules` row per market. No venue is asked. The crypto rows the sector scan lists carry no size rules, so on that path the answer reads unknown for every crypto row; the ticker field reads rules off each connector's table and answers all three.

The timeframe row

The asset class sets which four timeframes the buttons offer. Each button draws on while its timeframe is ticked, and reads its key at the press from the row the class shows. Crypto is the fast set and every other class shares the slower one.

`src/trading/ata_spm.py` — the two sets

```
CRYPTO_TIMEFRAMES = ("5m", "1h", "1d", "1w")
SLOWER_TIMEFRAMES = ("1h", "1d", "1w", "1M")
```

A fresh board carries the whole set its class lists, so all four draw on until one is pressed off. A timeframe press made before any sector exists is held on the board and given to the sector the next press adds.

Scan Now

Scan Now reads one press. What it reads depends on the field.

The field	What the press reads
a ticker the class lists	that one market, on every ticked timeframe

The field	What the press reads
a sector name the map lists	that sector's own assets
empty	the class's markets in its own order, until the hit target lands

`src/gui/main_tabs/market_inspector_surface.py` — the button and what it says

```
SCAN_NOW_LABEL = "Scan Now"
SCAN_NOW_TOOLTIP = (
    "Scan this sector now on the timeframes ticked beside it, without "
    "waiting for a rotation."
)
```

The operator asked for the empty field to walk the sector by volume.

> *ATA-SMP - Scan Now - Seems to be expecting a Ticker field entry. This is incorrect. If Ticker field has no entry, this button must scan sector's markets (on each selected TF) in order of decreasing volume until x total number of "hits" (across all selected TFs for the sector; add under settings, default is 3) are found.*

He had asked for the button twice before it worked.

> *ATA-SMP - Please proceed back to ATA-SMP and get my Scan button working. Unitize and apply same work flow under the correct Issue. It still does nothing as of the build I am currently running.*
>
> *Simulator - Scan Now - I should not have to have anything in the Ticker field. Stocks is saying 'no ticker list'. This is scaffolded, unverified garbage. I watched you perform various operations while working on this. How the fuck is it that after I launch the build, not a god damn thing has improved with this fucking button.*

Every mention of the Simulator in those words names this tab.

> *Make sure you correct my incorrect verbatims because SImulator does not have Scan Now...Inspector and ATA-SMP do.*

The button is disabled and reads `Scanning…` from the press until the answer or the failure crosses back. A press made while a scan runs writes a busy line to the Activity Log at warning.

`src/trading/ata_spm.py` — the label a running scan draws

```
SCAN_BUSY_LABEL = "Scanning…"
SCAN_BUSY_TEXT = "ATA-SPM scan pressed while a scan is running; press ignored"
```

Scan All

A second button sits beside Scan Now at its size, in both builds. One press walks every class the sector menu holds, in menu order, on every timeframe that class scans, over every market its list holds, with no stop at a hit count.

`src/gui/main_tabs/market_inspector_surface.py` — the second button

```
SCAN_ALL_LABEL = "Scan All"
SCAN_ALL_PART = "scan-all"
```

The operator asked for it, and for the confirmation timers beside it.

> *ATA-SMP - Add a Scan All button which will scan every market in every TF for every market in every sector. Have noticed that scans run really fast which is good but just do not to risk overloading a free API or something. For the attached image, I want the Confirmation Timers for prior Hits to appear here. Can be just a small ticker pair with a theme-consistent timer which triggers the rescan on completion.*

A Scan Now pressed after a Scan All drops those sectors before it walks, so it walks its own class and not every class.

`src/trading/ata_spm.py` — what the counter says during a walk

```
SCAN_ALL_PROGRESS_FORMAT = (
    "Scanning {asset_class} ({at} of {sectors}) · {read} of {total} · {hits} hit(s)"
)
SCAN_ALL_LISTING_FORMAT = (
    "Scanning {asset_class} ({at} of {sectors}) · reading the market list"
)
```

The hits count over every sector, because the chime sounds when that count rises. A per-sector reset would leave the second sector's first hits silent.

The pace a walk holds

Scan All keeps every read to a host at least a published gap. The crypto venue publishes ten requests a second for its public endpoints; the chart venue publishes no limit, so a refusal is its only signal.

`src/trading/ata_spm.py` — the gaps and the waits

```
EXCHANGE_PACE_S = 0.2
YAHOO_PACE_S = 0.5
EXCHANGE_RATE_LIMIT_WAIT_S = 10.0
YAHOO_RATE_LIMIT_WAIT_S = 60.0
```

The gap is measured from the end of the last read. A refusal is read as a wait, never as a burst: the walk holds that host for the venue's own retry figure when the answer carried one, else the table's seconds, and reads that market once more. A second refusal stands. Scan Now reads through no pace.

`src/trading/ata_spm.py` — the read every walk goes through

```
def paced_read(pace: Any, host: Any, read: Any) -> tuple:
    """One read held to ``pace`` on ``host``, retried once on a rate refusal."""
```

A walk hands both its vote step and its chart step one market memo, so a venue is asked once per market per timeframe rather than twice.

`src/trading/ata_spm.py` — the memo both steps read

```
class MarketMemo:
    """One market's candles, read once and answered to every later caller."""
```

The order a walk reads

Every class is walked largest first, by the figure that class publishes. The open entry's first line names the source, the markets read in order, and the count not read.

Class	The figure the order uses
crypto with a connector	the venue's own 24-hour quote volume
crypto with no connector	name order on the public products list, no volume figure
stocks	the screener's own volume ranking
commodities	the last complete daily bar's volume times its close
forex	map order; every pair's bar carries volume 0, so no figure exists

`src/gui/main_tabs/market_inspector_surface.py` — the ranked list one class answers

```
  def class_markets(asset_class: Any, connectors: Any = None) -> list:
      """Every market a class holds as an ``ata_spm.MarketOrder``, largest first."""

  def class_volumes(exchange_connectors: Any) -> dict:
  def listing_volumes(rows: Any) -> dict:
```

A stablecoin is dropped from a crypto ordering. A market the venue no longer trades is placed on the order's own dead list rather than in the walk.

`src/trading/ata_asset_maps.py` — the figure a non-crypto class is ranked by

```
  VOLUME_TIMEFRAME = RA_TIMEFRAME
  VOLUME_WINDOW_DAYS = 7
  NO_VOLUME_FIGURE = 0.0

  def venue_quote_volume(listing: Any, read: Any = None) -> Optional[float]:
      """``complete_bar``'s volume times its close, or ``NO_VOLUME_FIGURE``."""
```

The bar measured is the newest one stamped before today, which is a closed session, so the figure does not move through the day. Volume times close puts it in dollars beside crypto's quote volume.

`src/trading/ata_spm.py` — the lines the order takes

```
  VOLUME_LINE_FORMAT = "{asset_class} by volume"
  MAP_ORDER_LINE_FORMAT = "{asset_class} in map order"
  ORDER_LINE_FORMAT = "Order by {source}: {markets}"
  ORDER_UNREAD_FORMAT = "{markets} ({unread} not read)"
  ORDER_UNFIGURED_FORMAT = "{source}, {unfigured} with no figure last by name"
  MAP_ORDER_SOURCE_TEXT = "map order, no volume figure"
```

A market the venue no longer trades never reaches the list and is never fetched. A futures product the venue answers no candle for on any granularity is dropped where the list is built, and both counts sit on the order line.

`src/trading/ata_spm.py` — the two tails the order line carries

```
ORDER_DEAD_FORMAT = "{line}; {count} not trading, never fetched: {names}"
ORDER_NO_CANDLE_FORMAT = (
    "{line}; {count} dropped, the venue answered no candle "
    "on any granularity: {names}"
)
```

A dropped product stays reachable by name: typed into the ticker field it still resolves and still scans. The candle route serves two granularities and the weekly line rolls up from the daily one, so the read asks the hourly granularity, then the daily one, and stops at the first that answers.

`src/trading/ata_asset_maps.py` — the two readings behind the rule

```
def venue_granularities(venue: Any) -> tuple:
def candle_served(symbol: Any, read: Any = None) -> bool:
```

A hit, and the target that stops a walk

A hit is a vote the live trade gates would fire on. The zone's count, the Activity Log's hit lines and the approval bucket count one thing.

`src/trading/ata_gate_scan.py` — the one verdict

```
    @property
    def would_fire(self) -> bool:
        """True while either chain would fire, which is what a post needs."""
        return self.firing_side != NO_SIDE
```

Each market is read on every ticked timeframe first, because the gate chains read a market's other timeframes through the panel rows. Votes are then judged in timeframe order. The hit that reaches the target ends the walk, and markets after it are not read.

`src/trading/ata_spm.py` — the target, and what an unreadable count reads as

```
DEFAULT_HITS_PER_SCAN = 3

def hits_target(asked: Any) -> int:
    """``asked`` as a whole number, or ``DEFAULT_HITS_PER_SCAN`` below one."""
```

An empty box, a zero and a negative number all read as three, so no scan can stop before it has read anything.

A reversal vote is a market on one timeframe whose twelve-voter consensus and its Bollinger voter name the same non-neutral direction. It is still read and still shown, in each sector's reversal-call count, and it no longer decides what publishes.

`src/trading/ata_spm.py` — what a reversal vote is

```
    @property
    def is_reversal(self) -> bool:
        """The consensus and the band voter naming one non-neutral direction."""
        if SignalDirection.NEUTRAL in (self.direction, self.band_direction):
            return False
        return self.band_direction == self.direction
```

`src/trading/ata_spm.py` — the lines each kind of entry takes

```
VOLUME_META_FORMAT = "{read} market(s) read · {hits} hit(s) · {stop}"
STOPPED_AT_TARGET_TEXT = "stopped at target"
SECTOR_EXHAUSTED_TEXT = "sector exhausted"
SECTOR_LINE_FORMAT = "{sector} ({asset_class})"
SECTOR_META_FORMAT = "{assets} asset(s) · {votes} vote(s) · {calls} reversal call(s)"
MARKET_LINE_FORMAT = "{ticker} in {asset_class}"
MARKET_META_FORMAT = "1 market on {venue} · {votes} vote(s) · {hits} hit(s)"
MARKET_TIMEFRAME_FORMAT = "{candles} candle(s) · {reading}"
MARKET_HIT_READING = "{direction} vote, hit"
MARKET_REFUSED_READING = "{direction} vote, refused by the gates"
MARKET_NO_VOTE_READING = "no vote"
```

The zone's status line counts sectors and markets apart, so a run holding both counts both.

`src/trading/ata_spm.py` — what one walk answers and how far it goes

```
NO_HIT_TARGET = 0

def walks_order(scan: Any) -> bool:
    """True for a ``walk_all`` sector and for a ``hit_target`` above ``NO_HIT_TARGET``."""
```

A board builds one walking sector per class, tells the counter which list it is reading, and runs them. The walk hands a callable one progress record after each market and each hit, and one builder writes the line each record carries.

`src/trading/ata_spm.py` — the walk and the records it hands back

```
def compute_all(self, ticked: Any, hits: Any) -> None:
def _scan_until_hits(sector: Any, sources: Any, target: Any, on_progress: Any) -> tuple:
def market_line(progress: Any) -> str:

class ScanProgress:
class WalkProgress:
```

What the field draws while a walk runs

The field draws the Indicator Voting Panel of the market under read, and nothing else. The panel is one row per timeframe that answered candles, one cell per voter, plus the Net, Comp and Conf columns, in the colours the Live tab's own panel uses. The title names the market, the word `reading`, and the tally of bullish, bearish and neutral votes.

`src/gui/main_tabs/market_inspector_surface.py` — one builder, two callers

```
def panel_grid(symbol: Any, label: Any, rows: Any, lines: Any = ()) -> Optional[dict]:
    """One voting grid, for a called asset or for the market under read."""
```

The operator asked for that panel and for the text to go.

> *ATA-SMP - The scanner freaking works but we do have some issues. Derivatives sector list is empty. Metals only has four in its list and I am pretty sure its a more complex sector than this but could be wrong. I see a series of floating windows rapidly appearing and disappearing over the ATA-SMP panel once the scan gets going. The field where the IVP fields are supposed to appear during the read initially displays text but this gets very small very fast before disappearing completely and the floating windows appear with some containing text before vanishing. I was able to get three hits and three charts generated. Very good progress.*
>
> *Observed a very strange issue with the lower information panel when attempting to scroll down to see more text. This seemed to trigger a chart re-read loop of some kind. During this I saw the rapidly spawning and despawning windows again. Need to get rid of the informational text during reads and only display the really nice looking IVP information that has only been visible in the larger floating windows up to this point. The IVP data keeps the symbology consistent and its really well designed from what I have seen of it. We have no use a for a repeating spool of identical data lines.*

The field holds where the operator left it. Neither host drags its scroll to the bottom on a progress. The grid asks 592 pixels: a 44-pixel timeframe column, six 58-pixel voter columns, two 52-pixel columns and a 96-pixel Conf column. It draws whole at a 1400-pixel tab. At 900 and 700 the last three columns sit past the pane edge with a horizontal scroll bar under them.

`src/gui/main_tabs/market_inspector_surface.py` — the grid, its widths and its two callers

```
PANEL_SUMMARY_FORMAT = "▲ {bullish}  ▼ {bearish}  ■ {neutral}"

def panel_column_widths() -> tuple:
def panel_table(rows: Any) -> list:
def panel_summary_text(rows: Any) -> str:
def panel_tally(row: Any, direction: Any) -> int:
def scan_panel(progress: Any) -> Optional[dict]:
```

`voting_panel` is the called asset's grid, drawn in an open entry with its gate chain lines under it. `scan_panel` is the same grid for the market under read, with no gate line, because no chain has run over that market.

`src/trading/ata_spm.py` — the rows one market answers

```
def panel_rows_for(symbol: Any, timeframes: Any) -> dict:
```

The per-market read line still exists, with the same four things, written to the Activity Log instead of the field.

`src/trading/ata_spm.py` — the line each market read writes

```
SCAN_MARKET_LINE_FORMAT = "{symbol} · {timeframes} · {votes} vote(s) · {verdict}"
SCAN_MARKET_HIT_FORMAT = "hit on {labels}"
SCAN_MARKET_BLOCKED_FORMAT = "no hit: the gates blocked {names}"
SCAN_MARKET_NO_VOTE_TEXT = "no hit: no vote"
SCAN_MARKET_UNREAD_TEXT = "no candles"
```

The counter

The counter sits at the right end of the stepper's position row, just above the field, in both builds. It is empty while no scan runs.

The operator placed it there.

> *Can see the scan counter in the upper left corner of the lower section. Would rather have this counter just above and at the right corner of the same field. Then have the field display the results of each scanned market.*

`src/trading/ata_spm.py` — what the counter says during one class's walk

```
SCAN_PROGRESS_FORMAT = "Scanning {asset_class} · {read} of {total} · {hits} hit(s)"
SCAN_LISTING_FORMAT = "Scanning {asset_class} · reading the market list"
```

What a scan reports

An empty table can mean three different things. The note under it says which: no scan has run yet, a scan is running, or a scan finished and found nothing.

`src/gui/main_tabs/market_inspector_surface.py` — the sentence an empty table carries

```
def empty_table_text(scan_state: Any, noun: Any) -> str:
    """The sentence an empty table carries for one scan state.

    ``SCAN_NOT_ASKED``, ``SCAN_RUNNING`` and ``SCAN_FINISHED`` each get
    their own wording, so the three never read alike.
    """
```

`scan_state` reports the same three states to both renderers, so the React side draws the note the way the window does.

A scan that stops part way reports no scan at all. `MarketInspector.scan_universe` clears the pairs and the scan second before it runs the pair test, and sets the second only once that test answers, so a scan the analyzer stopped leaves no timestamp and no pairs from the scan before it. `inspector_scan_age` reads a cleared second as `NO_SCAN_AGE`, which no freshness window accepts, so the sector charts refetch rather than draw from a scan nobody finished.

`src/gui/main_tabs/market_inspector_surface.py` — the age a cleared second reads

```
def inspector_scan_age(inspector: Any) -> float:
    """The seconds since the Market Inspector's last universe scan."""
```

A scan ends with one note on the Activity Log. The note names the markets no venue lists, the timeframes no venue serves, the markets that answered no candle, and each market no built bot variant trades with that market's own reason.

`src/trading/ata_spm.py` — three of the four sentences a note can carry

```
UNLISTED_TEXT = "No configured venue lists {symbols}."
UNSERVED_TEXT = "No venue serves {labels}."
UNTRADEABLE_TEXT = "Read and not traded, no bot variant trades {symbols}: {reason}"
```

`src/gui/main_tabs/market_inspector_surface.py` — the fourth

```
NO_CANDLE_TEXT = "No candles came back for {symbols}."
```

The note groups markets by reason and writes one sentence per group. A symbol is named as unread only when every ticked timeframe came back empty for it. No market leaves the scan: a named market keeps its place in the asset list, the charts and the reports.

`src/trading/ata_spm.py` — the separator between two reasons

```
UNTRADEABLE_SEPARATOR = ". "
```

Each timeframe keeps that market's last close beside the candle count it already keeps, and the note reads those closes rather than asking a venue.

`src/trading/ata_spm.py` — the closes the note reads

```
def scanned_closes(timeframes: Any) -> dict:
    """The last close each market answered, the first timeframe that read one."""
```

A timeframe holding too few candles is reported, never voted. The floor is thirty candles, the number the gate scan already requires.

`src/trading/ata_spm.py` — the floor, and the line it is reported on

```
MIN_CANDLES_TO_VOTE = ata_gate_scan.MIN_CANDLES_FOR_TA

TIMEFRAME_VOTE_FORMAT = (
    "{votes} vote(s), {unread} without candles, {short} under {floor} candles"
)
```

The floor earns its place. On twenty-five hourly candles seven of the twelve voters abstain for want of history, and the remaining five still produce a direction.

The scan also writes to the log, one line per phase. Every line is written at info on the `acervator` logger, so the system log and the Console pane carry it.

`src/trading/ata_spm.py` — the lines one press writes

```
SCAN_PRESSED_TEXT = (
    "ATA-SPM scan pressed: {asset_class} on {timeframes}, "
    "ticker '{ticker}', target {hits} hit(s)"
)
MARKET_READ_TEXT = "ATA-SPM read {symbol} on {label} from {venue}: {candles} candle(s)"
MARKET_EMPTY_TEXT = "ATA-SPM read {symbol} on {label} from {venue}: no candles"
MARKET_REFUSED_TEXT = "ATA-SPM read {symbol} on {label} from {venue}: refused, {reason}"
HIT_TEXT = "ATA-SPM hit: {call}"
SCAN_FINISHED_TEXT = "ATA-SPM scan finished: {headline} · {meta}. {method}"
SCAN_NOTE_TEXT = "ATA-SPM scan finished: {note}"
SCAN_EMPTY_TEXT = "ATA-SPM scan finished: no sector to scan"
SCAN_FAILED_TEXT = "ATA-SPM scan failed: {error}"
```

Each line crosses to the Activity Log on one signal carrying the message and its level. A read that answered candles draws at info. A read that answered none, the busy line, and a finished line with no vote draw at warning. A scan that raised draws at error.

`src/gui/market_inspector.py` — the signal that carries a phase line

```
    scanLogged = Signal(str, str)
```

The candle source names where it read, what it read, and why it read nothing.

`src/trading/ata_asset_maps.py` — the four refusals a read can name

```
UNMAPPED_TEXT = "no map lists {symbol}"
NO_VENUE_TEXT = "no configured venue lists {symbol}"
UNSERVED_TEXT = "{venue} does not serve {timeframe}"
NO_ADAPTER_TEXT = "no fetcher serves {venue}"
```

Each module names its own, so the scan note's sentence and one read's refusal are two constants and not one.

`src/gui/main_tabs/market_inspector_surface.py` — the source one read names

```
CANDLES_FROM_SCAN = "universe scan"

def sector_candle_read(symbol: Any, timeframe: Any, sources: Any) -> tuple:
    """Where one read went, what it read, and why it read nothing."""
```

A mapped name names its own venue. A crypto name served from the held scan names the scan, and one read through a connector names the exchange.

Opposing Pairs

The zone draws the pairs kept on the last scan. A market reaches long or short only when two or more of its timeframes sit at the same Bollinger extreme. A thirty-day return correlation in the configured negative window raises a candidate, and a test for a long-run equilibrium then decides it.

`src/trading/market_inspector.py` — `MarketInspector._find_opposing_pairs`

```
def _find_opposing_pairs(self, signals: list, closes_by_symbol: dict) -> list:
    """Enumerate long × short candidates; keep the cointegrated ones.

    The 30-day return correlation raises a candidate and no longer decides
    it: ``cointegration_test`` runs on the full close series and a pair it
    refuses never reaches the table.
    """
    longs = [s for s in signals if s.direction == "long" and s.score >= 0.3]
    shorts = [s for s in signals if s.direction == "short" and s.score >= 0.3]
```

Two markets can move opposite each other for a year with no relationship holding between them. The test is cointegration in both of its standard forms, taken from statsmodels, and a pair passes only when both reject the null of no cointegration: Engle-Granger at a significance of 0.05, and Johansen above its ninety-five per cent critical value for rank zero.

`src/trading/pair_selection.py` — what a pair has to clear

```
WINDOW_BARS = 365
MIN_OBSERVATIONS = 120
SIGNIFICANCE = 0.05
```

`src/trading/pair_selection.py` — the two published tests

```
def engle_granger_p_value(left: Any, right: Any) -> float:
    """The two-step p-value, from ``statsmodels.tsa.stattools.coint``."""

def johansen_trace(left: Any, right: Any) -> tuple:
    """The rank-zero trace statistic and its 95% critical value."""
```

A pair with fewer than 120 daily closes is not tested and not shown. A flat series is refused before the correlation package is imported, under a method and a reason of its own rather than as a test result.

`src/trading/pair_selection.py` — the refusal a flat series takes

```
METHOD_CORRELATION = "correlation"
FLAT_SERIES_DETAIL = "series holds one repeated price"

def is_flat(series: Any) -> bool:
```

The correlation test imports no package at all. `pearson_correlation` computes the coefficient, and from the Student t distribution its two-sided p-value, over numpy alone. The reading was swept against scipy 1.18.1 over 2,090 samples — sizes from three to four thousand, coefficients from minus one to plus one, and p-values inside a decade of the 0.05 the panes gate on — and both readings answer the same verdict in every one of them.

`src/trading/pair_selection.py` — the coefficient and its p-value

```
def pearson_correlation(left: Any, right: Any) -> tuple[float, float]:
    """Pearson's r over two samples, and its two-sided t-test p-value."""
```

`MarketInspector._pearson` reads that one function, so the thirty-day correlation that raises a candidate and the coefficient a card prints can never be two different formulae.

Both packages are imported inside the three functions that call them, and nowhere else, so the application starts before either is loaded. Both are declared, and the bundle carries both, so the test runs in the built application and in a source run alike. A missing package is re-raised as itself rather than recorded as a refused pair.

One package is named now, and the built application carries it. `scipy` is named by no module of the tree: the import set holds 20 outside packages where it held 21, and 17 of them are declared. `statsmodels` is the `cointegration` extra, and `CONSUMER_EXTRAS["build"]` in `tools/deps.py` installs that extra, so the shipped bundle carries the package and the Opposing Pairs table answers rows. The guard is what makes it an extra rather than a core requirement: an install from a consumer that does not name the extra runs without the cointegration test, and the scan's own status line names the package rather than the import error.

`src/gui/main_tabs/market_inspector_surface.py` — the line a stopped scan carries

```
ANALYZER_ERROR_FORMAT = "Analyzer error: {error}"
ANALYZER_MISSING_PACKAGE_FORMAT = (
    "Opposing Pairs needs {package}, which this build left out"
)
```

`analyzer_error_text` reads the second format only for an `ImportError` that names a package. Every other failure keeps the first.

`pyproject.toml` — the two packages the tests need

```
"scipy>=1.18.1",
"statsmodels>=0.15.0",
```

`pyproject.toml` — the package the cointegration test needs

```
[project.optional-dependencies]
cointegration = ["statsmodels>=0.15.0"]
```

`tools/spec_common.py` — the packages the bundle still leaves out

```
EXCLUDES: tuple[str, ...] = ("tkinter", "matplotlib", "PIL", ...)
```

A name in that list must not also sit in `dependencies`. The two statistics packages sit in `dependencies` and not in the list, which is what lets the Opposing Pairs table and the Bot Swarm Topologies zone draw rows.

Neither statistics package sits in that list, and the rule above still holds because neither sits in `dependencies` either. The list leaves them out so the bundle carries the cointegration test: measured with scipy refused, neither `statsmodels.tsa.stattools` nor `statsmodels.tsa.vector_ar.vecm` imports, so excluding scipy alone would strip that test as surely as excluding statsmodels. Every name the list does hold was read against the imports of the 498 modules under `src/` plus `main.py`. Seven of the nine are imported nowhere, and `matplotlib` and `PIL` are imported only where the built application never reaches. The Bot Swarm

Topologies zone draws rows either way, because three of its four detectors read the correlation test alone.

The zone's own line names the whole funnel: markets read, markets with a direction, pairs tested, and whether any held equilibrium.

`src/gui/main_tabs/market_inspector_surface.py` — the three counts and where each is read

```
  def scan_counts(signals: Any, tested: Any) -> dict:
      """Markets read, markets with a direction, and pairs the test ran on."""
```

Where the analysis happens

`MarketInspector` in `src/trading/market_inspector.py` owns the maths. One method drives the whole pipeline and keeps the results on the analyzer.

`src/trading/market_inspector.py` — `MarketInspector.scan_universe`

```
  def scan_universe(
      self,
      candles_by_symbol_by_tf: dict,
      active_symbols: set,
      closes_by_symbol: dict,
  ) -> None:
```

Two steps run under it.

Step	Produces
`_analyze_tf`	One `TimeframeAnalysis` per timeframe
`_score_market`	One `MarketSignal` per market

One analyzer instance serves two readers. `get_shared_inspector` returns it, the tab owns the fetch cycle and writes into it, and the per-bot Market Inspector page in the Bot Details dialog reads back out of it through `build_per_bot_view`. One analyzer, two readers, no second copy of the score.

`src/trading/market_inspector.py` — what one finished scan leaves on the analyzer

```
      last_signals: list
      last_tested: list
      last_closes: dict
```

`last_tested` holds one entry per candidate the equilibrium test ran on, whether it passed or not, and the Opposing Trades funnel line is the one reader of it.

The per-bot page is drawn from one description of the screen. It returns the rows, the groups, the words, the colours and the layout numbers as values. The Qt side draws them as widgets and the React side draws them as elements.

`src/gui/main_tabs/market_inspector_tab_surface.py` — `per_bot_view`

```
def per_bot_view(bot: Any) -> dict:
    """The per-bot Market Inspector screen as values, read off the shared
    analyzer's most recent scan.
```

The failure line draws as characters. Where the per-bot view cannot be built, the tab draws one amber line naming the failure. That line carries the caught error, which the application did not write, so the label is fixed to plain text and `FALLBACK_TEXT_FORMAT` carries no tag. An error holding <b> or an image address shows those characters, and the tab builds nothing from them. The Paper and Simulator forks draw the same line the same way, in `src/gui/paper/paper_market_inspector_tab.py` and `src/gui/simulator/sim_market_inspector_tab.py`.

`src/gui/live_settings/market_inspector_tab.py` — `MarketInspectorTabMixin._create_market_inspector_tab`

```
msg.setTextFormat(Qt.PlainText)
```

Fetching

`set_exchange_source` binds a connectors getter and the application scheduler, so the tab always sees the current connector dict rather than a snapshot taken at build time.

The fetch serves its last network result while that result stays young enough, and the Refresh button passes a flag that goes to the network regardless.

`src/exchange/market_inspector_fetcher.py` — `fetch_htf_universe`

```
async def fetch_htf_universe(
    exchange_connectors: dict,
    active_symbols: Optional[set] = None,
    top_n: int = DEFAULT_TOP_N,
    progress_cb=None,
    force_network: bool = False,
    min_refresh_s: float = DEFAULT_MIN_REFRESH_S,
) -> FetchResult:
```

Three helpers do the work inside it.

Helper	What it does
`_pick_universe`	Ranks each connector's bulk tickers by 24-hour quote volume
`_fetch_one_symbol`	Pulls per-symbol OHLCV on the connector's single-worker executor
`weekly_from_daily`	Derives the weekly series when the venue lists no weekly timeframe

The weekly series is one definition. It calls the stone tablets' own rollup with the week bucket moved onto Monday 00:00 UTC, the day the chart venue stamps its weekly bar with.

`src/exchange/market_inspector_fetcher.py` — the weekly line

```
def weekly_from_daily(daily: list[_Candle]) -> list[_Candle]:
    """One weekly _Candle per calendar week of daily, through weekly_rows_from_daily."""
```

The scan asks a venue only for what its own table lists.

`src/exchange/market_inspector_fetcher.py` — the timeframes one connector serves

```
DAILY_TIMEFRAME = "1d"
WEEKLY_TIMEFRAME = "1w"

def exchange_timeframes(connector: Any) -> tuple:
    """The timeframes one connector's own table lists, with 1w where 1d is."""
```

`src/trading/market_inspector.py` — the three the Refresh scan names

```
HTF_TIMEFRAMES = ("1d", "1w", "1M")
```

Every crypto row carries that table, and a listing answers whether it serves one timeframe. Where no table is found, the public route's own granularity set answers instead. A market must sit at the same Bollinger extreme on two of those timeframes to carry a direction, so the Refresh scan's two series decide it.

No code reads that tuple. `src/trading/market_inspector.py`, in `MarketInspector.scan_universe`, walks whatever timeframe keys the fetch put in the map, and `src/exchange/market_inspector_fetcher.py`, in `_fetch_one_symbol`, puts `1d` in and `1w` beside it. The tuple's `1M` entry names no rung the scan builds.

`src/trading/stone_tablets/ra_fetcher.py` — the granularities the public route serves

```
class CoinbasePublicCandles(ExchangeAdapter):
    GRANULARITY_S = {"1d": 86_400, "1h": 3_600, "5m": 300}
```

`src/exchange/market_inspector_fetcher.py` — the two product lists and the volume read

```
def public_products() -> tuple:
def trading_products(exchange_connectors: Any) -> tuple:
def public_candles(symbol: Any, timeframe: Any, bars: Any) -> list:
async def fetch_quote_volumes(
    exchange_connectors: dict, min_refresh_s: float = DEFAULT_MIN_REFRESH_S
) -> dict[str, float]:
```

`src/gui/main_tabs/market_inspector_surface.py` — the two readers the scan asks after the maps

```
def connector_candles(symbol: Any, timeframe: Any, connectors: Any) -> list:
def inspector_candles(symbol: Any, timeframe: Any) -> list:
def inspector_scan_age() -> float:
```

A product reaches a walk only while the venue says its status is online and its trading is not disabled. A name the map holds and the venue does not trade goes on the order's dead list.

`src/exchange/market_inspector_fetcher.py` — the week the rollup uses

```
DAY_MS = 86_400_000
MONDAY_OFFSET_MS = 4 * DAY_MS
DAYS_PER_WEEK = 7
WEEK_MS = DAYS_PER_WEEK * DAY_MS
```

A request to the public crypto route is held to 300 candles, and the bars a read asks for are held to that ceiling before the start is computed, so the window ends at the press.

`src/exchange/market_inspector_fetcher.py` — the window one read asks for

```
        count = min(int(bars), RA_CHUNK_DAYS)
```

The market-data cache is shared. A crypto name with a Refresh under fifteen minutes old uses the candles Refresh read; past that the scan reads again.

`src/gui/main_tabs/market_inspector_surface.py` — the window a held scan serves

```
CANDLES_FRESH_SECONDS = DEFAULT_MIN_REFRESH_S
```

The scan asks three questions in order and stops at the first that answers: a listing in the asset maps, a fresh inspector scan, then the connectors. A crypto scan with no exchange connected reads the products list and the candles through the application's own public route.

`src/gui/main_tabs/market_inspector_surface.py` — the three sources, in order

```
def sector_candles(symbol: Any, timeframe: Any, sources: Any) -> tuple:
    """The candles one market answers, from the first source that has them."""
```

`src/trading/ata_asset_maps.py` — the listing one name resolves to, and its candles

```
def listing_of(symbol: Any) -> Optional[AssetListing]:
def venue_candles(listing: Any, timeframe: Any) -> list:
def venue_candle_read(listing: Any, timeframe: Any) -> tuple:
```

`src/trading/ata_spm.py` — the markets one class and sector hold

```
def markets_of(asset_class: Any, sector: Any = "") -> tuple:
```

Both hosts hand the model its asset source and its candle source where they are built, so neither reads `None` for the life of the screen.

`src/gui/main_tabs/market_inspector_surface.py` and `src/gui/market_inspector.py` — the wiring

```
self.set_ata_sources(sector_assets, self.scanned_candles)
self.set_ata_sources(sector_assets, self._scanned_candles)
```

Three readers serve the sources: one answers the assets a named sector holds from the shipped sector map, one answers the four timeframes a class scans, and one names each market the note reports as read and not traded.

`src/gui/main_tabs/market_inspector_surface.py` — the assets one sector holds

```
def sector_assets(sector: Any, asset_class: Any) -> list:
```

`src/trading/ata_spm.py` — the timeframes a class scans, and the markets it did not trade

```
def timeframes_for(asset_class: Any) -> tuple:
def untradeable_markets(scan: Any) -> list:
```

The sector map itself is loaded by `load_sector_map` in `src/trading/topology_proposals.py`, which is the one copy the topology detectors read as well.

The venues and the timeframes each serves

The chart venue answers four intervals and spells two of them differently from the way the engine names them.

`src/trading/stone_tablets/ra_fetcher.py` — the engine's word, and the endpoint's

```
YAHOO_INTERVALS: dict[str, str] = {
    "1h": "1h",
    RA_TIMEFRAME: "1d",
    "1w": "1wk",
    "1M": "1mo",
}
```

It refuses an hourly window longer than 729 days: 729 answers and 730 returns an error. The adapter records that reach and moves the start of a longer request forward, so an over-long ask returns the candles that exist instead of nothing. The other three intervals answered a twenty-five year request in full, so no limit is recorded for them.

`src/trading/stone_tablets/ra_fetcher.py` — the days each interval reaches

```
YAHOO_REACH_DAYS: dict[str, int] = {
    "1h": 729,
    RA_TIMEFRAME: UNCAPPED_REACH_DAYS,
    "1w": UNCAPPED_REACH_DAYS,
    "1M": UNCAPPED_REACH_DAYS,
}
```

The futures route serves an hourly and a daily granularity at 350 candles a request, and its weekly line rolls up from its daily one.

`src/trading/stone_tablets/ra_fetcher.py` — the futures candle route

```
class CoinbaseFuturesCandles(ExchangeAdapter):
    exchange_id = "coinbase-futures"
    chunk_limit = FUTURES_CANDLE_CAP
    BASE_URL = "https://api.coinbase.com/api/v3/brokerage/market/products"
```

Every venue candle is stamped in seconds, the unit the connector's candles, the chart's time axis and the confirmation timer already read. The chart venue's futures list is read with its own timeout, and the crypto sector keeps its daily and weekly series on the exchange row.

`src/trading/ata_asset_maps.py` — the futures read, and the unit every row is stamped in

```
FUTURES_TIMEOUT_S = 20.0
MS_PER_S = 1000
VENUE_EXCHANGE: (RA_TIMEFRAME, "1w"),

def futures_listings(timeout_s: float = FUTURES_TIMEOUT_S) -> tuple:
    """The venue's futures rows, their figures, and a refusal where it answered none."""
```

The asset maps

One row describes one asset a sector holds, with the venue and ticker carrying it.

`src/trading/ata_asset_maps.py` — one asset, and where it is carried

```
@dataclass(frozen=True)
class AssetListing:
    """One asset a sector holds, with the venue and ticker carrying it.

    ``quote`` is the currency the venue prices ``ticker`` in, which
    ``YahooChartAdapter.fetch_chunk`` checks its answer against.
    """

    symbol: str
    quote: str = USD
    venue: str = NO_VENUE
    ticker: str = ""
```

Forex is the class built end to end, and its tiers are liquidity tiers. The major tier is the seven pairs that hold the dollar. The minor tier is every cross of two of those currencies with no dollar in it, twenty-one pairs. The exotic tier is defined and holds no pair, because no pair is named for it.

`src/trading/ata_asset_maps.py` — the two tiers that carry names

```
FOREX_MAJOR: tuple[AssetListing, ...] = tuple(
    _yahoo_fx(one)
    for one in (
        "EUR/USD",
        "USD/JPY",
        "GBP/USD",
        "USD/CHF",
        "AUD/USD",
        "NZD/USD",
        "USD/CAD",
    )
)

CROSS_ORDER: tuple[str, ...] = ("EUR", "GBP", "AUD", "NZD", "CAD", "CHF", "JPY")

FOREX_MINOR: tuple[AssetListing, ...] = tuple(
    _yahoo_fx(f"{base}/{quote}")
    for at, base in enumerate(CROSS_ORDER)
    for quote in CROSS_ORDER[at + 1 :]
)
```

Commodities carries funds and not futures. A futures chart longer than a few months joins two or more contracts together, and every join is a price step nobody traded. This platform sells against a dollar target, so an invented step moves every level behind it. A fund holds the metal, prices in dollars, and has no expiry and no roll.

`src/trading/ata_asset_maps.py` — the listed instrument for each metal

```
METALS_PHYSICAL: tuple[AssetListing, ...] = tuple(
    AssetListing(symbol=one, quote=USD, venue=VENUE_YAHOO, ticker=one)
    for one in ("GLD", "SLV", "PPLT", "PALL")
)
METALS_BASE: tuple[AssetListing, ...] = tuple(
    AssetListing(symbol=one, quote=USD, venue=VENUE_YAHOO, ticker=one)
    for one in ("CPER", "DBB")
)
```

The four commodity spot pairs stay in the map and stay unlisted. A metal spot quote is a dealer market with no public listing, so no free venue carries it, and all four spellings answer HTTP 404.

`src/trading/ata_asset_maps.py` — the map, its sector names and where each figure came from

```
SECTOR_MAJOR = "major"
SECTOR_MINOR = "minor"
SECTOR_EXOTIC = "exotic"
FOREX_EXOTIC: tuple[AssetListing, ...] = ()

METALS_SPOT: tuple[AssetListing, ...]
MAPS: dict[str, dict[str, tuple]]
MAP_SOURCES: dict[str, str]
```

MAPS is the one structure every ticker list, sector list and scan reads, and MAP_SOURCES names the published document each class's taxonomy came from.

Energy carries the two groups the published index names, petroleum and gas. Its four funds are USO for WTI crude, BNO for Brent crude, UGA for gasoline and UNG for natural gas. No fund holding heating oil or gasoil is listed, because both answered no rows.

src/trading/ata_asset_maps.py — the energy rows

```
ENERGY_PETROLEUM: tuple[AssetListing, ...] = tuple(
    AssetListing(symbol=one, quote=USD, venue=VENUE_YAHOO, ticker=one)
    for one in ("USO", "BNO", "UGA")
)
ENERGY_GAS: tuple[AssetListing, ...] = (
    AssetListing(symbol="UNG", quote=USD, venue=VENUE_YAHOO, ticker="UNG"),
)
```

The operator added the energy sector.

> *ATA-SMP - New Market Sector - Energy - Not sure how I forgot to add this...*
>
> *ATA-SMP - Energy sector also only has four entries in its list. Probably not complete...*

Every forex pair the map names sends volume 0 on every bar. Two of the twelve voters read volume, and the volume voter abstains on such a series rather than standing a number in. Spot currency trading has no single published volume anywhere, and a currency future's volume is not the pair's.

src/trading/indicators/zscore.py — which of the two values the reading holds

```
smoothed = _vwma(z_series, z_volumes, self.smoothing_period)
z_volume_weighted = smoothed is not None
z = z_raw if smoothed is None else smoothed
```

The daily open is kept in every formula that reads it. Three read it: the volume voter, Heikin Ashi and the landing strip. Each of the three is the published formula. A fund opens at an auction, so its open is a traded price beyond doubt, and a gap between an open and the previous close is what any daily bar does when the market stops trading between bars.

A venue row whose open or close sits outside its own high and low is not a candle. Each such row is refused on its own, the rest are kept, and nothing is clamped or invented.

src/trading/ata_asset_maps.py — the conversion, one row at a time

```
def _candles_of(ticker: str, rows: Any) -> list:
    """Every row ``candles_from_raw`` accepts, one row at a time.

    A venue row whose open or close sits outside its own high and low is not
    a candle, and it is counted into ``VENUE_ROWS_REFUSED_LOG``.
    """
```

The stocks list

At press time the stocks list is the chart venue's predefined most-active screener, unauthenticated, ranked by regular market volume, each quote carrying its sector.

`src/trading/ata_asset_maps.py` — the screener the list is read from

```
SCREENER_URL = "https://query1.finance.yahoo.com/v1/finance/screener/predefined/saved"
SCREENER_ID = "most_actives"
SCREENER_COUNT = 100
```

Where the screener refuses or answers no quote, the list is the portfolio equities the simulator holds: every non-crypto name the other maps do not carry, 47 names, in map order. The order line names which list was read.

`src/trading/ata_asset_maps.py` — the two lists, and the read behind the first

```
STOCKS_PORTFOLIO: tuple[AssetListing, ...] = tuple(
    AssetListing(symbol=one, quote=USD, venue=VENUE_YAHOO, ticker=one)
    for one in SYMBOLS
    if one not in CRYPTO_SYMBOLS and one not in _MAPPED_FUNDS
)

def screener_listings() -> tuple:
    """The screener's own quotes, through the request route the chart adapter uses."""
```

`src/gui/main_tabs/market_inspector_surface.py` — the two source lines

```
STOCKS_SCREENER_SOURCE_FORMAT = "{source}, {sectors} sector(s)"
STOCKS_PORTFOLIO_SOURCE_FORMAT = (
    "RA portfolio equities in map order, the screener refused: {refusal}"
)
```

The operator set the rule for a source that is not free.

> *If we are having difficulty sourcing the data for free we will need to implement an API driven solution or get clever...*

The futures list

Derivatives is not a class. Each futures product takes the class of its underlying, and the product list is read at press time from the venue's own public route, ranked by its own 24-hour volume times price.

The second sentence above is overtaken in part, and it is kept as written. A product on a crypto root now takes Futures / Perps. A press reads this list for the classes the placement can answer, and FUTURES_CLASSES derives that set from the placement's own tables.

```
# src/gui/main_tabs/market_inspector_surface.py, in FUTURES_CLASSES
FUTURES_CLASSES = tuple(
    sorted(
        {
            ata_spm.contract_class(one)
            for one in (
                ata_spm.CLASS_CRYPTO,
                *(row[0] for row in ata_asset_maps.NON_CRYPTO_ROOTS.values()),
            )
        }
    )
)
```

Driven over six product records written for the reading, with the home redirected and every network name refused, through `futures_listings` and `class_markets` on the tree before this change and on this one:

```
                                 scan before    scan now        recording
three contracts on a crypto root crypto         futures_perps   futures_perps
two contracts on a commodity root commodities   commodities     commodities
one contract on an index root    stocks         indices         indices
scan and recording agree         2 of 6         6 of 6
```

`src/trading/ata_asset_maps.py` — where the futures list comes from

```
FUTURES_PRODUCTS_URL = "https://api.coinbase.com/api/v3/brokerage/market/products"
FUTURES_PRODUCT_TYPE = "FUTURE"
FUTURES_SOURCE_TEXT = "coinbase futures and perpetuals, 24 h volume x price"
```

The operator reported the empty slots that rule closed.

> *Inspector -> ATA-SMP -> Market Sectors -> Several slots for derivatives are empty or were showing no candles returned.*

Right half: topology proposals

`MarketInspectorTopologies` in `src/gui/market_inspector_topologies.py` renders one card per proposal and opens a preview dialog on Preview.

The engine behind the cards takes a plain dictionary, which keeps it free of any exchange or bot-manager coupling. It unions the detectors, drops overlapping proposals by asset and caps the result.

`src/trading/topology_proposals.py` — `detect_all_topologies`

```
def detect_all_topologies(
    context: dict[str, Any],
    cap: int = PROPOSAL_CAP,
) -> list[dict[str, Any]]:
    """Union every detector's proposals, dedupe them and cap at ``cap``.
```

Four detectors feed it.

Detector	Shape	Members it needs
`detect_momentum_funnel`	Correlated cluster, leader into laggers	3
`detect_mean_reversion_pair`	Anti-correlated pair, wired both ways	2
`detect_sector_cluster`	Same-sector star, hub into spokes	4
`detect_distance_to_band`	One asset, scrum-deep into fold-deep	2

Three of the four read the closes a finished Inspector scan writes. The fourth reads the bot roster alone and pairs a scrum-deep bot with a fold-deep bot on the same asset.

Each detector runs in an arm of its own. `detect_mean_reversion_pair` is the only one that reads the cointegration test, so in a build without `statsmodels` it answers nothing and the other three still answer. Every proposal returned carries one sentence naming the contribution that did not run.

`src/trading/topology_proposals.py` — the sentence a proposal carries

```
UNAVAILABLE_KEY: str = "unavailable_tests"
UNAVAILABLE_FORMAT: str = "{detector} needs {package}, which this build left out"
```

The pane prints that sentence after its count, so a reading the application cannot take is named beside Refresh and not in the log alone. `unavailable_tests` reads the proposals on display and keeps the first of each repeated sentence, since one detector run stamps them all.

`src/gui/main_tabs/market_inspector_topologies_surface.py` — the count and what follows it

```
STATUS_COUNT_FORMAT = "{proposals} proposal(s); {dismissed} dismissed"
STATUS_UNAVAILABLE_FORMAT = "{count}  ·  {sentences}"
```

With `statsmodels` absent the line reads "2 proposal(s); 0 dismissed · Mean-reversion pair needs statsmodels, which this build left out". With every package present it reads the count alone.

Two sentences above describe a stamp the pane no longer depends on.

> *Every proposal returned carries one sentence naming the contribution that did not run.*
>
> `unavailable_tests` *reads the proposals on display and keeps the first of each repeated sentence, since one detector run stamps them all.*

A refresh that returns no proposal has nothing to carry the stamp. The pane now reads the import path whenever the stamp is absent. The count therefore carries the sentence with no proposal on display as well as with several.

```
def unavailable_sentences() -> list[str]:
    """Each unrun-contribution sentence read off the import path, importing nothing."""
```

`src/trading/topology_proposals.py, in unavailable_sentences` — the sentence with no proposal to ride on

Measured offscreen on both hosts, with the package refused at the import finder. With one proposal on display the line read "1 proposal(s); 0 dismissed · Mean-reversion pair needs statsmodels, which this build left out" before and after. With no proposal on display it read "0 proposal(s); 0 dismissed" before, and after it carries that

same sentence. The Qt pane and the React model report one string: both call the one helper.

Two helpers read from disk under `src/trading/`. `suggested_target_usd` sizes each proposed bot from the asset target defaults, and the sector map names each asset's sector.

The pane reads the detector again the moment a scan finishes, on its own Refresh press, and on its ten-minute timer. Both panes answer that read under one name, so the Qt widget and the React host take the same call. A pane whose read raises is asked once, the tab logs a warning, and the scan still finishes.

`src/gui/market_inspector.py` — the one refresh both hosts call

```
def _refresh_proposals(self) -> None:
    """Read the detectors again and redraw the pane, at most once per trigger."""
```

An empty pane says which of three states it is in.

`src/gui/main_tabs/market_inspector_topologies_surface.py` — the three sentences

```
def empty_proposals_text(scan_state: Any) -> str:
    """The sentence an empty proposals pane carries for one scan state."""
```

Before any scan it reads "No proposals yet. The detector reads the closes the scanner writes, so press Refresh on the left half first." While a scan runs it reads "A scan is running. Proposals are built from what it finds." Afterwards it reads "No proposals right now. Try Refresh, or wait for market state to shift."

The pane's button reads Refresh, because the zone title already says what is being refreshed.

`src/gui/main_tabs/market_inspector_topologies_surface.py` — the label and its width

```
REFRESH_TEXT = "Refresh"
REFRESH_TOOLTIP = "Rerun topology detectors on current market state."
REFRESH_WIDTH_PX = 96
HEADER_WORD_WRAP = True
```

`current_topology_proposals` answers two different things. It answers `None` when the right pane never built or refused the read, and a list when the pane answered, so an empty list means the pane holds no proposals. The reader in the Simulator tab passes both answers on rather than flattening them.

`src/gui/main_tabs/market_inspector_surface.py` — the two answers a reader gets

```
    def current_topology_proposals(self) -> Optional[list]:
        """The proposals on display, for a simulator to read and wire.

        ``None`` says the right pane never built or refused the read,
        and a list says the pane answered, so an empty pane and an absent
        one never look alike to a caller.
        """
```

Dismissal

Dismissing a card hides it for a day, and the dismissal survives a restart. The pane writes each held id against the second it lapses at into the operator's settings, under `topology_dismissed_proposals`.

`src/gui/market_inspector_topologies.py` — `_persist_dismissed`

```
def _persist_dismissed(self) -> None:
    """Write ``_dismissed`` to the store, recording a refusal."""
    if self._dismiss_store is None:
        return
    try:
        self._dismiss_store.set(DISMISS_SETTINGS_KEY, dict(self._dismissed))
```

A write the store refuses leaves the card hidden for the session and puts the cause on the line beside Refresh, which reads `Dismissal NOT saved:` in place of the count until the next write lands. The pane stays up either way.

`src/core/settings.py` — `AppSettings.topology_dismissed_proposals` is the field the write lands in. `set_dismiss_store` hands the pane the settings manager it persists through, and the pane never resolves settings itself. Entries already lapsed are dropped at load and written back out, so a day's suppression is not extended across a restart and the stored set does not grow without bound.

Adopt

The pane emits an adopt request and the window handles it. The handler counts the new bots and their combined budget, asks which of the proposed wires already exist and would change, and shows all of it before anything is created.

`src/gui/main_window.py` — `_adopt_topology_proposal`

```
def _adopt_topology_proposal(self, proposal: dict) -> None:
    """Confirm, open the wizard for each new bot, then emit `wire.created`."""
```

An adopt that aborts part way calls `_report_adopt_orphans`, which names the bot ids it created and left unwired. Adopt is the only control on this screen that creates anything.

While the saved fleet is sealed, Adopt shows none of that. The handler asks `src/gui/main_window.py, in _refuse_while_sealed` first, so the press puts one box over the window and one Console line saying to unlock first. No confirm box, no wizard, no wire and no snapshot file follow. The words and the measurement are on the Console page.

```
if self._refuse_while_sealed():
    return
```

Push to the Simulator, the Paper Trader and Live

Both candidate zones carry a Push to Sim button. An open Opposing Trades entry draws it under the pair's headline, and the topology preview draws it beside Adopt. One press sends the candidate's bots to the Simulator, which holds them under its Back Test mode and leaves that mode in force.

`src/gui/main_tabs/market_inspector_surface.py` — the one button both zones draw

```
def push_to_sim_actions() -> list:
    """Push to Sim, the one button an open candidate entry draws."""
    return [
        action_row(
            PUSH_TO_SIM_PART,
            PUSH_TO_SIM_LABEL,
            PUSH_TO_SIM_TOOLTIP,
            PUSH_TO_SIM_WIDTH_PX,
        )
    ]
```

A topology names several bots and the wires between them; an opposing pair names two assets and no wire. Both reduce to the same payload — rows of one market and one dollar target, plus the wires — and the Simulator turns each row into a held bot on the venue its Stone Tablet was recorded on.

`src/gui/main_tabs/market_inspector_topologies_surface.py` — a proposal's payload

```
def proposal_push_candidates(proposal: Any) -> dict:
    """One proposal as ``{"bots", "wires"}``, the payload a push takes.

    Each bot is a ``{"symbol", "target_usd"}`` row and each wire is a
    ``{"source_asset", "target_asset", "pct"}`` row, so the routing the
    proposal describes travels with the bots it names.
    """
```

An asset no Stone Tablet names has no tape for Back Test to walk, so the push is refused whole. The Activity Log names the asset, nothing is held, and the run mode does not move. The same refusal is written when no Simulator tab is built.

`src/gui/simulator/sim_trading_tab_surface.py` — the refusal the operator reads

```
PUSH_NO_TABLET_FORMAT = (
    "Push to Sim refused: no Stone Tablet names {assets}, so Back Test has no "
    "tape to walk. The run mode is unchanged."
)
```

A wire is refused the same way. Its two ends name markets, and the push creates one simulated bot per market, so a wire whose source or target names a market the push did not create has no bot to reach. A wire naming one market at both ends has only one bot to reach, which is what `detect_distance_to_band` builds: it pairs a scrum-deep live bot with a fold-deep live bot on the same market, and those two collapse to one simulated bot. Either gap refuses the whole push, names the wire and says why.

`src/gui/simulator/sim_trading_tab_surface.py` — the wire refusals

```
PUSH_WIRE_UNHELD_FORMAT = "{source} to {target}: nothing was pushed for {asset}"
PUSH_WIRE_SELF_FORMAT = (
    "{source} to {target}: both ends are the one bot pushed for {source}"
)
```

A Target Balance is refused the same way, and on both destinations by the same rule. Each candidate row names one market and one dollar target, and both pushes read that target through one admission: an endless figure, a figure that is not a number, a stored flag, a negative figure and text no dollar can be read from are each refused, the Activity Log names every market refused, nothing is held and the run mode does not move. A row naming no target, or naming zero, takes the configured default instead, which is the `target_balance` field default `BotConfig` itself declares, so a push and the Bot Wizard cannot name two different dollars.

`src/trading/container/config.py, in pushed_target_usd` — the one admission both destinations read

```
PUSHED_TARGET_DEFAULT: float = float(
    next(one.default for one in fields(BotConfig) if one.name == "target_balance")
)
```

A repeated name is refused too. One pair holds one rate, so a topology naming the same source and target twice refuses the whole push, and a candidate naming the same market twice holds one bot for it rather than two.

`src/gui/simulator/sim_trading_tab_surface.py` — the target and repeated-pair refusals

```
PUSH_TARGET_FORMAT = "{asset}: {target} is not a Target Balance a bot can trade"
PUSH_WIRE_TWICE_FORMAT = (
    "{source} to {target} is named twice, and one pair holds one rate"
)
```

A pushed bot is a simulated record, never a live bot and never an order. It reaches the Simulator's own fleet file and its own bus, and the live engine and the live logs see nothing of it. A pushed wire is the same: it is registered on the Simulator's own `SimWireManager` and never on the live `SmartWireManager`.

The Paper Trader has no mode a candidate could land under, so no push targets it. Live takes no push either. The two targets are named on the promotion pipeline page.

Push to Paper

That first sentence is overtaken, and it is kept above as it stands. The Paper Trader still has no mode a candidate could land under, and a push targets it anyway. Both candidate zones carry a Push to Paper button beside Push to Sim, and one press spawns one paper scrumming bot per market the candidate names.

`src/gui/main_tabs/market_inspector_surface.py` — the two buttons an open entry draws

```
def candidate_push_actions() -> list:
    """Both destination buttons an open candidate entry draws, Push to Sim then
    Push to Paper."""
    return [*push_to_sim_actions(), *push_to_paper_actions()]
```

The payload is the same one Push to Sim takes. Paper reads its venue live, so it needs no Stone Tablet: every pushed bot takes the venue the Paper Trader's own `PaperExchange` names. The wires travel with the bots, and a wire that cannot reach two different pushed bots refuses the whole push, exactly as it does on the Simulator.

`src/paper/fleet_source.py` — where a pushed candidate lands on Paper

```
def push_candidate(
    self,
    candidate: Any,
    exchange_id: str = "",
    ta_timeframe: str = "",
) -> dict:
```

No run mode is named anywhere on this press, and the Activity Log line names none. The Paper Trader page carries what the spawned bots read and what each refusal says. Live still takes no push.

ATA-SMP

The eight phases sit outside the screen. Phases one to three and phase eight run in `src/trading/ata_spm.py`, and phases four to seven run in `src/trading/ata_spm_push.py`. The screen reads what they produce and draws it; it computes none of it.

`src/trading/ata_spm.py` — the phases one press runs

```
def run(
    sectors: Any,
    asset_source: Optional[Callable] = None,
    candle_source: Optional[Callable] = None,
    engine: Optional[VotingEngine] = None,
    message_format: Optional[str] = None,
    clock: Optional[Callable] = None,
) -> AtaSpmRun:
    """Phases one, two, three and eight in order, as one ``AtaSpmRun``."""
```

Phase	Module	What it produces
1 Evaluate	`ata_spm.evaluate`	One scan per sector, per ticked timeframe
2 Identify	`ata_spm.identify`	The votes carrying a reversal, strongest first
3 Pull	`ata_spm.pull`	The chart, its bands, and one message per confirming voter
4 Format	`ata_spm_push.format_post`	One post per push target
5 Distribute	`ata_spm_push.distribute`	One delivery record per post
6 Ready to Send	`ata_spm_push.ReadyToSend`	The bucket the operator approves from
7 Follow-Up	`ata_spm_push.FollowUpWatch`	What the market did after a published call
8 Timeframes	`ata_spm.agreement_for`	Every timeframe the asset voted on, and their verdict

A run answers one readback line naming the sectors scanned, the calls found, the charts pulled and the markets the gates refused.

`src/trading/ata_spm.py` — the line a finished run leaves

```
PHASE_RUN_FORMAT = (
    "{phase}: {sectors} sector(s), {calls} call(s), {pulls} chart(s), "
    "{refused} refused by the gates"
)
```

Phase eight answers whether the other timeframes agree with the call. It walks every vote the same asset produced in the run, names the direction each one read, and lists the timeframes that read the opposite. A neutral vote is neither agreement nor contradiction.

`src/trading/ata_spm.py` — the three verdicts phase eight can write

```
AGREEMENT_AGREED_TEXT = "Every timeframe agrees."
AGREEMENT_SINGLE_TEXT = "Only one timeframe voted."
AGREEMENT_CONTRADICTED_FORMAT = "Contradicted on {labels}."
```

Phase eight also caps a scan. Each sector takes only as many timeframes as the rounds so far show will fit inside one candle of the shortest timeframe the class scans. The rest are recorded as deferred and named on the zone, so a shortened scan is visible rather than silent.

The gate chain over scanned prices

One judgement decides whether a market is charted and posted, and it is the live trade gate chain. If the chain says Acervator would take the trade, the chart is drawn and the post joins the approval bucket. If it does not, no chart is drawn and nothing reaches the bucket.

Both chains the bots build are evaluated, unchanged, against a context filled from one market's candles and its voting summary. Nothing on the live path changes: the bots build the only two contexts inside their own tick, and this is a third caller.

`src/trading/ata_gate_scan.py` — both live chains, over scanned prices

```
readings = read_chain(build_scrumming_scrum_chain(), context) + read_chain(
    build_scrumming_fold_chain(), context
)
```

Twenty-four gates run across the two chains, fourteen on the scrum side and ten on the fold side. Eighteen decide from price and settings alone. Six need a position the scan does not hold, and the scan never invents one. Four of the six stand down by name, and two publish a distance instead of a verdict.

`src/trading/ata_gate_scan.py` — the six, split by what a scan can honestly say

```
NOT_APPLICABLE_GATES = (
    "delta_positive",
    "interval",
    "tranches_queued",
    "smart_ceiling",
)
HYPOTHETICAL_GATES = ("hysteresis_scrum", "hysteresis_fold")
```

The four stood-down gates appear in the expansion by name, marked as not run, each with the reason. A post must never imply a gate latched when no gate ran.

`src/trading/ata_gate_scan.py` — the reason each stood-down gate publishes

```
NOT_APPLICABLE_REASONS = {
    "delta_positive": "a scan holds nothing, and no surplus exists to test",
    "interval": "the same surplus, and a scan holds none of it",
    "tranches_queued": "a scan has no fold queue",
    "smart_ceiling": "a scan holds nothing, and any ceiling test passes",
}
```

The two hysteresis gates run against a hypothetical entry at the scanned price and publish the price a reversal would have to reach, with no verdict attached. At the instant of the scan the pivot equals the price, so both sides would refuse; the number is the reading, and the refusal is not.

`src/trading/ata_gate_scan.py` — what a hypothetical row prints

```
HYPOTHETICAL_FORMAT = (
    "entry at ${price:.8f} would need ${required:.8f}, "
    "{distance:.2f}% away; no gate ran"
)
```

Opposing Trade Distance is one sum: the bot's scrumming interval percentage plus its trading fee percentage, clamped between 0 and 50. It is a formula over settings, so a scan computes it exactly as a bot does.

`src/trading/otd_math.py` — the whole formula

```
total = float(interval_pct) + float(fee_pct)
return max(OTD_MIN_PCT, min(OTD_MAX_PCT, total))
```

Current reading. The sum keeps a floor of two fees: where the interval sits under the fee, the distance is two fees. A scan holds no fills, so no venue fee raises the fee it reads from its settings.

`src/trading/ata_gate_scan.py` — `build_context`

```
held = settings if settings is not None else scan_settings(symbol)
distance_held = distance_settings(held)
```

Landing Strip is the second number. The band-proximity detector reads whether price has sat against a Bollinger band for a run of candles, and it answers a side. An upper strip is a sell reversal and a lower strip is a buy reversal. The side and the candle count are both published beside the distance.

`src/gui/main_tabs/market_inspector_surface.py` — the line carrying both numbers

```
GATE_DISTANCE_FORMAT = "opposing trade distance {pct:.2f}% · landing strip {strip}"
```

A second strip detector runs inside the gate scan and feeds the confidence floor rather than the direction. It lowers the floor the TA consensus must clear, the way the live tick lowers it, so the scan and a bot judge a

market at one bar. The one term a scan loses is the band-priority favour, which arms only on a signed delta, so a scan reads at a stricter floor than a bot holding a position. The floor is the chain's own; no second threshold is written on the posting path.

`src/trading/ata_gate_scan.py` — the scan's readings and the floor it reads

```
@dataclass(frozen=True)
class GateScan:
    """Every gate reading over one market, with the side that would fire."""

    confidence_floor: float = 0.0
```

Current reading, the line under the scan's floor. The scan's floor takes the line a bot's floor takes. The favours relax it no further than what the two heaviest voters of the scanned vote reach at full confidence, and never above 0.25. So a scan needs a third agreeing indicator wherever the plain floor needs one.

`src/trading/ata_gate_scan.py` — `confidence_floor`

```
return _skewed_confidence_floor(skew, getattr(summary, "signals", None) or [])
```

Current reading, the scan's floor and its strip. The scan's floor now keeps the line at every skew, so it can rise over 0.25. The scan's own copy of the strip rule is gone. It calls the function a bot calls, so a strip sets the scan's side only when three indicators agree. The scan's LS light reads cyan only while the TA light on its bank shows neither a refusal nor an override.

`src/trading/ata_gate_scan.py` — `build_context`

```
is_bullish, is_bearish = vote_sides(
    direction, confidence, floor, band, getattr(summary, "signals", None) or []
)
```

`src/trading/ta_engine.py` — the candle floor the engine sets

```
MIN_CANDLES_FOR_TA = 30
```

A chart with too few candles produces no gate scan at all. The floor is 30 candles, and the line says so rather than reporting an empty result as a clean one.

`src/gui/main_tabs/market_inspector_surface.py` — the two lines the expansion heads

```
NO_GATE_TEXT = "No gate ran. The chart carried too few candles."
GATE_COUNT_FORMAT = (
    "{ran} of {total} gate(s) ran · {latched} latched · {blocked} blocked "
    "· {stood_down} did not run"
)
```

A refused market is still on the record. Its gate scan is kept, so a reader sees that the market was judged, which gates blocked it and why.

`src/trading/ata_spm.py` — what is drawn and when

```
    image = (
        render_pull_image(vote, candles, max_supporting_indicators, messages)
        if gates.would_fire
        else ChartImage()
    )
```

What a scan shows of your circuit breakers

The scan reads both breaker thresholds and the soft cooldown off the settings it is handed, through the one declaration the trading engine reads, and closes its readings with a row named `circuit_breaker_hard`. The row carries no side, so no chain verdict reads it.

`src/trading/ata_gate_scan.py` — the reading that row carries

```
HARD_BREAKER_FORMAT = (
    "single-candle move {move:.2f}% against your hard {hard:.2f}% "
    "and soft {soft:.2f}% (cooldown {cooldown} candles, soft side {side})"
)
```

Driven on one bar spanning 12.00 % of its open, two scans in the same run:

```
stored soft / hard   state      the row's reading
5.0 / 10.0           blocked    move 12.00% against your hard 10.00% and
                                soft 5.00%, soft side scrum
25.0 / 35.0          latched    move 12.00% against your hard 35.00% and
                                soft 25.00%, soft side none
```

The two rows name different figures, so the reading follows the stored record rather than a figure held inside the scan. Before this the scan held its own copy of the move arithmetic and its own copy of the opening soft figure, named the hard field nowhere, and published no figure at all.

The verdict does not move. Over twenty scans across five tapes and four stored pairs, the firing side, the fire-or-refuse answer and every chain gate's state and wording are identical; the readings per scan go from 22 to 23 and the decided count from 16 to 17, which is the new row. A sweep of 63,168 candle and figure combinations reads the same soft-breaker sides before and after.

No store in the tree holds a breaker figure for a scan, so a scan started from this tab reads the declared 25.0 % and 35.0 % until a caller hands it a record carrying yours.

The voting panel a called asset carries

ATA-SMP carries its own Indicator Voting Panel. It is a clone of the live one, drawn inside an open entry, and it reads only the markets ATA-SMP scanned. No live bot's market reaches it, and it reaches no live bot.

`src/gui/main_tabs/market_inspector_surface.py` — the panel and what feeds it

```
def voting_panel(pull: Any) -> Optional[dict]:
    """The Indicator Voting Panel one scanned asset carries, as its rows.

    The rows are the timeframes ATA-SMP read for this asset alone, and
    every cell is drawn from ``indicator_panel_surface``.
    """
```

The settings page

The Settings button swaps the zone for a page of buttons that takes the whole left column, so nothing is squeezed into a strip and no scroll bar appears. That page is Level 1. A press on a venue button opens that venue's own sign-in page, which is Level 1A.

`src/gui/main_tabs/market_inspector_surface.py` — the two pages the zone shows

```
LEVEL_ONE = "level-1"
LEVEL_ONE_A = "level-1a"
```

Level 1 carries three groups and a Back button. SM Accounts holds one button per push target. Asset Category holds one button per asset class, and the pressed one is the class a scan uses. Settings holds the six values a phase reads.

`src/gui/main_tabs/market_inspector_surface.py` — the three group names

```
SM_ACCOUNTS_TITLE = "SM Accounts"
ASSET_CATEGORY_TITLE = "Asset Category"
ATA_SETTINGS_TITLE = "Settings"
```

Setting	Which phase reads it	Starts at
Max posts per hour	The send ceiling	unset, and an unset ceiling releases nothing
Max supporting indicators	Phase four, capping the evidence lines and the chart overlays	unset, and every confirming voter is drawn
Confirmation share %	Phase seven, sizing the target a call must reach	0, and a call reads open
Standardised message text	Phase three, wording each indicator message	the standard wording
Hits per scan	The walk's stop target	3
Confirmation read candles	Phase seven, spacing each read after the first	3

A held venue credential, the asset class a scan uses and a ticked timeframe are one state, painted in the theme's primary colour under its on-primary text in both builds.

`src/gui/main_tabs/market_inspector_surface.py` — the setting keys and the rows they draw

```
SETTING_CONFIRMATION_SHARE = "confirmation_share_pct"
SETTING_CONFIRMATION_CANDLES = "confirmation_candles"
SETTING_ROWS: tuple[tuple[str, str], ...] = (...)
COUNT_SETTINGS = (SETTING_HITS_PER_SCAN, SETTING_CONFIRMATION_CANDLES)

def setting_rows(settings: Any) -> list:
def set_setting(settings: Any, key: Any, typed: Any) -> None:
```

`SETTING_ROWS` names each row and `COUNT_SETTINGS` says which take a whole number, so both hosts draw the page and write a value with no host code of their own. A count read as text, or under one, takes its own default.

`src/trading/ata_spm_push.py` — the count read safely, and what persists

```
def read_candles(asked: Any) -> int:
    """``asked`` as a whole number, or ``DEFAULT_CONFIRMATION_CANDLES`` below one."""

PERSISTED_SETTINGS = ("hits_per_scan", "confirmation_share_pct", "confirmation_candles")
```

Three settings persist. Hits per scan, the confirmation share and the confirmation read candles are written on every change and read back when the page is built. The other three live for the life of the process.

`src/trading/ata_spm_push.py` — where the persisted settings sit

```
def settings_path() -> Path:
    """``Path.home() / STATE_DIR_NAME / ATA_SPM_SETTINGS_NAME``."""
```

A typed message format may carry four keys in braces. A format naming any other key, or one the formatter cannot parse, no longer stops the scan: the standard wording is written and one warning names the key.

`src/trading/ata_spm.py` — the keys a message format may carry

```
MESSAGE_FORMAT_KEYS = ("label", "reading", "direction", "confidence")
MESSAGE_FORMAT_REFUSED_LOG = (
    "ATA-SPM message format refused, %s; the standard wording is used. Keys: %s"
)
```

The push targets

Ten targets ship. Adding one is adding a row: no phase names a target, and every screen and folder writer reads the rows, so a new row needs no screen edit. Signal publishes no way for a program to post into a group and is not a row.

The operator asked for the three that joined last, and ruled on Signal.

> *ATA-SMP - Need to add post / message distribution for Discord, Telegram, WhatsApp, and Signal.*
>
> *ATA-SMP - For Signal, if its not being used for this sort of thing then we can skip it.*

Target	Body ceiling	Title ceiling	Counted in	Image
X	280	none	Weighted characters	1200 by 675

Target	Body ceiling	Title ceiling	Counted in	Image
Instagram	2200	none	Characters	1080 by 1350
LinkedIn	3000	none	Characters	1200 by 627
TikTok	4000	90	UTF-16 runes	1080 by 1920
Facebook	none published	none	Characters	1200 by 630
Threads	500	none	UTF-8 with emoji	1080 by 1350
Reddit	40000	300	Characters	1200 by 628
Discord	2000	none	Characters	1200 by 675
Telegram	1024	none	Characters	1280 by 720
WhatsApp	65536	none	Characters	1200 by 675

Every number came off the platform's own documentation and each one is cited on its own page. Where a platform publishes no number, the row records that rather than carrying a guess. TikTok's title is empty because the header alone is over 90.

The push target rules — one section per target, with the page each number came from

```
  Facebook   body ceiling: no number published
             the row carries NO_LIMIT_PUBLISHED, and phase four applies none
```

Two platforms named at the start are not rows, and both were left out for the same reason: a rendered chart has no route in. One publishes an idea through its website and states that it has no API for it. The other's upload endpoint accepts video only.

One row describes one target, and one tuple of rows is every screen's source.

`src/trading/ata_spm_push.py` — the row, and the names every screen reads

```
  @dataclass(frozen=True)
  class PushTarget:
      """One push target: its sections, its ceilings, its header and its image size."""

  PUSH_TARGETS: tuple[PushTarget, ...] = (...)
  TARGET_NAMES = tuple(one.name for one in PUSH_TARGETS)
  SECTION_CALL = "call"
  SECTION_INDICATORS = "indicators"
  COUNT_WEIGHTED = "weighted"
```

`src/trading/ata_spm_push.py` — the header one row takes, and the row that names its own

```
TARGET_X = "X"
TARGET_WHATSAPP = "WhatsApp"

PushTarget(
    TARGET_X,
    (SECTION_CALL, SECTION_INDICATORS),
    body_limit=280,
    count_unit=COUNT_WEIGHTED,
    header=X_HEADER,
    image_width_px=1200,
    image_height_px=675,
)

def target_header(target: Any) -> str:
    """The row's own header, ``FIXED_HEADER`` where the row names none."""
```

The post text

One routine writes every piece of post text. It puts the header on top, the evidence under it, and the organisation address at the bottom. No caller supplies the address and no caller can remove it. The body, the caption, the thread root and the title all carry it, on every target.

`src/trading/ata_spm_push.py` — the post, written once

```
def compose(lines: Any, header: Any = FIXED_HEADER) -> str:
    """``header`` over ``lines`` over ``ORGANIZATION_URL``.

    ``fit_to_target`` drops ``lines`` to reach a ceiling and reaches neither
    ``header`` nor ``ORGANIZATION_URL``.
    """
```

The header is a field on the venue's row, and every row takes the fixed header unless it names its own. Nine rows take it and the X row names a short header. A venue is changed by editing its row; nothing branches on a venue's name.

`src/trading/ata_spm_push.py` — the two headers and the address

```
FIXED_HEADER = (
    "This is not investment advice. It is a demonstration of Ekthelius's "
    "proprietary TA engine housed in the Acervator governance execution "
    "platform."
)
X_HEADER = "Not investment advice. Acervator TA engine demonstration."
ORGANIZATION_URL = "https://github.com/Acervator-LLC"
```

The fixed header measures 144 in every unit, because it is plain ASCII. On X, with the call line, the address at 23 weighted characters and two separators, the floor is 122 of 280, leaving 158 for evidence. The short header measures 57 weighted units and keeps three facts: that it is not investment advice, that it is Acervator, and that it is a TA engine demonstration. The words are the operator's to change; the three facts are not.

He asked for the shorter one.

Should shorten specifically for X and maintain a consistent level of valuable data...

Phase four holds each body under its target's ceiling. It drops evidence lines, longest first, and never the header or the address. A line drops whole, so no price or band value is cut mid-digit, and the post says how many sentences were left out.

`src/trading/ata_spm_push.py` — what the fitter drops, and what it will not

```
def fit_to_target(ranked: Any, target: Any) -> tuple:
```

Where a venue holds a title, the title stands on the first line, then a blank line, then the body.

The chart a post carries

The picture is drawn by the Charts tab's own renderer. No second drawing engine was written for posts, because two engines drift apart and only one of them is the screen the operator watches. The renderer's drawing state and paint routine sit in a plain object. The Charts tab's chart is that object with a window around it, and a post image is that object with a picture file around it.

`src/gui/native_chart.py` — one paint routine, two places to send it

```
class ChartPainter:
    """The chart's drawing state and its paint routine, with no window around it."""

        def paint_to(self, p: QPainter, w: int, h: int) -> None:
            """Draw the whole chart onto ``p`` over a ``w`` by ``h`` area.

            The painter's device is the caller's: a widget from ``paintEvent``
            and a ``QImage`` from ``render_chart_png``.
            """
```

The operator asked for the two to be one renderer.

> *Refining our chart renderer (the one currently used under the Asset Charts Tab) will allow us to upgrade the visuals for one and align this to visuals of out going SM posts thus unifying direct usage with SM exposure.*
>
> *Refining our chart renderer ... will allow us to upgrade the visuals for one and align this to visuals of out going SM posts thus unifying direct usage with SM exposure.*
>
> *let's migrate to the Charts tab and do its visual upgrades. After its chart renderer is fully upgraded, it will need to be migrated to ATA-SMP.*
>
> *After its chart renderer is fully upgraded, it will need to be migrated to ATA-SMP.*

The split was forced by where a scan runs. Pressing Scan Now hands the work to a background worker so the screen keeps drawing, and the drawing toolkit will not build a window on a background worker. Driven, it does not raise an error that could be caught; it kills the program. A picture file has no such rule, so the post image is drawn straight onto the file and no window is made.

`src/gui/market_inspector.py` — the worker the scan runs on

```
            self._scan_thread = threading.Thread(
                target=self._compute_scan,
                args=(settings.message_format, settings.max_supporting_indicators),
                name=ATA_SCAN_THREAD_NAME,
                daemon=True,
            )
```

The indicators on the picture are the ones that voted for the call, not the ones the Charts tab happens to have switched on. Each overlay in the chart's registry names the voter it draws, so the picture is chosen by the vote. All twelve voters carry an overlay, and the header line names any voter the cap cut.

`src/gui/native_chart.py` — the overlays one vote draws

```
def overlays_for_voters(voters: Any, max_overlays: int = NO_OVERLAY_CAP) -> tuple:
    """The overlay keys drawing ``voters``, and the voters no overlay draws."""
```

`src/gui/native_chart.py` — the overlay names its voter

```
    ChartOverlay(
        key="bb",
        label="BB",
        colour_field="chart_band",
        pane=PRICE_PANE,
        occludes=False,
        draw="_draw_bollinger",
        tooltip="Bollinger Bands (20, 2σ) with cloud fill",
        voter="bollinger_bands",
    ),
```

Max supporting indicators decides how many are drawn. It was already the cap on how many confirming sentences a post carries, and it is the same cap on the chart. No second setting was added, and an unset value caps nothing.

`src/trading/ata_spm.py` — the unset cap, and the voters one call confirmed

```
NO_INDICATOR_CAP = 0

def confirming_signals(vote: Any) -> list:
def timeframe_label(timeframe: Any) -> str:
```

`src/gui/native_chart.py` — what reached the picture and what did not

```
@dataclass
class ChartImage:
    """One rendered chart on disk, and what the renderer could not draw.

    ``drawn`` and ``undrawn`` name voters: ``undrawn`` holds the ones no
    overlay draws and the ones ``max_overlays`` cut.
    """
```

The chart states the call. Three marks are drawn, and each one is earned. A badge in the top right names the direction. A dashed rule and a triangle mark the last bar, which is the bar the vote was made on. A strip under the time axis carries one row per confirming voter: a square in the colour of the line that drew that voter, and the sentence phase three wrote for it. A voter with no line still gets a row, in the dim colour. Nothing is drawn that a voter did not read.

`src/gui/native_chart.py` — the direction and the readings the picture takes

```
        def set_call(self, direction: str, readings=()) -> None:
            """Take one reversal direction and one reading line per voter.

            ``readings`` are ``(voter, text)`` pairs and ``_draw_call`` paints
            them under the time axis.
            """
```

A chart with no call set draws no badge, no bar mark and no strip, and keeps the height it had before. The Charts tab is unchanged by any of it.

Phase three answers one record per call, and that record carries the file.

`src/trading/ata_spm.py` — the picture the pull holds

```
@dataclass
class ChartPull:
    """The chart one reversal call was made on, and its confirming messages.

    ``image`` is that chart rendered to a PNG, which the post's caption
    captions.
    """
```

A venue image whose caller names no theme paints in the theme the window is in, and the message stamp at its foot takes that theme's own axis text colour. A venue image carries no resize grip; only a chart in a window draws one.

`src/gui/native_chart.py` — the theme one image paints in

```
painter.set_theme(tokens if tokens is not None else theme_in_force())
```

Each venue takes its image at the size that venue's own page publishes for a single image post. The two Meta feeds take the tallest shape one of them accepts, which gives the panes the most room.

`src/trading/ata_spm_push.py` — the two figures each row carries

```
    image_width_px: int = DEFAULT_IMAGE_WIDTH_PX
    image_height_px: int = DEFAULT_IMAGE_HEIGHT_PX
```

A height the natural layout fits draws the natural layout, and any height left over goes to the price pane. A height the layout does not fit compresses in one order: each sub-pane shrinks from its readable height toward its fold height; then the reading strip folds from one row per reading to one row of voter names with their swatches; then the price pane shrinks from its layout floor toward its paint floor. The strip folds because the caption

already carries every reading's sentence.

`src/gui/main_tabs/market_inspector_surface.py` — the grid's own column widths

```
PANEL_TF_WIDTH_PX = 44
PANEL_CELL_WIDTH_PX = 58
PANEL_NET_WIDTH_PX = 52
PANEL_CONF_WIDTH_PX = 96
```

`src/gui/main_tabs/native_chart_surface.py` — the four heights the fold moves between

```
TAG_HEIGHT_PX = 14
SUB_PANE_PLOT_BANDS = 7
SUB_PANE_LABEL_PX = TAG_HEIGHT_PX
SUB_PANE_READABLE_PX = SUB_PANE_LABEL_PX + SUB_PANE_PLOT_BANDS * TAG_HEIGHT_PX
SUB_PANE_FOLD_PX = 28
PRICE_PANE_LAYOUT_FLOOR_PX = 220
PRICE_PANE_PAINT_FLOOR_PX = 120
```

A sub-pane's readable height is 112 pixels, a label row over seven plot bands, and it folds to 28.

A height under the least layout is refused by name. The folder keeps its text file and the emitter row for that venue reads not ok.

`src/gui/native_chart.py` — the refusal a short image takes

```
NO_IMAGE_HEIGHT = 0

def strip_folded(width: int, height: int, readings: Any) -> bool:
def _least_height_for_panes(width_px: int) -> int:
def _minimum_height_for_panes(width_px: int) -> int:

IMAGE_TOO_SHORT_NOTE = (
    "{width}x{height} cannot hold the panes: {least} px is the least "
    "height at that width."
)
```

Each chart image carries its wording on it: the header, the call's own headline, then the address. The message takes 55 pixels at the foot of the image, 8 of padding above, 8 below and 13 for each of its three lines. A chart given no message is 55 pixels shorter and carries none.

`src/trading/ata_spm.py` — the wording one image carries

```
def post_caption(vote: AssetVote) -> str:
    """The header, the call's headline and the address, as the image's foot."""
```

Each bucket entry draws that picture scaled to the entry's width: a thumbnail closed and a larger view open, each at the height its own shape needs. An entry whose post names no file on disk draws a rectangle strip.

`src/gui/main_tabs/market_inspector_surface.py` — the two sizes one entry draws

```
THUMBNAIL_WIDTH_PX = 120
THUMBNAIL_HEIGHT_PX = 36
PREVIEW_WIDTH_PX = 320
PREVIEW_HEIGHT_PX = 160
```

`src/gui/main_tabs/market_inspector_surface.py` — the picture one entry draws

```
NO_IMAGE_SIZE = (0, 0)

def chart_image(post: Any) -> tuple:
    """That post's own PNG as a data address and its size, or ``NO_IMAGE_SIZE``."""
```

The Qt entry decodes that address into an image; the page places one image element with the same address in the same box. Both hosts read one `post_chart` payload.

The venue folders

The chart root holds one folder per venue, named for it. Each venue folder holds that venue's own version of every call: the chart with the venue's message at its foot, the same message as plain text beside it, and for X and WhatsApp a compose address as an Internet Shortcut.

`src/trading/ata_post_paths.py` — the root and one venue's folder

```
ATA_POST_ROOT: Path = Path.home() / ".acervator_ata_posts"

def venue_post_root(venue: Any, root: Any = None) -> Path:
    """One venue's own folder under the post root."""
```

The operator asked for the folders and for the message on each image.

> *Ready to Send Bucket - Want a folder button that opens the chart image directory. Want the standardized message marked on each image. Want this loading push candidates via a functioning market scanner...*
>
> *Should have a catered folder for each venue.*
>
> *Seems most of this stuff is price gated so will also need a folder where charts customed for each venue can be viewed and manually posted.*

The files are written inside phase three from the candles already held, in the same call that draws the root picture, so no venue is asked for a candle twice. A market the live gate chains refuse writes nothing anywhere.

`src/trading/ata_venue_folders.py` — the folder write

```
def write_venue_posts(vote: Any, candles: Any, posts: Any) -> dict:
def write_text(path: Path, body: str) -> None:
```

`src/gui/native_chart.py` — the wording the painter stamps at the foot

```
        def set_caption(self, text: str) -> None:
```

`FormattedPost.folder_path` names the venue folder each post's files sit in.

`src/trading/ata_post_paths.py` — the files one venue folder holds

```
POST_TEXT_SUFFIX = ".txt"

def get_ata_post_root() -> Path:
def post_image_path(symbol: Any, timeframe: Any, stamp: Any) -> Path:
def prune_post_images(path: Path) -> tuple:
```

`src/trading/ata_spm_push.py` — what one post names on disk

```
@dataclass(frozen=True)
class FormattedPost:
    """One venue's post: its body, its title, and the files its folder holds."""

def venue_files(pull: Any, target: Any) -> tuple:
```

`src/trading/ata_venue_folders.py` — the two compose addresses

```
X_INTENT_FORMAT = "https://x.com/intent/post?text={text}"
WHATSAPP_INTENT_FORMAT = "https://wa.me/?text={text}"
INTENT_SHORTCUT_FORMAT = "[InternetShortcut]\nURL={url}\n"
```

The folder keeps the newest picture of each market and removes the older ones. A market is one asset on one timeframe, so one asset on the hourly chart and the same asset on the daily chart are two markets and each keeps its own newest picture.

`src/trading/ata_post_paths.py` — how many stay, and why that number

```
#: ``ata_spm_push.format_run`` keys its pulls by symbol and timeframe, last
#: write winning, so one image per market is every image a post can name.
POST_IMAGES_KEPT_PER_MARKET = 1
```

Nothing in the application can name an older picture, so nothing loses one. Phase seven does not need one either: it reads the chart again from the market data, never from the folder, and the post it writes is text.

The trim happens on the same press that draws a picture, so the folder is never larger than one scan's worth of pictures plus the newest of every market scanned before. It reads one folder, the one its own file names, and goes no deeper. A file whose name a post picture could not have produced is not touched. A picture being written is never taken: the protection is the operating system's own refusal, and a refused file survives, is reported as refused, and goes on the next press.

`src/trading/ata_post_paths.py` — the line every trim writes

```
PRUNE_LOG = (
    "ATA post store: removed %d image(s), reclaimed %d byte(s), kept %d, refused %d"
)
```

The Chart Folder button opens the root, so every venue folder is in view, and the press writes one Activity Log line naming the folder it opened. The folder's address is handed to the operating system's own handler. No

browser is driven and nothing is typed into one.

`src/gui/main_tabs/market_inspector_surface.py` — the line and the hand-off

```
CHART_FOLDER_OPENED_LOG = "ATA chart folder opened: %s"

def chart_folder_line(root: Any) -> str:
def page_links(page: Any) -> tuple:
def link_segments(text: Any) -> list:
```

`src/trading/ata_spm_push.py` — the hand-off to the operating system

```
def open_path(path: Any) -> bool:
```

`page_links` is the whole list a press may name on the open page. Any other address opens nothing and the window records the refusal.

Sending

Every post waits in the approval bucket. The operator approves or declines each one, and no button sends a declined post.

`src/gui/main_tabs/market_inspector_surface.py` — the five buttons the bucket carries

```
APPROVE_LABEL = "Approve"
DECLINE_LABEL = "Decline"
POST_SELECTED_LABEL = "Post Selected"
POST_ALL_LABEL = "Post All"
FULL_AUTO_LABEL = "Send Bucket Full Auto"
```

The three send buttons share one width, the width that already held the longest wording, and wrap at the count the zone holds: one at a 700-pixel tab, two at 900, three at 1400 and above.

`src/gui/main_tabs/market_inspector_surface.py` — the shared button width

```
BUCKET_BUTTON_WIDTH_PX = 184
```

Each post takes one of three routes. The route is decided by the post's own venue and what is held for it.

`src/trading/ata_spm_push.py` — the three routes

```
ROUTE_API = "api"
ROUTE_INTENT = "intent"
ROUTE_FOLDER = "folder"
```

Route	Which posts take it	What a press does
api	Discord and Telegram, once held and the ceiling is set	one request carrying the text and the picture
intent	X and WhatsApp	hands the shortcut to the operating system's handler; the picture is attached by hand

Route	Which posts take it	What a press does
folder	every other venue	hands the venue folder to the operating system's handler

Discord and Telegram are the two API senders. Both take a typed credential rather than a browser sign-in.

`src/trading/ata_spm_send.py` — the two senders

```
SEND_ROUTES = {
    TARGET_DISCORD: send_discord,
    TARGET_TELEGRAM: send_telegram,
}

def build_sender(settings: Any, transport: Any = None) -> ApiSender:
```

The two target names those keys use are declared beside the rows, in `src/trading/ata_spm_push.py`.

`src/trading/ata_spm_push.py` — which route a post takes, and which targets a sender serves

```
def route_for(post: Any, settings: Any) -> str:
def deliver_one(post: Any, sender: Any, now: Any) -> Any:
def sender_takes(sender: Any, target: Any) -> bool:
```

A sender serves every target while it names none of its own, which is what a `RecordedDestination` does, and otherwise only the ones it names.

One request per post, as multipart form data, through the same address opener the sign-ins use. A venue answering 429 is read for its wait. A 401 or 403 is read as the credential refused, with what the venue said. A caption over its venue's ceiling is refused before any request, with the ceiling and the measure named. Nothing retries by itself; the post waits in the bucket for the next press.

`src/trading/ata_spm_send.py` — the three refusals a send reports

```
RATE_LIMITED_FORMAT = "429, retry after {seconds:g} s"
CREDENTIAL_REFUSED_FORMAT = "{http}, the credential was refused: {said}"
HTTP_REFUSED_FORMAT = "{http}, {said}"
```

The operator set the bound on how far this goes.

> *This will have to do. Do not want to build something that risks myself or others being banned.*

All three send routes obey one ceiling, counted over the hour ending now. The ceiling starts unset, and an unset ceiling releases nothing: a fresh install cannot publish until the operator sets a number.

`src/trading/ata_spm_push.py` — the ceiling every send route obeys

```
def allows(self, now: float, ceiling: Any) -> bool:
    """Whether one more post fits under ``ceiling`` at ``now``."""
    limit = int(ceiling or NO_CEILING_SET)
    if limit <= NO_CEILING_SET:
        return False
    return self.sent_within_hour(now) < limit
```

`src/trading/ata_spm_push.py` — the two refusals a held post reports

```
NO_CEILING_TEXT = "Max posts per hour is unset. Nothing leaves."
RATE_HELD_TEXT = "{sent} post(s) sent this hour, ceiling {ceiling}."
```

A second guard holds a repost back. A ticker reaches one push target once an hour, whatever composed the post, so a second call on the same ticker is refused exactly like a repeat of the first.

`src/trading/ata_spm_push.py` — the hour the guard measures

```
def allows(self, now: float, symbol: Any, target: Any) -> bool:
    """Whether an hour has passed since ``symbol`` last reached ``target``."""
    gap = self.since(now, symbol, target)
    return gap is NO_SEND_RECORDED or gap >= SECONDS_PER_HOUR
```

`src/trading/ata_spm_push.py` — what a held post reports

```
REPOST_HELD_FORMAT = (
    "{symbol} reached {target} {minutes:.0f} minute(s) ago; "
    "one post per ticker per hour."
)
```

Each bucket entry's status line carries its venue, its approval state, and then the last press's outcome.

`src/trading/ata_spm_push.py` — the five outcomes a status line can carry

```
STATUS_SENT_FORMAT = "sent · {destination}"
STATUS_NOT_SENT_FORMAT = "not sent · {detail}"
STATUS_IN_FOLDER_TEXT = "in folder"
STATUS_INTENT_OPENED_TEXT = "intent handed to the OS"
STATUS_INTENT_READY_TEXT = "intent ready"
```

Before any press the badge carries the confirmation timer's status instead.

`src/gui/main_tabs/market_inspector_surface.py` — the badge one entry's head row draws

```
def bucket_badge(held: Any) -> tuple:
    """The last press's delivery, else the timer's own status."""
```

`src/trading/ata_venue_folders.py` — the text one post writes beside its picture

```
def post_text(post: Any) -> str:
```

Every press runs on its own thread, one per press, and records cross back to the window thread. A press made while one runs is refused on the Activity Log.

`src/gui/market_inspector.py` — the hand-off thread

```
HAND_OFF_THREAD_NAME = "ata-smp-hand-off"
HAND_OFF_BUSY_TEXT = "ATA-SPM %s pressed while a hand-off is running; press ignored"
```

`src/trading/ata_spm_push.py` — the lines a press writes

```
PRESS_LINE_FORMAT = "ATA-SPM hand-off: {line}"
NOTHING_APPROVED_TEXT = (
    "ATA-SPM Post All: no post is approved. Approve one, then press again."
)
FULL_AUTO_TOGGLED_FORMAT = "ATA-SPM Send Bucket Full Auto: {state}."
```

Every send writes one block on the Live tab's API Interaction Log naming the venue, the action, the post, the text length, the image size and the message id or the refusal. The recorded endpoint names the route with the token left out.

`src/trading/ata_spm_send.py` — the two recorded endpoints

```
DISCORD_LOGGED_ENDPOINT = DISCORD_API_BASE + "/<id>/<token>?" + DISCORD_WAIT_QUERY
TELEGRAM_LOGGED_ENDPOINT = TELEGRAM_API_BASE + "/bot<token>/" + TELEGRAM_SEND_PHOTO
```

Every entry the connector records reaches that log from whatever thread made it. The main window carries one signal and one receiver, and the Live tab registers the receiver, so an entry from a scan thread crosses the signal and lands on the GUI thread. The receiver refuses any other thread and writes a violation file instead.

`src/gui/main_window.py` — the crossing every API entry takes

```
    apiEntryLogged = Signal(object)

    def _cross_api_event(self, entry: Any) -> None:
    def _on_api_event(self, entry: Any) -> None:
```

The scan's own progress and its failure cross the same way, and the tab hands the inspector the Live tab's Activity Log pane. Where that pane is absent the phase lines reach the log alone, and the tab says so.

`src/gui/market_inspector.py` — the three signals and the log the tab writes to

```
    scanProgressed = Signal(object)
    scanFailed = Signal(str)
    ACTIVITY_INFO = "info"

    def _say(self, message: str, level: str = ACTIVITY_INFO) -> None:
    def _on_scan_progress(self, progress: Any) -> None:
    def _set_scan_busy(self, busy: bool) -> None:
    def set_activity_log(self, log: Any) -> None:
```

The venue sign-in pages

Level 1A shows the venue name, the address it posts to, the permissions it asks for, and one box per value that venue needs. It shows no other venue's boxes.

`src/trading/ata_spm_push.py` — what one press opens

```
    def open_credentials(self, target: Any) -> Optional[str]:
        """Show one push target's Level 1A page, and answer which target it draws."""
        found = push_target(target)
        if found is None:
            return None
        self.settings_open = True
        self.credential_target = found.name
        self.connect_result = None
        return self.credential_target
```

Each venue row declares two sets. The fields are the values the operator types, which a venue hands him when he registers an application. The issued set is the values the sign-in obtains, which are never typed.

Venue	Boxes on its page	The sign-in issues	Posts to
X	Client ID, Client secret	Access token, Refresh token	`https://api.x.com/2/tweets`
Instagram	App ID, App secret	Instagram user id, Access token	`/<IG_ID>/media` then `/<IG_ID>/media_publish`
LinkedIn	Client ID, Linkedin-Version	Access token	`https://api.linkedin.com/rest/posts`
TikTok	Client key, Client secret, Verified URL prefix	Open id, Access token, Refresh token	`/v2/post/publish/content/init/`
Facebook	App ID, App secret	Page id, Page access token	`/<page_id>/feed` and `/<page_id>/photos`
Threads	App ID, App secret	Threads user id, Access token	`/<threads-user-id>/threads` then `/threads_publish`
Reddit	App ID, App secret, Subreddit, User agent	Access token, Refresh token	`https://www.reddit.com/api/v1/access_token` then `/api/submit`
Discord	Webhook URL	none	the channel's own webhook
Telegram	Bot token, Chat id	none	the bot's `sendPhoto`
WhatsApp	none	none	the Click-to-Chat address in the venue folder

Every value came off that platform's own published documentation, recorded in the platform rules audit.

Seven venues use three-legged OAuth: not one issues a working token from an app id and a secret alone. The operator sends himself to the venue in a browser, approves there, and the venue sends a code back to the program. A venue reads as held on Level 1 only once both sets are in the vault, so the button turns on when the sign-in has actually run.

Venue	Approval address	Token address
X	`https://x.com/i/oauth2/authorize`	`https://api.x.com/2/oauth2/token`
Instagram	`https://www.instagram.com/oauth/authorize`	`https://api.instagram.com/oauth/access_token`
LinkedIn	`https://www.linkedin.com/oauth/native-pkce/authorization`	`https://www.linkedin.com/oauth/v2/accessToken`

Venue	Approval address	Token address
TikTok	`https://www.tiktok.com/v2/auth/authorize/`	`https://open.tiktokapis.com/v2/oauth/token/`
Facebook	`https://www.facebook.com/v25.0/dialog/oauth`	`https://graph.facebook.com/v25.0/oauth/access_token`
Threads	`https://threads.com/oauth/authorize`	`https://graph.threads.com/oauth/access_token`
Reddit	`https://www.reddit.com/api/v1/authorize`	`https://www.reddit.com/api/v1/access_token`

X, LinkedIn and TikTok require PKCE for a desktop program, so those three send a `code_challenge` on the approval call and a code verifier on the token call. Every route sends its own `redirect_uri` on both calls.

Three venues take a second call after the exchange: Instagram and Threads trade the first token for one that lasts sixty days, and Facebook trades for a long-lived user token and then reads the Page token off the account. Reddit authenticates with HTTP Basic, refuses a generic user agent, and issues no refresh token unless the request asks for a permanent duration. TikTok names its client field `client_key`. LinkedIn's native address takes no client secret at all.

Venue	How it renews
X	by itself, refresh token grant, which needs `offline.access`
Reddit	by itself, refresh token grant
TikTok	by itself, refresh token grant, valid 365 days
Instagram	by itself, once the token is 24 hours old
Threads	by itself, once the token is 24 hours old
Facebook	no renewal; a long-lived Page token carries no expiry date
LinkedIn	does not renew; partner-only, and he approves again every 60 days

Six of the seven bind a loopback listener. It binds the loopback address and nothing else, serves exactly one request, refuses a reply carrying a state it did not generate, and closes. It never outlives one sign-in.

`src/trading/ata_spm_signin.py` — the listener, and the two ports

```
LOOPBACK_HOST = "127.0.0.1"
EPHEMERAL_PORT = 0
FIXED_CALLBACK_PORT = 8723
CALLBACK_TIMEOUT_SECONDS = 180.0
TRANSPORT_TIMEOUT_SECONDS = 30.0
```

Four venues check the redirect character for character and publish no wildcard, so no address a random port produces can ever match. Those four bind one declared port, and Level 1A prints it. LinkedIn asks a native client for a random port and TikTok publishes a wildcard port, so those two keep the operating system's port.

Venue	The redirect to register
X	`http://127.0.0.1:8723/callback`
Instagram	`http://127.0.0.1:8723/callback`

Venue	The redirect to register
LinkedIn	nothing; its native page asks for no registered address
TikTok	`http://127.0.0.1:*/callback`
Facebook	`https://www.facebook.com/connect/login_success.html`
Threads	`http://127.0.0.1:8723/callback`
Reddit	`http://127.0.0.1:8723/callback`

Facebook is the seventh, and it opens no listener. Meta requires HTTPS for an OAuth redirect and publishes one desktop address, and it answers a desktop app in the fragment of that address rather than with a code. A fragment never leaves the browser, so the program draws the browser itself, for one sign-in at a time.

`src/trading/ata_spm_signin.py` — the desktop reply Facebook sends

```
DESKTOP_RESPONSE_TYPE = "token"
FACEBOOK_DESKTOP_REDIRECT = "https://www.facebook.com/connect/login_success.html"
```

`src/gui/sign_in_view.py` — the browser the program draws

```
class SignInView(QDialog):
    """One modal sign-in view, one web view, no address bar and no context menu."""
```

The sign-in view reaches exactly two addresses: the venue's approval address and the venue's published redirect. Every other host ends the sign-in on the spot. It opens no second window, cancels every download the venue's page asks for, and empties the cookie jar of the off-the-record profile it held. Reaching the redirect ends the sign-in and the reply's fragment is read. The operator closing the window ends it, and so does 180 seconds passing.

`src/trading/ata_spm_signin.py` — the two hosts one sign-in may reach

```
def sign_in_hosts(authorize_url: Any, redirect_address: Any) -> tuple:
    """The only two hosts one sign-in view may load."""
```

Discord and Telegram issue no token, so Connect checks that the typed value reads as the form the venue's own page prints. It opens no browser and reaches no venue. A webhook address or a bot token in another form is refused on the page, with the form named. WhatsApp needs no sign-in at all.

`src/gui/main_tabs/market_inspector_surface.py` — the two wordings a typed venue carries

```
SIGN_IN_TYPED_TEXT = (
    "Connect checks what you typed and holds it. No browser opens and no "
    "venue is reached."
)
SIGN_IN_NONE_TEXT = (
    "No sign-in. The post goes through the venue folder and the compose address."
)
```

Level 1A returns to Level 1 by itself only when the venue accepts. An empty box, a missing route and a refusal from the venue each keep the page open and print what failed.

`src/trading/ata_spm_push.py` — the four wordings Connect can print

```
  CONNECT_OK_FORMAT = "{target} accepted the credential."
  CONNECT_FAILED_FORMAT = "{target} refused the sign-in: {error}"
  MISSING_FIELD_FORMAT = "{label} is empty."
  NO_CONNECTOR_FORMAT = "No sign-in route wired for {target}."
```

A refusal names the venue's own reason. The published standard puts that reason in two fields, and only those two are read out.

`src/trading/ata_spm_signin.py` — the three refusal wordings

```
  HTTP_REFUSAL_FORMAT = "HTTP {status} {reason}"
  VENUE_SAID_FORMAT = "{http}, and the venue said {said}"
  VENUE_REASON_FORMAT = "{error}: {description}"
```

Every outcome writes one log line naming the target, whether it succeeded, and the detail. The message line takes one colour per outcome, read by both builds: no colour before a press, the theme's error colour on a refusal, and its success colour on an acceptance.

`src/gui/main_tabs/market_inspector_surface.py` — the colour one outcome takes

```
  NO_COLOR = ""
  MESSAGE_REFUSED_COLOUR = ds.ERROR
  MESSAGE_ACCEPTED_COLOUR = ds.SUCCESS

  def message_colour(answered: Any) -> str:
      """``NO_COLOR`` before a press, else the refused or the accepted colour."""
```

A pasted value is trimmed before it is judged and before it reaches a venue, and a box holding only spaces is still reported empty.

`src/trading/ata_spm_push.py` — what Connect reads, and what it answers

```
  def connect_credentials(self, target: Any) -> ConnectResult:
  def missing_field(self, target: Any) -> Optional[str]:
  def typed_credential(self, target: Any, field: Any) -> str:
  def set_credential_text(self, target: Any, field: Any, typed: Any) -> None:
  def store_credential(self, target: Any, field: Any, value: Any) -> bool:
  def set_connector(self, connector: Any) -> None:
  def set_vault(self, vault: Any) -> None:
  CONNECT_RESULT_LOG = "ATA-SPM sign-in on %s: accepted=%s, %s"
```

`src/trading/ata_spm_signin.py` — the two typed routes, and how a refusal is read

```
REFUSAL_BODY_LIMIT = 4096

class SignInError(Exception):
    """A venue's own refusal, carrying ``refusal_text``."""

def sign_in_discord(typed: Any, session: Any) -> Any:
def sign_in_telegram(typed: Any, session: Any) -> Any:
def urlopen_transport(request: Any) -> Any:
```

Each host builds the session those routes run in, through `sign_in_session` on its own side.

A refusal body is read up to that limit and no further.

`src/gui/sign_in_view.py` — what the sign-in view refuses

```
class SignInPage(QWebEnginePage):
    def createWindow(self, kind: Any) -> None:
        """None, so no second window opens."""

    def _refuse_download(self, item: Any) -> None:
    def _release(self) -> None:
    def reach(self, url: Any) -> bool:
```

The credential vault

The tokens are kept in one file outside the repository, encrypted under the same passphrase every other stored credential in the product uses.

`src/core/encryption.py` — where the tokens sit

```
DEFAULT_VAULT_PATH = Path.home() / ".acervator" / "ata_spm_credentials.json"
```

A keystroke writes memory and a finished box writes the vault. A box is finished when the operator leaves it. A held value stores under its own key and an emptied box deletes that key. A vault write runs the key derivation at the published iteration count, which is why a write on leaving the box costs one such write rather than one per keystroke.

`src/core/encryption.py` — the derivation cost

```
_KDF_ITERATIONS = 600_000  # OWASP PBKDF2-HMAC-SHA256 guidance
```

`src/trading/ata_spm_push.py` — the key one value is held under

```
VAULT_KEY_FORMAT = "{target}:{field}"

def vault_key(name: Any, field: Any) -> str:
```

`src/core/encryption.py` — the vault, and the two reads the page makes on every paint

```
class CredentialVault:
    def store(self, key: str, value: str, salt: str) -> None:
    def delete(self, key: str) -> None:

class FileVault(CredentialVault):
    """A ``CredentialVault`` reading ``DEFAULT_VAULT_PATH`` and writing it on store."""

def default_vault() -> FileVault:
def vault_phrase(username: str) -> str:
def has_exchange(key: str) -> bool:
def encrypt(value: str, phrase: str) -> str:
```

`has_exchange` answers whether a value is held without decrypting one, which is why Level 1A can ask it on every paint.

Nothing typed on the page reaches a view model, a render or a log. The page publishes each box as a name and a wording and no value, plus the keys the vault holds something for. A held box's wording says so, and typing into it replaces what is held when the operator leaves it.

`src/gui/main_tabs/market_inspector_surface.py` — what one page publishes

```
def credential_page(target: Any) -> dict:
    """One venue's page: its lines, its boxes as name and wording, and no value."""
```

`src/trading/ata_spm_push.py` — the keys the vault holds something for

```
def held_fields(self, target: Any) -> tuple:
```

`src/gui/main_tabs/market_inspector_surface.py` — the wording a held box carries

```
CREDENTIAL_HELD_PLACEHOLDER = "Held · type to replace"
```

`src/trading/ata_spm_push.py` — what a credential row on Level 1 publishes

```
CREDENTIAL_HELD_TEXT = "held"
CREDENTIAL_MISSING_TEXT = "not held"
```

A page the operator filled before a restart takes Connect without being typed again. There is no Save credentials control: Connect is what stores. The exchange keys sit in the settings file and the push-target tokens in the vault file, two files under one passphrase.

What each venue needs before its page can work

Every platform issues these values only to an application the operator has registered with that platform. The software cannot obtain a registration, so each page names what is needed and where to register.

Venue	Where to register	Review and cost
X	`https://console.x.com`	no review; a paid usage plan, about $0.015 a post and about $0.20 with a link

Venue	Where to register	Review and cost
Instagram	`developers.facebook.com/apps/creation/`	no review, free; a professional account on a Page, and a role on his own app
LinkedIn	`www.linkedin.com/developers/apps`	two tiers of review; a registered company, a verified Page, a screencast, and LinkedIn must switch on its native flow by hand
TikTok	`developers.tiktok.com/apps`	audit for a public post, free; until the audit, only he can see what it posts
Facebook	`developers.facebook.com/apps/creation/`	no review, free; a Page he administers and a role on his own app
Threads	`developers.facebook.com/apps/creation/`	no review, free; a Threads profile and a role on his own app
Reddit	`www.reddit.com/prefs/apps`	no review, free; register an app and pick a subreddit

Meta grants standard access to every permission automatically, and it reaches any account holding a role on the app, so Instagram, Facebook and Threads need no app review. TikTok restricts every post an unaudited client makes to private viewing. LinkedIn is the one venue that may refuse him outright, and its page states both gates rather than offering a Connect that looks like it will work.

A word on a page becomes a link only where it names a host and a path. Every address the open page publishes reaches the browser, and every address it does not publish opens nothing and is recorded as refused. Nothing the operator typed and nothing a venue answered is ever a link.

`src/gui/main_tabs/market_inspector_surface.py` — which words are links

```
def link_address(word: Any) -> str:
    """``word`` as an openable address where it names a host and a path, else empty."""
```

The hit chime

A hit sounds one chime the moment the window learns of it. A confirmation sounds one more. A failed call sounds nothing. The tone is the Audio Suite's own generator, one preset joining its table, rendered once per process to a short wav with a decay applied and played through one output on the tab.

`src/gui/audio_suite.py` — the chime

```
CHIME_PRESET = "Hit Chime"
CHIME_PARTIALS = [(1.0, 0.45), (2.0, 0.18), (3.0, 0.09), (4.16, 0.05)]
CHIME_SECONDS = 0.6
CHIME_BASE_HZ = 880.0
```

The chime plays at 0.5, the volume the suite's music player opens at. The Audio Suite holds no mute control, so the chime has no mute and no slider of its own.

The operator asked for the chime and the timer together.

> *Simulator - Scan Now - ... There should be a chime when there is a hit. Once a hit is made, an isolated confirmation read timer starts based on the original hit's TF. The confirmation read mechanic should be partially documented already if not implemented...*

The confirmation read timer

Each hit starts its own timer the moment it lands in the approval bucket. The first read waits for the candle the hit was read on to close. Every read after it waits a count of closed candles of the hit's own timeframe, and that count is a setting starting at three. A daily hit reads again three days on; a weekly hit three weeks on.

`src/trading/ata_spm_push.py` — the grid each read lands on

```
DEFAULT_CONFIRMATION_CANDLES = 3
ONE_CANDLE = 1

def next_read_ts(
    at_ts: Any, timeframe: Any, now: Any, candles: int = ONE_CANDLE
) -> float:
```

The operator set that spacing.

> *Confirmation timers are too short and should be a multiple of the timeframe on which the hit occurred i.e. a weekly chart confirmation read should not be happening again in a few days but in a few weeks and a daily chart should happen in a few days rather than a few hours.*

The grid is anchored at the first close, so a read the venue delayed and a retry both land back on it. A changed count re-anchors a running timer: it keeps the grid beginning at the hit's own candle close and takes the new step.

`src/trading/ata_spm_push.py` — the seconds one candle takes

```
TIMEFRAME_SECONDS = {"5m": 300, "1h": 3600, "1d": 86400, "1w": 604800}
MONTH_TIMEFRAME = "1M"
```

A weekly candle closes at Monday 00:00 UTC, the instant the fetcher's rollup and the chart venue both stamp a week with. A monthly candle closes at the first instant of the month.

A venue that has not published the awaited candle is read again after a short wait, a bounded number of times, then at the next close.

`src/trading/ata_spm_push.py` — the retry

```
FOLLOW_UP_RETRY_S = 60.0
FOLLOW_UP_RETRY_CAP = 5
```

The tab's own clock wakes once a second and hands every due timer to one worker thread. The worker reads the market's newest candles through the scan's own route and judges the call. The outcome crosses back to the window thread, which writes the entry, the field, the Activity Log and the emitter row.

The verdict counts closed candles and reads no clock; the timer reads the clock only to choose when to read. The candles counted are those opened at or after the call's bar open whose close time is at or before the read, so the call's own candle is candle one once it closes. A venue's partial bar is never read as a close.

`src/trading/ata_spm_push.py` — the candles one read counts

```
def closed_candles(candles: Any, timeframe: Any, closed_before: Any) -> list:
    """The candles of ``candles`` that opened at or after the call and have closed."""
```

Three outcomes exist.

`src/trading/ata_spm_push.py` — the three verdicts

```
OUTCOME_CONFIRMED = "confirmed"
OUTCOME_FAILED = "failed"
OUTCOME_OPEN = "open"
```

Price moving the favourable way as far as the confirmation target confirms the call. The target is a share of the run from the call's close to the Bollinger midline, and the midline is recomputed on every candle after the call rather than frozen at it. The share is the confirmation share setting, and a share of zero sets no target, so the read says so instead of claiming a result.

`src/trading/ata_spm.py` — the moving target phase seven measures against

```
def midline_after(candles: Any, at: Any) -> list:
    """The Bollinger middle band at every candle after index ``at``.

    ``BollingerBands`` computes its published band on each window, so the
    target phase seven measures against moves with the market.
    """
```

A target exists only while the midline lies on the side the call expects the run to go: above a bullish close, below a bearish one. While the midline lies behind the call, the read stays open and says so.

`src/trading/ata_spm_push.py` — whether a target exists yet

```
def midline_ahead(direction_text: Any, call_close: Any, midline: Any) -> bool:
    """Whether the midline lies on the side the call expects the run to go."""
```

The trend carrying on for three candles in a row fails the call. A continuation is evidence against a reversal call, so the two outcomes are not the same shape. The count is in the call's own timeframe, and no wall clock is read, so a daily call cannot fail on the afternoon it was made.

`src/trading/ata_spm_push.py` — the candles a failure needs

```
CONTINUATION_CANDLE_FLOOR = 3
```

A call short of that floor is not ready. The read then says how far the trend has run against the call instead of claiming a result, and no candle is invented to reach the floor.

`src/trading/ata_spm_push.py` — the wording a call under the floor carries

```
FOLLOW_UP_NOT_READY_FORMAT = (
    "the trend has held for {run} of the {floor} candles a failure needs, "
    "last close {close:g}, target {target:g} not reached"
)
```

A settled call stops being watched and an open one stays, so no call is posted on twice. A settled call's follow-up post is one per venue and lands in the bucket as the scan's own entries do. A timer stops when its read settles the call, or when its entry leaves the bucket.

`src/trading/ata_spm_push.py` — the status a timer publishes

```
BUCKET_META_FOLLOW_UP_FORMAT = "{target} · {state} · {follow_up}"
FOLLOW_UP_STATUS_OPEN_FORMAT = "open · next read {when}"
FOLLOW_UP_STATUS_SETTLED_FORMAT = "{state} · close {close:g}"
```

`src/trading/ata_spm_push.py` — the lines a timer and a read write

```
FOLLOW_UP_TIMER_LINE_FORMAT = (
    "ATA-SPM confirmation timer for {symbol} {label}: next read {when}"
)
FOLLOW_UP_READ_OPEN_LINE_FORMAT = (
    "ATA-SPM confirmation read for {symbol} {label}: {state}, next read {when}"
)
FOLLOW_UP_READ_SETTLED_LINE_FORMAT = (
    "ATA-SPM confirmation read for {symbol} {label}: {state}, close {close:g}"
)
```

The timer tiles

The region right of the Timeframe row, the scan line and the stepper holds one tile per watched call, in both builds. A tile reads the market and its timeframe on its first line, with a small red cross at the right, and the time to the call's next read on its second, counted down once a second on the tab's own clock. At zero the read fires and the tile reads `reading` until the outcome crosses back. An open call then counts down to its next close; a confirmed or failed call reads its outcome and its close. A settled call's tile stays until its entry leaves the bucket.

`src/trading/ata_spm_push.py` — what a tile reads

```
TIMER_PAIR_FORMAT = "{symbol} {label}"
TIMER_COUNTDOWN_FORMAT = "{hours:02d}:{minutes:02d}:{seconds:02d}"
TIMER_COUNTDOWN_DAYS_FORMAT = "{days}d {hours:02d}:{minutes:02d}:{seconds:02d}"
TIMER_READING_TEXT = "reading"
```

Tiles wrap by the region's width at 132 pixels each, the width the widest pair a class lists takes at the caption size. The region is 110 pixels tall, three tile rows, and scrolls past that, so the tiles never push the stepper and the field down the zone. While no call is watched the region reads "No confirmation timer running."

The cross opens one pop-up naming the pair. No leaves the tile, its countdown and its timer standing. Yes removes that tile and its timer and nothing else: the call keeps its entry in the bucket, the bucket keeps its count, and every other tile keeps its countdown. The Activity Log records the removal.

`src/gui/main_tabs/market_inspector_surface.py` — the cross and its question

```
  TIMER_CLOSE_PART = "timer-close"
  TIMER_CLOSE_TEXT = "×"
  TIMER_DROP_TITLE = "Delete confirmation timer"
```

The operator asked for the cross.

> *Confirmation Reading timers need a small red 'x' for their deletion and this provides a confirmation pop up.*

The pop-up comes from the host in both builds, the way the hit chime does, so both builds ask the same question.

`src/trading/ata_spm_push.py` — the rows the region draws, and what a removal takes

```
  @dataclass(frozen=True)
  class TimerTile:
      """One watched call's pair, its countdown and its outcome."""

  @dataclass
  class FollowUpTimer:
      """One call's next read, its read count and the count it steps by."""

  def timer_tiles(self) -> list:
  def drop_timer(self, value: Any) -> bool:
      """Drop one open timer from ``timers`` or one call from ``settled``."""
  def watch_run(self, run: Any, candles: Any) -> None:
  def take_outcome(self, outcome: Any, candles: Any) -> None:
  def after_scan(self, run: Any) -> None:
  def candle_close_ts(at_ts: Any, timeframe: Any) -> float:
```

The Qt widget draws those rows as frames and the page draws the same rows from its own skin, with the cross as its own element. The clock's tick hands the page the rows alone, so a countdown moves without the whole payload.

The tile takes its colours from the theme's own tokens, so a theme change moves them.

```
  tile ground     SURFACE_INPUT      border        OUTLINE
  the pair        TEXT_HIGH          countdown     PRIMARY
  reading         WARNING            confirmed     SUCCESS      failed  ERROR
```

The Qt widget draws those rows as frames and the page draws the same rows as its own elements. Each cross carries that row's own value, and both hosts reach one method on the tab, which opens the pop-up and drops the timer on a Yes.

`src/gui/market_inspector.py` — the Qt tiles and the question

```
class TimerTiles:
    def _build_tile(self, row: Any) -> QFrame:
    def show_tiles(self, rows: Any) -> None:

    def _ask_drop_timer(self, value: Any) -> None:
```

`src/gui/web/market_inspector.js` — the page's own cross

```
function TimerClose({ closeValue }) { ... }
```

The called markets the Charts tab keeps watching

Reaching the approval bucket does a second thing. The market joins a list the Charts tab keeps, so the operator can watch a called market long after the post about it has gone or been declined.

The board answers one row per market, never one per post. A market called on two timeframes is still one market being watched, and the row carries the timeframes it was called on and the vote of the most recent call.

`src/trading/ata_spm_push.py` — the row the Charts tab reads

```
@dataclass(frozen=True)
class WatchedMarket:
    """One market on the ATA-SMP chart list, and the call that queued it."""

    symbol: str
    vote: str = VOTE_NEITHER
    timeframes: tuple = ()
```

Three places hold a called market and the board reads all three, oldest first: the calls phase seven has settled, the calls it is still watching, and the posts sitting in the bucket now. Filling the bucket from a new scan therefore does not drop an older market from the chart list. The settled calls are read in key order, so the list comes back in the same order every time.

A watched call is told apart by its asset, its timeframe and its bar's open time, and by nothing else, so a hit recorded while a retired class name was chosen still resolves and draws its tile.

`src/gui/market_inspector.py` — what this screen offers the Charts tab

```
def watched_markets(self) -> list:
    """The markets ATA-SMP has called, for the Charts tab's second list.

    ``PushBoard.watched_markets`` is the one set phase seven also reads.
    """
    return self._push_board.watched_markets()
```

The screen hands that reader to the Charts tab when it is built. Nothing is copied across, so the two screens cannot disagree about which markets are called. What the Charts tab does with the list is on the Charts tab page.

Multi-Exchange Arbitrage

`src/trading/arbitrage.py` holds the cross-exchange price monitor and its spread tracking. No module under `src/` imports it. The zone names the venues in reach and stops there: it draws no arbitrage panel. The first two proposal forms are on screen; the third is not.

In development.

Phantom Bot HTF Signals

The zone is named, sized and drawn, and the row that builds it writes the same sentence on every call. Nothing feeds it.

`src/gui/main_tabs/market_inspector_surface.py` — the row that never varies

```
PHANTOM_HTF_ZONE = "phantom_htf"
PHANTOM_HTF_GROUP_TITLE = "Phantom Bot HTF Signals"
PHANTOM_HTF_UNWIRED_TEXT = "Phantom Bot source not wired."

[PHANTOM_HTF_ZONE, PHANTOM_HTF_GROUP_TITLE, PHANTOM_HTF_UNWIRED_TEXT],
```

In development.

The section above is overtaken, and it is kept as written. The zone now reads the fleet. It steps through one entry for each bot that runs a phantom timeframe, in fleet order. A bot whose phantom bots are off gets no entry. The Indicator Voting Panel does the same: it adds no phantom row for that bot.

`src/gui/main_tabs/market_inspector_surface.py` — `phantom_entries`

```
return [phantom_entry(one) for one in bots or [] if phantom_timeframe(one)]
```

The tab builder hands the zone one lookup into the bot manager. On each two-second refresh the zone looks up the bots the other tabs are fed, and draws them again.

`src/gui/main_tabs/market_inspector_tab.py` — `MarketInspectorTabMixin._build_market_inspector_tab`

```
self._market_inspector.set_phantom_source(self._fleet_bot)
```

One entry names the market, the bot's own timeframe and its phantom timeframe. The line under it gives Comp and Conf for each of the two. The next line gives the gate sentence for each, in the words of the bot's own GATES line.

```
ZZZ/USD 1h with phantom 1d
1h Comp +0.94 Conf 8% · 1d Comp +2.64 Conf 23%
1h gates: Scrum blocked adx_trend_suppression, efficiency_ratio_regime, overridden TA.
Fold blocked BB, MID, TA, TRNQ, zscore_extremity. · 1d gates: Scrum blocked FIRE,
adx_trend_suppression. Fold blocked BB, MID, TA.
```

Clicking the entry opens two grids. The first is the Indicator Voting Panel, with one row for each timeframe. The second is the nineteen gate lights, the scrum bank above the fold bank, with one row for each timeframe. Each light keeps the colour the History tab gives it.

The bot's row comes from its last tick. The phantom's row comes from the phantom's last read. On each pass of its loop, the phantom runs both of the bot's gate chains over the candles it read, at the bot's own settings. That reading changes no trade.

`src/trading/phantom_balance.py` — `PhantomBot._tick`

```
self.last_gates = timeframe_gates(
    self.symbol, self.timeframe, candles, summary, self.settings
)
```

A higher timeframe reads the market and not the position. The six gates that read the position do not run at the phantom read. Their lights stay grey, and the entry names them under the grid. The bot's own row holds those six.

```
1d position: Scrum TGT, INT, OTD and Fold TRNQ, CEIL, OTD did not run. They read
the bot's position, at 1h.
```

Piece	Module	Symbol
the bots the zone reads	`src/gui/main_tabs/market_inspector_surface.py`	`phantom_fleet`
the one active timeframe	`src/trading/phantom_balance.py`	`one_active_phantom_timeframe`
the bot's own lights	`src/trading/scrumming/snapshots.py`	`state_reading`
the phantom's lights	`src/trading/ata_gate_scan.py`	`scan_reading`
the gate sentence	`src/trading/scrumming/snapshots.py`	`bank_text`
Comp for each row	`src/trading/ata_gate_scan.py`	`composite_rows`
the gate light grid	`src/gui/main_tabs/market_inspector_surface.py`	`lights_panel`

Driven on a constructed bot over a stand-in venue that refuses every order, the zone's lights equal the lights the bot's own chains decide at both timeframes. One changed gate input moves one light, which shows the comparison can report.

```
bots in the fleet                          2, one running a phantom timeframe
entries in the zone                        1; 2 with the second bot's phantoms on
lights equal to the bot's own chains       19 of 19 at 1h, 19 of 19 at 1d
one gate input changed, lights that moved  1 at 1h, 1 at 1d
Qt and React fields compared               20 equal, 0 different
orders placed, venue sockets opened        0, 0
```

The asset logo library

An asset's mark is a library on disk, divided into a folder per asset class and a folder per sector inside it. The library is filled by a step the operator starts himself. Creating a position reads that library and never fetches.

`src/trading/logo_library.py` — where one asset's mark is filed

```
  def library_folder(asset_class: Any, sector: str = "") -> str:
      """``kept_folder(f"{asset_class}/{sector}")``, every segment sanitised."""
```

An asset with no sector tag sits in the class folder itself; no sector is guessed. Every folder segment is sanitised the way a file name already is, so a sector named with a space becomes one hyphenated folder.

The library covers every asset the connected venues list plus every asset the maps hold: 542 assets, filed into 25 folders.

Folder	Marks	Folder	Marks
crypto	361	crypto/interop	5
stocks/portfolio	47	crypto/oracle	5
forex/minor	21	commodities/energy	4
crypto/l1	18	crypto/depin	3
crypto/dex	9	crypto/identity	3
crypto/defi	8	crypto/wallet-infra	3
crypto/meme	8	commodities/industrial-metals	2
commodities/precious-metals	8	crypto/payments	2
crypto/ai	7	crypto/privacy	2
crypto/l2	7	crypto/stablecoin	2
forex/major	7	crypto/storage	2
crypto/gaming	6	crypto/rwa	1
		crypto/wrapped	1

The fill takes its asset list from the order rules already recorded for each venue, not from a fresh venue call. Nothing is asked of any exchange or broker while the library fills.

`src/trading/logo_library.py` — where the asset list comes from

```
  def venue_bases(document: Optional[dict] = None) -> tuple[str, ...]:
      """Every base asset the recorded venues list, sorted, each folded by ``underlying_base``."""
```

A venue lists some contracts on a multiple of an asset. The base is folded back onto the asset only when the remainder names an asset some source already lists, so no name is shortened on a guess.

```
1000A10   ->  A10      A10 is a recorded base
1000MOG   ->  MOG       MOG is a recorded base
1000PEPE  ->  PEPE      PEPE is a recorded base
1000SHIB  ->  SHIB      SHIB is a recorded base
00        ->  00        nothing is left after the digits
1INCH     ->  1INCH     INCH is not a recorded base
2Z        ->  2Z        Z is not a recorded base
```

Nothing in the running program starts the fill. It is started by hand, and it refuses outright while the application holds the lock on the runtime directory.

```
python -m src.trading.logo_library --reads 40
python -m src.trading.logo_library --list
```

`src/trading/logo_library.py` — the refusal while the fleet trades

```
TRADING_REFUSAL = (
    "the application is running and holds the instance lock; close it first"
)
```

The gap between two reads is 0.5 seconds, the figure the market scan already uses for a host that publishes no limit. The hold after a rate refusal is 60 seconds. A run is bounded by the number of addresses it may read, and a run that stops leaves the marks it kept on disk and the unresolved assets in a file beside them, so the next run reads only what is left.

`src/trading/logo_library.py` — where the unresolved assets are written

```
UNRESOLVED_NAME = "unresolved.json"
```

An asset written to that file is skipped by every later run, including one whose source has since come back.

`src/core/asset_logos.py` — the walk, its budget and what it answers

```
LOGO_TIMEOUT_S = 10.0

class LogoBudgetSpent(RuntimeError):
    """Raised where a run has read every address it was allowed."""

class LogoCache:
    def read_budget(self, count: int) -> None:
    def kept_path(self, symbol: str) -> Optional[Path]:

@dataclass(frozen=True)
class LogoAnswer:
    """One asset's kept file, or the reason no address served an image."""

def resolve(
    symbol: Any,
    candidates: Any,
    timeout_s: float = LOGO_TIMEOUT_S,
    *,
    page_url: str = "",
    no_source_reason: str = "",
    folder: str = "",
) -> LogoAnswer:
```

`src/trading/logo_library.py` — how a ticker is settled against the coin list

```
def choose_coin(ticker: Any, records: Any) -> Optional[dict]:
    """The coin a ticker names: by recorded name, by a lone rank, then by margin."""

def underlying_base(base: Any, known: Any) -> str:
```

Where a mark is looked for

A crypto asset's mark is looked up in two steps. One read names every coin and its id; a second read names the picture each id serves. A ticker is not a coin id, so an address built out of a ticker is a guess, and the address the ticker used to build serves a web page rather than an image.

`src/exchange/crypto_assets.py` — the retired guess

```
SYMBOL_ICON_URL = "https://www.cryptocompare.com/media/img/cc_icons/{symbol}.png"
```

`src/exchange/crypto_assets.py` — the two reads behind a coin's mark

```
COIN_LIST_URL = "https://api.coingecko.com/api/v3/coins/list"
COIN_MARKETS_URL = (
    "https://api.coingecko.com/api/v3/coins/markets"
    "?vs_currency=usd&per_page={size}&page=1&ids={ids}"
)
```

A ticker several coins carry is settled by the recorded name, then by a lone market rank, then by a rank far better than the next. A ticker it cannot settle is written down with its candidates and no address.

`src/trading/logo_library.py` — how far apart two ranks must sit

```
COIN_RANK_MARGIN = 10.0
```

Every other asset is read from the organisation's own domain, at the two standard locations a site serves its own icon at. Where neither answers an image, the organisation's own page is read and the icon that page declares is asked next, and last the share image the page declares. No third party is contacted, no key is held, and nothing has to appear on the screen.

`src/trading/ata_asset_maps.py` — the two standard locations, and the domains they are read from

```
ORGANISATION_ICON_FORMATS: tuple[str, ...] = (
    "https://{domain}/apple-touch-icon.png",
    "https://{domain}/favicon.ico",
)
ORGANISATION_SITES: dict[str, str]
```

`src/core/asset_logos.py` — the last location a mark is looked for

```
SHARE_IMAGE_TAG: re.Pattern[str]
```

`SHARE_IMAGE_TAG` matches a meta tag whose property or name is the page's own share image, folding case, so a page that answers its own document at every icon address still yields one picture.

`src/core/asset_logos.py` — the icons a page declares for itself

```
def declared_icons(page_url: Any, body: Any) -> tuple[str, ...]:
    """Every icon address one page declares, and its own share image last."""
```

A fund share and a currency pair answer their organisation's own site. Every other US-listed ticker answers the regulator's own company page, which is keyed on the ticker alone and needs no lookup, so an equity the map never carried still answers. A metal quote answers nothing, because a metal has no issuer.

`src/trading/ata_asset_maps.py` — the regulator route

```
REGISTRY_URL_FORMAT = (
    "https://www.sec.gov/cgi-bin/browse-edgar?action=getcompany"
    "&ticker={ticker}&type=10-K&dateb=&owner=include&count=10"
)
```

The regulator's own company page carries a web-address field and that field is empty, so the register names the company and not its site.

No logo read reaches a trading venue's own domain.

`src/trading/ata_asset_maps.py` — the domains no read reaches

```
VENUE_DOMAINS: frozenset[str] = frozenset({"coinbase.com", "sofi.com"})
```

A kept body must be an image, read from the body's own leading bytes rather than its length. A length floor alone kept a web page as a picture once; the signature does not.

`src/core/asset_logos.py` — the signatures a kept body must carry

```
IMAGE_SIGNATURES = (
    (b"\x89PNG\r\n\x1a\n", "png"),
    (b"\xff\xd8\xff", "jpg"),
    (b"GIF87a", "gif"),
    (b"GIF89a", "gif"),
    (b"\x00\x00\x01\x00", "ico"),
    (b"BM", "bmp"),
)
```

The file name is built from the symbol, with every character a file name may not carry replaced by one gap character, so a currency pair is kept under a hyphenated name.

`src/core/asset_logos.py` — the file name

```
def kept_name(symbol: str) -> str:
    """``symbol`` as the file name stem a kept logo takes, every other character ``KEPT_NAME_GAP``."""
    text = str(symbol).strip().upper()
    return "".join(
        one if one.isascii() and one.isalnum() else KEPT_NAME_GAP for one in text
    )
```

Reading a mark back by its symbol alone finds it at whatever depth it sits, so the bot list and the wizard need no knowledge of the folders.

What the library holds

Sector	Targets	Marks kept	Resolving nothing
crypto	453	362	91
stocks	47	38	9
currencies	28	24	4
commodities	14	10	4
total	542	434	108

The 434 marks take 7,455,763 bytes and carry 395 distinct pictures: 311 PNG, 82 JPEG, 37 ICO and 4 WebP, the smallest 660 bytes and the largest 175,010. Every picture two assets share belongs to one issuer, so seven fund shares share one asset manager's mark and each currency group shares its central bank's.

Seven reasons cover every asset that resolves nothing.

Reason	Assets
several coins carry the ticker at comparable market rank	67
no coin record carries the ticker, or its name	22
every address answered something that is not an image	7
the company no longer trades	4

Reason	Assets
a metal quote with no issuer	4
several coins carry the ticker and none carries a market rank	2
the domain names a trading venue	2

The 22 with no coin record are venue index and derivative products. Four organisations answer this platform's own user agent with a refusal, and one serves a certificate this machine cannot verify. Nothing was relaxed to reach any of them.

Two sentences this page used to carry are overtaken, and both are quoted whole.

> *"A stock, a fund share, a metal and a currency pair do not resolve the way a coin does."*
>
> *"the 47 stocks rows have no logo address a company's own domain is not a fact this repository holds for a ticker, and no keyless service keyed on a ticker was accepted"*

A stock and a fund share now resolve from the organisation's own domain, and 43 of the 47 stock rows carry a logo address. A metal quote still resolves nothing.

Opening an asset's own organisation

Every address offered as openable is checked for its scheme first, and a refused one is an empty address carrying the reason it is empty. Some schemes open a local file or run a script instead of visiting a site, and an address for an asset arrives from outside this repository. The check reads the same allowed set the guarded fetch reads, so the two cannot drift apart.

`src/core/safe_url.py` — the check every offered address passes

```
def openable_url(
    url: object,
    allowed_schemes: Optional[Iterable[str]] = None,
) -> tuple[str, str]:
```

The file, javascript, vbscript and data schemes open nothing. An empty address answers that no address is known.

Every caller now reads that reason instead of dropping it. A mark whose address was refused says so on its own tooltip and writes the same line to the log, and the coin index records the refusal against the ticker it belongs to. A refused address can therefore be told from an asset that never had one.

Where a mark is drawn

Six places draw an asset icon and all six read the library. A row draws the kept file or the lettered disc and never waits on a network read.

`src/gui/main_tabs/bot_status_table_surface.py` — the size a row draws

```
LOGO_SIZE_PX = 32
ROW_HEIGHT_PX = LOGO_SIZE_PX + 2 * ROW_LOGO_MARGIN_PX
```

Six of the 434 marks offer only 16 pixels, because their own source serves a 16-pixel icon, and those six draw at their own size in both builds. A row with no mark draws its symbol exactly as it does today. None of the kept

images enters the repository: the library directory is ignored.

`src/gui/bot_wizard.py` — the one reader every drawing place uses

```
KEPT_LOGOS = LogoCache()
```

`resolve` is never called from a wizard, and nothing there fetches. The six places that draw an asset icon are the three bot wizards and the three Extractor bot tables, one pair each under the live, the paper and the simulator directories.

```
src/gui/bot_wizard.py                       src/gui/widgets/extractor_bot_table.py
src/gui/paper/paper_bot_wizard.py           src/gui/paper/paper_extractor_bot_table.py
src/gui/simulator/sim_bot_wizard.py         src/gui/simulator/sim_extractor_bot_table.py
```

What the emitters write

Every row below goes through one wire and nothing else. A topic declared in `src/core/emit_contracts.py` is a tracked emitter, and a run that never fires it is reported.

The operator set what the network is for.

> *Entire idea for the Emitter Network is to provide us signals for verifying proper software function resulting from all user actions or automated sequences.*
>
> *Entire idea for the Emitter Network is to provide us signals for verifying proper software function resulting from all user actions or automated sequences...*
>
> *For this, will need to verify and / or wire in Emitter Network and update what the Inspector's subsystems (including ATA-SMP) write to the logs. Entire idea for the Emitter Network is to provide us signals for verifying proper software function resulting from all user actions or automated sequences...*

Topic	Written	Carries
`market_inspector.scan_started`	one per Refresh	forced, connector count, active symbols
`market_inspector.scan_finished`	one per Refresh	market count, duration, signals, pairs, source, error
`inspector.ata.scan_pressed`	one per press	whether a thread started, against True
`inspector.ata.scan_started`	one per press	the thread's name, against the scan thread's name
`inspector.ata.market_read`	one per market read	the candle count, against the vote floor
`inspector.ata.scan_finished`	one per press	markets read, against markets listed
`inspector.ata.volume_order`	one per class read	markets with a figure, against markets listed
`inspector.ata.hit`	one per hit	the running count, against the target
`inspector.ata.ticker_resolved`	one per press with text typed	the class the text landed in, against the class chosen
`inspector.scan.progress`	every ten markets and at the end	class, read, total, hits
`inspector.scan.list_source`	one per list read	class, source, count, the names not trading
`inspector.ata.candidate`	one per bucket entry	whether the image is on disk

Topic	Written	Carries
`inspector.ata.image_size`	one per venue image	the written size, against the row's size
`inspector.ata.handoff`	one per post a press took	venue, route, outcome, destination
`inspector.ata.sent`	one per request	venue, symbol, timeframe, status, message id, elapsed
`inspector.ata.chime`	one per chime	its cause, the hits it covers, whether the player took it
`inspector.ata.follow_up_read`	one per read	candles counted, against candles the clock says have closed

Each topic is named by one constant, so a topic and its writer cannot drift apart.

```
SCAN_PRESSED_PIN, SCAN_STARTED_PIN, MARKET_READ_PIN, SCAN_FINISHED_PIN
CANDIDATE_PIN, HANDOFF_PIN, SENT_PIN, IMAGE_SIZE_PIN
CHIME_PIN, FOLLOW_UP_READ_PIN
```

Bridge

Three methods serve this screen, and the renderer modules carry the matching names.

Bridge method	Serves
`market_inspector.state`	The scanner and its two tables
`market_inspector_tab.state`	The per-bot page in the Bot Details dialog
`market_inspector_topologies.state`	The proposal cards

Each bridge method reads one object, and the two hosts build one each.

Object	Where it is built	What it is
`MarketInspectorTab`	`src/gui/market_inspector.py`	the Qt widget tree, its steppers and its clock
`MarketInspectorScreenModel`	`src/gui/main_tabs/market_inspector_surface.py`	the same screen as values, which the React host draws
`AtaSpmSettings`	both of the above	the six settings, the vault and the sign-in route
`SectorScan`	`src/trading/ata_spm.py`	one sector's or one market's walk and its result
`SignInSession`	`src/trading/ata_spm_signin.py`	one sign-in's listener, its state value and its transport
`ScanNowButton`, `PostChart`, `TimerTiles`, `TimerClose`, `ZONE_COUNTER`	`src/gui/web/market_inspector.js`	the page's own elements, drawn from `ata_spm_skin`
`PaneWidthPage`	`src/gui/market_inspector.py`	the Qt page that relays its own width to its rows

The window relays that width on every splitter move, which is what keeps the button row and the settings rows inside their zone.

`src/gui/market_inspector.py` — the redraws a width change and a progress run

```
    def _size_zones(self) -> None:
    def _relay_bucket_row(self) -> None:
    def _render_ata_row(self) -> None:
    def _show_lines(self, rows: Any) -> None:
    def _show_panels(self, panels: Any) -> None:
    def _show_panel_lines(self, rows: Any) -> None:
    def _show_actions(self, actions: Any) -> None:
    def _drop_panel_box(self) -> None:
```

`src/gui/main_tabs/market_inspector_tab.py` — where the Activity Log pane is handed over

```
    def _wire_ata_activity_log(self) -> None:
```

Each of those hides a widget before it reparents it, so no parentless widget is ever shown and no window floats over the panel. The proposals pane builds its own detector context, and a scan writes one sector's votes at a time.

```
MainWindow._build_topology_proposals    the detector context
SectorBoard.load_run                    the bucket one finished run replaces
ata_spm._vote_one                       one market's vote on one timeframe
market_inspector_topologies.pane_view    the pane as values
registry._rollup                        the one calendar-bucket rollup
```

Which model answers, by host

Two hosts draw this screen, and each reads a model of its own.

host	the model it reads	wired by
the Qt window	the tab's own model	`set_exchange_source`
the Electron shell	`market_inspector_surface.pane_model`, the one the bridge keeps between calls	nothing

`src/gui/main_tabs/market_inspector_tab.py`, in `MarketInspectorTabMixin._build_market_inspector_tab`, is the only caller of `set_exchange_source`. `MarketInspectorReactTab` subclasses `MarketInspectorTab` without overriding that method, so the one call wires both Qt variants.

The React variant reaches no bridge method. `src/gui/react_market_inspector_tab.py`, in `HOST_SCRIPT`, replaces the page's own `acervatorMarketInspectorAction` with a console line carrying `SCREEN_PREFIX`, `MarketInspectorPage.javaScriptConsoleMessage` reads that line, and `run_action` calls the inherited `_start_fetch` on the tab's own getter and scheduler.

Diagram: flowchart, left to right

- Refresh on the page -> **HOST_SCRIPT console line**

- HOST_SCRIPT console line -> **javaScriptConsoleMessage**
- javaScriptConsoleMessage -> **run_action**
- run_action -> **_start_fetch, the wired getter**
- _start_fetch, the wired getter -> **_fill_signal_rows, _fill_pair_rows**
- _fill_signal_rows, _fill_pair_rows -> **push, screen_push_script**
- push, screen_push_script -> **setTab on the page**

The scan's rows come back as a push and never as a response. `_fill_signal_rows` and `_fill_pair_rows` write the tab's own model, and `push` sends `build_view_model` of that model through `screen_push_script`.

`src/gui/react_market_inspector_tab.py` — the three pushes the page is drawn by

```
def screen_push_script(model: dict) -> str:
def tile_push_script(rows: list) -> str:
def topology_push_script(model: dict, preview_at: Optional[int]) -> str:
```

Topology Proposals is a payload of its own. `TopologiesPaneHost.view_model` builds it from the pane's own `TopologiesPaneModel` and `topology_push_script` sends it, so that pane draws from neither model in the table above.

`market_inspector.state` is reached by `loadScreen` and by the page's own `acervatorMarketInspectorAction`, both of which need a host supplying `global.acervator`. The Electron shell supplies one and the Qt web view does not.

That function routes `scan-now`, `scan-all`, the zone steps, the ticker field, the class and timeframe boxes, the push parts, the credential fields and the settings. It carries no branch for `REFRESH_PART`, so a Refresh press on a host reading `global.acervator` answers null and asks the bridge for nothing.

What a Refresh publishes about the connectors

A Refresh hands the live `{exchange_id: connector}` dictionary to the scheduler, and `src/gui/main_tabs/market_inspector_surface.py`, in `MarketInspectorScreenModel.fetch_call`, holds that same dictionary on `scheduled`. The published payload carries values in its place.

`src/gui/main_tabs/market_inspector_surface.py` — what `scheduled` publishes

```
def connector_states(connectors: Any) -> list:
def scheduled_fetch(found: Any) -> list:
```

Each connector publishes one `[name, connected]` pair, sorted by name. The name is the connector's own `exchange_id`, or the key the main window filed it under where the connector answers no id, and `connected` is its `is_connected`. One scheduled fetch publishes as that pair list followed by its force flag, so a wired source draws as `[["binance", true]], false]`.

the screen reads	from
the venue name	`exchange_id`, declared abstract on `ExchangeInterface`

the screen reads	from
whether it is signed in	`is_connected`, True after `connect` and False after `disconnect`

No live object reaches the payload. `src/gui/web/market_inspector.js`, in `checkShapes`, needs `scheduled` to be a list and reads no entry inside it.

A payload the bridge cannot write

`src/core/desktop_bridge.py`, in `encode_frame`, writes one response as one line. A value JSON has no spelling for is reported in place rather than taken out on the whole frame.

`src/core/desktop_bridge.py` — the three the encoder handles

```
  def json_writable(payload: Any) -> Any:
  def unwritable_text(response: Any, exc: BaseException) -> str:
  def encode_frame(response: dict) -> bytes:
```

the frame holds	what is written
`nan`, `inf` or `-inf`	`null` in that field
a value whose type has no encoder	`<unwritable TypeName>` in that field
a circular reference	an error frame carrying the request's own id

`encode_frame` catches the `TypeError`, the `ValueError` and the `RecursionError` that `json.dumps` raises, logs the type and the message under `acervator.core.desktop_bridge`, and returns a line for each of the three. The frontend's parser has a frame to read for every one of them, and the screen answers in place of going silent.

The sentence the declared derivation cost replaces

The sentence below stands in an earlier section and no longer describes the code. The earlier text stays where it is.

Earlier sentence	What the code does now
"`src/core/encryption.py` — the derivation cost", and the `_KDF_ITERATIONS` line under it	The cost is declared as `KDF_ITERATIONS`, a public name carrying the OWASP figure in its own docstring. The figure is unchanged at 600,000 rounds, and a search for `_KDF_ITERATIONS` finds nothing.

The narrative for the whole screen is on the tabs page. The tab index is on the subsystem README.

Bot Swarm Tab

Reference. The capital reinforcement network: the nodes, the wires between them, and the accounting that lands routed profit.

What builds it

`BotSwarmTabMixin._build_bot_swarm_tab` in `src/gui/main_tabs/bot_swarm_tab.py` constructs the visualizer and adds it to the tab row. The tab holds three sub-tabs.

Sub-tab	State
Bot Swarm	The live fleet
Simulator Swarm	Rows the Simulator registers while a run plays
Paper Swarm	Chrome only — see paper-trader.md

The builder asks `variant_surface` for the tab class under the screen name `BOT_SWARM`. A build stamped for Qt gets the old widget. Every other build gets the React tab, which holds the whole screen in one web view. React is the running choice.

`src/gui/main_tabs/bot_swarm_tab.py` — `BotSwarmTabMixin._build_bot_swarm_tab`

```
from ..variant_surface import BOT_SWARM, surface_class

built = surface_class(BOT_SWARM)()
```

The nodes

Each bot draws as one locust card, painted with QPainter. A theme module supplies the palette and a particle module the drifting motes.

`src/gui/visualizer/bot_node.py` — `BotNodeWidget`

```
class BotNodeWidget(QWidget):
    """Paint one bot as a locust, fed by set_bot_data and animate.

    paintEvent draws the wings, abdomen, thorax and head, then the
    symbol, the P/L and the bot_id as text.
    """
```

In the React build the card is drawn by `bot_node.js` from the payload `bot_node_surface` publishes. The tab page carries that module beside the wire canvas, the quick routing matrix and the theme palettes.

`src/gui/react_bot_swarm_tab.py` — `TAB_SCRIPT_ASSETS`

```
TAB_SCRIPT_ASSETS: tuple[str, ...] = (
    "vendor/react.production.min.js",
    "vendor/react-dom.production.min.js",
    "design_tokens.js",
    "shared_widgets.js",
    "header_strip.js",
    "visualizer_themes.js",
    "bot_node.js",
    "wire_canvas.js",
    "quick_routing.js",
    "bot_visualizer.js",
)
```

The grid of locust cards is the whole screen. The wire overlay covers it, because the grid is the only surface whose coordinates a wire can be drawn against.

The live rows

A strip above the locust grid holds one dense row per live bot: the id, the pair, the mode, the timeframe, the capital, the state, the price, the profit and the trade count. The strip is hidden while the fleet is empty. The same fleet load that draws the locusts fills it.

`src/gui/bot_visualizer.py` — `BotVisualizationTab._sync_live_rows`

```
def _sync_live_rows(self, bot_statuses: list[dict]) -> None:
    """Hold one ``_live_bot_rows`` row per bot in one fleet load.

    A row is rebuilt when the identifier mask changes the id it
    shows, and a bot no longer in the load loses its row.
    """
```

The React build fills the same rows through `BotVisualizerModel.sync_live_rows` and draws them in `LiveRows`.

`src/gui/main_tabs/bot_visualizer_surface.py` — `BotVisualizerModel.sync_live_rows`

```
def sync_live_rows(self, bot_statuses: list) -> None:
    """Hold one ``self.live`` row per bot in one fleet load.

    A row is rebuilt when ``masked`` changes the id it shows, and a
    bot no longer in the load loses its row.
    """
```

A stored reading that is text, a bool, not finite or too large to hold reads zero in a money or count column, so no stored value stops a row drawing. The identifier privacy dot masks the id column in both builds.

Drawing a wire

Drag from one node to another. Wires that arrive on the bus draw through the same handler, which is the path the startup restore and the topology adopt both take, so one subscription covers every source.

`src/gui/bot_visualizer.py` — `_on_external_wire_created`

```
def _on_external_wire_created(self, event) -> None:
    """Add the ``wire.created`` event's wire to ``_wires`` when new.

    A repeated pair updates its pct and adds no second entry.
    """
```

The quick routing matrix is the bulk alternative: tick source bots, tick destination bots, and every pair lands at once. Before that runs, the handler refuses four things and says which.

`src/gui/visualizer/quick_routing.py` — `QuickRoutingMatrix._on_connect_clicked`

```
    if not (0.0 <= pct <= 100.0):
        self._reject(f"Rate {pct} is outside 0–100%.")
        return
    if not sources:
        self._reject("No SOURCE bots are checked.")
        return
    if not dests:
        self._reject("No DESTINATION bots are checked.")
        return
    if pct <= 0:
        self._reject(
            "Rate is 0% — that would create wires that route " "nothing."
        )
        return
```

`_confirm_mass` puts a count and a description in front of the operator before a bulk connect or disconnect runs.

The flow along a wire

One `QTimer` on the Bot Swarm tab drives the whole animation. `BotVisualizationTab._animate` in `src/gui/bot_visualizer.py` runs on every tick: it moves each locust card on, adds the elapsed time to every wire's phase, and asks the wire overlay to repaint. `_WireCanvas.paintEvent` in `src/gui/visualizer/wire_canvas.py` draws the frame, and the travelling dot sits at `pulse_percent(phase)` along the wire.

The timer's interval is `FRAME_INTERVAL_MS` in `src/gui/main_tabs/bot_visualizer_surface.py`, worked out rather than written down: the largest whole number of Windows scheduler ticks that fits inside one frame at `TARGET_FRAMES_PER_S`. A wait is rounded up to the next `SCHEDULER_TICK_MS`, so asking for the 33 ms that `1000 / 30` reads as delivers 46.875 ms — 21 frames a second, not 30. Two ticks is 31.25 ms, so the timer asks for 31 ms and the flow runs at 30 frames a second.

Every frame reports itself. `_WireCanvas.paintEvent` hands its start time and its own duration to `FrameCadence`, and once a `CADENCE_WINDOW_S` window is folded `BotVisualizationTab._report_frame_cadence` emits `swarm.11.003.invariant.frame_cadence`. The record carries the window's frame count, the mean, median, 95th and worst gap between paints, the median and worst paint duration, and how many gaps ran past `FRAME_BUDGET_MS`. A stutter is one long gap among short ones, so `FrameCadence.reading` reports the median and the 95th apart — a mean alone hides it.

The React Bot Swarm page draws no flow. Its `wire-canvas` element is `display: none`, `WireOverlay` in `src/gui/web/bot_visualizer.js` redraws only when the wire list, the opacity or the theme changes, and no caller on the page sends the `animate` action that `apply_action` in `src/gui/main_tabs/bot_visualizer_surface.py` provides. Read on the running page with 38 bots and 33 wires loaded, it calls `requestAnimationFrame` zero times and `setInterval` zero times.

Routing the profit

`SmartWireManager` in `src/trading/smart_wire.py` owns the topology and one ledger per bot.

Method	What it does
`register_wire`	Adds an edge carrying its own percentage
`unregister_wire`	Drops one edge
`distribute_fold_profit`	Moves that percentage of realised profit to each wired target
`book_transfer`	Books one transfer onto both ledgers — the two wire totals and the target's provenance line for the funder
`export_wires`, `import_wires`	Carry the topology through a save and a restart
`export_ledgers`, `import_ledgers`	Carry the provenance through the same

The percent a wire is allowed to carry

One rule decides whether a stored percent is a number, and every reader of the topology reads it through that rule, so a percent the Bot Swarm tab draws as an em dash can never route a share.

`src/trading/smart_wire.py` — `routable_pct` and `pct_kind_refusal`

```
  def routable_pct(value) -> Optional[float]:
      """One stored wire percent as a float, or None when no reading admits it."""
```

A bool is refused by name. `float(True)` answers 1.0, so a saved `true` would otherwise stand as a one percent wire nobody configured. `register_wire` and `import_wires` refuse that kind before they store anything, `distribute_fold_profit` routes no share from a percent it cannot admit, and `export_wires` drops such a row from the save and logs which route is gone. `PaperWireManager` and `SimWireManager` read the same two functions, so the three engines admit and refuse the same values.

Three readings on the per-bot ledger answer where a bot's capital came from and how much of it has aged enough to move on.

Reading on `BotLedger`	Answers
`predominant_source`	Which bot sent the most of what this one holds
`mature_profit_total`	Profit above the bot's maturity threshold
`mature_profit_available`	The mature figure less what a saved ledger recorded as allocated

Maturity is a growth threshold, not an age and not a share. A bot's capital is mature once it is worth more than three times what it started with, and the mature figure is then the whole profit above that starting capital. Below the threshold the figure is zero.

`src/trading/smart_wire.py` — `mature_profit_usd`

```
  MATURE_GROWTH_PCT: float = 200.0

  if value < basis * (1.0 + MATURE_GROWTH_PCT / 100.0):
      return 0.0
  return value - basis
```

One function decides maturity everywhere. The per-bot ledger reads it against the seed capital, and the header strip reads it against the exchange cost basis of each bot's holdings, so the two readings cannot disagree about what mature means.

`src/trading/smart_wire.py` — `BotLedger.mature_profit_total`

```
return mature_profit_usd(
    self.starting_balance, self.starting_balance + self.total_profit
)
```

What the provenance breakdown counts

The breakdown is one line per funder, biggest amount first. A line is the running total that funder has supplied to this bot: the seed the bot was created with, plus every wire transfer that funder has sent it since. The seed sits under the key `SEED` when no bot funded the creation, and under the funding bot's own id when one did.

One method writes the map. Every live transfer books through `src/trading/smart_wire.py`, in `book_transfer`, which moves three figures together — the source ledger's outgoing total, the target ledger's incoming total, and the target's breakdown line for the source. Both live transfer paths call it: `src/trading/smart_wire.py`, in `distribute_fold_profit` on a compounded fold, and `src/trading/scrumming/wire_routing.py`, in `_route_scrum_proceeds_via_wires` on a scrum, a manual scrum and a detonation. A transfer cannot reach the totals without reaching the breakdown.

Now: a scrum or a manual scrum books each wire's share of its surplus alone, and a detonation books no transfer (`src/trading/scrumming/wire_routing.py, in _route_scrum_proceeds_via_wires`).

`src/trading/smart_wire.py` — `book_transfer`

```
def book_transfer(self, source_id: str, target_id: str, usd: float) -> bool:
    """Book one wire transfer onto the two ledgers it moves money between."""
```

An amount no reading admits books nothing and answers `False`, so the totals and the breakdown a ledger already holds stay readable. A breakdown line already unreadable keeps the figure it has, because adding to it would turn a line the tab draws as an em dash into one it draws as money.

The predominant funder row reads the same map through `src/trading/smart_wire.py`, in `BotLedger.predominant_source`, which answers the largest line other than `SEED`. A bot no wire has funded has no such line, so the row reads `— (SEED-funded only)`.

Paper and the Simulator book the same three figures in their own managers, `src/paper/paper_wire.py`, in `PaperWireManager._book`, and `src/simulator/sim_wire.py`, in `SimWireManager._book`. The same wiring reads the same breakdown on all three screens.

What the group draws now

The Provenance & Mature-Profit Spawn State group draws three rows: the starting balance, the predominant funder and the provenance breakdown. It drew three more — the mature profit total, the mature profit allocated to spawns and the mature profit available — and those three came off every screen that shows the group.

The heading is written once, and the three screens in the table below read it from there, so changing the heading is one edit. A single ampersand in a box title is read as a keyboard shortcut mark, so the doubled form

is what the three boxes are given and one ampersand is what each screen draws. The Electron panel reads the same words out of the shared model.

`src/gui/main_tabs/bot_swarm_tab_surface.py` — the heading and the three rows

```
PROVENANCE_GROUP_TITLE = "Provenance & Mature-Profit Spawn State"
PROVENANCE_GROUP_TITLE_QT = PROVENANCE_GROUP_TITLE.replace("&", "&&")

STARTING_ROW_LABEL = "Starting balance (seed):"
PREDOMINANT_ROW_LABEL = "Predominant funder (PPS):"
PROVENANCE_ROW_LABEL = "Provenance breakdown:"
```

Each of the three could only ever read `$0.0000`. Nothing in the program works out the allocated figure; three places write it and each only copies it back out of a saved row, so a zero saved comes back as a zero. The total and the available figure are worked out from a ledger's `total_profit`, and the only method that adds to `total_profit` is called from one place that nothing calls. A money figure that can only read zero tells the operator nothing, because he cannot tell a true zero from a reading that was never fed.

The sum itself is sound and stays where it is. `mature_profit_usd`, `BotLedger.mature_profit_total` and `BotLedger.mature_profit_available` are unchanged, and the header strip still reads maturity against the exchange cost basis of each bot's holdings. Only the three rows on the tab came off.

Screen	Where
Live	`src/gui/live_settings/bot_swarm_tab.py`
Paper	`src/gui/paper/paper_bot_swarm_tab.py`
Simulator	`src/gui/simulator/sim_bot_swarm_tab.py`
The shared model the panel reads	`src/gui/main_tabs/bot_swarm_tab_surface.py`

The sentences this overtakes, from the start of this section and from the block above:

> *The Provenance & Mature-Profit Spawn State group draws three rows: the starting balance, the predominant funder and the provenance breakdown.*
>
> *A single ampersand in a box title is read as a keyboard shortcut mark, so the doubled form is what the three boxes are given and one ampersand is what each screen draws.*
>
> `PROVENANCE_GROUP_TITLE = "Provenance & Mature-Profit Spawn State"`

The true sentences. The heading is `Provenance`. It names the three rows the group holds and names nothing else, because the clause that named mature-profit spawn state named three readings that came off the box.

`src/gui/main_tabs/bot_swarm_tab_surface.py` — the heading and the three rows

```
PROVENANCE_GROUP_TITLE = "Provenance"
PROVENANCE_GROUP_TITLE_QT = PROVENANCE_GROUP_TITLE.replace("&", "&&")

STARTING_ROW_LABEL = "Starting balance (seed):"
PREDOMINANT_ROW_LABEL = "Predominant funder (PPS):"
PROVENANCE_ROW_LABEL = "Provenance breakdown:"
```

The heading carries no ampersand, so the doubled form and the plain form are the same word and every screen that shows the group draws `Provenance`. The three Qt screens read `PROVENANCE_GROUP_TITLE_QT` and the two panel scripts read the title out of the payload, so none of them spells the words and the three builds cannot disagree about them.

The three rows are unchanged. Each keeps the label above, the breakdown row keeps `PROVENANCE_STYLE` and its word wrap, the funder row draws `– (SEED-funded only)` for a bot that only ever held its seed, and an em dash for a ledger that refuses to name a funder.

The summary box at the top of the tab is headed `Swarm Connections & Capital Flow`, and its words are declared in the same place as the provenance heading.

`src/gui/main_tabs/bot_swarm_tab_surface.py` — the two headings and the forms Qt is given

```
SUMMARY_GROUP_TITLE = "Swarm Connections & Capital Flow"
PROVENANCE_GROUP_TITLE = "Provenance"
# A QGroupBox title reads a single ampersand as a mnemonic mark, so Qt takes two.
SUMMARY_GROUP_TITLE_QT = SUMMARY_GROUP_TITLE.replace("&", "&&")
PROVENANCE_GROUP_TITLE_QT = PROVENANCE_GROUP_TITLE.replace("&", "&&")
```

The three Qt screens read `SUMMARY_GROUP_TITLE_QT`, and each panel script reads its title out of the payload, so no drawing surface spells either heading. The derived form is `Swarm Connections && Capital Flow`, so each box draws the same words it has always drawn.

The sentence this overtakes. The page carried no sentence naming where the summary heading is spelled. Changing either heading is one edit in one file, and the doubled form stays correct for whatever wording the declaration holds.

One function bounds what a bot may send away. It reads the target balance, the band edges, the ammunition the next fold needs and the cash on hand, and returns the exportable share as a percentage. A bot never exports the capital it is holding to buy the dip it is waiting for.

`src/trading/smart_wire.py` — `compute_safe_outflow_pct`

```
def compute_safe_outflow_pct(
    scrum_profit_usd: float,
    target_balance_usd: float,
    current_price: float,
    band_lower: float,
    band_upper: float,
    next_fold_ammo_usd: float,
    current_cash_usd: float,
    compound_growth_pct: float,
    retained_this_cycle_usd: float,
) -> float:
```

Landing the profit

The USD arrives at the destination and spreads evenly across the open fold tranches. Where no tranche stands, it parks instead, and the parked pool drains into the next tranche that opens. Parked credits are routed income, never a tranche.

`src/trading/scrumming/wire_routing.py` — `apply_wire_income`

```
  def apply_wire_income(self, usd: float, source: str, ref: str = "") -> dict:
      """Apply incoming Smart Wire USD to this bot's fold queue.
```

Each tranche keeps its wire-credit provenance, held by three methods.

Method	What it does
`_add_wire_credits`	Appends one detail entry per arrival
`_compact_wire_credits`	Bounds how many entries a tranche holds
`_roll_wire_credit_overflow`	Folds everything past the cap into a rolled total that keeps the sums exact

The tranche book

`FoldTrancheAccountingMixin` in `src/trading/scrumming/fold_tranches.py` owns the queued fold tranches, the discharge order the buy path reads, and the lifetime counters. It places no orders.

Four methods carry the fold through, in this order.

Method	Step
`_fold_eligible_tranches`	Selects what may fold at the current price
`_fold_discharge_order`	Fixes the order they discharge in
`_plan_fold_consumption`	Builds the plan
`_settle_fold_plan`	Books it

Merge, despawn, clear

Three functions collapse or remove a tranche. No other code path does.

Merge. `_top_up_remnant_fold_tranches` folds a new sell's USD into part-spent tranches instead of opening more records, and `_bound_new_fold_tranches` merges one sell's fresh slice back to a single record when the fill arrived as many lots. Both reduce the record count and move USD; neither loses any.

A fourth function combines a standing pair within a tenth of a percent

The sentence at the head of this subsection is overtaken. It is not edited and not removed. It read:

> *Three functions collapse or remove a tranche. No other code path does.*

A fourth now does. `src/trading/scrumming/fold_tranches.py`, in `_combine_fold_tranches_within_distance`, combines two standing fold tranches whose reference prices sit inside a tenth of a percent of each other. It runs each time a sale queues a tranche, after the top-up.

`src/trading/scrumming/sizing.py`, in `merge_fold_tranches_within_distance`, holds the arithmetic, and `src/trading/stack_math.py`, in `within_merge_distance`, holds the figure. The Stack ladder read that same declaration already.

The survivor keeps the lower reference price, which is the stricter fold threshold, sums the cash and the units, takes the units-weighted original cost and names both events. It moves USD and loses none, as the two merges above it do, and it brings the opened counter down by what it combined so `created - closed - discarded == standing` still holds.

Driven on a hand-built pair in one run: refs 0.05 percent apart combined, two records into one carrying $100.0000 and 1.000000 units at a cost basis of 85.00000000; refs 0.5 percent apart stayed as two.

A standing Stack rung also combines with a new rung inside the same distance, through `src/trading/stack_math.py`, in `merge_into_standing_rungs`. A rung holding a venue order is never a candidate, since combining it would need that order cancelled.

Despawn. Aged tranches are removed outright. The record leaves the book, the queue total recomputes from what is left, and the count lands in the discarded ledger rather than the closed one, because a closed tranche is one that folded. That split holds one invariant true.

`src/trading/scrumming/fold_tranches.py` — `_despawn_aged_tranches`

```
def _despawn_aged_tranches(self, now: Optional[float] = None) -> dict:
    """Remove tranches older than ``config.tranche_despawn_days``.

    Removing a fold tranche recomputes ``_fold_queue_usd``, the
    derived aggregate behind the tick's ``_fold_queue_usd == 0``
    short-circuit and the panel's "Parked USD", and counts the
    record in ``_tranches_discarded_lifetime``, never in
    ``_tranches_closed_lifetime``, because a closed tranche is one
    that FOLDED. That split keeps
    ``created - closed - discarded == standing`` true.
```

An aged stack tranche still holding a resting limit order is kept and counted separately, since removing it would leave that order on the book with nothing tracking it. The threshold reads the same shared helper the settings spinbox reads, so the sweep and the control cannot disagree about what a stored value means.

The sweep's report now names its counts the way the preview beside it already did, so the two agree on the word as well as on the number. The key set below replaces the one in the block above, and the line the operator reads says removed where it used to say delisted. Nothing outside the sweep read the old names.

```
report = {
    "threshold_days": _days,
    "fold_removed": 0,
    "stack_removed": 0,
    "stack_kept_live_order": 0,
    "ageless_kept": 0,
    "usd_removed": 0.0,
}
```

Two removers touch the stack ledger, not one. The age-driven sweep is one and the operator's own clear is the other, and both move the same discarded counter.

```
  def clear_stack_tranches(self, reason: str = "operator") -> dict:
      """Discard this bot's standing Stack tranches, trading nothing."""
```

Clear. `clear_fold_tranches` discards every queued tranche and trades nothing. Holdings, the main lots, the target balance and the parked wire credits all survive it, and the report warns when parked credits remain.

One admission rule for every lifetime counter

The invariant above holds only while every term of it is read the same way. One rule reads them all: `as_whole_count`, in `src/trading/container/config.py`. It admits a whole number and nothing else. A stored flag, a text number, a not-a-number, an endless number, a fraction and a whole number too large for a float each answer "no usable count", and none of them raises.

Seven counters read it: the four fold counters, the scrum-sells counter and the two stack counters.

Reader	What a refused counter does there
`src/trading/scrumming/state_io.py`, in `admitted_counter`	Lands zero and names the bot, the field and the stored type in the log
`src/gui/live_settings/fold_tokens.py`, in `fold_lifetime_counter`	Returns no count and logs the same three facts
`src/gui/live_settings/stack_tranches_tab.py`, in `_stack_lifetime_counter`	The same, for the two stack counters
`src/gui/main_tabs/fold_tranches_tab_surface.py`, in `guarded_count`	The same reading, for the second build
`src/gui/main_tabs/stack_tranches_tab_surface.py`, in `lifetime_counter`	The same reading, for the second build

The Paper and Simulator panels are forks of the Live ones and read the same rule. Both fold forks call `fold_lifetime_counter` in `src/gui/live_settings/fold_tokens.py`, the one module all three fold panels share. Each stack fork carries `_stack_lifetime_counter`, matching the Live panel's own, so no screen reaches across from the Simulator into Live to read a count.

Panel	Reader
`src/gui/paper/paper_fold_tranches_tab.py`	`fold_lifetime_counter`
`src/gui/simulator/sim_fold_tranches_tab.py`	`fold_lifetime_counter`
`src/gui/paper/paper_stack_tranches_tab.py`	`_stack_lifetime_counter`
`src/gui/simulator/sim_stack_tranches_tab.py`	`_stack_lifetime_counter`

`src/gui/main_tabs/fold_tokens_surface.py` holds a third copy of `compose_cycle_close_ratio`, and no counter reaches it. That module reads no bot and no stored field: `src/core/desktop_bridge.py` registers its `view_model` as the handler for the `fold_tokens.state` method, and `src/gui/web/fold_tokens.js` draws that payload's colours, sort orders, column tooltips, row heights and step names. Its ratio copy answers only a request that names the step and carries its own three numbers.

`admitted_counter` sits on both halves of the save path, so a counter the panel will not print is also a counter the file will not store. A stored flag used to be written back as the count one, which made the fault permanent; it is now written back as zero, with the field named in the log.

On screen a refused counter draws an em dash and never a number. The Cycle close ratio row and the Fill ratio row each say a counter is unreadable rather than work out a share from the terms they can still read. The Clear Lifetime Counters button stays live and its confirmation prints the dash, so an unreadable count is never the reason an operator cannot zero the ledger.

Where the fleet comes from

One file is the only source of the bots, and of their wires, ledgers and tranches with them. `StateManager` in `src/core/state_manager.py` reads and writes it. Two restore methods rebuild the fleet and its topology from it, and one save method writes it back. A restored bot connects to nothing and places no order until the operator starts it.

`src/trading/container/restore.py` — `StateRestoreMixin.restore_bots_from_state`

```
def restore_bots_from_state(self, state: dict) -> list[str]:
    """Recreate every persisted bot in IDLE state; return the ids restored.

    Connects to no exchange and places no order. Each restored bot carries
    a ``_restored`` flag and stays IDLE until the operator starts it, which
    is what triggers the exchange sync before any trading.
    """
```

`restore_smart_wires_from_state` in the same mixin rebuilds the topology. `delete_bot` is the only path that removes a record.

Capital claims

One reservation per bot stands against a wallet. A total above the wallet figure is refused whole, and the free figure reports what is left. The claims settle against the venue, because the exchange is the authority.

`capital_registry` — `CapitalRegistry.request_reservation` That file and that method are gone. The claims table lives in `src/trading/capital_reservation.py` now, as `CapitalReservationRegistry`, and `in reserve` is the call that takes a claim. It stores a `Reservation` and returns its token rather than a granted figure, and `in effective_available` answers what is left. The block below records the older contract.

```
"""Create or replace ``bot_id``'s ``Reservation`` and return
``(granted, reason, granted_usd)``.

A total above the ``_wallet_usd`` figure is refused whole with
``granted_usd`` 0, and a ``_wallet_usd`` of None grants unchecked.
"""
```

The reservation table reaches the frontend through its own bridge method, named in the table at the foot of this file. The Qt panel beside it, `CapitalRegistryPanel`, is declared and no screen builds it.

The dollar registry and its table are gone

The paragraph above describes a subsystem the tree no longer holds. Nothing imported the dollar registry, so nothing built one, and the table it fed could draw no row.

Four modules are removed: the registry in `capital_registry`, the Qt table in `capital_registry_panel`, the view model in `capital_registry_surface`, and the renderer page in `src/gui/web/capital_registry.js`.

The bridge no longer offers the method. `build_registry` in `src/core/desktop_bridge.py` returns 72 methods, and none of them starts with `capital_registry`. The renderer manifest lists 68 modules, and every one has a file on disk.

The slot that held the dollar registry is removed with it. `BotManager` carried a `_capital_registry` attribute, a setter with no caller, and five methods that read it. Every method they called on it — `request_reservation`, `release_reservation`, `grow_reservation`, `get_reservations` and `reconcile_with_exchange` — has no definition anywhere in the tree, so filling the slot would have raised on the first call. The attribute, the setter and the five readers are gone.

Claims in asset units are a different mechanism and they stay. `CapitalReservationRegistry` in `src/trading/capital_reservation.py` holds one claim per bot in target-asset units. The saved claim state held 38 of them on 2026-09-13. A bot resolves that registry itself, through `CapitalReservationMixin._crr`, and claims on its first tick.

Bridge

Seven methods serve this screen, and the renderer modules carry the matching names.

Bridge method	Serves
`bot_visualizer.state`	The whole tab
`bot_swarm_tab.state`	The three sub-tabs and the header row
`bot_node.state`	One locust card
`wire_canvas.state`	The wire overlay on the grid
`quick_routing.state`	The quick routing matrix
`capital_registry.rows`	The reservation table
`fold_tranches_tab.state`	The tranche book

The row for `capital_registry.rows` names a method the bridge no longer offers. Six methods serve this screen now. `build_registry` in `src/core/desktop_bridge.py` is the table that decides, and it registers the other six.

2026-09-07 15:58 - no issue recorded - ten single-word tabs, each on its own ground

The Bot Swarm is a proprietary capital reinforcement network system that allows profit to be dynamically and strategically routed between positions with the primary intention of this being to accelerate accumulation curves. Smart Wires are dragged between active bots or the quick connection matrix can be used to route multiple streams to different destinations. Each wire can carry a different percentage amount of profit. Profit sent over Smart Wires are registered at the destination as Wire Credits and these are then distributed into standing Fold Tranches which allow them to have a Surplus that will be spent to increase the Target Balance up to the Maximum Growth Per Cycle. Yes, that is probably a mouth full but hopefully the settings and names are, for the most part, self-explanatory.

The tab is now called Swarm. It sits sixth on the bar, on the gold ground with red text.

Each bot draws as one locust card on the grid, and wires drag between nodes.

`src/gui/visualizer/bot_node.py` — `BotNodeWidget`

```
class BotNodeWidget(QWidget):
    """Paint one bot as a locust, fed by set_bot_data and animate.

    paintEvent draws the wings, abdomen, thorax and head, then the
    symbol, the P/L and the bot_id as text.
    """
```

`SmartWireManager` in `src/trading/smart_wire.py` owns the topology and one ledger per bot. One function bounds what a bot may export, reading the target balance, the band edges and the ammunition the next fold needs, which stops a bot sending away the capital it holds for its own dip.

`src/trading/smart_wire.py` — `compute_safe_outflow_pct`

```
def compute_safe_outflow_pct(
    scrum_profit_usd: float,
    target_balance_usd: float,
    current_price: float,
    band_lower: float,
    band_upper: float,
    next_fold_ammo_usd: float,
    current_cash_usd: float,
    compound_growth_pct: float,
    retained_this_cycle_usd: float,
) -> float:
```

At the far end, arriving USD spreads across the open fold tranches, or parks as a pending wire credit while no tranche stands.

`src/trading/scrumming/wire_routing.py` — `apply_wire_income`

```
def apply_wire_income(self, usd: float, source: str, ref: str = "") -> dict:
    """Apply incoming Smart Wire USD to this bot's fold queue.
```

Three functions collapse or remove a tranche and no others: merge, despawn and clear. Despawn removes an aged record outright rather than holding it in a delisted state, and the count lands in the discarded ledger rather than the closed one.

`src/trading/scrumming/fold_tranches.py` — `_despawn_aged_tranches`

```
_days = self._despawn_threshold_days()
report = {
    "threshold_days": _days,
    "fold_delisted": 0,
    "stack_delisted": 0,
    "stack_kept_live_order": 0,
    "ageless_kept": 0,
    "usd_delisted": 0.0,
}
```

The fleet, and its wires, ledgers and tranches with it, comes from one state file alone. A restored bot connects to nothing and places no order until the operator starts it.

`src/trading/container/restore.py` — `StateRestoreMixin.restore_bots_from_state`

```
def restore_bots_from_state(self, state: dict) -> list[str]:
    """Recreate every persisted bot in IDLE state; return the ids restored.
```

Three sub-tabs open the screen: Bot Swarm, Simulator Swarm and Paper Swarm. The header row holds the drag hint, an identifier privacy dot that masks the bot hashes and the symbol labels together, the Privacy Mode button, and three controls. Exchange filters both the swarm and the quick routing scope. Theme lists the four palettes. The Wires slider sets wire opacity from 0 to 100 %.

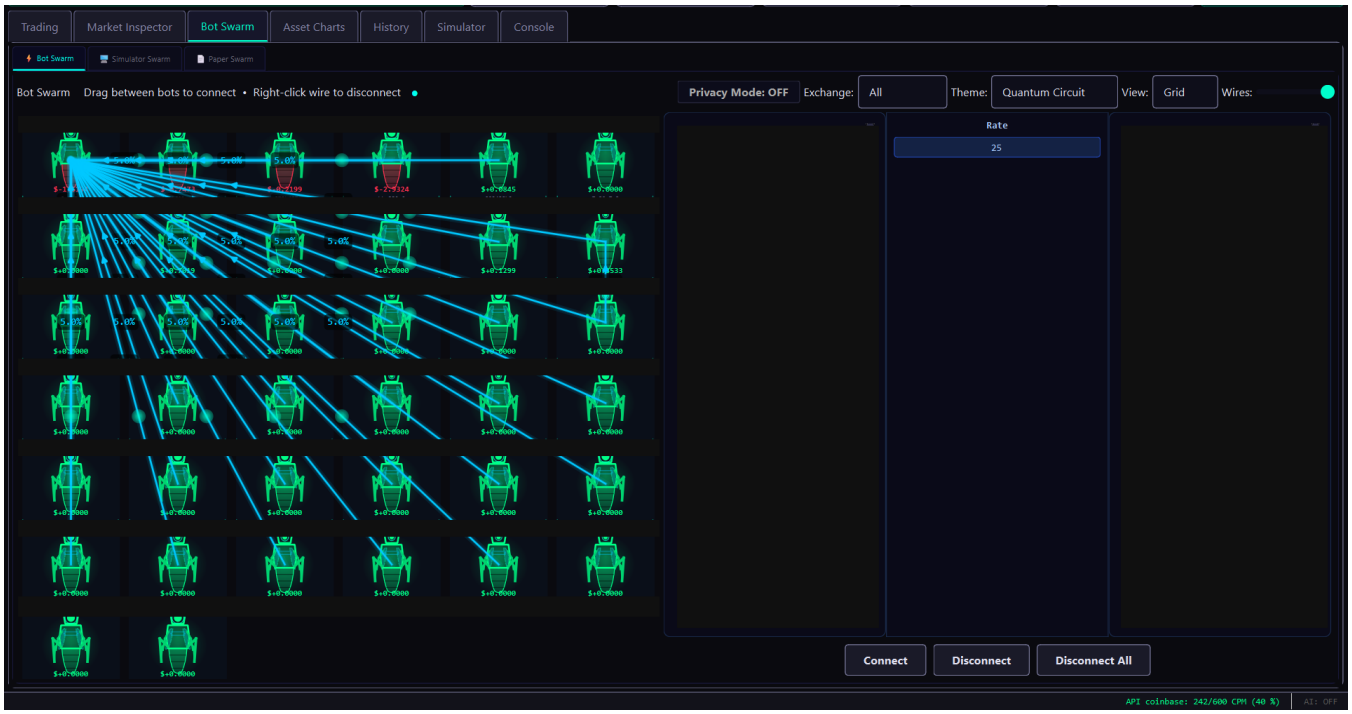
The quick routing matrix fills the right half in three zones: the source list, the Rate field between them, and the destination list. Every checked source wires to every checked destination at that rate, and four conditions are refused out loud before anything is created.

`src/gui/visualizer/quick_routing.py` — `QuickRoutingMatrix._on_connect_clicked`

```
if not (0.0 <= pct <= 100.0):
    self._reject(f"Rate {pct} is outside 0–100%.")
    return
if not sources:
    self._reject("No SOURCE bots are checked.")
    return
if not dests:
    self._reject("No DESTINATION bots are checked.")
    return
if pct <= 0:
    self._reject(
        "Rate is 0% — that would create wires that route " "nothing."
    )
    return
```

A confirmation puts the count in front of the operator before Connect, Disconnect or Disconnect All runs.

(Grid View)



Each card carries the symbol above, the realised profit below it, and the first eight characters of the bot id at the foot. The profit colour is the theme's success colour at zero and above and the error colour below, and the abdomen gradient takes that same colour with its alpha scaled by the size of the figure.

Privacy Mode is off in the figure, so the symbol and the id both draw plain. One field id covers all three readings on a card, the symbol, the short bot id and the tooltip, so a card is never half masked.

`src/gui/visualizer/bot_node.py` — the symbol, drawn through the one mask

```
p.drawText(
    QRectF(0, 2, w, 11),
    Qt.AlignCenter,
    _mask_or(symbol, "bot_swarm.identifiers"),
)
```

Theme names the palette the canvas paints in. The module declares four, and the figure sits on Quantum Circuit. The palette decides the locust body, the success and error colours the profit line takes, and the four corner brackets around each card, which draw in the second accent at low alpha.

`src/gui/visualizer/themes.py` — the palette the figure is on

```
"quantum": {
    "name": "Quantum Circuit",
    "bg": QColor(10, 15, 25),
    "accent": QColor(0, 200, 255),
    "accent2": QColor(0, 255, 200),
    "success": QColor(0, 255, 136),
    "warning": QColor(255, 180, 0),
    "error": QColor(255, 50, 80),
```

The wires cover the grid, because the grid is the only surface whose coordinates a wire can be drawn against. Each wire carries its own percentage, drawn beside it. Drag between two nodes to create a wire, and right-click a wire to remove it.

A wire's percentage sits in a rounded dark badge at the midpoint of the curve, so a wire crossing a card still reads. The Wires slider at the top right sets the opacity of the whole overlay rather than of any one wire.

`src/gui/visualizer/wire_canvas.py` — the badge under the label

```
  label = f"{pct}%"
  font = QFont("Consolas", 8, QFont.Bold)
  p.setFont(font)
  fm = p.fontMetrics()
  tw = fm.horizontalAdvance(label) + 8
  badge = QRectF(label_pt.x() - tw / 2, label_pt.y() - 9, tw, 18)
```

React draws this tab. The choice is made in one place, under the screen name `BOT_SWARM`. A build stamped for Qt gets the old widget. Every other build gets the web one, which holds the tab's page in a single view and answers the calls the main window already made.

`src/gui/main_tabs/bot_swarm_tab.py` — `BotSwarmTabMixin._build_bot_swarm_tab`

```
      def _build_bot_swarm_tab(self) -> None:
          """Build the Bot Swarm tab and add it to the main tab widget."""
          from ..variant_surface import BOT_SWARM, surface_class

          try:
              built = surface_class(BOT_SWARM)()
```

Every header control reports back to Python. The privacy dot, the Privacy Mode button, the Exchange and Theme pickers and the Wires slider each send one action. The screen then redraws from the answer.

`src/gui/react_bot_swarm_tab.py` — `BotSwarmReactTab.take`

```
          def take(self, params: dict) -> dict:
              """Apply one action to the model, push the payload and return it."""
              answer = surface.apply_action(self._state, params)
              self._last_model = answer
              if self._page_ready:
                  self._run(push_script(answer))
              return answer
```

A drag between two bots opens the rate box. A drag between two bots already wired opens the disconnect confirmation. A drag released on empty space offers the source bot's outgoing wires. One method decides which of the three it is, so both screens ask the same question in the same words.

`src/gui/main_tabs/bot_visualizer_surface.py` — `BotVisualizerModel.finish_drag`

```
    def finish_drag(self, source_id: str, target_id: str) -> Optional[dict]:
        """Set ``panel`` to what a wire drag from ``source_id`` asks for next.

        ``WireBoard.finish_drag`` decides the outcome; this turns it into
        the rate box, the disconnect confirmation or the wire picker the
        screen shows.
        """
```

The window a bot row opens

Clicking a bot opens its own settings window. Which window opens depends on the build. The React build draws the header, the tab strip and every tab inside one embedded browser; the Qt build draws the widget tree those tabs describe. The Simulator opens the same window from its own bot rows.

`src/gui/main_window.py` — `_on_bot_clicked`

```
from .variant_surface import BOT_LIVE_SETTINGS, surface_class

_cls = surface_class(BOT_LIVE_SETTINGS)
dlg = _cls(current_bot, self._bot_manager, self)
```

The React side is a subclass, not a rewrite. It replaces only the method that builds the widgets, so an edit still travels the same route into the bot. `_mark_changed` records the field against what the bot holds now, and `_apply_changes` writes it. The window carries no second copy of that logic.

Now: each box on the React Settings tab keeps the operator's edit until Apply, and the window records it only where the Qt control could hold it, a list pick as its item (`src/gui/main_tabs/live_settings_tab_surface.py, in admitted_edit`).

`src/gui/react_bot_live_settings.py` — `BotLiveSettingsReactDialog.apply_edit`

```
field = field_for(str(asked.get("name") or ""))
if not field:
    return
self._mark_changed(field, asked.get("value"))
```

Each tab is drawn by its own renderer module into a named empty space the window leaves for it. `TAB_PLAN` pairs the tab with that space, its module and the surface that builds its payload from the bot. A tab whose surface refuses the bot draws the reason in that space rather than nothing.

`src/gui/react_bot_live_settings.py` — `TAB_PLAN`, the first two rows

```
("Status", "status-page", "acervatorLiveStatus", "live_status_tab_surface"),
(
    "Settings",
    "settings-page",
    "acervatorLiveSettingsTab",
    "live_settings_tab_surface",
),
```

A refusing tab costs one tab, never the window. On the Qt side `_add_guarded_tab` and `_install_guarded_tab` build each tab and, on a refusal, add a page naming the kind that refused and the value, so the other six tabs still open and the operator's click still reaches a window. On the React side `tab_payload` answers a fault payload for that one space, with its import and its model lookup inside the same guard, because `redraw` fills every space in one loop.

`src/gui/bot_live_settings.py` — `_add_guarded_tab`

```
try:
    page = self._wrap_scrollable(build())
except Exception as tab_exc:
```

2026-09-09 22:12 - #128 - one view, growth stages, and wires that hang

List View is gone. The header row now holds the drag hint, the identifier privacy dot, the Privacy Mode button, Exchange, Theme and the Wires slider. The View picker between Theme and Wires is removed, and the grid of locust cards is the whole left pane.

`src/gui/bot_visualizer.py` — the grid goes straight into the row

```
            inner_hbox = QHBoxLayout()
            # No gap: _quick_routing_matrix takes every pixel the grid leaves.
            inner_hbox.setSpacing(0)
            inner_hbox.addWidget(self._grid_widget)
            inner_hbox.addWidget(self._quick_routing_matrix, stretch=1)
```

The card is smaller and the grid is wider

A card asks for 88 by 78 pixels, down from 112 by 98. That is 6,864 pixels of area against 10,976, a drop of 37 per cent, and the grid fits eight columns where it fitted six. The three text rows are measured in points rather than in card units, so they survive the smaller card unchanged; the insect body scales by the same ratio as the card.

`src/gui/visualizer/bot_node.py` — the card size

```
CARD_WIDTH_PX = 88
CARD_HEIGHT_PX = 78
```

Four growth stages, keyed on realised profit

Each card sits in one of four stages. The stage comes from the bot's realised profit measured against its cost basis, both pulled from the exchange, and the 200 per cent floor is the same number the Smart Wire maturity test reads.

Stage	Realised growth	Dominant colour	Decoration
Hopper	0 to 99 %	grey and black	wing buds, two tergites
Fledgling	100 to 199 %	red and white	three tergites
Immature adult	200 to 299 %	silver	full grown, four tergites

Stage	Realised growth	Dominant colour	Decoration
Mature adult	300 % and over	gold	three crown spines

The stage names are the published ones. They come from the FAO Desert Locust Guidelines, part one, Biology and behaviour, by Symmons and Cressman, which names the hopper, the fledgling, the immature adult and the mature adult in that order. The module records that source beside the names.

`src/gui/visualizer/growth_stage.py` — the source and the floors

```
STAGE_SOURCE = (
    "FAO Desert Locust Guidelines 1. Biology and behaviour, Symmons and Cressman"
)

HOPPER_FLOOR_PCT = 0.0
FLEDGLING_FLOOR_PCT = 100.0
IMMATURE_FLOOR_PCT = MATURE_GROWTH_PCT
MATURE_FLOOR_PCT = 300.0
```

One function turns the two exchange figures into the percentage. It returns nothing at all when the basis is absent or not above zero, which is how a bot the venue has not answered for is kept out of a stage it has not earned.

`src/gui/visualizer/growth_stage.py` — `realised_growth_pct`

```
    if not math.isfinite(basis) or not math.isfinite(profit) or basis <= 0.0:
        return None
    return profit / basis * PCT_SCALE
```

A second function reads the stats dict the fleet load carries and refuses a bot whose exchange reading is stale. A card with no reading draws as a Hopper and prints an em dash where the percentage goes, never a zero.

`src/gui/visualizer/growth_stage.py` — `growth_pct_from_stats`

```
    if fresh <= 0.0:
        return None
    return realised_growth_pct(stats.get(BASIS_KEY), stats.get(REALISED_KEY))
```

The two figures reach the screen through the bot status snapshot, which now carries them beside the year-to-date sums.

`src/trading/bot_container.py` — `get_status`, the three added stats

```
                "realized_pnl_exchange": round(
                    float(getattr(self.stats, "realized_pnl_exchange", 0.0) or 0.0), 4
                ),
                "cost_basis_total_exchange": round(
                    float(getattr(self.stats, "cost_basis_total_exchange", 0.0) or 0.0),
                    4,
                ),
                "exchange_data_fresh_ts": float(
                    getattr(self.stats, "exchange_data_fresh_ts", 0.0) or 0.0
                ),
```

Every stage colour is a theme token

No card holds a colour of its own. Each theme carries a body tone and a trim tone for each stage, and the trim is also the colour the growth figure is printed in. The bot id line reads its colour from the same place.

`src/gui/theme_engine.py` — the stage tokens on `ThemeTokens`

```
    locust_hopper_body: str = "#4a4a52"
    locust_hopper_trim: str = "#9a9aa6"
    locust_fledgling_body: str = "#c4303f"
    locust_fledgling_trim: str = "#f2f2f7"
    locust_immature_body: str = "#9ea4ad"
    locust_immature_trim: str = "#e2e6ec"
    locust_mature_body: str = "#b8912c"
    locust_mature_trim: str = "#f0c64a"
    locust_id_text: str = "#8c8ca8"
```

Every text colour on a card was measured against every canvas ground. One hundred and sixty readings were taken, five app themes by four canvas palettes by eight text roles, and all one hundred and sixty clear the WCAG AA floor of 4.5 to 1. On the Quantum Circuit ground the symbol reads 13.35 to 1, the positive P/L 14.30 to 1, the negative P/L 5.32 to 1 and the bot id 5.87 to 1. The four growth figures read 6.89, 17.18, 15.31 and 11.76 to 1.

`src/design_system.py` — the measure used

```
def validate_contrast(fg: str, bg: str, large_text: bool = False) -> tuple[bool, float]:
```

What the card says without a hover

Four readings, in the order they are drawn. The symbol names the market. The P/L is the realised figure in dollars. The growth figure beneath it is the percentage that placed the card in its stage. The short bot id at the foot separates two bots on one market. A one-letter stage badge sits in the top right corner.

`src/gui/visualizer/bot_node.py` — the growth figure and the badge

```
            p.drawText(
                QRectF(0, h - 31, w, 9),
                Qt.AlignCenter,
                growth_text(growth_pct),
            )
```

The tooltip carries the stage in words and the same percentage, so the colour on the card is never the only place the stage is stated.

The animation repaints eight times less often

Every coroutine in this application runs on the window's own thread, so a card that repaints on every frame of the 33 millisecond timer spends that thread. A still card now repaints every fourth frame. Measured over 300 frames, both builds repaint a still card 75 times, which is 7.58 times a second. A card holding a trade ring or a speck still repaints on every frame until they fade. Across a fleet of 38 cards that is 288 repaints a second where it was 1,152.

`src/gui/visualizer/bot_node.py` — `animate`

```
            self._since_repaint += dt
            busy = bool(self._trade_pulses or self._particles)
            if not busy and self._since_repaint < REPAINT_INTERVAL_S:
                return False
            self._since_repaint = 0.0
            self.update()
            return True
```

A wire hangs as a catenary

A wire is no longer a curve bent through a midpoint. It is the curve a cable makes when it hangs between two points, which has a published equation. The wire carries 18 per cent more cable than the straight distance between the two bots, and the two wires of a bidirectional pair hang at different depths.

`src/gui/main_tabs/wire_canvas_surface.py` — `catenary_parameter`

```
    wanted_ratio = math.sqrt(length * length - drop * drop) / span
    ceiling_ratio = math.sinh(CATENARY_U_MAX) / CATENARY_U_MAX
    if wanted_ratio >= ceiling_ratio:
        return None
```

Driven on a 200 pixel span, the sampled curve measures 1.17972 times its chord against the 1.18 asked for, sags 55.03 pixels, and lands exactly on both bots. A wire between two bots in the same column has no span, and falls back to the straight line between them.

`src/gui/main_tabs/wire_canvas_surface.py` — `catenary_curve`, the sample loop

```
    for at in range(count):
        x_at = x_from + span * at / (count - 1)
        points.append([x_at, lift - a * math.cosh((x_at - vertex_x) / a)])
```

No two flow-rate labels overlap

The percentage badge goes where the curve is flattest, which is the lowest point of the hanging wire. Where that place is already taken, the badge walks outward along the curve and then downward in 20 pixel steps until it is clear.

`src/gui/main_tabs/wire_canvas_surface.py` — `place_badge`

```
    for at in badge_order(points, flattest):
        box = badge_rect(points[at], advance_px)
        if not any(rects_overlap(box, one) for one in placed):
            return box
```

Measured on a grid of 38 bots carrying 152 wires, with the real 33 pixel width of a printed rate label: 114 pairs of badges sit on each other at the flattest point, and none does after the walk. Both numbers come from the same run, so the zero is a result rather than an empty instrument.

One painter each side reads one set of geometry. The Qt sheet imports the curve and the placement from the same module the React payload is built from, so the two cannot drift.

`src/gui/visualizer/wire_canvas.py` — what the Qt sheet imports

```
from ..main_tabs.wire_canvas_surface import (
    arrow_points,
    catenary_curve,
    label_text,
    place_badge,
    point_at,
    pulse_percent,
    slack_for_offset,
)
```

Right-clicking a wire measures against the same curve, so the wire the pointer finds is the wire on the screen.

Qt against React

Twenty-two measured values were read off the Qt tab and out of the React payload and compared: the card size, the grid columns and spacing, the repaint interval, the theme count, the opacity slider, the absence of a View control, the tooltip, the four stage names, labels, initials and floors, the tergite counts, the crown spines, the wing spreads, the body scales, the stage colour table, the bot id colour and the absent-growth text. Twenty-two of twenty-two matched. The comparison reported one real difference on its first run, a grid of eight columns on the Qt side against six on the React side, which is what it exists to catch.

2026-09-09 22:12 - #128 - the Swarm tab loses List View

The Swarm tab no longer offers a choice of view. The grid of locust cards is the whole screen, and the View picker that used to sit between Theme and Wires is gone from both builds. The table below carries the row for every file the change touched; the rows in the table above are the state before it.

Qt file	React module	Uses React	Bridge	Manifest	Registers in Electron	Ships in the build	RENDERS	Scope
`bot_swarm_list`	removed	-	-	-	-	-	-	deleted
`src/gui/bot_visualizer.py`	`bot_visualizer.js`	yes	yes	yes	yes	yes	yes	in scope
`src/gui/visualizer/bot_node.py`	`bot_node.js`	yes	yes	yes	yes	yes	yes	in scope
`src/gui/visualizer/wire_canvas.py`	`wire_canvas.js`	yes	yes	yes	yes	yes	yes	in scope
`src/gui/visualizer/growth_stage.py`	shared, no module of its own	-	-	-	-	yes	yes	in scope
`src/gui/react_bot_swarm_tab.py`	host for `bot_visualizer.js`	yes	yes	yes	yes	yes	yes	in scope
`src/gui/theme_engine.py`	`design_tokens.js`	-	yes	yes	yes	yes	yes	in scope

The Qt widget file went. `BotListView` and `LaneWireCanvas` had one caller, the Swarm tab, and nothing else built either of them. The live bot settings window draws its own wire table from `QTableWidget`, so it lost nothing.

`bot_swarm_list` — what was deleted

```
class BotListView(QTableWidget):
class LaneWireCanvas(QWidget):
class BotSwarmLaneAllocator:
```

List View is removed whole. The surface `bot_swarm_list_surface.py` and the module `bot_swarm_list.js` are deleted. The bridge method, the manifest line and the renderer page's script tag that reached them are gone.

`src/gui/web/bot_swarm_tab.js` — the shell that still names the list globals

```
  var LIST_API = "acervatorSwarmList";
  var LIST_LOADER = "acervatorLoadBotSwarmList";
```

Nothing defines those two globals now. The bot settings shell reads them in four places and guards every read, so the list region draws nothing and raises nothing.

`src/gui/react_bot_swarm_tab.py` — `TAB_SCRIPT_ASSETS`, the tail

```
    "quick_routing.js",
    "bot_visualizer.js",
)
```

One control in `tools/conversion_state.py` named the deleted file. A control pointed at a file that no longer exists reports `born React` rather than `paired`, which reads as a pass and proves nothing, so it now names a Qt file that is still paired.

`tools/conversion_state.py` — `CONTROLS`

```
CONTROLS = ("bot_visualizer", "theme_engine", "design_tokens")
```

Both builds were opened with the change in place. Qt draws eight main tabs and React draws ten, the same counts as before, and the Swarm tab in each carries no View picker, no row list and no lane sheet.

2026-09-23 - #239 - Disconnect All clears the key the fleet is stored under

Every control the tab carries

The tab holds three sub-tabs, six header controls and three routing buttons. A wire also carries two menus of its own.

Control	What it does	Behind it
Bot Swarm, Simulator Swarm, Paper Swarm	Picks the layer on show	the sub-tab bar
Identifier privacy dot	Masks the bot hashes and the symbol labels together	`_toggle_bot_swarm_identifier_mask`
Privacy Mode	Turns every mask on, or every mask off	`_on_privacy_mode_btn_clicked`
Exchange	Filters the swarm and the routing scope	`_on_exchange_filter_changed`
Theme	Picks one of the four palettes	`_on_theme_changed`
Wires	Sets the overlay opacity, 0 to 100 %	`_on_wire_opacity_changed`
Connect	Wires every ticked source to every ticked destination	`connect_clicked`
Disconnect	Removes the wire on every ticked pair	`disconnect_clicked`
Disconnect All	Removes every wire in the swarm	`disconnect_all_clicked`
Right-click a wire	Offers that wire, and both directions when both are wired	`_show_disconnect_menu`
Drag onto empty space	Offers the bot's own wires, one entry each	`_show_disconnect_picker`

Every one of the three routing buttons refuses out loud rather than doing nothing. With no bot ticked, Connect and Disconnect both answer **"No SOURCE bots are checked."**

What Disconnect All clears

A wire is stored once, at the top level of the fleet file, under `smart_wires`. `StateManager.save_state` writes that list from `SmartWireManager.export_wires`, and `restore_smart_wires_from_state` reads it back at the next launch.

`src/gui/main_tabs/bot_visualizer_surface.py` — the key a wire is stored under

```
WIRES_KEY = "smart_wires"
SOURCE_KEY = "source_id"
TARGET_KEY = "target_id"
```

`clear_all_routes` empties that list and answers the pairs that were in it. Both builds call it: the Qt tab through `_clear_all_routes_in_state`, and the React page through

`LiveRoutingTab.clear_all_routes`. Each answered pair becomes one `wire.removed` on the bus, which `BotManager._on_wire_removed_mgr` turns into an `unregister_wire` on the topology's owner. The canvas empties as those messages arrive, and the next launch reads an empty list and paints nothing back.

`src/gui/main_tabs/bot_visualizer_surface.py` — `clear_all_routes`

```
def clear_all_routes(state: dict) -> list:
    """Empty ``WIRES_KEY`` and report the pairs ``stored_wires`` found.

    Each reported pair is one ``wire.removed`` the caller emits.
    """
```

The sentence this overtakes. The page carried no sentence naming the key Disconnect All cleared. The older wording that describes the routes a wire is stored as lives on in `apply_routes`, which Connect and Disconnect still use; only the clear and the paint read `smart_wires`.

The question, on both screens

The Qt build puts the question in a message box. The React page now draws the same two sentences itself, from the words the surface already published to it, and sends the operator's reply back with the press. A press that carries no reply is refused, because a bulk change nobody confirmed must not go through.

`src/gui/main_tabs/quick_routing_surface.py` — `LiveRoutingTab.ask`

```
def ask(self, title: Any, body: Any) -> bool:
    """Take the next reply in ``answers``, or no once they run out."""
```

The React page takes the wires onto its board

The React host subscribes to `wire.created` and `wire.removed` at build, the two messages every wire reaches a screen as, and retracts both subscriptions when the tab closes. Before that it held none, so its wire board drew nothing however many wires the fleet carried.

`src/gui/react_bot_swarm_tab.py` — `BotSwarmReactTab._subscribe_to_wires`

```
for topic, action in (
    (surface.WIRE_CREATED_TOPIC, "wire_created"),
    (surface.WIRE_REMOVED_TOPIC, "wire_removed"),
):
```

2026-09-23 - #239 - every control read on the running program

What each control does, in both builds

The Qt tab carries **33** controls and the React page carries **21**. Every one was pressed or read on the running program, against a read-only copy of the saved fleet: 38 bots, 33 wires, 51 ledgers.

Control	What it does	Qt	React
Bot Swarm, Simulator Swarm, Paper Swarm	Picks the layer on show	yes	yes
Identifier privacy dot	Masks the bot hashes and the symbol labels together, in the shared register	yes	yes
Privacy Mode	Turns every mask in the shared register on, or every one off	yes	yes
Exchange	Narrows the swarm and the routing scope to one exchange	yes	yes
Theme	Picks one of the four palettes	yes	yes
Wires	Sets the overlay opacity, 0 to 100 %	yes	yes
Source ticks, Destination ticks	Pick the bots a routing press acts on	yes	yes
Rate	The share a new wire carries, 0 to 100 %	yes	yes
Connect	Writes one wire per ticked pair into the stored wire list	yes	yes
Disconnect	Removes the stored wire on every ticked pair	yes	yes
Disconnect All	Empties the stored wire list	yes	yes
Drag between two bots	Asks for a share, then stores that wire	yes	yes
Right-click a wire	Removes that wire, and offers both directions when both are wired	yes	**absent**
Drag onto empty space	Offers the bot's own wires, one entry each	yes	yes
+ Add Sim Bot, Run All, Stop All	Adds a Simulator bench row, and flips every row's own button	yes	**draws, acts on nothing**
+ Add Paper Bot, Start All, Stop All	Adds a Paper bench row, and flips every row's own button	yes	**draws, acts on nothing**
A bench row's asset, preset and capital	The values that row carries	yes	**absent**
A bench row's Run or Start	Flips that row's own button and status	yes	**absent**
A bench row's cross	Drops that row	yes	**absent**

The three routing buttons refuse out loud rather than doing nothing. Read on the running program: no source answers **"No SOURCE bots are checked."**, no destination answers **"No DESTINATION bots are checked."**, a rate that is not a number answers **"Rate 'abc' is not a number. Enter a percentage between 0 and 100."**, a rate of 0 answers **"Rate is 0% — that would create wires that route nothing."**, and a rate of 140 answers **"Rate 140.0 is outside 0–100%."**

A `yes` in the React column means the control acts when it is pressed. It does not mean the operator can reach it: only the sub-tab bar and the header strip paint on that page today. The section on how wide the tab has to be carries the measurement.

The key every connectivity press writes

The sentence this overtakes, from the section above:

> *The older wording that describes the routes a wire is stored as lives on in* `apply_routes`*, which Connect and Disconnect still use; only the clear and the paint read* `smart_wires`*.*

The true sentence. `apply_routes` now writes `WIRES_KEY`, so every connectivity press reads and writes the one list `stored_wires`, `clear_all_routes` and `hydration_plan` already read. Connect adds a record, a second Connect on the same pair replaces its share rather than adding a second record, and Disconnect drops it. Read on the running program with 33 wires stored: a Connect takes the list to **34** and a Disconnect takes it to **32**.

`src/gui/main_tabs/bot_visualizer_surface.py` — `apply_routes`

```
def apply_routes(state: dict, add: list, remove: list) -> dict:
    """Write each ``add`` pair into ``WIRES_KEY`` and drop each ``remove`` pair.

    ``stored_wires``, ``clear_all_routes`` and ``hydration_plan`` read
    that same list.
    """
```

The Qt tab calls the same function, so the widget and the page store a wire the same way. The right-click menu and the drag picker both land in `remove_wire`, which now drops the pair from the store as well as from the canvas, and the drag that makes a wire stores it when the operator accepts the share.

A direct write is refused when the load it came from carries no fleet, because `_load_bot_state_dict` answers an empty dict when the read fails. Read on the running program: a write from an empty load leaves the file byte-identical and logs the refusal; the same call from a real load moves the file.

The dot and the Privacy Mode button read one register

`PrivacyMaskRegistry` owns all 19 masks and the Trading tab writes the same one. Both builds now write it and both read it back.

- The dot flips `bot_swarm.identifiers`, the glyph turns from ● to ■, and the routing row labels turn from the symbol and short id to `**** [********]`.
- Privacy Mode sets all 19 masks on, or all 19 off, and its own word follows.
- **The dot also rewrites the Privacy Mode button.** Before this unit the button read `Privacy Mode: OFF` while one mask was on.

`src/gui/main_tabs/bot_visualizer_surface.py` — `any_mask_on`

```
def any_mask_on(self) -> bool:
    """Whether ``mask_registry`` holds any mask on.

    Answers ``masked`` or ``any_masked`` with no registry.
    """
```

What the page does not carry yet

The page draws the six bench layer buttons and no bench row. Each button carries no handler, so pressing all six leaves the row count at 0. The page has no right-click menu on a wire either: nothing under `src/gui/web/` binds a context menu. The Simulator bench belongs to #117 and the Paper bench to #19; a bench row's own Run and Start buttons flip a label and a status and start no session in either build.

The issue #117 and issue #19 clause above is overtaken. It is not edited and not removed. It read:

> *The Simulator bench belongs to #117 and the Paper bench to #19; a bench row's own Run and Start buttons flip a label and a status and start no session in either build.*

Issue #117 closed on 2026-09-19 and issue #19 closed on 2026-09-26, so neither pointer names open work. One surface declares both benches now, and the React page names the bench eleven times.

```
src/gui/main_tabs/bot_visualizer_surface.py
    SIM_BENCH_ID_FORMAT      "SIM-{idx:02d}"
    PAPER_BENCH_ID_FORMAT    "PAP-{idx:02d}"
    in sim_summary           the three lines under the Simulator bench rows
    in paper_summary         the same for Paper, with the bench capital
    in bench_id_text         a row's own id
    in bench_toggled         what a row's Run press answers
    in bench_status_style_sheet   the status colour

src/gui/web/bot_visualizer.js   names the bench on 11 lines
```

Whether a bench row starts a session is a product choice, so nothing here proposes one.

How wide the Swarm tab has to be

The Qt tab refuses to draw narrower than **1086 px**: the locust grid asks 786 px across its eight columns and the Quick Routing panel asks 278. Asked for 700 or for 900 the tab is 1086 wide both times, so in a pane under 1086 px the Wires slider and the three routing buttons sit past the right edge. At 1400 the whole tab draws — 38 locusts, 33 wires with their share labels, the two tick columns, the Rate box and the three buttons.

The React page fits every width sideways and **none of it downwards**. At 700, 900 and 1400 the body needs no sideways scroll and every header control and routing button reports its right edge inside the viewport. Read down the page instead: `tab-body` is 873 px tall and stacks all three layer panes in the flow — 287 + 293 + 293 — because a pane's own `display: flex` beats the `hidden` attribute. Inside the 287 px that leaves, `header-row` measures 900 px tall over children of 17 to 24 px, so `inner-row` starts at y = 931 with a height of 0, `quick-routing` has a height of 0, a routing button sits at y = 1754, and `wire-canvas` is `display: none`. The captured page shows the tab bar and the header strip and nothing under them. The locust grid, the wires and the routing panel are in the document and are not on the screen.

Where the wire list sits once a password is set

The sentence this overtakes, from the section on what Disconnect All clears:

> *A wire is stored once, at the top level of the fleet file, under* `smart_wires`*.*

The true sentence. That is where the wire list sits on an install carrying no password. With a password set, `smart_wires` moves inside one sealed field and the top level carries a plain-text latch in its place. `src/core/state_manager.py, in _sealed_for_disk` moves `bots`, `bot_count`, `smart_wires` and `smart_wire_ledgers` into that field on every save, and `in unseal` puts all four back for a session holding the key. Read on a state file saved with a password: the top level holds five fields — `fleet_sealed`, `sealed_fleet`, `saved_at`, `saved_at_human` and `version` — and not one of the four fleet fields. Read on the same material saved with no password: the same seven fields the page describes, and no latch.

What a routing row reads while the fleet is sealed

The sentence this stands beside, from the section on the privacy dot. It is not edited and not removed:

The dot flips `bot_swarm.identifiers`*, the glyph turns from ● to ■, and the routing row labels turn from the symbol and short id to* `**** [********]`*.*

The current reading. A row label is the pair and the short id while the fleet is open. While the fleet is sealed there is no pair to read, and the row says so in a word instead of drawing a question mark. `src/gui/main_tabs/quick_routing_surface.py, in SEALED_SYMBOL` holds that word and `in LiveRoutingTab.state_symbol` returns it when `src/core/state_manager.py, in fleet_sealed` answers True. `in QuickRoutingModel.symbol_for` carries it into the label, and `src/gui/visualizer/quick_routing.py, in QuickRoutingMatrix._symbol_for` answers the same word on the Qt panel.

Read on a hand-built fleet of two, once sealed with a password and once saved without one:

```
the file   privacy dot   symbol_for    the drawn row label
sealed     off           SEALED        SEALED [UNIT-A1]
sealed     on            SEALED        **** [********]
plain      off           AAA/BBB       AAA/BBB [UNIT-A1]
plain      on            AAA/BBB       **** [********]
```

The privacy dot still masks a sealed row, so the sentence above holds unchanged for what the dot does.

? is what the row reads for a bot the stored settings cannot answer for, and that case is still a question mark. `src/gui/main_tabs/quick_routing_surface.py, in UNKNOWN_SYMBOL` holds it. A sealed fleet and a bot with no stored symbol are two different readings and the row no longer draws them alike.

Both builds draw the word. The label is formatted before the panel is handed its values, so the React page draws `SEALED [UNIT-A1]` from the same string the Qt panel draws.

Owed, one line. `docs/Acervator-Product-Manual.pdf` needs a rebuild for this page, `docs/manual/08-tabs/bot-swarm.md`. Another branch holds that binary and this change does not touch it.

The grid is the only surface that draws a bot's figures

The sentences this overtakes, from the section on the live rows:

> *A strip above the locust grid holds one dense row per live bot: the id, the pair, the mode, the timeframe, the capital, the state, the price, the profit and the trade count. The strip is hidden while the fleet is empty. The same fleet load that draws the locusts fills it.*

The true sentences. No swarm tab draws a one-line bot row. The locust grid is the only surface that draws a bot's figures and it carries them under the icon. Driven with a six-bot fleet, Bot Swarm, Simulator Swarm and Paper Swarm build no row between them.

Each locust draws eight strings. The pair sits above the icon. Under the icon sit the realised growth figure, the profit and loss, the short id, the run state, the trade count and the last price. The mode, the candle length and the budget are not on the card, because the running-bot window already shows them.

The card is 88 by 108 pixels. It grew downward by three lines and the locust did not move: `BODY_BOX_HEIGHT_PX` pins the body box at the former card height, so the body centre and the body scale read what they read before.

`src/gui/main_tabs/bot_node_surface.py, in BotNodeModel.paint`

```
cx, cy = width / 2, BODY_BOX_HEIGHT_PX / 2
scale = min(width, BODY_BOX_HEIGHT_PX) / SCALE_DIVISOR * BODY_SCALE_RATIO[stage]
```

The three new lines are drawn at five point Consolas, each in a rectangle the full width of the card and ten pixels tall. The widest string each field can produce, measured with the machine's own Consolas loaded:

line	longest string	width
pair	thirteen characters	65 px
profit and loss	sign, seven digits, four decimals	80 px
run state	`COOLDOWN`	32 px
trade count	four digits and the word	44 px
last price	eight decimals, or seven digits	52 px

Every one fits the 88-pixel card. The Qt card and the React card take the same sizes from `bot_node_surface`, so the two builds draw the same eight strings in the same places.

CITED AS ABSENT

The path below is named on this page only to record which renderer page the dollar registry used before its removal. It does not open on disk.

- `src/gui/web/capital_registry.js` — the renderer page the dollar registry fed. Commit `d72f69cf` removed the registry, its Qt table, its view model and this page together. The renderer manifest names 68 modules and every one has a file on disk.

Back to the subsystem index.

Asset Charts

Reference. One candlestick panel per traded symbol.

What builds it

The builder mixin makes the tab and adds it to the row. The tab is a scroll area holding one panel per active bot.

`src/gui/main_tabs/charts_tab.py` — `ChartsTabMixin._build_charts_tab`

```
def _build_charts_tab(self) -> None:
    """Build the Charts tab and add it to the main tab widget."""
    self._charts_tab = TradeChartsTab()
    self._main_tabs.addTab(self._charts_tab, CHARTS_TAB)
```

Issue #450 renamed the tab from Asset Charts to Charts, and `CHARTS_TAB` holds that one word.

One method rebuilds that panel set from the current roster.

`src/gui/widgets/trade_charts_tab.py` — `TradeChartsTab.update_charts`

```
    def update_charts(
        self,
        bot_statuses: list[dict],
        bot_manager=None,
        exchange_connectors: dict = None,
    ) -> None:
        """Create or update chart panels for each bot.
```

Three more methods keep a panel current.

Method	What it does
`_on_tf_changed`	Changes one panel's timeframe, leaving the others alone
`log_trade`	Adds a fill marker to the panel that owns the symbol
`push_synthetic_candles`	Feeds a panel whose venue returned no history

The chart widget

`CandlestickChart` in `src/gui/native_chart.py` paints each panel with QPainter. No browser and no WebEngine dependency.

Four record types carry everything drawn over the candles.

Three do now. `GridLine` is gone, and no painter in either build ever drew from it.

Record	Draws
`Candle`	One open, high, low, close and volume bar
`TradeMarker`	An executed buy or sell
`PositionMarker`	A holding, with a distinct icon for invisible and visible

The position record is the one that carries the visibility mode, which is what tells an invisible holding from one resting on the book.

`src/gui/native_chart.py` — `PositionMarker`

```
    @dataclass
    class PositionMarker:
        """Active position shown on chart with visibility mode icon."""

        price: float
        side: str  # "buy" or "sell"
        visibility: str  # "internal" (invisible) or "orderbook" (visible)
        level: int = 0  # Grid level index
        filled: bool = False
        asset_held: float = 0.0
```

The setters name what they place.

Setter	Places
`set_candles`	The candle series

Setter	Places
`add_marker`	One executed fill
`set_trade_history_markers`	The fills already on record
`set_positions`	The holdings, invisible and on book
`set_tranche_floors`	The standing fold-tranche floors
`set_target_balance_lines`	The dollar target
`set_fire_armed_state`	An armed manual fire
`set_source_label`	The feed the candles arrived on
`set_error`	A failure message in place of the panel body

The mouse handlers give the panel a crosshair with a price and time readout, drag panning and a wheel zoom bounded by `_effective_visible_start` and `_effective_visible_count`.

Indicators on the chart

The chart calls the engine rather than computing anything of its own. Five series arrive that way, and an entry with no value is skipped when the panel paints.

`src/gui/native_chart.py` — `CandlestickChart._compute_indicators`

```
def _compute_indicators(self):
    """Fill ``_bb_data``, ``_vortex_data``, ``_macd_data``,
    ``_stochrsi_data`` and ``_ichimoku_data`` from the engine
    classes. A ``None`` entry marks a candle with no value, and
    ``paintEvent`` skips it.
    """
```

The toolbar under each panel holds the timeframe picker and eight toggles.

`src/gui/main_tabs/native_chart_surface.py` — `INDICATOR_TOGGLES`

```
INDICATOR_TOGGLES = (
    ("bb", "BB", "#50a0f0"),
    ("vortex", "Vortex", "#00ff88"),
    ("macd", "MACD", "#ffcc00"),
    ("stochrsi", "SRsi", "#ff9060"),
    ("ichimoku", "Ichi", "#c080ff"),
    ("volume", "Vol", "#80ffcc"),
    ("slingshot", "Sling", "#ff4488"),
    ("bbullseye", "BBull", "#ff00aa"),
)
```

Volume starts on and the other seven start off.

`src/gui/main_tabs/native_chart_surface.py` — `INDICATOR_DEFAULTS`

```
INDICATOR_DEFAULTS = {
    "bb": False,
    "vortex": False,
    "macd": False,
    "stochrsi": False,
    "ichimoku": False,
    "volume": True,
    "slingshot": False,
    "bbullseye": False,
}
```

Sling and BBull paint a placeholder shape rather than the indicator, and each says as much in its own tooltip. Issue #430 carries one further disagreement between a tooltip on this toolbar and what the chart paints.

The issue #430 clause above is overtaken. It is not edited and not removed. It read:

> *Issue #430 carries one further disagreement between a tooltip on this toolbar and what the chart paints.*

Issue #430 closed on 2026-09-09. No tooltip on this toolbar names a colour, so no tooltip can disagree with what the chart paints. The Vortex entry names its two lines, its period and its pane.

```
src/gui/native_chart.py
    tooltip="Vortex Indicator (VI+ / VI-, period 14, sub-pane)"

tooltips declared in that module   14
tooltips naming a colour            0
```

The complete voter set and the maths behind each indicator belong to the Indicator Voting Panel.

Where the candles come from

Three sources in order: the connected exchange first, a public API second, and the last successful fetch third. Each source that fails is recorded and the next one runs.

`src/exchange/chart_data.py` — `ChartDataFetcher.fetch`

```
# --- Source 1: Exchange OHLCV via CCXT ---
if exchange is not None:
    try:
        candles, source = await self._fetch_exchange(
            exchange, symbol, timeframe, limit
        )
        if candles:
            self._set_cache(symbol, timeframe, candles)
            return candles, source
```

With every source exhausted the fetcher returns an empty list and a label naming what failed, and the panel draws that label rather than an empty grid.

`src/exchange/chart_data.py` — `ChartDataFetcher.fetch`

```
# All sources failed
error_detail = " | ".join(errors) if errors else "No sources available"
```

Each row arrives as an `OHLCVCandle`.

The other chart

`src/gui/tradingview_chart.py` holds the QWebEngineView chart, which runs HTML and JavaScript inside the Chromium that PySide6 already ships. The Asset Charts tab does not use it; the panels above are painted natively. Its one construction site is the stock window, and the application builds no stock window, so no screen draws it today.

`src/gui/stock_main_window.py` — the one construction site

```
from .tradingview_chart import TradingViewChart

self._chart = TradingViewChart(symbol="AAPL", theme="dark")
```

Bridge

Three methods serve this screen.

Bridge method	Renderer module
`native_chart.state`	`native_chart.js`
`trade_charts_tab.state`	`trade_charts_tab.js`
`tradingview.chart`	`tradingview_chart.js`

2026-09-08 13:30 - #407 - a Live / ATA-SMP toggle over the arrows

The tab now walks two lists of markets. Live is the markets the bots are trading. ATA-SMP is the markets the TA engine has called and queued. One selector serves both: the arrows, the ticker menu and the readout all follow whichever list the toggle names.

The toggle is a button on its own row, above the blue arrows and the ticker menu. It reads the list on screen, so the button says Live while the Live list is up and ATA-SMP while the called markets are up.

`src/gui/main_tabs/trade_charts_tab_surface.py` — the two lists and their words

```
LIST_LIVE = "live"
LIST_ATA = "ata_smp"
LIST_MODES = (LIST_LIVE, LIST_ATA)
LIST_LIVE_TEXT = "Live"
LIST_ATA_TEXT = "ATA-SMP"
```

Pressing the toggle swaps the list, points the chart at that list's own selected market, and writes its header. Each list remembers where it was, so coming back to Live returns to the asset that was on screen.

`src/gui/main_tabs/trade_charts_tab_surface.py` — pressing the toggle

```
def toggle_list(self) -> str:
    """Move the arrows to the other list and answer the mode on screen."""
    self.list_mode = LIST_ATA if self.list_mode == LIST_LIVE else LIST_LIVE
    self.calls.append([SELECT_LIST_TOGGLED, self.list_mode, len(self.list_order())])
    self._follow_shown()
    self._label_shown()
    self._feed_shown()
    return self.list_mode
```

A market joins the ATA-SMP list when it reaches the Ready to Send bucket. It does not have to be posted. A call that is queued and then declined is still tracked, because the trigger is the bucket rather than the delivery.

The tab holds no register of its own. It reads the markets through a callable the Market Inspector hands it, so the list the operator walks and the calls phase seven is watching are one set of markets read twice.

`src/gui/main_tabs/market_inspector_tab.py` — where the two are joined

```
def _wire_ata_chart_list(self, inspector: Any) -> None:
    """Point the Charts tab's ATA-SMP list at ``inspector``'s ``PushBoard``.

    The Charts tab is built first, so it reads the board through a
    callable instead of holding a second copy of the markets.
    """
```

No entry is ever invented. With nothing bound, or with an empty bucket, the ticker offers one line reading No called market, the readout says 0 of 0, the arrows go grey, and a note beside the toggle says what the list is waiting for.

`src/gui/main_tabs/trade_charts_tab_surface.py` — the empty state's own words

```
ATA_EMPTY_TICKER_TEXT = "No called market"
ATA_EMPTY_HINT = "A market joins this list when it reaches Ready to Send."
```

An ATA-SMP market has no bot behind it, so the chart draws its candles without the trade markers, the target lines, the tranche floors and the fire glow that a traded asset carries. Nuclear Mode returns the tab to the Live list before it draws, because a Nuclear scenario belongs to a bot.

Figures. This page carries no figure. The screen changed on this date and no capture of the toggle row exists yet, so the paragraphs above are the only record of it.

2026-09-08 13:30 - #407 - the arrows now reach the renderer

The React half of this tab drew the arrows and the ticker menu, and pressing either one changed nothing. The buttons called the bridge with the step it wanted, and the handler read every other request field and dropped that one, so the answer that came back was the state the tab was already in.

`src/gui/main_tabs/trade_charts_tab_surface.py` — the three the handler now reads

```
        params.get("step_by"),
        params.get("pick_at"),
        params.get("toggle_list", False),
```

The same tab also refused to draw at all under the renderer. One of its own checks read a name the file never declared, which stopped the whole payload before a single row was placed. The check now compares the number of rows the Python side declares against the number the renderer draws, which is the disagreement it was written to catch.

`src/gui/web/trade_charts_tab.js` — the check, repaired

```
  // The surface and this module must name the same number of tab rows.
  function checkSlots(model) {
    var content = objectField(model, CONTENT);
    if (!owns(content, LAYOUT_SLOTS)) {
      return;
    }
    if (content[LAYOUT_SLOTS] !== LAYOUT_SLOT_COUNT) {
      chartFaults.push(
        fault(CONTENT, LAYOUT_SLOTS, DISAGREES_FAULT, LAYOUT_SLOT_COUNT)
      );
    }
  }
```

2026-09-13 15:40 - #23 - the Charts tab draws in the desktop window

The tab draws under the window the operator launches. `ChartsTabReact` in `src/gui/react_charts_tab.py` builds one web view and loads one page. The page carries every script and every rule inside it, so it fetches nothing.

`src/gui/react_charts_tab.py` — the modules the page carries, in load order

```
def roster() -> tuple[str, ...]:
    """Every module the page carries, in load order.

    ``STYLE_SOURCE_ASSETS`` leads: ``trade_charts_tab.js`` reads a payload
    colour back to its token through ``shared_widgets.variableFor``.
    """
    return STYLE_SOURCE_ASSETS + CHILD_MODULES + (PANEL_MODULE,)
```

`trade_charts_tab.js` draws the tab. `native_chart.js` draws the candles into the slot that module keeps. Both register with the shell's own panel host, so the page in the window and the page in the shell run the same two modules.

The page reaches the model over one console channel. Each ask leaves the page as a line under `ASK_PREFIX`, and `ChartsTabPage` hands that line to `run_ask`. The answer returns by the ask's own number. The arrows, the ticker menu and the TF picker each move the asset the window holds.

`src/gui/react_charts_tab.py` — the call the dashboard pass makes

```
        def update_charts(
            self,
            bot_statuses: list,
            bot_manager=None,
            exchange_connectors: Optional[dict] = None,
        ) -> None:
            """Rebuild the asset list for one pass of bot statuses and redraw."""
            self._model.update_charts(bot_statuses, bot_manager, exchange_connectors)
            self.redraw()
```

`fetch_chart_data` refetches the asset on screen. `set_ata_source` takes the callable behind the ATA-SMP list. Both answer the calls the main window and the Market Inspector tab already make on the Qt tab.

The tab's own style sheet

`src/gui/web/trade_charts_tab.css` paints the chrome around the chart. It holds no colour of its own. Every rule names a design token, and `design_tokens.js` writes each token onto the page as a CSS variable.

`src/gui/web/trade_charts_tab.css` — the arrows and the list toggle

```
[data-part="asset-prev"],
[data-part="asset-next"],
[data-part="chart-list-toggle"] {
  background: var(--SURFACE_CONTROL, var(--btn-bg));
  color: var(--PRIMARY, var(--accent));
  border: 1px solid var(--GLOW_PRIMARY_EDGE, var(--border));
  border-radius: 4px;
  font-family: inherit;
  font-size: 12px;
  font-weight: bold;
  cursor: pointer;
}
```

An arrow with one asset to walk goes grey. The panel takes a border and a rounded ground. The header line, the toolbar and the toggle row each take a rule between them. The readout beside the arrows and the empty-list hint take the low text colour at ten pixels.

The chart keeps a host of its own

`native_chart.js` opens a React root of its own. It draws into `chart-host`, a child element the tab declares and never fills. One element under two roots lost the whole tab the moment the waiting line cleared.

`src/gui/web/trade_charts_tab.js` — the slot the chart draws into

```
    var hostProps = { key: CHART_HOST_PART, style: { flex: AUTO, overflow: HIDDEN } };
    hostProps[PART_ATTR] = CHART_HOST_PART;
    hostProps[BOT_ATTR] = text(props.botId);
```

A missing chart module says so on the slot. `renderChartMounts` asks the panel host to mount the slot with no model, and the host writes the reason it refused where the candles belong.

The tab redraws when its width changes. `resizeEvent` re-arms a short timer, and the timer pushes the payload again. The chart then takes the width the tab has.

What the page draws

The page draws the Live toggle, the two arrows, the ticker menu and the readout saying which asset of how many. Under them one panel header names the symbol, the price and the bot state. The chart draws the candles, the Bollinger cloud, the Ichimoku cloud, the last price on the right axis, the volume strip and the Vortex, MACD and Stochastic RSI panes. Eight indicator switches close the panel, with Sling and BBull off.

The legend beside the feed name carries its own colour. `native_chart_surface` holds the two Qt style sheets for it, and the tab surface sends them as `legend_styles`.

Back to the subsystem index.

Where the chart draws

The Charts tab host mounts the chart module inside itself. The tab draws in the window the operator launches, not only in the shell. `CHILD_MODULES` in `react_charts_tab.py` names the one module the tab slot takes.

```
modules the page registered   header_strip, native_chart, trade_charts_tab
panel faults                  none
drawn from forty candles      40 candles, 40 wicks, 40 volume bars
indicator panes               Vortex, MACD, Stochastic RSI
```

With the chart module absent the page still draws its own chrome, and the panel host records the loss rather than drawing an empty slot.

```
native_chart.js registered no panel to draw
```

2026-09-15 10:38 - #55 - one painter draws the chart in both builds

The chart area of the Charts tab is the image one painter paints, in both builds. The Qt build's chart widget already painted through that routine; the React build now shows the same image where its own drawing used to be. The ATA-SPM post images come from the same routine, so one chart looks one way in the Qt tab, in the React tab and in a post.

`src/gui/native_chart.py` — the routine both builds and ATA-SPM paint through

```
    def paint_image(
        painter: ChartPainter,
        width_px: int,
        height_px: int,
        device_pixel_ratio: float = 1.0,
    ) -> QImage:
        ratio = float(device_pixel_ratio) if float(device_pixel_ratio) > 0 else 1.0
        image = QImage(
            int(round(int(width_px) * ratio)),
            int(round(int(height_px) * ratio)),
            QImage.Format_ARGB32,
        )
        image.setDevicePixelRatio(ratio)
        image.fill(painter.BG_TOP)
        image_painter = QPainter(image)
        painter.paint_to(image_painter, int(width_px), int(height_px))
        image_painter.end()
        return image
```

Fed the same hundred candles, the same label and the same six overlays, the three agree byte for byte at 1200 pixels wide: the Qt widget, the React image and the ATA-SPM image all read `d8932829dcf760c1`. A planted change to one candle's close moves the Qt widget and the React image to the same new digest, `4dbcb29e360c363e`, so the agreement is a reading and not an artefact. The ATA-SPM image for its own inputs is unchanged by this work.

How the React page gets the image

The tab holds one painter beside its model. Each time the page draws, the chart slot asks the tab for the image at the slot's own width and height and at the page's device pixel ratio. The tab feeds the painter what the panel holds and answers a PNG as a data URI, which the slot shows as one image element.

`src/gui/react_charts_tab.py` — what the painter is fed, as the Qt tab feeds its chart

```
        def _feed_painter(self) -> None:
            """Give ``painter`` what ``PanelSink`` holds, as the Qt tab gives its chart."""
            panel = self._model.panel
            painter = self._painter
            painter.symbol = str(panel.label)
            painter.set_timeframe(str(panel.chart_timeframe or panel.timeframe))
            painter.set_candles(
```

The image carries the display's pixels. At a device pixel ratio of 1.25 the page asked for 1201 by 731 CSS pixels and received a 1501 by 914 pixel image; at 2x, 600 by 492 CSS pixels received 1200 by 984. In the header band the 2x image holds 395 distinct colours where the 1x image scaled up holds 305, and the thinnest vertical run in the price pane is one device pixel where the scaled 1x image needs two.

The page no longer carries `native_chart.js`. The tab's list of child modules is empty, the page registers two modules, and the file stays in the tree unmounted.

`src/gui/react_charts_tab.py` — the modules the chart slot mounts

```
CHILD_MODULES: tuple[str, ...] = ()
```

The eight toggles reach the painter

Each check box under the chart now asks the tab to switch that overlay on the painter, and reads the painter's own state back off the answer. The Qt build's boxes drive the same switch on the same painter.

`src/gui/web/trade_charts_tab.js` — a click on one box

```
  function overlayChosen(key, on) {
    if (!global.acervator || typeof global.acervator.call !== "function") {
      loadFault = NO_BRIDGE;
      return Promise.resolve(null);
    }
    var params = {};
    params[TOGGLE_KEY_PARAM] = key;
    params[TOGGLE_ON_PARAM] = Boolean(on);
    return global.acervator
      .call(METHOD, params)
```

Each of the eight, flipped from its starting state, moves the image to a new digest in both builds, and the two builds' digests agree on every flip; flipped back, each returns to the starting digest. A toggle left alone answers the last image with no repaint. The starting states are six on and two off: BB, Vortex, MACD, SRsi, Ichi and Vol on; Sling and BBull off.

What a repaint costs

The tab answers the last image again when nothing it reads has changed. When something has, the painter repaints and the PNG is encoded on the window's own thread. Measured ten runs each, six overlays on, one hundred candles:

width x height	ratio	image	paint	PNG + base64
1200 x 492	1x	1200x492	7.6 ms	21.6 ms
1920 x 492	1x	1920x492	9.0 ms	31.5 ms
1200 x 492	2x	2400x984	18.0 ms	76.9 ms
1920 x 492	2x	3840x984	22.1 ms	112.9 ms

The dashboard pass calls `update_charts` every two seconds, and the header label carries the price, so a moving price repaints on most passes. Candles refetch at most every 30 seconds.

Live candles reach the React chart

The React tab's fetch handed the fetcher's rows to `list`, and the fetcher's row type is not iterable, so every live fetch ended in an error line and no candles. The rows now pass through `candle_row`, which reads an object, a dict or a six-number row.

`src/gui/main_tabs/trade_charts_tab_surface.py` — the fetch, repaired

```
            if rows:
                self.panel.set_candles([candle_row(one) for one in rows])
                self.panel.set_source(answered)
```

Figures. `artifacts/issue-55/` holds the Qt tab, the React tab and the ATA-SPM image before and after, for the same hundred candles. The React page draws the tab header line and the painter draws its own header line under it, so the label reads twice on that build.

2026-09-19 08:23 - #55 - every Voting Panel indicator on the chart

His words, 2026-08-21: *"Asset Charts Tab - Restructure - Must now display only one chart at a time with all indicators from the Indicator Voting Panel loaded and properly visualized."* And 2026-09-08: *"Want charts tab to only display one richly detailed and Acervator-annonated chart at a time with all Indicators visible by default excluding that might visually oclude BB or ICHI as Sling or Z-Score Algo Point. All indicators will need toggle check boxes at the bottom."*

The five series the indicator classes now answer

ADX, Supertrend, Z-Score, KER and RSI each gain one method beside their vote, named as the Vortex and Ichimoku classes name theirs. The vote reads that method, so the line on the chart and the vote in the Voting Panel are one arithmetic.

`src/trading/indicators/adx.py` - the series and the vote reading it

```
    def lines(self, candles: list) -> tuple[_Line, _Line, _Line]:
        """+DI, -DI and ADX, one entry per candle, for a chart to draw.
```

```
        di_plus_series, di_minus_series, adx_series = self.lines(candles)
```

class	`lines` answers, one entry per candle
`ADXIndicator`	+DI, -DI and ADX
`SupertrendIndicator`	the line, its side and the ATR under it
`ZScoreIndicator`	a `ZScoreBar`: the smoothed score, the two projected prices and the window figures
`KaufmanERIndicator`	the ratio
`RSIIndicator`	Wilder's RSI
`SlingshotIndicator`	a `SlingshotBar`: the squeeze state, the release, the snapback and the momentum

The vote is unchanged. `compute` from the commit before and `compute` from this one ran over every prefix of 600 recorded A15 1h candles, six classes, every Signal field compared: 0 of 600 prefixes differ for any class. The control: shifting one class's series by one candle makes the same comparison report 549 to 589 differing prefixes.

Fourteen overlays, seven sub-panes

The overlay registry now holds the twelve Voting Panel indicators in the panel's own order, then the two overlays that draw no voter: the Z-Score algo point and BB Bullseye.

`src/gui/native_chart.py` - one of the six new entries

```
    ChartOverlay(
        key="adx",
        label="ADX",
        colour_field="chart_last_price",
        pane=SUB_PANE,
        occludes=False,
        draw="_draw_adx",
        tooltip="ADX (14) with +DI and -DI, 0..100, 20 and 25 ruled, sub-pane",
        voter="adx",
    ),
```

overlay	pane	what it paints	ruled
ADX	sub-pane	+DI, -DI and ADX on 0..100	20 and 25
STrd	price pane	the Supertrend line, green under price while bullish, red over it while bearish	
ZSc	sub-pane	the smoothed score on a symmetric scale of at least 3	+2 and -2
KER	sub-pane	the Efficiency Ratio on 0..1	
RSI	sub-pane	Wilder's RSI on 0..100	30 and 70
ZPt	price pane	the resistance and support prices the averaged reversals project, dashed	

Slingshot now draws the indicator's own marks. Before, the painter re-derived a bandwidth squeeze of its own and drew marks the indicator never fired; now each diamond is a bar the class read as a squeeze release, signed by its momentum, and each circle a bar it read as a Bollinger snapback.

`src/gui/native_chart.py` - the slingshot marks

```
                if bar.released and bar.momentum != 0.0:
                    marks.append(("release", bar.momentum > 0.0))
                if bar.snapback:
                    marks.append(("snapback", "bull" in bar.snapback))
```

The sub-panes sit under the volume strip in registry order: Vortex, MACD, Stoch RSI, ADX, Z-Score, KER, RSI. Each takes 60 px at the chart's natural height. When the window gives less, every sub-pane shrinks alike, down to 28 px, so the price pane keeps its 220 px and every pane stays on screen.

`src/gui/native_chart.py` - the pane heights

```
SUB_PANE_H = 60
SUB_PANE_MIN_H = 28
PRICE_PANE_LAYOUT_H = 220
PRICE_PANE_MIN_H = 120
```

The legend

A band under the OHLC row names every overlay with the last candle's value of its series, in the overlay's colour. An overlay that is off reads `off` in grey. The band wraps to the chart's width. It is painted by the painter, so the Qt widget, the React image and the ATA-SPM post images all carry it.

`src/gui/native_chart.py` - what one legend entry carries

```
                found.append(
                    {
                        "key": overlay.key,
                        "label": overlay.label,
                        "text": text,
                        "colour": self.overlay_colour(overlay).name(),
                        "on": on,
                        "has_value": value is not None,
                    }
                )
```

Fourteen toggles

The toggle row under the chart holds one box per overlay, in the same order. The three occluders start off: Sling, ZPt and BBull, each a mark or a fill over the price pane. Every other indicator starts on.

`src/gui/main_tabs/native_chart_surface.py` - the occluders

```
INDICATOR_OCCLUDES = {
    "bb": False,
    "vortex": False,
    "macd": False,
    "stochrsi": False,
    "ichimoku": False,
    "volume": False,
    "slingshot": True,
    "adx": False,
    "supertrend": False,
    "zscore": False,
    "ker": False,
    "rsi": False,
    "zscore_point": True,
    "bbullseye": True,
}
```

Both hosts flip an overlay through one method on the painter. The Qt check box calls it, and the React box asks the tab, which calls it.

`src/gui/native_chart.py` - the Qt press

```
            self._chart.set_overlay(name, on)
```

The two emitters

`charts.indicator.toggled` is written once per press, with the key, the state asked and the variant; `ok` is true when the painter's state after the press is the state asked. `charts.indicators.drawn` is written once the candles arrive and the series recompute, with the symbol, the timeframe, the keys shown and each one's legend value; `ok` is true when every shown overlay holds a value on the last candle. Both go through the emitter network's one wire and nothing else.

`src/gui/native_chart.py` - the two names

```
  TOGGLED_PIN = "charts.indicator.toggled"

  DRAWN_PIN = "charts.indicators.drawn"
```

Read off the running program

The real window in each build, the home on a scratch directory, every socket but loopback refused, a two-bot fleet read off a copy of the operator's `bot_state.json`, and the real fetcher fed 100 recorded A15 1h candles by a stand-in connector:

reading	Qt	React
toggles under the chart, in order	14: BB, Vortex, MACD, SRsi, Ichi, Vol, Sling, ADX, STrd, ZSc, KER, RSI, ZPt, BBull	the same 14 on the page
starting states	11 on, Sling, ZPt and BBull off	the same
legend entries	14, 11 with a value, 3 reading `off`	the same 14 in the image answer
ADX on the last candle, by hand and on the painter	20.16, +DI 10.91, -DI 40.24	the same
Supertrend	63,003.61, bearish	the same
Z-Score	-1.24	the same
KER	0.7617	the same
RSI	25.44	the same
each new toggle pressed off then on	the line gone and back, the image digest moved and returned, one emitter record per press	the same through the page's box
sub-pane rules	RSI 30 and 70, ADX 20 and 25, Z-Score +2 and -2: 692 to 727 pixels on each ruled row differ from a repaint with no rules, 0 on the row above and 0 on a control row	the same
`bot_state.json` after every press	byte-identical	byte-identical
the planted legend value	the legend reads 99.99 where the series reads 25.44, and the comparison reports it	the same

The two builds paint one image: the painter's image for the same candles reads one digest in both.

Sentences this entry overtakes

They were not reworded. They are quoted here.

`docs/manual/08-tabs/asset-charts.md, under 'Indicators on the chart'` - "The chart calls the engine rather than computing anything of its own. Five series arrive that way, and an entry with no value is skipped when the panel paints." Fourteen series arrive that way now.

`docs/manual/08-tabs/asset-charts.md, under 'Indicators on the chart'` - "The toolbar under each panel holds the timeframe picker and eight toggles." Fourteen toggles.

`docs/manual/08-tabs/asset-charts.md, under 'Indicators on the chart'` - "Volume starts on and the other seven start off." Eleven start on and three start off.

`docs/manual/08-tabs/asset-charts.md, under 'Indicators on the chart'` - "Sling and BBull paint a placeholder shape rather than the indicator, and each says as much in its own tooltip." Both paint the indicator, and neither tooltip says placeholder.

`docs/manual/08-tabs/asset-charts.md, under 'The eight toggles reach the painter'` - "The starting states are six on and two off: BB, Vortex, MACD, SRsi, Ichi and Vol on; Sling and BBull off." Eleven on and three off.

Figures. `artifacts/u55/C2/` holds the Qt window and the React page at 1400 by 900 and at 1960 by 1200, the painter's image, and one venue PNG, before and after.

2026-09-19 17:40 - #55 - the painter's skin and the Acervator annotations

His words, 2026-08-21: *"Charts did get a slight visual over all during the last past some instances ago but I am still not completely satisfied."* And 2026-09-08: *"I want our chart renderer to be top notch. I want to be able to add modern features and graphics and styling to it so that we can help asset trading migrate away from the pastel and bland."*

Every colour is a theme token

The painter holds no colour of its own. Each colour it paints is one role in its palette table, and each role names a theme field and an alpha. A theme press on the Theme menu now reaches the painter in both builds: the window's theme switch calls the Charts tab, the tab calls the painter, and the painter resolves every role again. The toggle boxes under the chart and the two position labels above it read their colours from the painter, so they follow the press too. The toggle table in the chart's surface carries a theme field name per entry, not a hex string. A theme that lacks a field keeps the default theme's value for that role and the emitter row names how many roles it did resolve.

`src/gui/native_chart.py` - the roles added this day

```
        PALETTE_ROLES: dict[str, tuple[str, int]] = {
            "PANEL_SOURCE_TEXT": (PANEL_SOURCE_FIELD, 255),
            "TAG_TEXT": ("chart_bg_top", 255),
            "STRIP_FILL": ("chart_band", 40),
            "STRIP_EDGE": ("chart_band", 210),
            "FILL_TAG_SURFACE": ("chart_bg_top", 215),
```

`src/gui/main_window.py` - the one line the theme switch gained

```
                self._charts_tab.set_theme(tm.current)
```

At build the painter starts in the theme the window is in, and the venue image paints in that theme too when its caller names none.

`src/gui/native_chart.py` - the theme in force

```
    def theme_in_force():
        """The ``ThemeTokens`` ``ThemeManager.apply_theme`` last painted."""
        return THEMES.get(applied_theme(), DEFAULT_THEME_TOKENS)
```

Crisp at every device pixel ratio

The paint pass reads the device pixel ratio of the surface it paints and holds one device pixel as a logical length. The grid, the time ticks and the pane separators draw with anti-aliasing off, one device pixel wide, on coordinates moved to the centre of a device pixel. The wick is a share of one candle column and never under one device pixel; the candle border is one device pixel; the indicator lines keep their logical width and stay anti-aliased. Read on a 1200 pixel image at ratio 1.0 the pens are 0.8, 1.0, 1.2, 1.4, 1.5 and 1.56 logical pixels; at ratio 2.0 the set gains 0.5, the one-device-pixel pen.

`src/gui/native_chart.py` - the device pixel and the snap

```
            ratio = self._device_ratio(p)
            px = 1.0 / ratio

            def snap(value: float) -> float:
                """``value`` moved to the centre of the device pixel it falls in."""
                return (int(value * ratio) + 0.5) / ratio
```

The type

The header symbol is set in the design system's UI family at the card-title size, the OHLC row in the mono family at the label size, and the legend, the axes, the tags and the caption in the mono family at the caption size. The legend wraps at the chart's width as before; at 1400, 1366 and 1200 pixels it takes two rows and its widest row ends before the price scale.

`src/gui/native_chart.py` - the fonts

```
    def caption_font() -> QFont:
        """The mono font the legend, the axes, the tags and the caption are set in."""
        return design_font(ds.FONT_FAMILY_MONO, CAPTION_FONT_PX)

    def header_font() -> QFont:
        """The UI font the header symbol is set in."""
        return design_font(ds.FONT_FAMILY_UI, HEADER_FONT_PX, bold=True)
```

Every fill as the two glyphs

A fill is drawn as one of the two hermetic glyphs the Simulator's playback draws, from one definition both read: a sell as `dissolve`, a triangle pointing down as an outline, and a buy as `reform`, a triangle pointing up filled. The definition moved from the Simulator's surface to the chart's surface, and the Simulator imports it from there. The glyph sits on the candle whose interval holds the fill's stamp, one candle column wide and a twentieth of the price pane tall, with a tag naming the role and the price beside it. A fill whose stamp falls before the window or past its end is counted and not drawn.

`src/gui/main_tabs/native_chart_surface.py` - the one definition

```
MARK_GLYPHS = {
    SCRUM_SIDE: {
        "name": DISSOLVE_GLYPH,
        "points": [[-0.5, -0.5], [0.5, -0.5], [0.0, 0.5]],
        "filled": False,
    },
    FOLD_SIDE: {
        "name": REFORM_GLYPH,
        "points": [[-0.5, 0.5], [0.5, 0.5], [0.0, -0.5]],
        "filled": True,
    },
}
```

The two glyphs are hermetic characters

Two sentences above are overtaken. Each is quoted whole, with the true sentence under it.

Overtaken: *"A fill is drawn as one of the two hermetic glyphs the Simulator's playback draws, from one definition both read: a sell as* `dissolve`*, a triangle pointing down as an outline, and a buy as* `reform`*, a triangle pointing up filled."*

True now: a fill is drawn as one of two hermetic characters the Simulator's playback draws, from one definition both read. A sell draws U+1F761, named ALCHEMICAL SYMBOL FOR DISSOLVE, in gold. A buy draws U+1F75F, named ALCHEMICAL SYMBOL FOR PRECIPITATE, in blue. The alchemical block carries no COAGULATE character, and precipitate is the standard's name for a substance leaving solution and re-forming solid, which is what a fold does.

Overtaken: *"The glyph sits on the candle whose interval holds the fill's stamp, one candle column wide and a twentieth of the price pane tall, with a tag naming the role and the price beside it."*

True now: the character sits on the candle whose interval holds the fill's stamp, set at a twentieth of the price pane in the symbol family, with a tag naming the role and the price beside it. One candle column still sets the mark's footprint for placing that tag. The corner list and the fill flag are gone with the triangles they described, and so is the outline width, because a drawn character takes no outline.

```
# U+1F761 is named ALCHEMICAL SYMBOL FOR DISSOLVE, the scrum's operation.
DISSOLVE_GLYPH = "\U0001f761"
# U+1F75F is named ALCHEMICAL SYMBOL FOR PRECIPITATE, a substance leaving
# solution and re-forming solid, which is the fold's operation.
REFORM_GLYPH = "\U0001f75f"
MARK_GLYPHS = {
    SCRUM_SIDE: {"name": DISSOLVE_GLYPH},
    FOLD_SIDE: {"name": REFORM_GLYPH},
}
MARK_GLYPH_FAMILY = ds.FONT_FAMILY_GLYPH
```

The family resolves to Segoe UI Symbol, and both characters resolve on it. The Simulator's two playback windows draw the same two characters from the same object.

The fills come from two figures of the bot. Each tab subscribes to the bus topic every fill crosses and records it, so a fill made after launch draws on the next status tick. And each standing fold tranche the bot holds was made

at a scrum: its reference price is that sell and its stamp is that sell's time, so the tab draws each one as a scrum glyph. Before this day nothing called the tab's trade recorder, so no fill had ever reached the chart.

`src/gui/widgets/trade_charts_tab.py` - the subscription and the handler

```
            get_event_bus().subscribe(FILLED_TOPIC, self._on_trade_filled)
```

```
        def _on_trade_filled(self, event) -> None:
            """Record one ``trade.filled`` bus event through ``log_trade``.

            The next ``update_charts`` tick draws it as a glyph on its candle.
            """
```

The target, the floors, the strip, the glow and the position

Each annotation carries a tag at the right edge, a badge filled in the line's colour with the theme's ground colour for text. The target line and the ceiling line read `TARGET` and `CEILING` with their prices. Each tranche floor keeps its dashed line; floors whose tags would overlap share one tag naming their count and their price span, so 16 floors in one pane read as seven tags. The landing strip is a band between the side's Bollinger band and its tolerance edge, over the strip's candles at the bot's own timeframe, tagged with the side and the count; it draws from the bot's own proximity reading. The fire glow keeps its gradient and gains `SCRUM ARMED` or `FOLD ARMED`. The position is the venue's average entry with the units held, drawn as its line, its icon and a `POSITION` tag.

`src/gui/native_chart.py` - the strip's bounds

```
        def _strip_bounds(self) -> Optional[tuple]:
            """The landing strip's two prices, ``(top, bottom)``, or None with no strip set.

            The band lies between the side's Bollinger band and its tolerance
            edge, ``(upper - lower) * tolerance_pct / 100`` inside it.
            """
```

`src/gui/main_tabs/trade_charts_tab_surface.py` - the readers both tabs share

```
def landing_strip(bb: Any, timeframe: Any = "") -> Optional[dict]:
```

```
def position_reading(bot: Any) -> Optional[dict]:
```

```
def tranche_scrums(tranches: Any, bot_id: Any, symbol: Any) -> list:
```

The theme and annotation emitters

`charts.theme.applied` is written once per theme applied, with the theme, the variant and the symbol; `ok` is true when every palette role resolved from that theme. `charts.annotations.drawn` is written when the set of annotations the paint pass drew changes, with the counts drawn against the counts fed that fall inside the pane, and the fills fed and the fills off the window in its context. Both go through the emitter network's one wire.

`src/gui/native_chart.py` - the two names

```
  THEME_PIN = "charts.theme.applied"

  ANNOTATIONS_PIN = "charts.annotations.drawn"
```

The skin readings, off the running program

Both builds, the real window, the home on a scratch directory, every socket but loopback refused, one A15/USD bot's figures off a scratch copy of the operator's fleet file, and 100 recorded A15 1h candles rolled from the 5m tablet by the registry's own rollup:

reading	Qt	React
each of the five Theme menu entries pressed	79 of 79 roles move on the four non-default themes, 0 on the default; the toggle boxes and the two labels restyle	the same 79; the image digest moves on each press and the page's boxes recolour
literal colours in the painter and its surface	3 and 17 before, 0 and 0 after; a literal planted into one draw call reads 1	the same files
two fills handed to the tab's handler, one live-shaped fill on the bus, three standing scrums off the record	6 fills fed, 2 drawn as triangles on candles 60 and 80 of 100, 4 off the window and counted	the same 6, the same 2 triangles at the same vertices
the glyph corners against the Simulator's	one object, `MARK_GLYPHS`	the same object
55 floors off the record	55 lines fed, 16 inside the pane, 7 tags	the same
the target and ceiling prices	equal to the anchor and the cycle ceiling over the units held and the quote rate; outside the pane on these candles	the same
the position	the venue's average entry off the record, one `POSITION` tag	the same
the strip	the bot's own detector reads none on any window of this tablet; a chosen 3-candle upper strip draws its band between the detector's own band and tolerance edge, tagged `STRIP upper 3c`	the same
the glow	`SCRUM ARMED` tagged	the same
pens at ratio 1.0 and 2.0	0.8 to 1.56 logical, then 0.5 joins at 2.0; anti-aliasing off eight times for the grid and the separators, on again after each	the same
a theme with one role	no crash; that role takes the planted colour, every other keeps the default, the emitter row reads 5 of 79 and false	the same
the legend at 1400, 1366 and 1200	two rows, none clipped	the same
`charts.theme.applied`	7 rows, 79 of 79 on each	7 rows
`charts.annotations.drawn`	2 rows, drawn equal to fed inside the pane	2 rows
`bot_state.json` after every press	byte-identical; a copy with one byte appended reads a different digest	the same

reading	Qt	React
the venue image under Neon Light and Cyberpunk Dark	the top-left pixel equals each theme's ground and the foot pixel its lower ground; the two files differ; the message stamp is drawn in the theme's own axis text colour	one painter

Two sentences this entry overtakes

They were not reworded. They are quoted here.

`docs/manual/08-tabs/asset-charts.md, under 'The chart widget'` - "The setters name what they place." The table under it lacks `set_landing_strip`, which places the landing strip band.

`docs/manual/08-tabs/simulator.md, under 'The two glyphs'` - "One definition, `MARK_GLYPHS`, names both marks, and both hosts read it". The definition now lives in the chart's surface, and the Charts painter is a third reader.

Figures. `artifacts/u55/C3/` holds the Qt window and the React page at 1400 by 900 and at 1960 by 1200, the painter's image at ratio 1.0 and 2.0 with a crop of each, and the venue image under two themes, before and after.

2026-09-19 20:10 - #55 - the ATA-SMP list draws the venue image's picture

His words, 2026-09-08: *"After a market qualifies, has its chart rendered with the appropriate confirming indicators and test, and is to be sent to the Ready to Send bucket the Charts tabs will continue to track said market under a ATA-SMP chart list."* And 2026-09-19: *"After its chart renderer is fully upgraded, it will need to be migrated to ATA-SMP."* A called market on the ATA-SMP list now draws the picture its venue images carry: the candles the call was made on, at the call's timeframe, the overlays of its confirming voters and no other, the reversal badge, the reading strip and the caption band. The Live list is unchanged.

What a called market's chart draws now

The chart takes the call's own candles, so it draws the same bars the venue image drew, and it fetches nothing for that market. The overlays switched on are exactly the ones the venue image switched on, the badge names the same direction, the strip carries the same readings and the foot carries the root image's caption. The bot annotations stay off: no fill glyph, no floor, no target line, no landing strip, no fire glow, no position marker.

`src/gui/widgets/trade_charts_tab.py` - what one call puts on the panel

```
        def _draw_call(self, entry: dict, call) -> None:
            """Draw ``call`` on the panel: its candles at its timeframe, its
            overlays, its badge and its caption, with no bot annotation.

            The overlay set the Live list showed is held in ``_live_overlays``
            until ``_leave_call`` restores it.
            """
            chart = self._panel.chart
            if not self._live_overlays:
                self._live_overlays = [
                    key for key, on in chart.overlays_shown().items() if on
                ]
            self._clear_annotations()
            chart.set_timeframe(str(call.timeframe))
            self._panel.show_only(list(call.overlays))
            chart.set_candles(list(call.candles))
            chart.set_call(call.direction, call.readings)
            chart.set_caption(call.caption)
```

The toggle boxes at the bottom follow the picture. The panel's own `show_only` flips the painter and sets each box to match, with the boxes' signals blocked so no press is counted, and the React page reads the same set off the payload it is drawn from.

Where the call comes from

Phase three keeps, on the chart it pulled, the candles the images drew and the overlay keys they switched on. The Inspector's sector board turns each pulled call into one record per symbol when the run lands, and the Charts tab reads that record through a second callable beside the one that lists the markets.

`src/trading/ata_spm.py` - one call as the tab draws it

```
@dataclass(frozen=True)
class ChartCall:
    """One call as the Charts tab draws it: the picture its venue images carry.

    ``overlays`` are the keys the images switched on, ``readings`` the
    ``(voter, message)`` pairs ``set_call`` takes, ``caption`` the root
    image's ``post_caption`` and ``candles`` the ``chart_candles`` rows.
    """
```

`src/gui/main_tabs/market_inspector_tab.py` - the second callable

```
        charts.set_ata_source(inspector.watched_markets)
        # MarketInspectorTab keeps its SectorBoard as _ata_board and offers no accessor.
        board = getattr(inspector, "_ata_board", None)
        if board is None or not hasattr(charts, "set_ata_call_source"):
            logger.debug("no SectorBoard reachable; ATA-SMP charts draw no call")
            return
        charts.set_ata_call_source(board.chart_call)
```

The newest call per symbol is the one drawn, which is the vote the list shows. A market on the list with no call held, because no run in this session called it, draws as before: its candles fetched, no badge.

The Live set held and given back

The overlay set the Live list showed is held the moment a call is drawn and put back when the list returns to Live or moves to a market with no call. The badge, the strip and the caption clear at the same moment.

`src/gui/widgets/trade_charts_tab.py` - leaving a call

```
        def _leave_call(self) -> None:
            """Clear the call's badge and caption and restore the Live overlay set."""
            chart = self._panel.chart
            chart.set_call("", ())
            chart.set_caption("")
            if self._live_overlays:
                self._panel.show_only(self._live_overlays)
                self._live_overlays = []
            if self._showing_ata():
                self._clear_annotations()
```

The tab's ATA-SMP readings, off the running program, both variants

The real window in each variant, the home on a scratch directory, every socket but loopback refused, a connector shaped like Coinbase Exchange serving the operator's own tablet copies, his scan shape (crypto, nothing typed, 1hr 1d 1wk, target 3), then the Charts tab's toggle pressed:

reading	Qt	React
the list after the press	A27 bear 1d, AXS bear 1h	the same
the chart	A27, 1d, 213 candles	the same
overlays on	adx, ichimoku, ker, macd, slingshot, supertrend, vortex; no other	the same
badge and strip	bearish, 7 readings; caption 3 lines	the same
floors, target line, glow, strip, position	none	none
boxes checked at the bottom	the same seven	the same seven off the page
X, Instagram, TikTok images against the chart	overlays equal, badge equal, candles 213 and 213	the same
a market with no voter, planted	50 candles, no overlay, the badge and nothing else	the same
`charts.ata.rendered` rows	one per call drawn, `ok` True	the same
the scratch `bot_state.json`	the same bytes before and after every press	the same

The rendered emitter

`charts.ata.rendered` writes when a called market's chart draws: the overlays the painter shows against the call's, with the symbol, the timeframe, the list, the widget's size, the direction, the candle count and the variant.

`src/gui/widgets/trade_charts_tab.py` - the row

```
                _emit(
                    ATA_RENDERED_PIN,
                    actual=shown,
                    expected=sorted(call.overlays),
                    context={
                        "symbol": call.symbol,
                        "timeframe": call.timeframe,
                        "list": LIST_ATA,
                        "width": chart.width(),
                        "height": chart.height(),
                        "direction": call.direction,
                        "candles": len(call.candles),
                        "variant": "qt",
                    },
                )
```

The sentence the ATA-SMP picture overtakes

It was not reworded. It is quoted here.

`docs/manual/08-tabs/asset-charts.md, under 'a Live / ATA-SMP toggle over the arrows'` - "An ATA-SMP market has no bot behind it, so the chart draws its candles without the trade markers, the target lines, the tranche floors and the fire glow that a traded asset carries." The sentence holds, and holds on screen now: on `5819ced6` the last bot's 55 floors, its target line and its glow stayed drawn when the list moved to a called market, and the candles were fetched. A called market's candles are the call's own, and its picture carries the call's overlays, badge, strip and caption, which the sentence does not name.

Figures. `artifacts/u55/C5/` holds the tab's ATA-SMP chart in both variants, the three comparisons side by side and the seven venue images.

2026-09-19 20:53 - #55 - the page's chart answers the pointer as the widget does

The React page's chart takes the same five pointer inputs the Qt widget takes, and answers each one the same way to the candle: a hover draws a crosshair with its readout, a wheel tick zooms about the pointer, a drag pans, a double-click fits every candle again, and a leave takes the crosshair off. The mechanism differs by input. A hover repaints nothing: the page draws the crosshair itself on a canvas over the image, from the geometry the painter exports beside every image. A zoom, a pan and a reset move the painter's window and repaint the image, as the widget repaints itself.

One arithmetic, called by both

The widget's handlers and the page's asks reach the same methods on `ChartPainter`, and those methods call the pure functions `native_chart_surface` already held: `zoom_window`, `pan_start`, `zoom_factor`, `clamp_y_zoom`, `effective_visible_start`, `effective_visible_count` and `fmt_price`. The drag state moved from the Qt widget onto the painter, so both variants keep it in one place.

`src/gui/native_chart.py` - the wheel, on the painter both variants hold

```
        def wheel_turned(
            self, x: int, wheel_delta: float, width_px: int, control_held: bool = False
        ) -> bool:
            if not self._candles:
                return False
            if control_held:
                self._y_zoom_pct = clamp_y_zoom(
                    self._y_zoom_pct * zoom_factor(wheel_delta)
                )
                self._emit_view(
                    VIEW_CAUSE_Y_ZOOM, self._visible_start, self._visible_count
                )
                self._repaint()
                return True
            window = zoom_window(
                wheel_delta,
                int(x),
                int(width_px),
                len(self._candles),
                self._visible_start,
                self._visible_count,
            )
            if window is None:
                return False
            self._visible_start = window["start"]
            self._visible_count = window["count"]
            self._emit_view(VIEW_CAUSE_ZOOM, window["start"], window["count"])
            self._repaint()
            return True
```

`CandlestickChart.wheelEvent` calls `wheel_turned` with the event's x, its angle delta, the widget's width and whether Ctrl is held. `CandlestickChart.mouseMoveEvent` calls `pointer_moved`, which places the crosshair and pans through `pan_to` while a drag is active; `mousePressEvent` and `mouseReleaseEvent` call `pointer_pressed` and `pointer_released`; `mouseDoubleClickEvent` calls `view_reset`; `leaveEvent` calls `pointer_left`. The grip drag along the bottom strip stays on the widget, which is the only variant that draws a grip.

The page's asks

The page sends each input that moves the window as one ask on the bridge method `trade_charts_tab.view`, with the pointer's x and y, the host's width and height and the page's device pixel ratio. `ChartsTabReact.chart_view` answers it on the tab's painter through the same methods.

`src/gui/react_charts_tab.py` - the ask answered

```
            if action == surface.VIEW_ACTION_WHEEL:
                moved = painter.wheel_turned(
                    x,
                    float(asked.get(surface.VIEW_DELTA_PARAM) or 0.0),
                    width,
                    bool(asked.get(surface.VIEW_CONTROL_PARAM, False)),
                )
            elif action == surface.VIEW_ACTION_PRESS:
                painter.pointer_pressed(x)
            elif action == surface.VIEW_ACTION_DRAG:
                moved = painter.pan_to(x, width)
            elif action == surface.VIEW_ACTION_RELEASE:
                painter.pointer_released()
            elif action == surface.VIEW_ACTION_RESET:
                painter.view_reset()
                moved = True
```

The answer carries the window's start and count and, when the window moved, the repainted image with its geometry, so a zoom costs one round trip. The image key now carries the window and the price padding, so the two-second `update_charts` pass answers the zoomed image again instead of the fitted one. A drag sends one move at a time; a move that arrives while one is in flight waits, and the latest goes when the answer lands.

The crosshair drawn by the page

Every image answer carries `geometry`: the pane rect, the price scale, the candle column width, the window, and for each visible candle its time label and the six readout lines the painter formats with `readout_lines` - the same function `paint_to` draws the widget's readout from - with the theme's colours as CSS text. The page's `pointermove` handler draws the dotted crosshair, the price badge, the time badge and the readout box on the canvas from those numbers, at the positions `paint_to` uses, and writes what it drew on the chart host as `data-crosshair-x`, `data-crosshair-y`, `data-crosshair-candle` and `data-readout`. A `pointerleave` clears the canvas and the attributes. The price badge at the pointer's row is the one text the page formats itself, from `PRICE_FORMAT_BANDS`, the bands `fmt_price` reads.

`src/gui/main_tabs/native_chart_surface.py` - the readout both draw from

```
def readout_lines(candle: CandleLike) -> list[list[str]]:
    accent = READOUT_ROLE_UP if candle.close >= candle.open else READOUT_ROLE_DOWN
    change = candle.close - candle.open
    return [
        ["O", fmt_price(candle.open), READOUT_ROLE_LIGHT],
        ["H", fmt_price(candle.high), READOUT_ROLE_LIGHT],
        ["L", fmt_price(candle.low), READOUT_ROLE_LIGHT],
        ["C", fmt_price(candle.close), accent],
```

The two pointer emitters

Two pins through `signal_contract.emit`. `charts.view.changed` writes each time the window moves, with `actual` the window the painter holds, `expected` the window the surface function answered, and the cause: `zoom`, `y_zoom`, `pan` or `reset`. `charts.crosshair.shown` writes at most once per quarter second per painter, with `actual` the candle `candle_at_x` places under the pointer and `expected` the candle the drawing pass placed: the widget's `paint_to` on the Qt build, the page's throttled `crosshair` ask on the React build.

The five inputs read off the running program

Both variants built the real `MainWindow` with the home on scratch, every socket but loopback refused, one A15/USD bot off a scratch copy of `bot_state.json`, 100 A15 1h candles rolled from a copy of the operator's 5m tablet, at a device pixel ratio of 1.25, the widget and the page's chart host both 1358 pixels wide.

input	Qt widget	React page
pointer at x 679 in the price pane	candle 52; readout O 64,634.01, H 64,846.02, L 64,632.71, C 64,741.54, delta +107.53 (+0.17%), V 445	candle 52; the same six lines to the character
one wheel tick in at x 679	window None, None to 7, 85	7, 85; the image digest moved
a drag of 120 px left	start 7 to 15	7 to 15
a double-click	None, None; padding 1.0	None, None; padding 1.0
a leave	`_mouse_x` None; the crosshair pixels gone	the canvas holds 0 painted pixels; the attributes gone
a pointer in the header band	no readout	no readout, no canvas pixels

With `ZOOM_IN_FACTOR` set to 0.5 in the React process only, the same wheel tick reads 26, 50 on the page against 7, 85 on the widget, so the equality reading can fail. The scratch `bot_state.json` and the tablet copy read the same digest after every input in both variants.

What a hover and a zoom cost

Measured on the page at 1.25, the chart host 1358 by 561 CSS pixels: a hover draws in 0.3 to 0.4 ms per move over 60 moves, the widget's repaint reading 13.9 ms per move over the same 60. A wheel tick on the page runs 66 to 83 ms from the ask to the image placed - 14.3 ms to paint, 41.1 ms to encode a 222 KB PNG at 1698 by 701 pixels, the rest transport and the 30 KB geometry - against 13.4 ms on the widget. A hover on the page repaints nothing.

Figures. `artifacts/u55/C4/` holds the crosshair and readout in both variants on the same candle, and both after the same wheel tick.

2026-09-20 05:40 - #55 - the frame: one control row, compact toggles, the value field on the ATA-SMP picture

His words, 2026-09-20: *"it does not appear that the GUI Archetype organized all of the elements properly using any kind of rules as we still have a significant void space and poorly placed or incorrect icons"*; *"TA values are smashed together in a long row above the chart when they should be stacked in the lower left corner in their own field with a dark background and these only need to be present for ATA-SMP posting"*; *"Check boxes for indicators are over large and taking up too much real estate."* The tab now lays its controls by one rule, in both builds.

The layout rule

One control row sits at the top of the tab. From the left: the list toggle, the previous arrow, the ticker menu, the next arrow, the `n of N` counter, the empty-list hint when the ATA-SMP list is empty, the `TF:` word and the timeframe menu, then a stretch, then the two legend labels and the source label at the right edge. Every control in the row is `CONTROL_HEIGHT_PX` tall and the gap between two controls is `SELECTOR_SPACING_PX`. The type is the design system's: the buttons and the menus at `TYPE_SMALL` in `FONT_FAMILY_UI`, the counter and the labels at `TYPE_CAPTION`, the two arrow glyphs at `ARROW_GLYPH_PX` in `FONT_FAMILY_GLYPH`. The chart fills the tab's width from the row's bottom edge to the toggle row. The toggle row is one line of fourteen boxes in `CHART_OVERLAYS` order, left-aligned, with no stretch between them; each box is `TOGGLE_BOX_PX` square, under the caption line height, and its label is at `TYPE_CAPTION` in the overlay's colour. The tab draws no legend band. The ATA-SMP picture draws the value field at the lower-left corner of the price pane. Nothing else moves.

`src/gui/main_tabs/native_chart_surface.py` - the numbers both builds read

```
CONTROL_HEIGHT_PX = 26
ARROW_GLYPH_PX = 16
ARROW_GLYPH_FAMILY = ds.FONT_FAMILY_GLYPH
CONTROL_FONT_FAMILY = ds.FONT_FAMILY_UI
CONTROL_FONT_PX = ds.TYPE_SMALL
CAPTION_PX = ds.TYPE_CAPTION
TOGGLE_BOX_PX = 12
```

Who lays the row

The Qt tab lays one `QHBoxLayout`. `ChartPanel` builds the `TF:` label, the timeframe menu, the two legend labels and the source label as before, and the tab places them in its row through `timeframe_widgets` and `legend_widgets`. The panel keeps the chart and the toggle row.

`src/gui/widgets/trade_charts_tab.py` - the panel's controls join the row

```
            for widget in self._panel.timeframe_widgets():
                widget.setFixedHeight(CONTROL_HEIGHT_PX)
                control_row.addWidget(widget)
            control_row.addStretch()
            for widget in self._panel.legend_widgets():
                widget.setFixedHeight(CONTROL_HEIGHT_PX)
                control_row.addWidget(widget)
            layout.addLayout(control_row)
```

The React page draws the same row in `ControlRow`, from the sizes and the families `selector_values` carries, and the surface's `LAYOUT_SLOTS` reads three: the control row, the chart and the toggle row. The page no longer draws a header line of its own over the image; the painter's header line inside the image is the one both builds show.

The arrows draw their glyphs

The arrows carry U+25C0 and U+25B6. In the UI family those glyphs are 6 by 7 pixels at 12 pixels, a mark under a quarter of the button's height, which reads as an empty button. The buttons now set `FONT_FAMILY_GLYPH` at `ARROW_GLYPH_PX`, and the glyph reads 12 by 13 pixels in a 34 by 26 button.

`src/gui/design_system.py` - the family for symbol glyphs

```
FONT_FAMILY_GLYPH = "'Segoe UI Symbol', 'DejaVu Sans', 'Apple Symbols', sans-serif"
```

The toggle boxes

Each Qt box takes its own style sheet: the label in the overlay's colour at the caption size, the indicator `TOGGLE_BOX_PX` square with a one-pixel border. The page sizes each box's input from the same number, carried as `toggle_box_px` in the panel chrome.

`src/gui/native_chart.py` - one box's style sheet

```
TOGGLE_STYLE_FORMAT = (
    "QCheckBox {{ color: {color}; font-size: {font_px}px; spacing: {gap}px; }}"
    "QCheckBox::indicator {{ width: {box}px; height: {box}px; "
    "border-width: 1px; border-radius: 2px; }}"
)
```

The value field on the ATA-SMP picture

The legend band under the OHLC row is gone from every surface: the OHLC row sits directly over the price pane and the chart's natural height loses the band's rows. The values now draw as a field at the lower-left corner of the price pane, only on a chart that carries a call. Every venue PNG and the tab's ATA-SMP list call `set_call` with a direction; the Live list never does, so the tab shows no field. The field holds one caption-size line per overlay that is on, the label in the overlay's colour and the value in the axis text colour, on the theme's `chart_bg_top` at alpha 205 so the candles under it stay faint, inside a one-pixel border in `chart_grid`, `FIELD_PAD_PX` in from the pane's left edge and bottom edge. Lines past the pane's room are not drawn.

`src/gui/native_chart.py` - what makes a chart the ATA-SMP picture

```
        def draws_value_field(self) -> bool:
            """True when the chart carries a call, which makes it the ATA-SMP picture.

            Every venue PNG and the tab's ATA-SMP list call ``set_call`` with a
            direction; the Live list never does, so the tab draws no field.
            """
            return CALL_DIRECTION_ROLES.get(self._call_direction) is not None
```

A market on both lists keeps one symbol and draws two pictures, the bot's and the call's, so a list press now clears the followed symbol before the chart follows the shown market. Before this, a market that was both traded and called kept the bot's picture on the ATA-SMP list.

Read off the running program, both builds

Both builds built the real `MainWindow` with the home on scratch, every socket but loopback refused, one A15/USD bot off a scratch copy of `bot_state.json`, 100 A15 1h candles rolled from a copy of the 5m tablet, at the display's ratio of 1.25, the window 1920, 1400 and 900 pixels wide, each under `python -m pdb` with a breakpoint on the first statement of `_draw_legend`.

reading	before, Qt / React	after, Qt / React
distinct y of the controls	8, 42, 78 / 12, 50, 115	8 / 12
the chart's top edge against the controls' bottom	108 against 104 / 136.4 against 134.4	42 against 34 / 50.8 against 38
the arrow glyph	Segoe UI 12 px, 59 pixels of ink in the grabbed button / the browser's fallback	Segoe UI Symbol 16 px, 138 pixels of ink / the same family and size on the page
the box	indicator 16 by 16 in a 24 px box, label 9 px / input 13 by 13, label 9 px	indicator 12 by 12 in a 14 px box, label 10 px / input 12 by 12, label 10 px; the caption line height 12
`_draw_legend` on the Live tab, the presses and ten repaints	19 hits / 14 to 16 hits	0 hits / 0 hits
the natural chart height at 1886 wide	764 / 764	732 / 732
the field on one planted call, on the tab	none	191 by 51, three lines, at the pane's lower-left corner in both builds; gone again on the Live list
the field on the six venue sizes	none	191 by 51 with three voters, 191 by 181 with thirteen overlays on, inside the price pane on every size

With the row's spacing planted to zero in the driver's process only, the gaps between the controls read 0 in both builds against 8 unplanted, so the reading can fail.

Figures. `artifacts/u55/C6/` holds the tab, the arrow buttons, one box, the window and the page at each width before and after, the six venue PNGs with three voters and with all twelve, and the tab on the planted call.

Five sentences the frame overtakes

"The toggle is a button on its own row, above the blue arrows and the ticker menu." - the toggle is now the first control of the one row.

"A band under the OHLC row names every overlay with the last candle's value of its series, in the overlay's colour. An overlay that is off reads `off` in grey. The band wraps to the chart's width. It is painted by the painter, so the Qt widget, the React image and the ATA-SPM post images all carry it." - no surface carries the band; the ATA-SMP picture carries the field, and an overlay that is off has no line in it.

"The legend wraps at the chart's width as before; at 1400, 1366 and 1200 pixels it takes two rows and its widest row ends before the price scale." - the field does not wrap; each value is one line.

"The React page draws the tab header line and the painter draws its own header line under it, so the label reads twice on that build." - the page draws no header line of its own.

"The header line, the toolbar and the toggle row each take a rule between them." - the page has no toolbar and no header line; the toggle row keeps its rule.

2026-09-20 06:10 - #55 - the trade events from the History; the floors gone

His words, 2026-09-20: *"Dotted yellow lines for prior orders are supposed to be gone. Supposed to have hermetic symbols for trade events based on the History."* The chart draws no tranche floor on any surface, and every trade event of the shown market is drawn as its hermetic glyph from the History tab's rows, in both builds.

The rule

No dashed floor line, no `FLOOR` tag and no `N FLOORS` tag on the Qt widget, the React page's image or any venue PNG. For each row the History tab holds for the shown symbol, one glyph: a sell as `dissolve`, a buy as `reform`, on the candle whose interval holds the row's stamp, tagged with the role and the price, `SCRUM` for a sell and `FOLD` for a buy. A row outside the window is counted and not drawn. The rows are the venue's own fills, and the History tab is the one place that fetches them; the Charts tab never calls a venue. A fill the bus carries after launch draws at once and is not doubled when the History's next fetch carries it. The ATA-SMP picture and every venue PNG carry no glyph.

Where the trade events come from

The History tab holds the venue's fills in `_all_trades`, the time of its last fetch in `_last_fetched_ts` and whether a fetch is in flight in `_fetch_in_flight`; see the History tab. The window builds it after the Charts tab, so the Charts tab reads it at each `update_charts` tick under the name the window gives it, `_history_tab`. One surface reader takes the three names, one reader turns the rows of the shown symbol into fills, and one join keeps each bus fill the venue has not answered for.

`src/gui/main_tabs/trade_charts_tab_surface.py` - the rows to fills

```
def history_fills(rows: Any, symbol: Any) -> list:
    """One fill per History row of ``symbol``: the venue's id, side, price and stamp, the role from the side.

    A row whose side is neither buy nor sell, or whose price or stamp is
    not positive, gives no fill.
    """
```

`src/gui/main_tabs/trade_charts_tab_surface.py` - the join

```
def merged_fills(venue: Any, logged: Any) -> list:
    """``venue`` fills, then each ``logged`` fill no venue fill names.

    A venue fill names a logged fill when both carry the same venue id, or
    when their prices sit within ``FILL_MATCH_PRICE`` and their stamps
    within ``FILL_MATCH_WINDOW_S`` of each other.
    """
```

The bus fill carries no venue id today, so every join is by stamp and price. A fill the venue answers as several rows at several prices keeps its bus glyph beside them until the next launch.

The one ask, and the landing

When the History has never fetched and no fetch is in flight, the Charts tab asks `refresh()` once in its life, the same call the window makes when the History tab is activated. The Charts tab also connects the History's `history_refreshed` signal once, so the rows landing redraw the shown market without waiting for the two-second tick. The window's own rule keeps the History fresh after that.

`src/gui/main_tabs/trade_charts_tab_surface.py` - the one time the tab asks

```
def history_never_fetched(state: dict) -> bool:
    """True when the History has never landed a fetch and none is in flight: the one time the Charts tab asks.
"""
    return state["fetched_ts"] == NEVER_FETCHED_TS and not state["in_flight"]
```

`src/gui/widgets/trade_charts_tab.py` - the Qt tab's draw

```
        def _draw_history(self) -> None:
            """Put the shown market's trade events on the chart: the History's fills
            for its symbol, then each bus fill since launch the venue has not
            answered for.

            A History that has never fetched is asked to ``refresh`` once, the
            call the window makes on activation; ``history_refreshed`` is
            connected once so the landing redraws without waiting for the tick.
            A called market on the ATA-SMP list gets none.
            """
```

The React widget hands the same three names to its model through `set_history`, and the model's `feed_history` runs in `update_charts` before the bot's own annotations. Both builds read the same glyph set on the same candles from one painter.

The floors gone

`ChartPainter` no longer holds `_tranche_floors`, `set_tranche_floors` or `_draw_floors`; `FLOOR_LINE` is no longer a palette role; the two floor tag formats are gone. The Qt tab no longer reads the bot's lots, the React model no longer carries floors, and the page no longer stamps a floor count. The annotation emitter `charts.annotations.drawn` names fills, target lines, the strip, the glow and positions, and no floors; it now also fires when the fed count or the off-window count moves, so a row landing outside the window writes a row. The standing scrums off the bot's fold tranches are no longer a source of markers: the History's rows are the venue's answer for the same sells.

The History readings, off the running program, both builds

Both builds, the real window, the home on a scratch directory, every socket but loopback refused, a two-bot fleet off a scratch copy of one seed record, 100 recorded A15 1h candles rolled from the 5m tablet by the registry's own rollup and shifted by whole hours so the newest candle opens at the current hour, and a History venue on loopback answering `get_my_trades` with 40 A15/USD rows (26 on distinct candles inside the window, 14 older than it) and 6 A14/USD rows, through the History tab's own fetch path:

reading	Qt	React
`_draw_floors` and `set_tranche_floors` on the painter	absent; 0 floors held; the emitter names no floor	the same

reading	Qt	React
the History at the tab's first tick	never fetched; the tab asks once; the venue is asked once per market, 2 of 2; 46 rows land in 0.21 s	the same
the shown market's rows against its glyphs	40 rows for A15/USD, 40 markers held, 26 drawn on 26 candles, 14 counted off the window	the same 40, 26 and 14
a six-notch zoom to 36 candles, then a reset	5 drawn and 35 off; back to 26 and 14	the same
one bus fill on the newest candle	drawn at once, 27 of 41; the History's next fetch carrying it leaves 27 of 41	the same
a row planted inside the window, then one outside	28 drawn of 42; then 28 drawn of 43 with 15 off	the same
A14/USD shown, then A15/USD again	6 rows, 4 drawn, 2 off; back to 28 of 43	the same
the History tab after these fetches	49 of 49 trades shown, page 1 of 1	the same
the venue image with a call	no glyph, no floor	one painter

Figures. `artifacts/u55/C7/` holds the tab, the chart and the window at 1920 and 1400 in both builds, before and after.

Six sentences the History glyphs overtake

They were not reworded. They are quoted here.

"`set_tranche_floors` | The standing fold-tranche floors" - the setter table row; the setter no longer exists and nothing places a floor.

"The fills come from two figures of the bot. Each tab subscribes to the bus topic every fill crosses and records it, so a fill made after launch draws on the next status tick. And each standing fold tranche the bot holds was made at a scrum: its reference price is that sell and its stamp is that sell's time, so the tab draws each one as a scrum glyph." - the fills come from the History tab's rows for the shown symbol, then the bus fills since launch the venue has not answered for; the fold tranches are no longer a source.

"Each tranche floor keeps its dashed line; floors whose tags would overlap share one tag naming their count and their price span, so 16 floors in one pane read as seven tags." - no floor draws on any surface.

"`charts.annotations.drawn` is written when the set of annotations the paint pass drew changes, with the counts drawn against the counts fed that fall inside the pane, and the fills fed and the fills off the window in its context." - it is also written when the fills fed or the fills off the window change, and it names no floor.

"`55 floors off the record | 55 lines fed, 16 inside the pane, 7 tags | the same`"-a reading of the floors, which no longer draw.

"An ATA-SMP market has no bot behind it, so the chart draws its candles without the trade markers, the target lines, the tranche floors and the fire glow that a traded asset carries." - the sentence holds; there are no tranche floors on any market now.

2026-09-20 09:45 - #55 - the oscillators: one readable sub-pane height, whole-pixel pens, tags that never cross

His words, 2026-09-20: *"Oscillators are not being drawn at the highest quality and are too compressed vertically to be read clearly."* And: *"it does not appear that the GUI Archetype organized all of the elements properly using any kind of rules."* Three rules now govern the seven sub-panes, the pens and the fill tags, and each is proved on the running program in both builds. The GUI Archetype carries no layout or draw-quality rule, so the rules are stated here and read off the widget and the page.

The height rule

Every sub-pane on the Charts tab draws at one readable height. The figure is derived from what a sub-pane holds: a label row one tag tall over a plot of five bands one tag tall each - room over the upper ruled level, the upper level, the value tag at the middle, the lower level, and room under the lower level. A tag is 14 px, so a sub-pane is 14 + 5 x 14 = 84 px. The lines, the histogram and the ruled levels plot under the label row and never through it. The value tag sits at the plot's middle. The price pane never falls under 220 px.

`src/gui/main_tabs/native_chart_surface.py` - the figure beside the pane numbers

```
TAG_HEIGHT_PX = 14
#: A sub-pane's label row, one tag tall, over its plot.
SUB_PANE_LABEL_PX = TAG_HEIGHT_PX
#: The tag-tall bands a sub-pane's plot holds: over the upper ruled level, the
#: upper level, the value tag at the middle, the lower level, under the lower level.
SUB_PANE_PLOT_BANDS = 5
#: The height every sub-pane draws at on the Charts tab: the label row over
#: the plot's bands. A host shorter than the panes' height scrolls.
SUB_PANE_READABLE_PX = SUB_PANE_LABEL_PX + SUB_PANE_PLOT_BANDS * TAG_HEIGHT_PX
#: The least height a sub-pane folds to on a fixed-height venue image.
SUB_PANE_FOLD_PX = 28
```

The chart's height is the natural height of the panes on. With the volume strip and the seven sub-panes it is 28 + 18 + 220 + 28 + 7 x 84 + 18 = 900 px. A panel taller than that gives the rest to the price pane. A panel shorter than that keeps the chart at 900 and scrolls it.

`src/gui/native_chart.py` - the widget's height

```
        def _apply_height_for_panes(self) -> None:
            """Set the minimum height to ``_natural_height_for_panes`` at the widget's width.

            ``_height_override`` from a grip drag wins when it is taller. The
            scroll area holding the chart scrolls when its viewport is shorter
            than this height; a taller viewport's extra height goes to the price pane.
            """
            target = self._natural_height_for_panes(self.width())
```

The scroll hosts

On the Qt panel the chart sits in a scroll area with a vertical bar as needed and no horizontal bar. A wheel over the chart zooms the window, as before, and never scrolls the area, because the chart takes the wheel. The bar scrolls. The toggle row sits under the scroll area at the tab's foot.

`src/gui/native_chart.py` - the scroll area in the panel

```
            self._scroll = QScrollArea(self)
            self._scroll.setAccessibleName(CHART_SCROLL_NAME)
            self._scroll.setWidgetResizable(True)
            self._scroll.setFrameShape(QFrame.NoFrame)
            self._scroll.setHorizontalScrollBarPolicy(Qt.ScrollBarAlwaysOff)
            self._scroll.setVerticalScrollBarPolicy(Qt.ScrollBarAsNeeded)
            self._scroll.setWidget(self._chart)
            layout.addWidget(self._scroll)
```

On the page the chart mount scrolls the same way. The image host inside it is never under the natural height, so a shorter mount scrolls and the toggle row under the mount stays at the panel's foot. The wheel listener on the host already takes the wheel for the zoom.

`src/gui/web/trade_charts_tab.js` - the mount and the host

```
      style: {
        flex: AUTO,
        minHeight: ZERO_PX,
        overflowX: HIDDEN,
        overflowY: AUTO,
        display: FLEX,
        flexDirection: COLUMN
      }
```

```
    into.style.minHeight = height(
      answer[IMAGE_NATURAL_HEIGHT] === undefined
        ? answer[IMAGE_HEIGHT]
        : answer[IMAGE_NATURAL_HEIGHT]
    );
```

The image the page asks for is never under the natural height either.

`src/gui/react_charts_tab.py` - the image's height

```
            natural = int(self._painter._natural_height_for_panes(width))
            minimum = int(self._painter._minimum_height_for_panes(width))
            height = max(natural, asked_height)
```

A fixed-height venue image keeps the fold of 2026-09-19: each sub-pane shrinks from the readable height toward 28 px, then the reading strip folds, then the price pane shrinks toward 120 px, and a height under the least layout is refused by name.

The quality rule

Every stroke pen's width is its logical width scaled by the device pixel ratio to whole device pixels, never under one. The five logical widths 0.8, 1.0, 1.2, 1.4 and 1.5 draw 1, 1, 1, 1 and 2 device pixels at ratio 1.0 and 1, 1, 2, 2 and 2 at 1.25. Lines draw with anti-aliasing on. Axis rules - the grid, the pane separators, the ruled levels, the reference lines and the MACD zero line - draw with anti-aliasing off, one device pixel wide, on coordinates snapped to the centre of a device pixel. Bars - the volume bars and the MACD histogram - draw with anti-aliasing off on whole device pixels. Text stays at the caption size in the mono family.

`src/gui/main_tabs/native_chart_surface.py` - the pen width

```
def device_pen_width(width_px: float, ratio: float) -> float:
    """``width_px`` logical pixels scaled by ``ratio`` to whole device pixels, never
    under one, given back in logical pixels."""
    scale = float(ratio) if float(ratio) > 0 else 1.0
    return max(1, round(float(width_px) * scale)) / scale
```

`src/gui/native_chart.py` - one ruled level

```
                def _rule_line(y: float, colour=None, dashed: bool = True) -> None:
                    """One ruled level across the pane at ``y``: one device pixel, aliased, snapped."""
                    pen = QPen(colour if colour is not None else self.GRID_MINOR, px)
                    if dashed:
                        pen.setStyle(Qt.DashLine)
                    p.setRenderHint(QPainter.Antialiasing, False)
                    p.setPen(pen)
                    p.drawLine(QPointF(ML, snap(y)), QPointF(w - MR, snap(y)))
                    p.setRenderHint(QPainter.Antialiasing, True)
```

The tag rule and the readout

Every glyph draws first. A fill tag is then placed at the right of its glyph, or at the left when the right would cross the price scale. A tag that would cross a glyph or a tag already drawn is not drawn. The glyph count does not move. Merging two tags into one naming a count was not chosen, because a count names no price and no role.

`src/gui/native_chart.py` - where a tag goes

```
        def _tag_place(
            self,
            right_x: float,
            left_x: float,
            y: float,
            width: float,
            drawn: list,
            scale_x: float,
        ) -> Optional[QRectF]:
            """The rect a fill tag draws in: ``TAG_H_PX`` tall at ``right_x`` when
            ``_tag_clear``, else at ``left_x`` when clear, else None and no tag."""
            for x in (right_x, left_x):
                rect = QRectF(x, y, width, TAG_H_PX)
                if self._tag_clear(rect, drawn, scale_x):
                    return rect
            return None
```

The crosshair's readout names every fill on the candle under the pointer. One row per fill follows the six candle rows: its label is the side's glyph character, a sell as a filled triangle pointing down and a buy as one pointing up, and its text is the tag's text in the marker's colour. The painter's own tip and the page's canvas tip both draw the rows, because the page reads them off the geometry the image answer carries.

`src/gui/main_tabs/native_chart_surface.py` - one readout row for a fill

```
def fill_row(fill: FillLike) -> list[str]:
    """One readout row for ``fill``: the side's glyph, the tag's text and its role."""
    is_buy = str(fill.side).lower() == FILL_BUY_SIDE
    return [
        READOUT_BUY_GLYPH if is_buy else READOUT_SELL_GLYPH,
        FILL_TAG_FORMAT.format(label=fill_label(fill), price=fmt_price(fill.price)),
        fill_role(fill),
    ]
```

The oscillator readings, off the running program, both builds

Each build ran the real window with the home on a scratch directory, every socket but loopback refused, two bots off a scratch copy of one record, 100 A15 1h candles and a loopback History venue answering 40 fills, 26 inside the window. The chart's host was resized to 690, 900 and 1100 px. Each run was driven under the debugger with a breakpoint on the paint pass, ignored on every crossing and counted at the end.

reading	before, Qt / React	after, Qt / React
seven sub-panes at a 690 px host	54.0 px each, the price pane 220, no scroll	84.0 px each, the price pane 220, the chart 900 tall, the bar's range 210
at 900	60.0 each, 388	84.0 each, 220, range 0
at 1100	60.0 each, 588	84.0 each, 420, range 0
the toggle row	under the chart	under the scroll host at the tab's foot: y 738 on Qt, 740.8 on the page, at 690
the wheel over the chart	zooms 100 to 85 candles	zooms 100 to 85; the bar stays at 0

reading	before, Qt / React	after, Qt / React
the stroke pens in device pixels, ratio 1.0	0.8, 1.0, 1.2, 1.4, 1.5, 1.7	1, 2
ratio 1.25	1.0, 1.25, 1.5, 1.75, 1.875, 2.12	1, 2
the fills at 100 candles	26 glyphs, 26 tags, 24 overlapping pairs	26 glyphs, 15 tags, 0 overlapping pairs
at 36 candles	5 glyphs, 5 tags, 0 pairs	5 glyphs, 4 tags, 0 pairs
the squat venue, Facebook 1200 by 630, eight voters	seven sub-panes at 40 px, the strip folded, the price pane 226	the same
the readable figure planted to 20	-	seven sub-panes at 20.0, the chart 452 tall; 84.0 restored
the tag rule planted off	-	26 tags, 24 overlapping pairs; 15 and 0 restored

The crops at 1.25, four times enlarged: before, the RSI line smeared over two to three device rows and the 30 and 70 rules over two faint rows; after, the line two device pixels and each rule one crisp dashed row, the label in its own row over the plot, the value tag legible. Renders sit under `artifacts/u55/C8/` before and after.

Four sentences the oscillators overtake

They were not reworded. They are quoted here.

"Each takes 60 px at the chart's natural height. When the window gives less, every sub-pane shrinks alike, down to 28 px, so the price pane keeps its 220 px and every pane stays on screen." - each takes 84 px on the tab at every window height, and the panel scrolls when the window gives less; only a fixed-height venue image shrinks a sub-pane, toward 28 px.

"`SUB_PANE_H = 60`" and "`SUB_PANE_MIN_H = 28`" - the two names are gone; the painter reads the surface's readable figure and its fold floor.

"The wick is a share of one candle column and never under one device pixel; the candle border is one device pixel; the indicator lines keep their logical width and stay anti-aliased. Read on a 1200 pixel image at ratio 1.0 the pens are 0.8, 1.0, 1.2, 1.4, 1.5 and 1.56 logical pixels; at ratio 2.0 the set gains 0.5, the one-device-pixel pen." - every stroke pen is now a whole number of device pixels; at ratio 1.0 the set is 1 and 2, at 1.25 it is 0.8 and 1.6 logical, which are 1 and 2 device pixels.

"The glyph sits on the candle whose interval holds the fill's stamp, one candle column wide and a twentieth of the price pane tall, with a tag naming the role and the price beside it." - the glyph holds; its tag draws only where it crosses no glyph and no other tag, and the readout names the fill under the pointer.

2026-09-23 17:30 - #55 - the oscillators are a third taller, and the bottom bar drags both heights

His words, 2026-09-23: *"Oscillator default height should be increased by 30% - Oscillator and chart height needs to be adjustable by dragging the bottom bar. Said adjustments must move all subsequent elements down and not overlap them"*.

The default height

A sub-pane's plot now holds seven tag-tall bands, not five: two over the upper ruled level, the upper level, the value tag at the middle, the lower level, and two under it. A tag is 14 px, so a sub-pane is 14 + 7 x 14 = 112 px. Read off the running chart before the change it was 84.0 px. His figure, 84 x 1.3, is 109.2 px, which is not a whole number of bands; seven bands is the least count that reaches it, and it gives the plot the same room above the upper level as below the lower one. The rise is 33.3%.

`src/gui/main_tabs/native_chart_surface.py` - the band count

```
#: The tag-tall bands a sub-pane's plot holds: two over the upper ruled level,
#: the upper level, the value tag at the middle, the lower level, two under the
#: lower level. Seven is the least count whose height clears 84 px by the 30%
#: the operator asked for: 14 + 7 x 14 = 112, a rise of 33.3%.
SUB_PANE_PLOT_BANDS = 7
```

The chart's natural height follows. With the volume strip and the seven sub-panes it is 28 + 18 + 220 + 28 + 7 x 112 + 18 = 1096 px, where it was 900. A panel shorter than that scrolls to it.

The bottom bar

Three short dashes draw across the chart's bottom edge, in a strip 8 px tall. Pressing in that strip and dragging down sets the chart's height, and every sub-pane keeps the share of the chart it holds at the natural height. One drag therefore raises the price pane and every oscillator together. Dragged up, the height stops at the natural 1096 and the sub-panes return to 112 px. The pointer turns to the vertical resize arrows over the strip on the Qt widget.

`src/gui/main_tabs/native_chart_surface.py` - the share the drag gives a sub-pane

```
def sub_pane_height(dragged_height_px: int, readable_height_px: int) -> int:
    scaled = SUB_PANE_READABLE_PX * int(dragged_height_px) // int(readable_height_px)
    return max(SUB_PANE_READABLE_PX, scaled)
```

`CandlestickChart.mouseMoveEvent` drives it on the widget, and `chart_view` drives it on the page from the same press, drag and release the page already sends. Both call `set_pane_drag_height`, so both builds answer one grab of the bar the same way.

Nothing below the drag overlaps

Inside the chart the panes are laid one under the next: each pane's top is the pane above's bottom, so a drag moves every pane under it down by the height it won. Outside the chart the panes sit in a scroll host, with the toggle row under the host, so the host's own height never changes and the toggle row never moves. Read on the running program at 700, 900 and 1400 px wide:

drag	sub-pane	price pane	vortex	rsi	time axis	chart bottom
at rest	112.0	46.0 - 266.0	294.0 - 406.0	966.0 - 1078.0	1078.0 - 1096.0	1096
+260	138.0	46.0 - 344.0	372.0 - 510.0	1200.0 - 1338.0	1338.0 - 1356.0	1356
+4000	520.0	46.0 - 1410.0	1438.0 - 1958.0	4558.0 - 5078.0	5078.0 - 5096.0	5096
dragged up	112.0	46.0 - 266.0	294.0 - 406.0	966.0 - 1078.0	1078.0 - 1096.0	1096

On the Qt panel at 700 px wide the scroll host reads 0.0 - 668.0 and the toggle row 674.0 - 686.0 at every one of those positions; at 900 the host reads 0.0 - 878.0 and the row 884.0 - 896.0; at 1400 the host reads 0.0 - 1078.0 and the row 1084.0 - 1096.0. On the page at 700 the chart mount reads 50.8 - 656.8 and the toggle row 660.6 - 674.6 at every position, and the image inside the mount grows from 1146.8 to 4002.8 while the mount scrolls.

The height and drag readings, off the running program, both builds

Each build ran with the home on a scratch directory and every socket but loopback refused, 100 A15 1h candles, seven sub-panes on, the panel at 700, 900 and 1400 px wide. The Qt numbers come from `ChartPainter.geometry_payload`, which reports what the last paint laid out; the page's numbers come from the same payload carried on the image answer, and its element rectangles from the page's own `getBoundingClientRect`.

reading	before, Qt / React	after, Qt / React
each sub-pane at rest, every width	84.0 / 84.0	112.0 / 112.0
the chart's natural height	900 / 900	1096 / 1096
the bar dragged +260	84.0, the chart 1160 / no answer, the image 900	138.0, the chart 1356 / 138.0, the image 1356
the bar dragged +4000	84.0, the chart 4900 / no answer, the image 900	520.0, the chart 5096 / 520.0, the image 5096
the bar dragged up	84.0, the chart 900 / 900	112.0, the chart 1096 / 1096
the panes' tops and bottoms, every position	each top the one above's bottom	the same
the toggle row against the scroll host	674.0 under 668.0 / 660.6 under 656.8	the same at every drag position
the band count planted back to 5	-	84.0 at rest and the natural height 900, in both builds; 112.0 restored

Renders sit under `artifacts/u55/C9/`, one per width and drag position in each build.

Three sentences the drag overtakes

They were not reworded. They are quoted here.

"A tag is 14 px, so a sub-pane is 14 + 5 x 14 = 84 px." - a sub-pane's plot holds seven bands, so a sub-pane is 14 + 7 x 14 = 112 px.

"With the volume strip and the seven sub-panes it is 28 + 18 + 220 + 28 + 7 x 84 + 18 = 900 px." - it is 28 + 18 + 220 + 28 + 7 x 112 + 18 = 1096 px.

"each takes 84 px on the tab at every window height" - each takes 112 px at every window height, and a drag of the bottom bar raises that figure with the chart.

2026-09-23 22:10 - #55 - the trade markers draw on every timeframe

His words, 2026-09-23: *"Historical trade markers ... do not consistently appear and I could only get them to do so on the weekly timeframe during the previous test."* The chart now draws a fill's glyph at 5m, 1h, 1d and 1w, and where a timeframe cannot reach a fill it says so in the header.

The window rule and what each timeframe reaches

One fetch asks for `FETCH_LIMIT` candles, and a fill can only be drawn on a candle the fetch brought back. The window a chart draws is therefore the candle count times the timeframe's own length, and a fill older than the first candle is counted, named in the header, and not drawn.

`FETCH_LIMIT` is 300. One request carries at most 300 candles on the Coinbase candles route, which the tree records as `RA_CHUNK_DAYS`, and the Advanced Trade route's own ceiling is 350. The lower of the two is the figure the chart asks for, so no fetch asks a route for more than it serves. The fetch is one request either way, because the count is a parameter of that one call.

`src/gui/main_tabs/trade_charts_tab_surface.py` - the one declaration

```
#: Candles one fetch asks for, and so the span a fill's glyph can land in;
#: 300 is the Coinbase candles route's one-request ceiling, recorded as
#: ``RA_CHUNK_DAYS``. ``src.gui.widgets.trade_charts_tab`` imports this name.
FETCH_LIMIT = 300
```

What 300 candles reach, from `TIMEFRAME_SECONDS`:

timeframe	window span	reaches a fill this old
5m	1.042 days	one day
1h	12.500 days	twelve and a half days
1d	300.000 days	ten months
1w	2100.000 days	five years nine months

A five-minute chart cannot reach a fill five months old. Reaching 152 days at 5m needs 44,011 candles, which is 147 requests and 0.014 px a candle in the pane; at 1h it needs 3,668 candles and 12 requests. The daily and the weekly reach every fill a bot of this fleet holds, and the two short timeframes draw what they reach and name the rest.

One declaration of the candle count

`FETCH_LIMIT` was declared in two files with the same value. Nothing compares the two, so the two copies could disagree without anything reporting it. The surface declares the name now, and the Qt tab imports it.

`src/gui/widgets/trade_charts_tab.py` - the import

```
from ..main_tabs.trade_charts_tab_surface import (
    FETCH_LIMIT,
    bot_timeframe,
    fill_record,
```

The header names the fills it could not draw

The paint pass counts each fill it refused, under the reason it refused it: older than the first candle, past the last one, or at a price outside the drawn scale. The header carries one phrase per reason that counted anything, beside the symbol, the timeframe, the source and the candle count. A chart with nothing refused carries none of them, and its picture is unchanged.

`src/gui/native_chart.py` - the three phrases

```
FILLS_OLDER_HEADER_FORMAT = "{count} {fills} older than this chart"
FILLS_NEWER_HEADER_FORMAT = "{count} {fills} newer than this chart"
FILLS_OFF_SCALE_HEADER_FORMAT = "{count} {fills} off the price scale"
```

`charts.annotations.drawn` carries the same counts split by reason: `fills_older_than_window`, `fills_newer_than_window` and `fills_off_price_scale`, with `fills_off_window` the sum of the first two. It previously folded a fill at a price off the drawn scale into `fills_off_window`, which named the window for an absence the window did not cause.

The marker readings per timeframe

One market, 37 fills fed, read off the running program in both builds at 700, 900 and 1400 px wide. The candles come from a read-only copy of one five-minute stone tablet on disk, folded to each timeframe by the registry's own rollup; the fills carry the fill ages of the running fleet, read from a read-only copy of its state file. No venue was contacted, every socket but loopback was refused, and no bot was started and no order placed, priced or cancelled.

timeframe	candles before	candles after	window before	window after	drawn before	drawn after	header after
5m	100	300	0.347 days	1.042 days	**0**	**6**	31 fills older than this chart
1h	100	300	4.167 days	12.500 days	12	**18**	19 fills older than this chart
1d	100	213	100.0 days	213.0 days	31	**37**	none
1w	31	31	217.0 days	217.0 days	37	37	none

The daily and the weekly answered fewer candles than the 300 asked for, because the tablet on disk holds 212.771 days; a venue serving the whole 300 reaches 300 days at 1d.

Every marker sat on the candle holding its own stamp at every timeframe and every width, with no fault in any reading. One fill planted at 400 days, outside every window including the weekly's, raised every timeframe's refused count by exactly one and turned the daily and the weekly from no phrase to `1 fill older than this chart`.

Read off the saved renders rather than off the painter: at the weekly, before and after differ by **0 pixels** in both builds and in the image the page is handed, and the planted phrase differs by 720 pixels inside rows 14 to 22, the header's own band. At 700 px wide the longest header this draws ends 89 px clear of the right edge, so no phrase is clipped at the narrowest width.

A paint pass costs 40.58 ms at 300 candles against 27.64 ms at 100, at 700 px; 28.10 against 15.82 at 900; 30.51 against 19.43 at 1400. The tick that redraws the chart runs every two seconds.

Renders sit under `artifacts/u55/C11a/`, one per build, stage, width and timeframe.

Sentences the window rule overtakes

They were not reworded. They are quoted here.

"A fill whose stamp falls before the window or past its end is counted and not drawn." - it is counted, named in the header under the reason it was refused, and not drawn.

"A row outside the window is counted and not drawn." - a row outside the window is counted, named in the header, and not drawn.

"`charts.annotations.drawn` is written when the set of annotations the paint pass drew changes, with the counts drawn against the counts fed that fall inside the pane, and the fills fed and the fills off the window in its context." - the context now carries the fills fed, the fills off the window, and that figure split into the fills older than the window, the fills newer than it and the fills at a price off the drawn scale.

2026-10-16 - #230 #223 - one unreadable price costs one candle, and a stored flag never prints a price

The admission rule

Every price the chart reads goes through one admission step before anything is drawn with it. `as_finite_float` admits a value only when its exact type is int or float and the number is finite, so a flag, a word, a missing value, a not-a-number, an infinity and an integer too large to hold are all refused. A refused candle is left out of the series and counted. A refused price in the header is named rather than printed. Nothing refused is ever turned into a zero.

Where the admission sits

`src/gui/main_tabs/native_chart_surface.py`, in `admit_candles`

```
def admit_candles(candles: Sequence[Any]) -> tuple:
    kept = [one for one in candles if candle_prices(one) is not None]
    return kept, len(candles) - len(kept)
```

`candle_prices` reads a candle's open, high, low and close through `as_finite_float` and answers None when any one of them is refused. Both builds take their series through `admit_candles`: the Qt widget in `src/gui/native_chart.py`, in `ChartPainter.set_candles`, and the React page through `src/gui/main_tabs/native_chart_surface.py`, in `ChartModel.set_candles`. `price_range` admits again before it fits the axis, so a caller that reaches it directly cannot hand it a price it would refuse.

`price_range` answers None when no candle carries a readable high and low. The Qt widget then draws `NO_PRICE_RANGE_TEXT` through `_paint_no_range`, and `build_view_model` publishes the same phrase as the chart's empty message.

`row_prices` admits one venue row for the React page's feed in `src/gui/react_charts_tab.py`, in `_feed_painter`, so one unreadable row costs that row alone. A row too short to carry the four prices is refused with the rest. `_feed_painter` counts the rows it dropped and hands that count to `set_candles` as `refused`, which adds it to the candles `admit_candles` refuses again, so the count line names every candle missing from the picture. A series that arrives empty rewrites the count line rather than leaving the last one standing.

What the chart says about a candle it could not read

`candle_count_text` writes the count line. A series taken whole reads exactly as it always has.

`src/gui/main_tabs/native_chart_surface.py` - the three phrases

```
HEADER_COUNT_FORMAT = "{count} candles"
REFUSED_COUNT_FORMAT = "{count} candles, {refused} unreadable"
ALL_CANDLES_REFUSED_TEXT = "No candle carried a readable price"
```

`build_view_model` publishes the same figure as `refused_candles`, beside `candle_count`.

What the header says about a price it could not read

`src/gui/main_tabs/trade_charts_tab_surface.py`, in `header_label`, is the one header step both builds call: the Qt tab from `src/gui/widgets/trade_charts_tab.py`, in `_label_current`, and the React page from `_label_shown` in the surface. It answers three different things.

the saved price	the header
no price saved yet	the symbol alone, as it stands
a number `as_finite_float` admits	`{symbol} • ${price:.8f} • {state}`
anything `as_finite_float` refuses	`{symbol} • price unreadable • {state}`

A saved zero is a number, so it draws as `$0.00000000`. A price that was never saved draws the symbol alone. The two are no longer the same picture.

`PRICE_LABEL_FORMAT` was declared twice, once in the surface and once in `src/gui/widgets/trade_charts_tab.py`. The tab's copy is gone and both builds read the surface's.

The header step runs after the draw

`update_charts` places the header step below the candle draw in both builds: after `_draw_history` and `_decorate_current` in the Qt tab, and after `feed_history` and `_feed_shown` in the surface. A header the tab cannot write costs the operator nothing he can see.

The readings, off the running program, both builds

One chart built inside a shown parent, fed 30 candles, painted over a fixed 900 by 700 area. No venue was contacted, no bot was started, and no order was placed, priced, previewed or cancelled. The runtime tree was read with a redirected home.

A clean series: 42638 drawn pixels, `30 candles`, price axis 98.34 to 116.16. Before and after differ by **0 pixels** across all 630000 of them, and every published reading is unchanged.

One candle of thirty carrying an unreadable high:

the value in one candle's high	candles drawn before	candles drawn after	the chart before	the chart after
a text number	0	**29**	113 px of 42638	`29 candles, 1 unreadable`
a word	0	**29**	113 px of 42638	`29 candles, 1 unreadable`
a stored flag	30 at $1.00	**29**	a wrong scale, drawn	`29 candles, 1 unreadable`

the value in one candle's high	candles drawn before	candles drawn after	the chart before	the chart after
a missing price	0	**29**	113 px of 42638	`29 candles, 1 unreadable`
an integer too large to hold	0	**29**	113 px of 42638	`29 candles, 1 unreadable`
a not-a-number	0	**29**	895 px of 42638	`29 candles, 1 unreadable`
an infinity	0	**29**	113 px of 42638	`29 candles, 1 unreadable`
a zero	30	30	drawn	drawn, a readable zero
a negative	30	30	drawn	drawn, a readable negative

Three of thirty refused reads `27 candles, 3 unreadable`, and thirty of thirty reads `No candle carried a readable price`. The two builds answer the same kept series candle for candle at every count.

The saved price in the header, read in both builds:

the saved price	the header before	the header after
a text number	`$123.45000000` in Qt, the draw lost in React	`price unreadable`
a word	the draw lost in both	`price unreadable`
a stored flag	`$1.00000000` in both	`price unreadable`
a missing price	the symbol alone	the symbol alone
an integer too large to hold	the draw lost in both	`price unreadable`
a not-a-number	`$nan` in Qt, the symbol alone in React	`price unreadable`
an infinity	`$inf` in both	`price unreadable`
a zero	the symbol alone	`$0.00000000`
a readable price	`$101.25000000`	`$101.25000000`

The two builds disagreed on three of those kinds before this entry and agree on all of them after.

Sentences the admission rule overtakes

They were not reworded. They are quoted here.

"The tab is a scroll area holding one panel per active bot." - the tab holds one panel and one shown market, stepped between two lists by the arrows.

"One method rebuilds that panel set from the current roster." - `update_charts` rebuilds the two market lists and redraws the one panel.

2026-10-16 - #230 #223 - the other five stored prices admit before the chart draws them

What the candle admission did not reach

The candle series and the header price were admitted first. Five more stored prices still reached the paint, or the row builder, with a bare conversion. Each is read off the running program below.

the stored price	before	after
a position price that is a word, a missing value, a huge integer or a not-a-number	232 px of 42638 drawn	42638, chart intact
a target anchor or ceiling that is a word or a huge integer	7397 px of 42638 drawn	42638, chart intact
a marker price that is a text number, a not-a-number or an infinity	a marker drawn at a nonsense price	dropped
a fed candle whose field is a word, a missing value or a huge integer	**0 of 30** rows reached the chart	**29 of 30**
a synthetic candle with the same	the whole push refused	29 of 30, `29 candles, 1 unreadable`

A text number, a flag and a not-a-number in a fed candle built thirty rows before, carrying the wrong value into the picture.

Where each admission sits

`src/gui/native_chart.py`, in `set_positions`, keeps the positions whose price `as_finite_float` admits, so one refused position costs its own marker. `ChartModel.set_positions` in `src/gui/main_tabs/native_chart_surface.py` does the same for the React page.

`set_target_balance_lines` reads both prices through `as_finite_float` in both builds. A refused price becomes None, which is what None has always meant for that line: the line is hidden.

`set_trade_history_markers` in the widget and `set_markers` in the model read the marker price through `as_finite_float`. A marker without a readable price is dropped, as a marker without a positive price already was.

`src/gui/main_tabs/trade_charts_tab_surface.py`, in `candle_row`, answers None for a candle it cannot read, and `candle_rows` drops it and keeps the rest. The three feeds that built rows one by one call `candle_rows` now: the ATA-SMP call, the history feed and the synthetic push.

`src/gui/main_tabs/native_chart_surface.py`, in `synthetic_candle`, reads a candle by attribute or by key, so a candle built without a fetch is admitted the same way a fetched one is. `push_synthetic_candles` in `src/gui/widgets/trade_charts_tab.py` counts what it dropped and hands that count to `set_candles`, so the count line names it.

`CANDLE_FIELDS` and `CANDLE_PRICE_FIELDS` name the six fields and the four prices once, so the two builds cannot disagree on which fields are read.

The readings

One chart built inside a shown parent, fed thirty candles, painted over a fixed 900 by 700 area. A clean chart draws 42638 pixels; the reading's control reports 0 for a planted empty chart.

A clean series still draws identically: **0 pixels differ** across all 630000 of them against the tree before this work began, and every published reading is unchanged.

No venue was contacted, no bot was started, and no order was placed, priced, previewed or cancelled. The runtime tree was read with a redirected home.

Sentences the five admissions overtake

They were not reworded. They are quoted here.

"One candle as six numbers, from an `OHLCVCandle`, a dict or a six-number row." - `candle_row` answers six numbers for a candle it can read, and None for one it cannot, and `candle_rows` keeps those it could.

"Store the grid levels. Nothing paints them." - still true. The grid level list is written and read zero times, and `GridLine` is constructed zero times, in both builds. Layer 2 of #230 is unbuilt.

Layer 2 is built now, and the setter that sentence described is gone. The section below quotes what went with it.

The zero a bot carries before its first fetch

`src/trading/container/config.py` declares `current_price: float = 0.0`, so a bot that has not fetched a price yet reports zero rather than nothing. Printing that as a dollar figure would hand the operator a price he would believe.

`header_label` answers four different things, and the two builds answer the same.

the saved price	the header
no price saved at all	the symbol alone
a zero	`{symbol} • no price yet • {state}`
anything `as_finite_float` refuses, or a negative	`{symbol} • price unreadable • {state}`
a readable price above zero	`{symbol} • ${price:.8f} • {state}`

A negative reads as unreadable rather than as a price, because a market price below zero is not a price the chart can place.

Two sentences the four header answers overtake

They were not reworded. They are quoted here.

"A saved zero is a number, so it draws as `$0.00000000`." - a saved zero names `PRICE_NOT_YET_NOTE`, because the zero a bot carries before its first fetch is not a price it measured.

"| a readable zero | `$0.00000000` |" - a readable zero reads `no price yet`.

2026-10-19 - #230 - the header names the candle it left out, and the dead bullseye list goes

What the count line reached, and what the operator saw

`candle_count_text` wrote the refusal into the chart's status text, and the status text is only drawn when the chart holds no candles at all. A series with twenty-nine readable candles and one unreadable one therefore drew the plain count and said nothing about the candle missing from the picture. The refusal was counted, published and never painted.

`header_parts` is now the one step that writes the count piece, and it writes it through `candle_count_text`.

`src/gui/main_tabs/native_chart_surface.py`, in `header_parts`

```
  if candle_count:
      parts.append(candle_count_text(candle_count, refused_count))
```

`build_view_model` hands it `model.refused_candles`, and `src/gui/native_chart.py`, in `ChartPainter._draw_header`, calls `header_parts` instead of building the same list from its own format strings. The Qt header and the React page's `header_text` cannot drift apart on the count, because one function writes it for both.

What the widget says, read off the pixels

One chart rendered through its own `paintEvent` at 1000 by 630, offscreen, with the home directory redirected. Each figure below is the count of pixels that differ from the background gradient `paint_to` fills first. The drawn string is read by matching a candidate's own render against the header band's pixels, so the reading comes from the picture and not from the source.

the reading	before	after
a clean series of thirty readable candles	801085 px, `30 candles`	801085 px, `30 candles`
one candle's high is a text value	801904 px, `29 candles`	802148 px, `29 candles, 1 unreadable`
one candle's high is a missing value	801904 px, `29 candles`	802148 px, `29 candles, 1 unreadable`
one candle's high is a stored flag	801904 px, `29 candles`	802148 px, `29 candles, 1 unreadable`
one candle's high is a not-a-number	801904 px, `29 candles`	802148 px, `29 candles, 1 unreadable`
one candle's high is an endless value	801904 px, `29 candles`	802148 px, `29 candles, 1 unreadable`
one candle's high is a huge integer	801904 px, `29 candles`	802148 px, `29 candles, 1 unreadable`
every candle's high is a stored flag	1544 px, `No candle carried a readable price`	1544 px, the same

The match scores name the reading. `29 candles, 1 unreadable` scores 0.526 against the header before the change and **0.993** after it. A string the chart never drew scores 0.531, and a near miss of one digit scores 0.959, so a match above 0.98 is the string and nothing else is.

The positive control

The clean series draws **0 differing pixels** between the two trees, across all 630000 of them. A chart with nothing refused is the same picture it was.

The counter itself reports **0 px** for the background gradient read against itself and **1112 px** for a chart holding no candles, so it can report both a zero and a figure.

The bullseye list

`_bbullseye_data` was set to an empty list in `ChartPainter.__init__`, written nowhere else and read nowhere. `_draw_bullseye` draws the four BB Bullseye envelopes from `_bb_data`, the live Bollinger series, which is written in `_compute_indicators` and read in three places. The empty list is gone. The overlay, its key, its toggle and its four envelopes are untouched.

The grid line list stays, and here is why

`_grid_lines` is written by `set_grid_lines` and read zero times, and `GridLine` is constructed zero times. The React page's `GridLines` component draws `price_axis.grid.ticks`, the price axis ruling, not this list.

Removing it does not stop at the widget. `set_grid_lines` is named in `STEP_NAMES`, `grid_lines_held` is published by `build_view_model`, and `src/gui/web/native_chart.js` names `grid_lines_held` in `DECLARED_FIELDS`. Taking the list out takes a field out of the payload the Electron renderer is handed, which is a published contract rather than a price admission. That row stays open.

Readings this entry declares

The run named the host's own Consolas and Segoe UI font files to the Qt font database. The offscreen platform ships no font family at all, so without them every glyph renders as a box and the pixels carry a string's length rather than its letters. `pip install -e ".[dev]"` installs neither.

The match reading holds only where the band it reads is sparse. On a full chart the header band carries 820 ink pixels and a candidate either fits or does not. On a band crowded with candles every candidate fits, so the body message is read on a chart that drew no candles at all.

No venue was contacted, no bot was started, and no order was placed, priced, previewed or cancelled. The runtime tree was read with a redirected home.

A sentence this entry overtakes

It was not reworded. It is quoted here.

"Layer 2 of #230 is unbuilt." - the bullseye list is removed and the grid line list stays, named above with the published fields that hold it.

2026-10-20 - #230 - the grid line list, its setter and its published field go

What went, and what read it first

Nothing read the list. `_grid_lines` carried two writes and no reads. `grid_lines` on the page's model carried two writes and one read, and that read took the list's own length for `grid_lines_held`. `set_grid_lines` had no caller in either build, and `GridLine` was constructed nowhere.

The list, the setter, the record type and the published field are gone from both builds. `native_chart.js` no longer names `grid_lines_held` among its declared fields.

Why both sides had to change in one step

`native_chart.js`, in `checkFields`, walks every declared field and raises a missing-field fault on one the frame does not carry. Driven in node against the real served frame, the page before this change reports no fault on the frame as served and **one** fault, `grid_lines_held: missing`, on a frame with the field stripped. The page after this change reports no fault either way. Removing the field from the surface alone would have faulted on every frame.

What the operator sees, read off the pixels

One chart rendered through its own `paintEvent` at 1000 by 630, offscreen, with the home directory redirected. A clean series of thirty readable candles draws **0 differing pixels** across all 630000 of them, with 298608 ink pixels before and after.

the control	pixels differing
the same series rendered twice	0
a chart holding no candles	583542
twenty candles instead of thirty	77705
a different symbol in the header	1181

The offscreen platform reports no font family, so the run names six of the host's own font files to the Qt font database and the database then reports five families. The different-symbol row is what proves the comparison reads letters and not boxes.

What the frame carries

One real request answered by `serve`, over the real registry, written by `encode_frame`. The frame still says ok and still ends in one newline. It carried 47 fields and now carries 46. `grid_lines_held` is the only field gone, no field arrived, and no other field's value moved.

The ruling stays

The dotted price ruling is not this list. `src/gui/native_chart.py`, in `paint_to`, steps it from the fitted price range, and `src/gui/main_tabs/native_chart_surface.py`, in `price_grid`, answers the same steps for the page, which draws them in `src/gui/web/native_chart.js`, in `GridLines`. The page reads seven ticks at a step of five before and after.

Readings this removal declares

`STEP_NAMES` in `src/gui/main_tabs/native_chart_surface.py` is read zero times. Only the one entry naming the deleted setter went with it.

`refused_candles` is in the frame and is not in the page's declared fields, so `checkFields` does not guard it.

The docs archetype's hallucination rule checks a cited path, never a cited symbol. A page citing four symbols absent from disk reports no hallucination finding while its paths resolve.

No venue was contacted, no bot was started, and no order was placed, priced, previewed, amended or cancelled.

Two rows and two sentences this entry overtakes

They were not reworded. They are quoted here.

"| `GridLine` | A buy or sell level |" - the buy and sell levels the chart draws are `PositionMarker` records, placed by `set_positions`.

"| `set_grid_lines` | The buy and sell levels |" - `set_grid_lines` had no caller in either build.

"That row stays open." - the operator authorised the removal, and the list, its setter and the published field went together.

"the bullseye list is removed and the grid line list stays, named above with the published fields that hold it." - the grid line list is removed too, and `grid_lines_held` is gone from the frame and from the declared fields.

History Tab

Reference. Trade history pulled from the venues, paired with the platform's own trading logic and graded.

What builds it

One builder makes the tab, adds it to the row, and hands it the bot manager.

`src/gui/main_tabs/history_tab.py` — `HistoryTabMixin._build_history_tab`

```
def _build_history_tab(self) -> None:
    """Build the History tab and add it to the main tab widget."""
```

Fetching

Refresh starts an asynchronous fetch that asks every active exchange for the operator's own fills. The helper walks each exchange and symbol pair the bot manager exposes, and turns each venue trade into one row.

`src/exchange/history_helpers.py` — `fetch_all_history_chunked`

```
async def fetch_all_history_chunked(
```

The exchange is the authority here. The rows on screen are the venue's answer, and the platform's own log joins to them rather than replacing them.

The tab refreshes itself on activation when the last fetch is older than five minutes and no fetch is already in flight.

`src/gui/main_window.py` — `_on_main_tab_changed`

```
is_stale = last_ts == 0 or _t.time() - last_ts > 300
if is_stale and not in_flight:
    hist.refresh()
```

The venue call queue a refresh shares

One connector serves every bot on a venue and runs a single worker, so every caller on that venue shares one call queue eight deep. A refresh asks for one symbol at a time, and before each ask it waits for that queue to empty. The balance, ticker, portfolio and OHLCV reads the running platform polls therefore take the worker first. Past its budget the ask goes through behind two reserved slots instead, so a refresh is never the submission that fills the queue, and a busy venue changes when a refresh runs and never which rows it answers.

`src/exchange/ccxt_connector.py` — `CCXTConnector.await_bulk_read_slot`

```
    if depth <= BULK_READ_ADMIT_DEPTH:
        break
    if time.monotonic() - start >= budget_sec and depth <= admit_past_budget:
        break
    await asyncio.sleep(BULK_READ_WAIT_SLICE_SEC)
```

`src/exchange/history_helpers.py` — `_await_slot`

Measured on a fleet of thirty-nine bots on one venue, with two hundred platform reads issued while a refresh ran: forty-seven of those reads were refused at the queue's cap while the refresh held the worker back to back, and six once the refresh waited its turn. The same two hundred reads with no refresh running at all were refused five times, so the refresh now costs the platform about one read where it used to cost forty-two. Both runs returned the same 8,558 rows.

A refusal at the cap costs a symbol its rows, so the wait never ends while the queue is full and one refused submission earns a second wait.

`src/exchange/history_helpers.py` — `BULK_READ_ATTEMPTS`

```
    BULK_READ_ATTEMPTS = 2
```

A venue that does not serialise its calls needs no turn-taking, and the default waits for nothing.

`src/exchange/base.py` — `ExchangeInterface.await_bulk_read_slot`

```
    del budget_sec
    return 0.0
```

Where the trade record is kept

Refresh reads the venue live and keeps nothing. The year of trades the Simulator needs is kept separately, on disk, under the exchange history bucket. One file holds one exchange, one symbol and one year, and every file names its exchange inside itself.

`src/core/log_paths.py` — the bucket

```
    def get_exchange_history_dir() -> Path:
        """``exchange_history/`` bucket — the YTD trade files.
```

The files are filled from a transactions CSV exported from the exchange. The import keeps buys and sells, counts and drops everything that is not a trade, and refuses a file that is missing a column it needs. Re-importing an overlapping export adds only the rows whose id is new.

`src/exchange/ytd_csv_import.py` — the import

```
def import_ytd_csv(
    csv_path: Path,
    exchange_id: str,
    root: Optional[Path] = None,
) -> ImportResult:
    """Read ``csv_path`` and write ``exchange_id``'s trade files under
    ``get_ytd_root(root)``."""
```

Where the export covers a period and carries no trade for a symbol in it, that period is written to a gap record rather than filled. See the Simulator tab for the file format and the reader.

The import reads the export through a column map chosen by the exchange id it is given. One map exists, for Coinbase, and a Coinbase export writes the same files it always did. An exchange with no map is refused by name, with a line saying a sample export is needed, and nothing is written. The maps and the table of exchanges are in the Simulator tab.

`src/exchange/ytd_csv_import.py` — the map chosen at import

```
def import_ytd_csv(
    csv_path: Path,
    exchange_id: str,
    root: Optional[Path] = None,
) -> ImportResult:
    """Read ``csv_path`` and write ``exchange_id``'s trade files under
    ``get_ytd_root(root)``."""
    column_map = export_map_for(exchange_id)
```

Filtering, paging, export

Eight methods carry the controls under the table.

Method	What it does
`_populate_filter_options`	Fills the exchange, symbol and side pickers from the rows in hand
`_apply_filters`	Narrows the set
`_reset_filters`	Restores the full set
`_current_filters`	Reports the active state
`_remember_to_bound`, `_to_bound_ts`	Hold the upper date bound across a refresh
`_render_page`, `_prev_page`, `_next_page`	Page the result
`_export_csv`	Writes the current selection out

Where the cell values come from

One function returns the rendered page: the rows plus the pager state. It is the only disk read on that path.

`src/exchange/history_read_contract.py` — `build_page`

```
def build_page(
    filtered: list[dict],
    page: int = 0,
    bot_manager: Any = None,
    gate_index: Optional[dict] = None,
    voting_index: Optional[dict] = None,
) -> HistoryPage:
    """One rendered page: the ``HistoryRow`` list plus the pager state.

    ``build_join_indexes`` runs when neither index is supplied, and it is the
    only disk read on this path.
    """
```

One sentence above is overtaken. Quoted whole:

> *One function returns the rendered page: the rows plus the pager state. It is the only disk read on that path.*

The page builder still returns the rows and the pager state. It makes no disk read when the tab draws, because the tab hands it both join indexes. The tab reads the logs once per fetch, on its own thread, and the page builder's own read only runs for a caller that supplies neither index.

Now: the tab reads the logs once per fetch, in steps of about 8 ms on the window thread, after the first page draws (`src/gui/history_tab.py, in join_steps`).

Thirteen columns each yield one cell carrying a value, its text, its colour and its tooltip.

`src/exchange/history_read_contract.py` — `COLUMNS`, the first ten

```
COLUMNS: tuple[HistoryColumn, ...] = (
    HistoryColumn(0, "timestamp", "Timestamp (UTC)"),
    HistoryColumn(1, "exchange", "Exchange"),
    HistoryColumn(2, "symbol", "Symbol"),
    HistoryColumn(3, "bot", "Bot"),
    HistoryColumn(4, "side", "Side"),
    HistoryColumn(5, "amount", "Amount"),
    HistoryColumn(6, "price", "Price"),
    HistoryColumn(7, "cost", "Cost USD"),
    HistoryColumn(8, "fee", "Fee"),
    HistoryColumn(9, "trade_id", "Trade ID"),
```

Grade, Gates and Voting close the set. The ordering is fixed newest-first and nothing re-sorts.

`src/exchange/history_read_contract.py` — `ROW_ORDER`

```
ROW_ORDER = "timestamp_desc"
"""Newest first. ``fetch_all_history_chunked`` sorts ``reverse=True``,
``HistoryTab`` calls no ``setSortingEnabled``, and nothing re-sorts."""
```

The table renders those fields and derives none of them. A cost, a grade, a colour or a gate light on screen is always the contract's answer, which is what keeps a second implementation of History from growing behind the renderer.

The footer's two strings follow the same rule. The contract declares the page counter and the summary line, and the tab calls the contract for both, so one arithmetic produces each string for every surface that draws it.

`src/exchange/history_read_contract.py` — `page_label`, the counter the tab calls

```
def page_label(page: int, total: int) -> str:
    """The footer's page counter, or "No matches" when ``total`` is zero."""
    if total == 0:
        return "No matches"
    page = clamp_page(page, total)
    return f"Page {page + 1} / {page_count(total)} ({total} trades)"
```

Grading

One record and its price context go in, and an unweighted mean comes out. An axis with no input is skipped, so it neither credits nor penalises.

`src/trading/trade_grader.py` — `grade_trade`

```
def grade_trade(record: TradeRecord, ctx: PriceContext) -> TradeGrade:
    """Grade one ``record`` against its ``ctx``.

    ``TradeGrade.overall_numeric`` is the unweighted mean of the sub-scores
    whose inputs were present, or 0.5 when no axis could be scored.
    """
    exec_score, exec_bps = _score_execution(record, ctx)
    timing_score, mfe, mae = _score_timing(record, ctx)
    strategic_score, sb_delta = _score_strategic(record, ctx)
    outcome_score, realized = _score_outcome(record, ctx)
```

The mean becomes a letter at fixed boundaries.

`src/trading/trade_grader.py` — `_letter_from_numeric`

```
def _letter_from_numeric(num: float) -> str:
    """Map ``num`` in [0.0, 1.0] to ``A+``, ``A``, ``B``, ``C``, ``D`` or ``F``."""
    if num >= 0.93:
        return "A+"
    elif num >= 0.85:
        return "A"
    elif num >= 0.70:
        return "B"
    elif num >= 0.55:
        return "C"
    elif num >= 0.40:
        return "D"
    else:
        return "F"
```

`grade_trades` runs the batch. The regime tag reaches the grade's regime and its rationale without a sub-score of its own.

Gate analysis

Four helpers in `src/exchange/history_helpers.py` join the gate log to a trade.

Helper	What it does
`build_page_gate_index`	Buckets the gate log entries by bot id and minute
`lookup_gate_entry`	Picks the closest entry to a trade
`gate_cell_text`	Renders the cell text
`gate_cell_tooltip`	Renders its tooltip

The lights themselves come from one vocabulary. Nineteen of them, the scrum bank then the fold, each carrying its bank marker, its label, its state and its colour. No Qt type crosses that boundary.

`src/trading/gate_vocabulary.py` — `gate_light_row`

```
def gate_light_row(
    scrum_armed: bool,
    fold_armed: bool,
    scrum_blockers: Optional[list] = None,
    fold_blockers: Optional[list] = None,
    landing_strip_side: str = "",
    evaluated: bool = True,
) -> list[dict]:
    """The nineteen lights, in draw order: the scrum bank then the fold.

    Each entry carries ``bank`` ("S" or "F"), ``label``, ``state`` and
    ``color``. Serialisable: no Qt type crosses this boundary.
    """
```

Five states, five colours.

`src/trading/gate_vocabulary.py` — `LIGHT_COLORS`

```
LIGHT_COLORS: dict[str, str] = {
    "override": "#22d3ee",
    "not_evaluated": "#333340",
    "blocked": "#ff3366",
    "passed": "#00cc55",
    "not_the_blocker": "#c8901e",
}
```

That vocabulary is shared on purpose. The Simulator's gate cell and this table read the same labels and the same blocker prefixes, so a gate added on one surface cannot be missing from the other. That shared seam is what makes the Simulator's gate comparison meaningful.

Each trade is joined to its gate entry by `lookup_gate_entry` in `src/exchange/history_helpers.py`. One threshold decides what counts as a match, and a trade with no entry inside it reads "no record" in the cell.

`src/exchange/history_helpers.py` — the join threshold

```
JOIN_TOLERANCE_SECONDS = 60.0
"""Largest gap ``_best_entry_within_window`` accepts between a trade and a log
entry, and the backdate the index builders apply to ``since``."""
```

Nothing measures coverage across a window.

`build_page_voting_index` and `lookup_voting_entry` do the same for the indicator voting snapshot behind each trade.

When the logs are read

The tab reads the gate log and the voting log once, when a fetch lands, on a thread of its own. Drawing a page and pressing Prev or Next read the indexes already in hand and touch no file.

Now: the read waits until the first page draws, then runs in steps of about 8 ms on the window thread (`src/gui/history_tab.py, in HistoryTab._start_join_index_build`).

`src/gui/history_tab.py` — `HistoryTab._start_join_index_build`

```
def _start_join_index_build(self) -> None:
    """Read the gate and voting indexes for ``_all_trades`` off this thread.

    ``_render_page`` then reads the held indexes, so no page draw and no
    pager press touches disk; the newest build wins and an older
    build's answer is dropped.
    """
```

The read keeps only the minutes a cell can ask for. A lookup reads a trade's own minute and the two beside it, so an entry in any other minute can never reach a cell and is not held.

`src/gui/history_tab.py` — `join_bucket_keys`

```
def join_bucket_keys(rows: list, bot_manager) -> set:
    """The ``(bot_id, minute)`` keys the gate and voting lookups can reach.

    ``lookup_gate_entry`` reads a row's own minute and the two beside it, so an
    index holding only these keys answers every lookup identically.
    """
```

Measured on the operator's own logs, with a six-megabyte active gate log, five rotated copies of fifty-two megabytes each and a sixteen-megabyte voting log: 3,494 rows over 35 pages. The gate read kept 2,895 minutes and the voting read 2,955, against 167,944 gate entries parsed before. The longest stretch in which the window could not repaint fell from 4.2 seconds to 0.06, which is what an idle window reads on the same instrument. Every Gates cell and every light row on four sampled pages matched the earlier values exactly.

Nothing paces the fetch itself. One tab activation asks the venue for fills once per bot symbol, and those calls share one worker thread and a queue of eight with every other venue reader on the platform.

`src/exchange/ccxt_connector.py` — the shared queue

```
SYNC_QUEUE_CAP: int = 8
```

The renderer

`HistoryWebTable` in `src/gui/react_history_panel.py` draws the table with React inside the Chromium PySide6 ships, and sits where the Qt table used to. One History tab, one renderer, one set of rows.

The Qt table takes no theme. The React table takes one because it builds its page in that theme, and the Qt table's chrome comes from the application stylesheet instead, so a theme handed to it reached no colour. The tab passes neither.

`src/gui/history_qt_table.py` — `HistoryQtTable`, what construction takes

```
        def __init__(self, parent=None) -> None:
            super().__init__(parent)
            self.setAccessibleName("Qt History Table")
            self._last_model: dict = {}
```

Driven with one row and two columns, 26 values read back off the real table under Cyberpunk Dark and then Neon Light: every colour and every string was identical and the only value that moved was the theme name the table had stored. The React table, driven the same two ways, moves all six of its page colours.

The Simulator front-load

Refreshing History hands the Simulator the year of live trades it needs before a parity run. The tab's signal connects to the Simulator's panel, and the builder makes that connection once both tabs exist.

`src/gui/history_tab.py` — `HistoryTab.history_refreshed`

```
history_refreshed = Signal(list)
```

Now: the signal is a `Signal(object)`, so it hands the rows over without converting them (`src/gui/history_tab.py, in HistoryTab.history_refreshed`).

Paper History

The manual's part list names a Paper History Tab beside this one. No such module exists. Issue #19 built the Paper tab and its log, so the ledger such a tab would read is on disk now, and no screen reads it. See paper-trader.md.

Bridge

Three methods serve this screen.

Bridge method	Serves
`history.view_model`	The whole screen, assembled from the read contract
`history_tab.chrome`	The surrounding controls
`react_history_panel.state`	The table

Three renderer files draw it: `history_tab.js` for the chrome, then `history_panel.js` and its stylesheet for the table.

2026-09-08 08:17 - no issue recorded - what the closed issues landed

The History Tab is able to pull trade history from all active exchanges via their respective APIs. It also pairs each imported trade with its in-platform trading logic and applies our trade grading system.

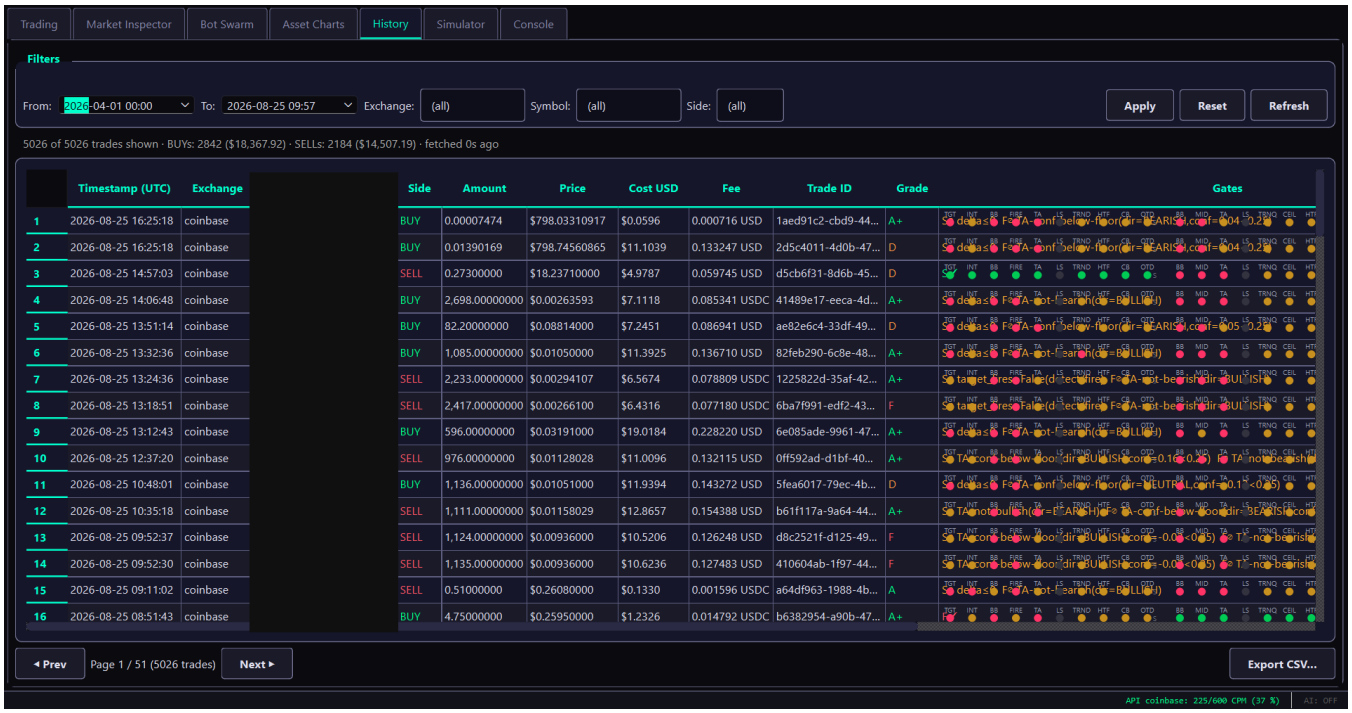
The tab keeps the name History. It sits eighth on the bar, on the gold ground with red text.

The tab fetches from every active venue and draws the rows with React. Every cell value comes from one read contract; the table renders those fields and derives none, which stops a second History growing behind the renderer.

`src/exchange/history_read_contract.py` — `build_page`

```
  def build_page(
      filtered: list[dict],
      page: int = 0,
      bot_manager: Any = None,
      gate_index: Optional[dict] = None,
      voting_index: Optional[dict] = None,
  ) -> HistoryPage:
```

The grade comes from the trade grader and the gate lights come from the same vocabulary the Simulator's gate cell reads. A refresh also hands the Simulator its year of live trades, so the tab's signal front-loads the parity run.



The History tab: filters, graded rows and the gate lights.

The Filters group carries From and To date pickers, then Exchange, Symbol and Side, each filled from the rows in hand. Apply narrows the set, Reset restores it, and Refresh goes back to the venues. The line under the group counts the retained rows against the loaded rows, both sides with their dollar totals, and the age of the last fetch.

Thirteen columns carry a row, newest first, and nothing re-sorts. BUY draws green and SELL red.

`src/exchange/history_read_contract.py` — `COLUMNS`, the first ten

```
COLUMNS: tuple[HistoryColumn, ...] = (
    HistoryColumn(0, "timestamp", "Timestamp (UTC)"),
    HistoryColumn(1, "exchange", "Exchange"),
    HistoryColumn(2, "symbol", "Symbol"),
    HistoryColumn(3, "bot", "Bot"),
    HistoryColumn(4, "side", "Side"),
    HistoryColumn(5, "amount", "Amount"),
    HistoryColumn(6, "price", "Price"),
    HistoryColumn(7, "cost", "Cost USD"),
    HistoryColumn(8, "fee", "Fee"),
    HistoryColumn(9, "trade_id", "Trade ID"),
```

Grade, Gates and Voting close the set.

The Gates cell opens with the blocker text, then the lights. Nineteen of them: the ten scrum gates TGT, INT, BB, FIRE, TA, LS, TRND, HTF, CB and OTD, then the nine fold gates BB, MID, TA, LS, TRNQ, CEIL, HTF, CB and OTD. Each light draws under its own label, and each bank ends with an S or an F marker. Five states, five colours.

`src/trading/gate_vocabulary.py` — `LIGHT_COLORS`

```
LIGHT_COLORS: dict[str, str] = {
    "override": "#22d3ee",
    "not_evaluated": "#333340",
    "blocked": "#ff3366",
    "passed": "#00cc55",
    "not_the_blocker": "#c8901e",
}
```

Green passed, red the gate that blocked, amber not armed for another reason, grey not evaluated at this candle, and cyan a landing-strip override. The blocker text and the light banks overlap in the figure; both occupy the one cell.

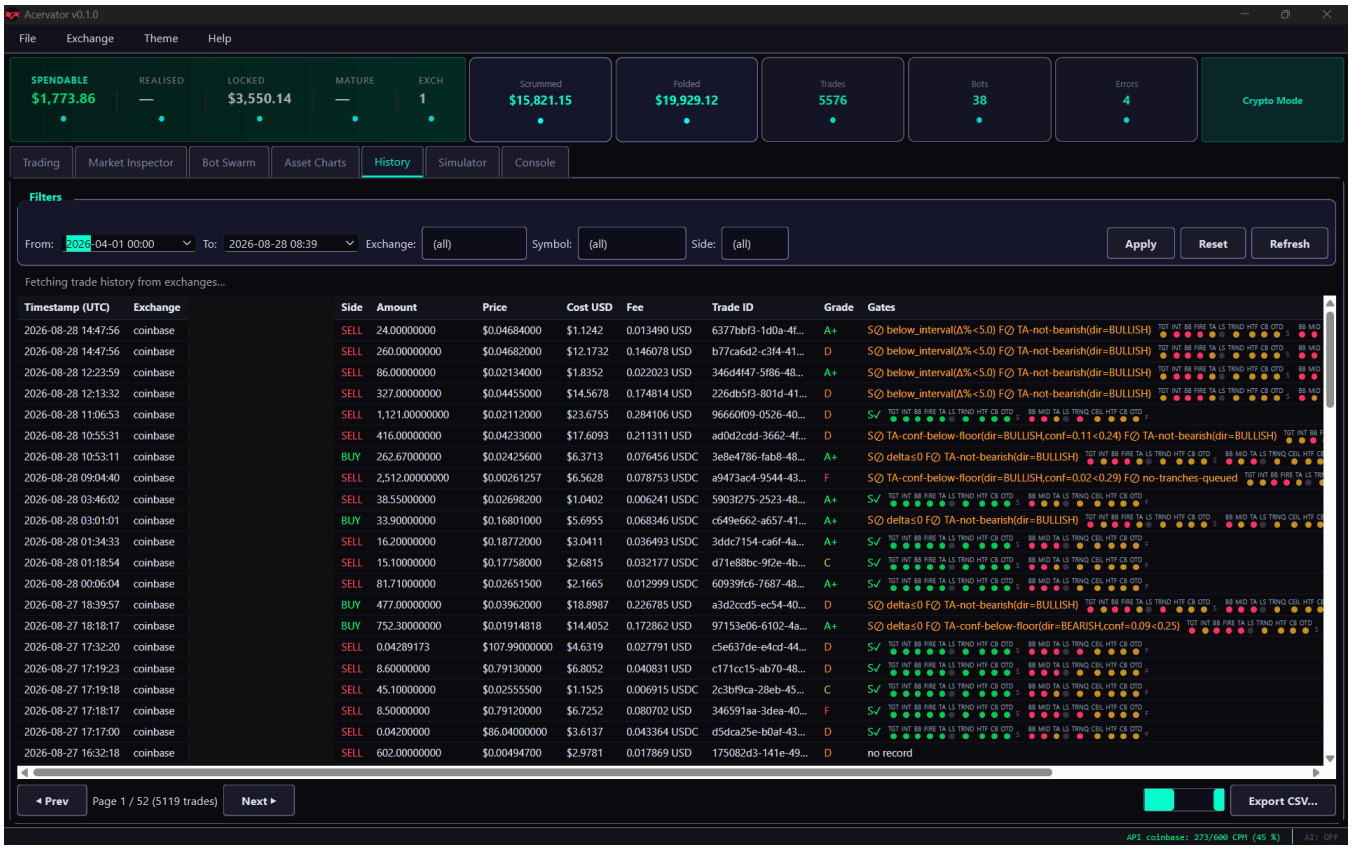
One sentence above is overtaken. Quoted whole:

> *The blocker text and the light banks overlap in the figure; both occupy the one cell.*

The blocker text and the light banks share the one cell and no longer overlap. The Qt table writes the text once, into the lights cell's own label, and leaves the table item under that cell blank. A row whose gate log holds no entry has no lights cell, and there the table item carries the `no record` text itself.

Prev, the page counter and Next page the result at the foot, and Export CSV writes the current selection out. The read contract declares both footer strings, and the tab calls it for both, so the counter and the summary read the same on this tab as on the React panel.

History Tab (React):



The same tab, captured while the asynchronous fetch runs. Five fixed status strings cover the states before any row exists, and the one on screen is the fetching entry.

`src/exchange/history_read_contract.py` — `STATUS_TEXT`

```
STATUS_TEXT = {
    "idle": "No history loaded yet — click Refresh.",
    "no_bot_manager": "Bot manager unavailable — cannot fetch history.",
    "no_async_loop": "Async loop not ready — try again after platform starts.",
    "fetching": "Fetching trade history from exchanges…",
    "timeout": "Fetch timeout (300s). Exchange may be rate-limited; try again.",
}
```

Two constants bound the fetch, in the same module: the poll runs on a 400 ms timer, so an observed latency is the true latency plus up to one interval, and the fetch is abandoned after three hundred seconds.

`src/exchange/history_read_contract.py` — the fetch bounds

```
FETCH_POLL_INTERVAL_S = 0.4
```

The header strip is visible here, which marks the screen as a live-trading tab. REALISED and MATURE draw the exchange figures the Live tab draws, which issue #418 fed. A row whose gate log holds no entry inside the join window reads `no record`.

Both columns draw an exchange figure here too, because one strip serves every tab that shows it. Where the venue has answered for no bot they stay empty, and Privacy Mode leaves an empty column empty.

Trade grading

The grader scores a trade that has already happened. It never feeds a trading decision, it never tunes a parameter, and no gate consults it. The same inputs always give the same grade.

Four axes are scored. An axis with no input is skipped: it neither credits nor penalises, and the overall number is the unweighted mean of the rest.

`src/trading/trade_grader.py` — `grade_trade`

```
exec_score, exec_bps = _score_execution(record, ctx)
timing_score, mfe, mae = _score_timing(record, ctx)
strategic_score, sb_delta = _score_strategic(record, ctx)
outcome_score, realized = _score_outcome(record, ctx)
```

Axis	What it reads	What a 1.0 means
Execution	fill price against the reference price at decision time	no slippage
Timing	the favourable excursion against the adverse one over the following candles	the trade went far more right than wrong before it closed
Strategic	the asset's discipline ratio before the trade against after it	the trade moved the ratio the operator's way
Outcome	realised profit per unit	a clear gain

The regime tag reaches the grade and the rationale string and gets no score of its own. Grading the same record with a regime tag and without one returns the same number, which is the control that proves it.

The scale is 0.0 to 1.0, not 0 to 100, and the mean becomes a letter at fixed boundaries.

`src/trading/trade_grader.py` — `_letter_from_numeric`

```
    if num >= 0.93:
        return "A+"
    elif num >= 0.85:
        return "A"
    elif num >= 0.70:
        return "B"
    elif num >= 0.55:
        return "C"
    elif num >= 0.40:
        return "D"
    else:
        return "F"
```

Driving the mapper across each boundary and just under it flips the letter exactly where the code says, in both directions.

One trap worth knowing. A trade with no gradeable input at all scores 0.5 and therefore reads as a **D**, not as ungraded. A fully specified trade scores 1.0 and reads A+. A column of Ds may mean the trades were poor, or it may mean the surrounding price context never arrived.

2026-10-06 - #1133 - the trade historian's own fetch

Refresh is not the only thing that asks the venue for fills. A launch also runs the trade historian, once for every market in the fleet, and the historian calls the venue directly rather than through the connector's paced read path.

`src/core/trade_historian.py` — the call

```
trades = self._exchange.fetch_my_trades(symbol, limit=limit) or []
```

The published limit

Coinbase publishes the limit for the private fills endpoint as ten requests per second per profile, with bursts up to twenty. One fetch every hundred milliseconds sits on that published figure, and the platform derives the interval from it rather than picking a number.

`src/core/trade_historian.py` — the interval

```
PUBLISHED_FILLS_RPS = 10.0
HISTORY_FETCH_INTERVAL_S = 1.0 / PUBLISHED_FILLS_RPS
```

The gate is one object for the whole program, so every market's fetch queues behind the last one. A second connector's threads wait their turn in the same queue, which is what keeps the request rate at the published figure no matter how many connectors a launch builds.

`src/core/trade_historian.py` — the gate

```
_FETCH_GATE = _HistoryFetchGate(HISTORY_FETCH_INTERVAL_S)
```

A refused fetch

A venue can refuse for reasons the platform does not control, so a refusal is tried again. The historian uses the retry the exchange paths already share: three attempts, doubling the wait each time, and only for the error classes that name a transient condition.

`src/core/trade_historian.py` — the attempt budget

```
HISTORY_FETCH_ATTEMPTS = 3
HISTORY_RETRY_BASE_DELAY_S = 1.0
```

A refusal is the venue answering normally, so it records nothing about the connection. Nothing on this path writes the lost-connection state that turns a venue's row red in Settings, and that state keeps its one writer in the credential validator.

`src/exchange/credential_state.py` — the lost-connection writer

```
def record_connection_lost(
    exchange_id: Any, when: Any = None, path: Any = None
) -> bool:
```

Once every attempt is spent the market gets no history, and the line above the table says so with the count.

`src/core/trade_historian.py` — the line

```
MISSING_HISTORY_FORMAT = (
    "Trade history missing for {refused} of {scanned} markets — "
    "the venue refused the fetch."
)
```

One sentence above is overtaken. Quoted whole:

> *The read contract declares both footer strings, and the tab calls it for both, so the counter and the summary read the same on this tab as on the React panel.*

The read contract still declares both footer strings and the tab still calls it for both. When a history fetch has been refused the Qt tab appends the missing-history line to the summary it got back, so on that one state the summary reads longer here than on the React panel.

`src/gui/history_tab.py` — the append

```
missing = self.missing_history_status()
self._set_status(f"{summary}  {missing}" if missing else summary)
```

Back to the subsystem index.

Console

Reference. A raw log tail in the upper pane and the emitter signal stream in the lower one.

What builds it

`ConsoleTabMixin._build_console_tab` in `src/gui/main_tabs/console_tab.py` builds both panes into a vertical splitter, weighted three to two, and starts three timers.

The builder does not hold the panes. It asks `variant_surface` for the tab class, takes the parts off it, and adds it to the tab row.

`src/gui/main_tabs/console_tab.py` — `ConsoleTabMixin._build_console_tab`

```
tab = surface_class(CONSOLE)()
self._console_tab = tab
self._console = tab.log_pane
self._signal_view = tab.signal_view
self._console_pause_btn = tab.pause_button
self._console_pause_indicator = tab.pause_indicator
```

Two classes answer that call. `ConsoleQtTab` builds the Qt panes, the control bar and the vertical splitter. `ConsoleReactTab` holds the same six parts and draws all of them in one web view. React is the running choice.

`src/gui/variant_surface.py` — the two loaders for the Console

```
def _qt_console() -> type:
    """Import and return the Qt Console tab."""
    from .qt_console_tab import ConsoleQtTab

    return ConsoleQtTab


def _react_console() -> type:
    """Import and return the React Console tab."""
    from .react_console_tab import ConsoleReactTab

    return ConsoleReactTab
```

`_start_console_timers` starts two timers, the signal drain and the console health. The pause refresh timer is built with the tab and started by the pause action.

`src/gui/main_tabs/console_tab.py` — `ConsoleTabMixin._start_console_timers`

```
self._signal_timer = QTimer(self)
self._signal_timer.setInterval(surface.DRAIN_INTERVAL_MS)
self._signal_timer.timeout.connect(self._drain_signals)
self._signal_timer.start()
```

Upper pane: the Python log

The upper pane is a read-only text view capped at two thousand blocks, which is what bounds its memory.

`src/gui/main_tabs/console_tab.py` — `ConsoleTabMixin._build_console_tab`

```
self._console = QPlainTextEdit()
self._console.setReadOnly(True)
self._console.setFont(QFont("Consolas", 9))
```

`ConsoleQtTab` builds that view. Under React the same calls land on `PagePane`, which holds the blocks and pushes them to the page.

The pushes of one event-loop turn coalesce into one. `redraw` starts a zero-interval single-shot timer and `_push_now` builds the payload and pushes it once when the timer fires, so a signals drain that appends 200 rows pushes the page once rather than 200 times. Measured on a Portfolio Battery walk, whose gate chain fires the TA engine's thirteen postcondition pins per evaluation into the drain: with a push per append the walk thread reached 5,000 bars in 257 seconds while the GUI thread rebuilt the page per row; with the pushes coalesced, 14 seconds.

`src/gui/react_console_tab.py` — the coalesced push

```
        def redraw(self) -> None:
            """Ask for one push of both panes on the next event-loop turn; the
            asks of one turn coalesce into one ``_push_now``."""
            if not self._redraw_timer.isActive():
                self._redraw_timer.start()
```

`src/gui/qt_console_tab.py` — `ConsoleQtTab.__init__`

```
self.log_pane = QPlainTextEdit()
self.log_pane.setReadOnly(True)
self.log_pane.setFont(
    QFont(surface.PANE_FONT_FAMILY, surface.PANE_FONT_POINT_SIZE)
)
self.log_pane.setMaximumBlockCount(surface.PANE_MAX_BLOCKS)
```

A log handler formats each record and hands the line to a relay. The relay carries it over a Qt signal to the thread that constructed it and paints it there, so a log call from any thread reaches the pane without touching the widget from that thread. Colour follows the level, and a message holding the phrase `INDICATOR PANEL` paints in the accent colour.

The handler attaches at two points, and both are needed because the logging engine clears propagation on the application logger.

`src/gui/main_tabs/console_tab.py` — `ConsoleTabMixin._build_console_tab`

```
logging.getLogger().addHandler(qt_handler)
logging.getLogger("acervator").addHandler(qt_handler)
```

The builder reads those two names from the surface and adds the tab's own handler to each.

`src/gui/main_tabs/console_tab.py` — `ConsoleTabMixin._build_console_tab`

```
for name in surface.HANDLER_LOGGERS:
    logging.getLogger(name).addHandler(tab.log_handler)
```

The handler level

The tab sets its own handler to debug and changes no logger's level. An earlier build raised the root logger and never restored it, which left the whole application logging at debug volume for the life of the process. Setting the handler instead keeps the verbosity local to this pane.

`src/gui/main_tabs/console_tab.py` — `ConsoleTabMixin._build_console_tab`

```
# Two disjoint attach points: `logging_engine` clears `acervator.propagate`.
for name in surface.HANDLER_LOGGERS:
    logging.getLogger(name).addHandler(tab.log_handler)
```

Each tab class sets that level on its own handler, from `HANDLER_LEVEL` in the surface.

`src/gui/qt_console_tab.py` — `ConsoleQtTab.__init__`

```
self.log_handler.setFormatter(
    logging.Formatter(surface.LOG_FORMAT, datefmt=surface.LOG_DATEFMT)
)
self.log_handler.setLevel(surface.HANDLER_LEVEL)
```

Pausing

Pause holds new lines in a bounded buffer. On resume the buffer replays and one notice names anything the cap dropped, so a gap is never silent.

`src/gui/main_tabs/console_log_handler.py` — `_QtLogHandler.set_paused`

```
def set_paused(self, paused: bool) -> None:
    """Hold new lines in ``_buffer`` while ``paused``, and drain them on resume.

    A drain that followed a ``_buffer_max`` overflow appends one notice line.
    """
```

`buffered_count` feeds the pause indicator, and Clear empties the pane.

Lower pane: the Emitter Network

The signal tail drains twice a second and renders the newest two hundred records. Each line carries a pass marker, a fail marker or a neutral one, then the signal name, the site, the actual value and the expected one. Records the slice drops are counted and announced with a gap marker drawn above the slice.

`src/gui/main_window.py` — `_drain_signals`

```
def _drain_signals(self) -> None:
    """Render sink records past `_signal_seq`, keeping the newest 200."""
```

The drain reads the process sink, the one `get_sink` answers on the GUI thread. A Simulator run routes its worker thread to the Simulator's own sink under the sim bucket, so the walk's pins, the TA engine's thirteen postcondition pins per evaluation among them, no longer reach this pane; the Simulator's own presses on the GUI thread and the one `sim.sink.routed` row each run leaves at its end do. See simulator.md.

`src/core/signal_contract.py` — the sink the drain reads

```
def get_sink() -> Optional[SignalSink]:
    """The sink the calling thread emits into: its `route_thread` sink when one
    is set, else the process sink."""
```

Seven counters ride along, published every five seconds: the sequence watermark, drain ticks, records read, records rendered, records the slice dropped, gap markers drawn, and health ticks seen. Ticks count invocations rather than records, which is what separates a quiet sink from a stopped timer.

`src/gui/main_window.py` — `_drain_signals`

```
# Counts invocations, not records: a quiet sink is not a stopped timer.
self._signal_drain_ticks = getattr(self, "_signal_drain_ticks", 0) + 1
```

Where the lower pane goes

The signal pane belongs to the Status tab once that screen exists. See system-status.md.

Bridge

Two methods serve this screen.

Bridge method	Serves	Renderer module
`console.tab`	Both panes	`console_tab.js`
`console.log_lines`	The log tail	`console_log.js`

The surface holds the two behaviours the tab owns rather than describes, so neither is written twice.

In the surface	Holds
`ConsolePane`	The same capped block buffer, at the same two-thousand-block limit
`SignalLedger`	The same seven counters
`format_record`	The same formatter the tab attaches to its handler

2026-09-07 15:58 - no issue recorded - ten single-word tabs, each on its own ground

This tab is focused on displaying Python activity and errors. The lower half, which is displaying the Emitter Network activity, will be migrated to the System Status Tab (under the Watchdog) which is to be built in the near future.

The tab keeps the name Console. It sits last on the bar, on the gold ground with red text.

The tab builds both panes into a vertical splitter. The upper one is a raw log tail: a handler formats each record and a relay paints it on the GUI thread, which lets a log call from any thread reach the pane safely.

`src/gui/qt_console_tab.py` — `ConsoleQtTab.__init__`

```
self.log_pane = QPlainTextEdit()
self.log_pane.setReadOnly(True)
self.log_pane.setFont(
    QFont(surface.PANE_FONT_FAMILY, surface.PANE_FONT_POINT_SIZE)
)
```

The tab sets its own handler to debug and changes no logger's level; an earlier build raised the root logger and never restored it.

`src/gui/qt_console_tab.py` — `ConsoleQtTab.__init__`

```
# The tab sets its own handler's level and leaves every logger level alone.
self.log_handler.setLevel(surface.HANDLER_LEVEL)
```

Pause holds lines in a bounded buffer and announces anything it had to drop.

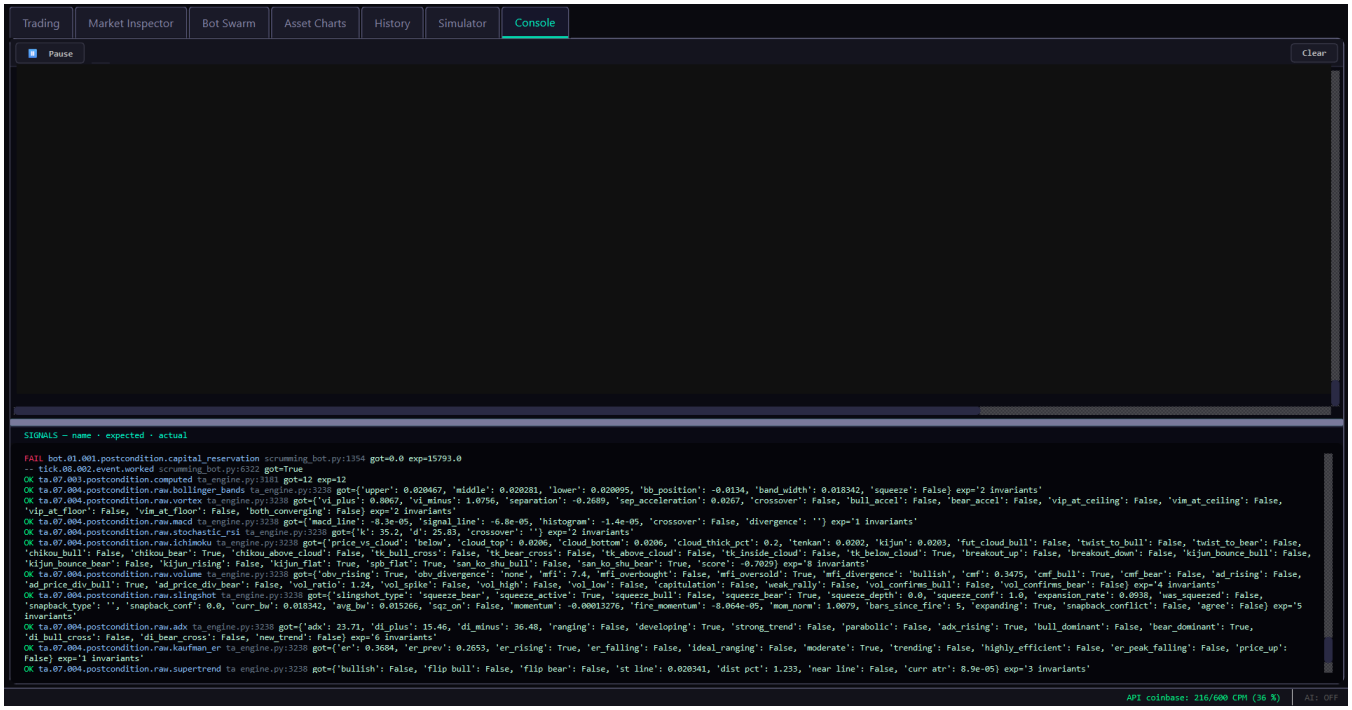
`src/gui/main_tabs/console_log_handler.py` — `_QtLogHandler.set_paused`

```
self._paused = bool(paused)
if not self._paused and self._buffer:
    for msg, r, g, b in self._buffer:
        self._append_signal.emit(msg, r, g, b)
    self._buffer.clear()
```

The lower pane is the Emitter Network. It reads the sink twice a second and renders the newest 200 records with their expected and actual values, keeping seven counters, which lets a quiet sink and a stopped timer read differently.

`src/gui/main_window.py` — `_drain_signals`

```
sink = get_sink()
if sink is None:
    return
new = sink.since(getattr(self, "_signal_seq", 0))
```



The Console tab: the log tail above, the Emitter Network below.

Pause and Clear sit above the upper pane. Pause holds lines in the bounded buffer and announces anything it had to drop when it resumes. Clear empties the pane.

Each line in the upper pane carries the time, the level, the logger name and the message, coloured by level. Three logger names appear in the figure.

React draws this tab. The choice is made in one place, under the screen name `CONSOLE`. A build stamped for Qt gets the pane set above. Every other build gets the web one, which holds the whole tab in a single view.

`src/gui/main_tabs/console_tab.py` — `ConsoleTabMixin._build_console_tab`

```
        from ..variant_surface import CONSOLE, surface_class

        tab = surface_class(CONSOLE)()
        self._console_tab = tab
        self._console = tab.log_pane
        self._signal_view = tab.signal_view
        self._console_pause_btn = tab.pause_button
        self._console_pause_indicator = tab.pause_indicator
```

Both tabs name the same six parts, so the pause, the drain and the health timer call one set of methods whichever side is built. The pause buffer, the level colours and the drop notice stay in the one handler both sides attach.

`src/gui/main_tabs/console_log_handler.py` — `_QtLogHandler.__init__`

```
    def __init__(self, text_edit, painter: Callable[[str, int, int, int], None] = None):
        """Paint into ``text_edit``, or into ``painter`` when one is given."""
```

A line reaches the page as markup the surface writes, with the message escaped, so a log message holding a tag is shown rather than read as one.

`src/gui/main_tabs/console_tab_surface.py` — `line_html`

```
  def line_html(text: Any, red: int, green: int, blue: int) -> str:
      """One console line as the markup a pane paints it with.

      The text is escaped, so a message holding a tag is shown rather than
      read as markup.
      """
```

Logger	Writes
`acervator.scrumming`	The bot warnings
`acervator.api`	The venue calls
`acervator.gui`	The panel feed

Five levels each carry a colour, and the figure shows three of them: warning, info and debug, with debug the most muted of the three. A sixth colour follows the text of the message rather than its level, and it marks the indicator panel feed.

`src/gui/main_tabs/console_log_handler.py` — the level colours

```
  COLORS = {
      "DEBUG": QColor(ds.TEXT_MUTED),
      "INFO": QColor(ds.TEXT_INACTIVE),
      "WARNING": QColor(ds.WARNING),
      "ERROR": QColor(ds.ERROR),
      "CRITICAL": QColor(ds.MAIN_LOG_CRITICAL),
  }
```

Two of those lines are worth reading against the code. The repeated warning naming a capital-reservation over-commit comes from `src/trading/capital_reservation.py`, where a claim whose total would pass the holdings is refused and the existing claims, the request and the holdings are named. The `FETCH_OHLCV` line names a twelve-indicator engine and lists all twelve indicators, which is the number the voting engine builds and the number the Indicator Voting Panel shows.

The lower pane opens with the header `SIGNALS — name · expected · actual`. Each record draws a pass marker, a fail marker or a neutral one, then the signal name, the site that emitted it, the actual value and the expected one. A satisfied expectation is recorded the same as a violated one, which keeps a call site that never ran distinct from one that always passed.

`src/gui/main_window.py` — `_drain_signals`, the three markers

```
if r.ok is True:
    mark, colour = "OK  ", ds.SUCCESS
elif r.ok is False:
    mark, colour = "FAIL", ds.ERROR
else:
    mark, colour = "--  ", ds.STATUS_NEUTRAL
```

The site is the file and the line of the frame that made the call, read off the stack at the moment of emission. It moves whenever the code moves. The numbers in the figure are the numbers those call sites held on the day of the capture, not the numbers they hold now.

`src/core/signal_contract.py` — `_caller_site`

```
def _caller_site(depth: int = 2) -> str:
    """Return `file:line` of the calling frame at the given stack `depth`."""
    try:
        f = sys._getframe(depth)
        return f"{Path(f.f_code.co_filename).name}:{f.f_lineno}"
```

The two snapshot emitters

Two symmetric records go to the bot log, and the Console tab is where an operator reads them. A blocked trade and a fired trade write the same shape, so the log holds both halves of the decision and not only the half that acted.

`src/trading/scrumming/snapshots.py` — `SnapshotEmitterMixin._emit_risk_gate_snapshot`

```
f"RISK GATE SNAPSHOT [{side.upper()}] "
f"risk_blockers={risk_blockers_sorted} "
f"ticker_last={ticker_last:.6g} "
f"panel={snapshot}"
```

The fired-side companion carries the same prefix shape.

`src/trading/scrumming/snapshots.py` — `SnapshotEmitterMixin._emit_trade_fire_snapshot`

```
f"TRADE FIRED SNAPSHOT [{side.upper()}] "
f"ticker_last={ticker_last:.6g} "
f"panel={snapshot}{extra_str}"
```

Method	Line prefix	Written when
`_emit_risk_gate_snapshot`	`RISK GATE SNAPSHOT [SIDE]`	a risk gate blocks a trade
`_emit_trade_fire_snapshot`	`TRADE FIRED SNAPSHOT [SIDE]`	a scrum or a fold actually fires

Both lines carry the last ticker price and a panel dictionary holding every voter's direction, confidence, weight and detail at that moment. The fire record adds the list of overrides that engaged, so a reader can tell an override-driven fire from a consensus-driven one without opening anything else.

The two code blocks above and the prefixes in the table are the shape these lines had before the tick-message rewrite. The current prefixes are `RISK GATE [SIDE]` and `TRADE FIRED [SIDE]`, and both write the same panel through one renderer.

Method	Line prefix	Written when
`_emit_risk_gate_snapshot`	`RISK GATE [SIDE]`	a risk gate blocks a trade
`_emit_trade_fire_snapshot`	`TRADE FIRED [SIDE]`	a scrum or a fold actually fires

The panel prints as a count and three direction groups rather than as a dictionary. A voter's weight is a configuration value that never moves between ticks, and a neutral voter's confidence is fixed at zero in the indicator code, so neither reaches the line. Direction, confidence and an indicator's own raw detail all remain.

`src/trading/scrumming/snapshots.py` — the grouped panel

```
  counts = ", ".join(f"{len(grouped[name])} {name.lower()}" for name in _PANEL_GROUPS)
  rows = [f"Panel {counts}."]
  for direction, voters in grouped.items():
      voters.sort(key=lambda one: (-one[0], one[1]))
```

True today: the measurement record is not in the tree, and the two figures it carried are kept here. The dictionary form measured 1,005 to 1,017 characters across 1,143 recorded snapshots, and the grouped form of the same panel measures 322.

Three further emitters sit in the same mixin: a trade notification carrying its own text prefix, a voting-panel snapshot at fire time, and a gate decision at fire time. The last two emit events rather than text lines, and the Console tab's signal pane reads them.

A missing capital reservation in the log

A bot claims the units it is allowed to work with so that a second bot on the same asset cannot sell them. Four places let a bot carry on when that claim is not there, and all four now write a warning naming what was lost. They appear in the upper pane at warning level.

`src/trading/scrumming/execution.py` — the one that bears on a sale

```
  logger.warning(
      "Bot %s sell of %.6f %s is NOT bounded by any other bot's "
      "capital reservation: the pre-check raised %s: %s. A sibling "
      "bot's claim on this asset is invisible to this sell.",
```

The other three name an allocation that is not held aside at admission, an Extractor whose base currency is empty, and a dollar grant made without an over-allocation check.

```
  src/trading/container/registry.py      at admission
  src/trading/extractor_bot.py           at the chunk-rate claim
  capital_registry                       at the dollar grant
```

All three of those lines have changed. `capital_registry` is removed, so no dollar grant is made and that warning can no longer be written. The admission warning in `src/trading/container/registry.py` is removed as well. It was never true. A bot claims its own capital on its first tick, through

`CapitalReservationMixin._crr`, so admission writes no claim and nothing is lost there. The Extractor writes a second warning now, described under the next heading but one.

The sale stops when the claim cannot be read

One sentence above is overtaken for the first of the four places, and it stays as written:

> *"Four places let a bot carry on when that claim is not there, and all four now write a warning naming what was lost."*

The true sentence for the place that bears on a sale: the warning is written and then the sale is refused. A check that cannot answer is not an allowance, which is what the open-orders read one screen below already does with a failure of its own.

`src/trading/scrumming/execution.py` — the refusal that follows the warning

```
self._emit_trade_notification(
    "SCRUM", "CANCELLED", "capital reservation unreadable"
)
return None
```

Driven on the real sell path with the claim read raising, twice: once on a claim table holding a quantity that is not a number, and once on a registry that could not be resolved. Both readings are the record the bot itself emitted.

```
before   SELL signal: 20.000000 @ $100.00000000 (VH:clean, MARKET)
         SCRUM SENT, venue order calls 1
after    SELL REFUSED (capital reservation unreadable)
         SCRUM CANCELLED, venue order calls 0
```

The operator sees a cancelled scrum with its reason in the upper pane, where a completed sale used to appear.

A claim file that will not read

A read can fail in two shapes, and only one of them raises. The claim file is read once, at start-up, by `_load` in `src/trading/capital_reservation.py`. A file that will not parse is caught there. The table is then empty because nothing was read, which is not the same as a file holding no claims.

The sale used to go through on that shape. The table read as empty, so `effective_available` reported the bot's whole holding as free, the pre-check passed it, and the operator saw a scrum sent.

`effective_available` now refuses to answer instead. It raises `ClaimsUnreadable`, which the pre-check one screen away already turns into a refusal. Both shapes of a failed read now end the same way.

Driven on the real sell path, with the claim file on disk holding bytes that will not parse. Both readings are the record the bot itself emitted.

```
before   SELL signal: 4.000000 @ $100.00000000 (VH:clean, LIMIT)
         SCRUM SENT, venue order calls 1
after    SELL REFUSED (capital reservation unreadable)
         SCRUM CANCELLED, venue order calls 0
```

The refusal names the file and what to do:

> *"the claim file reservation_state.json did not read at start-up, so no other bot's claim on the asset is known and an empty table here means nothing. Repair or move the file and restart"*

The writer holds back on the same shape. `_save` will not write a file it could not read, because writing it would replace the operator's claims with the few this run happens to know about. It logs an error naming the file and the count of changes that are not on the disk.

A claim that never reached the disk

The registry keeps the claim table in memory and writes it to a file. The writer is `_save` in `src/trading/capital_reservation.py`. A failed write does not stop the bot. It logs an error that names the file, the reason, and the count of changes that are not on the disk.

The line that records the claim now carries the answer as well. `CapitalReservationRegistry.reserve` logs the field `on_disk`. The line is an error when the value is False, and information when the value is True. Read the last line, not the first: a claim with `on_disk=False` exists for this run only, and the next start loses it.

A claim the bot refuses to place

The bot reads its exchange balance before it claims. The reader is `_get_cached_exchange_balance` in `src/trading/scrumming_bot.py`. The reader answers None when the venue call fails.

A balance that did not read is not a balance of zero, and it is not a licence to claim. `CapitalReservationMixin._ensure_capital_reservation` in `src/trading/scrumming/capital_reservation_mixin.py` stops on that answer. It places no new claim. It resizes no standing claim. It writes a warning that names the bot and the asset, and it tries again on the next call.

The Extractor refuses on the same answer

The Extractor reads its base currency the same way. The reader is `_read_base_holdings` in `src/trading/extractor_bot.py`, and it answers nothing when the venue call fails.

`set_initial_chunk_rate` stops on that answer. It places no claim, and it writes one warning at the upper pane that names the bot and the asset. The line reads:

```
Bot <id> could not read its <asset> balance; placing no claim, because no
holdings figure bounds it.
```

The Extractor claims once for each run, so it does not try again on a later call. A start reads the balance again.

The reader names the venue and the error

The refusal above tells you a claim was not placed. It does not tell you which venue would not answer, or what the venue said. The reader writes that itself, at warning level, so the whole failure sits on one line in this pane.

`src/trading/scrumming_bot.py` — `_get_cached_exchange_balance`

```
Bot <id> could not read its <asset> balance on <venue> (TimeoutError: the
venue balance endpoint did not answer). No holdings figure bounds a claim, so
no new claim is placed and no standing one is resized. Retrying next call.
```

Warning is the level this pane reads. The program runs its logging at information level, so a line written below that is dropped before any pane sees it. A venue that answered zero and a venue that refused used to look the same here; the line above tells them apart.

Driven with a venue stand-in whose balance call raises, the line was absent from a reading taken at warning level and present in a reading taken one level below it. It is now present in both. A neighbouring warning the same bot writes was present in all four readings, so the reading itself reports.

The bot named in a refused sale

A sale stops when another bot claims the units. The check is `effective_available` in `src/trading/capital_reservation.py`, and the caller is `src/trading/scrumming/execution.py`.

The refusal names the bots that hold the claim and the units each one holds. Stop a named bot to release its claim: `ScrummingBot.stop` calls `_release_capital_reservation`. The refusal names no page, because no page edits this table.

A claim that expires on its own clock

A bot pulses a heartbeat while it runs. A bot that stops pulsing leaves a claim that blocks every other bot on the same asset.

`prune_expired` in `src/trading/capital_reservation.py` drops a claim after HEARTBEAT_TTL seconds of silence, which is 120. `_prune_on_schedule` runs it once every HEARTBEAT_INTERVAL seconds, which is 30. Two methods call it: `effective_available`, before it answers, and `heartbeat`, after it stamps. A running bot stamps first, so it never drops its own claim. No setting changes either interval.

2026-09-23 - #34 - the drained record carries three more fields

`SignalSink.emit` in `src/core/signal_contract.py` stamps `cadence`, `budget_s` and `tab` on every record. The drain reads the same records, so every record this pane renders carries them. See system-status.md for what each field holds.

What the pane draws after the three fields land

The rendered line does not change. `_drain_signals` in `src/gui/main_window.py` reads five attributes off each record: `ok`, `name`, `site`, `actual` and `expected`. None of the three new fields is one of them.

`src/gui/main_window.py` - `_drain_signals`, the five it reads

```
view.appendHtml(
    f'<span style="color:{colour}">{mark}</span> '
    f'<span style="color:{ds.MAIN_LOG_NAME}">{_esc(r.name)}</span>'
    f'<span style="color:{ds.MAIN_LOG_SITE}"> {_esc(r.site)}</span>'
)
```

The seven counters do not change either. They count drain ticks, records read and records rendered, and no counter reads a field of the record.

2026-10-20 - #1392 - a gate refusal reads REFUSED, once a candle

The stage word

A trade notification carries a stage. `_emit_trade_notification` in `src/trading/scrumming/snapshots.py` writes it after the pair, and `stage_color` in `src/gui/main_tabs/status_log_surface.py` paints the line by reading it back out.

Three stages are passed anywhere in the tree.

Stage	What the bot did
`SENT`	an order is about to reach the venue
`FILLED`	the venue reported the fill
`REFUSED`	a gate stopped the trade and no order was sent

Ten sites used to pass `CANCELLED`. Every one of them is reached before `guarded_place_order`, so no order existed at any of them and nothing was pulled. `grep -rn '"CANCELLED"' src/trading/` now returns nothing.

`STAGE_COLORS` in `src/gui/main_tabs/status_log_surface.py` names `REFUSED`, so the pane paints it from the table rather than from `STAGE_DEFAULT_COLOR`. The colour is the same colour `CANCELLED` drew.

Two sentences and two readings above are overtaken. Quoted whole:

> *"The operator sees a cancelled scrum with its reason in the upper pane, where a completed sale used to appear."*

```
after    SELL REFUSED (capital reservation unreadable)
         SCRUM CANCELLED, venue order calls 0
```

True today: the operator sees a refused scrum with its reason in the upper pane, and the second line of both readings is `SCRUM REFUSED, venue order calls 0`. The code block one screen above, quoted whole:

```
self._emit_trade_notification(
    "SCRUM", "CANCELLED", "capital reservation unreadable"
)
```

True today, the same call:

```
self._emit_trade_notification(
    "SCRUM", "REFUSED", "capital reservation unreadable"
)
```

One line per reason per candle

`tick` evaluates every gate on every pass, and a pass happens every few seconds. On a five-minute timeframe the band position cannot move between most of those passes, so a refusal used to be announced again with identical numbers each time.

`_refusal_is_new` in `src/trading/scrumming/execution.py` decides whether an evaluation is news. It answers yes on the first call for a reason, on a candle different from the one the last call saw, when no candle is known, and when the price differs by `REFUSAL_HOLD_MOVE_FRACTION` from the price the previous call carried. Every verdict in `tick`'s scrum and fold chain now asks it before it writes anything.

The refusal itself is untouched. The gate refuses on the same inputs at the same threshold, and only the announcement is held.

Driven on the real `tick`, a bot built from invented configuration, 20 passes with the candle tape held still.

```
                                    before   after
refusal lines in bot.log                20       1
trade notifications                     20       1
venue order calls                        0       0
```

The bot sitting in a refusal stays visible. Each new candle carries one line, and a price move inside a candle re-opens the announcement on the next pass.

```
20 passes, nothing moving              before 20 lines    after 1 line
the candle rolls, nothing else moving  before 1 line      after 1 line
the band moves inside one candle       before 1 line      after 1 line
the band crosses the detect line       before 0 lines     after 0 lines, the trade fires
```

The gate's own reading was taken on a fresh bot at each of 30 band positions from 0.39 to 0.92, before and after: `_last_bb.bb_position`, `_bb_detect_thresholds` in `src/trading/scrumming/circuit_breakers.py`, whether the position is at or above the upper threshold, and the count of venue order calls. All 30 rows read the same both times. The gate closes from 0.390 through 0.873 with no order call, and opens from 0.889 through 0.920.

What is not held back

Four lines a pass are written with no guard: `DELTA`, `TA VOTE`, `BB PRIORITY SKEW` and `BULLSEYE UPPER`. They report the tick's state rather than a verdict. `FOLD HOLD (Position Ceiling)` is bounded by a counter of its own, one line per sixty passes, and `SCRUM SERVED BY STACK` and `HEDGE REBALANCE` each follow an action that happened once.

2026-10-21 - #1392 - the state lines take the same guard

One sentence above is overtaken. Quoted whole:

> *"Four lines a pass are written with no guard:* `DELTA`, `TA VOTE`, `BB PRIORITY SKEW` *and* `BULLSEYE UPPER`*."*

True today: those four lines ask `_refusal_is_new` before they write, as the verdicts do. Two more lines of the same kind went in with them. `READ` is the same report on the other side of the scrumming interval, written by the same branch of `tick`. `BULLSEYE LOWER` is the same line on the other band, written by `_tick_bullseye_notices` beside its upper twin. The rest of that sentence's paragraph stands: `FOLD HOLD (Position Ceiling)` is bounded by a counter of its own, and `SCRUM SERVED BY STACK` and `HEDGE REBALANCE` each follow an action that happened once.

What each line keys on

A refusal has one reason. A state line reports a reading that moves on every pass, so a key carrying the reading would be a new key each pass, the guard would answer new every time, and nothing would change. No key here carries a price, a confidence, a delta or a band position.

Each key is a fixed prefix plus the categorical state the line's own words already name, so a flip of that state re-announces at once.

Line	Key	Values it can take
DELTA	state_delta_<pos|neg>	2
READ	state_read	1
TA VOTE	state_ta_vote_<direction>	3
BB PRIORITY SKEW	state_bb_skew_<scrum|fold>	2
BULLSEYE UPPER	state_bullseye_upper_<wick|close>	2
BULLSEYE LOWER	state_bullseye_lower_<wick|close>	2

Twelve keys, every one from a bounded set. A key space that cannot grow with the reading cannot fall back to announcing every pass.

What it reads like now

Driven on the real `tick`, a bot built from invented configuration, with the candle tape held still.

```
                          20 passes      8 passes       8 passes
                          upper band     under interval lower band
                          before after   before after   before after
DELTA                        20     1
TA VOTE                      20     1
BB PRIORITY SKEW             20     1
BULLSEYE UPPER               20     1
READ                                        8     1
BULLSEYE LOWER                                          8     1
records per pass        [7,4,4,…] [7,0,0,…]
ta.voting events          1 each    1 each
venue order calls             0     0        0     0        0     0
```

A parked bot writes its state once and then writes nothing until something moves. The `ta.voting` event is untouched: it is a structured event the Indicator Voting Panel reads, it carries a silence budget in `src/core/emit_contracts.py`, and the run above reads it at one per pass both before and after. `bot.log` has no entry in that file and so no silence budget, which is why a quiet pass raises no fault.

The state still reaches him

```
                          before  after
the candle rolls             1       1      every line, every new candle
the next pass, same candle   1       0      the quiet one
the price moves              1       1      every line, at once
```

The gate did not move. The gate's own reading was taken on a fresh bot at each of 30 band positions, before and after: all 30 rows read the same both times, the gate closed from 0.390 through 0.873 with no venue order call and open from 0.889 through 0.920. The 35 message templates in `tick` and the 2 in `_tick_bullseye_notices` are unchanged, none lost and none gained. The skew itself still applies on every pass, because `_skewed_confidence_floor` is read above the line that announces it.

2026-10-28 - #1374 - a locked launch starts no bot and the Console says what it waits for

An optional password can seal the saved fleet. A launch that finds that password set, and holds no key for it, now draws every screen, loads no bot, and writes one Console line saying the fleet waits on the unlock. A locked launch is not an error and it writes no fault.

The three launches

`src/core/state_manager.py, in launch_classification` is the one answer every launch site reads, so no two sites can decide the lock differently. It reads the plain-text latch first, through `in latch_on`, and classifies the file through `in probe` only when that latch is set.

the saved file	the answer	what the launch does
no latch	`open`	restores the fleet and arms the start, as before
latch, key held	`unlocked`	the same, with the sealed payload opened
latch, no key	`locked`	draws every screen, loads no bot, records why

Driven on a hand-built fleet of two bots and one wire, the three launches give three different answers. Pointed at one file instead, all three give `open` and the reading reports them alike, so the three answers are a reading of the file and not a fixed result.

Where the latch is read

`main.py, in main` reads the classification before the fleet-start callback and before the fallback timer. Both the callback and the timer sit inside one branch a locked launch never takes, so no bot starts and nothing is scheduled.

Diagram: flowchart, top to bottom

- MainWindow built, every screen drawn -> **launch_classification**
- launch_classification -> **probe reads has_bots** *(open or unlocked)*
- probe reads has_bots -> **restore_bots_from_state**
- restore_bots_from_state -> **eligible bots**
- eligible bots -> **splash callback, with a 9500 ms timer behind it** *(more than none)*
- splash callback, with a 9500 ms timer behind it -> **run_fleet_sequence start**
- launch_classification -> **hold_fleet_for_unlock** *(locked)*
- hold_fleet_for_unlock -> **one Console line, no bot loaded, no timer armed**

The launch with no latch is unchanged. Measured over six saved-file shapes — no bots, one bot, two bots and a wire, nested lots and tranches, unicode and floats and nulls, and an unknown top-level field — `in latch_on`, `in probe`, `in has_saved_state` and `in load_state` answer the same through the files

before this section and the files after it. The control: one wire rate moved from 25.0 to 26.0 and the same comparison named `load_state` as different, so the six readings are a comparison that can fail.

What the Console says

`src/gui/main_tabs/main_window_surface.py` declares every line below, so the Qt build and the React build draw the same words.

when	the line
a locked launch	`The saved fleet is sealed. Every screen is open and no bot is loaded. The fleet starts when the unlock password is given.`
Start All while locked	the same line, and no bot is taken as eligible
a password that does not open	`That password did not open the saved fleet. The fleet stays sealed and no bot is loaded.`
a password that opens	`The saved fleet is open: N bot(s) loaded.`
the wires that came with it	`N Smart Wire(s) loaded with the fleet.`

`src/gui/main_window.py, in hold_fleet_for_unlock` writes the first line and latches the window, and `in run_fleet_sequence` asks `in fleet_locked` before it reads the fleet at all. Driven with one eligible bot and the window latched, the sequence answered 0, never read the bot map, and left that line in the Console. With the same bot and the window not latched it read the bot map and went on into the start path, so the latch is what stops it.

The unlock given mid-session

`src/gui/main_window.py, in unlock_fleet` takes the password, and no restart is needed. The order is fixed: `in unlock` holds the key, `in load_state` opens the payload, `in restore_bots_from_state` and `in restore_smart_wires_from_state` put the fleet in the process, and the callback `in set_fleet_start_callback` holds then runs. `main.py` registers that callback, which asks `InstanceGuard.evaluate` again with the bots the unlock loaded, because the launch asked it with a fleet of none.

Driven on the sealed fleet of two:

the password	the answer	still locked	handed over	the fleet start
wrong	False	yes	nothing	did not run
right	True	no	both bots and the wire	ran once

An unattended restart writes no fault

`src/core/instance_guard.py, in authorise_auto_start` answers `WITHHELD_FLEET_SEALED` before it reads the ownership decision and before it can ask for consent, so a machine with nobody at it is never shown a question it cannot answer. Measured with every log record of the run counted:

the run	records	faults	consent asked	files written
the fleet is sealed	1 at INFO	0	0	none
the control: consent given, no exclusive handle	1 at ERROR	1	1	—

The control is a condition that genuinely is a fault, and it reads as one through the same instrument, so the zero above is a reading.

What a locked launch does not read

A locked launch reads no price. `src/exchange/data_pool.py, in register` has one caller, `src/trading/bot_container.py, in start`, and `in refresh_all_tickers` has one caller that walks the fleet's own connectors. `src/gui/main_window.py, in _pump_currency_rates` and `in _pump_market_pairs_scout` both read `_exchange_connectors`, and every line that writes that map sits inside `in _connect_broker_for_bot`, `in _connect_written_crypto_for_bot` or `in _connect_exchange_for_bot` — each of which takes a bot. No bot, no connector, no price. A launch with an empty saved file reads no price for the same reason, so a locked launch reads what a fleet of none reads.

Not drawn yet

No screen takes the password. `src/gui/settings_dialog.py` owns that screen and it is not built, so `in unlock_fleet` is reachable from code and not yet from a keystroke.

The Product Manual PDF rebuild is owed for this page.

2026-10-28 - #1374 - the screen that takes the password is built

Where the sentences about the unlock now read differently

Two readings above describe the build before the screens landed. They stand as written, and these are the readings that replaced them.

written above	what it reads now
"No screen takes the password. `src/gui/settings_dialog.py` owns that screen and it is not built, so `in unlock_fleet` is reachable from code and not yet from a keystroke."	Two screens take it. `src/gui/settings_dialog.py, in _create_fleet_lock_group` draws the Password and Confirm Password rows that set it, and `src/gui/fleet_unlock_dialog.py, in FleetUnlockDialog` takes it back. `in unlock_fleet` is reached from a keystroke
"The Product Manual PDF rebuild is owed for this page."	`docs/Acervator-Product-Manual.pdf` is rebuilt in this branch and carries this page

Where the unlock screen appears

`main.py, in main` hands `_ask_for_unlock` to the splash's finished hook inside the branch a locked launch takes, beside `in hold_fleet_for_unlock`. The screen therefore opens over a window that already drew every tab. Dismissing it leaves the fleet sealed and the Console line in place, and the Unlock the fleet now button on the Settings User tab raises the same screen again.

Diagram: flowchart, top to bottom

- launch_classification reads locked -> **hold_fleet_for_unlock**
- hold_fleet_for_unlock -> **one Console line, no bot loaded**
- one Console line, no bot loaded -> **the splash closes**
- the splash closes -> **ask_for_unlock**
- ask_for_unlock -> **unlock_fleet, the fleet loads** *(password given)*
- ask_for_unlock -> **the fleet stays sealed** *(Not now)*

- the fleet stays sealed -> **Settings, User tab, Unlock the fleet now**
- Settings, User tab, Unlock the fleet now -> **ask_for_unlock**

What the screen was measured doing

Driven as a built widget against a sealed fleet of two bots and one wire: the Unlock button is dead on an empty row; a wrong password answered `refused`, put the Console's own refusal sentence on the screen, left the fleet sealed and changed no file; the right password answered `opened` and both bots and the wire came out of the payload in the same session.

The control for that reading: the same screen pointed at a window answering no `unlock_fleet` answered `no_window`, reported no unlock, and said so on the screen. So a reading that reports a reachable screen can also report that no screen accepts the password.

The words on the screen are the Console's words. `src/gui/main_tabs/fleet_unlock_surface.py` takes its detail line from `src/gui/main_tabs/main_window_surface.py, FLEET_LOCKED_TEXT` and its refusal line from `FLEET_UNLOCK_REFUSED_TEXT`, so the screen and the Console cannot disagree.

The rows that set the password are on the Settings page. See the Settings page.

2026-10-28 - #1374 - a locked launch reads the prices a screen draws for a market

A locked launch now reads every price a screen draws for a market, not for a bot. The venue keys are sealed with the fleet, so each of these reads goes through a route that carries no key.

Where the sentence about prices now reads differently

The reading above stands as written. This is the reading that replaced it.

written above	what it reads now
"A locked launch reads no price."	A locked launch reads three prices: the rate strip, the Charts ATA-SMP list and the Market Inspector scans. The prices that come through a bot or a venue connection wait for the unlock

The prices a locked launch reads

The dashboard tick runs every two seconds. While the window is latched, it feeds the two price screens it owns, and no other launch takes that branch.

```
if self.fleet_locked():
    self._feed_locked_price_screens()
```

`src/gui/main_window.py, in _feed_locked_price_screens` rebuilds the Charts ATA-SMP list, fetches the market on screen with no connector, and asks `in _pump_currency_rates` for the rates.

the screen	the read	the route, no key
Live tab, the rate strip, on both builds	BTC/USD and ETH/USD	`src/exchange/currency_rate_monitor.py, in refresh_from_feed`, through `src/paper/live_feed_source.py, in LiveFeedSource.ticker`
Charts, the ATA-SMP list	the candles of the market on screen	`src/exchange/chart_data.py, in ChartDataFetcher.fetch`, which reads CoinGecko with no connector

the screen	the read	the route, no key
Market Inspector, Scan Now and Scan All	each scanned market's candles	`src/gui/main_tabs/market_inspector_surface.py, in sector_candle_read`, the public Coinbase route

The Market Inspector scans read on a locked launch before this change. The rate strip and the Charts ATA-SMP list did not.

The prices that wait for the unlock

the read	why it waits
the venue page bot rows, the bulk ticker refresher, the Charts live list, the header strip, the voting panel's votes	each one reads a bot, and no bot is loaded
the market pairs scout	it reads each venue's ticker list through a venue connection, and only a bot's start opens one
Market Inspector, Refresh	its markets are the connected venue's own list, read through a venue connection

On a locked launch the Refresh line names the seal instead of asking for an exchange. `src/gui/main_tabs/market_inspector_surface.py, SEALED_NO_CONNECTORS_TEXT` holds it:

```
The saved fleet is sealed, so no bot has opened a venue connection. Give the password: Settings, User tab, Unlock the fleet now.
```

The Paper and Sim tabs read their own public routes for their own bots, and the lock seals neither of those fleets.

What the price readings were measured doing

Driven through the real dashboard tick, with every venue answered by a stand-in shaped like the public replies the code parses:

```
latched, no bot       rate strip   both prices, on the Qt panel and the React line
                      Charts       96 candles from CoinGecko
                      Scan Now     300 candles from the public route
                      Refresh      the sealed line
not latched, no bot   rate strip   pending, Charts 0 candles, Refresh asks for an exchange
before this change    latched      rate strip pending, Charts 0 candles
```

The unlatched run is the control: the same reading reports the reads that did not run. A launch with no password reads the same through the files before this section and the files after it: with a fleet and a connector the tick schedules the same rate read, and with neither it schedules nothing.

The Product Manual PDF rebuild is owed for this page.

2026-10-28 - #1374 - a sealed fleet makes no new bot

While the saved fleet is sealed, New Bot and Adopt make no bot. Each press puts one box over the window and one line in the Console, and nothing opens and nothing is written. A bot made then could not be saved, so it would trade with no record that the next launch could load.

Where the press stops

`src/gui/main_window.py, in _refuse_while_sealed` is the one place every press stops. It reads `src/core/state_manager.py, in launch_classification`, which is the answer every launch site reads and the answer under which each save is refused.

```
  def _create_bot(self, exchange_id="", defaults_override=None):
      """Open the Bot Creation Wizard; ``defaults_override`` merges in."""
      if self._refuse_while_sealed(exchange_id):
          return
```

the press	how it reaches the stop
+ New Bot, Qt build	`src/gui/widgets/exchange_tab.py, in ExchangeTab.__init__` hands it to `in _create_bot`
+ New Bot, React build	`src/gui/react_exchange_tab.py, in ExchangeTabReact._answer` hands it to `in _create_bot`
Adopt, both builds	`in _adopt_topology_proposal` asks the stop before its confirm box and before its wire snapshot

These are the only presses that make a live bot. The restore at launch and the restore at the unlock bring back the saved fleet, and neither runs while it is sealed. Paper and Sim bots live in their own fleets, and the lock seals neither.

What he sees

Both builds draw the same box, because one window holds every page in both. `src/gui/main_tabs/main_window_surface.py` declares the words.

where	the words
the box title	Unlock the fleet first
the box and the Console line	The saved fleet is sealed. A bot made now could not be saved, so no bot is made. Unlock the fleet first: Settings, User tab, Unlock the fleet now.

The last sentence takes the button name from `src/gui/main_tabs/settings_dialog_surface.py, FLEET_LOCK_UNLOCK_BUTTON_TEXT`, so the route and the button cannot disagree. The API log records one `BOT_CREATE_REFUSED` line.

What was measured

Driven with the home redirected, against a sealed fleet of two bots, through the files before this section and the files after it. The wizard, the pre-flight check and the venue connect were stand-ins, and every other part was the real one. No socket left the machine.

the press	before	after
+ New Bot, Qt build	the wizard built, 1 bot made and not saved, and its Start reached the venue connect	the box and the line; no wizard, no bot, the file unchanged
+ New Bot, React build	the same	the same
Adopt, 2 bots and 1 wire	the confirm box, a snapshot file, 2 bots and 1 wire made and not saved	the box and the line; no confirm box, no snapshot, no bot, no wire

With no password set, and after an unlock, all three presses made the same bots through both sets of files, on both builds, and saved them. The control: one target balance moved from 200 to 201, and the comparison named the bot that differed.

An unlock that opens the key and not the fleet

`src/gui/main_window.py, in unlock_fleet` now answers False when the password opens the key file and the saved fleet does not open with that key. That happens when the two files come from two different seals, or the fleet file is damaged. The window stays held, no key is held, and the Console says:

```
That password opened the key file, but the saved fleet did not open with that key. The fleet stays sealed and no bot is loaded.
```

Measured both ways on a key file from another seal. Before, the unlock answered True, the window let go of the lock, the Console said the fleet was open with 0 bots, and the fleet start ran, while a save still left the file unchanged. After, the unlock answered False, the window stayed held, and the fleet start did not run.

Where an earlier sentence now reads differently

The sentence above stands as written. This is the reading that replaced it.

written above	what it reads now
"The Product Manual PDF rebuild is owed for this page."	`docs/Acervator-Product-Manual.pdf` is rebuilt with this section and carries this page

Back to the subsystem index.

System Status Tab

Reference. The screen is not built. The tab row carries an empty tab labelled Status, and issue #34 carries the build-out. Both halves the operator's text describes run today, and this file is those two halves.

The skeleton

The tab exists and draws three lines: its name, one sentence saying it is not built, and the issue that owns it. It reads no emitter, no heartbeat and no post-mortem.

`src/gui/main_tabs/system_status_tab_surface.py` — the whole empty state

```
HEADING = "Status"
ISSUE = 34
BUILT = False
STATE_TEXT = "This tab is not built."
ISSUE_TEXT = f"Issue #{ISSUE} carries the build-out."
```

Two frontends draw that one view model. `EmptyTabsMixin` in `src/gui/main_tabs/empty_tabs.py` builds the Qt tab, and `src/gui/web/system_status_tab.js` registers a panel with the Electron shell's panel host. `src.core.desktop_bridge` serves the model under `system_status_tab.state`.

Half one: the Emitter Network

One plain function is the whole connection point. A call site says what it expected and what it actually saw, and the record goes to the sink. It is a function rather than a bus subscription, so a call site needs no reference to anything, and with no sink installed the call is a dictionary lookup and a return.

`src/core/signal_contract.py` — `emit`

```
def emit(
    name: str,
    actual: Any,
    expected: Any = None,
    ok: Optional[bool] = None,
    context: Optional[dict] = None,
    every: float = 0.0,
    instance: Optional[str] = None,
    module: Optional[str] = None,
    duration: Optional[float] = None,
) -> Optional[Signal]:
```

A satisfied expectation records the same as a violated one. That is the design point. In a log that only records failures, a call site that never ran and a call site that always passed look identical. Here they do not.

Three constants bound what the sink holds: how many rows buffer before a write, how many stay in memory, and the size at which the file rotates.

`src/core/signal_contract.py` — the sink's bounds

```
DEFAULT_FLUSH_EVERY = 200
"""Rows buffered in memory before `flush()` writes them to disk."""

RETAIN_ROWS = 350_000
"""Signal records kept in memory at once; older ones are evicted from memory but remain
in the file on disk.
"""

MAX_FILE_BYTES = 50 * 1024 * 1024
"""Byte size at which the sink's file rotates to `.1`."""
```

The function stays one; the sinks are two. A thread that calls `route_thread` sends its own emits to another sink of the same class until it calls `unroute_thread`, and `get_sink` answers per thread. The Simulator routes each run's worker thread to its own sink under the sim bucket, so a run's rows never rotate the process sink; every other thread's emits reach the process sink as before. See simulator.md.

`src/core/signal_contract.py` — the routing

```
def route_thread(sink: SignalSink) -> None:
    """Send every `emit` raised on the calling thread to `sink` until `unroute_thread`."""
```

Tracked emitters

A second module declares every emitter. A contract names a topic, the fields a payload must carry, the fields it may carry, and the vocabulary a field is allowed to draw from.

`src/core/emit_contracts.py` — the gate-decision contract

```
  EmitContract(
      topic="bot.gate_decision",
      required=("symbol",),
      optional=(
          "scrum_armed",
          "fold_armed",
          "scrum_blockers",
          "fold_blockers",
          "scrum_fixture",
          "fold_fixture",
      ),
      description="Gate evaluation snapshot at fire time.",
  ),
```

`EmitObserver` records what actually arrived and reports each violation in one of three classes: a topic that never fired, a required field that was missing, and a value outside its vocabulary.

Topic	Required	Notes
`trade.filled`	side, amount, price	The trade kind is `type` here; the persisted log row calls it `action`
`bot.gate_decision`	symbol	Scrum and fold arm state, blockers and fixture at fire time
`pnl.event`	none	Realised, unrealised, symbol
`ta.voting`	none	Voting-engine snapshot

The note on the first row is the reason the module exists. A consumer read one key from a payload that carried another. Both sides succeeded, the consumer got nothing, and a coverage report read zero while hundreds of trades flowed past it. A key-name mismatch is invisible to the producer and to the consumer; only a declared contract makes it visible.

The current reading, 2026-10-29. The fill row of the table above now requires every field of the one fill shape, with exactly one of the two fee fields, and it names the trade kinds a fill may carry. The payload is read flat, because every fill is now sent flat. A payload that carries neither fee field reads as a missing field, and one that carries both reads as a bad value.

`src/core/emit_contracts.py, in CONTRACTS` — the fill contract

```
      EmitContract(
          topic=FILL_TOPIC,
          required=SETTLED_FILL_FIELDS,
          optional=FILL_FEE_FIELDS,
          exclusive=FILL_FEE_FIELDS,
          vocab={"side": FILL_SIDES, "type": TRADE_TYPES},
          nested_key=None,
```

The observer already has a formatter that turns its counts into operator-facing lines. Its one caller was the Nuclear fleet controller writing a run report; the Simulator rebuild deleted that file, so nothing calls it and no screen reads it.

`src/core/emit_contracts.py` — `format_observer_lines`, what it returns

```
d = obs.to_dict()
out = [
    f"Emit contracts: {d['topics_seen']}/{d['topics_declared']} "
    "topic(s) observed",
]
```

Half two: the Watchdog

The Watchdog runs outside the process that can crash. It launches the application as a subprocess, tees both output streams to the console and to its own log, and polls a heartbeat file. A heartbeat whose modification time stops advancing past the stall threshold counts as a hang. When the child dies the Watchdog writes a post-mortem, which is the record an in-process hook cannot produce for a hard abort or an operating-system kill.

`acervator_watchdog.py` — the stall threshold and the poll

```
DEFAULT_STALL_SECONDS = 60  # must match or exceed Acervator's longest sync call (CCXT: ~15s typical, 30s timeout)
HEARTBEAT_POLL_INTERVAL = 2.0
```

Every post-mortem is a bundle of logs, and bundles once grew without limit. Three constants bound them now. Two govern the rotation and the third only warns.

`acervator_watchdog.py` — the rotation bounds

```
POSTMORTEM_KEEP_LATEST: int = 20  # most-recent N bundles preserved
POSTMORTEM_MAX_AGE_DAYS: int = 30  # anything older is dropped
POSTMORTEM_SIZE_WARN_BYTES: int = 5 * 1024 * 1024 * 1024  # 5 GB warning threshold
```

The Watchdog writes to files under the log root, and nothing in the application reaches into it. Surviving the application dying is the whole point of it.

Until the tab lands

The Console tab's lower pane is the Emitter Network readout. It renders the same sink records, capped at the newest 200 per drain.

2026-09-08 08:17 - #34 - what the closed issues landed

The manual marks this screen System Status Tab (To Be Built) and it is a skeleton today.

This tab consists of two distinct but closely related parts. The Emitter Network is an embedded system of data activity detectors intended to allow for detailed subsystem performance monitoring. The Watchdog is the raw signal capture for the Emitter Network’s output.

The tab is now called Status. It sits ninth on the bar, on the gold ground with red text.

The screen is not built. The tab row now carries a System Status skeleton, which draws its name, one sentence saying it is not built, and the issue that owns it. Issue #34 carries the build-out. Both halves run today, and the Console tab shows the first of them.

`src/gui/main_tabs/system_status_tab_surface.py` — the whole empty state

```
HEADING = "Status"
ISSUE = 34
BUILT = False
STATE_TEXT = "This tab is not built."
ISSUE_TEXT = f"Issue #{ISSUE} carries the build-out."
```

The Emitter Network is one plain function and one sink. A call site says what it expected and what it actually saw, and a satisfied expectation is recorded the same as a violated one, which keeps a call site that never ran distinct from one that always passed.

`src/core/signal_contract.py` — `emit`

```
def emit(
    name: str,
    actual: Any,
    expected: Any = None,
    ok: Optional[bool] = None,
    context: Optional[dict] = None,
    every: float = 0.0,
    instance: Optional[str] = None,
    module: Optional[str] = None,
    duration: Optional[float] = None,
) -> Optional[Signal]:
```

A second module declares every emitter. A contract names a topic's required fields and its vocabulary, and the observer reports a topic that never fired, a missing field or a value outside that vocabulary.

`src/core/emit_contracts.py` — the gate-decision contract

```
EmitContract(
    topic="bot.gate_decision",
    required=("symbol",),
    optional=(
        "scrum_armed",
        "fold_armed",
        "scrum_blockers",
        "fold_blockers",
        "scrum_fixture",
        "fold_fixture",
    ),
    description="Gate evaluation snapshot at fire time.",
),
```

The Watchdog is `acervator_watchdog.py` at the repository root. It launches the application as a child process, tees both output streams to its own log, polls the heartbeat file, and writes a post-mortem after the child dies.

`acervator_watchdog.py` — the stall threshold and the poll

```
DEFAULT_STALL_SECONDS = 60  # must match or exceed Acervator's longest sync call (CCXT: ~15s typical, 30s timeout)
HEARTBEAT_POLL_INTERVAL = 2.0
```

Post-mortem bundle rotation

The Watchdog writes a post-mortem bundle every time the application dies. Each bundle copies the runner's console log, the crash log, the fault-handler log and the thread-violation log. The console log runs to hundreds of megabytes. A hang writes two bundles: one while the child is still alive, and one after the Watchdog has stopped it. Per-run log rotation capped the size of one run's logs and capped nothing about the number of bundles, so weeks of restarts grew without limit until the log directory took the host down.

`acervator_watchdog.py` — `prune_postmortem_bundles`

```
def prune_postmortem_bundles(
    keep_latest: int = POSTMORTEM_KEEP_LATEST,
    max_age_days: int = POSTMORTEM_MAX_AGE_DAYS,
    log_dir: Path | None = None,
) -> tuple[int, int]:
```

It keeps the most recent bundles, drops anything past a maximum age whatever its position, and runs at two sites: once at Watchdog startup, which clears what earlier runs left behind, and once after each post-mortem is written, which holds the ceiling during a long session.

`acervator_watchdog.py` — the three constants

```
POSTMORTEM_KEEP_LATEST: int = 20  # most-recent N bundles preserved
POSTMORTEM_MAX_AGE_DAYS: int = 30  # anything older is dropped
POSTMORTEM_SIZE_WARN_BYTES: int = 5 * 1024 * 1024 * 1024  # 5 GB warning threshold
```

The third is a warning threshold read by `report_log_dir_footprint` and it deletes nothing, so two constants govern the rotation and one reports on it. The only other inputs are the log directory itself, derived from the home directory, and the literal bundle-name prefix inside the function. Neither is a rotation constant.

No test in this repository references any of the three names. The cap runs and nothing holds it in place.

2026-09-09 14:30 - #34 - the tab draws the emitter network by subsystem and by tab

The screen is built. The Status tab no longer draws the empty state; it draws the emitter network, divided two ways.

The top of the screen names the feed: the file the sink writes, how many records it has taken this run, how many are still in memory, how many were evicted and how many were dropped. Under it, one strip of totals across the whole network. With no sink in the process the feed line reads one sentence instead, and every count falls to zero.

`src/gui/main_tabs/system_status_tab_surface.py` — the tab a subsystem's emitters serve

```
TAB_BY_SUBSYSTEM = {
    "apitest": ENGINE_GROUP,
    "bot": ENGINE_GROUP,
    "charts": "Charts",
    "console": "Console",
    "exchange": "Live",
    "extractor": ENGINE_GROUP,
    "fleet": "Sim",
    "gui": ENGINE_GROUP,
    "history": "History",
    "instance": ENGINE_GROUP,
    "sim": "Sim",
    "swarm": "Swarm",
    "ta": ENGINE_GROUP,
    "tick": ENGINE_GROUP,
    "topology": ENGINE_GROUP,
    "trading": "Live",
    "ytd": ENGINE_GROUP,
}
```

By subsystem is one panel per prefix, seventeen of them. Each panel carries a light, the tab it serves, six counts, and a table of that subsystem's own emitters with the cadence each one declares, how many times it fired, how many of its retained records failed, and the last value it reported.

By tab is one row per tab on the bar, ten of them, and an eleventh named Engine for the emitters that serve no single screen. A row names the subsystems it holds and adds their counts together. Paper, Inspector, Accumulation and Status hold no subsystem at all, and each says `no emitters` rather than drawing a light. That absence is the reading, not a gap.

Green, Yellow, and why Red cannot show

Two lights, and the screen says so in its own legend.

`src/core/signal_contract.py` — the two states the report returns

```
HEALTH_STATES = (HEALTH_GREEN, HEALTH_YELLOW)
"""The states `subsystem_health` returns; `HEALTH_RED` is not one of them."""
```

Green means every always-on emitter the subsystem declares has fired and no retained record of it failed. Yellow means a record failed, or an always-on emitter never fired at all. A subsystem that has recorded nothing carries no light and reads `no records this run`, which keeps a quiet subsystem separate from a healthy one.

Red waits on issue #14. A hung subsystem sends nothing, so silence is the only sign of it, and silence can only be read against a rhythm each emitter is expected to keep. No emitter declares one, so nothing returns Red and no light on this screen can show it.

The issue #14 clause above is overtaken. It is not edited and not removed. It read:

Red waits on issue #14. No emitter declares one, so nothing returns Red and no light on this screen can show it.

Issue #14 closed on 2026-09-01, and the rhythm it asked for is declared. Every emitter carries a cadence category, and an always-on pin has an age above which it reads late.

```
src/core/signal_contract.py

CADENCE_ALWAYS_ON          "always_on"
CADENCE_TOGGLE             "toggle"
CADENCE_CATEGORIES         the two above
ALWAYS_ON_STALE_AFTER      10.0 seconds
ALWAYS_ON_WORST_HEALTHY_GAP   4.736 seconds
in cadence_verdict         answers on_time, stale or late per pin
```

The second half of the clause still holds. `HEALTH_RED` is declared and is not one of the states `subsystem_health` returns, so no light on this screen shows Red. Whether a late always-on pin should turn a subsystem Red is a product choice, so nothing here proposes one.

What the log record carries

The persisted row now names the subsystem it belongs to, so the report and the log divide the network the same way.

`src/core/signal_contract.py` — `Signal.to_json`

```
        payload = {
            "ts": self.ts,
            "seq": self.seq,
            "module": self.module,
            "name": self.name,
            "subsystem": subsystem_of(self.name),
            "kind": self.kind,
```

`subsystem_of` is the one function that splits a name, and `SignalSink.by_subsystem` calls it too, so the row on disk and the panel on screen cannot disagree.

Measured on this build

The registry declares 78 emitters across 17 subsystems. The issue body's figure of 70 is older than the registry.

```
declared emitters          78
  always-on                16
  toggle                   62
subsystems                 17
no call site in src/       27    fleet 8, sim 14, ta 2, ytd 3
```

The twenty-seven with no call site can never fire, so their subsystems read silent whatever the platform does. Two of them are always-on, which is why Sim cannot reach Green today. Those emitters belong to the Simulator rebuild and the emitter network item, not to this screen.

What is still open

The Console tab keeps its Emitter Network pane; moving it here is not done. The Watchdog is not on this screen yet. Both are named in issue #34 and neither is built.

Back to the subsystem index.

Where the Status tab draws

The window builds this panel under both builds. `SystemStatusTabMixin` appends it with no variant seam, because no Qt widget ever drew this tab. The two builds draw the same picture at every width.

```
class            SystemStatusReactPanel
accessible name  Status
module loaded    system_status_tab
style sheet      system_status_tab.css, 25 rules
drawn            17 subsystems, 11 tab groups, 78 emitter rows
```

2026-09-20 06:18 - #34 - the register equals the code

The register in `src/core/signal_contract.py` now names every emitter the code calls, and only those. Every name under `src/` that reaches `emit` has an entry with its kind, and every entry has a call site. The tab map lives in the same module.

`src/core/signal_contract.py` - one entry per group, with its reason

```
_ALWAYS_ON_PINS: dict[str, str] = {
    "charts.indicators.drawn": (
        "the same `fetch_chart_data` pass calls `set_candles` on every fetch "
        "that returns candles"
    ),
```

```
_TOGGLE_PINS: dict[str, str] = {
    "inspector.ata.scan_pressed": "the operator presses Scan Now or Scan All",
```

An entry is a name and one line. The line is the reason the kind was read at the call site, in the form the retired roster used. The two groups keep their kinds: a loop or a timer reaches an always-on pin on every pass; a toggle pin needs a trigger, and silence from it is normal. `CADENCE_BY_NAME` is built from the two groups as before.

The tab map in the register

The tab a subsystem serves is read from `TAB_BY_SUBSYSTEM` and `tab_of` in `src/core/signal_contract.py`, beside `subsystem_of`. The map names `inspector` to Inspector and `status` to Status. The surface imports the map and the function; it holds no copy.

`src/core/signal_contract.py` - `tab_of`

```
def tab_of(subsystem: str) -> str:
    """Return the tab `subsystem`'s emitters serve, or `ENGINE_GROUP` when none does."""
    return TAB_BY_SUBSYSTEM.get(subsystem, ENGINE_GROUP)
```

`src/gui/main_tabs/system_status_tab_surface.py` - the import

```
from src.core.signal_contract import (
    ENGINE_GROUP,
    HEALTH_GREEN,
    HEALTH_STATES,
    HEALTH_YELLOW,
    TAB_BY_SUBSYSTEM,
    get_sink,
    subsystem_health,
    tab_of,
)
```

What entered and what left

Thirty-six names entered: 7 `charts.*`, 15 `inspector.*`, 14 `sim.*`. One of them, `charts.indicators.drawn`, is always-on: the 2000 ms dashboard timer schedules `fetch_chart_data`, and that pass calls `set_candles` on every fetch that returns candles. The other thirty-five need a press, a pointer, a call, or a run the operator starts.

Twenty-seven entries left, each with no call site under `src/`: every `fleet.03.*` entry (8), every `sim.06.*` entry (14), `ta.07.001` and `ta.07.002`, and every `ytd.10.*` entry (3). Two of them were always-on, `sim.06.011.postcondition.price_chart.fed` and `sim.06.012.postcondition.gate_status.rendered`, and they held the Sim panel at Yellow with `always-on silent 2`.

Overtaken sentences on this page

Each sentence below stands as written above. The sentence under it is the current reading.

> `src/gui/main_tabs/system_status_tab_surface.py` — *the tab a subsystem's emitters serve*

The map is in `src/core/signal_contract.py`, as `TAB_BY_SUBSYSTEM`, with `inspector` and `status` added.

> ***By subsystem*** *is one panel per prefix, seventeen of them.*

Sixteen. The `fleet` and `ytd` prefixes hold no entry after this change, and `inspector` holds fifteen.

> *Paper, Inspector, Accumulation and Status hold no subsystem at all, and each says* `no emitters` *rather than drawing a light.*

Inspector holds `inspector`. Paper, Accumulation and Status still say `no emitters`.

> *The registry declares 78 emitters across 17 subsystems.*

The register declares 87 emitters across 16 subsystems.

> *The twenty-seven with no call site can never fire, so their subsystems read silent whatever the platform does. Two of them are always-on, which is why Sim cannot reach Green today.*

Those twenty-seven are gone. No entry lacks a call site, and the Sim panel's `always-on silent` reads 0.

> *drawn 17 subsystems, 11 tab groups, 78 emitter rows*

Drawn: 16 subsystems, 11 tab groups, 87 emitter rows.

Measured on this change

Read off the running program in both variants, on a scratch home with every socket but loopback refused, the tab shown, the page read at the widget.

```
declared emitters          87
  always-on                15
  toggle                   72
subsystems                 16
names in code not declared  0
entries with no call site   0
drawn                      16 subsystems, 11 tab groups, 87 emitter rows
Inspector tab row          inspector
Sim panel always-on silent 0
```

Before the change the same reading gave 17 subsystems, 80 rows on a page where two undeclared names had already fired at build, `always-on silent 2` on Sim, and `no emitters` on the Inspector row. One Scan Now on the loopback stand-in then raised an `inspector` panel with 11 undeclared rows under Engine. After the change the same press fills the Inspector panel: 15 declared, 11 fired, 0 undeclared, and the panel sits under the Inspector row.

The page still draws once, when the window builds. A Status tab opened after a scan shows what the sink held at build. The redraw is a later unit of #34.

2026-09-23 - #34 - the record carries its kind, its budget and its tab

`SignalSink.emit` in `src/core/signal_contract.py` stamps three more fields on every record it builds. `Signal.to_json` writes them to the line on disk. `read_records` restores them. A line on `session.jsonl` is eighteen fields. A reader can now judge a silence from the line alone. Before this change a reader had to import the register of the build that wrote the line.

field	what it holds	the function that gives it
`cadence`	`always_on` or `toggle`, empty for a name the register does not hold	`cadence_of`
`budget_s`	the seconds an always-on emitter may stay silent, empty for every other emitter	`always_on_stale_after`
`tab`	the tab the emitter's subsystem serves	`tab_of`

`src/core/signal_contract.py` - `SignalSink.emit`, the stamp

```
                _cadence, _tab = self._declared_for(_name)
                sig = Signal(
                    cadence=_cadence,
                    budget_s=(
                        always_on_stale_after(_window)
                        if _cadence == CADENCE_ALWAYS_ON
                        else None
                    ),
                    tab=_tab,
                )
```

`_window` is the widest `every=` window the sink holds for that emitter's identity. An emitter with no window keeps the flat budget, `ALWAYS_ON_STALE_AFTER`, ten seconds. An emitter with a window keeps

ALWAYS_ON_SLACK times that window, which is 2.11 times. A toggle keeps no budget. _declared_for reads the register once per name and keeps the answer, so the register is not walked per record.

What a restored record answers

read_records reads each of the three through its own check: _as_cadence admits only a member of CADENCE_CATEGORIES, _as_budget admits only a finite number above zero, and _as_tab admits only a string with text in it. A value outside a check restores empty and the record is kept, the way a bad dt or a bad duration is kept today. A line written before this change carries none of the three, so all three restore empty and every other field restores as before.

src/core/signal_contract.py - read_records, the restore

```
                        cadence=_as_cadence(d.get("cadence")),
                        budget_s=_as_budget(d.get("budget_s")),
                        tab=_as_tab(d.get("tab")),
```

Overtaken by the record's three new fields

The sentence below stands as written above. The sentence under it is the current reading.

> *The persisted row now names the subsystem it belongs to, so the report and the log divide the network the same way.*

The row names the subsystem, the emitter's kind, the emitter's budget in seconds and the tab the subsystem serves. Four of the row's eighteen fields come from the register rather than from the call, and the Signal.to_json block above carries three more keys after context.

Read on this change

Read off the running program on a scratch home, with every socket but loopback refused, and on a read-only copy of a rotated file from the operator's own log root.

```
a line on session.jsonl          18 fields, 25 of 25 lines
a line on session.digest.jsonl   19 fields, the three plus folded
an always-on emitter, no window  ta.07.003.postcondition.computed  budget_s 10.0
an always-on emitter, 30 s window console.14.001.invariant.records_rendered
                                                    budget_s 63.34459459459459
a toggle                         trading.12.001.postcondition.tab_assembled
                                                    budget_s empty, tab Live
the operator's rotated file      92,500 lines, 92,500 records restored,
                                 all three empty on every record
```

Before the change the same reading gave fifteen fields on every line and no cadence, budget_s or tab on any of them.

The panels do not read the three fields yet. subsystem_health still counts silence since launch, so the light is still Green or Yellow and never Red. Reading a budget off the record is a later unit of #34.

2026-09-23 - #34 - the qt bundle ships the shell renderer

This tab draws through a React page, and the page host sits in the renderer folder beside the application. A qt build shipped no copy of that folder. The tab then raised a missing shell asset and drew an empty pane. Every build now ships the folder.

`tools/spec_common.py` - the pair every build copies

```
def renderer_candidate(project_root: str) -> tuple[str, str]:
    """The (source, destination) pair of ``SHELL_RENDERER``, which every variant ships."""
```

`shell_candidates` now holds what the Electron shell alone needs: the three shell files and the Electron runtime. A React build ships those, and no other variant does.

The overtaken sentence about the bundle

The sentence below stands as written above. The sentence under it is the current reading.

> *The window builds this panel under both builds.*

The window builds this panel under both builds. Before this change only a React bundle carried the page host, so only a React bundle could draw the panel. Both bundles carry it now.

Read off the launched bundles

Both variants were built from one commit and launched with no network: a scratch home, an application container with no capability, and the name resolver mapped away. The Status tab was pressed through its accessibility action. The page was read off the accessibility tree.

```
qt bundle before     no desktop folder in the bundle
                     1 missing-asset line in system.log
                     0 panels, 0 emitter rows, empty pane

qt bundle after      desktop/renderer, 8 files, 38,592 bytes
                     0 missing-asset lines in system.log
                     16 panels, 87 emitter rows, 11 tab rows

react bundle after   desktop, 11 files: 3 shell files and the renderer
```

A build that names a renderer folder which is not on disk ships no renderer, and its Status tab raises the missing asset again. That is how the reading is known to be able to fail.

The crash log and the five writers that fill it

`~/.acervator_logs/crash_<date>_<time>.log` is the file that holds a fault. One file per run. The Watchdog copies it into every post-mortem bundle, beside the console log and the fault-handler log.

Five writers reach it. `main.py` installs all five at import, before the window exists.

writer	what reaches it	installed at
`sys.excepthook`	an exception no `except` caught	`main.py`
`threading.excepthook`	the same, on a worker thread	`main.py`
`sys.unraisablehook`	a failure the interpreter cannot propagate	`main.py`

writer	what reaches it	installed at
the Qt message handler	every message Qt emits	`main.py`
`_CrashLogHandler`	every log record at ERROR and above	`main.py`

The fifth is the one that sees a fault the other four cannot. A broad `except` that reports through `logger.error` and returns reaches no excepthook at all. `logging_engine` clears `acervator.propagate`, so the handler attaches to two logger names.

`main.py` — the two attach points

```
# Two disjoint attach points: `logging_engine` clears `acervator.propagate`.
CRASH_LOG_HANDLER_LOGGERS = ("", "acervator")
```

A record that carries no exception of its own is written with the exception live at the call site, labelled `exception live at log time`. A record that carries one is labelled `exception declared on the record`.

A fault that cannot be printed is still named

`traceback.format_exception` reads each frame's source line. It raises when the module it needs is gone, and an exception whose own `__str__` raises defeats it as well. Both happened on the operator's build, and the handler died without writing.

`_crash_record` names the exception type and its message first, through `_safe_text`, and adds the frames second. The type and the message therefore reach the file when no frame can be read. A frame that cannot be read is written as `<frame unreadable>` and the walk continues past it.

A repeating entry is counted, not repeated

An entry equal to the one before it is counted. The count lands as a `REPEAT` entry when a different entry arrives, at `CRASH_REPEAT_RELEASE_AT`, or at exit. The first copy of the entry is always written immediately.

```
[..] [QT_WARNING] [thread=MainThread] DirectWrite: CreateFontFaceFromHDC() failed ...
[..] [REPEAT]     [thread=MainThread] previous QT_WARNING entry repeated 58 more times
```

No category is silenced and no distinct entry is dropped. One font warning that repeats for the life of the process costs two lines instead of hundreds.

Read off the crash log

One crash log per run, read back off disk. A fault whose message cannot be rendered, raised through the installed `sys.excepthook`:

```
                       crash log lines    the exception named in it
before                        2           no
after                        16           yes
```

The same fault caught by a broad `except` and reported with `logger.error`:

```
before                        2          no
after                        15          yes
```

Fifty-nine copies of the font warning, then the fault:

```
before                       63          no    (59 of the 63 are the warning)
after                        19          yes   (1 warning, 1 REPEAT line)
```

The `before` column is the operator's own file: 63 lines, 59 of them the font warning, three boot lines, one `QFont::setPointSize` warning, and no crash.

The hardest case, with the frame source cache broken and the message unrenderable at once, so nothing about the exception can be formatted:

```
before                        2          no
after                         9          yes   (all three frames named)
```

A fault reported below ERROR reaches none of the five

`_CrashLogHandler` is attached at ERROR, so it is the fifth writer only for a record at ERROR or above. A broad `except` that reports the same fault with `logger.debug` builds a record no writer reads, and the crash log gains nothing.

The second place an exchange tab's bot rows are refreshed is the privacy-widget sweep, which the privacy-mode button on each exchange tab runs. Its own `except` reported at DEBUG. It now hands the failure to a throttled fault, which reports the first failure at ERROR and names what the operator loses.

`src/gui/main_window.py` — the two faults the sweep reports through

```
        _tab_bots_fault = ThrottledFault(
            "the exchange tab bot list",
            "Every bot row on each exchange tab keeps its last figures and "
            "stops refreshing.",
        )
        _tab_privacy_fault = ThrottledFault(
            "the exchange tab privacy button",
            "The privacy button on each exchange tab stops following the "
            "privacy mode it is set to.",
        )
```

The throttle is the same one the window already uses for its currency-rate and market-pairs pumps. The first failure reports and opens a thirty-second window; further failures inside that window are counted and not written, so a sweep that keeps failing costs one line rather than one per press.

Read off the crash log, the privacy sweep

The shipped method driven five times with one exchange tab whose bot-row refresh raises, and a second tab beside it whose refresh succeeds:

```
                        crash log lines    the fault named in it
before                         0           no
after                          9           yes
reports emitted                 1          from five failures
neighbour tab refreshed         5 times     in both readings
```

The fault reaches the file as a `LOG_ERROR` entry naming the feature, what is lost, the retry window and the cause, followed by the frame inside `refresh_all_privacy_widgets`.

A stopped work loop names the restart it needs

The platform runs its venue reads on one background work loop. A fifty- millisecond timer advances that loop, and the slot that drives it is `main.py, in _make_async_pump_timer`. When the call that queues the loop's next step raises, the loop is not advanced, every bot stops reading its venue, and nothing a running platform does recovers it.

`main.py, in PumpStallReport` is what the slot now reports through. The first raising pump writes one line naming what stopped and what is lost. Further raising pumps inside the thirty-second window are silent. Once the run of raising pumps has lasted the venue-stall span, one further line names the restart.

```
BACKGROUND WORK STOPPED - the loop that runs every bot's venue reads was not
advanced. Every bot stops reading its venue and every figure on screen keeps
its last value. Cause: TypeError: __init__() should return None, not 'NoneType'

RESTART NEEDED - the loop that runs every bot's venue reads has not advanced
for 1 minute, over 1193 pump(s). Nothing recovers it while the platform runs.
Close the platform and start it again. Cause: TypeError: __init__() should
return None, not 'NoneType'
```

The span is the one already in the tree. `STALL_THRESHOLD_SECONDS` is the same sixty seconds the Exchange sub-tab's red line reads, so the Console and that line cannot disagree about when a stop has begun. The window between reports is `DEFAULT_BASE_COOLOFF_S`, the thirty seconds every other feature fault uses.

Read off the crash log, the background work loop

The real pump slot fired every fifty milliseconds for seventy seconds, in a failing arm and a healthy arm of the same run, with an ordinary queued callback in both arms read the same way.

```
                                  before      after
crash log lines                    37912        126
Console lines                       1354          7
lines naming a restart                 0          1
queued callbacks that ran, failing     0          0
queued callbacks that ran, healthy  1390       1390
```

The healthy arm is the control. It ran for the same seventy seconds, past the sixty-second span, and wrote nothing in either reading, so the lines above are a reading of the stop and not of the pump's ordinary work. The failing arm's queued callback never runs in either reading: this names the stop, it does not prevent it.

The two signatures one state produces

The crash record for the recorded run holds two different failures. Driven on this interpreter, both come from one state: an object whose type is named `NoneType` and which an `is None` test refuses.

```
TypeError: __init__() should return None, not 'NoneType'
    raised where CPython compares what __init__ answered against its own None

AttributeError: 'NoneType' object has no attribute 'partition'
    raised in the standard library's traceback reader, one line below its own
    is-None guard
```

With the interpreter's own None both sites behave: a constructor raises nothing and the traceback reader answers None cleanly. So the two signatures are one fault seen twice, and the exit dialog's missing-callable message is that same state at a third site.

Overtaken by the pump's own report

The sentence below stands as written above. The sentence under it is the current reading.

> *One font warning that repeats for the life of the process costs two lines instead of hundreds.*

That holds for a warning that repeats with nothing between it. A raising pump wrote an `ASYNCIO` entry and a `LOG_ERROR` entry alternately, so no entry ever equalled the one before it and the count never fired: 1,354 raising pumps wrote 37,912 lines in seventy seconds. The pump now reports once per window, so the same seventy seconds cost 126 lines.

What this does not name

The state itself is not produced by any Python in this repository. Counted across the source tree and the entry point: classes named `NoneType`, nothing. Application imports of the two installed packages that define one, nothing, against sixteen for the exchange library as a control. Calls that hand raw object pointers to a C library, nothing. Across the 699 installed extension modules the only interpreter runtime named is `python314.dll`, on 591 of them, with a name that must appear reading 699 and a name that must not reading zero.

So nothing below the report is repaired here, and the report is what a restart is now made for.

The heartbeat file and the root that holds it

The Watchdog half above says what the heartbeat is for. It polls the file, and a modification time that stops advancing past the stall threshold counts as a hang. The file is `heartbeat.txt`, one per log directory. It holds a single line: a timestamp and the process id of the run that wrote it.

`main.py` — the line written on every tick

```
f.write(f"{datetime.now().isoformat()} pid={os.getpid()}\n")
```

The line is rewritten every two seconds for the life of the process. The modification time is the signal the Watchdog reads. The contents only name which run stamped it.

`main.py` — the timer that drives the write

```
heartbeat_timer.start(2000)
```

The directory is `~/.acervator_logs/`, the same root that holds the crash log, the fault-handler log and the stale-binary marker. One environment variable moves all four. Set `ACERVATOR_CRASH_LOG_ROOT` to a directory and every one of them is written there instead.

`main.py` — the resolver the four writers share

```
_override = os.environ.get(CRASH_LOG_ROOT_ENV)
return _P(_override) if _override else _P.home() / ".acervator_logs"
```

An empty value counts as unset, so the root falls back to the home directory. A run that redirects the home directory carries the heartbeat with it, whether the variable is set or not.

Read off the resolved path

The path the shipped resolver returns, read in three environments:

```
home            variable        heartbeat written to
scratch A       unset           scratch A / .acervator_logs
scratch A       scratch B       scratch B
real home       scratch B       scratch B
```

Every log directory the override moves

The four writers named above are not the whole set. The resolver those two lines show now lives in `src/core/log_paths.py`, under the name `resolve_log_root`, and the entry point calls it there. Thirteen things follow it.

The four entry-point files:

```
crash_<timestamp>.log            the crash log
faulthandler_<timestamp>.log     the fault-handler log
STALE_DIST_WARNING.txt           the stale-build marker
heartbeat.txt                    the file the Watchdog polls
```

The nine bucket directories, one per bucket function:

```
activity/           get_activity_dir
api/                get_api_dir
console/            get_console_dir
trade/              get_trade_dir
trade/pnl/          get_pnl_dir
exchange_history/   get_exchange_history_dir
_meta/              get_meta_dir
reports/            get_reports_dir
sim/                get_sim_dir
```

The variable is `ACERVATOR_CRASH_LOG_ROOT`. Set it to a directory and all thirteen are written there. An empty value counts as unset.

With the variable unset every one of the thirteen resolves under the home tree, at the same place it resolved before the buckets were wired to the resolver. The trade log, the gate log and the daily profit files do not move unless the variable is set.

The root is read on each call, not once when the module loads. A run that sets the variable after the module has been imported still moves all nine buckets.

Read off the resolved paths, all nine buckets, in two environments:

```
variable        where the nine resolved         entry point agreed
unset           home / .acervator_logs / *      yes, 9 of 9
scratch B       scratch B / *                   yes, 9 of 9
```

2026-10-04 - #410 - the periodic collection reads the young objects on eleven ticks in twelve

Python's own automatic memory collector is switched off at startup. A timer on the thread that draws the window collects in its place, every five seconds. That has not changed, and the switch-off has not changed. What changed is how deep each collection reads.

A collection has three depths. The shallowest reads only the objects made since the last collection. The deepest reads every live object the program holds, and so touches every page of memory the program occupies. Until this change every tick ran the deepest one, twelve times a minute.

Now eleven ticks in twelve run the shallowest read, and the twelfth runs the deepest. A collection still happens every five seconds, so nothing is collected less often than before. One full read a minute replaces twelve.

`main.py` - the depth each tick reads

```
def periodic_gc_generation(tick: int) -> int:
    """Return the gc generation the periodic collection reads on tick, counted from 1."""
    return 2 if tick % GC_FULL_EVERY_TICKS == 0 else 0
```

The timer still belongs to the thread that draws the window, so the collector still never runs on an exchange worker thread. That is the whole reason the automatic collector is switched off, and it is untouched.

Measured on the collection depth

Both sides were driven on one machine, on a heap holding 343,283 tracked objects built by importing all 487 modules under `src`, with the collector's own `gc.DEBUG_STATS` report supplying the object counts and `time.perf_counter` the clock. Each side ran one twelve-tick round, which is one minute of the real timer.

```
                    seconds per collection      objects read per collection
before              0.079518 mean               352,491 mean
                    0.090910 worst              352,502 worst

after               0.015458 mean                37,630 mean
                    0.081259 worst              352,503 worst

per minute          0.954211s  ->  0.185494s    4,229,892  ->  451,568
```

The deepest read costs the same as it always did; the worst single tick is that read, and it is unchanged. What falls is how many times a minute the program pays for it. Objects read per minute falls by 9.37 times, and that figure is the one the page-fault rate follows. The seconds column moves with whatever else the machine is doing - the same pair of rounds read a ratio of 4.05, 4.64 and 5.14 across three runs with the live platform sharing the machine - while the objects column read 9.37 every time.

The timer was then driven through a real Qt event loop for twenty-four ticks. Twenty-two shallow reads and two deep ones ran, `gc.get_stats()` reported twenty-four collections, every one of them ran on the thread that draws the window, and the automatic collector stayed off throughout.

A third depth, read every fourth tick, was measured and left out. It held the same number of objects as leaving it out did - a peak of 126,004 above a quiet heap either way - and cost more time, so it earns nothing.

`gc.set_threshold` cannot be part of this. While the automatic collector is switched off the thresholds are never read. Measured: the most aggressive threshold the interpreter accepts, then 20,000 unreachable reference cycles made, gives **0** collections with the collector off and **858** with it on.

What this does not prove

The crash the switch-off guards against cannot be shown to stay away by any reading taken here. That needs his own machine, his own fleet, and hours of running. What to watch for: a hard exit with no traceback, and a new file under `~/.acervator_logs/` named `faulthandler_*.log` naming a thread that is not the one drawing the window.

The deepest read now runs once a minute, so reference cycles that outlive one shallow read wait up to a minute instead of five seconds. Measured on a heap driven at 9,000 such objects every five seconds, the peak held 126,004 objects above a quiet heap against 26,998 before. Objects not in a reference cycle are freed the instant nothing points at them, and that is untouched. What to watch for: a resident set that climbs through a minute and does not fall back.

2026-10-17 - #62 - the ladder weighs rarity, and a topic's silence has a reading

The signal ladder evicted by position alone, so the newest routine reading pushed out the oldest rare one. Three bounds now sit beside the size bound, and a bus topic's silence has a declared reading.

The loudest writer declares a window

`src/trading/ta_engine.py`, in `compute_all`, holds the two loudest emitters on the ladder. The first fires once per call. The second fires once per indicator per call. Both now pass a window and a market key.

`src/trading/ta_engine.py` — the window

```
  TA_EMIT_EVERY = DIGEST_MIN_INTERVAL
```

The window is the digest ladder's own admission window, imported rather than copied, so one figure cannot become two. `instance` keys the window per market and timeframe, so one market's reading never suppresses another's. A suppressed observation is folded into the next admitted record's `count`, and `emit` never suppresses a failing check.

Driven on a 260-candle tape with a sink under a temporary home: one compute admitted 13 records, the same market computed again admitted 0, a second market admitted 13 of its own, and 13 returned after `reset_throttle`.

An age bound beside the size bound

`src/core/signal_contract.py`, in `SignalSink._retire_aged`, unlinks a rotated backup whose last write is older than the bound.

```
MAX_FILE_AGE_SECONDS = 14.0 * 24.0 * 3600.0
```

Fourteen days. The sweep runs on every `_roll` call, so a ladder below its size bound still retires its tail instead of holding it for ever. It reaches backups only. The live file is never unlinked, renamed or truncated, so no handle the sink holds is disturbed, and a backup that cannot be unlinked is left alone and not counted.

This bound does not widen the ladder's time window. It bounds the footprint in time. The window is widened by the rate limit and the share bound.

Driven with five planted backups, three of them past the bound: the three went and `age_retired` read 3. With every backup inside the bound, none went and the counter read 0. With the bound set to `0.0`, the same three aged backups stayed and the counter read 0.

A per-emitter share of one file

`src/core/signal_contract.py`, in `SignalSink._admitted_lines`, charges each record's bytes to its identity and refuses a passing record from an identity already at its share.

```
MAX_IDENTITY_SHARE = 0.10
```

Ten distinct identities are needed to fill one file at this share, so no single writer can crowd a file alone. The charge resets when the file rotates. Two records are never refused: one whose check failed, which is the reading worth finding, and one from a new identity past `max_identities`. A refused record's identity stands on the digest ladder, where its next admitted line carries the `folded` count of what was suppressed.

Driven on a 20,000-byte file, which gives one identity 2,000 bytes: a loud identity emitting 300 passing records put 6 of them on the live file and had 294 refused, the one rare identity's record was on the live file, and the loud identity's failing record was on the live file after its share was spent. With the share set to `0.0`, all 300 were on the live file and nothing was refused.

A reading for silence on a topic

A file of written records cannot report silence, because a record is only ever written when something happened. `src/core/emit_contracts.py`, in `EmitObserver.silence`, reads each declared topic against the budget its contract declares.

reading	what it means
`unbounded`	the topic declares no budget, so silence from it is not evidence
`never`	a budget is declared and the topic has not emitted at all; its producer is absent, which is not the same as stopped
`within`	the last emission is inside the budget
`overdue`	the topic emitted and then stopped past its budget

`EmitContract` carries the budget as a declared field, `silence_budget`. Two topics declare one: `bot.gate_decision` and `ta.voting`, the two a bot's own evaluation loop drives. `trade.filled`,

pnl.event, market_inspector.scan_started and market_inspector.scan_finished declare none, because a fill, a P&L movement and a scan each need a discrete trigger and silence from them says nothing. Those four read unbounded for ever.

```
LOOP_DRIVEN_SILENCE_BUDGET = STALE_AFTER
```

STALE_AFTER is the age at which the network already reads a pin stale, imported rather than copied, so the bus reading and the pin reading cannot drift to two ages. silence takes its clock from the caller and falls back to time.monotonic(), so a reader replaying compressed time supplies its own.

Driven on a fresh observer: the two budgeted topics read never before any emission and the four unbudgeted read unbounded. One ta.voting emission later, that topic read within a second on and overdue a second past its budget. trade.filled read unbounded a million seconds on.

Where the readings land

No new reader was built. Six readings joined SignalSink.health(), which src/gui/main_tabs/system_status_tab_surface.py already reads, and one key joined EmitObserver.to_dict(), which src/simulator/sim_bus.py already reads at a run's close.

reading	what it carries
age_retired	backups the age bound unlinked from the main ladder
digest_age_retired	the same for the digest ladder
max_file_age	the age bound in force
share_refused	passing records the share bound kept off the live file
share_budget	bytes one identity may fill between rotations
share_identities	identities holding a charge since the last rotation

format_observer_lines prints one tally line for the silence readings, then one line per topic reading never or overdue. A topic reading unbounded is counted and not listed, because a list of such topics would read as a list of faults.

Overtaken by the two new ladder bounds

Each sentence below stands as written above. The sentence under it is the current reading.

> *Three constants bound what the sink holds: how many rows buffer before a write, how many stay in memory, and the size at which the file rotates.*

Five. MAX_FILE_AGE_SECONDS bounds the age of a rotated backup, and MAX_IDENTITY_SHARE bounds the share of one live file a single identity may fill with passing records.

What the new bounds do not prove

Every figure above was read from a drive under a temporary home, with HOME, USERPROFILE, LOCALAPPDATA and APPDATA redirected before the first import and Path.home() printed to prove it. None of it was read from a running platform. What the window and the share bound do to a real session's megabytes per hour is unmeasured until a launch writes a real signals/session.jsonl. What to watch for: share_refused climbing while digest_folded climbs with it, and age_retired holding at 0 until a ladder's tail is fourteen days old.

2026-10-18 - #143 - the Watchdog names one encoding for the child, the pipe and the log

The Watchdog launches the platform, reads back everything it prints, and writes that into `console_<timestamp>.log`. Three ends carry the same text: what the child writes, what the pipe hands back, and what the file holds. A text stream that names no encoding falls back to the machine's locale, so each end picked its own and the operator's machine decided which.

The rule: one encoding, named at every end. `acervator_watchdog.py` holds it as two values.

`acervator_watchdog.py` — the one encoding and its error policy

```
CHILD_IO_ENCODING = "utf-8"
CHILD_IO_ERRORS = "replace"
```

Four sites read them. `ChildRunner.child_env` sets `PYTHONIOENCODING` for the child, so the child writes UTF-8. `ChildRunner._start_internal` names the same encoding on the pipe it reads and on the console log it opens. `ChildRunner._rotate_if_needed` names it again on the two files it reopens. `_set_relay_encoding` sets this process's own `sys.stdout`, which `ChildRunner._reader_loop` relays each child line to.

Both launch branches, frozen and source tree, build their environment in `ChildRunner.child_env`, so the two ends cannot drift apart.

What a failure cost before this

Driven on a Windows host whose locale reads `cp1252`, with a child made to print an arrow, an em dash and a zero-width joiner:

what the ends named	what happened
neither end, the shape before this	**the child died**, return code 1, and the log held its traceback instead of the line
the pipe only	the reader thread raised and the whole stream was lost
the child only	the reader thread raised and the whole stream was lost
both ends	return code 0, all three glyphs on disk and in the tail

Only the last row works. Naming the encoding at one end alone makes it worse, not better.

Why the error policy is not strict

`CHILD_IO_ERRORS` is `replace` at both ends, and each half earns its place.

where	strict instead	`replace`
the child's own output	a lone surrogate, which a Windows filename can carry, kills the child	the child survives and the glyph is substituted
the pipe the parent reads	a byte that is not UTF-8 loses the whole stream, 0 characters read	47 characters read, nothing lost

The machine identity read at launch

`src/core/instance_guard.py` decides whether another copy already holds the lock. It reads the machine identity before the window opens. `_machine_id_darwin` runs `ioreg` and `_machine_id_linux` reads `/etc/machine-id`. Both now name the encoding and both use `replace`, because a `UnicodeDecodeError` is not one of the faults either handler catches, and an uncaught one there ends the launch.

Measured on the one encoding

Two instruments ran over `git ls-files '*.py'`, 615 files, each proved on a planted fixture pair first.

```
shape                       before      after     the control
file read and file write    3 sites     3 sites   250 calls name the encoding
the pipe read              14 sites    11 sites    20 -> 23 calls name it
```

The three sites that moved are the two Watchdog launches and the `ioreg` call. The count is conserved: three left the target column and three entered the control column. Of the 3 file sites, two are not file objects at all, and the third is in the paused USB key reader.

The remaining 11 pipe sites sit outside the launch path: the paused USB key reader holds 3, the build and gate tooling holds 4, and the review harness holds 4.

The drive ran on a scratch home with `HOME`, `USERPROFILE`, `LOCALAPPDATA` and `APPDATA` redirected before the first import and `Path.home()` printed to prove it. It wrote one file, under that scratch home. What a real launch does to a real console log is unmeasured until one runs. What to watch for: a `console_<timestamp>.log` that holds an arrow or an em dash where it used to hold a `UnicodeEncodeError` traceback, and a `watchdog_errors.log` with no `_reader_loop: could not relay` line.

Settings

Reference. `SettingsDialog` in `src/gui/settings_dialog.py`. The two pages below are the ones Part 3 names.

The pages

One method adds them, in one order.

`src/gui/settings_dialog.py` — `SettingsDialog._setup_ui`

```
tabs.addTab(self._create_user_tab(), "User")
tabs.addTab(self._create_exchange_tab(), "Exchanges")
tabs.addTab(self._create_trading_tab(), "Trading")
tabs.addTab(self._create_ta_tab(), "TA Indicators")
tabs.addTab(self._create_phantom_tab(), "Phantom Bots")
tabs.addTab(self._create_theme_tab(), "Theme")
tabs.addTab(self._create_sound_tab(), "Sound")
tabs.addTab(self._create_sms_tab(), "SMS")
tabs.addTab(self._create_ai_monitor_tab(), "AI Monitor")
```

The count is a declaration, not a number in this manual

That block is a reading, and every removal has moved it. Eleven became ten, ten became nine, and the next removal will move it again. The rule replaces the figure. The page count and the control count are two declarations, and the answer is whatever those two hold on the day you read them.

`src/gui/main_tabs/settings_dialog_surface.py` — the two the count comes from

```
TAB_TITLES = (
    USER_TAB, EXCHANGE_TAB, TRADING_TAB, TA_TAB, PHANTOM_TAB,
    THEME_TAB, SOUND_TAB, SMS_TAB, AI_TAB,
)

CONTROL_SPECS: tuple[dict, ...] = (
    {"tab": USER_TAB, "group": "", "name": "username", "kind": LINE, ...},
    ...
)
```

Both builds read the same two, so a page or a control that leaves a declaration leaves the Qt window and the shell together. Count the entries in the first for the pages and the entries in the second for the controls. Every dated section further down this page records what one removal took; none of them owns the count.

Read on 13 September 2026, in one run, off the built Qt window and off the payload the shell draws:

	the declaration	the Qt window	the shell payload
Pages	9	9 tabs	9
Controls	65	not counted whole	65 specs
Tick boxes	23	23 drawn	23
Drop-downs	5	5 drawn	5
Radio buttons	0	0 drawn	0

The Qt window was read by walking its own widget tree, so the tick-box and drop-down rows are drawn counts and the whole-dialog figure of 65 is the declaration's. The counts fell by one when the Exchange drop-down became the `chosen_venue` label. The browser page was rendered as well and reported 0 radio inputs, against 23 checkbox inputs on the same reading.

What each page persists

`_save` writes and `_load_current` reads back. A control neither method touches keeps its build-time default for the life of the process and reaches no engine.

Page	Controls	`_save` writes	`_load_current` restores
User	1	the row	the row
Exchanges	5 plus 3 buttons	on Add and Remove	the configured list
Trading	6	all six	four of six
Profit Folding	11	the whole group	`active` alone
TA Indicators	12	nothing	nothing
Phantom Bots	13	nothing	nothing

Page	Controls	_save writes	_load_current restores
Theme	6	all six	the theme and the accent
Logging	6	the whole group	nothing
Sound	10 plus 6 buttons	nothing	nothing
SMS	17	nothing	nothing
AI Monitor	7	all seven	all seven

AppSettings in src/core/settings.py is the whole schema, and a key it does not declare is refused outright rather than stored somewhere unread.

src/core/settings.py — SettingsManager.set

```
def set(self, key: str, value: Any) -> None:
    """Set the ``AppSettings`` attribute *key* and call ``_save``.

    A *key* that ``AppSettings`` does not declare raises ``KeyError``.
    """
    with self._lock:
        if not hasattr(self._settings, key):
            raise KeyError(f"Unknown setting: {key}")
```

Four pages hold controls _save never reads, and the schema declares no field any of them could land in: TA Indicators, Phantom Bots, Sound and SMS. Issue #423 carries all four and counts the controls.

The issue #423 clause above is overtaken, and so is every later clause on this page that names it. None is edited and none is removed. This one read:

> *Four pages hold controls _save never reads, and the schema declares no field any of them could land in: TA Indicators, Phantom Bots, Sound and SMS. Issue #423 carries all four and counts the controls.*

Issue #423 closed on 2026-09-25. The schema declares a field for each of the four pages now, and each group has a named reader.

```
src/core/settings.py, in AppSettings

ta_indicator_weights        the TA Indicators sliders, one key per weight
default_enable_phantoms     the phantom master box
default_phantom_timeframe   the one timeframe a new bot starts with
sound                       the Sound page's nine switches and its volume
message_channels            the SMS page's twenty-two values
```

```
src/trading/ta_engine.py, in weights_from_settings   reads ta_indicator_weights
src/core/sound_engine.py, in sound_config_from_settings   reads sound
src/core/sms_engine.py, in sms_config_from_settings       reads message_channels
src/gui/main_tabs/bot_wizard_surface.py   DEFAULT_ENABLE_PHANTOMS_KEY,
                                          DEFAULT_PHANTOM_TIMEFRAME_KEY
```

The Exchanges page is the exception to the whole pattern. _add_exchange and _remove_exchange write through the settings manager as the operator uses them, rather than waiting for Save.

The Sound page reaches its engine by one other route. The slider handler builds a fresh configuration from every checkbox, pushes it in, and clears the sample cache, because each sample bakes its volume at synthesis time.

`src/gui/settings_dialog.py` — `_on_sfx_volume_changed`

```
se = get_sound_engine()
new_cfg = SoundConfig(
    enabled=self._sound_enabled.isChecked(),
    buy_sound=self._sound_buy.isChecked(),
    sell_sound=self._sound_sell.isChecked(),
    error_sound=self._sound_error.isChecked(),
    bot_state_sound=self._sound_state.isChecked(),
    fire_sound=self._sound_fire.isChecked(),
    track_sound=self._sound_track.isChecked(),
    profit_sound=self._sound_profit.isChecked(),
    drip_sound=self._sound_drip.isChecked(),
    volume=v / 100.0,
)
```

That path runs when the slider moves or a test button plays a sample, and at no other time.

The engine now drops its own clips. `src/core/sound_engine.py`, in `SoundEngine.update_config`, takes the new configuration and clears the cache itself, so no caller reaches into the engine to do it. The handler on this page hands the configuration over and stops there. The same method also removes the directory the old clips were written to, which the page used to leave behind on every slider step.

`src/core/sound_engine.py` — `SoundEngine.update_config`

```
self._config = config
self.cleanup()
self._temp_dir = None
self._available = False
self._cache = {}
```

The parent section, 08-tabs.md, carries the figure for each page and names every control on it.

Settings, User page

The whole page is one form row. No credential field, no licence field and no key import.

`src/gui/settings_dialog.py` — `_create_user_tab`

```
def _create_user_tab(self) -> QWidget:
    w = QWidget()
    form = QFormLayout(w)
    self._username = QLineEdit()
    form.addRow("Username:", self._username)
    return w
```

`_load_current` reads the stored value and `_save` writes it back through the settings manager, which persists the schema under a home-relative directory as a TOML file, or as JSON when no TOML writer is available. One lock guards every read and write, shared across every manager instance.

What a credential store would rest on

Both halves already exist, and the User page reads neither.

The first is encryption. A passphrase and a random salt derive a 256-bit key, and each token carries its own salt and nonce.

`src/core/encryption.py` — `derive_key`

```
  def derive_key(passphrase: str, salt: bytes) -> bytes:
      """Derive a 256-bit AES key from *passphrase* + *salt* via PBKDF2."""
```

`KeyringManager` holds the passphrase in the operating system's credential store. Without the `cryptography` package a fallback keystream with an HMAC-SHA256 tag takes over.

The second is the hardware-key path. A credential list encrypts against a USB volume's serial and writes at that mount point, so the file decrypts on that stick and nowhere else.

`src/core/usb_auth.py` — `write_auth_file`

```
  def write_auth_file(
      usb_path: Path,
      volume_serial: str,
      credentials: list[dict],
  ) -> Path:
      """Encrypt *credentials* under ``_derive_usb_key`` and write ``AUTH_FILENAME``.

      Each dict carries ``exchange_id``, ``api_key``, ``api_secret`` and
      ``passphrase``, and the written path under *usb_path* is returned.
      """
```

`find_auth_volume` matches the volume serial that `read_auth_file` needs to derive the same key.

Settings, Exchanges page

`_create_exchange_tab` draws the Exchange Status array and a form to add one venue: the Exchange row, an API key, an API secret, and a passphrase field that appears only when the exchange needs one.

The array is the only control that chooses a venue. One button per venue the sector serves, and a press binds the form to that venue. The Exchange row is a label, not a picker, so the form cannot name a venue other than the one whose button is pressed.

`src/gui/settings_dialog.py` — `_bound_venue` and `_draw_chosen_venue` read the one pressed row; `src/gui/main_tabs/settings_dialog_surface.py` — `row_venue_id` gives the id and `chosen_venue_text` gives the words.

The array lists every venue `venues_for_class` answers for the sector, and every one of them binds. Crypto draws 21 buttons and all 21 bind, Robinhood among them.

With no button pressed the row reads `NO_VENUE_CHOSEN`, and Test Connection and Test and Add Exchange refuse with `NO_VENUE_FEEDBACK` before reading a credential.

Three venues need a passphrase, and the field appears for those alone.

`src/exchange/ccxt_connector.py` — `PASSPHRASE_EXCHANGES`

```
PASSPHRASE_EXCHANGES: set[str] = {
    "kucoin",
    "okx",
    "bitget",
}
```

The key field accepts a Coinbase CDP key string as well as a plain key, and the secret field accepts a PEM elliptic-curve private key. Escaped newlines inside a pasted PEM convert on the way in.

Three handlers sit behind the buttons.

Handler	What it does
`_test_api_connection`	Authenticates against the venue and reports through `_set_feedback`
`_add_exchange`	Runs that test first and stores nothing when it fails
`_remove_exchange`	Drops the selected entry

A venue ccxt carries no entry for takes its own connector instead. `src/exchange/api_validator.py`, in `validate_credentials`, asks `crypto_connector_class` first and routes to `hand_written_result`, which reads that connector's `credential_refusal` and reaches no venue. Robinhood's override in `src/exchange/robinhood_connector.py`, in `credential_refusal`, loads the typed secret as its Ed25519 signing key and names a key it cannot load.

The stock wing

The same page in the stock wing shows a banner and disables Add and Test.

`src/gui/settings_dialog.py` — `_create_exchange_tab`

```
_banner = QLabel(
    "<b>Stock Wing:</b> equity-broker integration is "
    "queued — no live brokers are wired up yet. The "
    "list below shows the planned brokers; Add / Test "
    "are disabled until the broker connectors ship. "
    "Use the Crypto Wing for active trading today."
)
```

`EQUITY_EXCHANGE_IDS` fills the list with the planned brokers, each labelled as planned. Crypto is the wing that trades today.

What adding one does

The new exchange routes to the crypto layer or the stock layer, and the method removes the placeholder tab if one stood there.

`src/gui/main_window.py` — `add_exchange_tab`

```
def add_exchange_tab(self, exchange_id: str, display_name: str) -> None:
    """Add exchange tab to the correct layer (crypto or stock)."""
    is_equity = self._is_equity_exchange(exchange_id)
```

The method emits a signal naming which layer the tab actually landed in, read back from both layer widgets rather than from the argument it was given.

Bridge

Two methods serve this screen.

Bridge method	Serves	Renderer module
`settings_dialog.state`	The dialog and every page on it	`settings_dialog.js`
`exchange_tab.state`	One venue's tab	`exchange_tab.js`

2026-09-06 00:52 - #423 - the Settings dialog draws in React

The File menu opens this dialog, and which one it opens depends on the build. The React build draws every page below inside one embedded browser. The Qt build draws the widget tree the pages describe. One line in the running window picks the side.

`src/gui/main_window.py` — `_open_settings`

```
from .variant_surface import SETTINGS_DIALOG, surface_class

_cls = surface_class(SETTINGS_DIALOG)
dlg = _cls(self._settings, self._status_log, self, wing=_wing)
```

`_wing` is the sector the header strip's class group has active. `src/gui/main_tabs/header_strip.py`, in `active_sector` answers it, and raises `SectorUnknown` for a window holding none. `_open_settings` and `_add_exchange` catch that, write the fault to the status log and open nothing, so neither screen can open on a sector the operator is not on.

The React side is a subclass, not a rewrite. It replaces only the method that builds the widgets, so the loading and the saving below are the same code on both sides. Each named control gets a small holder that answers the same calls the widget answered, and the page reads and writes those holders.

`src/gui/react_settings_dialog.py` — `SettingsDialogReact._setup_ui`

```
self._model = surface.SettingsDialogModel(wing=self._wing)
self._model.build()
self._build_holders()
self._web = QWebEngineView(self)
self._web.setPage(SettingsDialogPage(self))
self._web.setHtml(dialog_html(self._theme_name()))
```

An edit on the page arrives as one line the host reads, and the holder takes it the way the control takes a stored value. A radio inside a group box clears the others, which is what `QGroupBox` does for the widget side.

`src/gui/react_settings_dialog.py` — `SettingsDialogReact.apply_edit`

```
  self._holders[name].admit(asked.get("value"))
```

Settings > User Tab

This needs to be built out to accept and contain individual end user credentials. This will also be where a license is entered or a keyfile is imported depending on how we design the authentication piece.

The dialog holds the pages below, added in one order.

`src/gui/settings_dialog.py` — `SettingsDialog._setup_ui`

```
  tabs.addTab(self._create_user_tab(), "User")
  tabs.addTab(self._create_exchange_tab(), "Exchanges")
  tabs.addTab(self._create_trading_tab(), "Trading")
  tabs.addTab(self._create_ta_tab(), "TA Indicators")
  tabs.addTab(self._create_phantom_tab(), "Phantom Bots")
  tabs.addTab(self._create_theme_tab(), "Theme")
  tabs.addTab(self._create_sound_tab(), "Sound")
  tabs.addTab(self._create_sms_tab(), "SMS")
  tabs.addTab(self._create_ai_monitor_tab(), "AI Monitor")
```

`_save` and `_load_current` decide which of them persists, and each page says which under its own heading. Issue #423 carries the four pages whose controls Save never reads.

Issue #423 closed on 2026-09-25, and `src/core/settings.py, in AppSettings` declares a field for each of the four pages. The declaration and its readers are named under What each page persists above.

The User page itself is one form row, and the settings manager persists it. No credential field, no licence field and no key import.

`src/gui/settings_dialog.py` — `_create_user_tab`

```
  def _create_user_tab(self) -> QWidget:
      w = QWidget()
      form = QFormLayout(w)
      self._username = QLineEdit()
      form.addRow("Username:", self._username)
      return w
```

Both halves a credential store would rest on already exist, and the page reads neither. `src/core/encryption.py` encrypts a secret with AES-256-GCM under a key derived from a passphrase and a random salt.

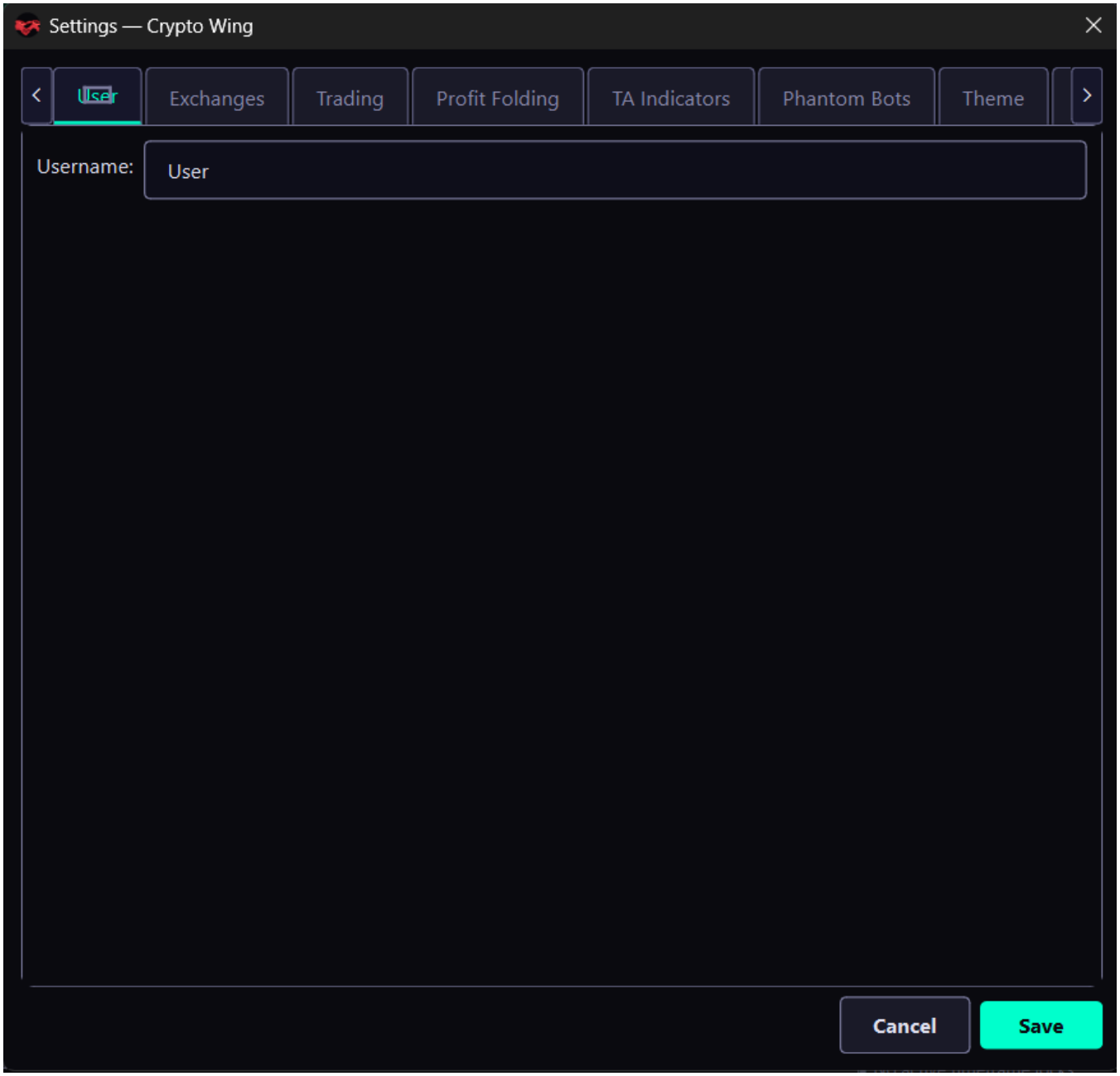
`src/core/encryption.py` — `derive_key`

```
  def derive_key(passphrase: str, salt: bytes) -> bytes:
      """Derive a 256-bit AES key from *passphrase* + *salt* via PBKDF2."""
```

`src/core/usb_auth.py` writes an encrypted credential file keyed to a USB volume's serial, which decrypts on that stick and nowhere else.

`src/core/usb_auth.py` — `write_auth_file`

```
def write_auth_file(
    usb_path: Path,
    volume_serial: str,
    credentials: list[dict],
) -> Path:
```



Settings, the User page.

The title bar names the wing the dialog was opened for. The arrows at each end of the page row scroll it, because the eleven do not fit the dialog's minimum width. Cancel and Save close the dialog, and Save always closes it, reporting any group that failed to persist. The page itself carries one row, Username, and nothing else.

That one row is not cosmetic. The stored username keys the credential vault. Adding a venue on the next page builds a master phrase from that name, then encrypts the API key and the API secret with the phrase.

`src/gui/settings_dialog.py` — `_add_exchange`, the phrase and the encryption

```
master = f"qat_{self._sm.get('username', 'user')}_vault"
config.api_key_enc = encrypt(key, master)
config.api_secret_enc = encrypt(secret, master)
```

The figure shows a stored value, User. The declared default is an empty string, and the settings object always declares the field, so the fallback written into the phrase never fires. An install where the row was never filled encrypts under a phrase with an empty middle. Change the name on a machine that already holds credentials and the stored secrets stop opening, because the connect path rebuilds the same phrase from the new name. Issue #436 carries it.

The issue #436 clause above is overtaken. It is not edited and not removed. It read:

> *Change the name on a machine that already holds credentials and the stored secrets stop opening, because the connect path rebuilds the same phrase from the new name. Issue #436 carries it.*

Issue #436 closed on 2026-10-19. The phrase format is written once and one function builds it, so no caller spells it out.

```
src/core/encryption.py          MASTER_FORMAT = "qat_{username}_vault"
src/core/encryption.py, in vault_phrase      the one builder
src/core/encryption.py, in hold_master_key   the held key every site answers from
sites naming vault_phrase       22 occurrences
sites spelling the format out    1, the declaration itself
```

What the renaming of a machine's username should do to stored secrets is a product choice, so nothing here proposes one.

`src/gui/main_window.py` — the decrypt side, rebuilding the same phrase

```
master = f"qat_{self._settings.get('username', 'user')}_vault"
api_key = decrypt(exch_config["api_key_enc"], master)
api_secret = decrypt(exch_config["api_secret_enc"], master)
```

Three sites write that phrase out as a literal, one to encrypt and two to decrypt, and a fourth declares the format none of them reads. The same issue carries that half.

`src/gui/main_tabs/settings_dialog_surface.py` — the declaration

```
MASTER_FORMAT = "qat_{username}_vault"
```

Settings > Exchanges

The page draws the Exchange Status array and adds one venue: the Exchange row, an API key, an API secret, and a passphrase field that appears only when the venue needs it. The array is the only control that chooses a venue, and the Exchange row is a label that repeats the pressed button's own words.

`src/gui/main_tabs/settings_dialog_surface.py` — `exchange_status_rows` gives the array its venues, `row_venue_id` the pressed row's id, and `chosen_venue_text` the words the Exchange row reads.

Three venues need the passphrase field, and it appears for those alone.

`src/exchange/ccxt_connector.py` — PASSPHRASE_EXCHANGES

```
PASSPHRASE_EXCHANGES: set[str] = {
    "kucoin",
    "okx",
    "bitget",
}
```

The key field accepts a Coinbase CDP key string and the secret field a PEM elliptic-curve private key. The connection test runs before anything is stored, and adding one routes the new tab to the crypto or the stock layer.

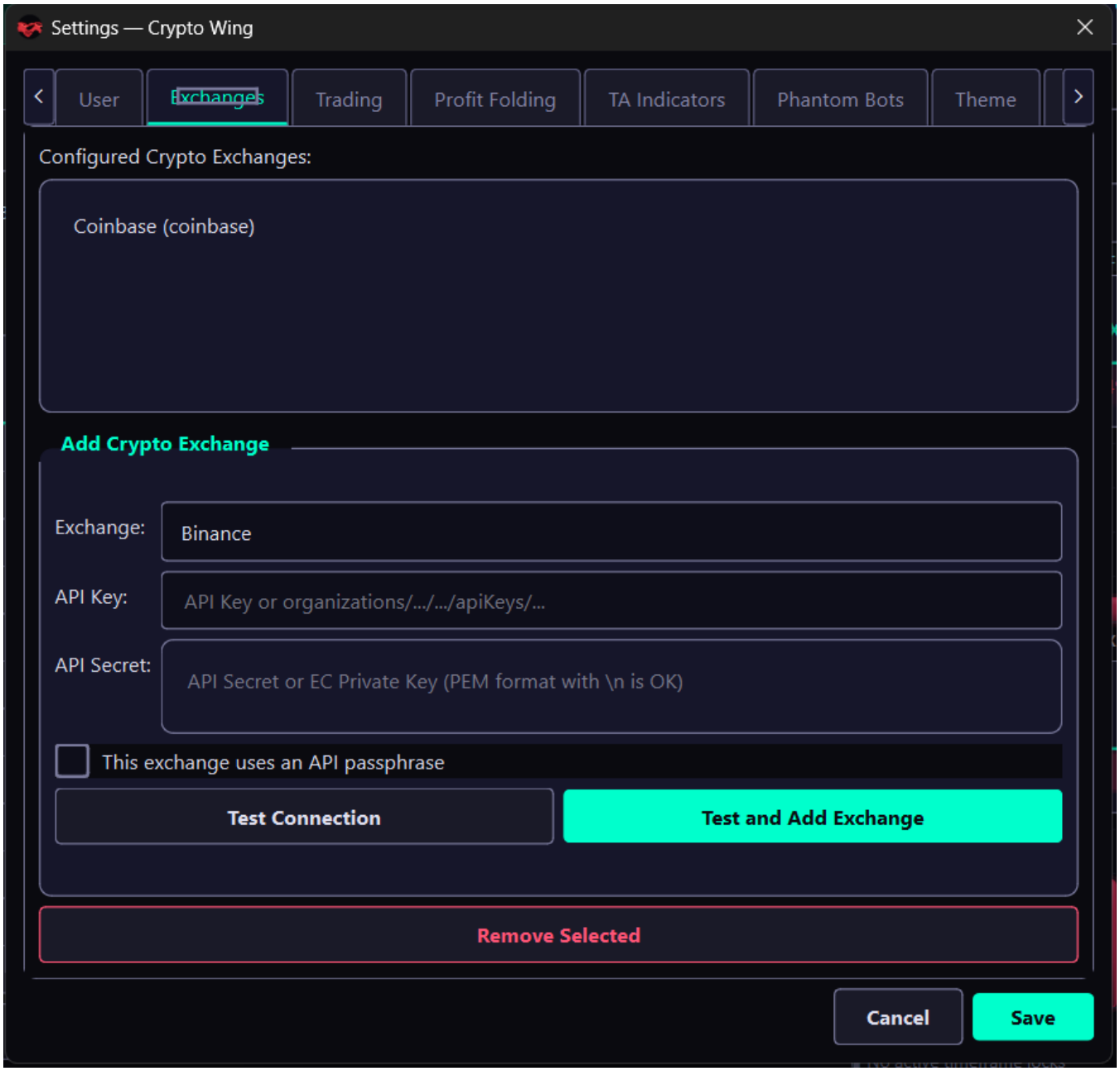
`src/gui/main_window.py` — `add_exchange_tab`

```
def add_exchange_tab(self, exchange_id: str, display_name: str) -> None:
    """Add exchange tab to the correct layer (crypto or stock)."""
    is_equity = self._is_equity_exchange(exchange_id)
```

The same page in the stock wing shows a banner and disables Add and Test, because no broker connector ships yet.

`src/gui/settings_dialog.py` — `_create_exchange_tab`

```
_banner = QLabel(
    "<b>Stock Wing:</b> equity-broker integration is "
    "queued — no live brokers are wired up yet. The "
    "list below shows the planned brokers; Add / Test "
    "are disabled until the broker connectors ship. "
    "Use the Crypto Wing for active trading today."
)
```



Settings, the Exchanges page.

Configured Crypto Exchanges lists what the manager returns for this wing, each entry naming its display name and its exchange id. The stock wing lists the equity ids instead and disables both buttons.

The Add Crypto Exchange group holds four rows. Exchange reads the pressed button's venue. API Key takes a plain key or a Coinbase CDP key string, and its placeholder shows the CDP shape. API Secret takes a plain secret or a PEM elliptic-curve private key, and escaped newlines inside a pasted PEM convert on the way in. The passphrase checkbox appears for the three venues above.

The figure predates the array and shows the picker the array replaced. With no button pressed the Exchange row reads `NO_VENUE_CHOSEN` and the passphrase box stays clear. A press sets that box for the three venues that need one and clears it for the rest. It also wipes the feedback line, so an answer about the last venue cannot stand as an answer about this one.

`src/gui/settings_dialog.py` — `_on_exchange_changed`

```
def _on_exchange_changed(self) -> None:
    eid = self._bound_venue()
    self._draw_chosen_venue()
    needs_pp = eid in self._passphrase_exchanges
    self._pp_check.setChecked(needs_pp)
    self._sync_passphrase_row()
    self._api_feedback.setText("")
```

The page hides the passphrase field until you tick the box, which is why the figure shows the box and no field under it. The box carries the state; the field only collects the value.

`src/gui/settings_dialog.py` — the field the box reveals

```
self._new_passphrase = QLineEdit()
self._new_passphrase.setEchoMode(QLineEdit.Password)
self._new_passphrase.setPlaceholderText(
    "Passphrase set when creating API key"
)
self._new_passphrase.setVisible(False)
```

The words in that field are now one declaration, read by two screens. The block above stands as written, and this is what the field reads today.

`src/gui/settings_dialog.py` — the field reads the declaration

```
from .main_tabs.init_wizard_surface import PHRASE_FIELD_PROMPT

self._new_passphrase = QLineEdit()
self._new_passphrase.setEchoMode(QLineEdit.Password)
self._new_passphrase.setPlaceholderText(PHRASE_FIELD_PROMPT)
```

The install wizard wrote its own words for the same fact. Both screens now report the same 46 characters, and which venues ask for a passphrase did not change. Trading tab, under the ratio and the spare record, carries the reading.

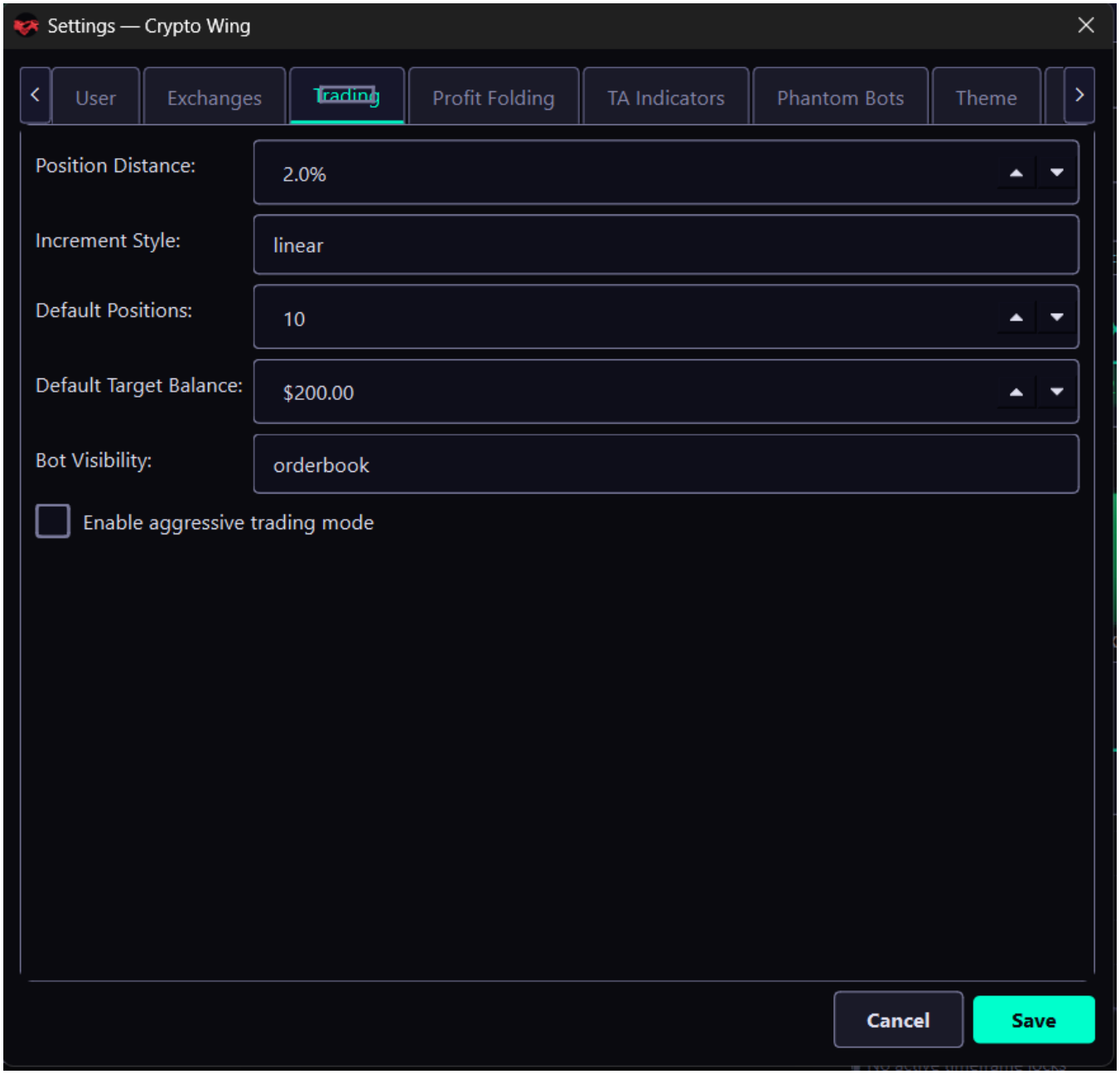
Test Connection runs the check and reports the answer. Test and Add Exchange runs the same check first and stores nothing when it fails. Remove Selected drops the highlighted entry.

Both buttons go dead for the length of a call, so a second click cannot start a second test. Test and Add returns on a failed check, which is what keeps a venue that did not answer out of the store.

`src/gui/settings_dialog.py` — `_add_exchange`, the guard before the store

```
if key and secret:
    result = self._test_api_connection()
    if result is None or not result.success:
        return
```

Settings > Trading



Settings, the Trading page.

Six rows carry the defaults a new bot starts from, not a running bot's settings.

Three of those six rows are off the page. The descriptions below, and the code blocks under them, record what the page held before the removal. Each removal has its own dated section further down this page.

the row	the dated section that records its removal
Position Distance	2026-09-11 - Position Distance is off the Trading page
Increment Style	2026-09-11 - Increment Style is off the Trading page
Default Positions	2026-09-11 - Default Positions is off the Trading page

The strip list quoted further down this section now says in its own words that it is a retirement record and not a set of settings.

`src/trading/container/config.py` — what the strip list says about itself

```
#: A retirement record, not a setting set. Each name is a setting removed from
#: the product, kept so a config stored before its removal still builds;
#: `_sanitize_deprecated_kwargs` drops each before `BotConfig.__init__`.
#: Add no name. Only `bulk_trading` is read, by `bot_config_kwargs`.
```

Position Distance - Sets the spacing a new bot puts between its positions. Key `position_distance_pct`, 1.0 % to 50.0 %, at 2.0 %.

The dialog stores the figure and reads it back the next time the page opens. No bot ever sees it. The bot factory strips the name before it builds a config, so the value cannot reach a bot by any route.

`src/gui/settings_dialog.py` — the Position Distance row

```
self._pos_distance = QDoubleSpinBox()
self._pos_distance.setRange(1.0, 50.0)
self._pos_distance.setSuffix("%")
self._pos_distance.setDecimals(1)
form.addRow("Position Distance:", self._pos_distance)
```

The strip list holds eleven names. This page contributes one of them and the Profit Folding page below contributes four more.

`src/trading/container/config.py` — _DEPRECATED_KWARGS

```
_DEPRECATED_KWARGS: frozenset = frozenset(
    {
        "bulk_trading",  # renamed to stack_mode
        "bulk_partial_on_return",
        "market_check_interval",
        "position_count",
        "position_distance_pct",
        "fold_mode",
        "fold_target",
        "fold_target_count",
        "profit_fold_pct",
        "distribute_target",
        "distribute_target_count",
    }
)
```

Increment Style - Chooses whether that spacing stays even or widens on a curve. Key `increment_style`, linear or logarithmic, at linear.

The dialog stores the choice and restores it on the next open. A bot takes its own increment style from the wizard rather than from this page.

`src/gui/settings_dialog.py` — the Increment Style row

```
self._increment_style = QComboBox()
self._increment_style.addItems(["linear", "logarithmic"])
form.addRow("Increment Style:", self._increment_style)
```

Default Positions - Sets how many positions a new bot opens with. Key `default_position_count`, 1 to 100, at 10.

The dialog stores the count and restores it. Nothing outside the dialog reads the name, so no bot opens with it.

`src/gui/settings_dialog.py` — the Default Positions row

```
self._default_positions = QSpinBox()
self._default_positions.setRange(1, 100)
form.addRow("Default Positions:", self._default_positions)
```

Default Target Balance - Sets the dollar figure the wizard's own Target Balance field opens at. Key `default_target_balance`, $1.00 to $1,000,000.00, at $200.00.

This is the one row on the page that reaches a new bot. The wizard's parameter page opens its Target Balance field at the stored figure.

`src/gui/bot_wizard.py` — the wizard field that reads the stored default

```
self._target_balance = QDoubleSpinBox()
self._target_balance.setRange(1.0, 1000000.0)
self._target_balance.setDecimals(2)
self._target_balance.setPrefix("$ ")
self._target_balance.setValue(defaults.get("default_target_balance", 200.0))
```

Bot Visibility - Chooses whether a new bot rests its orders on the book or tracks them inside the platform. Key `bot_visibility`, orderbook or internal, at orderbook.

Save writes the name and the load path never reads it. The box opens on orderbook whatever was stored, and a Save with no edit writes that first entry back over the stored value. A bot takes its visibility from the wizard, under a different name.

`src/gui/settings_dialog.py` — the Bot Visibility row

```
self._visibility = QComboBox()
self._visibility.addItems(["orderbook", "internal"])
form.addRow("Bot Visibility:", self._visibility)
```

Enable aggressive trading mode - Starts a new bot pricing for an immediate fill. Key `aggressive_trading`, a checkbox, off.

The same gap as the row above. Save writes it, the load path skips it, and the box opens clear. The wizard carries its own aggressive trading checkbox, and that is the one a new bot reads.

`src/gui/settings_dialog.py` — the aggressive trading row

```
self._aggressive = QCheckBox("Enable aggressive trading mode")
form.addRow(self._aggressive)
```

Save writes all six keys. The load path restores the first four, which is why the last two rows on the page open at their built-in defaults.

`src/gui/settings_dialog.py` — `_load_current`, the four it restores

```
self._pos_distance.setValue(self._sm.get("position_distance_pct", 2.0))
self._default_positions.setValue(self._sm.get("default_position_count", 10))
self._default_balance.setValue(
    self._sm.get("default_target_balance", 200.0)
)
style = self._sm.get("increment_style", "linear")
idx = self._increment_style.findText(style)
if idx >= 0:
    self._increment_style.setCurrentIndex(idx)
```

Three of the four reads in that block belonged to rows now off the page. The page saves and restores Default Target Balance alone out of the four.

OVERTAKEN, and the sentences and the block above are kept as written. The load path reads back every row, not the first four. One declaration lists the rows the dialog persists, and the save and the load walk that same declaration, so no row can be written without also being loaded. It holds nine rows today, Bot Visibility and Enable aggressive trading mode among them, and the groups below it load the same way.

`src/gui/settings_dialog.py` — `SettingsDialog._load_current`

```
for key, read, show, fallback in self._stored_rows():
    self._show_stored(key, show, self._sm.get(key, fallback), read)
for group, rows in self._stored_groups():
    stored = self._sm.get(group, {})
    for key, read, show, fallback in rows:
        self._show_stored(
            f"{group}.{key}", show, stored.get(key, fallback), read
        )
```

Settings > Profit Folding

The dialog draws no Profit Folding page. `TAB_TITLES` in `src/gui/main_tabs/settings_dialog_surface.py` holds no such entry, so neither build lays one out.

The settings record declares no folding group either. `AppSettings` in `src/core/settings.py` is the whole schema, and no field on it holds a folding value.

Folding is switched per bot, on the running bot's own settings row. The field is `profit_folding_active`, declared on `BotConfig`.

`src/trading/container/config.py` — the switch a fold reads

```
profit_folding_active: bool = True
```

A fold is bounded in dollars and by nothing else. The planner takes the tranches the market price reaches and the money left under this cycle's growth cap, and it stops when the money runs out. One dollar cap bounds a fold, and no count does carries that measured over five queued tranches.

Settings > TA Indicators



Settings, the TA Indicators page.

One slider per weight, twelve in all, each labelled from its key and each running 0.00 to 2.00. The value beside a slider follows it as it moves, and the starting values are the weights themselves.

Indicator weight - Sets how far one indicator's vote counts against the other eleven when the engine totals a direction. The key behind each slider, and the value it starts at, are the twelve below.

`src/trading/ta_engine.py` — `DEFAULT_WEIGHTS`

```
DEFAULT_WEIGHTS = {
    "bollinger_bands": 1.0,
    "vortex": 0.9,
    "macd": 1.2,
    "stochastic_rsi": 1.0,
    "ichimoku": 1.1,
    "volume": 0.8,
    "slingshot": 1.0,
    "adx": 1.0,
    "kaufman_er": 1.0,  # Perry Kaufman Efficiency Ratio (regime classifier)
    "supertrend": 1.0,  # Olivier Seban ATR-trailing trend (reactive flip)
    "zscore": 0.9,  # Statistical extremity over a longer window than BB
    # Both are built on Wilder's RSI, so this vote overlaps stochastic_rsi.
    "rsi": 0.8,
}
```

Since 2026-10-28 a new bot does not start at these figures. It starts at twelve zeros of its own, written by `src/trading/ta_engine.py, in new_bot_weights`, and takes this page's figures at the next Save. The sliders still open at the figures above. The dated entry for that day at the end of this page carries the readings.

A weight moved here reaches nothing. Save reads no widget on this page, the load path restores none, and the settings schema declares no field for indicator weights. The voting engine takes a weights argument and falls back to the defaults above, and no construction site under `src/` supplies one from the settings store. The page label promises an adjustment the engine never sees. Issue #423 carries it.

Issue #423 closed on 2026-09-25. The schema declares `ta_indicator_weights`, one key per weight slider, and `src/trading/ta_engine.py, in weights_from_settings` reads that group and hands the engine the stored weights.

```
src/core/settings.py, in AppSettings           ta_indicator_weights: dict
src/gui/main_tabs/settings_dialog_surface.py   TA_WEIGHT_GROUP_KEY
src/trading/ta_engine.py                       reads the group from the store
```

`src/trading/ta_engine.py` — `VotingEngine.__init__`, the fallback every construction site takes

```
self.weights = weights or DEFAULT_WEIGHTS.copy()
```

Where an indicator weight lands

Each of the twelve sliders is a named control, saved under one key and restored from it. Save writes all twelve; reopening the page shows what was saved. The key is `ta_indicator_weights`, and an empty one means every indicator votes at its published figure.

`src/core/settings.py` — the stored group

```
# Indicator name -> voting weight. Empty means every indicator votes at the
# figure ta_engine.DEFAULT_WEIGHTS declares for it.
ta_indicator_weights: dict = field(default_factory=dict)
```

A bot reads the group when it is built, so a weight changed here applies to the next bot created and to every bot on the next launch. A bot already ticking keeps the figures it started with until it is rebuilt.

Diagram: flowchart, left to right

- slider — TA Indicators page -> **K]**
- ("ta_indicator_weights -> **weights_from_settings**
- weights_from_settings -> **ScrummingBot — ta_weights**
- ScrummingBot — ta_weights -> **VotingEngine**
- VotingEngine -> **each indicator's weight**

`src/trading/ta_engine.py` — the one reader every path goes through

```
def weights_from_settings(stored: Optional[dict]) -> dict[str, float]:
    found = dict(DEFAULT_WEIGHTS)
    for name, figure in (stored or {}).items():
        ...
        found[name] = number
    return found
```

An entry naming no indicator, an entry that is not a number, and a negative figure are each dropped with a line in the log, so a hand-edited settings file cannot give an indicator a weight the engine refuses to explain.

One declaration of the twelve figures

`DEFAULT_WEIGHTS` is the only place the twelve figures are written. The voting engine builds one indicator per key, in that order, and the page reads the same mapping for its starting positions. Neither holds a second copy, so the two cannot drift apart.

`src/trading/ta_engine.py` — `_create_indicators`

```
return [
    INDICATOR_CLASSES[name](weight=self.weights.get(name, default))
    for name, default in DEFAULT_WEIGHTS.items()
]
```

The figures stay as they are

Driven on a record shaped the way the restore path reads `bot_state.json`, a restored bot arrives with no stored weight of its own and takes the declared figures. A zero written into the declaration would therefore reach every bot already on disk and stop all twelve indicators contributing, while the vote counts stayed the same. The figures are unchanged for that reason.

```
declared defaults   consensus 0.1684   BULLISH   votes 6/4/2
all twelve at zero  consensus 0.0      NEUTRAL   votes 6/4/2
```

A zero for one indicator is a different matter and is available today: setting a single slider to 0.00 removes that indicator's vote and leaves the other eleven voting.

Save hands the figures to a bot already running

Save is the only route. Pressing it writes the twelve figures and then hands them to every bot the manager holds. Each bot rebuilds its twelve indicators and votes on the new figures from its next tick. No timer and no other control reaches a bot already running.

`src/gui/main_window.py` — the one caller, inside the handler Save emits to

```
if self._bot_manager:
    self._bot_manager.push_ta_weights(
        weights_from_settings(
            self._settings.get("ta_indicator_weights", {})
        )
    )
```

Launch takes a different route and pushes nothing. `set_ta_weights` holds the figures for the restore path to build each bot with, and `push_ta_weights` is the only one that reaches a bot already built.

Diagram: flowchart, left to right

- Save button -> **settings_changed**
- settings_changed -> **push_ta_weights**
- push_ta_weights -> **each bot's VotingEngine**
- each bot's VotingEngine -> **the next tick's vote**
- launch -> **set_ta_weights, held only**
- set_ta_weights, held only -> **the next bot built**

Driven on three stand-in bots: every one read its MACD weight at 1.20 before the push and 0.30 after it, and a fourth holder that answers no such call was skipped, 3 of 4. Holding alone moved nothing, and the same reading stayed at 1.20.

Two voters build their own engine and take no weight

The detonation check and the initial-entry check each build a fresh voting engine, so the twelve figures reach neither. Initial entry gates the first buy on the direction that engine reports. Detonation cannot fire in any case: its confidence bar starts at 0.50, and one 240-candle tape put the highest consensus at 0.1242 with the figures declared and 0.1016 with all twelve at 0.30.

Current reading, the first buy. The initial-entry check still votes at the published figures. Its bearish vote now needs three agreeing indicators on that same vote, more than the 0.1966 two reach with all twelve voting. A bot whose own twelve figures are all zero, which every new bot holds until Save, buys on the direction alone as before.

`src/trading/scrumming/tick_phases.py` — `TickPhaseMixin._tick_initial_entry`

```
if not three_agree(
    summary.consensus_confidence,
    summary.signals,
    self._voting_engine.indicator_weights,
):
```

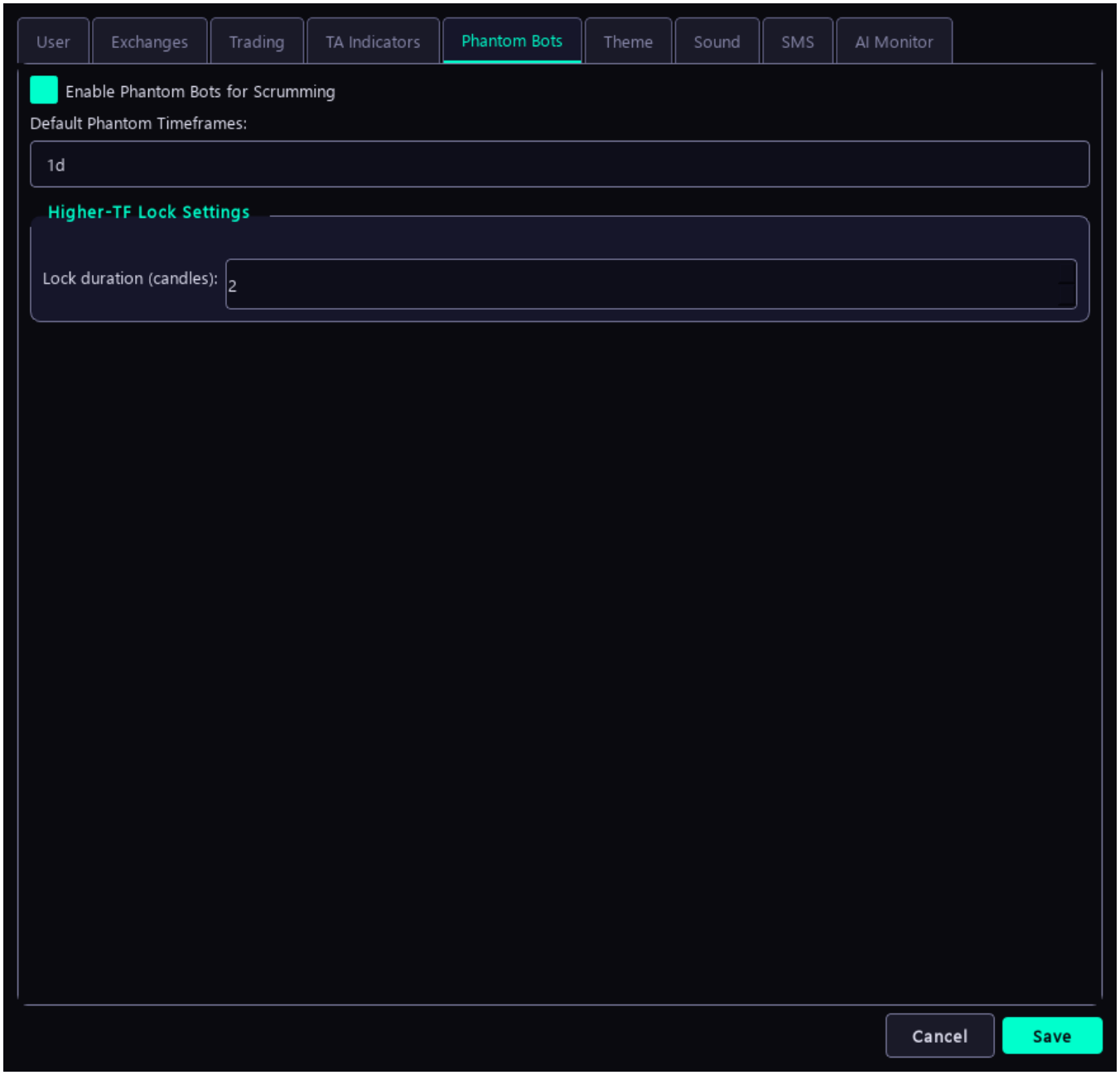
A phantom keeps the figures its set was created with

A phantom set is built from the parent's weights at the moment it is created. The push reaches the parent's own voter and not a phantom already running, so a phantom votes on the older figures until its set is created again.

Each drawn row is named by its indicator

On the drawn page every weight row carries the key the engine uses, which is the same name its slider answers to, and the printed words sit in the row's own label. Read off the rendered page: twelve rows, sixty-six drawn controls, and every row named for its indicator rather than for the words beside it.

Settings > Phantom Bots



Settings, the Phantom Bots page.

The master checkbox, eleven timeframe boxes — 1m, 5m, 15m, 30m, 1h, 2h, 4h, 6h, 12h, 1d and 1w — and one Higher-TF Lock Settings group holding Lock duration in candles, 1 to 10. The build checks 5m, 15m, 1h, 4h and 1d, which is the state the figure shows.

Enable Phantom Bots for Scrumming - Turns the phantom overrides on for a new bot. Per-bot key `enable_phantoms`, a checkbox, checked at build.

This is the only control on the page that starts on. Save never reads it, so the page stores nothing under the name it shows.

`src/gui/settings_dialog.py` — the phantom master switch

```
self._phantoms_enabled = QCheckBox(
    "Enable Phantom Bots for Scrumming"
)
self._phantoms_enabled.setChecked(True)
layout.addWidget(self._phantoms_enabled)
```

Default Phantom Timeframes - Chooses which charts a phantom watches. Per-bot key `phantom_timeframes`, eleven boxes, five of them checked at build.

One box per timeframe in a single row. The build decides which five open checked, and those five are the ones the figure shows.

`src/gui/settings_dialog.py` — the box built for each timeframe

```
cb = QCheckBox(tf)
cb.setChecked(tf in ["5m", "15m", "1h", "4h", "1d"])
self._phantom_tf_checks[tf] = cb
tf_grid.addWidget(cb)
```

The row is now one drop-down holding the same eleven names, and the page stores the one name it shows. The opening choice is the daily chart, which is the only entry above the wizard's own hourly default that every venue offers.

`src/gui/settings_dialog.py` — the one drop-down that replaced the grid

```
layout.addWidget(QLabel("Default Phantom Timeframes:"))
self._phantom_timeframe = QComboBox()
for tf in PHANTOM_TIMEFRAMES:
    self._phantom_timeframe.addItem(tf, tf)
self._phantom_timeframe.setCurrentIndex(
    self._phantom_timeframe.findData(PHANTOM_TIMEFRAME_DEFAULT)
)
layout.addWidget(self._phantom_timeframe)
```

Lock duration (candles) - Sets how many candles a higher-timeframe lock holds an opposing trade back for. Per-bot key `lock_candle_count`, 1 to 10, at 2.

The one spin box on the page, inside its own group. It opens at 2 and Save reads it no more than it reads the other two.

`src/gui/settings_dialog.py` — the lock duration box

```
lock_group = QGroupBox("Higher-TF Lock Settings")
lock_form = QFormLayout(lock_group)
self._lock_candles = QSpinBox()
self._lock_candles.setRange(1, 10)
self._lock_candles.setValue(2)
lock_form.addRow("Lock duration (candles):", self._lock_candles)
```

The page now stores that figure. One row carries the store key and the control it reads, so Save writes it and the load path puts it back.

`src/gui/main_tabs/settings_dialog_surface.py` — the row both surfaces persist

```
("default_lock_candle_count", "lock_candles", LOCK_CANDLE_DEFAULT),
```

Both wizard surfaces open their own lock box on the stored figure, so a new bot starts on the number the page shows. A run with no settings file at all opens at 2, the figure the store itself declares.

`src/gui/bot_wizard.py` — the wizard's lock box, opened on the store

```
self._stored_lock_candles = int(
    (defaults or {}).get("default_lock_candle_count")
    or AppSettings().default_lock_candle_count
)
```

Driven on one store, typed as 7 and saved: the page reopens at 7, the React page paints 7, and both wizard lock boxes open at 7. A settings file written before the key existed still loads, and its box opens at 2.

```
store before   KeyError 'Unknown setting: default_lock_candle_count'
store after    7
page reopen    Qt 7   React 7   React payload 7
wizard boxes   Qt 7   React 7   with no store at all 2
old file       loads, theme carried, box opens at 2
```

The figure stops at the bot's phantom coordinator and no code reads it. The lock guard that read it has been retired, so the coordinator keeps the number, holds no lock, and has no method that makes one.

`src/trading/phantom_balance.py` — the coordinator keeps the figure

```
def __init__(self, lock_candle_count: int = 2, bus=None) -> None:
    self.lock_candle_count = lock_candle_count
    self._locks: list[TradeLock] = []
```

Driven on two coordinators, one at the default and one at seven. Each carries the figure it was built with and each reports an empty lock list.

```
coordinator at 2   lock_candle_count 2   active locks []
coordinator at 7   lock_candle_count 7   active locks []
lock members left on the coordinator   _locks   get_active_locks   lock_candle_count
```

The wizard's own key still stops before the bot. The bot configuration declares no field of that name, and the bot class takes no argument for it.

```
BotConfig has a lock_candle_count field        False
bot_config_kwargs carries lock_candle_count    False
ScrummingBot.__init__ has lock_candle_count    False
```

The third reading has moved. `ScrummingBot.__init__` takes a `lock_candle_count` keyword argument, and `phantom_init_kwargs` in `src/trading/container/config.py` carries it from the wizard's collected config, so the figure reaches the new bot's coordinator. The first two readings stand: the bot

configuration still declares no field of that name and `bot_config_kwargs` still carries none.

```
BotConfig has a lock_candle_count field        False
bot_config_kwargs carries lock_candle_count    False
ScrummingBot.__init__ has lock_candle_count    True
```

Those eleven boxes are the eleven timeframes the voting engine already weights, from 1m at 0.3 up to 1w at 1.6, so a slower chart counts for more when several are combined.

These three controls carry the same gap the TA Indicators page carries. Save reads none of them, the load path restores none, and the schema refuses a key it does not declare rather than storing it somewhere unread.

`src/core/settings.py` — `SettingsManager.set`

```
with self._lock:
    if not hasattr(self._settings, key):
        raise KeyError(f"Unknown setting: {key}")
    setattr(self._settings, key, value)
    self._save()
```

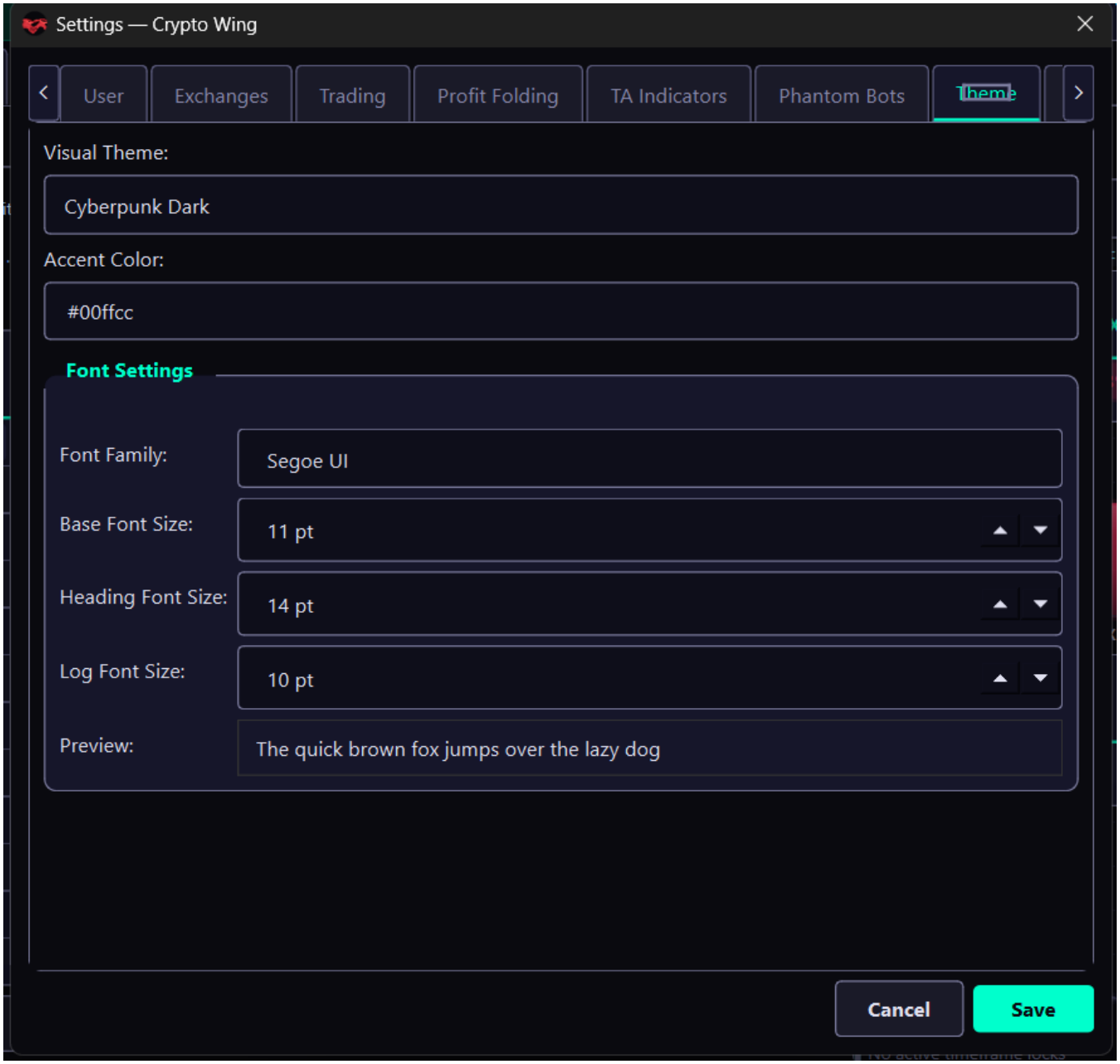
Issue #423 carries this page. The per-bot equivalents do persist: the wizard's phantom page emits the same three names into the bot's own config, and the Live Bot Settings dialog edits them there.

Issue #423 closed on 2026-09-25. The schema declares `default_enable_phantoms` and `default_phantom_timeframe`, and `src/gui/main_tabs/bot_wizard_surface.py` names both keys as `DEFAULT_ENABLE_PHANTOMS_KEY` and `DEFAULT_PHANTOM_TIMEFRAME_KEY`, which is where each one reaches a new bot.

`src/gui/bot_wizard.py` — `PhantomConfigPage.get_config`, the three keys that do reach a bot

```
return {
    "enable_phantoms": self._enable.isChecked(),
    "phantom_timeframes": checked,
    "lock_candle_count": self._lock_candles.value(),
}
```

Settings > Theme



Settings, the Theme page.

The theme picker, the accent colour field and a Font Settings group. Six controls and one preview.

Visual Theme - Chooses the palette the whole window paints in. Key `theme`, five display names — Cyberpunk Dark, Neon Light, Classic Terminal, Minimal Modern and Glass & Metal — at Cyberpunk Dark.

This is the one control on the page that reaches the running window. Save emits a signal, the main window reads the stored theme back, and the window repaints without a restart.

`src/gui/main_window.py` — `_on_settings_changed`

```
theme = self._settings.get("theme", "cyberpunk_dark")
self._switch_theme(theme)
```

A stored name the theme table does not hold opens on Cyberpunk Dark. The store accepts any text under that key, and the theme manager refuses a name it does not carry, so every store read passes its value through a guard first. Driven on ten stored values, five of them names the table does not hold: without the guard the application refused five and painted five, and with it all ten paint, while the five real names still paint their own five colours.

`src/gui/theme_engine.py` — `stored_theme`

```
def stored_theme(name: object) -> str:
    """Return ``name`` when ``THEMES`` holds it, else ``DEFAULT_THEME_NAME``."""
    return name if isinstance(name, str) and name in THEMES else DEFAULT_THEME_NAME
```

The guard covers the read the block above shows and the read at startup, which also says in the log when a stored name was not recognised. It covers the dialog's own page read as well: that one never refused, because the page palette falls back on its own table, so the guard there removes a second fallback and one copy of the default name rather than a refusal. Driven on eight stored values, four of them names the table does not hold: the page drew 8 of 8 and refused none both before and after, and all four unknown names paint a picture identical to Cyberpunk Dark and different from Neon Light.

`src/gui/react_settings_dialog.py` — `_theme_name`

```
        def _theme_name(self) -> str:
            """The stored theme the page paints in, always a name ``THEMES`` holds."""
            if not self._sm:
                return DEFAULT_THEME_NAME
            return stored_theme(self._sm.get("theme", DEFAULT_THEME_NAME))
```

`main.py` — the startup read

```
stored_name = settings.get("theme", DEFAULT_THEME_NAME)
theme_name = stored_theme(stored_name)
if theme_name != stored_name:
    log_manager.warning(
        f"Stored theme {stored_name!r} is not a known theme; "
        f"painting {theme_name}"
    )
theme_mgr.apply_theme(theme_name, app)
```

The palette reaches every Qt control and the Settings dialog's own page. It does not reach the React tab pages: each of those paints from the one design system, and their panes carry their own ground on every row. Rendered offscreen at 64,680 pixels a page, the Market Inspector, History, Console and Alerts pages come back identical under a stored Cyberpunk Dark and a stored Neon Light, and the Console page is identical even when the theme is handed straight to it.

`src/gui/react_history_panel.py` — the six values a page receives

```
    return {
        "--bg": colors["bg"],
        "--text": colors["text"],
        "--grid": colors["grid"],
        "--border": colors["border"],
        "--accent": colors["accent"],
        "--btn-bg": colors["btn_bg"],
    }
```

The main tab bar is the one React surface that follows a switch, and it follows it in part. Rendered at 8,232 pixels, one colour of eight moves from `#fcee0a` to `#f0cf1f` and returns when the theme returns; the bar's ground does not move.

`src/gui/react_main_window.py` — `MainTabBookReact.set_theme`

```
        def set_theme(self, name: str) -> None:
            """Repaint the bar's tabs from ``name``'s own tab tokens."""
            self._theme = name
            self._push_tabs()
```

Accent Color - Sets the highlight colour. Key `accent_color`, a free-text field, at `#00ffcc`.

A plain text box. The default is placeholder text, not a value, so the field opens empty until something is typed or a stored value loads. Save stores what is there and the load path puts it back.

`src/gui/settings_dialog.py` — the Accent Color field

```
layout.addWidget(QLabel("Accent Color:"))
self._accent_color = QLineEdit()
self._accent_color.setPlaceholderText("#00ffcc")
layout.addWidget(self._accent_color)
```

An empty box means the theme's own accent. A hex colour in the box paints over it. Everything else is refused, and the theme's own accent paints.

The colour itself is now measured. A dark hex no longer paints on a dark theme as typed: `#1a1a2e` on the Cyberpunk Dark ground measures 1.16 to 1, where each theme's own accent measures 4.95 to 1 or better against its own ground.

Under the 4.5 floor that measure applies, `#1a1a2e` is refused and the Cyberpunk Dark accent paints in its place. All five themes clear that floor with their own accent, so every theme paints as it did.

The same colour can pass on one theme and fail on another. White is taken on the three dark themes and refused on Neon Light and Minimal Modern.

A refusal goes to the application log with the colour, the theme, the ground, the measured ratio and the floor. The box keeps what was typed.

The measure lives in `src/core/contrast.py`. It needs no chart library, so the built application reaches it.

`src/gui/theme_engine.py` — the gate a typed colour passes

```
def accented(theme: ThemeTokens, accent: object) -> ThemeTokens:
    taken = stored_accent(accent)
    if not taken:
        return theme
    clears, ratio = validate_contrast(taken, theme.bg_primary)
    if not clears:
        logger.warning(
            ACCENT_REFUSED.format(
                accent=taken,
                ratio=ratio,
                theme=theme.name,
                ground=theme.bg_primary,
                floor=AA_FLOOR,
            )
        )
        return theme
    return replace(theme, accent_primary=taken)
```

`src/gui/theme_engine.py` — what the box is read through

```
def stored_accent(value: object) -> str:
    """Return ``value`` when it is a hex colour, else ``THEME_ACCENT``.

    The store's accent field is free text, and a value ``generate_qss`` cannot
    read either drops its declaration or escapes it, so the reads in ``main``
    and in the main window pass through here.
    """
    if not isinstance(value, str):
        return THEME_ACCENT
    asked = value.strip()
    return asked if _ACCENT_HEX.match(asked) else THEME_ACCENT
```

Both fallbacks now come from one declaration. The store declares the theme field and the accent field. The theme engine and the settings dialog surface read them rather than writing either value out again.

`src/core/settings.py` — `AppSettings`, the one declaration

```
    theme: str = VisualTheme.CYBERPUNK_DARK.value
    accent_color: str = ""
```

`src/gui/theme_engine.py` — the accent sentinel, read from that field

```
THEME_ACCENT = AppSettings().accent_color
```

`src/gui/main_tabs/settings_dialog_surface.py` — both row fallbacks

```
THEME_DEFAULT = AppSettings().theme
ACCENT_DEFAULT = AppSettings().accent_color
```

`src/gui/settings_dialog.py` — the theme row, in `_stored_rows`

```
                (
                    "theme",
                    self._theme_combo.currentData,
                    lambda value: self._show_data(self._theme_combo, value),
                    THEME_DEFAULT,
                ),
```

Neither value changed. The accent is empty and the theme is Cyberpunk Dark, as before. Moving the store's two fields to another value moved five surfaces together: the row table, the React model, the bridge payload, the launch resolution and the Qt dialog. The same move before the change left every one of those five where it was.

The engine keeps a second theme name of its own, and that one answers a different question: which theme paints when a stored name is not a theme at all. It is taken from the engine's own Cyberpunk Dark tokens, so the name it falls back to is always a name the theme table carries. Both names read Cyberpunk Dark today.

A store saved before this change keeps every value it holds. One saved with Neon Light and the accent `#ff8800` opened on Neon Light and that colour on all four surfaces that show them, and the contrast measure still refused the colour at 2.2 to 1 against the Neon Light ground and painted that theme's own accent.

Three hex digits or six, either case, surrounding spaces trimmed. Each value below was typed in and the accent button was then read off an offscreen render.

typed	the accent button paints
`#ff0000`	#ff0000
`#0f0`	#00ff00
`#FF00AA`	#ff00aa
`#ff0000`	#ff0000
`red`	the theme's accent
`rgb(255,0,0)`	the theme's accent
`#00ffc`	the theme's accent
`#00ffccff`	the theme's accent
`notacolour`	the theme's accent
`#ff0000;`	the theme's accent
`url(x)`	the theme's accent
empty, spaces, or no value at all	the theme's accent

A colour name is refused on purpose: the box takes a hex colour, and the four forms Qt would also read bring a wider parser with nothing to paint that hex cannot.

The refusal matters because the box is free text. A value that ends one declaration and opens another repaints the window, and that was measured: with `red; } QWidget { background-color: #000000` standing in the accent token, a plain pane rendered 3,840 pixels of #000000 where its ground is #0a0a0f. The same string typed into the box now leaves that pane on its ground. A hex colour carries no semicolon and no brace, so no accepted value can reach that.

Six surfaces carry the accent. The figures are pixel counts off an offscreen render of each one.

surface	pixels
accent button	3,289
full progress bar	2,092
ticked box	464
slider handle	164
selected tab underline	134
focused text box border	304

A typed `#ff0000` moves all six, on all five themes. Text a theme paints from its own `text_accent` stays where it was: on Classic Terminal 160 pixels of #00ff00 remain after the override, 96 in the tab label and 64 in the checkbox label.

`main.py` — the startup read

```
    stored_hex = settings.get("accent_color", THEME_ACCENT)
    accent = stored_accent(stored_hex)
    if not accent and str(stored_hex).strip():
        log_manager.warning(
            f"Stored accent colour {stored_hex!r} is not a hex colour; "
            f"painting the {theme_name} accent"
        )
    theme_mgr.apply_theme(theme_name, app, accent)
```

A refused value is named on the Activity Log when the dialog saves and when the Theme menu switches. The box keeps what was typed, so the value and the line that says it did not paint are both on screen.

`src/gui/main_window.py` — `_switch_theme`, the refusal

```
            taken = stored_accent(accent)
            if not taken and str(accent or "").strip():
                self._status_log.log(
                    f"Accent colour {accent!r} is not a hex colour; "
                    f"painting the {name} accent.",
                    "warning",
                )
```

Driven with `notacolour` against Neon Light, that line read: Accent colour 'notacolour' is not a hex colour; painting the neon_light accent.

A colour the measure refuses now reaches the same pane. The theme engine builds one refusal line, the engine writes it to the application log, and the main window puts the same line on the Activity Log, so the two carry one set of figures.

`src/gui/theme_engine.py` — `accent_refusal`, the one line both carry

```
def accent_refusal(theme: ThemeTokens, accent: object) -> str:
    taken = stored_accent(accent)
    if not taken:
        return ""
    clears, ratio = validate_contrast(taken, theme.bg_primary)
    if clears:
        return ""
    return ACCENT_REFUSED.format(
        accent=taken,
        ratio=ratio,
        theme=theme.name,
        ground=theme.bg_primary,
        floor=AA_FLOOR,
    )
```

`src/gui/main_window.py` — `_switch_theme`, the pane he reads

```
            refusal = accent_refusal(THEMES[stored_theme(name)], taken)
            if refusal:
                self._status_log.log(refusal, "warning")
```

Driven through that method with a real Activity Log pane, `#1a1a2e` typed against Cyberpunk Dark put this on the pane: Accent colour #1a1a2e measures 1.16 to 1 against the cyberpunk_dark ground #0a0a0f, under the 4.5 floor; painting the cyberpunk_dark accent. The same press with `#ff8800`, which measures 8.25 to 1 on that ground, put no refusal on the pane.

The React tab pages do not follow the typed value. Each page receives an `--accent` variable built from a theme name alone, so the seven rules in the Settings page's own stylesheet that read that variable stay on the theme's accent.

`src/gui/main_tabs/react_history_panel_surface.py` — `palette`, which takes a theme and no accent

```
def palette(theme: str) -> dict:
    """The six chrome colours one theme paints the page in.

    A theme the table does not carry falls back to the default theme.
    """
    colors = THEME_CHROME.get(theme) or THEME_CHROME[DEFAULT_THEME]
    return {
        key: colors[source] for key, source in zip(PALETTE_KEYS, PALETTE_SOURCE_KEYS)
    }
```

Contrast is not checked. Every theme's own accent clears WCAG 2.2 AA against the ground it sits on, and a typed one is painted as given.

Font Family - Sets the typeface for application text. Key `font_family`, an editable combo over twelve named families, at Segoe UI.

The box is editable, so a family outside the twelve can be typed in. Save stores this value and the three sizes below it. Nothing reads any of the four, and the load path does not restore them either, so the whole Font

Settings group reopens at its built-in figures.

`src/gui/settings_dialog.py` — the Font Family row

```
  self._font_family.addItems(fonts)
  self._font_family.setCurrentText("Segoe UI")
  self._font_family.setToolTip("Font family for all application text")
  font_form.addRow("Font Family:", self._font_family)
```

Base Font Size - Sets the size of ordinary interface text. Key `font_size`, 8 pt to 24 pt, at 11 pt.

The preview line below the group follows this box and the family box as either changes. That preview is the only place either value has an effect.

`src/gui/settings_dialog.py` — the Base Font Size row

```
  self._font_size = QSpinBox()
  self._font_size.setRange(8, 24)
  self._font_size.setValue(11)
  self._font_size.setSuffix(" pt")
  self._font_size.setToolTip("Base font size for all UI text")
  font_form.addRow("Base Font Size:", self._font_size)
```

Heading Font Size - Sets the size of headings and stat-card values. Key `heading_font_size`, 10 pt to 32 pt, at 14 pt.

The widest of the three ranges. The preview does not follow this box.

`src/gui/settings_dialog.py` — the Heading Font Size row

```
  self._heading_size = QSpinBox()
  self._heading_size.setRange(10, 32)
  self._heading_size.setValue(14)
  self._heading_size.setSuffix(" pt")
  self._heading_size.setToolTip("Font size for headings and stat card values")
  font_form.addRow("Heading Font Size:", self._heading_size)
```

Log Font Size - Sets the size of the Activity Log and API Log text. Key `log_font_size`, 8 pt to 18 pt, at 10 pt.

The narrowest of the three ranges, and the last row before the preview.

`src/gui/settings_dialog.py` — the Log Font Size row

```
  self._log_font_size = QSpinBox()
  self._log_font_size.setRange(8, 18)
  self._log_font_size.setValue(10)
  self._log_font_size.setSuffix(" pt")
```

Preview redraws in the chosen family and base size as either changes. It is a sample line, not a setting, and nothing stores it.

The five theme names come from one mapping built off the five theme objects.

`src/gui/theme_engine.py` — `THEMES`

```
THEMES: dict[str, ThemeTokens] = {
    t.name: t
    for t in [CYBERPUNK_DARK, NEON_LIGHT, CLASSIC_TERMINAL, MINIMAL_MODERN, GLASS_METAL]
}
```

Save writes the theme, the Accent Color field and all four font values.

`src/gui/settings_dialog.py` — `_save`, the theme and font keys

```
"theme": lambda: self._theme_combo.currentData(),
"accent_color": lambda: self._accent_color.text().strip(),
"font_family": lambda: self._font_family.currentText(),
"font_size": lambda: self._font_size.value(),
"heading_font_size": lambda: self._heading_size.value(),
"log_font_size": lambda: self._log_font_size.value(),
```

The load path reads back the theme and the accent field only, so the four font rows open at Segoe UI, 11, 14 and 10 whatever was stored.

Of the six, the theme alone reaches the window. `main.py` reads it at startup, and the window reads it again in `_on_settings_changed` when the dialog saves.

The other five are stored and never read again. Each palette carries its own typeface in the theme object it declares, so the theme decides the font and the four font rows decide nothing. The accent field has the same shape: declared, written, read back into its own line edit, and painted nowhere.

`src/core/settings.py` — `AppSettings`, the five that are stored and unread

```
accent_color: str = "#00ffcc"

# These four mirror the font widgets in settings_dialog._create_theme_tab.
font_family: str = "Segoe UI"
font_size: int = 11
heading_font_size: int = 14
log_font_size: int = 10
```

Of those five, the accent field now reaches the window. The four font rows do not. The stored default for the accent is empty, so a store that has never been typed into leaves every theme on its own accent rather than handing Cyberpunk Dark's to the other four.

`src/core/settings.py` — `AppSettings`, the accent field today

```
    theme: str = VisualTheme.CYBERPUNK_DARK.value
    # Empty leaves each theme's own accent painting. theme_engine.stored_accent
    # takes a hex colour here and refuses anything else.
    accent_color: str = ""
```

The Font Settings group is gone. Font Family, Base Font Size, Heading Font Size and Log Font Size are no longer on the Theme page, the preview line went with them, and the four stored keys are no longer fields of the

settings record. The theme declares the typography instead: every palette carries a typeface, a monospace face and four sizes.

`src/gui/theme_engine.py` — the typography tokens a theme declares

```
    # Font
    font_family: str = "'Rajdhani', 'Orbitron', 'Segoe UI', sans-serif"
    font_mono: str = "'JetBrains Mono', 'Fira Code', 'Consolas', monospace"
    font_size: str = "13px"
    font_size_small: str = "11px"
    font_size_large: str = "16px"
    font_size_title: str = "20px"
```

The monospace face and the large size are gone from the palette. No rule in the style-sheet text named either one, so neither is a value a palette declares, and a palette now carries a typeface and three sizes.

`src/gui/theme_engine.py` — the typography tokens a theme declares today

```
    # Font
    font_family: str = "'Rajdhani', 'Orbitron', 'Segoe UI', sans-serif"
    font_size: str = "13px"
    font_size_small: str = "11px"
    font_size_title: str = "20px"
```

Three of the five palettes name a typeface of their own. Classic Terminal takes a monospace family, Minimal Modern and Glass & Metal each name their own, and the other two keep the declared default above.

```
# CLASSIC_TERMINAL
font_family="'JetBrains Mono', 'Fira Code', 'Consolas', monospace",

# MINIMAL_MODERN
font_family="'SF Pro Display', 'Inter', 'Segoe UI', sans-serif",

# GLASS_METAL
font_family="'Exo 2', 'Rajdhani', 'Segoe UI', sans-serif",
```

The stylesheet generator reads those tokens, so the typeface and the size arrive with the palette rather than from a control.

`src/gui/theme_engine.py` — `generate_qss`, the global rule

```
QWidget {{
    background-color: {t.bg_primary};
    color: {t.text_primary};
    font-family: {t.font_family};
    font-size: {t.font_size};
    selection-background-color: {t.accent_primary};
    selection-color: {t.bg_primary};
}}
```

A typeface the operator sets would have split that. The theme engine audits a palette as one set and says so in its own opening words, and a size set outside the palette moves the text the audit was taken against while nothing asks for the audit again.

`src/gui/theme_engine.py` — the module's own rule

```
Every ``ThemeTokens`` pair meets WCAG 2.2 AA contrast, and a changed hex value
needs a fresh audit.
```

The Theme page now carries two controls, the theme picker and the accent field. The dialog payload carries 68 controls where it carried 72, the rendered page draws 55 where it drew 59, and Save writes nine top-level keys where it wrote thirteen. The five generated stylesheets are unchanged byte for byte, because the removal touched no palette.

Dropping the monospace face and the large size did touch the palette, so each of the five style sheets is shorter than that. 08-tabs.md holds the character count each one writes today.

A settings file written before the removal still loads. The load copies only the fields the record declares, so a font key in such a file is read past and every other setting arrives at its stored value.

`src/core/settings.py` — `SettingsManager._apply_dict`

```
defaults = asdict(AppSettings())
for key, default_val in defaults.items():
    if key in data:
        setattr(self._settings, key, data[key])
```

Driven on a file holding all four font keys beside nine other settings: nothing raised, and the username, the target balance, the visibility, the aggressive flag, the theme, the accent, the three phantom values, the indicator weights and the logging group all arrived as stored.

The Simulator's tone, nigredo

Every theme has a second tone, and one widget tree paints in it: the Simulator tab. The tone is the theme's own seven ground tokens moved part of the way to black. The accent, the text, the borders and the chart colours stay the theme's, so a Simulator screen reads as the same theme, darker.

`src/gui/theme_engine.py` — the derivation

```
NIGREDO_FRACTION = 0.25

NIGREDO_GROUNDS = (
    "bg_primary",
    "bg_secondary",
    "bg_tertiary",
    "bg_card",
    "bg_input",
    "bg_hover",
    "bg_selected",
)


def nigredo(theme: ThemeTokens, fraction: float = NIGREDO_FRACTION) -> ThemeTokens:
    moved = {
        name: toward_black(getattr(theme, name), fraction) for name in NIGREDO_GROUNDS
    }
    return replace(theme, **moved)
```

One function moves a colour. Each sRGB channel is scaled by one minus the fraction and rounded, so a fraction of 0 leaves the colour as it is and a fraction of 1 gives black. The function takes six-digit hex and refuses any other form.

```
def toward_black(colour: str, fraction: float) -> str:
    keep = 1.0 - fraction
    return "#" + "".join(
        f"{round(int(channel, 16) * keep):02x}" for channel in found.groups()
    )
```

The fraction is one number. Every theme's ground, its darkened ground, and the contrast of that theme's body text on each, WCAG 2.2 ratio:

```
theme             ground    nigredo   text      ratio on ground  ratio on nigredo
cyberpunk_dark    #0a0a0f   #08080b   #e0e0f0   15.13            15.33
neon_light        #f5f5fa   #b8b8bc   #1a1a2e   15.70             8.63
classic_terminal  #0a0a0a   #080808   #00ff00   14.43            14.60
minimal_modern    #fafafa   #bcbcbc   #1a1a1a   16.67             9.16
glass_metal       #1c1c24   #15151b   #d0d0e0   11.10            11.93
```

Every row is above 4.5 to 1. On the three dark themes the ratio rises. On the two light themes it falls, because their text is dark and a darker ground moves toward it; it stays above 4.5 to 1 up to a fraction near 0.45, and at 0.6 Neon Light reads 2.80 and fails. On a near-black theme the ground moves by two to seven per channel, which the picture reports and the eye does not; the raised grounds, the cards and the tables, move by more.

The stylesheet carries the tone. `generate_qss` ends with the theme's rules restated a second time, from the darkened tokens, each rule under one selector that names a Qt dynamic property.

`src/gui/theme_engine.py` — the scope and the block

```
TONE_PROPERTY = "tone"
NIGREDO = "nigredo"

NIGREDO_SELECTOR = f'QWidget[{TONE_PROPERTY}="{NIGREDO}"]'


def generate_qss(theme: ThemeTokens) -> str:
    return f"{_theme_qss(theme)}\n/* --- Simulator: {NIGREDO} --- */{nigredo_qss(theme)}\n"
```

The block's first rule, under Cyberpunk Dark:

```
QWidget[tone="nigredo"], QWidget[tone="nigredo"] QWidget {
    background-color: #08080b;
    color: #e0e0f0;
    font-family: 'Rajdhani', 'Orbitron', 'Segoe UI', sans-serif;
    font-size: 13px;
    selection-background-color: #00ffcc;
    selection-color: #08080b;
}
```

A widget that sets the property `tone` to `nigredo` paints in the tone, and so does every widget under it, including a dialog it parents. The Simulator tab sets it in its constructor, in both builds. A theme switch re-applies the whole stylesheet, so the tone follows the theme with no second step.

`src/gui/simulator/sim_trading_tab.py` — the tab takes the tone

```
self.setProperty(TONE_PROPERTY, NIGREDO)
self.setAttribute(Qt.WA_StyledBackground, True)
```

A React page reads the tone the same way. The page's `:root` chrome comes from `page_html`, which takes a tone and puts the two grounds through the same function; a theme switch runs `repaint_pages`, which reads the tone off each web view's property and pushes that view its own chrome.

`src/gui/react_history_panel.py` — the page chrome in a tone

```
if tone == NIGREDO:
    ground = toward_black(ground, NIGREDO_FRACTION)
    button = toward_black(button, NIGREDO_FRACTION)
```

Diagram: flowchart, left to right

- _switch_theme -> **ThemeManager.apply_theme**
- ThemeManager.apply_theme -> **generate_qss**
- generate_qss -> **_theme_qss: every widget**
- generate_qss -> **nigredo_qss: the tone=nigredo tree**
- _switch_theme -> **repaint_pages**
- repaint_pages -> **palette_script: a Live view**
- repaint_pages -> **palette_script with NIGREDO: a Sim view**

The Simulator page under the Simulator tab states what each build paints in the tone and what was read off the renders.

One answer says which theme the application is painted in. The theme engine records the colours it last painted, and every site that needs them reads that record, including the theme holder itself. A holder built fresh on a menu press and a chart asking later now give the same answer, accepted accent included.

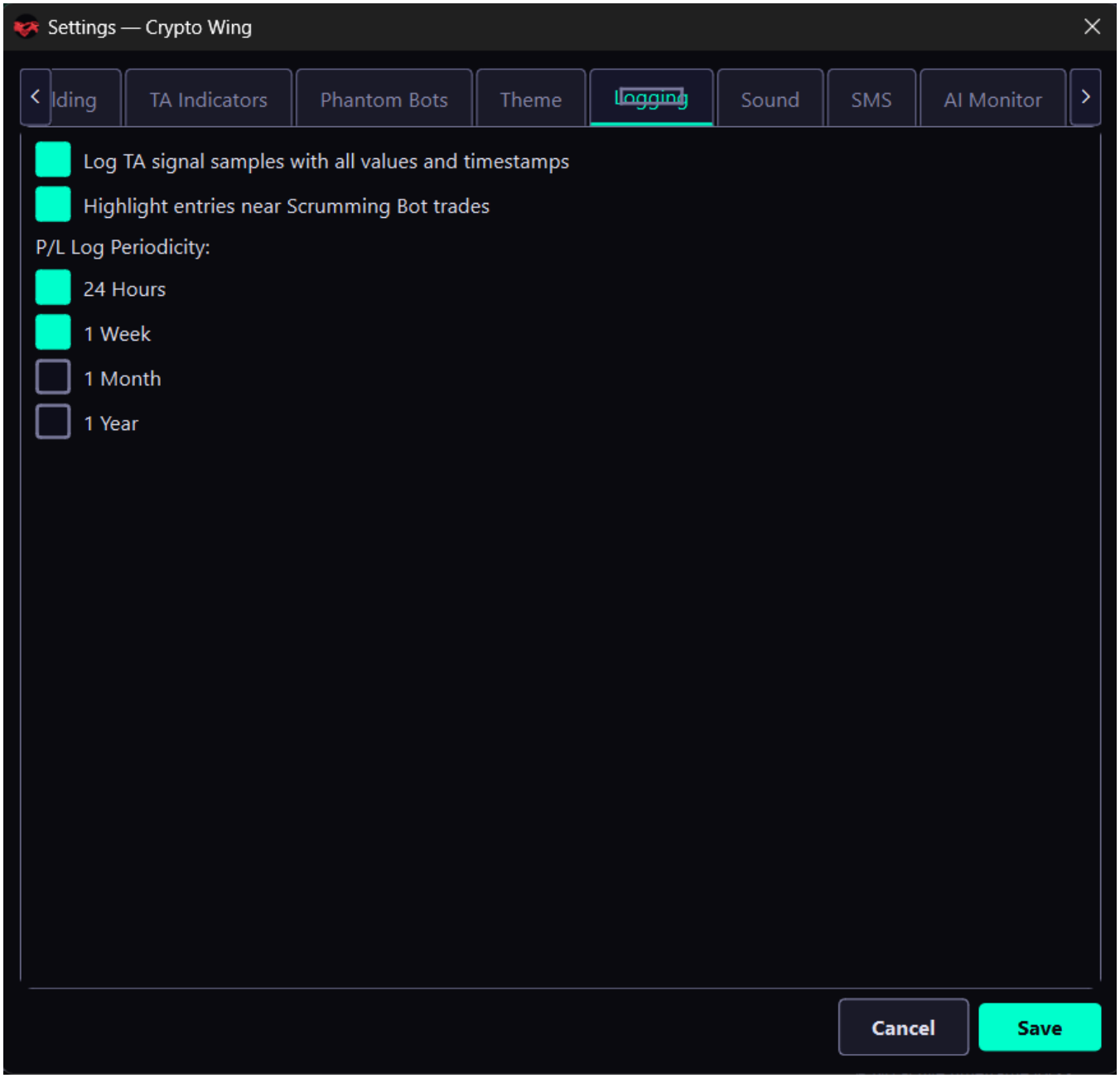
`src/gui/theme_engine.py` — `applied_tokens`, the one answer

```
  def applied_tokens() -> ThemeTokens:
      """Return the ``ThemeTokens`` ``ThemeManager.apply_theme`` last painted.

      Every site that paints from the application's theme reads this one answer,
      including ``ThemeManager.current``, so a second manager and a rebuild from
      ``applied_theme`` cannot hold different accents.
      """
      return _applied_tokens
```

Four sites ask: the holder that applied the theme, a holder built later, the Charts tab, and the native chart's own colour roles. Driven with Cyberpunk Dark and the accent `#ff8800`, all four read that accent. Moving the record to Glass Metal moved all four together; on the tree before this change two of the four moved, one held its own copy and one answered nothing at all.

Settings > Logging



Settings, the Logging page.

Two checkboxes and four periodicity boxes. Every key below sits inside one stored `data_logging` group.

Log TA signal samples with all values and timestamps - Writes each indicator reading out with its value and its time. Key `ta_signal_logging`, a checkbox, checked at build.

The first of the two flags on the page, and the one that decides whether the indicator record is written at all.

`src/gui/settings_dialog.py` — the TA logging checkbox

```
self._ta_logging = QCheckBox(
    "Log TA signal samples with all values and timestamps"
)
self._ta_logging.setChecked(True)
layout.addWidget(self._ta_logging)
```

Highlight entries near Scrumming Bot trades - Marks the log lines that sit close to a fire, so a decision can be read in its own context. Key `highlight_trade_proximity`, a checkbox, checked at build.

The second flag, built the same way and checked at build like the first.

`src/gui/settings_dialog.py` — the highlight checkbox

```
self._highlight_trades = QCheckBox(
    "Highlight entries near Scrumming Bot trades"
)
self._highlight_trades.setChecked(True)
layout.addWidget(self._highlight_trades)
```

P/L Log Periodicity - Chooses the windows a profit and loss figure is summarised over: 24 Hours, 1 Week, 1 Month and 1 Year. Key `active_periodicities`, four boxes written as a list, with the first two checked at build.

Four separate boxes rather than one control. Save walks them in page order and writes the checked ones into a list.

`src/gui/settings_dialog.py` — the four periodicity boxes

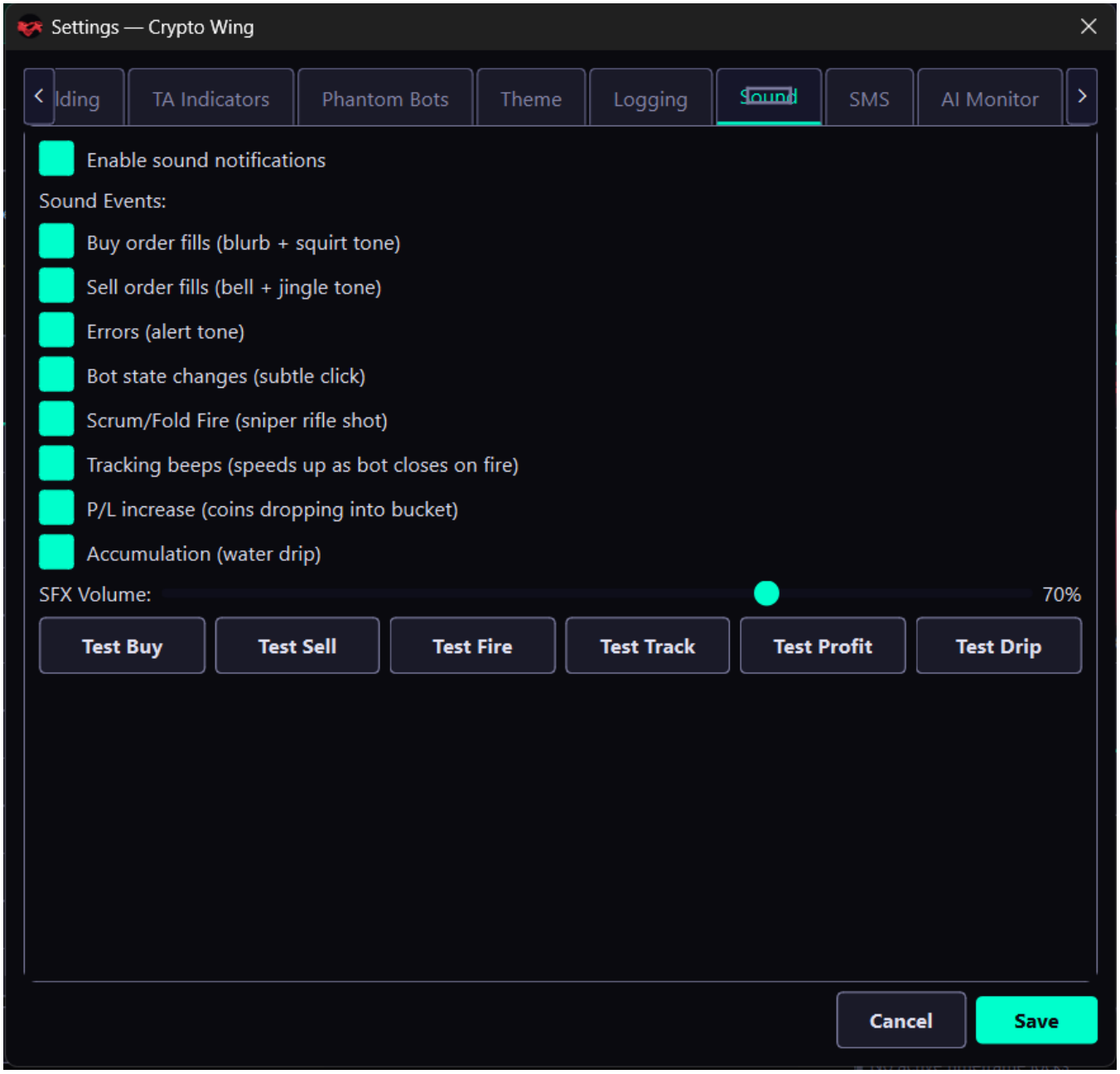
```
self._log_24h = QCheckBox("24 Hours")
self._log_24h.setChecked(True)
self._log_1w = QCheckBox("1 Week")
self._log_1w.setChecked(True)
self._log_1m = QCheckBox("1 Month")
self._log_1y = QCheckBox("1 Year")
```

Save collapses all six into one dictionary holding the two flags and the list of active periodicities. The load path reads none of them back, so every open shows the build state rather than the stored one. No reader outside the dialog reads the group either, which leaves the six controls writing to a key nothing consults. Issue #423 carries this page.

In development. What the load path should restore for this page has not been settled against the rest of the dialog, so nothing is proposed here.

Issue #423 closed on 2026-09-25, so the pointer names no open work. What the load path should restore for this page is still unsettled, and it is a product choice, so nothing is proposed here either.

Settings > Sound



Settings, the Sound page.

The master switch, eight event checkboxes, a volume slider and six test buttons. Every checkbox is checked at build.

Each checkbox names its sound in its own label, and the key behind each control is a field of the sound configuration the block below builds.

Enable sound notifications - Turns every sound on or off at once. Key `enabled`, on.

The engine tests this flag before it tests any other, so a clear box silences the page whatever the eight below are set to.

`src/gui/settings_dialog.py` — the master switch

```
self._sound_enabled = QCheckBox("Enable sound notifications")
self._sound_enabled.setChecked(True)
self._sound_enabled.setToolTip("Master switch for all audio notifications")
```

Current reading, 2026-10-29. The words are the same. Each Settings control's sentence is now the `tooltip` of its entry in the control declarations, and the dialog sets them all once its tabs are built.

`src/gui/settings_dialog.py` — `_describe_controls`

```
def _describe_controls(self) -> None:
    for spec in sds.CONTROL_SPECS:
        said = spec.get("tooltip")
        held = getattr(self, f"_{spec['name']}", None)
        if said and held is not None:
            set_description(held, said)
```

Buy order fills - Plays a low rising tone when a buy fills. Key `buy_sound`, on.

The label carries the sound's own name in brackets, as every box on this page does. The engine plays this one under the name buy.

`src/gui/settings_dialog.py` — the buy fill box

```
self._sound_buy = QCheckBox("Buy order fills (blurb + squirt tone)")
self._sound_buy.setChecked(True)
self._sound_buy.setToolTip(
    "Plays a low bubbly rising tone when a buy order is filled"
)
```

Sell order fills - Plays a bright bell when a sell fills. Key `sell_sound`, on.

The counterpart to the box above, played under the name sell.

`src/gui/settings_dialog.py` — the sell fill box

```
self._sound_sell = QCheckBox("Sell order fills (bell + jingle tone)")
self._sound_sell.setChecked(True)
self._sound_sell.setToolTip(
    "Plays a high bright bell tone when a sell order is filled"
)
```

Errors - Plays an alert tone on an error. Key `error_sound`, on.

One of the two boxes on the page with no tooltip of its own.

`src/gui/settings_dialog.py` — the error box

```
self._sound_error = QCheckBox("Errors (alert tone)")
self._sound_error.setChecked(True)
layout.addWidget(self._sound_error)
```

Bot state changes - Plays a click when a bot starts, pauses or stops. Key `bot_state_sound`, on.

The other box with no tooltip. The engine plays it under the name state, not under the key.

`src/gui/settings_dialog.py` — the bot state box

```
self._sound_state = QCheckBox("Bot state changes (subtle click)")
self._sound_state.setChecked(True)
layout.addWidget(self._sound_state)
```

Scrum/Fold Fire - Plays a rifle shot when a scrum or a fold executes, and on a Manual Fire. Key `fire_sound`, on.

This is the first of the four the engine reads with a default rather than straight off the configuration. A stored configuration written before these four existed leaves all four on.

`src/gui/settings_dialog.py` — the fire box

```
self._sound_fire = QCheckBox("Scrum/Fold Fire (sniper rifle shot)")
self._sound_fire.setChecked(True)
self._sound_fire.setToolTip(
    "Synthesized rifle shot plays when a scrum or fold\n"
    "actually executes. Also plays on Manual Fire."
)
```

Current reading, 2026-10-29. The words are the same, and the dialog sets them from this box's control declaration, as it does the master switch above.

Tracking beeps - Paces a beep by the scrum phase: silent in SEARCH, slow in TRACK, fast in FIRE. Key `track_sound`, on.

The tooltip gives the two paces in milliseconds: 800 in TRACK and 200 in FIRE.

`src/gui/settings_dialog.py` — the tracking box

```
self._sound_track = QCheckBox(
    "Tracking beeps (speeds up as bot closes on fire)"
)
self._sound_track.setChecked(True)
```

P/L increase - Plays coins dropping into a bucket on any trade that realised a profit. Key `profit_sound`, on.

The tooltip is the only place on the page that names the condition exactly: realised profit above zero, on grid and scrumming bots alike.

Read today: the coins follow the profit a fill's announcement carries. A fold, a Manual Fire, cartridge or wire-stack buy, and a manual tranche fire carry the target growth they applied. A detonation carries the

position's value above its anchor target, and an Extractor exit carries its gain. Every other fill, a scrum and a distribution sale among them, carries $0.00 and plays no coins.

Now: the wire stack is gone, and the growth a cartridge, Manual Fire or manual tranche fire carries reads each tranche's sale price less its wire share (`src/trading/scrumming/sizing.py, in discharge_gain_usd`).

```
# src/gui/main_tabs/main_window_surface.py, in trade_filled_sounds
    if profit > 0:
        played.append(PROFIT_SOUND)
```

`src/gui/settings_dialog.py` — the profit box

```
self._sound_profit = QCheckBox("P/L increase (coins dropping into bucket)")
self._sound_profit.setChecked(True)
```

Accumulation - Plays a water drip on a FOLD, and on nothing else. Key `drip_sound`, on.

The last of the nine. Its tooltip calls a fold the canonical accumulation moment and states that a scrum or a distribution does not fire it.

`src/gui/settings_dialog.py` — the accumulation box

```
self._sound_drip = QCheckBox("Accumulation (water drip)")
self._sound_drip.setChecked(True)
```

SFX Volume - Sets how loud every sample plays. Key `volume`, 0 % to 100 %, at 70 %.

Test Buy, Test Sell, Test Fire, Test Track, Test Profit and Test Drip play one sample each. They are buttons, not settings.

One handler is the only path from this page to the engine. It builds a configuration from every checkbox and the slider, pushes it in and clears the sample cache, because each sample bakes its volume at synthesis time.

`src/gui/settings_dialog.py` — `_on_sfx_volume_changed`

```
se = get_sound_engine()
new_cfg = SoundConfig(
    enabled=self._sound_enabled.isChecked(),
    buy_sound=self._sound_buy.isChecked(),
    sell_sound=self._sound_sell.isChecked(),
    error_sound=self._sound_error.isChecked(),
    bot_state_sound=self._sound_state.isChecked(),
    fire_sound=self._sound_fire.isChecked(),
    track_sound=self._sound_track.isChecked(),
    profit_sound=self._sound_profit.isChecked(),
    drip_sound=self._sound_drip.isChecked(),
    volume=v / 100.0,
)
```

That path runs when the slider moves or a test button is pressed, and at no other time. Save reads no widget on this page and the schema declares no sound field, so nothing here survives the dialog closing. A checkbox cleared and the dialog saved comes back checked. Issue #423 carries it.

Issue #423 closed on 2026-09-25. The schema declares `sound`, which holds the Sound page's nine switches and its volume, and `src/core/sound_engine.py, in sound_config_from_settings` reads that group back.

The engine now drops its own clips. `src/core/sound_engine.py`, in `SoundEngine.update_config`, takes the new configuration and clears the cache itself, so no caller reaches into the engine to do it. The handler on this page hands the configuration over and stops there. The same method also removes the directory the old clips were written to, which the page used to leave behind on every slider step.

`src/core/sound_engine.py` — `SoundEngine.update_config`

```
self._config = config
self.cleanup()
self._temp_dir = None
self._available = False
self._cache = {}
```

`src/core/sound_engine.py` — `SoundConfig`, the ten fields and their defaults

```
enabled: bool = True
buy_sound: bool = True
sell_sound: bool = True
error_sound: bool = True
bot_state_sound: bool = True
fire_sound: bool = True
track_sound: bool = True
profit_sound: bool = True
drip_sound: bool = True
volume: float = 0.7
```

Every tick box reaches the engine on its own. Each box's toggle runs the same push the slider runs, so a box cleared with nothing else touched silences that sound at once.

`src/gui/settings_dialog.py` — each box wired to the push

```
for _key, name in SOUND_CONFIG_FIELDS:
    getattr(self, f"_{name}").toggled.connect(self._push_sound_config)
```

The push reads the slider itself, so one method serves a box, the slider and a Test button alike.

`src/gui/settings_dialog.py` — the push

```
def _push_sound_config(self) -> None:
    self._on_sfx_volume_changed(self._sound_volume.value())
```

The React page draws the same ten controls. Its edit route named one handler, for the slider, and that handler wanted an argument the page never sends, so the press raised and reached nothing. Every sound control names the push, which wants no argument.

`src/gui/react_settings_dialog.py` — the edit route

```
EDIT_HANDLERS: dict[str, str] = {
    "sms_carrier": "_fill_gateway_email",
    surface.VOLUME_NAME: "_push_sound_config",
    **{name: "_push_sound_config" for _key, name in surface.SOUND_CONFIG_FIELDS},
}
```

The settings file holds the page as one group, and the schema declares that group from the engine's own defaults, so the two cannot disagree over what the page holds.

`src/core/settings.py` — the group on the schema

```
sound: dict = field(default_factory=lambda: asdict(SoundConfig()))
```

Save and load walk one row list for the group, the way the Profit Folding and AI Monitor groups do, so no row reaches the file without also reaching a control. The slider shows whole percent and the engine holds a fraction, so the volume row divides out and multiplies back.

`src/gui/settings_dialog.py` — the sound rows

```
switches = tuple(
    (
        key,
        getattr(self, f"_{name}").isChecked,
        show(getattr(self, f"_{name}")),
        getattr(built, key),
    )
    for key, name in SOUND_CONFIG_FIELDS
)
```

One reader carries a stored group into the engine, and it keeps the default for anything the generators cannot take. A switch reads as on or off: the play gate tests it for truth, so a non-empty word would open a sound the operator closed. A volume reads as a finite number from zero to one: each generator multiplies it into every sample before packing the samples, and a volume of four leaves the engine holding no clips and playing nothing.

`src/core/sound_engine.py` — the reader

```
def sound_config_from_settings(stored: Optional[dict]) -> SoundConfig:
    taken = SoundConfig()
    for name, value in (stored or {}).items():
        if name in SOUND_SWITCHES:
            if isinstance(value, bool):
                setattr(taken, name, value)
```

The engine takes the stored group at startup, before the first fill, so a silenced sound stays silent whether or not the operator opens this page.

`main.py` — the startup push

```
    get_sound_engine().update_config(
        sound_config_from_settings(settings.get("sound", {}))
    )
```

Two of the nine switches gate a clip no trade path plays. Nothing calls the buy helper or the sell helper, so Test Buy and Test Sell are the only presses that reach those two samples. Both boxes store and reload, and the sound itself waits on a caller.

`src/core/sound_engine.py` — the two helpers with no caller

```
    def play_buy(self) -> None:
        self.play("buy")

    def play_sell(self) -> None:
        self.play("sell")
```

A filled order now plays its own clip. `src/gui/main_window.py`, in `_on_trade_filled_sfx`, reads the fill's side and calls the buy helper on a filled buy and the sell helper on a filled sell. One function decides every clip a fill plays: `src/gui/main_tabs/main_window_surface.py`, in `trade_filled_sounds`. The engine still tests each clip's own switch, so a box cleared here keeps that clip silent while the rest of the fill still sounds.

Nine places in the bot emit a fill. Four pass the fill's fields as keywords and five nest them under one `data` field. `main_window_surface.py`, in `fill_fields`, reads the fill at either level, so the manual scrum, the manual fold, the auto detonation, the manual tranche fold and the self destruct reach the sounds alongside the five that already did.

`src/gui/main_tabs/main_window_surface.py` — the side decides the clip

```
    taken_side = (side or "").upper()
    if taken_side == BUY_SIDE:
        played.append(BUY_SOUND)
    elif taken_side == SELL_SIDE:
        played.append(SELL_SOUND)
```

The current reading, 2026-10-29. Every place in the bot that books a fill now sends it in one flat shape, so `fill_fields` reads one level only. It reads the fill through `read_settled_fill`, which refuses a fill that is not in that shape, and all eighteen kinds of fill reach the sounds.

`src/gui/main_tabs/main_window_surface.py` — the one shape the sounds read

```
        return read_settled_fill(data).payload()
```

The six Test buttons carry a part name of their own, and the page's click listener reports the plain button part alone, so a press on the React page reports nothing to the dialog. Those six are buttons, not settings, and the switches beside them do not depend on them.

`src/gui/web/settings_dialog.js` — the two part names

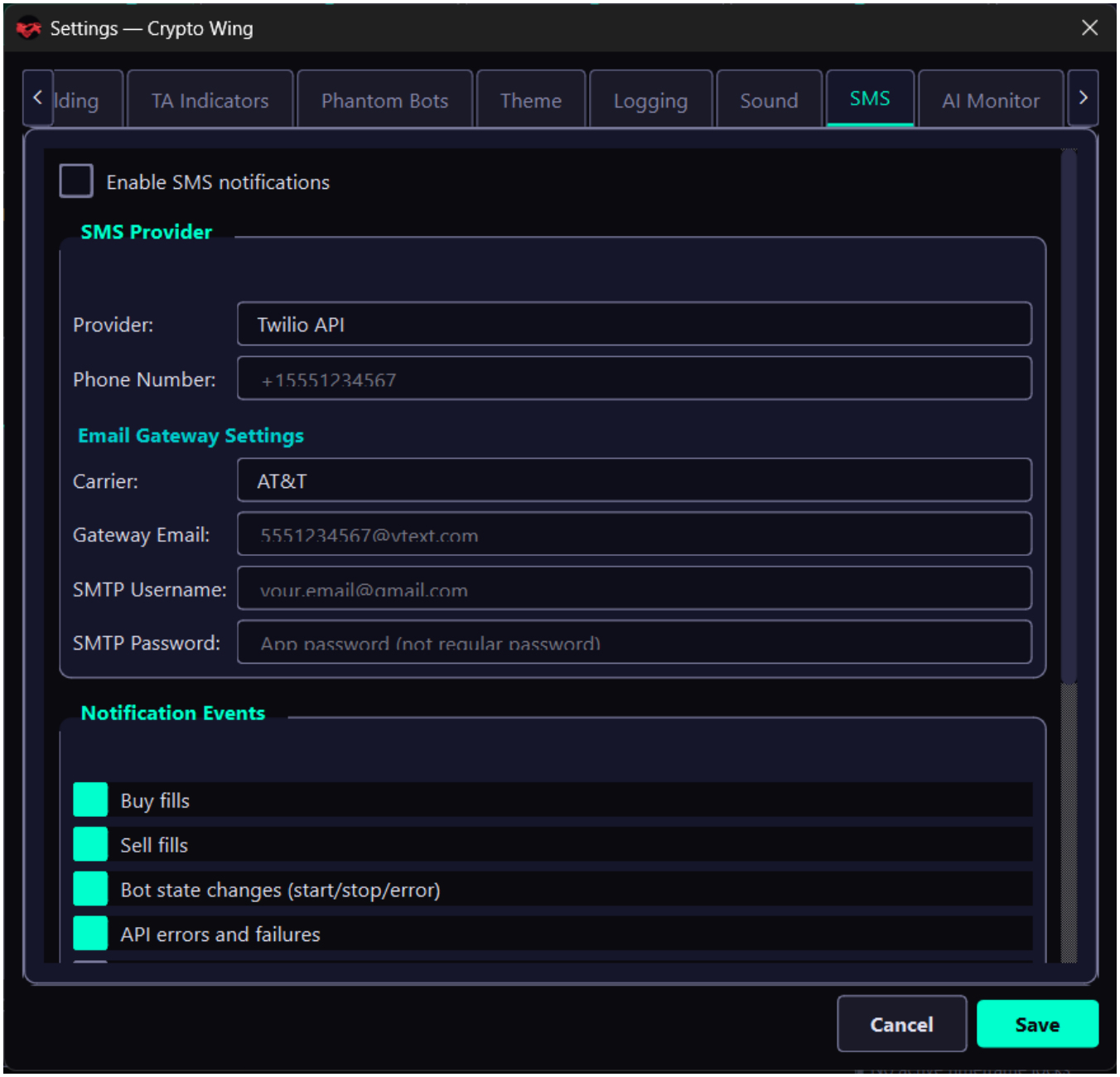
```
var BUTTON_PART = "button";
var SOUND_BUTTON_PART = "sound-button";
```

A press on the React page now reaches the dialog. The page's click listener in `src/gui/react_settings_dialog.py`, in `HOST_SCRIPT`, recognises the sound button part and reports the sound the pressed button names. `SettingsDialogReact.run_action` routes that report to `SettingsDialog._test_sound`, which pushes the page's switches and plays the one clip. A name the page did not draw is refused and written to the log, because `SOUND_PRESS_NAMES` holds the six names `SOUND_TEST_BUTTONS` declares.

`src/gui/react_settings_dialog.py` — the branch that reports a Test press

```
if (part === "sound-button") {
  console.log(
    "acervator-act:" +
      JSON.stringify({ sound: at.getAttribute("data-name") })
  );
  return;
}
```

Settings > SMS



Settings, the SMS page.

A scrolling page holding the master switch and three groups.

Seventeen controls, and the key behind each is a field of the message configuration the engine reads.

Enable SMS notifications - Turns every message on or off at once. Key `enabled`, clear at build.

The only control on the page outside the three groups, and the only checkbox here that opens clear rather than checked.

`src/gui/settings_dialog.py` — the SMS master switch

```
self._sms_enabled = QCheckBox("Enable SMS notifications")
self._sms_enabled.setToolTip(
    "Send text messages to your phone for trading events"
)
layout.addWidget(self._sms_enabled)
```

Current reading, 2026-10-29. The words are the same, and the dialog sets them from this box's control declaration.

Provider - Chooses how a message leaves the machine: a carrier's email gateway, or the Twilio web interface. Key `provider`, at the email gateway. Nothing loads a stored choice into the picker, so it opens on the first entry whatever the figure shows.

Two entries, both written as display labels. The engine tests a short code rather than either label, so neither entry matches what the send path looks for.

`src/gui/settings_dialog.py` — the Provider row

```
self._sms_provider = QComboBox()
self._sms_provider.setMinimumHeight(28)
self._sms_provider.addItems(["Email-to-SMS Gateway", "Twilio API"])
pf.addRow("Provider:", self._sms_provider)
```

Phone Number - Sets the number every message goes to. Key `phone_number`, empty, with a placeholder showing the international shape.

The placeholder is the only guidance on the row. It is grey sample text, not a value, so the field starts genuinely empty.

`src/gui/settings_dialog.py` — the Phone Number row

```
self._sms_phone = QLineEdit()
self._sms_phone.setMinimumHeight(28)
self._sms_phone.setPlaceholderText("+15551234567")
pf.addRow("Phone Number:", self._sms_phone)
```

Carrier - Names the carrier whose gateway address a message would be built for. Ten entries, opening on the first. It has no stored field of its own, and nothing on the page reads the choice.

The ten names come from one mapping in the engine. The last entry is a manual option whose address is deliberately empty.

`src/gui/settings_dialog.py` — the Carrier row

```
self._sms_carrier = QComboBox()
self._sms_carrier.setMinimumHeight(28)
from src.core.sms_engine import CARRIER_GATEWAYS

for carrier in CARRIER_GATEWAYS:
    self._sms_carrier.addItem(carrier)
pf.addRow("Carrier:", self._sms_carrier)
```

Gateway Email - Sets the full gateway address a message is sent to. Key `gateway_email`, empty.

The row a carrier choice would otherwise fill in. Nothing builds the address from the picker above, so this field is typed by hand or left blank.

`src/gui/settings_dialog.py` — the Gateway Email row

```
  self._sms_gateway = QLineEdit()
  self._sms_gateway.setMinimumHeight(28)
  self._sms_gateway.setPlaceholderText("5551234567@vtext.com")
  self._sms_gateway.setToolTip("Full email address for carrier SMS gateway")
  pf.addRow("Gateway Email:", self._sms_gateway)
```

Current reading, 2026-10-29. The words are the same, and the dialog sets them from this row's control declaration.

SMTP Username - Sets the mailbox a message is sent from. Key `smtp_username`, empty.

The first of the two mailbox rows. Its placeholder shows an ordinary address.

`src/gui/settings_dialog.py` — the SMTP Username row

```
  self._sms_smtp_user = QLineEdit()
  self._sms_smtp_user.setMinimumHeight(28)
  self._sms_smtp_user.setPlaceholderText("your.email@gmail.com")
  pf.addRow("SMTP Username:", self._sms_smtp_user)
```

SMTP Password - Sets that mailbox's application password. Key `smtp_password`, masked, empty.

The only masked field on the page. Its placeholder says an application password rather than the account password.

`src/gui/settings_dialog.py` — the SMTP Password row

```
  self._sms_smtp_pass = QLineEdit()
  self._sms_smtp_pass.setMinimumHeight(28)
  self._sms_smtp_pass.setEchoMode(QLineEdit.Password)
  self._sms_smtp_pass.setPlaceholderText(
      "App password (not regular password)"
  )
  pf.addRow("SMTP Password:", self._sms_smtp_pass)
```

Buy fills - Sends a message when a buy fills. Key `notify_buy_fills`, checked at build.

The first of the eight event rows, and one of the four that open checked.

`src/gui/settings_dialog.py` — the buy fills row

```
  self._sms_buy = QCheckBox("Buy fills")
  self._sms_buy.setChecked(True)
  ef.addRow(self._sms_buy)
```

Sell fills - Sends a message when a sell fills. Key `notify_sell_fills`, checked at build.

The counterpart to the row above, built and checked the same way.

`src/gui/settings_dialog.py` — the sell fills row

```
  self._sms_sell = QCheckBox("Sell fills")
  self._sms_sell.setChecked(True)
  ef.addRow(self._sms_sell)
```

Bot state changes - Sends a message when a bot starts, stops or errors. Key `notify_bot_state_changes`, checked at build.

The label names the three states in brackets, which the page's own text does not repeat anywhere else.

`src/gui/settings_dialog.py` — the bot state row

```
  self._sms_state = QCheckBox("Bot state changes (start/stop/error)")
  self._sms_state.setChecked(True)
  ef.addRow(self._sms_state)
```

API errors and failures - Sends a message when a venue call fails. Key `notify_errors`, checked at build.

The last of the four rows that open checked.

`src/gui/settings_dialog.py` — the API errors row

```
  self._sms_errors = QCheckBox("API errors and failures")
  self._sms_errors.setChecked(True)
  ef.addRow(self._sms_errors)
```

P/L threshold alerts - Sends a message when profit or loss passes a dollar figure. Key `notify_pl_threshold`, clear at build.

The first row in the group that opens clear. It gates the dollar figure on the row below it.

`src/gui/settings_dialog.py` — the P/L alert row

```
  self._sms_pl = QCheckBox("P/L threshold alerts")
  ef.addRow(self._sms_pl)
```

P/L threshold - Sets that dollar figure. Key `pl_threshold_amount`, $1 to $100,000, at $100.

The one spin box in the events group, and the only row there that is not a checkbox.

`src/gui/settings_dialog.py` — the P/L threshold row

```
  self._sms_pl_amount = QDoubleSpinBox()
  self._sms_pl_amount.setMinimumHeight(28)
  self._sms_pl_amount.setRange(1, 100000)
  self._sms_pl_amount.setValue(100)
  self._sms_pl_amount.setPrefix("$")
  ef.addRow("P/L threshold:", self._sms_pl_amount)
```

Low balance warnings - Sends a message when a balance runs low. Key `notify_balance_warning`, clear at build.

No figure on the page sets what counts as low, so the row carries the switch alone.

`src/gui/settings_dialog.py` — the low balance row

```
  self._sms_balance = QCheckBox("Low balance warnings")
  ef.addRow(self._sms_balance)
```

Exchange connection status - Sends a message when a venue connects or drops. Key `notify_connection_status`, clear at build.

The last event row, and the third of the three that open clear.

`src/gui/settings_dialog.py` — the connection status row

```
  self._sms_connection = QCheckBox("Exchange connection status")
  ef.addRow(self._sms_connection)
```

What the eight event rows reach today

The eight sentences above are the specification. Five of the eight rows now govern a real send.

`src/core/notifications.py, in attach_to_bus` subscribes the fill topic, and `src/core/notifications.py, in get_notification_manager` makes that subscription as it builds the manager. A filled trade therefore raises `BUY_FILLED` or `SELL_FILLED` with no window in the path. `src/core/notifications.py, in _send_sms` hands the engine the event, and `src/core/sms_engine.py, in send` reads `EVENT_SWITCHES` to find the switch that gates it. `Buy fills` gates `buy_fill` and `Sell fills` gates `sell_fill`.

`P/L threshold alerts` is reached from `src/core/notifications.py, in check_pnl_milestone`, which now reads `pl_threshold_amount` and raises one alert for each new whole multiple of it. `Exchange connection status` is reached from `src/exchange/ccxt_connector.py, in _alert_venue`, for a lost connection and for a venue that answers again.

The figure `check_pnl_milestone` is handed is the fleet's realised total, the sum of each bot's `stats.realised_pnl`. That field had no writer and read zero, so the alert never fired. The exchange walk now writes it with the bot's realised figure from the exchange's own fill history. The walk runs at most once in 300 seconds, so the alert follows a fill by up to 300 seconds.

Read today: the realised figure moves when the fill is announced, while the bot holds a complete fill history from its last walk. The window checks the milestone every two seconds, so the alert follows the fill by up to two

seconds. The walk then writes the exchange's own figure over it.

```
# src/trading/scrumming/reconciliation.py, in ReconciliationEngineMixin._join_fill_history
        health = compute_position_health(held, self.config.symbol.split("/")[0])
        self.stats.realized_pnl_exchange = float(health.realized_pnl_usd)
        self.stats.realised_pnl = float(health.realized_pnl_usd)
```

```
# src/trading/scrumming/reconciliation.py, in refresh_exchange_position_health
                self.stats.realized_pnl_exchange = self.stats.realised_pnl = float(
                    _ph.realized_pnl_usd
                )

# src/gui/main_window.py, in _refresh_dashboard
                        self._notif_manager.check_pnl_milestone(
                            agg.get("total_realised_pnl", 0)
                        )
```

Three rows still wait on a trigger of their own.

Row	What is missing
Bot state changes	Nothing raises `BOT_STARTED`, `BOT_STOPPED` or `BOT_ERROR`
API errors and failures	No alert event names a failed venue call. `src/exchange/ccxt_connector.py, in note_venue_refusal` raises one for a lost connection alone
Low balance warnings	No alert event names a low balance

The number the send reaches is one field. `src/core/notifications.py, in configure_sms` writes the number the Notifications and Alerts tab types into `SMSConfig.phone_number`, which is the field `Phone Number` on this page stores.

The Product Manual PDF rebuild is owed for this page.

Max messages per hour - Caps how many messages one hour may carry. Key `max_messages_per_hour`, 1 to 100, at 20. Below the area the figure shows.

The first of the two rows in the rate limiting group.

`src/gui/settings_dialog.py` — the hourly cap row

```
self._sms_max_hour = QSpinBox()
self._sms_max_hour.setMinimumHeight(28)
self._sms_max_hour.setRange(1, 100)
self._sms_max_hour.setValue(20)
rf.addRow("Max messages per hour:", self._sms_max_hour)
```

Min time between messages - Sets the wait between two messages. Key `cooldown_seconds`, 5 to 300 seconds, at 30. Below the area the figure shows.

The second rate limiting row, and the last control on the page. It is the one spin box here that carries a unit in its own suffix.

`src/gui/settings_dialog.py` — the cooldown row

```
self._sms_cooldown = QSpinBox()
self._sms_cooldown.setMinimumHeight(28)
self._sms_cooldown.setRange(5, 300)
self._sms_cooldown.setValue(30)
self._sms_cooldown.setSuffix(" sec")
rf.addRow("Min time between messages:", self._sms_cooldown)
```

The Carrier list fills from one mapping of carrier name to gateway address.

`src/core/sms_engine.py` — `CARRIER_GATEWAYS`, the first four

```
CARRIER_GATEWAYS = {
    "AT&T": "{number}@txt.att.net",
    "T-Mobile": "{number}@tmomail.net",
    "Verizon": "{number}@vtext.com",
    "Sprint": "{number}@messaging.sprintpcs.com",
```

None of it reaches the SMS engine. Save reads no widget on this page and the schema declares no SMS field. Neither provider label the combo offers matches the value the engine tests for, and the page carries no field for the three Twilio credentials the send path reads.

`src/core/sms_engine.py` — `SMSConfig.provider`, the value the send path tests

```
    provider: str = "email_gateway"
```

Issue #423 carries the page.

Issue #423 closed on 2026-09-25. The schema declares `message_channels`, which holds the SMS page's twenty-two values, and `src/core/sms_engine.py, in sms_config_from_settings` reads that group back.

`src/core/sms_engine.py` — `SMSConfig`, the three credentials no row fills

```
twilio_sid: str = ""
twilio_auth_token: str = ""
twilio_from_number: str = ""
```

The page persists now. Save writes one stored group and the dialog reads that group back on the way in, so every row reopens on the figure it was left at.

The group is named for the channels this page is to carry rather than for the one it carries today, so a channel added beside the text messages joins it instead of renaming it. One constant in the engine holds that name and every reader takes it from there, so the schema field is the only other place the name is written.

`src/core/settings.py` — the group the schema declares

```
    # The SMS page's twenty-two values. The name carries the channels the page
    # is to hold; sms_engine.sms_config_from_settings reads this group back.
    message_channels: dict = field(default_factory=lambda: asdict(SMSConfig()))
```

One list pairs each stored field with the control carrying it, and the save and the load both walk that list, so no row can be written without also being read.

`src/gui/main_tabs/settings_dialog_surface.py` — the first rows of the pairing

```
SMS_CONFIG_FIELDS = (
    ("enabled", "sms_enabled"),
    ("provider", "sms_provider"),
    ("phone_number", "sms_phone"),
```

The stored group reaches the engine at two moments. The startup path hands it over before the first fill, and the window hands it over again when the dialog saves, so a changed figure applies without a restart.

`main.py` — the startup push, before the first fill

```
    get_sms_engine().update_config(
        sms_config_from_settings(settings.get("message_channels", {}))
    )
```

Each provider entry carries a short code beside its printed label, and the picker stores the code rather than the label. The send path tests that code, so the Twilio entry reaches the Twilio route and the gateway entry reaches the mail route.

`src/gui/main_tabs/settings_dialog_surface.py` — the two provider entries

```
SMS_PROVIDERS = (
    ("Email-to-SMS Gateway", EMAIL_GATEWAY),
    ("Twilio API", TWILIO),
)
```

Carrier has a stored field of its own, and picking one fills the Gateway Email row below it from the number already typed. The row is filled only by that choice, never by a save or by opening the page, so an operator who never touches the picker is never given an address for a carrier they are not on.

`src/gui/settings_dialog.py` — the fill a carrier choice runs

```
built = gateway_address(
    self._sms_carrier.currentText(), self._sms_phone.text()
)
if built:
    self._sms_gateway.setText(built)
```

A gateway takes the national digits alone, so a leading country digit is dropped and a number of any other length builds nothing. The manual choice at the end of the list builds nothing by design, and a row the carrier cannot fill keeps whatever is in it.

`src/core/sms_engine.py` — the address a carrier builds

```
  def gateway_address(carrier: str, number: str) -> str:
      template = CARRIER_GATEWAYS.get(carrier, "")
      if not template:
          return ""
```

Five rows are new. Three fill the Twilio credentials the send path reads, with the auth token masked like the mailbox password above it, and two set the mail server and the port the email route opens. The page holds twenty-two controls now, one for every field of the message configuration.

`src/gui/settings_dialog.py` — the mail server and port rows

```
  self._sms_smtp_server = QLineEdit()
  self._sms_smtp_server.setMinimumHeight(28)
  self._sms_smtp_server.setPlaceholderText(built.smtp_server)
  pf.addRow("Mail Server:", self._sms_smtp_server)
```

The figure that set what counts as a low balance is gone. No row set it, nothing read it, and no sentence above asks for one, so the low balance row carries its switch alone as it always did. A settings file still holding the old key loads with every other value intact, and the engine names the key it dropped.

`src/core/sms_engine.py` — the line the engine prints for a name it does not hold

```
          logger.warning(
              "message_channels %r is not a message setting; ignored", name
          )
```

A value the engine cannot use is refused instead of reaching it. A provider outside the two entries, a switch that is neither on nor off, a cap or a cooldown outside its range, and a carrier with no gateway are each dropped with a line on the log and the build figure kept.

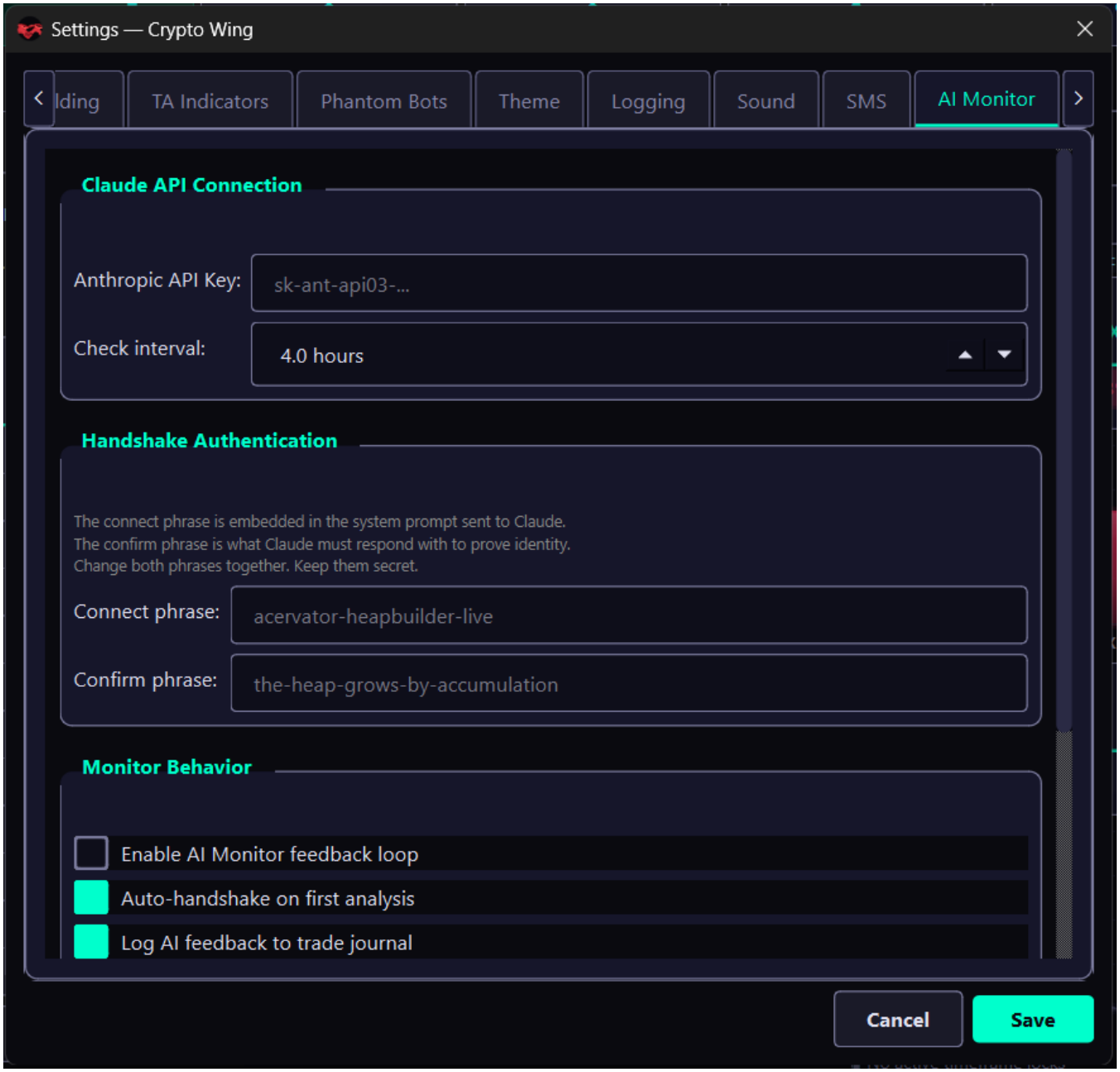
`src/core/sms_engine.py` — the range each numeric row may hold

```
  FIELD_BOUNDS: dict[str, tuple[float, float]] = {
      "smtp_port": (1, 65535),
      "pl_threshold_amount": (1.0, 100000.0),
      "max_messages_per_hour": (1, 100),
      "cooldown_seconds": (5, 300),
  }
```

The eight event rows wait on a trigger. Each one names an event type the engine checks before it sends, and nothing in the application calls the send method with one of those types, so no fill, no bot state and no venue error reaches a message yet.

In development.

Settings > AI Monitor



Settings, the AI Monitor page.

A scrolling page holding four groups.

Seven settings, and every key below sits inside one stored `ai_monitor` group.

Anthropic API Key - Holds the key the monitor authenticates with. Key `api_key`, masked, empty.

The first row of the Claude API Connection group. It is masked like the SMS password, and its placeholder shows the shape of a key rather than a key.

`src/gui/settings_dialog.py` — the API key row

```
self._ai_api_key = QLineEdit()
self._ai_api_key.setMinimumHeight(28)
self._ai_api_key.setEchoMode(QLineEdit.Password)
self._ai_api_key.setPlaceholderText("sk-ant-api03-...")
af.addRow("Anthropic API Key:", self._ai_api_key)
```

Check interval - Sets how long the monitor waits between two reviews. Key `interval_hours`, 0.5 to 24.0 hours, at 4.0.

The one figure on the page. It carries a decimal place, so half an hour is the shortest wait it accepts.

`src/gui/settings_dialog.py` — the check interval row

```
self._ai_interval = QDoubleSpinBox()
self._ai_interval.setMinimumHeight(28)
self._ai_interval.setRange(0.5, 24.0)
self._ai_interval.setValue(4.0)
self._ai_interval.setSuffix(" hours")
self._ai_interval.setDecimals(1)
af.addRow("Check interval:", self._ai_interval)
```

Connect phrase - Sets the phrase the platform puts into the prompt it sends. Key `connect_phrase`, empty.

The first of the two handshake rows. A grey note above the pair states the rule the two follow, and the placeholder shows the shape of a phrase.

`src/gui/settings_dialog.py` — the connect phrase row

```
self._ai_connect_phrase = QLineEdit()
self._ai_connect_phrase.setMinimumHeight(28)
self._ai_connect_phrase.setPlaceholderText("acervator-heapbuilder-live")
hf.addRow("Connect phrase:", self._ai_connect_phrase)
```

Confirm phrase - Sets the phrase the answer must carry back to prove it came from the same conversation. Key `confirm_phrase`, empty. Change both phrases together and keep both secret.

The second handshake row, built like the first. Neither field is masked.

`src/gui/settings_dialog.py` — the confirm phrase row

```
self._ai_confirm_phrase = QLineEdit()
self._ai_confirm_phrase.setMinimumHeight(28)
self._ai_confirm_phrase.setPlaceholderText("the-heap-grows-by-accumulation")
hf.addRow("Confirm phrase:", self._ai_confirm_phrase)
```

Enable AI Monitor feedback loop - Turns the monitor on. Key `enabled`, clear at build.

The one control on the page that opens clear. Save emits a signal and the main window reads the whole group back, so this box and the key beside it decide whether the header reads READY or OFF.

`src/gui/main_window.py` — `_on_settings_changed`, the monitor branch

```
ai_cfg = self._settings.get("ai_monitor", {})
if self._bot_manager:
    self._bot_manager.configure_live_monitor(ai_cfg)
    if ai_cfg.get("enabled") and ai_cfg.get("api_key"):
        self._ai_monitor_label.setText("AI: READY")
```

Auto-handshake on first analysis - Runs the phrase exchange on the first review rather than waiting for one to be asked for. Key `auto_handshake`, checked at build.

The first of the two Monitor Behavior rows that open checked.

`src/gui/settings_dialog.py` — the auto-handshake row

```
self._ai_auto_handshake = QCheckBox("Auto-handshake on first analysis")
self._ai_auto_handshake.setChecked(True)
bf.addRow(self._ai_auto_handshake)
```

Log AI feedback to trade journal - Writes each answer into the trade journal. Key `log_feedback`, checked at build.

The last control before the status group, and the second of the two that open checked.

`src/gui/settings_dialog.py` — the journal logging row

```
self._ai_log_feedback = QCheckBox("Log AI feedback to trade journal")
self._ai_log_feedback.setChecked(True)
bf.addRow(self._ai_log_feedback)
```

Connection Status closes the page, below the area the figure shows: a status line, a journal hash, a completed-check count and a Test Handshake button. The three readings report and the button acts. None of the four stores anything.

Save writes all seven controls into one dictionary and the load path reads all seven back, which makes this the one page besides User whose whole state round-trips.

`src/gui/settings_dialog.py` — `_load_current`, the seven it restores

```
ai = self._sm.get("ai_monitor", {})
self._ai_api_key.setText(ai.get("api_key", ""))
self._ai_interval.setValue(ai.get("interval_hours", 4.0))
self._ai_connect_phrase.setText(ai.get("connect_phrase", ""))
self._ai_confirm_phrase.setText(ai.get("confirm_phrase", ""))
self._ai_enabled.setChecked(ai.get("enabled", False))
self._ai_auto_handshake.setChecked(ai.get("auto_handshake", True))
self._ai_log_feedback.setChecked(ai.get("log_feedback", True))
```

Test Handshake runs no handshake. It checks that the key and both phrases hold text, then writes either an error line or the message saying the handshake runs on the next bot cycle.

`src/gui/settings_dialog.py` — `_test_ai_handshake`

```
self._ai_status.setText("Settings saved — handshake runs on next bot cycle")
self._ai_status.setStyleSheet("color: #00ddff; font-weight: bold;")
```

The button reports the fields it read, never a venue answer. Issue #327 carries that.

The issue #327 clause above is overtaken. It is not edited and not removed. It read:

> *The button reports the fields it read, never a venue answer. Issue #327 carries that.*

Issue #327 closed on 2026-10-22. The button still reads the fields and no venue, and its stated purpose now says so, which is what the issue asked for.

```
src/gui/main_tabs/settings_dialog_surface.py, in test_ai_handshake
    "Report whether the AI Monitor has what it needs to connect."
    reads   ai_api_key, ai_connect_phrase, ai_confirm_phrase

src/gui/main_tabs/settings_dialog_surface.py   AI_TEST_BUTTON_TEXT = "Test Handshake"
src/gui/main_tabs/settings_dialog_surface.py   AI_SAVED_TEXT names the next bot cycle
```

Six of the seven do reach the engine. `configure_live_monitor` in `src/trading/bot_container.py` reads five of them and builds or clears the monitor, and the main window reads the journal switch when an answer arrives. The auto-handshake box is the seventh: written, read back, and read by nothing else.

`src/trading/bot_container.py` — `configure_live_monitor`, the five keys it reads

```
self._live_monitor = LiveMonitor(
    api_key=settings["api_key"],
    journal=journal,
    interval_hours=settings.get("interval_hours", 4.0),
    connect_phrase=settings.get("connect_phrase", ""),
    confirm_phrase=settings.get("confirm_phrase", ""),
)
```

2026-09-11 - Save keeps what the store holds

`_save` and `_load_current` read one declaration. `_stored_rows` names every setting the store holds at its top level. `_stored_groups` names every group the store holds as one key, with the rows inside it. One row carries the store key, the read off its control, the write back into that control, and what stands in for a store without the key. Both methods walk the same rows. No setting can be written without also being loaded.

`src/gui/settings_dialog.py` — one row of `_stored_rows`

```
("bot_visibility",
 self._visibility.currentText,
 lambda value: self._show_text(self._visibility, value),
 "orderbook"),
```

`_show_stored` puts one stored value into one control. A value the control refuses leaves that control on its build figure and names the key in the log. One unreadable entry in the settings file cannot stop the dialog opening.

`SettingsDialogReact` replaces `_setup_ui` alone, so the React build and the Qt build load and save the same way.

A stored number now loads on the React build. `_StoredWriter` admits a stored figure against the control's own span, which it reads through `minimum` and `maximum`. The React build's `PageNumber` answers both off the control's spec, and `PageDecimal` answers `decimals` for a number box that keeps them. Without those answers every stored number on this build was refused: driven with a stored SFX Volume of 0.25, the slider opened at 70 and 20 keys were marked refused, among them the volume, the twelve indicator weights and four message figures. The same run now opens the slider at 25 and marks none.

`src/gui/react_settings_dialog.py` — the span a stored figure is admitted against

```
def minimum(self) -> Any:
    return self._span()[0]

def maximum(self) -> Any:
    return self._span()[1]
```

What each page restores now

This table replaces the restores column in What each page persists. Every other column holds.

Page	Controls	`_save` writes	`_load_current` restores
User	1	the row	the row
Exchanges	5 plus 3 buttons	on Add and Remove	the configured list
Trading	6	all six	all six
Profit Folding	11	the whole group	the whole group
TA Indicators	12	nothing	nothing
Phantom Bots	13	nothing	nothing
Theme	6	all six	all six
Logging	6	the whole group	the whole group
Sound	10 plus 6 buttons	nothing	nothing
SMS	17	nothing	nothing
AI Monitor	7	all seven	all seven

The fourteen rows a Save overwrote

Save wrote these fourteen rows and no load read them back. A Save with no edit wrote the build figure over the stored value. Each of the fourteen now reopens on what was stored.

Page	Rows
Trading	Bot Visibility, Enable aggressive trading mode
Profit Folding	Distribution Mode, Profit Folding Target, Fold to X# of buy positions, Upward Distribution Target, Distribute to X# of sell positions
Theme	Font Family, Base Font Size, Heading Font Size, Log Font Size

Page	Rows
Logging	Log TA signal samples with all values and timestamps, Highlight entries near Scrumming Bot trades, P/L Log Periodicity

What each row means has not changed. No reader outside the dialog reads any of the fourteen. Whether each one is wired to a reader or taken off the page is a separate question, and the Settings audit carries it row by row.

The sentences this section replaces

Each sentence below stands in an earlier dated section and no longer describes the code. The earlier text stays where it is.

Earlier sentence	What the code does now
"Save writes the name and the load path never reads it."	The load path reads `bot_visibility`. The box opens on the stored name.
"The box opens on orderbook whatever was stored, and a Save with no edit writes that first entry back over the stored value."	The box opens on the stored name. A Save with no edit writes that same name back.
"Save writes it, the load path skips it, and the box opens clear."	The load path reads `aggressive_trading`. The box opens ticked when the store holds true.
"Save writes all six keys. The load path restores the first four, which is why the last two rows on the page open at their built-in defaults."	The load path restores all six keys on the Trading page.
"This is the one control on the page the load path restores, so it is the only one of the six that reopens on what was stored."	The load path restores all six keys in the stored `profit_folding` group.
"Nothing reads any of the four, and the load path does not restore them either, so the whole Font Settings group reopens at its built-in figures."	Nothing outside the dialog reads the four font keys. The load path restores all four, so the Font Settings group reopens on what was stored.
"The load path reads none of them back, so every open shows the build state rather than the stored one."	The load path reads all three `data_logging` keys back. Every open shows the stored state.
"In development. What the load path should restore for this page has not been settled against the rest of the dialog, so nothing is proposed here."	The load path restores the whole group: both flags and the list of active periodicities.

2026-09-11 - The Username row and the vault phrase

Driven on the real dialog and the real vault. The home was redirected into a scratch directory before `src.core.settings` was imported, so `_DEFAULT_DIR` bound under that directory. No stored credential of the running install was read.

Every reader of the stored name

The row reaches four readers. Each one ran with a generated name, and each printed the phrase it built.

Site	What it does with the name	Driven result
`main.py`, the startup read	Decides whether to write the default	reads the stored name

Site	What it does with the name	Driven result
`src/gui/settings_dialog.py`, `_add_exchange`	Builds the phrase, encrypts the key and the secret	encrypts under `qat_<name>_vault`
`src/gui/main_window.py`, `_connect_exchange_for_bot`	Rebuilds the phrase, decrypts	opens the stored secret
`src/gui/widgets/api_tester_tab.py`, `_do_connect`	Rebuilds the phrase, decrypts	opens the stored secret

Nothing else reads the stored name. No bot reads it. No log line carries it. No title bar shows it.

`_stored_rows` carries the row that loads the stored name into the control and saves the control back. `SettingsDialogReact` replaces `_setup_ui` alone, and the page declares the row as one line control, so both builds carry the same row.

`src/gui/init_wizard.py` collects a name into `get_results`. `get_results` has no caller, and `InitWizard` is never built. The name on the User page comes from the startup write.

What a rename does to a stored secret

A secret went in under one name. The row then changed to a second name, and Save ran.

Question	Observed
Does a box warn?	No box appears
Is the rename refused?	No. The store holds the new name
Is the stored token rewrapped?	No. The token stays the one that was written
Does the secret open?	No. Both decrypt sites refuse

Both decrypt sites raise `Decryption failed — wrong passphrase or corrupted data`. The bot connect reports `Connection failed`, and the API tester label reads `Failed`. Neither message names the rename.

The earlier name typed back opens the same secret again. Nothing records that earlier name, and nothing shows it after the change.

No code handles the rename. The unlock the name will belong to is the Quintessence Wallet, and the Quintessence Wallet is not built.

The empty middle

`get('username', 'user')` on a store written without the entry returns an empty string, so the `'user'` fallback in the three phrase literals never fires. Such a store builds the phrase `qat__vault`. The startup write closes that gap on the first run.

`MASTER_FORMAT` formats to the same text the three literals carry.

2026-09-11 - The exchange picker and the venue list

The Exchange row on the Exchanges page offers fifteen crypto venues. This section records which list fills it, which build paints it, and every place the chosen venue id arrives.

Which build paints the picker

OVERTAKEN, and this section is kept as written. There is no picker. The Exchange Status array is the only control that chooses a venue, and the Exchange row of the Add form is a label reading `chosen_venue_text`. The two item lists this section traces — `crypto_exchange_items` and `exchange_items` in `src/gui/main_tabs/settings_dialog_surface.py` — are gone with the picker, and so is the `exchange_items` field of `view_model`. Both builds now take the chosen venue from `row_venue_id` over the array's one selected row.

Two builds draw the row. `src/_variant.py` sets `DEFAULT_VARIANT = REACT`, so a checkout with no baked variant file and no `ACERVATOR_VARIANT` draws the React page:

```
resolve_variant()                 react
draws_react('Settings dialog')    True
surface_class('Settings dialog')  SettingsDialogReact
```

The React page is `src/gui/react_settings_dialog.py`, and it reads its picker items from `src/gui/main_tabs/settings_dialog_surface.py` — `exchange_items`. The Qt build, `src/gui/settings_dialog.py` — `_create_exchange_tab`, reads `SUPPORTED_EXCHANGES` from the connector directly.

Diagram: flowchart, left to right

- ccxt_connector.SUPPORTED_EXCHANGES — + exchange_label -> **settings_dialog.py — _create_exchange_tab**
- ccxt_connector.SUPPORTED_EXCHANGES — + exchange_label -> **settings_dialog_surface.py — crypto_exchange_items**
- settings_dialog_surface.py — crypto_exchange_items -> **react_settings_dialog.py — the loaded page**
- settings_dialog_surface.py — crypto_exchange_items -> **view_model — settings_dialog.state**

The two lists, entry by entry

The page once carried its own frozen copy of the fifteen labels. Three readings were taken and each offered the same fifteen entries, in the same order, label for label:

Reading	Where it comes from	Count
The connector registry	`SUPPORTED_EXCHANGES` through `exchange_label`	15
The surface list	`settings_dialog_surface.exchange_items`	15
The loaded React page	the `<option>` elements the page drew	15

In order: Binance (US address refused), Bitfinex (untested), Bitget (untested, passphrase required), Bitstamp (untested), Bybit (US address refused), Coinbase, Cryptocom (untested), Gateio (untested), Gemini (untested), Huobi (US account restricted), Kraken (untested), Kucoin (untested, passphrase required), Mexc (untested), Okx (untested, passphrase required), Poloniex (US account restricted). Every label except Coinbase carries a note, because `exchange_label` marks a venue whose address is refused, or whose terms refuse the account, or that is untested, or that needs a passphrase.

Binance stands first because the list is in id order and `binance` sorts first.

Every reader of the chosen venue id

The picker writes its id into an `ExchangeConfig`, and `add_exchange` stores it keyed by that id. `gemini` was chosen on both builds and each reader printed what it saw.

Reader	What it saw
`SettingsManager.list_exchanges`	`exchange_id: 'gemini'`
`SettingsManager.get_exchange`	the same entry, matched on the id
`_load_current`, the Qt Configured list	`Gemini (gemini)`
The React Configured list	`Gemini (gemini)`
`_remove_exchange`, which reads the id back out of that row	the store emptied
`trading_tab_surface.exchange_display_name`	`Gemini`, the tab caption
`trading_tab_surface.is_equity_exchange`	`False`, so the crypto layer
`CCXTConnector.__init__`	the registry accepts the id

Five further sites read the stored id rather than the picker: the startup loop in `main.py` that builds one exchange tab per entry, the startup credential report and the bot-connect credential lookup in `src/gui/main_window.py`, the wizard's venue choices, and the API panel's stored-credential lookup. Each matches on the same `exchange_id` field.

Nothing else reads it. No bot reads the picker, and no gate reads it. `src/core/usb_auth.py` looks like a reader and is not one: it reads `CredentialVault.list_exchanges`, a separate store the picker never writes to.

The drift, measured

The connector is the authority over the id. A venue it does not carry is refused before any object exists:

```
CCXTConnector('notavenue') refused  Unsupported exchange 'notavenue'.
Supported: ['binance', 'coinbase', 'kraken', ... 'cryptocom']
```

With the frozen copy in place, one venue added to the connector alone reached the Qt picker and not the React page:

Reading	Before the connector gained a venue	After
The Qt picker	15	16
The surface list	15	15
The bridge list	15	15

A venue offered on the page and missing from the registry would be stored and then refused at the first connect. A venue in the registry and missing from the page would never appear.

What changed

`crypto_exchange_items` in `src/gui/main_tabs/settings_dialog_surface.py` now builds the pairs from `SUPPORTED_EXCHANGES` and `exchange_label`, imported when first asked so the module still loads no exchange library. The frozen copy is gone, and the registry is the only list of venues.

The screen offers the same fifteen venues in the same order as before. The Qt picker, the surface list, the bridge list and the stock-wing list were read from a tree at the earlier commit and from the changed tree, and the two readings agree in every entry. The same connector mutation now reaches all three crypto readings together.

Which venues are supported is still decided in one place only: `src/exchange/ccxt_connector.py`.

2026-09-11 - The API Key row, from the typed string to the stored token

The API Key row on the Exchanges page was typed into on the running React page, the Add button was pressed, and every place the typed string could come to rest was then searched. This section records what was found. The string used was a throwaway that is not a key, and no venue was contacted.

Where the typed key goes, and where it stops

The row is an entry field and nothing else. The operator fills it, the Add handler reads it once, and the row is cleared.

Diagram: flowchart, left to right

- the API Key row — new_api_key -> **api_validator.validate_credentials — the venue check**
- api_validator.validate_credentials — the venue check -> **core.encryption.encrypt — PBKDF2 then AES-256-GCM**
- core.encryption.encrypt — PBKDF2 then AES-256-GCM -> **settings.toml — exchanges api_key_enc**
- settings.toml — exchanges api_key_enc -> **core.encryption.decrypt**
- core.encryption.decrypt -> **CCXTConnector.sync_connect — apiKey**

`src/gui/settings_dialog.py` — `_add_exchange`, the two lines the row causes

```
master = f"qat_{self._sm.get('username', 'user')}_vault"
config.api_key_enc = encrypt(key, master)
```

Moment	What happens to the row
At dialog load	Nothing. No stored value is ever put back into it
On the Save button	Nothing. The row is not in the save path
On Test and Add Exchange	The venue check runs, then the key is encrypted and the file is written
Straight after that press	The row is cleared
On the next cycle	Nothing. No running bot re-reads the row
On restart	The stored token is decrypted fresh on each connect

A thirty-two character key was stored as a token of one hundred and four characters, and the longer Coinbase string as one of one hundred and fifty-six. The typed string appears nowhere in the stored entry.

Every reader of the stored key

Reader	What it does
`src/gui/main_window.py` — `_connect_exchange_for_bot`	decrypts it for a bot connect
`src/gui/widgets/api_tester_tab.py` — `_do_connect`	decrypts it for the API panel
the same two methods, one step earlier	refuse the connect while the field is empty
`src/gui/main_window.py`, the startup credential report	reads presence only, never the value

Both decrypt paths were driven and each recovered the typed string byte for byte, then handed those same bytes to the connector. Nothing in the frontend reads the stored field, measured by searching the whole index in three naming conventions. A separate credential store in `src/core/encryption.py` carries a field of the same name, and this row never writes there.

The clear-text hunt, twenty places

Twenty places were searched for the typed string after the press. Three hold it, and each of the three is a consumer that cannot do its work without it.

Place	Result
every file under the settings directory	not found
every log file the run produced	not found
the payload the dialog pushes to the page	not found
the page store values, labels, tooltips, calls, prints and boxes	not found
the rendered page markup	not found
the feedback line and the added-exchange box	not found
the message the bot-connect reader returned	not found
the API panel result view and status label	not found
every log record the run emitted, and the API Log ring buffer	not found
the API Key field the page draws	found, because the field shows what is typed
the venue check the Add ran first	found, because it authenticates with the key
the connector call each reader makes	found, because it authenticates with the key

Two controls prove the search can report. The typed string was written into the stored settings file, the search named that file, the file was put back and its digest matched. The typed string was then written through the application's own logger, the search named both the log file and the captured record, and that file was put back to the same digest.

```
hunt with the plant      [('...\.acervator\settings.toml', 1218)]
bytes unchanged by hash  True
log file hunt with it    [('...\.acervator_logs\console\system.log', 10258)]
bytes unchanged by hash  True
```

A wrong Username refuses rather than returning a wrong key, and the refusal names no part of the key:

```
decrypt refused                 ValueError: Decryption failed - wrong passphrase or corrupted data
the refusal names the typed key  False
```

What the row echoes

The row is drawn as a text field, so the key is readable on screen while it is typed. Three credential rows on the same dialog are drawn as password fields:

Row	Control kind	Echo
API Key	line	none
API Secret	text area	none, and a text area carries no echo mode
Passphrase	line	password
Anthropic API Key	line	password
SMTP Password	line	password

The Qt build sets no echo mode on this row either, so both builds agree. Masking the key alone would hide nothing, because the secret beside it authenticates with it and stays in view. The complete change reaches the API Secret row, which is held as its own unit, so it is recorded here and not made.

The Coinbase CDP key string

The page says the row takes a plain key or a Coinbase CDP key string. Both were typed and both round-trip:

```
the CDP string decrypts byte for byte                True
the stored entry carries the CDP string in the clear  False
```

The row treats the two alike. It accepts any string, and the venue check in front of the store is what refuses a key the exchange will not take. The key shape is read later, at connect, by `src/exchange/ccxt_connector.py` — `sync_connect`, which reports the format it detected and writes only the last four characters of the key into the API log.

2026-09-11 - The API Secret row converts a pasted PEM

The row takes a plain secret or a PEM elliptic-curve private key. A paste that carried every newline as the two characters backslash and n is converted before the secret is encrypted, so the stored token holds the real key and every venue is handed the same bytes.

`src/core/encryption.py` holds the two functions the conversion rests on. `looks_like_pem` reports whether the text carries both `PEM_ARMOUR` lines. `unescape_pem_newlines` replaces every escaped newline, and breaks the armour lines off the body when a PEM arrives on one line.

Where the conversion runs

Site	What it converts
`src/gui/settings_dialog.py` — `_typed_secret`	the row, read by the venue check and by the store write
`src/gui/main_tabs/settings_dialog_surface.py` — `_typed_credentials`	the same row in the Qt-free model
`src/exchange/ccxt_connector.py` — `sync_connect`	a Coinbase secret stored before the dialog converted

`SettingsDialogReact` inherits `_add_exchange`, so the React build runs `_typed_secret` through a `PageTextArea` holder.

`_typed_secret` converts only a value `looks_like_pem` accepts. A plain secret that carries a literal backslash and n reaches the store unchanged.

What the row stores

A throwaway PEM whose body reads THIS-IS-NOT-A-KEY was pasted into the row and the Add button was pressed. The stored token was then decrypted.

```
the escaped paste          3 escaped, 0 real newlines
decrypted out of the store 0 escaped, 3 real newlines
```

A PEM pasted with real newlines already is stored byte for byte, and so is a plain secret carrying a literal backslash and n.

What each venue is handed

At the point `sync_connect` builds the CCXT config, from the stored value the escaped paste produced:

Venue	Escaped	Real newlines	Usable PEM
coinbase	0	3	yes
kraken	0	3	yes

A second connect on the same stored value produces the same bytes, so the Coinbase branch in `sync_connect` never converts a converted secret twice.

What the row does not mask

The row is a text area and a text area carries no echo mode, so the secret is readable on screen while it is typed. The API Key row beside it is a text field with no echo mode either, and the two authenticate as a pair. Masking one of them hides nothing, and masking this one would mean replacing the text area with a single line that cannot take a multi-line PEM. Both rows are recorded and neither is changed.

2026-09-11 - The Passphrase row and the venue that needs one

The box labelled **This exchange uses an API passphrase** decides whether the field under it is drawn. Picking Kucoin, Okx or Bitget ticks the box, and the field appears with it on the React build and on the Qt build alike.

Which trigger draws the field

One method decides the field's visibility, and three triggers call it.

`src/gui/settings_dialog.py` — `_sync_passphrase_row`

```
def _sync_passphrase_row(self) -> None:
    self._new_passphrase.setVisible(self._pp_check.isChecked())
```

Diagram: flowchart, left to right

- Click on the box, Qt -> **_sync_passphrase_row**
- Click on the box, React page -> **_sync_passphrase_row**
- Venue pick, _on_exchange_changed -> **_sync_passphrase_row**
- _sync_passphrase_row -> **new_passphrase drawn or hidden**

Trigger	Where it is wired
A click on the box	`_pp_check.toggled`, on the Qt build
A click on the box	`SettingsDialogReact.apply_edit`, on a `pp_check` edit from the page
A venue pick	`_on_exchange_changed`, which the React dialog inherits and runs

`_on_exchange_changed` sets the box with `setChecked` and then calls the method. Qt emits `toggled` from `setChecked` and the React page-backed holder emits nothing, so the direct call is what draws the field on each build.

The Qt-free model carries the same rule for the page the dialog opens on. `SettingsDialogModel.on_exchange_changed` calls its own `show_passphrase`, so the Exchanges page draws the field for a venue that needs one from the first paint.

The row read on each venue

Driven on the real React page and on the Qt dialog, with the Exchanges tab open.

Venue	Needs one	Box ticked	Field drawn
kucoin	yes	yes	yes
okx	yes	yes	yes
bitget	yes	yes	yes
binance	no	no	no
coinbase	no	no	no
kraken	no	no	no

Both builds report that table. A direct click on the box draws the field for a venue that needs no passphrase, and a second click hides it again.

What the typed value reaches

The field takes focus once it is drawn, and the typed string runs the whole stored path.

Step	What it carries
The field	the typed passphrase
`validate_credentials`	the same string, as its fourth argument

Step	What it carries
`config.passphrase_enc`	an AES-256-GCM token, 84 characters
`decrypt`	the typed passphrase, byte for byte

Test Connection answers `Kucoin requires an API passphrase.` while the field is empty, and the venue is never reached. With the field filled, the connector is handed the key, the secret and the passphrase together.

Where the venue set is declared

Three declarations hold the same three ids.

Declaration	Read by
`src/exchange/ccxt_connector.py` — `PASSPHRASE_EXCHANGES`	the Qt dialog, `src/exchange/api_validator.py`, `sync_connect`
`src/gui/main_tabs/settings_dialog_surface.py` — `PASSPHRASE_EXCHANGE_IDS`	the React dialog and the Qt-free model
`src/gui/main_tabs/init_wizard_surface.py` — `PASSPHRASE_EXCHANGE_IDS`	the first-run wizard

`_sync_passphrase_row` reads the box and no set, so the field follows the box whichever declaration ticked it.

2026-09-11 - The saved exchange list

The list under Configured Crypto Exchanges is the stored `exchanges` field. Each entry is one saved venue: its id, its display name, three encrypted tokens, an enabled flag and two hardware fields. The list was driven on the running React page, two venues at a time.

Where the list is written

Two buttons write it, and Save is not one of them.

Diagram: graph, left to right

- Test and Add Exchange — add_btn -> **settings_dialog.py — _add_exchange / _remove_exchange**
- Remove Selected — remove_btn -> **settings_dialog.py — _add_exchange / _remove_exchange**
- settings_dialog.py — _add_exchange / _remove_exchange -> **settings.py — add_exchange / remove_exchange**
- settings.py — add_exchange / remove_exchange -> **settings.toml — exchanges**

On React a click on a list line reports its index, and `run_action` turns that into `setCurrentRow`, so Remove Selected has a row to drop. `_save` never names `exchanges`, so pressing Save changes nothing here.

Every reader of the saved list

Ten readers, all through `SettingsManager.list_exchanges` or `get_exchange`.

Site	What it does with the list
`main.py`	One `add_exchange_tab` per entry, in stored order

Site	What it does with the list
`src/gui/main_window.py` — `_report_stored_credentials_on_startup`	An empty list is a warning and an early return
`src/gui/main_window.py` — `_connect_exchange_for_bot`	Walks the list for the bot's id; a missing id and a missing key are two refusals
`src/gui/main_window.py` — `_sync_exchange_tabs`	Adds a tab for every listed id that has none
`src/gui/main_window.py` — `_create_bot`	Hands the whole list to `BotCreationWizard`
`src/gui/widgets/api_tester_tab.py` — `_do_connect`	Walks the list for the picked id and decrypts its tokens
`src/gui/settings_dialog.py` — `_load_current`	Fills the on-screen list, filtered by wing
`src/gui/main_tabs/settings_dialog_surface.py` — `_load_current`	The same fill in the Qt-free model
`src/gui/main_tabs/header_strip_surface.py` — `configured_exchange_count`	The length, and nothing else
`src/gui/main_tabs/trading_tab_surface.py` — `live_view_model`	`layer_exchanges` splits it into the crypto and stock layers

`src/core/usb_auth.py` calls a different `list_exchanges`, the one on `CredentialVault`. That is a separate store this list never writes to. No JavaScript file reads the stored list: the page is handed `listed_exchanges`, words already formatted.

Two adds, in the order added

Two venues were added through the page, each with its own generated credentials.

Reading	Answer
Order added	`binance`, `kucoin`
Order stored	`binance`, `kucoin`
Binance key decrypts to its own	True
Kucoin key decrypts to its own	True
Kucoin passphrase decrypts to its own	True

Each entry carries its own tokens. `list_exchanges` and `get_exchange` both deep-copy, so no reader can change the stored list by holding what it returned.

Removing one

The first line was clicked and Remove Selected pressed. The store kept the other entry, and its three tokens still decrypt.

Reading	Answer
Lines on screen before	`Binance (binance)`, `Kucoin (kucoin)`
Lines on screen after	`Kucoin (kucoin)`
The store after	one entry, `kucoin`
The survivor's key decrypts	True
The survivor's passphrase decrypts	True

Reading	Answer
The feedback line	`Binance removed.`

Adding a venue the list already holds

`add_exchange` stores one entry per venue id, so a second add is never a second entry. It now keeps that entry's place, and a credential token the new entry leaves empty keeps the token the stored entry carried.

The press	What the store holds after
The same venue with new credentials typed	One entry, carrying the newly typed tokens
The same venue with both rows empty	One entry, byte for byte what it held before
The same venue with the key typed and the secret empty	One entry, byte for byte what it held before

With two venues stored, re-adding the first leaves the order as `binance`, `kucoin`. The on-screen list is asked whether a line already names the venue, so one venue is drawn once:

Reading	Answer
Lines on screen before the re-add	`Binance (binance)`, `Kucoin (kucoin)`
Lines on screen after the re-add	`Binance (binance)`, `Kucoin (kucoin)`
Entries in the store	2

`listed_exchange_position` in `src/gui/main_tabs/settings_dialog_surface.py` answers that question, and both the Qt dialog and the Qt-free model call it.

The feedback line still reads `added without credentials` after a press with both rows empty, which names what was typed rather than what is stored. With the key typed and the secret empty it reads `added with credentials (verified)`, because it tests the key alone while `_add_exchange` needs both rows before it contacts a venue or encrypts anything.

What the readers see on an empty list

An empty list is the state on a fresh install. Every reader answers, and none raises.

Reader	Empty	One entry	Two entries
`main.py` venue tabs	none	`kucoin`	`binance`, `kucoin`
`configured_exchange_count`	0	1	2
The crypto layer	placeholder shown	opens on `kucoin`	opens on `binance`
The Settings page list	none	`Kucoin (kucoin)`	`Binance (binance)`, `Kucoin (kucoin)`
A bot asking for `kucoin`	`Exchange kucoin not found in settings.`	connects	connects
The API panel asking for `kucoin`	`No stored credentials for Kucoin.`	decrypts	decrypts

An entry stored with no tokens reads differently from a missing entry: the bot path answers `No API credentials for Kucoin.` instead.

Removing the venue a bot uses

Nothing on this path can see a bot. The dialog is handed the settings manager and the status log, and nothing else.

Reading	Answer
Refused	False
Warned about the bot	False
Silently allowed	True
What the bot's next connect reads	`Exchange binance not found in settings. Add it in Settings first.`

`_sync_exchange_tabs` only adds a tab, so the removed venue's tab stays on screen until the next launch.

OVERTAKEN, and the table above is kept as written. Remove Selected now refuses this press while a bot on the venue still trades, and it names the bot. The section *2026-10-29 - A venue a running bot trades on is kept* holds the reading.

2026-09-11 - Position Distance is off the Trading page

The Trading page carries five rows. The Position Distance row is removed. The row set a percent the dialog stored and read back, and no bot read the value.

Three files declared the row and every declaration is gone.

File	What it declared
`src/core/settings.py`	the `position_distance_pct` field on `AppSettings`
`src/gui/settings_dialog.py`	the `QDoubleSpinBox` and its `_stored_rows` entry
`src/gui/main_tabs/settings_dialog_surface.py`	the `pos_distance` spec, its Trading layout step and its `PERSISTED_ROWS` entry

`_DEPRECATED_KWARGS` in `src/trading/container/config.py` keeps the name. A `bot_state.json` still carrying the key restores without a `TypeError`.

Why the row had no wizard control to open

The row set a percent from 1.0 to 50.0 at 2.0, spacing a bot's positions. Three wizard controls carry a spacing and none carries that quantity.

Control	Key	Range	At	What it spaces
Split Distance	`split_distance`	0.10 % to 20.00 %	1.00 %	one Stack tranche from the next
Spacing	`stack_spacing_mode`	Linear, Quadratic, Exponential	Linear	how that tranche gap grows
Opposing Trade Interval	`scrumming_interval_pct`	0.10 % to 20.00 %	1.00 %	the price travel a reversal needs

The key belongs to the retired Grid Bot. The module docstring of `src/gui/bot_wizard.py` names the mode and its rows: "Grid Bot: Investment Amount, Position Count/Distance/Increment". `BotMode` declares `SCRUMMING` and `EXTRACTOR`, and `bot_config_kwargs` names "the retired Grid checkbox" in the same file as the strip list.

A store that still carries the key

`SettingsManager._apply_dict` walks the fields `AppSettings` declares and copies the ones the file carries. A key the file holds and `AppSettings` does not declare is passed over. A store holding `position_distance_pct = 7.5` opens, its `username` and `default_target_balance` read back, the dialog draws its controls, and the next Save writes the file without the key.

The sentences the removal replaces

Each sentence below stands in an earlier section and no longer describes the page. The earlier text stays where it is.

Earlier sentence	What the page does now
"Six rows carry the defaults a new bot starts from, not a running bot's settings."	Five rows carry those defaults.
"Position Distance - Sets the spacing a new bot puts between its positions." and the three sentences and the code block under it	The page carries no Position Distance row, and no control on it sets position spacing.
"The strip list holds eleven names. This page contributes one of them and the Profit Folding page below contributes four more."	The strip list holds the same eleven names. The Profit Folding page contributes four and this page contributes none.
"Save writes all six keys."	Save writes five keys.
"The load path restores all six keys on the Trading page."	The load path restores all five keys on the Trading page.
"`src/gui/settings_dialog.py` — `_load_current`, the four it restores", and the `self._pos_distance.setValue` line inside that block	`_stored_rows` carries no `position_distance_pct` row, so the load path names four keys and the page has five rows.
"`| Trading | 6 | all six | four of six |`" in What each page persists	Trading, 5 controls, `_save` writes all five, `_load_current` restores all five.
"`| Trading | 6 | all six | all six |`" in What each page restores now	The same five.

2026-09-11 - Increment Style is off the Trading page

The Trading page carries four rows. The Increment Style row is removed. The row stored one of two words and read it back, and nothing in the program read the word.

Three files declared the row and every declaration is gone.

File	What it declared
`src/core/settings.py`	the `increment_style` field on `AppSettings`, and the `IncrementStyle` enum that supplied its default and its two words
`src/gui/settings_dialog.py`	the `QComboBox` and its `_stored_rows` entry
`src/gui/main_tabs/settings_dialog_surface.py`	the `increment_style` spec, its Trading layout step, its `PERSISTED_ROWS` entry, and the `INCREMENT_KEY` and `INCREMENT_DEFAULT` names

`_DEPRECATED_KWARGS` in `src/trading/container/config.py` never held this name, so the strip list is unchanged.

The bot config field of the same name stays

`increment_style` names a field on two objects that never meet. `AppSettings` held the page's word. `BotConfig` declares its own at `src/trading/container/config.py, in _BOT_CONFIG_SCRUMMING_ONLY_FIELDS`, `_BOT_CONFIG_SCRUMMING_ONLY_FIELDS` names it in `src/trading/container/config.py` so an Extractor is refused it, and `src/trading/container/restore.py, in restore_bots_from_state` copies it out of a saved bot. All three stay. The page never wrote any of them, and a bot built through the real creation path holds the same `linear` it held before.

Nothing reads the bot field either. `git grep "\.increment_style"` finds no reader, and the same search shape finds thirteen reads of `.target_balance`, so the absence is a fact about the code rather than about the search.

OVERTAKEN, and the sentences above are kept as written. The bot config field has since gone too. The name occurs 19 times in the tree and all 19 are on this page, so no module declares it, names it in a mode-only set, or copies it out of a saved bot. The same count on `target_balance` reads 677 occurrences across 89 files, so the zero is a reading rather than a search that found nothing.

Why no wizard control could open at it

A row on this page earns its keep by opening a wizard field at the stored figure, the way Default Target Balance opens the wizard's Target Balance. No wizard control sets an increment style. The wizard collects 52 keys and none is this name. Its nearest control is Spacing, a different quantity under a different key.

	Increment Style	Spacing
key	`increment_style`	`stack_spacing_mode`
choices	linear, logarithmic	Linear, Quadratic, Exponential
at	linear	Linear
what it shapes	how the gap between a bot's positions grows	how the gap between Stack tranches grows

The word belongs to the retired Grid Bot. The module docstring of `src/gui/bot_wizard.py` names the mode and its rows: "Grid Bot: Investment Amount, Position Count/Distance/Increment". The other two of that trio, `position_count` and `position_distance_pct`, already sit on the strip list as grid legacy. This one stayed on `BotConfig` instead, unread.

One place still carries the word `logarithmic` for a spacing, and it carries it only to replace it. `_sanitize_deprecated_kwargs` at `src/trading/container/config.py, in _sanitize_deprecated_kwargs` promotes `stack_spacing_mode="logarithmic"` to `"quadratic"`.

A store carrying the key after the removal

`SettingsManager._apply_dict` walks the fields `AppSettings` declares and copies the ones the file carries. A key the file holds and `AppSettings` does not declare is passed over rather than refused. A store holding `increment_style = "logarithmic"` opens, its `username` and `default_target_balance` read back, the page draws its controls, and the next Save writes the file without the key.

	before the removal	after the removal
drawn controls on the page	70	69
the Increment Style row is drawn	yes	no

	before the removal	after the removal
the Default Positions row beside it	drawn	drawn
`username` read back off the store	`u10_operator`	`u10_operator`
`default_target_balance` read back	777.0	777.0
`BotConfig.increment_style` on a new bot	`linear`	`linear`

The sentences this removal replaces

Each sentence below stands in an earlier section and no longer describes the page. The earlier text stays where it is.

Earlier sentence	What the page does now
"Six rows carry the defaults a new bot starts from, not a running bot's settings."	Four rows carry those defaults.
"Increment Style - Chooses whether that spacing stays even or widens on a curve." and the two sentences and the code block under it	The page carries no Increment Style row, and no control on it chooses how spacing grows.
"A bot takes its own increment style from the wizard rather than from this page."	No wizard control sets an increment style, and nothing reads the `BotConfig` field of that name.
"Save writes all six keys."	Save writes four keys.
"The load path restores all six keys on the Trading page."	The load path restores all four keys on the Trading page.
"`src/gui/settings_dialog.py` — `_load_current`, the four it restores", and the three `self._increment_style` lines inside that block	`_stored_rows` carries no `increment_style` row, so the load path names three keys and the page has four rows.
"`| Trading | 6 | all six | four of six |`" in What each page persists	Trading, 4 controls, `_save` writes all four, `_load_current` restores all four.
"`| Trading | 6 | all six | all six |`" in What each page restores now	The same four.

The 2026-09-11 Position Distance section above restated the same rows at five. Four is the count after both rows are gone.

2026-09-11 - Default Positions is off the Trading page

The Trading page carries three rows. The Default Positions row is removed. The row stored a count between 1 and 100 and read it back, and nothing outside the dialog read the count.

Three files declared the row and every declaration is gone.

File	What it declared
`src/core/settings.py`	the `default_position_count` field on `AppSettings`
`src/gui/settings_dialog.py`	the `QSpinBox` with its `setRange(1, 100)` and its form row, and its `_stored_rows` entry
`src/gui/main_tabs/settings_dialog_surface.py`	the `default_positions` spec, its Trading layout step, and its `PERSISTED_ROWS` entry

`AppSettings` no longer declares the name, so `SettingsManager.set` refuses it. Driven after the removal, `set("default_position_count", 42)` raised `KeyError: 'Unknown setting: default_position_count'`. That is the schema doing its stated job rather than a fault.

The bot-side spelling stays on the strip list

`position_count` is the name a bot-side kwarg would carry, and `src/trading/container/config.py, in _DEPRECATED_KWARGS` holds it beside `position_distance_pct`. Both stay on the list, so a `bot_state.json` still carrying either key restores without a `TypeError`. `_sanitize_deprecated_kwargs` drops `position_count` before `BotConfig.__init__`, and `BotConfig` declares no field of that name and no field of this row's name.

The word belongs to the retired Grid Bot. The module docstring of The module docstring of `src/gui/bot_wizard.py` names the mode and its rows: "Grid Bot: Investment Amount, Position Count/Distance/Increment". `BotMode` at `src/trading/container/config.py, in BotMode` declares `SCRUMMING` and `EXTRACTOR` alone. With this row gone, all three of that group are off the page.

No wizard control counts the positions a bot opens with

A row on this page earns its keep by opening a wizard field at the stored figure, the way Default Target Balance opens the wizard's Target Balance. The wizard collects 52 keys and none of them is a count of positions a new bot opens with. The nearest control is Tranche Count, and it counts something else.

	Default Positions	Tranche Count
key	`default_position_count`	`stack_tranche_count_target`
range	1 to 100	2 to 20
at	10	3
what it counts	positions a new bot opens with	Stack tranches created from one SCRUM

The meaning, the range and the default all differ, so the row had no wizard field to open at its figure.

One site in the repository counts positions, and it counts ones already open. `src/gui/main_tabs/stock_main_window_surface.py, in open_position_count` reads a live status list and reports how many positions stand now, on the stock window. That is a runtime count rather than an opening count, and it never touches this row's name.

A legacy store after the removal

`SettingsManager._apply_dict` walks the fields `AppSettings` declares and copies the ones the file carries. A key the file holds and `AppSettings` does not declare is passed over rather than refused. A store holding `default_position_count = 55` opens, its `username` and `default_target_balance` read back, the page draws its controls, and the next Save writes a file without the key while leaving the stored `username` intact.

	before the removal	after the removal
drawn controls on the page	69	68
control specs in the payload	70	69
the Default Positions row is drawn	yes	no
the Default Target Balance row beside it	drawn	drawn

	before the removal	after the removal
keys and groups one Save wrote	14	13
`username` read back off the store	`u11_driver`	`u11_driver`
a legacy store's `username`	`u11_legacy`	`u11_legacy`
a legacy store's `default_target_balance`	321.0	321.0
a legacy store's `default_position_count`	55	not answered
`stack_tranche_count_target` on a new bot	3	3

The sentences this row's removal replaces

Each sentence below stands in an earlier section and no longer describes the page. The earlier text stays where it is.

Earlier sentence	What the page does now
"Six rows carry the defaults a new bot starts from, not a running bot's settings."	Three rows carry those defaults.
"Default Positions - Sets how many positions a new bot opens with." and the key line, the two sentences and the code block under it	The page carries no Default Positions row, and no bot holds a count of positions it opens with.
"The dialog stores the count and restores it."	The dialog stores and restores nothing under this name.
"Save writes all six keys."	Save writes three keys.
"The load path restores all six keys on the Trading page."	The load path restores all three keys on the Trading page.
"`src/gui/settings_dialog.py` — `_load_current`, the four it restores", and the `self._default_positions.setValue` line inside that block	`_stored_rows` carries no `default_position_count` row, so of the four lines in that block one remains and the page has three rows.
"`| Trading | 6 | all six | four of six |`" in What each page persists	Trading, 3 controls, `_save` writes all three, `_load_current` restores all three.
"`| Trading | 6 | all six | all six |`" in What each page restores now	The same three.

The 2026-09-11 Increment Style section above restated the same rows at four. Three is the count after all three of the Grid Bot group are gone.

2026-09-11 - Default Target Balance opens the wizard's field

The Trading page carries three rows. This row is the one that reaches a new bot. It stores a dollar figure between $1.00 and $1,000,000.00 under the key `default_target_balance` and opens at $200.00. The row is verified and unchanged.

The row, drawn and driven

React draws the row. The build answers `react`, `draws_react('Settings dialog')` answers `True`, and the class is `SettingsDialogReact`. The Trading tab draws 68 controls and this row is one of them.

What was driven	What it answered
the drawn node	`INPUT`, `type="number"`, `min="1"`, `max="1000000"`

What was driven	What it answered
$777.00 typed on the page	the holder answered 777.0
Save pressed on the page	the store answered 777.0
a fresh manager on the same folder	777.0
the page reopened	the row drew 777.0

The round trip holds. A figure typed on the page survives a Save, a new manager and a new page.

Every reader of the stored figure

Seven sites name the key. One of them reads it outside the Settings dialog.

Site	What it does with the figure
`src/core/settings.py, in AppSettings`	declares `default_target_balance: float = 200.0`
`src/gui/main_tabs/settings_dialog_surface.py, in PERSISTED_ROWS`	the React model's save and load row
`src/gui/settings_dialog.py, in _stored_rows`	`_stored_rows`, the read on Save and the write on load
`src/gui/bot_wizard.py, in TradingParamsPage.__init__`	the wizard's Target Balance field opens here
`src/gui/main_window.py, in _adopt_topology_proposal`	writes an override for one wizard opening
`src/gui/main_tabs/bot_wizard_surface.py, in DEFAULT_TARGET_BALANCE_KEY and _apply_stored_target_balance`	the React wizard module's own read
`src/gui/main_tabs/bot_wizard_surface.py, in DEFAULT_TARGET_BALANCE_KEY`	names the key in the payload catalogue

The tab conversion table gives the wizard's seam row: both builds open `BotCreationWizard`. **+ New Bot** reaches `src/gui/bot_wizard.py, in TradingParamsPage.__init__`, and the React wizard module is a second reader that the button does not reach.

Nothing under `desktop/` names the row. The control specs travel from Python into the page payload, and the drawn node carries `data-name="default_balance"` as data.

Where the figure stops being this setting

Diagram: flowchart, left to right

- Settings row — default_target_balance -> **SettingsManager — settings.toml**
- SettingsManager — settings.toml -> **C"]**
- main_window._create_bot — get_all(-> **bot_wizard.py, in TradingParamsPage.__init__ — Target Balance field**
- bot_wizard.py, in TradingParamsPage.__init__ — Target Balance field -> **TradingParamsPage.get_config — target_balance**
- TradingParamsPage.get_config — target_balance -> **bot_config_kwargs**
- bot_config_kwargs -> **BotConfig.target_balance**

`src/gui/bot_wizard.py, in TradingParamsPage.get_config` renames the figure `target_balance`. From that step on the figure is the bot's own setting.

Driven with $4,321.55 stored:

Step	What it answered
the wizard's Target Balance	4321.55
keys the wizard collected	52
`default_target_balance` among them	no
`target_balance` among them	4321.55
`bot_config_kwargs` `target_balance`	4321.55
the built config	`BotConfig`
`BotConfig.target_balance`	4321.55
`BotConfig` holds `default_target_balance`	no

Two more stored figures opened the wizard at themselves: $777.00 and $12.75. Market Inspector topology adoption hands `defaults_override` in place of the stored figure, and $25.00 opened the wizard at $25.00.

The range, at both ends and past them

The page, the wizard and this manual declare the same range. The page and the wizard hold a figure to that range in the same way.

Written to the store	The store holds	The page draws	The wizard opens at
1.00	1.0	1	1.0
1000000.00	1000000.0	1000000	1000000.0
0.50	0.5	1	1.0
1000001.00	1000001.0	1000000	1000000.0

The row holds a typed figure to the range before the store sees it, so the page stores no figure outside the range. A figure written into the file by hand meets the nearer end on the page and in the wizard alike, so no bot opens at a refused figure.

A store with the figure removed

What was driven	What it answered
`AppSettings` declares	200.0
a settings file with the key deleted	200.0
the wizard on that store	200.0
the wizard on an empty defaults dict	200.0

`get_all()` is `asdict` of a dataclass that declares the field, so the dict the wizard takes always carries the key. `_apply_dict` keeps the declared default for a field the file omits. The wizard's own fallback at `src/gui/bot_wizard.py, in TradingParamsPage.__init__` fires only where `main_window.py, in _create_bot` hands `{}`, which is a run with no settings manager, and the fallback gives the same $200.00.

The page's build figure for this control is $1.00, because the spec carries a range and no `value`. The store always answers, so the page draws $1.00 only where it refuses the stored figure.

What this section adds

Nothing on the page, in the store or in the wizard changed. The row matches every sentence written about it, and this section records the driven readings.

Two gaps stand outside this row and sit here as measured facts.

Gap	What was measured
a text figure in the settings file	`src/gui/settings_dialog.py, in _show_stored` catches it, logs "Settings kept the default for default_target_balance", and draws $1.00. `src/gui/bot_wizard.py, in TradingParamsPage.__init__` carries no guard, so `QDoubleSpinBox.setValue(str)` raises `TypeError` and `BotCreationWizard` cannot open. Only an edited file puts a text figure in the store
a write to a control the page already drew	The React page draws every control with `defaultValue`, which seeds the DOM once. A write from Python after the page is up moved the store and not the drawn figure: the node held 321 while the holder answered 654.0. This row stands clear of it, because `_load_current` runs before the page mounts and nothing writes the row afterwards

2026-09-12 - Bot Visibility opens the wizard's Order Visibility control

The Trading page row stores `orderbook` or `internal` under the key `bot_visibility` and opens at `orderbook`. The row is now the default the bot creation wizard's own control opens on. It was unwired, and removing it was refused: the behaviour it names runs on the bot, and the wizard carries a control for it.

The row and the wizard control, side by side

	The Settings row	The wizard control
Where	`src/gui/main_tabs/settings_dialog_surface.py, in CONTROL_SPECS`	`src/gui/bot_wizard.py, in TradingParamsPage.__init__`
Label	`Bot Visibility:`	`Order Visibility:`
Choices	`orderbook`, `internal`	`orderbook`, `internal`
Opens at	`orderbook`	the stored name, `orderbook` when the store holds none
What it writes	the store key `bot_visibility`	the config key `visibility`

The two agree on meaning, on both choices and on the default, so the row is a legitimate application default for the control.

Every reader of the stored visibility

Four sites name the key. One of them reads it outside the Settings dialog.

Site	What it does with the name
`src/core/settings.py, in AppSettings`	declares `bot_visibility: str = BotVisibility.ORDERBOOK.value`
`src/gui/settings_dialog.py, in _stored_rows`	`_stored_rows`, the read on Save and the write on load

Site	What it does with the name
`src/gui/main_tabs/settings_dialog_surface.py, in PERSISTED_ROWS`	the same row on the Qt-free surface
`src/gui/bot_wizard.py, in TradingParamsPage.__init__`	the wizard's Order Visibility control opens here

Before this section there was no reader outside the dialog. The bare name was searched in all 1081 files `git ls-files` reports, and the search was proved against `default_target_balance`, read out of the settings dict, and against `ai_monitor`, read through `getattr`. Both controls answered with their known reading sites, so the zero was a measurement.

Where the stored name reaches the bot

Diagram: flowchart, left to right

- Settings row — bot_visibility -> **SettingsManager — settings.toml**
- SettingsManager — settings.toml -> **C"]**
- main_window._create_bot — get_all(-> **bot_wizard.py, in TradingParamsPage.__init__ — Order Visibility control**
- bot_wizard.py, in TradingParamsPage.__init__ — Order Visibility control -> **TradingParamsPage.get_config — visibility**
- TradingParamsPage.get_config — visibility -> **BotConfig.visibility**
- BotConfig.visibility -> **scrumming_bot.py, in ScrummingBot.__init__ — _invisible**

`BotConfig.visibility` at `src/trading/container/config.py, in BotConfig` is where the name stops being this setting. From there the bot owns it, and `src/trading/scrumming_bot.py, in set_visibility_live` changes it live on one bot without touching the store.

The round trip, with a Save that changed nothing

Driven on the class the running build opens. `resolve_variant` answered `react` and `surface_class('Settings dialog')` returned `SettingsDialogReact`. The home was redirected into a scratch directory before the settings module was imported, so the settings default directory bound under that directory.

Stored	The box opened at	The store after a Save with no edit	The box on reopening
`internal`	`internal`	`internal`	`internal`
`orderbook`	`orderbook`	`orderbook`	`orderbook`

Two different stored names gave two different readings, so the reading discriminates. The payload the dialog pushes to the page carries 69 control specs, and the figure is the same before and after.

What a new bot opens at, and what an existing bot keeps

Driven	What it answered
`internal` stored, the wizard opened	`internal`
`orderbook` stored, the wizard opened	`orderbook`
a name the two items do not offer	`orderbook`, the build figure
app setting `internal`, a saved bot holding `orderbook`	the restored config holds `orderbook`
app setting `orderbook`, a saved bot holding `internal`	the restored config holds `internal`

A restored bot reads `cfg.get("visibility", "orderbook")` from its own saved config at `src/trading/container/restore.py, in restore_bots_from_state` and reads no application setting. Every `SettingsManager.get` call of the run was recorded with its calling file: the wizard and the restore paths read the key no times, and the Settings dialog read it once. The recorder saw the read it was meant to see, so the two zeros are measurements.

The sentence this wiring replaces

Earlier sentence	What the code does now
"A bot takes its visibility from the wizard, under a different name."	A new bot still takes its visibility from the wizard, and the wizard's control now opens at this row's stored name.

The two revert sentences in the same paragraph were already retired by Save keeps what the store holds, and that section's reading was re-driven here and holds.

What the wiring adds

One lookup on one control, in the file the + New Bot button reaches. Nothing a running bot does changed, nothing on the Settings page changed and no order placement path changed.

Two absences stand outside this row and sit here as measured facts.

Absence	What was measured
the shell's wizard request carries no stored defaults	`src/gui/web/bot_wizard.js, in press` calls `bot_wizard.state` with the step values alone. `src/gui/main_tabs/bot_wizard_surface.py, in view_model` reads `defaults` from the request, so the Qt-free wizard surface receives an empty bag on every call and `_apply_stored_target_balance` falls back to its built-in figure
the Qt-free wizard surface has no visibility lookup	`src/gui/main_tabs/bot_wizard_surface.py, in _apply_stored_target_balance` applies the stored target balance and nothing applies a stored visibility. Its `visibility` combo in `COMBO_FIELDS` carries the same two entries as the Qt one and opens at index 0

OVERTAKEN, and the row above is kept as written. The lookup is there now. `src/gui/main_tabs/bot_wizard_surface.py, in _apply_stored_visibility` reads the stored key and is one of five stored reads the surface runs together, beside the target balance, the aggressive flag, the phantom switch and the lock count. The two entries are unchanged.

2026-09-12 - Enable aggressive trading mode opens the wizard's own box

The Trading page row stores a boolean under the key `aggressive_trading` and opens clear. The row is now the default the bot creation wizard's own Aggressive Trading checkbox opens on. It was unwired, and removing it was refused: the behaviour it names runs on the bot, and the wizard carries a checkbox for it.

The Settings box and the wizard box, side by side

	The Settings row	The wizard box
Where	`src/gui/main_tabs/settings_dialog_surface.py, in CONTROL_SPECS`	`src/gui/bot_wizard.py, in TradingParamsPage.__init__`
Text	`Enable aggressive trading mode`	`Aggressive Trading (force IOC-limit takers)`
Choices	on, off	on, off
Opens at	off	the stored flag, off when the store holds none
What it writes	the store key `aggressive_trading`	the config key `aggressive_trading`

The two agree on meaning, on both choices and on the default, and here the two keys are the same word.

The two sets that share the word

The bare name `aggressive_trading` stands in 19 python lines of the 1081 files `git ls-files` reports. They split into two sets, and no line crosses between them.

Set	Lines	What they are
the application setting	3	the `AppSettings` field, the dialog's `_stored_rows` row, the same row on the Qt-free surface
a bot's own config field	16	the `BotConfig` field, the restore read, the bot's own `_aggressive`, the live toggle, the wizard write and the live settings box

The bot's own field is a separate setting with its own screen row, and this section changes nothing in that set.

Every reader of the stored flag

Four sites name the application key. One of them reads it outside the Settings dialog.

Site	What it does with the flag
`src/core/settings.py, in AppSettings`	declares `aggressive_trading: bool = False`
`src/gui/settings_dialog.py, in _stored_rows`	`_stored_rows`, the read on Save and the write on load
`src/gui/main_tabs/settings_dialog_surface.py, in PERSISTED_ROWS`	the same row on the Qt-free surface
`src/gui/bot_wizard.py, in TradingParamsPage.__init__`	the wizard's Aggressive Trading box opens here

Before this section there was no reader outside the dialog. The bare name was searched in every tracked file, and the search was proved against `default_target_balance`, read out of the settings dict, and against `ai_monitor`, read through `getattr`. Both controls answered with their known reading sites, so the zero was a measurement.

Where the stored flag reaches the bot

Diagram: flowchart, left to right

- Settings row — aggressive_trading -> **SettingsManager — settings.toml**
- SettingsManager — settings.toml -> **C"]**

- main_window._create_bot — get_all(-> **bot_wizard.py, in TradingParamsPage.__init__ — Aggressive Trading box**
- bot_wizard.py, in TradingParamsPage.__init__ — Aggressive Trading box -> **TradingParamsPage.get_config — aggressive_trading**
- TradingParamsPage.get_config — aggressive_trading -> **BotConfig.aggressive_trading**
- BotConfig.aggressive_trading -> **scrumming_bot.py, in ScrummingBot.__init__ — _aggressive**
- scrumming_bot.py, in ScrummingBot.__init__ — _aggressive -> **scrumming_bot.py, in _open_stack_from_scrum — OrderType.IOC_LIMIT**

`BotConfig.aggressive_trading` at `src/trading/container/config.py, in BotConfig` is where the flag stops being this setting. From there the bot owns it, and `src/trading/scrumming_bot.py, in set_aggressive_live` changes it live on one bot without touching the store.

The round trip on both choices

Driven on the class the running build opens. `resolve_variant` answered `react` and `surface_class('Settings dialog')` returned `SettingsDialogReact`. The home was redirected into a scratch directory before the settings module was imported, so the settings default directory bound under that directory. No venue was contacted. A name-lookup guard refused eight lookups, every one of them the wizard's own market scan.

Stored	The box opened at	The store after a Save with no edit	The box on reopening
on	on	on	on
off	off	off	off

Two different stored flags gave two different readings, so the reading discriminates. The payload the dialog pushes to the page carries 69 control specs and the page draws 68, the same two figures before and after.

What a new bot carries, and what an existing bot keeps

Driven	What it answered
on stored, the wizard opened	on
off stored, the wizard opened	off
on stored, through `bot_config_kwargs` and `make_bot_config`	`BotConfig.aggressive_trading` on
off stored, the same path	`BotConfig.aggressive_trading` off
the setting on, a saved bot holding off	the restored bot holds off
the setting off, a saved bot holding on	the restored bot holds on

A restored bot reads `cfg.get("aggressive_trading", False)` from its own saved config at `src/trading/container/restore.py, in restore_bots_from_state` and reads no application setting. Every `SettingsManager.get` and `get_all` call of the run was recorded with its calling file. The wizard and the restore paths read the key no times, and the driver read it once. The recorder saw the read it was meant to see, so the two zeros are measurements.

What this wiring retires

Earlier sentence	What the code does now
"The wizard carries its own aggressive trading checkbox, and that is the one a new bot reads."	A new bot still reads the wizard's checkbox, and that checkbox now opens at this row's stored flag.

The two revert sentences in the same paragraph were already retired by Save keeps what the store holds, and that section's reading was re-driven here and holds.

What the one lookup adds

One lookup on one checkbox, in the file the + New Bot button reaches. Nothing a running bot does changed, nothing on the Settings page changed and no order placement path changed.

The wizard path this wiring does not reach

The Electron shell's wizard request carries no stored defaults, so the Qt-free wizard surface opens its box at the build figure whatever the store holds.

Absence	What was measured
the shell's wizard request carries no stored defaults	`src/gui/web/bot_wizard.js` never names `defaults`, and `src/gui/main_tabs/bot_wizard_surface.py, in view_model` reads `defaults` out of the request. A request with no bag left `target_balance` at its built-in 200.0
the Qt-free wizard surface has no aggressive lookup	`bot_wizard_surface.view_model` with a bag holding the flag on still answered off. `CHECK_FIELDS` in `src/gui/main_tabs/bot_wizard_surface.py` seeds the box at `False` and nothing reads the store
a hand-edited word in the store reads as on	The store declares a boolean. A word put there by hand ticks the box on the Settings page and in the wizard alike, because both admit the value as `bool(value)`

OVERTAKEN, and the row about the missing lookup is kept as written. The lookup is there now. `src/gui/main_tabs/bot_wizard_surface.py, in _apply_stored_aggressive` reads the stored key and falls back to the seeded `False` only when the store holds none. The seeded figure is unchanged.

2026-09-12 - Profit Folding Active waits on the wizard's folding page

Driven on the React build, which is what the running application draws. The home was redirected into a scratch directory before `src.core.settings` was imported, so the settings directory and the log root bound under that directory. No stored setting of the running install was read and no venue was contacted.

The row stays on the page exactly as it is. It stores and restores correctly, and the value it stores has no reader. The one place a new bot could read it is a wizard page no route reaches, so a default pointed at that page would change no reading. The row therefore waits, and this section records what it waits on.

The Settings box and the wizard box it would open

	The Settings row	The wizard control
Where	`settings_dialog_surface`	`bot_wizard`
Kind	checkbox	checkbox
Text	`Profit Folding / Upward Distribution Active`	`Enable Profit Folding`
Choices	on, off	on, off
Default	on	on
What it writes	the store key `profit_folding.active`	the bot config key `profit_folding_active`

The two agree on meaning, on both choices and on the default.

Every reader of the stored group

Nothing outside the dialog. The bare name `profit_folding` returns 52 hits across the 1,081 files the repository tracks, 41 of them Python. Eight name the application setting, and all eight are the dialog or the declaration behind it.

Site	What it is
`settings`	the field defaults for the group
`settings`	the `AppSettings` field that declares the group
`settings`	the manager docstring naming the group
`settings_dialog`	the row the save and the load path both walk
`settings_dialog_surface`	the group in the save list
`settings_dialog_surface`	the group key
`settings_dialog_surface`	the same six rows on the Qt-free surface
`settings_dialog_surface`	the group read off the page

The other 33 name a bot's own field `profit_folding_active`, which is a different name and a different thing. That field is read seven times inside the trading engine and gates the fold in `src/trading/scrumming_bot.py, in _apply_fold_target_growth` and in `_preview_fold_growth`.

The nested getter and setter on `SettingsManager` still have no callers. The bare names `get_nested` and `set_nested` return two hits each, and both are the declaration and the class docstring in `src/core/settings.py`.

The round trip of the whole stored group

All six keys in the group survive a Save that changed nothing, in both stored states. The ten top-level rows and the two sibling groups came back unchanged across the same Save.

Key	Stored	The control showed	The store after Save
`active`	`false`	`false`	`false`
`mode`	`logarithmic`	`logarithmic`	`logarithmic`
`fold_target`	`most_recent_buy`	`most_recent_buy`	`most_recent_buy`
`fold_target_count`	`9`	`9`	`9`

Key	Stored	The control showed	The store after Save
`distribute_target`	`x_sell`	`x_sell`	`x_sell`
`distribute_target_count`	`7`	`7`	`7`

The same read with the group stored at its defaults reported all six kept as well, so the reading tells the two states apart.

Why no route reaches the wizard's folding page

The wizard registers six pages and routes to five. `nextId` was read from each page in turn, with that page set as the start page.

Diagram: flowchart, left to right

- Trading Mode -> **Select Asset Pair**
- Trading Mode -> **Extractor Pool**
- Select Asset Pair -> **Trading Parameters**
- Extractor Pool -> **Trading Parameters**
- Trading Parameters -> **Phantom Bots**
- Phantom Bots -> **finish**
- Trading Parameters -> **finish**
- Profit Folding and Upward Distribution -> **finish**

No page answers the folding page. The only branch that would is guarded behind `src/gui/bot_wizard.py, in is_grid`, and `is_grid()` returns `False` always. The Qt-free wizard surface states the same in a named constant, `UNREACHABLE_PAGES` at `bot_wizard_surface`, and its view model reports the folding page unreachable.

A second cut sits behind the first. `BotCreationWizard.get_bot_config` never calls the folding page's `get_config`. A Scrumming bot collects 52 keys and `profit_folding_active` is not among them, against the six keys that page's own `get_config` returns.

What a new bot opens at whatever is stored

Driven	Reading
the group stored with `active` off, the wizard's box	on, the build figure
`profit_folding_active` among the keys the wizard collects	no
`profit_folding_active` on the new bot's config	on
the declared default on the bot config field	on
`profit_folding_active` carried when anything collects it	yes

The carriage from the wizard to the bot is already built, so the field arrives the moment the folding page is reached and collected. An existing bot is untouched either way: it takes the flag from its own saved config at `src/trading/container/restore.py, in restore_bots_from_state`, which reads no application setting.

The sentence this section leaves standing

`Profit Folding / Upward Distribution Active - Turns the whole page on.`

The box does not turn the page on. It carries no toggle connection and no enable rule on either build, so the three groups below it stay live with the box clear. The sentence stands as the specification and the page does not yet meet it.

What waits on what

The blocker	What it needs
the wizard's folding page is unreachable	a route to the page and a call to its `get_config`
the box does not turn the page on	an enable rule over the three groups below it, which are the other five rows
the Electron shell's wizard request carries no stored defaults	the request carrying the defaults bag the receiving surface reads

Until the first of the three lands, a stored choice on this row has nowhere to arrive, and the row is left as it stands.

2026-09-12 - Distribution Mode is off the Profit Folding page

The Profit Folding page carried one master checkbox and three groups of radio buttons. The Distribution Mode group is removed. It offered two choices, `equal` and `logarithmic`, stored the chosen word under the key `mode` inside the `profit_folding` group, and read it back. Nothing outside the dialog read the word.

Three files declared the row and every declaration is gone.

File	What it declared
`src/core/settings.py`	`FoldDistributeMode`, and the `mode` field on `ProfitFoldingSettings`
`src/gui/settings_dialog.py`	`FOLD_MODE_BUTTONS`, the `QGroupBox` with its two `QRadioButtons`, and the `_stored_groups` row
`src/gui/main_tabs/settings_dialog_surface.py`	`MODE_GROUP_TITLE`, `EQUAL_MODE`, `LOG_MODE`, `FOLD_MODE_BUTTONS`, the `fold_equal` and `fold_log` specs, the `GROUPS` entry, the Profit Folding layout step, and the `_folding_rows` entry

The two spellings this row carried

The Settings dialog stored the word under `mode`. The bot wizard's own Distribution Mode group writes the same word under `fold_mode`. The two names addressed the same choice, and the dialog said so itself: the tuple holding this row's two buttons was named `FOLD_MODE_BUTTONS` while the row it fed wrote `mode`.

What matched	The Settings row	The wizard's group
the group title	Distribution Mode	Distribution Mode
the two controls	`fold_equal`, `fold_log`	`fold_equal`, `fold_log`
the two stored words	`equal`, `logarithmic`	`equal`, `logarithmic`
the default	Equal checked at build	Equal checked at build

What matched	The Settings row	The wizard's group
the key it wrote	`profit_folding.mode`	the bot config key `fold_mode`

`fold_mode` stays in `_DEPRECATED_KWARGS` at `src/trading/container/config.py, in _BOT_CONFIG_EXTRACTOR_ONLY_FIELDS`, so a `bot_state.json` still carrying it restores without a `TypeError`. `_sanitize_deprecated_kwargs` drops it before `BotConfig.__init__`, and `BotConfig` declares no field of that name. The wizard's group is the only control left that writes it, and it sits on the wizard page no route reaches.

Because `mode` never matched a strip-list name, the count at Settings > Trading stands as written: the Profit Folding page contributes four of the eleven names, and this row was never one of them.

Why no fold takes a distribution mode

The row offered a choice between two ways of spreading a fold across positions. No code in the engine offers that choice. Every fold path was read for a mode parameter and none takes one.

Where	What it does
`src/trading/scrumming_bot.py, in _apply_fold_target_growth`	drains fold surplus into one target balance; no positions, no spread
`src/trading/smart_wire.py, in distribute_fold_profit`	splits realised fold profit across wires to other bots by per-wire percent
`src/trading/scrumming/wire_routing.py, in _spread_wire_usd_over_fold_queue`	adds wire income evenly to every standing fold tranche; the even split is fixed and has no alternative
`stack_math fold_ladder_prices`	places fold prices from `stack_spacing_mode` and decides no size
`profit_fold apply_profit_fold`	grows one target balance; its own docstring records that no module imports it

The one mode-like selector near folding refuses this row's value. `src/trading/stack_math.py, in SPACING_MODES` holds `quadratic`, `fibonacci`, `linear` and `exponential`. `level_multipliers("logarithmic", 3)` raises `ValueError`, while `level_multipliers("quadratic", 3)` returns `[1.0, 4.0, 9.0]`.

The two choices belong to the retired Grid Bot. The earliest tracked copy of the strip list names the block in its own words, as grid-legacy fields kept so a stored bot does not crash on load, and `src/trading/container/restore.py, in restore_bots_from_state` records that the grid mode was removed.

A six-key group opening with five rows

`SettingsManager._apply_dict` copies the whole `profit_folding` dict off the file, so a store written before the removal still carries its `mode` entry and still opens. The page reads back the five keys it has rows for, and the next Save writes the group without the sixth.

	before the removal	after the removal
control specs in the payload	69	67
control specs on the Profit Folding tab	11	9
groups declared on the surface	13	12

	before the removal	after the removal
a Distribution Mode group is drawn	yes	no
rows the `profit_folding` group declares	6	5
keys a legacy store's group holds on opening	6	6
keys the group holds after one Save	6	5
keys and groups one Save wrote	13	13
`active` read back off a legacy store	`False`	`False`
`fold_target`	`most_recent_buy`	`most_recent_buy`
`fold_target_count`	`9`	`9`
`distribute_target`	`x_sell`	`x_sell`
`distribute_target_count`	`7`	`7`
`username` beside the group	`u16_driver`	`u16_driver`
`bot_visibility` beside the group	`internal`	`internal`

Every sibling in the shared group reads back at its stored value, and Save raised no warning box.

The sentences the page keeps that no longer describe it

Each sentence below stands in an earlier section and no longer describes the page. The earlier text stays where it is.

Earlier sentence	What the page does now
"One master checkbox and three groups of radio buttons, eleven controls in all."	One master checkbox and two groups of radio buttons, nine controls in all.
"Distribution Mode - Chooses whether a fold spreads evenly across the chosen positions or on a curve." and the key line, the two sentences and the code block under it	The page carries no Distribution Mode group, and no fold in the engine takes a distribution mode.
"`| Profit Folding | 11 | the whole group |` `active` alone |" in What each page persists	Profit Folding, 9 controls, `_save` writes the whole group, `_load_current` restores the whole group.
"`| Profit Folding | 11 | the whole group | the whole group |`" in What each page restores now	The same nine controls and five keys.
"| Profit Folding | Distribution Mode, Profit Folding Target, Fold to X# of buy positions, Upward Distribution Target, Distribute to X# of sell positions |" in The fourteen rows a Save overwrote	Thirteen rows, four of them on this page. Distribution Mode is off the page.

The master switch above this row gated five rows. It gates four now, and the four are Profit Folding Target, Fold to X# of buy positions, Upward Distribution Target and Distribute to X# of sell positions.

2026-09-12 - Profit Folding Target waits on its count row

The third row on the Profit Folding page offers three choices: fold to all buy positions, to a counted number of them, or to the most recent. It stores the chosen word under the key `fold_target` inside the `profit_folding` group and reads it back. Nothing outside the dialog reads the word, and no fold in the engine reaches a chosen set of buy positions. The row is therefore dead, and it stays on the page until the count row beside it comes off with it.

What the three buttons choose, and what reads the word

Build	The controls
React, the build that runs	`fold_all`, `fold_x` and `fold_recent`, three radio specs under `FOLD_GROUP_TITLE` in `src/gui/main_tabs/settings_dialog_surface.py`
Qt, preserved beside it	three `QRadioButton` inside `QGroupBox("Profit Folding Target")` in `src/gui/settings_dialog.py`

Both write `profit_folding.fold_target` through one `_stored_groups` row, and `FOLD_TARGET_BUTTONS` fixes the read order that decides the stored word.

The bare name `fold_target` was searched over the 1077 tracked files outside `dist/` and `docs-archive/`. It returns 16 hits: three declare the row, five belong to the bot wizard's own group, and eight are the strip list and this manual quoting it. No site reads the word.

Read shape	Sites
`getattr(x, "fold_target", ...)`	0
`x["fold_target"]`	0
`sm.get_nested("profit_folding", "fold_target")`	0
`getattr(bot.config, "fold_target", ...)`	0
`cfg.fold_target`	0

The same five shapes, spelled for the sibling key `profit_folding_active`, return 16 sites in the same files, so the zeros read the repository rather than a blind search. `FoldTarget` in `src/core/settings.py` has two sites, both its own declaration.

`fold_target` stays in `_DEPRECATED_KWARGS` at `src/trading/container/config.py, in _BOT_CONFIG_EXTRACTOR_ONLY_FIELDS`, so a `bot_state.json` still carrying it restores without a `TypeError`. `BotConfig` declares no field of that name, and `_sanitize_deprecated_kwargs` drops the key before `BotConfig.__init__`. This is where the row differs from Profit Folding Active: `profit_folding_active` is a real `BotConfig` field, so that row has a destination behind one cut route. `fold_target` has no field to land in, so repairing the wizard's folding page would not give it one.

Why no fold reaches a chosen set of buy positions

Every fold path was asked for its runtime signature. None takes a target of positions or a count of them.

Where	What it does
`profit_fold` `apply_profit_fold`	grows one dollar target balance; its `target` parameter is annotated `float`, and its own docstring records that no module imports it
`src/trading/scrumming_bot.py, in _apply_fold_target_growth`	drains fold surplus into one target balance; no positions, no set
`src/trading/smart_wire.py, in distribute_fold_profit`	splits realised fold profit across wires to other bots by per-wire percent
`src/trading/scrumming/wire_routing.py, in _spread_wire_usd_over_fold_queue`	adds wire income evenly to every standing fold tranche; the share is fixed at the dollars divided by the tranche count

Where	What it does
`src/trading/scrumming/fold_tranches.py, in _fold_eligible_tranches`	picks the tranches a fold may reach by price alone
`src/trading/scrumming/fold_tranches.py, in _fold_discharge_order`	orders those tranches by `ref`, highest first
`stack_math fold_ladder_prices`	places fold prices and decides no size; its `levels` counts prices on a ladder

The two paths that do decide which tranches a fold reaches were driven over three tranches at refs 120, 110 and 100. `_fold_discharge_order` returned them in price order and not in `created_ts` order, so "most recent" is not even the order the queue discharges in. `_fold_eligible_tranches` at 105 returned the two tranches above that price and at 50 returned all three, so the set is price and no count narrows it.

The even spread handed each of three tranches the same share, and the share is the dollars divided by the tranche count. It is fed by wire income rather than by a fold, and no second way of spreading exists.

The one selector near folding refuses this row's words. `SPACING_MODES` in `src/trading/stack_math.py` holds `quadratic`, `fibonacci`, `linear` and `exponential`. `level_multipliers("x_buy", 3)` raises `ValueError`, while `level_multipliers("linear", 3)` returns `[1.0, 2.0, 3.0]`.

The count that has no other referent

The count row beside this one, Fold to X# of buy positions, is not a setting in its own right. It is the number one of these three choices uses, and four readings tie it to this row.

What ties them	Measured
the group	the spin `fold_x_count` carries the group title `Profit Folding Target`, which is this row's title
the layout	the Qt-free surface puts `fold_x` and `fold_x_count` in one row tuple, and the Qt tab puts them in one `QHBoxLayout`
the words	the spin's spec carries `label` `None` and no `text`. Every word beside it on screen belongs to the radio `fold_x`
the sentence	"Fold to X# of buy positions - Sets that counted number" points at this row's sentence for what "that" names

Driven with this row's three radios taken out, the group titled `Profit Folding Target` holds one control: a spin box named `fold_x_count`, with no label and no text. Taking this row off alone would leave a numeric box with no words, under a title describing a choice the page no longer offers.

The two rows are one decision, so one change takes both off together.

A five-key group opening with the row gone

`SettingsManager._apply_dict` copies the whole `profit_folding` dict off the file, so a store written before the row comes off still carries its `fold_target` entry and still opens. The figures below come from driving `SettingsDialogReact` on one store holding all five keys, with the row present and then with the row taken out.

	the row present	the row gone
the dialog built	yes	yes
control specs in the payload	67	64

	the row present	the row gone
control specs on the Profit Folding tab	9	6
control specs in the Profit Folding Target group	4	1
a legacy store's five keys opened	yes	yes
keys the group holds after one Save	5	4
keys and groups one Save wrote	13	13
warning boxes raised	0	0
`active` read back off a legacy store	`False`	`False`
`fold_target_count`	9	9
`distribute_target`	`x_sell`	`x_sell`
`distribute_target_count`	7	7

Every one of the four siblings in the shared group reads back at its stored value, both before and after the Save, and the four read back again when the store is reopened.

With the row present, all three stored words round-trip. Each opens the page on its own button, each is read back as itself, and each survives a Save that changed nothing.

Stored `fold_target`	**The radio ticked**	**The store after a Save with no edit**
`all_buy`	`fold_all`	`all_buy`
`x_buy`	`fold_x`	`x_buy`
`most_recent_buy`	`fold_recent`	`most_recent_buy`

The claim the page makes that the page does not keep

The sentence "Three radio buttons and the count box the middle one gates" stands in an earlier section. No gate exists. Driven on `SettingsDialogReact`, the count box stays able to take an edit with each of the three radios ticked in turn, and the same reading returns a refusal when the box is deliberately disabled. The Qt folding tab makes no `toggled` connection and calls no `setEnabled`, and the Qt-free surface holds no enable rule for either name.

The two sentences the engine no longer keeps

The sentence below stands in an earlier section and no longer describes the engine. The earlier text stays where it is.

Earlier sentence	What the engine does now
"Profit Folding Target - Chooses which buy positions a fold reaches: all of them, a counted number of them, or the most recent."	No fold chooses a set of buy positions. A fold reaches the tranches whose reference price is above the market and discharges them in price order.
"Three radio buttons and the count box the middle one gates."	The count box is live with any of the three radios ticked.

2026-09-12 - Fold to X# of buy positions takes the whole target group off the page

The Profit Folding page carried one master checkbox and two groups of radio buttons. The `Profit Folding Target` group is removed, all four controls together: the three radio buttons and the count box inside them. The group offered a choice of which buy positions a fold reaches and a number for the middle choice. No fold in the engine offers that choice, and no fold counts the positions it reaches.

Three files declared the group and every declaration is gone.

File	What it declared
`src/core/settings.py`	`FoldTarget`, and the `fold_target` and `fold_target_count` fields on `ProfitFoldingSettings`
`src/gui/settings_dialog.py`	`FOLD_TARGET_BUTTONS`, the `QGroupBox` with its three `QRadioButtons` and its `QSpinBox`, and the two `_stored_groups` rows
`src/gui/main_tabs/settings_dialog_surface.py`	`FOLD_GROUP_TITLE`, `FOLD_ALL`, `FOLD_X`, `FOLD_RECENT`, `FOLD_TARGET_BUTTONS`, the four control specs, the `GROUPS` entry, the Profit Folding layout step, and the two `_folding_rows` entries

`FOLD_COUNT_DEFAULT` stays, because the sell-side count row reads it.

The four controls the group held

The control	What it stored
the radio `fold_all`, "Fold to ALL buy positions", checked at build	`profit_folding.fold_target` as `all_buy`
the radio `fold_x`, "Fold to X# of buy positions:"	the same key as `x_buy`
the radio `fold_recent`, "Fold to most recent buy positions"	the same key as `most_recent_buy`
the spin `fold_x_count`, 1 to 100, at 5, with no label and no text	`profit_folding.fold_target_count`

The spin carried the group title of the three radios and shared one layout row with `fold_x`. Every word beside it on screen belonged to that radio, so the count had no words of its own and no meaning apart from the choice it sized.

What the number was meant to count

The bare name `fold_target_count` appears at 11 sites across the 1077 tracked files outside `dist/` and `docs-archive/`: three declared this row, two belong to the bot wizard's own count, and six are the strip list and this manual quoting it. Six read shapes were spelled for the name and each returned nothing.

Read shape	Sites
`getattr(x, "fold_target_count", ...)`	0
`getattr(bot.config, "fold_target_count", ...)`	0
`x["fold_target_count"]`	0
`get_nested(..., "fold_target_count")`	0
`get(..., "fold_target_count")`	0
`cfg.fold_target_count`	0

The same six shapes, spelled for the sibling key `profit_folding_active`, return 17 sites over the same files, so the zeros read the repository rather than a blind search.

`fold_target` and `fold_target_count` both stay in `_DEPRECATED_KWARGS` at `src/trading/container/config.py, in _BOT_CONFIG_EXTRACTOR_ONLY_FIELDS`, so a `bot_state.json` still carrying either restores without a `TypeError`. `BotConfig` declares neither name as a field, and `_sanitize_deprecated_kwargs` drops both before `BotConfig.__init__`.

One dollar cap bounds a fold, and no count does

`src/trading/scrumming/tick_phases.py, in _tick_execute_fold` is the live fold-back route. It asks `_fold_eligible_tranches` which queued tranches the market price reaches, sorts them by `initial_buy_price`, then hands the sorted list and one dollar figure to `_plan_fold_consumption` at `src/trading/scrumming/fold_tranches.py, in _plan_fold_consumption`. That dollar figure is `cycle_growth_cap_usd` less what the cycle has already consumed.

Driven over five queued tranches at refs 150 down to 110, on a bot config carrying `fold_target_count = 2`:

What was asked	Tranches the fold reached
a cap of $1000	5 of 5
a cap of $25	3 of 5, one of them part-consumed
a cap of $0	0 of 5

The stored count of 2 narrowed nothing. The two tighter caps show the reading can report a narrowed plan, so the unnarrowed plan at $1000 is a reading.

`_fold_eligible_tranches` narrows by price alone: at a market price of 100 it returned all five, and at 135 it returned the two tranches above that price. `_fold_discharge_order` returned the five in `ref` order, highest first. The same five rows in `created_ts` order, newest first, are the reverse of that, so "most recent" is not the order the queue discharges in.

`_spread_wire_usd_over_fold_queue` handed each of the five the same $10 share out of $50. The share is the dollars divided by the tranche count, it is fed by wire income rather than by a fold, and it reaches every standing tranche.

`fold_ladder_prices` at `stack_math` takes `levels`, which counts prices on a ladder. `fold_ladder_prices(100.0, 4)` returned four prices and decided no size and no position.

Nine calls that refused a counted fold

Each live fold path was asked for a count or a target by keyword.

The call	What it did
`_plan_fold_consumption(eligible, cap, count=2)`	`TypeError`, unexpected keyword argument
`_plan_fold_consumption(eligible, cap, fold_target_count=2)`	`TypeError`, unexpected keyword argument
`_fold_eligible_tranches(100.0, 1.0, count=2)`	`TypeError`, unexpected keyword argument
`_fold_eligible_tranches(100.0, 1.0, fold_target='x_buy')`	`TypeError`, unexpected keyword argument
`_fold_discharge_order(count=2)`	`TypeError`, unexpected keyword argument

The call	What it did
`_fold_discharge_order(most_recent_first=True)`	`TypeError`, unexpected keyword argument
`_spread_wire_usd_over_fold_queue(30.0, [], count=2)`	`TypeError`, unexpected keyword argument
`apply_profit_fold(..., fold_target_count=2)`	`TypeError`, unexpected keyword argument
`apply_profit_fold(..., target='x_buy')`	`TypeError`, a word compared against a number

The `target` parameter of `apply_profit_fold` is a dollar balance, not one of this group's three words, and `profit_fold` records in its own module docstring that no module imports it.

A five-key store opening on three rows

`SettingsManager._apply_dict` copies the whole `profit_folding` dict off the file, so a store written before the removal still carries its `fold_target` and `fold_target_count` entries and still opens. The page reads back the three keys it has rows for, and the next Save writes the group without the other two.

	before the removal	after the removal
the dialog built	yes	yes
control specs in the payload	67	63
control specs on the Profit Folding tab	9	5
control specs in the `Profit Folding Target` group	4	0
a `Profit Folding Target` group is drawn	yes	no
an `Upward Distribution Target` group is drawn	yes	yes
groups declared on the surface	12	11
rows the `profit_folding` group declares	5	3
`ProfitFoldingSettings` fields	5	3
keys a legacy store's group holds on opening	5	5
keys the group holds after one Save	5	3
warning boxes raised	0	0
`active` read back off a legacy store	`False`	`False`
`distribute_target`	`x_sell`	`x_sell`
`distribute_target_count`	`7`	`7`
`username` beside the group	`u18_driver`	`u18_driver`
`bot_visibility` beside the group	`internal`	`internal`

Every one of the three remaining siblings reads back at its stored value and survives a Save that changed nothing. No holder named `_fold_all`, `_fold_x`, `_fold_x_count` or `_fold_recent` exists on the dialog any more, while `_dist_x_count` still does.

The sentences both rows leave behind

Each sentence below stands in an earlier section and no longer describes the page. The earlier text stays where it is.

Earlier sentence	What the page does now
"One master checkbox and three groups of radio buttons, eleven controls in all."	One master checkbox and one group of radio buttons, five controls in all.
"Profit Folding Target - Chooses which buy positions a fold reaches", the two sentences and the code block under it	The page carries no `Profit Folding Target` group.
"Three radio buttons and the count box the middle one gates."	Neither the radios nor the count box exists, so nothing is left to gate.
"Fold to X# of buy positions - Sets that counted number. Key `fold_target_count`, 1 to 100, at 5."	The page sets no count, and one dollar cap bounds how many tranches a fold reaches.
"Save collapses the two target groups into one dictionary."	Save collapses one target group, the sell side.
"Four of the six keys then fail a second time further down: the two target names and their two counts are four of the eleven the bot factory strips."	Two of the three keys the page still stores are on the strip list: `distribute_target` and `distribute_target_count`.
`"| Profit Folding | 11 | the whole group | the whole group |"` in What each page restores now	Five controls, and three keys in the group.
"| Profit Folding | Distribution Mode, Profit Folding Target, Fold to X# of buy positions, Upward Distribution Target, Distribute to X# of sell positions |"	Two rows on this page, Upward Distribution Target and Distribute to X# of sell positions.

The master switch above the removed group gated four rows. It gates two now, and the two are Upward Distribution Target and Distribute to X# of sell positions.

2026-09-12 - Upward Distribution Target takes the sell-side group off the page

The Profit Folding page carried one master checkbox and one group of radio buttons. The `Upward Distribution Target` group is removed, all four controls together: the three radio buttons and the count box inside them. The group offered a choice of which sell positions a distribution reaches and a number for the middle choice. No distribution in the engine offers that choice, and no distribution counts the positions it reaches.

Three files declared the group and every declaration is gone.

File	What it declared
`src/core/settings.py`	`DistributeTarget`, and the `distribute_target` and `distribute_target_count` fields on `ProfitFoldingSettings`
`src/gui/settings_dialog.py`	`DIST_TARGET_BUTTONS`, the `QGroupBox` with its three `QRadioButtons` and its `QSpinBox`, the `QRadioButton` import, the two `_stored_groups` rows, and `_picked` with `_show_picked`
`src/gui/main_tabs/settings_dialog_surface.py`	`DIST_GROUP_TITLE`, `DIST_ALL`, `DIST_X`, `DIST_RECENT`, `DIST_TARGET_BUTTONS`, `FOLD_COUNT_DEFAULT`, the four control specs, the `GROUPS` entry, the Profit Folding layout step, the two `_folding_rows` entries, and `_picked` with `_show_picked`

`_picked` and `_show_picked` held one caller each in each file, the `distribute_target` row, so both pairs go with it.

The group's four controls on the sell side

The control	What it stored
the radio `dist_all`, "Distribute to ALL sell positions", checked at build	`profit_folding.distribute_target` as `all_sell`
the radio `dist_x`, "Distribute to X# of sell positions:"	the same key as `x_sell`
the radio `dist_recent`, "Distribute to most recent sell positions"	the same key as `most_recent_sell`
the spin `dist_x_count`, 1 to 100, at 5, with no label and no text	`profit_folding.distribute_target_count`

The spin carried the group title of the three radios and shared one layout row with `dist_x`. Every word beside it on screen belonged to that radio, so the count had no words of its own and no meaning apart from the choice it sized. The two rows are one decision, so one change takes both off together.

What the three sell-side words chose

The bare name `distribute_target` appeared at 16 sites across the 1077 tracked files outside `dist/` and `docs-archive/`, and `distribute_target_count` at 13. Three of each declared these rows, five of the first and three of the second belong to the bot wizard's own copy, one of each is the strip list, and the rest are this manual quoting the keys. Six read shapes were spelled for each name and every one returned nothing.

Read shape	Sites
`getattr(x, "distribute_target", ...)`	0
`getattr(bot.config, "distribute_target", ...)`	0
`x["distribute_target"]`	0
`get_nested(..., "distribute_target")`	0
`get(..., "distribute_target")`	0
`cfg.distribute_target`	0

The same six shapes, spelled for the sibling key `profit_folding_active`, return 17 sites over the same files, so the zeros read the repository rather than a blind search. A planted file spelling all six shapes for `distribute_target` grew every one of the six counts.

`distribute_target` and `distribute_target_count` both stay in `_DEPRECATED_KWARGS` at `src/trading/container/config.py, in _BOT_CONFIG_EXTRACTOR_ONLY_FIELDS`, so a `bot_state.json` still carrying either restores without a `TypeError`. `BotConfig` declares neither name as a field, and `_sanitize_deprecated_kwargs` drops both before `BotConfig.__init__`. A `make_bot_config` call asking for all four of `distribute_target`, `distribute_target_count`, `profit_folding_active` and `scrum_fold_pct` kept the last two and built a config carrying neither removed name.

One accumulator bounds a distribution, and no count does

`src/trading/scrumming/tick_phases.py, in _tick_distribute` is the live route, and `src/trading/scrumming_bot.py, in tick` calls it. It sells `_dist_accumulator` units of the asset, takes the lesser of that figure and the venue balance, then walks `_main_lots` sorted by `initial_buy_price` and opens one fold tranche per lot it consumes.

Driven over five held lots, on a config carrying `distribute_target = 'x_sell'` and `distribute_target_count = 2`:

What was asked	Lots the distribution reached
an accumulator of 5.0 units	5 of 5
an accumulator of 2.5 units	2 of 5
an accumulator of 1.0 units	1 of 5
an accumulator of 0.0 units	0 of 5, and no sell placed

The stored count of 2 narrowed nothing. The three smaller accumulators show the reading can report a narrowed walk, so the unnarrowed walk at 5.0 units is a reading. The venue balance is the only other bound: at a balance of 2.0 units against an accumulator of 5.0 the sell was 2.0 units and the walk reached 2 of 5.

The five lots were built with buy-price order and age order deliberately opposite. The walk consumed `['p150_oldest', 'p140']`, which is buy price highest first, and the newest-first order is the exact reverse of it. "Most recent" is not the order a distribution takes.

One DIST sell leaves one tranche behind. `_bound_new_fold_tranches` merges the slice the build loop appended back to a single record, so the number of positions a distribution opens is 1 at an accumulator of 5.0 units and 1 at 3.0.

Current reading. The distribution no longer sells one accumulator. Each fold that buys more units than its scrum sold queues one set: those extra units beside the price the fold paid. A sale takes only the sets whose price, raised by the Minimum Opposing Trade Distance, the market has reached, and the sold units come off those sets alone. The venue bound reads the free units in the balance the venue reports. With Stack Mode on, each set gets one ladder, never one per tick, and a fold whose own ladder opens queues no set.

`src/trading/scrumming/distribution.py` — `dist_sets_clearing`

```
  return [one for one in sets if dist_floor_price(one, otd_pct) <= price]
```

Nine sell-side calls that refused a count

Each sell-side path was asked for a count or a target by keyword.

The call	What it did
`_tick_distribute(..., count=2)`	`TypeError`, unexpected keyword argument
`_tick_distribute(..., distribute_target='x_sell')`	`TypeError`, unexpected keyword argument
`_tick_distribute(..., distribute_target_count=2)`	`TypeError`, unexpected keyword argument
`_execute_sell(1.0, 200.0, None, count=2)`	`TypeError`, unexpected keyword argument
`_settled_sale_proceeds(1.0, 200.0, label='DIST', count=2)`	`TypeError`, unexpected keyword argument
`_bound_new_fold_tranches(0, count=2)`	`TypeError`, unexpected keyword argument
`_apply_scrum_fold_pct(0, 100.0, 1.0, distribute_target='x_sell')`	`TypeError`, unexpected keyword argument

The call	What it did
`scrum_ladder_prices(100.0, 'x_sell')`	`ValueError`, levels must be an int
`split_scrum_into_tranches(..., spacing_mode='most_recent_sell')`	`ValueError`, unknown spacing_mode

`scrum_ladder_prices(100.0, 3)` returned `[101.0, 104.0, 109.0]`, three prices on an upward ladder, and decided no position and no set. `SPACING_MODES` in `src/trading/stack_math.py` holds `quadratic`, `fibonacci`, `linear` and `exponential`, and none of this group's three words.

The count default that kept the constant alive

`FOLD_COUNT_DEFAULT` at `src/gui/main_tabs/settings_dialog_surface.py` had exactly one reader, the `distribute_target_count` row. That row is gone, so the constant goes with it. The bare name fell from 3 sites to 1, and the one left is this manual.

Name	Sites before	Sites after
`distribute_target`	16	13
`distribute_target_count`	13	10
`DistributeTarget`	2	0
`DIST_TARGET_BUTTONS`	6	0
`DIST_GROUP_TITLE`	7	0
`FOLD_COUNT_DEFAULT`	3	1
`dist_x_count`	8	4

Every site still standing is either this manual or the bot wizard's own copy of the same three words, which carries its own controls and its own count of 1 to 50.

A six-key store opening on one row

`SettingsManager._apply_dict` copies the whole `profit_folding` dict off the file, so a store written before the removal still carries all six keys the group ever held and still opens. The page reads back the one key it has a row for, and the next Save writes the group without the other five.

	before the removal	after the removal
the dialog built	yes	yes
control specs in the payload	63	59
control specs on the Profit Folding tab	5	1
control specs in the `Upward Distribution Target` group	4	0
an `Upward Distribution Target` group is drawn	yes	no
groups declared on the surface	11	10
rows the `profit_folding` group declares	3	1
`ProfitFoldingSettings` fields	3	1
keys a legacy store's group holds on opening	6	6
keys the group holds after one Save	3	1
warning boxes raised	0	0

	before the removal	after the removal
`active` read back off a legacy store	`False`	`False`
`username` beside the group	`u19_driver`	`u19_driver`
`bot_visibility` beside the group	`internal`	`internal`
the Qt dialog built on a legacy store	yes	yes
a `_dist_x_count` holder on the Qt dialog	yes	no

`active` is the one remaining sibling and it reads back at its stored value on every run, before and after, and survives a Save that changed nothing. The three rows the dialog keeps beside the group, `username`, `bot_visibility` and the stored group itself, all read back unchanged.

With the group present, all three stored words round-tripped. Each opened the page on its own button, each was read back as itself, and each survived a Save that changed nothing.

Stored `distribute_target`	**The radio ticked**	**The store after a Save with no edit**
`all_sell`	`dist_all`	`all_sell`
`x_sell`	`dist_x`	`x_sell`
`most_recent_sell`	`dist_recent`	`most_recent_sell`

What the Profit Folding page holds now

One checkbox, `Profit Folding / Upward Distribution Active`, and nothing else. It is the page's only control and the only `profit_folding` key the dialog stores. The Settings dialog draws no radio button on any tab any more: the three removed here were the last three `RADIO` control specs on the surface.

The sentences the sell side leaves behind

Each sentence below stands in an earlier section and no longer describes the page. The earlier text stays where it is.

Earlier sentence	What the page does now
"Upward Distribution Target - The same three choices on the sell side. Key `distribute_target`, at all sell positions."	The page carries no `Upward Distribution Target` group.
"The sell-side group is built the same way as the buy-side group above, with its own count box at 5 and its own all-sell button checked at build."	Neither group exists.
"Distribute to X# of sell positions - Sets the sell-side count. Key `distribute_target_count`, 1 to 100, at 5."	The page sets no count, and the accumulated asset bounds how many lots a distribution reaches.
"Save collapses the two target groups into one dictionary."	Save collapses no target group. It writes one key, `active`.
"Save collapses one target group, the sell side."	Save collapses none.
"Two of the three keys the page still stores are on the strip list: `distribute_target` and `distribute_target_count`."	The page stores one key and it is not on the strip list.
"`FOLD_COUNT_DEFAULT` stays, because the sell-side count row reads it."	The sell-side count row is gone and so is the constant.

Earlier sentence	What the page does now
"The master switch above the removed group gated four rows. It gates two now, and the two are Upward Distribution Target and Distribute to X# of sell positions."	The master switch gates no row.
"One master checkbox and three groups of radio buttons, eleven controls in all."	One master checkbox, one control in all.

The master switch is the last row on the page, and its own verdict was that it waits on the wizard's folding page. Nothing on this page gates anything now.

2026-09-12 - The phantom master box becomes a new bot's phantom default

The Phantom Bots page's master checkbox now stores a boolean and the bot creation wizard's own phantom box opens on it. Driven on the React build, which is what the running application draws. The home was redirected into a scratch directory before the settings module was imported, so the settings directory and the log root bound under that directory. No stored setting of the running install was read and no venue was contacted.

The Settings master box and the wizard's phantom box

	The Settings row	The wizard box
Where, Qt	`src/gui/settings_dialog.py, in _create_phantom_tab`	`src/gui/bot_wizard.py, in PhantomConfigPage.__init__`
Where, React	`src/gui/main_tabs/settings_dialog_surface.py, in CONTROL_SPECS`	`src/gui/main_tabs/bot_wizard_surface.py, in _apply_stored_phantom_enable`
Text	Enable Phantom Bots for Scrumming	Enable Phantom Bots
Choices	on, off	on, off
Opens at	on	the stored default, off when no bag arrives
What it writes	the store key `default_enable_phantoms`	the config key `enable_phantoms`

The two agree on meaning and on both choices. They disagree on the name, and the names are kept apart on purpose.

The two phantom names that share a word

The page's own name and a bot's own name are two settings, not one. The store accepts the first and refuses the second, driven:

```
store accepts key default_enable_phantoms
store refuses key enable_phantoms   -> KeyError 'Unknown setting: enable_phantoms'
store refuses key phantoms_enabled  -> KeyError 'Unknown setting: phantoms_enabled'
```

Set	Occurrences in `src`	What they are
the application setting	6	the `AppSettings` field, the Qt dialog's row, the same row on the Qt-free surface, the two wizard lookups
a bot's own config field	27	the constructor argument, the restore read, the bot's own flag, the live toggle, the wizard write and the live settings box

Set	Occurrences in `src`	What they are
a bot's own saved record	10	the key the container writes per bot and the restore path reads back

Counted with `git grep -ow` over the 1081 files the index reports. Three files hold a name from both sets and each is a crossing point: the store's declaration names the bot key in its comment, and the two wizard files write the bot key from the application default.

Every reader of the stored phantom default

Five sites name the application key. Two of them read it outside the Settings dialog.

Site	What it does with the default
`src/core/settings.py, in AppSettings`	declares `default_enable_phantoms: bool = True`
`src/gui/settings_dialog.py, in _stored_rows`	the Qt row, read on Save and written on load
`src/gui/main_tabs/settings_dialog_surface.py, in PERSISTED_ROWS`	the same row on the Qt-free surface
`src/gui/bot_wizard.py, in PhantomConfigPage.__init__`	the Qt wizard's phantom box opens here
`src/gui/main_tabs/bot_wizard_surface.py, in _apply_stored_phantom_enable`	the Qt-free wizard's box opens here

Before this section there was no reader at all, and no store key to read. The dialog saved 14 groups and none of them carried the box; it saves 15 now.

Where the stored default reaches a phantom bot

Diagram: flowchart, left to right

- Settings row — default_enable_phantoms -> **SettingsManager — settings.toml**
- SettingsManager — settings.toml -> **C"]**
- main_window._create_bot — get_all(-> **bot_wizard.py, in PhantomConfigPage.__init__ — Enable Phantom Bots**
- bot_wizard.py, in PhantomConfigPage.__init__ — Enable Phantom Bots -> **PhantomConfigPage.get_config — enable_phantoms**
- PhantomConfigPage.get_config — enable_phantoms -> **F"]**
- main_window.py, in _create_bot — ScrummingBot(enable_phantoms= -> **scrumming_bot.py, in ScrummingBot.__init__ — _phantoms_enabled**
- scrumming_bot.py, in ScrummingBot.__init__ — _phantoms_enabled -> **tick_phases.py, in _tick_initialise — create_phantom_set**
- tick_phases.py, in _tick_initialise — create_phantom_set -> **main_window.py, in _refresh_dashboard — the Comp field**

The constructor argument at `src/trading/scrumming_bot.py, in ScrummingBot.__init__` is where the flag stops being this setting. From there the bot owns it, and the Comp field on the TA Indicator Voter Panel is the last reader. That field already refuses a phantom timeframe at or below the parent's own rank, at `src/gui/main_window.py, in _refresh_dashboard`, so the higher-timeframe rule is not new here.

The round trip on both phantom choices

Stored	React box on reopen	React wizard box	Qt box on reopen	Qt wizard box	The config a new bot reads
off	off	off	off	off	off
on	on	on	on	on	on

Two different stored flags gave two different readings on every column, so the reading discriminates. Driven before the change with the box ticked, both wizards answered off, which is the reading this section repairs.

What a new bot opens with, and what a restored bot keeps

Driven through the real restore path with the application default set on.

The saved record	What the restored bot holds
its own flag off	off
its own flag on	on
no flag at all	on, which is the constructor's own default

A restored bot reads its own saved record at `src/trading/container/restore.py, in restore_bots_from_state` and reads no application setting. A record holding off stayed off while the store held on, so the application default cannot reach a bot already on disk.

The sentences this wiring leaves standing

Earlier sentence	What the code does now
"This is the only control on the page that starts on. Save never reads it, so the page stores nothing under the name it shows."	It still opens on. Save reads it and stores it under `default_enable_phantoms`, so the name on screen and the name in the store differ.
"These three controls carry the same gap the TA Indicators page carries. Save reads none of them, the load path restores none, and the schema refuses a key it does not declare rather than storing it somewhere unread."	Save reads the master box and the load path restores it. The other two controls on the page still carry the gap.
"Enable Phantom Bots for Scrumming - Turns the phantom overrides on for a new bot."	Unchanged, and true for the first time: the wizard's own box opens on this row.

The wizard request this wiring still waits on

The Electron shell's wizard request carries no stored defaults, so the Qt-free wizard opens its box at the build figure whatever the store holds.

Absence	What was measured
the shell's wizard request carries no stored defaults	`src/gui/web/bot_wizard.js` names `defaults` no times, and `src/gui/main_tabs/bot_wizard_surface.py, in view_model` reads it out of the request. The lookup added here fires on any call that carries a bag, and the shipped call carries none
the bridge hands the request through unchanged	`src/core/desktop_bridge.py, in build_registry` maps the wizard method straight to the surface, so nothing on the way in adds the stored settings

This absence is shared with every other stored default the wizard opens on, so it belongs to the wizard request rather than to this row.

Rows 34 and 35 stay for their own units

Row	Why it did not come with this one
34, Default Phantom Timeframes	Its eleven boxes are not a declared control on either surface, so the one list that carries Save and load cannot hold them. The rule for this group turns eleven boxes into one selection above the parent's own timeframe, which is a change of control, not a change of wiring
35, Lock duration (candles)	It is a declared control and would persist through the same one line, but its destination is the wizard's own lock box, which is a second lookup this row does not make

Measured on the page's declared controls: the Phantom tab carries two of them, the master box and the lock spin box, and the timeframe grid is neither.

2026-09-12 - The timeframe grid becomes one phantom selection

The Phantom Bots page's eleven tick boxes are now one drop-down, the page stores the name it shows, and the bot creation wizard opens its own phantom box on that name. A name at or below the bot's own TA Timeframe, or one the venue does not offer, is refused on the wizard page rather than dropped in silence. Driven on the Qt dialog, the React host dialog and the shared surface. The home was redirected into a scratch directory before the settings module was imported, so the settings directory and the log root bound under that directory. No stored setting of the running install was read and no venue was contacted.

What the page offered before and what it offers now

	Before	Now
Control	eleven tick boxes in one row	one drop-down
Choices	eleven, five ticked at build	eleven, one selected at build
Opens at	5m, 15m, 1h, 4h and 1d	1d
Declared as a control	no	yes, `phantom_timeframe`
In the list that carries Save and load	no	yes
Store key	none, the store refused four candidate names	`default_phantom_timeframe`

The selection round-tripping through the store

Driven on the shared surface and on both dialog hosts against one scratch store.

```
he picks               : 12h
save reported          : {'saved': 16, 'failed': [], 'closed': True}
store fields changed   : ['default_phantom_timeframe', 'ta_indicator_weights']
stored value           : '12h'
read back off disk     : '12h'
a fresh dialog shows   : 12h
pressing Save again keeps it: '12h'
```

The same run before the change reported one changed field, the indicator weight bag, and the store refused every candidate name for this row.

```
refuses phantom_timeframes         -> KeyError 'Unknown setting: phantom_timeframes'
refuses default_phantom_timeframes -> KeyError 'Unknown setting: default_phantom_timeframes'
refuses default_phantom_timeframe  -> KeyError 'Unknown setting: default_phantom_timeframe'
refuses phantom_timeframe          -> KeyError 'Unknown setting: phantom_timeframe'
```

Why the daily chart is the opening choice

A phantom must sit above the bot's own TA Timeframe, and the wizard's timeframe drop-down opens on the hourly chart. Of the five entries above it, only one is offered by every venue the application knows.

Timeframe	Above the hourly chart	Offered by coinbase	Offered by kraken
2h	yes	yes	no
4h	yes	no	yes
6h	yes	yes	no
12h	yes	no	no
1d	yes	yes	yes
1w	yes	no	yes

Measured against the venue lists the application holds.

```
   coinbase: offers ['1m', '5m', '15m', '30m', '1h', '2h', '6h', '1d']
     kraken: offers ['1m', '5m', '15m', '30m', '1h', '4h', '1d', '1w']
    binance: offers ['1m', '5m', '15m', '30m', '1h', '2h', '4h', '6h', '12h', '1d', '1w']
```

The store declares the opening choice and the page reads it from there, so the schema and the screen cannot drift apart.

`src/core/settings.py` — the one declaration of the opening choice

```
# The one phantom timeframe a new bot starts with. 1d is the only entry
# above the wizard's 1h ta_timeframe default that every venue offers;
# coinbase has no 4h, 12h or 1w, and kraken no 2h or 6h.
default_phantom_timeframe: str = "1d"
```

Where the stored selection reaches a phantom bot

Diagram: graph, left to right

- Settings row — default_phantom_timeframe -> **SettingsManager — settings.toml**
- SettingsManager — settings.toml -> **main_window.py, in _create_bot — settings.get_all**
- main_window.py, in _create_bot — settings.get_all -> **bot_wizard.py — set_parent_timeframe**
- bot_wizard.py — set_parent_timeframe -> **PhantomConfigPage.get_config — phantom_timeframes**
- PhantomConfigPage.get_config — phantom_timeframes -> **F"]**
- main_window.py, in _create_bot — ScrummingBot(phantom_timeframes= -> **tick_phases.py, in _tick_initialise — create_phantom_set**

- tick_phases.py, in _tick_initialise — create_phantom_set -> **main_window.py, in _refresh_dashboard — the Comp field**

The Comp field rank-weights the parent's Net with every phantom summary above the parent's own rank, and skips the rest at `src/gui/main_window.py, in _refresh_dashboard`. One selection above the parent means one phantom reaches that field and none is skipped.

The refusal the wizard page shows

Both refusal reasons were driven on the shared surface and on the Qt page.

```
  stored  1d parent  1h -> selection ['1d'] refusal ''
  stored  4h parent  1h -> selection []     refusal 'phantom_not_offered'
      the box says: Timeframe 4h: REFUSED, coinbase does not offer it
  stored  5m parent  1h -> selection []     refusal 'phantom_not_higher'
      the box says: Timeframe 5m: REFUSED, a phantom must sit above the bot's own TA Timeframe of 1h
  stored  1d parent  1d -> selection []     refusal 'phantom_not_higher'
      the box says: Timeframe 1d: REFUSED, a phantom must sit above the bot's own TA Timeframe of 1d
```

A box the operator has already ticked keeps his choice, so the stored name seeds a page he has not touched and never overwrites a page he has.

What a bot already on disk does

A running bot's phantom set never came from this page and still does not.

Read	What it answers
`src/trading/bot_container.py, in get_full_state`	writes the enable flag into the bot's record and writes no timeframe list
`src/trading/container/restore.py, in restore_bots_from_state`	builds the bot without a timeframe argument
`src/trading/scrumming_bot.py, in ScrummingBot.__init__`	the bot therefore takes its class default

Driven on a bot built the restore way, before and after this change, against a parent on the hourly chart.

```
  built the restore way      : ['5m', '15m', '30m', '1h', '1d']
  class default              : ['5m', '15m', '30m', '1h', '4h', '1d']
  survives the Comp skip     : ['1d']
  silently skipped           : ['5m', '15m', '30m', '1h']
```

Identical on both sides of the change. Four of the five a restored bot runs sit at or below the hourly parent and never reach the Comp field, and the sixth is dropped because coinbase has no four-hour chart. That belongs to row 131, which owns a bot's own phantom selection.

The sentences this change leaves standing

Earlier sentence	What the code does now
"Default Phantom Timeframes - Chooses which charts a phantom watches."	One chart, not several. The name it shows is the chart the one phantom watches.
"Per-bot key `phantom_timeframes`, eleven boxes, five of them checked at build."	Eleven names in one drop-down, one selected at build, and the page's own store key is `default_phantom_timeframe`.
"One box per timeframe in a single row. The build decides which five open checked, and those five are the ones the figure shows."	One drop-down in that row. The figure shows the eleven boxes the page no longer draws.
"These three controls carry the same gap the TA Indicators page carries. Save reads none of them, the load path restores none."	Save reads two of the three and the load path restores both. The lock spin box still carries the gap.
"The one spin box on the page, inside its own group. It opens at 2 and Save reads it no more than it reads the other two."	It still opens at 2 and Save still does not read it. The other two are now read.

What this row still waits on

Absence	What was measured
the wizard still draws eleven boxes	`src/gui/bot_wizard.py, in PhantomConfigPage.__init__` builds eleven tick boxes, so the operator can still tick several. One selection is seeded and not enforced there; the wizard's own control belongs to the bot's phantom rows
the figure on this page shows the old grid	`p39-i0.png` was captured from the eleven boxes and has not been retaken
the rank order is written twice	`src/trading/phantom_balance.py, in TIMEFRAME_ORDER` and `src/gui/main_tabs/bot_wizard_surface.py, in PHANTOM_TIMEFRAMES` hold the same eleven names in the same order, and the Qt-free surface takes no import, so nothing reconciles them. Measured identical today

2026-09-12 - Every retired setting name says what it is

Eight settings came off these pages, and their names did not all leave the tree with them. A name sitting in a comment with no explanation reads as a setting, so each one that stays now says what it is.

Where the retired names still sit

Counted over the git index, matches rather than lines, in both the code spelling and the words the page shows.

the name	matches	where they are
`position_distance_pct`	9	this page 8, the strip list 1
`increment_style`	21	this page 15, the bot config 2, the restore path 1, a debug report 1
`default_position_count`	9	this page only
`position_count`	19	this page 14, the strip list 1, a creation log line 1, a census note 1, a stock helper 2
`fold_mode`	10	this page 4, the two wizard files 5, the strip list 1

the name	matches	where they are
`fold_target`	31	this page 24, the two wizard files 5, the strip list 1
`fold_target_count`	23	this page 20, the two wizard files 2, the strip list 1
`distribute_target`	34	this page 27, the two wizard files 5, the strip list 1
`distribute_target_count`	21	this page 18, the two wizard files 2, the strip list 1
`profit_fold_pct`	3	this page 1, a wizard docstring 1, the strip list 1
`upward_distribution`	1	a wizard docstring, and nowhere else

The settings store and both settings dialog modules hold none of them. Thirteen camelCase spellings were searched beside the snake_case ones and all read zero, so no shipped JavaScript names a retired setting.

What emptying the strip list does

A record shaped like `bot_state.json` was built carrying thirty config keys, of which eleven are the retired names, and driven through all three build routes with the strip list as shipped and then emptied.

```
the list as shipped, 11 names
  make_bot_config(**stored)             OK  symbol='A15/USD'
  bot_config_kwargs then make_bot_config OK  kwargs=18 retired leaked=0
  restore_bots_from_state               OK  restored=1 skips=0

the list emptied
  make_bot_config(**stored)             TypeError: unexpected keyword
                                        argument 'bulk_trading'
  bot_config_kwargs then make_bot_config OK  kwargs=18 retired leaked=0
  restore_bots_from_state               OK  restored=1 skips=0
```

Dropping the eleven one at a time raises that error naming each key in turn, eleven times out of eleven, so every entry is load-bearing on that route. The launch restore names each kwarg it wants one at a time rather than passing the stored config whole, so the restore arm does not move. A retired key that did reach the factory is caught at `src/trading/container/restore.py, in restore_bots_from_state`, which skips the bot and leaves its record on disk.

What the declaration says now

The eleven entries stay. The comment above them states why.

`src/trading/container/config.py` — the strip list, with its own description

```
#: A retirement record, not a setting set. Each name is a setting removed from
#: the product, kept so a config stored before its removal still builds;
#: `_sanitize_deprecated_kwargs` drops each before `BotConfig.__init__`.
#: Add no name. Only `bulk_trading` is read, by `bot_config_kwargs`.
```

The names taken out, and the ones left standing

occurrence	what happened to it
the parameter page's `get_config` docstring	removed. It listed four retired keys the method does not emit, and carried two version names. `upward_distribution` had no other occurrence in the tree and now has none
the restore path's Grid-legacy comment	it read "nothing here reads them" above three lines that read three of them. It now states what the hand-written kwarg list does
the stack-mode fallback comment	it named no key. It now names `bulk_trading`
the eleven strip-list entries	kept, for the measured reason above
the two wizard files' folding groups	kept. The wizard's folding page is unreachable, and the rows that own those files are still open
the creation log line's `position_count` read	kept. It is the row that owns the line that always prints zero
the handoff files, the archive chronicle, the census note, the debug report	kept. Each is a dated record of an earlier state, not a description of a control

2026-09-12 - The Logging page comes off the Settings dialog

The Logging page carried two checkboxes and four periodicity boxes. All six are removed, and the tab goes with them, because every control on the page wrote into one stored group that nothing outside the dialog ever read.

Each of the three names on the page was judged on its own evidence. Two of them name logging the platform already writes under a different name. The third names a summary no code in the tree writes at all.

Three files declared the page and every declaration is gone.

File	What it declared
`src/core/settings.py`	`LogPeriodicity`, the whole `DataLoggingSettings` class, and the `data_logging` field on `AppSettings`
`src/gui/settings_dialog.py`	`PERIOD_BUTTONS`, `DEFAULT_PERIODS`, `_create_logging_tab` with its six check boxes, its `addTab` step, `_ticked_periods`, `_show_periods`, and the three `_stored_groups` rows
`src/gui/main_tabs/settings_dialog_surface.py`	`LOGGING_TAB`, `LOGGING_HEADING`, `LOGGING_GROUP_KEY`, `LOGGING_FLAG_DEFAULT`, `PERIOD_CONTROLS`, `DEFAULT_PERIODS`, the six control specs, the tab title, the layout page, the heading entry, `_logging_rows`, `_logging_group`, and the save-group entry

The six controls the page held

The control	What it stored
the box `ta_logging`, "Log TA signal samples with all values and timestamps", checked at build	`data_logging.ta_signal_logging`
the box `highlight_trades`, "Highlight entries near Scrumming Bot trades", checked at build	`data_logging.highlight_trade_proximity`
the box `log_24h`, "24 Hours", checked at build	one entry in `data_logging.active_periodicities`
the box `log_1w`, "1 Week", checked at build	the same list
the box `log_1m`, "1 Month", clear at build	the same list
the box `log_1y`, "1 Year", clear at build	the same list

The four periodicity boxes were one setting spread over four controls. Save walked them in page order and wrote the ticked ones as a list.

Nothing outside the dialog read the group

Each of the four stored names occurs in exactly three files, and all three are the declaration plus the two dialog surfaces. The scan walked every Python and JavaScript file under the source, tooling and desktop trees, plus the entry point.

```
data_logging               3 files
ta_signal_logging          3 files
highlight_trade_proximity  3 files
active_periodicities       3 files
    src/core/settings.py
    src/gui/settings_dialog.py
    src/gui/main_tabs/settings_dialog_surface.py
```

Every writer of every log file the platform keeps sits in one module, `src/core/logging_engine.py`, and that file holds no occurrence of any of the four names. The same count over the word "timestamp" in the same file returns three, so the zeros read the file rather than a blind search.

Each indicator reading is already written with its value and its time

The voting panel snapshot writes one record for every fired trade, carrying the whole vote that drove it. Each vote is one indicator's own reading, and it carries its own clock value beside the record's.

`src/trading/indicators/types.py` — one indicator's output

```
@dataclass
class Signal:
    """One indicator's output at a point in time."""

    indicator: str
    timeframe: str
    direction: SignalDirection
    confidence: float  # 0.0 - 1.0
    weight: float = 1.0
    details: dict = field(default_factory=dict)
    timestamp: float = field(default_factory=time.time)
```

Driven through the real writer into a scratch directory, one record landed with the reading, the weight, the raw values and two separate times:

```
row timestamp  2026-09-12T17:07:05.855494+00:00
signal in row  {"indicator":"rsi","timeframe":"1h","direction":-1,
                "confidence":0.72,"weight":1.4,
                "details":{"rsi":71.25,"period":14},
                "timestamp":1789232825.8554606,"abstained":false}
```

That is what the removed checkbox described, already written, for every fired trade, with no setting consulted. The operator's own file carries it: his `voting.log` measured 8,078,180 bytes on the day of this change.

Every row in the two decision logs already sits at a fire

The gate log and the voting log are both written once per fired trade, never per tick. Each file therefore holds fire-adjacent rows and nothing else, and each row already names the action that fired it.

`src/core/logging_engine.py` — the voting writer's own contract

```
"""Write one entry with category "voting" to ``voting.log``.

``ScrummingBot._emit_voting_panel_snapshot_at_fire`` reaches this once
per fired trade, not per tick.
"""
```

A flag that marked the lines near a fire would mark every line in both files, so it distinguishes nothing. The record type does declare a `highlight` field, and nothing in the tree sets it and nothing reads it.

No profit and loss figure is summarised over any window

The cascade declares four windows and creates a directory for each one at construction. Only the daily writer has a caller.

`src/core/logging_engine.py` — the four windows, and the three that nothing fills

```
PERIODS = {
    "daily": 1,
    "weekly": 7,
    "monthly": 30,
    "yearly": 365,
}
```

The three builders that would fill the other three directories have no call site anywhere. Parsed from every Python file under the source, tooling and desktop trees, plus the entry point:

What was counted	Result
call sites of the weekly, monthly and yearly builders	0
occurrences of those three names, as a control	9
files under `tests/` naming any of the three	0

The nine occurrences are the three definitions and six mentions inside docstrings, so the zero is a count of calls and not a fact about the file set.

The runtime tree reads the same way. The daily directory holds 66 files and the other three have held none since they were created on 2026-06-09. A driven run reproduced it exactly: one daily file written, and nothing in weekly, monthly or yearly.

The three builders are gone

The cascade now declares one window. `PnLCascade` in `src/core/logging_engine.py` holds a single `PERIODS` key, `daily`, and one writer, `record_daily`. The three builders and the helper that wrote their files are removed.

The cascade makes one directory. A start reaches `daily/` only. It no longer makes `weekly/`, `monthly/` or `yearly/`.

What was counted	Before	After
occurrences of the three builder names in the tree	9	0
windows the cascade declares	4	1
directories the cascade makes at a start	4	1
the daily writer	present	present

The runtime tree keeps every file it holds. The daily directory held 67 files on 2026-09-13, and the other three held none. Nothing was written or deleted under the runtime tree to prove this.

A store carrying the retired group still opens

`SettingsManager._apply_dict` walks the declared fields and takes only the keys a file shares with them, so a stored group with no field behind it is passed over and every other value loads. A file holding the three retired keys beside thirteen other settings was driven through the real store:

	before the removal	after the removal
the dialog built	yes	yes
tabs in the payload	11	10

	before the removal	after the removal
a Logging tab is drawn	yes	no
layout pages in the payload	11	10
control specs in the payload	68	62
control specs on the Logging tab	6	0
check boxes on the built window	30	24
the six retired labels found on that window	6	0
stored groups the dialog persists	4	3
rows the retired group declared	3	0
other settings read back at their stored value	13 of 13	13 of 13
the retired group read off the store	a dict of three keys	`None`
anything raised on load	no	no

The thirteen siblings include the username, the theme, the accent colour, the folding group, the monitor group and the indicator weights, and each one reads back at its stored value. The three remaining groups still round-trip through a Save.

Both columns were driven, on the unchanged tree and on the change, from one store holding the same fifteen keys. The reader that finds none of the six labels on the new window found all six on the old one, so the zero is a fact about the dialog and not about the reader.

What a live trade still writes

Nothing about a live trade changed. The same three writers were driven on the branch and each produced what it produced before: the voting record with its indicator reading and its two times, the gate record with its armed flags and its blocker list, and one daily profit and loss file.

The sentences this section leaves behind

Each sentence below stands in an earlier section and no longer describes the dialog. The earlier text stays where it is.

Earlier sentence	What the dialog does now
"Two checkboxes and four periodicity boxes. Every key below sits inside one stored `data_logging` group."	The dialog has no Logging page, and the store declares no such group.
"Log TA signal samples with all values and timestamps - Writes each indicator reading out with its value and its time."	The voting record writes each indicator reading with its value and its time, for every fired trade, with no setting in front of it.
"The first of the two flags on the page, and the one that decides whether the indicator record is written at all."	No flag decides it. The record is written on every fired trade.
"Highlight entries near Scrumming Bot trades - Marks the log lines that sit close to a fire."	Both decision logs hold one row per fired trade, so every row already sits at a fire.
"The second flag, built the same way and checked at build like the first."	Neither flag exists.
"P/L Log Periodicity - Chooses the windows a profit and loss figure is summarised over."	No window is summarised. The daily file is the only one anything writes.

Earlier sentence	What the dialog does now
"Four separate boxes rather than one control. Save walks them in page order and writes the checked ones into a list."	The four boxes are gone and the list is no longer stored.
"Save collapses all six into one dictionary holding the two flags and the list of active periodicities."	Save writes three groups and none of them is this one.
"No reader outside the dialog reads the group either, which leaves the six controls writing to a key nothing consults."	The six controls are off the page, so nothing writes to the key.
"In development. What the load path should restore for this page has not been settled against the rest of the dialog, so nothing is proposed here."	The page is gone, so nothing restores it.
"The load path restores the whole group: both flags and the list of active periodicities."	No group, and no rows to restore.
"| Logging | Log TA signal samples with all values and timestamps, Highlight entries near Scrumming Bot trades, P/L Log Periodicity |"	No rows on this page, because there is no page.
"Eleven tabs -- User, Exchanges, Trading, Profit Folding, TA Indicators, Phantom Bots, Theme, Logging, Sound, SMS and AI Monitor"	Ten tabs, and Logging is not one of them.

What is still owed

The roll-up itself is absent, and taking the control off the page does not build it. A weekly, monthly or yearly profit and loss file needs something that calls the builder, and no such caller exists. That is a behaviour to build, not a setting to wire, and it belongs to whatever owns the profit and loss evidence layer.

In development.

2026-09-12 - The AI Monitor page reaches the monitor it names

Seven settings, one store group, one carry. The page was driven end to end on the React build, which is what the application draws, and again on the Qt build. Six of the seven arrive at the monitor. The seventh still arrives nowhere.

`src/core/settings.py` - the one declaration behind all seven

```
@dataclass
class AIMonitorSettings:
    """Field defaults for the ``AppSettings.ai_monitor`` group."""

    api_key: str = ""
    interval_hours: float = 4.0
    connect_phrase: str = ""
    confirm_phrase: str = ""
    enabled: bool = False
    auto_handshake: bool = True
    log_feedback: bool = True
```

What the page was asked and what it answered

A store holding an obvious fake key and both phrases was opened, the dialog was built, and every control was read off the running object. All seven opened on the stored figures on both builds. Typing into them and pressing Save wrote all seven back. Reopening a fresh manager read all seven.

```
api_key         wrote the fake key          read the fake key
interval_hours  wrote 1.5                   read 1.5
connect_phrase  wrote u75-connect-phrase    read u75-connect-phrase
confirm_phrase  wrote u75-confirm-phrase    read u75-confirm-phrase
enabled         wrote True                  read True
auto_handshake  wrote False                 read False
log_feedback    wrote False                 read False
```

The key is masked on both builds

One entry on the control list carries the masking, so neither build can lose it without the other losing it too. On the drawn page the key row is a password field, read off the browser twice in one process and `password` both times. On the Qt build the same row reads `EchoMode.Password`.

`src/gui/main_tabs/settings_dialog_surface.py` - the masked row

```
{
    "tab": AI_TAB,
    "group": AI_API_GROUP_TITLE,
    "label": "Anthropic API Key:",
    "name": "ai_api_key",
    "kind": LINE,
    "min_height": SMS_CONTROL_MIN_HEIGHT,
    "echo": "password",
    "placeholder": "sk-ant-api03-...",
}
```

Neither phrase row is masked, which is what the page already says about them.

No provider was contacted to prove any of this

The two phrases and the journal hash were read out of the composed prompt, which is built without a request. The request arm itself was reached with the transport made unimportable, so the arm ran and no socket opened.

```
connect phrase in the prompt  True
confirm phrase in the prompt  True
journal hash in the prompt    True
the key in the prompt         False
handshake arm reached         no transport, no call
with no key                   {'authenticated': False, 'message': 'No API key'}
analysis with no key          {'status': 'disabled'}
```

The key never enters the prompt. It goes in one header field and nowhere else.

A check interval the monitor cannot count is refused

The page holds the figure between half an hour and twenty four hours, so nothing the operator types can leave that range. The store is a file, and a figure that reaches it another way used to go straight through. Zero, a negative and a true flag each made the review due on every dashboard pass; a word and an empty value each raised inside the due test, once per pass, with nothing on screen.

`src/trading/live_monitor.py` - the refusal

```
    @classmethod
    def wait_hours(cls, raw) -> float:
        """Return ``raw`` as the wait ``should_check`` counts, else the default.

        A stored ``interval_hours`` of zero, a negative, a string or None would
        make ``should_check`` answer True on every tick or raise inside it, so
        ``configure_live_monitor`` cannot hand either through.
        """
        hours = float(raw) if isinstance(raw, (int, float)) else 0.0
        if not math.isfinite(hours) or hours <= 0:
            logger.warning(
                "LiveMonitor: check interval %r is not a wait; using %.1fh",
                raw,
                cls.DEFAULT_INTERVAL_HOURS,
            )
            return cls.DEFAULT_INTERVAL_HOURS
        return hours
```

Driven through the real manager, each refused figure now holds four hours and the review waits for it. The log names the figure it refused.

```
interval 'four'  held 4.0  due right after a check  False
interval 0.0     held 4.0  due right after a check  False
interval -3.0    held 4.0  due right after a check  False
interval None    held 4.0  due right after a check  False
interval 1.5     held 1.5  due right after a check  False
interval 1.5     held 1.5  one interval later       True
```

The journal switch now writes a line

The switch is read when an answer arrives. The note it built was the record type the monitor's own journal keeps, and the journal the window holds is a different one that keeps a different entry. Driven with the note the window built, the write produced no file and no line, and the window logged that the note was not written.

```
record handed over     TradeRecord, from live_monitor
journal files          []
lines on disk          0
what it raised         'TradeRecord' object has no attribute 'symbol'
```

`src/gui/main_window.py` - the note as the held journal takes it

```
if ai_cfg.get("log_feedback"):
    try:
        # _journal is reconciliation.TradeJournal, so the note
        # goes in through record_from_trade as a JournalEntry.
        self._journal.record_from_trade(
            bot_id="AI_MONITOR",
            symbol="SYSTEM",
            action="AI_FEEDBACK",
            side="neutral",
            price=0.0,
            quantity=0.0,
            reason=feedback[:500],
        )
    except Exception:
        logger.exception(
            "AI feedback note was not written to " "the journal"
        )
```

The same call now lands one row, and the journal counts it.

```
journal files        ['journal_2026-09-12.jsonl']
on disk              AI_MONITOR SYSTEM AI_FEEDBACK | the answer text
lines on disk        1
entries in memory    1
statistics           {'total_entries': 1, 'actions': {'AI_FEEDBACK': 1}}
```

What the drawn page now does with a figure it refuses

The number box reports whatever the browser read out of it, and an empty box reads as no figure at all. That reached the control, the control refused it, and the refusal left the dialog through the channel the page's console lines arrive on. The figure was kept and nothing said so.

`src/gui/react_settings_dialog.py` - the refused edit

```
try:
    self._holders[name].admit(asked.get("value"))
except Exception as exc:  # noqa: BLE001
    # A cleared number box reports null, which admit refuses; redraw
    # puts the held figure back instead of leaving the box empty.
    logger.warning(
        "Settings page sent %s a value its control refuses: %s",
        name,
        exc,
    )
    self.redraw()
    return
```

Driven on the drawn page, a cleared box is now named in the log and the held figure is kept. A typed phrase and a typed figure both still arrive, which is what proves the channel itself is alive.

```
typed 2.5 into the box        the dialog holds 2.5
typed a phrase into the row   the dialog holds the phrase
cleared the box               the dialog still holds 2.5, the refusal is logged
```

The box itself still draws empty until the next figure is typed, because the drawn controls take their figure once, when the page first builds them. Every control on every page of this dialog takes its figure that way, so a figure the dialog pushes after the page is up reaches the store and not the box. That is one change to how the page builds a control, and it is owed.

In development.

The drawdown share says what it is

The monitor's own journal keeps the deepest fall from peak as a share of peak, and the name said nothing about that. The figure the review line prints and the figure the report file publishes are both unchanged.

`src/trading/live_monitor.py` - the running deepest fall

```
if s["peak"] > 0:
    dd_pct = (s["peak"] - trade.portfolio_value) / s["peak"] * 100
    s["max_dd_pct"] = max(s["max_dd_pct"], dd_pct)
```

Driven across a peak of two hundred and a fall to one hundred, the share reads twenty five at one hundred and fifty, holds at one hundred and eighty, and reads fifty at one hundred. The report's own field keeps the name it published.

```
value 200.0  peak 200.0  share  0.0
value 150.0  peak 200.0  share 25.0
value 180.0  peak 200.0  share 25.0
value 100.0  peak 200.0  share 50.0
report       {'advantage_pct': -50.0, 'max_dd': 50.0}
```

Auto-handshake still reaches nothing, and why it is not wired

The box stores and restores. Nothing reads it. The phrase exchange it describes is what the review already does on its own, every time the connection is not yet proved, and it asks no switch.

`src/trading/live_monitor.py` - the review's own exchange

```
async def analyze(self, portfolio=0.0, passive=0.0, bots=0) -> dict:
    """Send performance snapshot. Auto-handshakes if needed."""
    if not self._enabled:
        return {"status": "disabled"}
    if not self._authenticated:
        hs = await self.handshake()
        if not hs.get("authenticated"):
            return {"status": "auth_failed", **hs}
```

The exchange has one caller in the whole application, and that is the line above. The box's ticked position is therefore the only behaviour there is, and its cleared position asks for an exchange the operator requests,

which nothing can request: Test Handshake requests none. Wiring the box today would make its cleared position a second way of turning the monitor off, which the box above it already does. The box waits on a Test Handshake that performs one.

In development.

2026-09-13 - The credential pair is masked and Username is not

The API Key row and the API Secret row are both hidden while they are typed. Two sections above record the pair as readable and say the change was left to a unit. This is that change. One declaration carries it, and the page reads that declaration for both rows.

`src/gui/main_tabs/settings_dialog_surface.py` - the two specs

```
{
    "label": "API Key:",
    "name": "new_api_key",
    "kind": LINE,
    "echo": "password",
},
{
    "label": "API Secret:",
    "name": "new_api_secret",
    "kind": TEXT_AREA,
    "echo": "password",
},
```

How a multi-line secret is hidden

A single-line box hides its characters by taking a password type. A text area has no type attribute at all, so it cannot. The running build draws the page inside a web view, and that engine offers a style property that hides the characters of any element. The style sheet applies it to a text area carrying the masking mark.

`src/gui/web/settings_dialog.css` - the one rule a text area needs

```
textarea.acervator-settings-dialog[data-part="control"][data-echo="password"] {
  -webkit-text-security: disc;
}
```

The three options and the reading that refused two

Three things were driven on the loaded page. The engine supports the property. A text area reports no type. A single-line box handed a two-line value gives one line back, so drawing the secret as a single line would flatten a pasted PEM and the venue would refuse the key.

```
CSS.supports("-webkit-text-security", "disc")   True
the text area's type attribute                  null
two lines into a single-line box, read back      "...secretsecond-line"
two lines into the text area, read back          "...secret\nsecond-line"
```

Both halves now end in the same state

Read off the rendered page, with the pair beside the passphrase row that was already hidden. All three report the same hiding style.

```
API Key      INPUT     type password   data-echo password   text-security disc
API Secret   TEXTAREA  type null       data-echo password   text-security disc
Passphrase   INPUT     type password   data-echo password   text-security disc
```

The hidden characters measured in the page's own layout

A hidden row draws one repeated mark for every character, so two values of the same length draw to the same size while two of different lengths still differ. The key row was measured across its rendered width and the secret row down its rendered height.

```
                       before     after
key, 200 wide chars    2254 px    986 px
key, 200 thin chars     593 px    986 px
key, 600 thin chars               2937 px
secret, 400 wide        206 px     98 px
secret, 400 thin         62 px     98 px
secret, 8 thin                      58 px
```

What a hidden row still saves

Both rows were typed through the page and both getters the add path reads returned what was typed. The newline inside the secret survived, so the PEM conversion still receives the bytes it converts.

```
the add path reads the key      "u159-not-a-real-key-0000"
the add path reads the secret   "u159-not-a-real-secret\nsecond-line"
a newline survived the row      True
```

The hints above the rows stay readable

Hiding the characters of a value does not hide the placeholder, so the Coinbase key shape and the PEM note still show on an empty row.

```
API Key      placeholder text-security none, colour rgb(117, 117, 117)
API Secret   placeholder text-security none, colour rgb(117, 117, 117)
Passphrase   placeholder text-security none, colour rgb(117, 117, 117)
```

Username is not a credential row and stays readable

The Username row keeps a plain text field. It was typed on the loaded page, read back off the object the save path reads, saved, then read again from a second dialog built over the same store.

```
username row as drawn    INPUT  type text  text-security none
typed on the page        u159-fake-operator
read off the object      u159-fake-operator
store after Save         u159-fake-operator
a second dialog reads    u159-fake-operator
the reloaded page shows  u159-fake-operator
```

What the Qt build still shows

The Qt dialog builds its own widgets and reads neither spec, so both rows there are unchanged and both stay readable. A Qt text edit has no echo mode. The only precedent for hiding one is the init wizard, which paints the text transparent. That also hides the placeholder and leaves the characters selectable, so applying it would trade one visible fault for two. The Qt pair is left as it is.

`src/gui/init_wizard.py` - the Qt precedent, and what it costs

```
self._api_secret.setStyleSheet(
    "color: transparent; background-selection-color: transparent;"
)
```

2026-09-13 - The first-run path keys no vault to a placeholder

A launch on a machine that has never run Acervator no longer writes a name into the store. The startup block reads the stored name, reports a first run in the log, and leaves the entry alone, so nothing claims the operator is called User.

`main.py` - the first-run block, after the change

```
stored_version = settings.get("app_version", "")
current_version = _acervator_version

if not settings.get("username", ""):
    log_manager.info("First run — no stored name, no wizard")
```

The version write that sat beside the name write is not lost. The next block compares the stored version against the running one, and on a fresh store those differ, so the version is written there as it always was.

One declaration builds the phrase

The phrase the credential vault encrypts under is now written once, in the module that holds the encryption. Every site that seals or opens a stored credential calls it, so the four sites cannot disagree about the text.

`src/core/encryption.py` - the declaration and the fallback

```
MASTER_FORMAT = "qat_{username}_vault"
UNNAMED_OPERATOR = "user"


def vault_phrase(username: str) -> str:
    return MASTER_FORMAT.format(username=str(username) or UNNAMED_OPERATOR)
```

The seven callers are the encrypt site on the Exchanges page, the five decrypt sites that open a stored secret, and the drawn page's own phrase builder.

```
src/gui/settings_dialog.py        _add_exchange                seals
src/gui/main_window.py            _connect_exchange_for_bot    opens, the ccxt venues
src/gui/main_window.py            _broker_credential           opens, the broker venues
src/gui/main_window.py            _written_crypto_credential   opens, the hand-written venues
src/gui/widgets/api_tester_tab.py _do_connect                  opens
src/core/encryption.py            default_vault                opens the file vault
src/gui/main_tabs/settings_dialog_surface.py  _master_phrase   the drawn page
```

The fallback fires on a store with no name

Driven on an empty directory, with the home redirected before the first import. The store declares the field, so a read returns the declared empty string and the fallback in each literal never had a chance to fire. The one declaration reads that empty string and substitutes the unnamed operator.

```
stored username            ''
phrase, before the change  'qat__vault'     fallback fired  False
phrase, after the change   'qat_user_vault' fallback fired  True
the drawn page and the declaration agree    True
sealed under the new phrase, opens again    True
```

An existing install opens the same secrets

A store that already carries a name is untouched. The phrase built from a name is the same text before and after, and a secret sealed under the earlier phrase opens under the phrase the changed tree builds.

```
phrase on a named store, before   'qat_FAKEOPERATOR_vault'
phrase on a named store, after    'qat_FAKEOPERATOR_vault'
same phrase                        True
a secret sealed before opens now    True
```

Only a store with no name at all reads a different phrase than it did, and no such store can hold a credential, because the startup write filled the entry before any venue could be added.

The wizard's secret row starts hidden

The install wizard builds its own widgets and reads neither the page spec nor the style sheet the drawn page uses. Its key row masked its value at build while its secret row did not, because the hiding lived only inside the Show credentials handler, which nothing called until the box was ticked. The handler now runs once as the wizard is built, and it carries the sheet as one declaration.

`src/gui/init_wizard.py` - the hidden state and the build seed

```
SECRET_HIDDEN_STYLE = (
    "color: transparent; selection-color: transparent; "
    f"placeholder-text-color: {ds.TEXT_PLACEHOLDER};"
)
...
self._toggle_visibility(self._show_key.isChecked())
```

Read off the rendered rows, with a monospace font loaded so two values of one length differ only in ink. A selection is read by comparing two such values rather than by counting the highlight, which moves with the text width.

Row and state	which characters paint	the hint paints	a selection shows which
Secret, at build, before	896	1122	896
Secret, at build, after	0	1122	0
Secret, Show ticked, after	896	1122	896
Secret, hidden again, after	0	1122	0

The Show ticked row is the control: it reports on every reading, so each zero is a fact about the state and not about the instrument. The hint now survives the hide, which the earlier sheet removed along with the characters. The key and passphrase rows report a masked display and a refused copy at build, both before and after, so the pair ends in the same state.

What the install path still leaves open

A Qt text area hands its value to the clipboard whatever colour it paints in, while a single-line row in password mode refuses the copy. Measured on every state of the secret row, the copy returned the typed value; on the key row it returned nothing. No style sheet closes that, and the wizard draws on no path today.

```
API Secret     copy hands over the value   True   in all four states
API Key        copy hands over the value   False  at build and hidden
```

The wizard's name box still fills itself with User and still refuses an empty name. That collection reaches no caller, and the unlock it would feed is the Quintessence Wallet, so it waits on the wallet rather than being removed.

2026-09-13 - One venue list, one passphrase set

Four copies of two facts were measured, and the copies are gone. The connector decides which venues exist and which of them ask for a passphrase. Every screen now reads those two answers instead of keeping its own.

```
def exchange_ids() -> tuple:
    from ...exchange.ccxt_connector import SUPPORTED_EXCHANGES

    return tuple(sorted(SUPPORTED_EXCHANGES.keys()))
```

The screen offers the same fifteen venues in the same order as before, and the same three still ask for a passphrase. Nothing about what is offered changed.

Where the two lists were declared

Six declarations were found, and three of them were copies of the other three.

Fact	Declaration	Where
the venues	`SUPPORTED_EXCHANGES`	the connector, the authority
the venues	a frozen tuple	the install wizard page
the venues	a frozen tuple	the one-window bot runner
the passphrase venues	`PASSPHRASE_EXCHANGES`	the connector, the authority
the passphrase venues	a frozen set	the Settings page
the passphrase venues	a frozen tuple	the install wizard page

Each copy is now a small function that asks the connector when it is called. The import sits inside the function, so loading a screen module still loads no exchange library and reads no settings file.

The falsifier, one entry at a time

One venue was taken out of the connector inside a driving process, and every consumer was then asked what it offers. The tree itself still holds fifteen.

Reading	Consumers asked	Still holding the removed venue
Before the change	9	4
After the change	9	0

The same was done with one passphrase venue, against every place that decides whether the field is drawn.

Reading	Consumers asked	That did not follow the connector
Before the change	14	9
After the change	14	0

The four that carried the venue were the install wizard's ids, its labels, its bridge list and the bot runner's model. The nine that ignored the passphrase change included the ticked box and the drawn field on the Settings page, which is what the operator sees.

Why that matters, and it is not tidiness. A venue offered on a screen and absent from the connector is stored and then refused, before any object exists.

```
CCXTConnector('gemini') refused: Unsupported exchange 'gemini'. Supported:
['binance', 'coinbase', 'kraken', 'kucoin', 'bybit', 'okx', 'gateio', 'bitget',
'huobi', 'mexc', 'bitfinex', 'poloniex', 'bitstamp', 'cryptocom']
```

Every reader of the chosen venue, driven

Five readers were named earlier as out of reach without a live window. All five were driven in one run, with real tab widgets and a real wizard, against a store holding two venues: one with an obvious fake credential and one with none.

Reader	What it answered
the startup tab loop	two crypto tabs, Gemini and Kraken, no stock tab

Reader	What it answered
the startup credential report	Gemini stored and unverified, Kraken with none
the bot connect lookup	both refusals, before anything was contacted
the bot wizard's venue picker	Gemini and Kraken, Gemini selected
the API panel's stored lookup	the no-credentials refusal

One step is still out of reach. The Qt wizard is opened behind a modal call that waits for a person, so the handler that opens it cannot be run to the end without one. The wizard itself was built with the stored entries and its picker read.

The diagnostic tool converts for every venue

The operator tool asked for a key at the keyboard and then converted a pasted private key's escaped newlines for Coinbase alone. Every other venue received an unusable key. The conversion now runs on the way in, for every venue, gated on the key's own armour lines.

```
secret = input("  API Secret: ").strip()
if looks_like_pem(secret):
    secret = unescape_pem_newlines(secret)
```

The tool also opened its fifteen public endpoints through the plain library call, which allows a file scheme, under a suppression marker that sat on the line above the real one. It now opens them through the project's own scheme-restricted request and opener.

What the docstrings claimed

Two screen modules said in their own opening text that a value changed on one side alone is reported. Nothing reads prose, and the tree holds no test that compares them.

```
git ls-files tests/     conftest.py, 6 Solidity files, 1 data file, 150 reports
pytest --collect-only   no tests collected
```

The claim is removed rather than reworded. Each module now says plainly that nothing compares its written-out values, and names the values that are not written out and therefore cannot drift: the twelve indicator weights, the venue list and the passphrase set.

Two things this section does not change. Which venues are offered is still decided in one place only, the connector, and no venue was added or removed. The install wizard still writes its own shorter passphrase note beside a venue name while the Settings page writes the connector's longer one, because changing either would change words on a screen.

2026-09-13 - The exchange screen follows the store

Three things this page said were not things the program had done. A removed venue kept its tab until the next launch. An add reported a venue verified when nothing had been contacted. An add named the rows that were typed rather than the entry that was stored. Each one now reads off what happened.

The tab bar takes a venue off

The sync added and never dropped. It now asks each layer for the ids the store no longer lists, takes those tabs off the bar, and returns how many went.

`src/gui/main_window.py` — `_drop_unlisted_exchange_tabs`, called by the sync

```
_gone = self._drop_unlisted_exchange_tabs(_wanted)
```

Driven on the running window, crypto layer, reading the tab captions and the rendered bar after each press.

The press	Captions before	Captions after
Remove Selected on Kraken	`Kraken`, `Kucoin`	`Kucoin`
Test and Add Exchange for Gemini	`Kucoin`	`Kucoin`, `Gemini`
Remove Selected on the last two	`Kucoin`, `Gemini`	`Get Started`

Before the change all three presses left the same three captions and the same rendered bar, so the picture agreed with the list and both were wrong. The sync still adds, which is the second row above.

The Get Started page comes back because each layer now keeps its placeholder object instead of dropping the reference on its first add. A tab taken off is asked to stop its news ticker and its data-pull countdown first; both keep running otherwise, because a tab removed from a bar is still a child of the layer's stack.

Nothing here changes what a removal permits. The store drops the entry exactly as it did, no bot is consulted and none is stood down.

The add message reads the stored entry

The phrase came from the API Key row. It now comes from the entry the store holds after the write, through one function both builds call.

`src/gui/main_tabs/settings_dialog_surface.py` — `stored_credential_phrase`

```
def stored_credential_phrase(entry: Any) -> str:
    holder = entry if isinstance(entry, dict) else {}
    if any(holder.get(name) for name in CREDENTIAL_FIELDS):
        return ADDED_WITH_CREDENTIALS
    return ADDED_WITHOUT_CREDENTIALS
```

The press	The line before	The line after
Key typed, secret empty, nothing stored	`Kucoin added with credentials (verified).`	`Kucoin added without credentials.`
Both rows empty over stored credentials	`Kucoin added without credentials.`	`Kucoin added with stored credentials.`
Both rows empty, nothing stored	`Kucoin added without credentials.`	`Kucoin added without credentials.`

The Activity Log line and the confirmation box take the same phrase, so the three surfaces cannot disagree. The third row is the control and it did not move.

The first row is the press row 8 measured. A key with no secret never reaches the connection check and never reaches the store, so nothing was contacted and nothing was stored, and the word verified had no grounds behind it. The word is gone rather than reworded: no add can earn it without a connection, and the venue's own answer is already reported on that same line by the check that got it.

A blank re-add still keeps the stored tokens, which is row 8's repair. The second row above is that press, and the entry it names carries the same two tokens it carried before.

What could not be driven

`QMessageBox.information` waits for a person, so the add cannot be driven past it on either build. The two lines written before it were read off the running dialog, and the box text was read off the Qt-free model, which records the box instead of showing it. All three take their phrase from the one function above.

2026-09-13 - The bot's increment style comes off BotConfig

The Increment Style row left the Trading page on 11 September and the bot field of the same name stayed. Nothing wrote that field and nothing read it. Both of its declarations are now gone.

```
# src/trading/container/config.py, both lines removed
increment_style: str = "linear"          # the BotConfig field
"increment_style",                       # the Scrumming-only name
```

The sweep was run twice on the same name, once matching case and once ignoring it, because a case-sensitive search has already missed a site twice in this audit.

probe	before	after
occurrences in code	2	0
occurrences repo-wide, matching case	22	20
occurrences repo-wide, ignoring case	22	20
writers anywhere	0	0
readers anywhere	0	0

The twenty that remain are this page, the Trading tab page and one captured report. None of them is code.

A record carrying the retired name still restores

The restore path builds its arguments through one helper, and that helper keeps only the names the dataclass declares. A record holding a name the product has retired loses it at that filter instead of raising.

```
record            43 stored keys, four of them retired names
restored          1 bot, ScrummingBot, on the read-only restore placeholder
fields compared   65
fields different  1, the removed name
raised            nothing
```

The creation path reads that same declaration, so one removal reaches both paths at once. A wizard-shaped dictionary driven through the same two calls gave the same 65 compared against 1 different, and a record holding only a venue, a pair and a mode still restored with nothing raised.

The bot's own behaviour does not move

One restore and one tick were driven before the change and after it, against the read-only placeholder the restore path already uses. That placeholder answers False to every order method, so no order can be placed from the run.

reading	before	after
tick readings compared	10	10
tick readings different	-	0
what the tick raised	the placeholder has no ticker reader	the same

The comparison can report a difference. A control run moved the stored target balance and the stored read rate, and the same comparison then named three fields instead of one and two tick readings instead of none.

The bot's spacing style was already gone

The bot-side spacing style was removed with the Settings-side one. Nothing was left to take out.

```
spacing_style   0 occurrences in src, tests, tools, desktop and dev_harness
                4 occurrences repo-wide, both files dated history
```

The wizard stops advertising the retired Grid Bot

The wizard's opening description named a mode the product does not build, listed three rows already off the Settings dialog, and carried a version string. A second description inside the parameter page named the same mode again, with two more version strings.

```
# src/gui/bot_wizard.py, the opening description now reads
#   Accumulation Bot: Target Balance, Scrumming Interval, TA Engine, Phantoms
#   Extractor Bot: base-currency chunk, artillery sizing, compounding tier
#   Both: Visibility, Aggressive Trading
```

The page subtitle that named the retired mode is now one sentence with no version in it. No caller can draw it: the mode page answers False to the grid question on every call, so that branch has never been reached.

The retired names this unit left in place

Each of these carries a reason, and none of them is in the wizard's own text.

where	what it says	why it stays
the launcher	two screen strings naming the retired mode	that file does not pass its own coding verdict, so nothing ships from it
the mode page	a comment beside a constant return	it states why the method returns a constant
the restore path	one comment naming the retired mode	it explains why a stored mode is skipped
a captured report	one occurrence of the removed name	captured output, already owned by another row

The second wizard module keeps its stored-default read

The second wizard module reads a stored target balance that never arrives, because the shell's request carries the step values alone. The read itself is live: the model calls it on every build, and three sibling reads of the same bag sit beside it, two of them built deliberately by the phantom rows above.

```
_apply_stored_target_balance   called on every model build
sibling reads of the same bag  3
```

Removing it would delete one of four reads of one bag and settle three rows that are still open. It stays, and the empty bag is what those rows address.

2026-09-13 - The shell's wizard request carries the store

The desktop shell asks for the Create Auto Trader pages and sends the step values alone. The running program now puts its own stored settings into that request before the pages are laid out, so the wizard the shell draws opens on the same figures as the wizard the Qt window opens.

```
# src/gui/main_tabs/bot_wizard_surface.py, the handler the bridge registers
def live_view_model(params: dict, live: Any) -> dict:
    asked = dict(params or {})
    manager = getattr(live, "settings_manager", None)
    if asked.get("defaults") is None and hasattr(manager, "get_all"):
        asked["defaults"] = manager.get_all()
    return view_model(asked)
```

The bag the request carries went from no keys to seventeen. That was read on the request the shell sends when the New Bot space mounts, and again on the request every control press sends, so a press does not put the wizard back on its built-in figures.

What the request now carries

Five Settings rows reach a control on the shell build. Two of them are lookups this section adds to the wizard surface; the other three were already written and had nothing to read.

Settings row	store key	wizard control	opened at	opens at
Default Target Balance	`default_target_balance`	Target Balance	200.00	the stored figure
Bot Visibility	`bot_visibility`	Order Visibility	Order Book	the stored name
Enable aggressive trading mode	`aggressive_trading`	Aggressive Trading	clear	the stored flag
Enable Phantom Balance Bots for Scrumming	`default_enable_phantoms`	Enable Phantom Bots	clear	the stored flag
Lock duration (candles)	`default_lock_candle_count`	Candles to lock	2	the stored figure

The two new lookups sit beside the three already there, and all five run inside the model build, before the page is drawn.

```
# src/gui/main_tabs/bot_wizard_surface.py, inside _build
self._apply_stored_target_balance()
self._apply_stored_visibility()
self._apply_stored_aggressive()
self._apply_stored_phantom_enable()
self._apply_stored_lock_candles()
```

The five figures on both builds

One store, written with five values that differ from every built-in figure, was read by both wizards. Both opened on the same five.

control	the store	the Qt wizard	the shell wizard
Target Balance	777.00	777.00	777.00
Order Visibility	internal	internal	internal
Aggressive Trading	on	on	on
Enable Phantom Bots	on	on	on
Candles to lock	7	7	7

Before the change the shell wizard opened at 200.00, Order Book, clear, clear and 2, on the same store. A bot created from those pages carried those five figures, so the shell built a different bot from the Qt window on identical settings. It no longer does.

An older store still opens the wizard

A store written before these rows existed carries none of the five keys. The request then hands over the twelve keys it does hold, every lookup falls to its own built-in figure, and the wizard lays out its six pages as usual.

```
bag keys 12    target balance 200.00    visibility Order Book
               aggressive clear         phantoms clear
               lock candles 2           pages 6
```

The shell reaches the store only while the trading program serves the bridge. Started as the surfaces alone, with no running system behind it, the bridge has no settings manager to read and the wizard opens on its built-in figures.

2026-09-13 - The store refuses a value of the wrong type

Two rows turned out to be one fault. `AppSettings` names a type for each of its seventeen fields, and nothing anywhere held a stored value to that type. Driven across all seventeen, the setter took a word where a number was declared and a word where a switch was declared, on seventeen of seventeen.

The store now answers for its own declaration. One function reads the type each field's own default carries, and both write paths ask it.

```
# src/core/settings.py
DECLARED_TYPES: dict[str, type] = {
    name: type(value) for name, value in asdict(AppSettings()).items()
}


def declared_type_holds(key: str, value: Any) -> bool:
    ...
```

A switch is refused where a number is declared, and a whole number is admitted where a dollar figure is declared, because a settings file writes 350 and reads 350 back.

What the two write paths do now

`SettingsManager.set` raises and names the field, the type it takes and the type it was handed. The load path never raises, because a settings file that stops the program starting is worse than one field on its default.

path	a value of the wrong type	what the operator sees
`set`	`TypeError: Setting 'aggressive_trading' takes bool, not str`	the Settings page's own save loop names the field and keeps going
the load path	the field's declared default, and one warning line	`Settings kept the default for default_target_balance: the file holds str, the field takes float.`

Neither path changes a value into something else. A word is never turned into a number, because a changed value is a second way to be wrong.

A settings file that stops the bot wizard opening

A hand-edited file holding a word where the target balance belongs stopped the wizard being built at all. The Settings page caught the same value and drew one dollar; the wizard had no guard beside its two siblings, which already refuse an unknown visibility name and admit only a switch.

surface	before	after
the Qt wizard	`TypeError` at construction, no window	opens, Target Balance $200.00
the shell wizard	`TypeError` at model build, no pages	opens, Target Balance 200.00
the Settings page	opened, drew `$1.00`	opens, draws $200.00

Both wizards now keep their own figure when the value handed to them is not a number, and both still open on a figure the operator did set.

```
stored 777.00     Qt wizard 777.00      shell wizard 777.00
stored a word     Qt wizard 200.00      shell wizard 200.00
stored 350        Qt wizard 350.00      shell wizard 350.00
```

A switch that read on whatever the file held

The aggressive trading flag is declared as a switch, and a word in the file read as on across every surface at once. Both surfaces agreed with each other, which is why nothing reported it.

surface	before, file holds the word no	after
the Qt wizard box	on	clear
the shell wizard box	on	clear
the Settings page box	on	clear
the config a new bot is built from	on	clear

A switch the operator really did set still reaches the wizard on: stored on opens the box on, which is the control that proves the reading is the store's and not the guard's.

A store at plausible values loads unchanged

Every one of the seventeen fields was written at a value the operator could hold — a venue entry with both tokens, a target balance of 350.00, a chosen theme, an accent colour, twelve indicator weights, the sound group and the message channels group. It was written through both writers the store uses.

```
fields compared 17   different 0    written as settings.toml
fields compared 17   different 0    written as settings.json
control: the same 17 with two words planted -> different 2
         default_target_balance and aggressive_trading, both on their defaults
```

The control is the second half of that block. The comparison reports a difference when there is one, so the two zeroes above are a reading and not an empty instrument.

The creation record names figures the wizard carries

Every bot ever created logged `Positions=0`. The record read a name off the wizard's dictionary that no branch of the wizard writes — it is a retired name, listed among the settings the bot config drops. An Extractor recorded a balance of zero for the same reason, because that mode's dictionary carries no target balance either.

```
# src/gui/main_window.py
if config.get("mode") == "extractor":
    _created = (
        f"Chunk size=${config.get('extractor_chunk_size_usd', 0):.2f}, "
        f"Scan top={config.get('extractor_scan_top_n', 0)}"
    )
else:
    _created = (
        f"Balance=${config.get('target_balance', 0):.2f}, "
        f"Tranche count={config.get('stack_tranche_count_target', 0)}"
    )
```

Both names are the wizard's own labels: Tranche Count on the parameters page and Chunk size (USD) on the Extractor page.

mode	before	after
Scrumming	`Balance=$200.00, Positions=0`	Balance=$200.00, Tranche count=3
Extractor	`Balance=$0.00, Positions=0`	Chunk size=$100.00, Scan top=8

Moving the two controls moves the line: a tranche count of 9 and a target balance of 512.00 recorded Balance=$512.00, Tranche count=9.

The increment style sentence

The Trading page description above says a bot takes its own increment style from the wizard rather than from this page. That was true of neither half by the time it was written, and it is true of nothing now.

what the sentence says	what is there
a wizard control sets an increment style	no wizard control does, on either build
a bot carries an increment style of its own	the bot config field was removed on 13 September

Nothing sets an increment style and nothing reads one. The concept the sentence reaches for is Tranche Spacing, which the wizard does carry, and the derived placement the engine works out from Opposing Trade Distance is what decides where a tranche lands.

2026-09-13 - The wizard's Profit Folding page is removed

The bot creation wizard registered six pages and routed to five. The sixth was Profit Folding & Upward Distribution, and it is gone from both builds. Nothing a person can press changed, because nothing could reach it.

Driven with the home redirected into a scratch directory before the first product import, so the settings directory and the log root bound under that directory. No stored setting of the running install was read, no credentials file was opened and no venue was contacted.

Which switch is which

Three controls wear almost the same words. Only one of them does anything, and this section removed neither that one nor the Settings row.

The words on screen	Where it lives	What happened to it
Profit Folding Active	a running bot's own settings	untouched. It gates the fold and it works
Profit Folding / Upward Distribution Active	the Settings dialog's Profit Folding page	untouched. It is the last row on that page
Enable Profit Folding	the wizard's folding page	removed with the page it sat on

The removed one is the wizard's. A bot already running reads its own flag from its saved record, and that reading is unchanged.

```
  if not self.config.profit_folding_active:
```

The page that no route reached

`nextId` was read from each page in turn with that page set as the start page, on both modes. No page answered the folding page in either.

```
before, accumulation      before, extractor
  asset  -> params          asset  -> params
  mode   -> asset           mode   -> pool
  params -> phantom         params -> finish
  pool   -> params          pool   -> params
  folding -> finish         folding -> finish     no page answers folding
  phantom -> finish         phantom -> finish
```

The Qt-free surface said the same in a named constant rather than by omission, and published it as a payload key. Both builds now register five pages, every one of the five is on a route, and the constant and its key are gone with the page that was the only entry either ever held.

The bot the wizard builds, before and after

The collection was driven on both builds and both modes, and every key and every value was compared.

Arm	keys before	keys after	key set identical	values different
Qt, accumulation	49	49	yes	0
shell, accumulation	49	49	yes	0
Qt, extractor	49	49	yes	0
shell, extractor	49	49	yes	0
Qt, on stored figures	49	49	yes	0
shell, on stored figures	49	49	yes	0

The last two arms opened the wizard on a stored bag rather than on the build figures, and that bag moved five values on both builds either side of the change: target balance 200.00 to 512.00, the visibility from order book to internal, aggressive trading on, phantoms on, and the lock from 2 candles to 5. A bag that moved nothing would have made those two arms prove nothing.

The same comparison reported ten differences in the routing and payload readings taken in the same run, so a zero on the six config arms is a fact about the bot and not a blind comparator.

An Extractor still carries `profit_folding_active`, because both builds write it from a literal on the extractor branch and never from the folding page.

```
config["enable_phantoms"] = False
config["profit_folding_active"] = False
```

The stored record that still loads

A record shaped like `bot_state.json` was written carrying all six of the page's names, and restored through the two calls the restore path makes.

Stored name	On the built config, before	after
`fold_mode`	absent	absent
`fold_target`	absent	absent
`fold_target_count`	absent	absent

Stored name	On the built config, before	after
`distribute_target`	absent	absent
`distribute_target_count`	absent	absent
`profit_folding_active`	False	False

Sixteen fields were compared and none differed. Nothing raised on either side. The five names stay on the retirement record in the container config, which is what drops them, so a record written before the page came off still builds.

The fold ladder half nobody called

`stack_math` declared a matched pair of ladder-price helpers. The scrum half is called when a stack is split into tranches; the fold half had one definition and no call, no import and no aliased binding anywhere in the tree.

```
  prices = scrum_ladder_prices(
```

The fold half is removed. The shared helper keeps its direction argument, because the Extractor passes the fold sign to the placement floor on a live path of its own.

The sentences the wizard page leaves behind

Each sentence below stands in an earlier section and no longer describes the tree. The earlier text stays where it is.

Earlier sentence	What is there now
"the wizard's folding page is unreachable - a route to the page and a call to its `get_config`"	The page is removed, so there is nothing to route to
"the box does not turn the page on - an enable rule over the three groups below it"	The wizard's box and its three groups are gone together
"No page answers the folding page. The only branch that would is guarded behind `is_grid()`"	That branch is removed and `is_grid` keeps its one live caller on the parameter page
"The field arrives the moment the folding page is reached and collected."	No page collects it. A new accumulation bot takes the declared default on its config
"Until the first of the three lands, a stored choice on this row has nowhere to arrive"	The first will not land. The Settings row's own answer is now free of the wizard
"The wizard registers six pages and routes to five."	It registers five and routes to all five
"the wizard lays out its six pages as usual"	It lays out five
"its view model reports the folding page unreachable"	The view model no longer carries an unreachable list, because no page is

The Settings dialog's own Profit Folding row is untouched by this section and still holds the page alone.

2026-09-13 - The Settings dialog's Profit Folding page is removed

The page held one checkbox and nothing else. That checkbox reached no bot, no engine and no fold, so the page is gone from both builds and the dialog opens on nine tabs. Folding is still switched on and off per bot, on the running bot's own settings row, and that row is untouched.

The orphaned target-growth module goes with the page. `apply_profit_fold` had no caller anywhere in the tree, and the glossary named it as the fold's entry point while the live path is the fold tranches.

Driven with the home redirected into a scratch directory before the first product import, so the settings directory and the log root bound under that directory. No stored setting of the running install was read, no credentials file was opened and no venue was contacted. One connection to a venue host was attempted on purpose and refused.

The three switches, and the one this section removed

Three controls wear almost the same words. Only one of them changes what a bot does, and this section left that one alone.

The words on screen	Where it lives	What happened to it
Profit Folding Active	a running bot's own settings	untouched. It gates the fold and it works
Profit Folding / Upward Distribution Active	the Settings dialog's Profit Folding page	removed with the page
Enable Profit Folding	the wizard's folding page	removed earlier, with that page

The stored names behind them are three different words, which is what keeps them apart.

```
per-bot switch    profit_folding_active     a BotConfig field, 10 read sites
Settings switch   profit_folding.active     an AppSettings group, 0 readers
wizard switch     -                         no longer in the tree
```

The nine tabs the dialog draws now

Both builds take the tab list from one declaration, so removing the page removed the tab from the Qt window and from the shell together.

	before	after
Tabs	10	9
Controls on the dialog	67	66
Controls on the Profit Folding tab	1	0
Layout entries	10	9
Groups the dialog persists	5	4
Keys a Save writes	14	13

The tab list after the removal, read off the payload the shell draws:

```
User, Exchanges, Trading, TA Indicators, Phantom Bots,
Theme, Sound, SMS, AI Monitor
```

A tab holding no control is worse than no tab. What a reader of this page expects to find there is one master checkbox and three groups of radio buttons, eleven controls in all, and none of the eleven remains. Where folding is switched now is the running bot's own settings row, which the dated sections above already describe.

The stored group a settings file still carries

A settings file written before the removal still carries its `profit_folding` entry. The loader copies only the fields the settings record declares, so an entry it no longer declares is passed over and every other value arrives.

Driven on a file carrying ten keys, one of them the folding group:

	before	after
Fields on the settings record	17	16
The folding group is one of them	yes	no
Every other stored key arrived	yes	yes
Anything raised	no	no

The next Save writes the file without the group.

The per-bot switch, driven either side

A record shaped like `bot_state.json` was written, restored through the two calls the restore path makes, and the bot it built was driven with the switch on and then off. The readings are the same on both sides of the removal.

The switch	applier returned	target after	preview returned	the gate read
on	$6.00	$200.00 to $206.00	$6.00	on
off	$0.00	held at $200.00	$0.00	off

Two controls prove the comparison can report a difference rather than agreeing by being blind. Moving the units the preview is asked about took its answer from $6.00 to $4.00, and lowering the growth cap from three percent to one took the applier from $6.00 to $2.00 and the target to $202.00.

```
[switch] ON   apply 6.0  target 206.0  preview 6.0  gate true
[switch] OFF  apply 0.0  target 200.0  preview 0.0  gate false
[control] units 0.2        preview 4.0     moved
[control] growth cap 1.0%  apply   2.0     moved
```

The record and the wizard collection go through one declaration, so creation and restore cannot drift. Both carried twelve values into the config, sixty-four config fields were compared between the two, and none differed, on both sides of the removal.

The orphaned target-growth module

`profit_fold` held one function and forty-six lines. Its own module docstring recorded that no module imported it, and that was still true: four occurrences in the product, all four inside that file, three of them in the docstring and one on the definition line.

The live fold path is the tranche book, and the glossary now says so.

```
_fold_eligible_tranches      picks the tranches a fold may reach
_apply_fold_target_growth    grows the target from the fold surplus
```

The sentences the Settings folding page leaves behind

Each sentence below stands in an earlier section and no longer describes the tree. The earlier text stays where it is.

Earlier sentence	What is there now
"One master checkbox and three groups of radio buttons, eleven controls in all."	The page is removed and the dialog draws nine tabs
"Every key below sits inside one stored `profit_folding` group."	No page writes that group and the settings record no longer declares it
"The stored group holds one field after those removals."	The group is gone. A stored file still carrying it loads and the field is passed over
"Profit Folding / Upward Distribution Active - Turns the whole page on."	No page remains for it to turn on
"This is the one control on the page the load path restores."	The load path restores no control on that page
"What the Profit Folding page holds now - One checkbox and nothing else."	It holds no checkbox, because the page is removed
"The master switch is the last row on the page, and its own verdict was that it waits on the wizard's folding page."	Both pages are removed. The verdict is settled by removal, not by waiting
"`SettingsManager._apply_dict` copies the whole `profit_folding` dict off the file, so a store written before the removal still carries its `mode` entry and still opens."	It copies the fields the record declares, and the folding group is no longer one. The store still opens

The glossary entry that named the removed module as the fold's entry point is answered in place, in the ADR index and glossary.

2026-09-13 - A value pushed after the page is drawn

The dialog keeps its figures in Python and draws them in a web page. Until this section, a figure written after the page was up moved the store and left the screen alone. The page now takes every push.

```
# src/gui/react_settings_dialog.py, the write and the push it starts
def set_value(self, name: str, value: Any) -> None:
    self._model.admit(name, value)
    self.redraw()
```

Where a push used to stop

React draws each control with a seeded value, which the browser reads once when the node is made. Every ordinary property is redrawn on a later push — whether a row is hidden, whether a button is greyed, the choices in a drop-down — but the figure inside a box is not one of those. The node kept whatever it was born with.

Diagram: flowchart, left to right

- the operator's own edit -> **the store**
- a write from the program -> **the store**
- the store -> **the payload**
- the payload -> **the page redraws**

- the page redraws -> **hidden, greyed, choices**
- the page redraws -> **the figure in the box**

What each drawn control takes now

Each draw ends by writing the store's figure into every drawn control whose figure differs. A tick box takes a tick, a drop-down takes a row number, and every other control takes its text. Nothing is written where the two already agree, so a redraw that changes nothing touches nothing.

```
// src/gui/web/settings_dialog.js
function draw(target, node) {
  var root = rootFor(target);
  global.ReactDOM.flushSync(function () {
    root.render(node);
  });
  syncDrawn(target);
  return target;
}
```

The dialog draws 66 controls across nine tabs. Sixty-five of them take a seeded figure and are reached by this. The sixty-sixth is the configured exchange list, which is drawn as a list of lines and was always rebuilt.

Tab	Controls	Tab	Controls
User	1	Theme	2
Exchanges	6	Sound	10
Trading	3	SMS	22
TA Indicators	12	AI Monitor	7
Phantom Bots	3		

The credential rows, cleared and read off the page

Adding a venue empties the API Key, API Secret and Passphrase rows. Both figures below were read off the drawn nodes, never off the store.

Driven on the rendered page	Before	Now
the key row after the clear	still showed the typed key	empty
the secret row after the clear	still showed the typed secret	empty
the key row after a tab switch	still showed the typed key	empty
Lock duration after a write of 3	showed 7	shows 3
the phantom box after a tick	showed clear	shows ticked
Default Phantom Timeframes after a write of `1d`	showed `4h`	shows the row for 1d

A push cannot land on a half-typed value

The one control the page leaves alone is the one under the operator's cursor. A figure being typed has not been reported yet, so the store is behind the screen for that box and only for that box. Skipping it is what keeps a push from wiping out half a typed key.

```
// src/gui/web/settings_dialog.js, inside syncDrawn
var typing = owner ? owner.activeElement : null;
if (node === typing || kind === LIST_KIND) {
  return;
}
```

Driven both ways on the same push: with the key row focused and holding a half-typed value, the page kept it; with the same row left, the same push wrote the store's empty figure in. The skip is what saves it.

An emptied number box is no longer reported

A number box with nothing in it is not a figure. The page used to hand the empty box back as nothing at all, the store refused it, and a warning line was written for a value nobody had entered. The page now puts the held number back in the box and reports nothing.

```
// src/gui/react_settings_dialog.py, the page's own reader
var value = readValue(node, kind);
if (typeof value === "number" && !isFinite(value)) {
  node.value = api.valueOf(name);
  return false;
}
```

The TA row width the page checked against

The page checks the shape of everything it is handed. Its check on the indicator weight rows wanted three fields and every row carries four, the fourth being the slider's own name. The check reported twelve faults on every load, one for each indicator, and reports none now.

```
// src/gui/web/settings_dialog.js
var TA_ROW_WIDTH = THIRD + STEP;
```

The six earlier rows this section drove

Every one of these was built on the older behaviour and every one was driven again afterwards.

Earlier work	Driven after the change
the API Key and API Secret masked together	all three credential rows still masked, read off the drawn page
the wizard's stored defaults, filled before the draw	the five stored figures still reach the wizard, against the built-in figures with no store
Enable Phantom Balance Bots for Scrumming	opens on the stored setting
Default Phantom Timeframes	opens on the stored name
Lock duration (candles)	opens on the stored figure

Earlier work	Driven after the change
the wizard's Enable Phantom Bots and Active Timeframes	both open on the stored setting, driven either way

2026-09-13 - The radio drawing code comes off the Settings dialog

The dialog draws no radio button on any tab. The last three went off the page with the sell-side target group, and the machinery that drew them stayed behind: a kind constant, two lookup tables, a holder class, the grouping that cleared a button's siblings, a browser branch and a style rule. None of it had anything left to draw, and all of it is gone.

Driven with the home redirected into a scratch directory before the first product import, so the settings directory and the log root bound under that directory. No stored setting of the running install was read, no credentials file was opened, no bot was built and no venue was contacted. One connection to a venue host was attempted on purpose and refused.

What drew a radio, and what asked for one

Four files carried the machinery. Zero control specs asked for it.

What it was	Where it lived
the kind constant, and its rows in the signal and painted tables	`src/gui/main_tabs/settings_dialog_surface.py`
the holder class and the sibling grouping	`src/gui/react_settings_dialog.py`
the input-type branch and the shared group name	`src/gui/web/settings_dialog.js`
the style rule the tick box shared with it	`src/gui/web/settings_dialog.css`

The count either side, over the five files that make the dialog:

	before	after
Lines naming a radio	26	0
Occurrences, case-insensitive	31	0
Control specs of that kind	0	0

The dialog either side of the removal

Three surfaces were read in one run, before on a clean copy of the default branch and after on the branch:

	before	after
Tabs, in the Qt window and in the shell payload	9 and 9	9 and 9
Control specs the payload carries	66	66
Tick boxes the Qt window draws	23	23
Drop-downs the Qt window draws	6	6
Radio buttons the Qt window draws	0	0
Radio inputs the browser page draws	0	0
Checkbox inputs the browser page draws	207	207

The whole payload, the whole Qt reading and the whole rendered-page reading compare identical either side. The 207 is the positive control: the counter that reported zero radio inputs saw 23 tick boxes on each of the nine tab readings, so the zero is a fact about the page rather than about the counter.

A settings file written while the radio buttons were there

The sell-side group stored one of three words, and the buy-side group stored one of another three. A settings file written before the groups came off still carries all of them.

```
buy side     all_buy    x_buy    most_recent_buy
sell side    all_sell   x_sell   most_recent_sell
```

Driven on a file carrying four top-level keys, one of them a folding group of six:

	after
Anything raised	no
Username arrived	yes
Default Target Balance arrived	yes, 310.0
The folding group	not declared, passed over

The sentence this removal leaves standing

None. The Profit Folding description above names no control and no radio button. Nothing on the dialog draws a radio button, and after this change nothing in the code can.

2026-09-13 - The distribution statistic comes off the bot record

One field on a bot's statistics record counted dollars toward a distribution the platform no longer performs. It was declared once and written by nothing, and it is gone.

Driven the same way as the section above, with the home redirected before the first product import. No stored record of the running install was read, no credentials file was opened, no bot was built and no venue was contacted.

One declaration, no writer, no reader

`src/trading/container/config.py` — the line that went

```
accumulated_distribute: float = 0.0  # Tracks toward extended position
```

The name appeared once in the whole tree, the same count case-sensitive and case-insensitive, and that one appearance was its own declaration. No line set it, no line read it, and no page of this manual named it.

A saved bot record carrying the retired figure

A save copies the statistics out with `asdict`, so the retired figure was written into every saved record. A restore copies back only the names the record still declares, and passes over the rest.

`src/trading/container/restore.py` — the gate a stored figure meets

```
saved_stats = bot_data.get("stats", {})
for key, val in saved_stats.items():
    if hasattr(bot.stats, key):
        setattr(bot.stats, key, val)
```

Driven on a record shaped like a saved bot, carrying every declared field, the retired figure at 12.5, and one planted name that has never been declared:

	before	after
Fields the record declares	36	35
Keys in the stored record	37	37
Stored figures the gate admitted	36	35
Stored figures the gate passed over	1	2
Anything raised	no	no

The planted name is the positive control. It was passed over on both sides, so the gate can report a skip, and the retired figure moving from admitted to passed over is the whole of the change.

The directive this removal breaks

Nothing. The name appears in zero lines under `docs/`, so no sentence anywhere describes it. Its own comment named bookkeeping for the distribution mechanics the Profit Folding sections above already record as removed.

2026-10-04 - Exchange Status, the sector's venues in three colours

The list at the top of the Exchanges page is headed Exchange Status. It carries every venue the active sector can trade through, not only the venues already configured. Each line is a button, and pressing one points the Add form below at that venue.

Which venues the panel lists

The sector decides the list. `venues_for_class` answers it, and a venue serving more than one sector is answered for each of them.

`src/gui/main_tabs/settings_dialog_surface.py` — `exchange_status_rows`

```
for venue in sorted(acs.venues_for_class(wing)):
    state = venue_state(venue, held)
    name = venue.capitalize()
```

Driven on the real dialog for all six sectors, with every outbound socket refused:

Sector	Venues the registry answers	Rows drawn
Crypto	15	15
Stock	10	10
Commodities	1	1
Forex	0	1 note

Sector	Venues the registry answers	Rows drawn
Indices	0	1 note
Futures / Perps	0	1 note

A sector no venue serves draws one line saying so, rather than an empty box. Coinbase is listed under Crypto, Stock and Commodities, because it serves all three.

What the three colours mean

Grey is the resting colour. It says nobody has checked, not that something failed. Green says a credential check succeeded. Red says a check reached the venue and the venue refused it.

State	Colour token	Colour
Credentials validated	`SUCCESS`	`#00ff88`
API connection lost	`ERROR`	`#ff3366`
No valid credentials	`STATUS_NEUTRAL`	`#8899aa`

Where the colour comes from

`src/exchange/credential_state.py` keeps one record per venue, under `~/.acervator/exchange_credential_state.json`. The credential checker is the only writer, and it writes only after a venue has answered.

`src/exchange/api_validator.py` — `validate_credentials`

```
from .credential_state import record_validated

record_validated(exchange_id)
```

A missing key or a missing passphrase is refused before any call is placed, and those refusals record nothing, so the venue stays grey.

Diagram: flowchart, left to right

- Test Connection -> **api_validator — validate_credentials**
- api_validator — validate_credentials -> **credential_state — exchange_credential_state.json** *(answered)*
- credential_state — exchange_credential_state.json -> **Exchange Status — _refresh_exchange_status** *(read on redraw)*

A redraw reaches no venue

Drawing the panel reads the stored record and nothing else. Measured with every outbound socket refused and a refusal counter watching: ten redraws of the crypto panel, fifteen rows each time, and the counter never moved off the one refusal its own control produced.

A recorded green does not fade with time. The colour reports the last recorded answer, and a timer turning it grey would say nobody had checked when somebody had. A venue that stops answering turns red through its next check.

Pressing a line

A press selects the line and points the Exchange row of the Add form at that venue, then writes the venue's name into the feedback line.

`src/gui/settings_dialog.py` — `_open_credentials_for_row`

```
self._exchange_list.setCurrentRow(int(at))
self._on_exchange_changed()
self._set_feedback(
    sds.VENUE_FORM_BOUND.format(name=venue.capitalize()), "info"
)
```

Every venue the array draws binds. Driven over all six sectors with the home redirected and no venue contacted: 91 of 91 buttons that name a venue bound the form to that venue, 21 of 21 under crypto, with the Exchange row, the checked button and the feedback line all naming the one venue.

Both builds draw one panel

The Qt build and the React page read the same rows from the same surface, so the heading and the line text cannot drift apart. Read back off the drawn React page beside the Qt widgets: the heading is Exchange Status on both, and all fifteen crypto lines match character for character.

How current a colour is

Each colour is set by one event, and the panel reads the record those events leave. A colour is therefore exactly as current as the last event that wrote it.

Event	What it records	What writes it
A venue answered an authenticated call	green	Test Connection, and a running bot's first balance fetch after it connects
A venue did not answer a call	red	the connector, on the call that failed
A venue rejected the credential	grey	the connector, on the call it refused
A venue asked for a slower rate	no change	nothing is written

`src/exchange/ccxt_connector.py` - `note_venue_refusal`

```
kind = classify_venue_refusal(exc)
if kind == VENUE_LOST:
    if recorded_state(exchange_id) != CONNECTION_LOST:
        record_connection_lost(exchange_id)
elif kind == VENUE_CREDENTIAL_REJECTED:
    forget(exchange_id)
```

A slower rate writes nothing on purpose. ccxt files every rate limit under its own `NetworkError` class, so a check reading that class alone would draw a venue red every time the venue asked it to wait. One launch produced 785 such refusals.

`src/exchange/ccxt_connector.py` - `_refusal_order`

```
  (VENUE_CREDENTIAL_REJECTED, _ccxt_error_types(("AuthenticationError",))),
  (VENUE_RATE_LIMITED, _ccxt_error_types(("RateLimitExceeded", "DDoSProtection"))),
  (VENUE_ANSWERED, _ccxt_error_types(("ExchangeNotAvailable", "InvalidNonce")) + (HTTPError,)),
  (VENUE_LOST, _ccxt_error_types(("NetworkError",)) + (TimeoutError, URLError)),
```

Driven on one venue holding a green record, with every outbound socket and every name lookup refused:

Exception raised into the connector	State the store then held
a dropped connection	red
a rate limit	green, unchanged
a rejected credential	grey

Diagram: flowchart, left to right

- a bot's exchange call — _call_sync -> **classify_venue_refusal** *(refused)*
- classify_venue_refusal -> **credential_state — exchange_credential_state.json** *(did not answer)*
- classify_venue_refusal -> **nothing written** *(rate limit)*
- a bot's exchange call — _call_sync -> **credential_state — exchange_credential_state.json** *(answered)*
- credential_state — exchange_credential_state.json -> **Exchange Status** *(read on redraw)*

When a colour is stale

Every event above fires on a call to the venue. A venue nobody calls keeps the colour its last call left, so the age of a colour is the age of that call.

What the line shows	How old the colour is
green, with bots running on this venue	set when the connection opened, and set again on the first call that succeeds after one fails
green, with no bots on this venue	as old as the last Test Connection
red	the moment a call failed, and it stays until a call succeeds
grey	nothing has been recorded for this venue

A green line means no call has failed to reach this venue since the colour was recorded. It is not a promise that the venue is answering now.

The store already stamps the time of each record, and the panel does not draw it. The field is there to read.

`src/exchange/credential_state.py - recorded_at`

```
def recorded_at(exchange_id: Any, path: Any = None) -> Optional[float]:
    """When one venue was last checked, as epoch seconds, or None."""
    entry = _read(path).get(normalise_id(exchange_id))
```

The sentences the Exchange Status panel replaces

Each sentence below stands in an earlier section and no longer describes the code. The earlier text stays where it is.

Earlier sentence	What the code does now
"`_create_exchange_tab` lists the configured exchanges and offers a form to add one: the exchange picker, an API key, an API secret, and a passphrase field that appears only when the exchange needs one."	The page lists every venue the sector can trade through, coloured by its last credential check, above the same add form.
"Configured Crypto Exchanges lists what the manager returns for this wing, each entry naming its display name and its exchange id."	Exchange Status lists every venue the sector serves, each line naming the venue and its exchange id.
"The stock wing lists the equity ids instead and disables both buttons."	Every sector lists its own venues. The stock wing still disables both buttons.
"| `_remove_exchange` | Drops the selected entry |"	`_remove_exchange` drops the stored entry and its recorded check, and the line returns to grey.
"A venue that stops answering turns red through its next check."	A venue that stops answering turns red on the call that failed, with no check pressed.

2026-10-06 - Six sectors on the Exchanges page

The sector table, overtaken

OVERTAKEN, and the table above is kept as written. Coinbase is registered under all six sectors, so the three sectors recorded there as serving no venue each draw one venue row. Driven again on the real dialog, one sector at a time, with the home trees redirected and every outbound socket refused:

Sector	The table above records	Measured now
Crypto	15 rows	15 rows
Stock	10 rows	10 rows
Commodities	1 row	1 row
Forex	1 note	1 row, `Coinbase (coinbase)`
Indices	1 note	1 row, `Coinbase (coinbase)`
Futures / Perps	1 note	1 row, `Coinbase (coinbase)`

The sector and product tree records the same six registrations.

The Add box is named after the sector

The box under the list takes its title from the sector the page was opened on. The sector's display name and its venue noun fill the title, so a sector outside the crypto and stock wings is named rather than drawn as crypto.

`src/gui/main_tabs/settings_dialog_surface.py` — `add_group_title`

```
return ADD_GROUP_FORMAT.format(
    name=acs.display_name(sector), noun=acs.venue_noun(sector)
)
```

Read off the drawn Qt box beside the title the view model publishes to the React page, for all six sectors:

Sector	The box's title	Both builds agree
Crypto	Add Crypto Exchange	yes

Sector	The box's title	Both builds agree
Stock	Add Stock Broker	yes
Commodities	Add Commodities Exchange	yes
Forex	Add Forex Exchange	yes
Indices	Add Indices Exchange	yes
Futures / Perps	Add Futures / Perps Exchange	yes

Both builds call the one function, so the two titles cannot drift apart.

Which tab each route opens

Three routes reach this dialog. Driven on a live window, with the credential store empty and every outbound socket refused:

Route	Tab it opens on
Exchange ■ Add Exchange	Exchanges
An asset class's Add Exchange button	Exchanges
File ■ Settings	User

The asset class button was pressed under each of the six sectors in turn. Every press opened the Exchanges tab, and the panel listed that sector's venues. The refusal counter stayed at zero across all eight routes, against a control that moved it to one on a single deliberate reach for a socket.

2026-10-09 - Exchange Status is an array of buttons, not a list

The panel is now an array of rectangular buttons. Four buttons sit across every row. One page holds sixteen buttons. A pages button sits under the array, and it appears only when the sector serves more than sixteen venues. Neither build draws a scroll bar.

The list sentences, replaced

OVERTAKEN, quoted whole:

> *"The list at the top of the Exchanges page is headed Exchange Status."*
>
> *"Each line is a button, and pressing one points the Add form below at that venue."*

True today: the panel at the top of the Exchanges page still carries the heading Exchange Status, and it draws an array of buttons rather than a column of lines. Each button is one venue. Pressing one still points the Add form below at that venue, and it still reports the same phrase.

OVERTAKEN, quoted whole:

> *"`src/gui/main_tabs/settings_dialog_surface.py` — `exchange_status_rows`"*

True today: `exchange_status_rows` still answers every venue the sector serves, and the drawing path is now `exchange_status_page`. That function takes a page number and answers one page of buttons. Each button carries the index into `exchange_status_rows` that a press reports, so the press path did not move.

What draws the array

Both builds read one surface. Neither computes a cell of its own.

Build	What draws the array	What draws the pages button
Qt	`src/gui/settings_dialog.py`, in `_draw_venue_array`	the same function
Browser	`src/gui/web/settings_dialog.js`, in `VenueArray`	`in VenuePagesButton`

The surface answers both. `src/gui/main_tabs/settings_dialog_surface.py`, in `exchange_status_page`, gives one page of buttons. In `venue_pages_button`, it gives the control's text, its tooltip and its shown flag.

Diagram: flowchart, left to right

- venues_for_class -> **exchange_status_rows**
- exchange_status_rows -> **exchange_status_page**
- exchange_status_rows -> **venue_pages_button**
- exchange_status_page -> **Qt — _draw_venue_array**
- exchange_status_page -> **Browser — VenueArray**
- venue_pages_button -> **Qt — _draw_venue_array**
- venue_pages_button -> **Browser — VenueArray**

The shape of one page

Two figures fix the shape, and both live on the surface.

Figure	Name	Value
Buttons across a row	`VENUE_GRID_COLUMNS`	4
Buttons on a page	`VENUE_PAGE_HOLDS`	16

A short last row widens its buttons to fill the array, so the array ends with no gap. The row, the column and the span of every button come from `venue_cell`.

A page holding fewer than four buttons narrows its grid to the count it holds. `venue_grid_shape` answers that width, and both builds draw only the columns the page fills. The Qt build stretches the first of them and leaves the rest at zero. The browser build declares that many grid tracks, in `src/gui/web/settings_dialog.js`, in `venuePageColumns`.

Buttons on the page	Grid the surface answers	Spans
1	1 row, 1 column	1
3	1 row, 3 columns	1, 1, 1
5	2 rows, 4 columns	1, 1, 1, 1, 4
6	2 rows, 4 columns	1, 1, 1, 1, 2, 2
13	4 rows, 4 columns	twelve of 1, then 4
16	4 rows, 4 columns	sixteen of 1

A sector no venue serves draws its note as one disabled button. That page holds one button in a one-column grid, so the note spans the whole array. Read back off the drawn page, the note button carries no venue id, reports disabled, and draws at `repeat(1, minmax(0, 1fr))`.

The pages button appears at seventeen venues

Driven on the real surface with the venue registry standing in at each count, and with the home trees redirected:

Venues the sector serves	Pages	Pages button drawn	First page holds
15	1	no	15
16	1	no	16
17	2	yes	16
32	2	yes	16
33	3	yes	16

Sixteen venues fill one page exactly, so a sector at the limit draws neither a pages button nor a scroll bar.

Measured page counts, all six sectors

Read back off the browser page's own drawn tree, with the home trees redirected and no venue contacted:

Sector	Venues	Pages	Buttons on page 1	Buttons on page 2	Pages button
Commodities	16	1	16	—	hidden
Crypto	21	2	16	5	shown
Forex	6	1	6	—	hidden
Futures / Perps	16	1	16	—	hidden
Indices	13	1	13	—	hidden
Stocks	19	2	16	3	shown

Every page drew four columns. No page drew a scroll bar.

The pages button moves the set

The button steps to the next page, and the last page steps back to the first. `venue_next_page` answers that step, and `venue_page_offset` answers the first row index of a page.

Driven on the two sectors that fill more than one page:

Sector	Page 1 holds	Page 2 holds	Venues on both pages	Venues reached
Crypto	16	5	0	21 of 21
Stocks	16	3	0	19 of 19

No venue appears on two pages, and every venue in the sector is reachable. A control drawing the same venues on both pages is the failure this reading catches. A planted payload doing exactly that read red, so the reading can fail rather than pass on anything.

Every button carries the Sector button skin

A venue button takes the skin the Sector buttons take. The accent is that venue's own state colour. `src/gui/main_tabs/asset_class_surface.py`, in `button_style`, gives the skin, and `venue_button_style` fills it with the state colour.

The three colours keep the meanings the table above records. Green says a credential check succeeded. Red says a venue refused a check. Grey says nobody has checked.

The browser page reads the Qt style sheet through `src/gui/web/header_strip.js`, in `styleOf`. That module is now loaded on the dialog page, so the skin reaches the buttons. Read back off the drawn page, every venue button on all eight sector pages carried between eleven and thirteen style declarations, and the pages button carried eleven.

A press still points the Add form at its venue

A press reports the row index into `exchange_status_rows`, exactly as the list lines did. `src/gui/settings_dialog.py`, in `_open_credentials_for_row`, binds the Add form and writes the phrase.

Driven on the crypto sector, with every outbound socket refused:

Row pressed	Venue	The Add form bound to	The phrase reported
15	okx	`okx`	Enter credentials for Okx.
16	poloniex	`poloniex`	Enter credentials for Poloniex.
0	alpaca	nothing	Alpaca has no credential form on this tab.
20	webull	nothing	Webull has no credential form on this tab.

Row 16 is the first button of page two, and it bound the form like any other. A venue with no credential form on this tab reports so, and binds nothing.

What the browser build holds of its own

The view model answers page zero. The browser host holds the page the operator paged to. `src/gui/react_settings_dialog.py`, in `_venue_array_payload`, reads the surface at that page and puts the answer on the payload. In `_next_venue_page`, it steps the page and redraws.

What it holds	Where
The current page number	`src/gui/react_settings_dialog.py`, in `_venue_array_payload`
The step to the next page	`in _next_venue_page`
The row a press bound	`in currentRow` on `PageList`

The Qt build holds the same page number in `_venue_page` and steps it in `_next_venue_page`, so the two builds page the same way.

More sentences the Exchange Status panel replaces

Each sentence below stands in an earlier section and no longer describes the code. The earlier text stays where it is.

Earlier sentence	What the code does now
"The list at the top of the Exchanges page is headed Exchange Status."	The panel keeps that heading and draws an array of buttons, four across and sixteen to a page.
"Each line is a button, and pressing one points the Add form below at that venue."	Each button is one venue, and pressing one still points the Add form at that venue.
`"| Sector | Venues the registry answers | Rows drawn |"`	The registry answers 16 commodities, 21 crypto, 6 forex, 16 futures / perps, 13 indices and 19 stocks venues.

Earlier sentence	What the code does now
"`src/gui/main_tabs/settings_dialog_surface.py` — `exchange_status_rows`"	`exchange_status_page` draws the panel, and `exchange_status_rows` answers the whole sector behind it.
"ten redraws of the crypto panel, fifteen rows each time"	A redraw still reaches no venue. The crypto panel now draws sixteen buttons on page one and five on page two.
"all fifteen crypto lines match character for character"	Both builds draw the same twenty-one crypto venues from one surface, sixteen on page one and five on page two.

2026-10-09 - The first-run wizard offers every venue the platform holds

Page two of the setup wizard listed the venues the ccxt library carries. A venue with a hand-written connector was absent, so a venue the Exchanges tab accepts could not be chosen during first-time setup.

Both screens now read one answer. `src/gui/main_tabs/asset_class_surface.py`, in `known_venues`, holds every venue id the platform has: both crypto connector registries, `EQUITY_VENUES`, every venue `EXTRA_VENUE_CLASSES` adds a sector for, and every venue the market-rules recording holds rows for.

Diagram: flowchart, left to right

- CRYPTO_CONNECTORS -> **crypto_venues**
- SUPPORTED_EXCHANGES -> **crypto_venues**
- crypto_venues -> **known_venues**
- EQUITY_VENUES -> **known_venues**
- EXTRA_VENUE_CLASSES -> **known_venues**
- recorded_venue_classes -> **known_venues**
- known_venues -> **exchange_ids, the wizard's page two**
- known_venues -> **venues_for_class, one sector**
- venues_for_class, one sector -> **exchange_status_rows, the Exchanges tab**

What each screen offers

The wizard has no sector control, so it offers the whole set. The Exchanges tab narrows the same set to the sector the operator is on.

Sector	The tab offers	Venues the tab offered and the wizard did not, before	After
crypto	21	alpaca, ibkr, robinhood, schwab, tastytrade, webull	none
stocks	19	alpaca, etrade, fidelity (not offered on any tab), ibkr, interactivebrokers, schwab, tastytrade, tdameritrade, webull	none
commodities	16	alpaca, etrade, ibkr, schwab, tastytrade, webull	none
forex	6	ibkr	none
indices	13	alpaca, etrade, ibkr, schwab, tastytrade, webull	none

Sector	The tab offers	Venues the tab offered and the wizard did not, before	After
futures / perps	16	ibkr, schwab, tastytrade, webull	none

The wizard offered 15 venues and offers 25. Robinhood is one of them, and the operator can pick it and carry it to the credentials page.

What the wizard does with the pick

Step	Where	Driven result
The list is drawn	`src/gui/main_tabs/init_wizard_surface.py`, in `exchange_items`	25 entries, Robinhood at position 20
The pick is held	`in selected_exchange`	`robinhood`
The pick is handed on	`in get_results`	`exchange_id` reads `robinhood`
A written connector is tested	`src/exchange/api_validator.py`, in `validate_credentials`	reads `crypto_connector_class` before the ccxt path

The Qt page and the browser page draw the same list. `src/gui/init_wizard.py` fills its drop-down from `exchange_items`, and `src/gui/web/init_wizard.js`, in `exchangeItems`, reads the same entries off the payload.

A test with no venue bound

`src/gui/main_tabs/init_wizard_surface.py`, in `test_api`, set its no-venue path and then read the venue's name, which raised. It now writes `Choose an exchange first.` and returns, so no credential reaches a validator under no venue.

What did not change

`PASSPHRASE_EXCHANGES` still names bitget, kucoin and okx, and only those three draw the passphrase row and the `(requires passphrase)` note. Driven over six venues: the three show the row, and coinbase, robinhood and schwab do not.

The wizard still pre-selects the first entry of its list. That entry was binance and is now alpaca.

The venue lists this did not reach

List	Where	Why it was left
the API tester's picker	`src/gui/main_tabs/api_tester_tab_surface.py`, in `supported_exchanges`	`src/gui/widgets/api_tester_tab.py`, in `_do_connect`, builds a `CCXTConnector` for the id, so a hand-written venue would be offered and then refused
the lite bot runner's picker	`src/gui/main_tabs/live_bot_window_surface.py`, in `offered_exchange_ids`	the window drops the group box holding its pickers, and nothing builds `LiveBotWindow`
the Simulator's venue set	`src/simulator/portfolios.py`, in `crypto_venues`	it labels a sim symbol's sector and offers nothing to choose from
the bot wizard's picker	`src/gui/main_tabs/bot_wizard_surface.py`, in `exchange_items`	it lists the configured exchanges, not a registry
the visualizer's picker	`src/gui/main_tabs/bot_visualizer_surface.py`, in `exchange_items`	it lists the fleet's own exchanges, not a registry

2026-10-11 - The Verification Code row and the venue that needs one

One venue hands out no session until the operator approves this program on the venue's own web page, and that page answers with a short code. The page redirects nowhere, so nothing in this program can collect the code for him. He reads it off the page and pastes it into a row the Add form now draws.

`src/gui/main_tabs/settings_dialog_surface.py`, in `credential_form`, answers `code_row` for that venue, and both builds read that one answer. `src/stocks/etrade_connector.py`, in `VERIFICATION_CODE_VENUES`, is the set it reads, so the row drawn and the leg the connector walks cannot disagree.

What the operator does

Four steps, in this order, on the Exchanges tab.

1. Press the venue's button. The form draws an extra row, **Verification Code**, and the line under the form says what to do with it. 2. Type the API key and secret and press Add. The venue answers with a web address, which the line under the form prints. 3. Open that address, sign in at the venue, and approve this program. The venue then shows a short code. 4. Paste the code into **Verification Code** and press Add again.

Diagram: flowchart, left to right

- Press the venue button -> **credential_form answers code_row**
- credential_form answers code_row -> **Verification Code row drawn**
- Add, code row empty -> **begin_authorization, leg one**
- begin_authorization, leg one -> **the address the line prints**
- the address the line prints -> **Operator approves at the venue**
- Operator approves at the venue -> **Add, code pasted**
- Add, code pasted -> **complete_authorization, leg three**

- complete_authorization, leg three -> **the venue grants a session**

`src/gui/settings_dialog.py`, in `_connect_verification_code_venue`, holds one connector per venue while this runs, because the token leg one takes must outlive the press that walked it. `src/gui/main_tabs/settings_dialog_surface.py`, in `connect_verification_code_venue`, decides which of the two legs a press walks: an empty row walks leg one, a filled row walks leg three.

What is stored and what is not

The code is read, sent and dropped. Nothing writes it to disk, to a log, or into the answer the form reads back.

Value	Where it lives	How long
API key and secret	the credential vault, under `api_key_enc` and `api_secret_enc`	until removed
the request token	the held connector, in memory	five minutes
the verification code	nowhere	the one press that sends it
the access token	the held connector, in memory	to the end of the US Eastern day

`src/core/settings.py`, in `ExchangeConfig`, gains no field for this row. `src/trading/ata_spm_signin.py`, in `credential_store_fields`, still answers two field names for every venue the Exchanges tab offers.

Why the approved browser sign-in does not serve

`src/trading/ata_spm_signin.py` carries this program's three-legged browser sign-in, and it does not fit this venue. It is an OAuth 2.0 flow that waits on a redirect; this venue is an OAuth 1.0a flow that redirects nowhere.

What the browser sign-in assumes	What this venue does
`LoopbackReceiver` waits for the venue to redirect a browser back	`src/stocks/etrade_connector.py`, in `OAUTH_CALLBACK`, is `oob`, so the venue shows a code instead of redirecting
`code_challenge` and `new_verifier` build a proof the venue checks	the venue signs each leg instead, at `src/stocks/etrade_connector.py`, in `request_signature`
`SignInSession.approve` sends a response type, a state and a redirect address	`src/stocks/etrade_connector.py`, in `authorize_url`, takes a key and a token and nothing else
`GRANT_AUTHORIZATION_CODE` names the grant posted	`src/stocks/etrade_connector.py`, in `ACCESS_GRANT_ROUTE`, takes a verifier

The code row read on each venue

Driven on the real Qt dialog with the Exchanges tab built and the crypto wing open, pressing each venue's own button.

Venue	Needs a code	Code row drawn	Key and secret rows
etrade	yes	yes	drawn
coinbase	no	no	drawn
no venue pressed	no	no	drawn

Measured over every venue `src/gui/main_tabs/asset_class_surface.py`, in `known_venues`, offers: 1 of 26 draws the row and 25 do not. An invented venue id draws no row.

A sentence this page carries that the screen now reads differently

OVERTAKEN, quoted whole:

> `PASSPHRASE_EXCHANGES` *still names bitget, kucoin and okx, and only those three draw the passphrase row and the* `(requires passphrase)` *note.*

True today: `src/exchange/ccxt_connector.py`, in `PASSPHRASE_EXCHANGES`, names four venue ids — bitget, kucoin, okx and okxus — so four of the 26 venues draw the passphrase row and 22 do not.

2026-10-11 - The stock wing takes a credential

The Exchanges page on the stock wing drew its form and then switched every control off. `src/stocks/alpaca_connector.py`, in `BROKER_CONNECTORS`, names eight brokers and `broker_connector_class` builds one for each, so the form now settles each control against the venue the operator pressed.

Which wing a sector lands on

`src/gui/main_tabs/settings_dialog_surface.py`, in `wing_or_default`, collapses six sectors onto two wings. `CLASS_WINGS` names one mapping, stocks onto the stock wing; every other sector falls to `DEFAULT_WING`, and an unknown word answers crypto.

Sector	Wing	Venues offered	Brokers among them
Crypto	crypto	24	8
Stocks	stock	21	8
Commodities	crypto	21	8
Forex	crypto	16	6
Indices	crypto	20	8
Futures / Perps	crypto	18	5

21 venues sit on the stock wing and 99 venue-and-sector cells on the crypto wing. Every one of the eight brokers serves the stocks sector and at least one sector that lands on the crypto wing, so each one reached an enabled form on one sector and a dead form on stocks.

The seven controls, settled one at a time

`src/gui/settings_dialog.py`, in `_create_exchange_tab`, switched off seven widgets and hung one sentence on all seven. Each one now answers for itself.

Control	State	What decides it
API Key	enabled	19 of the 21 stocks venues hold a connector; each broker's own `connect` takes this row
API Secret	enabled	`src/stocks/broker_base.py`, in `open_session`, refuses an empty secret
passphrase box	enabled	4 of the 21 stocks venues are in `src/exchange/ccxt_connector.py`, in `PASSPHRASE_EXCHANGES`
passphrase row	enabled	drawn for those 4, by `src/gui/settings_dialog.py`, in `_sync_passphrase_row`
Verification Code	enabled	`src/stocks/etrade_connector.py`, in `VERIFICATION_CODE_VENUES`, names etrade

Control	State	What decides it
Test and Add	enabled for 19, refused for 2	`src/gui/main_tabs/settings_dialog_surface.py`, in `connector_held`
Test Connection	enabled for 12, refused for 9	`src/gui/main_tabs/settings_dialog_surface.py`, in `credential_route_held`

`credential_route_held` asks whether `src/exchange/api_validator.py`, in `validate_credentials`, holds a route that reaches no session: a hand-written connector's own `credential_refusal`, or a venue id `src/exchange/ccxt_connector.py`, in `SUPPORTED_EXCHANGES`, names. Nine of the 21 stocks venues hold neither — alpaca, etrade, fidelity (not offered on any tab), ibkr, interactivebrokers, schwab, tastytrade, tdameritrade, webull — and their Test Connection carries `NO_CREDENTIAL_TEST_TIP` in place of a press that cannot answer.

`connector_held` asks whether any registry builds a connector at all. `interactivebrokers` and `tdameritrade` are the two venue ids that answer no, so their Add press is refused and the feedback row carries `NO_CONNECTOR_PROMPT`.

What each broker's form fills

Broker	API Key row	API Secret row
alpaca	the key id, `src/stocks/alpaca_connector.py`, in `connect`	the secret key
etrade	the consumer key, `src/stocks/etrade_connector.py`, in `connect`	the consumer signing key
fidelity	not offered on any tab, so no form takes its key	not offered
ibkr	the bearer token, `src/stocks/ibkr_connector.py`, in `connect`	unused
robinhood	no typed row; `src/trading/ata_spm_signin.py`, in `BROWSER_AUTHORIZATION_SECTORS`, sends it to the browser press	no typed row
schwab	the bearer token, `src/stocks/schwab_connector.py`, in `connect`	unused
tastytrade	the grant's refresh token, `src/stocks/tastytrade_connector.py`, in `connect`	the client secret
webull	the app key, `src/stocks/webull_connector.py`, in `connect`	the app signing key

No broker needs a row the form did not already draw, so the row set gained nothing: 0 rows added, 0 removed.

Where the Add press stores and where it tests

Diagram: flowchart, top to bottom

- Add press -> **browser authorization?**
- browser authorization? -> **connect_browser_venue** *(yes)*
- browser authorization? -> **connector_held?** *(no)*
- connector_held? -> **NO_CONNECTOR_PROMPT, nothing stored** *(no)*
- connector_held? -> **takes_verification_code?** *(yes)*
- takes_verification_code? -> **connect_verification_code_venue** *(yes)*
- takes_verification_code? -> **credential_route_held?** *(no)*
- credential_route_held? -> **validate_credentials, then store** *(yes)*
- credential_route_held? -> **store the key and secret** *(no)*

- store the key and secret -> **the Start press opens the session**

`src/gui/settings_dialog.py`, in `_add_exchange`, ran the credential test before it stored, for every venue. A venue with no route failed that test, so no broker credential could be stored on either wing. The test now runs only where `credential_route_held` answers a route.

Driven on the real screen

The Qt Exchanges tab built on the stocks sector, each broker's own button pressed, with the home redirected and no venue contacted. `isHidden` is the row reading: an offscreen dialog answers `isVisible` false for every row, drawn or not.

Venue	Key and Secret rows	Code row	Test Connection	Add press
alpaca	drawn	absent	refused	takes a press
etrade	drawn	drawn	refused	takes a press
fidelity	not offered on any tab, so absent	absent	refused	refused
ibkr	drawn	absent	refused	takes a press
robinhood	absent	absent	refused	Connect with a Browser
schwab	drawn	absent	refused	takes a press
tastytrade	drawn	absent	refused	takes a press
webull	drawn	absent	refused	takes a press
coinbase	drawn	absent	takes a press	takes a press
okx	drawn, passphrase drawn	absent	takes a press	takes a press
interactivebrokers	drawn	absent	refused	refused
tdameritrade	drawn	absent	refused	refused

A value typed into every drawn row reads back at the consumer that uses it: the key at `_new_api_key`, the secret at `src/gui/settings_dialog.py`, in `_typed_secret`, the passphrase at `_new_passphrase` once the box is ticked, and the code at `_new_verification_code` for etrade alone. Blinding the reading flips it: disabling the key row reads disabled, and hiding the code row reads absent.

The Add press driven on webull stored the key and the secret, and both read back out of `src/core/settings.py`, in `SettingsManager.get_exchange`, decrypted with `src/core/encryption.py`, in `vault_phrase`. `validate_credentials` was called 0 times on that press and 1 time on the same press for coinbase, so the route reading is not blind. The same press on tdameritrade is refused, writes the no-connector sentence and stores nothing.

The React store draws the same form

`src/gui/main_tabs/settings_dialog_surface.py`, in `SettingsDialogModel._setup`, switched the same seven controls off over `STOCK_DISABLED_CONTROLS`. `on_exchange_changed` now reads one `credential_form` answer for the row set, both presses and the Add button's words, which is the answer `_sync_credential_kind` reads.

Three readings moved with it:

- `show_key_rows` hides the API Key, API Secret and passphrase box for a browser-authorization venue, which the Qt build already did and the store did not.

- `bind_venue_row` writes `venue_form_bound`'s sentence, the one `src/gui/settings_dialog.py`, in `_open_credentials_for_row`, writes.
- `src/gui/react_settings_dialog.py`, in `PagePress`, takes `setText` and `setToolTip`. `_sync_credential_kind` already called both, and the holder carried neither, so a venue press raised on the React build before it drew anything. `src/gui/web/settings_dialog.js`, in `DialogButton`, reads the texts bag before its layout node's words, so the Add button draws the words the store holds.

Sentences this page carries that the screen now reads differently

OVERTAKEN, quoted whole:

> *The same page in the stock wing shows a banner and disables Add and Test.*

True today: the stock wing shows a banner and settles each control against the pressed venue. Add takes a press on 19 of the 21 stocks venues and Test Connection on 12.

OVERTAKEN, quoted whole:

> `EQUITY_EXCHANGE_IDS` *fills the list with the planned brokers, each labelled as planned. Crypto is the wing that trades today.*

True today: `src/gui/main_tabs/asset_class_surface.py`, in `venues_for_class`, fills the list, and `BROKER_CONNECTORS` holds a built connector for all eight brokers on it. No venue is labelled planned.

OVERTAKEN, quoted whole:

> *The same page in the stock wing shows a banner and disables Add and Test, because no broker connector ships yet.*

True today: eight broker connectors ship, and the page refuses only the two presses a venue cannot answer.

OVERTAKEN, quoted whole:

> *The stock wing lists the equity ids instead and disables both buttons.*

True today: the stock wing lists the 21 venues the stocks sector serves, and both buttons are refused only on the venue ids that hold no connector.

OVERTAKEN, quoted whole:

> *The passphrase checkbox appears for the three venues above.*

True today: `PASSPHRASE_EXCHANGES` names four venue ids — bitget, kucoin, okx and okxus — so the box is ticked for four venues.

OVERTAKEN, quoted whole:

> *A press sets that box for the three venues that need one and clears it for the rest.*

True today: a press sets the box for those same four venues and clears it for the rest.

OVERTAKEN, quoted whole:

```
_banner = QLabel(
    "<b>Stock Wing:</b> equity-broker integration is "
    "queued — no live brokers are wired up yet. The "
    "list below shows the planned brokers; Add / Test "
    "are disabled until the broker connectors ship. "
    "Use the Crypto Wing for active trading today."
)
```

True today: the banner reads from `STOCK_BANNER_TEXT`, which names the eight brokers, says the form takes each one's key and secret, and says Test Connection answers where a credential can be checked without a session. The page carries that block twice and both are overtaken by this one sentence.

OVERTAKEN, and the paragraph above is kept as written. The banner now reads from `stock_banner_text`, which counts and names the brokers `offered_brokers` answers. Seven are named, and Fidelity is not one of them, because no tab offers it.

```
# src/gui/main_tabs/settings_dialog_surface.py, in offered_brokers
    listed = acs.venues_for_class(class_of_wing(STOCK_WING))
    return tuple(sorted(one for one in listed if broker_connector_class(one)))
```

OVERTAKEN, quoted whole:

```
self._set_feedback(
    sds.VENUE_FORM_BOUND.format(name=venue.capitalize()), "info"
)
```

True today: `_open_credentials_for_row` writes `sds.venue_form_bound(venue)`, which reads `credential_form` and so names the credential the pressed venue takes. `VENUE_FORM_BOUND` is still the constant this page cites and no caller formats it.

2026-10-16 - The lite bot runner keeps its pickers

The lite bot runner built its configuration grid on a group box the builder then dropped. The toolkit freed that box together with the six fields it owned, while the window was still being built, and printed one line about it. The grid now stays on the one group box that is returned, and the START and STOP row sits in that grid as its third row. `src/gui/live_bot_window.py`, in `_build_controls`, returns that box.

Driven on the real window, shown and read back: the venue picker, the base picker, the target-asset picker, the target-dollar box and the two credential fields all answer after construction, and they answer again after the window is closed and reopened. A value typed into each reaches `src/trading/container/config.py`, in `make_bot_config`, and the two credentials reach `src/exchange/ccxt_connector.py`, in `CCXTConnector.connect`. The toolkit's refusal line is gone from the build.

`src/gui/main_tabs/live_bot_window_surface.py`, in `build_view_model`, publishes the same reading to the React side: one configuration group, thirteen grid cells in the window's own order, and `controls_reach_the_window` True. Both builds were driven with the same six values and handed their worker the same seven arguments.

Nothing builds this window. `LiteLiveBotWindow` is constructed only by `src/gui/live_bot_window.py`, in `main`, which is the module's own standalone entry point, and nothing asks the bridge for `live_bot_window.state`. The repair gives the six fields back. It does not give the operator a way in.

OVERTAKEN, quoted whole:

the lite bot runner's picker	`src/gui/main_tabs/live_bot_window_surface.py`**, in** `offered_exchange_ids`	**the window drops the group box holding its pickers, and nothing builds** `LiveBotWindow`

True today: the window keeps the group box holding its pickers. Nothing builds the window, and that half alone is why the venue list did not reach this picker.

A stored figure no control can hold draws a dash

Every numeric control on this window reads its stored figure through one admission before the control sees it. `src/trading/container/config.py, in as_box_figure` is that admission, and it refuses six kinds: a stored flag, a text number, a not-a-number, an endless number, an integer too large for a decimal, and a finite figure outside the control's own span.

A refused figure leaves the control at the bottom of its span drawing the dash the indicator panel draws for a cell it cannot read. The log names the key. Save then **leaves the stored figure alone** until the operator sets one, so a figure nobody entered is never written.

The three rows, drawn and driven, in both builds

The window was built against a settings file under a throwaway home. Each kind went in, every control was read back, Save was pressed with nothing touched, and the file was read back.

Stored value	Target balance draws	Save writes
`250.0`	`$250.00`	`250.0`, unchanged
`250`	`$250.00`	`250.0`
`"250"`	the dash	nothing; `"250"` stays
`"abc"`	the dash	nothing; `"abc"` stays
`True`	the dash	nothing; `True` stays
an integer of 400 digits	the dash	nothing; it stays
a not-a-number	the dash	nothing; it stays
an endless number	the dash	nothing; it stays

Lock candle count answers the same way, and a stored `250` on that control now draws the dash rather than the top of its span. An unknown theme name leaves the build's theme selected, names itself in the log, and Save keeps the stored name.

Twenty-seven rows were driven across the three settings and both builds. Save changed nothing on all twenty-seven, and the two builds agreed on every row.

The control that proves the reading

Moving a control and then pressing Save writes the figure the operator set: target balance `2.0`, lock candle count 2, theme `glass_metal`. So the twenty-seven unchanged rows are a reading that could have reported the other answer.

Where each piece sits

What	Where
the admission	`src/trading/container/config.py, in as_box_figure`
the figure a decimal control may hold	`src/gui/main_tabs/settings_dialog_surface.py, in seed_decimal`
the figure a whole-number control may hold	`src/gui/main_tabs/settings_dialog_surface.py, in seed_whole`
the writer that admits before it writes	`src/gui/settings_dialog.py, in _put_stored`
the refusal, logged and marked	`src/gui/settings_dialog.py, in _show_stored`
the dash drawn on the control	`src/gui/settings_dialog.py, in _mark_refused`
the row Save leaves alone	`src/gui/settings_dialog.py, in _untouched_since_refusal`
the same reading for the React page	`src/gui/main_tabs/settings_dialog_surface.py, in _load_stored`

2026-10-16 - The faults a control reader may raise

A reader that cannot answer a control raises, and the load and save path catches a named set rather than every exception. The dialog names that set once and the page surface names the same five faults again, so the two builds pass over the same refusals and neither swallows a fault of another kind.

`src/gui/settings_dialog.py` - `READER_FAULTS`

```
  READER_FAULTS = (LookupError, TypeError, ValueError, ArithmeticError, AttributeError)
```

Three readers catch that set. The refusal mark and the row Save leaves alone catch it in the dialog, and the page surface catches it in its own reading. Each one marks the one control it read and leaves the rest of the window drawn.

The writer that admits a figure before it writes is a class rather than a closure, so the control it marks is a plain attribute on the writer and reaching that control needs no override, in `src/gui/settings_dialog.py`, in `_StoredWriter`.

2026-10-18 - The launch line reports what the last call answered

Launch writes one Activity Log line for each configured exchange. The line used to say a credential was stored and unverified. It now says what the venue's last authenticated call answered, and it still contacts nothing.

`src/gui/main_window.py, in _report_stored_credentials_on_startup` reads the settings entry for the stored blob and the record `src/exchange/credential_state.py` keeps. Reading that record places no venue call.

What the record holds	The line reads	Level
the venue answered	credentials stored, accepted at a date and time	info
the venue did not answer	credentials stored, the venue did not answer at a date and time	warning
nothing	credentials stored, no answer on record	info

The third row covers two cases and the line says so rather than claiming the venue was never asked. `src/exchange/ccxt_connector.py, in note_venue_refusal` drops the record when a venue rejects a credential, so a rejected key and a venue nobody has called read the same.

The authenticated call launch already places

Nothing is pressed for it.

Step	Where
the saved bots are restored	`main.py, in main`
the splash is handed the fleet-start callback	`main.py, in main`
the consent dialog is skipped where this machine already owns the fleet	`src/core/instance_guard.py, in authorise_auto_start`
every restored bot is started, one at a time	`src/gui/main_window.py, in run_fleet_sequence`
the venue's client is built with the decrypted key, and markets load	`src/exchange/ccxt_connector.py, in sync_connect`
the account balance is read on that client	`src/gui/main_window.py, in _connect_exchange_for_bot`
the account's own trade history is read, on its own thread	`src/exchange/ccxt_connector.py, in _scan_trade_history`

The splash runs 8.5 s of phases and then waits 750 ms, so the fleet start fires about nine seconds after launch. `main.py` holds a 9500 ms fallback timer for the same callback.

Diagram: flowchart, left to right

- launch -> **the startup line**
- credential_state — exchange_credential_state.json -> **the startup line** *(read, no venue call)*
- launch -> **run_fleet_sequence — about nine seconds later**
- run_fleet_sequence — about nine seconds later -> **_connect_exchange_for_bot — balance read**
- _connect_exchange_for_bot — balance read -> **credential_state — exchange_credential_state.json** *(answered or refused)*

The launch balance read now reaches the record

That read does not pass through `src/exchange/ccxt_connector.py, in _call_sync`, so neither recorder saw it and its answer was written nowhere. It now records an answer on the way out, and a refusal in the handler that already catches one. No venue call is added: the call was already being placed.

A key revoked between two sessions is refused by that read. The record kept the green its last good session left.

The sentence this section replaces

The earlier text stays where it is.

Earlier sentence	What the code does now
"the startup credential report | Gemini stored and unverified, Kraken with none"	The report names what each venue's last authenticated call answered, or that no answer is on record. A venue holding no credential still reports none.

2026-10-18 - A format check records a format check

A green on the Exchange Status panel means a venue answered an authenticated call. Nothing else writes a green.

A venue with its own written connector carries no ccxt entry, so pressing Test Connection for it fetches no market and reads no balance. The check that runs instead reads the key and loads the secret as a signing seed, and it contacts nothing.

`src/exchange/api_validator.py, in hand_written_result` — the check that asks no venue

```
  refusal = connector.credential_refusal(api_key, api_secret, passphrase)
```

That check used to call the recorder whose contract is "one venue answered an authenticated call", so the venue drew green on the shape of the credential alone. Its own message says what it did: no venue was contacted, no market count and no balance were read.

The record carries a third state

`record_format_checked` holds what a format check learns. It refuses where the record already holds a state only a venue's own answer writes, so a shape check can neither replace a green nor clear a red.

`src/exchange/credential_state.py, in record_format_checked`

```
  if recorded_state(exchange_id, path) in VENUE_ANSWERS:
      return False
  return _record(exchange_id, FORMAT_CHECKED, when, path)
```

`VENUE_ANSWERS` holds the two states a venue writes. `RECORDED_STATES` is those two and the format check.

What the panel draws

The third state draws the resting colour, never green. A reading the platform could not take is drawn as an absence, and the words carry what happened. The panel still paints three colours.

The credential	The record	The row	The words
its shape passes and no venue was asked	`format_checked`	the resting colour	credential format checked, no venue asked
a venue accepted it	`validated`	green	credentials validated
a venue refused it	nothing	the resting colour	no valid credentials: never asked, or the venue refused one
none is stored	nothing	the resting colour	no valid credentials: never asked, or the venue refused one

`src/gui/main_tabs/settings_dialog_surface.py` — the words each state draws

```
VENUE_STATE_WORDS = {
    VENUE_STATE_VALIDATED: "credentials validated",
    VENUE_STATE_LOST: "API connection lost",
    VENUE_STATE_FORMAT_CHECKED: "credential format checked, no venue asked",
    VENUE_STATE_UNSET: "no valid credentials: never asked, or the venue refused one",
}
```

The last two rows read alike on purpose. `src/exchange/ccxt_connector.py, in note_venue_refusal` drops the record when a venue rejects a credential, so a rejected key and a venue nobody has called hold the same record. The words name both causes rather than letting one colour stand for both.

Driven on the real path

Seven credentials through `src/exchange/api_validator.py, in validate_credentials`, with the home redirected and the venue replaced by a double that answers or refuses in process. No venue was contacted.

The credential	The row drew before	The row draws now
its shape passes, no venue asked	green, credentials validated	the resting colour, credential format checked, no venue asked
a venue accepted it	green, credentials validated	green, credentials validated
a venue refused it	the resting colour	the resting colour, with both causes named
none is stored	the resting colour	the resting colour, with both causes named
a green on record, then a format check	green	green
a red on record, then a format check	green	red
a shape the check refuses	the resting colour	the resting colour

Row two is the control. A credential a venue accepted still draws green.

Row six was a second false reading. The format check called the recorder that clears a red and sends the restored-connection alert, so a shape check turned a venue's own refusal green and reported the connection restored. The red now stands until a venue answers.

The startup line reads the same record

`src/gui/main_window.py, in _report_stored_credentials_on_startup` writes one Activity Log line for each configured exchange from this record. A format check leaves no venue answer, so that line reports an absence.

Driven on the real method with the window's log, spool and settings replaced by doubles, and the home redirected:

The record	The line	Level
`validated`	credentials stored, accepted at a date and time	info

The record	The line	Level
`lost`	credentials stored, the venue did not answer at a date and time	warning
`format_checked`	credentials stored, no answer on record	info
nothing	credentials stored, no answer on record	info

The detail beside the accepted line used to say that a venue with its own connector records a green on the credential alone. That is what this change stopped, so the detail now says only a venue answering an authenticated call records it. The detail beside the third and fourth rows names all three of their causes.

Two sentences the third state replaces

Each sentence below stands in an earlier section and no longer describes the code. The earlier text stays where it is.

Earlier sentence	What the code does now
"Grey is the resting colour. It says nobody has checked, not that something failed."	Grey is still the resting colour and still says nothing failed. It now covers a venue nobody has asked and a credential whose format was checked, and the row's words tell the two apart.
"grey | nothing has been recorded for this venue"	Grey covers a venue with nothing on record and a venue holding a format check. Neither is a venue answer, so both draw the resting colour.

2026-10-18 - A Username change asks before it reseals the stored secrets

The name in the Username row is the passphrase every stored venue secret is sealed under. A Save that changes that name over a stored credential now puts the consequence on screen and waits for an answer. Nothing reseals without the press, and an answer of No leaves both the name and the stored secrets exactly as they were.

One reading, read by both save paths

`src/gui/main_tabs/settings_dialog_surface.py`, in `username_rename_consequence`, answers whether a Save would orphan anything. It calls the case dangerous only where the name changed **and** a venue holds a sealed token, which is the one case that leaves a secret under a phrase nothing rebuilds. `credential_bearing_venues` names those venues, reading the same `CREDENTIAL_FIELDS` per entry that `stored_credential_phrase` reads. No token is opened to answer, and no value is read.

`src/gui/settings_dialog.py`, in `_settle_username_rename`, reads that answer and puts the question. It runs before `_save` walks its write loop, so the loop writes whichever name the answer left in the row.

`src/gui/main_tabs/settings_dialog_surface.py`, in `SettingsDialogModel._hold_username_over_credentials`, reads the same answer. The Qt-free model carries no press, so it refuses that one case: the stored name goes back into the row and the question is recorded as a box.

What each answer does

The answer	The stored name	The stored secrets
Yes	the new name is written	every sealed token is resealed under the new phrase
No	the stored name goes back in the row	untouched
the reseal could not finish	the stored name goes back in the row	untouched
the question could not be put	the row is skipped for that Save	untouched

The reseal covers both stores a phrase keys

`src/core/encryption.py`, in `rekey_credential_stores`, is the one caller-facing function. It covers the `_enc` tokens the settings store holds per venue and the `FileVault` the ATA-SPM pages push through. `rekeyed_token` opens one token under the old phrase and seals it again under the new one.

Each venue is opened and resealed whole before anything is written, and the settings store takes one write, so a venue the old phrase cannot open keeps the tokens it had and is named on the Activity Log. A vault holding no entry writes no file, so an install that never pushed through the ATA-SPM pages gains no file from a rename.

Diagram: flowchart, top to bottom

- Press Save on the User page -> **username_rename_consequence**
- username_rename_consequence -> **Write the row** *(the name is the same)*
- username_rename_consequence -> **Write the row** *(no venue holds a token)*
- username_rename_consequence -> **Put the question on screen** *(a venue holds a token)*
- Put the question on screen -> **Put the stored name back** *(No)*
- Put the question on screen -> **rekey_credential_stores** *(Yes)*
- rekey_credential_stores -> **The settings store, one write**
- rekey_credential_stores -> **The FileVault, written only where one entry moved**
- rekey_credential_stores -> **Write the row**

Driven on the real method

The press is the one outward edge replaced, and the replacement counts its calls, so a run that never put the question cannot read as a run that did. Everything else is real: the surface reading, `rekey_credential_stores`, and a `SettingsManager` in a temporary directory holding one venue with three sealed tokens. The home was redirected before the first import and `Path.home()` was printed from inside the run. No stored setting of the running install was read, no credential file was opened and no venue was contacted.

#	Reading	Result
-	-------	------
1	a secret sealed under one name, opened under another	refused
2	**control** — a secret sealed and opened under the same name	opens
3	every stored token before the rename	opens
4	the question was put	once

#	Reading	Result
5	the row after an answer of Yes	carries the new name
6	every stored token after an answer of Yes	opens
7	the row after an answer of No	carries the stored name
8	every stored token after an answer of No	opens
9	**control** — the question on an unchanged row	not put
10	**control** — the question on a store holding no credential	not put
11	the vault file after a rename over an empty vault	not written
12	the Qt-free model over a stored credential	the row put back, one box recorded

Rows 9 and 10 are the controls for the question. They prove the box appears on the one case that needs it and on no other, so rows 4 to 8 are the question working and not a box on every Save.

The empty row

`src/core/encryption.py`, in `vault_phrase`, substitutes `UNNAMED_OPERATOR` for an empty name, so an install whose row was never filled seals under that one phrase. `main.py` reads the stored name on a launch and writes nothing when it is empty, so that install is reachable from a fresh one. Filling the row later is a rename like any other, and the question is put the same way.

Four sentences this change and the one declaration replace

Each sentence below stands in an earlier section and no longer describes the code. The earlier text stays where it is.

Earlier sentence	What the code does now
"The declared default is an empty string, and the settings object always declares the field, so the fallback written into the phrase never fires. An install where the row was never filled encrypts under a phrase with an empty middle."	The fallback fires. `vault_phrase` reads the declared empty string and substitutes `UNNAMED_OPERATOR`, so an install whose row was never filled seals under that one phrase, not under a phrase with an empty middle.
"Change the name on a machine that already holds credentials and the stored secrets stop opening, because the connect path rebuilds the same phrase from the new name."	A Save that changes the name over a stored credential puts the consequence on screen first. On Yes every sealed token is resealed under the new phrase; on No the stored name goes back in the row. The stored secrets open either way.
"Three sites write that phrase out as a literal, one to encrypt and two to decrypt, and a fourth declares the format none of them reads."	One declaration, `MASTER_FORMAT`, is filled by one function, `vault_phrase`. Eight sites call that function: one seals, one seals and opens, five open, and one is the Qt-free model's own phrase builder. No site writes the format out.
"`src/gui/main_tabs/settings_dialog_surface.py` — the declaration"	The declaration is in `src/core/encryption.py`, beside `UNNAMED_OPERATOR` and `vault_phrase`. The surface reads that function and declares no format of its own.

The sentences the declared derivation cost replaces

Each sentence below stands in an earlier section and no longer describes the code. The earlier text stays where it is.

Earlier sentence	What the code does now
"`src/core/encryption.py` — `derive_key`", and the signature under it, in both of the two sections that carry it	`derive_key` takes a third argument, a keyword-only `iterations`, and runs the declared `KDF_ITERATIONS` where it is left out. A seal that recorded an earlier cost hands back the cost it recorded.
"`src/core/encryption.py` - the declaration and the fallback", and the `vault_phrase` body under it that returns `MASTER_FORMAT` alone	`vault_phrase` reads `held_master_key` first and answers `master_key_phrase` of the held key where one is held. Where none is held it returns `MASTER_FORMAT` on the stored name exactly as the block shows, which is every install no password has been set on.

2026-10-22 - One declaration opens both Phantom Bots boxes

The master box and the lock box each had their opening figure typed into several files. The master box was the one that disagreed: five files said on and four said off, and a wizard handed an empty bag of defaults opened the box clear while the Settings page opened it ticked.

Both figures are now declared once, in the settings module, and every screen reads that one name.

```
# src/core/settings.py, the two declarations
DEFAULT_ENABLE_PHANTOMS = True

DEFAULT_LOCK_CANDLE_COUNT = 2


@dataclass
class AppSettings:
    default_enable_phantoms: bool = DEFAULT_ENABLE_PHANTOMS
    default_lock_candle_count: int = DEFAULT_LOCK_CANDLE_COUNT
```

On is the figure kept. It is what the store's own schema already declared, and `SettingsManager.get_all` answers it on a machine with no settings file, so every wizard the main window opens has always opened with the box ticked. Two is the figure kept for the lock box, because every site already read two.

Where each figure used to be written down

Nine sites named the master box default and four of them disagreed with the store. Seven sites named the lock figure and all seven agreed by luck.

Screen or module	Master box	Lock box
`src/core/settings.py`, in `AppSettings`	the declaration	the declaration
`src/gui/main_tabs/settings_dialog_surface.py`, `PERSISTED_ROWS`	reads it	reads it
`src/gui/main_tabs/settings_dialog_surface.py`, `CONTROL_SPECS`	reads it	reads it
`src/gui/settings_dialog.py`, in `_create_phantom_tab`	reads it	reads it
`src/gui/settings_dialog.py`, in `_stored_rows`	reads it	reads it
`src/gui/main_tabs/bot_wizard_surface.py`, `CHECK_FIELDS` and `NUMBER_FIELDS`	reads it	reads it
`src/gui/bot_wizard.py`, in `PhantomConfigPage`	reads it	reads it
`src/gui/paper/paper_bot_wizard.py`, in `PaperPhantomConfigPage`	reads it	reads it

Screen or module	Master box	Lock box
`src/gui/simulator/sim_bot_wizard.py`, in `SimPhantomConfigPage`	reads it	reads it
`src/gui/live_settings/phantom_bots_tab.py`, in `PhantomBotsTabMixin`	from the bot	reads it
`src/gui/paper/paper_phantom_bots_tab.py`	from the bot	reads it
`src/gui/simulator/sim_phantom_bots_tab.py`	from the bot	reads it
`src/trading/phantom_balance.py`, in `TimeframeCoordinator`	-	reads it
`src/trading/scrumming_bot.py`, in `ScrummingBot`	reads it	from its coordinator
`src/trading/container/restore.py`, in `restore_bots_from_state`	reads it	from the saved record

What moving the one declaration does

The declaration was set to off and seven, two figures no site held, and every screen was built again. Fourteen lock readings moved from two to seven together, and the master box went clear on all four wizard surfaces and on the Settings page. Neither figure moved anywhere before the change.

```
no store, declaration at on and 2      three wizards on, lock 2; Settings on, lock 2
no store, declaration at off and 7     three wizards clear, lock 7; Settings clear, lock 7
```

A stored figure still wins

A store holding off, nine and 4h opens all three wizards and the Settings page at off, nine and 4h, whatever the declaration says. A running bot carrying a saved nine shows nine on its own Phantom Bots tab on Live, Paper and the Simulator.

```
declaration off and 7, store on and 9     every wizard opens on, lock 9
declaration off and 7, bot saved 9        every running-bot page shows 9
```

Two sentences this one declaration leaves standing

Each sentence below stands in an earlier section and no longer describes the code. The earlier text stays where it is.

Earlier sentence	What the code does now
"The one spin box on the page, inside its own group. It opens at 2 and Save reads it no more than it reads the other two."	It still opens at 2, and the 2 is now the one declared figure rather than a figure typed into this page. Save reads it and stores it.
The Settings-row table whose Lock duration row shows an opened-at of 2, and whose master box row above it shows an opened-at of clear	The opened-at column is the one declared figure for both rows: ticked for the master box, 2 for the lock box. The stored column is unchanged.

2026-10-23 - Eighteen citations on this page name their module

Eighteen citations in this page's own prose named a line number. The code above each of those lines has moved, and the site each citation named has since been removed, so none of the eighteen can name a live symbol. Each now names its module, and the words beside it still name the subject. No sentence above this one changed.

The fold-ladder helper and the target-growth module are already answered in the orphaned target-growth module. The two subjects below had no answer anywhere on the page, and these are it.

The stored-group reader sites

Earlier citation	What is there now
the eight sites under Every reader of the stored group, each named with a line number	No site is in the tree. The bare name `profit_folding` returns no hit in any of the three modules those rows name. Each row now names its module with no line number
the Settings row and the wizard box above them, each named with a line number	Both pages are removed, which the rows on the removal above already state. Each now names its module

The unreachable-page constant

Earlier citation	What is there now
the named constant on the Qt-free wizard surface, held at a line number	The constant is in no file under `src/`. That surface declares no page list, and the folding page the constant named is removed with the rest of the wizard's folding path

2026-10-24 - One declaration names each venue field and its default

Adding a venue writes a record of eight fields. The field names and the field defaults were two separate lists matched by their position, and the two lists sat apart with other names between them. Inserting one field into one list alone moved every later default by one place, and nothing reported it.

Driven on the real Add Exchange step with one field inserted ahead of the rest, the venue was added switched off, its hardware mode held an empty string, and its stored passphrase field held the word true. Nothing raised.

`src/gui/main_tabs/settings_dialog_surface.py` — one row per field

```
EXCHANGE_CONFIG_ROW: tuple[tuple[str, Any], ...] = (
    ("exchange_id", EMPTY_TEXT),
    ("display_name", EMPTY_TEXT),
    (KEY_FIELD, EMPTY_TEXT),
    (SIGNING_FIELD, EMPTY_TEXT),
    (PHRASE_FIELD, EMPTY_TEXT),
    ("enabled", True),
    ("hardware_mode", False),
    ("hw_volume_serial", EMPTY_TEXT),
)
```

`src/gui/main_tabs/settings_dialog_surface.py, in add_exchange` builds the record from that one list, so each field carries its own default whatever order the list holds. The same insertion driven again left

every field on its own default, and a venue added with no insertion is unchanged.

The four stored groups were already addressed by name. Saving with the group list reversed writes exactly what saving in the declared order writes, measured on this page's own save path. The reading is shown able to report by a third arrangement whose groups are paired with the wrong rows.

```
declared order   ai_monitor, ta_indicator_weights, sound, message_channels
reversed order   the same four keys hold the same rows
mis-paired list  the AI rows land under the weights key, and it is seen
```

A stored settings file written before this change still loads. Sixteen stored rows across the four groups and the Username row each reached their own control, before and after, with the read-back identical.

2026-10-25 - #665 - a stored choice outside its list says so on the control

Settings outlive the lists they were chosen from. A word stored under `bot_visibility` that the Bot Visibility list no longer offers used to raise two different faults on the two builds. The Qt window raised a refusal naming the word. The running React window raised a missing-method fault instead, because the one reader both windows share asked the drop-down whether it takes typed text, and only the Qt drop-down answers that question.

`src/gui/settings_dialog.py` — the reader asks whether the control can answer

```
def _combo_takes_typed_text(combo: object) -> bool:
    """Whether ``combo`` answers ``isEditable`` and reports it takes typed text.

    The page-backed drop-down draws a fixed list and declares no such method,
    so a control that cannot answer is read as taking no typed text.
    """
    asked = getattr(combo, "isEditable", None)
    return bool(asked()) if callable(asked) else False
```

`src/gui/react_settings_dialog.py, in PageCombo` now answers the same question with False, because the page draws a fixed list. Both windows therefore take one route for a word the list does not hold, and the word on the control's refusal is the stored word.

The operator is told on the control. A refused choice hangs a sentence on the drop-down itself, which both windows draw as the control's hover text. The sentence is declared once, in `src/gui/main_tabs/settings_dialog_surface.py, in refused_item_tooltip`, and names the stored key rather than a figure, because a list of words is no figure.

```
stored bot_visibility 'both'
  Qt     box opens on 'orderbook'   hover text names bot_visibility
  React  box opens on 'orderbook'   hover text names bot_visibility
  both   LookupError: this list holds no row for both

stored bot_visibility 'internal'
  Qt     box opens on 'internal'    no hover text, no refusal
  React  box opens on 'internal'    no hover text, no refusal
```

Save leaves the stored word alone while the box still holds what the refusal left in it, so the word on disk is not replaced by a choice the operator never made.

The colour a refused number is drawn in does not reach a drop-down on the React window. `src/gui/web/settings_dialog.js, in controlProps` draws the hover text for every control and reads no style for one, so the sentence is the channel both windows share.

A settings file written before this change still loads. Eighteen declared top-level fields read back with none differing, all five stored groups read back with their own field counts, and the out-of-list word survives on disk.

2026-10-25 - #665 - the AI Monitor's opening wait is declared once

Row 262. The four-hour wait the Check interval box opens on was written in seven places. Six of the seven were copies, and one copy held the whole number four where the other six held four point zero.

One declaration now serves every site. It is the stored field, and the page, both windows and the monitor all read it.

`src/core/settings.py, in AIMonitorSettings`

```
    api_key: str = ""
    interval_hours: float = 4.0
    connect_phrase: str = ""
```

The page builds that group's own opening figures once and reads the wait off them, the way it already reads the message channel's figures.

`src/gui/main_tabs/settings_dialog_surface.py, in AI_LOAD_KEYS`

```
    AI_BUILT = AIMonitorSettings()

    AI_LOAD_KEYS = (
        ("api_key", "", "ai_api_key"),
        ("interval_hours", AI_BUILT.interval_hours, "ai_interval"),
        ("connect_phrase", "", "ai_connect_phrase"),
    )
```

The same figure opens the box on both windows and stands behind the Save and load row, so the Qt window and the React window cannot disagree about it.

`src/gui/settings_dialog.py, in _create_ai_monitor_tab`

```
    self._ai_interval.setRange(0.5, 24.0)
    self._ai_interval.setValue(sds.AI_BUILT.interval_hours)
    self._ai_interval.setSuffix(" hours")
```

The monitor's own name for the wait is now a read of that field rather than a second figure, so `wait_hours` refuses a stored figure against the same number the box opens on.

`src/trading/live_monitor.py, in LiveMonitor`

```
DEFAULT_INTERVAL_HOURS = AIMonitorSettings.interval_hours
```

One line on the Console reads differently. A stored AI Monitor group that carries the switch and the key but no interval used to print the whole number; it now prints the same four point zero every other path already printed.

```
stored group with no interval
  before   AI Monitor enabled (phrase: '...', interval: 4h)
  after    AI Monitor enabled (phrase: '...', interval: 4.0h)

stored group carrying 4.0
  before   AI Monitor enabled (phrase: '...', interval: 4.0h)
  after    AI Monitor enabled (phrase: '...', interval: 4.0h)
```

Driven with the one declaration moved to eleven hours, every site followed it together, and the box span and the handshake switch beside it did not move.

```
the one declaration moved 4.0 -> 11.0
  stored field                 11.0
  monitor's own name           11.0
  the wait a bad figure keeps  11.0
  page load fallback           11.0
  React control opens on       11.0
  Qt control opens on          11.0
  Qt Save and load fallback    11.0
  Console line, no interval    11.0h
  box span                     (0.5, 24.0)   unmoved
  handshake switch             True          unmoved
```

Before the change the same move reached the monitor alone and left five copies and the Console line at four. Every file is byte identical after the move was put back.

2026-10-26 - #665 - the venue record, the add box and the retired mode's name

The Add press hands the store the record the store declares. The Qt-free model built a plain dictionary of eight names. The store's own method takes a declared record of ten, and converts it with a call that only a declared record answers. Nothing reached that path, so nothing had failed yet; driven against the real store it raised and stored no entry.

`src/gui/main_tabs/settings_dialog_surface.py, in SettingsDialogModel.add_exchange`

```
config = ExchangeConfig(exchange_id=eid, display_name=eid.capitalize())
```

The model now builds the same record the Qt press builds, so the two field sets cannot drift apart again. The page's stand-in store takes the declared record too, which keeps it from accepting a shape the real store refuses.

```
                            before                      after
what the model hands        a dictionary of 8 names     the declared record
the real store's answer     raised, 0 entries stored    1 entry stored
the two MCP token fields    absent from the model       carried by the record
```

The add box no longer promises a tab that is already there. The box text ended with one sentence whatever the bar held. A venue already drawn in the Configured list got the same promise as a venue the bar had never seen.

`src/gui/main_tabs/settings_dialog_surface.py, in added_box_text`

```
ADDED_BOX_TAB_NEW = "The exchange tab will appear in the main window."
ADDED_BOX_TAB_HELD = "The exchange tab is already in the main window."
```

Both builds read that one answer, because the React dialog inherits the Qt press. The Remove Selected line now names the tab as well; before, only the Activity Log said a tab had gone.

```
add, venue not listed      Alpaca has been added without credentials.
                           The exchange tab will appear in the main window.
add, venue already listed  Alpaca has been added without credentials.
                           The exchange tab is already in the main window.
removal                    Alpaca removed. Its exchange tab comes off the
                           main window.
```

The retired mode's name is off every surface that advertised it. Nine files outside the bot wizard named a mode the engine does not declare. Each is a write site: a card, an About box, a tool tip, a log line or a package docstring.

site	what it said
`src/gui/launcher.py`	both mode cards, now reading the surface's own text
`src/gui/main_tabs/launcher_surface.py`	the crypto subtitle, a feature line and the stocks features
`src/gui/main_tabs/main_window_surface.py`	the About box and the abbreviation tool tips
`src/gui/main_window.py`	the abbreviation tool tips, a bot-ready record and the Manual Fire refusal
`src/gui/main_tabs/stock_main_window_surface.py`	the stock About box
`src/stocks/stock_bot.py`	a strategy line and two settings its own mode list never offered
`src/gui/main_tabs/settings_dialog_surface.py`	the water-drip tool tip
`src/gui/settings_dialog.py`	the same tool tip, second copy
`src/exchange/ccxt_connector.py`	an order's data-usage line
`src/exchange/robinhood_connector.py`	the same line, second copy
`src/trading/__init__.py`	the package docstring

The restore path is the one read site and it still reads the old word. A saved record carrying it is named on the Console and skipped by name, and its record stays on disk.

`src/trading/container/restore.py, in StateRestoreMixin.restore_bots_from_state`

```
three saved records driven, one carrying the old word
  restored          the scrumming record and the extractor record
  skipped by name   the legacy record, "legacy grid mode (unrestorable)"
  records on disk   all three, untouched
```

Two places keep the word for a reason. The trade and reconciliation records name the legacy side values a stored row can still carry, so a reader of an old row is not left guessing.

```
src/trading/analytics_engine.py, the side field
src/trading/reconciliation.py, the action field
```

2026-10-26 - #665 - the nested pair and the log field are off the tree

Two readings on this page name a getter and setter on the settings store, and a field on the log record. Both were true when they were taken. Neither name is in the source tree now.

`src/core/settings.py, class SettingsManager` — what the store declares

```
reads and writes   get, set, get_all, reset_defaults
venue entries      add_exchange, get_exchange, list_exchanges, remove_exchange
the removed pair   get_nested, set_nested
```

`src/core/logging_engine.py, class LogEntry` — the record's five fields

```
timestamp: str = ""
category: str = ""
exchange: str = ""
bot_id: str = ""
data: dict = field(default_factory=dict)
```

Every stored field is still read and written. The pair took a group name and a key and reached the same group dictionary that `get` and `set` reach, so no field lost a route. A line written now carries no key for the removed field, and a line already on disk keeps it: every reader takes a stored line through a plain JSON parse, and no site builds the record back out of one.

The two names were counted over the tracked source tree, with a name from the same module as the figure that shows the count reports.

```
name              occurrences under src/   occurrences on this page
get_nested                             0                          4
set_nested                             0                          1
SettingsManager                       19                       many
```

Where the earlier settings-store entries now read differently

Five readings above name the removed pair or the removed field. They stand as written, and these are the readings that replaced them.

written above	what it reads now
"The nested getter and setter on `SettingsManager` still have no callers. The bare names `get_nested` and `set_nested` return two hits each, and both are the declaration and the class docstring in `src/core/settings.py`."	Both are off the store. It declares eight public methods and neither name is among them
"The record type does declare a `highlight` field, and nothing in the tree sets it and nothing reads it."	The record declares five fields and none of them is that one. A line written now carries no such key
"`sm.get_nested("profit_folding", "fold_target")`" in the read-shape table	That shape cannot be written now. Its zero stands: no site reads the word, by any shape
"`get_nested(..., "fold_target_count")`" in the read-shape table	The same. The zero stands and the shape is gone
"`get_nested(..., "distribute_target")`" in the read-shape table	The same. The zero stands and the shape is gone

The second sentence above wraps across two source lines, so a search that reads one line at a time cannot see it. Measured on this page, it reads nothing per line and one across the folded text.

2026-10-26 - The Phantom Bots figure is retaken from the drawn page

The picture on this page was taken from the eleven-box grid. The page draws one drop-down now, so the picture showed a control the operator cannot find. It has been retaken from the running dialog.

What the retaken figure shows

The dialog was built and its Phantom Bots tab selected, then the picture was taken off the widget. Read off that page while it was open: one drop-down, one tick box, and no timeframe tick box at all. The drop-down holds eleven names, 1m through 1w, and opens on the daily chart.

`src/gui/settings_dialog.py, in SettingsDialog._create_phantom_tab`

```
self._phantom_timeframe = QComboBox()
for tf in PHANTOM_TIMEFRAMES:
    self._phantom_timeframe.addItem(tf, tf)
self._phantom_timeframe.setCurrentIndex(
    self._phantom_timeframe.findData(PHANTOM_TIMEFRAME_DEFAULT)
)
```

The tab strip in the retaken picture carries nine names and no Profit Folding tab, because that page is gone. Every character in the four labels was checked against the ink a missing character draws, and none of them drew as an empty box, so the words in the picture are the words the page holds.

Where the sentences about the old picture now read differently

Three readings above describe the picture that has been replaced. They stand as written, and these are the readings that replaced them.

written above	what it reads now
"The build checks 5m, 15m, 1h, 4h and 1d, which is the state the figure shows."	The picture shows one drop-down reading 1d. No box is checked, because the page draws no timeframe box
"One box per timeframe in a single row. The build decides which five open checked, and those five are the ones the figure shows."	The picture shows the drop-down in that row and the one name it holds
"the figure on this page shows the old grid" in the table of what this row waits on	The picture is retaken. That absence is closed

What the drop-down does not draw

The picture carries no arrow on the drop-down, and that is what the application draws. The theme reserves twenty-four points of width for the arrow and names no image for it, and no rule anywhere in the tree names the arrow itself, so the reserved strip is painted empty on every drop-down in the application.

`src/gui/theme_engine.py` — the rule that reserves the strip

```
QComboBox::drop-down {
    border: none;
    width: 24px;
}
```

2026-10-28 - #1374 - the Fleet Lock rows, and the screen that gives the password back

The User tab takes one optional password beside the Username. With a password set, the saved fleet and the venue vault are both sealed, and a launch starts no bot until the password is given. With no password set the page behaves as it always did.

`src/gui/main_tabs/settings_dialog_surface.py, in fleet_lock_intent` holds the whole decision, so the Qt build and the drawn build take and refuse the same thing.

The four controls

On screen	What it is	Module, symbol
Password	Takes a new password, hidden as it is typed	`src/gui/settings_dialog.py, in _create_fleet_lock_group`
Confirm Password	Takes it a second time	the same
Unlock the fleet now	Raises the unlock screen; live only while a key file exists and this session holds no key	`src/gui/settings_dialog.py, in _open_fleet_unlock`
Clear the lock	Presses in to arm the demolition; live only while a key file exists	`src/gui/settings_dialog.py, in _show_fleet_lock_state`

A line above the four says which state the install is in. `src/gui/main_tabs/settings_dialog_surface.py, in fleet_lock_words` holds both sentences.

What a Save does with the two rows

Nothing is written when a key is typed. Save is the only moment anything moves.

The rows	The answer	What happens
both empty	`nothing`	no question, no line, no file touched
one filled, one empty	`half_filled`	one Console line, no file touched
the two differ	`mismatch`	one Console line, no file touched
both match, no lock held	`set`	the question, then the seal
both match, a lock held and open	`change`	the question, then the reseal
both match, a lock held and shut	`change_needs_unlock`	one Console line, no file touched
Clear armed, Password row empty	`clear_needs_password`	one Console line, no file touched
Clear armed, Password row filled	`clear`	the question, then the demolition

Measured on the built page: the four refusing rows put 0 questions on screen and left both files byte-identical.

What changes on disk

Diagram: flowchart, top to bottom

- Save pressed -> **fleet_lock_intent**
- fleet_lock_intent -> **seal_master_key, write_master_key** *(set)*
- seal_master_key, write_master_key -> **reseal_on_disk, the latch goes on**
- reseal_on_disk, the latch goes on -> **rekey_credential_stores, the vault moves to the master key**
- fleet_lock_intent -> **seal_master_key over the key this session holds** *(change)*
- seal_master_key over the key this session holds -> **write_master_key, one file, nothing else moves**
- fleet_lock_intent -> **open_master_key with the typed password** *(clear)*
- open_master_key with the typed password -> **rekey_credential_stores, the vault moves back to the username**
- rekey_credential_stores, the vault moves back to the username -> **unseal_on_disk, the latch goes off**
- unseal_on_disk, the latch goes off -> **master_key_path unlinked**

He does	What moves
sets a password	the sealed key file appears under `src/core/log_paths.py, in resolve_state_root`. The saved fleet's top level drops from seven fields to five: the four fleet fields move inside one `sealed_fleet` token. Every stored venue secret moves from the Username phrase to the master key
changes it	the sealed key file only. `src/core/encryption.py, in seal_master_key` is given the key this session already holds, so the vault phrase and the fleet payload do not move
clears it	every stored venue secret moves back to the Username phrase, the saved fleet is written back in plain text, and the sealed key file is unlinked last

The key file is the last thing to go and the first thing to arrive, so a second press of Clear the lock is the recovery at every point a run can stop. `src/core/state_manager.py, in unseal_on_disk` is the only path that turns the latch off, and it keeps the key file where it could not write the fleet back.

The question before the seal

A password is never taken on the press alone. The box reads `This password becomes the only way into your lot book` and its words state that the password is never written down, that nobody can reset it, and that losing it loses every bot, every tranche and every lot the saved fleet holds. `src/gui/main_tabs/settings_dialog_surface.py, FLEET_LOCK_SET_QUESTION` holds them.

Measured: answering No left the key file absent and both files byte-identical. Answering Yes wrote the key file, turned the latch on, and moved one stored venue secret under the new key.

The demolition

Clear the lock presses in and stays in. While it is in, the line above the rows is replaced by the demolition itself: the sealed key file goes, the venue vault goes, and every venue has to be entered again by hand on the Exchanges page. Save then puts the same words in a box that opens on No.

Three things have to agree before anything is destroyed: the button is pressed in, the current password is in the Password row, and the box is answered Yes.

The arm	Key file after	Files changed
armed, Password row empty	present	none
armed, the box answered No	present	none
armed, the wrong password	present	none
armed, the right password, Yes	gone	the vault and the saved fleet, as stated

The unlock screen

`src/gui/fleet_unlock_dialog.py, in FleetUnlockDialog` is one screen, and it is reached two ways: `main.py, in main` raises it when the splash closes on a locked launch, and the Unlock the fleet now button raises the same screen at any time.

On screen	What it is
The saved fleet is sealed.	the headline
the Console's own locked-launch sentence	the detail, from `src/gui/main_tabs/main_window_surface.py, FLEET_LOCKED_TEXT`
Password	the one row, hidden as it is typed
a red line, empty at rest	the refusal, from `src/gui/main_tabs/main_window_surface.py, FLEET_UNLOCK_REFUSED_TEXT`
Not now	leaves the fleet sealed; the Escape key and the window X do the same
Unlock the fleet	hands the password to `src/gui/main_window.py, in unlock_fleet`

Unlock the fleet is dead while the row is empty, on both builds. `src/gui/main_tabs/fleet_unlock_surface.py, in takes_press` is the one reading behind it.

The four Fleet Lock controls are on the Qt page. The drawn build replaces `in _setup_ui` with one web view and holds no row of its own, so `src/gui/settings_dialog.py, in _fleet_lock_drawn` answers False there and a Save on that page touches no lock and puts no line on the Console. Measured both ways: the page that drew the group refused a half-filled pair with one line, and the page that drew none added no line and asked no question.

Measured on the built screen, against a sealed fleet of two bots and one wire:

The password	The answer	On screen	Bots loaded	Files changed
nothing typed	the button is dead	nothing	none	none
wrong	`refused`	the refusal line	none	none
right	`opened`	nothing	both	the fleet is open to every reader

Both builds draw the unlock from one declaration

`src/gui/main_tabs/fleet_unlock_surface.py, in build_view_model` serves the whole screen as one payload, `src/core/desktop_bridge.py` registers it under `fleet_unlock.state`, and `src/gui/web/fleet_unlock.js` draws it.

Handed the real payload, the drawn build found all 21 declared fields, reported 0 faults, and drew 15 nodes: the same headline, the same detail sentence, the same refusal line, the same two button labels, and the password row as a hidden input. The control: a payload with one field removed and the hidden row flipped to plain reported both faults by name and drew the row as plain text, so the clean reading is a reading.

What the arm with no password reads

A launch with no password set reads the same through the files before this section and the files after it. Measured over six saved-file shapes — no file, an empty file, one bot, two bots and a wire, nested lots and tranches, and unicode with floats and nulls — `in latch_on`, `in launch_classification`, `in probe`, `in has_saved_state` and `in load_state` all answered alike. The control: one wire rate moved from 25.0 to 26.0 and the same comparison named the difference.

`src/core/encryption.py, in vault_phrase` also reads the same both ways, over six usernames against three held-key states plus six seal-and-open round trips. `in username_phrase` now names the branch that build takes with no key held, and it is what `src/gui/settings_dialog.py, in _username_phrase` asks while a key is held, which is the one moment the two answers differ. The control: one username moved and the comparison named it.

One defect found in the same files

`src/gui/settings_dialog.py, in _clear_fleet_lock` first asked `src/core/encryption.py, in vault_phrase` for the Username phrase while the key was held. That call answers the master key phrase, so the clear resealed the vault under the phrase it was already under and reported success, and the stored secrets then opened under nothing. `in username_phrase` answers the same whether a key is held or not, and the clear asks it instead.

2026-10-28 - #305 - the AI note builds its journal when it is written

The journal switch still writes one line for each answer. The window no longer holds a trade journal from launch, so no launch makes the journal folder or reads the day files. `src/gui/main_window.py, in MainWindow._on_ai_feedback` builds the journal at the moment an answer arrives with the switch on, writes the note, and lets the journal go.

`src/gui/main_window.py` — `MainWindow._on_ai_feedback`, the note as it is written now

```
if ai_cfg.get("log_feedback"):
    try:
        from ..trading.reconciliation import TradeJournal

        TradeJournal().record_from_trade(
            bot_id="AI_MONITOR",
            symbol="SYSTEM",
            action="AI_FEEDBACK",
            side="neutral",
            price=0.0,
            quantity=0.0,
            reason=feedback[:500],
        )
```

Driven in a redirected home, on the tree before this change and on this tree:

```
switch on    one Console line    one journal line holding the note
switch off   one Console line    no journal line
window build, before   journal and recovery folders made
window build, now      no folder made
```

2026-10-28 - #1374 - the React page draws the Fleet Lock group from the declaration the Qt page reads

The React Settings page now draws the same Fleet Lock group as the Qt page: the state line, Password, Confirm Password, Unlock the fleet now and Clear the lock. A password can now be set, changed, cleared and given back on either build.

The drawn build now holds the four controls

The reading in the Fleet Lock section above stands as written. This is the reading that replaced it.

written above	what it reads now
"The four Fleet Lock controls are on the Qt page. The drawn build replaces `in _setup_ui` with one web view and holds no row of its own, so `src/gui/settings_dialog.py, in _fleet_lock_drawn` answers False there and a Save on that page touches no lock and puts no line on the Console."	Both pages draw the four controls. `src/gui/react_settings_dialog.py, in SettingsDialogReact._build_holders` gives the drawn build one holder for each control, so `in _fleet_lock_drawn` answers True on both builds and a Save on either page reaches `in _settle_fleet_lock`

One declaration, two builds

`src/gui/main_tabs/settings_dialog_surface.py, FLEET_LOCK_CONTROLS` names each control and every field it is drawn with. The Qt build reads it in `src/gui/settings_dialog.py, in _create_fleet_lock_group`, and the React page draws the rows, the presses and the group from the same entries.

```
FLEET_LOCK_CONTROLS: tuple[dict, ...] = (
    {
        "name": FLEET_LOCK_PHRASE_NAME,
        "kind": LINE,
        "label": FLEET_LOCK_PHRASE_LABEL,
        "echo": HIDDEN_ECHO,
        "placeholder": FLEET_LOCK_PHRASE_PLACEHOLDER_ABSENT,
        "description": FLEET_LOCK_PHRASE_DESCRIPTION,
    },
    ...
    {
        "name": FLEET_LOCK_CLEAR_NAME,
        "kind": PRESS,
        "text": FLEET_LOCK_CLEAR_BUTTON_TEXT,
        "checkable": True,
        "danger": True,
        "description": FLEET_LOCK_CLEAR_DESCRIPTION,
    },
)
```

Each description reaches its control through `src/gui/tooltips.py, in set_description` on both builds, in `src/gui/settings_dialog.py, in _describe_fleet_lock`.

Read off both built pages with no password set, every field matched: the label, the hidden row, the placeholder, the description, whether the control takes a press, whether Clear the lock stays pressed in and draws in the danger colour, and the state line.

```
Password             hidden on both    the same label, placeholder and description
Confirm Password     hidden on both    the same label, placeholder and description
Unlock the fleet now dead on both      no key file on disk
Clear the lock       dead on both      stays pressed in, danger, on both
```

A field cut from the declaration, reported by both builds

`src/gui/main_tabs/settings_dialog_surface.py, in fleet_lock_faults` names every field a control lacks. The Qt dialog and the React host each log one error line per fault, and `src/gui/web/settings_dialog.js, in checkFleetLock` reports the same fault from the page's own audit.

Measured with the `echo` field cut from the Password entry of a copy of the declaration:

	Qt build	React build
the fault named	`lock_phrase echo missing`	`lock_phrase echo missing`, and the page's own audit names it too
the Password row	drawn as plain text	drawn as plain text
with the field in place	0 faults	0 faults

Set, refused, opened, changed and cleared on the React page

Driven on the React page in a redirected home, against a saved fleet of two bots and one stored venue key:

step	what happened
both rows typed on the page, Save pressed	the box `This password becomes the only way into your lot book` asked once; answered Yes, the key file was written, the latch went on and the stored venue key moved under the master key
a locked launch, the wrong password	`refused`, the Console's refusal line, no file changed, no bot loaded
the right password	`opened`, both bots loaded, the fleet start ran once
a new password typed twice, Save	the box `The new password becomes the only way in`; the key file was rewritten, the old password no longer opened the key and the new one opened the same key
Clear the lock pressed in, the password typed, Save	the line above the rows showed the demolition and Confirm Password went dead; the box asked, the key file was removed, the latch went off and the venue key opened under the Username again

The control: the same steps on the page before this change found no Password row, wrote no key file and left the latch off.

The way back after Not now, on the React page

After Not now the fleet stays sealed with no bot loaded. On both builds the way back is Settings, User tab, Unlock the fleet now. Measured on the React page: the button was live while the key file was on disk and no key was held, its press raised the unlock screen, the right password loaded both bots, and the button then went dead.

The arm with no password, on both builds

A Save with both rows empty and no password set reads the same through the files before this section and the files after it, on the Qt page and on the React page: the same stored settings, no box, no key file, one Console line. The control: one Username moved before Save, and the comparison named the box and the Console line that moved.

The Product Manual PDF rebuild is owed for this page.

2026-10-28 - #665 - a new bot's twelve weights start at zero

A new bot now starts with all twelve indicator weights at zero, so its vote reads neutral until Save on this window hands it the figures the TA Indicators page holds. A bot already saved keeps the figures it votes at, and the twelve sliders still open at the published figures.

One declaration of a new bot's twelve

The one place a new bot's figures are written is `src/trading/ta_engine.py, in new_bot_weights`. The Qt wizard and the React wizard both put it in the new bot's own config through the same reader in the wizard surface.

```
NEW_BOT_WEIGHT = 0.0


def new_bot_weights() -> dict[str, float]:
    """The twelve weights a new bot votes at, ``NEW_BOT_WEIGHT`` under each name."""
    return dict.fromkeys(DEFAULT_WEIGHTS, NEW_BOT_WEIGHT)
```

`src/gui/main_tabs/bot_wizard_surface.py, in with_new_bot_weights`

```
if config.get("mode") == SCRUMMING_MODE:
    from ...trading.ta_engine import new_bot_weights

    config["own_ta_weights"] = new_bot_weights()
return config
```

The bot's own weights, and the page's

A bot holds its own twelve in its config and votes at them in place of this page's figures. A record saved before holds none, so a saved bot votes at this page's figures exactly as it did. Save hands every bot this page's figures and empties its own twelve, so a restart keeps the page's figures.

`src/trading/scrumming_bot.py, in ScrummingBot.__init__`

```
own_weights = config.own_ta_weights
if isinstance(own_weights, dict) and own_weights:
    ta_weights = weights_from_settings(own_weights)
...
self._ta_weights = ta_weights
self._voting_engine = VotingEngine(weights=ta_weights)
```

A stored value that is not a mapping is left out with a warning, and the bot votes at the stored figures.

`src/trading/scrumming_bot.py, in ScrummingBot.set_ta_weights`

```
self._ta_weights = dict(weights) if weights else None
self._voting_engine.set_weights(self._ta_weights)
self.config.own_ta_weights = {}
```

```
moment            new bot                       saved bot
create            its own twelve at zero        -
first vote        NEUTRAL at 0.0                this page's figures
restart           its own twelve at zero        this page's figures
Save              this page's figures           this page's figures
restart after     this page's figures           this page's figures
```

The Paper Trader and Simulator wizards write no weights, and their runs vote as before.

Driven on both trees

Each tree ran in its own redirected home, the earlier one checked out from git at the commit this change starts from. Three stores were read: none written, the twelve published figures written, and two figures moved. A new bot came through each live wizard and the calls the create path makes. A saved bot came through the restore path after the launch handed it the store.

```
store       new bot, before              new bot, after             saved bot, two trees
none        published, BULLISH 0.102     twelve zeros, NEUTRAL 0.0  published, 0 differences
published   published, BULLISH 0.102     twelve zeros, NEUTRAL 0.0  published, 0 differences
two moved   the store's, BULLISH 0.0988  twelve zeros, NEUTRAL 0.0  the store's, 0 differences
```

Moving one digit on one side of the saved-bot comparison reports two differences, both naming the moved MACD weight, so a zero here can report a change. A new bot's record keeps its zeros across a restart. After Save its record holds none, and a restart reads the store's figures.

Where four earlier entries now read differently

They stand as written, and these are the readings that replaced them.

written above	what it reads now
"A bot reads the group when it is built, so a weight changed here applies to the next bot created and to every bot on the next launch."	A saved bot still does. A new bot starts at twelve zeros of its own and takes this page's figures at the next Save
"The figures are unchanged for that reason."	The published figures are still unchanged. The zero sits in a new bot's own twelve, so no bot already on disk changes
"The detonation check and the initial-entry check each build a fresh voting engine, so the twelve figures reach neither."	The detonation check votes at the bot's own twelve. The initial-entry check still builds its own engine at the published figures
"Detonation cannot fire in any case: its confidence bar starts at 0.50"	The bar starts at 0.00, so a reading the vote reaches can fire it

The Product Manual PDF rebuild is owed for this page.

2026-10-28 - #1374 - a sealed fleet makes no new bot

While the saved fleet is sealed, New Bot and Adopt make no bot. The box they show names the Unlock the fleet now button on the User tab as the way to unlock first. It takes that name from `FLEET_LOCK_UNLOCK_BUTTON_TEXT` in `src/gui/main_tabs/settings_dialog_surface.py`, so the box and the button cannot disagree. The words and the measurement are on the Console page.

Where an earlier sentence now reads differently

The sentence above stands as written. This is the reading that replaced it.

written above	what it reads now
"The Product Manual PDF rebuild is owed for this page."	`docs/Acervator-Product-Manual.pdf` is rebuilt with this section and carries this page

2026-10-29 - A venue a running bot trades on is kept

Remove Selected keeps a venue while a bot on it still trades. The line under the Add form names each bot to stop, and the status log carries the same words. The press writes nothing: the stored entry, its sealed tokens and its recorded check all stay. Once every bot on the venue is idle or stopped, Remove Selected drops the entry as it did before.

A bot is on a venue when its own venue id is that venue's id. One stored entry serves every sector a venue trades, so a bot under any sector counts. Idle and stopped are the two states with no trading loop. Starting, running, paused, error and cooldown each keep a loop, and each keeps the venue.

The guard both builds call

The Qt dialog runs `SettingsDialog._remove_exchange` on the press. The React dialog inherits that method and reaches it through `ACTION_HANDLERS`, so one method serves both builds.

`src/gui/settings_dialog.py` — `_remove_exchange`, before any write

```
    refusal = remove_refusal(eid, self._window_fleet())
    if refusal:
        self._set_feedback(refusal, "warning")
        if self._status_log:
            self._status_log.log(refusal, "warning")
        return
    self._sm.remove_exchange(eid)
```

The fleet is the one the window that opened the dialog runs. A fleet that raises while it is read keeps the venue too, and the line says its bots could not be read.

`src/gui/main_tabs/settings_dialog_surface.py` — `remove_refusal`

```
    reader = getattr(fleet, "list_bots_by_exchange", None)
    if reader is None:
        return ""
    name = str(exchange_id).capitalize()
    try:
        statuses = [dict(one) for one in reader(exchange_id) or ()]
    except (AttributeError, KeyError, RuntimeError, TypeError, ValueError) as exc:
        return REMOVE_UNREAD_FORMAT.format(
            name=name, error=f"{type(exc).__name__}: {exc}"
        )
    blocking = [
        REMOVE_BLOCKER_FORMAT.format(
            symbol=one.get("symbol", ""),
            bot_id=one.get("bot_id", ""),
            state=one.get("state", ""),
        )
        for one in statuses
        if str(one.get("state", "")) not in NOT_RUNNING_STATES
    ]
```

`list_bots_by_exchange` also fills each venue tab's bot table, where only the active sector's bots are kept. The guard counts the bots of every sector.

`NOT_RUNNING_STATES` in `src/gui/main_tabs/main_window_surface.py` holds idle and stopped, the two states the main window already reads as not running.

The Qt-free model of the same press, `SettingsDialogModel.remove_exchange`, calls the same guard. Nothing in the running program presses the model.

Remove Selected on a venue a bot trades on

Driven on both builds and on the Qt-free model, before and after the change, in one run. The venue is Binance, holding a stand-in credential in a redirected home, and every outbound connection was refused. The bots are stand-ins that ask the venue for one price on each tick.

Bots on the venue	Before	After
one running	removed, and the bot kept calling the venue on the connector it holds: 12 calls at the press, 23 after 0.15 s more	refused, the bot named, nothing written
one running, one paused, one stopped	removed	refused, the running and the paused bot named, the stopped bot not named
one in cooldown after five failed calls	removed	refused, the bot named in cooldown
none	removed	removed, every reading the same
one stopped and one idle	removed	removed, every reading the same
one running on another venue	removed	removed, every reading the same

Both builds and the model gave the same answer in every row. In the three rows that still remove, seven readings were compared on each side and 0 differed. Changing one reading on one side made the same comparison report 1 difference, so a zero there can report a change.

The line the React page drew for the second row, read off the page:

```
Binance cannot be removed while a bot trades on it. Stop each bot first: STANDA/USD (bot u172b000, running), STANDB/USD (bot
u172b001, paused).
```

What a stopped bot does on its next Start

Once every bot on a venue is stopped, Remove Selected drops the entry. A stopped bot on that venue then meets its Start path with no entry to read. Driven on the real main window methods with the entry removed:

Start path	With the venue's entry removed
a ccxt venue, `_connect_exchange_for_bot`	refused: `Exchange binance not found in settings. Add it in Settings first.`
Robinhood crypto, `_connect_written_crypto_for_bot`	Settings opens to ask for the venue, then refused: `No API credentials for Robinhood. Add them in Settings. No connector was built.`
a broker venue with no session open, `_open_broker_session`	refused: `no API credentials are stored`
a broker venue with a session already open, `_open_broker_session`	the open session is reused and the store is not read

Where earlier sentences on Remove now read differently

They stand as written, and these are the readings that replace them.

written above	what it reads now
"Remove Selected drops the highlighted entry."	Remove Selected drops the highlighted entry once every bot on that venue is idle or stopped. While one is not, it keeps the entry and names each such bot
"| `_remove_exchange` | Drops the selected entry |"	`_remove_exchange` drops the selected entry only where `remove_refusal` answers no words
"`_remove_exchange` drops the stored entry and its recorded check, and the line returns to grey."	It does so only while no bot on the venue still trades
"Nothing on this path can see a bot. The dialog is handed the settings manager and the status log, and nothing else."	The dialog reads the fleet of the window that opened it, through `_window_fleet`
"| Refused | False |", "| Warned about the bot | False |" and "| Silently allowed | True |"	The press is refused and the refusal names the bot. Nothing is allowed in silence

The Product Manual PDF is not rebuilt with this section, and its rebuild is owed for this page.

The sentence above stands as written. This is the reading that replaced it.

written above	what it reads now
"The Product Manual PDF is not rebuilt with this section, and its rebuild is owed for this page."	`docs/Acervator-Product-Manual.pdf` is rebuilt with this section and carries this page

2026-10-29 - A stored venue no sector lists stays removable

Three venue ids are listed under no sector and offered on no tab: Fidelity, TD Ameritrade's id and the second Interactive Brokers id. An entry stored for one of them still draws on the Exchanges page, so Remove Selected can delete it. It draws in every sector, after that sector's own venues, in the warning colour.

```
# src/gui/main_tabs/settings_dialog_surface.py, in dialog_venue_states
    for entry in stored or ():
        named = entry.get(EXCHANGE_ID_KEY, "") if isinstance(entry, dict) else ""
        venue = str(named or "").strip().lower()
        if venue and not acs.venue_classes(venue):
            held[venue] = VENUE_STATE_NOT_OFFERED
```

A press on that row binds it for Remove Selected and offers nothing else. The key and secret rows hide, Add and Test Connection refuse the press, and the feedback row reads `NOT_OFFERED_PROMPT`.

```
# src/gui/main_tabs/settings_dialog_surface.py, in credential_form
    listed = bool(acs.venue_classes(venue_id)) if bound else True
```

Read with the home redirected and no venue contacted, in both builds, with one stored entry each for the three ids:

Reading	Before	Now
Sectors drawing the Fidelity entry	Stocks, Crypto, Commodities, Indices	all six
Sectors drawing the other two entries	none	all six
Add on the Fidelity row	takes a press	refused
Key and secret rows on the Fidelity row	drawn	hidden
Remove Selected on the Fidelity row	deletes it	deletes it, in all six sectors
A venue press on the React build	raised `AttributeError`	binds the row

On the React build every venue press raised `AttributeError`, because the API Secret field and the passphrase box took no `setVisible`. Every page control now takes it.

```
# src/gui/react_settings_dialog.py, in _Held.setVisible
    def setVisible(self, shown: Any) -> None:
        """Show or hide the control and redraw."""
        self._owner.set_visible(self._name, bool(shown))
```

The Product Manual PDF is not rebuilt with this section, and its rebuild is owed for this page.

Back to the subsystem index.

The Promotion Pipeline

Reference. A strategy earns its way to real money in four steps. Each step is a gate: it feeds the next one, and the next one refuses what the previous one could not produce.

Diagram: flowchart, left to right

- Market Inspector — scans markets, proposes topologies -> **Simulator — Stone Tablet history**
- Simulator — Stone Tablet history -> **Paper Trader — live feed, fake budget**
- Paper Trader — live feed, fake budget -> **Live — real orders on the venue**
- Market Inspector — scans markets, proposes topologies ..> **Live — real orders on the venue** *(adopt)*

What each step contributes

Step	Data source	What it must produce
Market Inspector	Higher-timeframe OHLCV from the connected venues	A signal table, an opposing-pair table and topology proposals
Simulator	Stone Tablets, read-only	The same gates latching on the same candles as live
Paper Trader	The live feed, in real time, against a budget that is not real	A trade record the History surface can grade
Live	The venue	Filled orders, and the exchange values every ledger reconciles to

One logic, three data sources

Live, Paper and the Simulator differ in one thing only: where the candles and the balances come from. The trading logic stays one body of pure code that all three call. Only the stateful shells fork, which keeps a simulated run from touching a live object.

The Simulator's fleet was the worked example. It built real bots against a fake exchange rather than a second bot class, and that fake exchange was a subclass of the same interface the live connector implements.

`FleetSimExchange`, in the Simulator fleet package the rebuild deleted

```
class FleetSimExchange(ExchangeInterface):
    """Real-symbol candle-driven fake exchange for Fleet Replay.
```

The rebuild replaced it. The Sim tab now reads Stone Tablet files through `TabletSource`, which holds no venue and defines no write.

The dashed edge

One edge skips the two middle steps. The Market Inspector's adopt path creates real bots and draws real wires on the live fleet, which is why the handler opens a confirmation naming the new bot count, the new budget and every wire the adopt would change.

`src/gui/main_window.py` — `_topology_wire_collisions`

```
def _topology_wire_collisions(
    self, wires: list, asset_to_bot: dict
) -> list[dict]:
    """Return one dict per proposal wire whose bot pair is already wired."""
```

The read-only edge into the Simulator was a different hook, and it said so in its own words. It let the sim replay a proposal's shape across sim bots and created nothing. The rebuild removed that hook, so no proposal reaches the Sim tab today.

`set_topology_getter`, in the Simulator tab the rebuild deleted

```
    READ ONLY. This is not the adopt path — adopting a proposal
    creates real bots and wires on the live fleet, while this only
    lets the simulator replay a proposal's shape across sim bots.
    """
```

Where the chain breaks today

The Paper Trader step has a module now, and issue #19 built it. No step hands its result to the next one: the Simulator hands nothing forward, Paper loads its own fleet from the stored bot record, and Live receives from the Market Inspector's adopt path instead. See paper-trader.md.

Back to the subsystem index.

Notifications and Alerts

Reference. Two channel forms and a Save button on the left. The Event Routing table and the Notification History table on the right.

What the Event Routing table is for

The table is the map from an event to the channels that carry it. One row per event. One column per channel. A cell reads `Yes` when the event reaches that channel and `No` when it does not.

The operator clicks a routing cell to change it. The click writes the channel list the notification manager reads, so the next alert of that event follows the new reading.

Diagram: flowchart, left to right

- click a routing cell -> **alerts_tab_surface, in toggle_route**
- alerts_tab_surface, in toggle_route -> **notifications, in set_route**
- notifications, in set_route -> **notifications, in set_rule**
- notifications, in set_rule -> **AlertRule.channels**
- a filled trade -> **notifications, in send**
- AlertRule.channels -> **notifications, in send**
- notifications, in send -> **the channel handler**

`src/gui/alerts_tab.py, in _toggle_route` is the one method both builds enter. The Qt table reaches it from `cellClicked`. The drawn page reaches it from `src/gui/react_alerts_tab.py, in _apply_request`, which reads the `route` the page sends. `src/gui/main_tabs/alerts_tab_surface.py, in toggle_route` holds the flip itself, so one gesture writes one rule.

The columns come from one declaration

`src/gui/main_tabs/alerts_tab_surface.py`, `ROUTING_COLUMNS` names one channel per column with the heading above it. `RULES_COLUMNS`, `RULES_COLUMN_COUNT` and `ROUTED_CHANNELS` are all read from it.

Column	Channel	What carries the message
Event	—	the event name, titled
Priority	—	the rule's priority word
In-App	`IN_APP`	the log spool the window reads
Telegram	`TELEGRAM`	the Telegram Bot API
SMS	`SMS`	`src/core/sms_engine.py, in send`
Sound	`SOUND`	the alert sound engine

The Qt table takes its count and its headings from those names. The drawn page takes the same headings from the `rules_columns` field that `src/gui/main_tabs/alerts_tab_surface.py, in build_view_model` writes. Removing one entry from `ROUTING_COLUMNS` removes that column from both builds together, so the two cannot disagree on a channel.

Which events reach the message channels

`src/core/notifications.py`, `EVENT_MESSAGE_KIND` pairs an event with the `src/core/sms_engine.py` switch that gates it. An event listed there is given the Telegram and SMS channels by `src/core/notifications.py, in message_channels_for`. An event left out reaches the in-app spool and the sound engine alone.

Eleven of the fourteen events are paired today. `Buy filled` and `Sell filled` are among them, so a filled trade reaches in-app, Telegram and SMS.

The switch on `Settings > SMS` still gates the send. `src/core/sms_engine.py, in send` refuses and writes the refusal to the log when the group is off, when no switch names the event kind, or when the rate caps are reached. A ticked SMS cell therefore routes the event; it does not override the switch.

The dollar figure in a fill message

`src/core/notifications.py, in fill_message` prints the kind, the units, the price and the dollars of one fill. The dollars are read by `src/core/emit_contracts.py, in fill_usd`, which uses the field names `FILL_USD_FIELDS` declares. All ten emitters of the fill event write that one name, and the `trade.filled` contract takes its optional dollar fields from the same tuple.

The current reading, 2026-10-29. The message prints the `usd` field of the fill that `read_settled_fill` returns, and the two names above are gone. Every place in the bot that books a fill sends it in one shape, and the contract for the fill event requires every field of that shape, the dollars among them. A fill that is not in that shape raises no alert, and the refusal is written to the log.

`src/core/notifications.py, in fill_message` — the dollars of the one shape

```
    usd = _as_number(fill.usd)
```

The state of this screen

`src/gui/variant_surface.py` registers both builders under the name `ALERTS`. Nothing calls `surface_class(ALERTS)`, and `src/gui/main_tabs/retired_tabs.py, in _install_retired_tab_sentinels` sets `_alerts_tab` to `None`. The running window therefore builds no Alerts page, and `src/gui/main_window.py, in _refresh_dashboard` skips it.

Both builds of the screen are complete and both draw the same six columns. The window does not open either one.

The engine behind this screen keeps running. The window builds the notification manager at launch, and every 2 seconds the refresh tick hands it the fleet P/L for the milestone check. The screen's own surface loads only when a request for this page arrives, so a session that never asks for it never imports it.

`src/gui/main_tabs/alerts_tab_surface.py` — `METHOD`, read from the one module that names every retired page's method

```
METHOD = retired_surfaces.ALERTS_TAB_METHOD
```

The Product Manual PDF rebuild is owed for this page.

Where an earlier sentence now reads differently

It stands as written. This is the reading that replaced it.

written above	what it reads now
"The Product Manual PDF rebuild is owed for this page."	`docs/Acervator-Product-Manual.pdf` is rebuilt with this page
"Both builds of the screen are complete and both draw the same six columns. The window does not open either one."	Both stay in the source folder. A packaged build carries neither: no call asks the screen registry for `ALERTS`, so the build leaves `src.gui.alerts_tab` and `src.gui.react_alerts_tab` out. The rule is on 17-what-the-build-includes.md
"The screen's own surface loads only when a request for this page arrives, so a session that never asks for it never imports it."	A packaged build does not carry the surface either, and its registry holds no handler for this page's method. A request for the page gets `UnknownMethod`, and no import runs. A run from the source folder still answers it

Back to the subsystem index.

Part 4

Live and Supporting Evidence

- YTD Exchange Data Covering the Entire Development Arc and Exchange Test Coverage

Part 5

Conceptual Hopscotch: The HOP Protocol, Rules Registry, and SADP v1 and v2

ADR Index & Glossary

Decision Records and Glossary

Reference. Where this repository records a decision, and the words the manual uses, each anchored to the module that implements it.

Where a decision is recorded

Two places hold the decisions this repository has made. Each answers a different question.

- `.claude/rules/` — three binding standards, on code comments, on documentation and on tests. `CLAUDE.md` at the root holds the repository rules above them: one role per folder, and no machine-specific path anywhere committed.
- `CHANGELOG.md` and the commit history. See 09-updates-and-versioning.md for what the changelog holds today.

Beside those, `docs/audits/` holds ten audit directories and one audit page. An audit records one measurement, and the issue it answers cites it by name.

The four places, overtaken

True today: two of the four places are gone, so two remain. The repository tracks no engineering-notes directory and no ledger file inside one. It tracks one Python module under `tests/`, its own conftest, so no decision in this repository is held as an executable guard.

Glossary

Every term below names something in the code, and carries the module or symbol that implements it. The technical-analysis vocabulary is in 07-indicators.md with the twelve published formulae, and is not repeated here.

The accumulation cycle

- **Target** — the dollar value a bot aims to hold in an asset. The scrum sells above it and the fold buys back beneath it.
- **Scrum** — a sell of the excess above the target. `ScrummingBot` in `src/trading/scrumming_bot.py` decides and places it.
- **Fold** — the buy back on the dip that follows a scrum, carried by `apply_profit_fold` in `profit_fold`.
- **Fold, corrected 2026-09-13** — the sentence above names a module that no code ever called, and it is removed. A fold runs on the tranche book: `_fold_eligible_tranches` picks the tranches the fold may reach, and `_apply_fold_target_growth` in `src/trading/scrumming_bot.py` grows the target from the surplus, gated by the bot's own Profit Folding Active row.
- **Interval** — the smallest price move between two fires, in percent. `scrumming_interval_pct` in `src/trading/container/config.py`, default 1.0.

- **Scrum/Fold cycle** — the pair, run against volatility rather than against a price forecast. A completed cycle ends holding more of the asset than it started with.
- **Price floor** — every unit bought back after a fold costs at or under the lot's original price. `_fold_eligible_tranches` in `src/trading/scrumming/tick_phases.py` enforces it.
- **Fold tranche** — one queued slice of scrummed dollars waiting for its buy back. `src/trading/scrumming/fold_tranches.py` holds the book.
- **Merge** — `_top_up_remnant_fold_tranches` folds a new sell's dollars into part-spent tranches, and `_bound_new_fold_tranches` collapses one sell's fresh slice into a single record when the fill arrived as many lots. Both reduce the record count without losing a dollar.
- **Despawn** — `_despawn_aged_tranches` removes a tranche older than the bot's threshold. It removes the record; it does not delist it, and the count lands in the discarded lifetime counter rather than the closed one.
- **Clear** — `clear_fold_tranches` discards every queued tranche and trades nothing.

Merge, despawn and clear are the only three functions that collapse or remove a tranche. 08-tabs/bot-swarm.md describes each in full.

Bots and what they hold

- **Scrumming Bot** — the accumulation bot, `ScrummingBot` in `src/trading/scrumming_bot.py`. One per traded symbol.
- **Extractor Bot** — `ExtractorBot` in `src/trading/extractor_bot.py`, a chunked state machine rather than a scrum/fold engine.
- **Smart Wire** — a capital reinforcement link between two bots, in `src/trading/smart_wire.py`.
- **Phantom balance** — a bot's view of holdings it may not sell yet, in `src/trading/phantom_balance.py`.
- **Capital claim** — one bot's hold on a quantity of an asset. `effective_available` in `src/trading/capital_reservation.py` answers holdings minus every other bot's claim on the same asset, never the caller's own.
- **Fleet** — the bots a run holds, with their wires, ledgers and tranches. All of it loads from `bot_state.json` through `StateManager`, which is the only source.

OVERTAKEN, and the Phantom balance entry above is kept as written. The module holds no view of holdings and no selling rule. It holds one read-only observer per timeframe. Each observer holds its own voting engine, runs it on its own candle length, keeps the summary, and builds no order and holds no position. The coordinator weights those summaries by timeframe rank and answers the parent bot with them. A bot's hold on a quantity of an asset is the Capital claim entry above, in a different module.

`src/trading/phantom_balance.py` — the module's own first line

```
"""``PhantomBot`` read-only TA observers, one per timeframe.
```

The classes behind the wire, the phantom balance and the fleet load:

```
class SmartWireManager: ...     # src/trading/smart_wire.py
class BotLedger: ...
class WireTransaction: ...

class PhantomBalanceManager: ...    # src/trading/phantom_balance.py
class PhantomBot: ...

class StateManager: ...     # src/core/state_manager.py
```

Deciding a trade

- **Voter** — one indicator's contribution to a direction and a confidence. `VotingEngine` in `src/trading/ta_engine.py` runs the twelve.
- **Gate chain** — the ordered checks a candidate trade passes before it fires, in `src/trading/gate_chain.py`. The sell chain and the buy chain differ, and 07-indicators.md lists both.
- **Heikin-Ashi candle** — a smoothed price bar. `detect_landing_strip_v2` in `src/trading/indicators/landing_strip.py` reads them.
- **Landing Strip** — a tightening detector, `detect_landing_strip_v2` in `src/trading/indicators/landing_strip.py`.
- **Band travel** — the mean-reversion read, `MRInspector` in `src/trading/mr_inspector.py`.
- **Trade grade** — a letter per completed trade, from four scored axes in `src/trading/trade_grader.py`. The market regime is recorded beside them and never scored. See 10-live-trade-history.md.

The gate chain's two classes and the grader's five entry points:

```
class GateChain: ...    # src/trading/gate_chain.py
class GateContext: ...

def grade_trade(record, ctx) -> TradeGrade: ...     # src/trading/trade_grader.py
def _score_execution(...): ...
def _score_timing(...): ...
def _score_strategic(...): ...
def _score_outcome(...): ...
```

Getting a strategy to real money

- **Promotion pipeline** — Market Inspector, then Simulator, then Paper Trader, then Live. Each step is a gate rather than a mode, and 08-tabs/promotion-pipeline.md holds the entry and exit criteria for each.
- **Gate-latch criterion** — the Simulator's own criterion. It passes when the gates latch identically on the same candles as live, not when profit or trade count matches. See 08-tabs/simulator.md.
- **Stone Tablet** — a stored candle history a Simulator run reads. `src/simulator/tablet_source.py` serves the candles, and `src/simulator/back_test.py` walks them through the live gate chain.
- **Paper Trader** — the step between Simulator and Live, defined by running the same logic against a real-time feed and a fake budget. Built under issue #19, in `src/paper/`. The shelved stock window still defaults to `paper_trader_available: bool = False`. See 08-tabs/paper-trader.md.

- **Proof of Accumulation** — the competition package, `src/competition/`. It holds the bot identity, the Merkle log, the challenge protocol, the token ledger and the local chain. Its two screens are shelved by the operator's own direction. See 08-tabs/proof-of-accumulation.md. The Simulator rebuild removed the fleet-replay files this glossary once named.

The development harness

- **Archetype** — a domain quality gate under `dev_harness/harness/`, reporting a `passed` boolean.
- **Handoff** — `ACERVATOR_HOP8.md`, measured for drift by `tools/hop_check.py`.
- **Rule id** — an RN name in `RULE_META`, in `unreached/core/rule_registry.py`. The registry has no production importer and the ids bind nothing. `RULE_META` now lives at `unreached/core/rule_registry.py`, moved out of `src/` because no file in the tree imports it.

11-hop-protocol-and-rules-registry.md describes all three.

The title page

- **Solve et coagula** — dissolve and reform, the sense the epigraph *Turbator aequilibrii dissolvendus reformandusque* carries on 01-title.md. One function letters SOLVE · ET · COAGULA around the outer arc of the harvest trophy, and that is the one place the phrase appears in the code.

```
def harvest_svg(d: TrophyData) -> str: ...      # src/competition/trophy_generator.py
```

The operator's own words

These name a posture rather than a component. They appear in the manual because the operator uses them, and no module implements any of them.

- **Accumulation trading** — the product category he places Acervator in: harvesting volatility to end holding more of the asset, rather than predicting a direction.
- **Conceptual hopscotch** — his name for a session hopping across a context boundary, which the handoff file exists to carry.
- **Ekthelius the Accumulator** — his handle.

Part 7

Recent Updates and Version Tracking

Part 8

The Development Chronicle As Told By Claude

Part 9

Live Trade History and Supporting Evidence

- VWAP Charts - YTD Data Analysis and Trade Grading - Deep Indicator Function Investigations Via Trade Logic Gate Logs

VWAP Charts and Trade Grading

Reference. What a year of live fills measures, how the volume-weighted average price is drawn from it, and what the platform's own grading and gate logs add to a fill.

The fill record is the operator's own venue export. It is not committed to this repository and no row of it is reproduced anywhere in this manual. Every figure below is an aggregate over that record.

The record behind the charts

The export holds 5,709 physical rows and 5,705 body rows. 5,661 of the body rows are fills.

Body rows by transaction type: Advanced Trade Buy 3,189; Advanced Trade Sell 2,472; Reward Income 16; Deposit 15; Buy 8; Sell 4; Withdrawal 1.

44 body rows carry no fill, and each one is accounted for by name: 16 Reward Income, 15 Deposit, 12 USDC cash legs of a trade priced elsewhere, and 1 Withdrawal. Nothing is dropped without a reason that says what it was.

40 distinct assets appear in the record. USD and USDC are quote currencies, which leaves 38 charted bases, one for each bot in the fleet, and each of the 38 has fills of its own.

Fills per asset, most to fewest: A01 900, A02 495, A03 459, A04 322, A05 261, A06 230, A07 226, A08 220, A09 196, A10 191, A11 178, A12 159, A13 147, A14 141, A15 124, A16 124, A17 119, A18 119, A19 113, A20 87, A21 83, A22 80, A23 77, A24 75, A25 74, A26 73, A27 73, A28 66, A29 63, A30 58, A31 25, A32 24, A33 23, A34 23, A35 16, A36 6, A37 6, A38 5. Those 38 counts sum to 5,661, the fill total above.

Subtotal is the price

A row carries both a Subtotal and a Total, and only one of them belongs in a VWAP. On every row the absolute quantity times the price equals the absolute Subtotal to within 5e-6. Total is that same figure plus the fee.

A fee is a transaction cost, not a price. Fee drag measures between 0.92 and 1.20 percent per asset, so a VWAP built on Total would sit roughly that far above the price the venue actually filled at, on every buy in the record.

How a VWAP is computed here

The running figure is a streaming sum of price times quantity over a streaming sum of quantity, both carried as floats. The price is the export's Price at Transaction column. The generator never reads Subtotal, and it does not need to. The section above measures quantity times price against Subtotal and finds the two equal to within 5e-6.

Run against the same rows with Python's exact `Fraction`, the two paths agree to 3.63e-15 across 26 assets, so the float path loses nothing a chart can show.

One control separates a weighted mean from an unweighted one. One unit bought at $10 and three at $20 give 17.50 by hand, and both the float path and the `Fraction` path return 17.50. An unweighted mean of the two prices returns 15.00. An implementation that forgot to weight by quantity would open a gap of 2.50 on that case rather than pass unnoticed.

What the combined chart plots

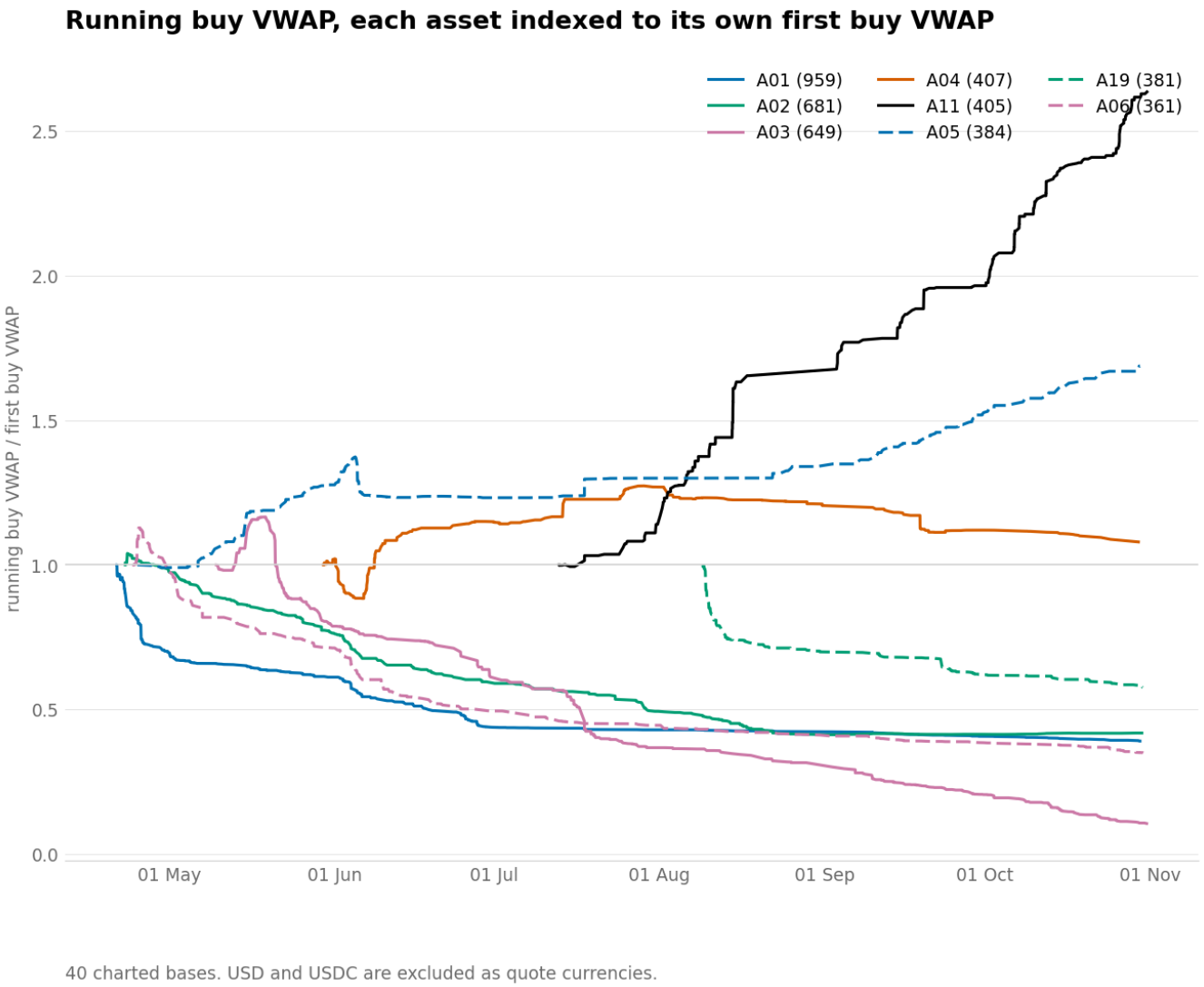
Each asset has a chart of its running buy VWAP against time. The line is the trajectory of the average as accumulation proceeds, and the shape of that trajectory is the reading.

Two metrics run through this part and they answer different questions. The trajectory answers what the units now held cost on average, and it keeps moving while the position is open. The sell-over-buy ratio answers whether the trades already closed sold above what they cost, and it is one settled figure per asset across the whole record. One reads the position, the other reads the realised trades. This part carries both, and neither stands in for the other.

The combined chart indexes every asset to its own first buy VWAP and plots the running buy VWAP divided by that first value, so an asset priced in cents and an asset priced in thousands share one axis and one scale.

Of the eight assets with the most fills, six trend down to between 0.4 and 0.6. A05 and A04 trend up.

Read the line as what it is. A falling running buy VWAP says the average cost of the base units now held fell as more were bought. It is not profit. It does not become profit until units are sold, and the index carries no sale in it at all.



The running buy VWAP of the eight assets with the most fills, each divided by its own first buy VWAP.

The eight lines are the eight assets with the most fills. Each one is that asset's running buy VWAP divided by its own first buy VWAP, which puts A10 at 5.71e-06 and A05 at 359.67 on one axis and one scale. The rule at 1.0 marks the first value. The legend names the asset and its fill count in three columns. The note under the axis reads:

> `38 charted bases. USD and USDC are excluded as quote currencies.`

Five pens carry the eight lines. The chart takes the series palette from `src/design_system.py` and keeps only the colours reaching a contrast ratio of 3.0 against the page background. Five of the eight qualify. The five run solid for the first five assets and dashed for the last three.

Series colour	Contrast on the page background	In the chart
#000000	21.00	yes
#0072B2	5.19	yes
#D55E00	3.87	yes

Series colour	Contrast on the page background	In the chart
#009E73	3.42	yes
#CC79A7	3.06	yes
#56B4E9	2.31	no
#E69F00	2.25	no
#F0E442	1.32	no

Measured over the same 5,661 fills, the eight lines end here:

Asset	Fills	First buy VWAP	Final buy VWAP	Final index
A01	900	1.5915	0.672605	0.423
A02	495	0.10842	0.044733	0.413
A03	459	0.11868	0.035741	0.301
A04	322	0.2478	0.298528	1.205
A05	261	359.67	485.048	1.349
A06	230	0.01581	0.006453	0.408
A07	226	0.05537	0.032580	0.588
A08	220	0.0508	0.022611	0.445

Six of the eight end below 1.0 and two end above it. Of the six, five land between 0.408 and 0.588, and A03 lands at 0.301.

The buy and sell ratio

The second metric is one figure per asset: the sell VWAP divided by the buy VWAP, across every fill in the record. Above 1.000 the average sale price beat the average buy price. Below it the average sale came in under the average cost. The figure reads how the closed trades performed, and nothing else.

No chart needs redrawing for it. Every per-asset chart already carries both lines, and the ratio is where the two of them finish: the sell line's last value over the buy line's last value. Both averages come from the same two columns as the charts, the export's Price at Transaction and its Quantity Transacted. Each side keeps its own pair of sums, so neither average pulls on the other.

Of the 38 charted bases, 35 have enough fills on both sides for the figure to mean anything. 23 of those finish above 1.000 and 12 below, and none lands exactly on it. A29 is the highest at 1.351 and A20 the lowest at 0.879.

Asset	Buys	Sells	Buy VWAP	Sell VWAP	Ratio
A01	538	362	0.672605	0.610801	0.908
A02	265	230	0.044733	0.0453964	1.015
A03	275	184	0.0357413	0.040867	1.143
A04	143	179	0.298528	0.328486	1.100
A05	140	121	485.048	536.675	1.106
A06	138	92	0.00645325	0.00627872	0.973

Asset	Buys	Sells	Buy VWAP	Sell VWAP	Ratio
A07	155	71	0.0325805	0.0317443	0.974
A08	140	80	0.0226113	0.0212018	0.938
A09	113	83	1.38181	1.34298	0.972
A10	104	87	5.98261e-06	6.10717e-06	1.021
A11	75	103	0.0297901	0.036577	1.228
A12	98	61	0.00762374	0.00758363	0.995
A13	69	78	14.6904	15.4413	1.051
A14	95	46	1996.6	2047.62	1.026
A15	95	29	68918	68393.6	0.992
A16	69	55	242.103	238.855	0.987
A17	66	53	0.36133	0.38167	1.056
A18	82	37	1.39501	1.40129	1.005
A19	57	56	0.00276124	0.00283892	1.028
A20	32	55	0.0336835	0.0296231	0.879
A21	43	40	9.06192	9.27909	1.024
A22	43	37	0.0881513	0.0882367	1.001
A23	43	34	82.9528	83.4024	1.005
A24	47	28	0.0116845	0.0116532	0.997
A25	41	33	0.857863	0.824958	0.962
A26	41	32	0.475448	0.497449	1.046
A27	45	28	0.198142	0.21519	1.086
A28	29	37	61.99	66.8959	1.079
A29	23	40	0.00205112	0.00277049	1.351
A30	27	31	2.03969	2.09155	1.025
A31	16	9	0.0828047	0.0787786	0.951
A32	8	16	0.431039	0.480832	1.116
A33	8	15	0.145768	0.164984	1.132
A34	10	13	0.174843	0.202243	1.157
A35	9	7	0.160515	0.164726	1.026
A36	3	3	46.4508	51.1833	not read
A37	3	3	0.0589505	0.0591535	not read
A38	1	4	2148.63	2576.19	not read

Three rows carry no ratio and the reason is the fill counts beside them. A38 has one buy, so its buy figure is a single price and not an average of anything. A36 and A37 have three buys and three sells each. A mean over three fills moves with any one of them, so a ratio built on it says more about which trade landed last than about how the asset performed. The table keeps both averages, since the export supports each one, and leaves the division out, since a number there would read as a result.

Every other row rests on at least seven fills a side, and 30 of the 38 rest on at least 23 a side.

The two metrics leave out different things and both gaps matter. The ratio counts only what completed on both sides, so it says nothing about units still held — and on most of these assets the held position is the larger part of the story. The trajectory counts every buy and no sale at all, so it says nothing about whether a sale ever cleared its cost. A cycle needs both readings. Either one alone overstates what it knows.

Traps in the export

Three shapes in the record break a reader that takes every row the same way.

A plain `Sell` row carries a negative quantity against a positive Subtotal. An `Advanced Trade Sell` row pairs those signs the other way round. Taking the sign from one column and the magnitude from the other turns four trades backwards.

One row carries a negative fee.

Three A27 rows carry a Subtotal that rounds to $0.00 against a quantity that is real. A filter that drops zero-value rows drops those base units with them.

Where the figures live

The charts are generated output, so they are not tracked. They belong beside the manual's other images in `artifacts/manual-figures/` under the repository root, a path `.gitignore` excludes. FIGURES.md inventories the images the manual itself embeds. No chart image is committed to this repository.

The 39 charts of this part live one level down, in a directory of their own beside the manual figures and under the same ignore rule. Keeping the two sets apart keeps the manual's own 38 images and this part's 39 charts from mixing. The combined view above sits in the first file below, and each charted base has one chart of its own.

```
docs/manual/figures/
    p<page>-i<index>.png        the manual's own 38 images
    vwap_combined.png           the eight-line combined chart
    vwap_a<NN>.png              one per charted base, 38 of them
    weekly_a<NN>.png            one weekly chart per charted base, 38 of them
    trace_a<NN>.png             one price trace per charted base, 38 of them
```

The charts are tracked, and they name no asset

Three sentences above are overtaken. They are quoted whole, and the sentence that replaces each one follows it.

> *The charts are generated output, so they are not tracked.*

The 39 charts are tracked, under `docs/manual/figures/`, so a clone can rebuild the manual without the venue export.

> *They belong beside the manual's other images in* `artifacts/manual-figures/` *under the repository root, a path* `.gitignore` *excludes.*

Both sets sit together in `docs/manual/figures/`, which `.gitignore` does not exclude.

> *No chart image is committed to this repository.*

Every chart is committed, and every one of them has the asset name painted out of its title, out of its price-axis label, and out of the combined view's legend. The file name carries the label this part uses, `vwap_a01.png` through `vwap_a38.png`, and never the asset.

The repository carries one manual, and that manual names no traded asset. The build that names them runs only from a checkout holding the unobscured charts, which this repository does not carry.

How to read a per-asset chart

Every chart carries two panels over one time axis.

The upper panel draws every fill as a dot at its price, buys in `COLORS["series"][0]` and sells in `COLORS["series"][5]`. Two lines cross it: the running buy VWAP and the running sell VWAP, in those same two colours. Each line steps at a fill of its own side and holds flat across a fill of the other side. The price axis turns logarithmic when the highest fill price exceeds the lowest more than eightfold, which covers A01 at 10.30 and A03 at 13.25 and no other asset. A chart of 30 fills or fewer marks each step with a dot: A31, A32, A33, A34, A35, A36, A37 and A38.

The lower panel draws net units accumulated — every bought quantity minus every sold quantity, running — in `COLORS["accent"]`, filled down to zero. It answers what the price panel cannot: whether the base position grew.

Every chart in this part draws its colours, its type sizes and its page size from `src/design_system.py`, which holds them under five names.

```
COLORS = {"bg": ..., "ink": ..., "accent": ..., "series": [...]}
TYPE = {"h1": ..., "body": ..., "cap": ...}
GRID = {"unit": 8, "page_w_in": 11.0, "page_h_in": 8.5}


def apply_rcparams(): ...


def contrast_ratio(fg: str, bg: str) -> float: ...
```

Read the buy line as a trajectory of average cost. A falling line says the average cost of the units bought so far fell. That fall is not profit, for the same reason the combined chart above carries none: no sale enters that line.

The Bollinger band behind the price

A grey channel sits behind the fills on the upper panel. It is a Bollinger band, and it is there so a trade at a volatility extreme can be seen rather than asserted. A scrum sells into the top of the channel and a fold buys into the bottom, and the channel puts both on the picture.

The band is the published one. A middle line is a simple moving average over a fixed number of bars, and the outer lines sit a fixed number of population standard deviations above and below it, taken over the same bars.

```
Middle = SMA(period)
Upper  = Middle + std_dev * sigma(period)
Lower  = Middle - std_dev * sigma(period)
```

The setting is the conventional one: period 20, width 2 standard deviations.

Where the band maths comes from

The platform already computes this band for its own trading decisions, and the charts read the same code rather than a second copy of the formula. The method returns one upper, middle and lower value per bar, and nothing before the twentieth bar, because no window has closed yet.

```
# src/trading/indicators/bollinger.py
BollingerBands(period=20, std_dev=2.0).bands(candles)
```

The deviation is the population one, divided by the window length, which is what Bollinger publishes. It is computed in `src/trading/indicators/helpers.py` and shared by every indicator that needs a rolling deviation.

What a fill at a band edge means

The upper line is two standard deviations above the recent average price, so a sell dot sitting on it or above it is a sale made while the price was at the top of its own recent range. The lower line is the mirror of that, so a buy dot on it or below it is a purchase made into the bottom of that range. Those two readings are the whole reason the band is drawn.

A dot inside the channel is an ordinary trade. Nothing about a fill at an edge says it was profitable. It says where in the recent price distribution the trade landed.

Which bar length the band uses

The export carries a timestamp and a price per fill and no candles, so the bars are built from the fills: the last fill in a bar closes it, and a bar with no fill closes where the one before it closed. That construction decides whether a short bar can carry a band at all. Twenty consecutive bars with no trade have no spread between them, so the deviation is zero and the band collapses onto the price line.

Measured over all 38 charted bases, counting every band the method returned and how many of them had zero width:

Bar length	Bands drawn	Zero width	Share	At or above upper	At or below lower	Fills in a banded bar
5 minutes	1,288,981	1,064,990	82.6%	2,921	3,134	6,868
15 minutes	429,202	315,054	73.4%	2,578	2,609	6,842
1 hour	106,790	52,205	48.9%	1,972	1,854	6,702
4 hours	26,182	4,994	19.1%	1,167	939	6,272
1 day	3,792	35	0.9%	605	460	4,955

At five minutes, 82.6 percent of the channel has no width. The band traces the price line as a hairline and shows no extreme at all. At one day, 0.9 percent has no width and the channel is wide enough to read across the whole window. The charts use one-day bars.

The cost of the coarse bar is the warm-up. Twenty one-day bars must close before the first band appears, so the first twenty days of each asset carry no channel, and 4,955 of the 6,997 fills land in a bar that has one.

The band edges, counted over every base

One row per charted base, in fill-count order. The two edge columns count fills against the band of the bar holding them, on one-day bars at period 20 and width 2.

Asset	Fills	Buy	Sell	At or above upper	At or below lower	In a banded bar	First buy VWAP	Final buy VWAP	Index	Window
A01	925	551	374	20	56	400	1.5915	0.652338	0.410	21 Apr to 25 Sep
A02	589	314	275	57	10	454	0.10842	0.0449093	0.414	22 Apr to 26 Sep
A03	537	315	222	13	28	409	0.11868	0.0257786	0.217	9 May to 27 Sep
A04	378	170	208	21	13	225	0.2478	0.277471	1.120	29 May to 27 Sep
A05	323	172	151	35	28	283	359.67	534.024	1.485	24 Apr to 26 Sep
A06	292	165	127	21	11	218	0.01581	0.00614089	0.388	24 Apr to 26 Sep
A08	274	165	109	20	19	209	0.0508	0.0223169	0.439	23 Apr to 27 Sep
A07	261	171	90	10	14	198	0.05537	0.0320055	0.578	3 May to 27 Sep
A10	237	122	115	25	24	167	5.71e-06	5.84811e-06	1.024	12 Apr to 27 Sep
A11	220	95	125	49	7	166	0.01722	0.033729	1.959	13 Jul to 25 Sep
A09	215	118	97	18	9	138	1.4622	1.38509	0.947	27 Apr to 27 Sep
A12	210	118	92	26	20	181	0.009814	0.00778479	0.793	27 Apr to 27 Sep
A13	205	93	112	21	7	170	15.7253	16.3105	1.037	9 May to 27 Sep
A14	169	108	61	18	31	149	2308.68	2037.61	0.883	23 Apr to 24 Sep
A16	157	80	77	13	20	141	273.4	243.486	0.891	2 May to 27 Sep
A17	155	82	73	10	4	132	0.35136	0.364156	1.036	7 May to 27 Sep
A15	152	107	45	11	39	143	77782.1	70016.9	0.900	23 Apr to 24 Sep
A18	148	97	51	18	22	123	1.4278	1.39586	0.978	23 Apr to 26 Sep

Asset	Fills	Buy	Sell	At or above upper	At or below lower	In a banded bar	First buy VWAP	Final buy VWAP	Index	Window
A19	147	79	68	2	3	40	0.00395	0.0024861	0.629	9 Aug to 27 Sep
A25	121	55	66	19	5	101	1.0673	0.850641	0.797	9 May to 27 Sep
A20	116	44	72	0	1	42	0.0697757	0.0309713	0.444	9 Aug to 27 Sep
A22	112	56	56	15	22	103	0.09814	0.0884351	0.901	25 Apr to 26 Sep
A24	108	62	46	4	3	73	0.02809	0.0104166	0.371	8 Jul to 27 Sep
A21	107	53	54	10	18	102	9.296	9.38981	1.010	24 Apr to 25 Sep
A23	101	52	49	11	24	88	85.32	84.0803	0.985	23 Apr to 25 Sep
A29	98	40	58	15	5	85	0.001603	0.00239143	1.492	7 Jul to 27 Sep
A30	98	40	58	21	2	69	2.411	2.2074	0.916	29 May to 27 Sep
A27	87	51	36	9	4	44	0.205737	0.197296	0.959	29 May to 25 Sep
A33	86	36	50	20	0	60	0.14222	0.155404	1.093	22 Aug to 26 Sep
A26	85	46	39	6	1	42	0.5153	0.471505	0.915	13 Jul to 27 Sep
A28	81	35	46	9	0	46	64.42	63.7886	0.990	29 May to 27 Sep
A32	49	15	34	15	0	37	0.42307	0.465448	1.100	23 Jul to 26 Sep
A31	42	21	21	9	4	28	0.09457	0.0825252	0.873	29 May to 24 Sep
A34	36	15	21	7	0	26	0.1696	0.179876	1.061	23 Jul to 26 Sep
A35	35	15	20	13	0	27	0.1695	0.166708	0.984	13 Jul to 25 Sep
A36	18	6	12	8	0	17	46.33	47.883	1.034	23 Jul to 25 Sep
A37	15	7	8	1	5	12	0.05997	0.057538	0.959	23 Jul to 25 Sep
A38	8	2	6	5	1	7	2148.63	2168.26	1.009	23 Jul to 21 Sep

Across the 38, 605 fills sit at or above the upper band and 460 at or below the lower one. A02 and A11 sell into the top most often, at 57 and 49. A01 buys into the bottom most often, at 56.

The newest export, beside the one the figures above measure

The figures and tables above were measured on an earlier export. A later one covers the same account over a longer window. Both sets of numbers are below, so a reader holding either can tell which is which.

The following sentence is overtaken:

> *The export holds 5,709 physical rows and 5,705 body rows. 5,661 of the body rows are fills.*

The later export holds 7,049 physical rows and 7,045 body rows, and 6,997 of the body rows are fills on a charted base.

The following sentence is overtaken:

> *Of the eight assets with the most fills, six trend down to between 0.4 and 0.6. A05 and A04 trend up.*

On the later export, five of the eight trend down to between 0.388 and 0.578, A03 falls further to 0.217, and A05 and A04 still trend up, to 1.485 and 1.120.

Side by side:

Reading	Earlier export	Later export
Physical rows	5,709	7,049
Body rows	5,705	7,045
Fills on a charted base	5,661	6,997
Charted bases	38	38
First fill	12 Apr	12 Apr
Last fill	3 Sep	27 Sep
Rows carrying no fill	44	48

The charted bases are the same 38 assets in both. The later export adds 1,336 fills and 24 days, and its extra rows carrying no fill are three more reward payments and one more cash leg.

One check separates a longer record from a different calculation. Of the 38 first buy VWAP values the table above publishes, 37 come out of the later export identical to the digit, and the one that differs is A20, published rounded to 0.069776 against a computed 0.0697757. A different price column or a different weighting would have moved all 38.

The October export, and the base that makes 39

A third export covers the same account to 19 October. The two readings above stand, and this one sits beside them. Every figure in this section is the output of `tools/build_product_manual.py`, in `measurement_lines`, which reads the exports `export_files` finds under `artifacts/evidence-input/transactions/`. That directory is excluded from the repository, so the record stays out of it and the figures drawn from it ship.

The following sentence is overtaken:

> *40 distinct assets appear in the record. USD and USDC are quote currencies, which leaves 38 charted bases, one for each bot in the fleet, and each of the 38 has fills of its own.*

41 distinct assets appear in the October export. USD and USDC are still the quote currencies, which leaves 39 charted bases, and each of the 39 has fills of its own. The 39th base opened on 4 October.

The following sentence is overtaken:

> *The later export holds 7,049 physical rows and 7,045 body rows, and 6,997 of the body rows are fills on a charted base.*

The October export holds 8,365 physical rows and 8,361 body rows, and 8,310 of the body rows are fills on a charted base.

The following sentence is overtaken:

> *Of the 38 charted bases, 35 have enough fills on both sides for the figure to mean anything. 23 of those finish above 1.000 and 12 below, and none lands exactly on it. A29 is the highest at 1.351 and A20 the lowest at 0.879.*

All 39 charted bases now carry fills on both sides. 32 finish above 1.000 and 7 below, and none lands exactly on it. A29 is the highest at 1.228 and A20 the lowest at 0.911.

The following sentence is overtaken:

> *Of the eight assets with the most fills, six trend down to between 0.4 and 0.6. A05 and A04 trend up.*

The eight assets with the most fills are no longer the same eight. A11 has more fills than A07 and takes its place. Five of the eight trend down, A03 as far as 0.137, and A04, A05 and A11 trend up, to 1.100, 1.639 and 2.403.

Three readings side by side:

Reading	Earlier export	Later export	October export
Physical rows	5,709	7,049	8,365
Body rows	5,705	7,045	8,361
Fills on a charted base	5,661	6,997	8,310
Charted bases	38	38	39
First fill	12 Apr	12 Apr	12 Apr
Last fill	3 Sep	27 Sep	19 Oct
Rows carrying no fill	44	48	51

The October export adds 1,313 fills and 22 days over the later one. Its three extra rows carrying no fill are all reward payments.

One check separates a longer record from a different calculation. The producer rebuilt the later export's table from that same export, and every cell it shares with the published one came back identical:

```
python -m tools.build_product_manual --measure
    rows compared                        38
    cells compared                      266
    cells differing                       0
control, the same table against the earlier export's
    cells differing                     229 of 266
```

The 38 first buy VWAP values the table above publishes are unchanged across the October export, every one of them. A first buy VWAP is the first buy's price, and no later fill can move it.

Every base, measured on the October export

One row per charted base, most fills first. The columns are the ones the export alone fixes.

Asset	Fills	Buy	Sell	First buy VWAP	Final buy VWAP	Index	Window
A01	950	567	383	1.5915	0.632117	0.397	21 Apr to 18 Oct
A02	657	351	306	0.10842	0.0452924	0.418	22 Apr to 19 Oct
A03	609	366	243	0.11868	0.0162426	0.137	9 May to 18 Oct
A04	404	190	214	0.2478	0.272606	1.100	29 May to 18 Oct
A05	371	204	167	359.67	589.423	1.639	24 Apr to 19 Oct
A11	336	151	185	0.01722	0.0413817	2.403	13 Jul to 19 Oct
A06	327	186	141	0.01581	0.00589118	0.373	24 Apr to 19 Oct
A08	313	190	123	0.0508	0.0223261	0.439	23 Apr to 18 Oct
A07	300	197	103	0.05537	0.0313047	0.565	3 May to 18 Oct
A10	269	142	127	5.71e-06	5.75974e-06	1.009	12 Apr to 18 Oct
A09	263	143	120	1.4622	1.43666	0.983	27 Apr to 18 Oct
A13	248	119	129	15.7253	17.411	1.107	9 May to 19 Oct
A12	242	139	103	0.009814	0.00790726	0.806	27 Apr to 19 Oct
A19	226	148	78	0.00395	0.0023844	0.604	9 Aug to 15 Oct
A17	191	104	87	0.35136	0.384832	1.095	7 May to 18 Oct
A14	190	121	69	2308.68	2075.55	0.899	23 Apr to 18 Oct
A16	186	97	89	273.4	248.764	0.910	2 May to 19 Oct
A15	181	123	58	77782.1	71198.7	0.915	23 Apr to 18 Oct
A18	172	114	58	1.4278	1.39522	0.977	23 Apr to 18 Oct
A25	160	81	79	1.0673	0.890555	0.834	9 May to 19 Oct
A29	155	74	81	0.001603	0.00293077	1.828	7 Jul to 19 Oct
A20	146	65	81	0.0697757	0.0281437	0.403	9 Aug to 19 Oct
A30	141	63	78	2.411	2.59862	1.078	29 May to 18 Oct
A24	140	85	55	0.02809	0.00931148	0.331	8 Jul to 17 Oct
A33	138	72	66	0.14222	0.171733	1.208	22 Aug to 18 Oct
A21	137	73	64	9.296	9.88602	1.063	24 Apr to 18 Oct
A22	132	70	62	0.09814	0.0881565	0.898	25 Apr to 18 Oct
A23	115	62	53	85.32	85.572	1.003	23 Apr to 17 Oct
A27	113	69	44	0.205737	0.198163	0.963	29 May to 18 Oct
A26	108	61	47	0.5153	0.466622	0.906	13 Jul to 18 Oct
A28	95	44	51	64.42	65.9488	1.024	29 May to 18 Oct
A32	69	26	43	0.42307	0.520594	1.231	23 Jul to 18 Oct

Asset	Fills	Buy	Sell	First buy VWAP	Final buy VWAP	Index	Window
A31	62	33	29	0.09457	0.0856605	0.906	29 May to 18 Oct
A35	49	25	24	0.1695	0.171694	1.013	13 Jul to 18 Oct
A34	45	21	24	0.1696	0.187527	1.106	23 Jul to 18 Oct
A36	25	11	14	46.33	50.3828	1.087	23 Jul to 14 Oct
A37	18	9	9	0.05997	0.0572927	0.955	23 Jul to 19 Oct
A38	17	8	9	2148.63	2294.25	1.068	23 Jul to 14 Oct
A39	10	7	3	2.0423	1.96971	0.964	4 Oct to 18 Oct

The buy and sell ratio on the October export

The sell VWAP over the buy VWAP, across every fill in the October export.

Asset	Buys	Sells	Buy VWAP	Sell VWAP	Ratio
A01	567	383	0.632117	0.587099	0.929
A02	351	306	0.0452924	0.0463992	1.024
A03	366	243	0.0162426	0.0198769	1.224
A04	190	214	0.272606	0.300622	1.103
A05	204	167	589.423	660.061	1.120
A11	151	185	0.0413817	0.0490044	1.184
A06	186	141	0.00589118	0.00576189	0.978
A08	190	123	0.0223261	0.0216727	0.971
A07	197	103	0.0313047	0.031177	0.996
A10	142	127	5.75974e-06	5.89461e-06	1.023
A09	143	120	1.43666	1.42274	0.990
A13	119	129	17.411	18.9641	1.089
A12	139	103	0.00790726	0.00806262	1.020
A19	148	78	0.0023844	0.00254737	1.068
A17	104	87	0.384832	0.412881	1.073
A14	121	69	2075.55	2155.22	1.038
A16	97	89	248.764	252.5	1.015
A15	123	58	71198.7	73581.2	1.033
A18	114	58	1.39522	1.40659	1.008
A25	81	79	0.890555	0.910984	1.023
A29	74	81	0.00293077	0.00359865	1.228
A20	65	81	0.0281437	0.0256386	0.911
A30	63	78	2.59862	3.01716	1.161
A24	85	55	0.00931148	0.00962266	1.033
A33	72	66	0.171733	0.194725	1.134

Asset	Buys	Sells	Buy VWAP	Sell VWAP	Ratio
A21	73	64	9.88602	10.4953	1.062
A22	70	62	0.0881565	0.0901305	1.022
A23	62	53	85.572	87.5001	1.023
A27	69	44	0.198163	0.21289	1.074
A26	61	47	0.466622	0.49167	1.054
A28	44	51	65.9488	72.1458	1.094
A32	26	43	0.520594	0.629924	1.210
A31	33	29	0.0856605	0.0897963	1.048
A35	25	24	0.171694	0.188452	1.098
A34	21	24	0.187527	0.218207	1.164
A36	11	14	50.3828	60.2623	1.196
A37	9	9	0.0572927	0.0574807	1.003
A38	8	9	2294.25	2724.79	1.188
A39	7	3	1.96971	1.89008	0.960

Three columns are not carried forward

The band table above counts fills against the band of the bar holding them, and those bars are built from the fills rather than from candles. No committed file builds them. The closest reconstruction returns 3,797 bands against the 3,792 that table counts, and 653 fills at or above the upper band against its 605. A figure that cannot be reproduced is not republished here, so this section carries the columns the export alone fixes and leaves the three band columns to the reading above.

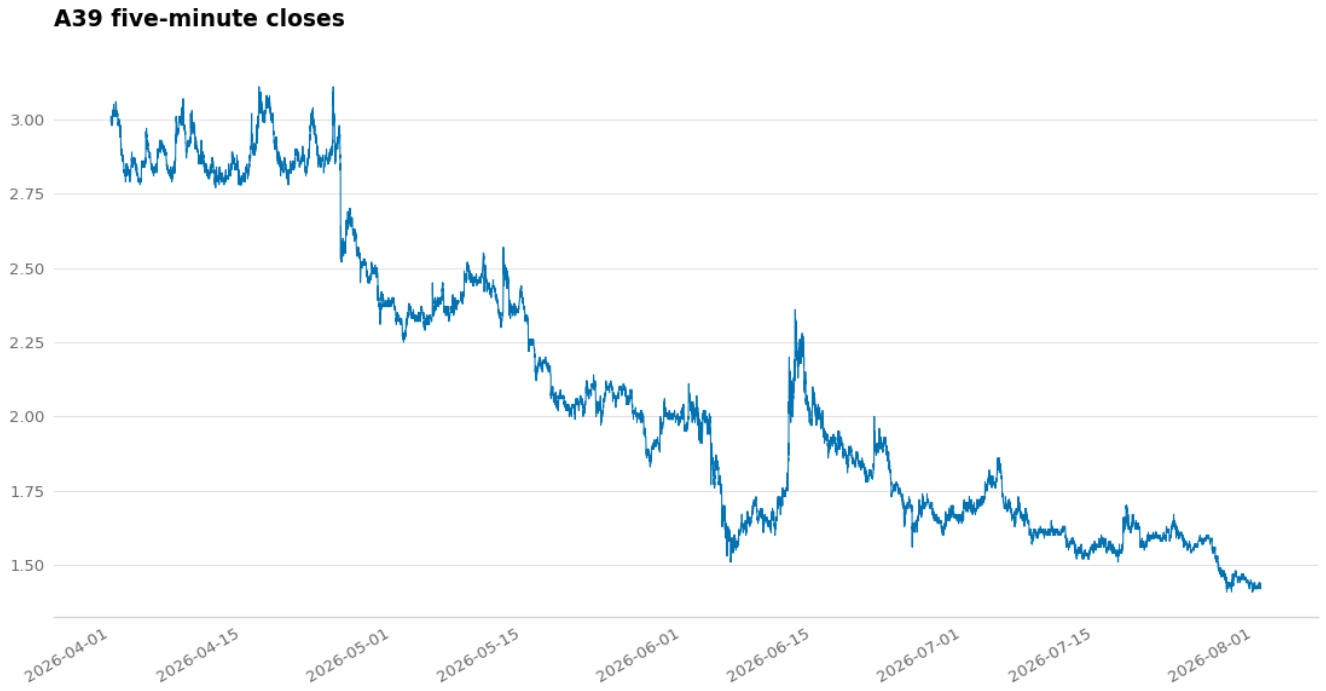
The combined chart near the top of this part draws the eight assets with the most fills of the earlier export. Those are no longer the eight with the most fills, and nothing in this repository redraws that chart.

A39, the base the October export adds

A39 has 10 fills, seven buys and three sells, between 4 October and 18 October. Net units rise from 12.128 to 14.895. The buy line runs from 2.0423 down to 1.96971 and the sell line ends at 1.89008, which puts the ratio at 0.960.



A39 weekly chart, 18 weekly candles built from 18,525 five-minute rows.



A39 price trace, 18,525 five-minute closes across the recorded window.

A39 has no position card. The venue's card set covers 37 of the 39 bases, and A33 is the other base without one.

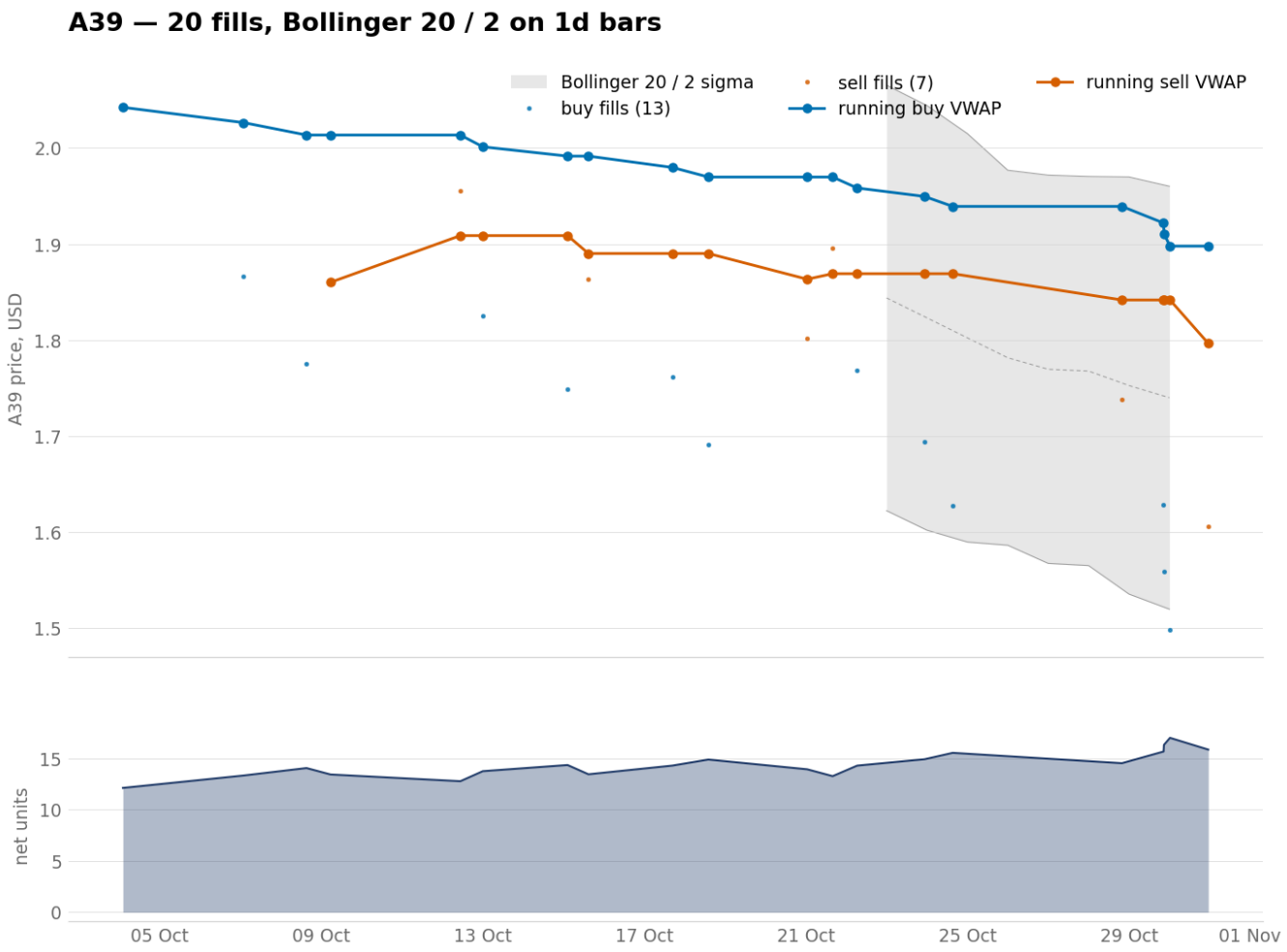
Now: the card set covers 37 of the 40 bases, and A33, A39 and A40 have no card (`tools/build_product_manual.py, in placard_sources`).

A39 has no VWAP chart. The 39 charts above arrived with the record, and no committed file draws a fortieth, so the weekly chart and the price trace carry this base. `tools/build_product_manual.py`, in `labels_across`, numbers a base that arrives late after the labels already published, so A01 through A38 keep the bases they have carried throughout and a new base takes the next number.

The following two sentences are overtaken:

> *A39 has no VWAP chart. The 39 charts above arrived with the record, and no committed file draws a fortieth, so the weekly chart and the price trace carry this base.*

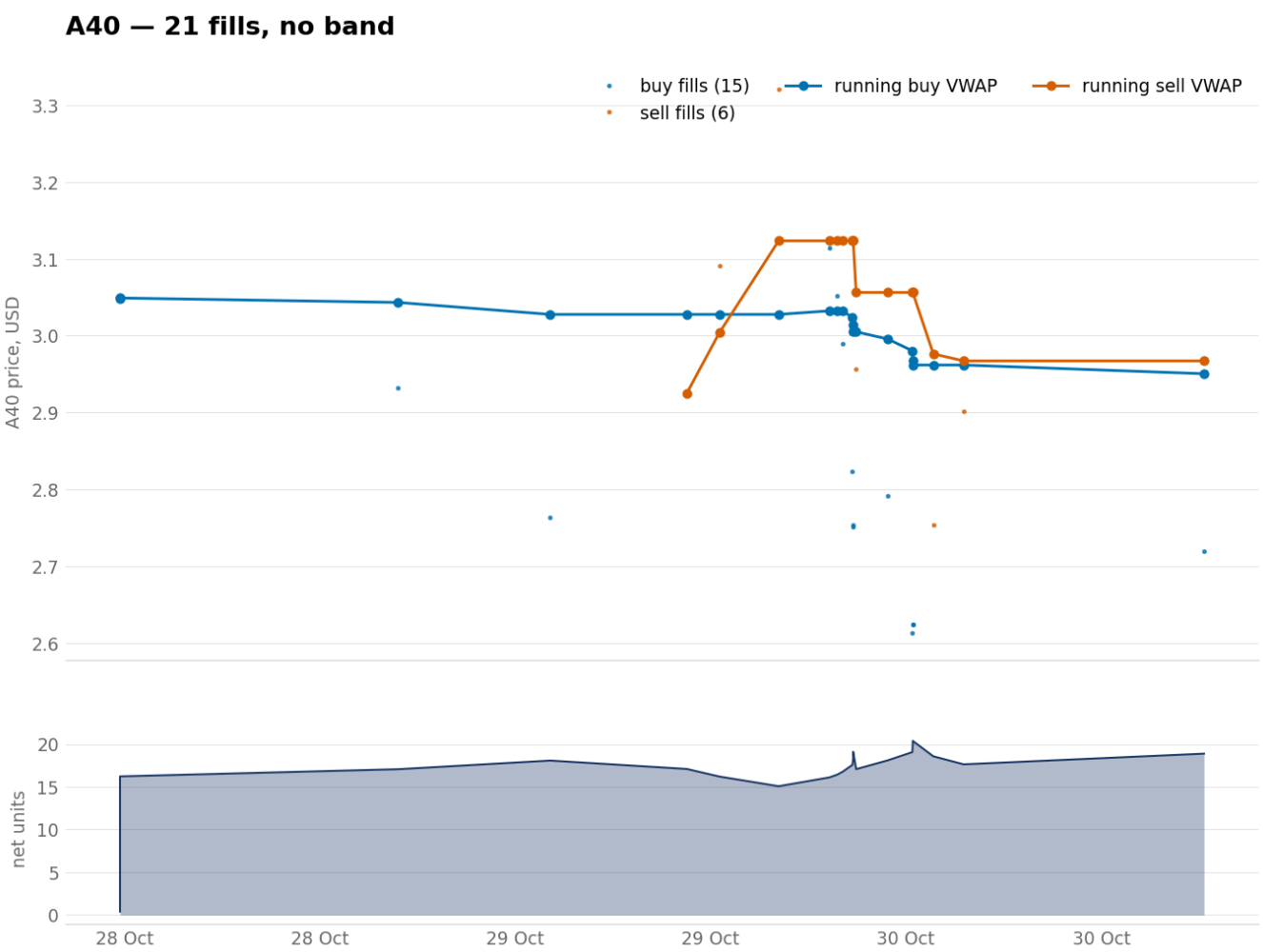
A39 has a VWAP chart. `_write_asset_vwap` in `tools/build_product_manual.py` draws one for every label the newest export carries, and that export carries A39.



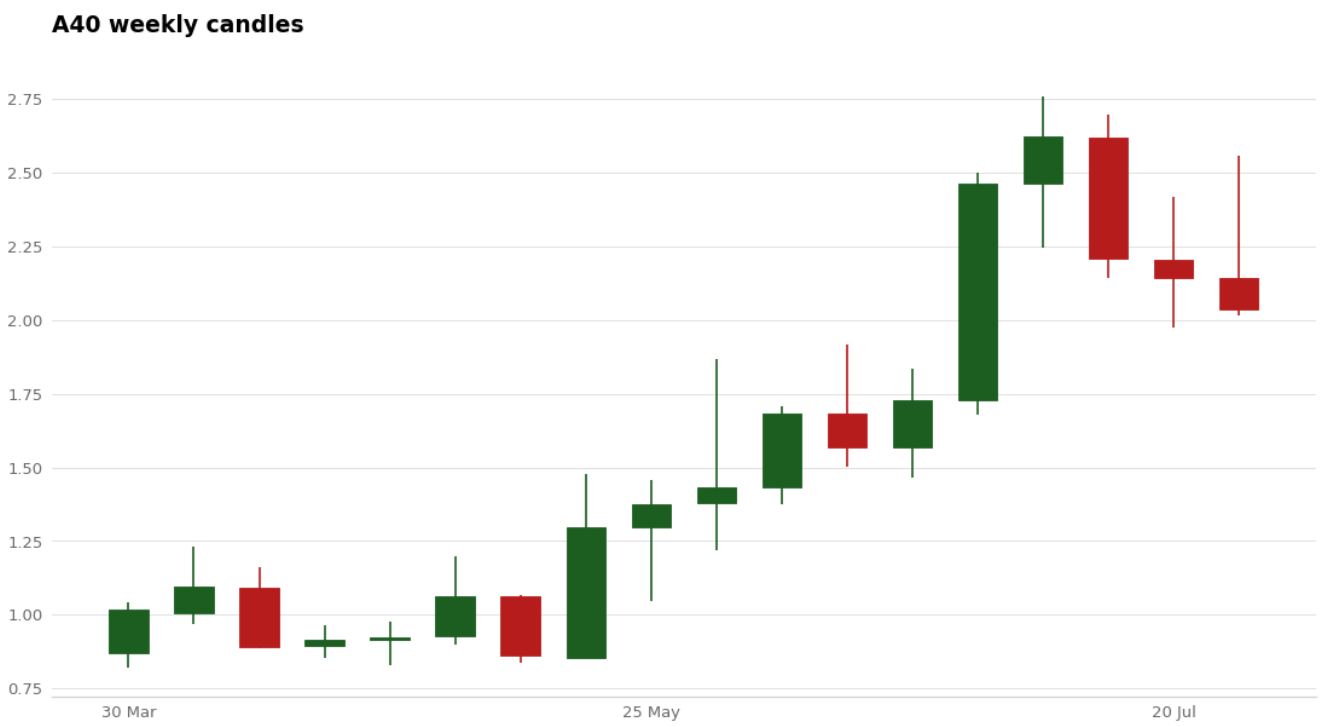
A39, 10 fills with every step dotted, and no band: its 15 one-day bars are fewer than the 20 the band needs.

Now: the A39 chart draws 20 fills, dots every step, and carries a band over the last 8 of its 27 one-day bars (`tools/build_product_manual.py, in fill_band`).

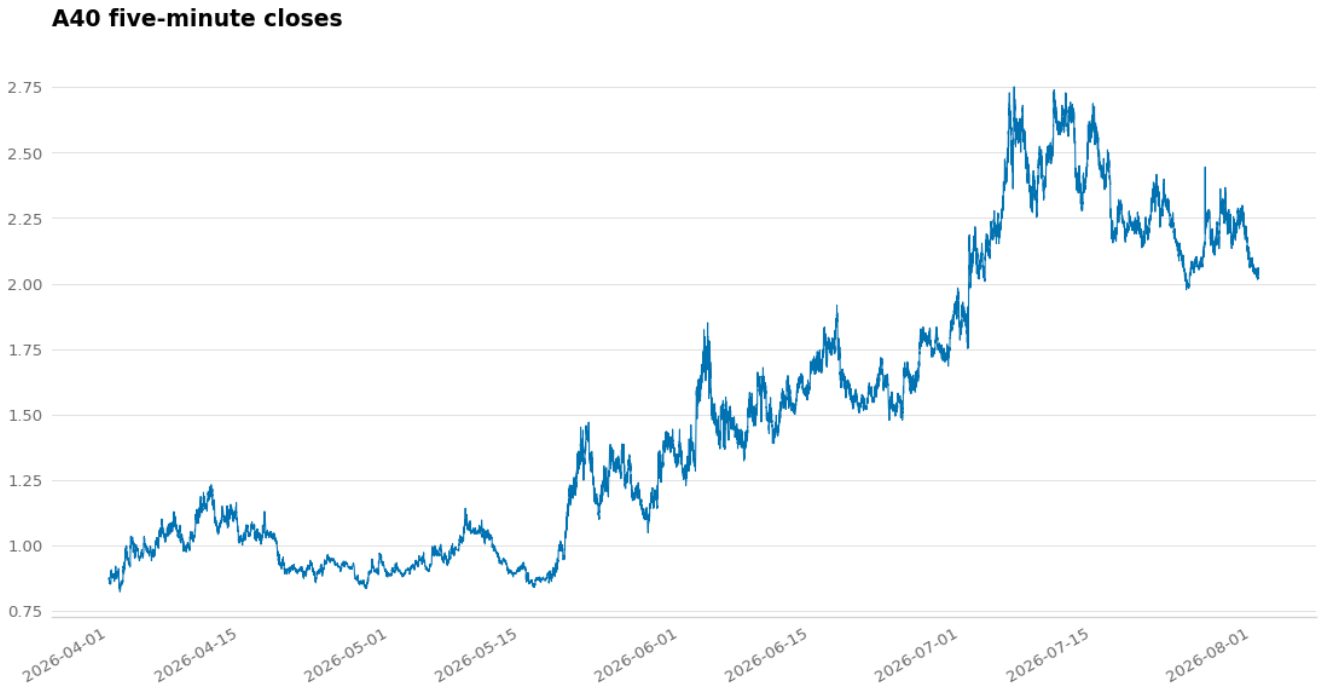
A40



A40, 21 fills with every step dotted, and no band: its 4 one-day bars are fewer than the 20 the band needs.



A40 weekly chart, 18 weekly candles built from 28,776 five-minute rows.



A40 price trace, 28,776 five-minute closes across the recorded window.

A40 has 21 fills, 15 buys and 6 sells, between 27 October and 30 October, and no position card. Its Stone Tablet rows end on 1 August, so its weekly chart and price trace stop before its first fill (`tools/build_product_manual.py, in tablet_candles`).

What this repository holds for these charts

The rows behind the charts take the shape `fetch_all_history_chunked` in `src/exchange/history_helpers.py` returns for the History tab, one dict per fill.

The closest committed figure to the buy line is the average entry price `compute_position_health` derives in `src/exchange/position_health.py` from the records a venue returns for its own trade history. The two differ, and the difference matters to a reader holding a chart beside the platform. The average entry follows the open position only: a buy re-weights it, a sell leaves it alone, and a full close resets it to zero. The buy line here counts every buy in the record and never resets.

```
if t.side == OrderSide.BUY:     # t is one get_my_trades record
    new_qty = qty + t.amount
    if new_qty > 0:
        avg_entry = (qty * avg_entry + t.amount * t.price) / new_qty
    qty = new_qty
elif t.side == OrderSide.SELL:
    qty -= t.amount
    if qty < 1e-12:
        qty = 0.0
        avg_entry = 0.0
```

No committed module computes an average sell price, and none computes a ratio of one average over the other. Five names were searched across the source, the harness and the tools, ignoring case, and every one of them returns no file. The control returns files for both of its terms and none for a coined one.

```
searched in src/, dev_harness/ and tools/, ignoring case
    avg_sell             0 files
    average_sell         0 files
    sell_vwap            0 files
    avg_sell_price       0 files
    sb_ratio             0 files
control
    avg_entry           13 files
    vwap                 6 files
    a coined term        0 files
```

The chart module computes no ratio either. It draws the two lines and stops there. The ratio table above comes from the export directly, on the same two columns the charts read.

No committed file produces these charts. A walk of every path any commit ever added, renamed or deleted reaches 1,626 distinct paths, and exactly one of them carries the letters vwap: a Simulator test for the price band. A pickaxe over every Python file in every commit returns nothing for the three names a chart builder would carry. The charts and the record behind them belong to the operator. This repository cannot regenerate either.

```
git log --all --diff-filter=ADR --name-only
    1,626 distinct paths
    test_vwap_band_scales_to_price             the one vwap name
                                                deleted; not in the tree

git log -S<name> -- '*.py'
    draw_combined         0 commits
    vwap_combined         0 commits
    buy_vwap              0 commits
control
    avg_entry            16 commits
    sync_ytd_trade_count  7 commits
    a coined term         0 commits
```

The builder draws every VWAP chart

`tools/build_product_manual.py` draws all 40 VWAP charts this part embeds: one for each of the 39 labelled bases, and the combined view. Below, each overtaken sentence appears whole, and what the code does today follows it.

Now: the builder draws 41 VWAP charts, one for each of the 40 labelled bases in the newest export and the combined view (`tools/build_product_manual.py, in _write_asset_vwap`).

> *No committed file produces these charts.*

`write_evidence_figures` draws them first among the evidence figures, before the build lays out a page.

```
out = _write_asset_vwap(figures_dir, labels, fills)
out += _write_asset_weekly(figures_dir, labels, candles)
```

> *The charts and the record behind them belong to the operator. This repository cannot regenerate either.*

The record still belongs to the operator and stays out of the repository. A checkout holding it under `artifacts/evidence-input/transactions/` regenerates every chart. The `--evidence-dir` flag points the builder at a copy held in any other folder.

```
python -m tools.build_product_manual --evidence-dir <a folder holding transactions/>
```

> *The 39 charts of this part, the combined view above included, come from one module, `vwap_charts`. It reads the export once, groups the fills by asset, and writes one PNG per charted base.*
>
> *The module is not committed to this repository. Neither is the export it reads.*

`_write_asset_vwap` takes the newest export's fills and groups them by label, never by asset.

Each group goes to `_draw_vwap`, and the whole set goes to `_draw_combined`. Every string a drawing function writes comes from the label, a count or a fixed word, so no title, axis label or legend carries an asset name. The repository holds the drawing. It does not hold the export.

```
groups: dict[str, list[Fill]] = {}
for fill in fills:
    groups.setdefault(labels[fill.asset], []).append(fill)
```

> *Every chart is committed, and every one of them has the asset name painted out of its title, out of its price-axis label, and out of the combined view's legend.*

The repository holds every chart, and no chart has anything painted out. The label sits in the title and the price-axis label of each per-asset chart, and in the legend of the combined view.

```
price.set_ylabel(f"{label} price, USD")
price.set_title(f"{label} — {len(fills)} fills, {note}")
```

> *The file name carries the label this part uses,* `vwap_a01.png` *through* `vwap_a38.png`*, and never the asset.*

The file names run from `vwap_a01.png` to `vwap_a39.png`, and none names the asset.

Now: the file names run to `vwap_a40.png`, one for each label (`tools/build_product_manual.py, in asset_figure_name`).

> *The combined chart near the top of this part draws the eight assets with the most fills of the earlier export. Those are no longer the eight with the most fills, and nothing in this repository redraws that chart.*

`_draw_combined` redraws it from the newest export. Its eight lines are the eight bases with the most fills there: A01 to A06, A08 and A11. The note under its axis counts the bases that export carries:

```
39 charted bases. USD and USDC are excluded as quote currencies.
```

Now: the eight lines are A01 to A06, A11 and A19, and the note counts 40 charted bases (`tools/build_product_manual.py, in _draw_combined`).

> *No committed module computes an average sell price, and none computes a ratio of one average over the other.*

`trajectory` in `tools/build_product_manual.py` computes the running sell VWAP each chart draws.

`measurement_lines` divides the final sell VWAP by the final buy VWAP for the ratio table on the October export.

```
ratio = row.sell_vwap / row.final_buy_vwap if row.final_buy_vwap else 0.0
```

> *The price axis turns logarithmic when the highest fill price exceeds the lowest more than eightfold, which covers A01 at 10.30 and A03 at 13.25 and no other asset. A chart of 30 fills or fewer marks each step with a dot: A31, A32, A33, A34, A35, A36, A37 and A38.*

On the newest export the axis turns logarithmic for A01 at 14.82 and A03 at 38.35, and for no other base. Four charts have 30 fills or fewer, and each marks every step with a dot: A36, A37, A38 and A39.

```
marker = "o" if len(fills) <= STEP_DOT_MAX_FILLS else "None"
```

Now: the axis turns logarithmic for A01 at 16.72 and A03 at 43.35, and the four charts that dot every step are A37, A38, A39 and A40 (`tools/build_product_manual.py, in _draw_vwap`).

What a redrawn chart reads

Every chart reads the newest export in the input folder: 8,310 fills on 39 bases, the last on 19 October. The fill count in each chart's title matches the table under "Every base, measured on the October export".

Now: every chart reads 9,052 fills on 40 bases, the last on 31 October, and each title's fill count matches the table `python -m tools.build_product_manual --measure` prints (`tools/build_product_manual.py, in measurement_lines`).

The band table, the table under "The 38 per-asset charts" and the description under each base's heading take their numbers from the earlier exports. They stand as published.

Where a one-day bar opens

No committed file builds them.

`fill_bars` builds the one-day bars the band reads. Every bar opens at 00:00 UTC. The last fill in a bar closes it, and a bar with no fill closes where the bar before it closed.

`fill_band` hands those bars to the platform's own band, at period 20 and width 2.

```
CHART_BAND = BollingerBands(period=20, std_dev=2.0)
BAND_BAR_MS = DAY_MS
```

The band table above counts one-day bars that open at 17:00 UTC. A recount of the same 6,997 fills at that boundary returns every row of that table exactly. At 00:00 UTC the recount returns the reconstruction the section on the three band columns quotes:

```
the 27 September export, period 20, width 2, one-day bars
                   bands   zero width   at or above upper   at or below lower   in a banded bar
bars at 17:00 UTC  3,792       35             605                 460              4,955
bars at 00:00 UTC  3,797       36             653                 455              4,975
```

The channel on a redrawn chart sits on bars that open seven hours later than the bars that table counts. The builder opens every bar at 00:00 UTC, the boundary `week_start_ms` also uses, on any machine that runs it.

The evidence slides

The same run writes the evidence slide show, Prototyping-Trading-Evidence.pptx, after the charts and before the PDF. Every number on a slide comes from the newest export, and every chart on a slide is one this run drew.

```
evidence = read_evidence(args.evidence_dir)
deck = write_evidence_deck(args.deck, args.figures_dir, evidence)
```

The 38 per-asset charts

One chart per charted base, in fill-count order. The table gives what each chart's two lines start and end at, and the window its axis spans.

Asset	Fills	Buy	Sell	First buy VWAP	Final buy VWAP	Index	Window
A01	900	538	362	1.5915	0.672605	0.423	21 Apr to 2 Sep
A02	495	265	230	0.10842	0.044733	0.413	22 Apr to 3 Sep
A03	459	275	184	0.11868	0.035741	0.301	9 May to 2 Sep
A04	322	143	179	0.2478	0.298528	1.205	29 May to 31 Aug
A05	261	140	121	359.67	485.048	1.349	24 Apr to 3 Sep
A06	230	138	92	0.01581	0.006453	0.408	24 Apr to 3 Sep
A07	226	155	71	0.05537	0.032580	0.588	3 May to 3 Sep
A08	220	140	80	0.0508	0.022611	0.445	23 Apr to 3 Sep
A09	196	113	83	1.4622	1.381809	0.945	27 Apr to 3 Sep
A10	191	104	87	5.71e-06	5.98261e-06	1.048	12 Apr to 3 Sep
A11	178	75	103	0.01722	0.029790	1.730	13 Jul to 3 Sep
A12	159	98	61	0.009814	0.007624	0.777	27 Apr to 3 Sep
A13	147	69	78	15.7253	14.690407	0.934	9 May to 1 Sep
A14	141	95	46	2308.68	1996.595	0.865	23 Apr to 21 Aug
A15	124	95	29	77782.1	68917.999	0.886	23 Apr to 3 Sep
A16	124	69	55	273.4	242.103	0.886	2 May to 1 Sep
A17	119	66	53	0.35136	0.361330	1.028	7 May to 3 Sep
A18	119	82	37	1.4278	1.395015	0.977	23 Apr to 3 Sep
A19	113	57	56	0.00395	0.002761	0.699	9 Aug to 31 Aug
A20	87	32	55	0.069776	0.033684	0.483	9 Aug to 2 Sep
A21	83	43	40	9.296	9.061918	0.975	24 Apr to 3 Sep
A22	80	43	37	0.09814	0.088151	0.898	25 Apr to 3 Sep
A23	77	43	34	85.32	82.952844	0.972	23 Apr to 1 Sep
A24	75	47	28	0.02809	0.011685	0.416	8 Jul to 31 Aug
A25	74	41	33	1.0673	0.857863	0.804	9 May to 3 Sep
A26	73	41	32	0.5153	0.475448	0.923	13 Jul to 30 Aug
A27	73	45	28	0.205737	0.198142	0.963	29 May to 28 Aug
A28	66	29	37	64.42	61.990016	0.962	29 May to 30 Aug
A29	63	23	40	0.001603	0.002051	1.280	7 Jul to 30 Aug
A30	58	27	31	2.411	2.039694	0.846	29 May to 3 Sep
A31	25	16	9	0.09457	0.082805	0.876	29 May to 22 Aug
A32	24	8	16	0.42307	0.431039	1.019	23 Jul to 3 Sep
A33	23	8	15	0.14222	0.145768	1.025	22 Aug to 3 Sep
A34	23	10	13	0.1696	0.174843	1.031	23 Jul to 3 Sep
A35	16	9	7	0.1695	0.160515	0.947	13 Jul to 2 Sep
A36	6	3	3	46.33	46.450791	1.003	23 Jul to 3 Sep

Asset	Fills	Buy	Sell	First buy VWAP	Final buy VWAP	Index	Window
A37	6	3	3	0.05997	0.058950	0.983	23 Jul to 16 Aug
A38	5	1	4	2148.63	2148.63	1.000	23 Jul to 25 Aug

Those 38 fill counts sum to 5,661, the fill total this part opens with. Eleven of the 38 end at or above 1.000 and 27 end below it.

The 39 charts of this part, the combined view above included, come from one module, `vwap_charts`. It reads the export once, groups the fills by asset, and writes one PNG per charted base.

The line each chart draws is a running mean of price weighted by quantity. The price is the export's Price at Transaction column and the quantity is its Quantity Transacted column. Buys and sells accumulate in two separate pairs of sums, so a fill on one side steps that side's line and leaves the other flat. The lower panel is the same walk of quantities, added on a buy and taken away on a sell.

The module is not committed to this repository. Neither is the export it reads.

```
def load_fills(src: Path):
    """Return the Fill list and a Counter keyed by drop cause."""
    ...
    price = parse_money(r[idx["Price at Transaction"]])
    qty = abs(parse_money(r[idx["Quantity Transacted"]]))
    ...
    fills.append(Fill(asset, when, side, price, qty))

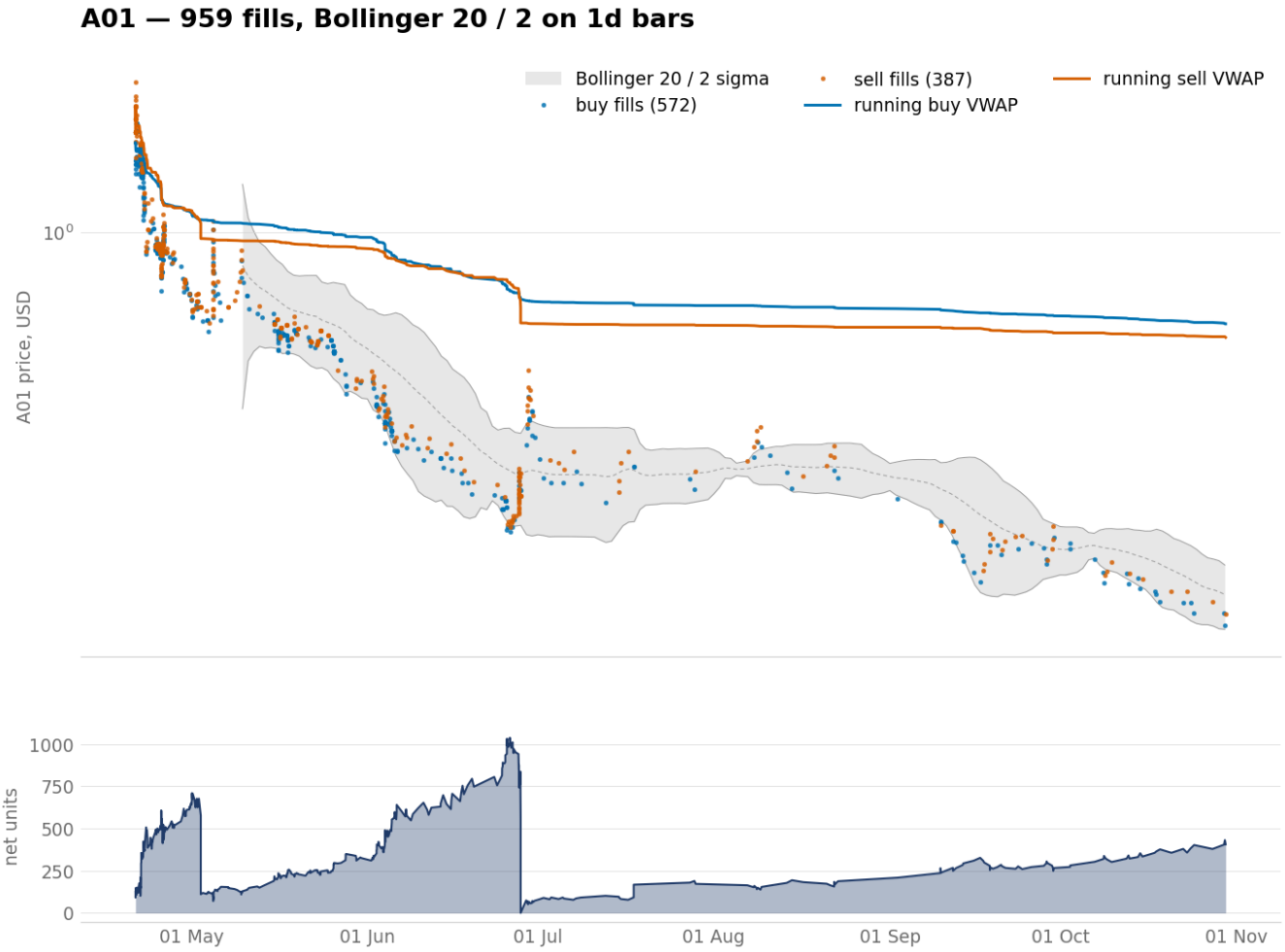

def trajectory(fills):
    """Return times and the running buy VWAP, sell VWAP and net units."""
    ordered = sorted(fills, key=lambda f: f.when)
    t, buy_v, sell_v, held = [], [], [], []
    bn = bd = sn = sd = pos = 0.0
    for f in ordered:
        if f.side == "buy":
            bn += f.price * f.qty
            bd += f.qty
            pos += f.qty
        else:
            sn += f.price * f.qty
            sd += f.qty
            pos -= f.qty
        t.append(f.when)
        buy_v.append(bn / bd if bd > 0 else float("nan"))
        sell_v.append(sn / sd if sd > 0 else float("nan"))
        held.append(pos)
    return t, buy_v, sell_v, held
```

Four of the 38 charts below carry a block of their own. Each one is a chart where the module takes a turn it does not take for the rest: a logarithmic price axis, the minor ticks that axis needs, a tick label small enough to go scientific, and a fill count low enough to mark every step. The other 34 are drawn by the two functions above and nothing else.

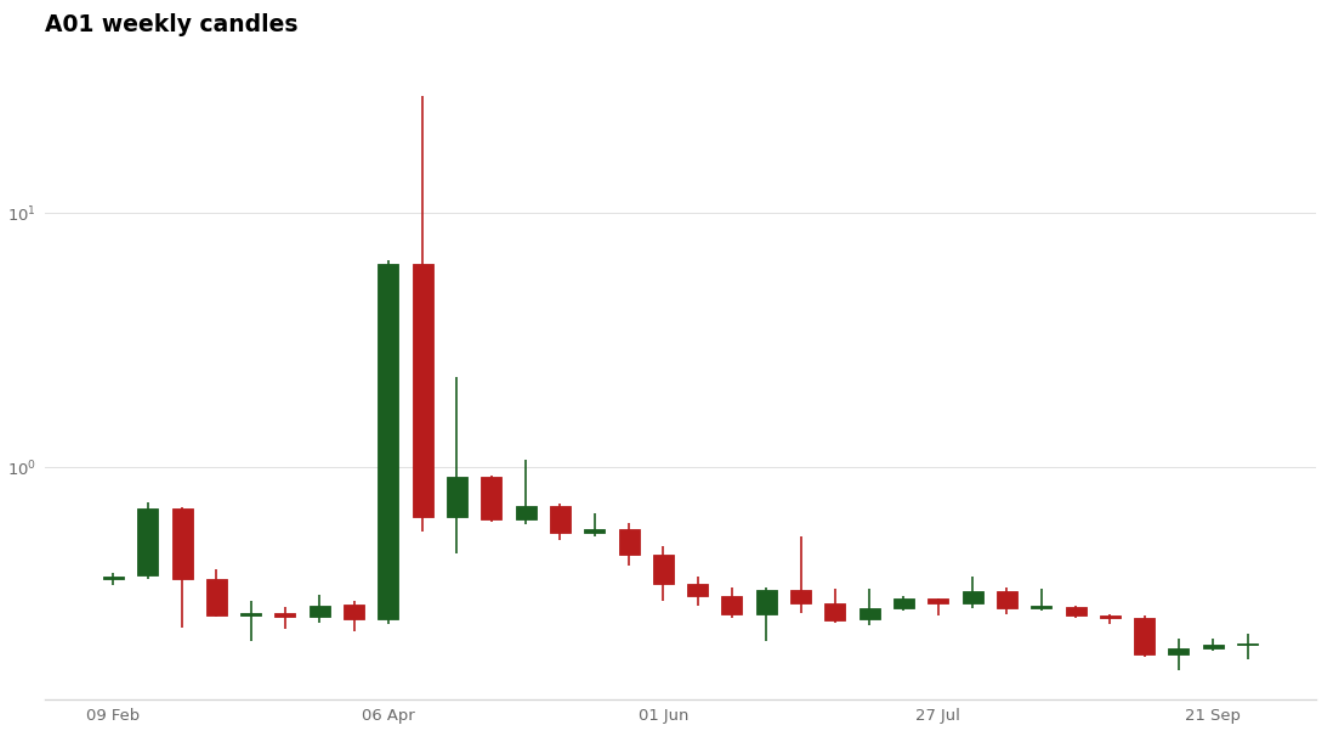
Each of the 38 headings below holds three charts for one base, under one label: its VWAP chart, its weekly chart and its price trace. The VWAP chart draws the fills. The weekly chart and the price trace draw the market those fills ran in, from the Stone Tablet rows the next two sections set out.

A fourth figure follows those three: the venue's own position card for that base, obscured as the position-card section below sets out. A33 is the one base the venue supplied no card for, and its heading holds three figures and a line saying so.

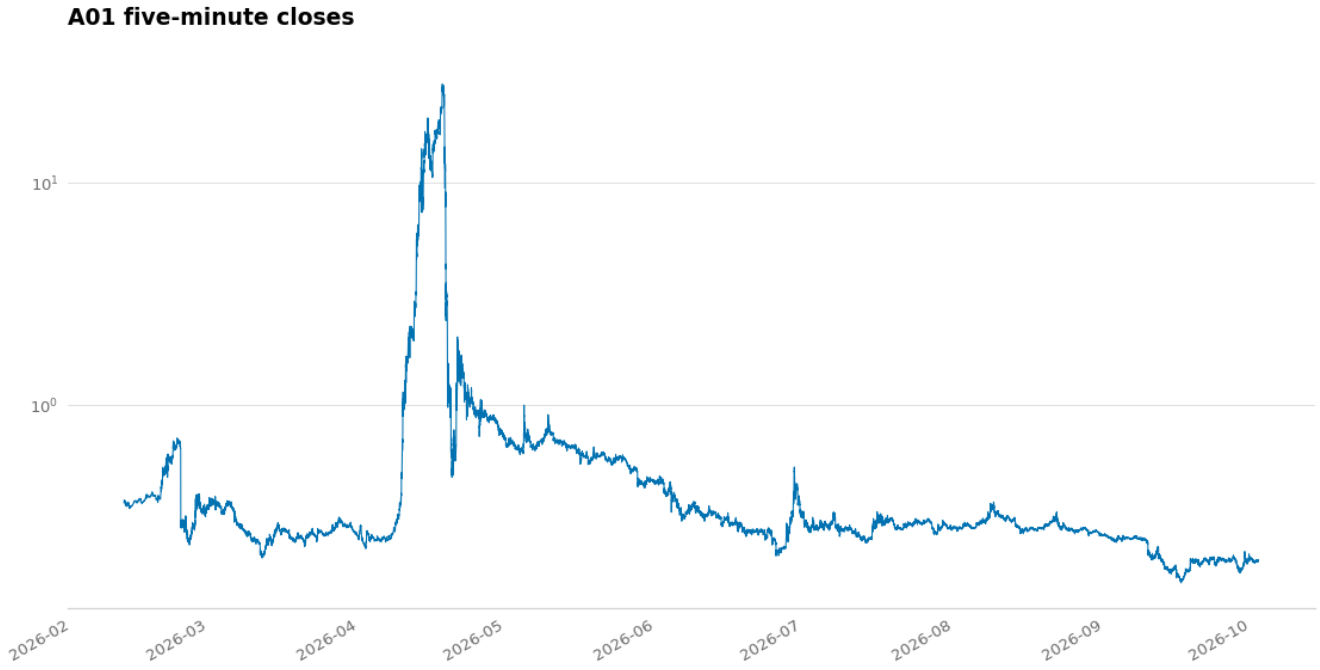
A01



A01, 900 fills, a logarithmic price axis, and a net-units panel that empties twice.



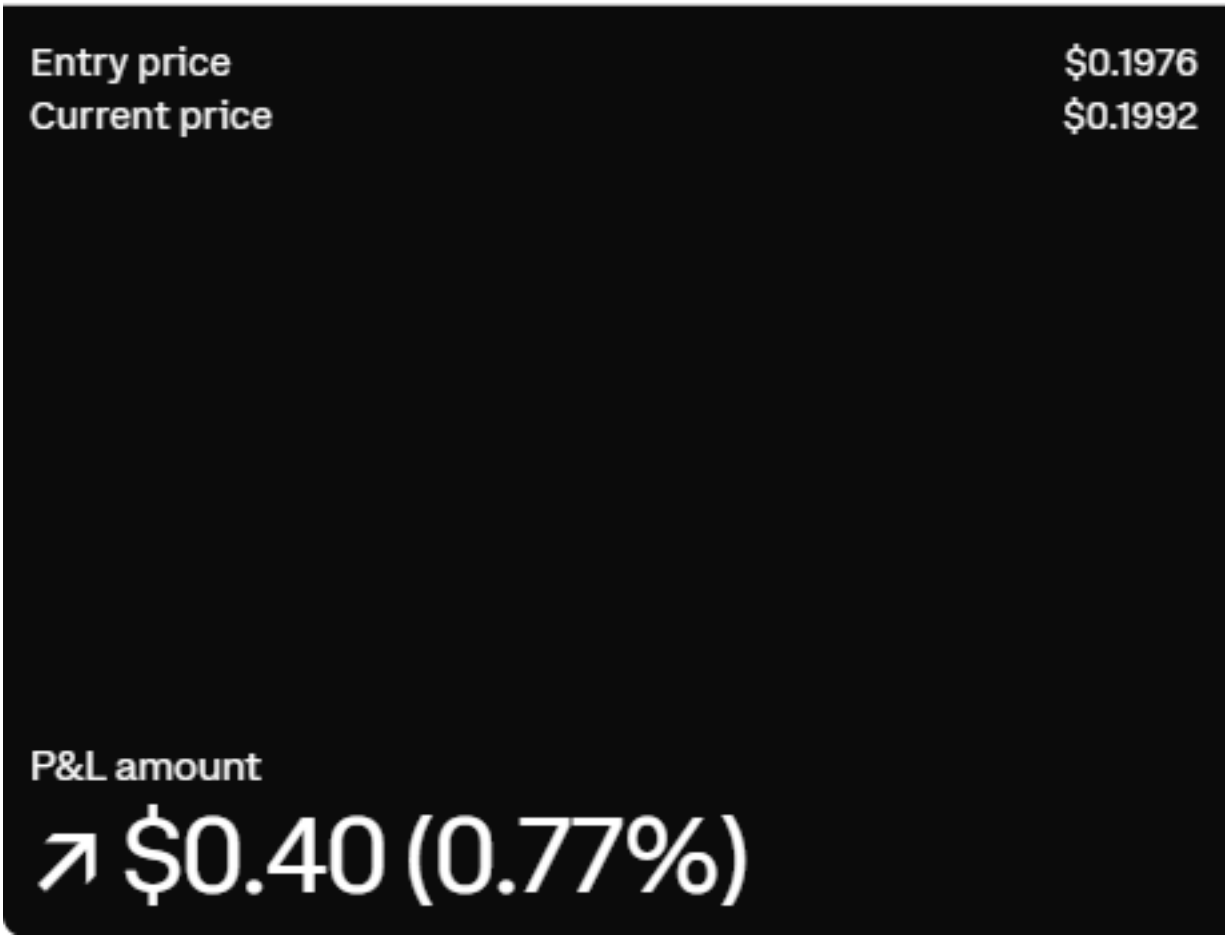
A01 weekly chart, 34 weekly candles built from 50,380 five-minute rows.



A01 price trace, 50,380 five-minute closes across the recorded window.

coinbase ADVANCED

A01

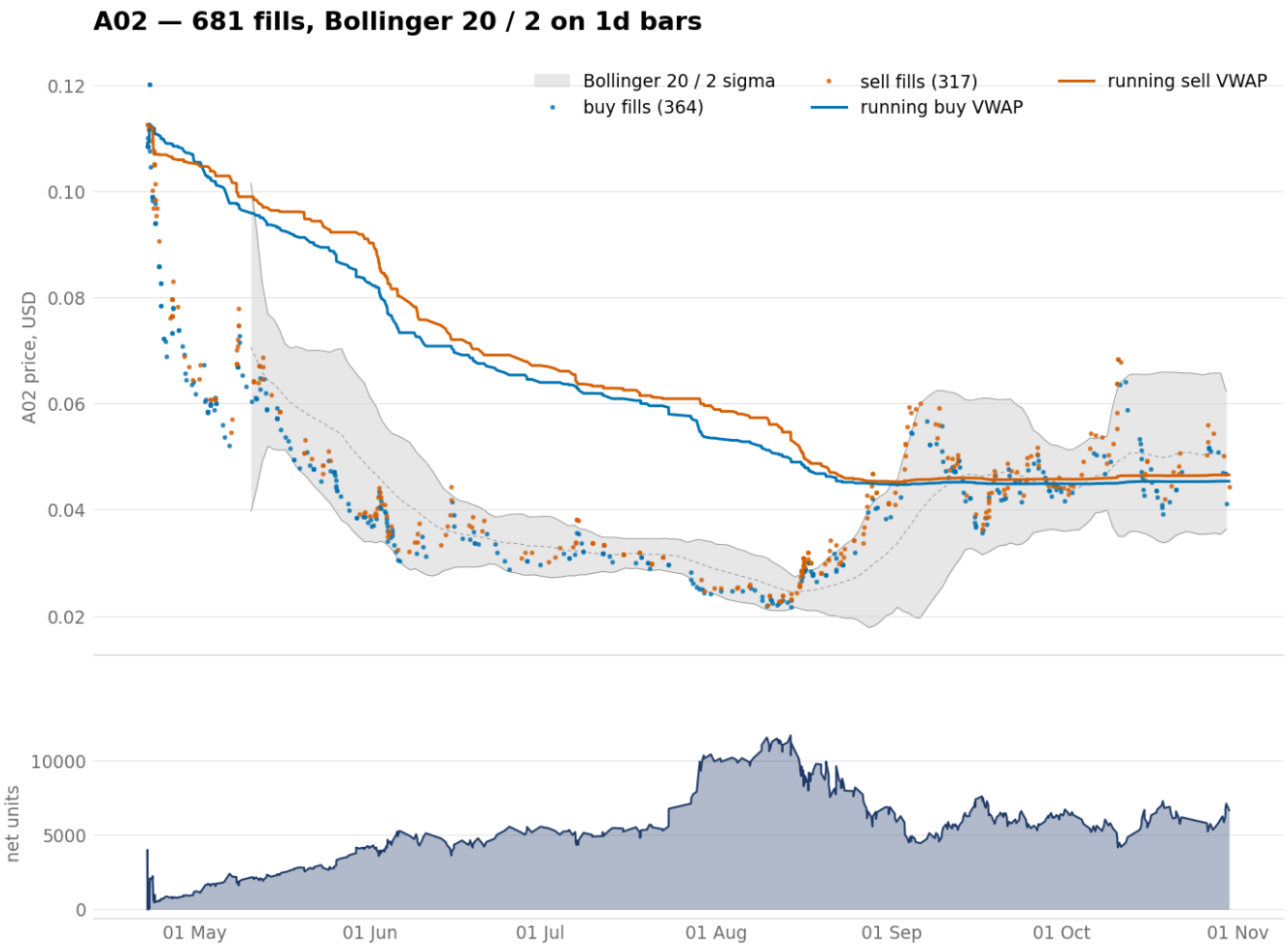


A01 position card, with the mark, the name and the code square painted out.

The axis is a measurement, not a choice. A01's fills run from 0.2118 to 2.1805, a span of 10.30, and the module turns the axis logarithmic above eight. A03 is the only other chart over that line.

```
    lo = min(f.price for f in fills)
    hi = max(f.price for f in fills)
    if hi / lo > 8:
        ax.set_yscale("log")
```

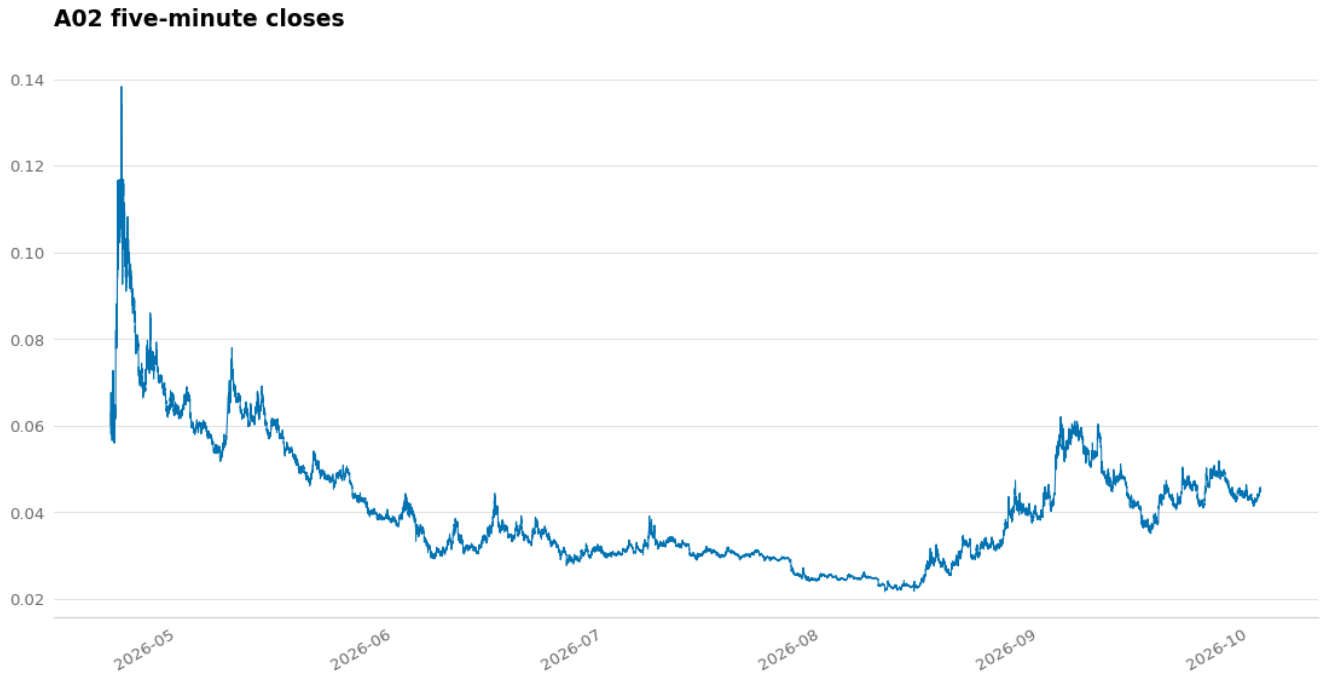
A02



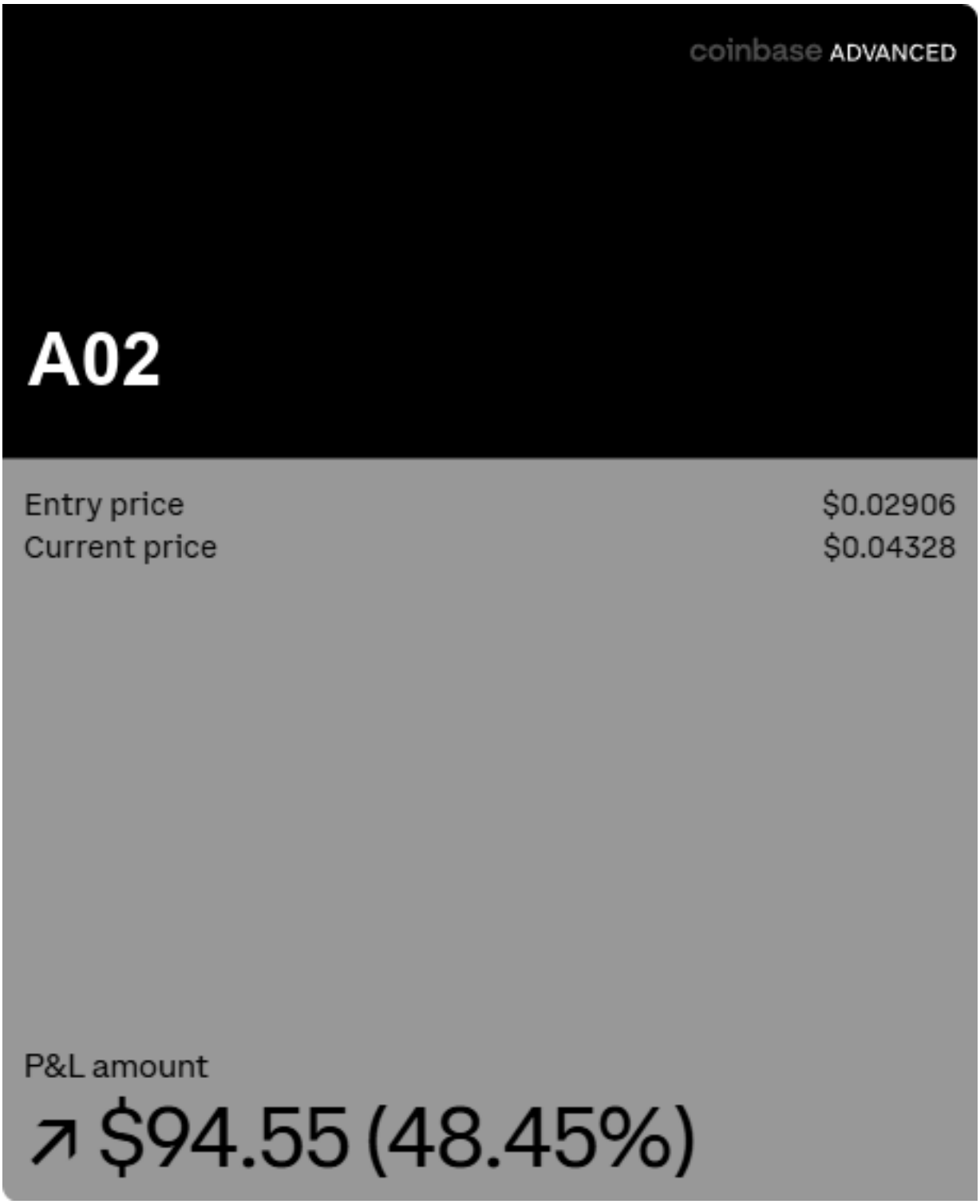
A02, 495 fills, with the sell line above the buy line for the whole window.



A02 weekly chart, 24 weekly candles built from 38,463 five-minute rows.

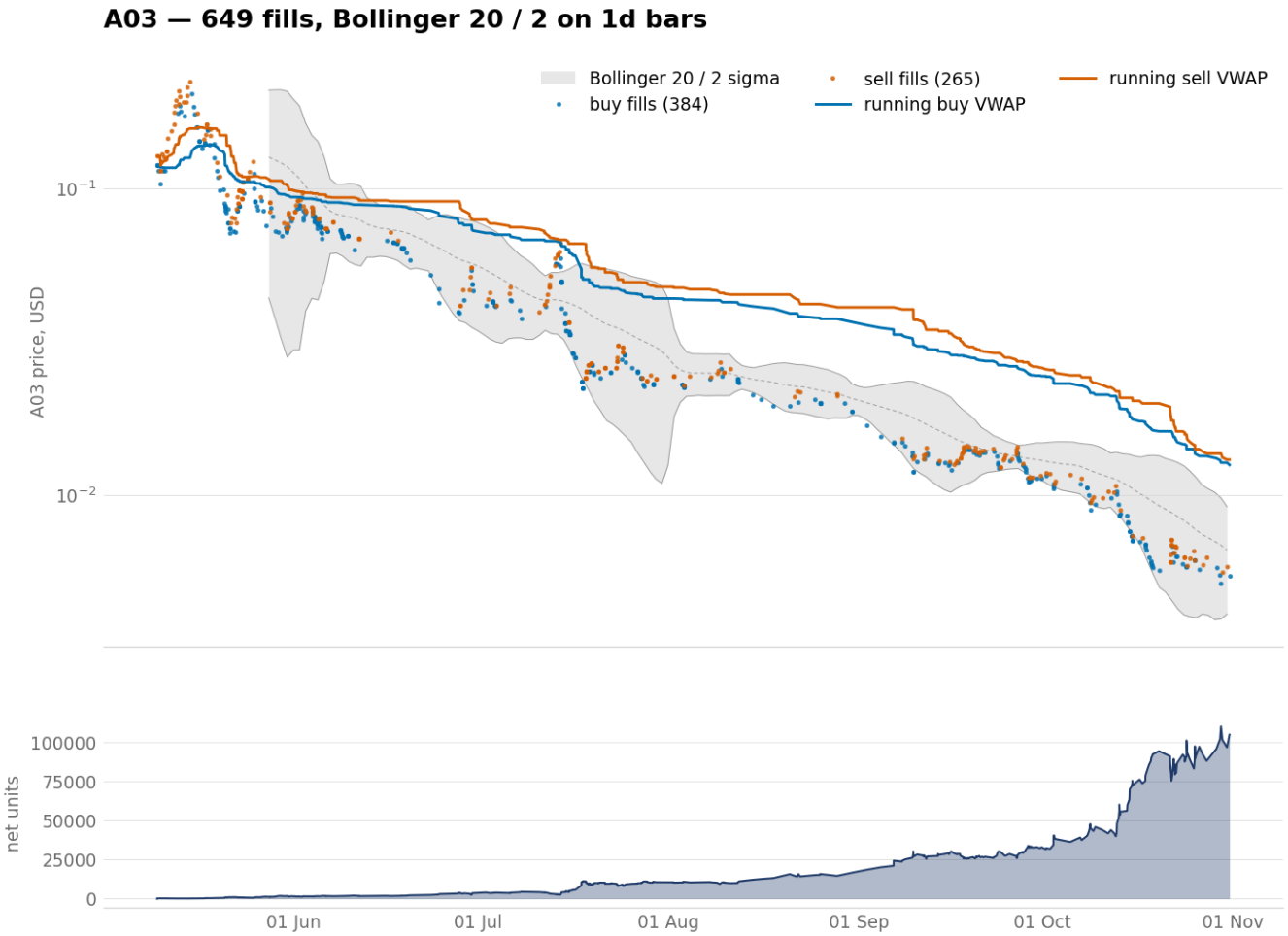


A02 price trace, 38,463 five-minute closes across the recorded window.



A02 position card, with the mark, the name and the code square painted out.

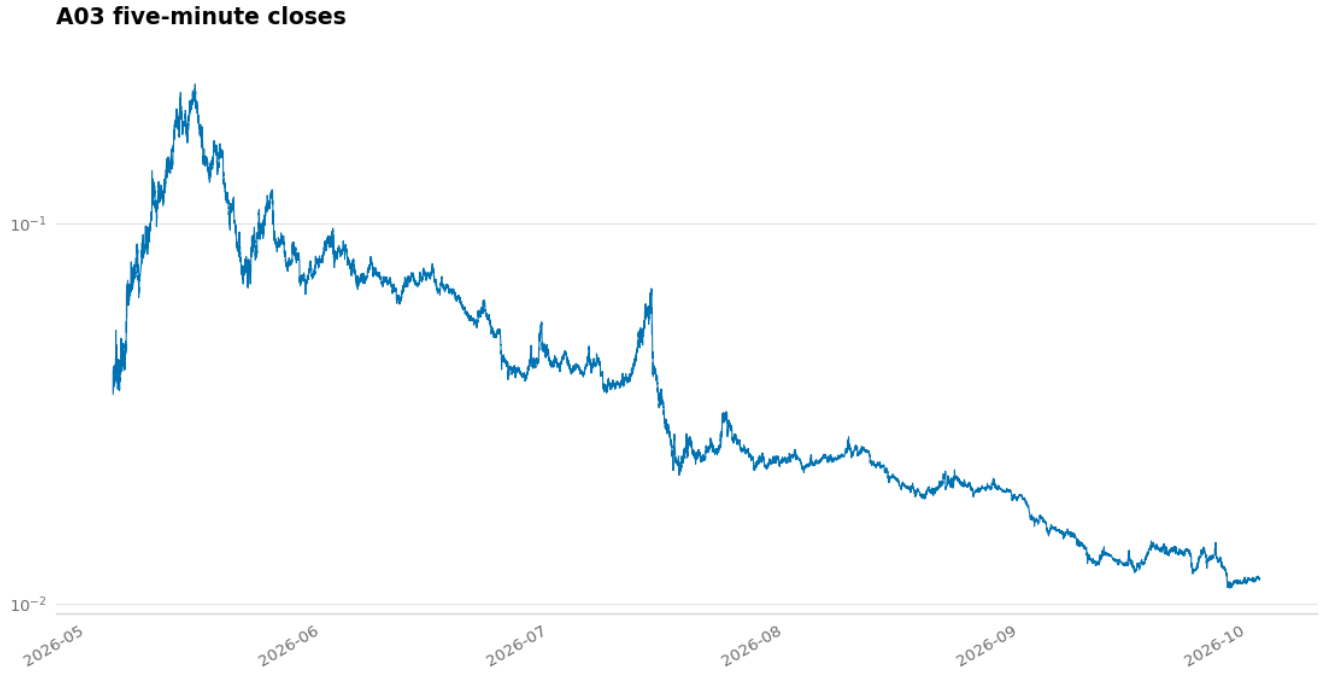
A03



A03, 459 fills, ending at the lowest index of the eight most-traded assets.

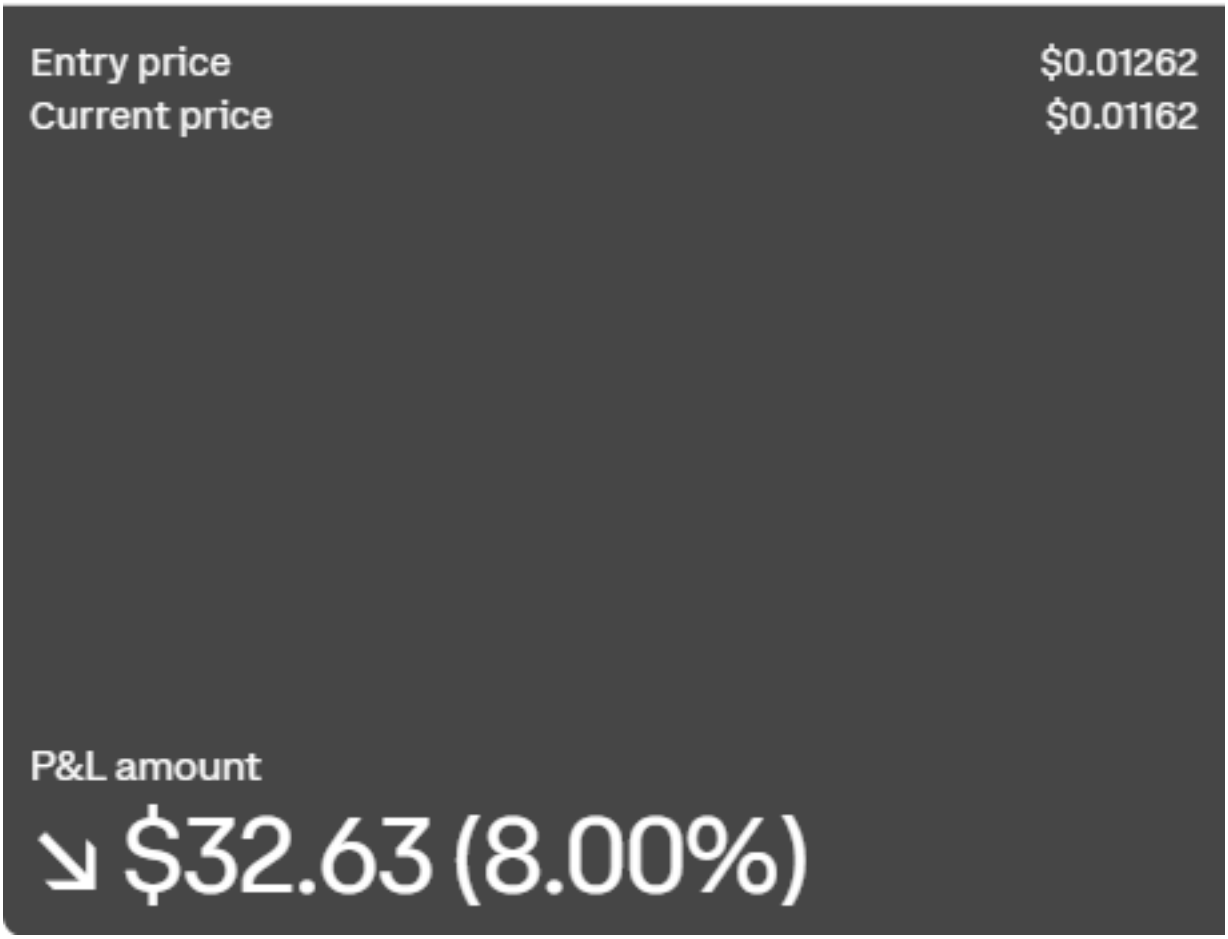


A03 weekly chart, 22 weekly candles built from 39,186 five-minute rows.



A03 price trace, 39,186 five-minute closes across the recorded window.

A03



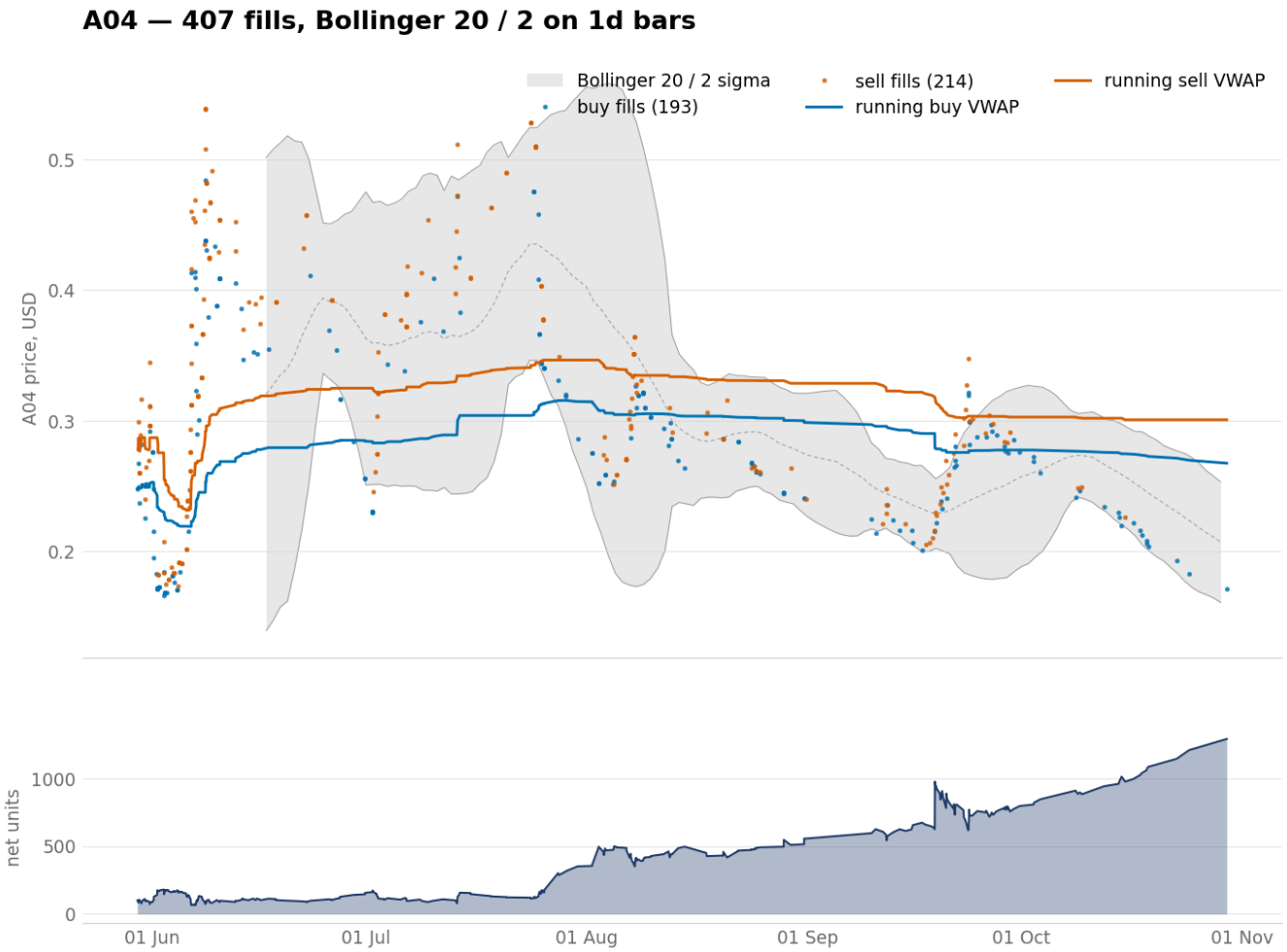
A03 position card, with the mark, the name and the code square painted out.

A logarithmic axis labels the powers of ten. A03's fills run from 0.01682 to 0.22281, a window holding one of them, and A01's window holds one as well. The module adds five minor ticks per decade and gives them the same price formatter as the major ones, which is what puts 0.2000 down to 0.0200 on this axis.

```
SUBS = (1.0, 2.0, 3.0, 5.0, 7.0)

ax.yaxis.set_minor_locator(LogLocator(base=10.0, subs=SUBS, numticks=99))
ax.yaxis.set_minor_formatter(FuncFormatter(fmt_price))
ax.tick_params(axis="y", which="minor", labelsize=TYPE["cap"]["size"])
```

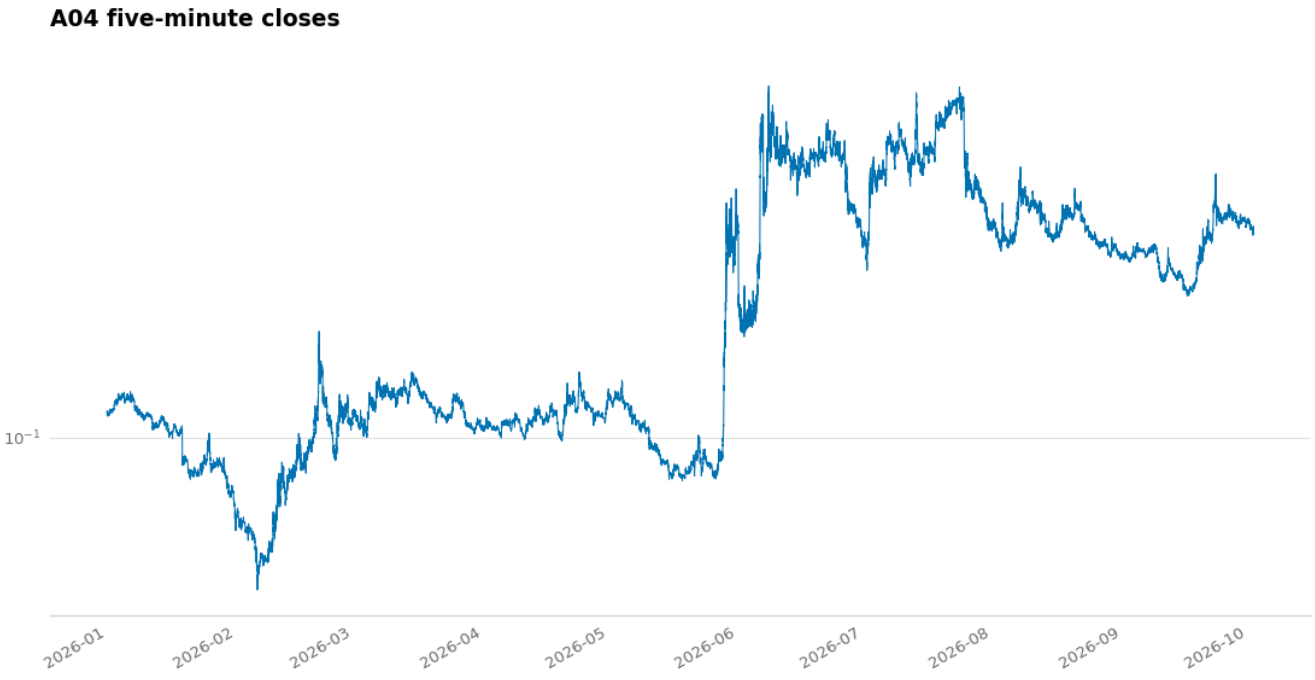
A04



A04, 322 fills, with more sells than buys and both lines rising.



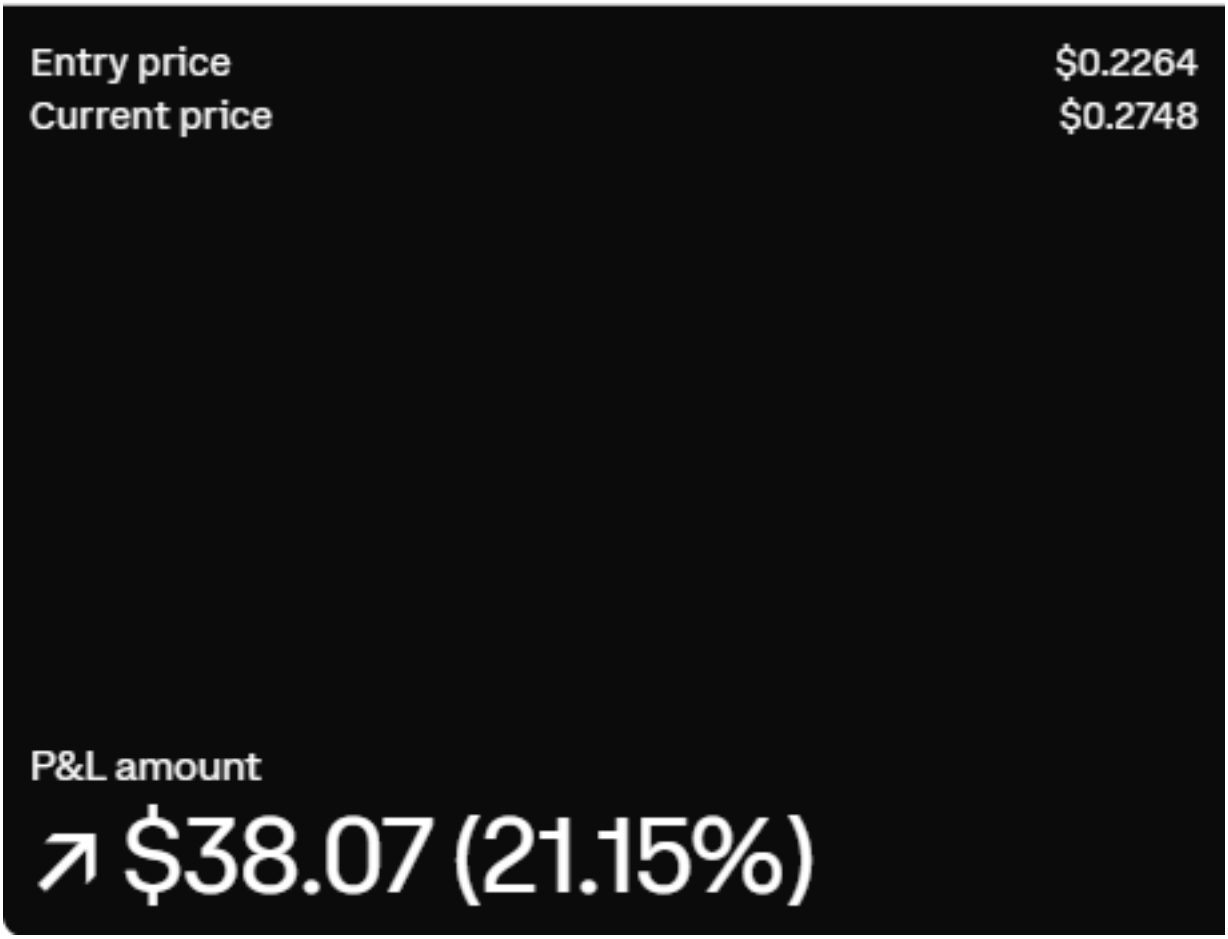
A04 weekly chart, 40 weekly candles built from 47,201 five-minute rows.



A04 price trace, 47,201 five-minute closes across the recorded window.

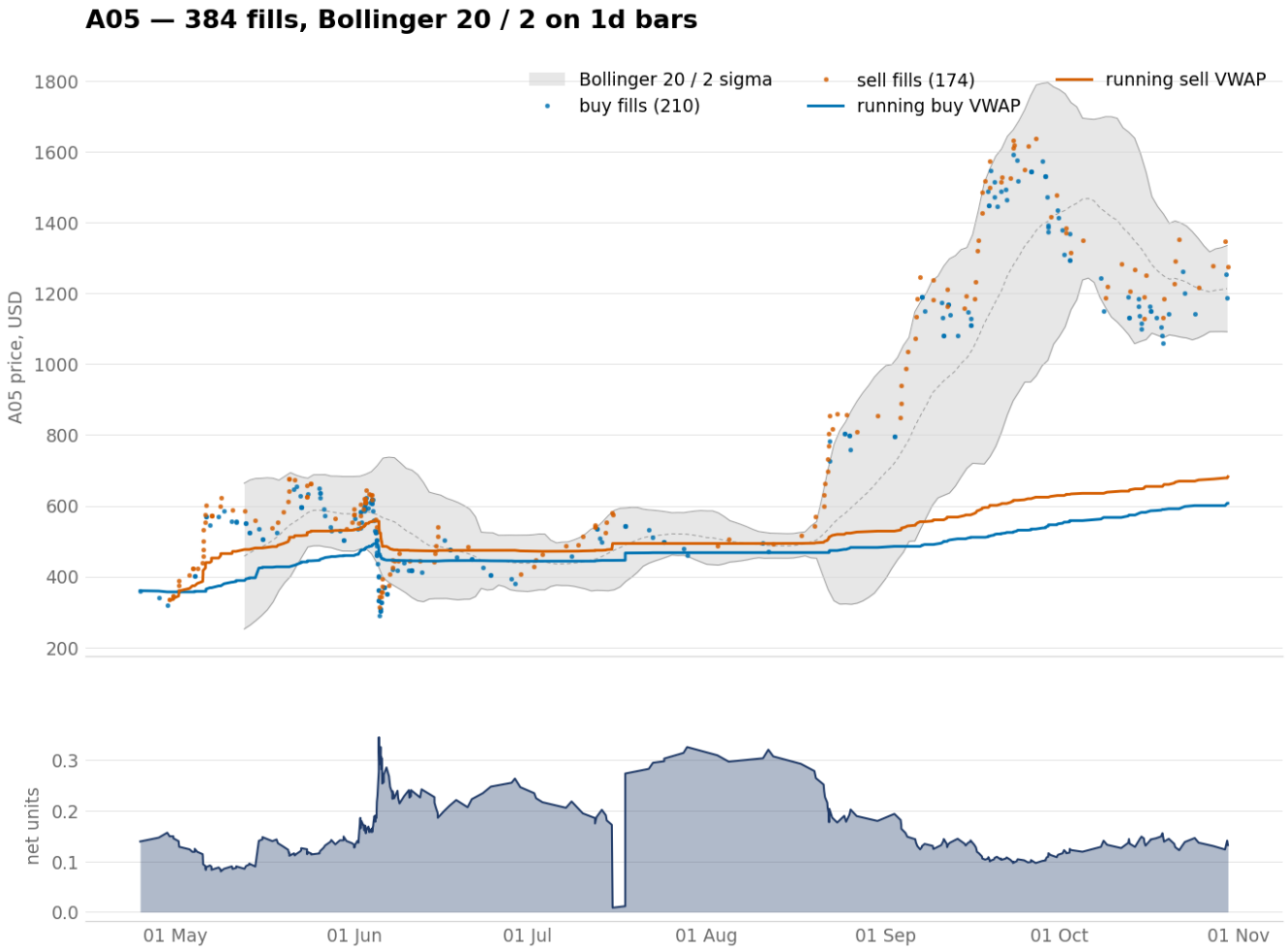
coinbase ADVANCED

A04



A04 position card, with the mark, the name and the code square painted out.

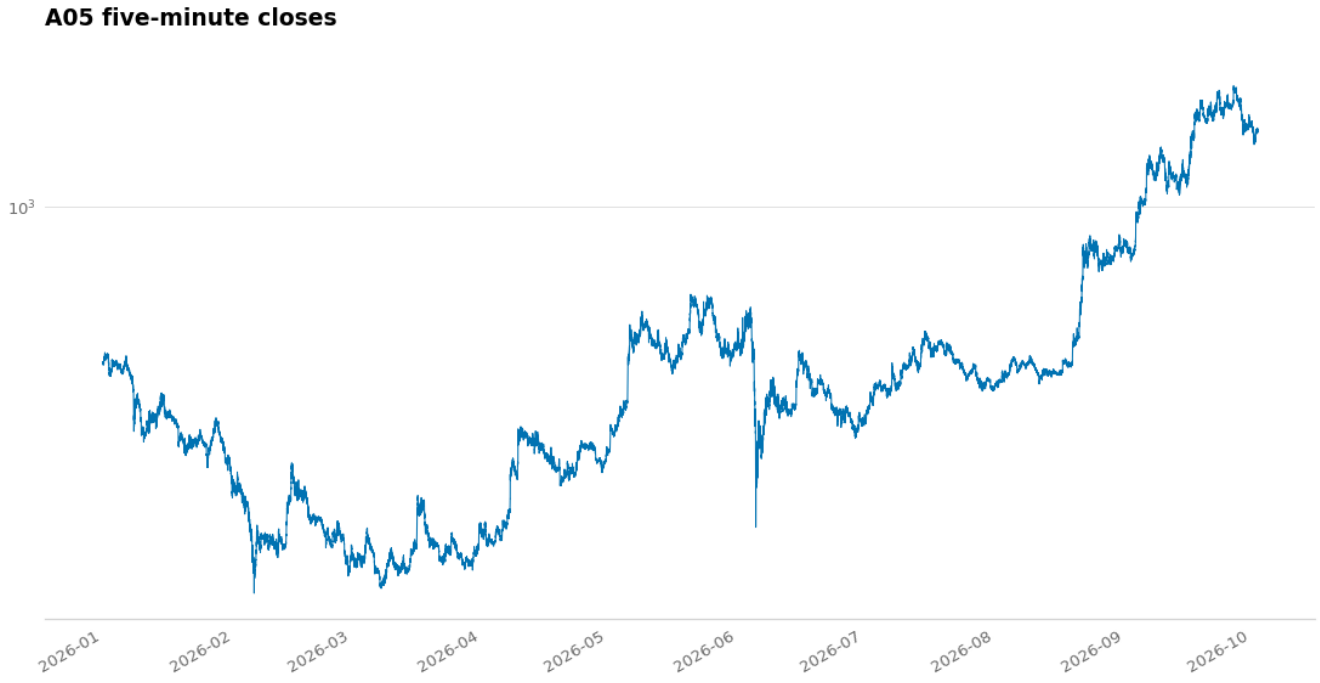
A05



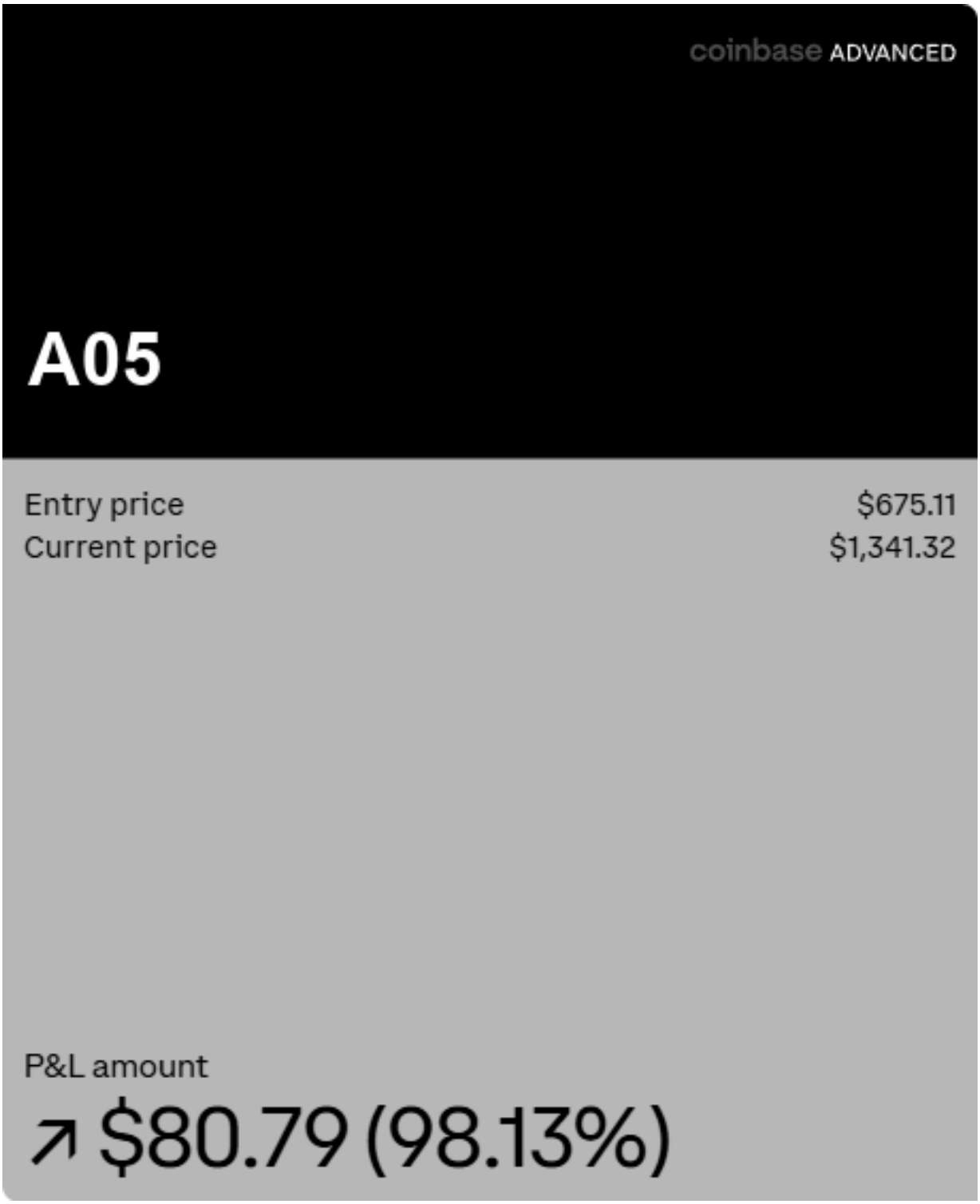
A05, 261 fills, the highest final index of the eight most-traded assets.



A05 weekly chart, 40 weekly candles built from 79,004 five-minute rows.

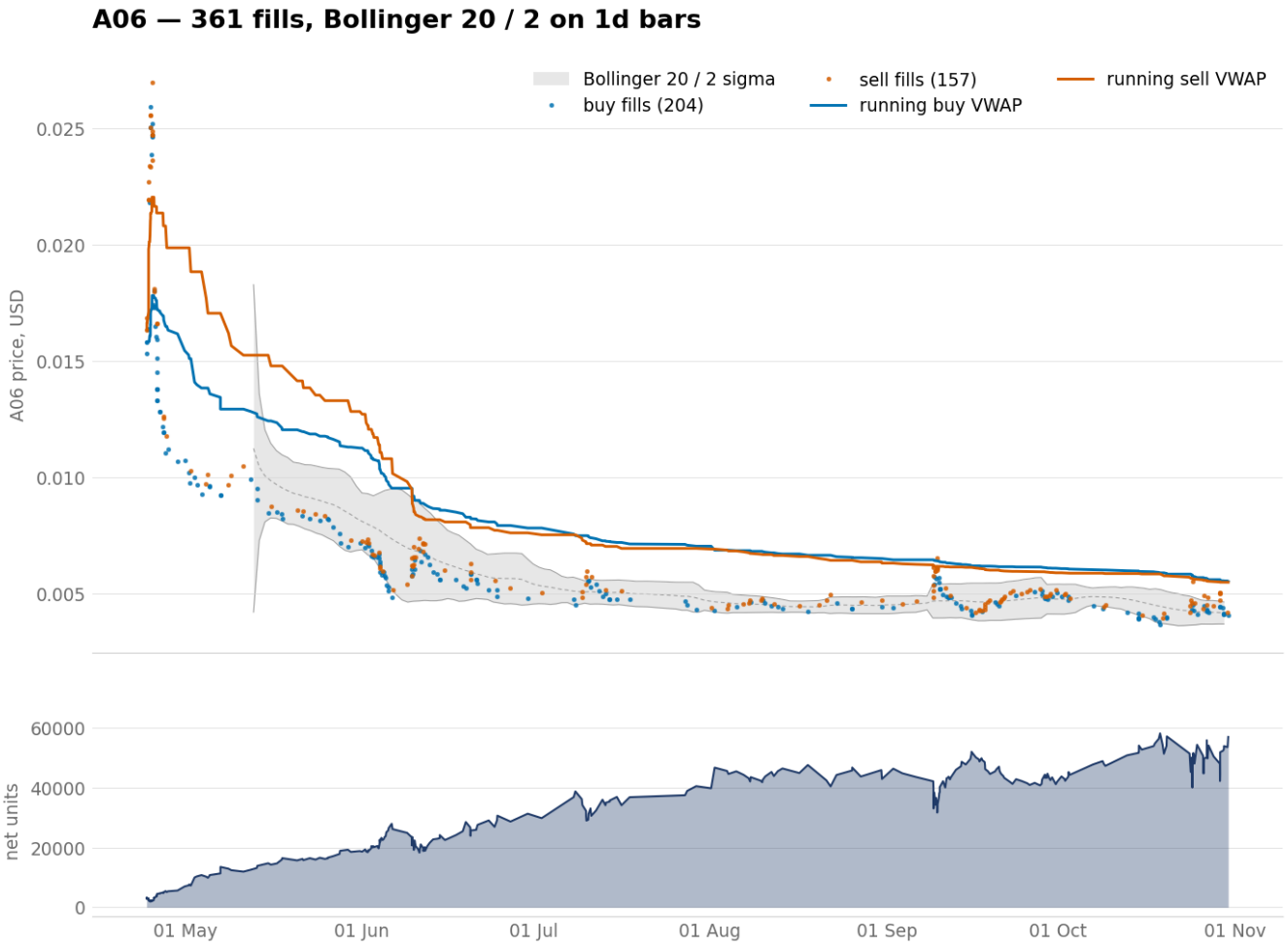


A05 price trace, 79,004 five-minute closes across the recorded window.

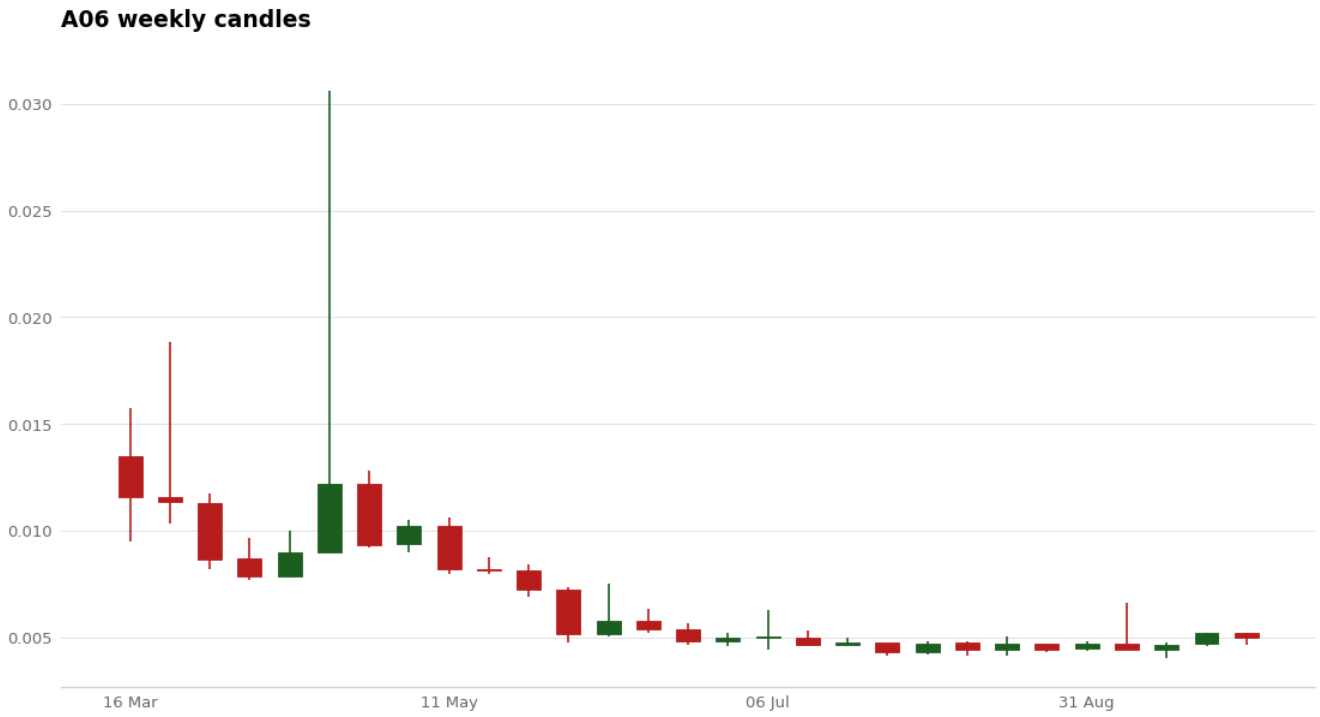


A05 position card, with the mark, the name and the code square painted out.

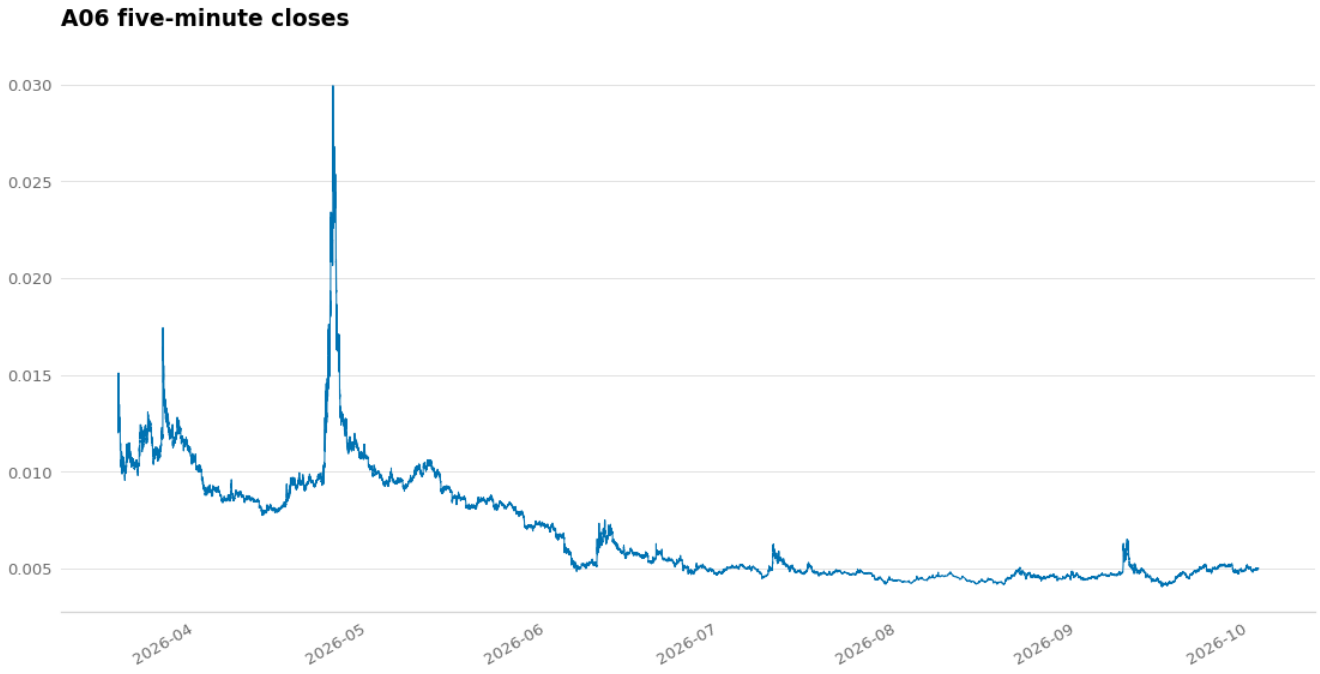
A06



A06, 230 fills, with the two lines converging from mid-June.



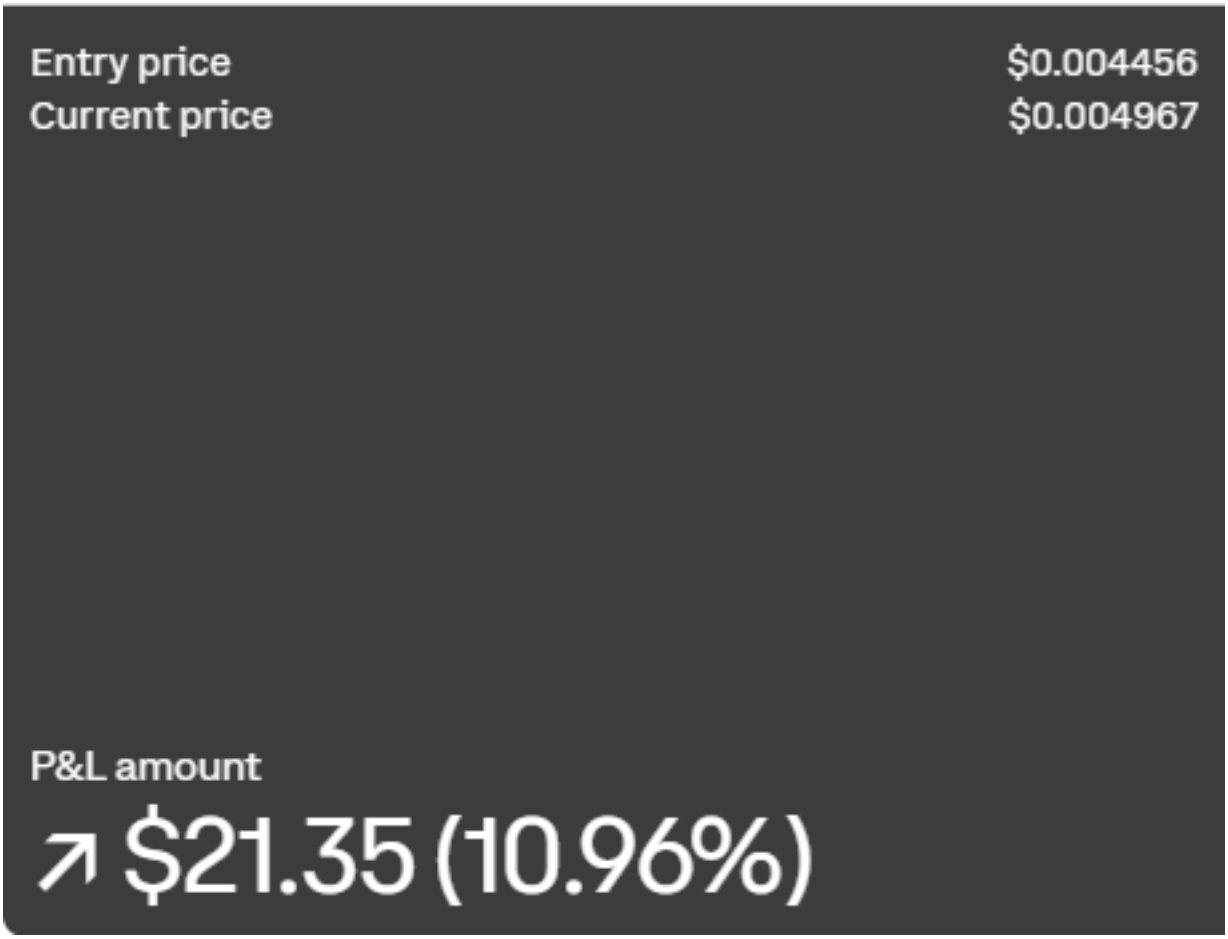
A06 weekly chart, 29 weekly candles built from 33,344 five-minute rows.



A06 price trace, 33,344 five-minute closes across the recorded window.

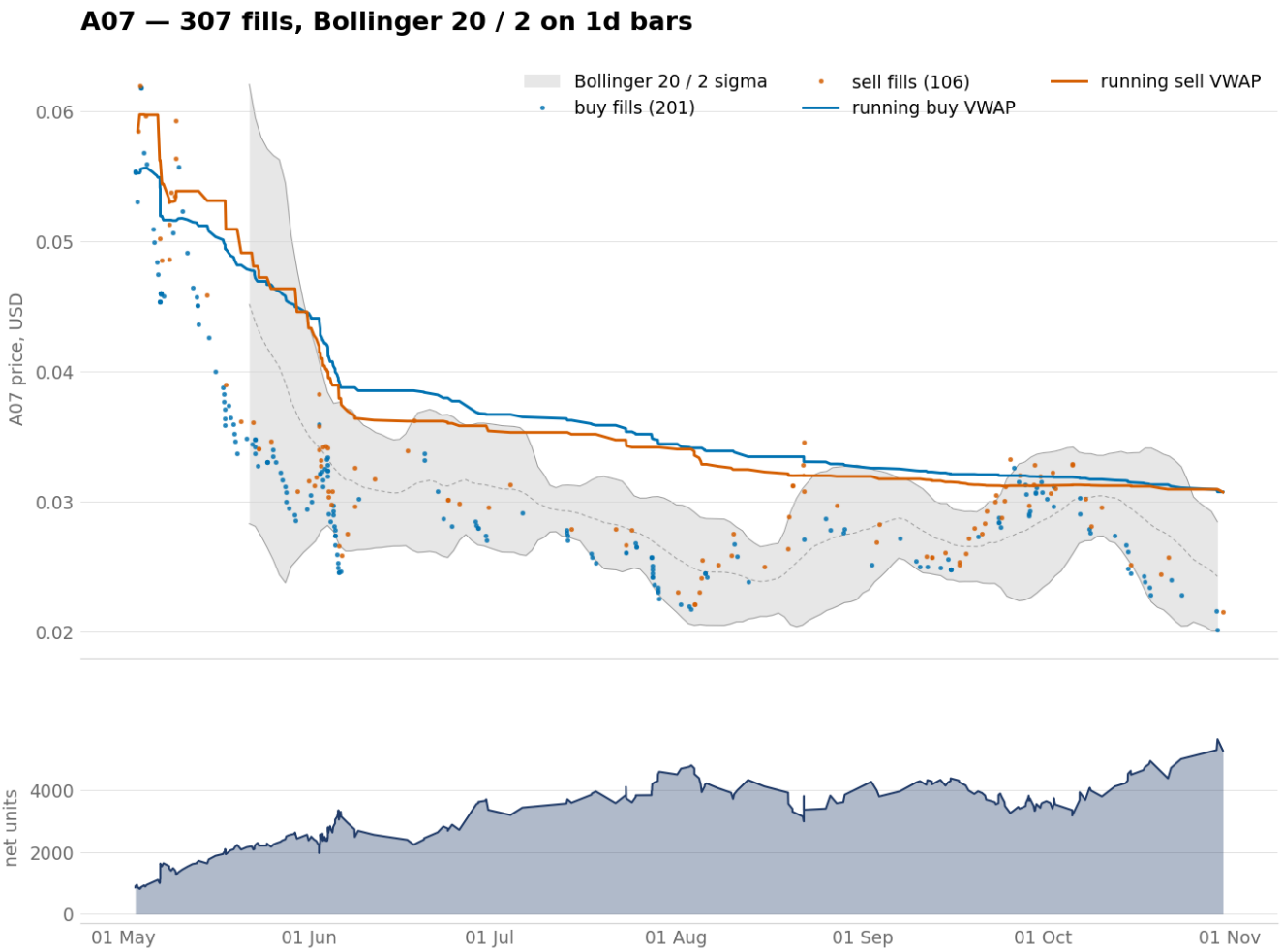
coinbase ADVANCED

A06

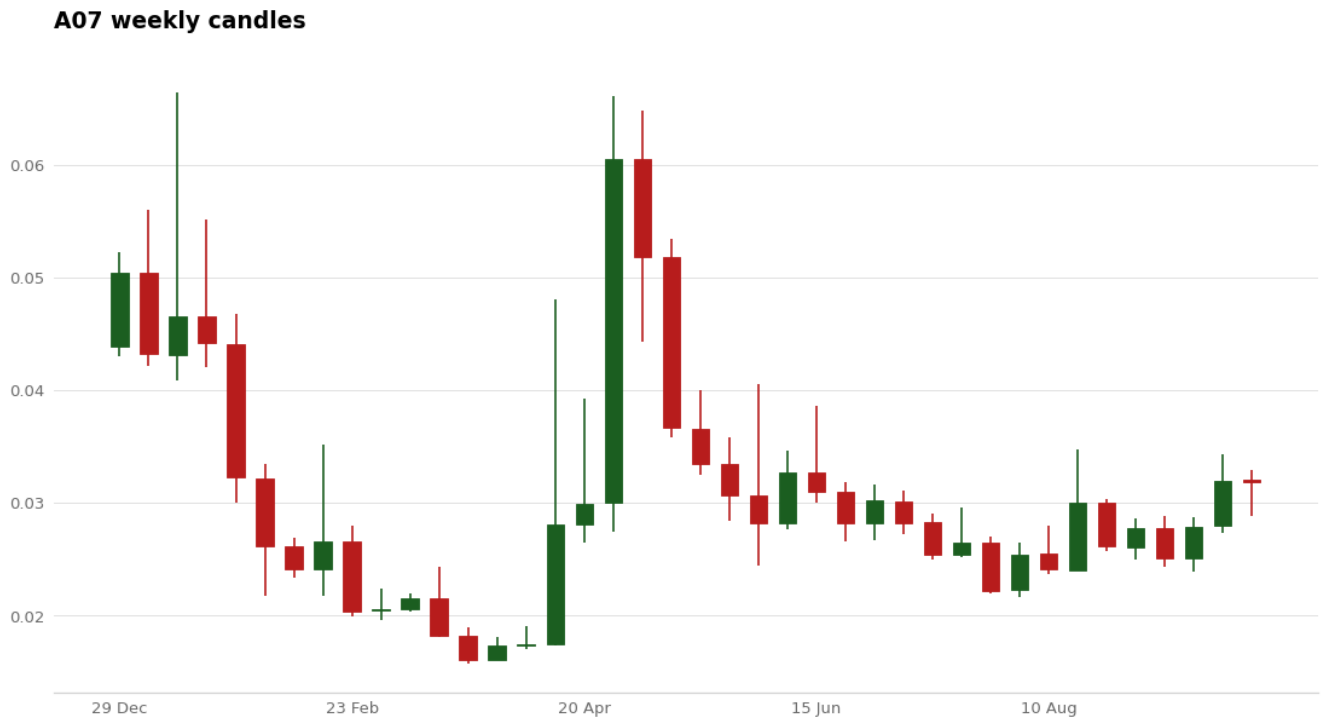


A06 position card, with the mark, the name and the code square painted out.

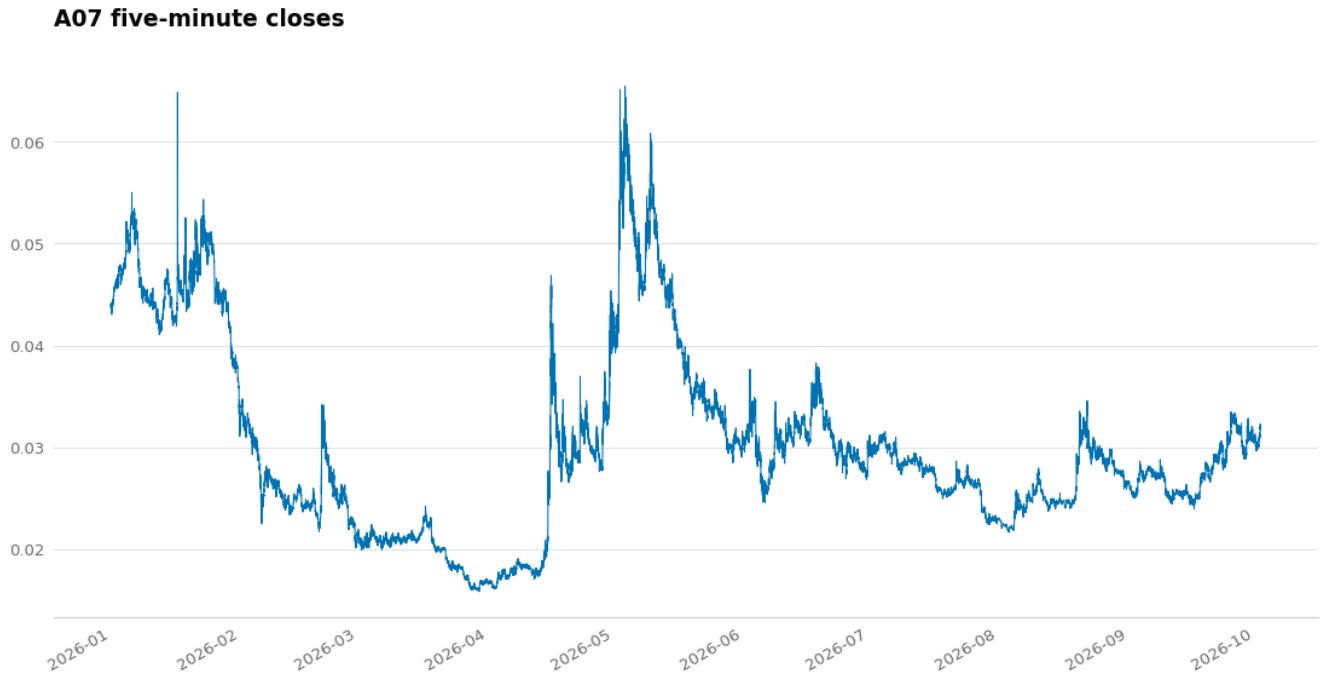
A07



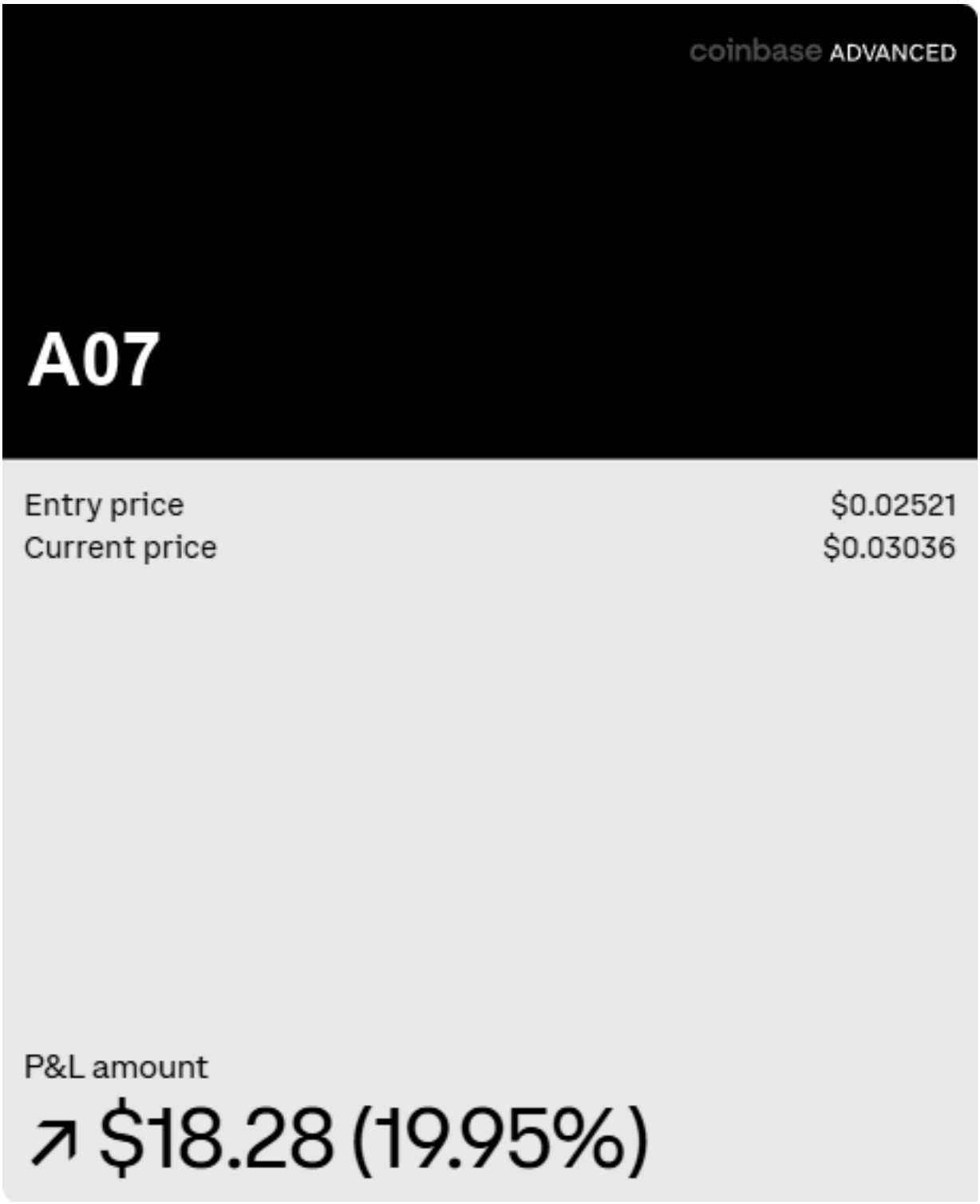
A07, 226 fills, with a step down at the start of June.



A07 weekly chart, 40 weekly candles built from 49,638 five-minute rows.

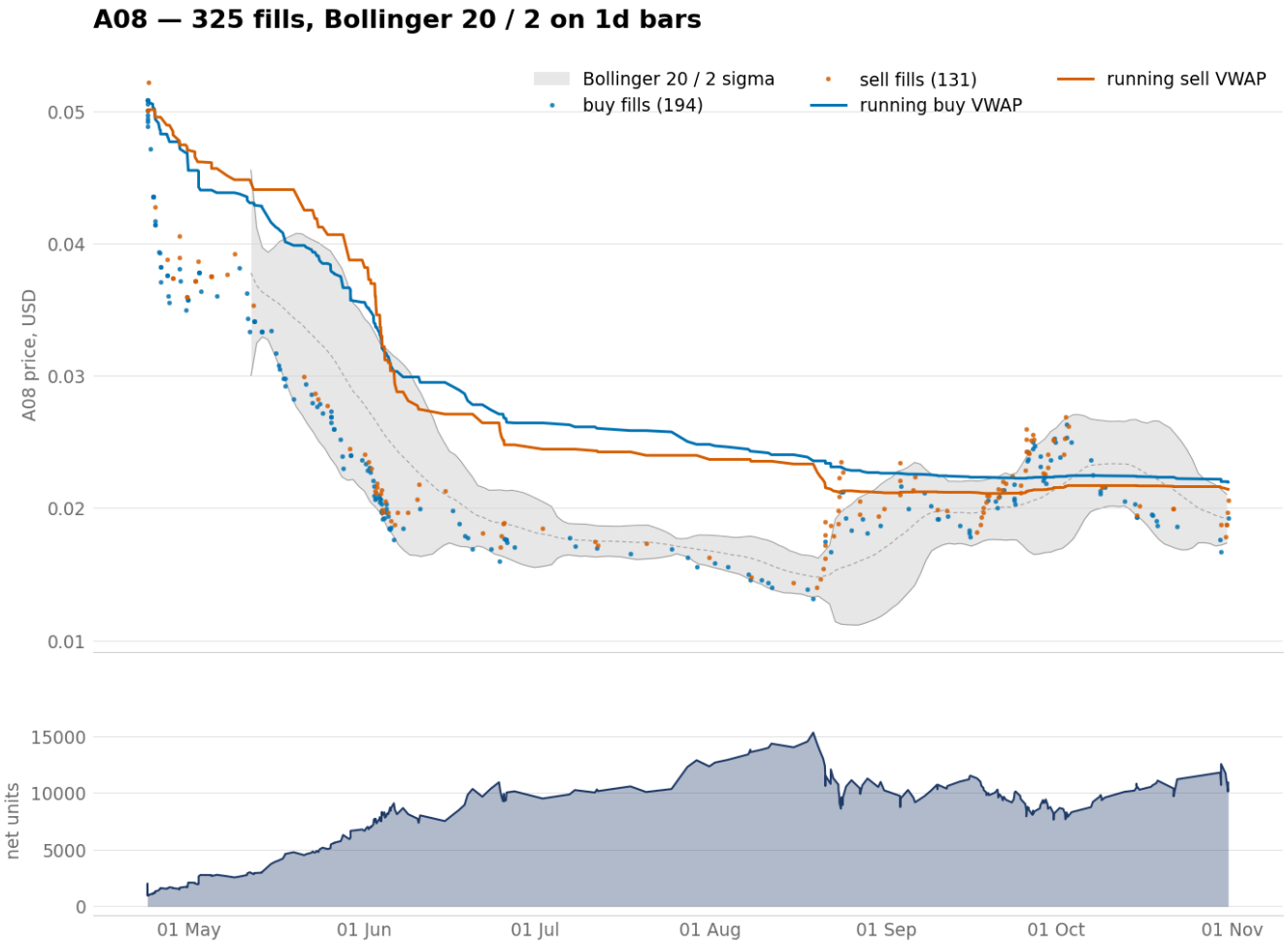


A07 price trace, 49,638 five-minute closes across the recorded window.

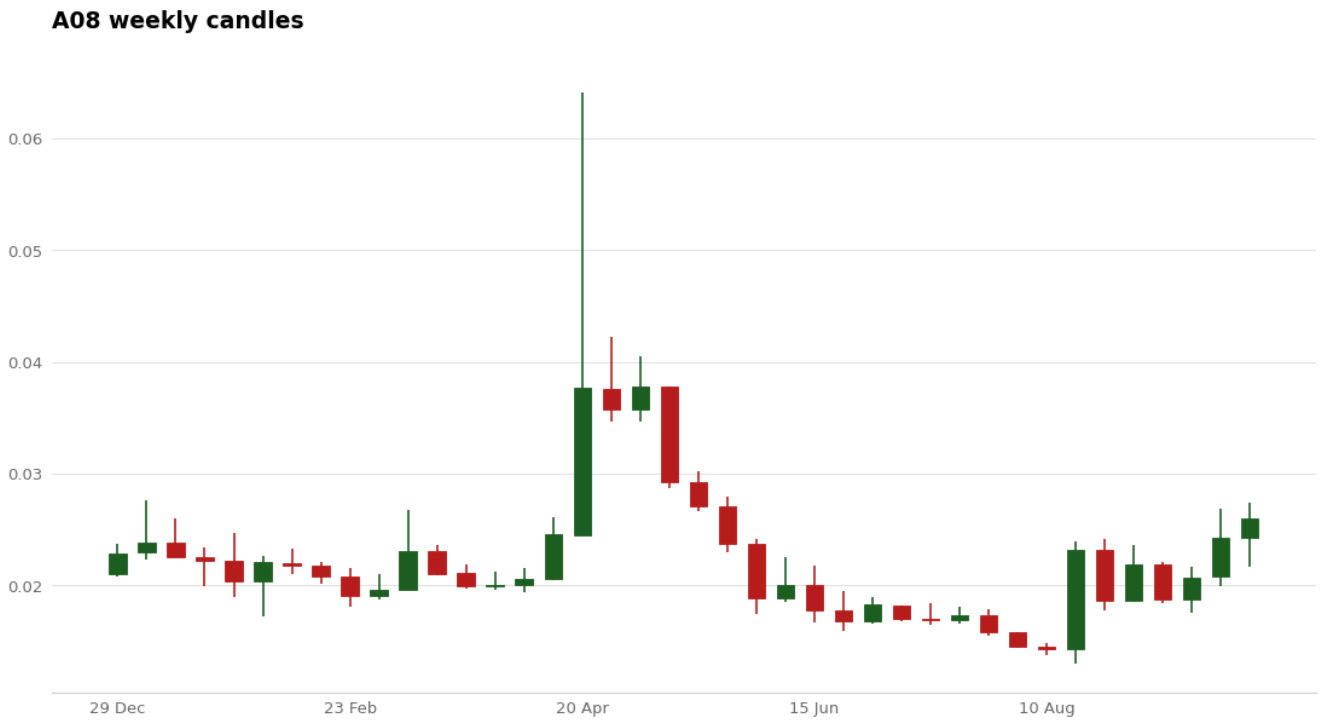


A07 position card, with the mark, the name and the code square painted out.

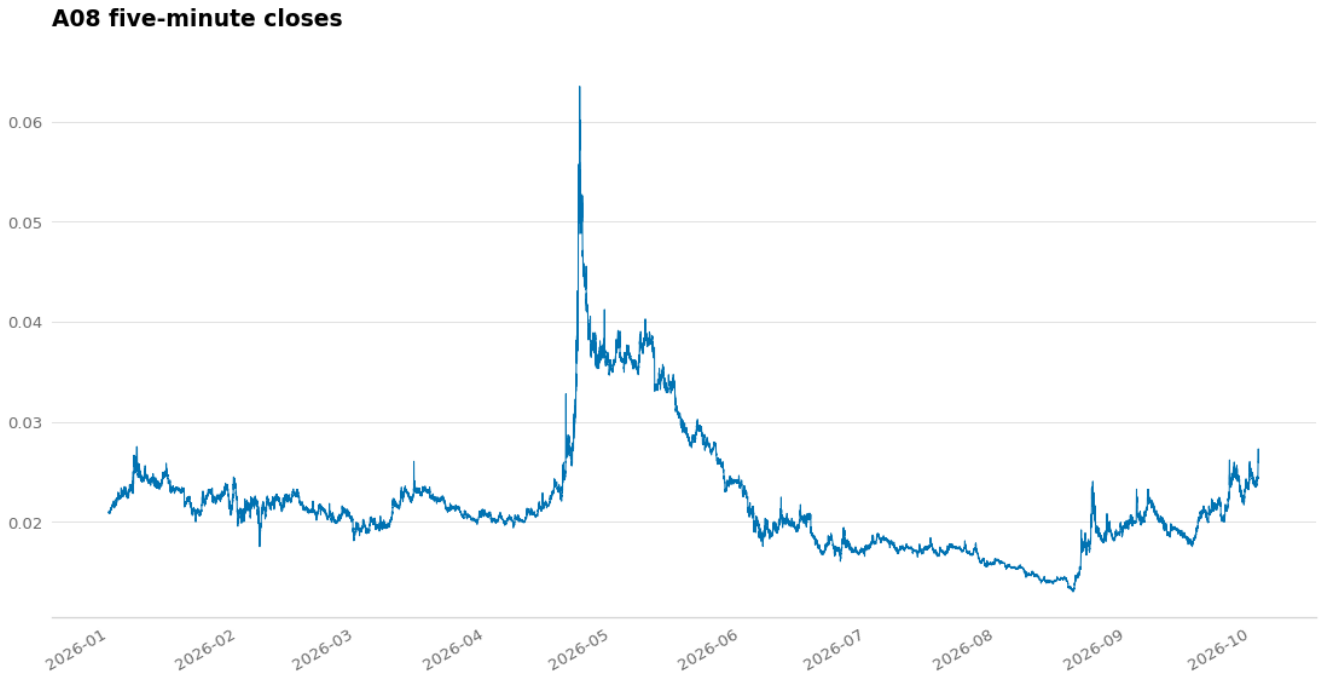
A08



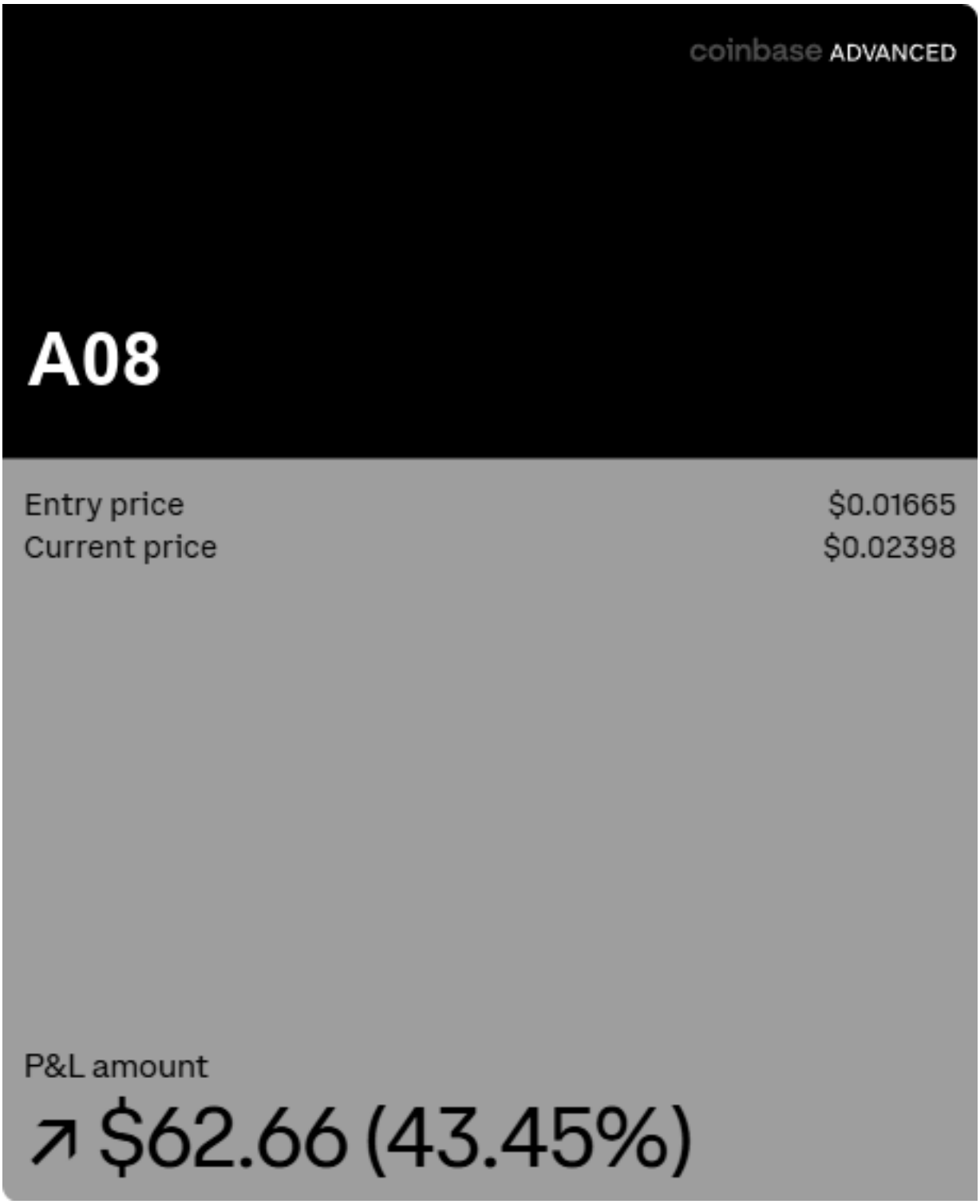
A08, 220 fills, with the sell line crossing below the buy line in mid-June.



A08 weekly chart, 40 weekly candles built from 52,751 five-minute rows.

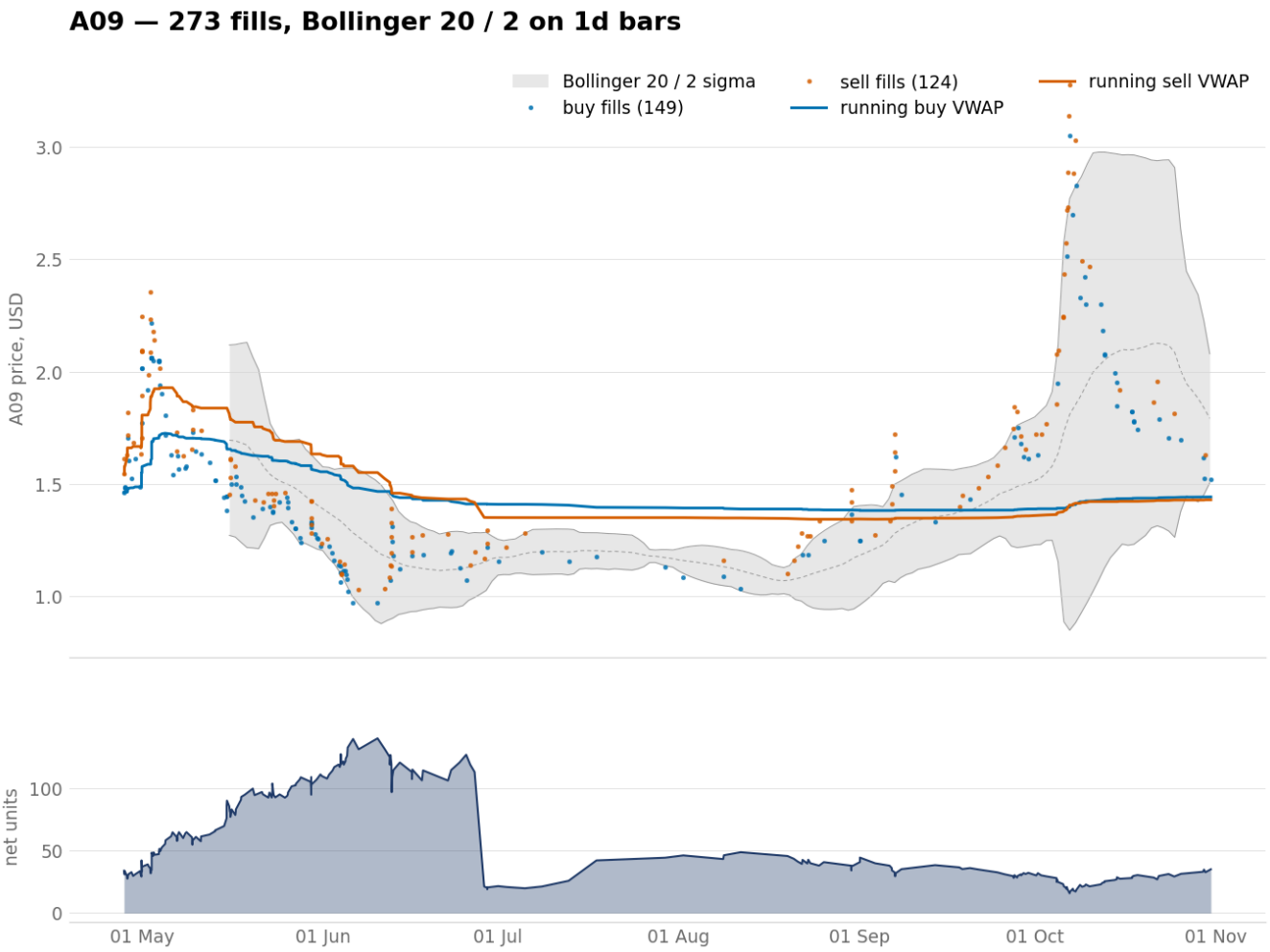


A08 price trace, 52,751 five-minute closes across the recorded window.



A08 position card, with the mark, the name and the code square painted out.

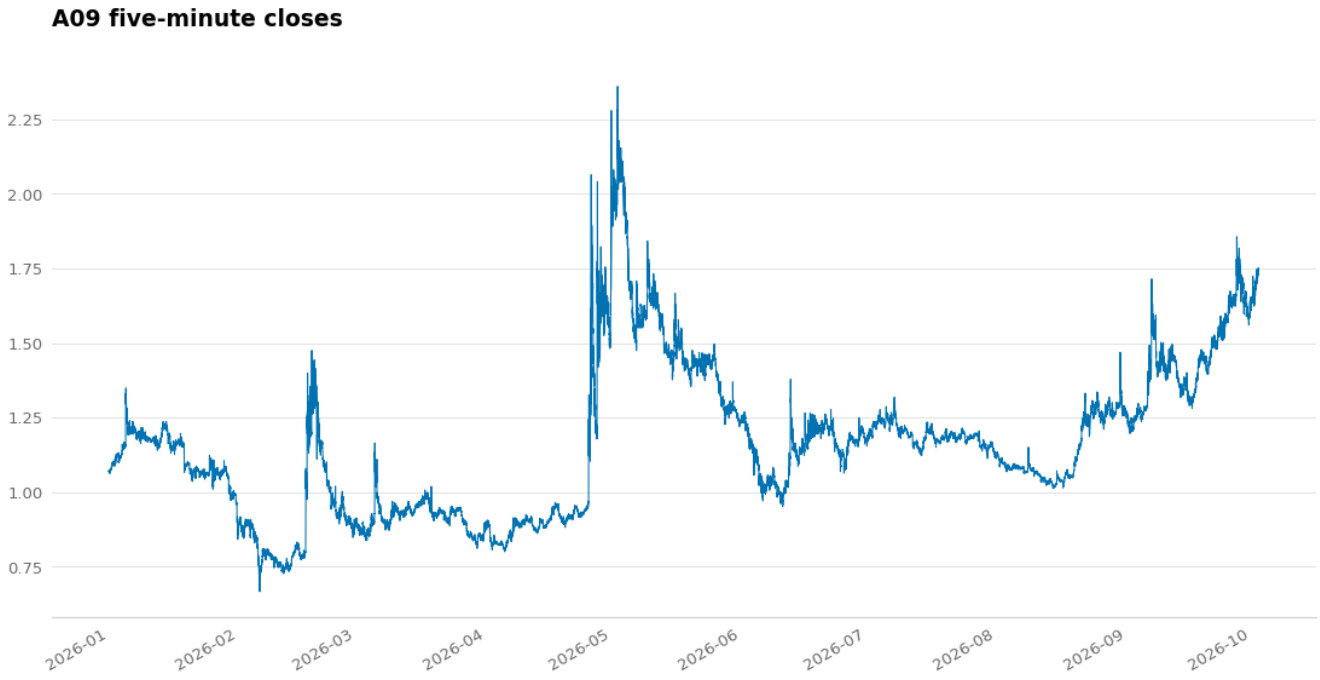
A09



A09, 196 fills, with both lines flat from July onward.



A09 weekly chart, 40 weekly candles built from 53,988 five-minute rows.



A09 price trace, 53,988 five-minute closes across the recorded window.

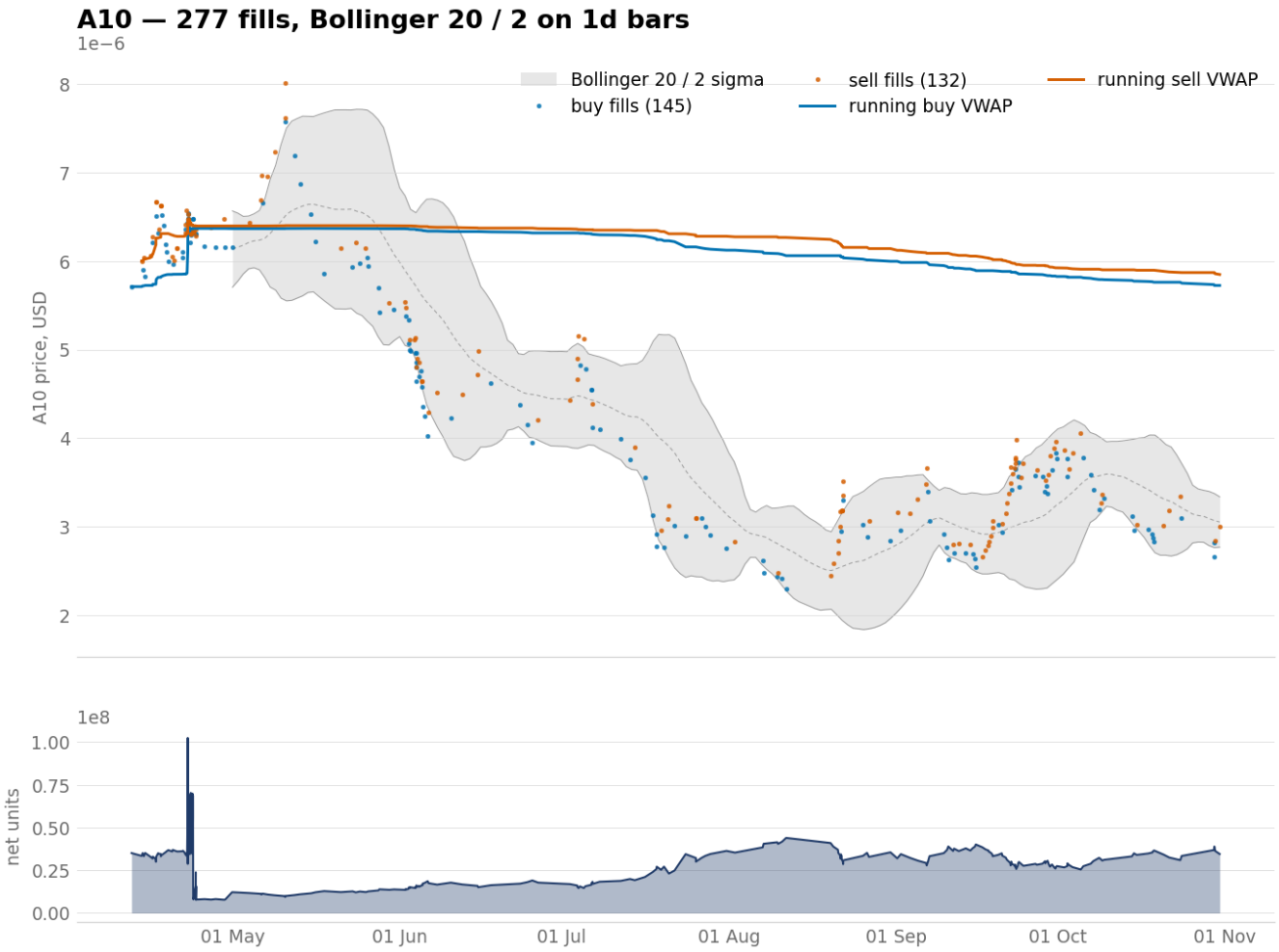
coinbase ADVANCED

A09



A09 position card, with the mark, the name and the code square painted out.

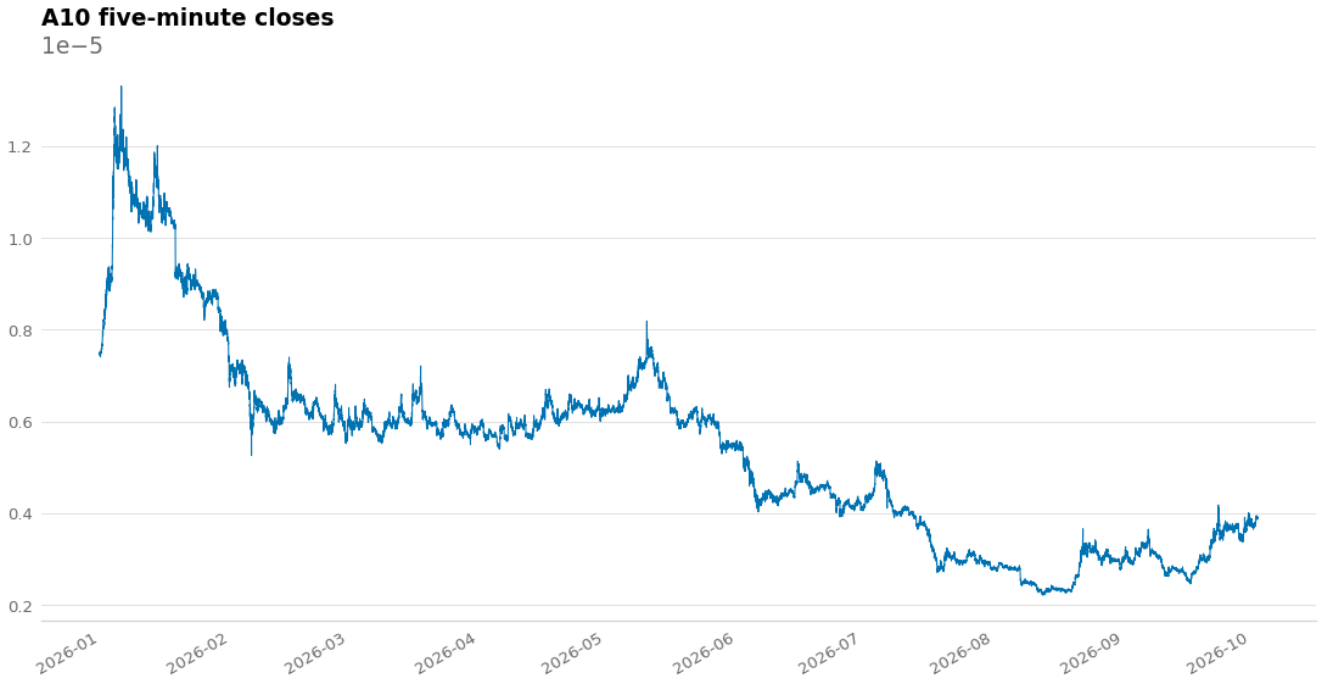
A10



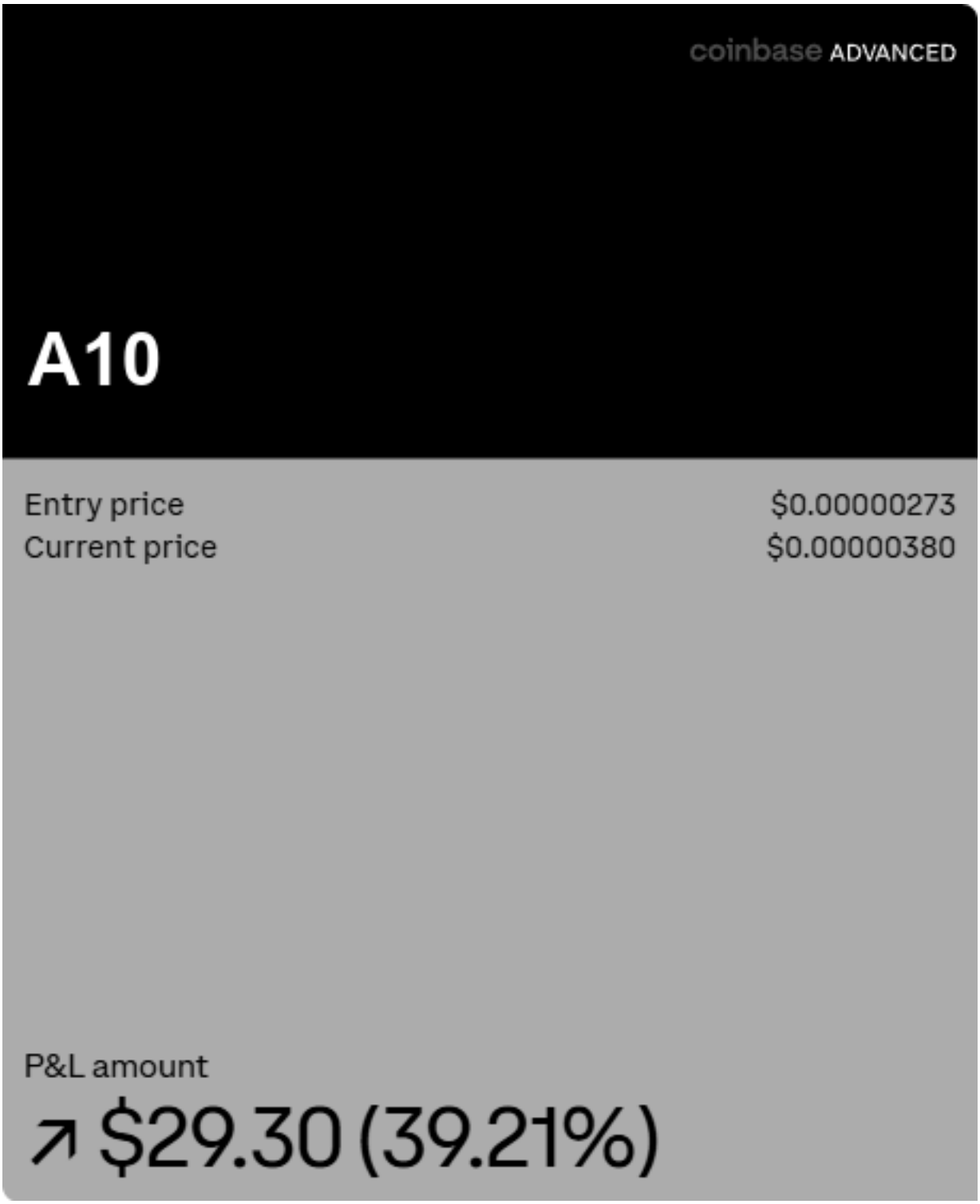
A10, 191 fills, priced near six millionths of a dollar.



A10 weekly chart, 40 weekly candles built from 72,287 five-minute rows.



A10 price trace, 72,287 five-minute closes across the recorded window.

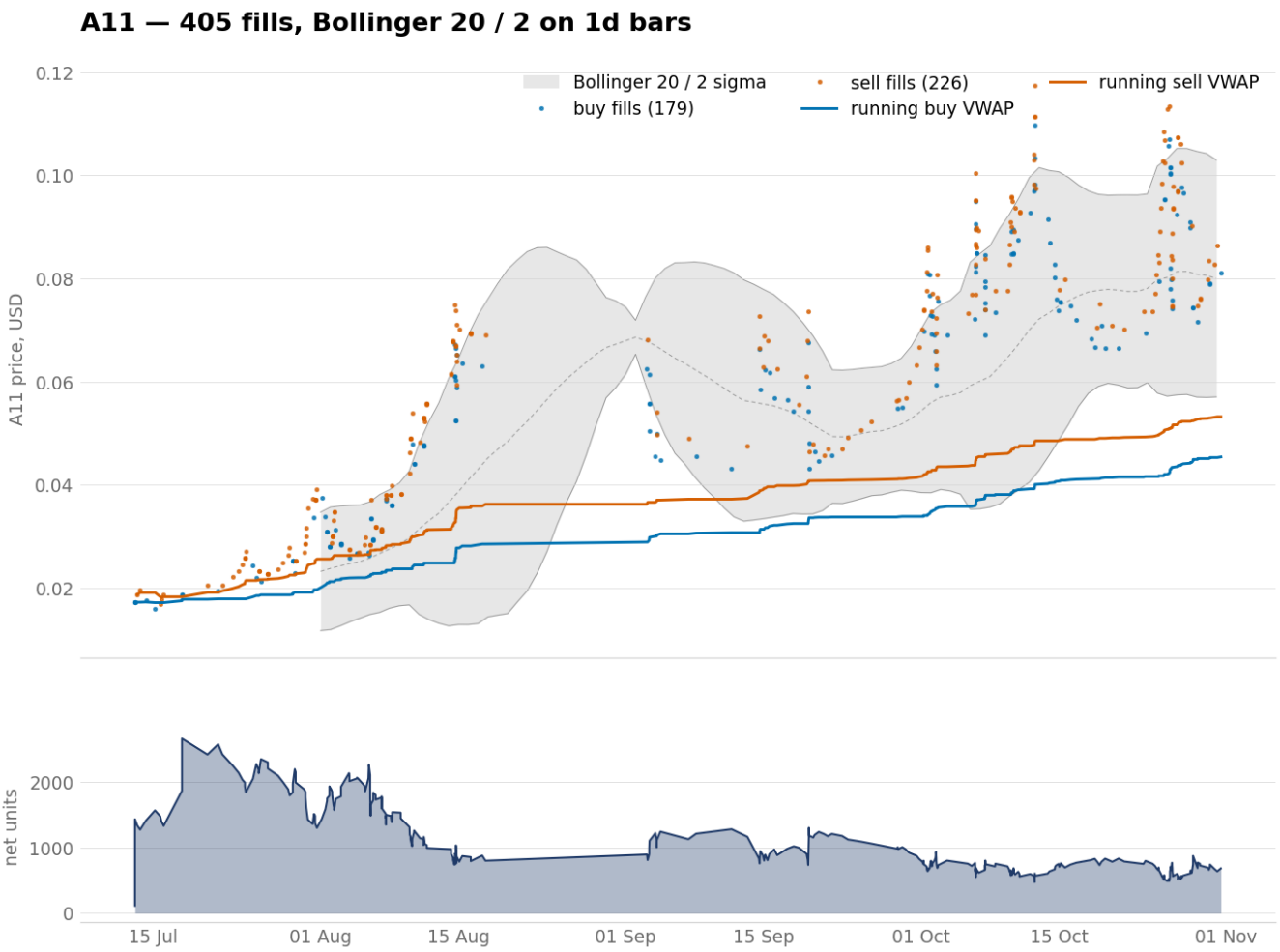


A10 position card, with the mark, the name and the code square painted out.

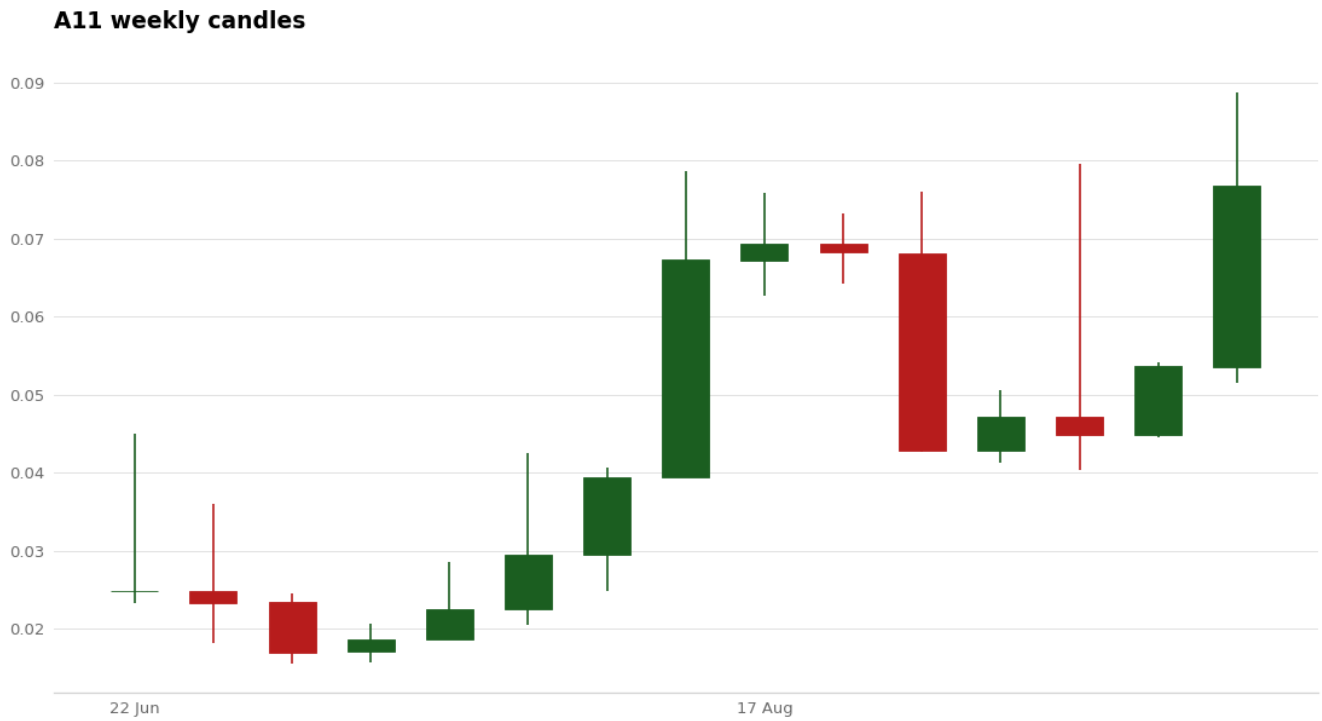
A10 is the only base whose fills fall under a tenth of a cent. The next lowest is A29 at 0.001479, so A10 alone reaches the last branch of the tick formatter, and this is the only chart of the 38 labelled in scientific notation.

```
def fmt_price(v, _pos=None) -> str:
    """Format a y-axis price tick, keeping a micro-cap significand visible."""
    if v == 0:
        return "0"
    if abs(v) >= 1000:
        return f"{v:,.0f}"
    if abs(v) >= 1:
        return f"{v:,.2f}"
    if abs(v) >= 0.001:
        return f"{v:.4f}"
    return f"{v:.2e}"
```

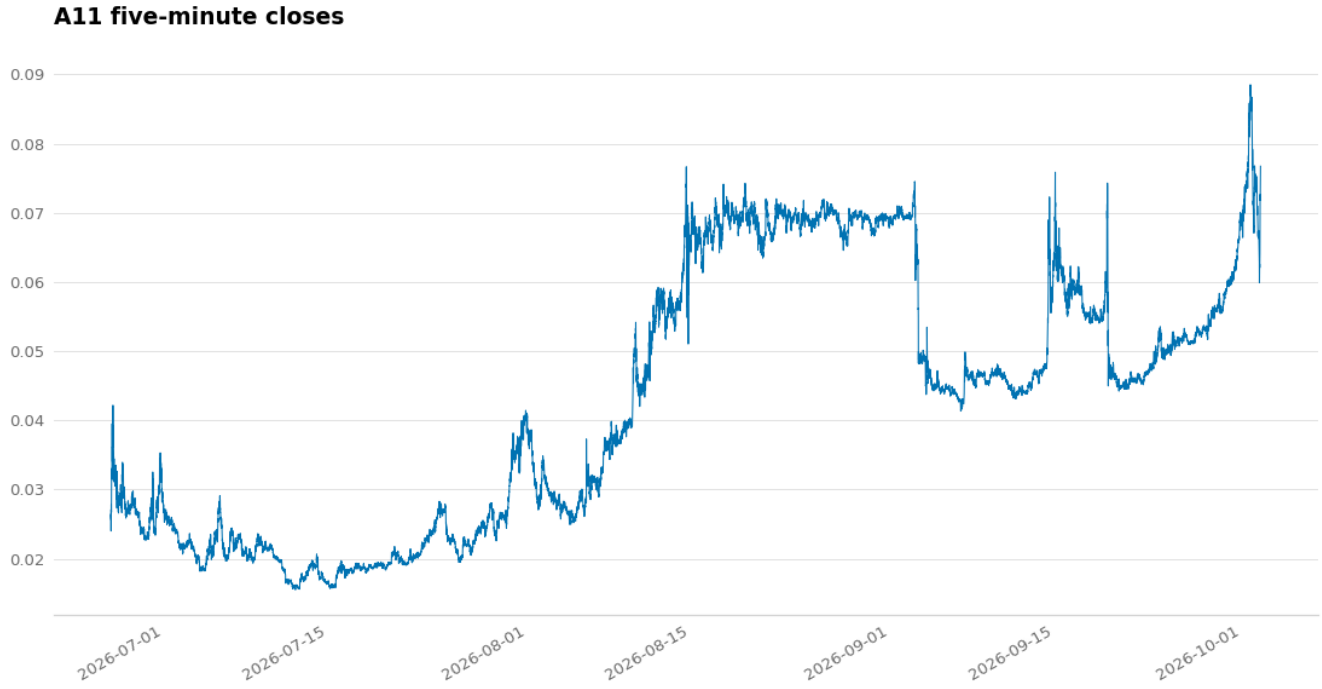
A11



A11, 178 fills, and the largest rise of the 38.



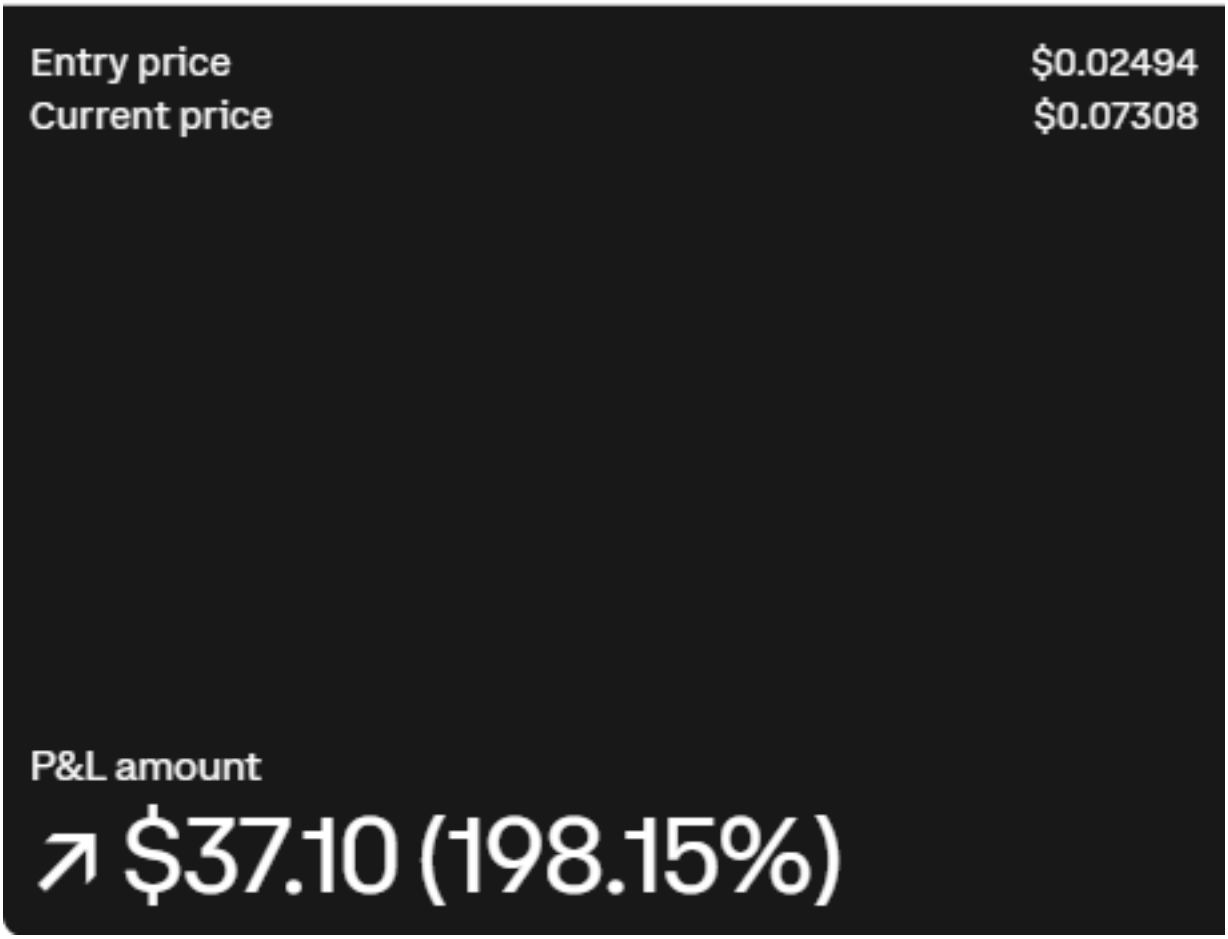
A11 weekly chart, 15 weekly candles built from 26,750 five-minute rows.



A11 price trace, 26,750 five-minute closes across the recorded window.

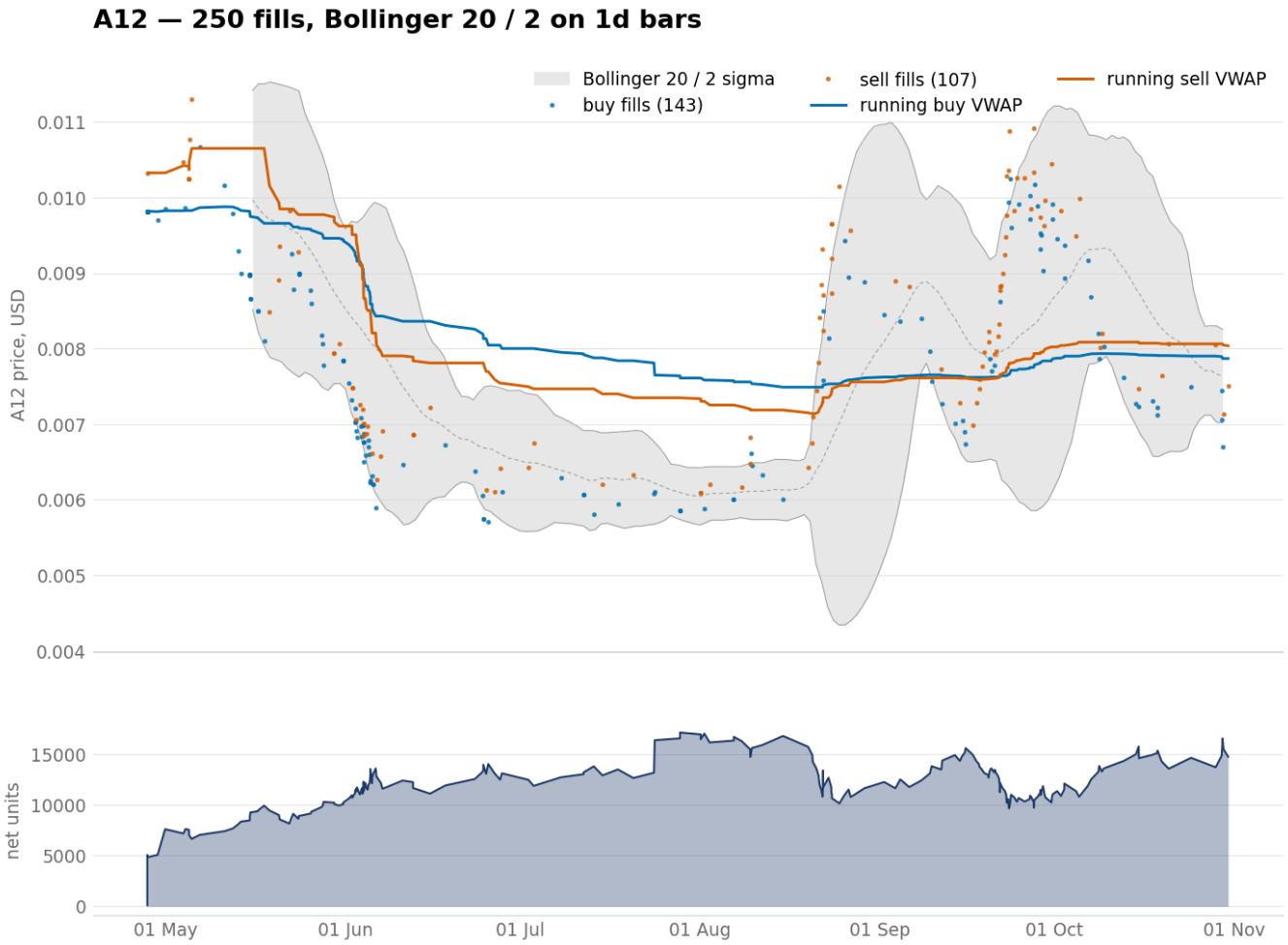
coinbase ADVANCED

A11



A11 position card, with the mark, the name and the code square painted out.

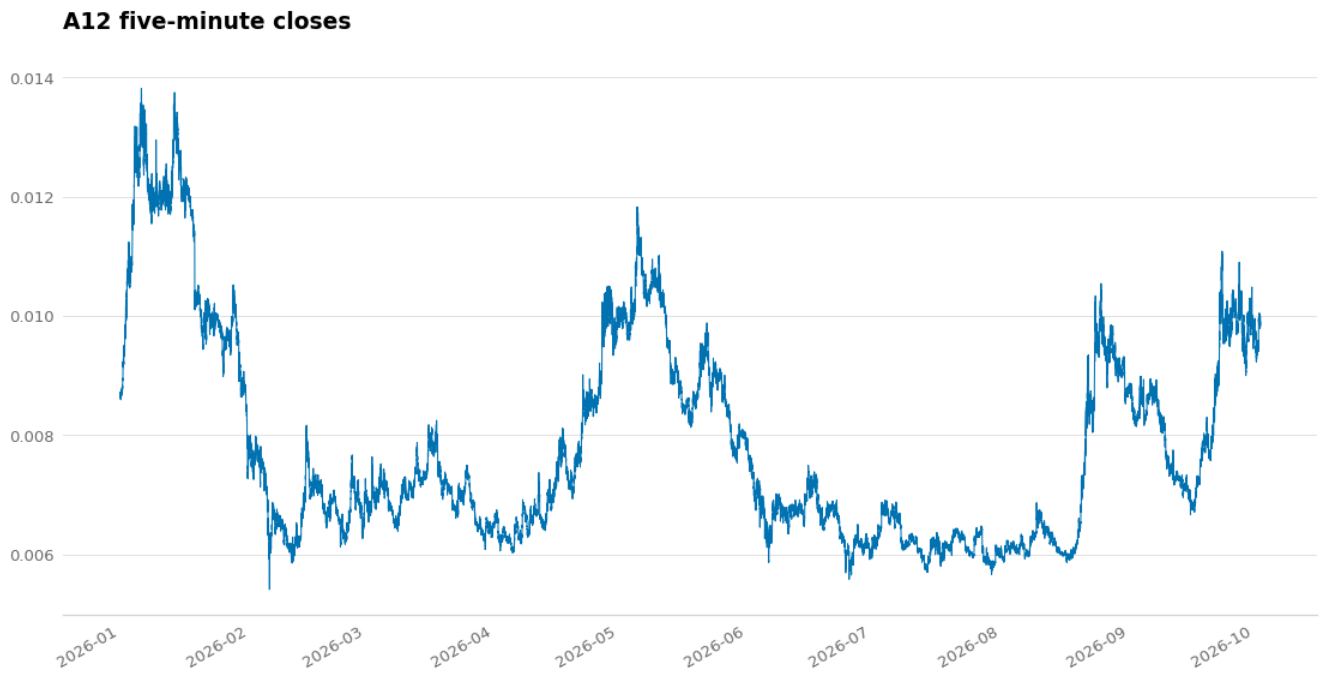
A12



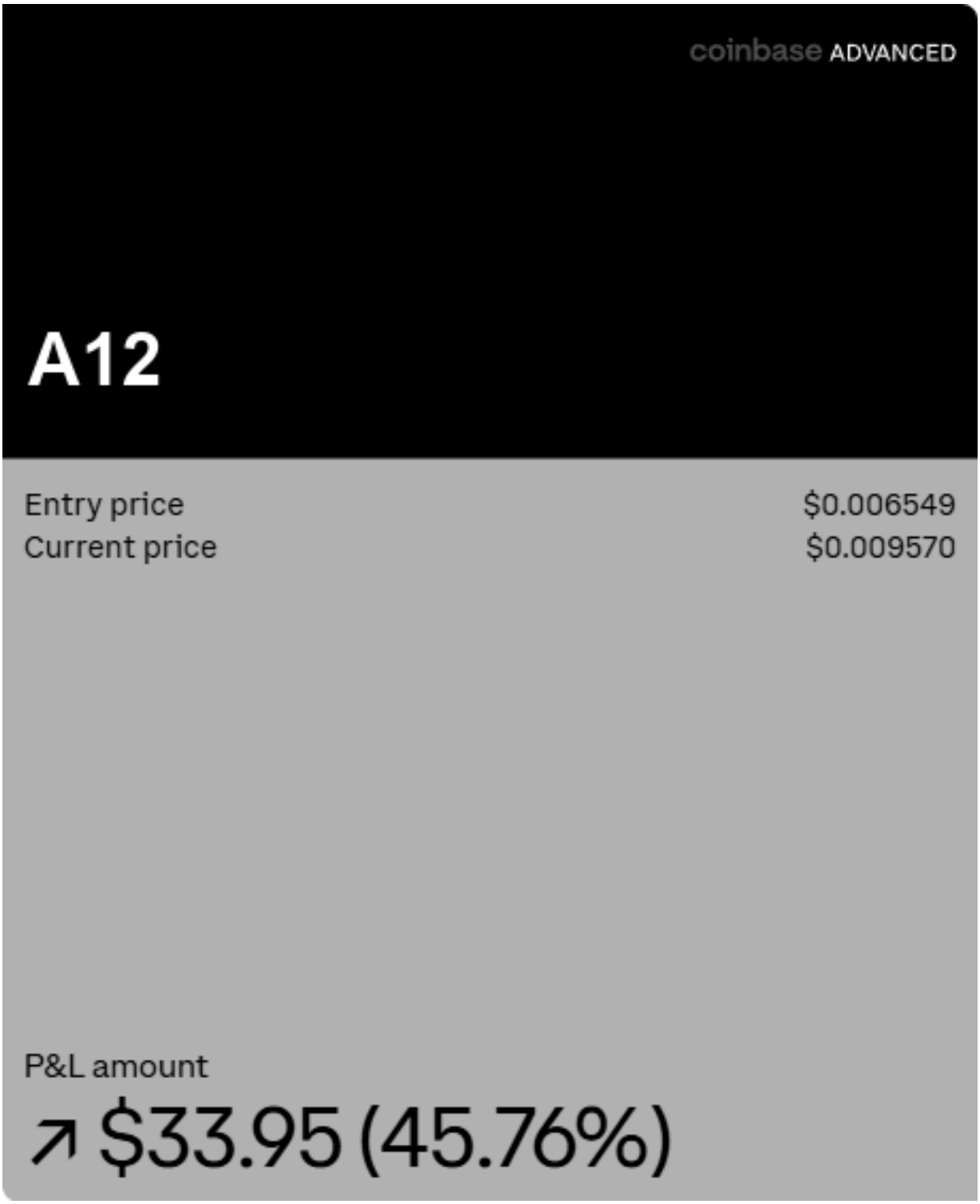
A12, 159 fills, with the sell line dropping below the buy line at the start of June.



A12 weekly chart, 40 weekly candles built from 78,332 five-minute rows.

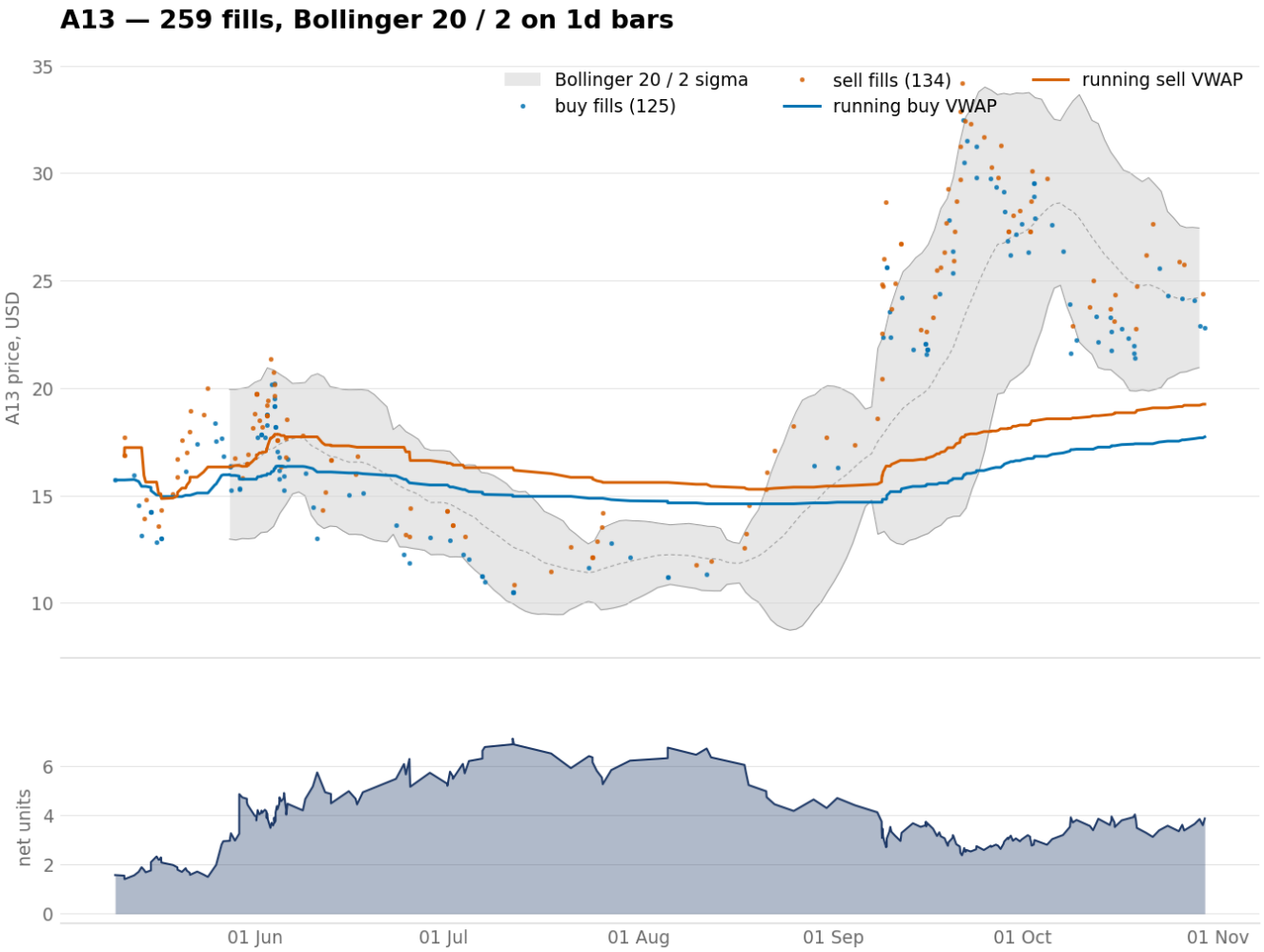


A12 price trace, 78,332 five-minute closes across the recorded window.



A12 position card, with the mark, the name and the code square painted out.

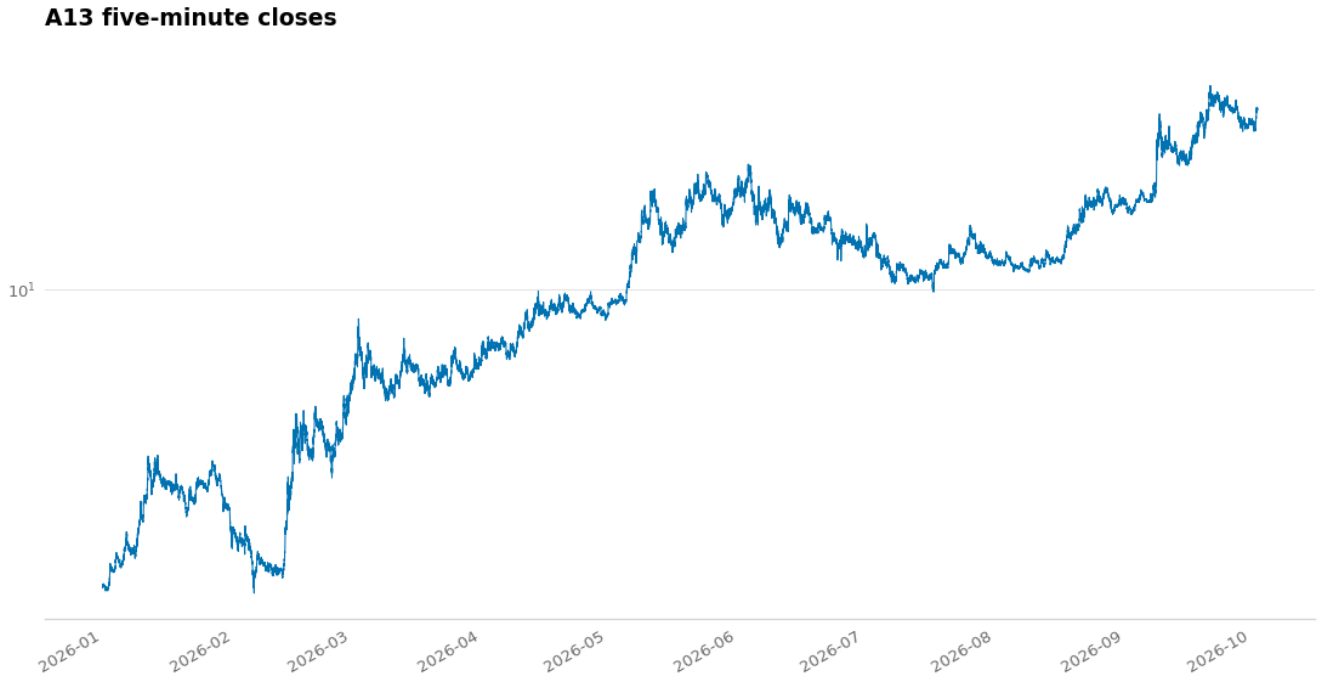
A13



A13, 147 fills, with the sell line above the buy line throughout.



A13 weekly chart, 40 weekly candles built from 76,386 five-minute rows.



A13 price trace, 76,386 five-minute closes across the recorded window.

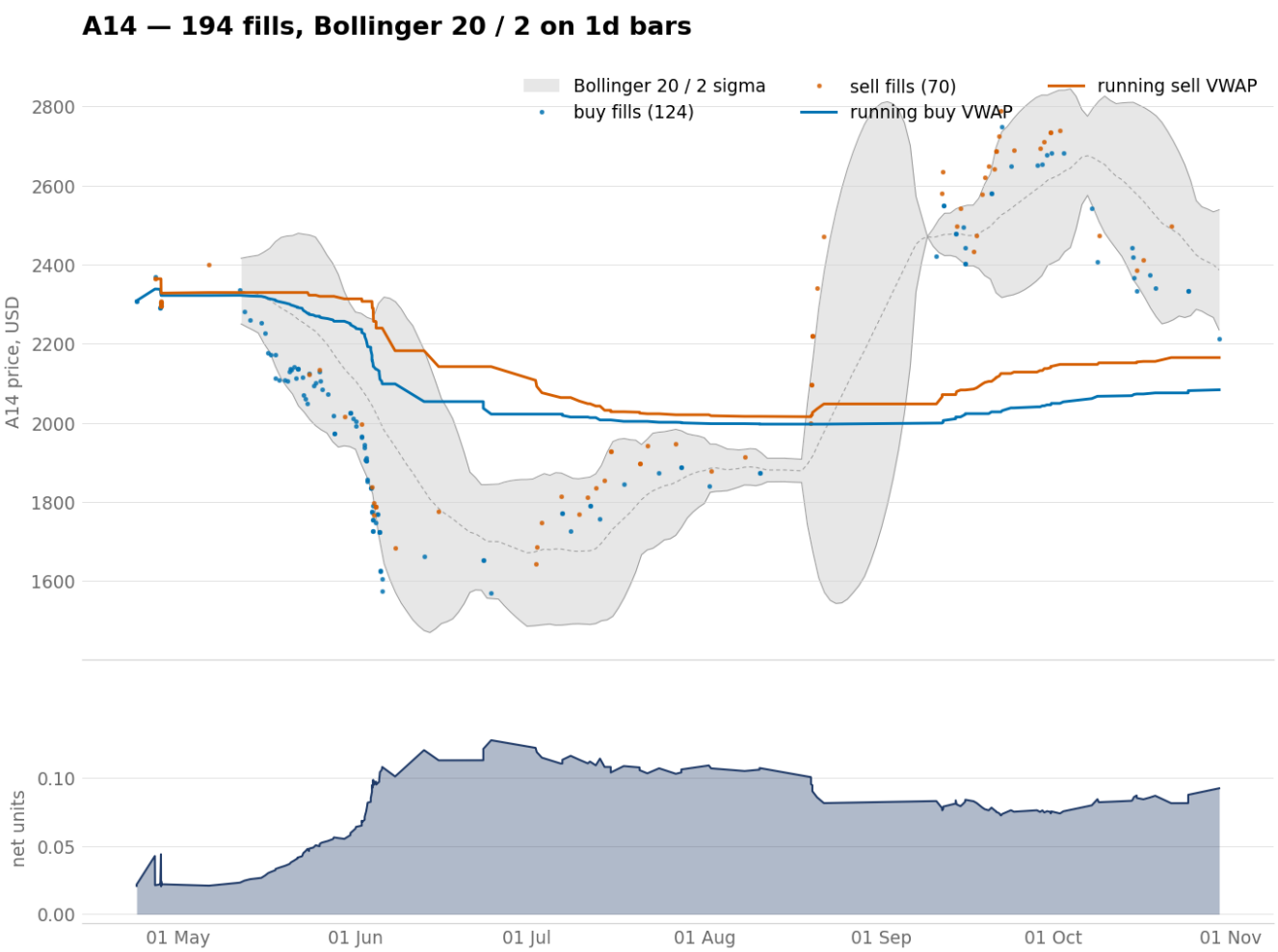
coinbase ADVANCED

A13



A13 position card, with the mark, the name and the code square painted out.

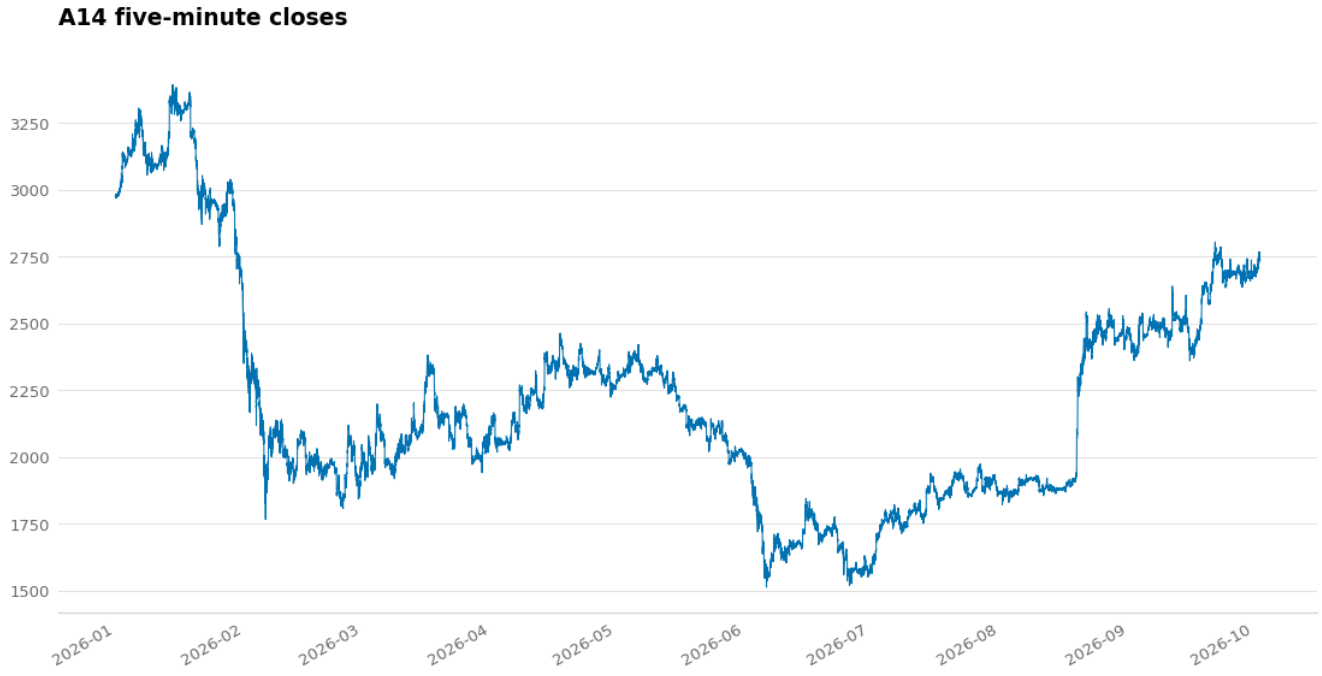
A14



A14, 141 fills, ending 21 August.



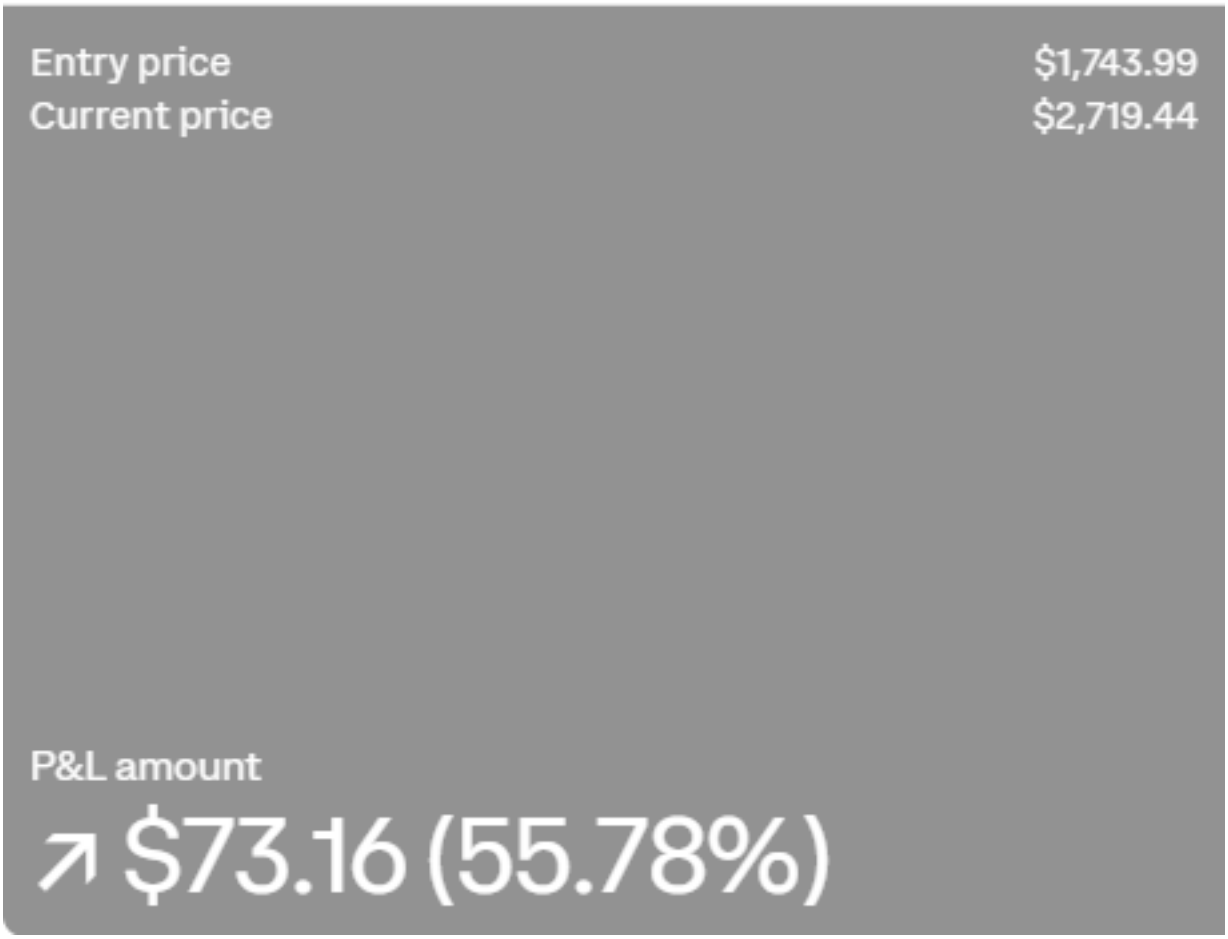
A14 weekly chart, 40 weekly candles built from 79,004 five-minute rows.



A14 price trace, 79,004 five-minute closes across the recorded window.

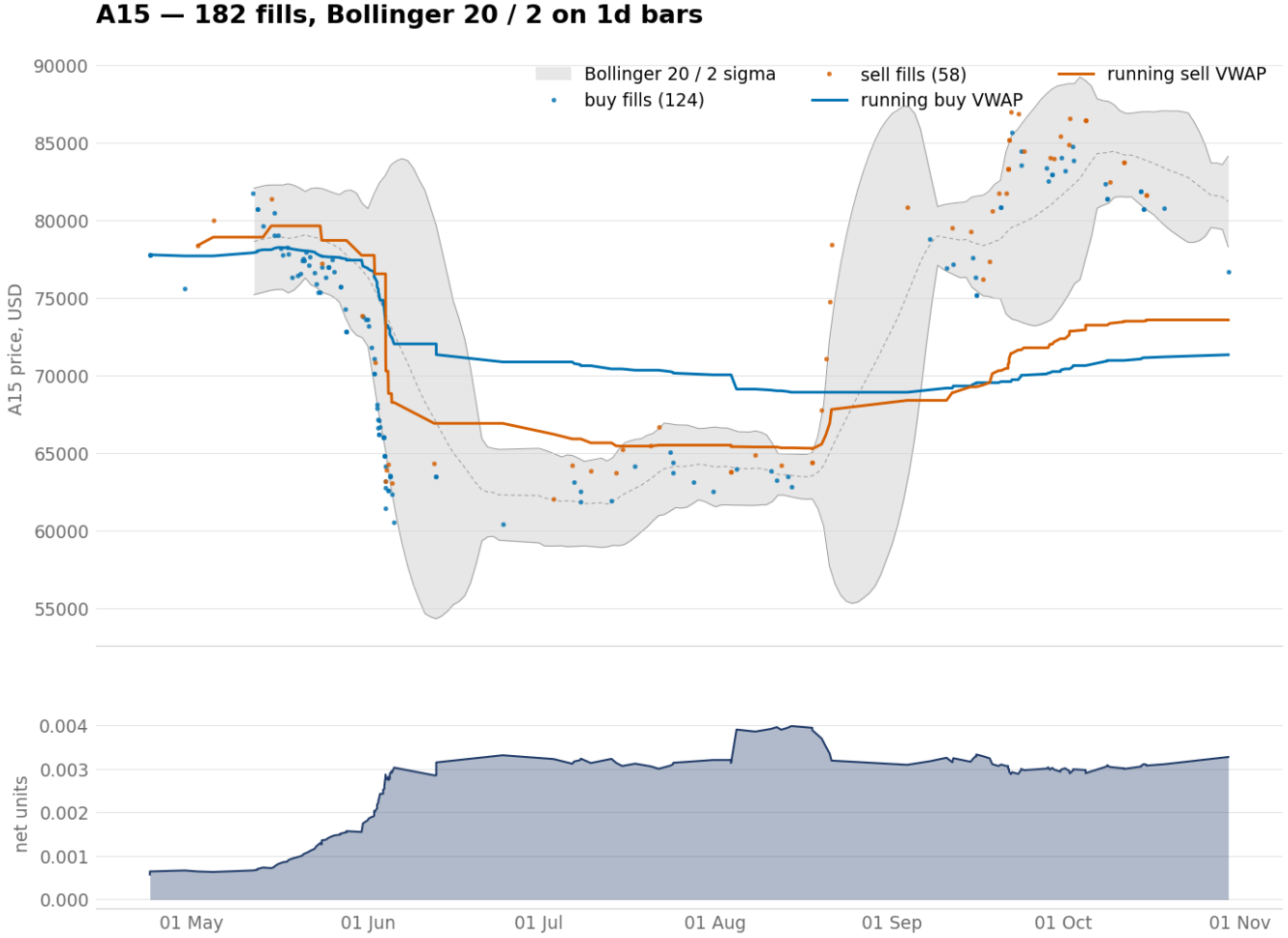
coinbase ADVANCED

A14



A14 position card, with the mark, the name and the code square painted out.

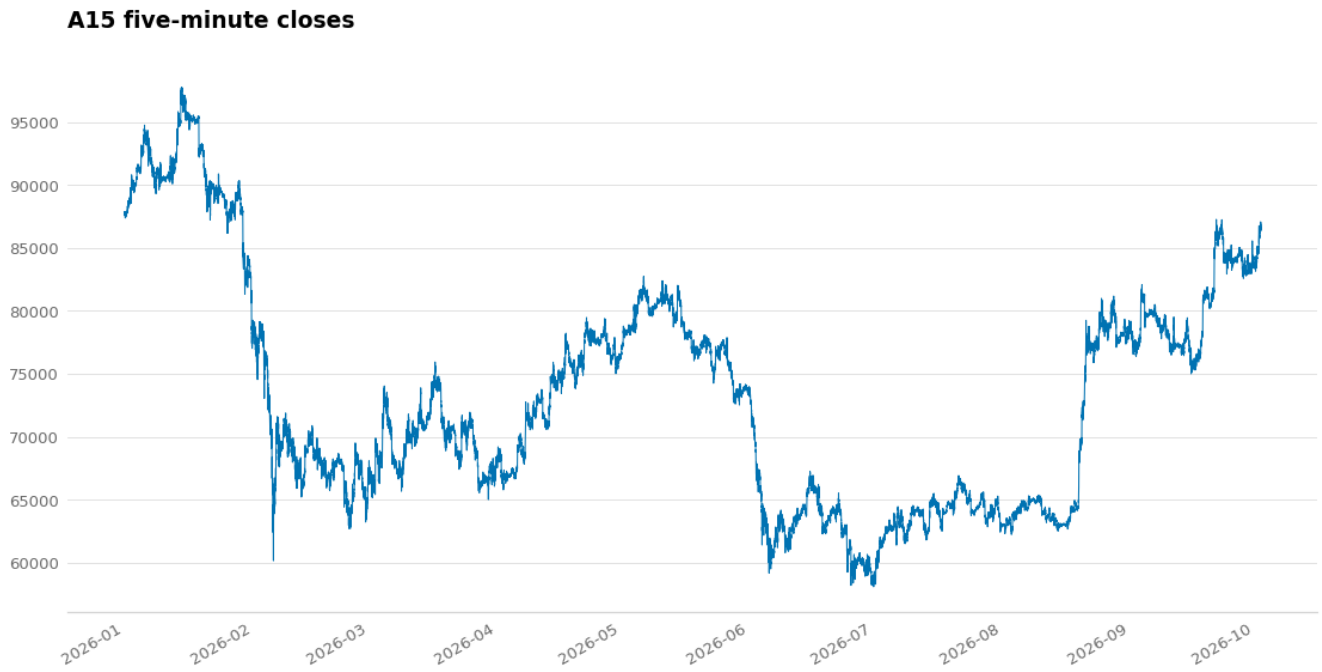
A15



A15, 124 fills, with the two lines splitting at the start of June.



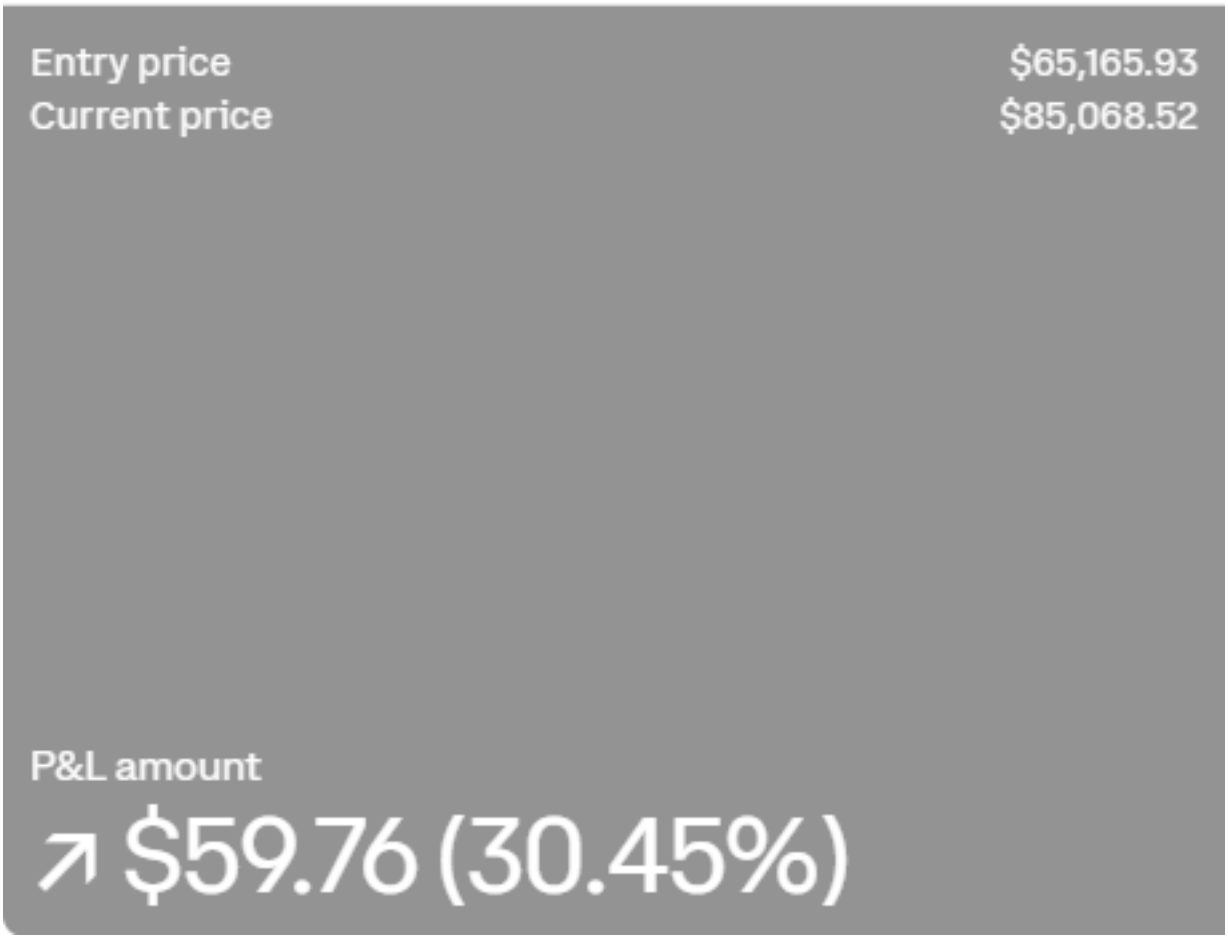
A15 weekly chart, 40 weekly candles built from 79,005 five-minute rows.



A15 price trace, 79,005 five-minute closes across the recorded window.

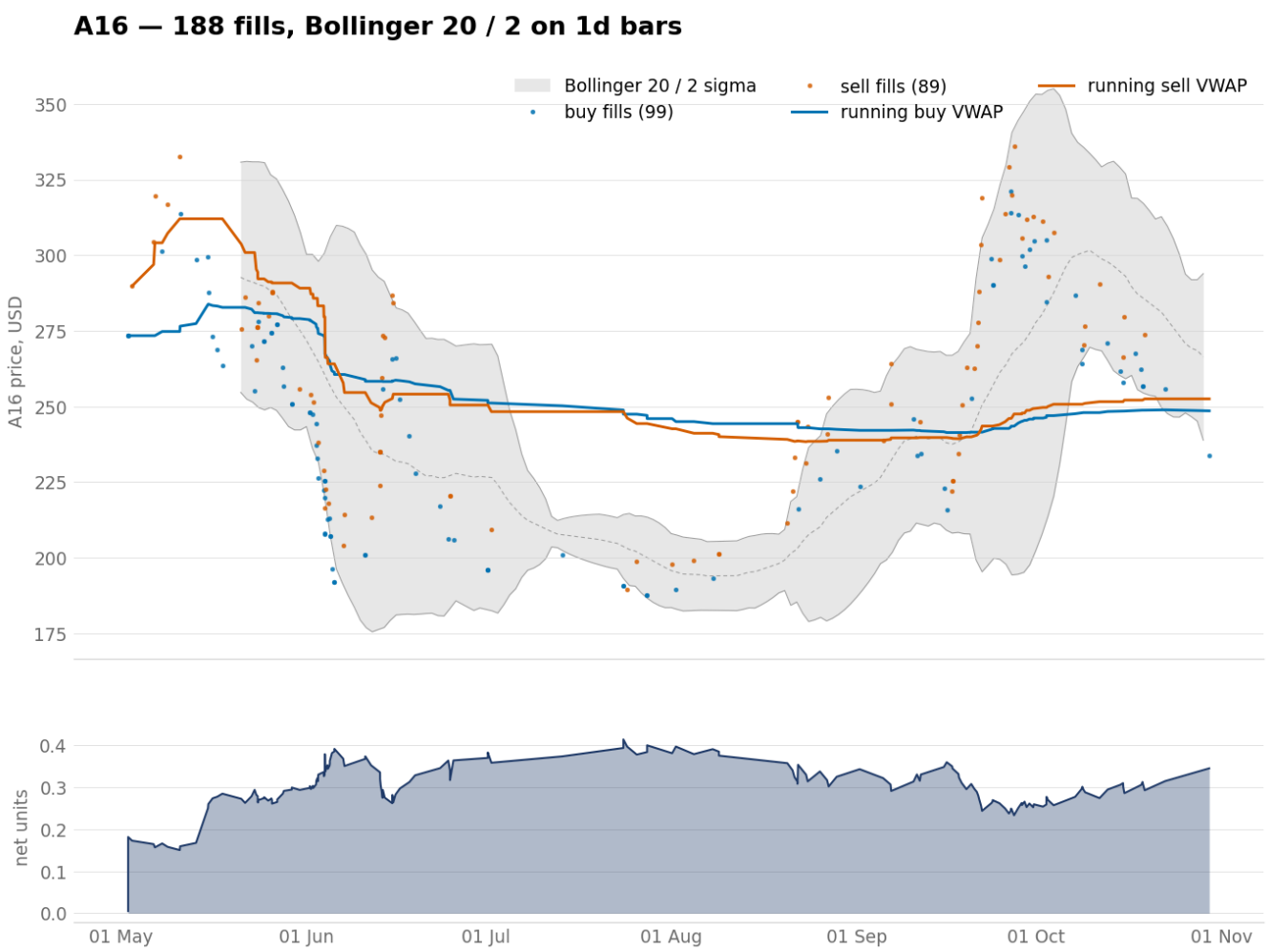
coinbase ADVANCED

A15



A15 position card, with the mark, the name and the code square painted out.

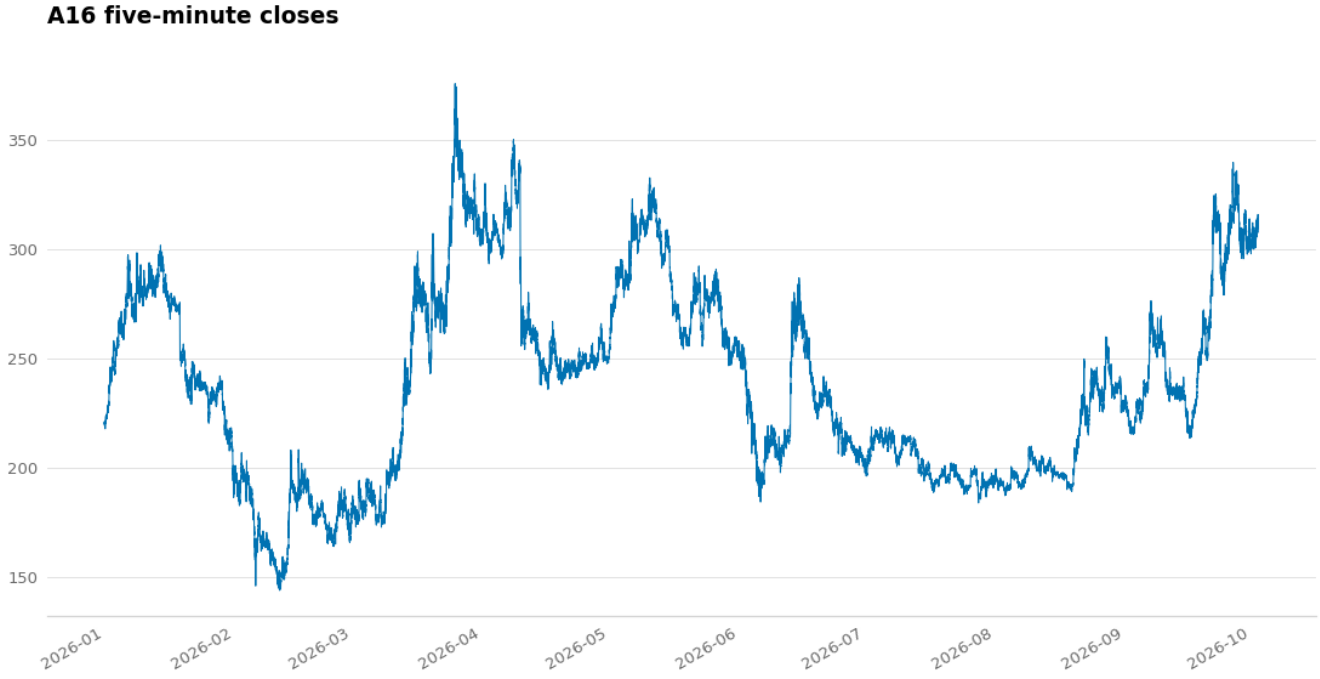
A16



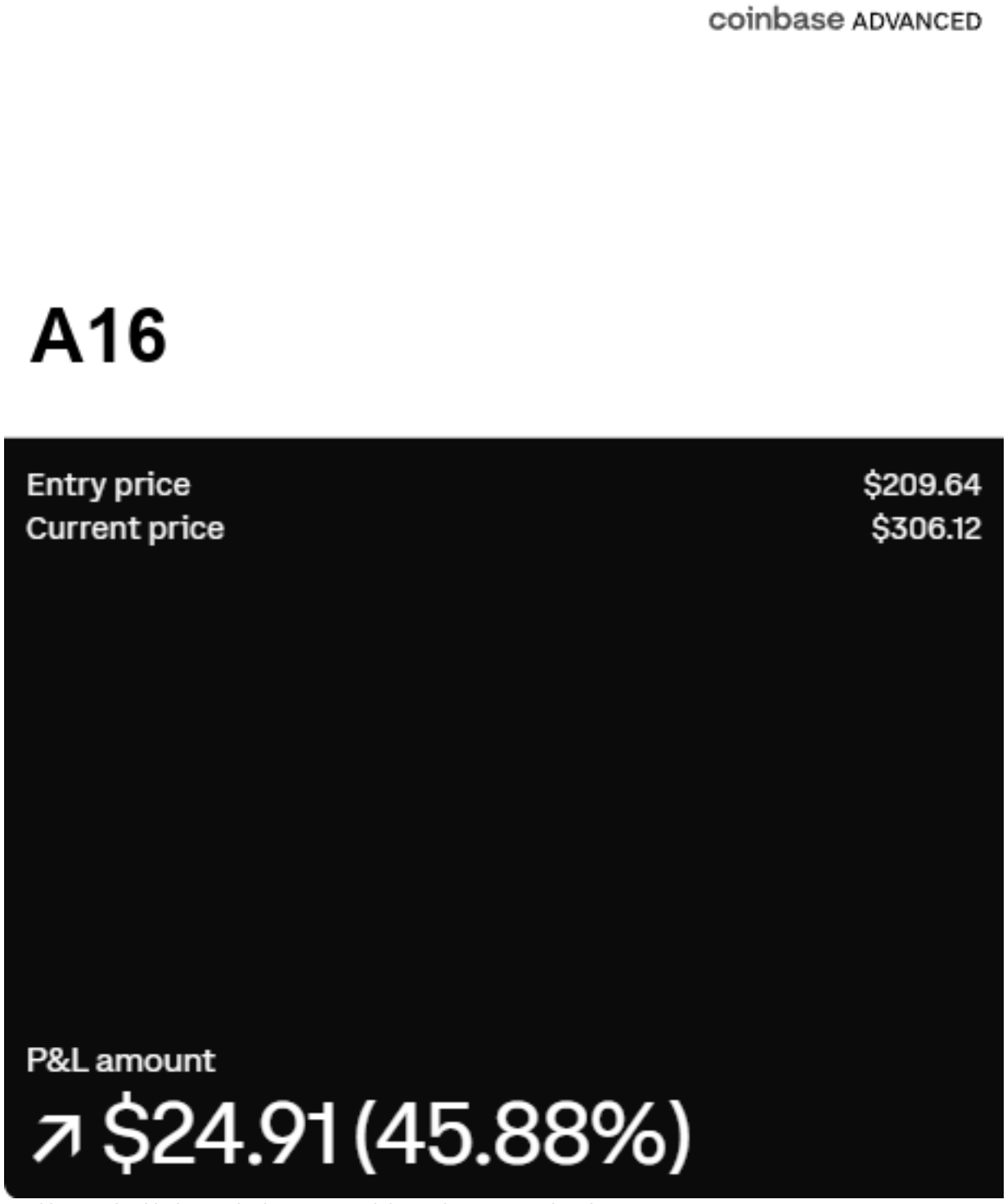
A16, 124 fills, with the sell line above the buy line until mid-June.



A16 weekly chart, 40 weekly candles built from 78,992 five-minute rows.

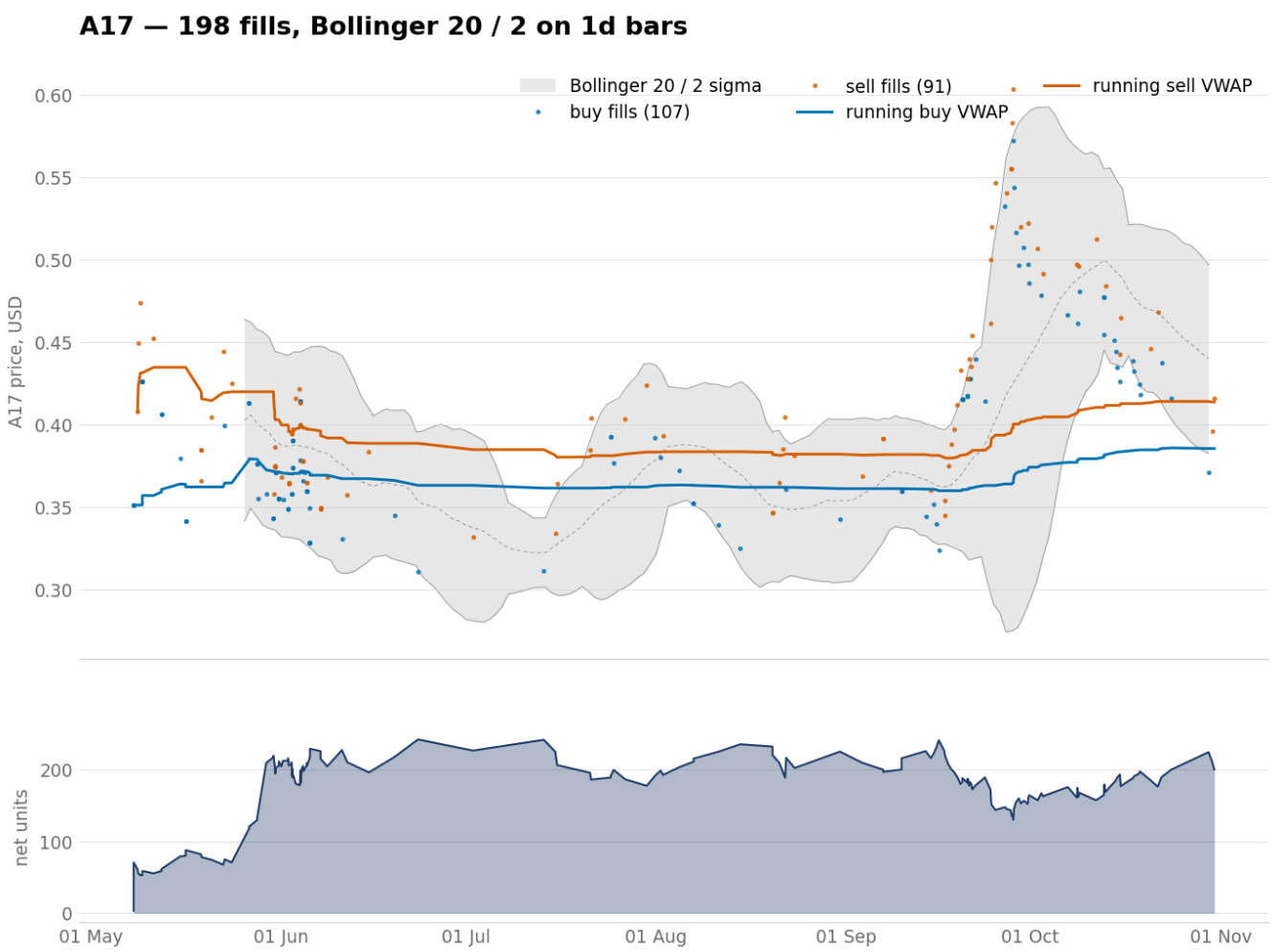


A16 price trace, 78,992 five-minute closes across the recorded window.



A16 position card, with the mark, the name and the code square painted out.

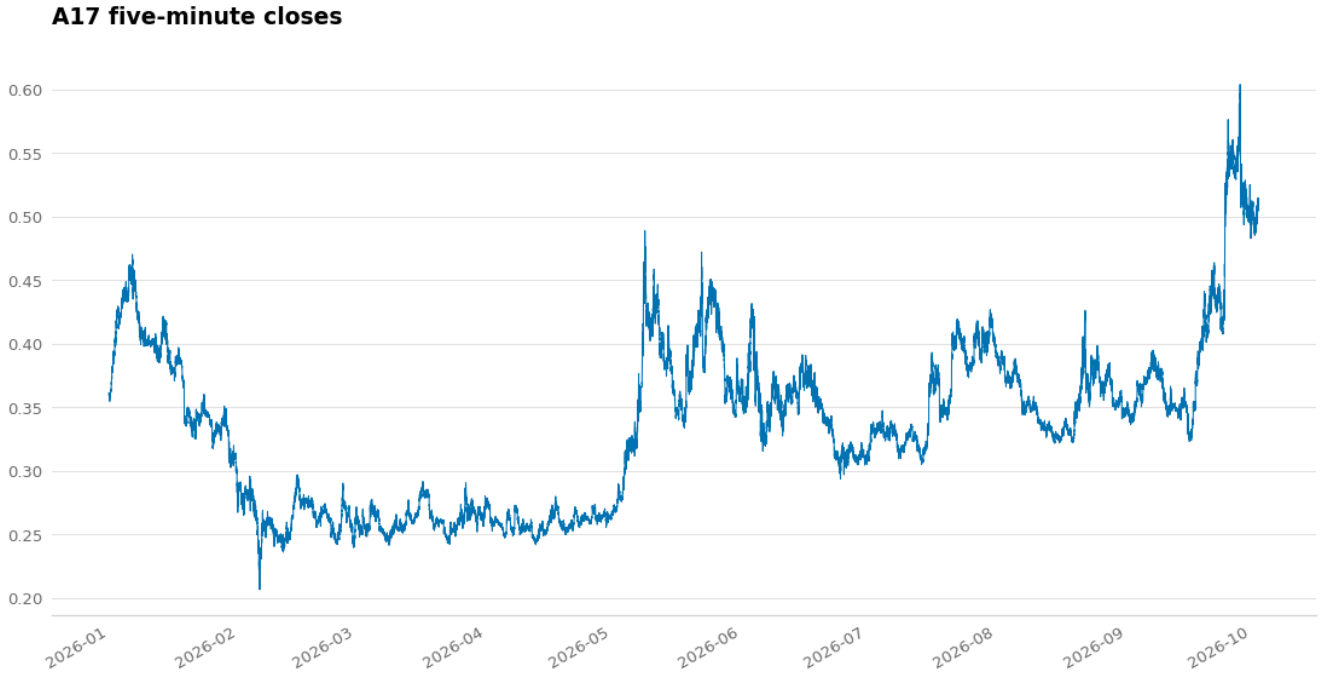
A17



A17, 119 fills, with both lines nearly flat after June.



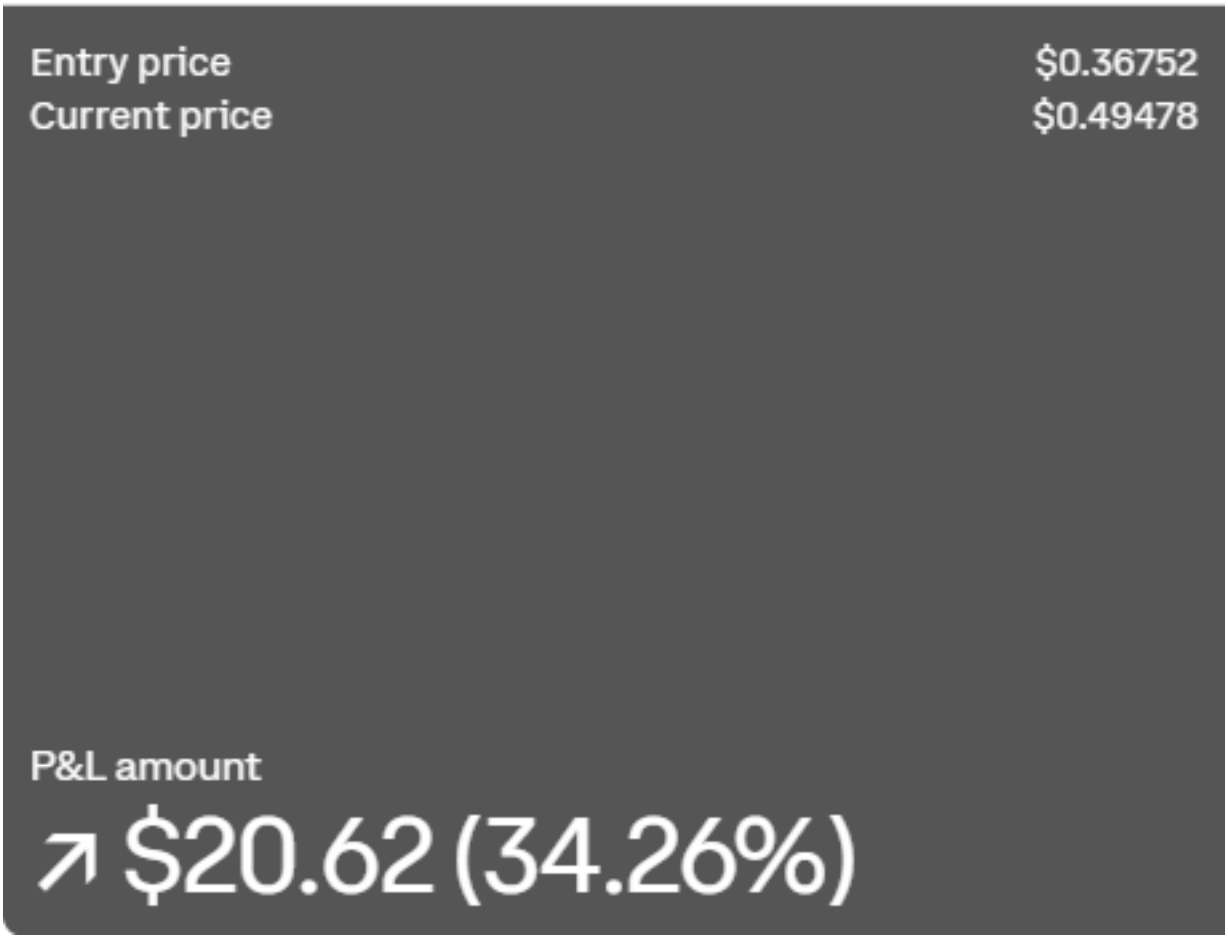
A17 weekly chart, 40 weekly candles built from 78,992 five-minute rows.



A17 price trace, 78,992 five-minute closes across the recorded window.

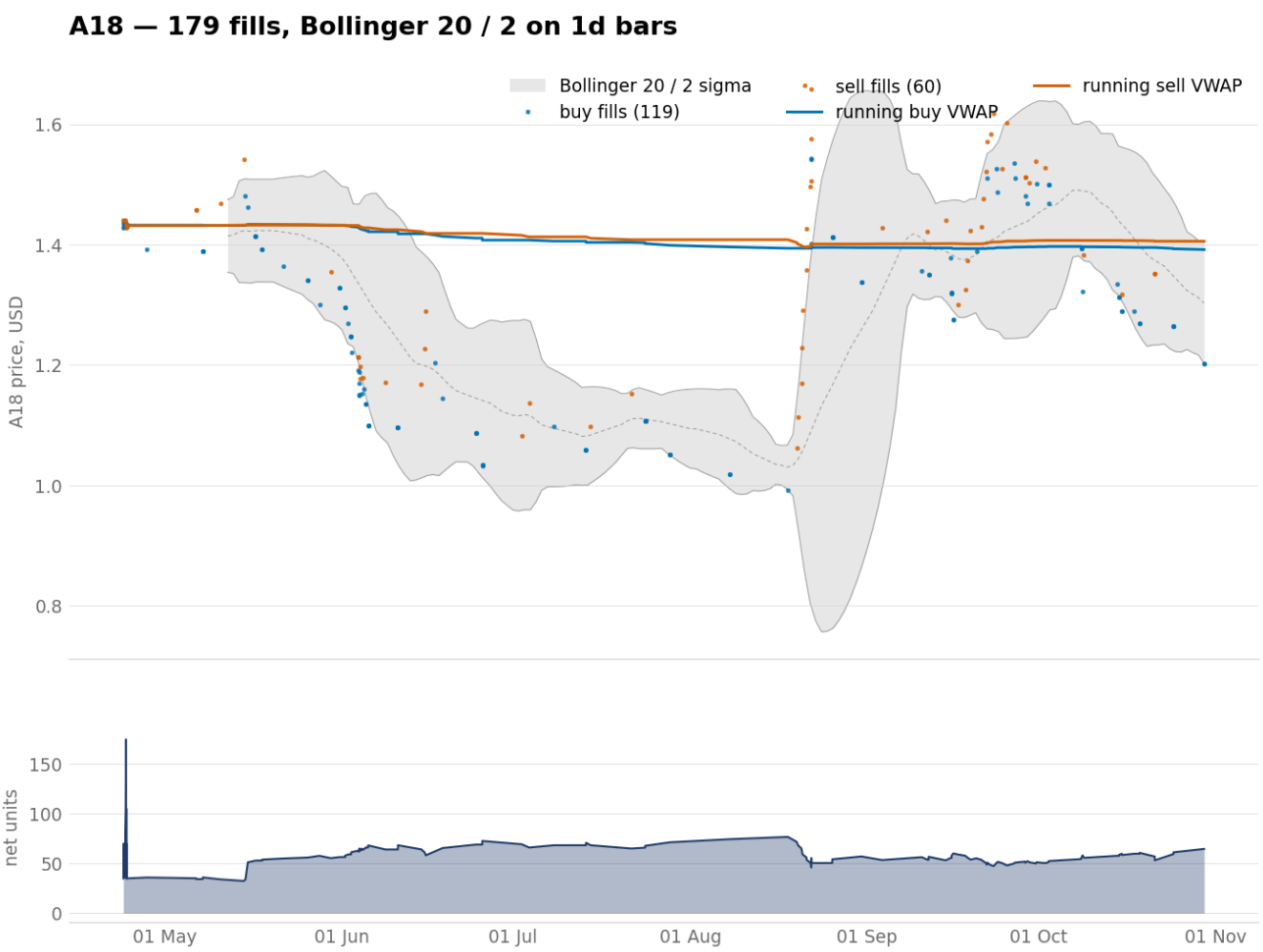
coinbase ADVANCED

A17



A17 position card, with the mark, the name and the code square painted out.

A18



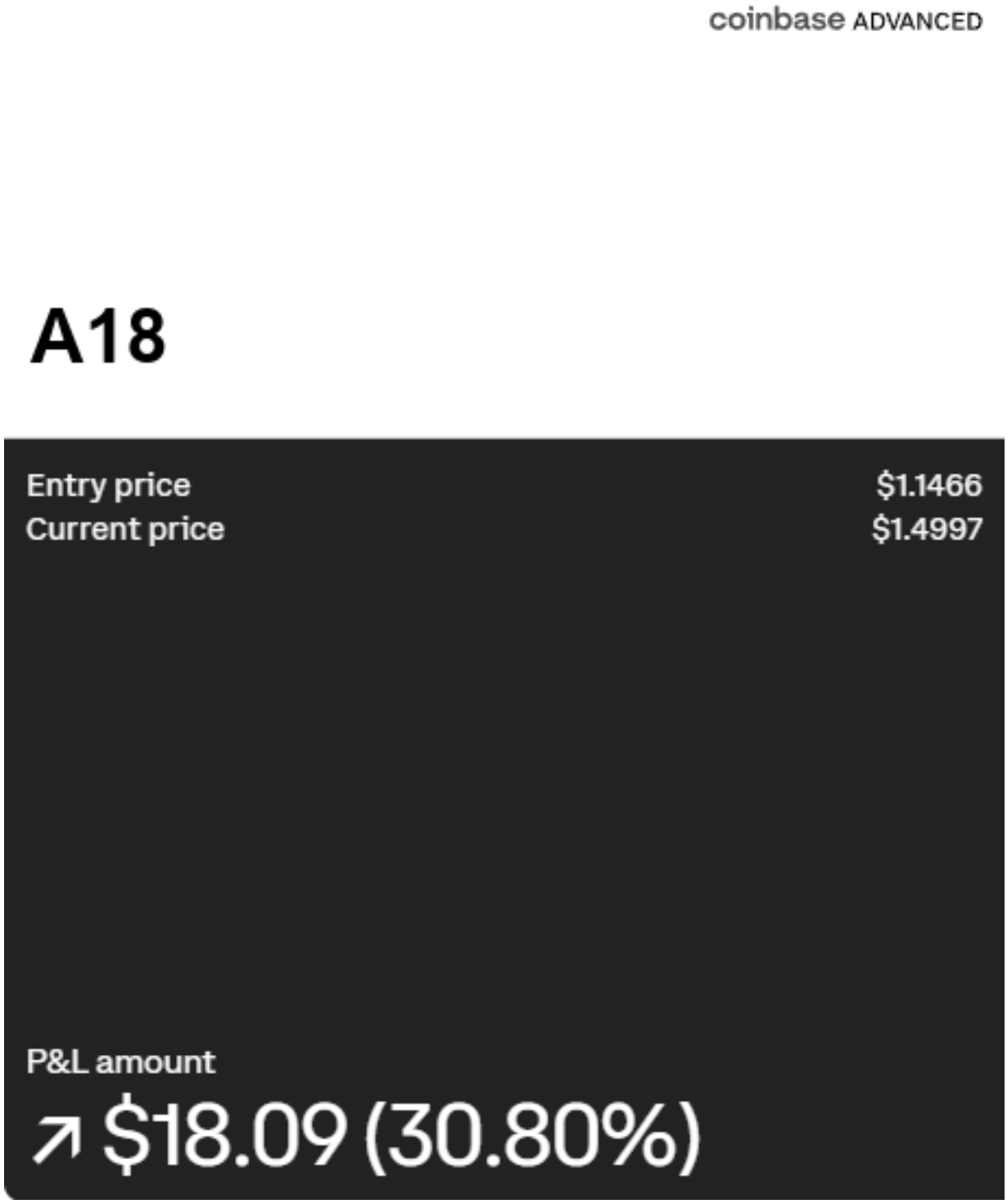
A18, 119 fills, with the two lines almost touching across the window.



A18 weekly chart, 40 weekly candles built from 79,004 five-minute rows.

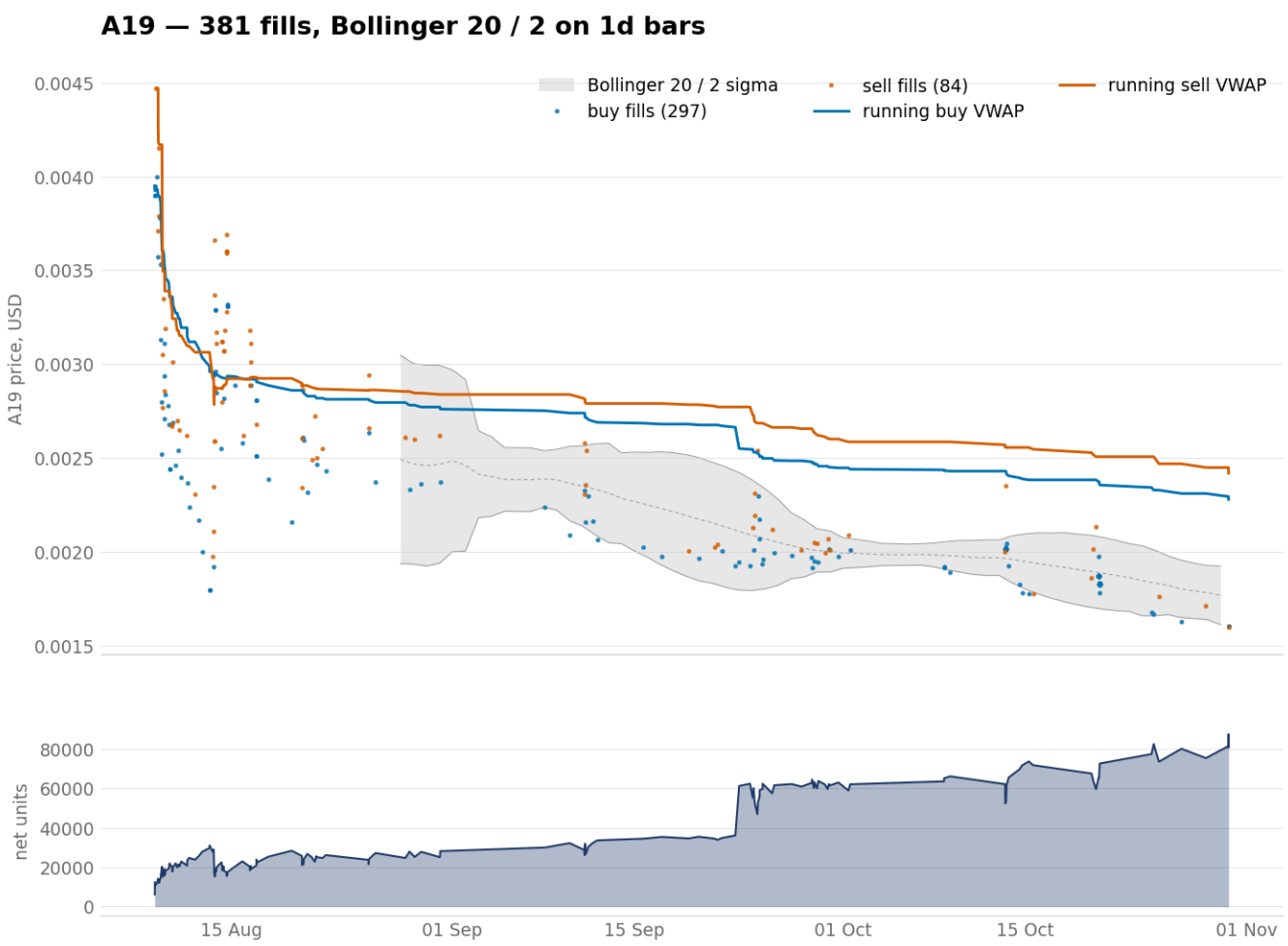


A18 price trace, 79,004 five-minute closes across the recorded window.

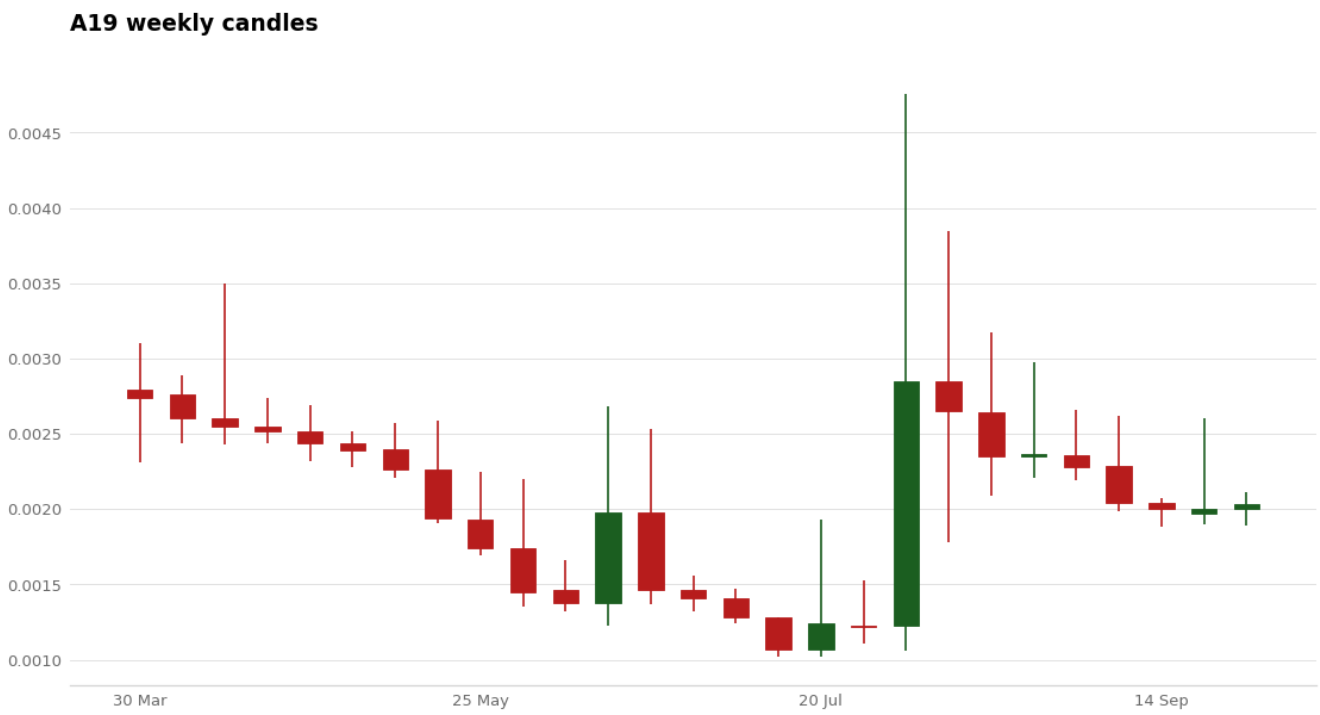


A18 position card, with the mark, the name and the code square painted out.

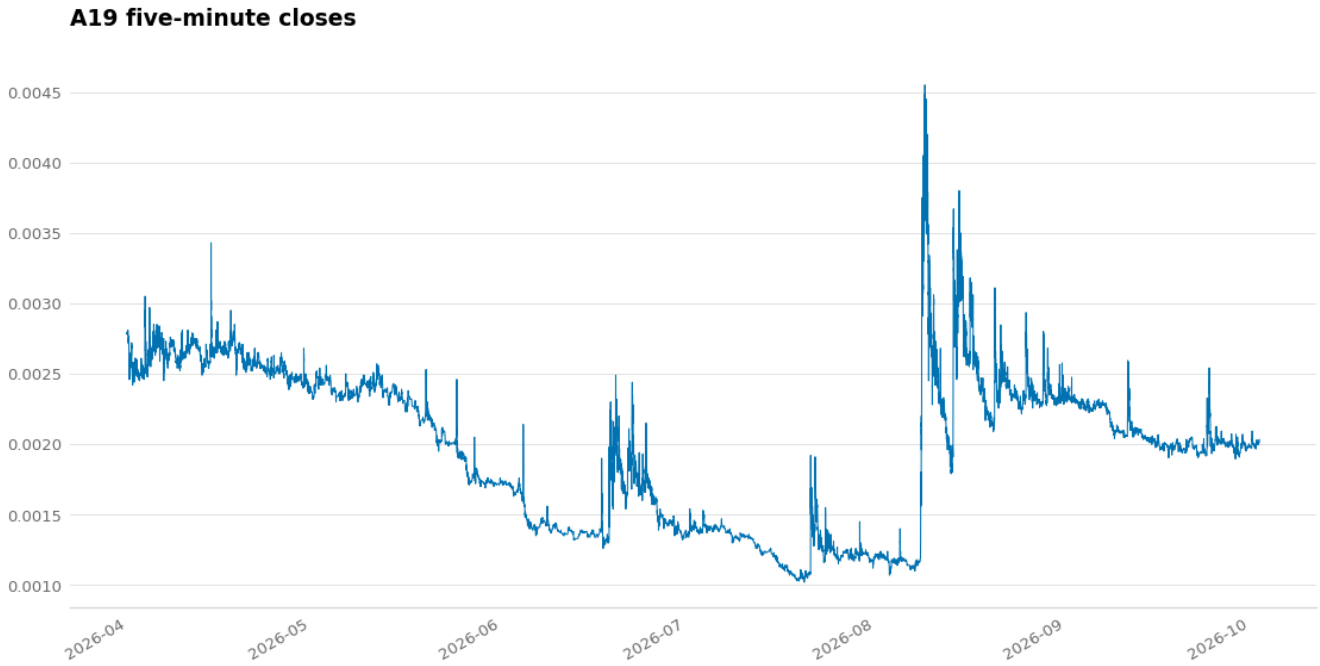
A19



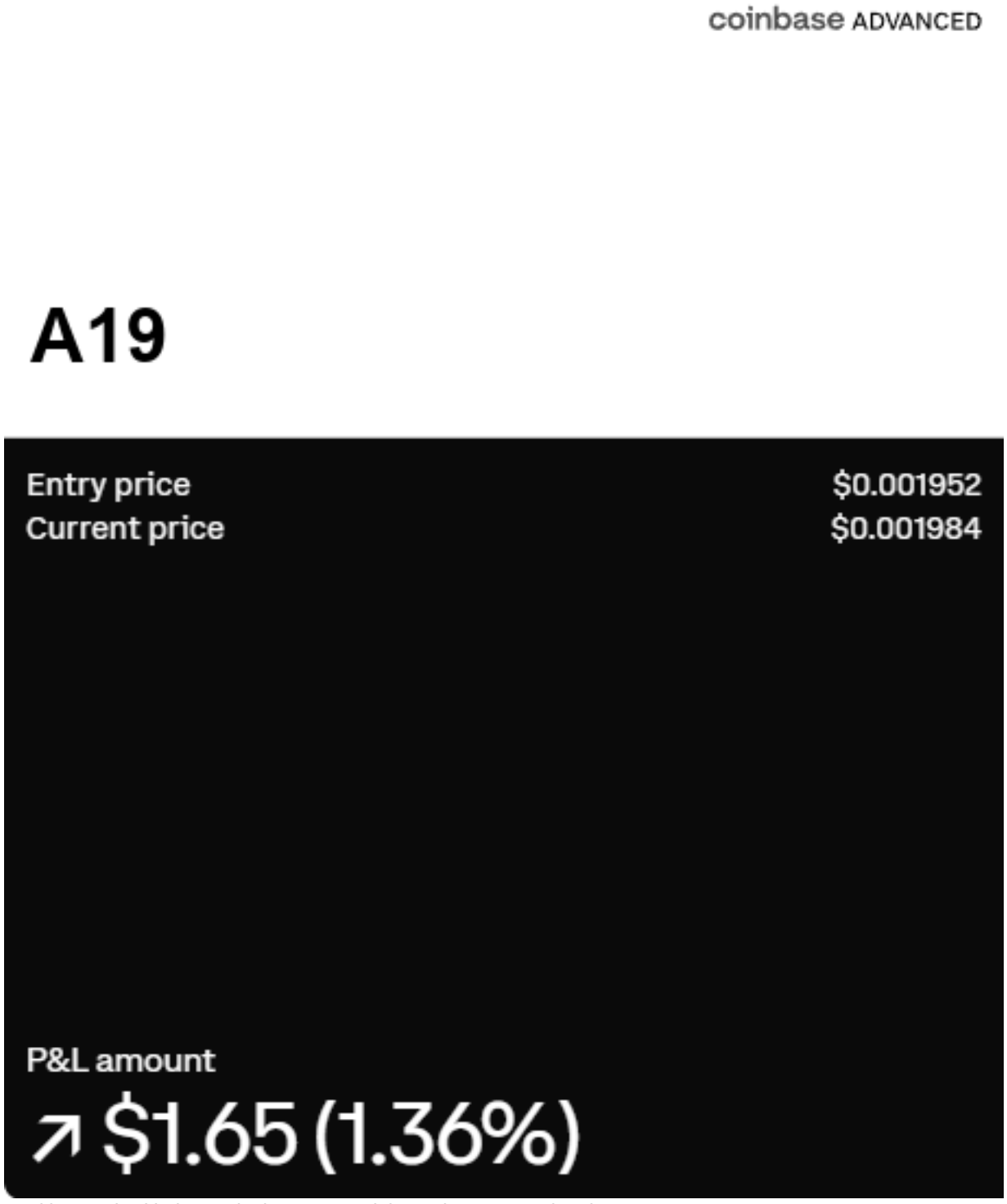
A19, 113 fills across 22 days in August.



A19 weekly chart, 27 weekly candles built from 15,736 five-minute rows.

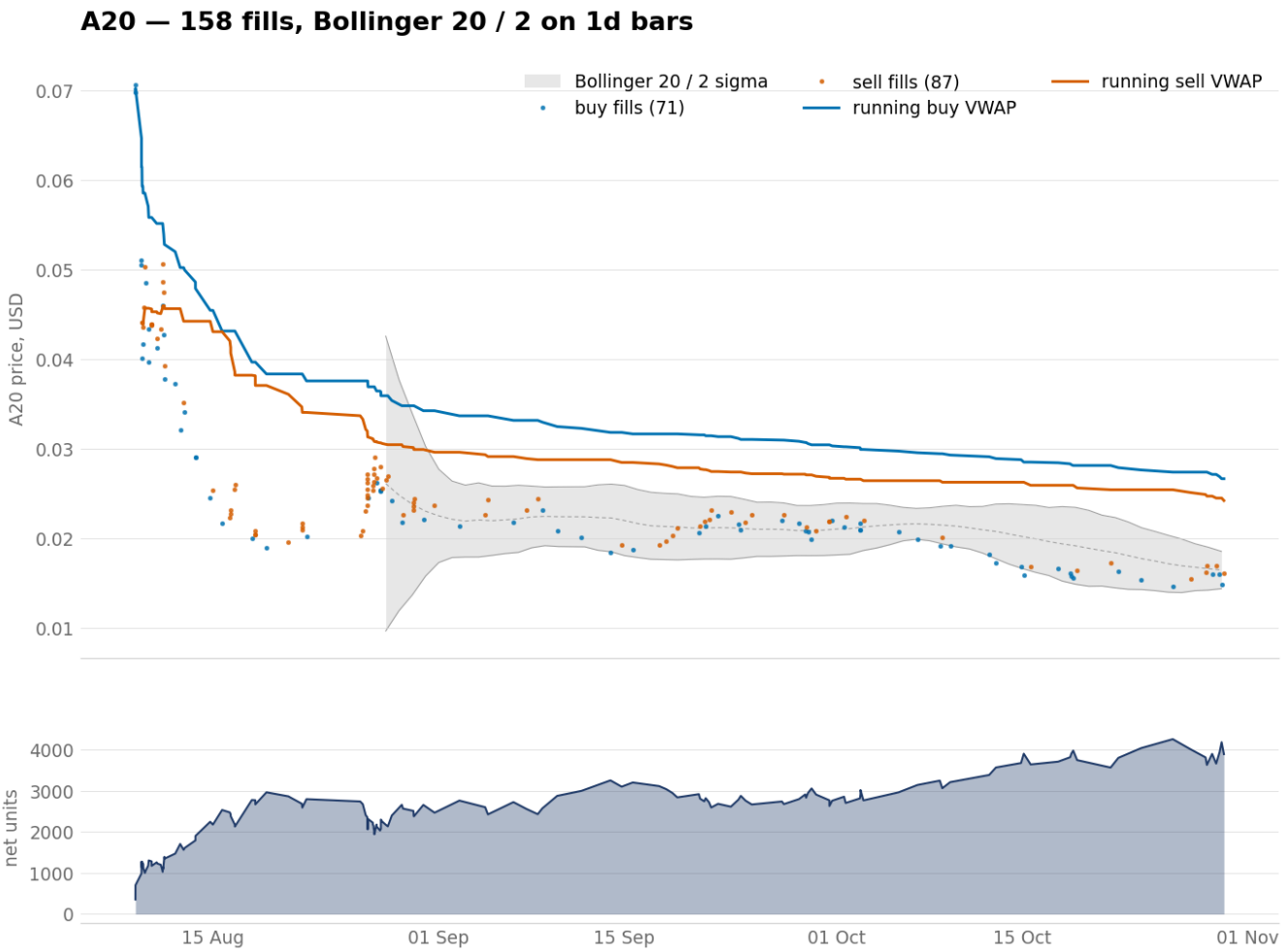


A19 price trace, 15,736 five-minute closes across the recorded window.

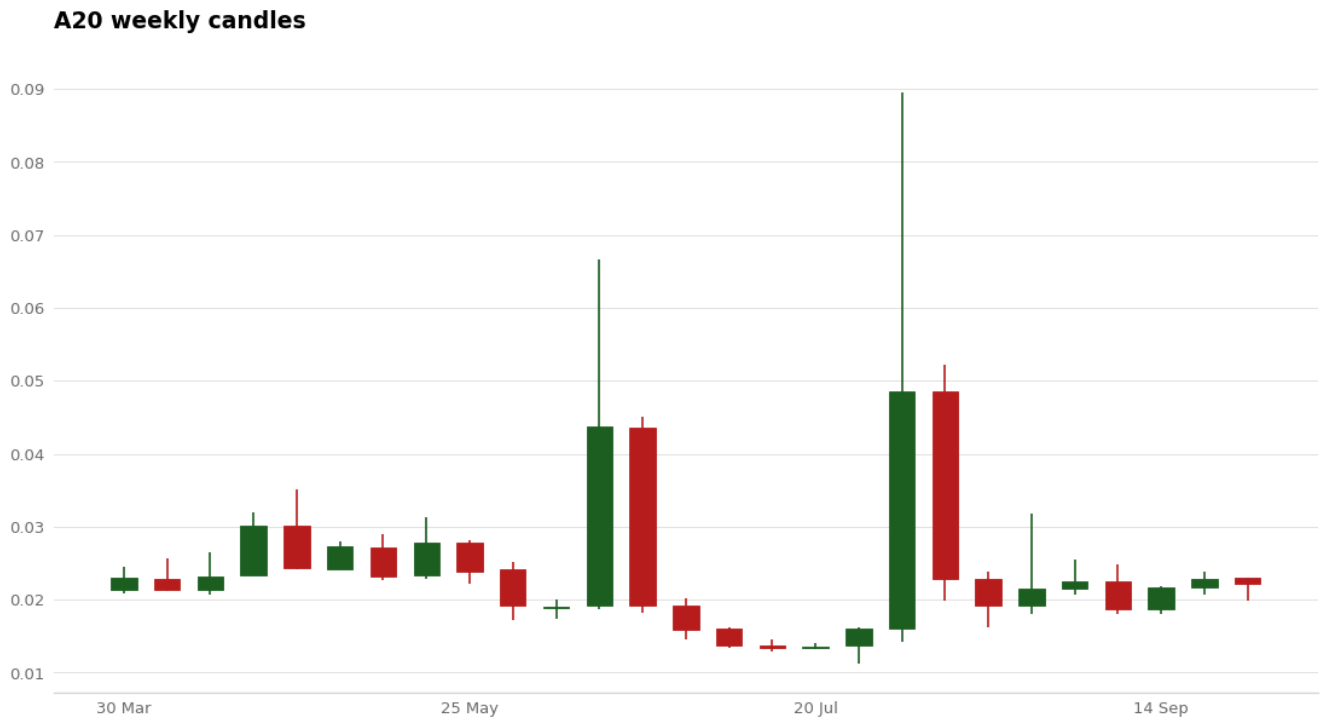


A19 position card, with the mark, the name and the code square painted out.

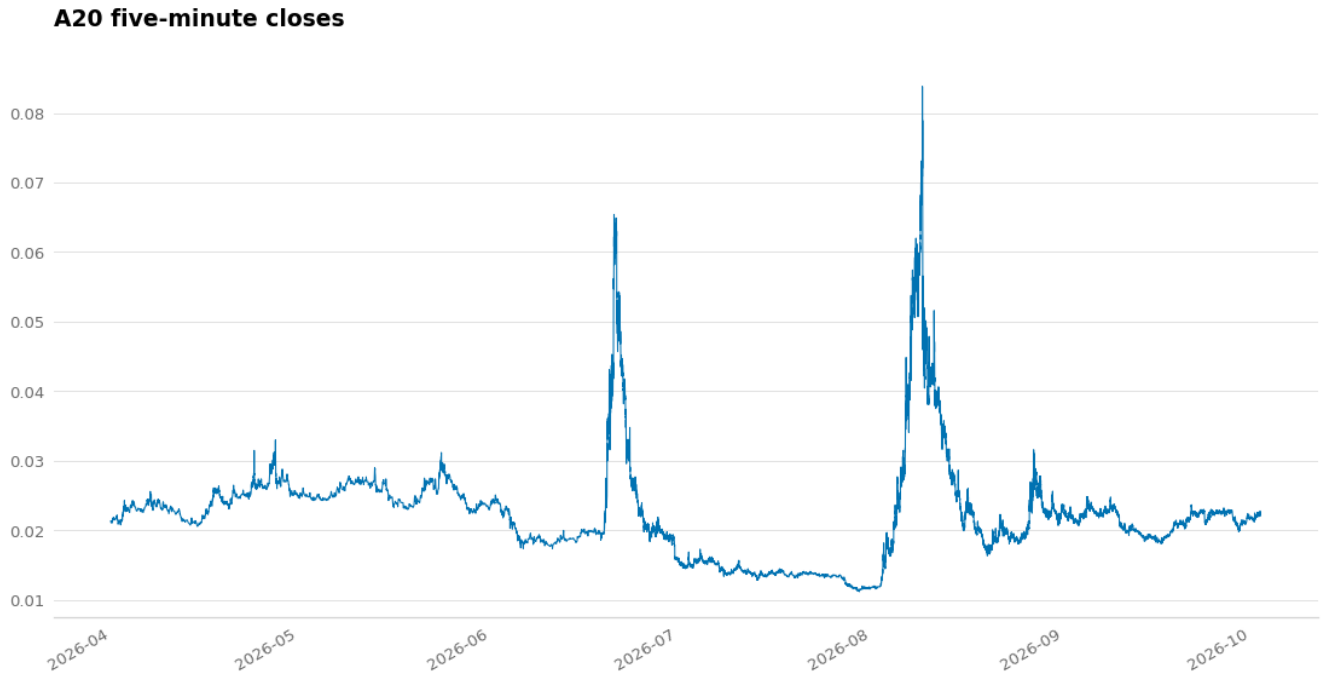
A20



A20, 87 fills, with the sell line crossing below the buy line in mid-August.



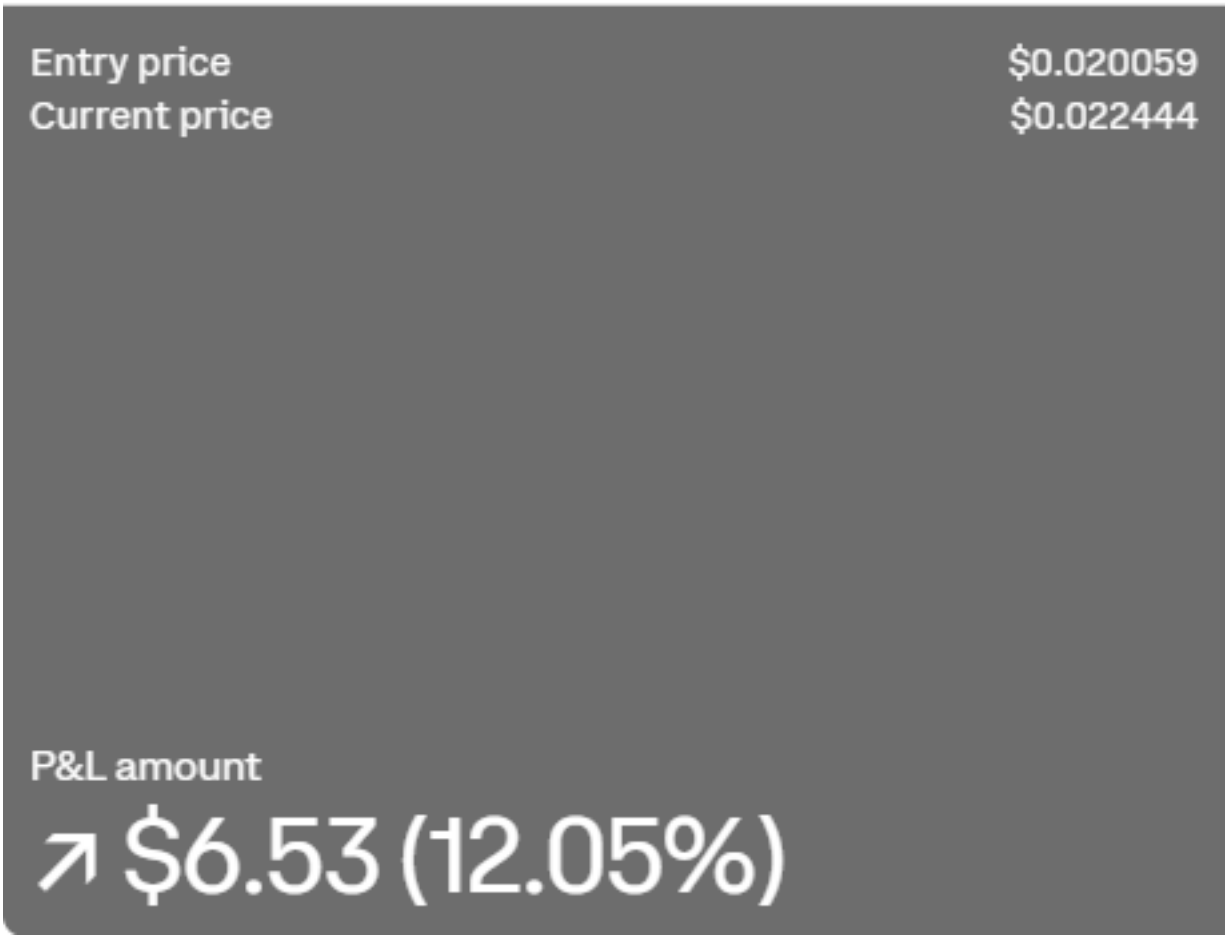
A20 weekly chart, 27 weekly candles built from 21,070 five-minute rows.



A20 price trace, 21,070 five-minute closes across the recorded window.

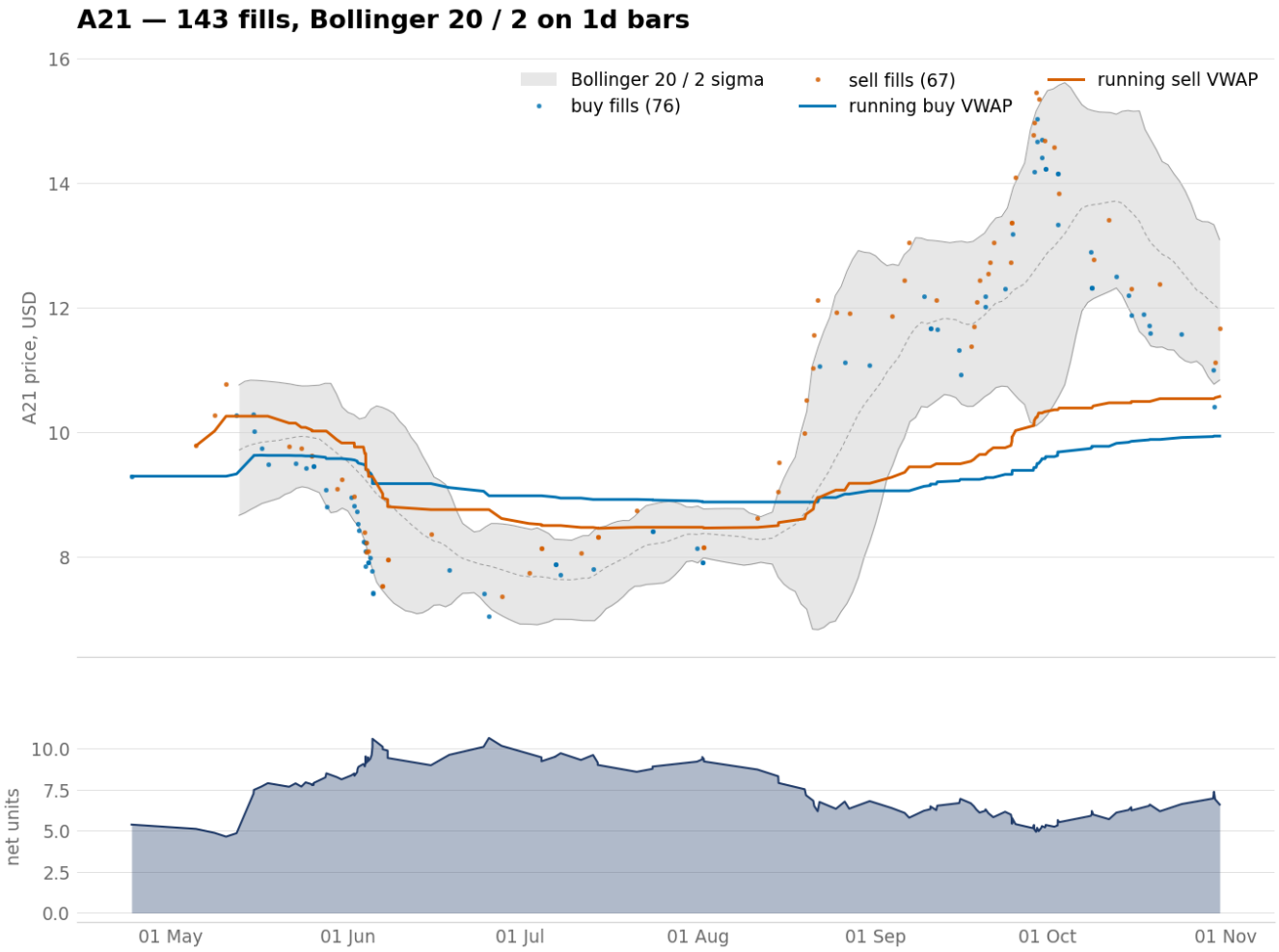
coinbase ADVANCED

A20



A20 position card, with the mark, the name and the code square painted out.

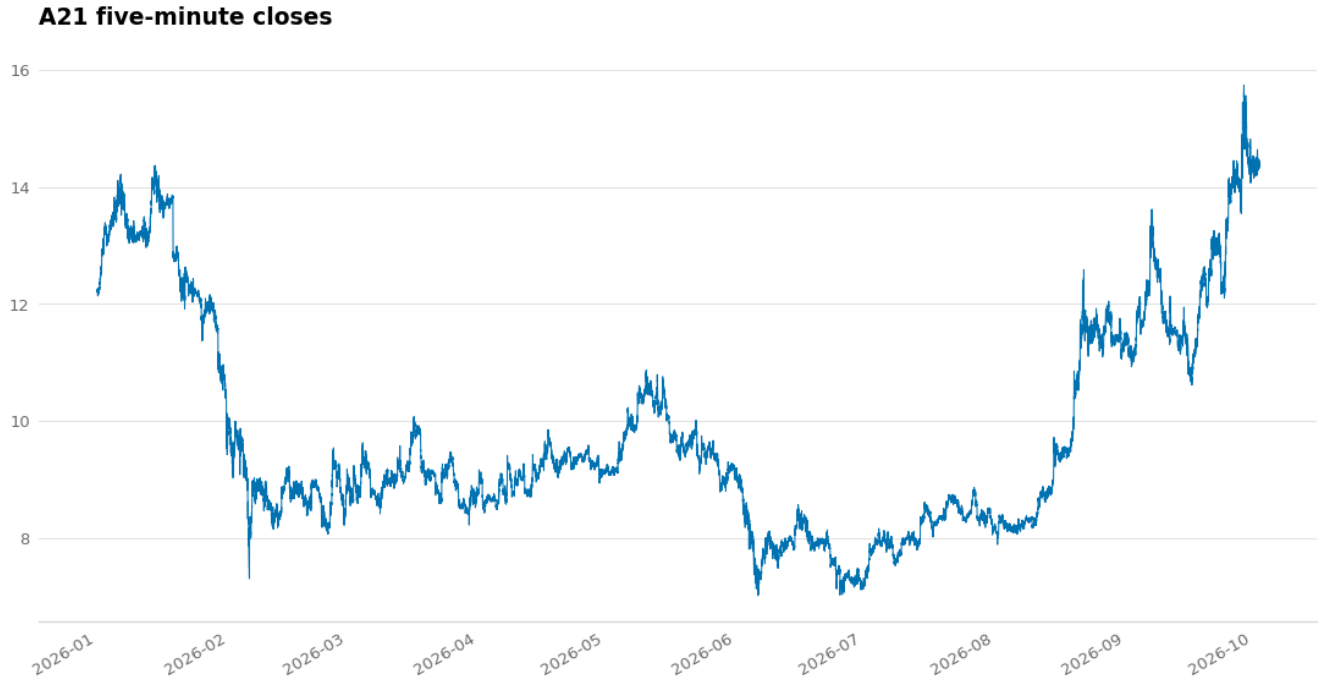
A21



A21, 83 fills, with both lines turning up in the last two weeks.

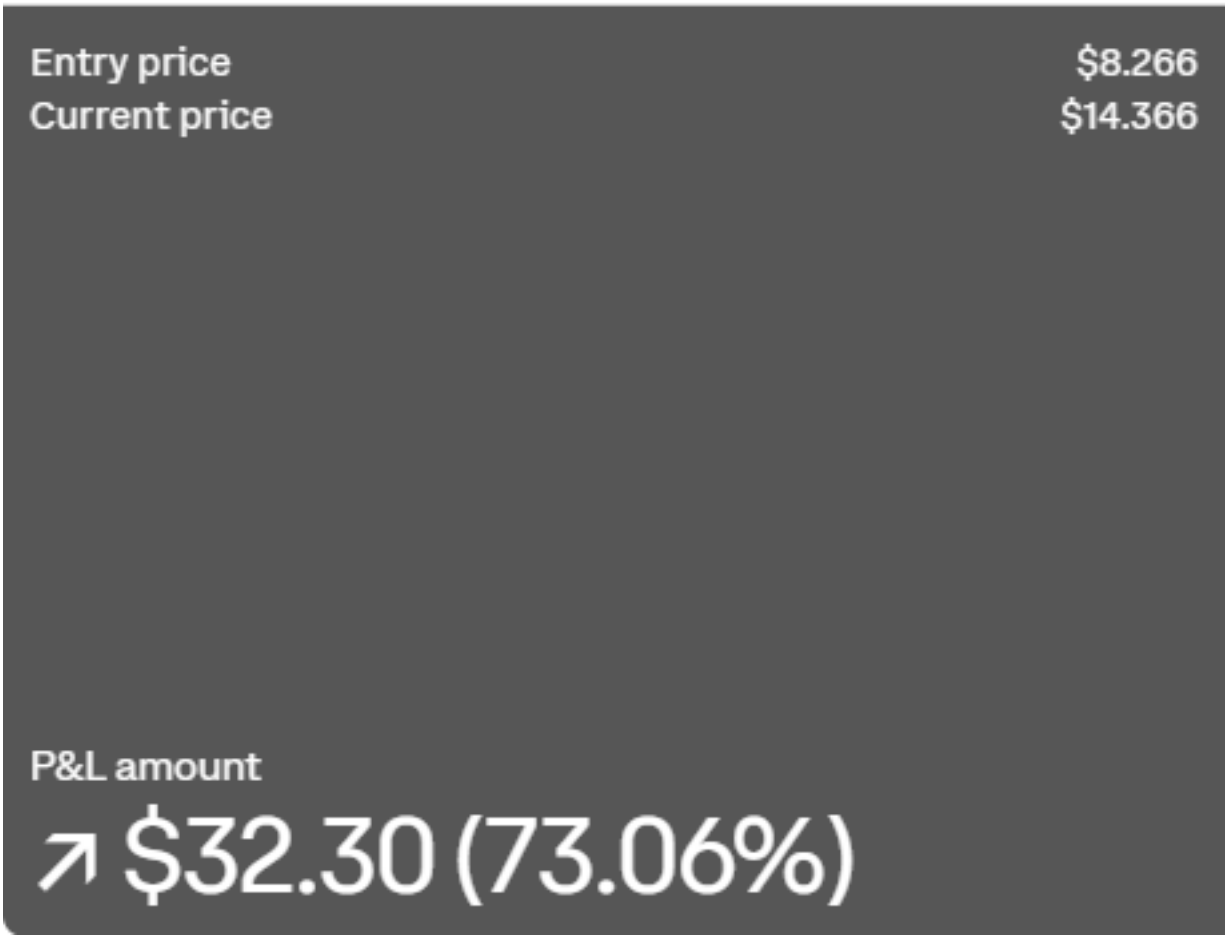


A21 weekly chart, 40 weekly candles built from 79,003 five-minute rows.



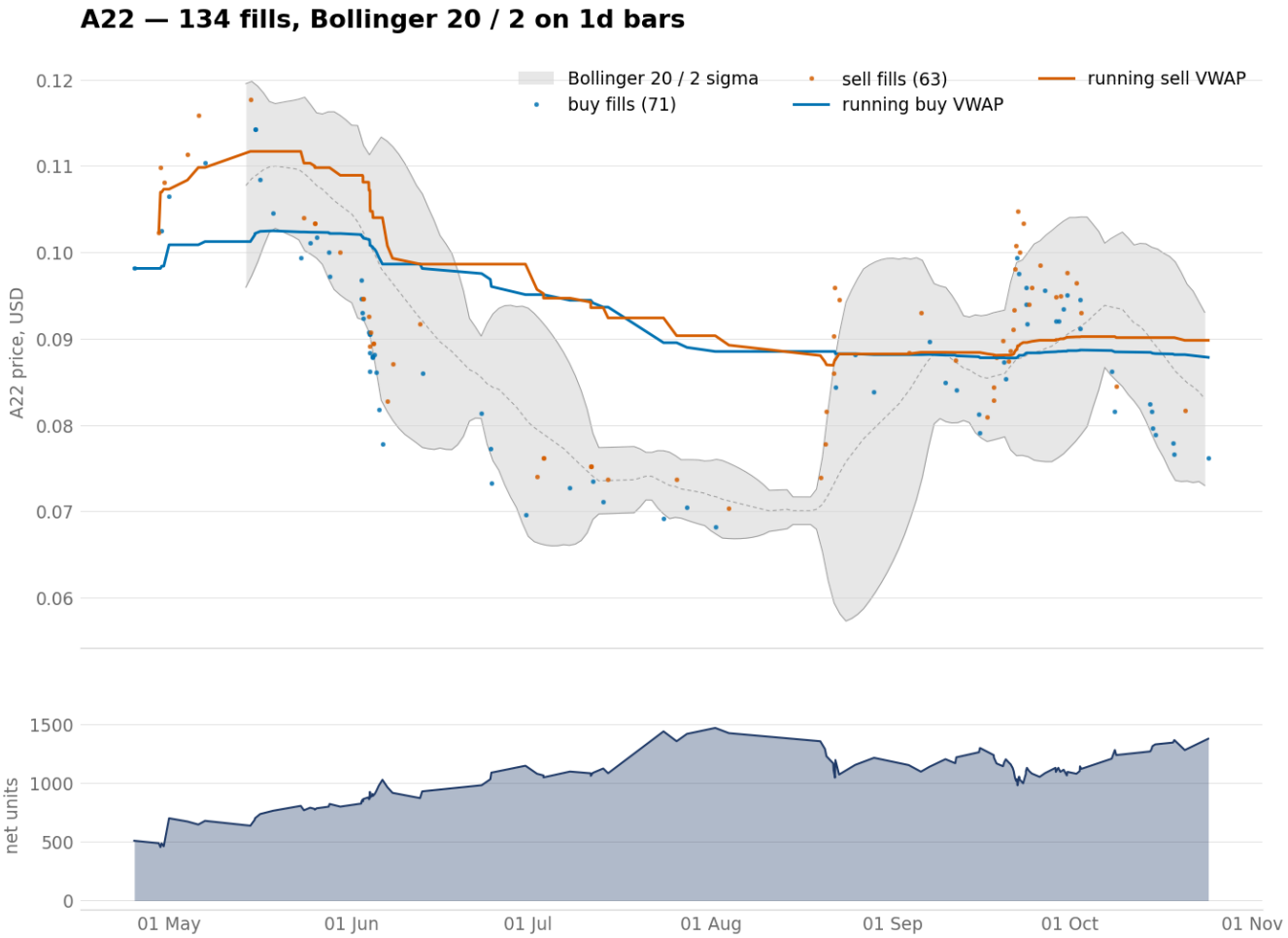
A21 price trace, 79,003 five-minute closes across the recorded window.

A21



A21 position card, with the mark, the name and the code square painted out.

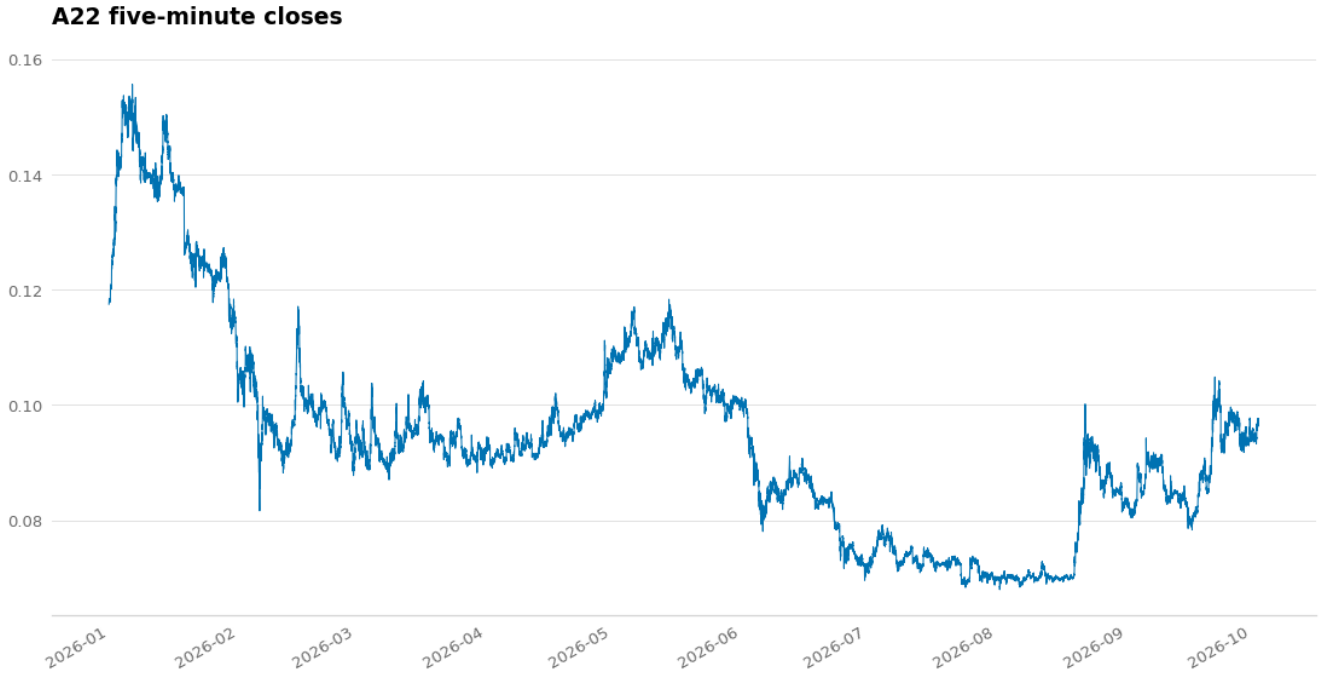
A22



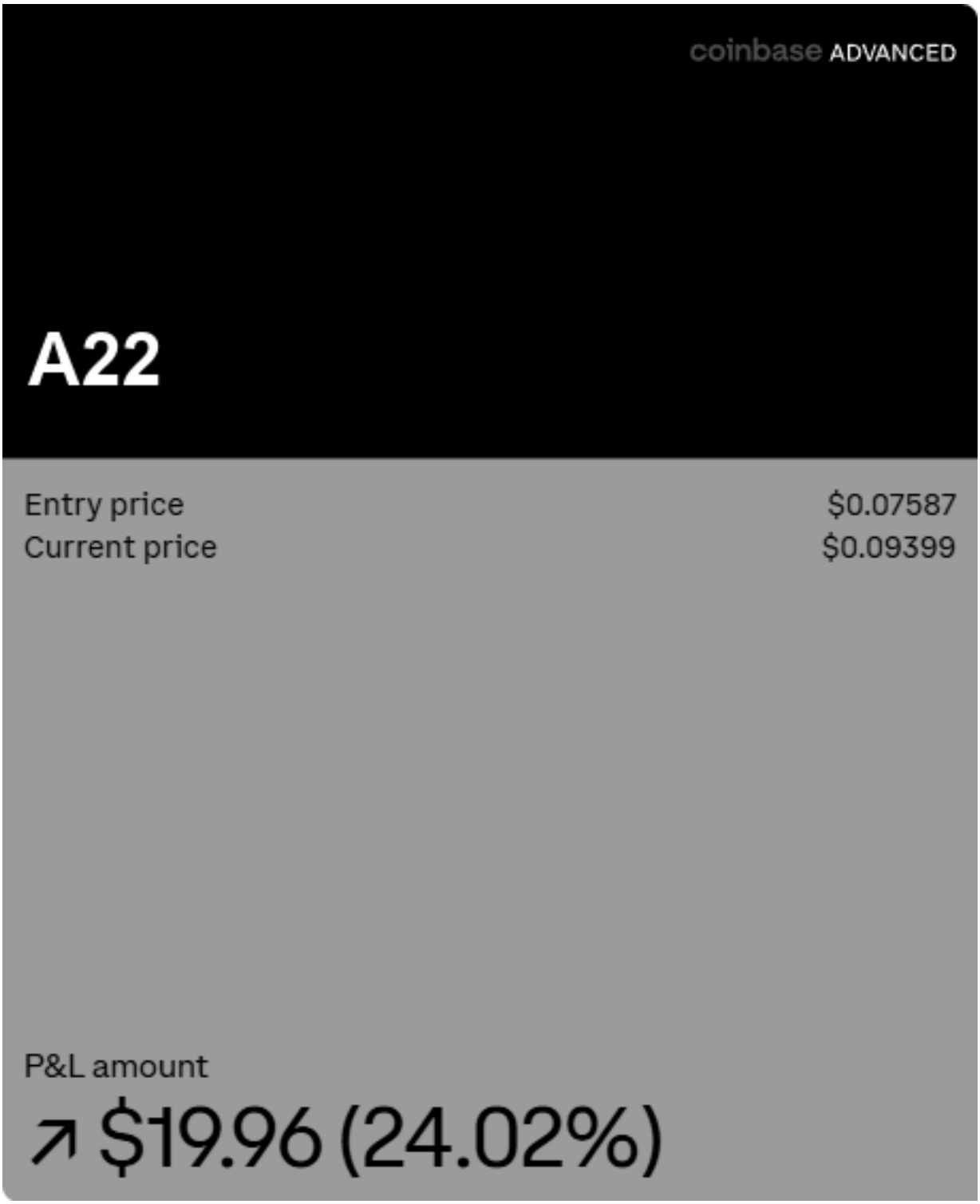
A22, 80 fills, with the two lines meeting in mid-August.



A22 weekly chart, 40 weekly candles built from 79,000 five-minute rows.

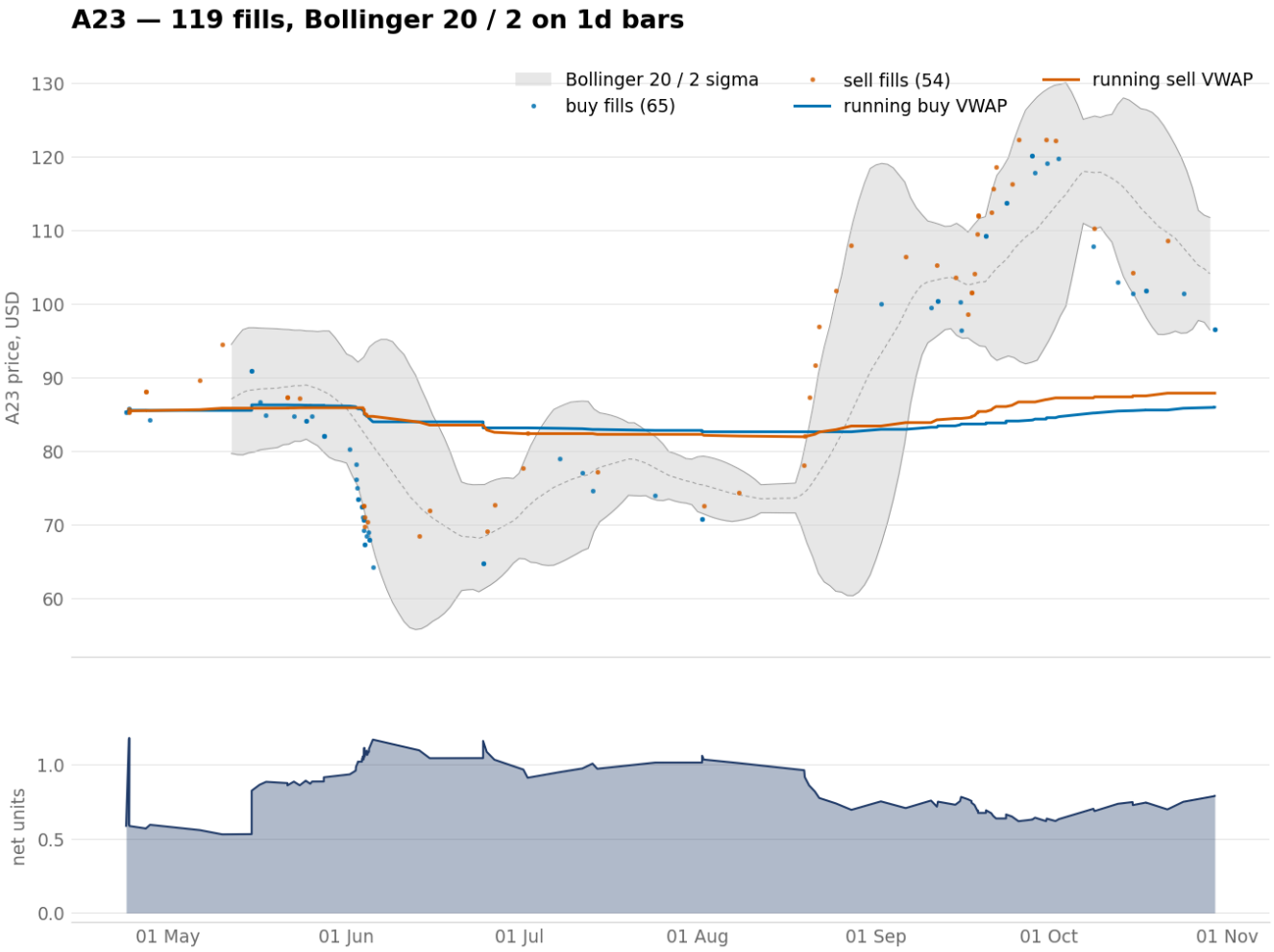


A22 price trace, 79,000 five-minute closes across the recorded window.



A22 position card, with the mark, the name and the code square painted out.

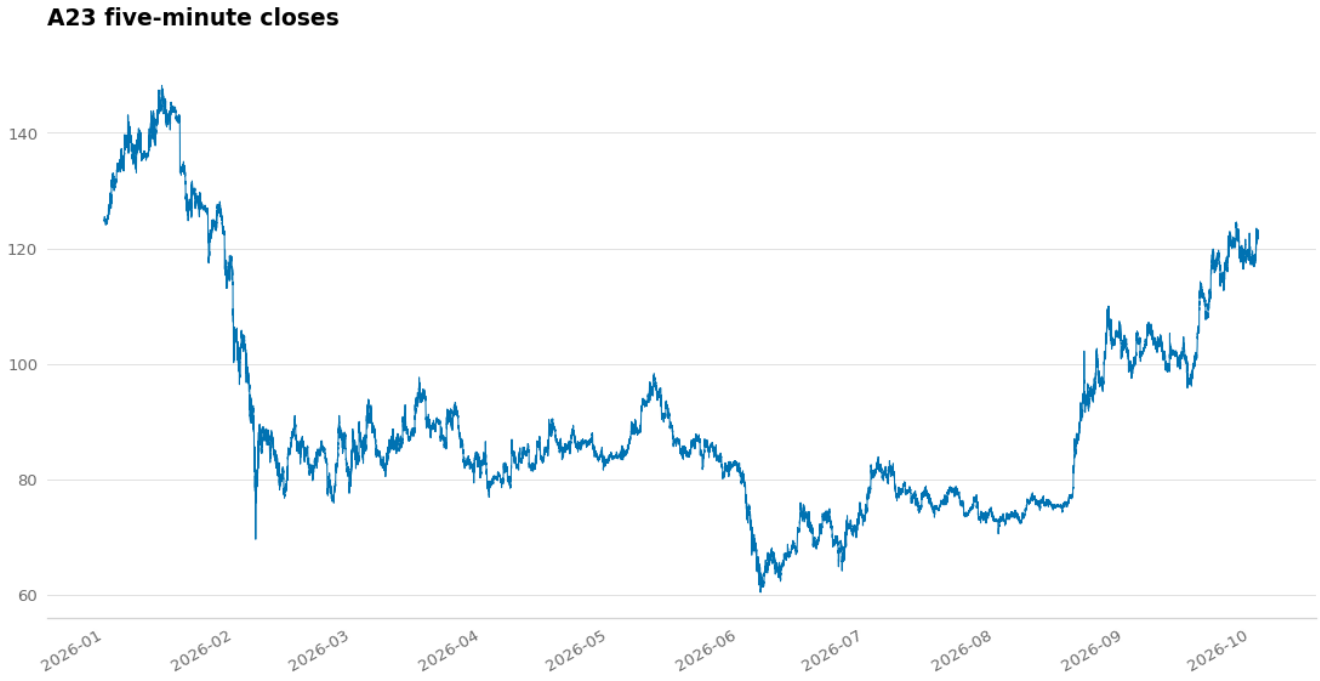
A23



A23, 77 fills, with the two lines within about a dollar of each other.



A23 weekly chart, 40 weekly candles built from 79,004 five-minute rows.



A23 price trace, 79,004 five-minute closes across the recorded window.

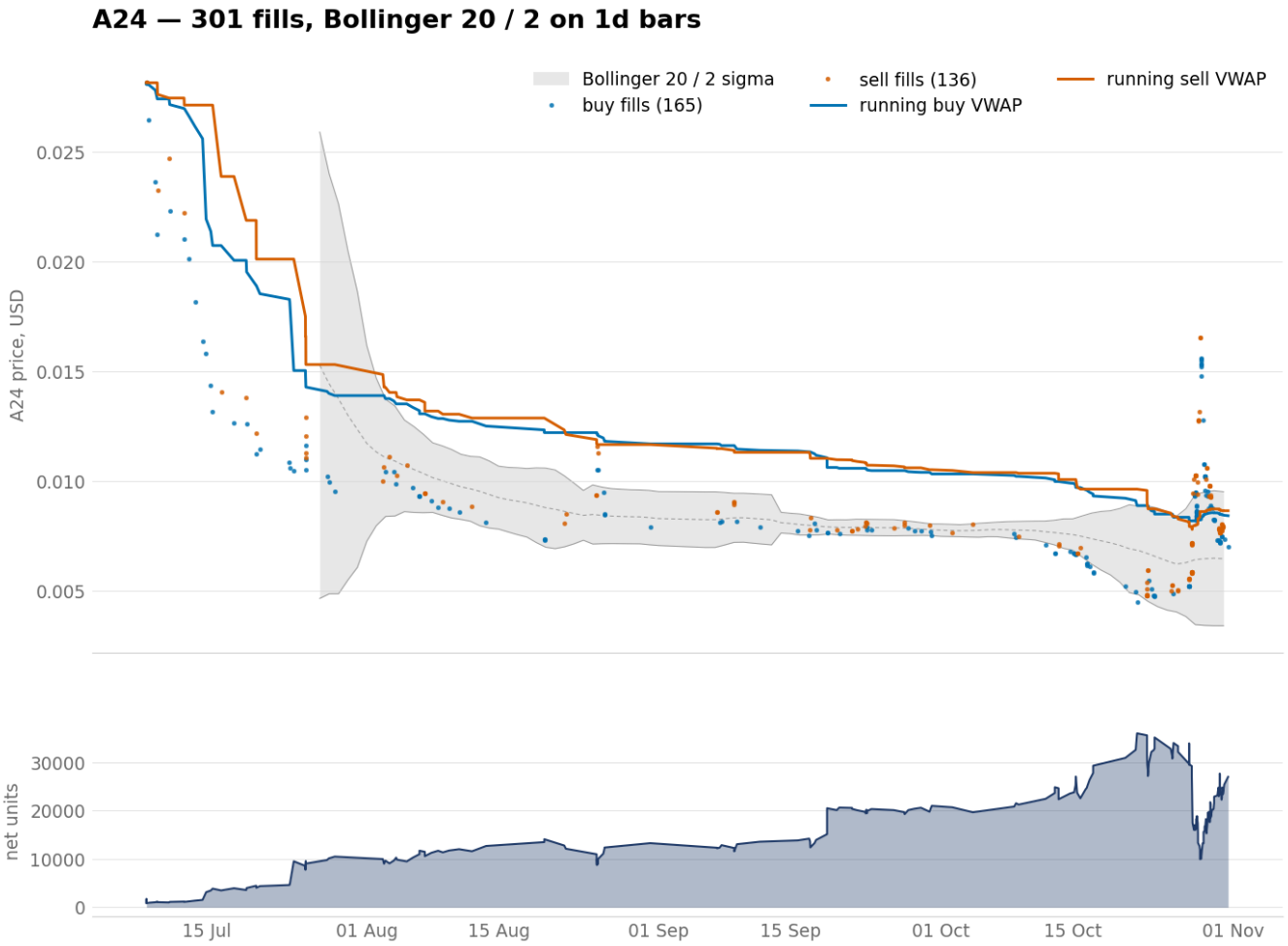
coinbase ADVANCED

A23

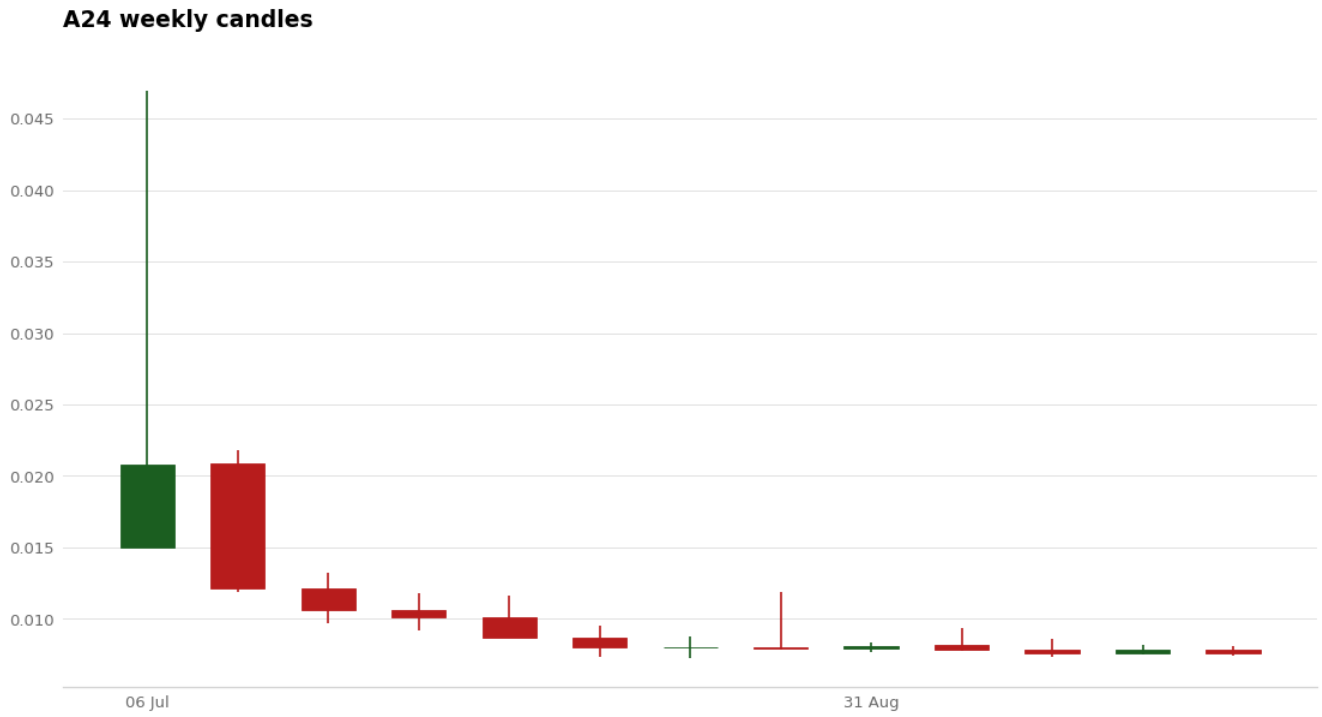


A23 position card, with the mark, the name and the code square painted out.

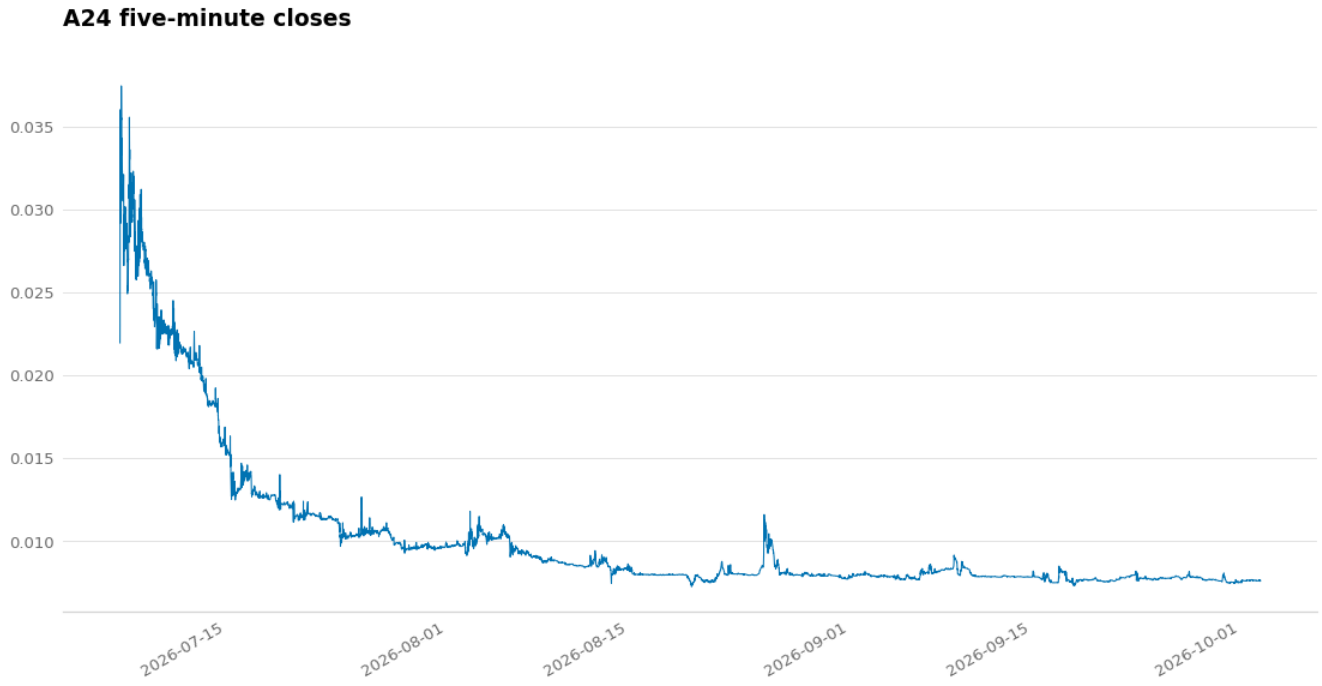
A24



A24, 75 fills, with both lines falling by more than half.



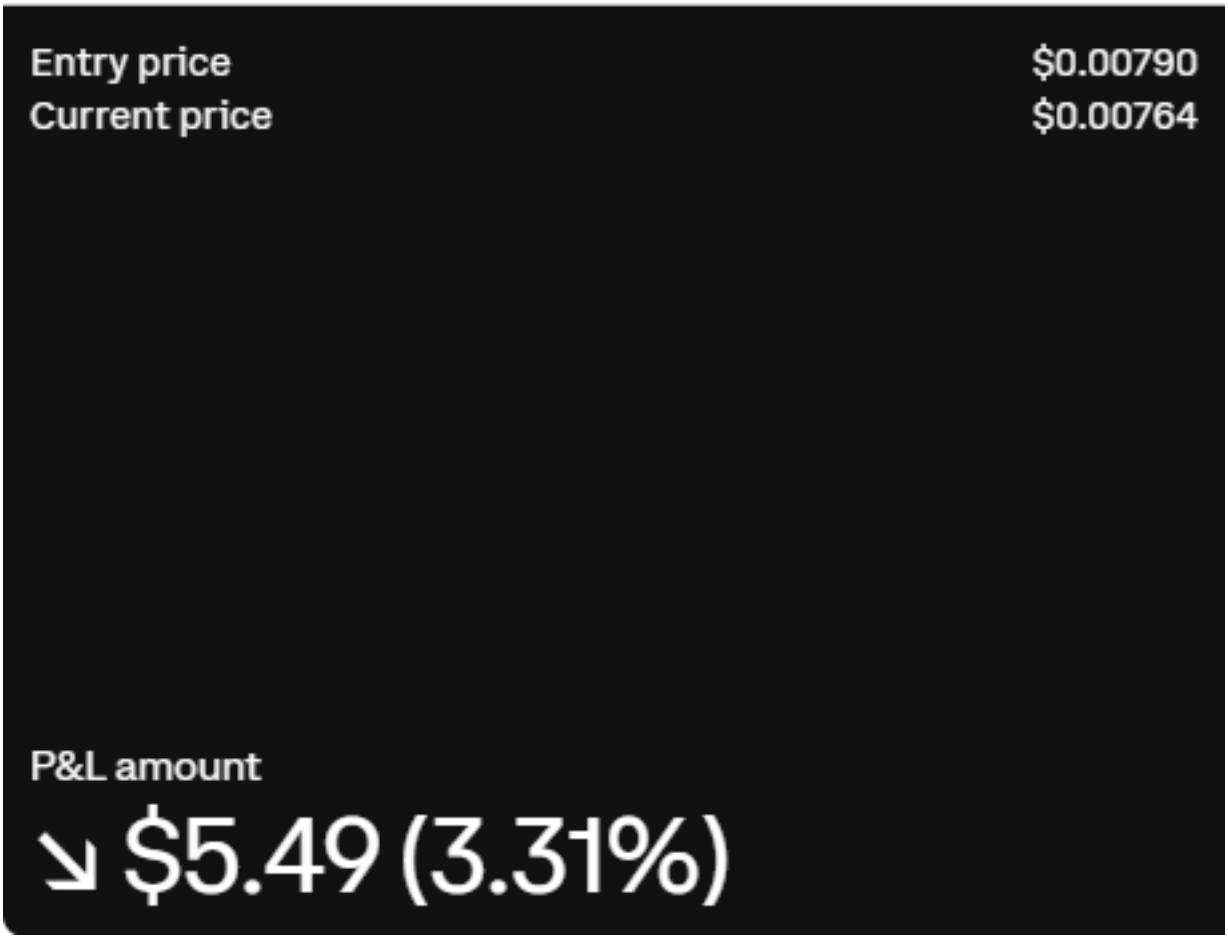
A24 weekly chart, 13 weekly candles built from 8,734 five-minute rows.



A24 price trace, 8,734 five-minute closes across the recorded window.

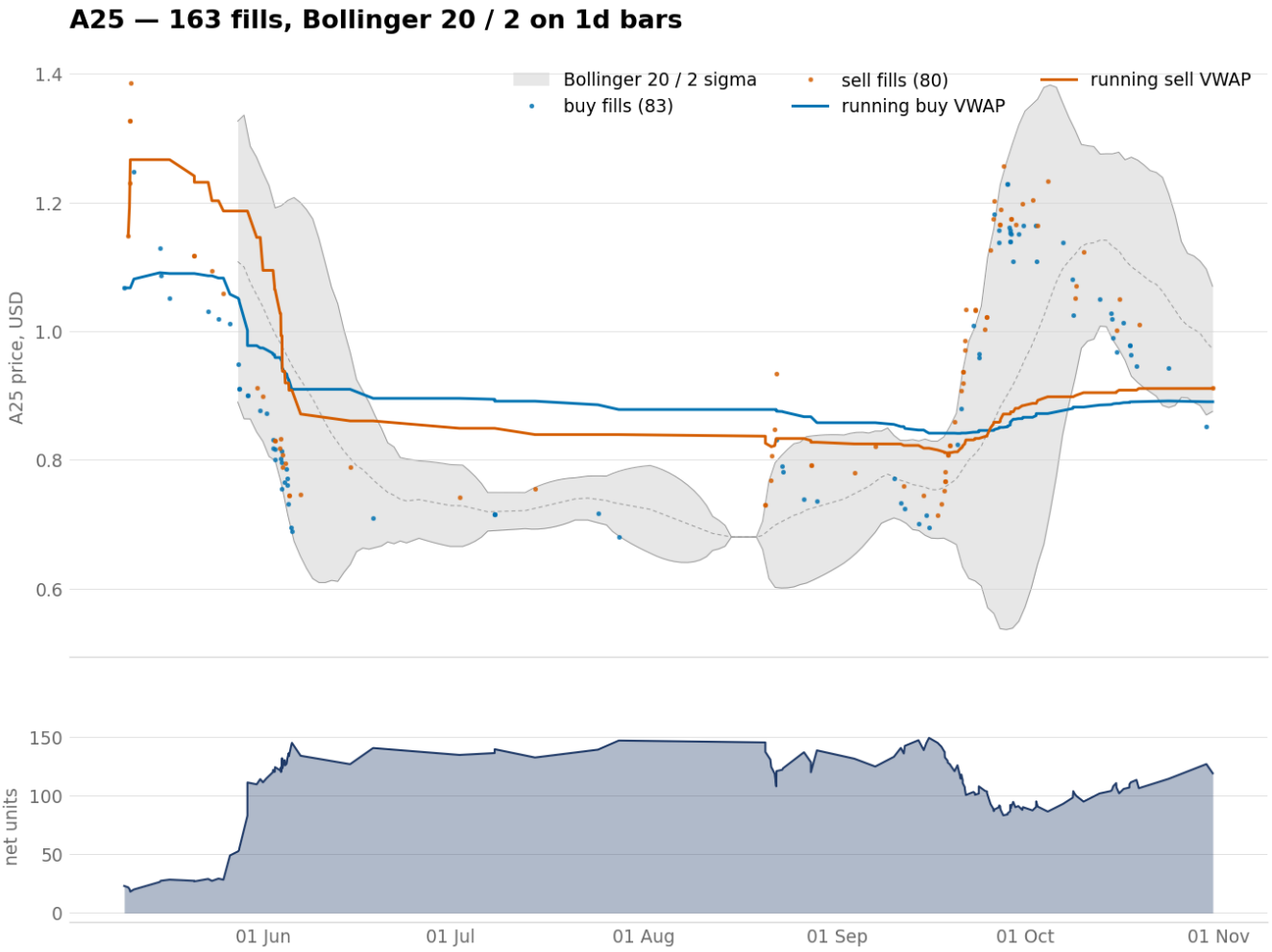
coinbase ADVANCED

A24



A24 position card, with the mark, the name and the code square painted out.

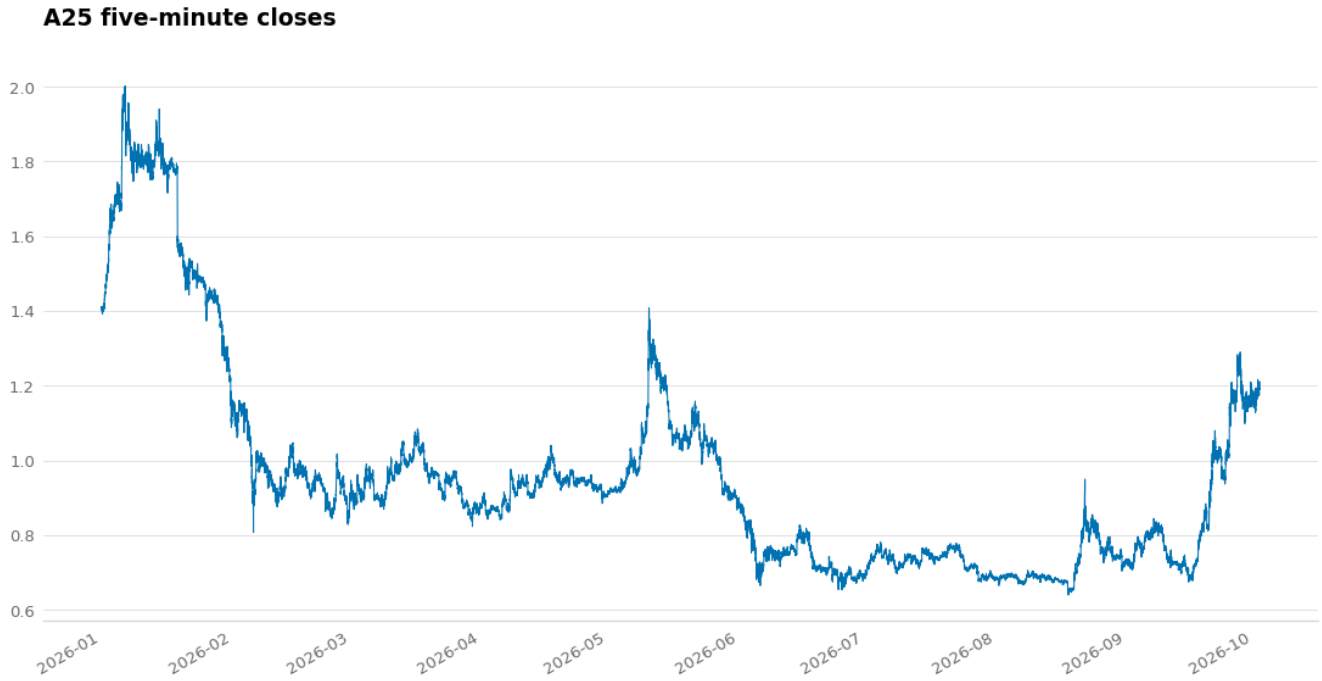
A25



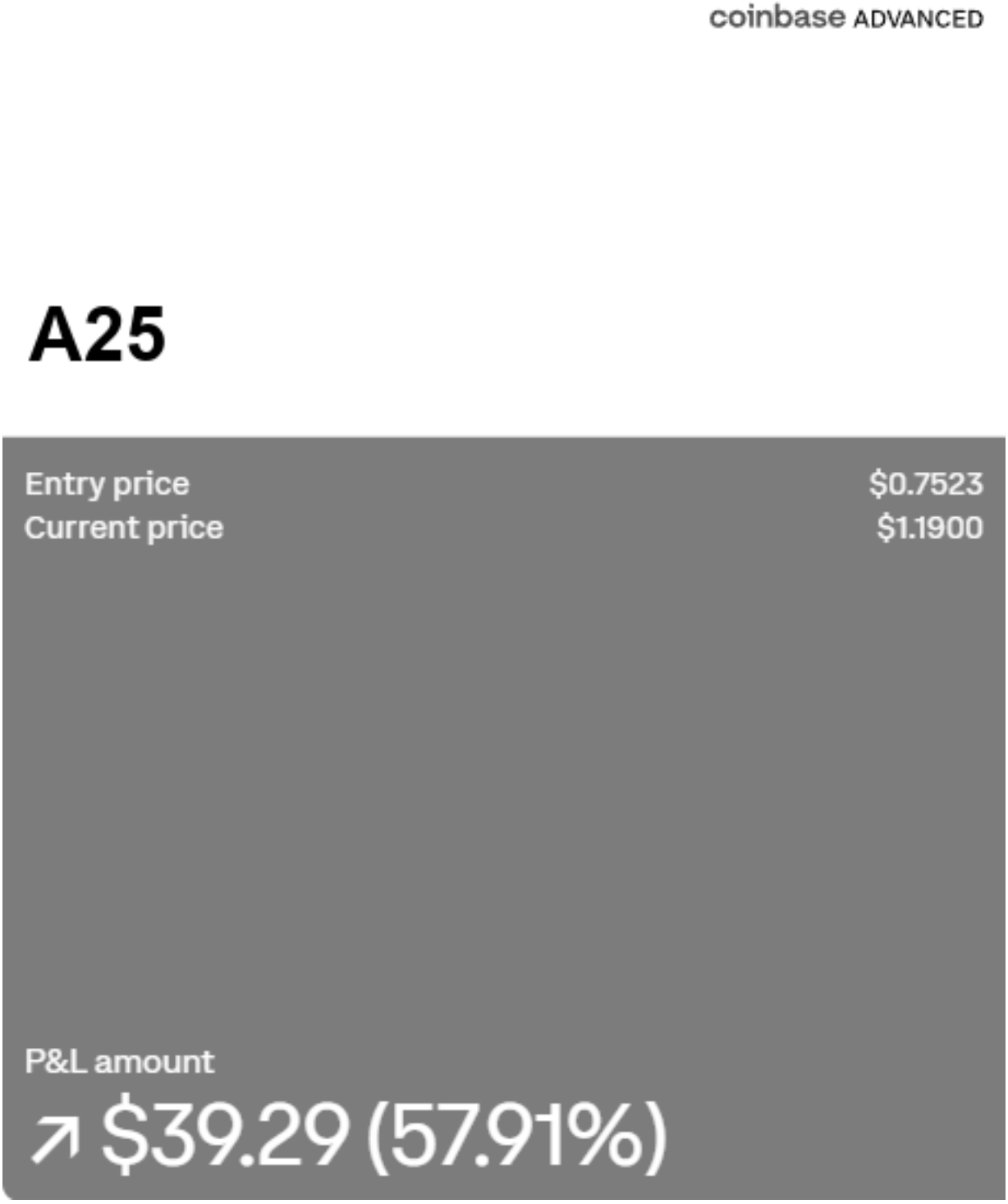
A25, 74 fills, with a step down at the start of June.



A25 weekly chart, 40 weekly candles built from 78,996 five-minute rows.

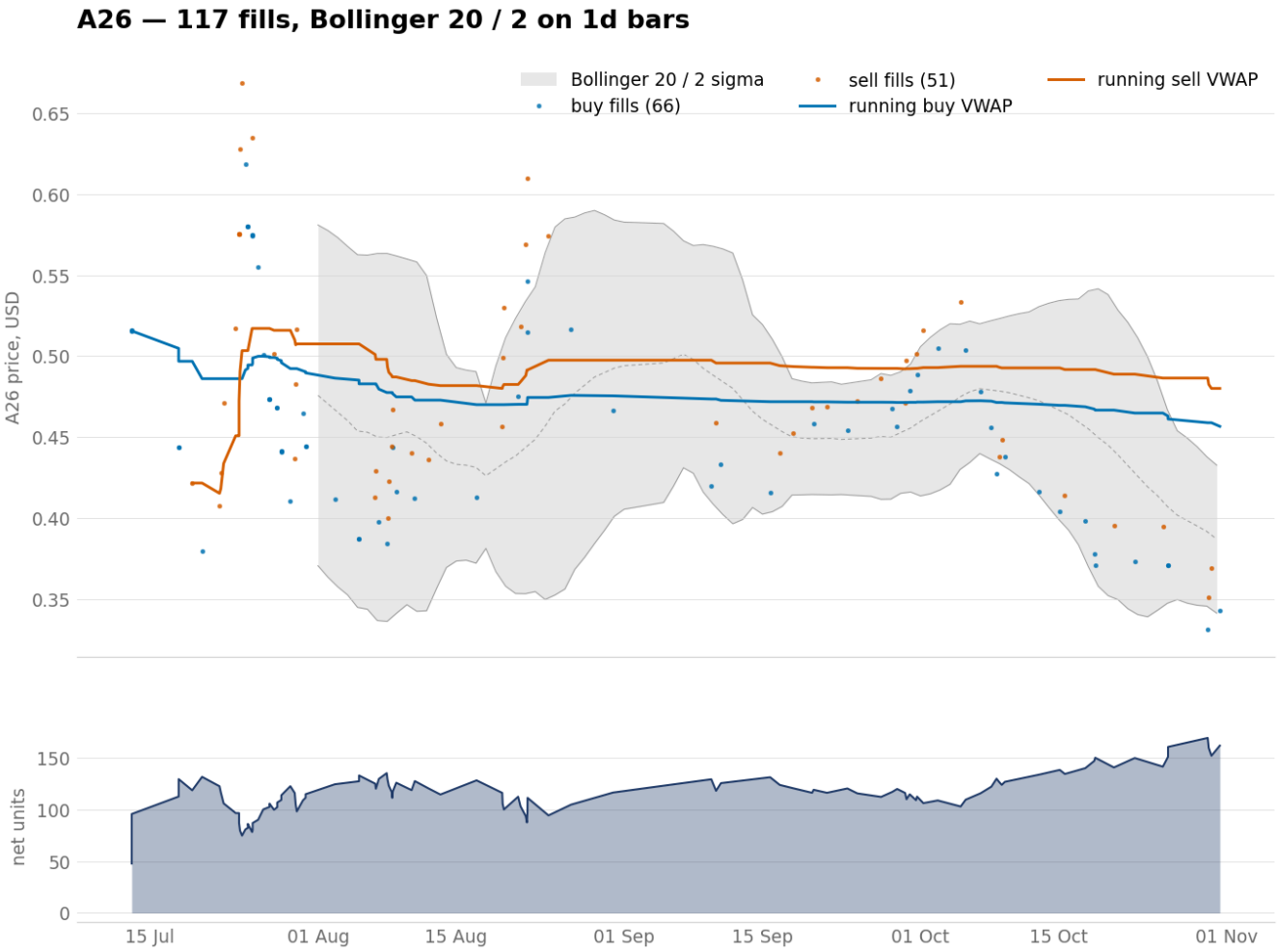


A25 price trace, 78,996 five-minute closes across the recorded window.

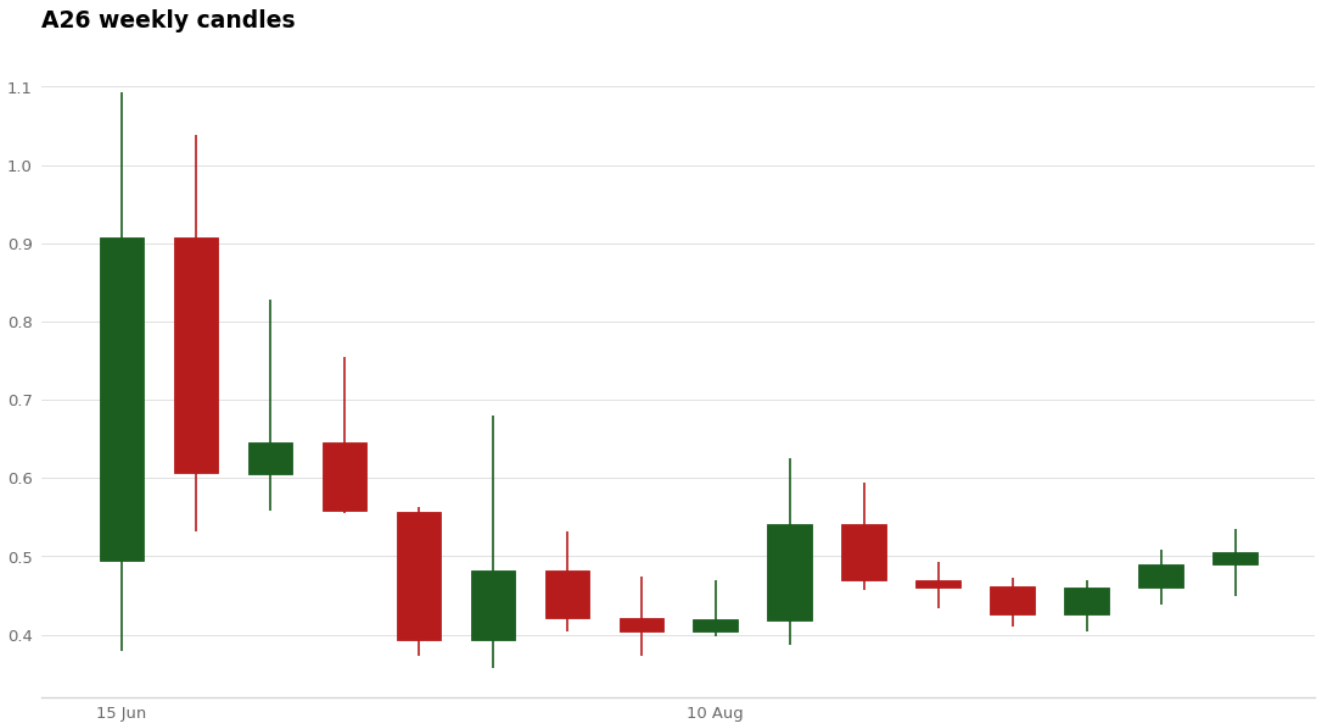


A25 position card, with the mark, the name and the code square painted out.

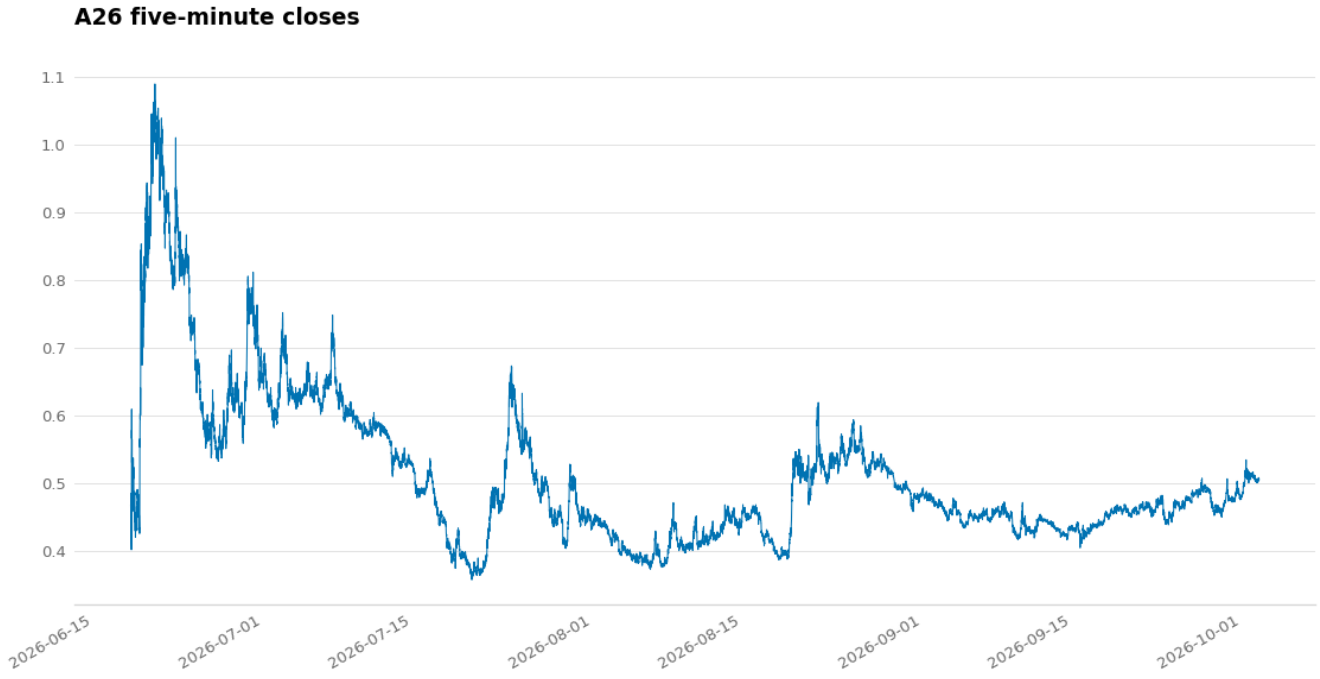
A26



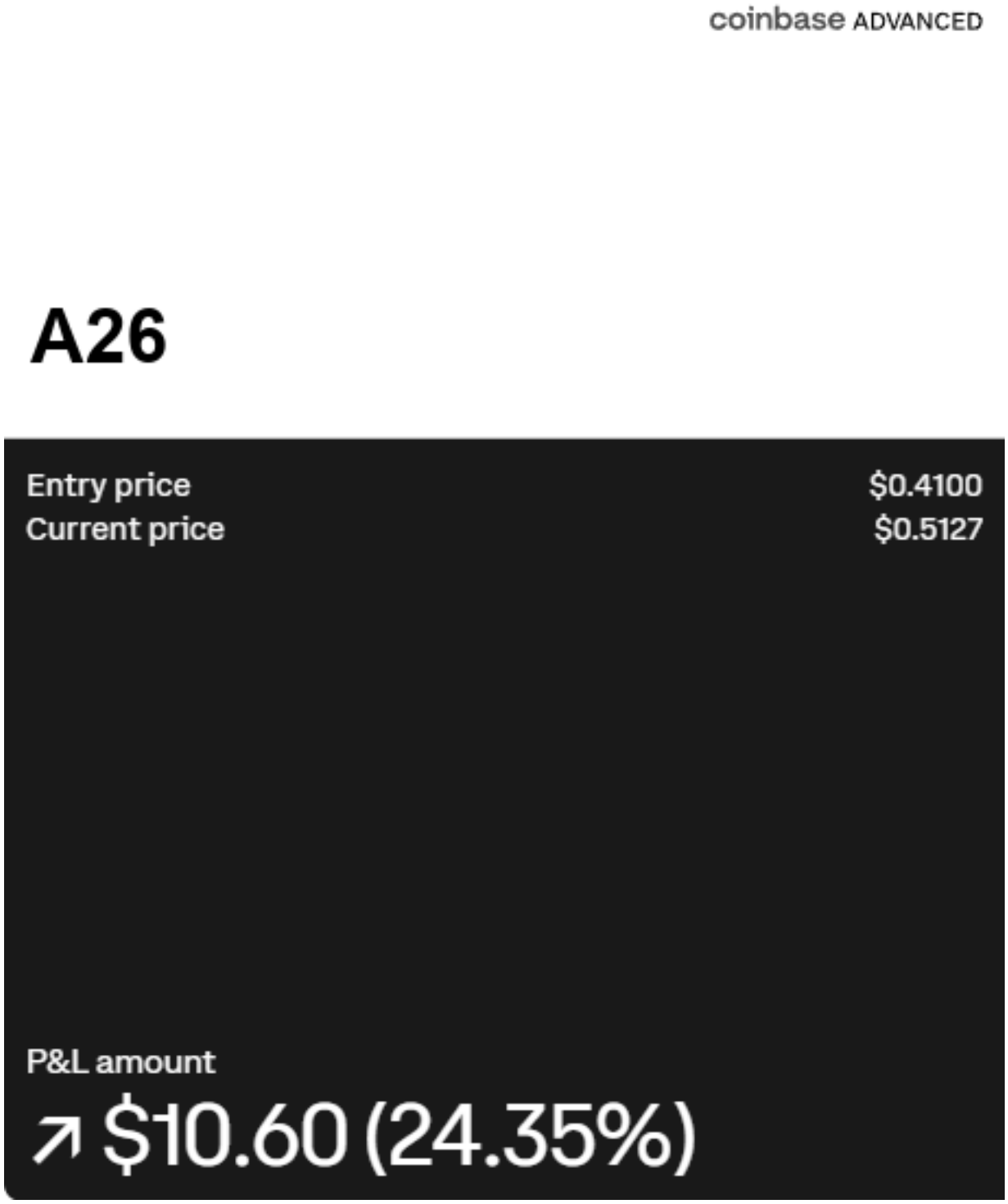
A26, 73 fills, with the sell line above the buy line from late July.



A26 weekly chart, 16 weekly candles built from 25,946 five-minute rows.

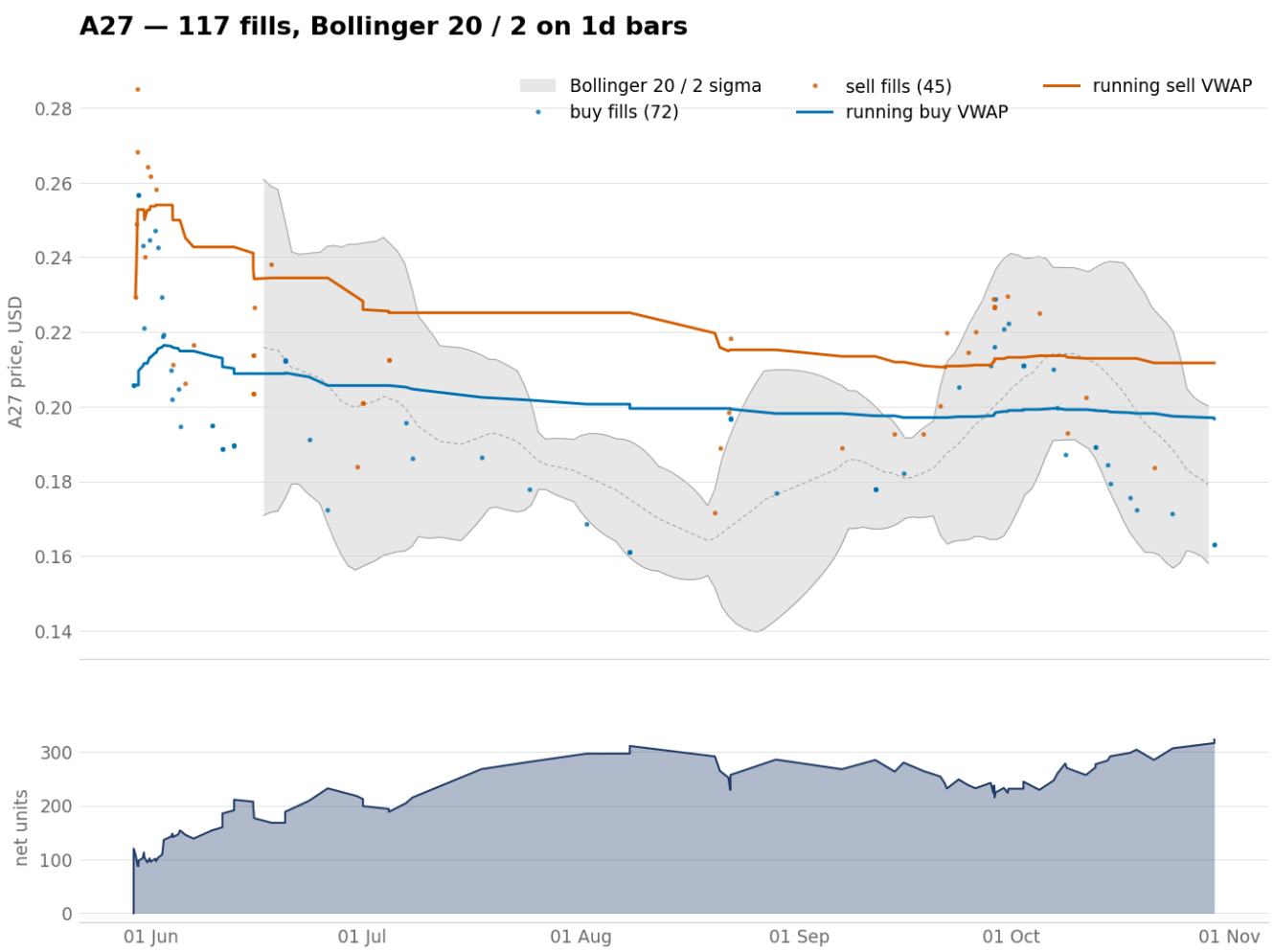


A26 price trace, 25,946 five-minute closes across the recorded window.



A26 position card, with the mark, the name and the code square painted out.

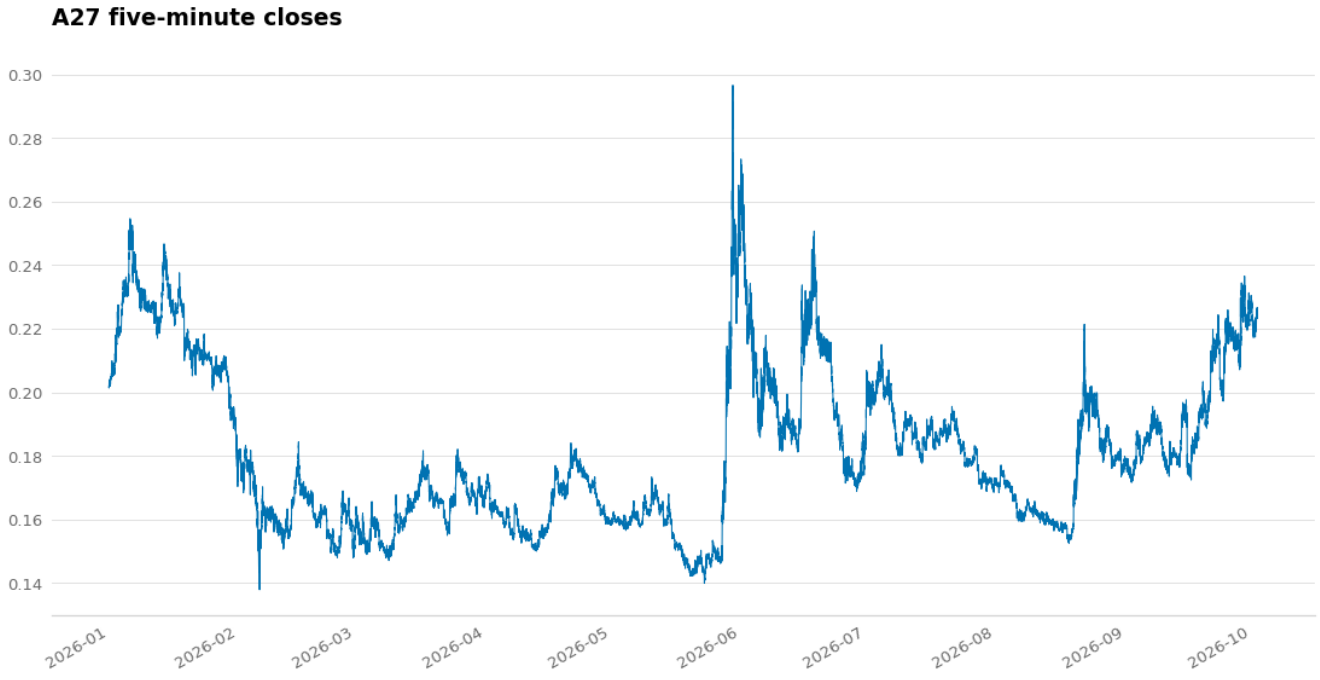
A27



A27, 73 fills, with the sell line about 0.02 above the buy line throughout.



A27 weekly chart, 40 weekly candles built from 79,003 five-minute rows.



A27 price trace, 79,003 five-minute closes across the recorded window.

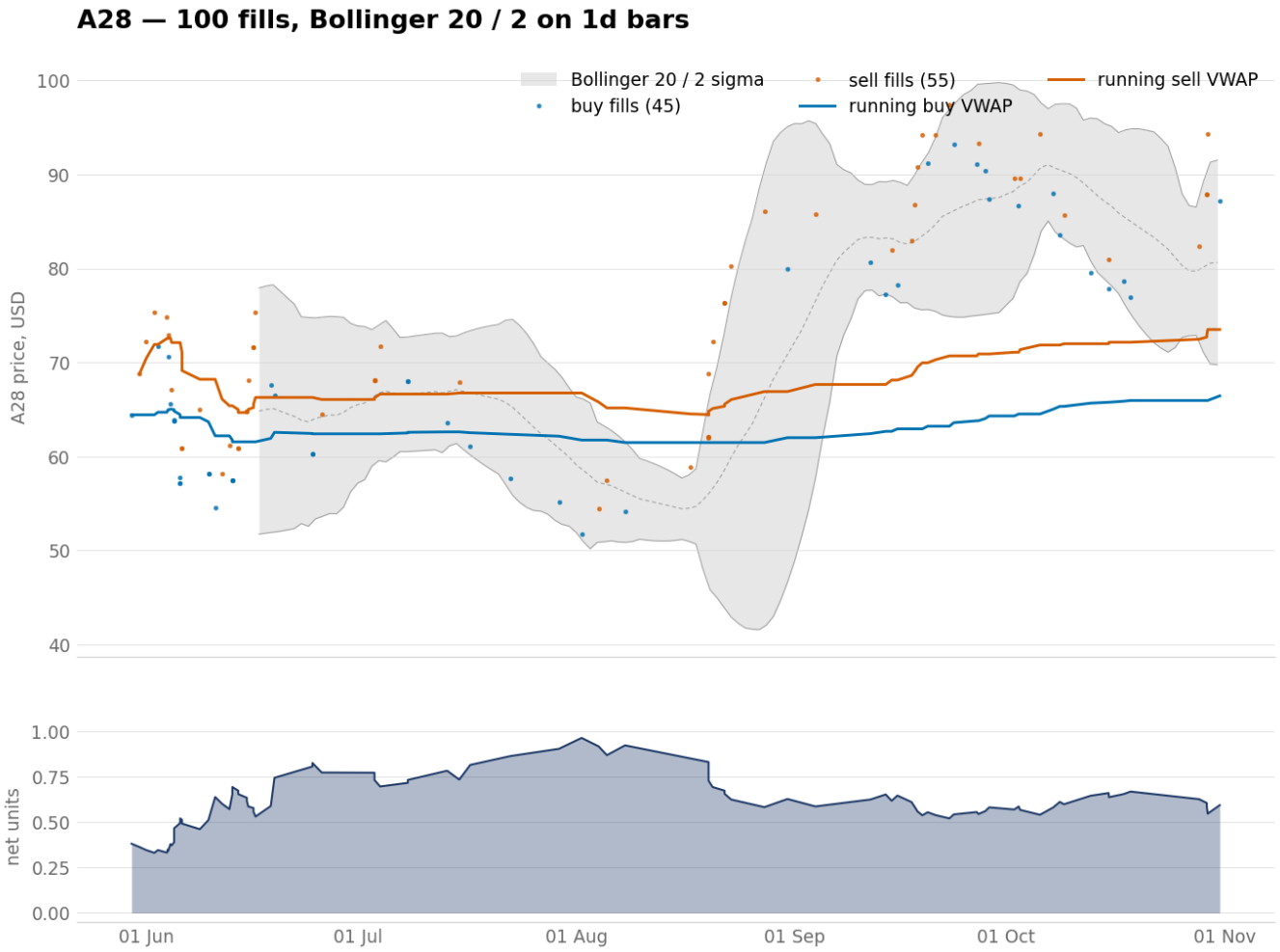
coinbase ADVANCED

A27



A27 position card, with the mark, the name and the code square painted out.

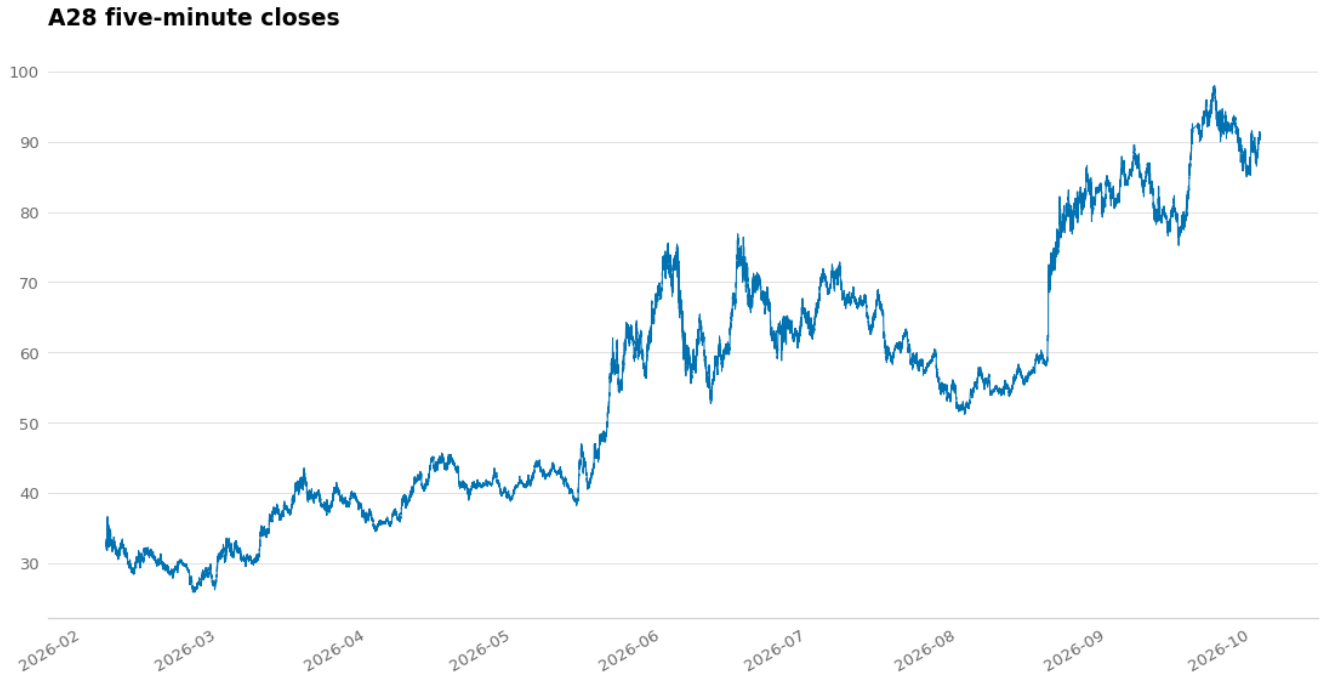
A28



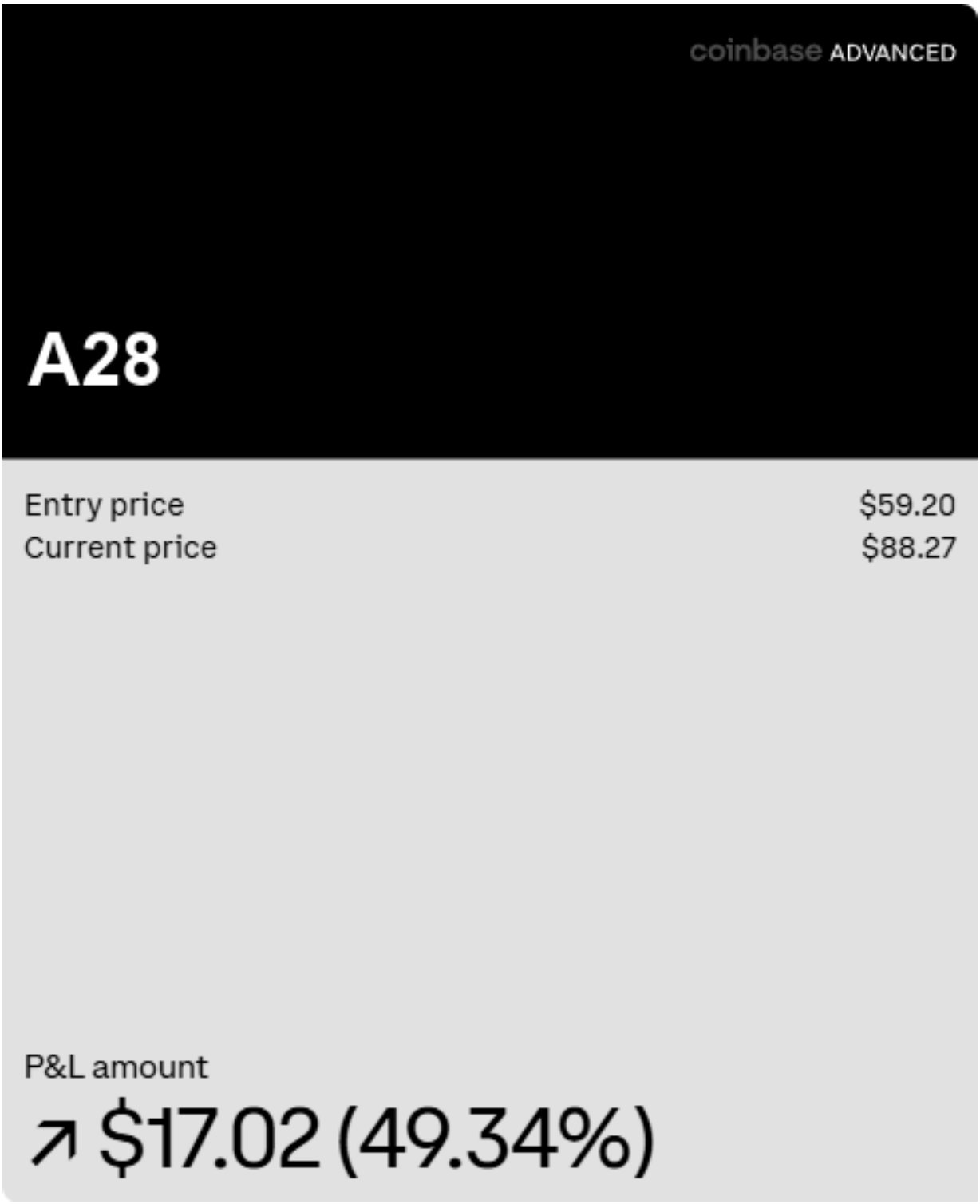
A28, 66 fills, with more sells than buys.



A28 weekly chart, 35 weekly candles built from 68,265 five-minute rows.

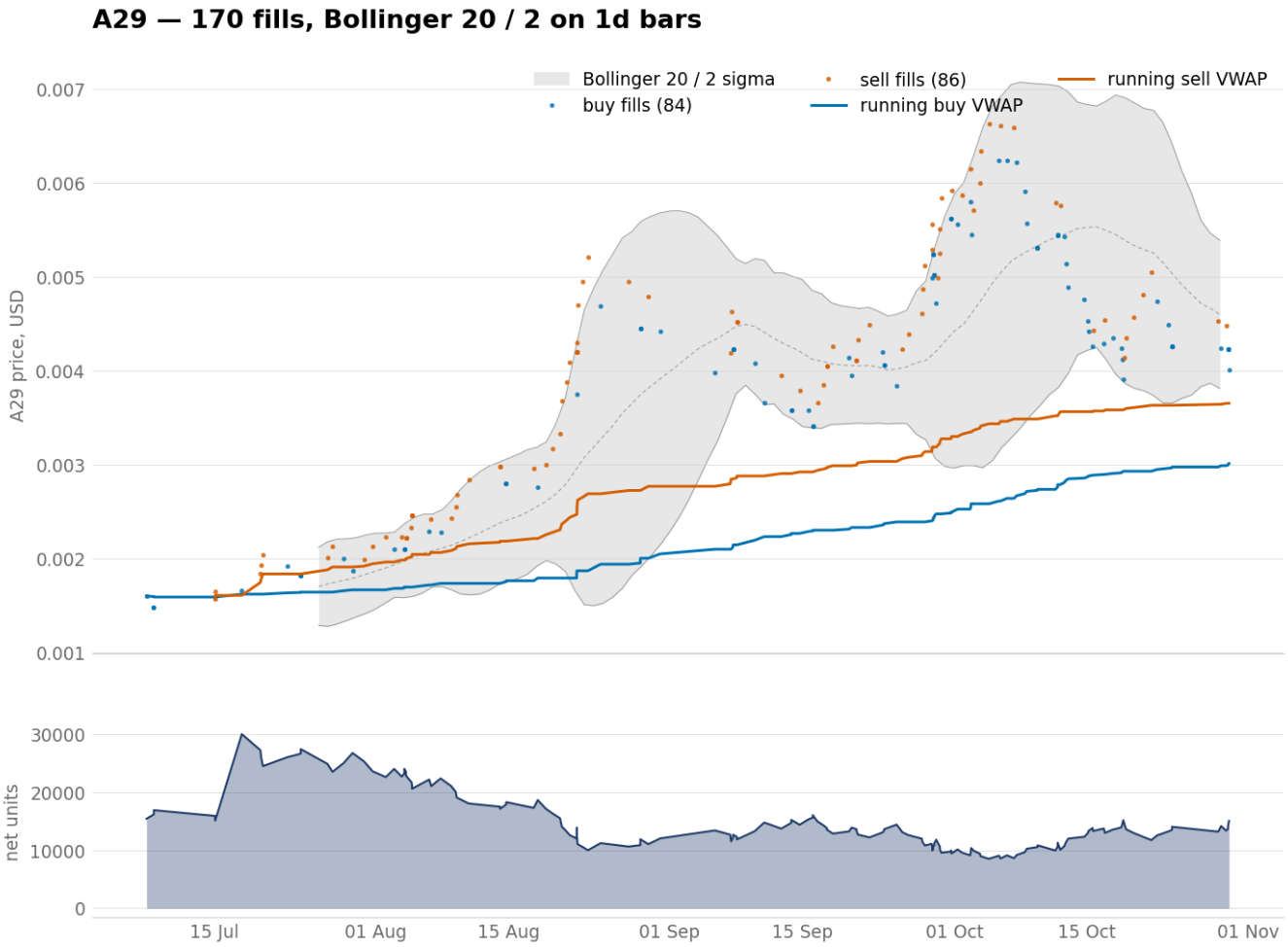


A28 price trace, 68,265 five-minute closes across the recorded window.



A28 position card, with the mark, the name and the code square painted out.

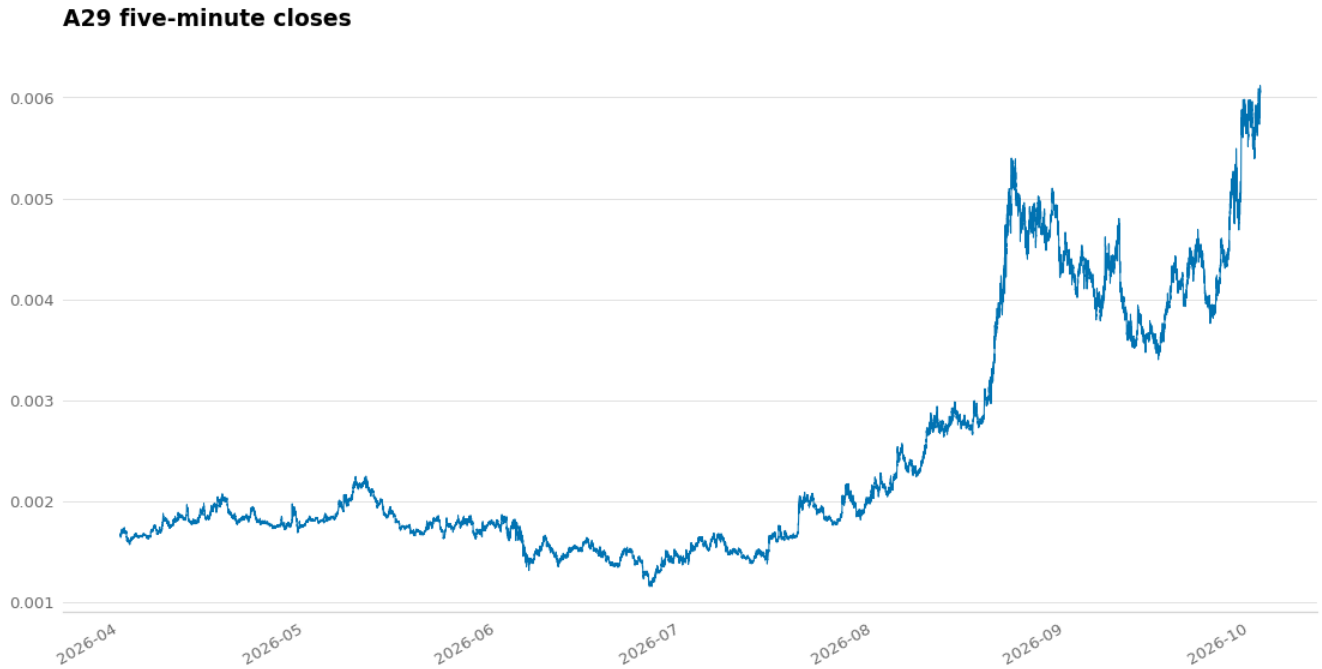
A29



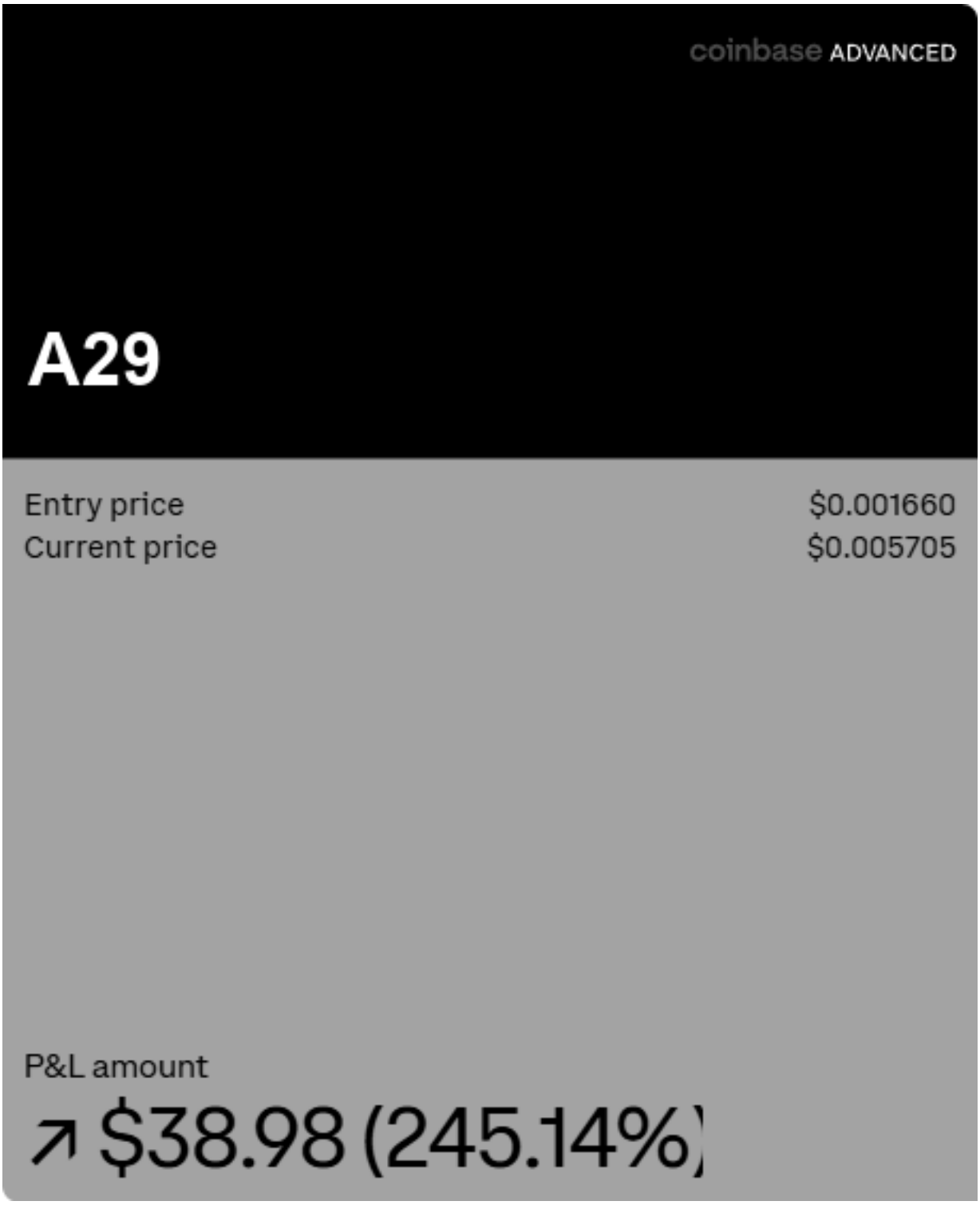
A29, 63 fills, with both lines rising through August.



A29 weekly chart, 27 weekly candles built from 51,228 five-minute rows.

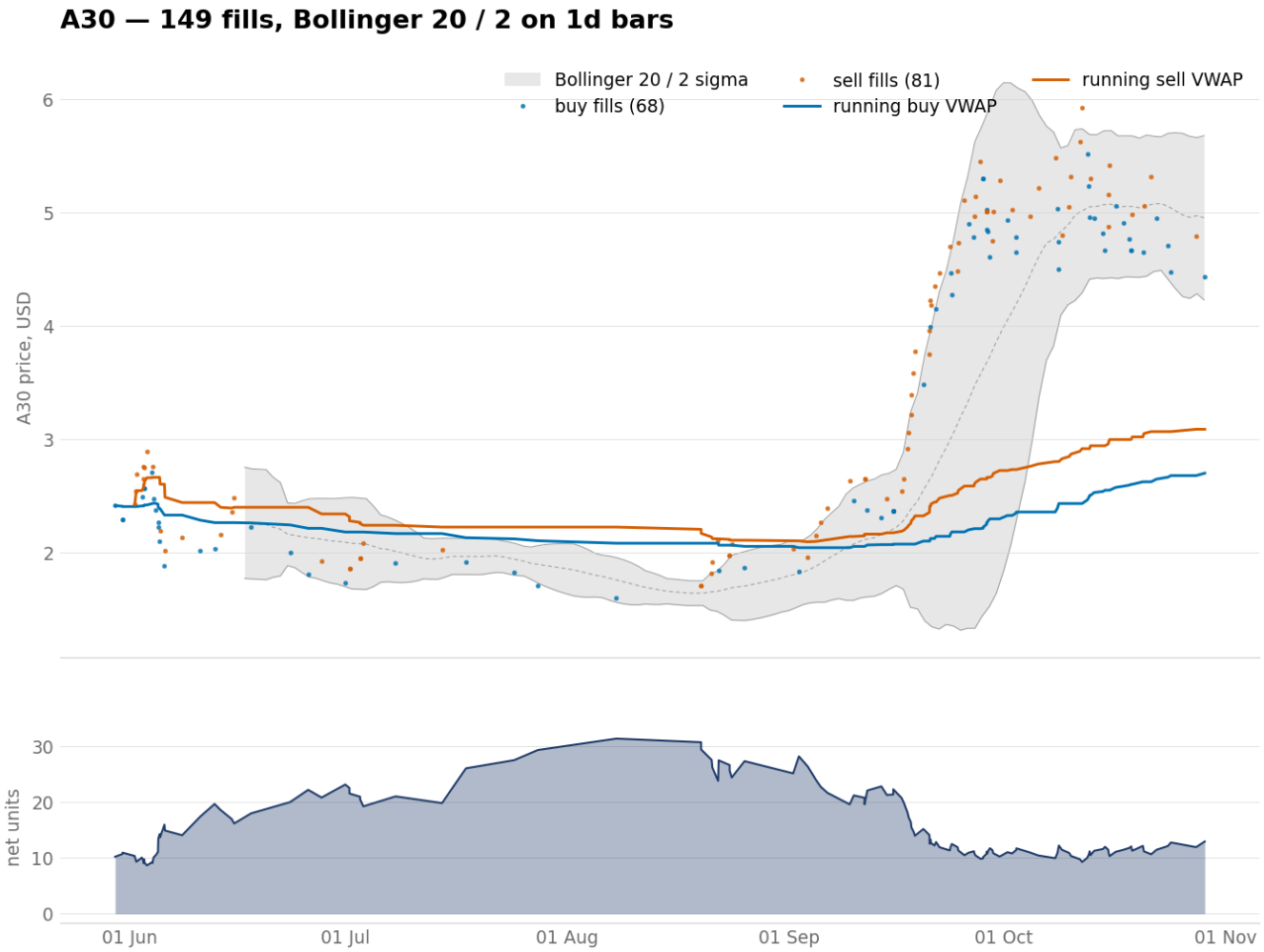


A29 price trace, 51,228 five-minute closes across the recorded window.



A29 position card, with the mark, the name and the code square painted out.

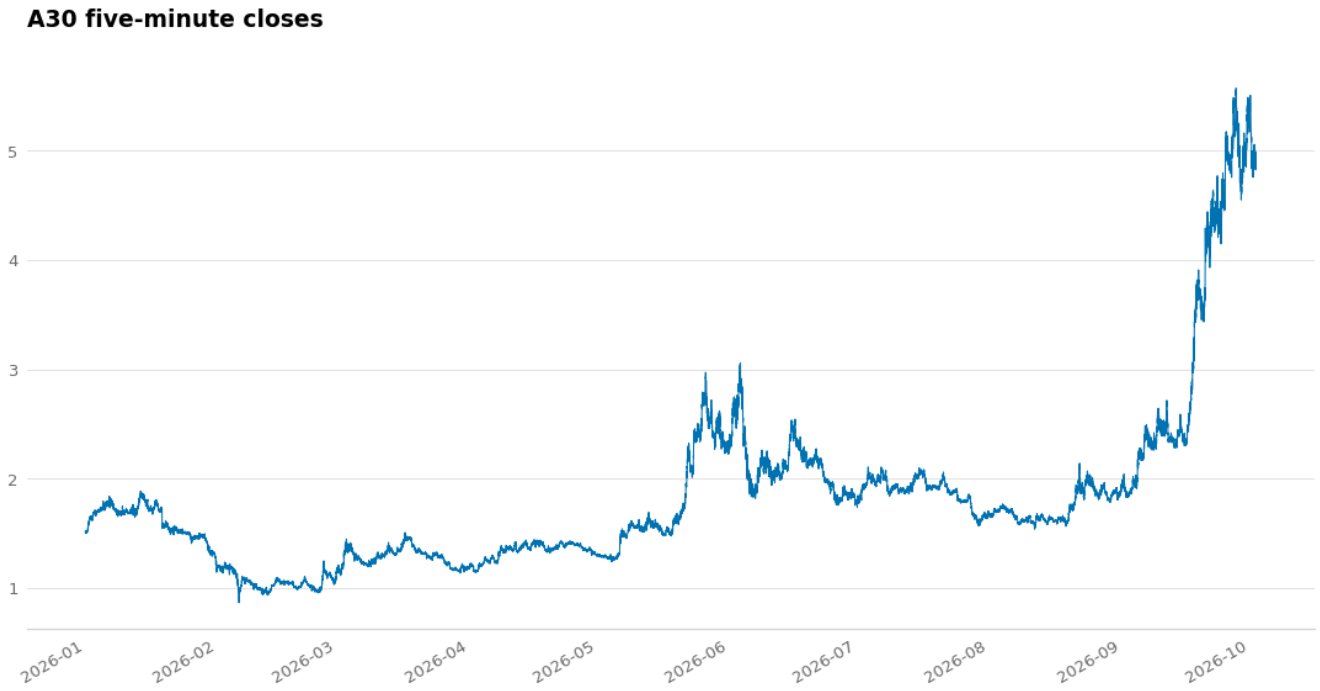
A30



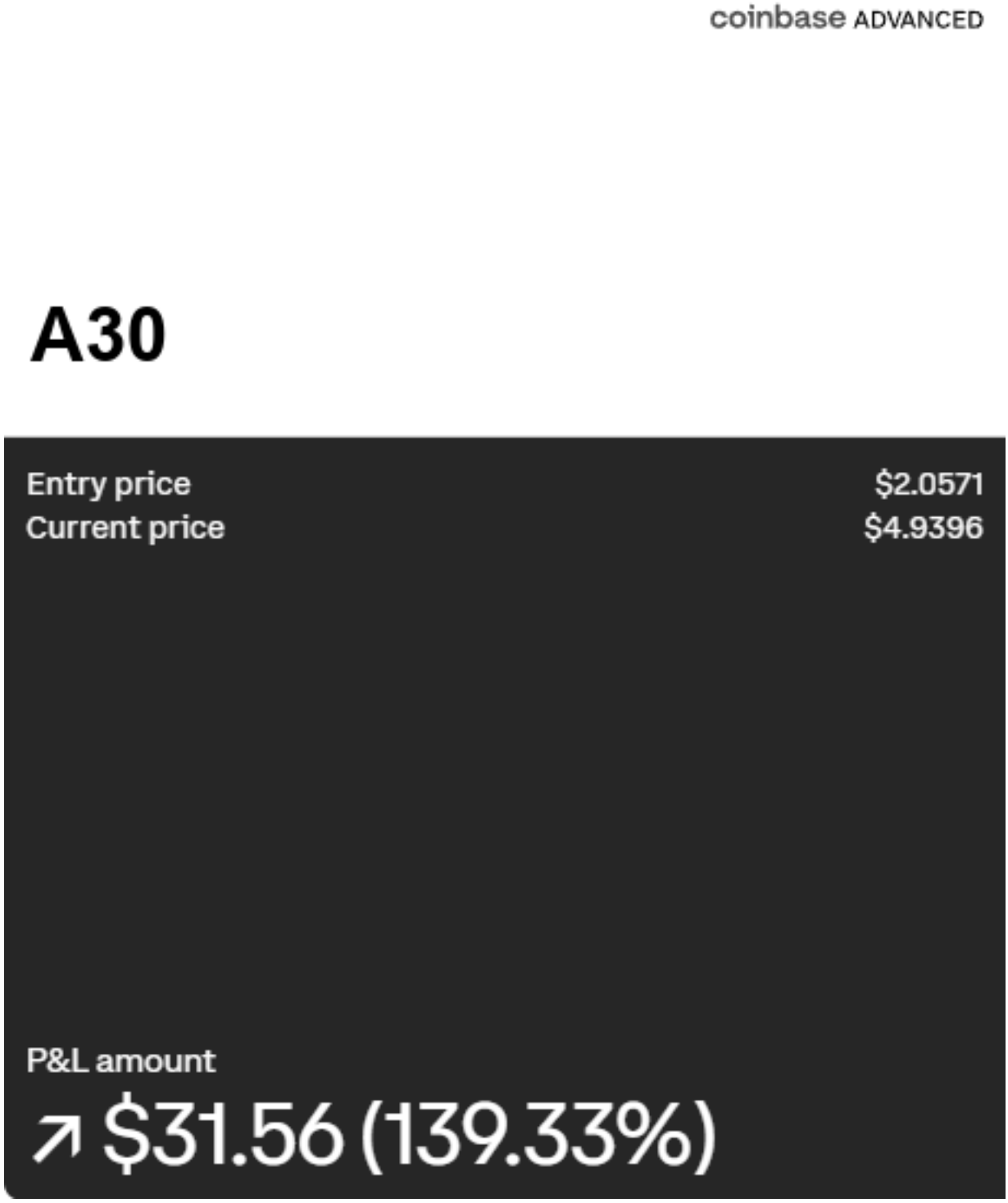
A30, 58 fills, with both lines falling steadily.



A30 weekly chart, 40 weekly candles built from 78,491 five-minute rows.

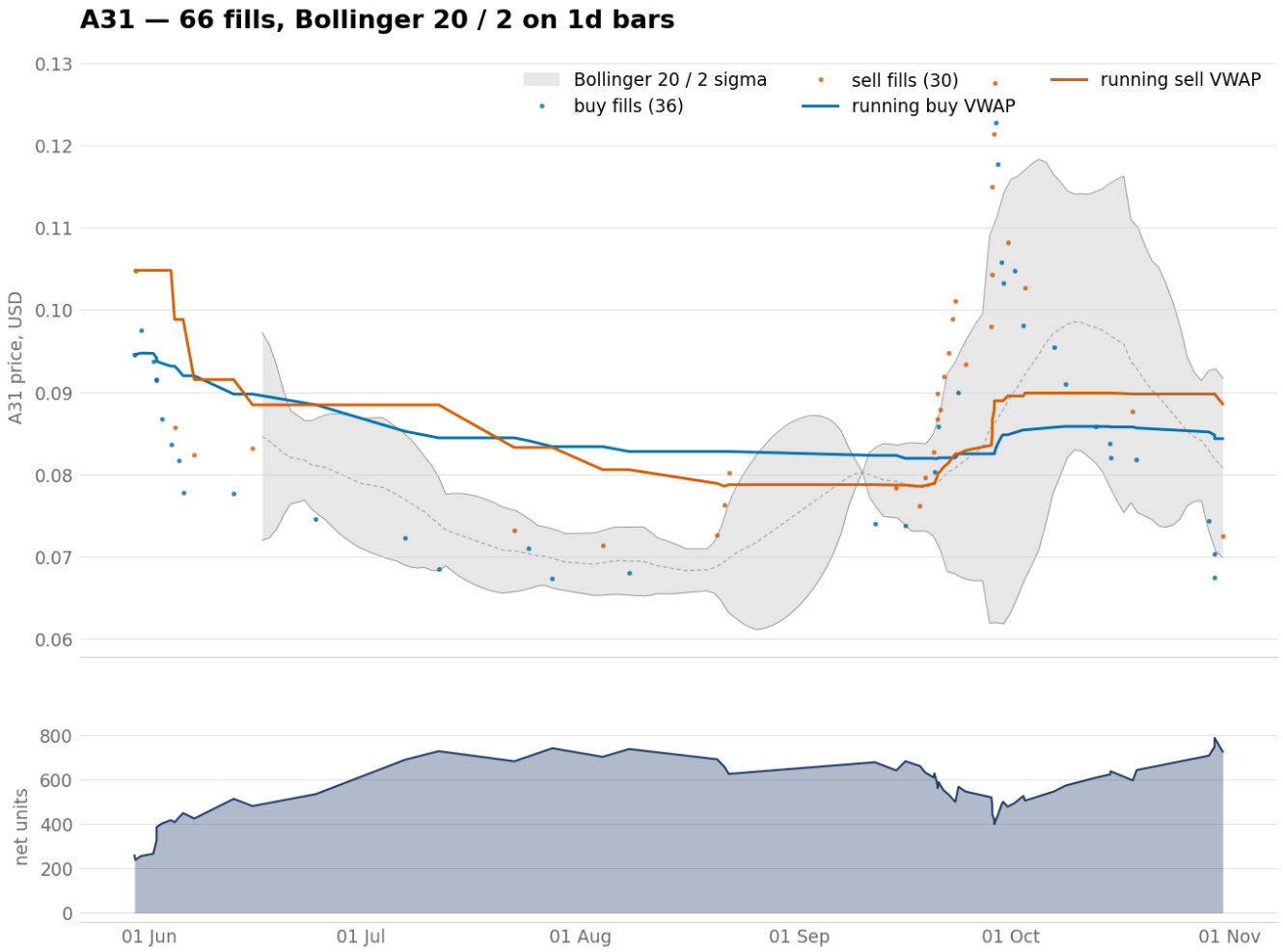


A30 price trace, 78,491 five-minute closes across the recorded window.



A30 position card, with the mark, the name and the code square painted out.

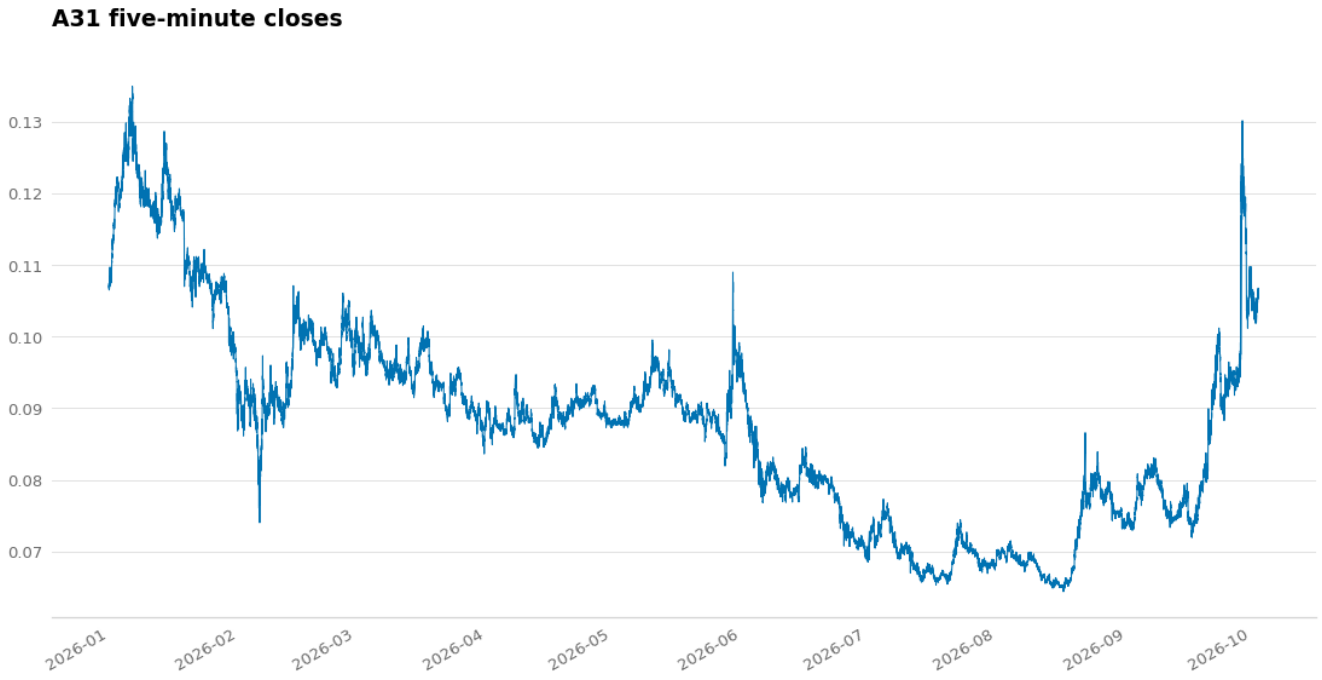
A31



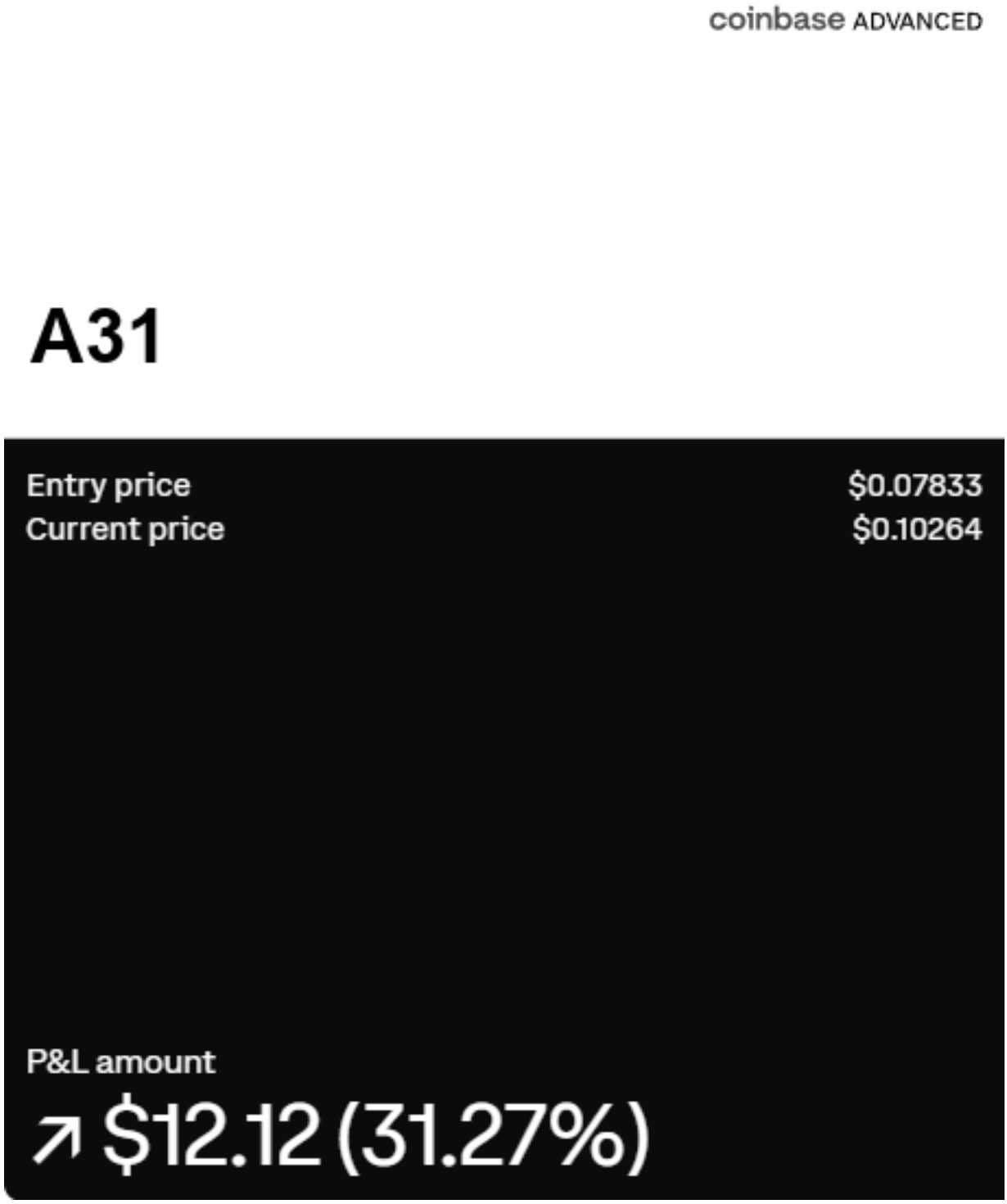
A31, 25 fills, thin enough for the two lines to carry a dot per step.



A31 weekly chart, 40 weekly candles built from 78,993 five-minute rows.



A31 price trace, 78,993 five-minute closes across the recorded window.



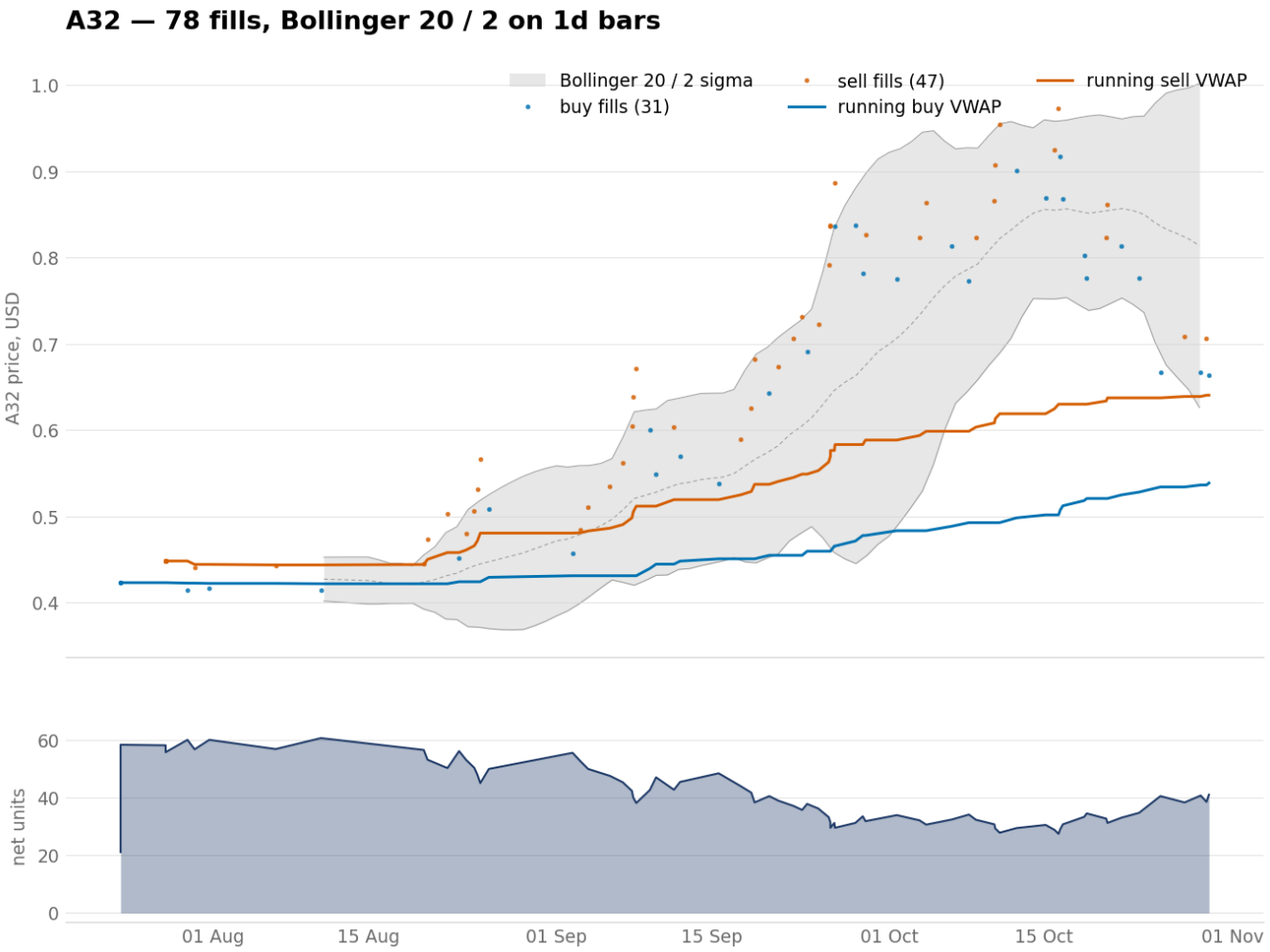
A31 position card, with the mark, the name and the code square painted out.

25 fills is under the module's threshold of 30, so both lines carry a marker at every step. Sixteen dots sit on the buy line and nine on the sell line. The seven charts after this one are drawn the same way.

```
THIN_FILLS = 30

dot = "o" if len(fills) <= THIN_FILLS else None
ax.plot(t, buy_v, color=COLORS["series"][0], lw=2.0, marker=dot,
        markersize=4, label="running buy VWAP")
```

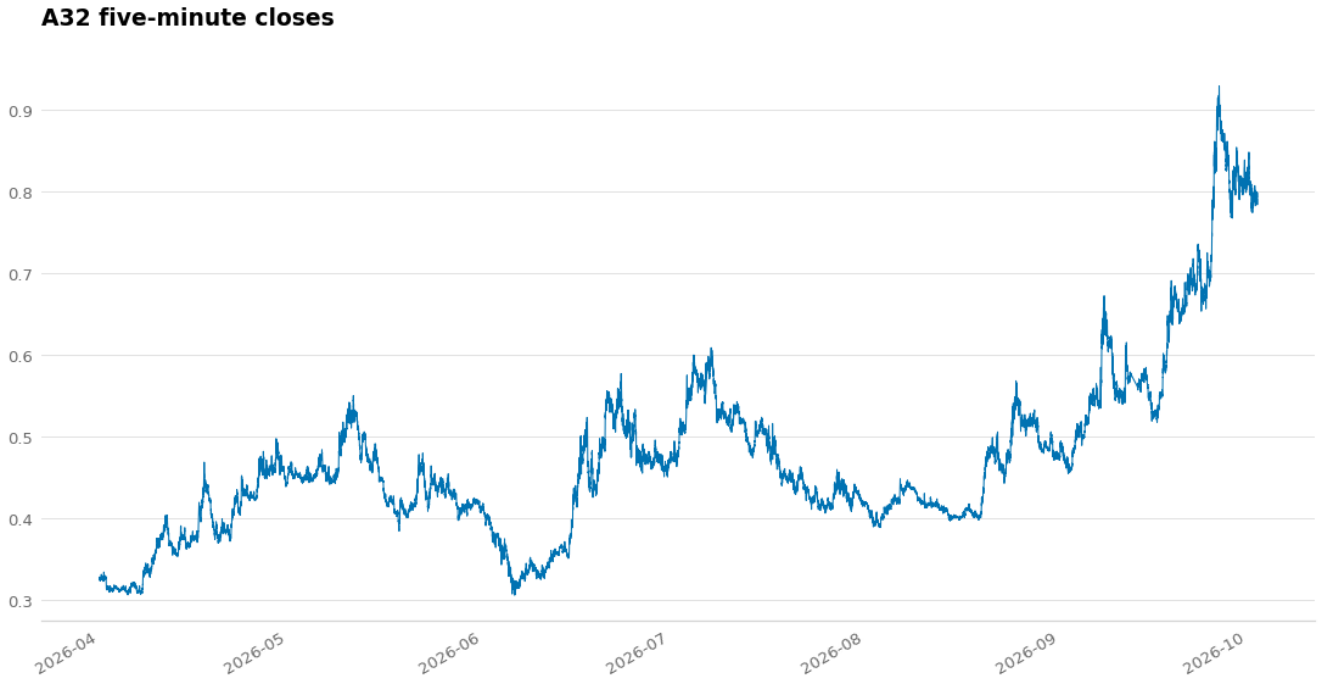
A32



A32, 24 fills, with eight buys and sixteen sells.



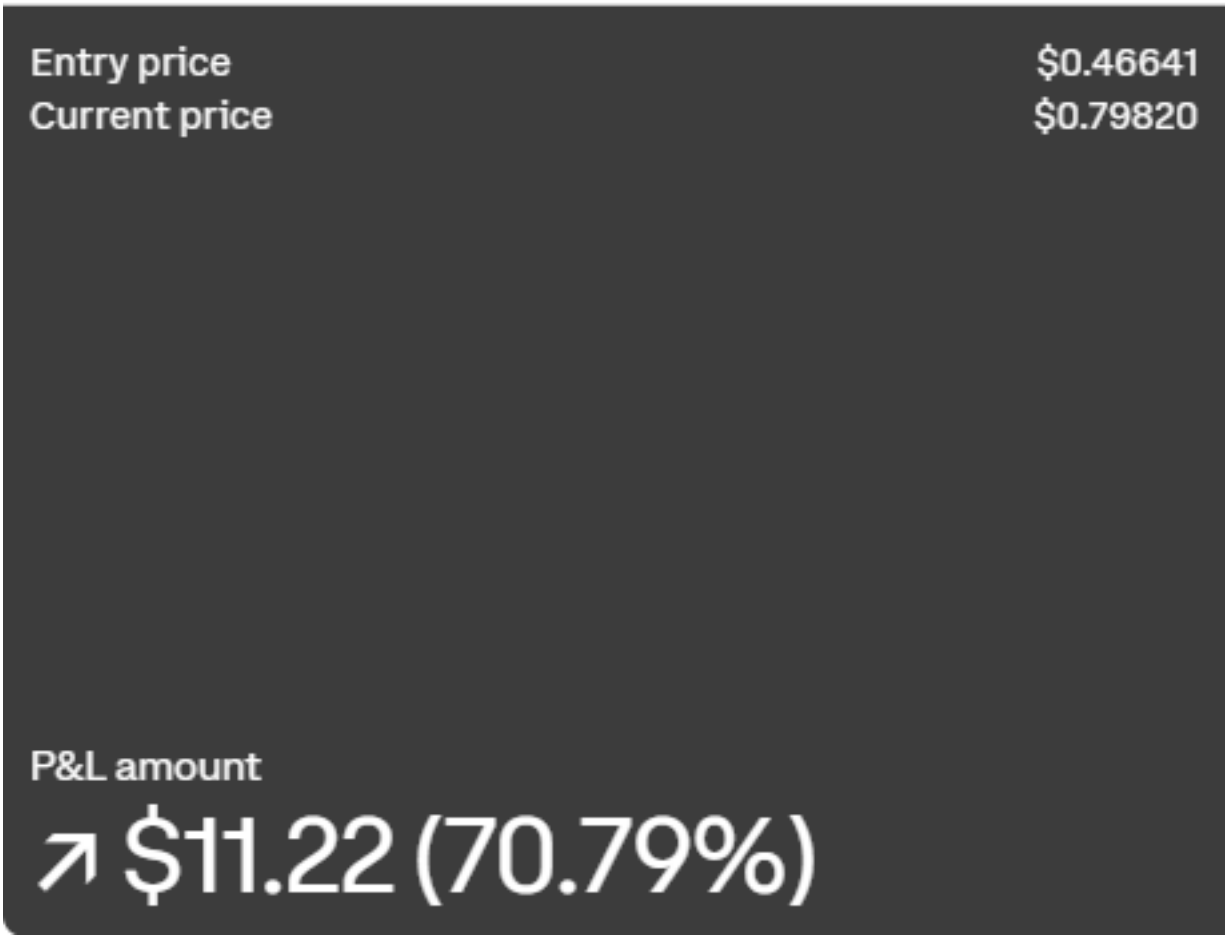
A32 weekly chart, 27 weekly candles built from 52,521 five-minute rows.



A32 price trace, 52,521 five-minute closes across the recorded window.

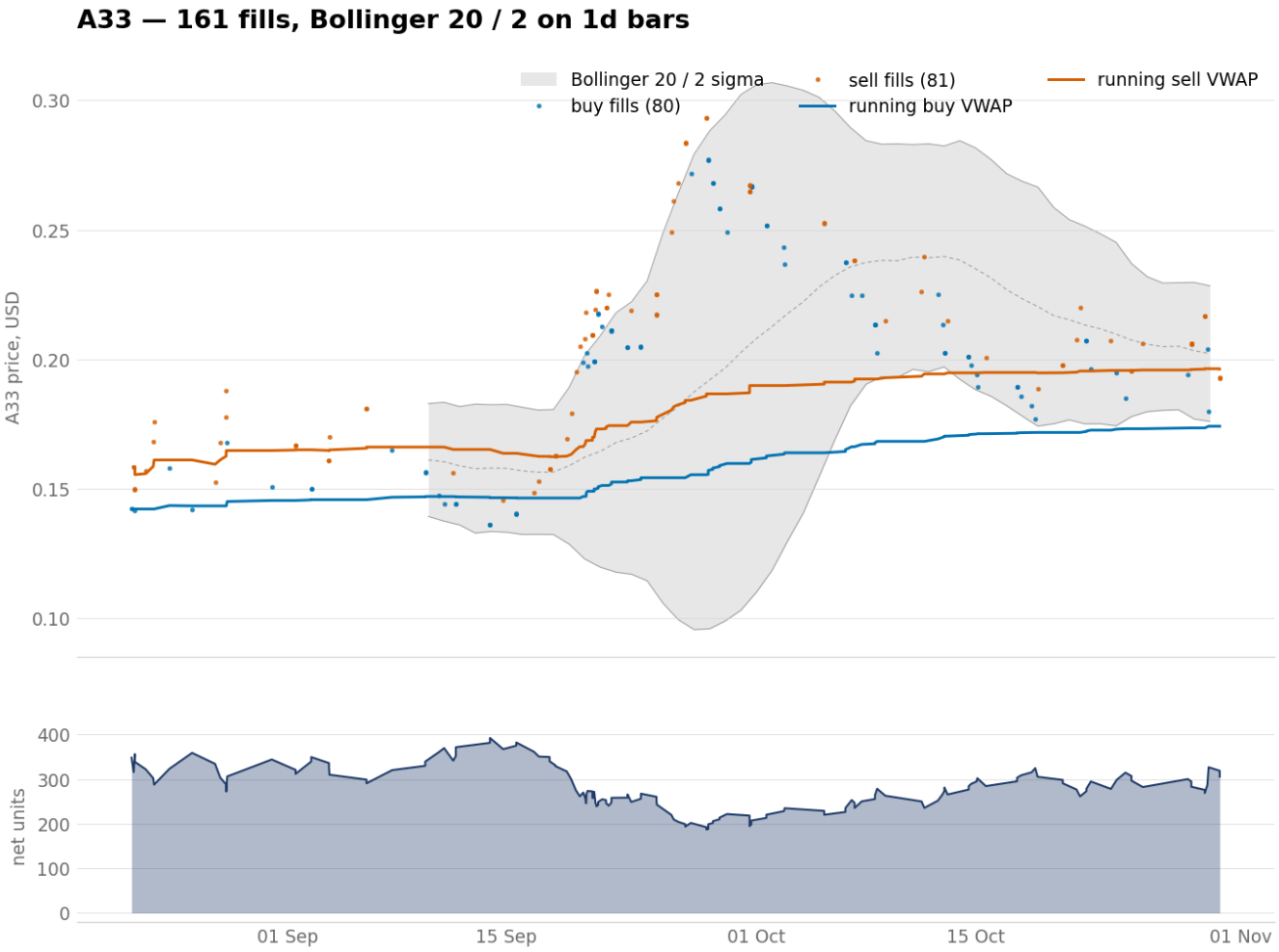
coinbase ADVANCED

A32

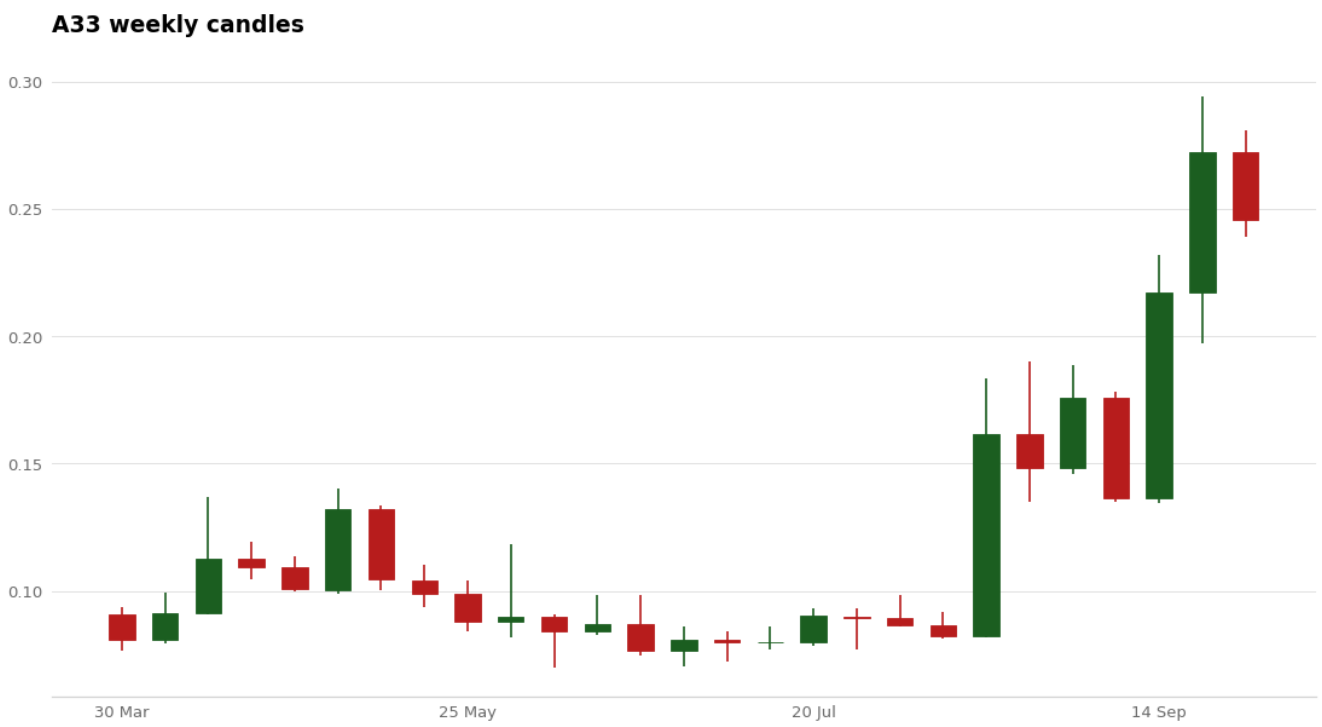


A32 position card, with the mark, the name and the code square painted out.

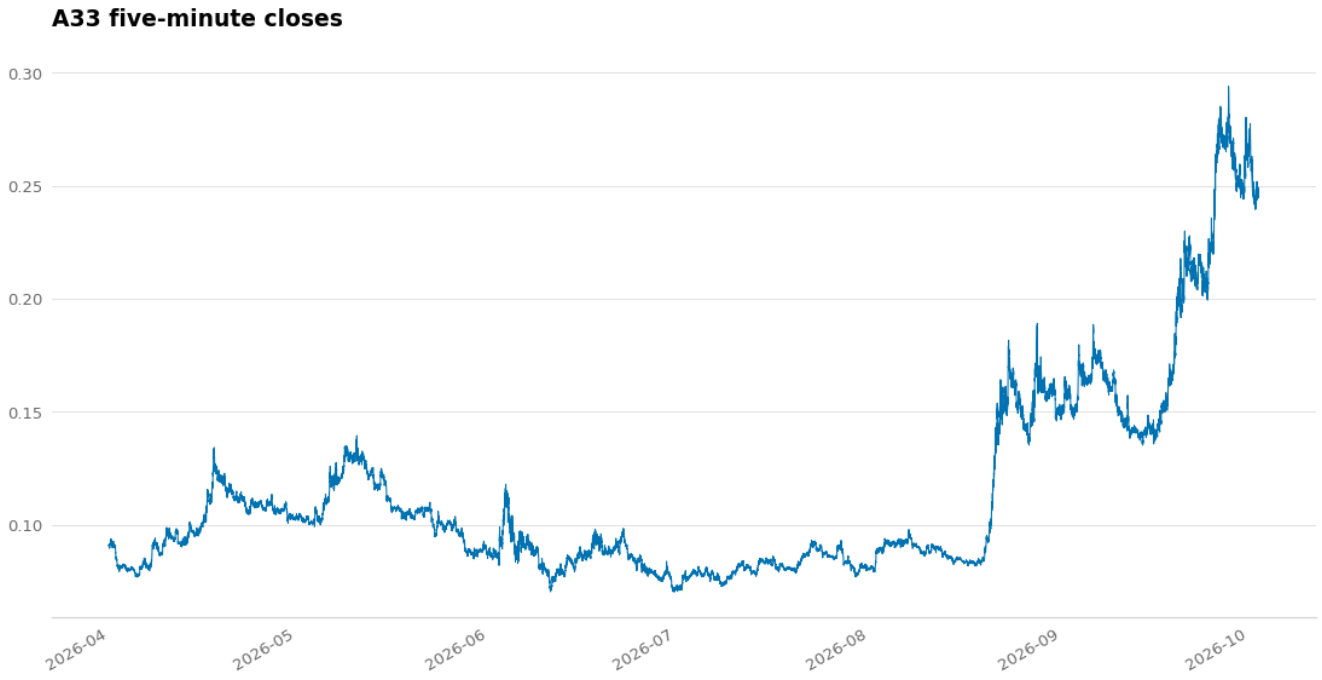
A33



A33, 23 fills across 12 days, the shortest window of the 38.



A33 weekly chart, 27 weekly candles built from 50,503 five-minute rows.

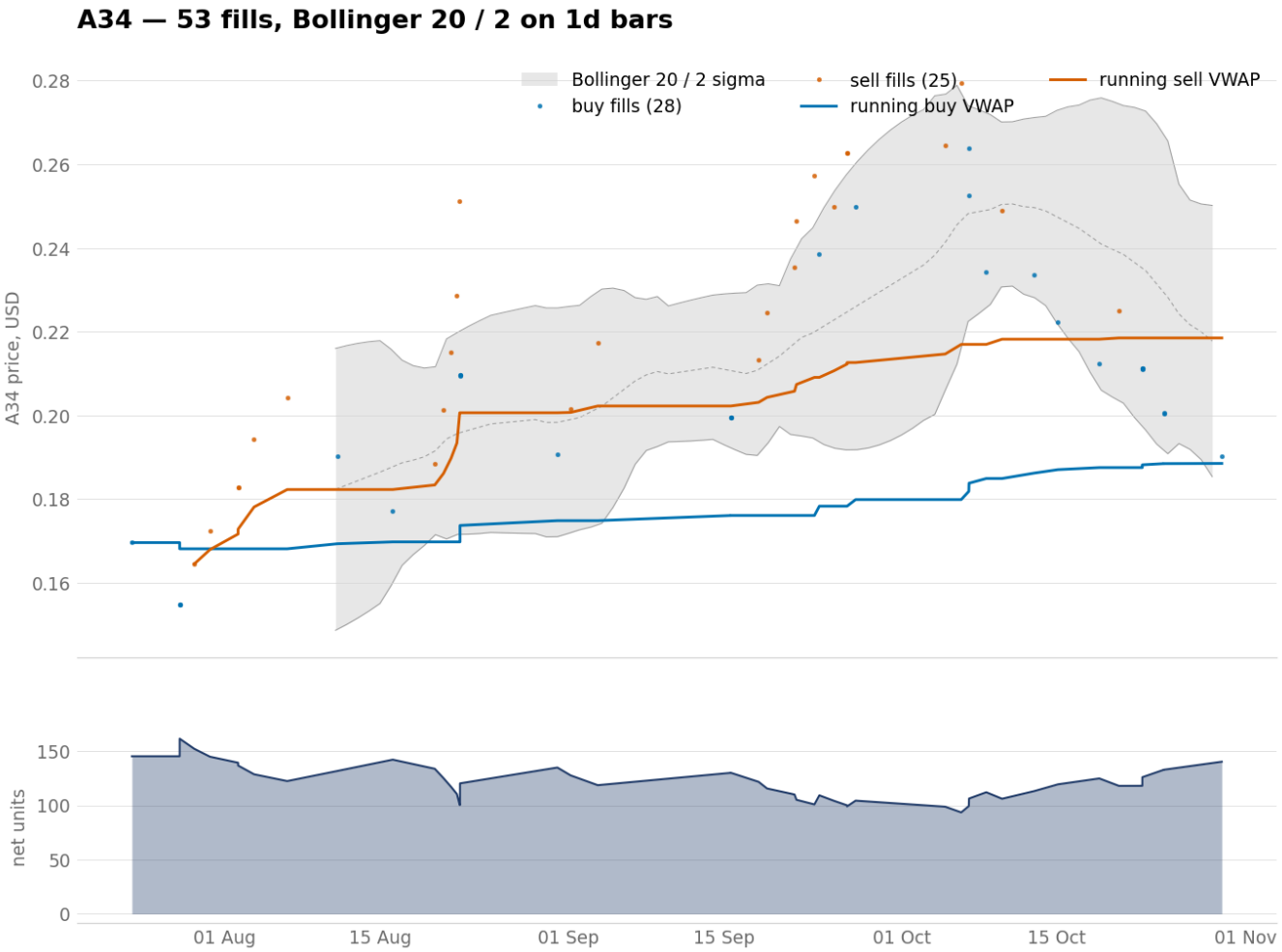


A33 price trace, 50,503 five-minute closes across the recorded window.

A33 has no position card. The venue's card set covers the other 37 bases.

A thin chart, and the shortest window here. Eight buys against fifteen sells. Both lines step up once, on 28 August.

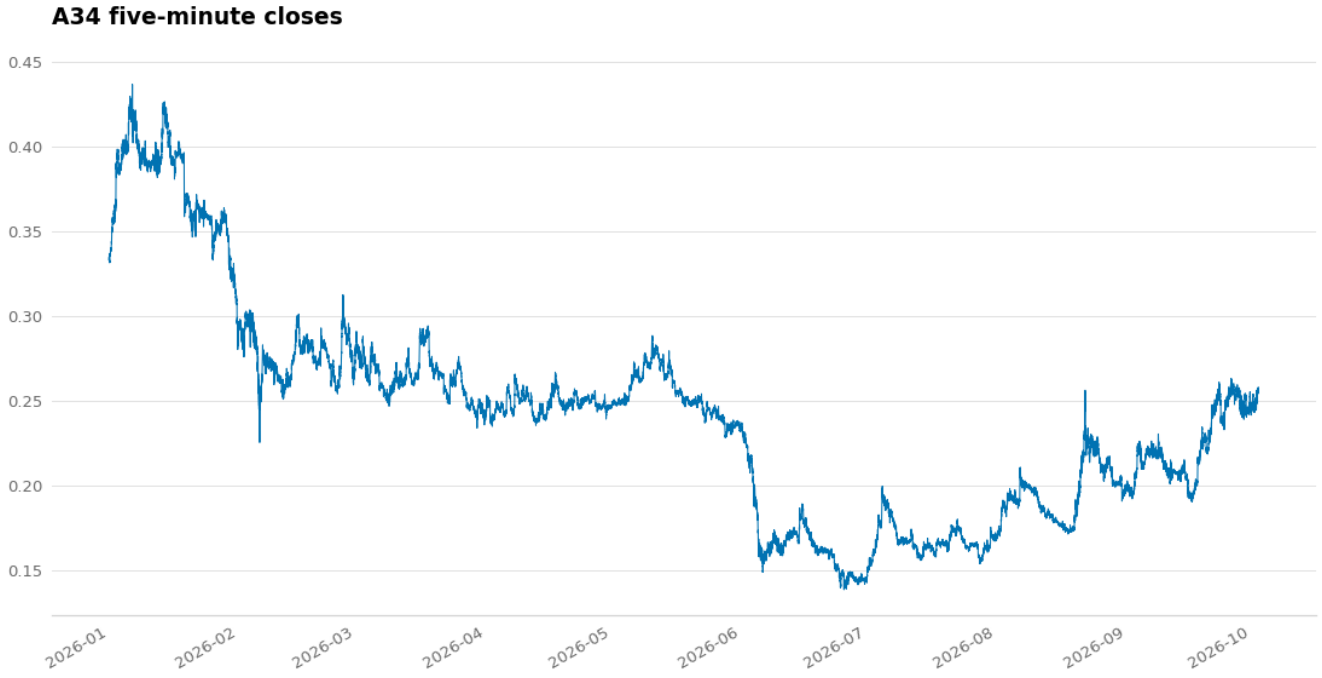
A34



A34, 23 fills, with the sell line stepping up sharply on 22 August.



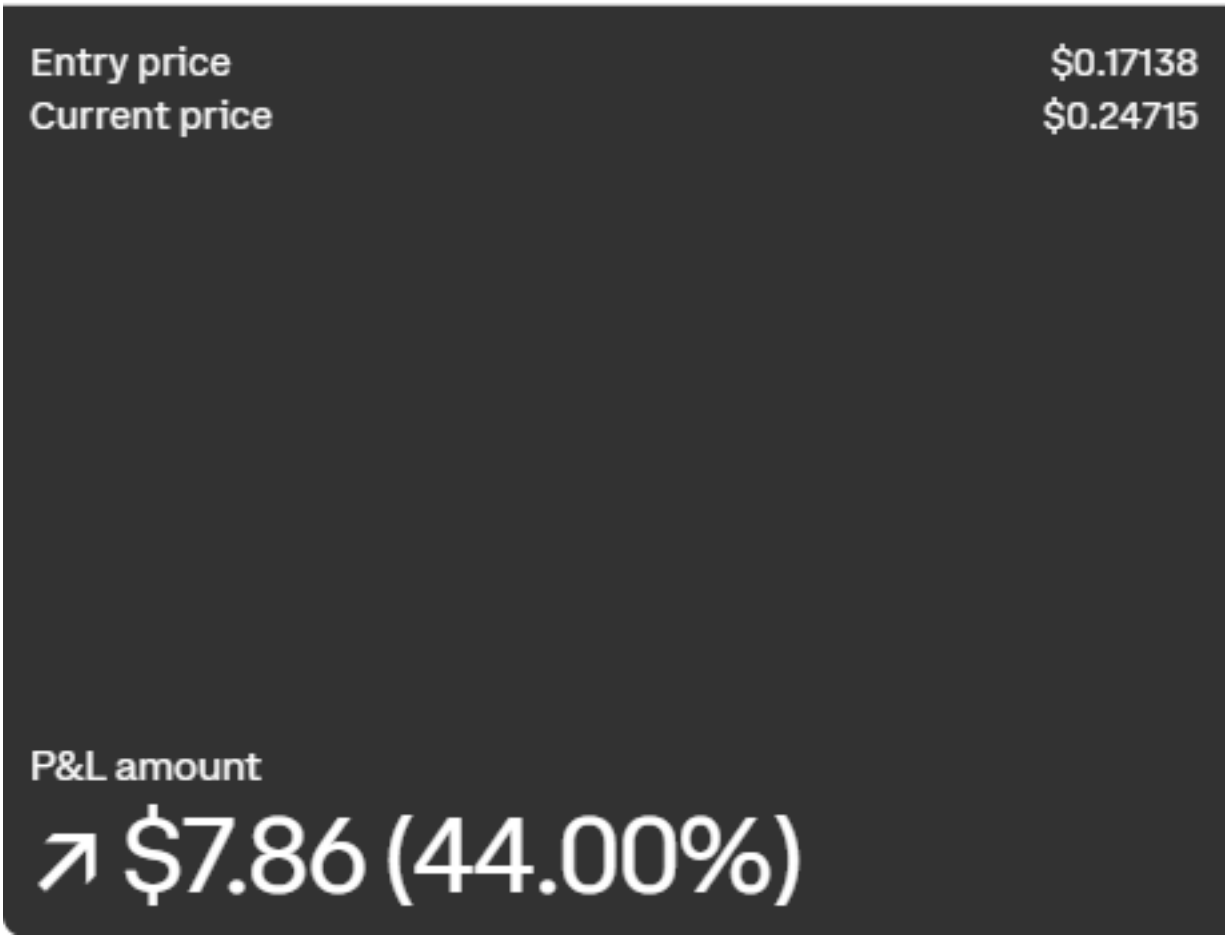
A34 weekly chart, 40 weekly candles built from 78,986 five-minute rows.



A34 price trace, 78,986 five-minute closes across the recorded window.

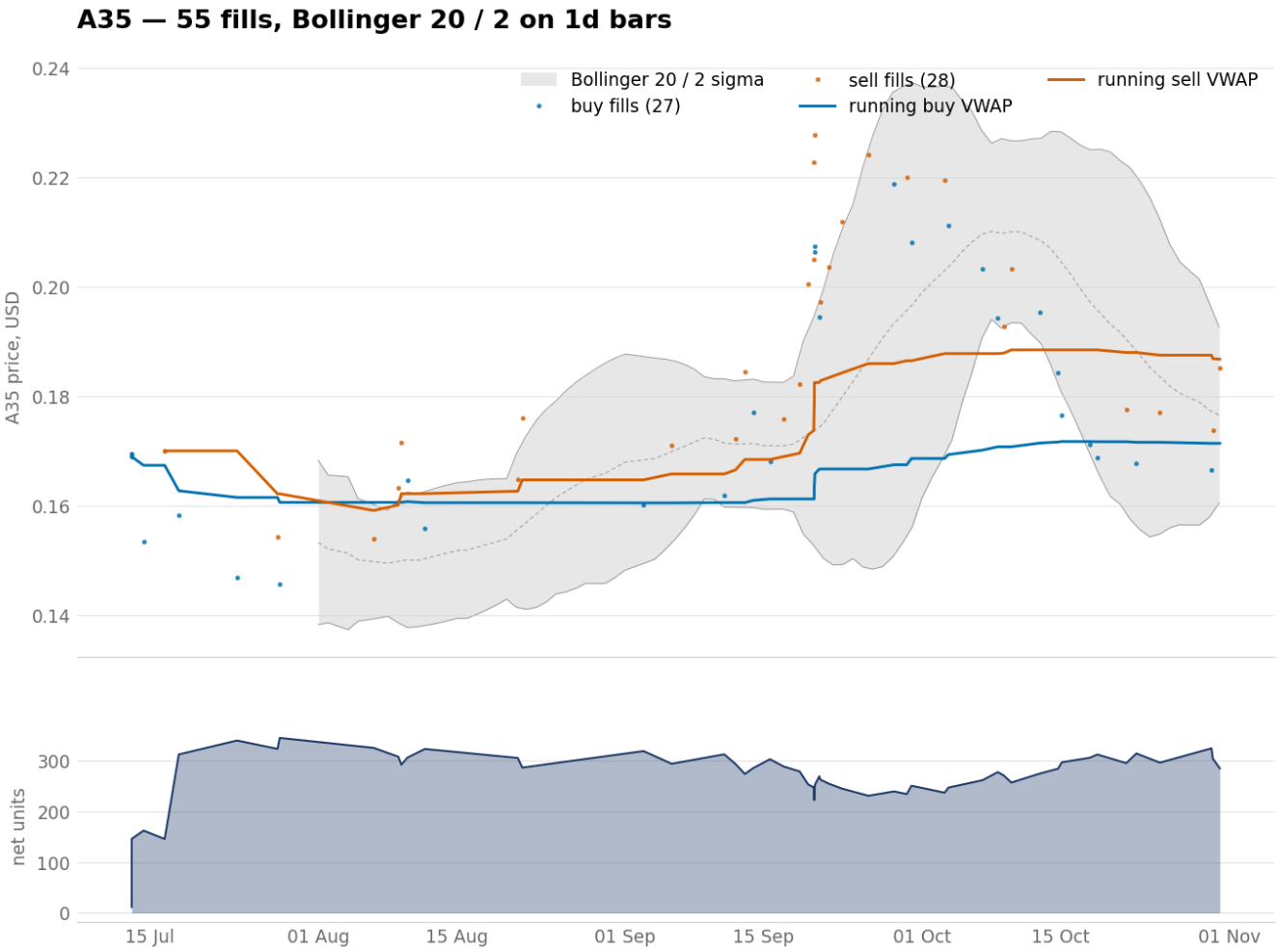
coinbase ADVANCED

A34



A34 position card, with the mark, the name and the code square painted out.

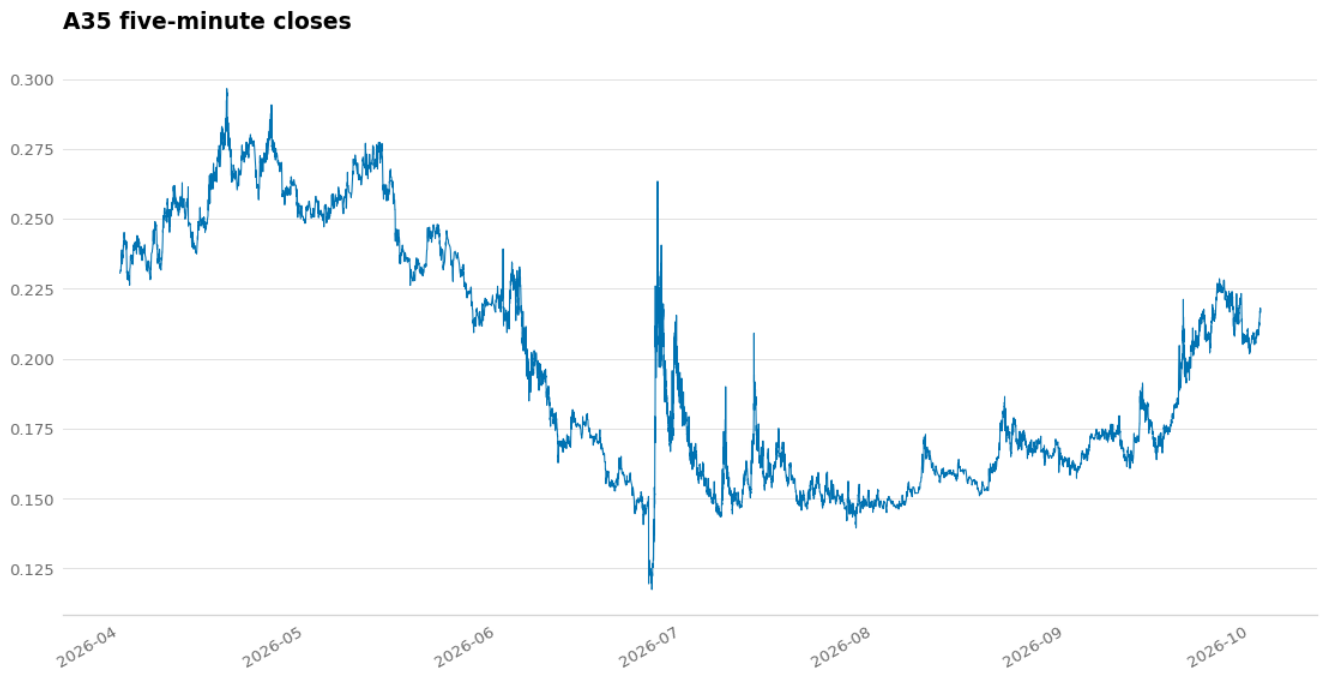
A35



A35, 16 fills, with the buy line flat from late July.

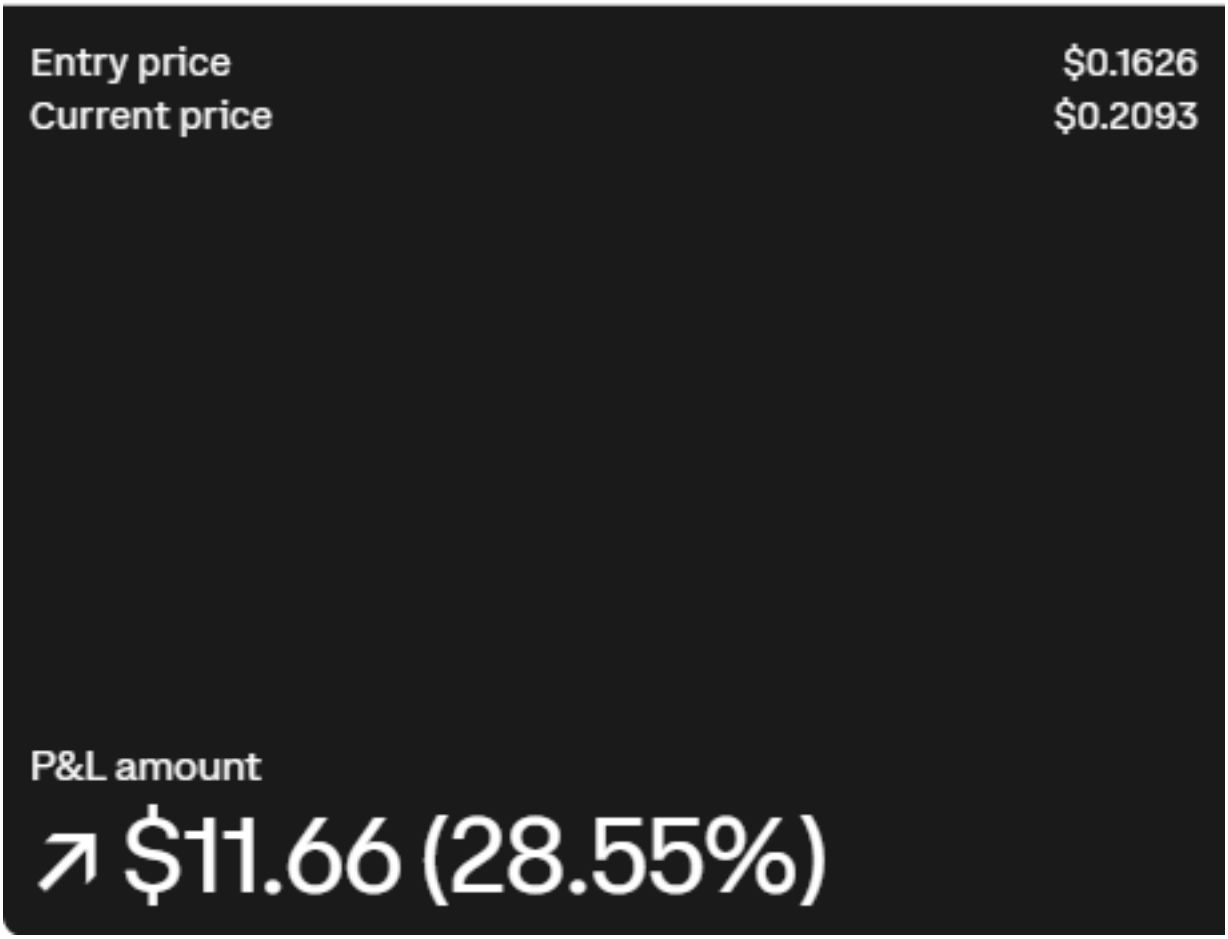


A35 weekly chart, 27 weekly candles built from 10,940 five-minute rows.



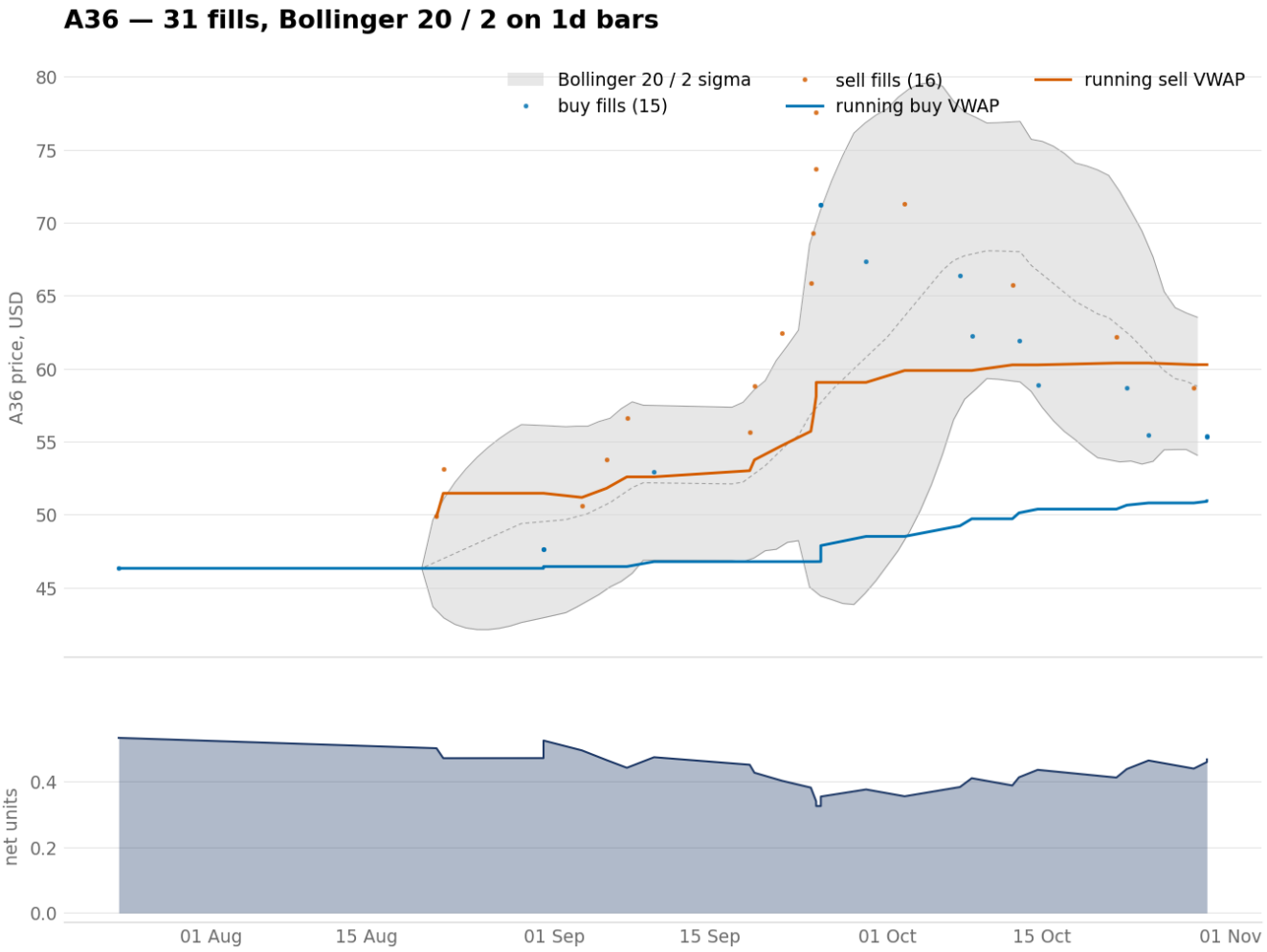
A35 price trace, 10,940 five-minute closes across the recorded window.

A35



A35 position card, with the mark, the name and the code square painted out.

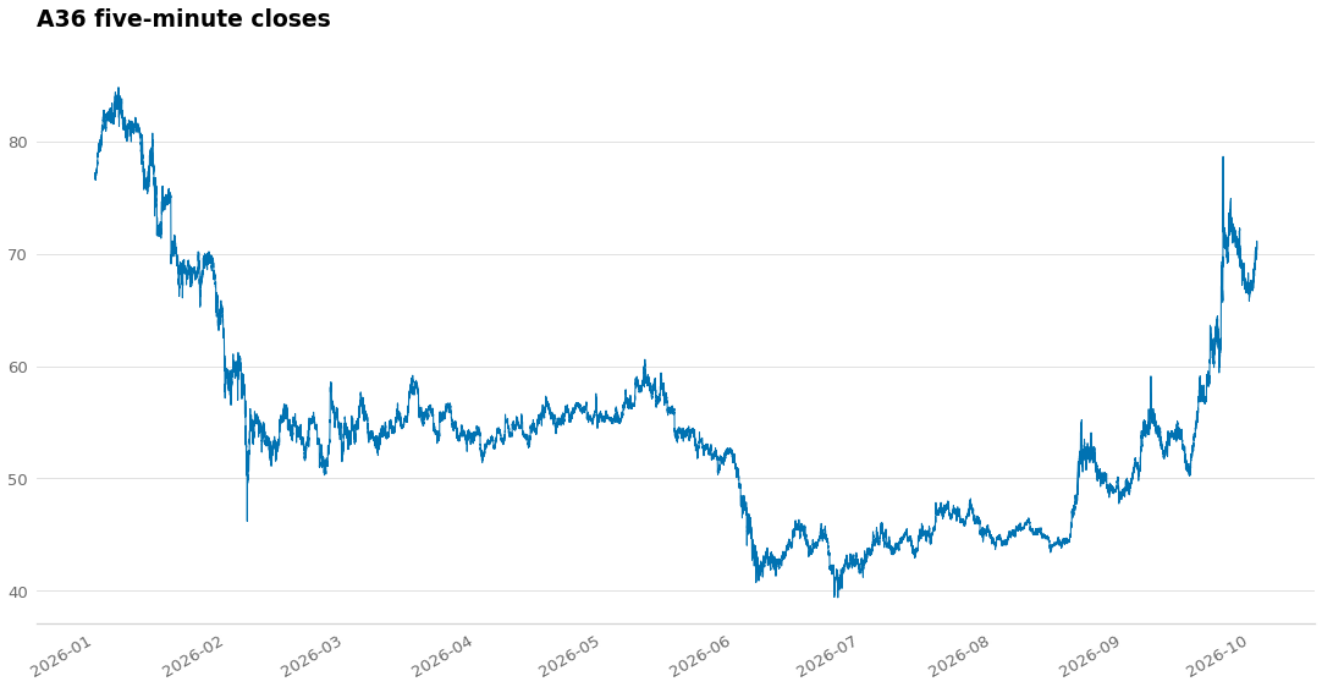
A36



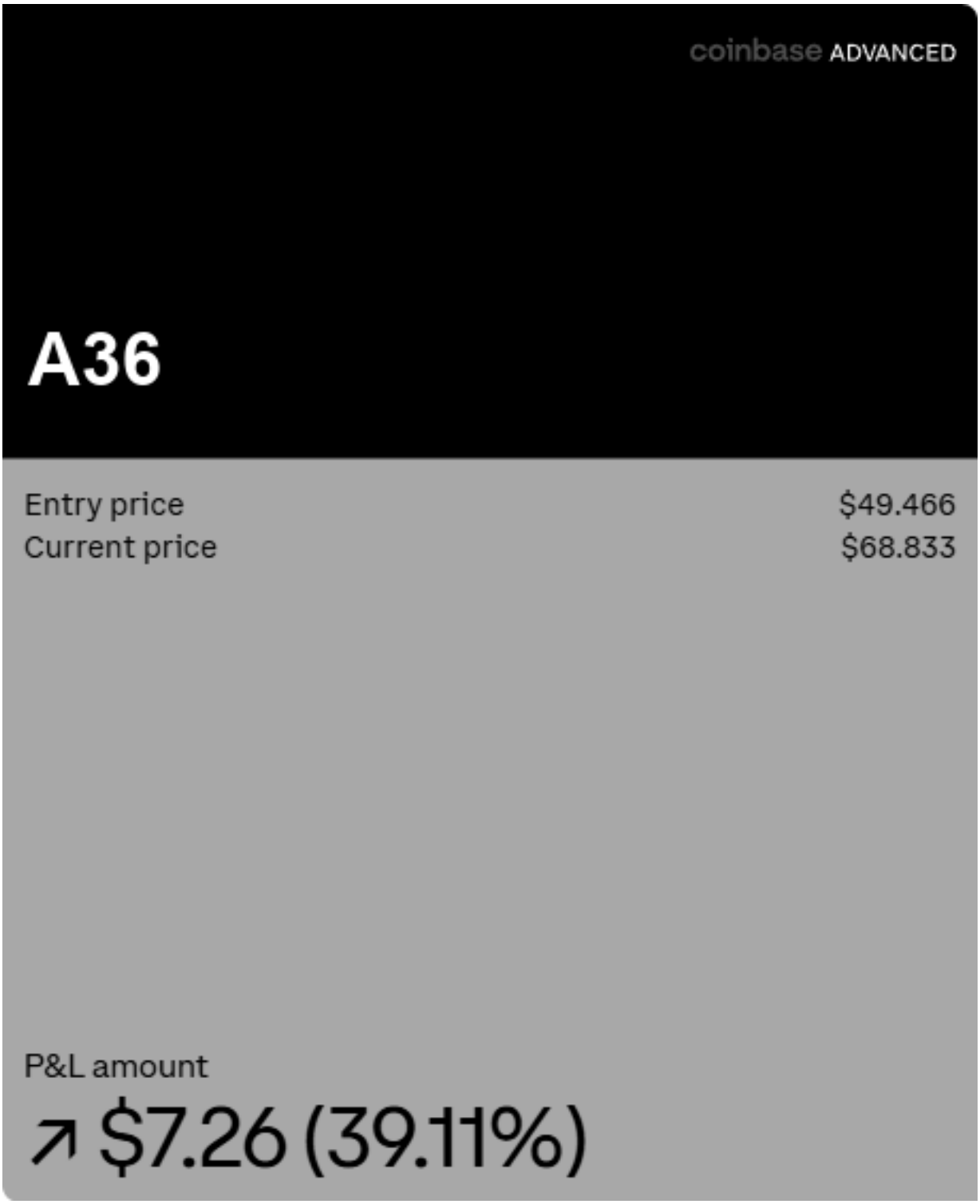
A36, 6 fills, three buys and three sells.



A36 weekly chart, 40 weekly candles built from 79,004 five-minute rows.

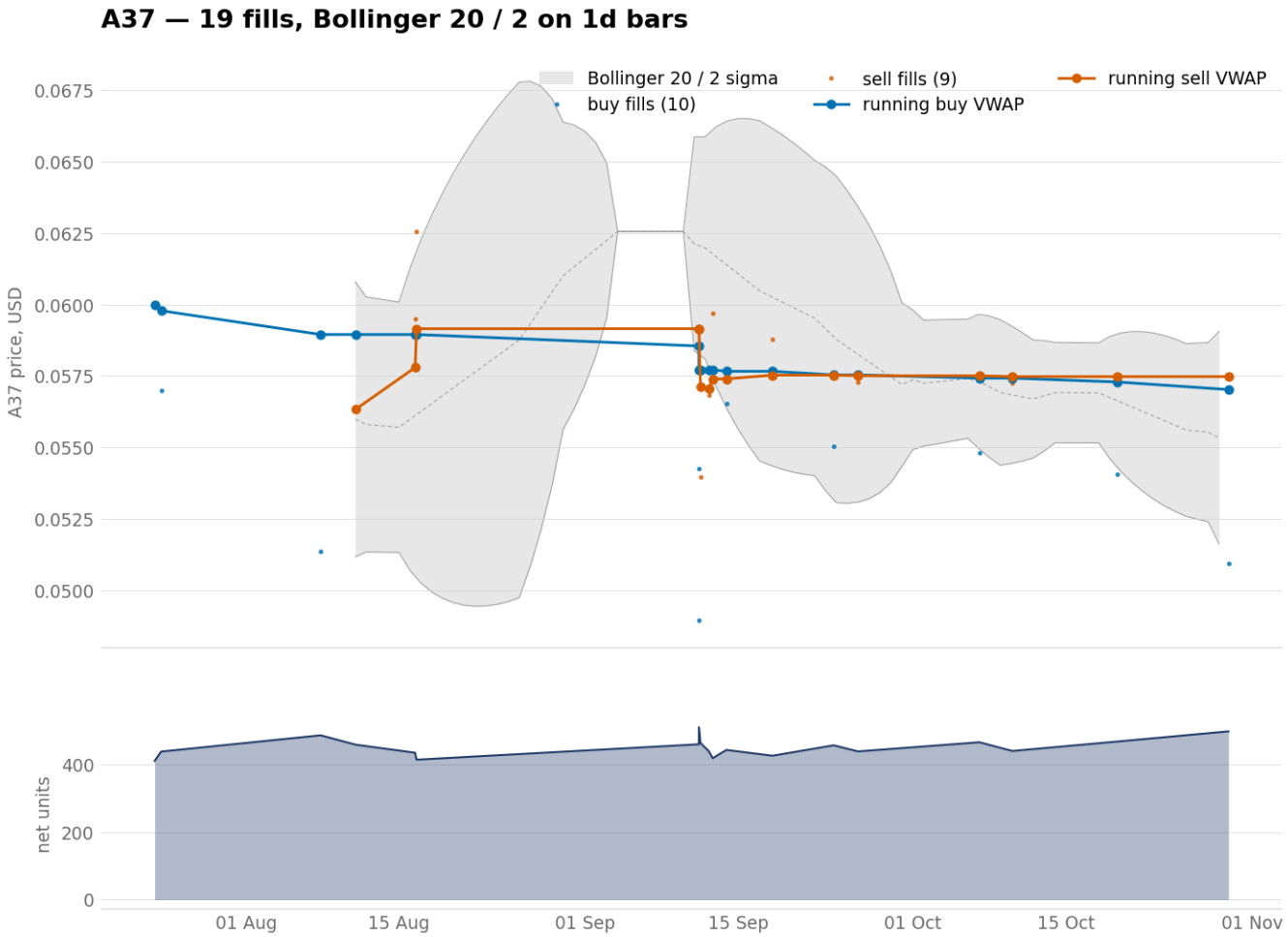


A36 price trace, 79,004 five-minute closes across the recorded window.



A36 position card, with the mark, the name and the code square painted out.

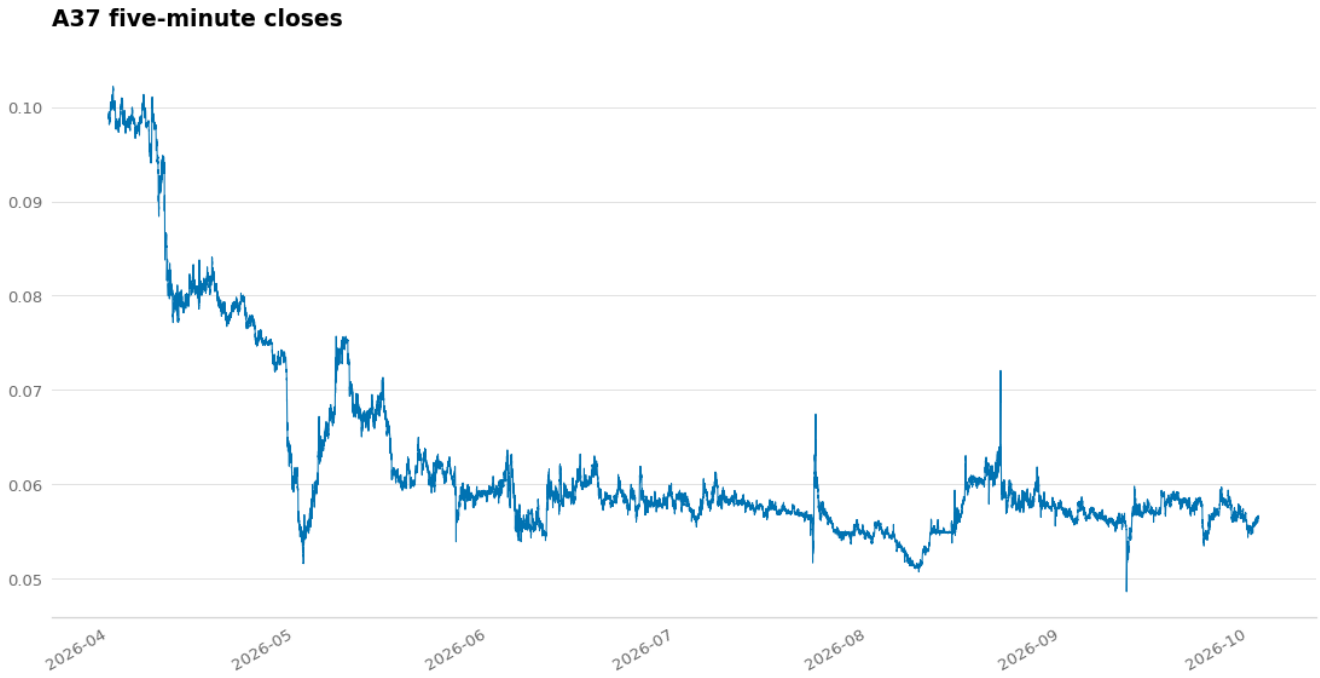
A37



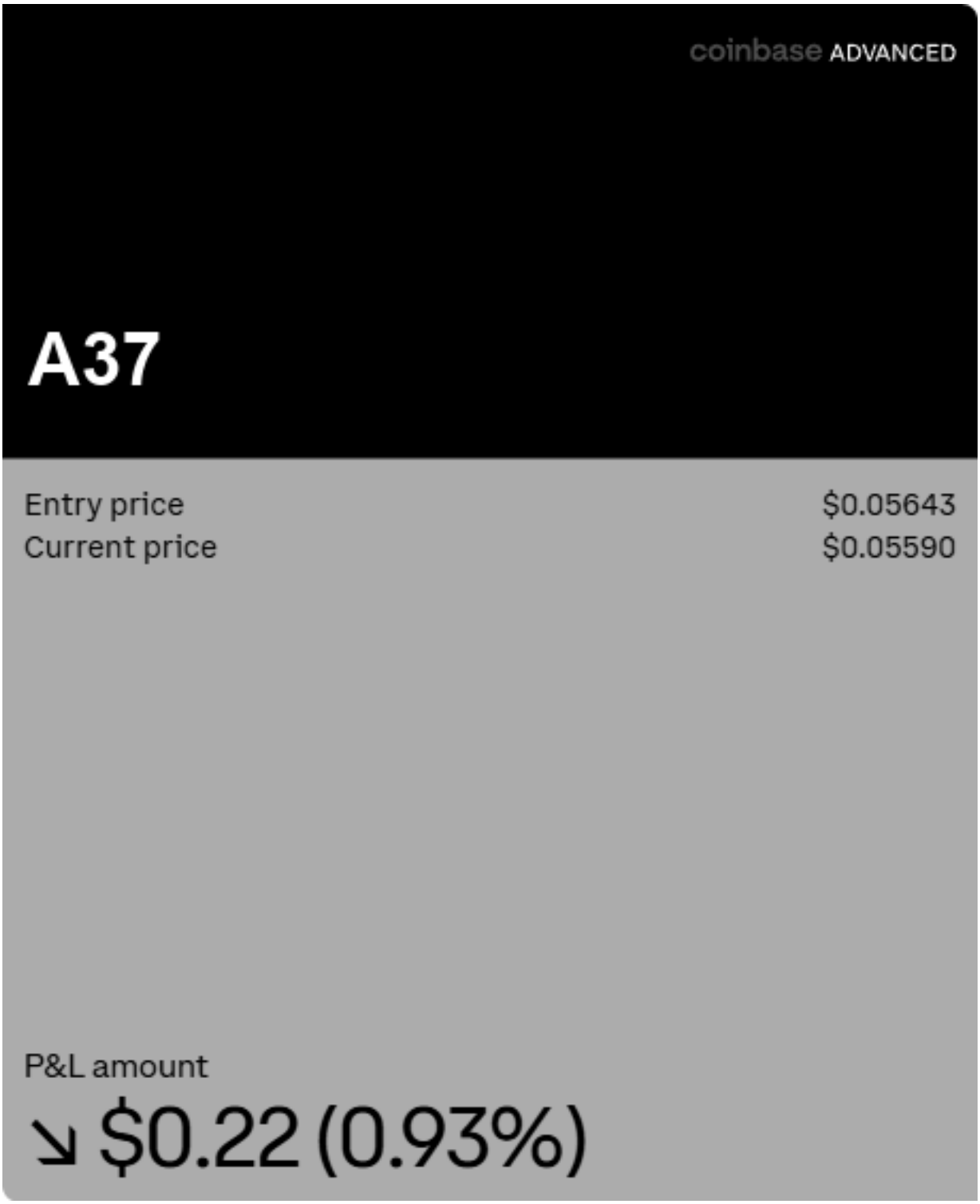
A37, 6 fills, closing 16 August.



A37 weekly chart, 27 weekly candles built from 49,654 five-minute rows.

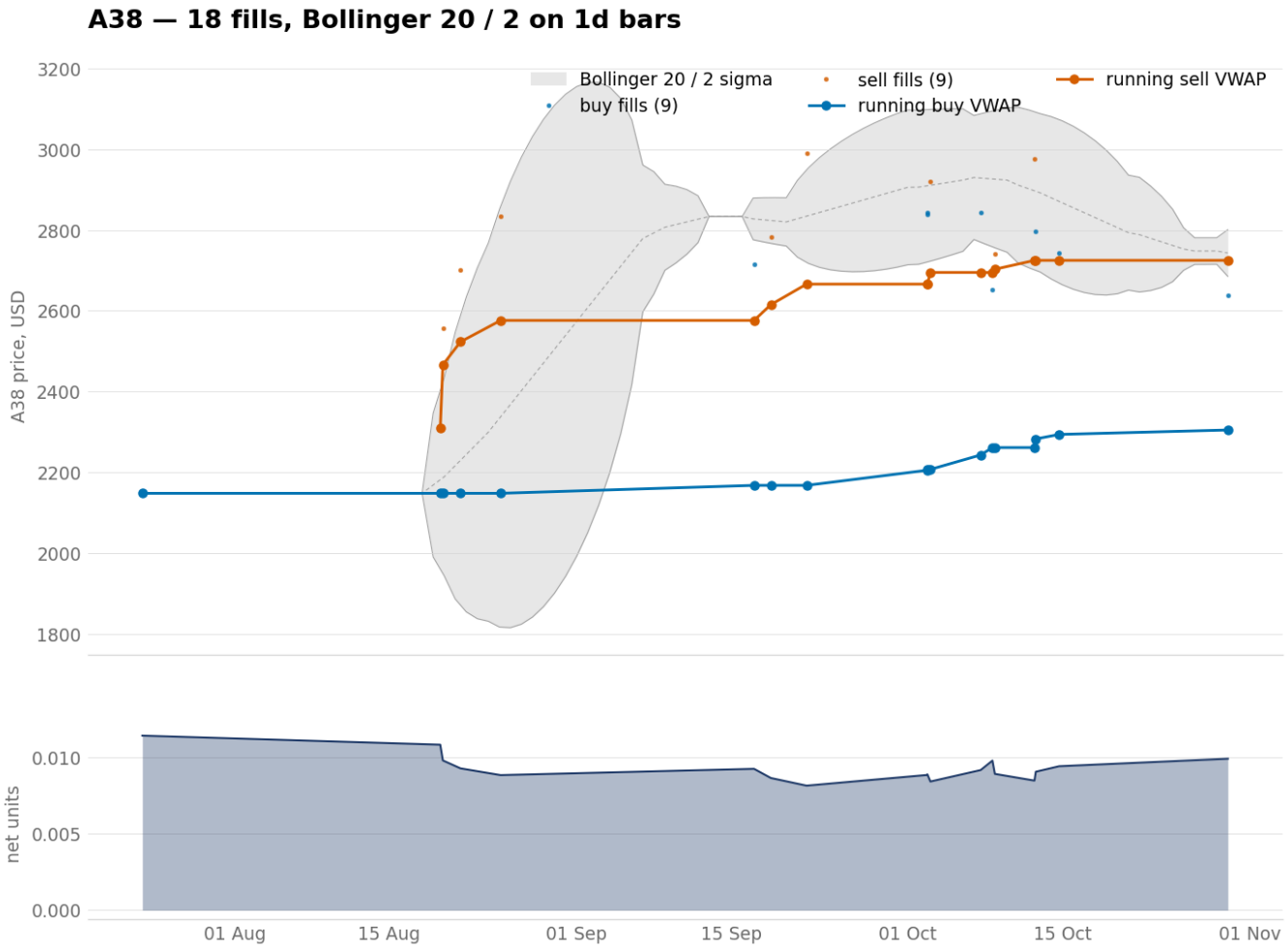


A37 price trace, 49,654 five-minute closes across the recorded window.



A37 position card, with the mark, the name and the code square painted out.

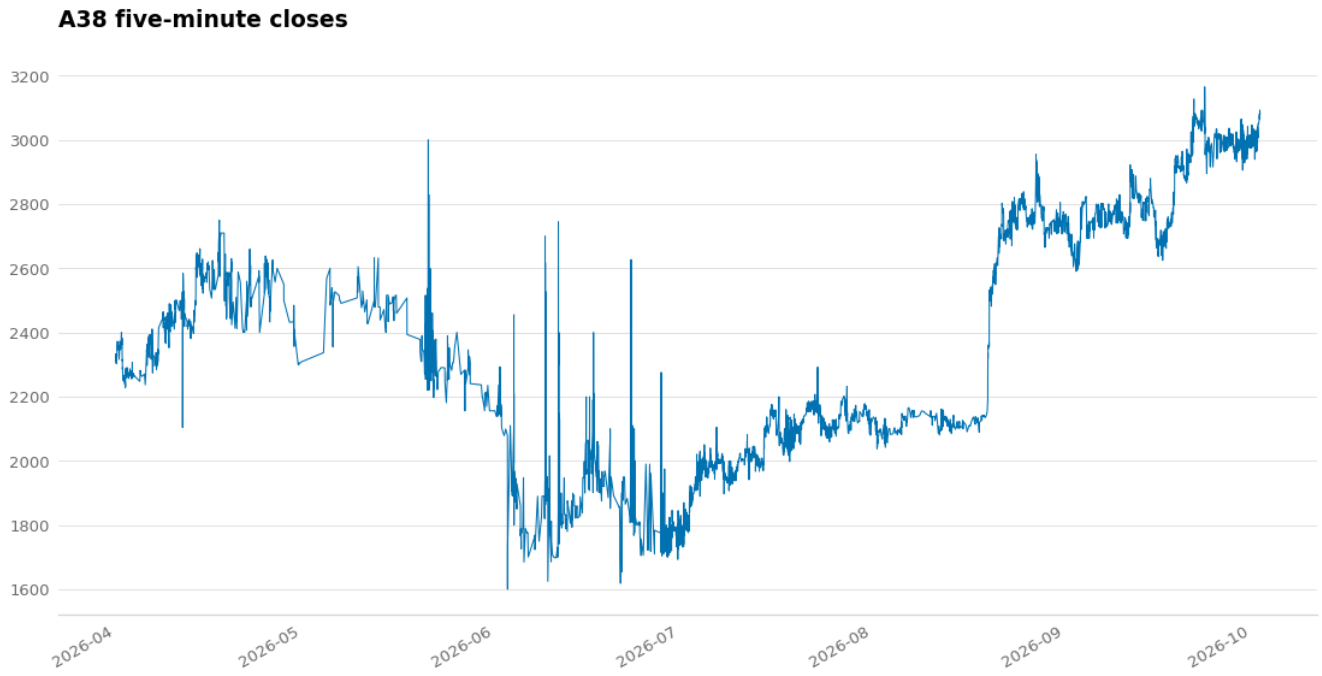
A38



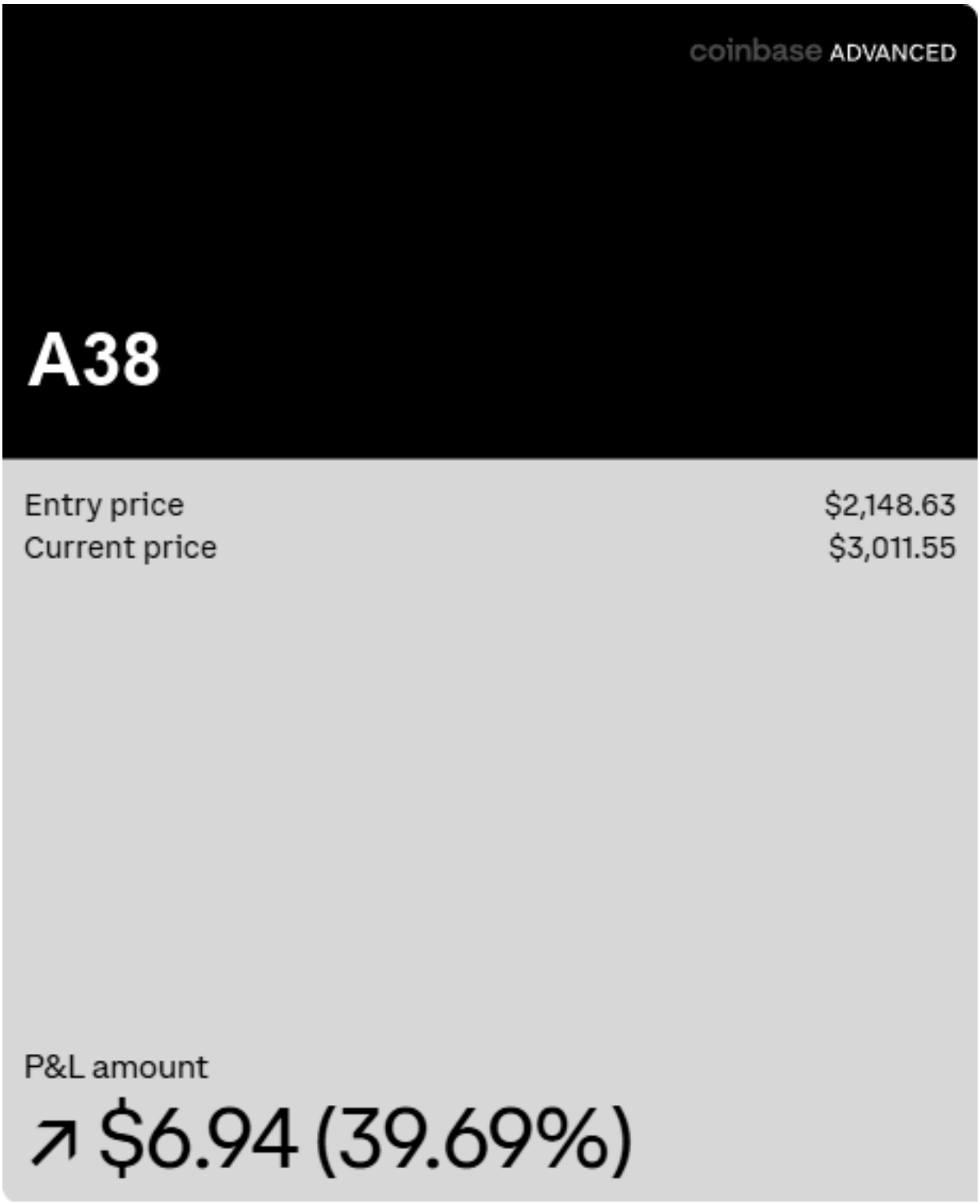
A38, 5 fills, one buy and four sells.



A38 weekly chart, 27 weekly candles built from 6,257 five-minute rows.



A38 price trace, 6,257 five-minute closes across the recorded window.



A38 position card, with the mark, the name and the code square painted out.

Trade grading

One function in `src/trading/trade_grader.py` grades a completed fill, on four axes and no more. Execution scores against a reference price at the decision, timing across the prices after the trade, strategy across the shift in the rolling S/B figure, and outcome against realised profit per unit. An axis whose price inputs are absent is left out, and the overall figure is the unweighted mean of the axes that scored — 0.5 when none of them could.

```
def grade_trade(record: TradeRecord, ctx: PriceContext) -> TradeGrade: ...


def _score_execution(...): ...
def _score_timing(...): ...
def _score_strategic(...): ...
def _score_outcome(...): ...
```

A second function maps that mean to a letter.

```
def _letter_from_numeric(num: float) -> str:
    if num >= 0.93:
        return "A+"
    elif num >= 0.85:
        return "A"
    elif num >= 0.70:
        return "B"
    elif num >= 0.55:
        return "C"
    elif num >= 0.40:
        return "D"
    else:
        return "F"
```

The market regime rides beside the grade rather than inside it. `PriceContext.regime_tag` reaches `TradeGrade.regime` and the rationale text, and it carries no sub-score of its own, which is why a rationale can read five terms while the grade rests on four.

`grade_trades` runs the batch.

The screen these grades reach is in the History tab reference.

Gate logs behind each trade

A fill says what happened. The gate log says which decision produced it.

`gate_light_row` in `src/trading/gate_vocabulary.py` returns nineteen lights for one decision, the scrum bank first and the fold bank second, each entry carrying its bank, label, state and colour. The History table and the Simulator draw their lights from that one function, so both surfaces name the same gates in the same order. The labels themselves, and the indicator readings behind them, are in the indicator reference.

`lookup_gate_entry` in `src/exchange/history_helpers.py` pairs each fill with the nearest gate entry for the same bot. The tolerance is 60 seconds either side of the fill, and a fill that finds an entry inside that

window carries its gate. A fill that finds none carries no gate.

```
JOIN_TOLERANCE_SECONDS = 60.0


def gate_cell_text(entry: Optional[dict]) -> str:
    if not entry:
        return "no record"
```

A fill with no entry reads "no record" in the Gates column, and its tooltip says no gate record joined to the trade. Nothing measures coverage over a set of fills, and nothing names a reason a fill has no gate.

One gate log per exchange and per sector

A decision is written to the venue it was taken on and the sector of the market it was taken in. The path is `trade/gate/<exchange>/<sector>/gate.log` under the log root, and the row carries its own sector in `data.asset_class`, so a row names the file it is in.

```
~/.acervator_logs/trade/gate/coinbase/crypto/gate.log
~/.acervator_logs/trade/gate/robinhood/crypto/gate.log
~/.acervator_logs/trade/gate/robinhood/stocks/gate.log
```

`src/core/log_paths.py, in gate_log_path` composes it, and `path_segment` folds each name to one lowercase directory, so a venue id cannot reach a parent directory through the path. `src/core/logging_engine.py, in LogManager._gate_writer_for` holds one writer per pair and opens it on the first decision, so every bot on one venue trading one sector appends to a single file.

The sector reaches the writer from the bot. `src/trading/scrumming/snapshots.py, in _emit_gate_decision_at_fire` reads `BotContainer._asset_class` for the traded symbol, which answers the sector the venue's market recording holds, then the sector the bot's own config declares, then crypto. The Simulator answers the same question through `src/simulator/sim_bus.py, in sim_asset_class`, so a sim decision is filed where its live twin is filed.

Each pair's active file rotates at 50 MB and keeps five backups, so one pair bounds at 300 MB and each further pair adds 300 MB of its own.

Reading every decision, wherever it was written

`src/core/log_paths.py, in gate_log_files` lists every gate log once: the pre-split file while it is still there, then each pair's archive members, rotations and active file. Every reader goes through it.

reader	what it draws
`src/trading/live_log_reader.py, in live_gate_decisions`	the entries the History tab and the parity tools join against
`src/exchange/history_helpers.py, in build_page_gate_index`	the per-page bot-and-minute index
`src/gui/history_tab.py, in read_join_indexes`	the same index, read off the GUI thread
`src/simulator/gate_log_source.py, in GateLogSource.files`	the Simulator's read-only `GateRow` records
`src/simulator/sim_bus.py, in sim_log_paths`	the paths the parity report prints

reader	what it draws
`tools/capture_live_baseline.py, in gate_members`	the baseline snapshot's latch distribution

Now: the History tab reads that index in steps of about 8 ms on the GUI thread, not off it (`src/gui/history_tab.py, in read_join_indexes`).

What became of the records written before the split

`src/core/logging_engine.py, in migrate_legacy_gate_logs` carries them, once, when the first `LogManager` of a build with this layout is built. It runs before any gate writer opens, so no handle is held on a file it moves.

Each pre-split file leaves every reader's view in one rename before any bucket file appears, under a `.consumed-` name the archive member will take. A file whose records all belong to one pair is then renamed again into that pair's archive, with no byte copied. A file holding more than one pair is streamed into a `.pending-` file per pair, and each is promoted in turn. A carry cut short leaves the `.consumed-` name in place and the next `LogManager` finishes it, so a decision is never recorded twice.

An archive member never rotates and nothing prunes it. The records are decisions behind real fills, not a running stream.

Measured over a full-size copy of the operator's own gate records, with no write to his tree:

```
in     6 files   167,947 decisions   268,798,267 bytes
out    7 files   167,947 decisions   268,798,267 bytes   1.81 seconds

gate/coinbase/crypto/archive/     6 files, the five rotations and the active file
gate/fleet_sim/crypto/archive/    1 file, 136 decisions an older simulator wrote
```

The byte totals are equal because the carry copies each line as the bytes on disk hold it. The two destinations are the two values the `exchange` field carries across those records: `coinbase` on 167,811 of them and `fleet_sim` on 136. No record carried a sector, so every one is filed under the sector `src/core/logging_engine.py, in default_gate_sector` answers, which is the sector `BotContainer._asset_class` answers for a market the recording labels none.

Two controls were read on the same copy. A comparison of the line sets before and against after reports `lost=1` when one decision is removed from the result and `duplicated=1` when one is repeated, so the equal reading above is a measurement. A carry interrupted after its first pair was promoted reads 3,245 of 3,381 decisions readable, 136 lost and 0 duplicated; the next `LogManager` reads 3,381, 0 lost and 0 duplicated.

Weekly candles behind the fills

The 38 charts above draw fills. This set draws the market those fills ran in. Each panel is one charted base and carries the same label, ten panels to a page.

The bars are built here, not fetched. `weekly_bars` in `tools/build_product_manual.py` groups a base's five-minute Stone Tablet rows by the Monday 00:00 UTC that `week_start_ms` returns for each stamp. Each group gives one bar: the first row's open, the highest high across the group, the lowest low across the group, and the last row's close. A week the venue recorded no trade in produces no bar.

One week of one series checks that rule against hand arithmetic. On A05, the week opening 2026-05-18 holds 2,016 five-minute rows.

```
weekly_bars   open 535.96  high 688.52  low 515.24  close 662.04
by hand       open 535.96  high 688.52  low 515.24  close 662.04
control       a last-open rule returns 662.10 for the open
```

The control matters because three of the four figures survive a wrong grouping. A rule taking the last open rather than the first moves only the open, and the comparison reports it.

A panel's price axis turns logarithmic when its highest high exceeds its lowest low more than eightfold, the same rule the per-asset charts above use.

One sentence above is overtaken. It is quoted whole, and the sentence that replaces it follows.

> *Each panel is one charted base and carries the same label, ten panels to a page.*

Each charted base has a weekly chart of its own, and it sits in that base's group above, beside that base's VWAP chart.

`_write_asset_weekly` in `tools/build_product_manual.py` writes one figure per base, named for the base's label and titled with it.

The price at the finest resolution the record carries

Five minutes is the finest resolution the Stone Tablets hold. `StoneTabletsRegistry` in `src/trading/stone_tablets/registry.py` stores every tablet at that step and rolls every longer timeframe up from it, so no finer price exists in this repository. These panels plot the close of every five-minute candle across the recorded window, one line per charted base.

The weekly bars above and these lines read the same rows and answer different questions. A weekly bar hides what happens inside its week, and inside the week is where the Scrum and Fold cycle works. A week that opens and closes at the same price can still hold the swings the engine sells into and buys back from.

Each charted base has a price trace of its own, and it sits in that base's group above, under the same label as that base's VWAP chart and weekly chart.

`_write_asset_traces` in `tools/build_product_manual.py` writes one figure per base, from the same five-minute rows the weekly bars are built from.

The venue's own position cards

The venue renders one card per open position. 37 cards cover the 38 charted bases, and A33 has none. Each card carries the entry price the venue holds for the open position, the current price, and the gain or loss in dollars and in percent.

A card's entry price is not the buy line of the chart above it. The card follows the open position only: a buy re-weights it, a sell leaves it alone, and a full close resets it to zero. The buy line counts every buy in the record and never resets. That is the same difference this part sets out against `compute_position_health` in `src/exchange/position_health.py`.

Each card is obscured before it is committed. `obscured_placard` in `tools/build_product_manual.py` converts the card to grey, paints the asset mark and the asset name out with the card's own ground colour, paints the code square out with the footer's ground colour, and writes the label where the name was. The venue's own figures are left as the venue rendered them.

`_write_asset_placards` in `tools/build_product_manual.py` writes one card per base, named for that base's label. Each card is embedded under its own heading above, beside that base's three charts. A base the venue supplied no card for is reported absent and stops the build, so no heading can carry another base's card.